台灣自編大學教科書

# 地圖建構與同步定位 (SLAM)

Simultaneous Localization and Mapping



大學教科書

李世淵 (機器人叫獸)

# 目錄

《地圖建構與同步定位》　全書九篇　三十四章


**第一篇　SLAM 與自主導航概論　（第 1–3 章）**

第 1 章　什麼是地圖建構與同步定位 ..... 4

第 2 章　世界知名大學 SLAM 課程導覽 ..... 27

第 3 章　機器人導航系統總覽 ..... 44

**第二篇　定位與導航感測器　（第 4–9 章）**

第 4 章　室內定位技術 ..... 64

第 5 章　無線電導航與衛星導航 ..... 84

第 6 章　地磁導航與電子羅盤 ..... 104

第 7 章　IMU 慣性導航 ..... 124

第 8 章　光達 LiDAR 介紹 ..... 146

第 9 章　相機與視覺導航感測器 ..... 166

**第三篇　SLAM 數學基礎　（第 10–12 章）**

第 10 章　座標系統與剛體運動 ..... 187

第 11 章　機率、雜訊與狀態估測 ..... 206

第 12 章　最佳化與圖論 SLAM ..... 228

**第四篇　2D SLAM 與掃地機器人　（第 13–16 章）**

第 13 章　2D 地圖建構 ..... 248

第 14 章　Gmapping：粒子濾波 SLAM ..... 268

第 15 章　Cartographer：Google 2D / 3D SLAM ..... 288

第 16 章　掃地機器人 SLAM ..... 310

**第五篇　3D 光達 SLAM 與自主巡檢　（第 17–19 章）**

第 17 章　3D 點雲與光達建圖 ..... 332

第 18 章　LOAM 與光達里程計 ..... 355

第 19 章　自主巡檢機器人與場域實務 ..... 377

**第六篇　Visual SLAM 與視覺導航機器人　（第 20–24 章）**

第 20 章　Visual SLAM 入門 ..... 398

第 21 章　對極幾何與相機運動估測 ..... 420
第 22 章　ORB-SLAM2 理論與實作 ..... 444
第 23 章　ORB-SLAM3 與 Visual-Inertial SLAM ..... 468
第 24 章　視覺導航機器人 ..... 490

**第七篇　微型機器人、無人機與邊緣 AI SLAM　（第 25–27 章）**

第 25 章　NanoSLAM 與低功耗 SLAM ..... 514
第 26 章　無人機 SLAM 與視覺慣性導航 ..... 538
第 27 章　多機器人 SLAM 與雲端 / 邊緣協作 ..... 562

**第八篇　ROS / Python / C++ 實作　（第 28–31 章）**

第 28 章　SLAM 程式設計環境 ..... 585
第 29 章　2D 光達小車建圖實作 ..... 610
第 30 章　RGB-D / 雙目相機 SLAM 實作 ..... 635
第 31 章　SLAM 專題與評量 ..... 662

**第九篇　AI、VLM 與語意導航 SLAM　（第 32–34 章）**

第 32 章　語意 SLAM：讓地圖帶有意義 ..... 692
第 33 章　模擬訓練與 Sim-to-Real ..... 711
第 34 章　VLM、語言導航與具身智慧 ..... 738

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第一篇 SLAM 與自主導航概論

# 第 1 章

# 什麼是地圖建構與同步定位

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

## 本章在全書的位置

在正式進入內容之前，先說明這一章在整本書中扮演的角色，以及它與後續各章的連結。這個「地圖」會在每一章開頭出現，幫助同學隨時知道自己走到哪裡、下一步要去哪裡——這正是本書所談的定位問題，只是換成了知識的空間。

**圖 1-0　第 1 章與全書的連結地圖**

【本章負責】建立 SLAM 的核心問題意識：機器人如何同時知道自己在哪、並畫出環境的樣子。

【往下延伸】

1.3 節的六大感測器　→　第二篇（第 4–9 章）逐一深入：室內定位、衛星、地磁、IMU、光達、相機

1.2 節的雞生蛋問題　→　第 11 章以機率濾波求解、第 12 章以圖優化求解

1.4 節的四大應用　→　第四篇掃地機器人、第五篇自主巡檢、第六篇 AR / VR、第七篇無人機

1.5 節的地圖表示　→　第 13 章佔據格、第 17 章點雲與八叉樹、第 32 章語意地圖

1.6 節的前端後端　→　第 12 章正式定義、第 15、18、22 章在真實系統中體現

1.7 節的技術挑戰　→　貫穿全書「常見失敗」的討論主軸

【先備知識】無。本章不需要任何數學或程式基礎，只需要好奇心。

## 學習目標

本章是全書的起點，目的在於建立同學對「地圖建構與同步定位」（Simultaneous Localization and Mapping，簡稱 SLAM）的整體圖像。學完本章之後，同學應能夠達成下列目標：

1. 說明 SLAM 的基本定義，並能以自己的話解釋「定位」（Localization）與「建圖」（Mapping）分別在解決什麼問題。
2. 解釋為什麼「定位」與「建圖」互為前提，形成雞生蛋、蛋生雞的循環依賴問題，以及 SLAM 如何同時估測兩者。
3. 列舉機器人常用的感測器（里程計、IMU、相機、光達、無線電、地磁），並說明各自量測的物理量與優缺點。
4. 舉例說明 SLAM 在自動駕駛、無人機、掃地機器人與 AR / VR 中扮演的角色。
5. 分辨 SLAM 系統的各種輸出形式：軌跡、2D 佔據格地圖、3D 點雲、語意地圖與拓樸地圖。
6. 完成本章實作活動，親身體會「感測器看到的世界」與「實際地圖」之間的差異。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
| --- | --- | --- |
| 同步定位與建圖 | SLAM | 機器人在未知環境中，一邊建立環境地圖、一邊估測自身位置的技術。 |
| 定位 | Localization | 估測機器人在地圖或世界座標中的位置與姿態（位姿，Pose）。 |
| 建圖 | Mapping | 利用感測器資料建立環境的幾何或語意表示。 |
| 位姿 | Pose | 位置（x、y、z）加上姿態（方向角），2D 常記為（x、y、θ）。 |

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 里程計 | Odometry | 由輪子編碼器或感測器累積推算的相對位移，會隨時間漂移。 |
| 慣性量測單元 | IMU | 由加速度計與陀螺儀組成，量測加速度與角速度。 |
| 光達 | LiDAR | 以雷射光飛行時間量測距離的感測器，可產生掃描線或點雲。 |
| 閉環偵測 | Loop Closure | 機器人回到走過的地點時，辨識出「這裡我來過」，用來修正累積誤差。 |
| 佔據格地圖 | Occupancy Grid Map | 把空間切成小格子，標記每格「有障礙、無障礙、未知」的 2D 地圖。 |
| 點雲 | Point Cloud | 由大量三維點組成的空間表示，常由 3D 光達或深度相機產生。 |
| 語意地圖 | Semantic Map | 除了幾何形狀，還標註「門、桌子、走廊」等意義的地圖。 |
| 拓樸地圖 | Topological Map | 以「節點與連線」描述地點與通路關係的地圖，類似捷運路線圖。 |

## 1.1 SLAM 的基本定義：Localization + Mapping

想像一個情境：你被蒙上眼睛，帶到一棟從來沒去過的建築物裡，然後把眼罩拿掉。你手上沒有平面圖，牆上也沒有任何指標。這時你會怎麼辦？大多數人會先環顧四周，記下「左邊有一扇門、前面是長走廊、右邊有樓梯」，然後開始走動。走著走著，你腦中會逐漸浮現這棟建築的樣子，同時你也隨時知道「我現在大概在剛剛那扇門的右前方十公尺」。

這個看似自然的過程，正是機器人領域中最經典、也最困難的問題之一：同步定位與建圖（Simultaneous Localization and Mapping，SLAM）。SLAM 要回答的核心問題只有一句話：

> **本書核心問題**
>
> 機器人沒有地圖，也不知道自己在哪裡，如何一邊走、一邊建立地圖、一邊修正自己的位置？

把這個問題拆開來看，SLAM 其實是兩個子問題的結合：

- 定位（Localization）：已知地圖，估測機器人自己在地圖中的位姿。例如：手機打開 Google Maps，GPS 告訴你「你在校門口」，這就是定位。
- 建圖（Mapping）：已知機器人每一刻的位姿，把感測器量到的環境資訊拼接成一張地圖。例如：測量員拿著儀器，在已知的測站位置上量測周圍地形，繪製地形圖。

單獨來看，這兩個問題都已經有成熟的解法。麻煩的是，真實世界的機器人往往「兩者都沒有」：進入新環境時，既沒有現成地圖可以定位，也沒有精準位姿可以建圖。SLAM 的精神，就是讓機器人同時估測這兩個未知量——一邊移動、一邊觀測、一邊把「我在哪裡」與「世界長什麼樣」這兩件事互相校正、越估越準。

MIT 的視覺導航課程（VNAV，16.485）將 SLAM 定義為：在沒有環境先驗知識的情況下，僅利用感測器資料，同時估測機器人的位姿（或整條軌跡）以及環境表示的問題。

CMU 的 16-833 課程則進一步強調，SLAM 必須在「感測不確定性」之下即時完成——感測器有雜訊、輪子會打滑、光線會變化，機器人永遠拿不到完美的量測值，因此 SLAM 本質上是一個「在不確定性中做最佳估測」的問題。這也是為什麼本書第三篇會花整整三章討論機率、濾波與最佳化的數學基礎。

### SLAM 這個名字的由來

SLAM 一詞源自 1980 年代末至 1990 年代機器人學界對「隨機建圖」（Stochastic Mapping）問題的研究。Smith、Self 與 Cheeseman 在 1986 年前後提出以機率方法同時估測機器人位姿與地標位置的架構，其後 Durrant-Whyte、Leonard、Dissanayake 等學者以擴展卡爾曼濾波器（EKF）建立了完整的理論基礎，並證明 SLAM 問題是「可收斂」的——只要持續觀測，地圖與定位的不確定性會逐漸下降。2000 年代之後，粒子濾波（如本書第 14 章的 Gmapping）與圖優化（如第 15 章的 Cartographer、第 22 章的 ORB-SLAM）陸續成為主流，SLAM 也從實驗室走進掃地機器人、自駕車與手機 AR，成為現代智慧機器人不可或缺的核心技術。

## 1.2 為什麼「定位」與「建圖」是雞生蛋、蛋生雞問題

要理解 SLAM 為什麼困難，關鍵在於看清楚定位與建圖之間的循環依賴：

- 想要建圖，必須先知道自己在哪裡。感測器量到「前方三公尺有一面牆」，這個資訊只有在知道「我現在的位姿」時才有意義——同樣一筆量測，機器人站在不同位置、朝向不同方向，牆就會被畫在地圖上完全不同的地方。
- 想要定位，必須先有地圖。定位的本質是「比對」：把眼前看到的景象，跟地圖中的特徵比對，才能推論出自己的位置。沒有地圖，就沒有比對的依據。

這就是典型的雞生蛋、蛋生雞（Chicken-and-Egg）問題：建圖需要定位，定位又需要地圖，兩者互為前提，似乎哪一個都無法先開始。

### 沒有 SLAM 會發生什麼事：誤差累積

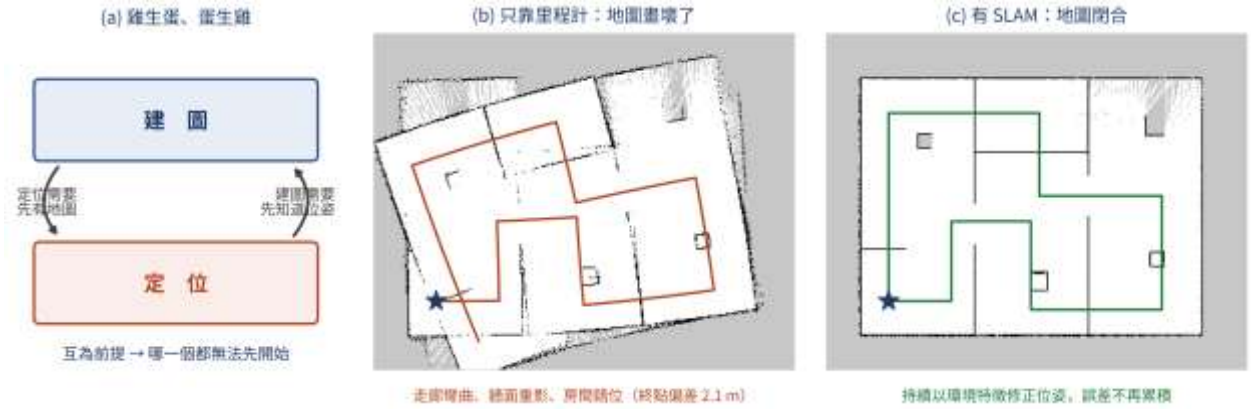
有同學可能會問：機器人不是有里程計嗎？輪子轉了幾圈、走了多遠，累加起來不就知道位置了？理論上是，但實務上里程計有一個致命弱點：漂移（Drift）。輪子會打滑、地毯與磁磚的摩擦力不同、輪徑有製造誤差，每一步的小誤差都會被「累加」進位置估測中，而且永遠不會自己消失。走十公尺可能只差幾公分，但繞行整層樓一圈之後，估測位置可能已經偏差數公尺——如果直接拿這樣的位姿去建圖，畫出來的走廊會彎曲、房間會錯位，同一面牆甚至會在地圖上出現兩次。

SLAM 打破這個僵局的策略，可以用一句話概括：不要把定位與建圖當成兩個先後的步驟，而是當成同一個估測問題，讓兩者互相修正。具體而言，SLAM 系統通常這樣運作：

1. 起步：機器人把出發點定為原點，此時地圖是空的，但位姿是確定的（我就在原點）。
2. 移動與預測：機器人前進一小步，用里程計或 IMU 預測新的位姿——這個預測帶有誤差。
3. 觀測與更新：感測器量測周圍環境，若量到的特徵與地圖中已有的特徵吻合，就用「吻合的程度」反過來修正位姿；同時把新看到的區域加入地圖。
4. 閉環修正：當機器人繞了一圈回到走過的地點，系統辨識出「這裡我來過」（閉環偵測，Loop Closure），此時可以一次性地把整條軌跡與整張地圖的累積誤差攤平修正，讓地圖「閉合」起來。

換句話說，SLAM 並不是奇蹟般地讓誤差消失，而是持續利用「環境本身」作為參考基準：牆壁不會動、柱子不會移位，只要機器人重複觀測到同樣的環境特徵，就能把漂移「拉回來」。這個觀念會貫穿全書——從第 11 章的貝氏濾波、第 12 章的位姿圖優化，到第 15 章 Cartographer 的閉環偵測，都是同一個精神在不同層次的展開。

**圖 1-1　雞生蛋問題，以及沒有 SLAM 時地圖會變成什麼樣子**




左圖把這個循環依賴畫成一個環：要建圖得先知道自己在哪，要定位又得先有圖可比對，兩者互為前提，看起來哪一個都無法先開始。

中間與右邊兩張圖則是同一台機器人、同一個場域、同一趟路線，用兩種方式建出來的地圖。這是用程式真的跑出來的模擬：機器人繞著走廊走一圈，一邊以 2D 光達掃描周圍，一邊把量到的牆面標進格子裡。

中圖只用里程計推算位姿。每一步的角度誤差極小（約百分之一度），走完一圈後終點只偏離起點 2.1 公尺——聽起來不多，但畫出來的地圖已經不能用了：整張圖被扭轉、走廊彎曲、同一面牆因為前後兩次經過時位姿不同而被畫成兩條、房間的隔牆彼此錯開。這正是前一段文字所描述的現象。

右圖的位姿經過修正，牆面筆直、隔間方正、地圖閉合。請注意兩張圖的感測資料完全相同——差別只在「機器人以為自己在哪裡」。這句話值得記住：SLAM 建不出好地圖的時候，問題幾乎永遠出在定位，而不是感測器。

**從這張圖可以先記住三件事**

一、地圖品質取決於位姿品質。感測器再準，位姿錯了，地圖就是歪的。

二、誤差不是突然出現的，而是一步一步累積的——中圖每一小段看起來都很合理，錯的是「加起來」。

三、SLAM 的修正力量來自環境本身：牆不會移動，只要重複觀測到同一面牆，就能把漂移拉回來。第 12 章會讓同學看到這個修正在數學上如何完成。

### 兩條解法脈絡：本書的主線預告

面對這個互相依賴的難題，四十年來的研究者發展出兩條主要的解法脈絡。它們構成了本書第三篇之後的主線，同學現在不需要理解細節，但先知道有這兩條路，後面讀起來會清楚得多。

- 機率濾波路線：既然無法確定機器人到底在哪，那就同時保留許多種可能性，再用感測器資料逐一淘汰不合理的假設。這條路發展出卡爾曼濾波與粒子濾波，是第 11 章的主題，也是第 14 章 Gmapping 的理論基礎。
- 最佳化路線：把「機器人走過的每個位置」與「感測器量到的每個約束」寫成一個巨大的方程組，再一次求出最符合所有觀測的解。這條路發展出圖優化與束調整，是第 12 章的主題，也是第 15 章 Cartographer 與第 22 章 ORB-SLAM 的核心。

這兩條路線並非互相取代。第 11 章會說明濾波方法的即時性優勢，第 12 章則會說明最佳化方法為何能回頭修正歷史錯誤。第 14 章與第 15 章更會讓同學在同一個場域親手比較兩者的差異——那時再回頭看本節，會對「雞生蛋問題」的解法有更立體的理解。

## 1.3 機器人如何感知世界：里程計、IMU、相機、光達、無線電、地磁

人類靠眼睛、耳朵、前庭系統（內耳的平衡器官）與肌肉本體感覺來感知「我在哪裡、周圍有什麼」。機器人也需要對應的感官。SLAM 所使用的感測器可以粗分為兩大類：

- 本體感測器（Proprioceptive Sensors）：量測機器人「自己」的運動狀態，例如輪子轉了多少（里程計）、機身加速度與轉動角速度（IMU）。它們不需要看外界，但誤差會累積。
- 外部感測器（Exteroceptive Sensors）：量測機器人「周圍環境」，例如相機拍到的影像、光達量到的距離、收到的無線電訊號強度、地球磁場方向。它們提供絕對或半絕對的參考，但容易受環境干擾。

SLAM 的藝術，就在於把「會漂移但連續平滑」的本體感測器，與「不漂移但有雜訊、可能瞬間出錯」的外部感測器融合起來，截長補短。以下逐一介紹本書後續章節會深入探討的六大類感測器。

### 一、里程計（Odometry）

輪型機器人最基本的感測器。在馬達軸或輪軸上安裝編碼器（Encoder），計算輪子轉動的圈數，配合輪徑與輪距，就能推算機器人前進的距離與轉彎的角度。里程計的優點是取樣頻率高、成本低、資料連續；缺點是前面提過的漂移——打滑、輪徑誤差、地面不平都會讓誤差不斷累積。在 ROS 系統中，里程計對應 odom 座標框架，是幾乎所有 SLAM 演算法的「運動預測」來源（詳見第 10 章與第 29 章）。

### 二、慣性量測單元（IMU）

IMU（Inertial Measurement Unit）通常包含三軸加速度計與三軸陀螺儀，有些還加上三軸磁力計。加速度計量測線加速度，陀螺儀量測角速度，把它們對時間積分，理論上可以推算速度、位置與姿態——這就是慣性導航。IMU 的取樣頻率極高（常見 100 Hz 到 1000 Hz），對快速運動非常敏感，是無人機與視覺慣性 SLAM（如第 23 章的 ORB-SLAM3）不可或缺的元件。但積分會放大雜訊與偏差，造成快速漂移，因此 IMU 幾乎一定要與其他感測器融合使用（詳見第 7 章）。

### 三、相機（Camera）

相機是資訊量最豐富、成本又最低的感測器：一張影像包含紋理、顏色、形狀、甚至語

意（這是門、那是人）。單目相機（Monocular）便宜但無法直接量測深度，會有尺度不確定性；雙目相機（Stereo）利用左右影像視差計算深度；RGB-D 相機（如 RealSense）直接輸出每個像素的距離。以相機為主的 SLAM 稱為 Visual SLAM，是本書第六篇的主軸。相機的弱點是依賴光線與紋理——黑暗、逆光、白牆、快速運動造成的模糊，都會讓視覺 SLAM 失效（詳見第 9 章、第 20 章）。

### 四、光達（LiDAR）

光達（Light Detection and Ranging）發射雷射光並量測反射回來的時間（Time of Flight），直接得到「這個方向、這個距離有東西」的精確幾何資訊。2D 光達旋轉掃描一個平面，輸出一圈距離值（LaserScan），是掃地機器人與教學小車的標準配備；3D 光達（多線雷射）同時掃描多個仰角，輸出三維點雲，是自駕車與自主巡檢機器人的核心感測器。光達的優點是量測精確、不受光線影響；缺點是價格較高、對玻璃與鏡面會失效、雨霧會造成雜訊（詳見第 8 章、第五篇）。

### 五、無線電（Radio）

GPS / GNSS 衛星定位是最廣為人知的無線電定位，但在室內、隧道、高樓峽谷中訊號會被遮蔽。室內環境則可利用 Wi-Fi 訊號強度指紋、藍牙 Beacon、UWB 超寬頻等技術進行定位。無線電定位的優點是提供「絕對位置」參考、不會漂移；缺點是精度有限（GPS 約數公尺、Wi-Fi 約數公尺、UWB 約十至三十公分）、易受多路徑反射干擾。實務上常作為 SLAM 的輔助資訊或初始化來源（詳見第 4 章、第 5 章）。

### 六、地磁（Geomagnetism）

地球本身就是一個大磁鐵，磁力計（電子羅盤）可以量測地磁方向，提供絕對航向角參考。更有趣的是，建築物內的鋼筋與設備會讓地磁產生獨特的局部扭曲，這些扭曲形成「地磁指紋」，可以用來做室內定位。地磁感測的優點是完全被動、零功耗需求極低；缺點是易受鐵磁物體干擾，必須經過校正（詳見第 6 章）。

| 感測器 | 量測內容 | 主要優點 | 主要弱點 | 本書章節 |
|---|---|---|---|---|
| 里程計 | 輪子轉動→相對位移 | 便宜、連續、高頻 | 打滑漂移、誤差累積 | 第 10、29 章 |
| IMU | 加速度、角速度 | 高頻、對快速運動敏感 | 積分漂移快 | 第 7、23 章 |
| 相機 | 影像（紋理、顏色、語意） | 資訊豐富、成本低 | 依賴光線與紋理 | 第 9、19–23 章 |
| 光達 | 雷射測距（2D / 3D） | 幾何精確、不受光線影響 | 價格較高、鏡面失效 | 第 8、13–18 章 |
| 無線電 | GPS / Wi-Fi / UWB 訊號 | 絕對位置、不漂移 | 精度有限、多路徑干擾 | 第 4、5 章 |
| 地磁 | 地球磁場方向與強度 | 被動、提供絕對航向 | 鐵磁干擾、需校正 | 第 6 章 |

**表 1-1　SLAM 常用感測器比較**

### 感測器如何在本書中展開

本節提到的六類感測器，每一類在第二篇都有專章討論。這裡先給出對照表，讓同學知道每個感測器的「深入版本」在哪裡，也方便日後查閱。

**表 1-2　感測器與專章對照**

里程計　→　第 3 章（航位推算）、第 13 章（與光達融合）、第 29 章（實作與校正）

IMU 慣性單元　→　第 7 章（原理、漂移、姿態濾波、預積分）

相機　→　第 9 章（成像模型）、第六篇（視覺 SLAM 全篇）

光達　→　第 8 章（原理與規格）、第四篇（2D）、第五篇（3D）

無線電　→　第 4 章（室內定位）、第 5 章（衛星導航）

地磁　→　第 6 章（電子羅盤與地磁指紋）

每一章的結構都相同：原理 → 規格與限制 → 失效情境 → 與其他感測器的互補 → 實作活動。

這個一致的結構，是為了讓同學能快速比較不同感測器，而不是把它們當成孤立的知識。

這裡先埋下一個貫穿全書的觀念：沒有任何一種感測器是全能的。光達精確卻看不懂玻璃、相機資訊豐富卻怕黑暗、IMU 反應快卻會漂移、衛星訊號準確卻進不了室內。第二篇的每一章都會誠實地討論該感測器「在什麼情況下會失敗」，而第 30 章的實驗更會讓同學親眼看到光達與相機在同一場域各自崩潰的樣子。理解限制，是本書從第一頁到最後一頁的核心信念。

## 1.4　SLAM 與自動駕駛、無人機、掃地機器人、AR / VR 的關係

SLAM 之所以在過去二十年從冷門學術題目變成兵家必爭的核心技術，是因為它剛好是四大熱門應用的共同地基。

### 自動駕駛（Autonomous Driving）

自駕車需要公分級的定位精度——GPS 的數公尺誤差足以讓車子跨越車道。因此自駕車普遍採用「高精地圖 + 即時定位」的策略：先以測繪車搭載 3D 光達與相機，用 SLAM 技術建立高精度地圖；行駛時再把即時感測資料與高精地圖比對，實現精準定位。在沒有高精地圖的區域（例如新開通的道路、停車場），車輛則必須即時執行 SLAM。本書第 17、18 章介紹的 3D 光達建圖與 LOAM 系列演算法，正是自駕車定位建圖的技術骨幹。

### 無人機（Drone / UAV）

無人機在室內、橋樑下方、隧道等 GPS 訊號被遮蔽（GPS-denied）的環境飛行時，只能依靠機載感測器定位。由於載重與電力極為有限，無人機偏好輕巧的「相機 + IMU」組合，也就是視覺慣性里程計（VIO）與視覺慣性 SLAM——這正是 ETH Zürich 與 UZH 研究群的強項，也是 MIT VNAV 課程以無人機為實驗平台的原因。2015 年後，DJI 等消費級無人機把 VIO 做進產品，讓無人機能在室內穩定懸停；學術端則持續往更小、更省電推進，例如本書第 25 章介紹的 NanoSLAM，讓 44 公克的微型無人機也能完全機載執行 SLAM。

### 掃地機器人（Robot Vacuum）

掃地機器人是 SLAM 最成功的大眾化案例。早期的掃地機採用「隨機碰撞」策略——撞到牆就轉彎，清掃路線雜亂無章、重複又遺漏。2010 年 Neato 率先把 2D 光達裝上掃地機，2017 年小米第一代光達掃地機器人以親民價格普及，讓 SLAM 技術首次大量走進一般家庭：機器人先建立房間地圖，再規劃弓字形路徑逐區覆蓋，效率與覆蓋率大幅提升。今天打開掃地機 App 看到的房間地圖、禁區設定、指哪掃哪，背後全是 SLAM（詳見第 16 章）。

### AR / VR（擴增實境 / 虛擬實境）

AR 眼鏡或手機 AR 要把虛擬物件「釘」在真實世界的桌面上，前提是裝置必須即時知道自己在空間中的位姿——這其實就是 SLAM，只是業界慣稱「空間定位」或「Inside-Out Tracking」。Apple 的 ARKit、Google 的 ARCore、Meta Quest 的頭盔追蹤，核心都是視覺慣性 SLAM。VR 頭盔用機身相機追蹤環境特徵，取代了早期需要外部基地台的定位方式。

可以說，SLAM 讓 AR / VR 從「固定座位的體驗」進化為「自由走動的體驗」。

**觀念小結**

這四大應用看似天差地遠，但底層都是同一個問題：在未知或部分未知的環境中，即時估測自身位姿並理解周圍空間。

差別只在於感測器組合（光達為主或相機為主）、運算資源（車載電腦或手機晶片）與精度要求（公分級或公寸級）。

這也是為什麼本書強調「一套觀念、多種平台」：學會 SLAM 的核心原理，就能在自駕車、無人機、掃地機器人與 AR 之間觸類旁通。

## 1.5 從教室到校園：高中與大學場域的 SLAM 教學案例

本書的一大特色，是把 SLAM 的學習「落地」在兩個真實的教學場域：高中的 AI 機器人課程，與大學的智慧機器人學程。同樣的核心觀念，會依照學習階段設計成不同深度的活動。

### 高中場景：看得懂、做得出來

高中階段的重點是建立直覺與動手經驗，不急著推導數學。以「教室」為實驗場域，可以設計以下循序漸進的活動：

- 活動一：人肉 SLAM。一位同學蒙眼、由同學口頭指揮在教室行走，行走者邊走邊口述「我覺得我在哪裡」，其他同學在紙上記錄實際位置與估測位置的差距，體會「沒有外部觀測時誤差如何累積」。
- 活動二：手機感測器初體驗。用手機的指南針、加速度計 App 觀察感測器數值，理解 IMU 與地磁感測的原理與雜訊。
- 活動三：掃地機器人觀察。觀察家用掃地機器人的清掃路線與 App 中的地圖，討論「它怎麼知道哪裡掃過了」。

- 活動四：模擬器建圖。在 ROS 模擬環境（如 TurtleBot3 模擬器）中操作虛擬小車建立教室地圖，不需要真實硬體也能完整體驗 SLAM 流程。

### 大學場景：能推導、能實作、能研究

大學部與專題階段，則以大學智慧機器人教室與校園為場域，把同一套觀念往下扎根、往上延伸：

- 基礎實作：以 2D 光達教學小車在教室與走廊執行 Gmapping 與 Cartographer 建圖（第 14、15、29 章），量化比較兩種演算法的地圖品質。
- 進階實作：以 RGB-D 相機執行 RTAB-Map 與 ORB-SLAM3（第 30 章），比較視覺 SLAM 與光達 SLAM 在同一場域的表現。
- 專題研究：機器狗校園巡檢、緊耦合光達慣性建圖、視覺導航無人機、多機器人協同建圖（第 18、22、24–26 章），銜接研究所等級的題目。

本書第 31 章提供六個完整專題與評分 Rubric，涵蓋從高中到大學專題的完整光譜。建議教師在學期初就讓學生看到這條「學習地圖」：知道現在學的每一個觀念，最終會通往哪一個看得見、摸得著的機器人系統。

## 1.6 SLAM 系統輸出：軌跡、2D 地圖、3D 點雲、語意地圖、拓樸地圖

SLAM 系統執行完畢後，到底會「產出」什麼？依照應用需求不同，輸出形式可以分為五大類。理解這些輸出形式，有助於在後續章節中快速掌握「這個演算法是在解決哪一種輸出」。

### 一、軌跡（Trajectory）

機器人從出發到結束，每一個時間點的位姿序列。軌跡是評估 SLAM 精度最直接的依據：把估測軌跡與真實軌跡（Ground Truth，通常由動作捕捉系統或高精度 GPS 提供）比較，計算絕對軌跡誤差（ATE）與相對位姿誤差（RPE）。EuRoC、TUM、KITTI 等公開資料集

（第 26、30 章）都提供真實軌跡供研究者評測。

## 二、2D 佔據格地圖（Occupancy Grid Map）

把地面切成固定大小的格子（例如每格 5 公分），每一格記錄「被佔據的機率」：黑色代表障礙物（牆、桌腳）、白色代表可通行空間、灰色代表尚未觀測的未知區域。這是室內輪型機器人與掃地機器人最常用的地圖形式，也是 ROS Navigation 導航堆疊的標準輸入（第 13 章）。

## 三、3D 點雲（Point Cloud）

由數十萬至數億個三維點構成的空間模型，每個點帶有座標（x、y、z），可再附加顏色或反射強度。3D 點雲能完整呈現立體結構——樓梯、天花板管線、桌面高度，是自主巡檢、工廠數位分身（Digital Twin）與自駕車高精地圖的基礎（第 17 章）。

## 四、語意地圖（Semantic Map）

在幾何地圖之上，為每個區域或物件貼上「意義標籤」：這一塊是門、那一塊是桌子、這條是走廊、那裡是充電站。語意地圖讓機器人不只知道「哪裡有障礙」，更知道「那個障礙是什麼」，因而能執行「去教務處」「避開樓梯」這類人類語言等級的任務。語意地圖是 AI 視覺模型（YOLO、SAM、VLM）與 SLAM 的交會點，也是第九篇（第 32–34 章）的主題。

## 五、拓樸地圖（Topological Map）

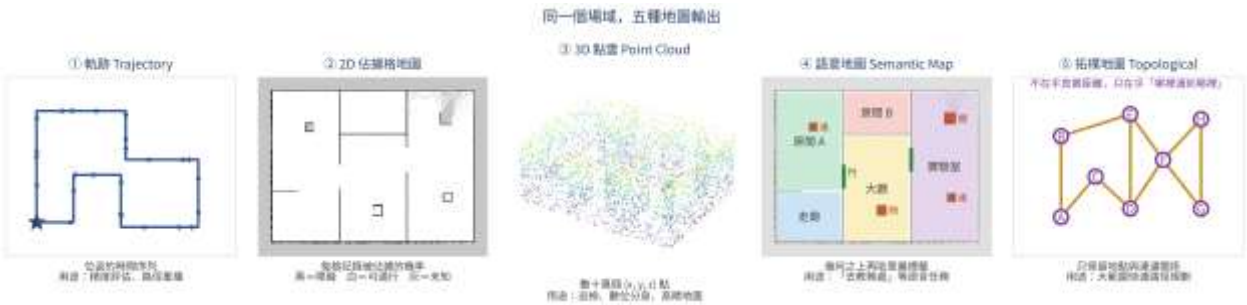
拋開精確的幾何座標，只保留「地點」與「地點之間的連通關係」，就像捷運路線圖不在乎車站的真實距離，只在乎「哪一站可以到哪一站」。拓樸地圖記憶體需求極低、規劃路徑極快，適合大範圍導航；實務系統常採「拓樸為骨架、格地圖為血肉」的混合式地圖。

| 輸出形式 | 資料結構 | 典型應用 | 代表章節 |
| --- | --- | --- | --- |
| 軌跡 | 位姿的時間序列 | 精度評估、路徑重播 | 第 31 章 |

| 輸出形式 | 資料結構 | 典型應用 | 代表章節 |
|---|---|---|---|
| 2D 佔據格地圖 | 機率格網 | 室內導航、掃地機器人 | 第 13–16 章 |
| 3D 點雲 | 三維點集合 | 空間掃描、自主巡檢、自駕車 | 第 17、18 章 |
| 語意地圖 | 幾何 + 物件標籤 | 語意導航、任務型機器人 | 第 24、32 章 |
| 拓樸地圖 | 節點與邊的圖 | 大範圍快速規劃 | 第 3、32 章 |

**表 1-3　SLAM 系統五大輸出形式**

**圖 1-2　同一個場域的五種地圖輸出**




這五張圖來自同一個模擬場域、同一趟掃描，差別只在「保留什麼、丟掉什麼」。① 軌跡只保留機器人自己的位姿序列，完全不記錄環境；② 佔據格地圖把地面切成格子，記錄每一格被佔據的機率，是第四篇的主角；③ 點雲保留三維結構，能看出牆的高度與天花板，是第五篇的基礎；④ 語意地圖在幾何之上疊加意義——這裡是走廊、那裡是實驗室、那個方塊是桌子，是第 32 章的主題；⑤ 拓樸地圖丟掉所有幾何，只留下地點與連通關係，像捷運路線圖。

值得注意的是資訊量的取捨。從①到⑤並不是「愈後面愈好」——拓樸地圖的資料量可能只有點雲的百萬分之一，但要讓機器人避開一張椅子，它毫無幫助；反過來，要規劃「從一樓大廳到三樓實驗室」的路線，點雲反而笨重得多。實務系統常同時維護多種地圖，例如用拓樸圖做全域規劃、用格地圖做局部避障，這就是 1.6 節所說的「拓樸為骨架、格地圖為血肉」。

同學可以把這張圖當成全書的目標清單：讀完第四篇會親手做出②，第五篇是③，第六篇的視覺 SLAM 會產生特徵點地圖與③的變體，而第 32 章要做的是④。現在看不懂細節沒關係，

但請先建立一個印象——SLAM 的「輸出」不是只有一種。

## 公式或流程

### 一、SLAM 的機率定義

雖然嚴謹的推導要到第 11 章才展開，本章先給出 SLAM 的標準數學表述，讓同學對「SLAM 在估什麼」有一個精確的錨點。定義以下符號：

- x(t)：機器人在時間 t 的位姿，2D 情況下為（x、y、θ）。
- m：環境地圖（可能是格地圖、地標集合或點雲）。
- z(1:t)：從時間 1 到 t 的所有感測器觀測（雷射掃描、影像特徵等）。
- u(1:t)：從時間 1 到 t 的所有控制輸入或里程計讀值。

完整 SLAM（Full SLAM）問題，是在給定所有觀測與控制之下，估測整條軌跡與地圖的聯合機率分布：

$$p(x(1:t), m \mid z(1:t), u(1:t))$$

線上 SLAM（Online SLAM）問題，則只估測「當下」的位姿與地圖，把過去的軌跡邊緣化掉，適合即時系統：

$$p(x(t), m \mid z(1:t), u(1:t))$$

用白話翻譯：SLAM 要找的答案，是「在我看到的所有東西、走過的所有路之下，最有可能的軌跡與地圖是什麼」。之所以寫成機率分布而不是單一數值，正是因為感測器有雜訊、運動有誤差——SLAM 給的答案永遠附帶「有多確定」的資訊。粒子濾波 SLAM（第 14 章）、EKF-SLAM（第 11 章）與圖優化 SLAM（第 12 章）本質上都是求解這個機率問題的不同數值方法。

### 二、SLAM 系統標準流程

現代 SLAM 系統普遍採用「前端 + 後端」的架構，流程如圖 1-3 所示：

**圖 1-3　SLAM 系統標準流程**

感測器資料（光達掃描 / 影像 / IMU / 里程計）

↓

前端（Front-End）：特徵提取、資料關聯、掃描匹配、里程計估測

↓

後端（Back-End）：濾波或圖優化，估測位姿與地圖的最佳解

↓

閉環偵測（Loop Closure）：辨識重訪地點，修正全域累積誤差

↓

輸出：軌跡 + 地圖（2D 格地圖 / 3D 點雲 / 語意地圖）

前端負責「把原始感測資料轉成幾何約束」，後端負責「在所有約束下求出最一致的答案」，閉環偵測則是修正長期漂移的關鍵武器。本書第二篇講感測器（前端的輸入）、第三篇講數學（後端的工具）、第四篇到第六篇講各種具體系統如何實現這條流程，第七篇之後則探討微型化、多機協作與 AI 語意化的前沿延伸。

## 1.7　SLAM 的技術演進與當代前沿

前面幾節介紹的是 SLAM 的基本問題與構成。本節把視野拉高，看這門學問四十年來如何演變，以及此刻正在發生什麼——這能幫助同學理解，本書後續章節所教的技術，各自處於這條演進線的哪個位置。

### 四個階段的演進

綜合近年的回顧文獻，SLAM 的發展大致可分為四個階段，而且每個階段並未取代前一

個，而是不斷疊加、互相補強：

1. 手工設計時期（1986–2010 年代）：研究者以人類對幾何的理解，手工設計特徵偵測器（如第 20 章的 SIFT、ORB）與濾波演算法（第 11 章）。這個時期奠定了 SLAM 的數學骨架，也是本書第三篇到第六篇的主要內容。
2. 深度學習時期（2015 年前後起）：以神經網路取代部分手工模組——學習來的特徵、學習來的深度估計、學習來的閉環偵測。本書第 9.4 節的單目深度估測、第 20.7 節的深度特徵，都屬於這個階段的成果。
3. 輻射場時期（2021 年起）：以神經輻射場（NeRF）與三維高斯潑濺（3D Gaussian Splatting，3DGS）重新定義「地圖」是什麼——地圖不再只是點雲或格子，而是一個能重新算繪出照片級畫面的場。
4. 基礎模型時期（2023 年起）：視覺語言模型與機器人基礎模型進入 SLAM，讓機器人從「知道形狀」走向「理解意義」。這是本書第 32 章的主題。

### 輻射場如何改變地圖的定義

第三階段的轉折特別值得說明，因為它改變了一個看似理所當然的前提：地圖應該長什麼樣。

傳統 SLAM 的地圖是佔據格（第 13 章）或點雲（第 17 章）——它們回答「哪裡有東西」，但無法回答「從這個角度看過去，畫面應該長什麼樣」。NeRF 以神經網路隱式地表示整個空間的輻射場，能從任意視角算繪出逼真的畫面；而 3DGS 則改採顯式表示，把場景建模為一群三維高斯基元的集合，每個基元有位置（均值）、共變異數、顏色與不透明度。

3DGS 之所以在機器人領域快速崛起，關鍵在於效率。不同於隱式表示需要對空間中每一個點查詢神經網路，3DGS 的可微分光柵化管線克服了大型神經網路訓練與算繪速度緩慢的瓶頸——而那正是 NeRF 系統難以用於機器人的主因。此外它的顯式結構讓地圖能被輕易編輯或形變，這對地圖修正與環境適應相當有利。

**觀念釐清：三種場景表示的取捨**

【隱式】以神經網路逼近輻射場。記憶體小，但每次查詢都要跑網路，速度慢。

【顯式】直接在學到的空間特徵（體素、雜湊格、高斯基元）上做算繪，不含神經元件。速度快，但記憶體需求高。

【混合】把學到的空間特徵與神經網路結合。訓練與算繪都能加速，同樣需要較多記憶體。

對機器人而言，速度與記憶體的權衡永遠存在——這與第 25 章「毫瓦級平台」的取捨是同一個問題，

只是換到了地圖表示的層次。

## 當代前沿的四個方向

綜觀近兩年的研究，SLAM 領域最活躍的方向可歸納為四項，同學若日後要投入研究，這是相當實用的地圖：

- 穩健性：讓輻射場 SLAM 能應付動態環境與運動模糊。近期已有專門處理模糊輸入的稠密神經 SLAM，以及針對演化中場景的高斯建圖方法；也有系統以 IMU 輔助處理模糊、滾動快門與雜訊嚴重的熱像影像——這正好呼應第 26.4 節討論的高速飛行三重打擊。
- 協作化：多台機器人共同建立高斯地圖。值得注意的是，傳統協作式 SLAM 有走向分散式以提升擴展性與穩健度的趨勢，但由於三維高斯訓練的運算成本高昂，目前協作式高斯 SLAM 反而以集中式設計居多——這與第 27.3 節的架構討論恰成對照，也說明技術選型如何受運算成本左右。
- 語言化：把語言能力融入地圖表示，讓機器人能以自然語言查詢空間。這是第 32 章

的主題，也是第 24.5 節 VLM 導航的延伸。

- 基礎模型化：以多視圖基礎模型支撐單目高斯 SLAM，或探討「對機器人而言，究竟什麼才是最好的三維場景表示」這類根本問題。這條線正把幾何 SLAM 與人工智慧推向同一個交會點。

### 本書的定位

讀到這裡，同學可能會問：既然輻射場與基礎模型如此熱門，為什麼本書大部分篇幅仍在講傳統的幾何方法？

答案有三層。其一，實務性：目前絕大多數真正部署在產品上的系統——掃地機器人、自駕車、巡檢機器狗、AR 眼鏡——骨幹仍是本書所教的幾何方法。它們成熟、可解釋、運算需求明確。

其二，基礎性：輻射場與基礎模型並未取代幾何，而是建立在幾何之上。3DGS SLAM 仍需要位姿估計、仍需要閉環修正、仍需要處理感測器的限制。不理解幾何，就無法判斷這些新方法在什麼時候會失效。

其三，教學性：本書的目標是培養能做出判斷的工程師，而非追逐術語的使用者。第 31 章的專題與第 32 章的討論會反覆強調同一件事——技術會不斷演進，但「理解限制、選擇合適工具」的能力不會過時。

## 案例

### 案例 1-1　小米掃地機器人：SLAM 走進千家萬戶

2017 年，小米推出第一代米家掃地機器人，機頂搭載一顆旋轉式 2D 光達，售價卻壓在傳統高階掃地機的一半以下。它的工作流程完整示範了本章介紹的 SLAM 循環：開機後光達每秒掃描周圍數千個距離點，機器人一邊行走一邊以粒子濾波演算法建立房間的佔據格地

圖，同時在地圖中定位自己；地圖完成後，路徑規劃模組把房間切成區塊，以弓字形路徑逐區覆蓋，確保不重複、不遺漏。這台機器讓「SLAM」這個原本只出現在論文中的名詞，第一次大規模進入一般家庭的客廳。對學習者的啟示是：SLAM 不是遙不可及的尖端科技，它的核心版本可以跑在一顆手機等級的處理器上——這也是本書相信高中生也能學會 SLAM 的原因。

## 案例 1-2　DARPA 地下挑戰賽：SLAM 的極限考場

2018 至 2021 年，美國國防高等研究計畫署（DARPA）舉辦地下挑戰賽（Subterranean Challenge），要求機器人團隊在人造隧道、地下城市與天然洞穴中自主探索、建圖並尋找目標物。地下環境集合了 SLAM 的所有噩夢：完全沒有 GPS、灰塵與煙霧干擾光達、黑暗讓相機失效、狹窄通道與垂直豎井考驗運動能力。參賽隊伍（包括 MIT、CMU、ETH 等名校團隊）普遍採用多感測器融合——光達 + IMU + 相機 + 熱像儀，並讓輪型機器人、四足機器狗與無人機組成異質團隊協同建圖。這場比賽大幅推進了光達慣性 SLAM（第 18 章）與多機器人 SLAM（第 27 章）的實用化，也證明了一件事：單一感測器、單一機器人的 SLAM 已趨成熟，未知嚴苛環境下的強韌性（Robustness）才是下一個十年的主戰場。

## 案例 1-3　手機 AR 量尺：口袋裡的視覺慣性 SLAM

打開 iPhone 內建的「測距儀」App，把鏡頭對準桌面移動幾秒，就能在空中拉出一條貼合真實世界的虛擬量尺。這背後是 ARKit 的視覺慣性 SLAM：手機相機以每秒 30 至 60 張的速度追蹤影像特徵點，IMU 以每秒數百次的頻率量測運動，兩者融合後即時估測手機的六自由度位姿，並重建出桌面的平面結構。這個案例適合作為課堂的第一個示範——不需要任何機器人硬體，每位同學的手機裡就住著一套工業級 SLAM 系統。教師可以讓同學實驗：在白牆（無紋理）、昏暗房間、快速甩動手機時量尺會如何失準，親身體會第 1.3 節所述相機感測器的限制。

### 案例 1-4 從 iMAP 到 3DGS：五年之間，地圖變成了什麼

本書的案例通常談的是成熟技術的應用，這一則例外——它談的是正在發生的事，目的是讓同學感受這個領域的變化速度。

2021 年，一項名為 iMAP 的工作首度展示了以神經隱式表示做即時 SLAM 的可能性，被視為輻射場 SLAM 的起點。此後短短數年間，相關研究呈爆炸性成長：有的以體素特徵格改良、有的改用特徵平面、有的結合點雲式的輻射場。到了 2023 年底，三維高斯潑濺出現，因其顯式表示與即時算繪能力，迅速成為新的主流方向。

這個變化的速度值得同學留意：從一個概念驗證，到成為整個社群的研究主軸，中間只隔了大約三年。相較之下，本書第 22 章的 ORB-SLAM 從 2015 年發表至今，仍是視覺 SLAM 的標準比較基準——十年不衰。

兩者的對比帶出一個值得深思的問題：什麼樣的技術能長存，什麼樣的會被快速取代？筆者的觀察是，解決「根本問題」的方法（如對極幾何、圖優化、閉環偵測）生命週期極長，而解決「表示方式」的方法（地圖用什麼格式儲存、畫面如何算繪）則迭代快速。這也是本書把大部分篇幅放在前者的原因。

對同學的建議是：把新技術當成「又一種表示方式」來理解，問它解決了舊方法的什麼限制、又付出了什麼代價（記憶體？運算？穩健度？）。有了這個框架，無論明年出現什麼新名詞，你都能快速定位它的價值。

## 實作活動

### 實作 1-1 感測器看到的世界 vs 實際地圖

目的：體會「以第一人稱視角記錄環境」與「俯視地圖」之間的落差，理解 SLAM 要解決的資料轉換問題。

### 器材

- 手機一支（錄影用），或教學用輪型機器人一台。
- 方格紙兩張、鉛筆、量尺（或捲尺）。

### 步驟

1. 分組（每組 3 至 4 人），選定一條室內路線：例如從教室後門出發，沿走廊直行、右轉、經過飲水機、回到前門。
2. 第一人稱記錄：一位同學手持手機以走路高度錄影走完全程，過程中口述看到的地標（「左手邊有滅火器」「前方是玻璃門」）。
3. 憑記錄畫地圖：回到教室後，只准看影片、不准回到現場，全組合作在方格紙上畫出「感測器看到的世界」——路線、轉角、地標的相對位置，並標注估計的距離。
4. 實地測量：帶著捲尺回到現場，實際測量走廊長度、轉角位置與地標間距，在第二張方格紙上畫出「實際地圖」。
5. 比對與討論：把兩張圖疊在一起，量測誤差最大的地方，討論誤差從哪裡來——距離感失準？轉彎角度估錯？地標記錯順序？

### 討論問題

1. 走得越遠，你們畫的地圖誤差越大還是越小？這與課文中的「漂移」有什麼關係？
2. 如果路線最後回到出發點，你們畫的終點有沒有跟起點重合？如果沒有，該怎麼「修正」整張圖？這個動作對應課文中的哪一個名詞？
3. 影片中哪些地標最容易辨認、哪些最容易搞混？這對「相機式 SLAM 喜歡什麼樣的環境」有什麼啟發？

### 延伸挑戰（大學部）

以手機錄製的影片作為輸入，在電腦上執行開源視覺 SLAM 系統（如 ORB-SLAM3 的單目模式），輸出估測軌跡，與你們手繪的地圖及實測地圖三方比較。此挑戰預告了第 22、23、

30 章的內容。

## 延伸閱讀

1. C. Cadena et al., "Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age," IEEE Transactions on Robotics, vol. 32, no. 6, 2016.（SLAM 三十年發展的權威回顧論文，全書多章將反覆引用）
2. H. Durrant-Whyte and T. Bailey, "Simultaneous Localization and Mapping: Part I & Part II," IEEE Robotics & Automation Magazine, 2006.（SLAM 經典入門教程，建立理論基礎）
3. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005.（機率機器人學聖經，本書第三篇數學基礎的主要參考）
4. MIT OpenCourseWare, 16.485 Visual Navigation for Autonomous Vehicles（VNAV）.（免費公開的視覺導航完整課程，含講義與實驗）
5. CMU 16-833 Robot Localization and Mapping 課程網站.（以狀態估測與最佳化為主軸的 SLAM 研究所課程）
6. Cyrill Stachniss（University of Bonn）Robot Mapping 系列課程影片，YouTube 免費公開.（以直觀動畫講解 SLAM 演算法，適合自學）
7. C. Campos et al., "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM," IEEE Transactions on Robotics, 2021.（本書第 23 章主角的原始論文）

## 全書導覽：三十四章的地圖

作為第一章的收尾，這裡提供一份全書的鳥瞰圖。同學不必現在就記住它，但在往後每次感到迷失時，可以回來看看自己在哪一段旅程上。

**圖 1-4　全書九篇的學習路徑**




## 三條可選的閱讀路線

- 高中 / 入門路線：第 1–3 章建立觀念 → 第 8、9 章認識光達與相機 → 第 13、16 章理解 2D 建圖與掃地機器人 → 第 28、29 章動手做模擬。全程不需深入數學，各章的實作活動多數可用模擬完成。
- 大學部完整路線：依序讀第一篇到第六篇，搭配第八篇的實作章節同步進行。第三篇的數學是關鍵門檻，建議搭配第 11、12 章的實作活動來建立直覺。
- 專題 / 研究路線：先快速瀏覽第一至三篇建立共同語言，再依研究方向深入——做光達選第五篇、做視覺選第六篇、做微型平台選第七篇、做 AI 整合選第九篇，最後以第 31 章的專題規劃收斂。

無論選擇哪條路線，本書建議的學習節奏都相同：先建立直覺、再理解原理、然後動手驗證、最後刻意尋找失效邊界。第 28.7 節會把這個節奏展開成五個具體階段。

## 小結

1. SLAM（同步定位與建圖）要解決的核心問題是：機器人在沒有地圖、不知道自己位置的情況下，一邊移動、一邊建立地圖、一邊修正自身位姿。
2. 定位需要地圖、建圖需要定位，兩者形成雞生蛋、蛋生雞的循環依賴；SLAM 的解法是把兩者視為同一個估測問題，利用環境觀測持續互相修正，並以閉環偵測攤平長期累積誤差。
3. 機器人的感測器分為本體感測（里程計、IMU，連續但漂移）與外部感測（相機、光達、無線電、地磁，提供環境參考但各有弱點）；SLAM 的精髓在於感測器融合、截長補短。
4. 自動駕駛、無人機、掃地機器人與 AR / VR 四大應用共用同一套 SLAM 地基，差別只在感測器組合、運算資源與精度要求。
5. SLAM 系統的輸出包含軌跡、2D 佔據格地圖、3D 點雲、語意地圖與拓樸地圖五大形式，各自對應不同的應用需求。
6. 數學上，SLAM 是在給定觀測 z 與控制 u 之下，估測軌跡 x 與地圖 m 的聯合機率分布；現代系統以「前端—後端—閉環」的架構求解。

下一章，我們將走訪 MIT、CMU、Stanford、ETH / UZH 與 Bonn / Freiburg 等世界知名大學的 SLAM 課程，看看國際一流學府如何安排這門學問的學習路徑，並從中提煉出本書「感測器 → 數學 → 前端 → 後端 → 閉環 → 專題」的教學主軸。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

## 第 2 章

## 世界知名大學 SLAM 課程導覽

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

## 本章在全書的位置

第 1 章建立了 SLAM 的問題意識，本章則回答一個實際的問題：這門學問，世界上最好的學校是怎麼教的？這不只是見聞的補充——本書的整體架構正是從這些課程的共同骨架提煉而來，因此讀懂本章，等於拿到了理解後續二十九章編排邏輯的鑰匙。

**圖 2-0 第 2 章與全書的連結地圖**

【承接前章】第 1 章的六大感測器、雞生蛋問題、五種地圖表示，在本章各校課程中都能找到對應的單元。

【本章負責】導覽五所頂尖大學的 SLAM 課程，並提煉出它們的共同架構。

【往下延伸】

2.1 MIT VNAV → 第 7 章 IMU 預積分、第 12 章流形最佳化、第 23 章視覺慣性 SLAM

2.2 CMU 16-833 → 第 11 章機率濾波、第 12 章因子圖與 GTSAM

2.3 Stanford → 第 3 章導航系統架構、第 29 章規劃與控制

2.4 ETH / UZH → 第 9 章相機模型、第 21 章對極幾何、第 26 章無人機 VIO

2.5 Bonn / Freiburg→ 第 11 章粒子濾波、第 13 章佔據格、第 14 章 Gmapping

2.6 共同架構 → 本書九篇的編排依據

【先備知識】無。本章是導覽性質，不涉及數學推導。

## 學習目標

工欲善其事，必先利其器；學欲深其道，必先觀其路。本章帶領同學「站在巨人的肩膀上」，導覽五所世界頂尖大學的 SLAM 相關課程，看看國際一流學府如何安排這門學問的學習路徑。學完本章之後，同學應能夠達成下列目標：

7. 說出 MIT、CMU、Stanford、ETH / UZH、Bonn / Freiburg 五個學術重鎮的 SLAM 課程名稱、主授教授與課程定位。
8. 比較各校課程的切入角度：視覺導航、機率推論、系統整合、視覺慣性、機率建圖，並說明其異同。
9. 歸納 SLAM 教材的共同架構：感測器 → 數學 → 前端 → 後端 → 閉環 → 專題，並對應到本書的九篇章節。
10. 知道各校公開的免費學習資源（OpenCourseWare、YouTube 課程影片、公開講義）在哪裡、如何取用。
11. 依照自己的程度與目標，規劃一條適合自己的 SLAM 自學路線。

## 核心概念

| 關鍵詞 | 英文 / 代號 | 概念說明 |
|---|---|---|
| 視覺導航 | Visual Navigation | 以相機為主要感測器的機器人導航技術，MIT VNAV 課程的主軸。 |
| 狀態估測 | State Estimation | 從帶雜訊的量測中推算系統狀態（位姿、速度、地圖）的數學方法。 |
| 機率圖模型 | Probabilistic Graphical Model | 以節點與邊表示隨機變數關係的模型，是現代 SLAM 後端的理論語言。 |
| 因子圖 | Factor Graph | 一種機率圖模型，把量測表示為「因子」，CMU 16-833 的核心工具。 |
| 視覺里程計 | Visual Odometry, VO | 只用相機影像逐幀估測相機運動的技術，UZH 課程的期末專題。 |
| 視覺慣性里程計 | VIO | 相機加 IMU 融合的里程計，手機 AR 與無人機的核心技術。 |
| 流形最佳化 | Manifold Optimization | 在旋轉群等彎曲空間上做最佳化的數學技術，MIT VNAV 的特色主題。 |

| 關鍵詞 | 英文 / 代號 | 概念說明 |
| --- | --- | --- |
| 佔據格建圖 | Occupancy Grid Mapping | Freiburg / Bonn 課程系統化整理的經典建圖方法。 |
| 開放式課程 | OpenCourseWare, OCW | 大學免費公開的完整課程教材，MIT OCW 是最著名的代表。 |
| 大規模開放線上課程 | MOOC | 如 edX、Coursera 上的線上課程，ETH 自主移動機器人課程即有 MOOC 版。 |

## 2.1 MIT VNAV：視覺導航、自駕車、無人機與最佳化

麻省理工學院（MIT）航空航天系開設的 16.485「自主載具視覺導航」（Visual Navigation for Autonomous Vehicles，簡稱 VNAV），由 Luca Carlone 教授主持的 SPARK 實驗室開授，是全世界最具代表性的視覺導航研究所課程之一。Carlone 教授本人是 SLAM 領域的重量級學者——第 1 章延伸閱讀中那篇 2016 年的 SLAM 回顧論文，他正是主要作者之一。

### 課程主軸與特色

VNAV 的課程設計有一條清晰的邏輯線：從幾何出發、以最佳化收斂、用無人機驗證。課程內容大致分為四大塊：

- 三維幾何基礎：旋轉表示（旋轉矩陣、四元數）、剛體變換、相機模型與二視圖幾何——對應本書第 10 章與第 21 章。
- 流形上的最佳化（Optimization on Manifolds）：旋轉不是普通的向量，不能直接做加減，必須在「流形」這種彎曲空間上進行最佳化。這是 VNAV 最具特色的數學主題，也是理解現代 SLAM 後端的鑰匙——本書第 12 章會以較平易的方式介紹其精神。
- 視覺前端：特徵提取與匹配、視覺里程計、地點辨識（Place Recognition）與閉環偵測——對應本書第 20 至 22 章。
- SLAM 後端與系統：位姿圖優化、Bundle Adjustment、視覺慣性融合，以及語意理解的前沿延伸——對應本書第 12、23、32 章。

### 教學方式：無人機團隊專題

VNAV 最令人印象深刻的是它的實作安排：學生分組操作真實無人機，逐週完成從姿態控制、軌跡追蹤、視覺特徵、視覺里程計到完整導航系統的實驗，期末以無人機競賽式專題收尾。這種「每一週的數學都立刻在飛行器上驗證」的設計，正是本書第八篇實作章節效法

的精神。

VNAV 的完整教材（講義、作業、實驗指引）已在 MIT OpenCourseWare 免費公開（課號 16.485，2020 年秋季版），是自學視覺導航最完整的免費資源之一。對本書讀者而言，建議在讀完第三篇數學基礎後，再回頭挑戰 VNAV 的講義，會有事半功倍的效果。

## 2.2 CMU Robot Localization and Mapping：機器人定位、建圖與後端最佳化

卡內基美隆大學（CMU）機器人研究所（Robotics Institute）是全球歷史最悠久、規模最大的機器人研究機構，其 16-833「機器人定位與建圖」（Robot Localization and Mapping）由 Michael Kaess 教授主講。Kaess 教授是 SLAM 後端演算法的代表性人物——現代圖優化 SLAM 廣泛使用的 iSAM / iSAM2 增量式平滑演算法與 GTSAM 函式庫，正是出自他與 Frank Dellaert 教授的研究團隊。

### 課程主軸與特色

16-833 的課程描述開宗明義指出：定位與建圖是行動機器人在真實世界運作的基礎能力，而比單獨解決這兩個問題更具挑戰的，是它們的組合——SLAM；穩健的解法必須能處理感測量測中固有的不確定性，同時即時提供定位與地圖估測。課程因此把重心放在「線性代數與機率圖模型交會處的高效機率推論演算法」，內容包含：

- 貝氏濾波家族：卡爾曼濾波、擴展卡爾曼濾波（EKF-SLAM）、粒子濾波與 FastSLAM——對應本書第 11、14 章。
- 平滑與圖優化：因子圖（Factor Graph）、最小平方、iSAM 增量式求解——對應本書第 12 章。
- 資料關聯與閉環：量測到底對應地圖中哪個地標？這是 SLAM 最容易出錯的環節。
- 最先進系統導讀：課程後段帶學生閱讀當代 SLAM 系統論文，理解理論如何落地。

### 與 MIT VNAV 的對照

如果說 VNAV 是「從視覺與幾何切入」，16-833 就是「從機率與推論切入」：VNAV 花大量篇幅講相機與流形，16-833 則把濾波、因子圖與稀疏線性代數講到透徹。兩門課合起來，恰好覆蓋現代 SLAM 前端與後端的完整知識版圖——這也是本書把第二篇（感測器）與第三篇（數學）分開安排、再於第四至六篇整合成系統的原因。

## 2.3 Stanford Robot Autonomy：自主機器人的定位、感知、SLAM、規劃

史丹佛大學的「機器人自主原理」（Principles of Robot Autonomy，課號 AA274A / CS237A）由航太系與電腦科學系合開，Marco Pavone 教授等人主持的自主系統實驗室（ASL）負責授課。與前兩門課不同，這門課的定位不是「把 SLAM 一個主題挖深」，而是「把自主機器人整個系統走一遍」。

### 課程主軸與特色

Stanford 這門課涵蓋自主機器人的完整技術堆疊，SLAM 是其中承上啟下的一環：

- 運動與控制：運動學模型、軌跡追蹤、回授控制——機器人「會動」的基礎。
- 感知：相機模型、影像特徵、物件偵測與機器學習感知——機器人「看得見」。
- 定位與 SLAM：貝氏濾波、蒙地卡羅定位、EKF-SLAM——機器人「知道自己在哪」。
- 規劃與決策：路徑規劃（A*、RRT）、軌跡最佳化、任務決策——機器人「知道要去哪、怎麼去」。
- 系統整合：全學期以 ROS 為骨幹，期末在模擬與實體機器人上整合出完整的自主導航系統。

### 對本書的啟示

Stanford 的課程設計回答了一個重要問題：SLAM 學了要幹嘛？答案是——SLAM 是

自主系統的中樞，往下承接感測器資料，往上支撐路徑規劃與任務執行。本書第 3 章的「導航系統總覽」與第 29 章的「Navigation Stack 導航」正是採用這個系統觀：讓同學永遠知道，眼前這個演算法在整台機器人中扮演什麼角色。這種「以系統帶演算法」的教學法，特別適合高中與大學部的專題導向課程。

## 2.4 ETH Zürich / UZH：視覺慣性里程計、VIO、MAV 無人機導航

瑞士蘇黎世聯邦理工學院（ETH Zürich）與蘇黎世大學（UZH）地理相鄰、師資互通，共同構成歐洲行動機器人研究的心臟地帶。這裡有兩條互補的課程脈絡，分別代表「系統派」與「視覺派」。

### ETH：自主移動機器人（Autonomous Mobile Robots）

由 Roland Siegwart 教授領銜的 ETH 自主系統實驗室（ASL）長期開設「自主移動機器人」課程，其同名教科書（MIT Press 出版，Siegwart、Nourbakhsh 與 Scaramuzza 合著）是行動機器人領域最廣泛使用的教材之一，涵蓋運動學、感知、定位與導航規劃。這門課也推出了 edX 平台上的 MOOC 版本，讓全世界的學習者免費修習。此外，ASL 在視覺慣性估測上的研究成果——ROVIO、OKVIS 等開源系統——已成為無人機與行動裝置定位的重要基礎建設，本書第 23、26 章會再遇到它們。

### UZH：行動機器人視覺演算法（Vision Algorithms for Mobile Robotics）

UZH 機器人與感知研究群（RPG）的 Davide Scaramuzza 教授，是視覺里程計與事件相機（Event Camera）研究的國際領軍人物。他開設的「行動機器人視覺演算法」課程，內容涵蓋影像形成、濾波、特徵提取、多視圖幾何、稠密重建、追蹤、影像檢索、事件視覺、視覺慣性里程計與 SLAM——課程網站直接點名，這些正是 Apple ARKit、Microsoft HoloLens、Meta Quest 與 NASA 火星直升機背後的演算法。

這門課最精彩的設計是「積木式期末專題」：每一講之後都有對應的實作練習，學生逐

週實作視覺里程計的一個模組（特徵提取、匹配、位姿估測、三角測量……），期末把所有積木組裝成一條完整可跑的視覺里程計管線，並在 KITTI 等真實資料集上驗證。所有課程影片與講義都在 RPG 網站與 YouTube 免費公開。本書第九章、第 20 至 21 章的知識架構，很大程度上參考了這門課的鋪陳順序。

## 2.5 University of Bonn / Freiburg：Robot Mapping 與 Cyrill Stachniss SLAM 課程

德國的弗萊堡大學（University of Freiburg）與波昂大學（University of Bonn）是機率機器人學（Probabilistic Robotics）的發源重鎮。Wolfram Burgard 教授（《Probabilistic Robotics》作者之一）在弗萊堡長期開設「行動機器人導論」與「機器人建圖」（Robot Mapping）課程；其學生 Cyrill Stachniss 教授其後在波昂大學延續並發揚這條教學路線，開設以 SLAM 與環境建模為主軸的完整課程。

### 課程主軸與特色

Freiburg / Bonn 路線的最大特色，是把機率方法講得極其系統化、極其循序漸進：

- 機率基礎：貝氏法則、運動模型、觀測模型——對應本書第 11 章。
- 濾波式 SLAM：EKF-SLAM、UKF、粒子濾波與 FastSLAM、Gmapping 的 RBPF 原理——對應本書第 11、14 章。
- 圖優化式 SLAM：最小平方、位姿圖、閉環與強健化——對應本書第 12 章。
- 建圖方法：佔據格建圖、雷射掃描匹配、地圖表示——對應本書第 13 章。

### 對自學者最友善的資源

Stachniss 教授把歷年課程完整錄影上傳 YouTube 免費公開，從五分鐘的單一觀念短片到九十分鐘的完整講課都有，搭配動畫與板書推導，被全球 SLAM 學習者公認為「最適合自學入門」的影音資源。特別值得一提的是，本書第 14 章的主角 Gmapping 正是出自 Stachniss 與 Grisetti、Burgard 的研究成果——讀完第 14 章後去看原作者親自講解，是最

原汁原味的學習體驗。

## 2.6 各校課程與本書章節的對照

在歸納共同架構之前，先做一件對同學很實用的事：把五所學校的核心單元，逐一對應到本書的章節。這張表有兩個用途——一是當你想深入某個主題時，知道該去看哪一門國際課程；二是當你修習這些線上課程時，知道可以回頭參照本書的哪一章。

**表 2-1 五校課程單元與本書章節對照**

【MIT VNAV：視覺導航與最佳化】

| | | |
|---|---|---|
| 影像形成與相機模型 | → | 第 9 章 |
| 流形上的最佳化 | → | 第 10 章（旋轉表示）、第 12 章（圖優化） |
| IMU 預積分 | → | 第 7.5 節 |
| 視覺慣性里程計 | → | 第 23 章 |

【CMU 16-833：定位、建圖與後端】

| | | |
|---|---|---|
| 貝氏濾波與粒子濾波 | → | 第 11 章 |
| EKF-SLAM | → | 第 11.6 節 |
| 因子圖與 iSAM2 | → | 第 12.6 節、第 27 章（多機器人） |
| 閉環偵測 | → | 第 20.7 節、第 22.5 節 |

【Stanford Robot Autonomy：系統整合】

| | | |
|---|---|---|
| 機器人系統架構 | → | 第 3 章 |
| 感知與狀態估測 | → | 第二篇、第 11 章 |

路徑規劃與控制 → 第 3.4 節、第 29.6 節

【ETH / UZH：視覺與無人機】

特徵偵測與匹配 → 第 20 章

多視圖幾何 → 第 21 章

視覺慣性融合 → 第 23 章

微型飛行器導航 → 第 26 章

【Bonn / Freiburg：機率建圖】

佔據格地圖 → 第 13 章

掃描匹配與 ICP → 第 13.3 節、第 17.3 節

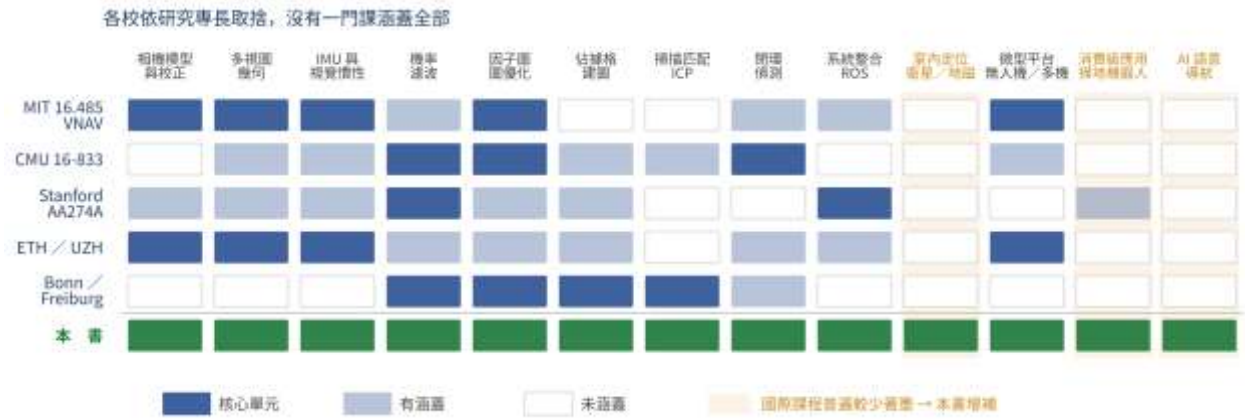
Gmapping 與 RBPF → 第 14 章

圖優化 SLAM → 第 12 章、第 15 章

觀察這張表，同學會發現一個現象：沒有任何一所學校的課程涵蓋全部主題。MIT 偏視覺慣性與最佳化、Bonn 偏機率建圖、Stanford 偏系統整合。這是合理的——一門十四週的課無法窮盡整個領域，各校自然依其研究專長取捨。

本書的編排策略因此是「取各校之長並補其缺口」：以 Bonn 的機率建圖為第三、四篇的骨幹、以 MIT 與 UZH 的視覺慣性為第六篇的主線、以 CMU 的因子圖為第十二章的核心、以 Stanford 的系統觀貫穿第三章與第八篇，再補上台灣課程實際需要卻在國際課程中較少著墨的部分——例如第 4 章的室內定位、第 6 章的地磁導航、第 16 章的掃地機器人，以及第九篇的 AI 語意導航。

**圖 2-1 五校課程的涵蓋範圍比較**




把表 2-1 的對照結果畫成矩陣，前一段文字所說的現象就一目了然了：五列代表五所學校，十三欄代表 SLAM 的主要主題，深色是該課程的核心單元、淺色是有涵蓋、空白是未涵蓋。

橫著看，每一所學校都有明顯的重心：MIT 與蘇黎世兩校的深色集中在左側的相機模型、多視圖幾何與視覺慣性；Bonn / Freiburg 的深色集中在中段的機率濾波、佔據格建圖與掃描匹配；CMU 落在機率與因子圖；Stanford 則是唯一把系統整合與 ROS 當成核心的一門。這正是 2.6 節所說的「依研究專長取捨」。

直著看則更有意思。有四欄幾乎整片空白——室內定位與衛星地磁、消費級應用、AI 語意導航，以及微型平台（只有兩校涵蓋）。這不是這些主題不重要，而是它們要嘛屬於產業而非研究前沿、要嘛演進太快而尚未穩定進入十四週的課程。

最後一列是本書。它填滿了全部十三欄，但同學應該理解這不代表本書比這些課程更深——恰恰相反，一本教科書的任務是建立完整的骨架與共同語言，而國際課程提供的是某個領域的縱深。兩者的正確用法是互補：以本書鋪面，以課程挖深。

## 2.7 SLAM 教材的共同架構：感測器 → 數學 → 前端 → 後端 → 閉環 → 專題

走訪五校之後，我們把鏡頭拉遠，會發現一個有趣的現象：儘管切入角度各異——MIT 從幾何、CMU 從機率、Stanford 從系統、蘇黎世從視覺、德國從建圖——這些課程的知識

骨架卻驚人地一致，都可以歸納為六個階段：

**圖 2-2 SLAM 教材的共同架構**

① 感測器（Sensors）：認識相機、光達、IMU 等資料來源與其誤差特性

↓

② 數學（Math）：座標變換、機率、濾波、最佳化——處理不確定性的語言

↓

③ 前端（Front-End）：特徵提取、資料關聯、掃描匹配、里程計

↓

④ 後端（Back-End）：濾波或圖優化，求出最一致的軌跡與地圖

↓

⑤ 閉環（Loop Closure）：辨識重訪地點，修正全域累積誤差

↓

⑥ 專題（Projects）：在真實或模擬平台上整合驗證，完成完整系統

本書的九篇結構，正是把這六階段展開並在地化的結果：第二篇對應「感測器」、第三篇對應「數學」、第四至六篇把「前端、後端、閉環」落實在 2D 光達、3D 光達與視覺三種系統上、第七至八篇對應「專題」、第九篇則延伸到各校課程尚在快速演進的 AI 語意導航前沿。

| 學校 / 課程 | 主授教授 | 切入角度 | 代表特色 | 對應本書 |
| --- | --- | --- | --- | --- |
| MIT 16.485 VNAV | Luca Carlone | 幾何與最佳化 | 流形最佳化、無人機專題 | 第 10、12、21 章 |

| 學校 / 課程 | 主授教授 | 切入角度 | 代表特色 | 對應本書 |
|---|---|---|---|---|
| CMU 16-833 | Michael Kaess | 機率推論 | 因子圖、iSAM / GTSAM | 第 11、12 章 |
| Stanford AA274A | Marco Pavone 等 | 系統整合 | ROS 全堆疊、期末整合 | 第 3、28、29 章 |
| ETH 自主移動機器人 | Roland Siegwart | 行動機器人總論 | 經典教科書、edX MOOC | 第 3、7 章 |
| UZH 視覺演算法 | Davide Scaramuzza | 視覺與 VIO | 積木式 VO 專題、事件相機 | 第 9、20、21 章 |
| Bonn / Freiburg | Stachniss、Burgard | 機率建圖 | YouTube 全公開、Gmapping | 第 11、13、14 章 |

**表 2-2　五校 SLAM 課程總覽與本書章節對應**

## 公式或流程

### 一、以「估測問題」統一五校課程

五校課程雖然風格迥異，但都圍繞著第 1 章介紹的同一條核心公式。回顧線上 SLAM 的機率定義：

$$p(x(t), m \mid z(1:t), u(1:t))$$

五校課程的差別，其實只是「攻打這條公式的不同兵法」：

- CMU 與 Freiburg / Bonn 專攻「如何求解這個機率分布」：濾波法（第 11 章）沿時間遞迴更新，圖優化法（第 12 章）把它轉成最小平方問題求解。
- MIT 與 UZH 專攻「觀測 z 從哪裡來」：相機影像如何變成幾何約束——特徵、對極幾何、重投影誤差（第 20、21 章）。
- Stanford 專攻「解出來之後怎麼用」：位姿與地圖如何餵給路徑規劃與控制，構成完整自主系統（第 3、29 章）。

### 二、SLAM 自學路線規劃流程

綜合五校資源，本書建議的自學路線如下，同學可依自己的起點選擇入口：

**圖 2-3 SLAM 自學路線建議流程**

起點判斷：我有程式與線性代數基礎嗎？

├─ 還沒有 → 先看 ETH 自主移動機器人 MOOC（觀念導向、門檻最低）

└─ 已具備 → 進入下一步

↓

想走視覺路線 → UZH 課程影片 + 逐週實作視覺里程計積木

想走機率路線 → Stachniss YouTube 課程 + 本書第三篇推導

↓

進階深化 → MIT VNAV OCW 講義（流形最佳化） + CMU 16-833（因子圖）

↓

系統整合 → 以 ROS 完成本書第八篇實作與第 31 章專題

## 2.8 開放教育的演進：從 MOOC 到具身智慧課程

本章介紹的課程有一個共同特徵：教材公開。這件事在今天看來理所當然，但它其實是過去十餘年才發生的變化，而且至今仍在演進。理解這個演進，對同學規劃自己的學習路徑很有幫助。

### 三個階段的變化

知識取得的方式在過去十幾年發生了根本改變。這個歷程大致始於 2011 年前後，當時史丹佛的教授們把人工智慧課程透過線上形式向所有人開放；在那之前，人工智慧一般只能透過數量有限的大學課程、或書籍與論文來學習，而且這些資源主要集中在已開發國家，新

興市場的學習者難以取得。大規模開放線上課程的出現，開啟了所謂「知識民主化」的進程。

對 SLAM 這類需要大量背景知識的領域，這個變化尤其關鍵。在 2010 年代之前，想學 SLAM 幾乎只能進入少數幾個研究室；今天，任何一位台灣的學生只要有網路，就能取得與麻省理工學生相同的講義與作業。本書案例 2-2 提到的 Stachniss 教授把整套課程放上影音平台，正是這個趨勢的代表。

### 第二階段：硬體導向的開放課程

純影片課程有一個限制：學生看得懂理論，卻沒有機器人可以驗證。因應這個缺口，出現了結合實體平台的開放課程。例如 2021 年三月，Duckietown 基金會發布了第一個以硬體為基礎的人工智慧與機器人大規模開放線上課程，名為「以 Duckietown 學自駕車」，在公開平台上免費提供；該課程讓學習者親手操作，使真實的機器人自己做決策並完成廣泛定義的任務，從理論、實作、到在模擬與真實機器人上部署，逐步進行；並為此從零設計了一個專門支援這種學習體驗的機器人平台。

這個模式與本書第八篇的設計理念一致：先在模擬中把流程與參數調通，再搬到便宜的實體硬體上驗證。第 28.1 節建議的容器化統一環境、第 29 章的「先模擬再實體」路徑，都是同一個思路。

### 第三階段：具身智慧進入課程

近兩年課程內容出現了新的變化。大型語言模型的突破正被應用於機器人領域，具身智慧讓機器人能理解自然語言指令並轉化為實體動作；而 Google DeepMind 與 OpenAI 等組織正在開發能從龐大資料集學習通用操作技能的機器人基礎模型，理解如何為特定應用微調這些模型，將成為必備能力。

與此同時，另一個方向也進入課程：由於雲端連線的限制，裝置端人工智慧推論的需求持續上升，現代課程越來越常涵蓋模型最佳化技術（量化、剪枝、知識蒸餾）與邊緣人工智慧框架及硬體加速。這正是本書第 25 章（毫瓦級平台）與第 33.8 節（邊緣部署）所討論的

內容——本書把這兩個主題納入，正是為了對齊這個教學趨勢。

### 對學習方法的三點啟示

1. 模擬優先：有研究者指出，模擬專案能以硬體學習約十分之一的成本，教會約八成的必要概念；等基礎穩固後，硬體工作會進行得更有效率，而過早嘗試複雜專案則會導致挫折。這與本書第八篇的建議完全一致。
2. 漸進式挑戰：建議的學習結構是逐級推進——基本運動（讓機器人走出方形、圓形、8 字形）、感測器整合（加入障礙偵測與避障）、環境建圖（以 SLAM 技術建立空間地圖）、物件操作（視覺偵測與抓取）、自主任務（把各項能力整合為完整任務）。對照本書，這五級分別對應第 3 章、第 8 章、第 13 至 15 章、第 24 章與第 31 章。
3. 線上課程與教科書互補：線上課程長於動態演示與最新進度，教科書長於系統性與可反覆查閱。本書建議的用法是：以本書建立架構、以線上課程補充直覺、以開源專案驗證理解——這三者的分工，正是第 28.6 節閱讀開源專案方法的延伸。

## 案例

### 案例 2-1　從課堂作業到改變產業：GTSAM 的故事

CMU 16-833 課堂上教的因子圖與增量式平滑，並不只是紙上談兵。Kaess 教授與 Dellaert 教授開發的 GTSAM（Georgia Tech Smoothing and Mapping）函式庫，把這些理論包裝成高效、開源的 C++ 工具箱，如今已是全球機器人公司與研究室的標準配備——從水下機器人、自駕車到 AR 裝置的後端最佳化，處處可見它的身影。這個案例告訴我們：一門課的核心作業（在 16-833 中，學生要親手實作因子圖求解器），可能就是未來產業基礎建設的雛形。本書第 12 章介紹 g2o、Ceres 與 GTSAM 時，同學將親手使用這個「從課堂長出來」的工具。

### 案例 2-2　YouTube 上的 SLAM 教授：知識公開的力量

Stachniss 教授從 2013 年起把 Freiburg 時期的 Robot Mapping 課程完整錄影公開，其後在波昂持續更新，累積數百支影片、被觀看數百萬次。世界各地無數自學者——包括許多台灣的研究生與工程師——都是靠這些免費影片入門 SLAM 的。同樣地，MIT 把 VNAV 完整放上 OCW、UZH 公開所有講義與影片、ETH 推出 MOOC。這股「頂尖大學知識公開」的浪潮，意味著學習 SLAM 的門檻已從「能否進入名校」變成「有沒有一份好的學習地圖與持續的毅力」。本書正是希望扮演那份用繁體中文寫成的學習地圖。

## 案例 2-3　台灣學生的視角：從大學教室到國際課程

對台灣的學習者而言，五校課程是很好的「進階疊代」素材，但直接啃原文教材常會遇到兩個障礙：一是數學符號與英文術語的雙重門檻，二是缺乏在地可操作的實驗場域。本書的策略是「在地打底、國際接軌」：先用中文與熟悉的場景（教室建圖、校園巡檢、掃地機器人）把觀念與實作打穩，每章的延伸閱讀再指向對應的國際課程單元。以第 14 章 Gmapping 為例：先在大學教室用小車完成建圖實作，再回頭看 Stachniss 講解 RBPF 的影片，多數同學會發現原本抽象的粒子濾波突然「看得見了」。

## 案例 2-4　為什麼還需要教科書：MOOC 時代的大學課程

本章介紹了大量免費的線上資源，這自然會引出一個尖銳的問題：既然頂尖大學的課程都能免費看，為什麼還要修課？為什麼還要有這本書？

這個問題在教育界確實被嚴肅討論。有學者指出，隨著教學數位轉型快速推進，大學已開始面臨來自大規模開放線上課程的重大競爭，因此必須讓自己與這些課程有所區隔，才能向學生說明三到五年學位的附加價值何在。

筆者認為，教科書與課堂在這個時代的價值，至少有四個是線上影片難以取代的：

- 在地化：國際課程以英文授課、以歐美場域為例。本書刻意採用台灣的專業術語（第 1 章的用語討論）、以大學校園與台灣的高中教室為場域、並納入台灣特別關切的主題（如第 6 章地磁導航在鋼構建築中的表現、第 16 章掃地機器人）。

- 系統性：線上課程各自獨立，學生需自行拼湊。本書的九篇架構、每章的「本章在全書的位置」、以及跨章的交叉引用，正是為了提供這種整體視野。
- 可查閱性：影片難以快速回查某個公式或參數。教科書的索引、目錄、章節編號，在實作卡關時遠比重看影片有效率。
- 失敗經驗的傳承：這是本書最刻意經營的部分。國際課程通常展示成功的結果，而本書每一章都討論「常見失敗」——第 14.6 節的四大失敗模式、第 26.5 節的安全原則、第 29.7 節的除錯順序。這些來自真實教學現場的經驗，是任何影片都不會告訴你的。

所以本書的定位很明確：它不與線上課程競爭，而是作為它們的地圖與註解。同學若能同時使用兩者——以本書建立骨架、以國際課程補充血肉——得到的會比任何單一來源都多。這也是案例 2-2 強調「知識公開的力量」時，真正想傳達的訊息：資源不是問題，如何組織資源才是。

## 實作活動

### 實作 2-1　國際 SLAM 課程資源探索與比較報告

目的：親手找到並瀏覽五校公開資源，建立自己的學習資源地圖，並練習閱讀英文課程網頁的能力。

**步驟**

5. 分組（每組 2 至 3 人），每組認領一所學校：MIT VNAV、CMU 16-833、Stanford AA274A、UZH 視覺演算法、Bonn Stachniss 課程。
6. 搜尋並開啟該課程的官方網站或 OCW / YouTube 頁面，找出：課程名稱與課號、主授教授、最近一次開課的主題週次表（Syllabus）。

7. 挑選一週你們「看得懂標題」的主題（例如 Kalman Filter、Feature Extraction），瀏覽該週講義的前五頁或影片的前十分鐘。
8. 製作一頁簡報回答：這門課假設學生已經會什麼？最先教什麼？作業或專題長什麼樣子？哪一部分對應本書的哪一章？
9. 各組上台報告三分鐘，全班合力完成一張「五校課程比較表」，貼在教室或課程網站上，作為整學期的參考地圖。

**討論問題**

6. 五校課程中，哪一門的入門門檻最低？哪一門最需要數學基礎？你的判斷依據是什麼？
7. 如果只能追蹤一個免費資源搭配本書自學，你會選哪一個？為什麼？
8. （大學部）比較 VNAV 與 16-833 的專題要求，哪一種評量方式更能檢驗「真的學會了 SLAM」？

**延伸挑戰（大學部）**

從 UZH 課程網站下載第一份實作練習（通常是特徵提取或相機模型），嘗試在自己的電腦上執行，並記錄遇到的環境設定問題與解決過程——這份紀錄將在第 28 章建置 SLAM 開發環境時派上用場。

## 延伸閱讀

4. MIT OpenCourseWare, 16.485 Visual Navigation for Autonomous Vehicles (VNAV), Fall 2020.（完整講義、作業與無人機實驗指引，免費公開）
5. CMU 16-833 Robot Localization and Mapping 課程網站（cs.cmu.edu/~kaess/teaching/16833）.（歷年課綱與閱讀清單）
6. Stanford AA274A / CS237A Principles of Robot Autonomy I 課程網站與公開講義.（自主機器人全堆疊課程）

7. R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, Introduction to Autonomous Mobile Robots, 2nd ed., MIT Press, 2011.（ETH 課程指定教科書，另有 edX MOOC 版本）
8. UZH Robotics and Perception Group, Vision Algorithms for Mobile Robotics 課程頁面（rpg.ifi.uzh.ch/teaching.html）.（全部講義與影片公開）
9. Cyrill Stachniss, Robot Mapping / SLAM 系列課程影片, YouTube（頻道：Cyrill Stachniss）.（最適合自學的 SLAM 影音資源）
10. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005.（Freiburg 路線的理論總集成，本書第三篇主要參考）
11. D. Scaramuzza and F. Fraundorfer, "Visual Odometry: Tutorial," IEEE Robotics & Automation Magazine, 2011.（視覺里程計的經典入門教程，分上下兩部）

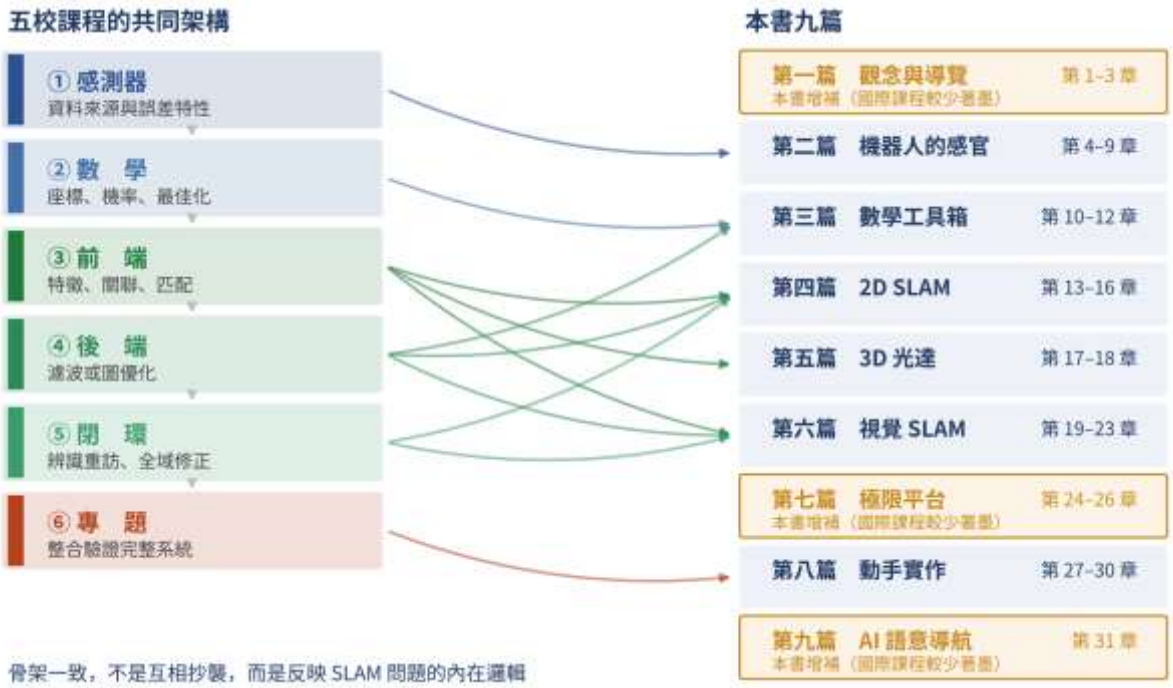
## 從課程導覽到系統架構

本章的最後，把視線轉回本書自身的旅程。

第 2.7 節歸納出五校課程的共同架構：感測器 → 數學 → 前端 → 後端 → 閉環 → 專題。這個架構之所以反覆出現，不是因為互相抄襲，而是因為它反映了 SLAM 問題的內在邏輯——你必須先有資料（感測器），才能談如何處理資料（數學）；必須先能估計相鄰時刻的關係（前端），才能談如何全域一致（後端）；而閉環是後端能力的極致展現，專題則是所有能力的整合驗收。

本書的九篇正是這個邏輯的展開，只是把每個環節都放大成完整的篇幅：

**圖 2-4 共同架構與本書九篇的對應**




【本書增補的部分】

第一篇　觀念與導覽　（國際課程通常直接進入技術）

第七篇　極限平台　（微型機器人、無人機、多機協作）

第九篇　AI 語意導航　（對齊具身智慧的教學趨勢）

下一章我們就要進入這個架構的第一站——不是感測器，而是比感測器更上位的東西：機器人導航系統的完整架構。第 3 章會回答「SLAM 在整個導航系統中處於什麼位置」，並介紹反應式、審議式與混合式三種架構、代價地圖與路徑規劃、以及輪型、履帶、四足、無人機等不同平台的導航需求差異。有了那張系統藍圖，第二篇的六種感測器才知道各自該安裝在什麼位置上。

## 小結

8. MIT VNAV 以幾何與流形最佳化為主軸，用無人機專題貫穿全學期，完整教材已在 OCW 免費公開。
9. CMU 16-833 把 SLAM 定位為「感測不確定性下的即時估測問題」，專攻濾波、因子圖與圖優化後端，是 iSAM / GTSAM 的發源地。
10. Stanford 機器人自主課程以系統整合為特色，SLAM 被放在「感知—定位—規劃—控制」的完整堆疊中教授，全程以 ROS 實作。
11. ETH 的自主移動機器人課程（含 edX MOOC）提供最平易的總論入口；UZH Scaramuzza 課程則以積木式視覺里程計專題深入視覺與 VIO。
12. Freiburg / Bonn 路線把機率建圖講得最系統化，Stachniss 教授的 YouTube 課程是公認最適合自學的免費影音資源。
13. 五校課程共享同一個知識骨架：感測器 → 數學 → 前端 → 後端 → 閉環 → 專題；本書九篇即依此架構展開，並以台灣校園場域在地化。

下一章，我們將把鏡頭從「課程」轉回「機器人本身」，綜覽機器人導航系統的完整架構——Sense–Think–Act 循環、地圖與定位的分類、以及 SLAM 在整個導航系統中的位置，為第二篇的感測器之旅做好準備。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

## 第 3 章

## 機器人導航系統總覽

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

## 本章在全書的位置

第 1 章說明了 SLAM 要解決什麼問題，第 2 章看了世界怎麼教這門學問。但這裡有一個容易被忽略的前提：SLAM 從來不是獨立存在的技術，它是導航系統中的一個環節。本章要畫出那張完整的系統藍圖，讓同學知道 SLAM 的上游是什麼、下游又接到哪裡去。

這一章是第一篇的收尾，也是理解全書架構的關鍵：第二篇的感測器要裝在這個系統的哪個位置、第三篇的數學支撐哪個模組、第八篇的實作又要把哪些元件接起來——答案都在本章的架構圖裡。

**圖 3-0　第 3 章與全書的連結地圖**

【承接前章】第 1 章的五種地圖表示與六大感測器，在本章被放進完整的系統架構中定位。

【本章負責】建立導航系統的整體藍圖：感知、決策、行動如何串接，SLAM 位於何處。

【往下延伸】

3.1 感知決策行動　→　貫穿全書的分層思想（第 15、18、26、32 章反覆出現）

3.2 地圖的種類　→　第 13 章佔據格、第 17 章點雲、第 20 章特徵地圖、第 32 章語意地圖

3.3 定位的種類　→　第 11 章重定位與綁架問題、第 23 章多地圖、第 29.5 節 AMCL

3.4 導航系統架構　→　第 29.6 節 Nav2 完整實作、第 31 章專題整合

3.5 ROS 的角色　→　第八篇全篇（第 27–30 章）

3.6 平台差異　→　第 16 章掃地機器人、第 18 章機器狗、第 26 章無人機

【先備知識】第 1 章的基本觀念。本章仍不涉及數學。

## 學習目標

前兩章我們認識了 SLAM 這個核心問題與世界名校的教學路線；本章把鏡頭拉高，綜覽一台自主導航機器人的完整系統架構，讓同學在深入各項技術之前，先擁有一張「全系統的地圖」。學完本章之後，同學應能夠達成下列目標：

12. 以 Sense–Think–Act 循環說明機器人系統的基本運作原理，並舉出各階段的具體例子。
13. 分辨四種主要地圖表示法（佔據格地圖、特徵地圖、點雲、語意地圖）的資料結構、優缺點與適用場景。
14. 區分相對定位、絕對定位、地圖定位與重定位四種定位方式，並說明各自的誤差特性。
15. 畫出「SLAM、路徑規劃、避障、控制」四大模組組成的導航系統架構圖，並說明模組間的資料流。
16. 說明 ROS / ROS 2 的核心概念（節點、話題、服務）以及它在 SLAM 教學與研究中的角色。
17. 比較掃地機器人、服務機器人、機器狗、無人機與自駕車五種平台的導航需求差異。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
| --- | --- | --- |
| 感知—決策—行動 | Sense–Think–Act | 機器人系統的基本循環：感測環境、做出決策、執行動作。 |
| 特徵地圖 | Feature Map | 只記錄環境中顯著地標（角點、柱子、標誌）位置的地圖。 |
| 相對定位 | Relative Localization | 由運動量測累積推算位姿（航位推算），誤差隨時間累積。 |
| 絕對定位 | Absolute Localization | 直接量測相對於全域參考系的位置（如 GPS、UWB），不漂移。 |
| 地圖定位 | Map-based Localization | 把即時感測資料與既有地圖比對來估測位姿，如 AMCL。 |
| 重定位 | Relocalization | 機器人「迷路」或被搬動後，在地圖中重新找回自己位置的能力。 |

| 中文名詞 | 英文名詞 | 概念說明 |
| --- | --- | --- |
| 綁架機器人問題 | Kidnapped Robot Problem | 機器人被突然移到未知位置的極端測試，檢驗重定位能力。 |
| 路徑規劃 | Path Planning | 在地圖中找出從起點到目標的可行且最佳路徑。 |
| 避障 | Obstacle Avoidance | 偵測並閃避地圖上沒有的動態或臨時障礙物。 |
| 代價地圖 | Costmap | 把障礙物加上安全膨脹距離的導航用地圖，分全域與局部兩層。 |
| 機器人作業系統 | ROS / ROS 2 | 機器人軟體開發框架，提供模組間通訊、工具與生態系。 |
| 節點與話題 | Node / Topic | ROS 的基本單元：節點是功能模組，話題是節點間的資料通道。 |

## 3.1 Sense–Think–Act：感知、決策、行動

所有自主機器人——無論是掃地機器人、機器狗還是自駕車——的運作都可以化約為一個不斷重複的循環：感知（Sense）、決策（Think）、行動（Act）。這個循環每秒執行數次到數百次，構成機器人「活著」的心跳。

- 感知（Sense）：透過感測器蒐集資訊。包含感知「外在世界」（光達掃描到前方有牆、相機看到行人）與感知「自身狀態」（輪子轉了幾圈、機身傾斜幾度）。第 2 篇介紹的六大感測器都屬於這一層。
- 決策（Think）：把感知資訊轉化為行動指令。這一層又可細分為多個子層：狀態估測（我在哪裡？——SLAM 與定位）、環境理解（周圍有什麼？）、任務規劃（我要去哪裡？）、路徑規劃（怎麼去？）與運動規劃（下一步怎麼動？）。
- 行動（Act）：把指令轉為馬達、舵機、螺旋槳的實際輸出，改變機器人與世界的狀態——而這又會改變下一輪感知到的內容，循環於是延續。

早期機器人學曾流行「感知—建模—規劃—行動」的嚴格序列式架構（Sense–Model–Plan–Act），每一輪都要先建完整世界模型再行動，反應遲鈍；1980 年代 Rodney Brooks 提出的行為式架構（Behavior-based Architecture）則主張跳過建模、讓感知直接驅動行為，反應快但難以完成複雜任務。現代機器人普遍採用混合式架構：底層用快速的反應式迴圈處理避障與平衡（毫秒級），上層用審議式的規劃處理建圖與導航（秒級）——SLAM 正是上層審議式架構的核心成員。

用一個生活比喻：騎腳踏車閃避突然衝出的小狗，靠的是不經大腦的反射（反應式）；規劃「怎麼從家裡騎到學校」，靠的是腦中的地圖與路線（審議式）。人如此，機器人亦然。

## 3.2 地圖的種類：Occupancy Grid、Feature Map、Point Cloud、Semantic Map

第 1 章我們從「輸出」的角度初識過地圖的樣貌；本節從「資料結構」的角度深入比較，

因為地圖表示法的選擇，直接決定了記憶體用量、演算法複雜度與應用能力。

### 一、佔據格地圖（Occupancy Grid Map）

把空間切成規則格網，每格儲存一個「被佔據的機率」（0 到 1 之間）。優點是結構簡單、查詢快速、天然支援路徑規劃與避障；缺點是記憶體隨面積線性成長、解析度固定（格子 5 公分就看不見 3 公分的物體），且不含任何語意。2D 佔據格是室內輪型機器人的主流選擇；3D 版本則常用八叉樹（Octree，如 OctoMap）壓縮儲存，只在有東西的地方細分格子。

### 二、特徵地圖（Feature Map）

反其道而行：不記錄「每一格有沒有東西」，只記錄環境中少數顯著地標的位置與描述——牆角、柱子、AprilTag、視覺特徵點。優點是極度精簡（一間教室可能只需幾十個地標）、與 EKF-SLAM 等估測理論天作之合；缺點是「地標以外的空間一無所知」，無法直接用來避障，且依賴環境中存在穩定可辨識的特徵。ORB-SLAM 系列建立的地圖點（Map Point）本質上就是視覺特徵地圖。

### 三、點雲地圖（Point Cloud Map）

直接保存感測器量到的原始三維點，不做格網化也不抽取特徵。優點是保真度最高、能呈現任意複雜的立體結構；缺點是資料量龐大（動輒數 GB）、缺乏「空與未知」的區別（沒有點的地方是沒東西？還是沒掃到？）、直接拿來規劃路徑並不方便。實務上點雲常作為「原始素材」，再依需求轉換為格地圖或網格模型。

### 四、語意地圖（Semantic Map）

在幾何之上加註意義：這塊區域是「廚房」、那個物體是「椅子」、這條線是「可通行的走廊」。語意地圖讓機器人能執行人類語言等級的任務，是服務機器人與第 32 章 AI 語意導航的基礎。它通常不是獨立存在，而是疊加在前三種幾何地圖之上的「標籤層」。

| 地圖類型 | 資料結構 | 記憶體 | 優點 | 限制 |
|---|---|---|---|---|
| 佔據格地圖 | 機率格網 | 中（隨面積） | 規劃避障方便、演算法成熟 | 解析度固定、無語意 |
| 特徵地圖 | 地標清單 | 極小 | 精簡、利於估測理論 | 不能避障、依賴特徵 |
| 點雲地圖 | 三維點集合 | 大 | 保真度最高、立體完整 | 難直接規劃、無空區概念 |
| 語意地圖 | 幾何 + 標籤 | 視底層而定 | 支援語言任務、人機互動 | 依賴 AI 辨識正確性 |

**表 3-1　四種地圖表示法比較**

## 3.3　定位的種類：相對定位、絕對定位、地圖定位、重定位

### 一、相對定位（航位推算）

從已知起點出發，靠里程計與 IMU 累加每一步的位移來推算目前位姿，又稱航位推算（Dead Reckoning）。這是大航海時代水手的方法：記錄航向與航速，累加推算船位。優點是自給自足、更新快速；致命傷是第 1 章反覆強調的漂移——誤差只增不減。

### 二、絕對定位

直接量測相對於全域座標系的位置：GPS 給出經緯度、UWB 基站給出室內座標、地磁羅盤給出絕對航向。優點是誤差有界、不隨時間惡化；缺點是依賴外部基礎設施（衛星、基站）、精度有限、更新率通常較低。

### 三、地圖定位

在「已有地圖」的前提下，把即時感測資料與地圖比對來推算位姿。ROS 生態中最經典的是 AMCL（Adaptive Monte Carlo Localization，自適應蒙地卡羅定位）：在地圖上撒出大量「猜測位置」的粒子，看哪些粒子預期看到的雷射掃描與實際掃描最吻合，逐步收斂到真實位姿（原理詳見第 11 章，實作詳見第 29 章）。掃地機器人第二次打掃時不必重新建圖，就是靠地圖定位。

**四、重定位**

如果機器人被抱起來搬到另一個房間（著名的「綁架機器人問題」），或視覺 SLAM 因快速轉動而跟丟，系統必須在不知道自己在哪的情況下，於整張地圖中重新找回位置——這就是重定位。它比一般定位困難得多，因為沒有「上一刻位置」可以參考，必須做全域搜尋。ORB-SLAM 系列以詞袋模型（Bag of Words）快速檢索相似畫面實現重定位（第 22 章）；這個能力也與閉環偵測共用同一套技術核心。

四種定位並非互斥，而是互補的層次：實務系統以相對定位提供高頻率的短期估測，以地圖定位或絕對定位提供不漂移的長期校正，並隨時準備在迷失時啟動重定位——這正是感測器融合思想在定位層的體現。

## 3.4　導航系統架構：SLAM、路徑規劃、避障、控制

有了地圖與定位，機器人還不會「走到目的地」。完整的導航系統由四大模組組成，彼此以明確的資料介面串接：

**圖 3-1　自主導航系統架構與資料流**




圖的左欄是五個模組依序串接的資料流，右欄則補上兩件文字不容易一眼看清的資訊：各模組的典型更新頻率，以及它在 ROS / Nav2 中對應的實際元件與本書章節。

更新頻率的差異值得特別注意。感測器每秒輸出數十至數百筆資料，控制器每秒執行五十到一百次，而全域規劃可能好幾秒才算一次——這不是誰比較重要，而是各自的任務性質不同：避免撞牆必須即時，重新規劃整條路線則可以慢慢來。第 29 章調整系統時，若發現機器人反應遲鈍或路徑抖動，第一件事就是檢查這條頻率階梯哪一環沒跟上。

最左邊那條紅色的迴路箭頭是整張圖的靈魂：機器人動了，環境相對於它就變了，於是必須重新感知。導航從來不是一條單向的管線，而是一個不斷繞回自己的閉迴路——這正是 3.1 節 Sense–Think–Act 的完整形態。

- SLAM / 定位模組是整個系統的「眼睛與內心地圖」：沒有正確的位姿，後面所有模組都在錯誤前提下工作——這就是為什麼本書用整本書的篇幅講清楚這一個模組。
- 全域規劃在靜態地圖上運作，回答「大方向怎麼走」；常用 A* 與 Dijkstra 等圖搜尋演算法。

- 局部規劃在機器人周圍數公尺的「局部代價地圖」上高頻運作，處理地圖上沒有的東西：走動的人、臨時擺放的椅子。代價地圖把障礙物向外膨脹一圈安全距離，確保機器人不會擦撞。
- 控制器把幾何路徑翻譯成馬達聽得懂的語言。差速輪機器人的指令通常就是一組（v、ω）：前進多快、轉多快。

這個架構在 ROS 生態中的具體實現，就是家喻戶曉的 Navigation Stack（ROS 1）與 Nav2（ROS 2）。第 29 章將帶同學把這整條管線在實體小車上跑起來。

## SLAM 在系統中的上下游：一張責任分界圖

前面說明了導航系統的四大模組。這裡特別釐清一件同學常混淆的事：SLAM 的責任邊界到底在哪裡？它不做什麼？

**圖 3-2　SLAM 的上下游責任分界**




這張圖把 SLAM 的責任切成四塊，由左而右就是一條資料鏈：上游餵給它什麼、它自己負責什麼、它明確不負責什麼、以及下游誰在用它的輸出。

中間那兩塊的對比是本節的重點。許多初學者會把「機器人走錯路」「撞到椅子」「停不準」全算在 SLAM 頭上，但這三件事分別屬於規劃器、局部避障與控制器。SLAM 只回答兩個問題：我在哪裡、環境長什麼樣。

這張分界圖有一個實用價值：除錯時能快速定位問題。第 29.7 節會提供一套系統性的排查順序，其邏輯正是依循這條上下游鏈——若機器人行為異常，先確認上游（感測器、TF、時間）是否正常，再看 SLAM 本身，最後才檢查下游的規劃與控制。許多同學花數小時調整規劃器參數，問題其實出在外參填錯——這正是不理解責任分界的代價。

## 3.5　ROS / ROS 2 在 SLAM 教學中的角色

如果說 SLAM 是導航的靈魂，ROS（Robot Operating System，機器人作業系統）就是承載靈魂的骨架。名字裡雖有「作業系統」，它其實是跑在 Linux 之上的機器人軟體框架，解決一個關鍵問題：讓感測器驅動、SLAM、規劃、控制這些來自不同作者的模組，能像積木一樣拼在一起。

### 三個核心概念

- 節點（Node）：一個獨立的功能程式。光達驅動是一個節點、Gmapping 是一個節點、路徑規劃是一個節點。
- 話題（Topic）：節點間的資料通道，採「發布 / 訂閱」模式。光達節點發布 /scan 話題，SLAM 節點訂閱它——雙方不必認識彼此，只要約好資料格式。
- 服務與動作（Service / Action）：一問一答式（查詢地圖）與長任務式（導航到目標點、可回報進度可取消）的通訊模式。

### 為什麼 SLAM 教學離不開 ROS

第一，標準化介面讓演算法可以「熱插拔」：同一台小車、同一份雷射資料，換一個節點就能比較 Gmapping 與 Cartographer 的建圖品質——這是本書第四篇實驗設計的基礎。第二，rosbag 錄放工具能把感測資料錄下來反覆重播，沒有機器人的同學也能用公開資料集完整練習 SLAM。第三，RViz 視覺化工具即時顯示地圖、位姿、粒子雲與規劃路徑，把抽象演算法變成看得見的動畫。第四，全球主流 SLAM 開源系統（Gmapping、

Cartographer、RTAB-Map、ORB-SLAM 的社群封裝）都提供 ROS 介面，站上 ROS 就站上了整個生態系。

### 從 ROS 1 到 ROS 2

ROS 1 誕生於 2007 年，已於 2025 年 5 月隨最後版本 Noetic 終止官方支援；新專案一律建議採用 ROS 2。ROS 2 重新設計了底層通訊（採用 DDS），支援即時性、多機器人與嵌入式平台，每年 5 月 23 日（世界烏龜日）發布一個新版本：偶數年為長期支援版（LTS，支援五年）、奇數年為一般版。截至本書撰寫時，常用版本包括 Humble（2022，LTS）、Jazzy Jalisco（2024，LTS）與最新的 Lyrical Luth（2026，LTS，搭配 Ubuntu 26.04）。本書實作章節以 LTS 版本為基準，並在第 28 章詳細說明環境安裝。教學上的好消息是：本章介紹的節點、話題等觀念在 ROS 1 與 ROS 2 中一脈相承，學會一套，兩邊通用。

### Nav2 的當代地位：從研究工具到產業標準

本節介紹了 ROS 在教學中的角色，這裡補充一個對同學規劃職涯有意義的觀察：ROS 生態中的導航堆疊，已經從研究工具演變為產業標準。

作為 ROS Navigation Stack 的後繼者，Nav2 建立在十五年的傳承之上，它把自駕車領域的技術降維、重新設計並最佳化，用於行動與地面機器人；它提供感知、規劃、控制、定位、視覺化與行為等完整功能，能從原始或預處理過的感測與語意資料計算環境模型、動態規劃路徑、產生可行的馬達指令、避開障礙，並組織更高層次的機器人行為。

值得注意的是它的產業採用程度：Nav2 是一套生產級的高品質導航框架，受到全球一百多家公司信賴；它讓行動機器人能在複雜環境中完成使用者定義的應用任務，且幾乎適用於任何一類機器人運動學。它的能力也不限於「從 A 點移動到 B 點」——還能設定中間位姿，並表達物件跟隨、完整覆蓋導航等其他類型的任務。

最後一點對本書特別有意義：「完整覆蓋導航」正是第 16 章掃地機器人的核心任務，而「物件跟隨」則呼應第 24 章的視覺導航。這說明 Nav2 不只是一個路徑規劃器，而是一

個可組合的任務框架——第 29.6 節會透過行為樹的概念說明它如何做到這件事。

**學習建議：ROS 技能的市場價值**

對台灣的學生而言，ROS 2 與 Nav2 的實作能力有幾個現實的價值：

• 產業共通語言：無論做 AGV、服務機器人、農業機器人或巡檢機器狗，多數團隊都用 ROS 2。

• 可累積的作品：第 31 章的專題成果能直接放進履歷，且他人可重現驗證。

• 銜接國際：本書第 2 章介紹的各校課程作業，多數也以 ROS 為平台。

本書第八篇（第 27–30 章）因此不只是教學單元，也是職能訓練——

第 28 章的環境建置、第 29 章的完整導航系統、第 31 章的專題評量，

構成一份可展示的實作履歷。

## 3.6 從掃地機器人到自駕車：不同平台的導航需求

同一套導航架構，落在不同平台上會長出截然不同的樣貌。理解「需求如何塑造系統」，是系統設計能力的起點。

- 掃地機器人：2D 平面、低速、環境半靜態。重點不是「從 A 到 B」而是「覆蓋每一寸地板」（Coverage Path Planning），感測器成本被壓到極致——一顆 2D 光達加碰撞與懸崖感測器就要打天下（第 16 章）。
- 室內服務機器人（送餐、導覽）：與人共處是最大挑戰。需要語意理解（電梯、櫃檯）、社交性避障（不從人群中硬穿）、以及長期運行下的地圖更新能力。

- 機器狗（四足機器人）：能上樓梯、跨障礙，導航從 2D 升級為 2.5D / 3D——地圖必須包含地形高度與可踏性，定位必須在劇烈震動下保持穩定，光達慣性緊耦合（第 18 章）成為標配。
- 無人機：完整 6 自由度、高速、載重與電力極度受限。偏好輕量的視覺慣性方案（第 26 章），且對延遲零容忍——定位慢半秒，飛機已經撞牆。
- 自駕車：速度快、後果重、範圍大。採高精地圖 + 多重冗餘感測（光達、相機、雷達、RTK-GNSS 缺一不可），定位精度要求公分級，且必須通過車規安全驗證。

| 平台 | 空間維度 | 速度 | 核心難點 | 典型感測組合 |
|---|---|---|---|---|
| 掃地機器人 | 2D | 慢 | 全覆蓋、成本極限 | 2D 光達 + 碰撞 + IMU |
| 服務機器人 | 2D + 語意 | 慢 | 人機共處、長期地圖 | 2D 光達 + 相機 + 超音波 |
| 機器狗 | 2.5D / 3D | 中 | 地形理解、震動 | 3D 光達 + IMU + 深度相機 |
| 無人機 | 3D（6DOF） | 快 | 重量功耗、零延遲 | 相機 + IMU（VIO） |
| 自駕車 | 2D 大範圍 | 極快 | 安全冗餘、公分級 | 3D 光達 + 相機 + 雷達 + RTK |

**表 3-2　五種平台的導航需求比較**

本書的章節安排正是沿著這條光譜前進：第四篇從最親民的掃地機器人與 2D SLAM 出發，第五篇進入機器狗與 3D 光達，第六、七篇走向視覺導航與無人機，讓同學在難度階梯上逐級而上。

**圖 3-3　五種平台的導航需求光譜**




把表 3-2 畫成並列的欄位後，一條清楚的光譜就浮現了：由左而右，空間維度從 2D 升到 3D、速度從慢到極快、感測組合從一顆 2D 光達擴張到光達加相機加雷達加 RTK。

請注意最底下那行章節標示——它幾乎就是本書第四篇到第七篇的順序。這不是巧合：本書刻意沿著這條難度光譜編排，讓同學從最親民、最便宜、最容易驗證的掃地機器人開始，逐級走向機器狗、無人機與多機協作。每往右一格，需要處理的自由度、速度與不確定性都會增加一個等級。

另一個值得玩味的觀察是自駕車：它的空間維度回到 2D（車子不會飛），但「2D 大範圍」帶來的是完全不同的難題——地圖大到無法整張載入、速度快到留給演算法的時間只有數十毫秒、後果重到必須多重冗餘。這說明一件事：維度低不等於簡單，真正決定難度的是需求的組合。

## 公式或流程

### 一、航位推算的誤差累積

以 2D 差速輪機器人為例，設時間 t 的位姿為 $(x(t)、y(t)、\theta(t))$，一個取樣週期 $\Delta t$ 內量得線速度 $v$ 與角速度 $\omega$，則航位推算的更新式為：

$$x(t+1) = x(t) + v \cdot \Delta t \cdot \cos\theta(t)$$

$$y(t+1) = y(t) + v \cdot \Delta t \cdot \sin\theta(t)$$

$$\theta(t+1) = \theta(t) + \omega \cdot \Delta t$$

注意每一步的 v 與 ω 都帶有量測誤差，而且誤差會透過 cos 與 sin「傳染」給位置——特別是角度誤差：航向偏 1 度，直走 10 公尺後位置就偏約 17 公分，而且之後的每一步都建立在這個歪掉的方向上。這條簡單公式，就是「為什麼純里程計不能撐起導航」的數學解釋，也是第 10 章座標運動學與第 11 章誤差傳播理論的預告。

## 二、導航問題的分解

本章介紹的系統架構，可以整理為導航三問與其對應模組：

**圖 3-4　導航三問**

問題一：我在哪裡？（Where am I?）

→ 定位 / SLAM（本書第一至六篇）

問題二：我要去哪裡？（Where am I going?）

→ 任務與目標決策（第 31、32 章專題）

問題三：我該怎麼去？（How do I get there?）

→ 路徑規劃 + 避障 + 控制（第 29 章 Nav2 實作）

這三問出自行動機器人學的經典表述（Leonard 與 Durrant-Whyte）：整本教科書的任務，就是把第一問回答到透徹，並讓同學有能力把答案接上第二、三問。

## 3.7 導航技術的當代演進：從模組化到基礎模型

本章描述的是經典的模組化導航架構——感知、定位、規劃、控制各司其職，模組之間以明確的介面串接。這個架構穩定、可解釋、易於除錯，是目前絕大多數實際部署系統的骨幹，也是本書第八篇實作的對象。

但近兩年出現了一條截然不同的路線，值得同學知道它的存在與定位。

### 第二條路線：導航基礎模型

傳統架構的每個模組各自訓練或設計；新路線則試圖用單一的大型模型端到端地完成導航。這類模型被稱為導航基礎模型或視覺語言動作模型。

其能力範圍相當驚人。以一項代表性工作為例，它提供統一的框架來處理多種任務——包含影像問答、影片問答與導航；在物件搜尋、追蹤與自動駕駛等多項基準上驗證，並在多種真實機器人平台上實測，包括人形機器人、四足機器人、無人機與輪型機器人，顯示出通往通用導航的可能性。

這個「同一套模型跨多種平台」的特性，與本書第 3.6 節的討論形成有趣的對照：本節說明不同平台的導航需求差異極大（輪型只需 2D、四足需地形評估、無人機需 6-DoF），而基礎模型路線正試圖用一套模型涵蓋這些差異。第 24.5 節與第 32 章會再回到這個主題。

### 世界模型：讓機器人「想像」未來

另一個新興概念是世界模型。傳統導航的規劃基於當前地圖；世界模型則讓機器人能預測「如果我往這走，接下來會看到什麼」，再據此選擇行動。近年已有研究把視覺語言世界模型用於階層式規劃與雙時域的導航預測，也有工作探討如何讓世界模型在連續環境中自我演化。

這個思路對本書的讀者並不陌生——第 11 章的貝氏濾波就是「預測—更新」的循環，只是預測的是位姿而非畫面。世界模型可以理解為把預測的對象從「狀態」擴展到「感知」，這是一個自然但強大的延伸。

### 成長的速度與務實的判斷

這個領域的成長速度值得留意。有整理指出，2025 至 2026 年間釋出的機器人基礎模型已超過九十個，涵蓋視覺語言動作模型、世界模型、人形機器人基礎模型、移動、導航與操作策略等類別；而視覺語言動作模型在頂級研討會的投稿量，從 2025 到 2026 年成長了約十八倍。

面對這樣的變化速度，同學該如何自處？本書的建議是三點：

1. 先掌握模組化架構：因為它是目前實際部署的主流，也是理解新方法的基礎——基礎模型仍然要輸出位姿或路徑點，仍然要面對感測器的限制，仍然要在真實世界中避障。
2. 理解新方法解決了什麼舊問題：基礎模型的價值在於泛化與語言介面，而非取代幾何精度。第 34.7 節會詳細討論它的四大風險，包括幻覺、系統性誤判、延遲與安全性。
3. 注意分層而非替換：目前最務實的整合方式，是讓基礎模型處理高階決策（秒級以上），幾何系統處理即時控制（毫秒級）。這正是第 32.6 節提出的分層架構，也是第 3.1 節「感知—決策—行動」思想的現代版本。

**觀念總結：兩條路線的分工**

【模組化架構】本書第一至八篇

優點：可解釋、可除錯、運算需求明確、不需訓練資料

限制：每個模組各自最佳化，未必達成全域最優；語意理解能力弱

【基礎模型路線】本書第 32 章

優點：泛化能力強、支援語言介面、可跨平台

限制：運算與記憶體需求高、可能產生幻覺、難以除錯與問責

【務實的整合】上層用基礎模型理解任務，下層用模組化架構保證安全。

這個分工原則在本書第 24、27、32 章會反覆出現。

## 案例

### 案例 3-1　送餐機器人的一天：架構總動員

觀察餐廳裡的送餐機器人完成一趟任務，本章所有觀念會一次到齊。開機：載入既有地圖，AMCL 地圖定位確認自己在充電站（地圖定位 + 重定位能力）。接單：任務系統指定 12 號桌（導航第二問）。出發：全域規劃在佔據格地圖上算出路徑（A*），沿途里程計與 IMU 提供高頻位姿（相對定位），雷射持續與地圖比對修正（地圖定位）。半路一位客人起身擋道：局部代價地圖瞬間更新，局部規劃繞出一條弧線（避障），控制器輸出新的（v、ω）。送達後返航充電。整趟任務中，SLAM 建好的那張地圖是一切的基礎——地圖歪一公尺，機器人就會把餐送進牆裡。

### 案例 3-2　綁架測試：把掃地機器人抱到另一個房間

許多同學家裡就能做這個經典實驗：讓掃地機器人在客廳工作到一半，把它抱起來放到臥室。早期低價機型會當場「精神錯亂」——以為自己還在客廳，對著臥室的牆重複碰撞；具備重定位能力的機型則會停頓數秒，旋轉一圈重新掃描，比對雷射輪廓與記憶中的全屋地圖，然後在 App 地圖上「瞬移」到正確的臥室位置繼續工作。這數秒鐘裡發生的，正是教科書上的綁架機器人問題求解：拋棄先前位姿假設、全域搜尋、多假設驗證、收斂。一個抱起來再放下的動作，就是一堂免費的定位理論課。

### 案例 3-3　從 Willow Garage 到 Nav2：ROS 生態的力量

2010 年前後，美國 Willow Garage 公司把 PR2 機器人與 ROS 一同開源，並整理出第一代 Navigation Stack——AMCL、代價地圖、全域與局部規劃器的完整組合。此後十餘年，全球數以千計的機器人（包括台灣許多大學實驗室的自製小車）不必重寫導航系統，站在同一套開源積木上疊出各自的應用。ROS 2 時代的 Nav2 更進一步引入行為樹（Behavior Tree）任務架構與外掛式規劃器。這段歷史說明了開源生態對機器人教育的意義：學生的心力可以集中在理解原理與創新應用，而不是重造每一顆輪子——這也是本書第八篇直接以 ROS 2 為實作骨幹的原因。

## 案例 3-4　語意進入導航堆疊：當分割模型接上 Nav2

本章介紹的模組化架構，近年正在發生一個安靜但重要的變化：語意資訊開始進入導航堆疊的核心。

傳統的代價地圖（第 29.6 節）只區分「可通行」與「有障礙」。但真實世界的通行性遠比這細膩——人行道與車道都是平坦的地面，幾何上沒有差別，語意上卻完全不同；草地可以壓過去，水窪最好繞開；玻璃在光達上不存在，在視覺上卻清晰可辨。

近期已有實際展示把分割基礎模型與 Nav2 結合，在戶外環境中分割人行道與路面來進行語意導航，並在一般等級的運算平台上運行。這意味著語意導航已經從論文走向可部署的系統。

對照本書的架構，這個變化發生在兩個層次的交界處：語意層（第 32 章的 AI 視覺模型）產出「這是人行道、那是車道」的判斷，幾何層（第 29 章的代價地圖）把這個判斷轉換為通行成本。兩者的介面很單純——語意結果變成代價地圖的一個圖層。這正是模組化架構的優勢：新能力能以圖層的形式加入，不需重寫整個系統。

對同學的啟示有兩點。其一，第 29.6 節談的「代價地圖分層設計」不只是實作細節，而是一個能容納未來擴充的架構決策。其二，若你要做語意導航的專題（第 24、31 章），最務實的切入點不是重寫導航系統，而是「多加一個語意圖層」——這個做法成本低、風

險小，而且能立即看到效果。

## 實作活動

### 實作 3-1　導航系統拆解與紙上設計

目的：不寫程式，先用「系統設計師」的眼光完成一次導航系統的完整拆解與規格設計，建立全局觀。

**步驟**

10. 分組（每組 3 至 4 人），每組抽選一種平台：掃地機器人、送餐機器人、校園巡檢機器狗、室內巡檢無人機、無人載具小巴。
11. 需求分析：列出該平台的運行環境（室內外？有人？地形？）、速度需求、失敗的後果嚴重度。
12. 感測器選型：從第 1 章表 1-1 的六大感測器中挑選組合，說明每一顆的用途與捨棄其他選項的理由（含成本考量）。
13. 地圖與定位選型：從表 3-1 選擇地圖表示法、從 3.3 節選擇定位策略，說明理由。
14. 繪製系統架構圖：仿照圖 3-1，畫出你們平台的完整資料流，標明各模組的更新頻率（哪些要毫秒級？哪些秒級即可？）。
15. 各組上台簡報五分鐘，其他組扮演「客戶」提出至少一個刁鑽問題（例如：玻璃隔間怎麼辦？機器人被小孩推走怎麼辦？）。

**討論問題**

9. 你們的平台中，哪一個模組失效會造成最嚴重的後果？系統該如何為它設計備援？
10. 如果預算砍半，你會先犧牲哪顆感測器？系統能力會退化成什麼樣子？

11. （大學部）你們設計的架構中，哪些模組可以直接使用 ROS 2 / Nav2 的現成元件？哪些必須自行開發？

**延伸挑戰（大學部）**

安裝 ROS 2 與 TurtleBot3 模擬器（安裝步驟預告於第 28 章，網路上有充足教學），在 Gazebo 模擬世界中啟動 Nav2，實際觀察 RViz 中的全域路徑、局部路徑與代價地圖，對照本章圖 3-1 指認每一個元件。

## 延伸閱讀

12. R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, Introduction to Autonomous Mobile Robots, 2nd ed., MIT Press, 2011.（本章系統觀的主要參考，第 1 章即以「導航三問」開場）
13. J. J. Leonard and H. F. Durrant-Whyte, "Mobile Robot Localization by Tracking Geometric Beacons," IEEE Transactions on Robotics and Automation, 1991.（「我在哪裡」三問的經典出處之一）
14. R. Brooks, "A Robust Layered Control System for a Mobile Robot," IEEE Journal of Robotics and Automation, 1986.（行為式架構的開山之作，理解架構演進必讀）
15. S. Macenski et al., "The Marathon 2: A Navigation System," IROS 2020.（Nav2 導航系統的官方論文）
16. ROS 2 官方文件（docs.ros.org）與 Nav2 官方文件（docs.nav2.org）.（實作章節前建議先瀏覽其觀念介紹頁）
17. S. Macenski et al., "Robot Operating System 2: Design, Architecture, and Uses in the Wild," Science Robotics, 2022.（ROS 2 設計理念與產業應用的權威介紹）

18. A. Hornung et al., "OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees," Autonomous Robots, 2013.（3D 佔據格地圖的代表作）

## 從系統藍圖到感測器：第一篇的收束

第一篇到這裡結束。三章的分工是這樣的：第 1 章建立問題意識（SLAM 要解決什麼）、第 2 章提供學習地圖（世界怎麼教、本書怎麼編）、本章畫出系統藍圖（SLAM 在整體中的位置）。

這三章刻意不含任何數學與程式，目的是先建立完整的觀念框架。筆者的教學經驗是：太早進入公式與程式碼，學生容易見樹不見林，做出了能跑的系統卻說不清楚它為什麼這樣設計。有了這個框架，後續的技術細節才有掛靠的地方。

### 圖 3-5　從第一篇通往第二篇

【第一篇建立的三個框架】

問題框架　→　定位與建圖互相依賴，必須同時解決（第 1 章）

學習框架　→　感測器 → 數學 → 前端 → 後端 → 閉環 → 專題（第 2 章）

系統框架　→　感知 → 定位建圖 → 規劃 → 控制（本章）

【第二篇要回答的問題】

機器人用什麼「看」世界？每一種感測器能做什麼、又會在什麼時候失敗？

第 4 章　室內定位　→　Wi-Fi、UWB、藍牙、視覺標記

第 5 章　衛星導航　→　GNSS 原理、都市峽谷、與慣性的組合

第 6 章　地磁導航　→　電子羅盤、磁干擾、地磁指紋

第 7 章　IMU　→　加速度計、陀螺儀、漂移、姿態濾波

第 8 章 光達 → 飛行時間、2D 與 3D、規格與盲區

第 9 章 相機 → 針孔模型、校正、單目雙目與 RGB-D

【第二篇的共同結構】每章都以「原理 → 規格 → 失效情境 → 互補 → 實作」展開。

特別提醒同學留意每一章的「失效情境」討論。第 4 章會談 Wi-Fi 訊號的波動、第 5 章會談都市峽谷的多路徑、第 6 章會談鋼構建築中的磁場扭曲、第 7 章會談 IMU 的漂移速度、第 8 章會談光達看不見玻璃、第 9 章會談相機的白牆詛咒。

這些不是負面資訊，而是工程判斷的基礎。到了第 30.7 節，同學會在同一個場域親眼看到光達與相機各自崩潰——那時再回頭看第二篇的每一章，會明白為什麼本書從一開始就這麼強調限制。

## 小結

14. 自主機器人以 Sense–Think–Act 循環運作；現代系統採混合式架構——底層反應式迴圈處理避障與平衡，上層審議式規劃承載 SLAM 與導航。
15. 四種地圖表示法各擅勝場：佔據格利於規劃避障、特徵地圖精簡利於估測、點雲保真度最高、語意地圖支援語言等級任務；實務常混合使用。
16. 定位分為相對定位（高頻但漂移）、絕對定位（不漂移但精度有限）、地圖定位（與既有地圖比對）與重定位（迷失後全域找回）；四者互補構成完整定位能力。
17. 導航系統由 SLAM / 定位、全域規劃、局部避障與運動控制四大模組串成資料流；SLAM 是一切下游模組的前提。

18. ROS / ROS 2 以節點、話題、服務的標準化介面串接模組，配合 rosbag 與 RViz 工具，成為 SLAM 教學與研究的事實標準；ROS 1 已於 2025 年終止支援，本書實作以 ROS 2 LTS 版本為基準。
19. 從掃地機器人到自駕車，導航需求隨空間維度、速度與安全要求逐級升高，感測器組合與系統設計亦隨之改變；本書章節即沿此難度光譜安排。

至此，第一篇「SLAM 與自主導航概論」告一段落——同學已經知道 SLAM 是什麼問題、世界名校怎麼教、以及它在整台機器人中的位置。下一章起進入第二篇，我們將逐一拆解機器人的「感官」：先從一個看似矛盾的問題開始——為什麼在室內，天上三十多顆 GPS 衛星幫不上忙？

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第二篇 定位與導航感測器

# 第 4 章

# 室內定位技術

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

## 本章在全書的位置

第一篇把 SLAM 的問題、世界各校的教法與導航系統的整體藍圖都交代完了。從本章開始進入第二篇，我們要一個一個檢視機器人身上的感測器——它們各自能回答什麼問題、又各自在什麼情況下失靈。

本章是第二篇的開場，主題是「室內定位」。它和後面幾章有一個關鍵的分工：第 5 章的衛星導航在室內失效，第 6 章的地磁與第 7 章的 IMU 能提供方向卻無法提供絕對位置，第 8 章的光達與第 9 章的相機則需要一張地圖才能定位。而本章討論的技術——Wi-Fi、藍牙、UWB、RFID 與視覺標記——是唯一一類「在室內直接給出絕對座標」的手段。

這件事對 SLAM 極為重要。第 1 章提過 SLAM 的兩大難題：累積漂移與閉環偵測。本章的技術正好從外部提供不漂移的絕對參考，成為第 12 章圖優化中的一條額外約束邊。換句話說，本章不是 SLAM 的替代品，而是 SLAM 的錨點。

**圖 4-0 第 4 章與全書的連結地圖**

【承接前章】

第 3 章的系統藍圖告訴我們「定位」是導航的一個模組；本章補上室內場域的解法。

第 3.3 節的「絕對定位」在室內要靠什麼？答案就是本章。

【本章負責】

無線電測距（Wi-Fi / BLE / UWB）＋ 地標識別（RFID / QR / AprilTag）＝ 室內絕對位置

【往下延伸】

→ 第 5 章 室外的對應解法：GNSS（與本章形成室內外互補）

→ 第 6、7 章 方向感測器：地磁與 IMU（補本章缺少的朝向與高頻更新）

→ 第 9、21 章 AprilTag 背後的相機內參與 PnP 數學

→ 第 12 章 把本章的絕對觀測寫成圖優化的一條約束邊

→ 第 24、29 章 AprilTag 與 Wi-Fi 在真實機器人上的整合實作

## 學習目標

從本章起，我們進入第二篇「定位與導航感測器」，逐一拆解機器人的各種「感官」。第一站是室內定位——這是最貼近同學日常生活的定位技術，也是理解「絕對定位」與「SLAM」如何互補的最佳起點。學完本章之後，同學應能夠達成下列目標：

18. 解釋 GPS 訊號為何在室內失效，並說明訊號衰減與多路徑效應的物理成因。
19. 說明 Wi-Fi RSSI 指紋定位的離線建庫與線上比對兩階段流程，並實作簡易版本。
20. 比較 BLE Beacon 的三種定位模式（鄰近偵測、三邊定位、指紋定位）與其精度差異。
21. 說明 UWB 的高精度來源，並區分 TWR 與 TDoA 兩種測距架構。

22. 比較 RFID、NFC、QR Code 與視覺標記（AprilTag / ArUco）作為「地標式定位」的原理與適用場合。

23. 說明室內定位與 SLAM 如何互補融合，並知道各技術在精度、成本、部署難度上的權衡。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
| --- | --- | --- |
| 接收訊號強度指標 | RSSI | 接收端量到的訊號強度，隨距離增加而衰減，是最便宜的測距依據。 |
| 指紋定位 | Fingerprinting | 先建立各位置的訊號特徵資料庫，再比對即時訊號找出最相似位置。 |
| 多路徑效應 | Multipath | 訊號經牆面反射後多條路徑抵達，造成測距偏長或不穩定。 |
| 非視距 | NLOS | 發射端與接收端之間被遮蔽，訊號需穿透或繞射，是誤差主因。 |
| 三邊定位 | Trilateration | 由三個以上已知點的距離求交點，推算未知點位置。 |
| 三角定位 | Triangulation | 由角度資訊（而非距離）推算位置。 |
| 超寬頻 | UWB | 使用極短脈衝、極寬頻譜的無線技術，可達公寸級甚至公分級測距。 |
| 雙向測距 | TWR | 標籤與基站來回通訊量測飛行時間，不需基站時間同步。 |
| 到達時間差定位 | TDoA | 比較訊號抵達多個基站的時間差，需基站精確同步，容量大。 |
| 低功耗藍牙信標 | BLE Beacon | 週期性廣播識別碼的低功耗藍牙裝置，常用於室內導引。 |
| 視覺基準標記 | Fiducial Marker | 如 AprilTag、ArUco，可由影像精確解算相對位姿的人工標記。 |

| 中文名詞 | 英文名詞 | 概念說明 |
| --- | --- | --- |
| 六自由度位姿 | 6-DoF Pose | 三軸位置加三軸旋轉，視覺標記可一次解出完整位姿。 |

## 4.1 為什麼室內不能只靠 GPS

走進百貨公司地下停車場，手機地圖上的藍點會開始亂飄，甚至直接顯示「無法定位」。這個日常經驗背後，是三個疊加的物理限制。

### 一、訊號實在太微弱

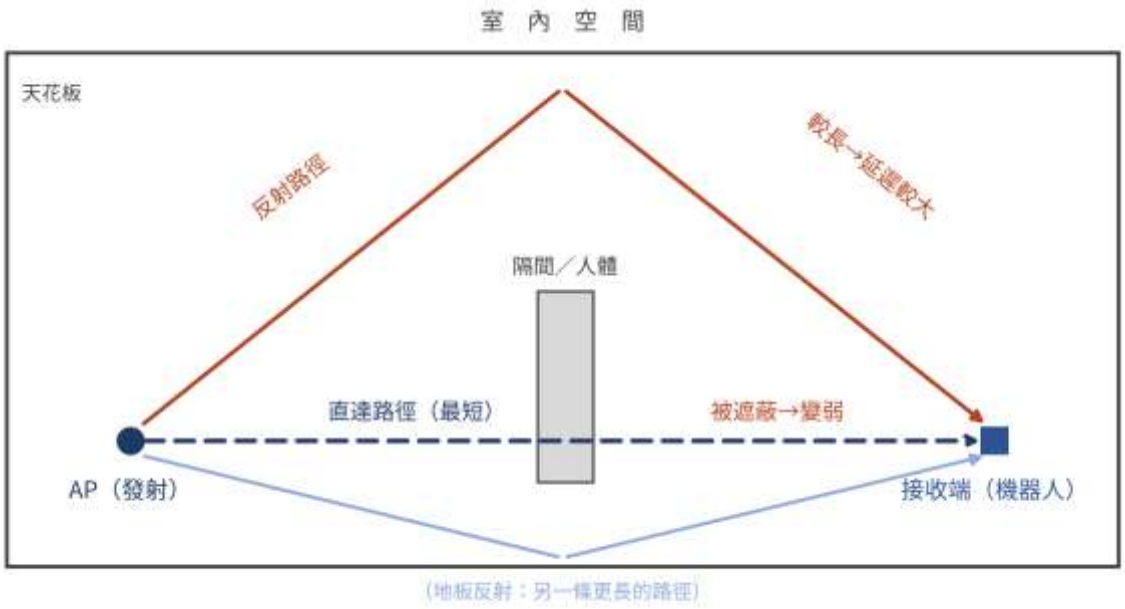
GPS 衛星在距地表約兩萬公里的中軌道上，發射功率僅約 25 至 50 瓦——大約是一顆家用燈泡的等級。訊號跋涉兩萬公里抵達地面時，強度已衰減到約 -130 dBm 這個量級，比環境的背景雜訊還要微弱，接收機必須靠「展頻編碼」的相關運算才能把它從雜訊中撈出來。這樣脆弱的訊號，遇到鋼筋混凝土樓板可能再衰減數十 dB，等同於把僅存的一絲訊號徹底掐熄。

### 二、多路徑效應讓測距失真

即使部分訊號從窗戶漏進室內，它往往已經在牆面、天花板、金屬櫃上彈跳過。GPS 的測距原理是「訊號飛行時間乘上光速」，反射路徑比直線路徑長，算出來的距離就會偏大。更麻煩的是，室內反射路徑往往比直達路徑更強（直達的被牆擋住了），接收機無從分辨，定位結果因此劇烈飄移——這正是停車場裡藍點亂跳的主因。

這個現象在幾何上很容易理解：接收端收到的不只一個訊號，而是一組延遲不同、強度不同的副本。測距演算法若誤把某個反射副本當成直達波，算出的距離就會「偏長」——而且永遠只會偏長，不會偏短，因為反射路徑必然比直線路徑遠。這種單向的系統性偏差無法靠多次平均消除，是室內無線定位最棘手的誤差來源。

**圖 4-1 室內多路徑效應示意**




圖中三條路徑同時抵達接收端：直達波被隔間或人體削弱，天花板與地板的反射波卻可能更強。測距演算法若誤取後者，算出的距離就比真實距離長——而且永遠偏長。

> **為什麼這件事和 SLAM 有關**
>
> 第 8 章的光達與第 21 章的視覺測距也會遇到類似的「假對應」問題：把錯誤的觀測當成正確的，導致估計被拉偏。
>
> 處理手法也相通——第 21 章的 RANSAC、第 12 章的強健核函數（Robust Kernel），本質上都是在說「不要相信少數異常觀測」。
>
> 所以本章雖然在談無線電，同學學到的「異常值會系統性地偏向某一側」這個直覺，在後面的幾何章節同樣適用。

## 三、幾何條件惡化

衛星定位需要至少四顆分布良好的衛星（三顆定位、一顆解算時鐘誤差）。室內能「看見」的衛星往往只剩靠近窗戶方向的少數幾顆，幾何分布極差。定位學上用精度稀釋因子（DOP，Dilution of Precision）描述這個現象：衛星擠在同一方向時 DOP 值飆高，即使每顆的測距都準確，交會出的位置誤差仍會被放大數倍。

**觀念釐清：室內定位的三難**

① 精度：要公尺級還是公分級？精度每提高一個數量級，成本往往上升一個數量級。

② 成本：需要在環境中安裝多少基礎設施（基站、信標、標記）？

③ 覆蓋：一棟樓？一層樓？一個房間？覆蓋範圍越大，部署與維護成本越高。

本章介紹的每一種技術，本質上都是在這三個維度上取捨的不同答案。

也正因為沒有任何一種室內定位技術能同時滿足三者，SLAM 才顯得如此重要：SLAM 不需要任何預先安裝的基礎設施，機器人靠自己的感測器就能在陌生環境建圖定位。但反過來說，SLAM 有漂移與重定位的弱點——這就是本章最後 4.7 節要談的融合主題。

## 4.2 Wi-Fi RSSI 指紋定位

Wi-Fi 是室內最普及的無線基礎設施——教室、辦公室、商場處處都有 AP（無線基地台），而且每支手機、每台筆電都內建接收器。既然訊號無所不在，能不能拿來定位？答案是可以，但方法與 GPS 完全不同。

### 為什麼不能直接用 RSSI 算距離

直覺想法是：訊號強度隨距離衰減，那麼量到強度就能反推距離，再用三邊定位求交點。理論上，自由空間中的訊號強度遵循對數距離路徑損耗模型：

$$RSSI(d) = RSSI(d_0) - 10 \cdot n \cdot \log_{10}(d / d_0)$$

其中 n 是路徑損耗指數（自由空間約為 2，室內因遮蔽常在 2 至 4 之間）。問題在於：室內的 n 值會隨牆壁材質、人流、家具擺設而劇烈變化，同一距離的實測 RSSI 可能相差 10 dB 以上，換算成距離誤差可達數公尺。此外人體本身就是很好的訊號吸收體——手機拿在手上與放在桌上，讀數就不一樣。因此室內 Wi-Fi 定位主流不走「測距」路線，而走「指紋」

路線。

### 指紋定位：兩階段流程

指紋定位的核心洞見是：不要試圖理解訊號如何傳播，只要記住「每個地點看起來像什麼樣子」。流程分為兩階段：

- 離線建庫階段（Offline / Training）：把場域劃分為格點（例如每 2 公尺一個參考點），在每個點上停留數秒，記錄所有可見 AP 的 MAC 位址與 RSSI 值，形成一組「訊號指紋」。全部參考點的指紋集合構成指紋資料庫（Radio Map）。
- 線上定位階段（Online / Positioning）：使用者裝置掃描當下的 AP 與 RSSI，與資料庫中每一筆指紋比對相似度，取最相似者（或前 K 個的加權平均）作為估測位置。

**圖 4-2　Wi-Fi 指紋定位的兩階段流程**

【離線階段：建立指紋資料庫】　（耗時，但只做一次）

① 在場域中選定參考點（例如每 2 公尺一點）

↓

② 在每個參考點停留數秒，記錄所有可見 AP 的 MAC 與 RSSI

↓

③ 存成資料庫：　位置(x, y) ↔ 指紋向量 F = [$RSSI_1$, $RSSI_2$, …, $RSSI_n$]

【線上階段：即時定位】　（快速，持續執行）

④ 裝置掃描目前的 RSSI，得到觀測向量 S

↓

⑤ 對資料庫中每一筆指紋 $F_i$ 計算距離 D(S, $F_i$)

↓

⑥ 取最相近的 K 筆，以其座標加權平均 → 輸出估計位置

關鍵觀念：這裡的「距離」是訊號空間中的距離，不是實體距離。

這個兩階段結構，同學在後面的章節會一再遇到。第 6 章的地磁指紋定位是同一套流程換成磁場強度；第 20 章的視覺詞袋（Bag of Words）閉環偵測也是同樣的邏輯——離線建立特徵字典，線上比對相似度。甚至第 14 章 Gmapping 的粒子權重計算，本質上也是在問「目前的觀測和地圖上這個位置該有的觀測有多像」。

理解這一點，同學就會發現：所謂「定位」，很大一部分是在做比對（Matching）。差別只在於比對的對象是訊號強度、磁場、影像特徵，還是點雲幾何。

### 比對演算法：K 最近鄰

最常用的比對方法是 KNN（K-Nearest Neighbors）。設當前量到的指紋向量為 S、資料庫中第 i 個參考點的指紋為 $F_i$，兩者的歐氏距離為：

$$D(S, F_i) = \sqrt{\sum_j (S_j - F_{ij})^2}$$

其中 j 掃過所有共同可見的 AP。取 D 值最小的 K 個參考點，把它們的座標做（加權）平均，就是估測位置。加權時常用距離倒數作為權重，稱為 WKNN。

### 優點與限制

- 優點：零硬體成本（利用既有 AP）、覆蓋範圍廣、手機即可實作，非常適合作為教學入門。
- 限制：精度通常在 3 至 8 公尺；指紋庫建置費工（一層樓可能需數百個參考點）；環境變動（AP 更換位置、隔間改裝、人潮）會使資料庫失效，需要定期重建。

近年改良方向包含：以深度學習取代 KNN 比對、結合手機 IMU 的行人航位推算（PDR）平滑軌跡、以及利用 Wi-Fi RTT（往返時間，IEEE 802.11mc）直接測距——後者可將精度提升至 1 至 2 公尺，但需要 AP 與裝置雙方都支援。

## 4.3　Bluetooth Beacon ／ BLE 定位

低功耗藍牙（BLE，Bluetooth Low Energy）信標是一顆鈕扣電池就能運作一到兩年的

小裝置，週期性廣播自己的識別碼。2013 年 Apple 推出 iBeacon 規格後大為普及，Google 也提出 Eddystone 規格。相較 Wi-Fi AP，信標的優勢在於「可以自己決定裝在哪裡」——密度與位置都能為定位需求量身設計。

### 三種使用模式

- 鄰近偵測（Proximity）：最簡單也最可靠。不算座標，只判斷「使用者是否靠近某信標」，並粗分為緊鄰（10 公分內）、近（約 1 公尺）、遠（10 公尺內）三級。博物館展品導覽、賣場櫃位推播多採此模式。對機器人而言，這相當於「通過了某個檢查點」。
- 三邊定位（Trilateration）：由三顆以上信標的 RSSI 換算距離後求交點。理論漂亮，但受制於 4.2 節談過的 RSSI 不穩定，實務精度多在 2 至 5 公尺，且需要良好的信標幾何布局。
- 指紋定位：與 Wi-Fi 指紋法完全相同，只是把 AP 換成信標。由於信標密度可控，精度通常略優於 Wi-Fi 指紋。

### 進階：藍牙尋向（Direction Finding）

藍牙 5.1 起加入尋向功能，透過天線陣列量測訊號的到達角（AoA）或離開角（AoD）。由於角度資訊比 RSSI 穩定得多，配合適當設計可把精度推進到公寸甚至公分級——這也是近年 BLE 定位最重要的技術躍進。代價是需要陣列天線硬體，成本與部署複雜度上升。

### 工程注意事項

- 廣播頻率與電池壽命是蹺蹺板：每秒廣播 10 次反應快但耗電，每秒 1 次省電但定位延遲明顯。
- BLE 使用 2.4 GHz 頻段，與 Wi-Fi、微波爐共用，人多的場合干擾顯著（人體含水量高，對 2.4 GHz 吸收特別強）。
- 信標需要定期巡檢換電池——一個常被忽略的長期維運成本，部署數百顆時尤其可觀。

## 4.4 UWB 超寬頻定位

如果說 Wi-Fi 與 BLE 是「用訊號強度猜距離」，超寬頻（UWB，Ultra-Wideband）就是「直接量時間算距離」——這是精度躍升的根本原因。

### 為什麼 UWB 這麼準

UWB 發射的是持續時間僅約數奈秒的極短脈衝，佔用高達 500 MHz 以上的頻寬。時間解析度與頻寬成反比，脈衝越短，接收端就越能精確判定「訊號是哪一刻抵達的」。由於光速約每奈秒 30 公分，能把到達時間解析到次奈秒級，距離就能算到公分級。

更關鍵的是抗多路徑能力：反射訊號比直達訊號晚到，一般窄頻訊號的波形拖得長，直達與反射會疊在一起難以分離；UWB 的脈衝極短，第一個抵達的尖峰（直達波）能與後續反射波清楚分開，接收機只取第一個峰值即可——這正是室內環境中最需要的特性。

### 兩種測距架構

- 雙向測距（TWR，Two-Way Ranging）：標籤發訊給基站、基站回覆，標籤依據來回總時間扣掉處理時間，算出單程飛行時間與距離。優點是不需基站間時間同步；缺點是每次測距都要來回通訊，標籤數量多時通道會塞車。
- 到達時間差定位（TDoA，Time Difference of Arrival）：標籤只單向廣播，多個基站記錄各自的到達時間，由伺服器比較時間差解算位置。優點是標籤只發不收、極省電，且系統容量幾乎不受標籤數限制；缺點是所有基站必須達到奈秒級的時間同步，部署要求較高。

### 精度的實話

廠商文宣常標榜「10 公分精度」，這在視距良好（LOS）、基站幾何布局理想的條件下確實可達成。但真實場域中，一旦人體或設備遮擋形成非視距（NLOS），誤差就會明顯上升；研究比較顯示，較新的 IEEE 802.15.4z 標準在 NLOS 條件下的平均測距誤差可控制在 0.2 公尺以內，較舊標準有明顯改善。工程上的務實預期是：良好部署下公寸級，遮蔽嚴重

處誤差可能達數十公分甚至更多。

### 應用現況

UWB 已從實驗室走入消費與工業場域：手機與追蹤標籤的精確尋物、汽車數位鑰匙（防中繼攻擊的安全測距）、工廠與倉儲的即時定位系統（RTLS）避免堆高機碰撞，乃至於運動賽事中的球體追蹤。對機器人而言，UWB 特別適合作為「不漂移的絕對參考」，與 SLAM 融合使用——這是 4.7 節的主題，也是無人機室內定位的常見方案。

## 4.5 RFID / NFC / QR Code 地標定位

前三節的技術都試圖「連續地」估測位置。本節介紹的是另一種思路：離散地標定位——不追求隨時知道座標，只求「經過特定地點時，能百分之百確定自己在那裡」。這種思路成本極低、可靠度極高，在工業場域反而更受歡迎。

### RFID（無線射頻識別）

被動式 RFID 標籤本身沒有電池，靠讀取器發出的電磁場感應供電並回傳識別碼，成本可低至數元台幣。依頻段分為低頻（LF，數公分）、高頻（HF，約 10 公分）與超高頻（UHF，數公尺）。倉儲 AGV 的經典做法是在地板下埋設 RFID 標籤陣列，車輛底部的讀取器經過時讀到編號，即可校正位置——精度取決於標籤密度，鋪得密就準。

### NFC（近場通訊）

可視為 HF RFID 的一個特化子集，作用距離僅約 4 公分。距離短反而成為優點：讀到就代表「幾乎貼在標籤上」，位置確定性極高。適合充電站對接、工具櫃取放確認等需要「零歧義」的場合。

### QR Code 與資料矩陣碼

QR Code 的最大優勢是零成本——一張印表機列印的貼紙就是一個地標，壞了撕掉重貼即可。這正是亞馬遜倉儲機器人（Kiva 系統）採用的方案：地板上規則貼滿 QR 碼，機器

人底部相機向下拍攝，讀到編號就知道自己在網格的哪一格，同時由影像中 QR 碼的位置與旋轉角推算細部偏移與朝向。數十萬台機器人在全球倉庫中運行，靠的就是這個看似樸素的方案。

| 技術 | 作用距離 | 成本 | 提供資訊 | 典型應用 |
|---|---|---|---|---|
| 被動 RFID | 公分至數公尺 | 極低 | 識別碼 | 倉儲地板標籤、物流追蹤 |
| NFC | 約 4 公分 | 極低 | 識別碼 | 充電對接、工具確認 |
| QR Code | 視相機而定 | 近乎為零 | 識別碼 + 粗略位姿 | 倉儲機器人、教學實驗 |
| AprilTag / ArUco | 公分至數公尺 | 近乎為零 | 完整 6-DoF 位姿 | 機器人導航、相機校正 |

**表 4-1　四種地標式定位技術比較**

## 4.6　視覺地標與 AprilTag / ArUco 定位

QR Code 能告訴機器人「我在幾號格子」，但如果我們想知道更精確的資訊——相機相對於標記的完整三維位置與朝向呢？這就是視覺基準標記（Fiducial Marker）的舞台。AprilTag（密西根大學開發）與 ArUco（西班牙 Córdoba 大學開發，已整合進 OpenCV）是兩套最廣泛使用的系統。

### 為什麼能解出完整位姿

原理其實很直觀：標記是一個已知邊長的正方形，四個角點的三維座標在標記自身座標系中完全已知（例如邊長 10 公分的標記，四角就是（0,0,0）、（0.1,0,0）、（0.1,0.1,0）、（0,0.1,0））。相機拍到這個標記時，四個角點在影像上的二維像素座標也可以精確偵測出來。已知四組「三維點對應二維點」，加上相機內參（第 9 章會詳細介紹），就能解出相機相對標記的旋轉與平移——這在電腦視覺中稱為 PnP 問題（Perspective-n-Point），本書第 21 章會深入探討。

### 偵測流程

16. 影像二值化：把灰階影像轉為黑白，凸顯標記的高對比邊界。
17. 輪廓偵測與四邊形篩選：找出封閉輪廓，保留近似正方形者。
18. 透視校正：把傾斜拍到的四邊形拉回正方形（單應性變換）。
19. 編碼解讀：讀取內部黑白格陣列的位元編碼，比對字典確認 ID，並利用內建的漢明碼糾錯拒絕誤判。
20. 位姿解算：以四角點與相機內參求解 PnP，輸出 6-DoF 位姿。

### AprilTag 與 ArUco 的比較

- AprilTag：糾錯能力與抗誤判率設計得更嚴謹，在遠距離、低解析度、部分遮蔽下仍相對穩健，學術與機器人領域偏好使用。
- ArUco：偵測速度快、字典可自訂大小、與 OpenCV 深度整合（幾行程式碼即可上手），教學與快速原型開發極為方便。

### 在機器人系統中的角色

- 絕對位姿校正：在充電站、電梯口、實驗室門口貼標記，機器人一看到就能消除累積漂移——這是最常見的用法。
- 精確對接：充電、取放料等需要公釐級對位的動作，靠視覺標記做最後階段的視覺伺服（第 24 章）。
- 無人機降落：降落平台上的標記讓無人機精準著陸。
- SLAM 系統的真值來源：教學實驗中，可用密集布置的標記提供近似真值軌跡，用來評估 SLAM 精度（第 31 章專題會用到）。

值得一提的是，視覺標記本質上是「人造的、保證可靠的特徵點」。理解它，其實就是理解第 20 章視覺 SLAM 中「自然特徵點」的簡化版——差別在於自然特徵不保證穩定、不保證唯一、也沒有已知尺寸，這正是視覺 SLAM 困難之所在。

## 4.7 室內定位與 SLAM 的融合

讀到這裡，同學應該已經發現一個模式：室內定位技術與 SLAM 的優缺點恰好互補。

| 特性 | 室內定位（Wi-Fi / BLE / UWB / 標記） | SLAM |
|---|---|---|
| 誤差行為 | 有界，不隨時間累積 | 短期精確，長期漂移 |
| 基礎設施 | 需事先部署與維護 | 不需要，隨到隨用 |
| 初始位置 | 開機即知道自己在哪 | 需從原點出發或執行重定位 |
| 地圖資訊 | 只有座標，不含環境結構 | 產出完整地圖，可供避障與規劃 |
| 更新頻率 | 較低（每秒數次） | 較高（每秒數十次） |

**表 4-2　室內定位與 SLAM 的互補性**

### 三種融合層次

- 鬆耦合（Loosely Coupled）：兩套系統各自獨立運作，各自輸出位置估計，再以卡爾曼濾波等方式加權融合。實作最簡單，是教學與快速原型的首選。
- 緊耦合（Tightly Coupled）：把室內定位的原始量測（UWB 測距值、標記角點）直接當作約束項，放進 SLAM 的後端最佳化中一起求解。精度最佳，因為所有資訊在同一個機率架構下被一致地處理——這也是第 12 章位姿圖優化的自然延伸：UWB 測距就是位姿與基站之間的一條「邊」。
- 輔助初始化與重定位：最務實的用法。SLAM 平時自行運作，只在開機時由 UWB 或視覺標記提供初始位姿，或在跟丟時提供重定位線索。以極低的整合成本，解決了 SLAM 最頭痛的兩個問題。

### 典型架構案例

一台校園服務機器人的完整定位系統可能長這樣：以 2D 光達 SLAM 為主力，提供每秒 10 次的位姿與地圖；在建築物各樓層電梯口與大廳貼 AprilTag，機器人每次經過就校正一

次絕對位姿；充電站前加裝 NFC 確保對接零誤差；跨樓層時由 BLE 信標判斷「現在在幾樓」（這是光達 SLAM 完全無法回答的問題，因為每層樓的雷射輪廓可能長得一模一樣）。每個技術各司其職，這正是系統工程的精髓——沒有萬用的感測器，只有恰當的組合。

## 公式或流程

### 一、三邊定位的數學

設有三個已知位置的基站 $A_1(x_1, y_1)$、$A_2(x_2, y_2)$、$A_3(x_3, y_3)$，量得標籤到各基站的距離分別為 $d_1$、$d_2$、$d_3$，則標籤位置 $(x, y)$ 滿足三個圓方程式：

$$(x - x_i)^2 + (y - y_i)^2 = d_i^2 \quad , \quad i = 1, 2, 3$$

把第一式分別減去第二、三式，平方項相消，得到兩條線性方程式，即可解出 $(x, y)$。

在理想無誤差情況下，三個圓恰好交於一點；但真實量測有誤差，三圓會交出一個「三角形區域」。此時改以最小平方法求解——找一個點使它到三個圓的偏差平方和最小：

$$\min \Sigma_i \left( \sqrt{(x - x_i)^2 + (y - y_i)^2} - d_i \right)^2$$

這個「用多餘量測對抗雜訊」的思路，正是第 12 章 SLAM 後端最佳化的核心精神——同學會發現，從三邊定位到 Bundle Adjustment，數學骨架是一脈相承的。

### 二、TWR 飛行時間計算

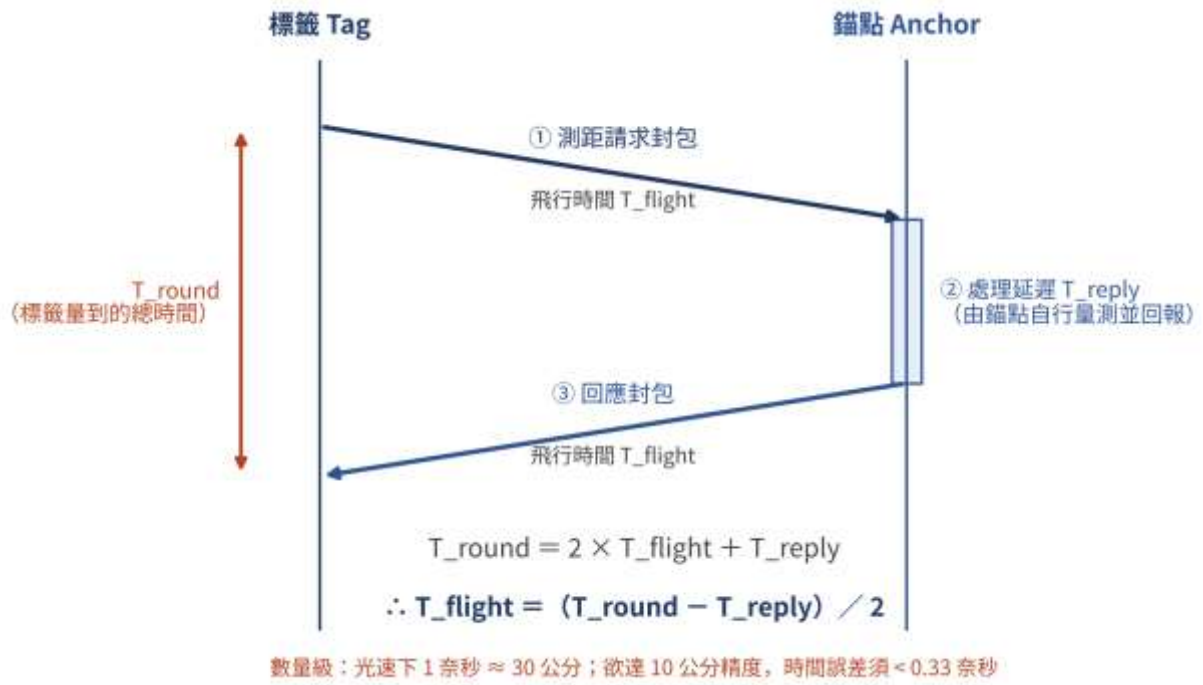
雙向測距中，設標籤發出訊號到收到回覆的總時間為 T_round，基站從收到訊號到發出回覆的處理時間為 T_reply，則單程飛行時間與距離為：

$$T_flight = (T_round - T_reply) / 2$$

$$d = c \times T_flight \quad (c \text{ 為光速，約 } 3 \times 10^8 \text{ m/s})$$

由此可見時間量測精度的重要性：光速下 1 奈秒的誤差就等於 30 公分的距離誤差；若要達到 10 公分精度，時間解析度必須優於 0.33 奈秒。這也解釋了為什麼一般 Wi-Fi 與藍牙晶片做不到——它們的時脈解析度遠不足以支撐這個等級的量測。

**圖 4-3　雙向測距（TWR）的時序關係**




這張圖解釋了 UWB 為什麼準，也解釋了它為什麼貴。T_reply 由錨點自己量測後回報，它消掉了兩端時鐘不同步的影響——這是 TWR 相對於單向測距（需要嚴格時鐘同步）的最大優勢。代價是每測一次距離要來回兩個封包，更新率受限，且每個標籤都要輪流與錨點對話，使得系統容量成為實務瓶頸。

## 三、室內定位技術選型流程

**圖 4-4　室內定位技術選型決策流程**

問題一：需要什麼精度？

├─ 公尺級即可 → Wi-Fi / BLE 指紋定位（成本最低）

├─ 公寸級 → UWB 或 BLE 尋向（需部署基站）

└─ 公分級 → UWB（LOS 良好）或視覺標記（近距離）

↓

問題二：可以在環境中安裝設備嗎？

├─ 完全不行 → 只能用 SLAM（第四至六篇）

├─ 可貼紙張 → QR Code / AprilTag（零成本）

└─ 可施工布線 → UWB 基站 / BLE 信標陣列

↓

問題三：需要連續軌跡還是離散校正？

├─ 連續 → Wi-Fi / BLE / UWB

└─ 離散校正 → RFID / NFC / 視覺標記 + SLAM 為主力

## 4.8　室內定位的當代演進：精度之外的競賽

本章前七節介紹的技術骨架在過去十年相當穩定，但 2025 至 2026 年間出現了幾項值得同學掌握的變化。有趣的是，這些變化的重點大多不在「更準」，而在「更容易部署」與「更普及」。

### 一、藍牙通道探測：挑戰 UWB 的低成本路線

藍牙 6.0 規範納入了通道探測（Channel Sounding）技術，以相位式測距（PBR）取代原本只能靠 RSSI 推估距離的做法。其理論精度約可達公尺級、在良好條件下可接近十公分等

級，而成本與功耗遠低於 UWB，並且能在既有藍牙晶片的演進路徑上實現。

不過產業界的評估相當務實：視線遮蔽、多路徑失真、擴充性與電池續航等現實限制，使它目前尚未成為大規模室內行動定位的可行方案。對同學而言，這是一個很好的教材——一項技術的理論精度與可部署性是兩件事，而工程選型永遠看後者。這正是 4.7 節選型流程圖想傳達的思維。

### 二、Wi-Fi 的 RTT 與 802.11az：標準有了，普及還沒

Wi-Fi 的往返時間測距（RTT，IEEE 802.11mc）能提供次公尺級精度，但支援度仍是瓶頸——據產業統計僅約三成的 Android 裝置支援，iOS 則未開放。改良版的單向 RTT 可在任意 AP 上運作，代價是精度退回三至五公尺，且需要大量後處理與校正。新一代的 802.11az 標準把定位能力正式納入 Wi-Fi 規範，被認為可望把精度推進到公寸等級，但基礎設施的更換週期以年計。

這帶出一個對工程師很重要的判斷：在校園或實驗室環境中，4.2 節的 RSSI 指紋法雖然「土」，卻是唯一在所有裝置上都能用的方法。這也是本章實作 4-1 選擇指紋法而非 RTT 的原因。

### 三、UWB：精度最高，但仍是高階配備

UWB 的定位精度可達三十公分以內，是本章所有無線技術中最準的。但它的普及受限於硬體成本、僅高階 iOS 與 Android 機型支援，以及需要佈建錨點的基礎設施需求。在倉儲、醫院、工廠等「場域固定且價值夠高」的應用中，UWB 已經是成熟方案；在消費性或教學場域，它仍然偏貴。

### 四、視覺標記的新進展：讓標記「隱形」

4.6 節的 AprilTag 與 ArUco 有一個長期被抱怨的缺點：黑白方塊很醜，貼在醫院、辦公室或家裡的牆上非常突兀。2026 年發表的 iMarkers 提出一個有趣的解法——用膽固醇型球狀反射體（CSR）取代印刷墨水，製成的標記對人眼幾乎不可見，但在特定偏振條件下對相機

清晰可辨。

其實測資料值得注意：偵測距離與同尺寸的印刷 ArUco 相當，位姿估測誤差僅差 1.3 至 3.9 個百分點，辨識耗時 21 至 105 毫秒；而在五勒克斯以下的極低照度環境中，偵測率仍有 78 至 89 個百分點，相較之下印刷標記只剩 6 個百分點。對於要在昏暗走廊或夜間巡檢的機器人，這是實質的改善。

另一條路線是用深度學習取代傳統的影像處理管線。例如 YoloTag 把物件偵測模型用於標記偵測，在無人機這類會劇烈晃動、影像常帶動態模糊的平台上，比傳統二值化流程更穩健。這與第 24 章 YOLO 在導航中的角色、第 32 章的 AI 輔助導航是同一條脈絡。

### 五、真正的趨勢：從「調校」走向「免校正」

把上述變化放在一起看，會浮現一個共同方向：部署的簡便性正在取代理論精度，成為系統選型的首要條件。自校正、免指紋採集的系統逐漸成為業界基準線，因為 4.2 節提過的指紋採集與維護成本，在真實商業場域中往往比精度差個一公尺更難接受。

**給同學的三個判準**

一、問「要多準」之前，先問「準了要做什麼」。導航到門口需要三十公分，找到某個房間只需要三公尺。

二、精度之外還有三個維度：部署成本、裝置相容性、維護負擔。本章表 4-1 與 4.7 節的選型流程就是在平衡這四者。

三、絕對定位不必單獨承擔全部精度——和第 7 章的 IMU、第 13 章的 SLAM 融合後，三公尺的 Wi-Fi 定位也足以支撐可靠的導航。這是 4.7 節的核心主張，也是本章最重要的觀念。

## 案例

## 案例 4-1 亞馬遜倉儲機器人：用貼紙撐起全球物流

亞馬遜（Amazon Robotics，前身 Kiva Systems）的倉儲機器人是「低科技解決高難度問題」的教科書案例。數十萬台橘色搬運機器人在全球倉庫中穿梭，它們的定位方案卻樸素得驚人：地板上以固定間距貼滿 QR 碼，機器人底部的向下相機讀取編號確認格位，再由 QR 碼在畫面中的偏移與旋轉計算細部位姿；格位之間則靠輪子里程計短距離推算。

這個方案的高明之處在於徹底利用了環境的可控性：倉庫是自家蓋的，地板可以隨意貼、光線可以自己打、路線可以規定成網格。既然如此，何必讓機器人辛苦地做視覺 SLAM？相較之下，服務機器人進入的是別人的建築、無法改造環境，才不得不倚賴 SLAM。這個對比給同學一個重要啟示：選擇定位技術之前，先問「我能改造環境到什麼程度」——這往往比演算法本身更關鍵。

## 案例 4-2 醫院裡的 UWB 即時定位系統

大型醫院常面臨一個昂貴的問題：輸液幫浦、超音波推車等移動式設備經常「不知道被推到哪裡去了」，護理人員每天花大量時間找設備，醫院只好超額採購。UWB 即時定位系統（RTLS）的解法是：在天花板部署基站網格，每台設備貼一顆 UWB 標籤，以 TDoA 架構單向廣播，伺服器即時解算所有標籤位置並顯示在樓層平面圖上。

這個案例展示了 TDoA 架構的價值：設備標籤只發不收，一顆電池能撐數年；且系統容量幾乎不受標籤數量限制，一家醫院上萬個標籤也不會塞車。醫院場域的挑戰則在於金屬設備與人流造成的 NLOS——這也是實務精度往往達不到文宣數字的原因。同樣的架構稍作調整，就能用於工廠的堆高機防撞、倉儲的人員安全區域管理，乃至於機器人的絕對位置校正。

## 案例 4-3 教室裡的 AprilTag 導航實驗

大學智慧機器人教室的一個典型教學實驗：在教室四面牆上貼十張 AprilTag，每張標記的世界座標事先量測記錄。教學小車搭載一顆前向相機，只要視野中出現任一標記，就能解算出自己相對該標記的 6-DoF 位姿，再換算為世界座標——瞬間獲得不漂移的絕對定位。

這個實驗的教學價值極高：第一，它讓同學親手驗證「四個角點加相機內參就能解出位姿」這件看似神奇的事；第二，把標記遮住一半、拉遠距離、關掉燈光，同學能親身體會視覺定位的脆弱點；第三，最精彩的是把它與光達 SLAM 並行執行，在 RViz 中同時顯示兩條軌跡——SLAM 軌跡平滑但慢慢偏離，AprilTag 軌跡跳動但始終正確。這張圖勝過千言萬語，一眼道盡本章 4.7 節的融合動機。

## 實作活動

### 實作 4-1　教室 Wi-Fi 訊號定位熱區圖

目的：親手完成一次完整的指紋定位流程——建庫、量測、比對、驗證，理解 RSSI 定位的能力與極限。本實作只需手機，適合高中與大學部共同進行。

**器材**

- 智慧型手機（Android 建議安裝 Wi-Fi 分析類 App；iOS 因系統限制較嚴，可改用 AirPort 工具或以 BLE 信標替代）。
- 方格紙或試算表軟體、捲尺、標示參考點的膠帶。

**步驟**

12. 場域規劃：選定一間教室，以捲尺量出長寬，在地面以 2 公尺間隔貼出格點標記（一間普通教室約 12 至 20 個參考點），並在方格紙上編號。
13. 離線建庫：在每個參考點站定，面向固定方向（例如一律面向講台，減少人體遮蔽造成的差異），記錄可見的前五強 AP 的名稱與 RSSI 值，每點量測三次取平均，填入試算表。
14. 繪製熱區圖：針對訊號最強的那台 AP，把各參考點的 RSSI 值以色階填入方格紙（強為紅、弱為藍），觀察訊號強度的空間分布是否符合「離 AP 越遠越弱」的預期。

15. 線上定位測試：由一位同學閉眼被帶到某個未告知的參考點，量測當下指紋；其他同學以 KNN 方法（計算與資料庫中每筆指紋的歐氏距離，取最相近的 3 筆平均座標）推算位置。
16. 誤差統計：重複測試至少 8 次，記錄每次的估測位置與真實位置，計算平均誤差與最大誤差。
17. 干擾實驗：請全班同學聚集在測試點周圍，重新量測同一點的指紋，比較 RSSI 變化與定位誤差的變化。

**討論問題**

19. 你們量到的平均定位誤差是多少？與課文提到的 3 至 8 公尺是否相符？誤差最大的參考點在哪裡？為什麼是那裡？
20. 人群聚集讓誤差變大了多少？這對「商場尖峰時段的室內導航」意味著什麼？
21. 如果把參考點間隔從 2 公尺縮小到 1 公尺，精度會提升多少？建庫工作量增加多少？這個代價值得嗎？
22. （大學部）若要把這套系統與第 14 章的光達 SLAM 融合，你會採用 4.7 節的哪一種耦合方式？為什麼？

**延伸挑戰（大學部）**

以 Python 撰寫 WKNN 定位程式，讀入建好的指紋資料庫與即時量測值，自動輸出估測座標與誤差統計；進一步比較 K = 1、3、5 時的精度差異，並嘗試加入手機 IMU 的步數偵測做軌跡平滑，觀察誤差是否下降。

## 延伸閱讀

20. P. Bahl and V. N. Padmanabhan, "RADAR: An In-Building RF-Based User Location and Tracking System," IEEE INFOCOM, 2000.（Wi-Fi 指紋定位的開山之作，必讀經典）

21. H. Liu et al., "Survey of Wireless Indoor Positioning Techniques and Systems," IEEE Transactions on Systems, Man, and Cybernetics, 2007.（室內定位技術的完整綜述）

22. IEEE Std 802.15.4z-2020, Enhanced Ultra Wideband (UWB) Physical Layers and Associated Ranging Techniques.（UWB 測距的現行標準文件）

23. E. Olson, "AprilTag: A Robust and Flexible Visual Fiducial System," ICRA, 2011.（AprilTag 原始論文，說明糾錯設計思路）

24. S. Garrido-Jurado et al., "Automatic Generation and Detection of Highly Reliable Fiducial Markers Under Occlusion," Pattern Recognition, 2014.（ArUco 原始論文）

25. OpenCV 官方文件之 ArUco 模組教學（docs.opencv.org）.（附完整程式範例，實作首選參考）

26. Bluetooth SIG, Bluetooth Core Specification 5.1 之 Direction Finding 章節.（AoA / AoD 尋向技術規格）

## 從室內定位到導航感測器：本章的位置回顧

本章介紹的技術有一個共同特徵：它們都需要在環境中預先佈建些什麼——AP、信標、UWB 錨點、RFID 標籤或視覺標記。這是它們的力量來源（提供不漂移的絕對位置），也是它們的根本限制（進入沒佈建過的環境就完全失效）。

接下來的五章，我們轉向另一類感測器：它們不需要佈建，只靠物理定律或環境本身既有的資訊工作。第 5 章的 GNSS 靠衛星，第 6 章的地磁靠地球磁場，第 7 章的 IMU 靠慣性，第 8、9 章的光達與相機靠環境的幾何與紋理。代價是——它們都不能直接給出「我在建築物裡的第幾公尺」這個答案。

這正是 SLAM 存在的理由。當沒有佈建、也沒有地圖時，機器人只能一邊走一邊畫地圖、一

邊用剛畫好的地圖定位自己。第三篇的數學、第四篇以後的演算法，全都在回答這個問題。而本章要同學記住的是：如果環境允許佈建，別逞強——一個三十塊錢的 AprilTag，有時候比一整套 SLAM 系統更可靠。

## 小結

7. GPS 在室內失效有三個原因：訊號極度微弱難以穿透建材、多路徑反射造成測距失真、可見衛星幾何條件惡化使誤差被放大。
8. Wi-Fi RSSI 因室內傳播環境複雜，不適合直接測距，主流採指紋定位：離線建立訊號資料庫、線上以 KNN 比對，精度約 3 至 8 公尺，零硬體成本但需維護資料庫。
9. BLE 信標可依需求布置，有鄰近偵測、三邊定位、指紋定位三種模式；藍牙 5.1 起的尋向功能以 AoA / AoD 大幅提升精度，代價是需要陣列天線。
10. UWB 以奈秒級短脈衝直接量測飛行時間，能分離直達波與反射波，是精度最高的無線定位技術；TWR 免同步但容量有限，TDoA 省電且容量大但需基站精確同步。
11. RFID、NFC、QR Code 提供離散但極可靠的地標定位，成本低廉，適合可改造環境的場域，亞馬遜倉儲機器人即為代表案例。
12. AprilTag / ArUco 等視覺標記能由四個角點與相機內參解出完整 6-DoF 位姿（PnP 問題），是機器人絕對位姿校正、精確對接與 SLAM 精度評估的利器。
13. 室內定位與 SLAM 恰好互補：前者誤差有界但需基礎設施，後者自給自足但會漂移；融合方式分鬆耦合、緊耦合與輔助初始化重定位三種層次。

下一章，我們把視野從室內拉到戶外，探討無線電導航與衛星導航——GPS / GNSS 究竟如何運作？為什麼 RTK 能把公尺級誤差壓到公分級？以及校園巡檢機器人如何在室內外之間無縫切換定位模式。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

## 第 5 章

## 無線電導航與衛星導航

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

## 本章在全書的位置

第 4 章解決了室內的絕對定位，本章處理室外——而且是地球上任何一處的室外。衛星導航是人類史上覆蓋範圍最大的定位基礎建設，也是機器人身上唯一「不必佈建任何東西就能知道自己在地球哪裡」的感測器。

但本章真正想讓同學建立的，不是「GPS 很好用」，而是三個更深的觀念。第一，GNSS 的數學骨架——未知數比觀測多一個（時鐘誤差），於是需要多一顆衛星——與第 11 章的狀態估測、第 12 章的圖優化是同一種思維。第二，幾何決定精度：DOP 的概念在第 21 章的對極幾何、第 17 章的點雲配準中會以不同面貌再度出現。第三，沒有任何單一感測器可靠，融合才是答案——5.6 節的 GNSS / INS / SLAM 三重架構，正是全書後半部的縮影。

### 圖 5-0　第 5 章與全書的連結地圖

【承接前章】

第 4 章　室內的絕對定位（需佈建）　→　本章：室外的絕對定位（不需佈建）

第 4 章的多路徑、NLOS、幾何條件惡化，在本章得到量化的名字：UERE 與 DOP

【本章負責】

測距交會的三維版本 + 時鐘未知數 = 虛擬距離定位

差分與載波相位 = 從公尺級推進到公分級

訊號品質判斷 = 機器人何時該相信 GNSS

【往下延伸】

→ 第 6、7 章　GNSS 失效時的遞補：地磁提供航向、IMU 提供短期推算

→ 第 11 章　本章的「多觀測解多未知數」正式化為狀態估測

→ 第 12 章　GNSS 觀測成為位姿圖中的絕對約束邊

→ 第 18、26 章　戶外光達 SLAM 與無人機導航中的 GNSS 角色

→ 第 26 章　GNSS 拒止環境下的自主導航

## 學習目標

上一章我們探討了室內定位；本章走出戶外，探討覆蓋全球的無線電與衛星導航系統。這是機器人「絕對定位」能力的主要來源，也是理解「為什麼自駕車與無人機仍然需要 SLAM」的關鍵一課。學完本章之後，同學應能夠達成下列目標：

24. 說明 GNSS 的三大組成（太空、控制、使用者區段）與衛星定位的基本原理，並解釋為何至少需要四顆衛星。
25. 區分三角定位、三邊定位與到達時間差定位三種幾何定位法的差異與適用場合。
26. 說明載波相位量測與 RTK 的原理，理解為何 RTK 能將公尺級誤差壓縮到公分級。
27. 比較 5G、LoRa、Wi-Fi RTT 等地面無線電定位技術在機器人應用上的潛力與限制。
28. 列舉並解釋無線電定位的主要誤差來源：電離層、對流層、多路徑、遮蔽、幾何條件與干擾。
29. 說明 GNSS 如何與 IMU、SLAM 融合，並設計一套校園巡檢機器人的室內外混合導航策略。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 全球衛星導航系統 | GNSS | 各國衛星定位系統的統稱，含 GPS、GLONASS、Galileo、BeiDou。 |
| 虛擬距離 | Pseudorange | 由訊號傳播時間乘光速得到的距離，因含接收機時鐘誤差故稱「虛擬」。 |
| 星曆 | Ephemeris | 衛星軌道參數，用來精確計算衛星在任一時刻的位置。 |
| 精度稀釋因子 | DOP | 描述衛星幾何分布對定位誤差的放大倍率，值越小越好。 |

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 載波相位 | Carrier Phase | 量測載波波形的相位，精度遠高於虛擬距離，是 RTK 的基礎。 |
| 整週波未定值 | Integer Ambiguity | 載波相位量測中未知的整數週期數，解出它才能達成公分級精度。 |
| 即時動態定位 | RTK | 以基準站差分修正達成公分級定位的技術。 |
| 差分 GNSS | DGNSS | 利用已知座標的基準站計算誤差修正量，播送給移動站使用。 |
| 星基增強系統 | SBAS | 以同步衛星播送修正訊息的廣域增強系統，如 WAAS、EGNOS。 |
| 網路 RTK | NTRIP | 透過網際網路傳送 RTK 修正資料的通訊協定。 |
| GNSS 拒止環境 | GNSS-denied | 衛星訊號被遮蔽或干擾而無法使用的環境，如室內、隧道、都市峽谷。 |
| 都市峽谷效應 | Urban Canyon | 高樓夾道遮蔽衛星並造成強烈多路徑反射的現象。 |

## 5.1 GPS / GNSS 基本原理

GPS（Global Positioning System）是美國於 1970 年代開始建置、1990 年代全面運作的衛星定位系統。今日它已不是唯一選擇：俄羅斯的 GLONASS、歐盟的 Galileo、中國的北斗（BeiDou）都已達成全球覆蓋，加上日本的 QZSS 與印度的 NavIC 兩套區域系統，統稱為全球衛星導航系統（GNSS，Global Navigation Satellite System）。截至 2026 年 3 月，四大系統在軌運作的衛星合計約 130 顆，其中 GPS 約 31 顆，分布於 6 個軌道面、約兩萬公里高的中軌道。2020 年後生產的智慧型手機大多同時支援四大系統——這正是近年手機定位明顯變準的原因：可用衛星變多，幾何條件自然變好。

### 三大區段

- 太空區段（Space Segment）：在軌衛星群。每顆衛星攜帶高精度原子鐘，持續廣播自己的識別碼、時間戳記與軌道參數（星曆）。
- 控制區段（Control Segment）：地面監測站網路與主控站。持續追蹤衛星、修正軌道與時鐘偏差，並把最新星曆上傳給衛星。
- 使用者區段（User Segment）：接收機。只接收、不發射——這是理解 GNSS 的關鍵：衛星並不知道你在哪裡，定位完全由你的接收機自行完成，因此使用者數量可以無限多。

### 定位的基本原理：測距交會

GNSS 定位的本質是三維的三邊定位。衛星在訊號中蓋上「我在時刻 t 發出這個訊號」的時間戳記；接收機收到時記下自己的時刻，兩者相減得到傳播時間，乘上光速就是距離。已知衛星位置（由星曆算出）與距離，接收機就位於一個以該衛星為球心的球面上。三顆衛星的三個球面相交，理論上就能定出位置。

### 為什麼需要第四顆衛星

問題出在時鐘。衛星上的原子鐘造價高昂、精度達奈秒級；但接收機裡只是一顆廉價石

英振盪器，與衛星時間隨時可能相差數微秒。而光速下 1 微秒就等於 300 公尺的距離誤差——這誤差會讓三個球面根本交不到一起。

解法非常巧妙：把接收機的時鐘誤差當作第四個未知數，一起解出來。未知數變成四個（x、y、z、Δt），就需要四條方程式，也就是四顆衛星。這也是為什麼由於接收機時鐘不準，量到的距離不能稱為真距離，而稱為虛擬距離（Pseudorange）。

這個設計的副作用是：GNSS 接收機在完成定位的同時，也把自己的時鐘校正到了原子鐘等級的精度。全球的電信基地台、電網、金融交易系統都靠 GNSS 授時同步——衛星導航系統對現代社會的重要性，其實遠超過「幫你導航」。

## 5.2 三角定位、三邊定位與時間差定位

無線電定位的幾何方法有三大類，同學務必分清楚——這三個名詞在中文文獻中常被混用，但原理截然不同。

### 一、三角定位（Triangulation）：用角度

量測目標相對於兩個已知基準點的方位角，兩條射線的交點即為目標位置。傳統測量學、航海用的無線電測向（RDF）、以及第 4 章提到的藍牙 AoA 尋向，都屬於這一類。特點是需要方向性天線或天線陣列，但不需要精確的時間量測。

### 二、三邊定位（Trilateration）：用距離

量測目標到各已知點的距離，求球面（或圓）的交點。GNSS、UWB 的 TWR 都屬於這一類。特點是需要精確的時間或訊號強度量測，但不需要知道方向。第 4 章的公式已經介紹過二維情況，GNSS 則是三維版本再加上時鐘未知數。

### 三、到達時間差定位（TDoA）：用時間差

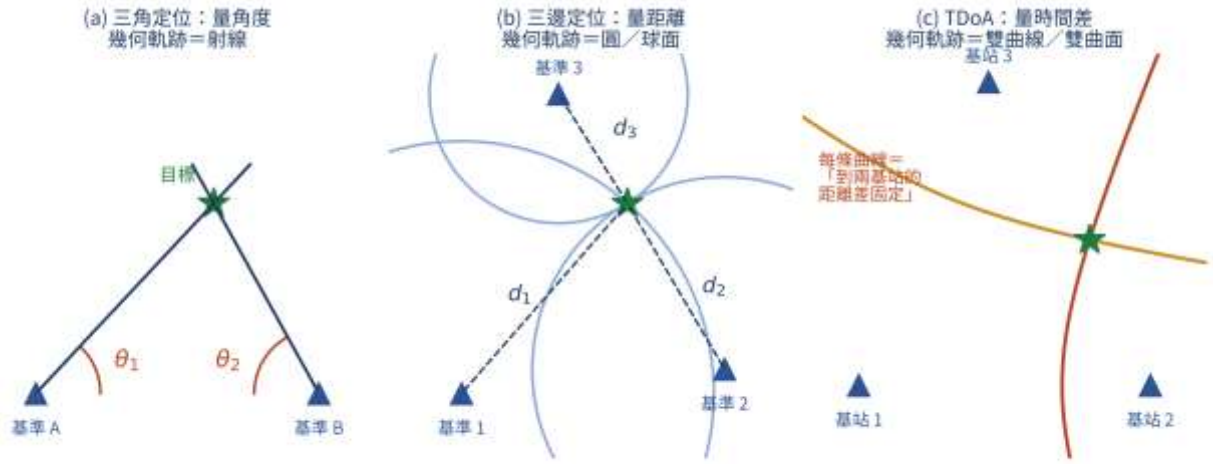
不量絕對距離，只量「訊號抵達兩個基站的時間差」。時間差固定的點集合，在幾何上構成一條雙曲線（三維中為雙曲面），兩組以上雙曲面的交點即為目標位置——因此又稱

雙曲線定位。第 4 章介紹的 UWB TDoA 架構、以及二戰時期的羅蘭（LORAN）長波導航系統都採用此法。特點是目標端只需發射或只需接收，極省電且系統容量大，代價是各基站必須精確時間同步。

| 方法 | 量測物理量 | 幾何軌跡 | 需要條件 | 典型應用 |
|---|---|---|---|---|
| 三角定位 | 角度 | 射線 | 方向性天線 | BLE AoA、無線電測向 |
| 三邊定位 | 絕對距離 | 圓 / 球面 | 精確計時或已知發射功率 | GNSS、UWB TWR |
| TDoA | 距離差 | 雙曲線 / 雙曲面 | 基站間精確同步 | UWB TDoA、LORAN |

**表 5-1　三種無線電定位幾何方法比較**

**圖 5-1　三種無線電定位幾何方法的軌跡比較**




這張圖是本節最值得記住的一頁。三種方法的差異不在技術細節，而在「一次量測把目標限制在什麼樣的幾何形狀上」——射線、圓（球面）、雙曲線（雙曲面）。定位的本質，就是用多個這樣的限制求交集。

值得注意的是，這個「觀測把狀態限制在某個流形上，再求交集」的思路，正是第 11 章狀態估測與第 12 章圖優化的幾何直覺。第 13 章的掃描匹配、第 21 章的對極幾何（一次對應把相機運動限制在對極線上），本質上都在做同一件事。同學若能在這裡建立起這個直覺，

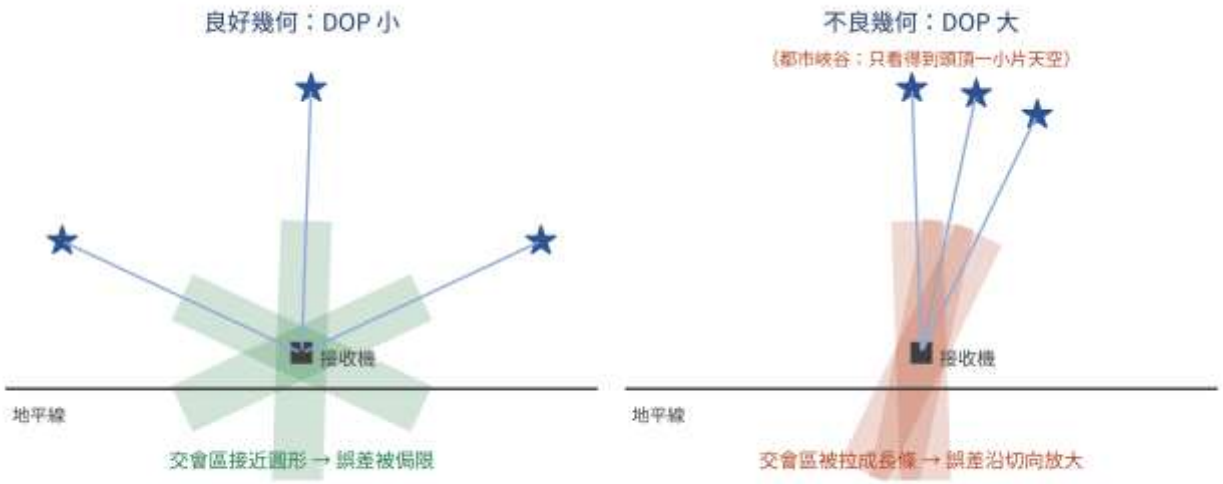
後面幾篇會輕鬆很多。

### 精度稀釋因子（DOP）

無論哪一種方法，基準點（衛星或基站）的幾何分布都會直接影響精度。想像兩顆衛星在天空中幾乎同一方向：它們的球面交角極小，量測誤差稍有變化，交點就會沿著切線方向大幅滑動。反之，衛星均勻分布在四面八方時，交角接近垂直，誤差被侷限在小範圍內。

這個放大效應以精度稀釋因子（DOP）量化：實際定位誤差約等於測距誤差乘上 DOP 值。開闊天空下 DOP 常小於 2，都市峽谷或室內則可能飆到 10 以上——這正是第 4 章所說「幾何條件惡化」的量化解釋。

**圖 5-2　衛星幾何分布對定位精度的影響（DOP）**




圖中每顆衛星畫出一條半透明的「測距不確定帶」——帶的寬度就是測距誤差，方向垂直於視線。左圖的衛星散布四方，三條帶子交會出接近圓形的小區域；右圖的衛星擠在頭頂一小片天空，帶子方向幾乎平行，交會區被拉成細長的斜條。同樣的測距誤差，右圖的位置不確定性大了好幾倍。

這解釋了幾個實務現象：為什麼垂直方向（高程）的 GNSS 精度總是比水平方向差約兩到三倍——因為地面以下沒有衛星，垂直方向的幾何先天就是單邊的；為什麼都市峽谷中「衛

星數很多但精度很爛」——數量不等於幾何；為什麼巡檢機器人應該把 HDOP 而非衛星數當作主要的品質指標。

> **DOP 的家族與門檻值**
>
> GDOP（幾何）、PDOP（位置）、HDOP（水平）、VDOP（垂直）、TDOP（時間）——機器人導航最常用 HDOP。
>
> 實務經驗值：HDOP < 2 為優良，2～5 可用，5～10 勉強，> 10 應視為不可信。
>
> 這組門檻值會在本章 5.7 節的巡檢系統設計與實作 5-1 的勘測表中直接派上用場。

## 5.3 RTK-GNSS 與高精度戶外定位

一般手機 GNSS 的精度約在 3 至 10 公尺，這對「找路」夠用，但對自駕車、農機自動駕駛、無人機測繪而言遠遠不足。要把誤差壓到公分級，需要兩項關鍵技術：差分修正與載波相位量測。

### 一、差分的思想

GNSS 的誤差有一大部分是「區域共通」的：電離層與對流層的延遲、衛星時鐘與軌道的微小偏差，在相距數公里的兩台接收機上幾乎完全相同。

差分 GNSS（DGNSS）利用這個特性：在一個座標已經精確測定的位置架設基準站（Base Station），基準站把「算出來的位置」與「已知的真實位置」相比，就能反推出當下的誤差量，再把修正量播送給附近的移動站（Rover）。移動站扣掉共通誤差後，精度可從公尺級提升到公寸級。

### 二、載波相位：精度躍升的關鍵

一般 GNSS 使用的是調變在載波上的「測距碼」，其碼片長度約數百公尺，量測解析度自然受限。但載波本身的波長只有約 19 公分（L1 頻段），如果能量測載波的相位，解析度

可達波長的百分之一，也就是公釐級。

代價是產生了一個新的未知數：接收機能精確量出「不滿一週的相位」，卻不知道訊號總共走了幾個完整波長——這就是整週波未定值（Integer Ambiguity）問題。RTK 系統的核心技術，就是快速且可靠地把這個整數解出來（稱為 Ambiguity Resolution 或「固定解」）。一旦固定成功，定位精度可達公分級；固定失敗時只能維持「浮動解」，精度退回公寸級。

## 三、RTK 系統架構

**圖 5-3　RTK-GNSS 系統架構**

基準站（Base）：座標已知，接收衛星訊號 → 計算誤差修正量

↓　（電台、4G / 5G 或網際網路 NTRIP 傳送）

移動站（Rover）：接收衛星訊號 + 基準站修正量

↓

雙差分運算 → 消除共通誤差 → 解算整週波未定值

↓

固定解（Fixed）：公分級定位　/　浮動解（Float）：公寸級定位

## 四、網路 RTK 與實務考量

自建基準站成本高、覆蓋有限，因此發展出網路 RTK：由國家或業者建置基準站網路，透過 NTRIP 協定以網際網路播送修正資料。台灣的內政部國土測繪中心即建置有 e-GNSS 系統，提供全台的即時定位服務。

- 基線長度限制：移動站與基準站距離越遠，「共通誤差」的假設越不成立，通常建議在數十公里內。

- 初始化時間：從開機到固定解通常需要數十秒，過隧道或樹下失鎖後需要重新固定。
- 仍需開闊天空：RTK 消除的是大氣與衛星誤差，對「遮蔽」與「多路徑」無能為力——樹下、高樓旁的 RTK 一樣不可靠。這點對機器人應用至關重要。

值得一提的是 Galileo 的高精度服務（HAS）等新一代技術，可透過衛星直接播送修正資料，達到約 20 公分等級的精度而不需地面基準站，這對大範圍戶外機器人應用是相當實用的折衷方案。

## 5.4 5G / LoRa / Wi-Fi RTT 在機器人定位的應用

衛星訊號來自兩萬公里外，天生脆弱。地面基地台的訊號則強得多，能不能拿來定位？這是近年通訊與定位融合的熱門方向。

### 一、5G 定位

5G 從標準制定之初就把定位納入設計，具備三項有利條件：毫米波與大頻寬帶來高時間解析度、大規模天線陣列（Massive MIMO）可量測到達角、小基站密集布建縮短了距離。5G 定位綜合使用到達時間差、到達角與往返時間等方法，在理想條件下可達公尺甚至次公尺級。對工廠 AGV、園區自駕車而言，這意味著「通訊與定位共用同一套基礎設施」，極具吸引力。限制在於：定位精度高度依賴基站密度，且需要電信業者開放相關能力。

### 二、LoRa 定位

LoRa 是長距離、低功耗的物聯網通訊技術，單一閘道器覆蓋可達數公里。它的定位能力來自 TDoA——多個閘道器記錄同一封包的到達時間差。精度僅約數十至數百公尺，遠不及前述技術，但勝在覆蓋廣、耗電極低、成本低廉。適用場景是「大範圍粗定位」：農場設備追蹤、山區資產監控，或第 4 章案例中「UWB 精確測距 + LoRa 長距離回傳」的混合架構。

### 三、Wi-Fi RTT（往返時間）

IEEE 802.11mc 標準引入的 Fine Timing Measurement（FTM）機制，讓裝置能與 AP

交換時間戳記，直接量測往返時間換算距離——這與第 4 章的 RSSI 猜距離有本質差異。精度可達 1 至 2 公尺，且不需要建立指紋資料庫。限制是 AP 與裝置雙方都必須支援 FTM，目前普及率仍是主要瓶頸。

| 技術 | 涵蓋範圍 | 典型精度 | 基礎設施需求 | 機器人應用定位 |
|---|---|---|---|---|
| GNSS | 全球 | 3–10 公尺 | 無（衛星已在天上） | 戶外主力 |
| RTK-GNSS | 基準站數十公里內 | 公分級 | 基準站或網路服務 | 農機、測繪、自駕 |
| 5G | 基站覆蓋範圍 | 公尺至次公尺 | 密集基站 | 園區、工廠 |
| LoRa | 數公里 | 數十至數百公尺 | 閘道器 | 大範圍粗追蹤 |
| Wi-Fi RTT | 室內數十公尺 | 1–2 公尺 | 支援 FTM 的 AP | 室內輔助 |

**表 5-2　無線電定位技術的機器人應用比較**

## 5.5　無線電定位誤差來源：多路徑、遮蔽、干擾

理解誤差，才能判斷什麼時候可以信任 GNSS、什麼時候必須切換到 SLAM。GNSS 的誤差可分為三大類。

### 一、傳播路徑誤差（可用差分消除）

- 電離層延遲：距地表約 50 至 1000 公里的電離層含大量自由電子，會延遲訊號。這是單頻接收機最大的誤差源，可達數公尺，且隨太陽活動劇烈變化。雙頻接收機可利用不同頻率的延遲差異加以消除——這也是雙頻 GNSS 較昂貴但精準得多的原因。
- 對流層延遲：大氣層底部的水氣與氣壓造成的延遲，約數十公分至數公尺，可用模型估計修正。
- 衛星時鐘與軌道誤差：控制區段已持續修正，殘餘誤差約在公尺以下。

### 二、局部環境誤差（差分無法消除）

- 多路徑：訊號經建築、地面、水面反射後抵達，路徑變長使測距偏大。這是都市與校園環境的主要誤差來源，可達數公尺甚至更多。抑制方法包括扼流圈天線、訊號品質篩選演算法，但無法根除。
- 遮蔽：建築、樹冠、隧道直接擋住訊號，導致可用衛星數不足或幾何條件惡化。台灣校園常見的大王椰子與榕樹樹冠，就足以讓 RTK 從固定解掉回浮動解。
- 都市峽谷效應：高樓夾道時，天空只剩狹長一條，可見衛星集中在同一方向使 DOP 惡化，同時牆面反射造成強烈多路徑——遮蔽與多路徑雙重打擊，這是自駕車在市區最頭痛的定位問題。

### 三、人為干擾

- 干擾（Jamming）：以強訊號淹沒微弱的衛星訊號，使接收機失鎖。市面上非法的小型干擾器即可癱瘓周圍數十公尺內的 GNSS。
- 欺騙（Spoofing）：發射偽造的衛星訊號，讓接收機算出錯誤但「看起來正常」的位置。這比干擾更危險——系統不會報錯，只會安靜地把你導向錯誤的地方。對無人機與自駕車而言，這是嚴肅的資安議題。

這三類誤差的存在，正是「機器人不能只靠 GNSS」的根本理由。一台在校園樹蔭下巡檢的機器人，隨時可能遇到訊號中斷、跳點數公尺、或 RTK 掉解——若導航系統完全依賴 GNSS，機器人就會在這些時刻失去方向。解法就是下一節的融合。

## 5.6 無線電導航與 IMU / SLAM 的融合

第 4 章我們看過室內定位與 SLAM 的互補；本節的 GNSS 與 IMU / SLAM 融合，是同一思想在戶外的展開，也是目前最成熟的感測融合應用之一。

### GNSS 與 IMU 的天作之合

這兩者的誤差特性恰好完全相反：

- IMU：更新率極高（數百 Hz）、短時間內極為精確、完全不受遮蔽與干擾影響；但積分漂移使誤差隨時間快速累積（第 7 章詳述）。
- GNSS：更新率低（1 至 10 Hz）、單次量測有雜訊；但誤差有界，長時間不會漂移。

融合的邏輯因此非常自然：以 IMU 提供高頻率的連續軌跡，以 GNSS 定期把累積漂移「拉回來」。這套組合稱為 GNSS / INS 組合導航，是航空、船舶、飛彈與自駕車的標準配備。實作上通常以卡爾曼濾波器（第 11 章）進行，濾波器會持續估計並補償 IMU 的偏差（Bias），使得即使 GNSS 短暫中斷（過隧道、樹下），IMU 仍能維持數秒到數十秒的可用精度——這段時間稱為「慣性導航的推算能力」。

### 加入 SLAM：三重保險

GNSS / INS 在長隧道或室內仍會失效，此時就需要 SLAM 接手。完整的三重架構如下：

**圖 5-4 GNSS / INS / SLAM 三重融合架構**

IMU（數百 Hz）：提供高頻連續運動估測，短期精確

+

GNSS（1–10 Hz）：戶外提供絕對位置，消除長期漂移

+

SLAM（10–30 Hz）：以環境特徵提供相對約束，GNSS 失效時的主力

↓

融合濾波器 / 圖優化後端（第 11、12 章）

↓

連續、不漂移、全天候可用的位姿估測

### 關鍵工程問題：座標系統一

融合前必須先解決一個容易被忽略的問題：GNSS 給的是經緯度（WGS84 地理座標），SLAM 用的是以出發點為原點的區域直角座標。兩者必須先轉換到同一個座標系——通常做法是把經緯度轉為東北天（ENU）局部直角座標，並以第一個 GNSS 定位點作為原點。此外還需要處理時間同步（各感測器時間戳記對齊）與外參校正（GNSS 天線、IMU、光達三者的相對安裝位置）。這些「不起眼」的工程細節，往往才是實作能否成功的決定因素——第 10 章的座標轉換與 TF Tree 會完整處理這個主題。

### 信任度切換

實務系統還需要一個「裁判」機制，判斷此刻該相信誰。常用依據包括：可用衛星數、DOP 值、RTK 解算狀態（固定解 / 浮動解 / 單點解）、以及 GNSS 與 SLAM 估測之間的差異是否過大。當機器人從戶外進入室內，系統應平滑地把權重從 GNSS 轉移到 SLAM，而不是突然跳變——這正是下一節校園場景的核心挑戰。

## 5.7 校園巡檢機器人的室內外混合導航

把本章與前一章的技術整合起來，我們可以完整設計一台校園巡檢機器人的導航系統。這個場景之所以經典，正是因為它涵蓋了所有困難情境。

### 場域分析：一趟巡檢的四種環境

- 開闊操場與停車場：天空開闊，GNSS / RTK 表現最佳，可作為定位主力。
- 樹蔭步道與建築物旁：遮蔽與多路徑同時發生，GNSS 可能跳點數公尺或掉解。此區必須以 SLAM 為主、GNSS 為輔，且要有能力判斷 GNSS 不可信。
- 穿堂與騎樓：半室內，GNSS 時有時無，是最容易出錯的「過渡區」。
- 室內走廊與教室：GNSS 完全失效，純靠光達 / 視覺 SLAM，並以第 4 章的 AprilTag 或 BLE 信標提供樓層資訊與絕對校正。

### 系統設計策略

21. 地圖分層：戶外使用以 GNSS 座標為基準的全域地圖，室內使用各棟建築的 SLAM 區域地圖，兩者以「門口」作為錨點連接，形成拓樸式的地圖網路（第 3 章的拓樸地圖在此派上用場）。
22. 感測器配置：RTK-GNSS 天線（戶外主力）+ IMU（連續性骨幹）+ 3D 或 2D 光達（SLAM 主力）+ 相機（視覺標記與語意辨識）。
23. 模式切換機制：以 GNSS 品質指標（衛星數、DOP、解算狀態）與 SLAM 匹配品質共同決策，在過渡區採漸進式權重調整而非硬切換。
24. 過渡區強化：在每棟建築出入口貼設 AprilTag，作為「戶外座標系」與「室內座標系」之間的橋樑——機器人一看到標記，就同時知道自己在兩個座標系中的位置，完成無縫接軌。
25. 失效備援：若所有定位手段同時異常（例如強烈干擾加上光達被遮），機器人應停止移動並回報，而非帶著錯誤位姿繼續行駛——安全設計的基本原則。

這套架構將在第 31 章的專題中實際落地。同學此刻只需掌握一個核心觀念：真實世界的機器人導航，從來不是「選一種最好的技術」，而是「讓多種技術在各自擅長的場合接力」。這正是本書第二篇逐一介紹六大感測器的用意——你必須認識每一位隊員的長處與弱點，才能當一個好教練。

## 公式或流程

### 一、虛擬距離方程式

設接收機位置為 $(x, y, z)$、時鐘偏差為 $\Delta t$，第 $i$ 顆衛星的已知位置為 $(x_i, y_i, z_i)$，量得的虛擬距離為 $\rho_i$，則：

$$\rho_i = \sqrt{(x - x_i)^2 + (y - y_i)^2 + (z - z_i)^2} + c \cdot \Delta t$$

其中 $c$ 為光速。式中未知數為 $x$、$y$、$z$、$\Delta t$ 共四個，因此至少需要四顆衛星建立四條方程式。由於方程式非線性，實務上以泰勒展開線性化後迭代求解，或以最小平方法處理多於四顆衛星時的冗餘量測——當可見衛星有 10 顆時，就有 10 條方程式解 4 個未知數，多出來的資訊正好用來抑制雜訊。

### 二、定位誤差與 DOP

定位誤差可近似分解為兩個因子的乘積：

$$\text{定位誤差} \approx \text{UERE} \times \text{DOP}$$

UERE（User Equivalent Range Error）是單一衛星的等效測距誤差（含電離層、對流層、多路徑等），DOP 則是幾何放大倍率。這條簡潔的關係式告訴我們兩條改善途徑：降低測距誤差（用雙頻、用差分修正），或改善幾何條件（用多星系、找開闊處）。同學可以用手機的 GNSS 檢視 App 實際觀察：走到大樓旁邊時，可見衛星數下降、DOP 上升，定位誤差圈立刻變大。

### 三、載波相位觀測式

載波相位觀測量 φ（以週為單位）與真實距離 r 的關係為：

$$\lambda \cdot \varphi = r + c \cdot \Delta t + \lambda \cdot N + \text{誤差項}$$

其中 λ 為載波波長（L1 約 19 公分），N 即整週波未定值——一個未知的整數。RTK 的全部精髓，就在於利用基準站與移動站的雙重差分消去大部分誤差項後，快速可靠地把 N 解算為整數。一旦 N 固定，λ · φ 就成為公釐級精度的距離量測，這就是公分級定位的來源。

## 四、GNSS 定位可信度判斷流程

**圖 5-5　機器人判斷是否信任 GNSS 的決策流程**

① 可見衛星數 ≥ 6 顆？　否 → 不信任，切換 SLAM 主導

↓ 是

② HDOP < 2.0？　否 → 降低權重

↓ 是

③ RTK 狀態為固定解？　浮動解 → 中等權重；單點解 → 低權重

↓ 固定解

④ 與 IMU / SLAM 推算位置的差異 < 門檻？　否 → 判定為多路徑跳點，拒絕本次量測

↓ 是

⑤ 採納為高權重絕對位置量測，校正累積漂移

# 5.8　衛星導航的當代變局：更強的訊號與更強的干擾

本章前七節描述的 GNSS 架構自 1990 年代定型以來相當穩定。但 2025 至 2026 年間，這個領域同時出現了兩股方向相反的力量——一股讓定位更準更強韌，另一股讓它更不可信。

兩者都直接影響機器人的系統設計，同學務必知道。

### 一、免費的高精度：Galileo 高精度服務

5.3 節介紹的 RTK 需要基準站與通訊鏈路，是它普及的最大障礙。歐盟的伽利略高精度服務（HAS）提出另一條路：直接在衛星訊號中免費廣播精密星曆與時鐘改正，接收機不必連網、不必訂閱，就能以精密單點定位（PPP）達到約二十公分的水平精度。

這對機器人教學是好消息——它把「公分等級定位」從需要基準站的專業設備，往消費級硬體推進了一大步。代價是 PPP 的收斂時間較長（通常數分鐘），不像 RTK 那樣瞬間固定，因此適合長時間運行的巡檢或農機，不適合走走停停的場域。

### 二、低軌道導航星座：訊號強度的世代差距

更根本的變化來自低軌道定位導航授時（LEO-PNT）。傳統 GNSS 衛星在兩萬公里的中軌道，訊號抵達地面只剩約 -130 dBm——5.1 節提過，這比環境雜訊還微弱。低軌衛星只在五百公里左右，距離縮短四十倍，接收功率因而大幅提升。

以一個代表性的商業系統為例，其規劃的最大接收功率約達 -136 dBW，業者宣稱可達傳統 GPS L1 C/A 訊號的百倍強度；單顆衛星的測距精度經在軌軟體更新後由 4.2 公分改善至約 1.5 公分，全星座部署後在開闊天空下的目標是水平約二公分、垂直約四公分。整個星座規劃二五八顆衛星，是 GPS 二十四顆基本星座的十倍以上；首批量產衛星預計於 2026 年十月發射，正式服務自 2027 年起。

對機器人而言，訊號強度的意義不只是「更準」，而是「在更多地方還收得到」——樹蔭下、建築物邊緣、室內靠窗處，這些 5.5 節列為失效區的場域，都可能重新變成可定位區。

### 三、另一股力量：干擾與偽造正在常態化

但同一時期，GNSS 遭受的人為干擾規模遠超過去。上述低軌實驗衛星在五百公里高空進行的觀測顯示，歐洲至中東一帶——西起法國、東至巴基斯坦邊境——存在大範圍的訊號干擾；在干擾最嚴重的區域，GPS 訊號強度由正常的約四十分貝衰減至僅約十分貝。研究團隊

的說法是：「我們本來就預期會看到一些干擾，但比我們想像的多得多。」

這件事把 5.5 節的「人為干擾」從教科書上的一小段，變成系統設計必須正面處理的常態條件。干擾（Jamming）讓接收機收不到訊號，機器人至少知道自己瞎了；偽造（Spoofing）更危險——它送出看似正常的假訊號，讓機器人「自信地走錯地方」。

值得注意的是，低軌星座的強訊號本身就是一種對抗手段：實測顯示其較強的訊號可使干擾源能夠癱瘓的面積縮減達九成五。但業者也坦承，近距離或高功率的干擾源在不利條件下仍可能奏效——沒有任何單一系統能免疫。

### 四、地面網路的補位：5G 定位

5.4 節介紹的 5G 定位也在持續演進。第三代合作夥伴計畫（3GPP）自 Release 17 起把高精度定位正式納入規範，Release 18 進一步強化載波相位定位與側行鏈路（Sidelink）定位——後者讓裝置之間可以直接測距，不必全部經過基地台。這對第 27 章的多機器人協作是值得注意的發展：一群機器人可以互相測距，形成區域性的相對定位網。

### 五、給機器人工程師的結論

把這四件事放在一起，會得到一個清楚的結論：GNSS 正在變得同時更強與更脆弱。更強，是因為訊號功率、改正服務與多系統多頻段都在進步；更脆弱，是因為它所處的電磁環境已經不再單純。

而工程上的應對方式，恰恰就是本章 5.6 節已經講過的：永遠不要讓任何單一感測器獨自承擔定位責任。無論 GNSS 變得多好，機器人都應該有能力偵測它何時不可信，並在那一刻優雅地切換到 IMU 推算與 SLAM。

> **一個實務判準：偵測偽造的三道防線**
>
> 一、合理性檢查：位置是否瞬間跳躍？速度是否超出機器人的物理極限？（第 7 章的 IMU 正好提供獨立的運動參考）

二、一致性檢查：GNSS 位置與 SLAM 估計的差異是否持續超過門檻？（這正是圖 5-5 決策流程的核心）

三、訊號層檢查：載噪比（$C/N_0$）是否異常地整齊劃一？真實衛星因仰角不同應有明顯差異，偽造訊號往往過於一致。

## 案例

### 案例 5-1　農機自駕：RTK 最成功的商業應用

在自駕車還在測試的年代，RTK 自動駕駛曳引機早已在全球農田中大規模商業運轉。農業場景之所以成為 RTK 的完美舞台，原因很值得同學思考：田地空曠無遮蔽，衛星訊號品質極佳；作業路徑是規則的直線來回，不需複雜決策；而精度需求卻極高——播種行距偏差幾公分，就會造成重疊或漏播，累積成可觀的產量與成本損失。

RTK 讓曳引機能以公分級精度沿著預定路線自動行駛，駕駛只需在田邊轉彎時介入（新型系統連轉彎也自動化）。這個案例展示了一個重要的工程判斷：當環境條件對某項技術極為有利時，就應該把它用到極致，而不必為了「技術完整性」硬加上不需要的感測器。反過來說，同一台曳引機開進果園樹冠下，RTK 立刻失效——這時才需要光達或視覺 SLAM 接手。技術選型永遠是場域決定的。

### 案例 5-2　都市峽谷中的自駕車定位

自駕車在市區行駛時面臨 GNSS 最惡劣的考驗：兩側高樓把天空切成一條窄縫，可見衛星從十餘顆掉到三、四顆且集中在同一方向，DOP 惡化數倍；更麻煩的是玻璃帷幕大樓的鏡面反射——接收機可能收到的是從對面大樓彈回來的訊號，算出的位置偏差可達數十公尺，而且系統往往不知道自己錯了。

主流自駕車的解法是徹底降低對 GNSS 的依賴：以事先測繪的高精地圖為基準，用車上的光達與相機即時比對地圖特徵（車道線、路緣、標誌桿、建物輪廓）來定位，GNSS 僅作

為初始化與粗略區域判斷之用。這正是第 17、18 章 3D 光達建圖技術的商業價值所在——當天上的訊號不可靠時，地面的環境結構才是最忠實的參考。

### 案例 5-3　無人機的 GPS 拒止飛行

消費級無人機在開闊處能靠 GNSS 穩定懸停，一旦飛進橋樑下方、室內廠房或森林，就進入所謂的 GNSS 拒止（GNSS-denied）環境。早期無人機在此情境下會失控漂移，甚至觸發返航卻撞上障礙物。

現代解法是視覺慣性里程計（VIO）：以下視或前視相機追蹤地面 / 環境的紋理特徵，配合 IMU 估測運動，完全不依賴外部訊號（第 26 章詳述）。這也解釋了為什麼無人機的視覺定位研究會集中在 ETH / UZH（第 2 章）——他們的核心命題正是「當衛星幫不上忙時，機器人如何只靠自己的眼睛飛行」。對本書讀者而言，這是一條清楚的學習線索：本章談的是「有訊號時怎麼用好」，第六、七篇談的則是「沒訊號時怎麼辦」。

## 實作活動

### 實作 5-1　校園 GNSS 訊號品質勘測

目的：以手機實測校園各處的衛星訊號品質，親身驗證遮蔽、多路徑與 DOP 對定位精度的影響，並產出一張「GNSS 可信度地圖」，作為第 31 章巡檢機器人專題的路線規劃依據。

#### 器材

- 智慧型手機，安裝可顯示衛星資訊的 App（可查詢「GNSS 檢視」「GPS Test」類工具，需能顯示衛星數、訊號強度與 DOP 或精度估計值）。
- 校園平面圖、記錄表、捲尺（選用）。

#### 步驟

18. 選點：在校園中選出至少 8 個具代表性的測點，涵蓋四種環境——開闊操場、樹蔭步道、建築物北側牆邊、穿堂或騎樓下、以及室內走廊各至少一點。

19. 量測：在每個測點靜置手機 60 秒後記錄：可見衛星總數、實際用於定位的衛星數、各衛星訊號強度分布、HDOP 值（或 App 顯示的精度估計）、以及顯示的經緯度座標。

20. 重複性測試：在同一測點間隔 5 分鐘再量一次，比較兩次的經緯度差異——這個差異就是定位的重複性誤差。

21. 繪製可信度地圖：把各測點依「衛星數與 DOP」分為三級（綠：可信賴，可用 GNSS 主導；黃：需降權，SLAM 輔助；紅：不可信任，須切換 SLAM），標示在校園平面圖上。

22. 多路徑觀察：在高樓玻璃帷幕旁與空曠處各站 2 分鐘，比較定位點的跳動幅度，並記錄各自的衛星訊號強度分布是否異常。

23. 綜合報告：依據可信度地圖，為一台校園巡檢機器人規劃一條巡檢路線，標明沿途各段應以何種定位方式為主導，以及應在哪些位置設置 AprilTag 校正點。

**討論問題**

23. 哪一個測點的定位重複性誤差最大？該處的衛星數與 DOP 值如何？符合本章 5.5 節的哪一類誤差來源？

24. 樹蔭下與建築物旁，哪一種遮蔽對定位的影響較大？你認為原因是什麼（提示：樹葉是部分穿透，牆壁是完全遮蔽但會反射）？

25. 若這台巡檢機器人只配置 GNSS 而沒有 SLAM，它在你們規劃的路線上會在哪些路段出問題？後果是什麼？

26. （大學部）依據圖 5-5 的決策流程，為你們的校園場域設定各項門檻值（衛星數、HDOP、位置差異），並說明訂定理由。

**延伸挑戰（大學部）**

以 Python 撰寫程式解析手機或 USB GNSS 接收器輸出的 NMEA 語句（GGA、GSA、GSV），自動記錄衛星數、HDOP 與定位品質指標，並繪製時間序列圖與軌跡圖。進一步嘗

試計算「靜態測點的位置散布標準差」，量化不同環境下的定位精度差異。這份程式將可直接沿用於第 31 章的巡檢專題。

## 延伸閱讀


27. P. Misra and P. Enge, Global Positioning System: Signals, Measurements, and Performance, 2nd ed., Ganga-Jamuna Press, 2011.（GNSS 原理的標準教科書）
28. B. Hofmann-Wellenhof, H. Lichtenegger, and E. Wasle, GNSS – Global Navigation Satellite Systems, Springer, 2008.（涵蓋四大系統與 RTK 的完整參考書）
29. P. D. Groves, Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems, 2nd ed., Artech House, 2013.（GNSS / INS 組合導航的權威著作，第 5.6 節主要參考）
30. 內政部國土測繪中心 e-GNSS 即時動態定位系統網站.（台灣網路 RTK 服務的官方資訊與使用說明）
31. RTKLIB 開源專案（rtklib.com）.（免費的 GNSS / RTK 處理軟體，可用於教學實驗與後處理分析）
32. European Union Agency for the Space Programme, Galileo High Accuracy Service (HAS) 官方說明文件.（新一代衛星直播修正服務）
33. 3GPP TS 38.305, NG-RAN Stage 2 Functional Specification of User Equipment Positioning.（5G 定位的標準規範文件）


## 從衛星到感測器：本章的位置回顧

第 4 章與本章合起來回答了一個問題：機器人要怎麼知道自己在世界的哪裡。答案是——如果環境允許（室內有佈建、室外有天空），就讓外部基礎建設告訴它。這是最省事、最不會漂移的做法。

但這兩章也同時劃出了邊界。室內沒佈建、室外沒天空的場域——隧道、地下停車場、倉庫深處、森林、災後建築、月球表面——機器人必須完全靠自己。接下來的第 6 至 9 章要逐一介紹它身上可用的「內部感官」：地磁羅盤、慣性量測單元、光達、相機。它們不需要任何外部設施，但也都有一個共同的致命弱點：會漂移。

而消除漂移的唯一方法，是讓機器人認出自己回到了走過的地方。這就是閉環偵測，也是 SLAM 之所以成立的理由。第三篇的數學與第四篇以後的演算法，全都在為這件事鋪路。本章留給同學的核心判斷是：當 GNSS 可用時，用它；當它不可用時，知道它不可用——後者才是真正困難、也真正重要的工程能力。

## 小結

14. GNSS 由太空、控制、使用者三大區段組成；接收機只收不發，以虛擬距離進行三維三邊定位，並因需同時解算時鐘偏差而至少需要四顆衛星。截至 2026 年，四大系統在軌衛星合計約 130 顆。
15. 無線電定位有三種幾何方法：三角定位用角度、三邊定位用距離、TDoA 用距離差；基準點的幾何分布以 DOP 量化，定位誤差約等於測距誤差乘上 DOP。
16. RTK 結合差分修正與載波相位量測達成公分級定位，關鍵在於解算整週波未定值；固定解為公分級、浮動解為公寸級，且對遮蔽與多路徑仍無能為力。
17. 地面無線電定位各有定位：5G 適合園區與工廠、LoRa 適合大範圍粗追蹤、Wi-Fi RTT 可提供室內公尺級測距，但都受限於基礎設施普及度。
18. GNSS 誤差分為可差分消除的傳播誤差（電離層、對流層、衛星時鐘軌道）、無法消除的局部誤差（多路徑、遮蔽、都市峽谷）與人為干擾（Jamming、Spoofing）。
19. GNSS 與 IMU 誤差特性互補，構成成熟的組合導航；再加入 SLAM 即成三重架構，可在戶外、過渡區與室內全程維持連續且不漂移的定位。

20. 校園巡檢機器人需依環境分區採用不同定位主力，並以視覺標記作為室內外座標系的橋樑；系統設計的核心觀念是「讓多種技術在各自擅長的場合接力」。

下一章，我們探討一個最古老卻仍在服役的導航手段——地磁。從航海羅盤到手機指南針，從鐵磁干擾校正到室內地磁指紋定位，看看地球本身的磁場如何為機器人提供絕對方向參考。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

## 第 6 章

## 地磁導航與電子羅盤

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

## 本章在全書的位置

第 4 章與第 5 章回答了「我在哪裡」，從本章開始，第二篇要轉向另一個同樣基本、卻更容易被忽略的問題：「我朝向哪邊」。

位置與朝向是兩回事。一台機器人就算知道自己站在走廊中央，若不知道自己面向哪一側，仍然寸步難行——它無法把「往前走三公尺」轉譯成正確的動作，也無法把感測器讀到的「前方兩公尺有牆」正確地標在地圖上。第 10 章會把這件事形式化為位姿（Pose）：位置加上朝向，六個自由度缺一不可。

本章的磁力計是機器人身上唯一能提供絕對航向的感測器——它不需要地圖、不需要佈建、也不會漂移。代價是它極度容易被干擾，而且干擾的來源往往就是機器人自己。這個「唯一絕對、但極易受污染」的性格，使它在感測器融合中扮演一個很特別的角色：平時貢獻不多，但在關鍵時刻無可取代。

**圖 6-0　第 6 章與全書的連結地圖**

【承接前章】

第 4 章　Wi-Fi 指紋定位　→　本章 6.3 節：換成磁場強度的同一套方法

第 5 章　GNSS 給位置但給不了靜止時的朝向　→　本章補上這一塊

【本章負責】

地磁三參數（F、D、I）　→　傾斜補償　→　磁航向　→　真航向

硬鐵 / 軟鐵校正：把變形的點雲矯正回理想的球

把「干擾」反過來當成「指紋」：室內地磁定位

【往下延伸】

→ 第 7 章　IMU：陀螺儀短期準、磁力計長期準，兩者互補成 AHRS

→ 第 10 章　航向角正式成為位姿的一部分

→ 第 14 章　地磁指紋比對 ≈ 粒子濾波的觀測模型

→ 第 26 章　無人機羅盤校正失敗的後果（案例 6-1）

→ 第 29 章　實體小車的 IMU / 磁力計安裝位置與干擾排查

## 學習目標

指南針是人類最古老的導航工具之一，卻仍活躍在每一支手機、每一台無人機之中。本章探討地球磁場如何為機器人提供絕對方向參考，以及它在室內為何會從「乾淨的方向感測器」變成「獨特的位置指紋」。學完本章之後，同學應能夠達成下列目標：

30. 說明地球磁場的成因與三個描述參數：磁偏角、磁傾角與磁場強度。
31. 說明磁力計如何量測磁場，並由三軸磁力計配合傾斜補償計算出航向角。
32. 解釋室內地磁指紋定位的原理，並說明「干擾」為何反而成為定位的資訊來源。
33. 區分硬鐵誤差與軟鐵誤差的成因，並執行八字形校正等實務校正程序。
34. 說明地磁感測在手機、無人機與機器狗中扮演的角色與已知限制。

35. 分析地磁、IMU 與視覺 SLAM 三者的誤差特性，說明其互補融合的邏輯。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
| --- | --- | --- |
| 地磁場 | Geomagnetic Field | 地球外核液態金屬對流產生的磁場，強度約 25 至 65 微特斯拉。 |
| 磁偏角 | Declination | 磁北與真北之間的水平夾角，隨地點與時間變化。 |
| 磁傾角 | Inclination / Dip | 磁力線與水平面的夾角，赤道近 0 度、兩極近 90 度。 |
| 磁力計 | Magnetometer | 量測磁場強度與方向的感測器，多採霍爾效應或磁阻原理。 |
| 電子羅盤 | Electronic Compass | 結合磁力計與加速度計，輸出航向角的模組。 |
| 航向角 | Heading / Yaw | 機器人前進方向與參考北方之間的夾角。 |
| 傾斜補償 | Tilt Compensation | 利用加速度計量得的姿態，把磁力計讀值投影回水平面。 |
| 硬鐵誤差 | Hard-iron Error | 載具上永磁材料造成的固定偏移，使量測球心偏離原點。 |
| 軟鐵誤差 | Soft-iron Error | 鐵磁材料扭曲外部磁場，使量測球面變形為橢球。 |
| 地磁指紋 | Magnetic Fingerprint | 建築物鋼構造成的局部磁場扭曲，可作為位置特徵。 |
| 世界地磁模型 | WMM | 由 NOAA 與 BGS 共同維護的全球磁場模型，每五年更新一次。 |
| 磁暴 | Geomagnetic Storm | 太陽活動引發的磁場劇烈擾動，會暫時降低羅盤精度。 |

## 6.1 地球磁場與磁力計

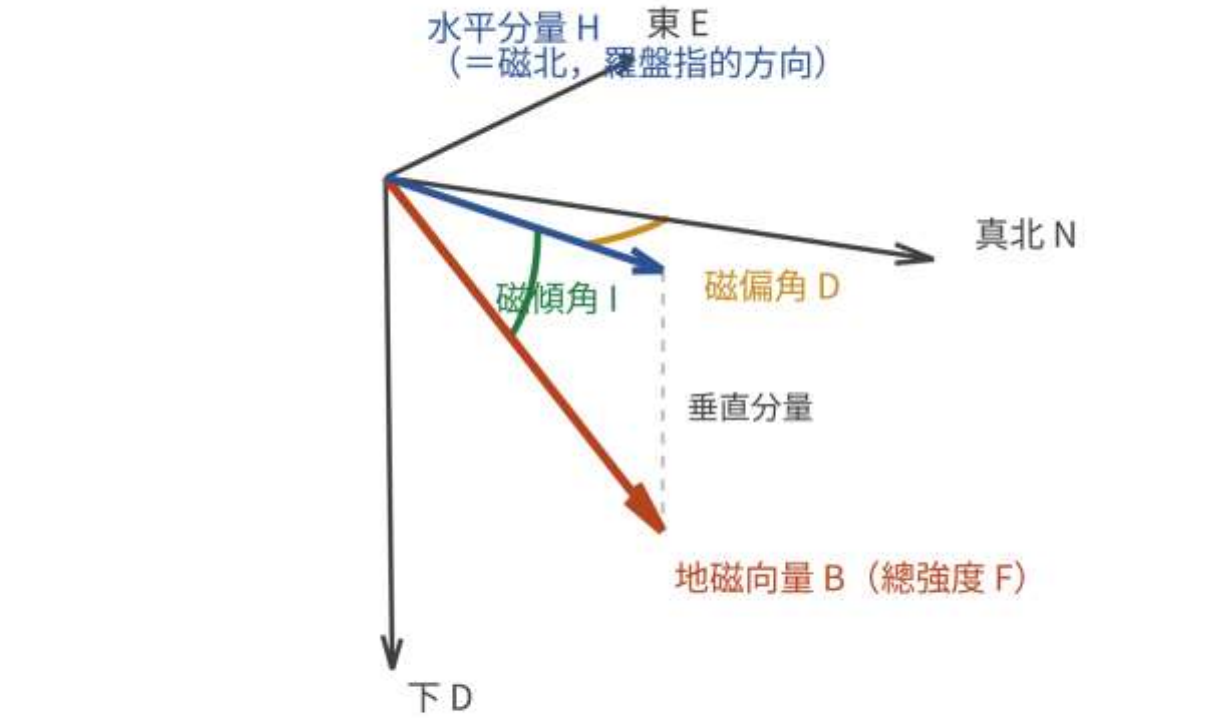
地球是一顆巨大的磁鐵。這個磁場並非來自某塊固態磁石，而是源自地球外核中高溫液態鐵鎳的對流運動——熾熱的導電流體在自轉與熱對流的共同作用下形成環流，如同一台巨型發電機持續產生磁場，這個機制稱為地球發電機（Geodynamo）。

### 地磁場的三個描述參數

在地表任一點，地磁場是一個三維向量，通常以三個參數描述：

- 磁場總強度（Total Intensity）：約在 25 至 65 微特斯拉（μT）之間，赤道附近較弱、高緯度較強。台灣約在 45 μT 上下。作為對照，一顆冰箱磁鐵的表面磁場強度可達數千微特斯拉——地磁其實非常微弱，這正是它容易被干擾的原因。
- 磁偏角（Declination）：磁北與真北的水平夾角。地圖與導航軟體使用的是真北，但羅盤指的是磁北，兩者的差值就是磁偏角。台灣地區的磁偏角約為西偏 4 度上下（隨地點與年份略有變化），也就是說羅盤指的「北」比真北偏西約 4 度。
- 磁傾角（Inclination）：磁力線與水平面的夾角。在磁赤道附近接近 0 度（磁力線平行地面），在磁極附近接近 90 度（磁力線垂直插入地面）。台灣約在 35 度左右——這個參數對機器人很重要，因為它意味著地磁向量有相當大的垂直分量，必須做傾斜補償才能正確算出航向。

**圖 6-1　地磁向量與三個描述參數**




台灣約為：F ≈ 45 μT　D ≈ 西偏 4°　I ≈ 35°　（I 不為零 → 必須做傾斜補償）

（圖中磁偏角已放大以利辨識）

這張圖把三個參數的幾何關係一次交代清楚。地磁向量 B 從原點指向斜下方（在北半球），它在水平面上的投影 H 就是羅盤所指的磁北；H 與真北之間的水平夾角是磁偏角 D，B 與水平面之間的夾角是磁傾角 I。

請特別注意台灣的磁傾角約為 35 度這件事——它意味著地磁向量有相當大的垂直分量。只要機器人稍微傾斜，這個垂直分量就會「漏」進水平軸的讀值，使航向算錯。這正是 6.2 節傾斜補償存在的理由，也是為什麼電子羅盤永遠需要搭配加速度計（第 7 章）才能運作。在磁赤道附近 I 接近零，傾斜補償的重要性會低得多；而在高緯度 I 接近 90 度時，水平分量 H 太小，羅盤幾乎失效——這就是 6.1 節提到的磁場失效區。

### 磁場會漂移：世界地磁模型

由於外核流體的運動並不穩定，地磁場持續緩慢變化，磁北極更是不斷移動——近數十年來它從加拿大北部往西伯利亞方向漂移，且自 1990 年代末以來速度明顯加快。因此，精確導航必須依賴持續更新的模型。

美國 NOAA 國家環境資訊中心（NCEI）與英國地質調查所（BGS）共同維護的世界地磁模型（WMM，World Magnetic Model）就是這個標準。它每五年更新一次，現行版本 WMM2025 於 2024 年 12 月發布，效期至 2029 年底；同時另有高解析度版本 WMMHR2025。這個模型被北約、美國國防部、航空與航海系統，以及每一支手機的指南針 App 所使用——同學手機上顯示的「真北」，背後就是這個模型在做磁偏角修正。

模型也標示出兩極附近的「磁場失效區」（Blackout Zone）：該處水平磁場強度過低，羅盤導航不再可靠。此外，太陽活動引發的磁暴會使磁偏角暫時偏離模型預測值——這對高緯度的航空作業影響尤其顯著。

### 磁力計的量測原理

- 霍爾效應（Hall Effect）：導體中通以電流，若受到垂直方向的磁場，載子會偏向一側形成橫向電壓，電壓大小正比於磁場強度。原理直觀、成本低，但靈敏度相對較低。
- 磁阻效應（Magnetoresistance）：某些材料的電阻會隨磁場改變。異向性磁阻（AMR）與穿隧磁阻（TMR）技術靈敏度高、功耗低，是目前手機與機器人常用的方案。

現代 MEMS 磁力計把三軸感測元件整合在一顆數公釐見方的晶片中，常與三軸加速度計、三軸陀螺儀封裝在一起，構成所謂的九軸 IMU 模組。要注意的是，磁力計輸出的原始資料是「感測器座標系中的三軸磁場分量」，距離「航向角」還有相當的處理距離——這正是下一節的主題。

## 6.2 電子羅盤與航向角

傳統羅盤的指針是一根可自由轉動的磁針，靠重力保持水平、靠磁力指向磁北。電子羅盤沒有活動部件，必須用數學完成同樣的工作——而這裡藏著一個許多初學者會踩到的陷阱。

### 最簡單的情況：完全水平

若磁力計恰好保持水平，航向角只需用水平面上的兩個分量計算：

$$\psi = atan2\left( -m_Y, m_x \right)$$

其中 $m_x$、$m_Y$ 是磁力計在水平面上的兩軸讀值，ψ 即磁航向角。使用 atan2 而非 atan 是為了正確判斷四個象限。

### 現實問題：機器人會傾斜

問題在於，機器人幾乎不可能永遠保持水平——小車經過斜坡、無人機前傾加速、機器狗上下樓梯，感測器都會跟著傾斜。由於台灣地區的磁傾角約 35 度，地磁向量有相當大的垂直分量；一旦感測器傾斜，這個垂直分量就會「洩漏」到水平兩軸上，造成航向角嚴重偏差。實測上，傾斜 10 度就可能造成數度到十餘度的航向誤差。

### 解法：傾斜補償

解法是先用加速度計量出重力方向，推算出滾轉角 φ 與俯仰角 θ，再把磁力計讀值旋轉回水平面：

$$m_x' = m_x \cdot \cos\theta + m_z \cdot \sin\theta$$

$$m_Y' = m_x \cdot \sin\varphi \cdot \sin\theta + m_Y \cdot \cos\varphi - m_z \cdot \sin\varphi \cdot \cos\theta$$

$$\psi = atan2\left( -m_Y', m_x' \right)$$

這就是為什麼電子羅盤模組一定要包含加速度計——單靠磁力計無法在傾斜狀態下正確工作。這也解釋了一個生活現象：手機指南針 App 通常會提示你「請將手機保持水平」，因為傾斜補償雖能修正，但加速度計在載具運動時會同時量到運動加速度與重力，導致姿態估計失準，進而使航向角變得不穩。

### 從磁航向到真航向

最後一步是加上磁偏角修正：

$$真航向 = 磁航向 + 磁偏角$$

在台灣，磁偏角約為西偏 4 度（依慣例記為負值），因此真航向約等於磁航向減去 4 度。精確的磁偏角值可由 WMM 模型依經緯度與日期查詢——這也是 GNSS 與地磁感測合作的一個小例子：必須先知道自己大概在哪裡（GNSS），才能查到正確的磁偏角修正量。

## 6.3 室內地磁指紋定位

前兩節把「干擾」視為敵人——建築物的鋼筋會扭曲地磁，使羅盤失準。但如果換個角度想：既然每個位置的扭曲模式都不一樣，而且相當穩定，那麼這些扭曲本身不就是絕佳的位置特徵嗎？這個反轉思維，正是室內地磁定位的核心洞見。

### 原理：把 bug 變成 feature

現代建築的鋼筋、鋼樑、電梯井、大型機具都由鐵磁材料構成，它們會使周圍地磁場產生局部扭曲。這些扭曲有三個對定位極為有利的性質：

- 空間獨特性：不同位置的磁場向量（大小與方向）各不相同，形成類似指紋的特徵。
- 時間穩定性：建築結構不會移動，因此磁場分布數月甚至數年不變——遠比 Wi-Fi 訊號穩定（AP 會被更換位置、人潮會造成起伏）。
- 零基礎設施：不需要安裝任何信標或基站，只要建立資料庫即可。這是它相對 UWB、BLE 的最大優勢。

### 流程：與 Wi-Fi 指紋法一脈相承

流程與第 4 章的 Wi-Fi 指紋定位幾乎相同：離線階段沿走廊或格點蒐集各位置的磁場特徵建庫；線上階段以即時量測比對資料庫求出最相似位置。

### 一個關鍵差異：資訊維度太低

但地磁指紋有一個先天弱點：Wi-Fi 指紋在一個點上可能量到十幾台 AP，形成十幾維的特徵向量，區別力很強；地磁指紋卻只有三個分量（甚至常只用磁場總強度這一維），區別力相對薄弱——走廊上相隔十公尺的兩個點，磁場強度完全相同並不罕見。

因此實務上的地磁定位很少採用「單點比對」，而是採用軌跡比對：讓使用者或機器人邊走邊記錄磁場序列，把這一整段「磁場曲線」與資料庫比對。一段曲線的獨特性遠高於單一數值，就像一段旋律比一個音符更容易辨識。常用方法包括粒子濾波（第 11 章）與動態時間校正（DTW）。

**實務精度與限制**

- 精度：搭配軌跡比對與行人航位推算（PDR），室內精度可達約 1 至 3 公尺，與 Wi-Fi 指紋法相當甚至更好。
- 建庫成本：與 Wi-Fi 指紋法相同，需要人工走訪蒐集，這是最大的部署障礙。
- 環境變動：大型金屬設備移位、建築改建會使局部資料庫失效；電梯運行、大型馬達啟停也會造成瞬時干擾。
- 磁暴影響：太陽活動雖會影響地磁，但室內定位比對的是「局部相對模式」而非絕對值，因此影響相對輕微。

## 6.4 鐵磁干擾與硬鐵 / 軟鐵校正

磁力計是最需要校正的感測器之一。原因很簡單：地磁只有約 45 μT，而機器人自己身上的馬達、電池、鋼製螺絲、金屬底盤所產生的磁場，往往遠大於此。若不校正，羅盤讀出來的幾乎是「機器人自己的磁場」而不是地球的。

**理想情況：一顆完美的球**

先建立一個關鍵的幾何直覺：把磁力計朝各個方向緩慢轉動一圈，蒐集所有三軸讀值並畫在三維空間中。由於地磁場的大小固定、只有方向改變，這些點理論上應該落在一個以原點為球心、半徑等於地磁強度的球面上。校正的全部工作，就是把實際量到的變形點雲「矯

正回」這顆理想的球。

### 硬鐵誤差：球心偏移

硬鐵誤差來自載具上會產生固定磁場的物體——永久磁鐵、被磁化的鋼件、馬達。它們在感測器座標系中疊加一個固定的磁場向量，效果是把整顆球平移離開原點。

校正方法相對簡單：找出量測點雲在各軸的最大值與最小值，取中點即為偏移量：

$$b = (m_max + m_min) / 2$$

$$m_corrected = m_raw - b$$

### 軟鐵誤差：球變橢球

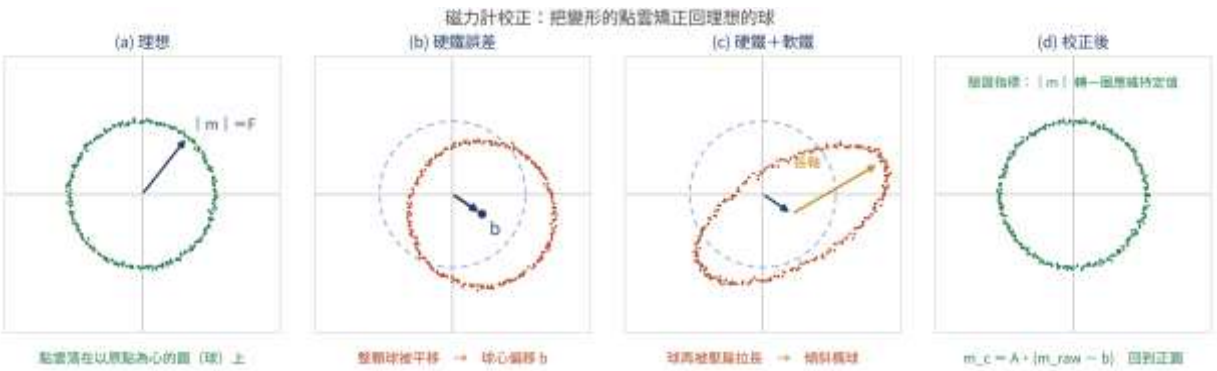
軟鐵誤差來自不會自己產生磁場、但會「扭曲」外部磁場的鐵磁材料——鐵板、鋼樑、機殼。它們讓磁力線在某些方向被聚集、某些方向被排開，效果是把球壓扁或拉長成橢球，甚至傾斜旋轉。

校正需要一個 3×3 的變換矩陣把橢球拉回正球：

$$m_corrected = A \cdot (m_raw - b)$$

矩陣 A 通常以橢球擬合（Ellipsoid Fitting）演算法從量測點雲估算。這也是為什麼完整的磁力計校正需要蒐集「各個方向」的資料——只在水平面轉圈，無法約束垂直方向的形變。

**圖 6-2　硬鐵與軟鐵誤差的幾何意義**




這四格圖是本節的全部精神。(a) 理想狀態下，把磁力計朝各方向轉一圈，所有讀值應落在

以原點為心、半徑等於地磁總強度的球面上。(b) 硬鐵誤差把整顆球「平移」——球的形狀沒變，只是球心跑到 b 的位置。(c) 軟鐵誤差進一步把球「壓扁拉長並旋轉」成傾斜的橢球。(d) 校正就是先減去偏移、再乘上一個把橢球拉回正球的矩陣。

理解了這張圖，三件實務上的事就變得理所當然：為什麼只在水平面轉圈不夠（那只約束了橢球的一個切面，垂直方向的形變完全沒被觀測到，所以要畫八字並翻轉）；為什麼必須在最終安裝狀態下校正（b 與 A 是「整台機器人」的性質，拆下來單獨校正量到的是另一組參數）；以及為什麼「轉一圈看總強度是否維持定值」是最好的驗證指標（校正成功等價於點雲回到正球，而正球的定義就是半徑處處相等）。

> **一個常見的誤解**
>
> 許多同學以為校正是「對準北方」。並不是——校正完全不需要知道哪邊是北。
>
> 校正只做一件事：把量測點雲的形狀矯正回球。至於這顆球上的哪個方向對應真北，是 6.2 節傾斜補償與 6.1 節磁偏角修正的工作。
>
> 這個「先修正感測器本身的系統誤差，再談它與世界的關係」的分工，在第 9 章的相機校正（內參 vs 外參）中會以完全相同的結構再出現一次。

### 實務校正程序

26. 八字形校正：手持裝置在空中畫「8」字並同時翻轉，讓感測器涵蓋盡可能多的三維姿態。手機 App 提示「請揮動手機」時，做的就是這件事。
27. 在最終安裝狀態下校正：必須把磁力計裝到機器人上、接好所有線路、電池就位之後再校正——因為硬鐵誤差是「整台機器人」造成的，拆下來單獨校正毫無意義。
28. 遠離環境干擾源：校正時應遠離鋼筋牆面、電腦、大型金屬桌，否則會把環境的軟鐵效應誤當成載具本身的誤差記錄下來。
29. 驗證：校正後再轉一圈，檢查磁場總強度是否維持穩定（應接近當地地磁強度且變化小），這是判斷校正是否成功最簡單的指標。

### 動態干擾：校正也救不了的部分

還有一類干擾無法靠事前校正解決：馬達電流變化產生的磁場。機器人加速時電流增大，磁場隨之改變，這是與運動狀態相關的動態誤差。緩解方法包括把磁力計安裝在遠離馬達與電源線的位置（無人機常把羅盤裝在延伸支架或 GPS 天線頂端就是這個原因）、以電流值建立補償模型，或在濾波器中降低磁力計在高電流時段的權重。

## 6.5 地磁導航在手機、無人機、機器狗中的角色

### 手機

手機中的磁力計肩負三項任務：指南針 App 的方向指示、地圖 App 的「視角朝向」（讓地圖隨你轉身而旋轉）、以及 AR 應用的初始方位設定。此外，磁力計也在感測器融合中提供「不漂移的航向參考」——單靠陀螺儀，手機的方向估計會在數分鐘內明顯偏離。

### 無人機

無人機是最依賴、也最容易被羅盤問題困擾的平台。飛控系統需要航向資訊才能正確解讀 GNSS 的位置變化並執行方向控制；羅盤一旦錯誤，會出現典型的「圓周飛行」（Toilet Bowl Effect）——無人機以為自己在往前飛，實際上繞著圈子越飛越偏。

- 起飛前校正是標準程序，且必須在實際起飛地點附近進行。
- 切勿在鋼筋結構物、鐵製圍籬、地下停車場、大型金屬屋頂附近校正或起飛。
- 專業機種常配置雙羅盤互相驗證，並在偵測到讀值異常時自動改用 GNSS 航向（由運動方向推算）。

### 機器狗與地面機器人

四足機器人與輪型機器人的姿態變化劇烈、機身金屬件多、馬達功率大，磁力計的工作環境相當惡劣。因此在室內場域，多數系統會刻意降低磁力計的權重，甚至完全不用——航向改由光達或視覺 SLAM 的掃描匹配結果提供，因為在有結構的室內環境中，「牆的方向」比「地磁」可靠得多。

反過來說，在戶外開闊、少金屬的環境（農田、草原、大型戶外場域），地磁仍是最經濟可靠的航向來源。這再次印證本書反覆強調的觀念：沒有絕對優劣的感測器，只有適合與不適合的場合。

## 6.6　地磁、IMU、視覺 SLAM 的互補關係

把地磁放進整個定位系統來看，它的角色可以用一句話概括：唯一能提供「絕對航向」的低成本被動感測器。

| 感測器 | 提供資訊 | 誤差特性 | 主要弱點 |
|---|---|---|---|
| 陀螺儀 | 角速度（積分得角度變化） | 短期極準，長期漂移 | 零偏漂移累積 |
| 加速度計 | 重力方向（滾轉、俯仰） | 長期無漂移 | 運動加速度污染 |
| 磁力計 | 磁北方向（航向） | 長期無漂移 | 鐵磁干擾、需校正 |
| 視覺 / 光達 SLAM | 相對位姿與環境結構 | 短期精確，長期漂移 | 依賴環境特徵 |
| GNSS | 絕對位置（可推算航向） | 誤差有界 | 室內失效、更新率低 |

**表 6-1　各感測器在姿態與航向估測中的角色**

### 三軸互補：AHRS 的邏輯

姿態與航向參考系統（AHRS，Attitude and Heading Reference System）正是把前三者組合起來的經典設計：

- 陀螺儀提供高頻率、平滑的角度變化，是姿態估計的骨幹。
- 加速度計以重力方向校正滾轉與俯仰的漂移——但它對航向（繞垂直軸旋轉）完全沒有約束力，因為繞著重力軸轉動，重力方向不會改變。
- 磁力計正好補上這個缺口：以磁北方向校正航向漂移。

這三者的分工可以這樣記憶：陀螺儀管「短期一切」，加速度計管「長期的上下」，磁力計管「長期的東西南北」。互補濾波器與擴展卡爾曼濾波（第 7、11 章）就是實現這個分工的數學工具。

### 加入 SLAM 之後

視覺或光達 SLAM 也能提供航向，而且在室內結構化環境中往往比磁力計可靠。因此現代室內機器人常見的策略是：

30. 系統啟動時，以磁力計提供粗略的初始航向，讓 SLAM 有一個合理的起點（避免地圖建在完全隨機的方向上）。
31. 運行期間，以 SLAM 的掃描匹配結果為主要航向來源，磁力計權重調低或僅作為異常監測。
32. 進入戶外或無結構空曠區域（SLAM 失效風險高）時，重新提高磁力計權重。
33. 若磁力計讀值與 SLAM 估計持續大幅不一致，觸發警示——這往往意味著機器人附近有大型金屬物體，或 SLAM 已經跟丟。

這個「依情境調整信任度」的設計思路，與第 5 章圖 5-5 的 GNSS 可信度判斷完全一致。同學至此應該能感受到，第二篇各章看似在講不同的感測器，實則反覆在談同一件事：認識每個感測器的誤差特性，然後讓它們在對的時機貢獻對的資訊。

## 公式或流程

### 一、地磁向量的分量表示

地磁場向量 B 在當地水平座標系（北 N、東 E、下 D）中可分解為：

$$B = (B_N, B_E, B_D)$$

由此可導出三個常用參數：

$$\text{水平分量}\quad H = \sqrt{(B_N^2 + B_E^2)}$$

$$磁偏角\quad D = \operatorname{atan2}(B_E, B_N)$$

$$磁傾角\quad I = \operatorname{atan2}(B_D, H)$$

$$總強度\quad F = \sqrt{(B_N^2 + B_E^2 + B_D^2)}$$

以台灣為例，總強度 F 約 45 μT、磁傾角 I 約 35 度，代入可得水平分量 H ≈ F · cos I ≈ 37 μT、垂直分量約 26 μT。垂直分量佔比可觀，這正是 6.2 節強調傾斜補償不可省略的量化理由。

## 二、磁力計校正模型

完整的校正模型同時處理硬鐵與軟鐵誤差：

$$m_{corrected} = A \cdot (m_{raw} - b)$$

其中 b 為 3×1 的硬鐵偏移向量（球心平移），A 為 3×3 的軟鐵校正矩陣（橢球還原為正球）。校正品質的驗證指標是：校正後在任意姿態下，磁場總強度應保持穩定：

$$\|m_{corrected}\| \approx F\ (當地地磁總強度，恆定)$$

實務上可計算一段旋轉資料中 ||m|| 的標準差，數值越小表示校正越成功。這是一個同學在實驗中可以立即計算、立即判斷的實用指標。

## 三、電子羅盤完整處理流程

**圖 6-3　電子羅盤從原始讀值到真航向的處理流程**

① 磁力計原始三軸讀值 m_raw

↓

② 硬鐵 / 軟鐵校正：$m = A \cdot (m_{raw} - b)$

↓

③ 由加速度計求姿態：滾轉角 φ、俯仰角 θ

↓

④ 傾斜補償：把 m 投影回水平面，得 $m_x'$、$m_Y'$

↓

⑤ 計算磁航向：$\psi = \text{atan2}(-m_Y', m_x')$

↓

⑥ 加磁偏角修正（由 WMM 依經緯度查得）→ 真航向

↓

⑦ 與陀螺儀融合（互補濾波 / EKF）→ 平滑且不漂移的航向輸出

## 6.7　地磁導航的當代發展：從人工校正到自動校正

地磁是一門老技術——指南針有近千年歷史。但它在機器人領域的應用，近年出現了兩個值得注意的轉折：模型的解析度大幅提升，以及校正從「人工儀式」走向「自動完成」。

### 一、更細的地磁模型：WMMHR

6.1 節提到的世界地磁模型（WMM）於 2024 年 12 月發布 WMM2025 版本。同時發布的還有高解析度版本 WMMHR2025，其空間解析度在赤道附近約為三百公里，相較於標準版約三千三百公里提升了約十倍。

這個差異對一般導航影響不大，但對需要高精度航向的作業——例如測量、航空、以及使用地磁輔助的長距離自主航行——意義明確：模型愈細，磁偏角的局部修正就愈準，真航向的誤差也就愈小。NOAA 建議使用者改用高解析度版本。

值得一提的是，這個模型本身就是「地圖」的一種——只是它畫的不是牆壁的位置，而是磁場的分布。同學可以把它視為一張全球尺度的、由國家機構維護的先驗地圖；而 6.3 節的室內地磁指紋地圖，則是同一個概念縮小到建築物尺度、由使用者自己建立的版本。第 13 章之後要建的佔據格地圖，又是另一種尺度與內容的地圖。「什麼是地圖」這個問題，比直覺上想的要寬廣得多。

## 二、不必再揮手機：在 SLAM 過程中自動校正

6.4 節描述的八字形校正，是磁力計使用上最大的實務障礙——它要求使用者做一個不自然的動作，而且在許多場合根本做不到。裝在列車、電梯或高速載具上的感測器無法被拿起來揮動；清潔機器人也不會自己翻滾。

近年研究提出的解法是：把校正本身納入 SLAM。一項 2024 年的代表性工作提出所謂的「同步定位、建圖與校正」架構，以因式分解的粒子濾波在估計位姿與磁場地圖的同時，一併估出磁力計的零偏——具體做法是對每個粒子各自維護一個卡爾曼濾波器來追蹤零偏，並利用空間鄰域分析判定閉環。實驗在購物中心與辦公室、公寓的移動機器人上進行，零偏估計結果可與作業系統內建的校正方法相當，而完全不需要使用者做任何校正動作。

對同學而言，這個方法的結構值得細看：它的骨架正是第 14 章 Gmapping 所使用的 Rao-Blackwell 化粒子濾波——位姿用粒子表示，其餘的量（在 Gmapping 中是地圖，在這裡是地圖加零偏）對每個粒子各自以解析方式維護。換句話說，這是同學在第四篇會親手實作的演算法，被換了一個應用對象。這也再次印證本書反覆強調的觀點：SLAM 的數學骨架是通用的，換掉感測器與觀測模型就能解決全新的問題。

## 三、深度學習進入地磁指紋定位

6.3 節指出地磁指紋法的根本困難是資訊維度太低——每個位置只有三個數字（或實務上常只用總強度一個數字），不同位置極易撞號。近年的研究方向是改用序列模型：不看單點讀值，而看機器人移動過程中整段磁場變化的「曲線形狀」，再以神經網路做比對。

這類方法已從早期的卷積網路，發展到結合注意力機制與長短期記憶網路的架構，在複雜室內環境中展現出比傳統 KNN 比對更好的強健性。其核心洞見與本書第 20 章的視覺詞袋、第 22 章的 ORB-SLAM2 閉環偵測一致：單一觀測不可靠，但一段觀測的序列具有高度的辨識性。

**把三件事串起來**

模型變細（WMMHR）→ 絕對航向更準；自動校正（SLAM 內建）→ 不再需要人工儀式；序列學習 → 低維度觀測也能定位。

三者的共同方向是：把原本需要人介入的步驟，交給演算法在運行中自己解決。

這正是本書第八篇實作章節的核心精神——一個好的機器人系統，應該讓使用者只負責「說要去哪裡」，其餘的都是系統的事。

## 案例

### 案例 6-1　無人機的「馬桶效應」：羅盤錯誤的教科書案例

無人機飛手圈中流傳一個生動的名詞：Toilet Bowl Effect（馬桶效應）——無人機在定點懸停模式下，不但停不住，反而像水沖進馬桶一樣繞著圈子越飛越大。這幾乎必然是羅盤問題。

原因可以這樣理解：飛控以為機頭朝北，實際上朝東北。當 GNSS 顯示飛機往東漂移，飛控便下令「往西修正」——但它認知中的西，實際上是西北。修正動作因此永遠帶著一個固定的角度偏差，位置誤差不但沒被消除，反而被推著沿切線方向旋轉，形成圓周軌跡。

常見肇因包括：在鋼筋陽台或金屬車頂上校正、起飛點下方有鋼筋或地下管線、機上加裝了含磁性的配件（某些相機雲台或喇叭含永磁）。這個案例的教學價值在於：它清楚展示了「航向誤差」與「位置誤差」如何耦合——航向錯了，所有基於位置的控制決策都會系統性地偏向同一側。這個觀念在第 10 章的座標轉換與第 29 章的導航實作中會再次出現。

## 案例 6-2　從干擾到資訊：室內地磁定位的商業化嘗試

2010 年代中期，芬蘭的 IndoorAtlas 等公司提出以地磁指紋做室內定位的商業方案，一度被視為「不需要任何硬體」的理想解——不必安裝信標、不必依賴 Wi-Fi，只要手機內建的磁力計即可。這個構想在購物中心、機場、展館的導覽應用中確實展示了公尺級的定位能力。

但商業化過程也暴露了現實限制：指紋庫必須人工走訪建立，一座大型商場需要數十小時的勘測工作；商場改裝櫃位、更換大型金屬展示架後，局部資料庫就得重測；且單靠地磁的區別力不足，實務上仍須融合 Wi-Fi、藍牙與手機 PDR 才能穩定運作。

這個案例給同學兩個啟示：第一，「零硬體成本」不等於「零部署成本」，資料蒐集與維護往往才是真正的門檻——這與第 4 章 Wi-Fi 指紋定位的結論完全一致；第二，在真實產品中，很少有單一技術能獨力撐起需求，融合幾乎是必然的答案。

## 案例 6-3　教室裡的磁場地圖：把鋼筋畫出來

一個在大學與高中課堂都極受歡迎的示範實驗：讓同學拿著手機的磁力計 App，緩慢走過教室的地板與牆邊，觀察磁場強度讀值的變化。

結果往往令同學印象深刻：在教室中央的空曠處，讀值穩定在約 45 μT 附近；一走近柱子或承重牆，數值可能瞬間跳到 60 μT 以上；把手機貼近電腦桌的金屬腳架或黑板的鐵框，讀值甚至可能翻倍或急遽下降。若把測得的數值依位置畫成色階圖，就會浮現出一張「隱形的鋼筋分布圖」——建築的骨架被磁場顯影了出來。

這個實驗一次達成三個教學目的：讓同學直觀感受地磁的微弱與易受干擾（6.1 節）、理解為何室內羅盤不可信（6.4 節），同時也親眼見證這些「干擾」的空間穩定性與獨特性，正是地磁指紋定位的基礎（6.3 節）。同一組資料，同時說明了「為什麼不能用」與「為什麼可以用」——這正是本章最核心的觀念轉折。

## 實作活動

### 實作 6-1　磁力計校正與教室地磁指紋地圖

目的：完成一次完整的磁力計校正，並實測教室的地磁分布，同時體會地磁作為「航向感測器」與「位置指紋」的雙重身分。本實作以手機即可進行，大學部可延伸至 Arduino / ESP32 硬體。

#### 器材

- 智慧型手機，安裝可顯示三軸磁力計原始數值的感測器 App（可搜尋「magnetometer」「sensor logger」類工具）。
- 教室平面圖、記錄表、色鉛筆或試算表軟體。
- （大學部選用）ESP32 或 Arduino 搭配 HMC5883L / QMC5883L / MPU-9250 等九軸模組。

#### 第一部分：校正與航向驗證

24. 未校正基準測試：在教室中央手持手機，緩慢原地旋轉一整圈，記錄磁場總強度 ||m|| 的最大值與最小值。理論上旋轉時總強度應保持不變，實測的變化幅度即反映未校正誤差的大小。
25. 八字形校正：依 App 或系統指示，手持手機在空中畫「8」字並翻轉各個角度約 20 秒，讓感測器涵蓋盡可能多的姿態。
26. 校正後驗證：重複步驟 1，比較校正前後總強度的變化幅度（可計算標準差），驗證校正成效。

27. 傾斜實驗：把手機水平朝向教室黑板，記錄航向角；再把手機前傾約 30 度、維持同一朝向，觀察航向讀值變化了多少。討論這與 6.2 節傾斜補償的關係。

28. 干擾實驗：把手機分別靠近筆電、金屬椅腳、消防栓、電源插座，記錄磁場總強度與航向角的變化幅度。

### 第二部分：地磁指紋地圖

29. 在教室地面以 1.5 至 2 公尺間隔標出格點（約 15 至 20 個點），並在平面圖上編號。

30. 在每個格點以固定姿態（手機水平、面向同一方向）停留 5 秒，記錄磁場總強度與三軸分量，每點量測三次取平均。

31. 繪製色階圖：以磁場總強度為色階值（低為藍、高為紅）填入平面圖，觀察是否浮現柱子、承重牆或大型金屬設備的輪廓。

32. 盲測驗證：由一位同學閉眼被帶至某格點，量測磁場後由其他同學比對資料庫推測位置，重複 6 次並統計正確率與平均誤差。

33. 軌跡比對進階：讓一位同學沿教室邊緣走一圈並連續記錄磁場序列，觀察這條「磁場曲線」是否比單點數值更容易辨識出走的是哪一段路線。

### 討論問題

27. 校正前後，旋轉一圈時磁場總強度的變化幅度各是多少？這個差異說明了什麼？

28. 傾斜 30 度造成的航向誤差有多大？如果一台機器人爬上 15 度的斜坡而沒有做傾斜補償，導航會出現什麼問題？

29. 你們的盲測正確率如何？失敗的案例多半發生在哪些格點？這印證了 6.3 節提到的哪一項限制？

30. 比較單點比對與軌跡比對的辨識效果，說明為何實務系統偏好後者。

31. （大學部）根據你們的教室磁場地圖，判斷這間教室適合用磁力計提供機器人航向嗎？若不適合，你會改用什麼方法（提示：參考 6.6 節）？

**延伸挑戰（大學部）**

以 ESP32 搭配九軸模組，撰寫程式完成下列功能：讀取三軸磁力計與加速度計原始值、實作硬鐵校正（最大最小值法）與橢球擬合軟鐵校正、實作傾斜補償並輸出真航向角（含台灣磁偏角修正）、最後以互補濾波器融合陀螺儀資料，比較融合前後航向輸出的平滑度與長期穩定性。這份程式可直接作為第 7 章 IMU 姿態估測的基礎。

## 延伸閱讀


34. NOAA NCEI, World Magnetic Model 2025 (WMM2025) 官方網站與技術報告.（含線上磁偏角計算工具，可查詢台灣各地數值）
35. British Geological Survey, Geomagnetism 教學資源（geomag.bgs.ac.uk）.（地磁基礎知識與模型說明）
36. M. Kok, J. D. Hol, and T. B. Schön, "Using Inertial Sensors for Position and Orientation Estimation," Foundations and Trends in Signal Processing, 2017.（慣性與磁感測融合的完整教程，可免費取得）
37. B. Li et al., "How Feasible Is the Use of Magnetic Field Alone for Indoor Positioning?," IPIN, 2012.（地磁室內定位可行性的代表性研究）
38. J. Haverinen and A. Kemppainen, "Global Indoor Self-localization Based on the Ambient Magnetic Field," Robotics and Autonomous Systems, 2009.（地磁指紋定位的早期經典論文）
39. J. F. Vasconcelos et al., "Geometric Approach to Strapdown Magnetometer Calibration in Sensor Frame," IEEE Transactions on Aerospace and Electronic Systems, 2011.（磁力計校正的嚴謹數學處理）
40. PX4 / ArduPilot 開源飛控之 Compass Calibration 官方文件.（無人機羅盤校正的實務指引與常見問題排除）

## 從方向到慣性：本章的位置回顧

本章給了機器人一個絕對的方向感，但也暴露了它的脆弱：一面鋼筋牆、一具運轉中的馬達，甚至機器人自己加速時的電流變化，都足以讓它指錯方向。而且磁力計的更新率不高，無法反映快速的旋轉。

下一章的 IMU 正好補上這兩個缺口。陀螺儀的更新率可達每秒數百次，對短時間內的姿態變化極為靈敏，而且完全不受磁場干擾；但它的輸出必須積分，誤差會隨時間累積漂移。於是出現了一個漂亮的分工：陀螺儀管短期、磁力計管長期，兩者再加上加速度計提供的重力方向，構成姿態與航向參考系統（AHRS）——這正是本章 6.6 節表 6-1 所整理的格局，也是第 7 章姿態濾波要正式處理的問題。

同學讀到這裡應該已經感覺到第二篇的節奏了：每一章介紹一個感測器，說明它能給什麼、又在什麼情況下說謊，然後指出下一章的感測器如何補上這個缺口。這條鏈走到第 9 章結束時，機器人身上所有的感官就都齊備了——接著第三篇要教的，是把這些彼此矛盾的讀值融合成一個最合理答案的數學。

## 小結

21. 地磁場源自地球外核液態金屬對流，強度僅約 25 至 65 μT，遠弱於日常金屬物體產生的磁場，因此極易受干擾；以磁偏角、磁傾角與總強度三個參數描述。
22. 磁北極持續朝西伯利亞方向漂移，精確導航須依賴世界地磁模型（WMM）；現行版本 WMM2025 於 2024 年 12 月發布，效期至 2029 年底，並另有高解析度版本 WMMHR2025。
23. 電子羅盤必須結合加速度計進行傾斜補償——台灣磁傾角約 35 度，地磁垂直分量可觀，未補償的傾斜會造成顯著航向誤差；最後再加磁偏角修正才得真航向。
24. 室內鋼構造成的磁場扭曲，在導航上是干擾，在定位上卻是穩定且獨特的位置指紋；因單點資訊維度低，實務多採軌跡比對配合粒子濾波，精度約 1 至 3 公尺。

25. 磁力計校正的幾何本質是把變形的量測點雲還原成正球：硬鐵誤差造成球心平移（以偏移向量 b 修正），軟鐵誤差造成球變橢球（以矩陣 A 修正）；校正須在最終安裝狀態下、遠離環境干擾處進行。

26. 無人機最依賴羅盤也最易受其害，羅盤錯誤會導致「馬桶效應」；室內機器人則常降低磁力計權重，改以 SLAM 掃描匹配提供航向。

27. 在 AHRS 架構中，陀螺儀管短期一切、加速度計管長期的上下、磁力計管長期的東西南北；加入 SLAM 後，磁力計常退居初始化與異常監測的角色。

下一章，我們深入第二篇最核心的感測器——IMU 慣性導航。加速度計與陀螺儀如何量測運動？為什麼積分會產生漂移？姿態濾波、IMU 預積分與視覺慣性緊耦合又是什麼？這些將是後續視覺 SLAM 與無人機導航章節的重要基礎。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

## 第 7 章

## IMU 慣性導航

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

## 本章在全書的位置

第 6 章的磁力計給了機器人一個絕對方向，但它慢、而且脆弱。本章的慣性量測單元補上相反的性格：極快、完全不受外界干擾，但只要時間一長就錯得離譜。

IMU 在全書中的地位很特殊。它既是第二篇的一員感測器，也是第六篇視覺慣性 SLAM 的主角之一。更重要的是，它是全書第一個「必須靠積分才能用」的感測器——這帶出了 SLAM 最核心的矛盾：遞推（Dead Reckoning）快而連續，卻必然漂移；觀測（Observation）慢而離散，卻不漂移。整本書從第 11 章的濾波、第 12 章的圖優化，到第 20 章之後的閉環偵測，都是在處理這個矛盾的不同面向。

因此本章請同學特別留意的不是 MEMS 的物理原理，而是 7.3 節的誤差成長規律與 7.5 節的預積分思想。前者解釋了「為什麼不能只靠 IMU」，後者則是現代視覺慣性系統得以即時運行的關鍵工程技巧。

**圖 7-0 第 7 章與全書的連結地圖**

【承接前章】

第 6 章 磁力計：絕對、不漂移，但慢且易受干擾

本 章 IMU ：快、不受干擾，但必然漂移 → 兩者互補成 AHRS

【本章負責】

比力與科氏力的量測原理 → 姿態解算 → 積分漂移（$t^2$ 與 $t^3$）

姿態濾波：互補 → Kalman → EKF

預積分：讓高頻 IMU 能進入低頻的最佳化問題

【往下延伸】

→ 第 11 章 本章的互補濾波與 EKF 在此被正式納入貝氏濾波架構

→ 第 12 章 預積分成為位姿圖中的一條邊（7.5 節的伏筆）

→ 第 18 章 LIO：光達慣性里程計

→ 第 23 章 ORB-SLAM3 的視覺慣性緊耦合（7.7 節的完整版）

→ 第 26 章 無人機：IMU 是飛控的心臟，失效即墜機

## 學習目標

IMU 是機器人身上最忙碌的感測器——它以每秒數百次的頻率不斷回報運動狀態，卻也是最容易「說謊」的一個。本章深入慣性導航的原理、限制與現代解法，這些內容將直接支撐第六篇的視覺慣性 SLAM 與第七篇的無人機導航。學完本章之後，同學應能夠達成下列目標：

36. 說明加速度計與陀螺儀的 MEMS 量測原理，並解釋加速度計為何在靜止時仍讀到約 9.8 的數值。
37. 定義滾轉、俯仰、偏航三個姿態角，並由加速度計計算滾轉與俯仰角。
38. 解釋積分漂移的成因，並說明位置誤差為何隨時間以平方甚至三次方成長。
39. 說明互補濾波器與卡爾曼濾波器的基本思想，並比較兩者的適用場合。

40. 說明 IMU 預積分要解決什麼問題，以及它為何是現代視覺慣性 SLAM 的關鍵技術。

41. 說明 IMU 在視覺慣性 SLAM 中的四大貢獻，並解釋鬆耦合與緊耦合的差異。

42. 說明 ORB-SLAM3 如何以最大後驗估測完成 IMU 初始化，以及初始化需要「運動激勵」的原因。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 慣性量測單元 | IMU | 整合加速度計與陀螺儀（有時含磁力計）的感測器模組。 |
| 比力 | Specific Force | 加速度計實際量到的量：運動加速度減去重力加速度。 |
| 科氏力 | Coriolis Force | MEMS 陀螺儀藉以量測角速度的物理效應。 |
| 姿態角 | Roll / Pitch / Yaw | 滾轉、俯仰、偏航，描述剛體在三維空間中的朝向。 |
| 零偏 | Bias | 感測器在零輸入時仍有的固定輸出誤差，會隨溫度與時間變化。 |
| 隨機游走 | Random Walk | 白雜訊經積分後產生的隨機漂移，誤差隨時間平方根成長。 |
| 積分漂移 | Integration Drift | 微小誤差經積分不斷累積，使估測值逐漸偏離真值。 |
| 互補濾波器 | Complementary Filter | 以高通與低通互補的方式融合陀螺儀與加速度計。 |
| 卡爾曼濾波器 | Kalman Filter | 在高斯雜訊假設下的最佳線性遞迴估測器。 |
| 擴展卡爾曼濾波 | EKF | 將非線性系統線性化後套用卡爾曼濾波的方法。 |
| IMU 預積分 | IMU Preintegration | 把兩關鍵幀之間的 IMU 量測預先積分為單一約束，避免重複計算。 |

| 中文名詞 | 英文名詞 | 概念說明 |
| --- | --- | --- |
| 緊耦合 | Tightly Coupled | 將 IMU 原始量測與視覺量測放進同一個最佳化問題求解。 |
| 最大後驗估測 | MAP | 在已知觀測下尋找機率最大的狀態估計，ORB-SLAM3 初始化的理論基礎。 |
| 尺度不確定性 | Scale Ambiguity | 單目相機無法確定真實尺寸，需由 IMU 或已知物體提供尺度。 |

## 7.1 加速度計、陀螺儀、磁力計

IMU（Inertial Measurement Unit，慣性量測單元）的核心是兩種感測器：加速度計量測線加速度、陀螺儀量測角速度。第 6 章介紹的磁力計常一併封裝，構成所謂的「九軸」模組。這三者的分工，正是第 6.6 節提過的：陀螺儀管短期一切、加速度計管長期的上下、磁力計管長期的東西南北。

### 一、加速度計：一顆彈簧上的質量塊

MEMS 加速度計的原理可以用一個極簡模型理解：一個微小的質量塊（Proof Mass）以彈性樑懸掛在晶片中。當晶片加速時，質量塊因慣性而相對位移，這個位移改變了電容極板間距，由電路量出電容變化即可換算加速度。整個結構刻在數平方公釐的矽晶片上，成本可低至數十元台幣。

這裡有一個關鍵觀念，初學者幾乎必定會困惑：把加速度計靜置在桌上，它讀到的不是 0，而是垂直方向約 9.8 m/s²。為什麼靜止的東西會有加速度？

**觀念釐清：加速度計量的是「比力」而非「加速度」**

加速度計實際量測的物理量是比力（Specific Force），定義為：

a_measured = a_true − g

其中 a_true 是載體相對慣性系的真實加速度、g 是重力加速度向量。

直觀理解：加速度計量的是「質量塊被彈簧推了多用力」。靜止時，彈簧必須施力抵抗重力把質量塊撐住，因此讀到約 9.8；

而自由落體時，質量塊與外殼一起下墜、彈簧完全不受力，加速度計反而讀到 0——這正是太空站「失重」的物理本質。

結論：使用加速度計前，必須先知道重力方向並扣除它，才能得到真正的運動加速度。而重力方向又必須由姿態決定——

這個「姿態與加速度互相依賴」的循環，正是慣性導航困難的根源之一。

## 二、陀螺儀：振動與科氏力

MEMS 陀螺儀量測的是角速度。它的原理利用科氏力（Coriolis Force）：晶片內的質量塊被驅動做高頻往復振動；當整體發生旋轉時，振動中的質量塊會受到垂直於振動方向的科氏力作用，產生可偵測的側向位移，位移量正比於角速度。

要特別注意：陀螺儀輸出的是角速度（每秒轉幾度），不是角度。要得到角度，必須對時間積分——而這正是漂移問題的來源（7.3 節）。

## 三、等級與規格

IMU 依精度分為數個等級，價格可以差上數千倍：

| 等級 | 典型零偏穩定度 | 價格級距 | 典型應用 |
|---|---|---|---|
| 消費級 MEMS | 每小時數十度 | 數十至數百元 | 手機、玩具、教學模組 |
| 工業級 MEMS | 每小時數度 | 數千至數萬元 | 無人機、機器人、AGV |
| 戰術級 | 每小時 0.1 至 1 度 | 數十萬元 | 導引、測繪、自駕車 |
| 導航級 | 每小時 0.001 度以下 | 數百萬元以上 | 航空、潛艦、太空 |

**表 7-1　IMU 精度等級與應用**

本書教學實驗使用的 MPU-6050、MPU-9250、ICM-20948、BNO055 等模組屬於消費級到工業級之間；無人機飛控（Pixhawk 系列）則多採工業級並常配置多顆互相驗證。理解等級差異很重要：同一套演算法在戰術級 IMU 上可以獨立推算數分鐘，在消費級模組上

可能數秒就失準——這決定了系統設計時能對 IMU 寄予多大的信任。

## 7.2 姿態估測：Roll、Pitch、Yaw

姿態（Attitude）描述剛體在空間中的朝向。航空領域慣用的歐拉角表示法有三個角度，同學可以用飛機來記憶：

- 滾轉角（Roll，φ）：繞機身前後軸旋轉——飛機左右翼上下擺動，機身「側傾」。
- 俯仰角（Pitch，θ）：繞機身左右軸旋轉——機頭上仰或下俯。
- 偏航角（Yaw，ψ）：繞垂直軸旋轉——機頭左右轉向，也就是第 6 章談的航向角。

### 由加速度計求滾轉與俯仰

在載體靜止或等速運動時，加速度計量到的就是重力在三個軸上的分量。既然知道重力必定指向地心（正下方），比對三軸讀值就能反推出感測器傾斜了多少：

$$\varphi = \operatorname{atan2}(a_y, a_z)$$

$$\theta = \operatorname{atan2}\left(-a_x, \sqrt{a_y^2 + a_z^2}\right)$$

### 關鍵限制：加速度計管不到偏航角

這裡有一個必須牢記的限制：加速度計無法量測偏航角。原因很直觀——把裝置水平放在桌上原地旋轉，重力方向始終不變，三軸讀值也就不變，加速度計對此完全「無感」。這正是第 6 章磁力計不可或缺的原因，也是純視覺 SLAM 需要其他資訊提供初始航向的原因。

### 第二個限制：加速度污染

上述公式有一個前提：加速度計量到的「只有重力」。但機器人一旦加速、轉彎、震動，讀值就是重力與運動加速度的疊加，此時算出的姿態角會嚴重失真。一台急加速的小車，用這個公式會誤判成「車頭上仰」——因為向前的加速度在感測器看來與「傾斜後重力的分量」無法區分。

這個限制帶來一個重要結論：加速度計適合提供長期平均的姿態參考，但不能用於瞬時姿態估測。而陀螺儀恰好相反——短期精確、長期漂移。兩者的互補性，就是 7.4 節姿態濾波的全部動機。

### 歐拉角的萬向鎖問題

歐拉角直觀好懂，但有一個著名缺陷：當俯仰角接近 ±90 度（機頭垂直朝上或朝下）時，滾轉與偏航兩個旋轉軸會重合，喪失一個自由度，稱為萬向鎖（Gimbal Lock）。這對無人機特技飛行或機械手臂是致命的。因此實務系統內部多改用四元數（Quaternion）表示姿態——它沒有奇異點、運算效率高，只是不易直觀理解。四元數的完整介紹留待第 10 章，本章先以歐拉角建立直覺即可。

## 7.3 積分漂移問題

慣性導航的美好願景是：只要有 IMU，機器人就能完全自給自足地知道自己的位姿——不需要衛星、不需要地標、不需要任何外部訊號。這個願景在理論上完全成立，在實務上卻幾乎不可能單獨實現。原因只有一個字：積分。

### 積分的雙重放大

從 IMU 讀值推算位姿，需要做積分：

$$\text{角速度 } \omega \xrightarrow{\text{積分一次}} \text{姿態角}$$

$$\text{加速度 } a \xrightarrow{\text{積分一次}} \text{速度} \xrightarrow{\text{再積分一次}} \text{位置}$$

積分是一個「只進不出」的運算：每一刻的微小誤差都會被永久累加進結果，而且永遠不會自己抵消。更糟的是位置需要積分兩次，誤差被放大兩輪。

### 誤差成長的數學

假設加速度計有一個固定的零偏誤差 b（例如 0.01 m/s²，這在消費級模組上很常見），則經過時間 t 後：

$$速度誤差 = b \cdot t$$

$$位置誤差 = (1/2) \cdot b \cdot t^2$$

代入數字感受一下：b = 0.01 m/s² 時，10 秒後位置誤差約 0.5 公尺，60 秒後約 18 公尺，300 秒後高達 450 公尺。這還只是加速度計的零偏；陀螺儀的零偏會造成姿態誤差隨時間線性成長，而姿態一旦錯了，重力就扣不乾淨，殘餘的重力分量又會被當成運動加速度積分兩次——這條路徑造成的位置誤差甚至隨 $t^3$ 成長，是實務上最主要的誤差來源。

**直觀理解：姿態誤差如何毒害位置估測**

假設姿態估計只偏了 1 度。此時系統認為的「垂直方向」與真正的垂直方向差 1 度，

扣除重力後會殘留一個水平分量：9.8 × sin(1°) ≈ 0.17 m/s²。

這個「不存在的加速度」經過兩次積分，10 秒後就造成約 8.5 公尺的位置誤差。

結論：在慣性導航中，姿態精度比加速度精度更關鍵。這正是 7.4 節姿態濾波如此重要的原因，

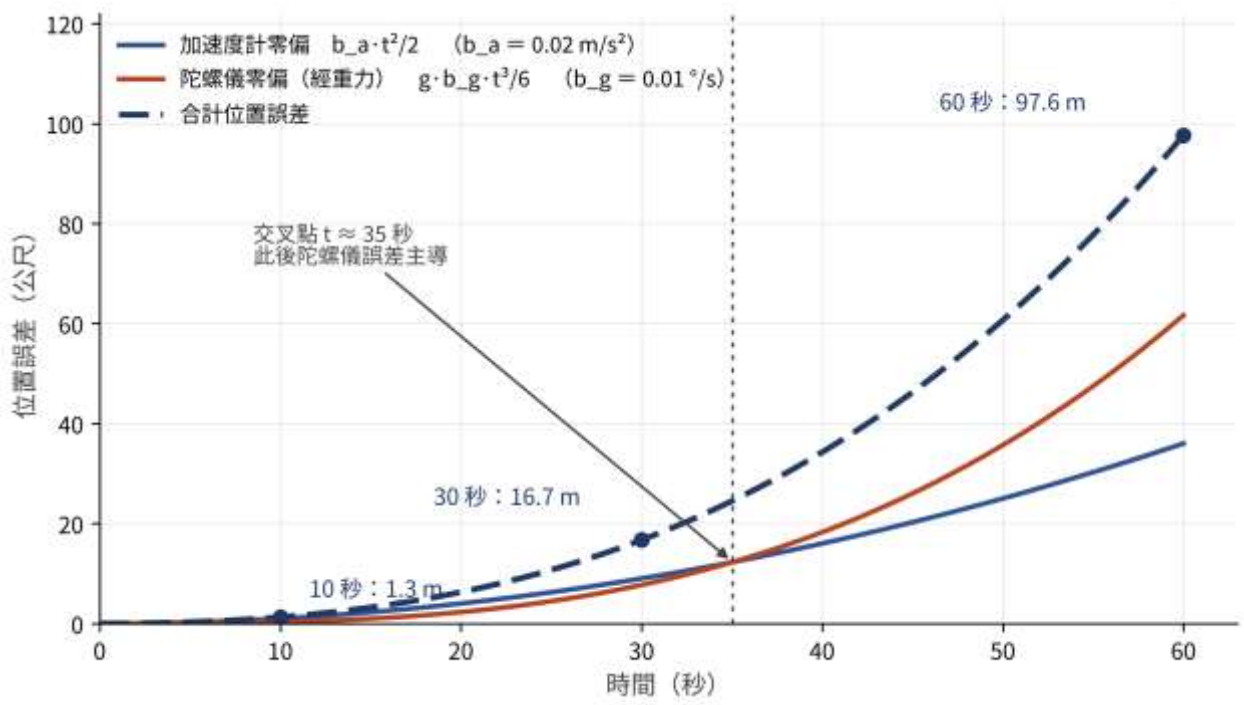
也是為什麼所有實用系統都必須引入外部資訊（磁力計、GNSS、視覺）來約束姿態。

## 誤差的三種來源

- 零偏（Bias）：零輸入時的固定輸出偏移。它並非常數，會隨溫度變化（開機暖機期間尤其明顯）與時間緩慢遊走。這是最主要的誤差源，也是各種濾波器與最佳化演算法都要「線上估計」的對象。

- 雜訊（Noise）：隨機的高頻擾動。積分後形成隨機游走，誤差隨時間的平方根成長——相較零偏的線性或平方成長，這一項反而不是主要問題。
- 尺度因子與軸不正交誤差：實際靈敏度與標稱值的偏差、三軸未完全垂直。可透過事前校正大幅減輕。

**圖 7-1　消費級 MEMS IMU 純慣性推算的位置誤差成長**




這張圖用具體數字說明了 7.3 節的全部重點。圖中採用典型消費級 MEMS 的零偏水準：加速度計零偏 0.02 公尺每平方秒（約 2 mg），陀螺儀零偏 0.01 度每秒。兩條誤差曲線的形狀截然不同——加速度計零偏造成的誤差以 $t^2$ 成長，陀螺儀零偏透過「姿態錯誤使重力扣不乾淨」的途徑造成 $t^3$ 成長。

請注意那個交叉點：約在三十五秒處，陀螺儀造成的誤差超越加速度計，此後完全主導。合計誤差在十秒時約 1.3 公尺、三十秒時約 16.7 公尺、六十秒時已接近百公尺。一分鐘，一百公尺——這就是為什麼沒有人會用消費級 IMU 單獨導航。

這張圖也解釋了兩件在工程上非常重要的事。其一，為什麼高階 IMU 的價格幾乎全花在陀螺儀上：因為 $t^3$ 項最終會主導，降低陀螺儀零偏的效益遠大於降低加速度計零偏。其二，為什麼視覺慣性系統的關鍵參數是「多久能得到一次視覺觀測」：只要在誤差尚小的區間內

（例如零點幾秒的關鍵幀間隔）就把 IMU 拉回正軌，這條曲線就永遠跑不到右半邊。第 23 章 ORB-SLAM3 能在無人機上達到公分級精度，靠的正是這個策略。

> **一個值得記住的數量級**
>
> 姿態誤差 $\propto t$（線性）、速度誤差 $\propto t^2$、位置誤差 $\propto t^3$ ——每多積分一次，時間的次方就加一。
>
> 這個「積分放大誤差」的規律不只屬於 IMU。第 13 章的輪式里程計、第 20 章的視覺里程計，全都是遞推式感測，也全都逃不過累積漂移。
>
> 差別只在成長速度：輪式里程計約為線性至平方、視覺里程計約為線性、IMU 是三次方——所以 IMU 的漂移最兇猛，也最需要被頻繁校正。

**務實的結論**

IMU 的正確用法不是「拿來獨立導航」，而是「拿來填補其他感測器的空隙」。消費級 IMU 的可用推算時間約在數秒等級，工業級可達數十秒，戰術級以上才能撐到分鐘等級。因此系統設計的核心問題永遠是：在 IMU 漂移到不可接受之前，下一次外部校正會在什麼時候到來？無論那個校正來自 GNSS（第 5 章）、光達掃描匹配（第 13 章）還是視覺特徵（第 20 章），道理都是一樣的。

## 7.4 姿態濾波：Complementary Filter、Kalman Filter、EKF

既然陀螺儀短期準、長期飄，加速度計長期準、短期受污染，答案已經呼之欲出：把兩者融合，各取所長。這就是姿態濾波要做的事。

**一、互補濾波器：最簡單而有效**

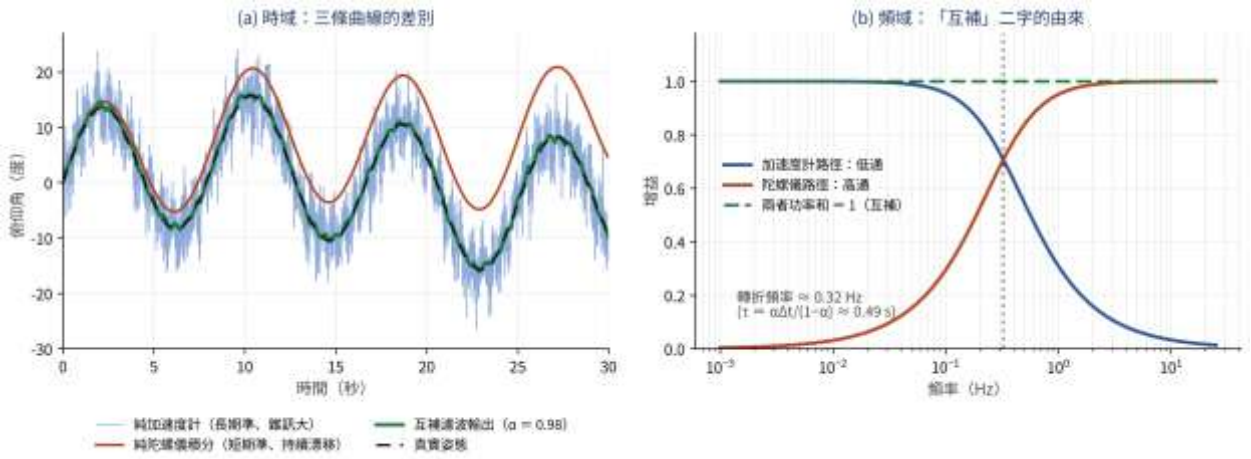
互補濾波器的想法極為直觀：對陀螺儀的積分結果取「高通」（相信它的快速變化）、對加速度計算出的角度取「低通」（相信它的長期平均），兩者加權相加：

$$\theta = \alpha \cdot (\theta_{prev} + \omega \cdot \Delta t) + (1 - \alpha) \cdot \theta_{accel}$$

其中 α 通常取 0.95 至 0.99 之間。α 越大表示越信任陀螺儀，輸出越平滑但抗漂移能力較弱；α 越小則越快跟隨加速度計，但也越容易被震動干擾。

- 優點：只需幾行程式碼、幾乎不佔運算資源、參數只有一個且物理意義清晰，非常適合 Arduino 等級的微控制器與教學實驗。
- 限制：沒有誤差的統計模型，無法估計零偏，也無法提供「估計有多不確定」的資訊。

**圖 7-2　互補濾波器的時域表現與頻域意義**




左圖是同學在實作 7-1 將親手畫出的結果。淺藍色是純加速度計計算的姿態：長期來看圍繞真值，但雜訊極大；紅色是純陀螺儀積分：曲線平滑漂亮，卻持續往上偏離；綠色是互補濾波的輸出，幾乎與黑色虛線的真實姿態重合。

右圖解釋了「互補」這兩個字從何而來。互補濾波器實際上是在做頻率分工：加速度計的訊號經過一個低通濾波器（只保留慢變化，濾掉震動雜訊），陀螺儀的訊號經過一個高通濾波器（只保留快變化，濾掉緩慢漂移），兩條路徑的響應恰好互補，加起來在所有頻率上都等於一。

轉折頻率由 α 與取樣週期決定：時間常數 $\tau = \alpha\Delta t / (1 - \alpha)$。以 $\alpha = 0.98$、$\Delta t = 0.01$ 秒計算，τ 約 0.49 秒，轉折頻率約 0.32 赫茲。這給了同學一個調參的物理依據：比這更慢的姿態變化交給加速度計判斷，比這更快的交給陀螺儀。若機器人運動劇烈、加速度污染嚴重，

就提高 α（相信陀螺儀久一點）；若陀螺儀品質差、漂移快，就降低 α。

這個「依訊號特性分配信任」的思想，在本書後面會不斷以更複雜的形式出現：第 11 章的卡爾曼增益就是依不確定度自動算出的 α，第 12 章圖優化中各條邊的資訊矩陣是同一件事的矩陣版本，而第 27 章多機器人系統中「該相信自己還是隊友」的判斷，本質上也是同一個問題。

## 二、卡爾曼濾波器：最佳線性估測

卡爾曼濾波器把問題提升到機率的層次。它不只估計「姿態是多少」，還同時追蹤「這個估計有多不確定」（以共變異數矩陣表示），並在每一步做兩件事：

34. 預測（Predict）：用陀螺儀讀值推算下一刻的姿態，同時讓不確定性增大——因為積分會累積誤差。
35. 更新（Update）：用加速度計的觀測修正預測值。修正的力道由「卡爾曼增益」決定：若預測的不確定性大、觀測的雜訊小，就多相信觀測；反之則多相信預測。

這個「預測—更新」的循環，正是第 1 章 SLAM 標準流程的縮影，也是第 11 章要深入展開的核心數學。卡爾曼濾波器的一大優勢是可以把零偏納入狀態向量一起估計——系統會自動學習「這顆陀螺儀今天偏了多少」，這是互補濾波器做不到的。

## 三、擴展卡爾曼濾波（EKF）：處理非線性

標準卡爾曼濾波要求系統是線性的，但姿態運動學充滿三角函數與四元數乘法，本質上是非線性的。EKF 的解法是在當前估計點附近做一階泰勒展開（求雅可比矩陣）將系統線性化，再套用卡爾曼濾波的公式。

EKF 是機器人領域最廣泛使用的姿態估測方法：無人機飛控（PX4、ArduPilot 的 EKF2 / EKF3）、ROS 的 robot_localization 套件、手機的感測器融合，背後幾乎都是 EKF。它的限制是：線性化在誤差很大時可能失準，且需要合理的初始估計才能收斂。

| 方法 | 運算需求 | 可估零偏 | 提供不確定度 | 適用場合 |
|---|---|---|---|---|
| 互補濾波器 | 極低 | 否 | 否 | 教學、微控制器、簡易平台 |
| 卡爾曼濾波器 | 中 | 是 | 是 | 線性系統、簡化姿態模型 |
| EKF | 中高 | 是 | 是 | 無人機飛控、機器人主流方案 |
| Mahony / Madgwick | 低 | 部分 | 否 | 開源飛控、AHRS 模組常用 |

**表 7-2　常見姿態濾波方法比較**

補充說明：Mahony 與 Madgwick 濾波器是介於互補濾波與卡爾曼之間的實用方案，以四元數表示姿態並採用梯度下降或比例積分修正，運算量小卻表現不俗，在開源飛控與 AHRS 模組中極為常見，適合作為實作進階的第一站。

## 7.5　IMU 預積分概念

本節進入現代視覺慣性 SLAM 的關鍵技術。內容較為進階，高中課程可只掌握「要解決什麼問題」的直覺，大學部則應理解其設計邏輯。

### 問題的起點：頻率不匹配

視覺 SLAM 的相機大約每秒 20 至 30 幀，IMU 卻高達每秒 200 至 1000 次。這意味著相鄰兩張影像之間，夾著數十到上百筆 IMU 量測。如何把這一大堆高頻資料，變成後端最佳化能用的「一條約束」？

### 樸素做法的致命問題

最直覺的做法是：把這段時間內的 IMU 量測直接積分，得到「從第 i 幀到第 j 幀的位姿變化」，當作一條約束加入最佳化。

但這裡有一個嚴重問題。積分加速度時，必須先扣除重力，而扣除重力需要知道姿態；

姿態又與第 i 幀的位姿估計有關。最佳化過程中，第 i 幀的位姿估計會被反覆更新——每更新一次，這段 IMU 積分就必須全部重算一遍。以上百筆量測、數十次迭代計算，運算量將完全無法即時處理。

### 預積分的核心洞見

預積分（Preintegration）的巧妙之處在於：把積分結果表示成「與初始位姿無關」的形式。

具體做法是，不積分「世界座標系中的位姿變化」，而是積分「第 i 幀機體座標系中的相對運動量」。這些相對量只依賴 IMU 的原始讀值與零偏估計，與第 i 幀在世界中的位置、朝向完全無關。於是最佳化過程中無論第 i 幀的位姿如何被更新，預積分結果都不必重算——只需在最外層做一次簡單的座標變換即可。

**圖 7-3　預積分解決的問題**

【樸素做法】

第 i 幀位姿更新 → 重力方向改變 → 整段 IMU 必須重新積分 → 運算量爆炸

【預積分做法】

預先積分「相對運動量」（與初始位姿無關）→ 只算一次

↓

最佳化時，第 i 幀位姿如何變動，都只需外層做一次座標變換

↓

運算量大幅下降，得以即時執行

### 預積分產出什麼

兩個關鍵幀之間的預積分，會產出三個相對量與一個不確定度：

- 相對旋轉 ΔR：這段時間機體轉了多少。
- 相對速度變化 Δv：速度改變了多少。
- 相對位置變化 Δp：在初始機體座標系中移動了多少。
- 共變異數矩陣：這條約束有多可信，用來決定它在最佳化中的權重。

### 零偏的處理

還有一個細節：積分時必須先扣掉零偏，但零偏本身也是待估計的未知數。若最佳化過程中零偏估計改變了，預積分不就又要重算？預積分理論的解法是同時計算「預積分結果對零偏的一階偏微分（雅可比）」，零偏微幅改變時只需用線性修正快速更新，不必重新積分。

預積分理論由 Forster 等人於 2015 年提出並嚴謹處理了旋轉群的流形結構，如今已成為所有主流視覺慣性系統的標準元件——VINS-Mono、OKVIS、ORB-SLAM3 無一例外。這也呼應第 2 章 MIT VNAV 課程為何要花大量篇幅講「流形上的最佳化」：旋轉不是普通的向量，處理它需要專門的數學工具。

## 7.6 視覺慣性 SLAM 中的 IMU 角色

為什麼視覺 SLAM 一定要加上 IMU？答案不是「多一個感測器多一份保險」這麼簡單——IMU 補上的是純視覺方法在原理上就辦不到的幾件事。

### IMU 的四大貢獻

36. 提供絕對尺度：單目相機有先天的尺度不確定性——同一段影像序列，可能是「大房間中的大運動」，也可能是「模型屋中的小運動」，純視覺無法分辨。IMU 量測的是有物理單位的加速度，積分後即可提供真實尺度。這是單目視覺慣性系統最重要的價值。

37. 提供重力方向：加速度計長期平均指向重力，使系統能確定「哪邊是下」。這讓滾轉與俯仰角成為可觀測量，地圖也能被擺正——對無人機控制與 AR 應用至關重要。
38. 填補視覺失效的空隙：快速轉動造成影像模糊、鏡頭被短暫遮擋、進入白牆或黑暗區域時，視覺前端會跟丟。IMU 在數秒內仍能維持可用的推算，讓系統撐過這段空白並重新接上——這大幅提升了系統的強韌性。
39. 提供高頻輸出：相機每秒 30 幀對飛行控制而言太慢。IMU 讓系統能以數百 Hz 輸出位姿，滿足即時控制迴路的需求。

### 視覺對 IMU 的回報

這個合作是雙向的。視覺提供的是 IMU 最缺的東西：不漂移的位置與姿態觀測。透過視覺約束，系統能持續估計並補償 IMU 的零偏，讓 7.3 節的漂移噩夢受到控制。兩者的關係可以這樣總結：IMU 給視覺尺度與重力，視覺給 IMU 校正與約束。

### 鬆耦合與緊耦合

- 鬆耦合（Loosely Coupled）：視覺系統與慣性系統各自獨立計算出位姿，再把兩個結果加權融合。優點是模組化、易實作、可直接沿用現成的視覺 SLAM；缺點是資訊在各自處理階段就已經被壓縮，融合時無法再回頭利用原始量測，精度受限。
- 緊耦合（Tightly Coupled）：把視覺的重投影誤差與 IMU 的預積分約束放進同一個最佳化問題，一起求解位姿、速度、零偏與地圖點。優點是所有資訊在同一個機率架構下被一致處理，精度顯著較佳；缺點是實作複雜、運算量大。

現代高精度系統一律採用緊耦合。這也再次呼應第 4 章 4.7 節與第 5 章的融合討論——同學應該已經發現，「緊耦合優於鬆耦合」是貫穿整本書的一個普遍原則：越晚壓縮資訊，越能求得最佳解。

## 7.7 ORB-SLAM3 與 IMU 緊耦合

ORB-SLAM3 是目前最完整的開源視覺慣性 SLAM 系統，由西班牙薩拉戈薩大學團隊開

發。它是第一套能同時支援純視覺、視覺慣性與多地圖 SLAM，並相容單目、雙目、RGB-D 相機以及針孔與魚眼鏡頭模型的系統。本節先建立整體概念，完整的系統剖析留待第 23 章。

### 核心創新：全程以最大後驗估測為基礎

ORB-SLAM3 最重要的貢獻，是一套完全建立在最大後驗估測（MAP，Maximum-a-Posteriori）之上的緊耦合視覺慣性系統——連 IMU 初始化階段也不例外。這使得系統能在室內外、大小場景中穩健即時運作，精度較先前方法提升約二至十倍。

### 為什麼 IMU 初始化這麼困難

系統剛啟動時，有一堆未知數必須同時確定：地圖的真實尺度、重力方向、加速度計與陀螺儀的零偏、以及初始速度。這些量彼此耦合，而且不是全部都能立刻觀測到。

早期系統（如 ORB-SLAM-VI）的初始化程序需要約 15 秒才能完成，這段期間系統精度不佳，嚴重影響穩健性與使用體驗。ORB-SLAM3 把初始化重新表述為一個最佳化問題——在 MAP 的框架下，同時求解尺度、重力方向、零偏與速度的最可能值，使初始化大幅加速且更為可靠。

### 運動激勵：初始化的必要條件

這裡有一個實作上必須知道的關鍵：IMU 初始化需要足夠的運動激勵（Motion Excitation）。原因在於，如果載體只是等速直線移動或完全靜止，加速度計讀到的幾乎只有重力，系統無法把「尺度」與「零偏」區分開來——數學上稱這些量不可觀測（Unobservable）。

因此實務操作時，啟動視覺慣性 SLAM 後應先做一段帶有加減速與旋轉的運動（拿著相機小幅度前後左右移動並轉動數秒），系統才能順利完成初始化。這是同學在第 30 章實作 ORB-SLAM3 時最常遇到的問題之一：「為什麼一直初始化不成功？」答案往往是動得不夠。

### 三執行緒架構中的 IMU

- 追蹤執行緒（Tracking）：對每一幀提取 ORB 特徵，最小化重投影誤差與慣性誤差以估計相機位姿；在視覺慣性模式下，還會一併估計速度與 IMU 零偏。
- 局部建圖執行緒（Local Mapping）：插入關鍵幀、剔除冗餘資訊，並執行局部 Bundle Adjustment——在慣性模式下即為視覺慣性 BA，同時以 MAP 估測完成並持續精修 IMU 參數。
- 迴路與地圖合併執行緒：偵測閉環與地圖重合，執行全域修正。

### 多地圖系統：跟丟不再是災難

ORB-SLAM3 的第二項創新是多地圖架構（Atlas）。當系統因長時間視覺資訊不良而跟丟時，它不會崩潰，而是開啟一張新地圖繼續運作；日後重新回到曾經走過的區域時，再把新舊地圖無縫合併。這對長時間運行的機器人極為實用——第 23 章會完整討論這個機制。

### 精度表現

在公開資料集上，ORB-SLAM3 的雙目慣性模式於 EuRoC 無人機資料集達到平均約 3.5 公分的精度；在代表 AR / VR 情境、以手持快速運動錄製的 TUM-VI 房間序列中，精度可達約 9 公釐。這個數量級足以說明：一顆便宜的相機加上一顆便宜的 IMU，在正確的演算法下，能達到令人驚訝的精度——而這正是本書希望同學親手驗證的事。

## 公式或流程

### 一、慣性導航基本運動學方程

設世界座標系中的位置為 p、速度為 v、姿態旋轉矩陣為 R；IMU 量得角速度 ω 與比力 a，重力向量為 g。連續時間的更新方程為：

$$\dot{R} = R \cdot [\omega]_\times$$

$$\dot{v} = R \cdot a + g$$

$$\dot{p} = v$$

其中 $[\omega]_\times$ 表示角速度的反對稱矩陣。這三條式子就是慣性導航的全部——但也正因為

要對它們積分兩次，才有了 7.3 節的漂移問題。離散化後的實作形式為：

$$R(k+1) = R(k) \cdot Exp(\omega \cdot \Delta t)$$

$$v(k+1) = v(k) + (R(k) \cdot a + g) \cdot \Delta t$$

$$p(k+1) = p(k) + v(k) \cdot \Delta t + (1/2) \cdot (R(k) \cdot a + g) \cdot \Delta t^2$$

## 二、零偏造成的誤差成長

若加速度計零偏為 b_a、陀螺儀零偏為 b_g，主要誤差項的成長行為為：

$$姿態誤差 \approx b_g \cdot t \quad (線性)$$

$$速度誤差 \approx b_a \cdot t + g \cdot b_g \cdot t^2/2$$

$$位置誤差 \approx b_a \cdot t^2/2 + g \cdot b_g \cdot t^3/6 \quad (三次方項主導)$$

注意第三式中含重力 g 的 $t^3$ 項：陀螺儀零偏透過「姿態錯誤 → 重力扣不乾淨 → 假加速度」的途徑，最終成為長時間慣性導航的主要誤差來源。這條式子解釋了為什麼高階 IMU 的價格幾乎全花在陀螺儀上。

## 三、互補濾波器完整實作流程

**圖 7-4 互補濾波器姿態估測流程**

① 讀取 IMU：三軸加速度 a、三軸角速度 ω

↓

② 由加速度計計算姿態：φ_acc = atan2(a_y, a_z)，θ_acc = atan2( - a_x, √(a_y² + a_z²))

↓

③ 由陀螺儀積分：φ_gyro = φ_prev + ω_x · Δt（俯仰同理）

↓

④ 互補融合：φ = α · φ_gyro + (1 - α) · φ_acc （α 約 0.95 ~ 0.99）

↓

⑤ 偏航角 ψ：由磁力計提供（第 6 章），或由陀螺儀積分但接受其漂移

↓

⑥ 輸出姿態（φ, θ, ψ），並更新 φ_prev、θ_prev 供下一輪使用

## 四、視覺慣性緊耦合的代價函數

緊耦合系統把所有資訊放進同一個最佳化問題，其代價函數的結構為：

$$\min \Sigma \| \text{視覺重投影誤差} \|^2 + \Sigma \| \text{IMU 預積分誤差} \|^2 + \| \text{先驗誤差} \|^2$$

第一項要求「地圖點投影到影像上的位置」與「實際觀測到的特徵點位置」盡量接近；第二項要求「相鄰關鍵幀之間的位姿變化」與「IMU 預積分推算的結果」盡量一致；第三項則保留先前最佳化的資訊。待求解的變數包含所有關鍵幀的位姿、速度、IMU 零偏與地圖點座標。

這正是第 12 章圖優化的完整形式——同學此刻只需理解一件事：所謂「緊耦合」，就是讓不同感測器的誤差項在同一條式子裡互相角力，最後求出一個對所有觀測都最合理的答

案。

## 7.8 慣性感測的當代發展：當 IMU 遇上深度學習

IMU 是一個成熟到近乎沉悶的領域——MEMS 的物理原理數十年未變，預積分的數學在 2015 至 2017 年間也已大致定型。但 2024 至 2026 年間出現了一條新路線，值得同學認識：用學習的方法處理 IMU 的誤差。

### 一、問題的癥結：零偏不是常數

回顧 7.3 節，IMU 誤差的主要來源是零偏。傳統做法把零偏當成「緩慢變化的隨機遊走」，以濾波器或最佳化估計它——7.4 節的 EKF 與 7.7 節的 ORB-SLAM3 都是這麼做的。

但真實的零偏行為更複雜：它隨溫度、電源電壓、機械應力變化，而且與感測器本身的非線性特性糾纏在一起。用一個簡單的隨機遊走模型去描述它，本質上是一種妥協。深度學習的切入點正在這裡——如果零偏的變化有規律，為什麼不讓模型去學？

### 二、學習零偏動態：以神經微分方程建模

2025 年的一項代表性工作提出以神經常微分方程（Neural ODE）直接建模零偏的連續時間動態，而非把它視為隨機遊走。這個方法有幾個設計上的巧思值得同學注意。

首先，訓練不需要零偏的真值——真值根本無法直接量測；研究者改以位姿真值建構損失函數，讓模型間接學會零偏。其次，由於 IMU 的運動學定義在矩陣李群上（第 10 章會正式介紹這個概念），作者把神經微分方程延伸到李代數的形式，使模型的輸出天生滿足旋轉的幾何約束。第三，IMU 的離散量測以三次 Hermite 樣條內插成連續的控制輸入，再採先陀螺儀、後加速度計的階層式訓練策略。

在 EuRoC 與 TUM-VI 資料集（同學在 7.6 節與實作中會用到的那兩套）以及實體實驗上，該方法明顯改善了純 IMU 積分的誤差，也提升了視覺慣性里程計的整體表現，且展現出跨資料集的泛化能力。

### 三、慣性定位的基礎模型

學習式 IMU 方法長期以來有一個共同弱點：模型在特定資料集上訓練，換一個平台就失效。2025 年 CVPR 的一項工作試圖以基礎模型的思路解決這件事——先在超過一百小時的多平台資料上預訓練，取得通用的運動知識；再以低秩調適（LoRA）針對新任務微調，可訓練參數僅約一百一十萬個；最後在部署時進行線上測試期調適，讓模型邊跑邊學。

其成果是相較於各自專用的模型，絕對軌跡誤差改善約三成六，同時能在機器人上以每秒兩百幀即時運行。這個「預訓練 - 微調 - 線上調適」的三段式結構，與第 33、34 章要介紹的視覺語言動作模型如出一轍，顯示機器人領域各個子題正在往同一種方法論收斂。

### 四、該怎麼看待這條路線

本書的立場是務實的：學習式方法目前的定位是改善 IMU 的輸入品質，而不是取代 7.5 節的預積分或 7.7 節的緊耦合最佳化。實務上最有價值的用法，是把學習模型放在最前端——先把原始讀值去噪、把零偏估得更準，再餵進既有的幾何系統。

這樣的分工有兩個好處：幾何系統的可解釋性與數學保證得以保留，而學習模型只負責它最擅長的「從資料中找出難以建模的規律」。同學在第 32 章會看到，這正是目前 AI 與 SLAM 整合最成熟的模式。

> **給同學的提醒**
>
> 無論模型多強，7.3 節的物理事實不會改變：積分必然放大誤差，$t^3$ 項終究會主導。
>
> 學習式去噪能把零偏降低一個數量級，意思是讓圖 7-1 的那條曲線往右平移——而不是讓它變平。
>
> 真正能讓曲線變平的，只有「不漂移的外部觀測」：視覺、光達、GNSS、閉環。這是 SLAM 存在的根本理由，也是本書後六篇的全部內容。

## 案例

## 案例 7-1 潛艦與飛彈：慣性導航的原初戰場

慣性導航最早的大規模應用出現在冷戰時期的潛艦與洲際飛彈上。潛艦下潛後完全收不到任何外部訊號，必須靠純慣性導航維持數週的航行；飛彈飛行途中不能依賴可能被干擾的外部訊號，同樣只能自主推算。

這些系統使用的是機械式或雷射環形陀螺儀等導航級元件，體積如公事包、重達數十公斤、造價數百萬美元，零偏穩定度比手機 IMU 好上百萬倍。即便如此，潛艦仍必須定期上浮或以其他方式校正——因為 7.3 節的數學是無情的：再小的誤差，乘上足夠長的時間都會變成大誤差。

這個歷史對比極具教學意義：同學手上數十元的 MPU-6050，物理原理與那些天價系統完全相同，差別只在精度等級。也因此，本書強調的重點不是「買更好的 IMU」，而是「用更聰明的演算法與感測器融合，讓便宜的 IMU 發揮最大價值」——這正是過去二十年 SLAM 研究最大的貢獻。

## 案例 7-2 手機的計步器與螢幕自動旋轉

同學口袋裡的手機，每天都在示範慣性感測的應用與限制。螢幕自動旋轉靠的是加速度計判斷重力方向——所以躺在床上平放手機時，它常常轉錯方向，因為此時重力垂直於螢幕，滾轉角在數學上不可觀測（7.2 節的限制在生活中現形）。

計步器則展示了另一種思路：它不積分加速度求位置（那會如 7.3 節所述迅速失準），而是偵測加速度訊號中的週期性峰值來「數步數」，再乘上估計的步長得到距離。這個方法巧妙地繞開了積分漂移——與其積分，不如辨識模式。這正是行人航位推算（PDR）的核心思想，也常與第 4 章的 Wi-Fi 指紋定位、第 6 章的地磁指紋定位搭配使用。

這給同學一個重要啟示：面對感測器的先天限制，除了「用更好的濾波器」之外，還有一條路是「換一種用法」——不要求它做它做不到的事。

### 案例 7-3 EuRoC 資料集：視覺慣性 SLAM 的共同考場

EuRoC MAV 資料集由 ETH Zürich 團隊發布，以微型飛行器搭載雙目相機與 IMU，在機房與工業廠房中錄製多段序列，並以動作捕捉系統與雷射掃描提供高精度真值。它已成為視覺慣性 SLAM 領域的標準評測基準——幾乎所有相關論文都會報告在 EuRoC 上的成績。

資料集刻意包含由易到難的序列：從緩慢平穩的飛行，到快速激烈、影像明顯模糊的機動。這種設計讓研究者能清楚看出演算法在什麼條件下開始失效。ORB-SLAM3 在此資料集上的雙目慣性模式達到平均約 3.5 公分精度，即為在此標準下的成績。

對同學而言，公開資料集的價值在於：不需要無人機、不需要動作捕捉系統，只要一台電腦，就能執行與世界頂尖研究相同的實驗並比較結果。第 30 章的實作將直接使用 EuRoC 資料集——這是「站在巨人肩膀上」最具體的實踐。

## 實作活動

### 實作 7-1 親手見證積分漂移

目的：透過實際量測與計算，親身體驗慣性導航的漂移現象，並實作互補濾波器比較融合前後的差異。這是本書第二篇最重要的一個實驗——它會讓同學對「為什麼需要 SLAM」產生深刻的體感。

**器材**

- 智慧型手機（安裝可記錄並匯出感測器原始資料的 App），或 ESP32 / Arduino 搭配 MPU-6050 / MPU-9250 模組。
- 電腦（Python 環境，需 numpy 與 matplotlib）、捲尺、平坦桌面。

**第一部分：靜止漂移測試**

34. 把裝置完全靜止平放在桌上，記錄 60 秒的三軸加速度與角速度原始資料並匯出。

35. 計算零偏：取前 10 秒資料的平均值。理論上靜止時角速度應為 0、加速度應只有重力，實測偏差即為零偏。記錄三軸陀螺儀零偏（單位轉換為度 / 秒）。

36. 積分實驗（不扣零偏）：把 60 秒的角速度直接積分求角度。裝置明明完全沒動，畫出的角度曲線偏離了多少度？

37. 積分實驗（扣除零偏）：扣掉步驟 2 算出的零偏後重新積分，比較兩條曲線。誤差減少了多少？還剩下多少？剩下的是什麼造成的（提示：7.3 節的三種誤差來源）？

38. 位置漂移：扣除重力後對加速度做兩次積分求位置。裝置靜止 60 秒，算出的「位置」跑了多遠？與 7.3 節的理論估算是否吻合？

### 第二部分：運動測試與濾波

39. 沿桌面把裝置以直線推動 1 公尺後停下，記錄全程資料。以兩次積分計算移動距離，與捲尺實測的 1 公尺比較，計算誤差百分比。

40. 把裝置緩慢傾斜至約 30 度後放回水平，重複數次，記錄資料。

41. 以 Python 分別計算三條姿態曲線：純陀螺儀積分、純加速度計計算、互補濾波融合（$\alpha = 0.98$），畫在同一張圖上比較。

42. 震動測試：輕敲桌面製造震動，觀察三條曲線的反應差異——哪一條會劇烈跳動？哪一條最平滑？哪一條長期最可靠？

43. 參數實驗：把 $\alpha$ 依序設為 0.90、0.95、0.98、0.999，比較輸出曲線的平滑度與跟隨速度，找出你認為最適合的值並說明理由。

### 討論問題

32. 靜止 60 秒後，你們算出的「位置」漂移了多少公尺？如果一台機器人靠這個結果導航，會發生什麼事？

33. 扣除零偏後誤差顯著下降，這說明零偏是主要誤差來源。但零偏會隨溫度與時間變化——這對「開機後立刻校正一次就好」的做法有什麼啟示？

34. 直線推動 1 公尺的實驗中，你們的誤差百分比是多少？誤差主要來自加速階段還是減速階段？為什麼？

35. 比較三條姿態曲線，說明陀螺儀與加速度計各自的優缺點如何在圖上具體呈現。

36. （大學部）依據你們量到的零偏值，套用 7.3 節的公式估算「位置誤差達到 1 公尺」需要多久。這個時間對機器人系統設計意味著什麼？

**延伸挑戰（大學部）**

其一：實作 Madgwick 或 Mahony 濾波器（兩者皆有公開的參考程式碼），以四元數表示姿態，與互補濾波器比較長時間運行的穩定度。其二：下載 EuRoC 資料集的一段序列，讀取其中的 IMU 資料與真值軌跡，實作純慣性推算並與真值比較，繪製誤差隨時間成長的曲線，驗證 7.3 節關於 $t^2$、$t^3$ 成長的理論。這份實驗將直接銜接第 30 章的 ORB-SLAM3 實作。

## 延伸閱讀

41. P. D. Groves, Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems, 2nd ed., Artech House, 2013.（慣性導航的權威教科書，第 4 至 5 章為 IMU 誤差模型的標準參考）

42. M. Kok, J. D. Hol, and T. B. Schön, "Using Inertial Sensors for Position and Orientation Estimation," Foundations and Trends in Signal Processing, 2017.（可免費取得的完整教程，含程式範例，強烈推薦大學部閱讀）

43. C. Forster et al., "On-Manifold Preintegration for Real-Time Visual-Inertial Odometry," IEEE Transactions on Robotics, 2017.（IMU 預積分理論的代表論文，第 7.5 節主要依據）

44. C. Campos et al., "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM," IEEE Transactions on Robotics, 2021.（第 7.7 節主要依據，第 23 章將完整剖析）

45. S. Madgwick, "An Efficient Orientation Filter for Inertial and Inertial/Magnetic Sensor Arrays," Technical Report, University of Bristol, 2010.（Madgwick 濾波器原始文件，附完整程式碼）

46. M. Burri et al., "The EuRoC Micro Aerial Vehicle Datasets," International Journal of Robotics Research, 2016.（EuRoC 資料集說明論文）

47. T. Qin, P. Li, and S. Shen, "VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator," IEEE Transactions on Robotics, 2018.（另一套經典的緊耦合視覺慣性系統，可與 ORB-SLAM3 對照閱讀）

## 從慣性到外感：本章的位置回顧

第 6 章與本章介紹的磁力計與 IMU，合稱本體感測器（Proprioceptive Sensors）——它們感知的是機器人自身的狀態：我朝向哪邊、我轉了多快、我受到多大的加速度。它們不需要環境，也因此在任何環境中都能運作。

但它們看不見世界。IMU 不知道前方有牆，磁力計不知道這條走廊是否走過。第 8 章的光達與第 9 章的相機屬於另一類——外感測器（Exteroceptive Sensors），它們直接量測環境的幾何與外觀。這兩類感測器的分工，是整個機器人感知系統的基本架構：本體感測器提供高頻而漂移的連續估計，外感測器提供低頻而穩定的修正。

下一章我們轉向光達。它不像 IMU 那樣猜測自己動了多少，而是直接以雷射量出「那面牆在三公尺外」。從飛行時間原理、2D 與 3D 光達的差異，到它在掃地機器人、倉儲 AGV 與自駕車中的角色，第 8 章將為第四篇的 2D SLAM 實作揭開序幕。

## 小結

28. 加速度計量測的是比力（運動加速度減重力），因此靜止時讀到約 9.8、自由落體時讀到 0；陀螺儀利用科氏力量測角速度，輸出必須積分才能得到角度。

29. 加速度計可由重力方向解算滾轉與俯仰角，但對偏航角完全無感，且在載體加速時會受運動加速度污染；歐拉角在俯仰角接近 ±90 度時會發生萬向鎖，實務多改用四元數。

30. 積分漂移是慣性導航的根本限制：加速度計零偏造成位置誤差隨 $t^2$ 成長，陀螺儀零偏透過「姿態錯誤使重力扣不乾淨」的途徑造成 $t^3$ 成長；姿態精度比加速度精度更關鍵。

31. 姿態濾波融合陀螺儀（短期準）與加速度計（長期準）：互補濾波器簡單高效但無法估零偏；卡爾曼與 EKF 能同時估計零偏與不確定度，是無人機飛控與機器人的主流方案。

32. IMU 預積分把兩關鍵幀之間的高頻量測積分為「與初始位姿無關」的相對量，使最佳化過程中不必反覆重算，是現代視覺慣性 SLAM 得以即時運行的關鍵技術。

33. IMU 在視覺慣性 SLAM 中提供四大貢獻：絕對尺度、重力方向、視覺失效時的空隙填補、高頻輸出；視覺則回報以不漂移的觀測來校正 IMU 零偏。緊耦合精度優於鬆耦合。

34. ORB-SLAM3 全程以最大後驗估測為基礎完成緊耦合視覺慣性 SLAM，連 IMU 初始化亦然，精度較先前方法提升約二至十倍；初始化需要足夠的運動激勵，否則尺度與零偏不可觀測。

下一章，我們轉向另一種截然不同的感測器——光達。它不像 IMU 那樣猜測自己動了多少，而是直接以雷射量出「那面牆在三公尺外」。從飛行時間原理、2D 與 3D 光達的差異，到它在掃地機器人、倉儲 AGV 與自駕車中的角色，第 8 章將為第四篇的 2D SLAM 實作揭開序幕。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

## 第 8 章

## 光達 LiDAR 介紹

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

## 本章在全書的位置

第 6 章與第 7 章的磁力計與 IMU 是本體感測器——它們感知機器人自身的狀態，卻看不見世界。從本章開始，第二篇轉向外感測器：直接量測環境幾何與外觀的感測器。

光達是其中最直接的一種。它不推算、不積分、不猜測，而是以雷射直接量出「那個方向有東西，距離三公尺」。這個「直接量測幾何」的性格，使它成為整本書第四篇與第五篇的主角——從第 13 章的佔據格地圖、第 14 章的 Gmapping、第 15 章的 Cartographer，到第 17、18 章的 3D 點雲與 LOAM，全都建立在本章所介紹的資料之上。

因此本章的角色是「規格與直覺的奠基」。同學讀完之後，再看到一份光達的規格表，應該能立刻判斷：這顆光達能不能看見椅腳、建出來的地圖會有多細、在玻璃帷幕的大廳裡會不會出事。這些判斷在第 29 章親手建圖時會天天用到。

**圖 8-0　第 8 章與全書的連結地圖**

【承接前章】

第 6、7 章　本體感測器：快、連續，但漂移

本　　章　外感測器：慢、離散，但不漂移　→　兩者合作才能消除累積誤差

【本章負責】

飛行時間測距　→　掃描　→　LaserScan / PointCloud 資料格式

規格的物理意義：量測距離、角解析度、掃描頻率、反射率

【往下延伸】

→ 第 13 章　LaserScan + 位姿　→　佔據格地圖（本章資料的第一個用途）

→ 第 14、15 章　Gmapping 與 Cartographer：2D 光達 SLAM 的兩大主流

→ 第 16 章　掃地機器人：本章案例 8-1 高度盲區的完整討論

→ 第 17、18 章　3D 點雲配準與光達里程計

→ 第 29 章　實體小車建圖：本章的規格判斷在此變成實際操作

## 學習目標

前幾章的感測器都在「猜測」——里程計猜位移、IMU 猜運動、Wi-Fi 猜距離。本章介紹的光達則不同：它以雷射光直接「量」出「那個方向、那個距離有東西」。這種精確的幾何量測，使光達成為 2D 與 3D SLAM 最重要的感測器之一。學完本章之後，同學應能夠達成下列目標：

43. 說明飛行時間（ToF）測距原理，並解釋為何光達能提供精確的距離量測。
44. 區分 2D 光達、3D 多線光達的結構差異，以及機械式與固態式掃描機構的優缺點。
45. 說明 LaserScan 與 PointCloud 兩種資料格式的內容與差異。
46. 解釋角解析度、量測距離、精度與反射率如何影響光達的實際表現。

47. 舉例說明光達在室內建圖、倉儲 AGV 與自駕車中的角色。

48. 比較光達與相機、IMU 的優缺點，說明為何實務系統常將三者融合。

49. 認識適合教學的低成本光達平台，為第四篇的 2D SLAM 實作做好準備。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 光達 | LiDAR | 以雷射光量測距離的感測器，全名為光偵測與測距。 |
| 飛行時間 | Time of Flight, ToF | 由光發射到反射返回的時間乘光速，計算距離。 |
| 脈衝式測距 | Pulsed / dToF | 發射短脈衝並直接量測往返時間，適合長距離。 |
| 調頻連續波 | FMCW | 連續調變雷射頻率，可同時量測距離與速度。 |
| 雷射掃描 | LaserScan | 2D 光達輸出的一圈距離值資料格式。 |
| 點雲 | Point Cloud | 3D 光達輸出的三維點集合。 |
| 線數 | Channels / Lines | 3D 光達垂直方向的雷射束數量，如 16、32、64、128 線。 |
| 視野角 | Field of View, FoV | 光達可掃描的角度範圍，分水平與垂直。 |
| 角解析度 | Angular Resolution | 相鄰兩個量測點之間的角度間隔，越小越密。 |
| 反射率 | Reflectivity | 物體表面反射雷射的比例，影響可偵測距離與訊號強度。 |
| 固態光達 | Solid-State LiDAR | 無大型旋轉機構的光達，含 MEMS、Flash、OPA 等架構。 |
| 多普勒效應 | Doppler Effect | FMCW 光達藉以直接量測目標徑向速度的物理現象。 |

## 8.1　光達的基本原理：Time of Flight 與掃描

光達（LiDAR）的全名是 Light Detection and Ranging——光偵測與測距。它的核心概念與第 5 章的 GNSS、第 4 章的 UWB 一脈相承：發射訊號、量測往返時間、乘上訊號速度得到距離。差別只在於光達用的是雷射光，而且距離短得多、精度高得多。

### 一、脈衝式飛行時間（dToF）

最直觀的測距法是直接飛行時間（direct ToF，dToF）：發射一個極短的雷射脈衝，計時它從發射到反射返回所經過的時間 t，距離即為：

$$d = (c \times t) / 2$$

其中 c 為光速，除以 2 是因為光走了「去程加回程」的來回距離。這個公式簡單，但對計時精度的要求極高——光速下 1 奈秒等於 30 公分，要達到公分級精度，計時必須精準到約 0.07 奈秒。這正是光達晶片的技術門檻所在，也是它比超音波、紅外線測距昂貴的原因。dToF 的優點是能量集中在短脈衝中、可測遠距離，是自駕車長距離光達的主流。

### 二、調頻連續波（FMCW）

另一種日益重要的方法是調頻連續波（FMCW，Frequency-Modulated Continuous Wave）：不發射脈衝，而是持續發射一道頻率隨時間線性變化（Chirp）的雷射，再把返回訊號與當下發射訊號混合，量測兩者的頻率差（拍頻）。頻率差同時編碼了距離與速度——這是 FMCW 最迷人的特性。

FMCW 的最大優勢是能利用多普勒效應直接量出每一點的徑向速度，不需要像脈衝式那樣比較連續多次量測才能推算速度；而且它採用同調偵測（Coherent Detection），能幾乎完全排除環境光與其他光達的干擾。代價是需要窄線寬雷射光源，製造難度高、成本較貴，目前主要用於高階車用光達與研究領域。近年矽光子技術的進展，已能在微米級平台上實現約 120 公尺偵測距離與 0.3 公尺精度的 FMCW 系統，被視為下一世代的重要方向。

### 三、為什麼需要掃描

單束雷射一次只能量測一個方向的距離——那只是一個「點」。要建立環境的輪廓或立體形狀，就必須讓雷射束掃過不同方向，這就是掃描（Scanning）機構的任務。掃描方式決定了光達的形態與價格，也是 8.2 節的主題。

**觀念釐清：光達、雷達、超音波的差異**

三者原理相同（發射訊號、量往返時間），差別在使用的波：

- 光達（LiDAR）：雷射光，波長短、方向性極佳，可達公分至公釐級精度與高解析度；弱點是玻璃、鏡面、雨霧。
- 雷達（Radar）：無線電波，穿透力強、不受雨霧影響、可測速；弱點是解析度低、難分辨物體形狀。
- 超音波（Ultrasonic）：聲波，極便宜、近距離可靠；弱點是範圍小、精度低、易受風與吸音材料影響。

自駕車常三者並用，正是取其互補——這與本書反覆強調的感測器融合思想完全一致。

## 8.2 2D 光達、3D 光達、多線雷射

依照掃描維度，光達可分為 2D 與 3D 兩大類；依照掃描機構，又可分為機械式與固態式。這兩個分類軸決定了光達的能力、形態與價格。

### 一、2D 光達：掃描一個平面

2D 光達讓單束雷射在一個水平面上旋轉一圈（常見 360 度或 270 度視野），量出這個平面上四面八方的距離，輸出一圈距離值。它就像一把在固定高度水平劃過空間的「雷射尺」。

2D 光達的優點是資料量小、運算輕、價格親民（教學型號約數千至上萬元台幣），是

掃地機器人、教學小車與室內 AGV 的標準配備，也是本書第四篇 Gmapping、Cartographer 實作的主角。它的先天限制是「只看得到一個高度切面」——放在 20 公分高處掃描，就看不到 30 公分高的桌面，也偵測不到懸空的橫桿或低矮的門檻。這個限制在 8.5 節與第 16 章掃地機器人的討論中會反覆出現。

## 二、3D 光達：多線與掃描

要獲得立體資訊，就需要在垂直方向也能量測。3D 光達主要靠兩種方式達成：

- 多線（Multi-beam）：在垂直方向堆疊多組雷射發射與接收器，每一組負責一個仰角，同時旋轉掃描。常見規格有 16 線、32 線、64 線、128 線——線數越高，垂直方向越密、立體重建越精細，價格也越貴。這是自駕車與自主巡檢機器人的主流。
- 單束垂直掃描：以震盪鏡面或旋轉稜鏡讓單束（或少數幾束）雷射在垂直方向也擺動，用較少的雷射組件覆蓋垂直視野，藉此降低成本。

## 三、機械式 vs 固態式

傳統光達靠馬達帶動整個光學組件或鏡面旋轉，稱為機械式。它視野大、技術成熟，但有明顯缺點：體積大、有旋轉部件易磨損、在震動環境中壽命受限、成本高。

固態光達（Solid-State LiDAR）則試圖減少或消除大型旋轉機構，主要有三種架構：

- MEMS：以晶片上的微振鏡偏轉雷射束逐線掃描。嚴格說是「半固態」，因為仍有微小的振鏡運動；目前是出貨量最大的固態方案。
- Flash：像相機閃光燈一樣，一次照亮整個場景，由 SPAD / APD 感測器陣列接收全部回波，完全無移動部件；優點是無掃描延遲，限制是照明能量分散導致距離較短。
- OPA（光學相位陣列）：透過調整多個波導發射器的相位，以純電子方式在微秒內偏轉光束、無任何機械運動；技術最先進但量產成熟度仍在推進中。

固態光達的整體趨勢是體積更小、更耐震、成本更低，正逐步從高階車用走向機器人與消費領域。對本書讀者而言，理解「機械式為主的今天」與「固態化的明天」這個演進方向，

有助於判讀市場上琳瑯滿目的產品。

## 8.3 光達資料格式：LaserScan 與 PointCloud

感測器量到的資料，最終要以特定格式交給 SLAM 演算法處理。光達在 ROS 生態中有兩種標準格式，分別對應 2D 與 3D。

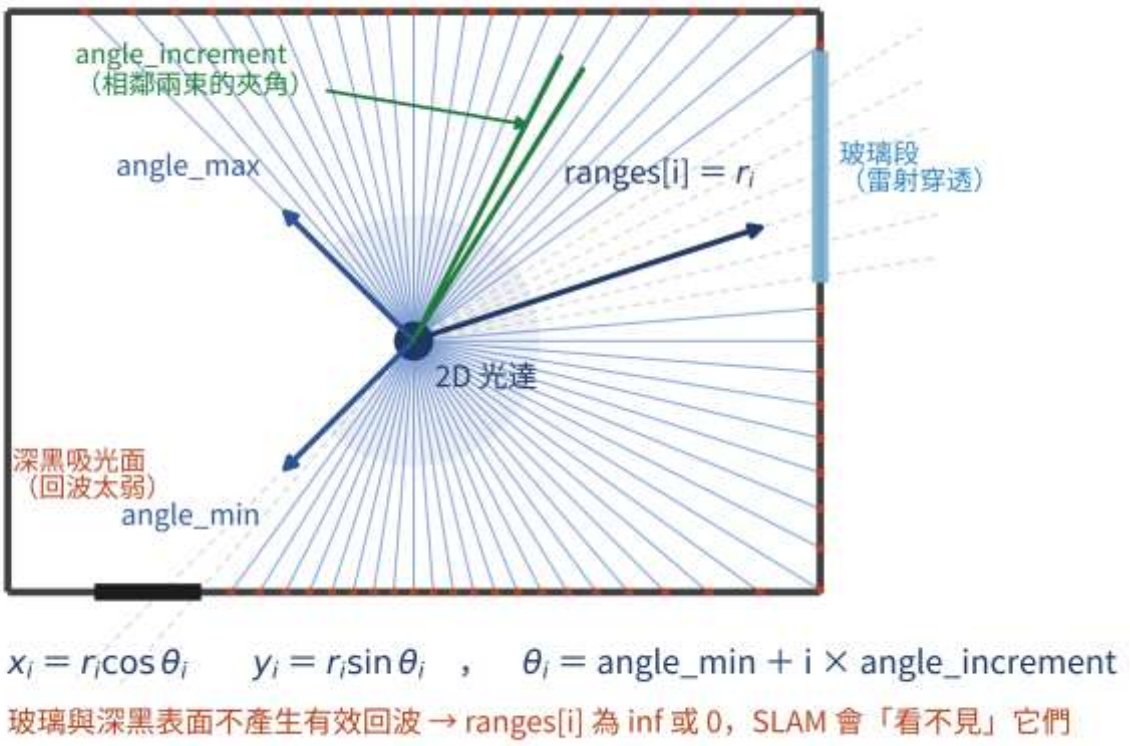
### 一、LaserScan：2D 光達的資料格式

2D 光達輸出的是一圈距離值，ROS 以 sensor_msgs/LaserScan 訊息表示。它的核心是一個距離陣列，配合幾個描述掃描範圍的參數：

- angle_min、angle_max：掃描的起始與結束角度。
- angle_increment：相鄰兩個量測點的角度間隔（即角解析度）。
- range_min、range_max：可信的最近與最遠量測距離。
- ranges[]：距離值陣列——這是資料主體，每個元素就是對應角度上量到的距離。
- intensities[]：反射強度陣列（選用）。

由於角度是規律遞增的，LaserScan 不需要為每個點單獨儲存角度，只要距離陣列加上起始角與角度間隔，就能還原每個點的方向。這種緊湊表示讓 2D 資料非常輕量——一圈掃描通常只有數百到上千個距離值。要在座標系中畫出某個點，只需簡單的極座標轉直角座標（8.5 節公式）。

**圖 8-1　2D 光達的掃描幾何與 LaserScan 資料結構**




這張圖把 LaserScan 訊息的每個欄位都放回它的幾何意義裡。光達位於原點，以 angle_min 為起點、每隔 angle_increment 發射一束雷射，直到 angle_max；每一束量回一個距離，依序存進 ranges 陣列。所以 ranges[i] 這個數字本身不帶方向資訊——方向必須由索引 i 反推：$\theta_i$ = angle_min + i × angle_increment。

這是初學者最常犯的錯誤之一：直接把 ranges 陣列畫出來，得到的是一條扭曲的曲線而非房間的形狀。必須經過圖中下方那兩條轉換式，把極座標的（r, θ）換成直角座標的（x, y），點雲才會「長得像房間」。同學在實作 8-1 會親手做這件事，而第 13 章把這些點標進格子的動作，正是建圖的第一步。

圖中還刻意畫出兩種失效情況。右牆的玻璃段讓雷射直接穿透，沒有回波；左下牆面的深黑吸光材質把光吸收殆盡，回波弱到無法判讀。兩者的結果相同——該方向的 ranges 值變成無限大或零，在 SLAM 眼中那裡「什麼都沒有」。於是地圖上出現一個本不該存在的開口，機器人可能一頭撞上玻璃門。

**玻璃問題的三種實務對策**

一、加裝超音波或紅外線近接感測器專門偵測玻璃——這是掃地機器人與服務機器人的標準做法。

二、在玻璃上貼可見的標示帶或 AprilTag（第 4.6 節），同時解決人與機器人的辨識問題。

三、在地圖上手動標註虛擬牆（Nav2 的 Keepout Zone，第 29 章會實作），讓路徑規劃直接避開。

記住一個原則：感測器看不見的東西，演算法再強也救不回來——這是第 32 章討論 AI 輔助導航時仍然成立的鐵律。

## 二、PointCloud：3D 光達的資料格式

3D 光達的資料是三維空間中的大量點，ROS 以 sensor_msgs/PointCloud2 訊息表示。與 LaserScan 不同，每個點都直接儲存自己的三維座標（x、y、z），可再附加反射強度、線號、時間戳記等欄位。

點雲的資料量遠大於 LaserScan：一台 64 線光達每秒可產生上百萬個點。這帶來兩個工程課題——龐大的傳輸與儲存需求，以及後續處理前通常需要的降採樣（第 17 章的 Voxel Grid）。點雲的處理有專門的開源函式庫 PCL（Point Cloud Library），第五篇會深入使用。

| 特性 | LaserScan（2D） | PointCloud2（3D） |
|---|---|---|
| 資料維度 | 一圈距離值（平面） | 三維點集合（立體） |
| 每筆資料量 | 數百至上千點 | 數萬至數百萬點 |
| 點的表示 | 距離 + 規律角度 | 直接存 x、y、z |
| 運算負擔 | 輕 | 重，常需降採樣 |

| 特性 | LaserScan（2D） | PointCloud2（3D） |
|---|---|---|
| 典型來源 | 2D 旋轉光達 | 多線或固態 3D 光達 |
| 本書應用 | 第 13 至 16 章 2D SLAM | 第 17 至 18 章 3D SLAM |

**表 8-1　LaserScan 與 PointCloud2 資料格式比較**

## 8.4　光達解析度、角解析度、量測距離與反射率

選用或評估一台光達，必須看懂它的規格表。本節說明四個最關鍵的參數，它們直接決定 SLAM 建圖的品質。

### 一、量測距離與精度

- 量測範圍：最近與最遠可量的距離。教學型 2D 光達常見 0.15 至 12 公尺，自駕車長距離光達可達 200 公尺以上。
- 精度（Accuracy）：量測值與真實距離的接近程度，常見規格為 ±2 至 ±3 公分。
- 精密度（Precision）：對同一目標重複量測的一致性。精度與精密度是兩回事——一把總是量多 5 公分的尺不精確但可能很精密，且系統性偏差可校正。

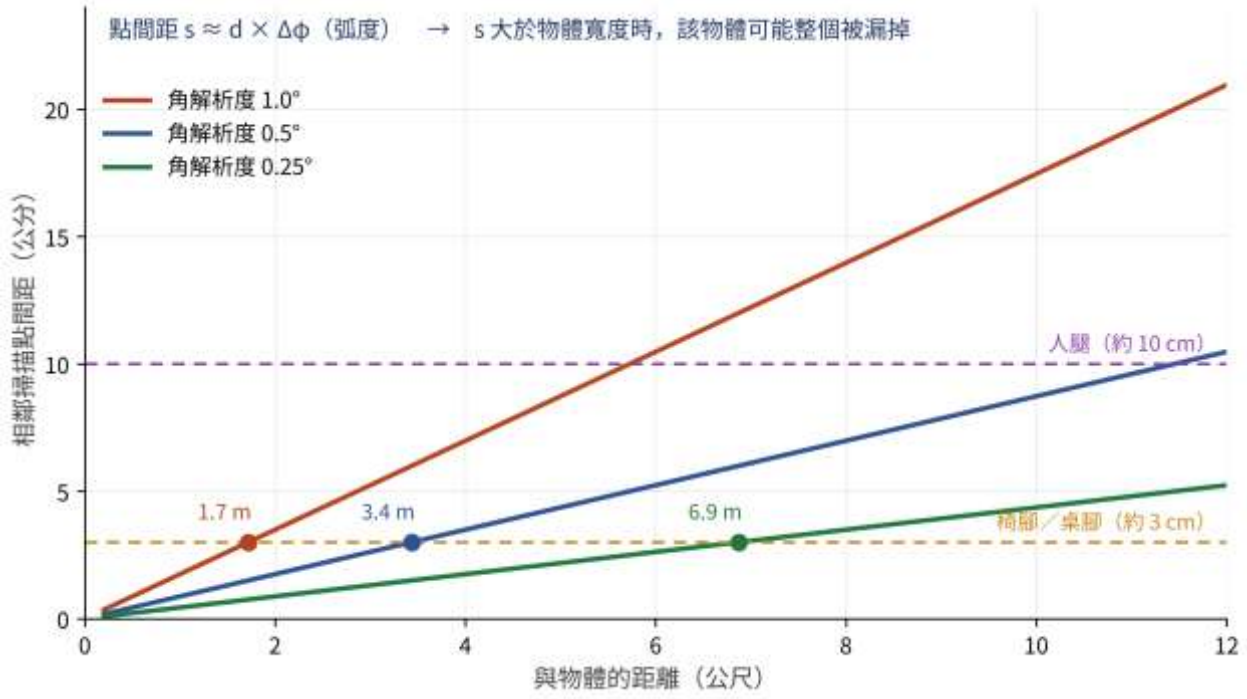
### 二、角解析度：地圖的細膩程度

角解析度是相鄰兩個量測點之間的角度間隔，它決定了地圖能呈現多細的結構。這裡有一個關鍵的幾何事實：同樣的角解析度，距離越遠，相鄰兩點的實際間距越大。

$$\text{相鄰點間距} \approx \text{距離} \times \text{角解析度（弧度）}$$

舉例：角解析度 1 度、距離 5 公尺時，相鄰兩點間距約 8.7 公分——這表示 5 公尺外一根 5 公分寬的椅腳，很可能整個落在兩束雷射之間而完全偵測不到。這個現象解釋了為什麼遠處的細小物體容易被光達漏掉，也是 8.5 節與第 16 章討論「掃地機器人為何撞到桌腳」的關鍵。

**圖 8-2　角解析度、距離與可偵測物體尺寸的關係**




角解析度是規格表上最容易被略過、卻最決定地圖品質的數字。它的效應很單純：相鄰兩個掃描點的間距隨距離線性增加，$s \approx d \times \Delta\varphi$。但把具體數字代進去之後，結論相當驚人。

以一支 3 公分寬的椅腳為例。角解析度 1 度的入門光達，只有在 1.7 公尺內才保證至少掃到一個點；0.5 度可延伸到 3.4 公尺；0.25 度則到 6.9 公尺。換句話說，同一間教室、同一張椅子，換一顆光達就可能從「地圖上清楚的四個點」變成「完全不存在」。

這解釋了幾件實務現象：為什麼便宜光達建出來的地圖「牆很直但家具很破碎」；為什麼掃地機器人常撞上椅腳（案例 8-1 的高度盲區只是原因之一，角解析度是另一個）；以及為什麼第 13 章的佔據格地圖需要多次觀測累積——單次掃描漏掉的細物體，移動到近處再掃一次就補得回來。

同學在選購或判讀規格時，可以把這張圖當成速查表：先想清楚「我最小要偵測多大的東西、在多遠處」，再回推需要的角解析度。這個由需求反推規格的思路，和第 4.7 節的室內定位選型流程是同一種工程素養。

## 三、掃描頻率

掃描頻率（Scan Rate，如 10 Hz 表示每秒轉 10 圈）影響機器人移動時的建圖品質。頻率太低而機器人移動快，同一圈掃描的起點與終點之間機器人已經移動了一段距離，造成掃描變形（Motion Distortion）——這在第 18 章的 3D 光達 SLAM 中是必須專門處理的問題。

### 四、反射率：被忽略卻關鍵的變數

光達依賴目標把雷射反射回來，因此目標的反射率至關重要。白牆反射率高、容易偵測；黑色物體吸收大部分雷射，反射微弱，可偵測距離大幅縮短——一台標稱 12 公尺的光達，面對黑色沙發可能只剩幾公尺。更棘手的是：

- 鏡面與玻璃：雷射被鏡面反射到別處而非返回，光達會「看穿」玻璃或量到鏡中的虛像，在地圖上造成錯誤——這是室內光達 SLAM 最常見的失敗來源之一。
- 高反光物體：金屬、反光標誌可能造成過強回波或多重反射。
- 液體表面：水面、光滑地面可能產生鏡面反射造成漏測。

因此光達規格表上的「最大距離」通常標註在特定反射率（如 10% 或 90%）條件下——同學看規格時務必注意這個前提，否則實際表現會與預期落差很大。

## 8.5 光達在室內建圖、倉儲 AGV、自駕車中的應用

### 一、室內建圖與服務機器人

2D 光達是室內建圖的主力。掃地機器人、送餐機器人、導覽機器人多以一顆水平 2D 光達搭配 Gmapping 或 Cartographer 建立佔據格地圖（第四篇）。室內環境結構分明（牆、門、走廊），光達的精確幾何量測能建出清晰穩定的地圖，這是視覺方法在白牆、昏暗環境下難以匹敵的。但前述的玻璃隔間、鏡面與單一高度切面問題，也讓純 2D 光達在現代玻璃帷幕建築中屢屢受挫，實務上常需搭配超音波或紅外線補足。

### 二、倉儲 AGV 與工廠自動化

倉儲自動導引車（AGV）是光達最成熟的工業應用之一。除了用於 SLAM 導航，光達

還兼任安全防護——許多工業光達具備安全認證，能在人員進入危險區域時即時觸發減速或停車。工廠環境相對受控（地面平整、光線穩定、佈局已知），使光達得以發揮最大可靠度。這也呼應第 4 章亞馬遜案例的觀念：環境越可控，技術選型越能簡化。

### 三、自駕車

3D 光達是多數自駕車的核心感測器（特斯拉的純視覺路線是著名的例外）。它提供精確的立體環境結構，用於障礙物偵測、可行駛區域判定，以及與高精地圖比對來定位（第 5 章都市峽谷案例）。自駕車對光達的要求最為嚴苛：長距離（看清高速前方）、高解析度（分辨行人與電線桿）、高更新率、以及嚴格的車規可靠度。這也是車用光達單價仍高（每顆約數百美元）、且積極往固態化與 FMCW 演進的原因。

### 四、其他新興應用

- 測繪與空間掃描：背包式或手持光達掃描建築、古蹟、工地，產生高精度 3D 模型（第 15、17 章）。
- 農林業：無人機載光達穿透樹冠量測林木高度與地形。
- 機器狗與巡檢：3D 光達配合 IMU 緊耦合，在無 GNSS 的室內外環境自主巡檢（第 18 章）。

## 8.6 光達與相機、IMU 的比較

把光達放進第二篇的感測器全景中比較，能更清楚看出它的定位。第 1 章的表 1-1 已做過總覽，此處針對建圖與定位任務做更深入的對照。

| 比較項目 | 光達 | 相機 | IMU |
|---|---|---|---|
| 直接量測 | 距離（幾何） | 亮度 / 顏色 | 加速度 / 角速度 |
| 深度資訊 | 直接、精確 | 需推算（單目無尺度） | 無 |
| 受光線影響 | 幾乎不受（主動光 | 嚴重（依賴環境光） | 不受 |

| 比較項目 | 光達 | 相機 | IMU |
|---|---|---|---|
| | 源） | | |
| 語意資訊 | 無（只有形狀） | 豐富（可辨識物體） | 無 |
| 資料量 | 中至大 | 大 | 小 |
| 成本 | 中至高 | 低 | 低至中 |
| 主要弱點 | 玻璃鏡面、雨霧、無語意 | 光線、無尺度、無深度 | 漂移 |

**表 8-2　光達、相機、IMU 在建圖定位任務上的比較**

### 三者的互補邏輯

這張表清楚顯示三者為何天生互補：

- 光達給精確的幾何與深度，但看不懂「那是什麼」，也怕玻璃鏡面。
- 相機給豐富的語意與顏色，但沒有可靠的深度與尺度，還怕黑暗逆光。
- IMU 給高頻的運動資訊填補空隙，但會漂移。

於是最強健的系統往往三者並用：光達 SLAM 建立精確幾何骨架、相機提供語意標註與閉環辨識、IMU 在光達掃描變形與快速運動時提供補償。第 18 章的光達慣性緊耦合（LIO）就是光達加 IMU 的代表，第 32 章的語意 SLAM 則是光達（或相機）加 AI 的代表。

### 光達 vs 視覺：一場沒有標準答案的辯論

在自駕車領域，「該不該用光達」曾是激烈爭論的話題。主張純視覺者認為：人類只用眼睛就能開車，且相機便宜、含語意、可辨識號誌；主張用光達者認為：光達提供的可靠深度是安全冗餘的關鍵，不該為了省成本而犧牲。這場辯論至今沒有定論，但它給同學一個重要啟示：感測器選型從來不是純技術問題，還牽涉成本、安全、法規與商業策略。作為工程師，理解每種感測器的能力邊界，才能在具體情境中做出有依據的判斷——這正是第二篇七章的共同目的。

## 8.7 低成本光達與教學型機器人平台

好消息是：光達已經便宜到能進入教室。本節介紹適合教學的光達與平台，為第四篇的實作預做準備。

### 一、入門級 2D 光達

近年低成本 2D 光達的普及，是 SLAM 教育得以下放到大學部甚至高中的關鍵。這類光達多採三角測距或低成本 ToF 原理，360 度掃描，量測範圍約 8 至 12 公尺，價格從數千元台幣起跳，並提供完整的 ROS 驅動——插上就能發布 LaserScan，直接餵給 Gmapping 或 Cartographer。它們的精度與穩定度雖不及工業型號，但對教學建圖已綽綽有餘。

### 二、教學型機器人平台

- TurtleBot 系列：ROS 官方的標準教學平台，搭載 2D 光達與完整軟體堆疊，是全球 SLAM 教學使用最廣的平台，網路教學資源極其豐富。本書第 29 章的實作以它為主要參考。
- 自組 ROS 小車：以差速輪底盤、馬達編碼器、微控制器搭配低成本 2D 光達與單板電腦（如 Raspberry Pi 或 Jetson Nano）自行組裝。成本較低、學習深度更高，適合大學部專題——同學能親手處理第 10 章的 TF 座標、第 29 章的里程計發布等每一個環節。
- 模擬平台：完全不需硬體。在 Gazebo / Foxglove 等模擬環境中，可放置虛擬光達小車進行完整 SLAM 實驗（第 28 章）。這讓高中課程與大班教學也能人人動手，是本書強烈推薦的入門途徑。

### 三、進階：3D 光達教學

低線數（如 16 線）的 3D 光達近年價格也大幅下降，配合 Jetson 等邊緣運算平台，已可用於大學部的 3D SLAM 專題（第 17、18 章）與大學的自主巡檢研究。對高中與大學部入門而言，仍建議從 2D 光達或模擬起步，把 SLAM 的核心觀念建立穩固後，再進入 3D 的世界。

無論選擇哪一種平台，本書的建議都一致：先在模擬中理解流程、再用便宜硬體驗證、最後才追求精度與規模。第四篇的每一個實作，都會標明模擬版與硬體版兩條路徑，讓不同資源條件的同學都能參與。下一章我們暫別「主動量測距離」的光達，轉向資訊最豐富、卻也最需要數學處理的感測器——相機，為第六篇的視覺 SLAM 鋪路。

## 公式或流程

### 一、飛行時間測距

脈衝式光達的基本測距公式：

$$d = (c \times t) / 2$$

其中 $c \approx 3\times10^8$ m/s 為光速、t 為脈衝往返時間、除以 2 因光走了來回雙程。由此可推得計時精度需求：

$$\text{距離誤差 } \Delta d = (c \times \Delta t) / 2$$

要達到 1 公分（0.01 m）精度，需要 $\Delta t \approx 0.067$ 奈秒——這說明光達計時電路的技術挑戰，也解釋了為何精確光達價格不菲。

### 二、LaserScan 點的座標轉換

2D 光達的第 i 個量測點以極座標（距離 $r_i$、角度 $\theta_i$）表示，其中角度可由掃描參數算出：

$$\theta_i = angle_min + i \times angle_increment$$

轉換到光達座標系的直角座標（用於建圖與顯示）：

$$x_i = r_i \times \cos\theta_i$$

$$y_i = r_i \times \sin\theta_i$$

這個簡單的極座標轉直角座標，是所有 2D SLAM 把「一圈距離值」變成「地圖上一圈點」的第一步。第 13 章的掃描匹配、第 14 章的 Gmapping，都建立在這個轉換之上。

## 三、角解析度與可偵測物體尺寸

相鄰兩束雷射在距離 d 處的間距，決定了能偵測的最小物體尺寸：

$$\text{點間距 } s \approx d \times \Delta\varphi \quad (\Delta\varphi \text{ 為角解析度，以弧度表示})$$

一個寬度小於 s 的物體，可能整個落在兩束雷射之間而被漏測。例如 $\Delta\varphi = 1°$（$\approx$ 0.0175 弧度）、$d = 5$ m 時，$s \approx 8.7$ cm。這條式子是理解光達「看得多細」的關鍵，也量化了「遠處小物體易漏測」的現象。

## 四、光達資料處理流程

**圖 8-3　光達從發射到 SLAM 輸入的資料流**

① 雷射發射脈衝 / 調頻波

↓

② 目標反射，感測器接收回波

↓

③ 計算飛行時間 → 距離 d（dToF）或拍頻 → 距離 + 速度（FMCW）

↓

④ 掃描機構提供角度資訊 → 每個點的方向

↓

⑤ 組成資料：2D → LaserScan（距離 + 角度） / 3D → PointCloud（x,y,z）

↓

⑥ 極座標轉直角座標 → 交給 SLAM 前端（掃描匹配、特徵提取）

# 8.8　光達的當代變局：價格崩落與固態化

本章前七節描述的光達技術架構相當穩定，但這個產業本身正在經歷劇烈的變化。對同學而言，最該知道的不是又有什麼新原理，而是一個單純的事實：光達正在從昂貴的研究設備，變成便宜的標準零件。

## 一、十年之間，價格掉了三個數量級

把幾個時間點排在一起，趨勢非常清楚。2016 至 2017 年間，自駕車研究常用的六十四線機械式光達，單價約八萬美元；2020 年，固態光達的代表產品約一千九百美元；2022 年，某款主流產品降至約五百美元；而到了 2026 年，同系列產品已進入量產並跌破兩百美元，中國市場的平均價格甚至約一百四十五美元。另有廠商以兩百美元以下為目標、並規劃朝一百美元推進。

驅動這個變化的是製造規模、關鍵元件自製化與產能擴張——其軌跡與過去太陽能模組、智慧型手機相機模組的成本曲線如出一轍。某主要廠商已針對超過四百萬台的訂單開始量產交付，並規劃將年產能提升至四百萬台。

對教學與研究的意義非常直接：本章 8.7 節所說「低成本光達」的門檻正在持續下降。五年前只有研究室買得起的 3D 光達，現在已經進入大學專題甚至高中社團的預算範圍。第 17、18 章的 3D 光達 SLAM，從「讀論文理解」變成「可以真的做做看」。

## 二、出貨結構：機器人正在追上車用

數字也顯示出應用面的變化。以一家主要廠商 2025 年的出貨為例，全年總出貨約一百六十二萬台，年增超過兩倍；其中先進駕駛輔助系統用途約一百三十八萬台，而機器人用途約二十三萬九千台——後者的年增幅達四倍以上，遠高於車用。該公司並預估 2026 年總出貨量將達三百萬至三百五十萬台。

機器人用光達的成長率高於車用，背後是掃地機器人、配送機器人、倉儲 AGV 與巡檢機器人的普及。這正是本書第四、五、八篇所針對的那些應用——同學所學的，正好是需求成長最快的那一塊。

## 三、固態化：從「會轉的」到「不會轉的」

8.2 節比較過機械式與固態式。這個轉換在產業上已基本定案：價格的崩落幾乎完全發生在固態架構上，因為沒有旋轉機構意味著更少的精密零件、更長的壽命、更小的體積，以及可以用半導體製程大量生產。

對系統設計者而言，這個轉變帶來一個觀念上的調整。機械式光達提供完整的三百六十度視野，固態光達則通常只覆蓋一個前向扇形。這意味著單顆感測器不再能看到全部——解法是裝多顆。於是「多感測器的外參校正」（各感測器之間的相對位姿）從一個偶爾的需求，變成每台機器人都必須處理的課題。第 17 章的點雲配準與第 10 章的座標轉換，正是處理這件事的工具。

## 四、調頻連續波與「4D 光達」

8.1 節提到的調頻連續波（FMCW）架構，近年被視為下一個世代的方向。它的關鍵優勢是在量距離的同時直接量出徑向速度——每個點除了 x、y、z 還多一個速度分量，因此常被稱為 4D 光達。

這對 SLAM 的意義值得同學思考。第 17 章之後會反覆遇到一個難題：動態物體（走動的人、行駛的車）會污染配準結果，使機器人以為自己移動了。若感測器本身就能告訴你「這些點正在移動」，動態物體的剔除就從困難的演算法問題變成單純的門檻判斷。此外 FMCW 對陽光與其他光達的干擾具有天然的抗性——這在光達即將裝滿街頭的時代是個實際的問題。

> **給同學的三個判斷**
>
> 一、不要以舊價格思考系統設計。「光達太貴所以用相機」這個論證，在 2026 年的成立範圍已經比五年前小得多。
>
> 二、但便宜不等於適合。價格崩落主要發生在車用規格的前向固態光達；室內機器人要的三百六十度小型光達，仍是另一個市場。

三、規格的物理意義不會因降價而改變。圖 8-2 的角解析度限制、玻璃與黑色表面的失效，在兩百美元的光達上與八萬美元的一樣成立。

## 案例

### 案例 8-1　掃地機器人的「隱形陷阱」：2D 光達的高度盲區

許多同學家裡的掃地機器人都上演過同樣的戲碼：它能靈巧地繞開沙發與牆壁，卻總是一頭撞上餐桌的椅腳，或卡在踢腳線邊緣。這不是機器人「笨」，而是 2D 光達的先天限制在現形。

2D 光達裝在機器人頂部某個固定高度（例如 8 公分處）水平掃描，它只「看得見」這個高度切面上的障礙。細瘦的椅腳在這個高度可能只佔一兩束雷射，若又距離較遠（8.4 節的角解析度問題），就可能整個落在雷射束的間隙中被漏掉。更低矮的障礙（電線、門檻、寵物碗）根本不在掃描平面上，光達完全無感。

這個案例生動說明了為何現代掃地機器人紛紛加裝其他感測器——前方的紅外線或超音波障礙偵測、底部的懸崖感測器、甚至改用 3D ToF 相機（第 16 章）。它也預告了一個重要觀念：SLAM 建的地圖只包含感測器「看得到」的世界，感測器的物理限制，直接決定了地圖的完整性與導航的安全性。

### 案例 8-2　自駕車的光達之爭：Waymo 與 Tesla 的路線分歧

自駕車產業存在一場著名的技術路線分歧。以 Waymo 為代表的主流陣營，車頂與車身佈滿多顆 3D 光達，主張光達提供的可靠深度是安全的基石；以 Tesla 為代表的另一陣營，則走純視覺路線，主張用足夠多的相機加上強大的 AI，就能像人類一樣只靠「看」來開車，且成本大幅降低、可大規模量產。

雙方各有理據：光達派強調惡劣光線與安全冗餘，視覺派強調成本、語意理解與可擴展

性。隨著車用光達持續降價與固態化、以及視覺 AI 持續進步，這場辯論仍在演變。對本書讀者，重點不在於「誰對誰錯」，而在於看懂背後的權衡：這正是第 8.6 節「感測器選型是技術、成本、安全、商業的綜合判斷」的最佳教材。

### 案例 8-3　背包式 3D 掃描：讓建築「數位分身」

把一台多線 3D 光達、IMU 與運算單元裝進背包，操作員背著它走過建築物，就能即時建立整棟樓的三維點雲模型——這就是背包式移動掃描（Backpack Mobile Mapping），Google 的 Cartographer 系統最初就展示過這類應用（第 15 章）。

走過走廊、樓梯、房間的過程中，系統靠 SLAM 即時拼接每一時刻的光達掃描，最終產出可用於建築資訊模型（BIM）、室內導航、古蹟保存的高精度數位分身。這個案例把本書的多個主題串在一起：ToF 測距（本章）、IMU 補償掃描變形（第 7 章）、3D 點雲處理（第 17 章）、閉環修正（第 15 章）。它也展示了 SLAM 的價值不僅在於「讓機器人動起來」,更在於「讓人類以前所未有的效率把真實空間數位化」——這是大學場域專題極具潛力的方向。

## 實作活動

### 實作 8-1　光達資料視覺化與規格解讀

目的：透過真實或模擬的光達資料，理解 LaserScan 的結構、親手完成極座標轉直角座標把「一圈距離值」畫成「環境輪廓」，並學會解讀光達規格表。本實作提供模擬與硬體兩條路徑，適合不同資源的班級。

#### 路徑 A：模擬（無需硬體，適合全班）

40. 在電腦上安裝 ROS 2 與 TurtleBot3 模擬環境（詳細步驟見第 28 章，或依官方教學），啟動 Gazebo 世界與虛擬小車。
41. 用 RViz 顯示 /scan 話題，觀察光達掃描的紅點如何隨小車移動而貼合牆面。

42. 用指令查看 LaserScan 訊息內容，找出 angle_min、angle_increment、range_max 等參數，記錄下來並計算這台虛擬光達的視野角與總點數。

43. 錄製一段 rosbag，匯出一圈 ranges 陣列。

**路徑 B：硬體（有低成本 2D 光達的班級）**

44. 將 2D 光達接上電腦，啟動其 ROS 驅動，確認 /scan 話題正常發布。

45. 把光達放在教室中央靜置，用 RViz 觀察掃描輪廓是否貼合四面牆與桌椅。

46. 分別把一塊白紙板、黑色書包、玻璃板（或鏡子）放在光達前方 2 公尺處，比較三者在 RViz 上的顯示差異，記錄玻璃與鏡面造成的異常。

47. 錄製一圈掃描資料匯出，供下一步分析。

**共同部分：資料分析（Python）**

48. 讀入匯出的 ranges 陣列，依 8.5 節公式把每個（距離、角度）轉為（x、y）直角座標。

49. 用 matplotlib 把所有點畫出來，觀察是否重現了環境的俯視輪廓（牆、桌、門口）。

50. 計算並標示：這台光達在 3 公尺與 6 公尺處，相鄰兩點的實際間距各是多少？討論這對偵測細小物體的影響。

51. 規格解讀：找一份真實光達的規格表（教學型或車用型皆可），指出其量測範圍、角解析度、掃描頻率、以及「最大距離」標註的反射率前提。

**討論問題**

44. 你畫出的點雲輪廓，哪些地方最清晰？哪些地方有缺口或雜點？缺口最可能對應環境中的什麼（玻璃？黑色物體？太遠？）？

45. （路徑 B）玻璃與鏡子在光達資料上造成了什麼異常？如果一台掃地機器人用這台光達建圖，遇到落地窗會發生什麼事？

46. 3 公尺與 6 公尺處的點間距差多少？若要可靠偵測 5 公分寬的椅腳，這台光達在多遠的距離內才辦得到？

47. 根據本章的比較（表 8-2），如果要為一台校園服務機器人選主要感測器，你會選光達還是相機？為什麼？需要補上什麼感測器？

**延伸挑戰（大學部）**

其一：撰寫一個 ROS 2 節點，訂閱 /scan、把 LaserScan 即時轉為點雲並發布，在 RViz 中同時顯示原始掃描與轉換結果。其二：實作一個簡單的「最近障礙物警示」——找出每一圈掃描中的最小距離與其方向，當小於安全門檻時輸出警告，這是第 29 章避障功能的雛形。

## 延伸閱讀

37. R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, Introduction to Autonomous Mobile Robots, 2nd ed., MIT Press, 2011.（第 4 章感測器部分含測距原理的系統介紹）

38. T. Raj et al., "A Survey on LiDAR Scanning Mechanisms," Electronics, 2020.（各種掃描機構——機械、MEMS、Flash、OPA——的完整比較）

39. You Li and Javier Ibanez-Guzman, "Lidar for Autonomous Driving," IEEE Signal Processing Magazine, 2020.（車用光達原理與系統的權威綜述）

40. ROS 官方文件 sensor_msgs/LaserScan 與 sensor_msgs/PointCloud2 訊息定義.（資料格式的第一手規格）

41. Point Cloud Library (PCL) 官方教學（pointclouds.org）.（3D 點雲處理的標準工具，第五篇實作將使用）

42. SLAMTEC RPLIDAR 系列與 Livox 系列產品技術文件.（入門級 2D 與低成本 3D 光達的規格與 ROS 驅動範例）

43. ROBOTIS TurtleBot3 e-Manual（emanual.robotis.com）.（教學平台的完整實作指引，含 SLAM 範例）

## 從距離到影像：本章的位置回顧

光達給了機器人一個近乎理想的幾何感官：直接、精確、不受光照影響。但它也有三個本質限制。其一，它只看得到幾何，看不到顏色、文字與語意——它知道前方有一個長方體，但不知道那是門還是櫃子。其二，它在無特徵的長廊或空曠大廳中會退化：每個方向看起來都一樣，掃描匹配無從判斷自己走了多遠（第 17.5 節會正式討論這個問題）。其三，它貴、重、耗電，雖然如 8.8 節所述這件事正在快速改變。

第 9 章的相機在這三點上恰好相反：它便宜、輕、省電，而且富含語意資訊——顏色、紋理、文字、物體類別。代價是它量不到絕對尺度（單目）、怕暗怕逆光、怕白牆。這兩種感測器的互補性，正是本章 8.6 節表 8-2 的核心，也是第五篇與第六篇分別以光達與視覺為主角的原因。

而第 23 章的 ORB-SLAM3 與第 18 章的 LIO-SAM 所代表的融合路線，則說明了業界的真實答案：在能負擔的前提下，兩者都裝。第 32 章會更進一步——當視覺語言模型能理解「那是一扇門」時，光達的幾何精度與相機的語意理解將合而為一，成為真正能聽懂人話的導航系統。

## 小結

48. 光達以飛行時間量測距離：脈衝式（dToF）直接量往返時間、能量集中適合長距離；FMCW 連續調頻，可同時量測距離與速度且抗干擾，但成本較高，是下一世代的重要方向。

49. 2D 光達掃描單一水平面、資料輕量價格低，是室內建圖主力；3D 光達以多線或垂直掃描獲得立體資訊；掃描機構從機械式走向 MEMS、Flash、OPA 等固態方案，趨勢是更小、更耐震、更便宜。

50. 2D 光達輸出 LaserScan（距離陣列 + 規律角度，緊湊輕量），3D 光達輸出 PointCloud（直接存三維座標，資料量大需降採樣）；兩者分別對應本書第四篇與第五篇。
51. 關鍵規格包含量測範圍、精度與精密度、角解析度（決定可偵測的最小物體，且同一角解析度下距離越遠點越疏）、掃描頻率（過低會造成運動變形）與反射率（黑色物體、玻璃鏡面是主要失效來源）。
52. 光達廣泛用於室內建圖、倉儲 AGV（兼安全防護）、自駕車（立體感知與地圖定位），以及背包式測繪、農林量測與機器狗巡檢等新興領域。
53. 光達給精確幾何但無語意且怕玻璃鏡面、相機給豐富語意但無可靠深度與尺度、IMU 給高頻運動但漂移；三者天生互補，最強健的系統往往並用。
54. 低成本 2D 光達與 TurtleBot 等教學平台，加上完全免費的模擬環境，讓 SLAM 教育得以下放到大學部與高中；本書建議「先模擬、再便宜硬體、後追求精度」的學習路徑。

下一章，我們轉向資訊最豐富的感測器——相機。從針孔相機模型、鏡頭畸變與校正，到單目、雙目、RGB-D 的差異，第 9 章將說明影像如何從一堆像素變成幾何資訊，為第六篇的視覺 SLAM 奠定基礎。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

## 第 9 章

## 相機與視覺導航感測器

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

## 本章在全書的位置

第 8 章的光達直接量出幾何，精確而可靠。本章的相機恰好相反——它量不到任何距離，卻帶來光達永遠給不了的東西：顏色、紋理、文字與語意。

這是第二篇的最後一章，也是整本書的轉折點。前八章累積下來的感測器，到此全部到齊：絕對位置（第 4、5 章）、絕對方向（第 6 章）、高頻運動（第 7 章）、精確幾何（第 8 章）、豐富語意（本章）。每一個都有它說謊的時候，沒有一個能單獨完成導航。

因此第三篇要教的，是把這些彼此矛盾的讀值融合成一個最合理答案的數學——第 10 章的座標與位姿、第 11 章的機率與狀態估測、第 12 章的最佳化與圖論。而本章的內容特別關鍵：針孔模型與內參矩陣 K 是整個第六篇視覺 SLAM 的地基，第 21 章的對極幾何、第 22 章的 ORB-SLAM2、第 23 章的 ORB-SLAM3，每一條式子裡都有 K。

**圖 9-0　第 9 章與全書的連結地圖**

【承接前章】

第 8 章　光達：幾何精確，但無語意、長廊退化、成本高

本　章　相機：便宜輕省電、富含語意，但無尺度、怕暗怕白牆

【本章負責】

針孔模型　→　內參矩陣 K　→　畸變與校正

三種相機取得深度的方式：單目、雙目、RGB-D

視覺的四種原始素材：特徵點、邊緣、光流、深度

【往下延伸】

→ 第 4.6 節　AprilTag 解位姿所需的 K，在此補齊

→ 第 20 章　特徵法與直接法：本章 9.6 節的完整展開

→ 第 21 章　對極幾何：兩張影像之間的運動估測

→ 第 22、23 章　ORB-SLAM2 / 3

→ 第 30 章　RGB-D 與雙目相機的實機建圖

→ 第 32、34 章　語意 SLAM 與 VLM 語言導航：本章 9.7 節的完整版

## 學習目標

相機是資訊最豐富、成本最低廉的感測器——一張影像同時包含形狀、紋理、顏色與語意。但影像的本質是「三維世界被壓扁成二維」，如何把遺失的深度找回來，正是視覺 SLAM 的核心挑戰。本章是第二篇的最後一章，也是第六篇視覺 SLAM 的入口。學完本章之後，同學應能夠達成下列目標：

50. 說明針孔相機模型，並寫出三維點如何投影到二維影像的數學關係。

51. 解釋相機內參與外參的意義，說明鏡頭畸變的成因與相機校正的目的。

52. 比較單目、雙目、RGB-D 三種相機取得深度的方式與各自的優缺點。

53. 說明特徵點、邊緣、光流三種視覺資訊的差異與用途。

54. 分析視覺導航相對於光達的優勢與限制，特別是尺度不確定性問題。

55. 說明從影像處理銜接到視覺 SLAM 的知識脈絡。

56. 初步認識 VLM 視覺語言模型與語意導航的概念，為第 32 章預做準備。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
| --- | --- | --- |
| 針孔相機模型 | Pinhole Camera Model | 描述三維點透過光心投影到成像平面的幾何模型。 |
| 焦距 | Focal Length | 光心到成像平面的距離，決定放大倍率與視野。 |
| 相機內參 | Intrinsic Parameters | 焦距、主點等相機自身的成像參數，構成內參矩陣 K。 |
| 相機外參 | Extrinsic Parameters | 相機在世界座標中的位置與姿態（旋轉 R 與平移 t）。 |
| 鏡頭畸變 | Lens Distortion | 鏡頭造成的影像變形，分徑向畸變與切向畸變。 |
| 相機校正 | Camera Calibration | 量測並估計內參與畸變係數的程序。 |
| 視差 | Disparity | 同一點在左右兩張影像中的水平位移，用於雙目測深。 |
| 基線 | Baseline | 雙目相機兩鏡頭之間的距離，影響測深範圍與精度。 |
| 特徵點 | Feature / Keypoint | 影像中易辨識、可重複偵測的顯著點，如角點。 |
| 光流 | Optical Flow | 影像中像素亮度隨時間的移動，反映運動資訊。 |
| 尺度不確定性 | Scale Ambiguity | 單目相機無法確定真實尺寸的先天限制。 |

| 中文名詞 | 英文名詞 | 概念說明 |
| --- | --- | --- |
| 視覺語言模型 | VLM | 能同時理解影像與語言的 AI 模型，用於語意導航。 |

## 9.1 針孔相機模型

要理解相機如何「看見」世界，必須先理解一個古老而優雅的模型——針孔相機（Pinhole Camera）。它的原型就是暗箱：一個密閉盒子的一面開一個小孔，外界的光線穿過小孔，在對面的內壁上投出一個上下顛倒的倒影。這個看似簡單的裝置，蘊含了所有相機成像的核心幾何。

### 投影的幾何

把針孔（光心，Optical Center）設為座標原點，光軸沿 Z 軸指向前方。空間中一個三維點 P =（X、Y、Z）發出的光線穿過光心，投射在焦距 f 處的成像平面上。由相似三角形，可得投影點的座標：

$$x = f \cdot X / Z$$

$$y = f \cdot Y / Z$$

這兩條式子雖然簡單，卻道破了視覺 SLAM 全部困難的根源——注意分母都是 Z（深度）。這意味著：

- 三維被壓成二維：三個座標（X、Y、Z）投影後只剩兩個（x、y），深度 Z 這個維度在投影過程中被「除掉」了。
- 深度資訊遺失：距離 2 公尺的小物體，與距離 4 公尺的大物體，可能在影像上佔完全相同的大小。單憑一張影像，無法區分「近的小東西」與「遠的大東西」。

這就是 9.5 節要深入討論的尺度不確定性的幾何本質。找回這個被除掉的 Z，正是雙目、RGB-D 相機以及視覺 SLAM 各種技術的共同目標。

### 從物理座標到像素座標

上面算出的（x、y）是成像平面上的物理座標（單位為公釐）。但數位影像用的是像素座標——以左上角為原點、以像素為單位。兩者的轉換需要三個參數：把公釐換算為像素的比例（fx、fy，把焦距用像素表示），以及影像中心（主點）的位置（cx、cy）。完整的

投影關係可寫成矩陣形式：

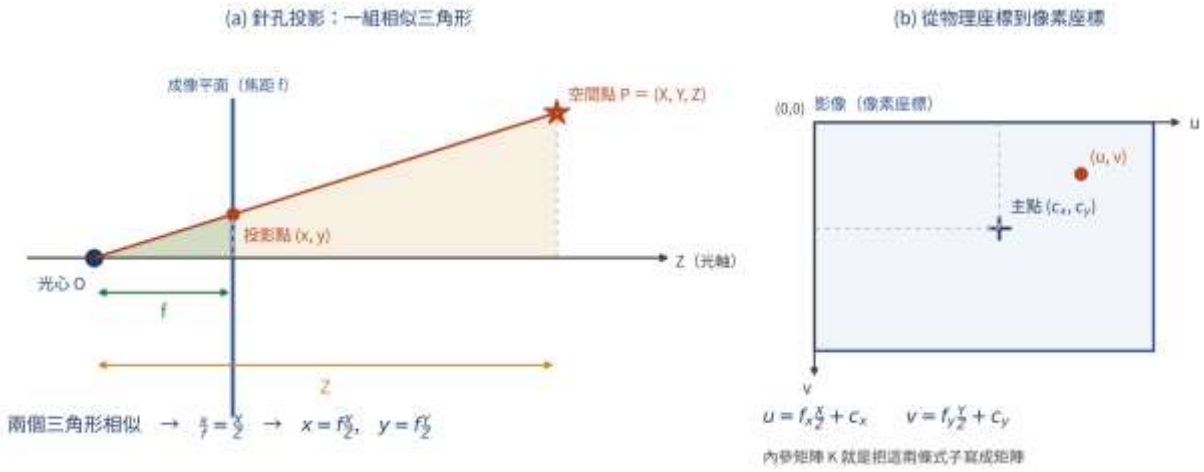
$$[u\ v\ 1]^T = (1/Z) \cdot K \cdot [X\ Y\ Z]^T$$

其中 K 稱為相機內參矩陣（Intrinsic Matrix）：

$$K = [fx\ 0\ cx;\ 0\ fy\ cy;\ 0\ 0\ 1]$$

這個 K 矩陣是視覺 SLAM 中最基礎、最常出現的物件。第 4 章的 AprilTag 解算、第 21 章的對極幾何、第 22 章的 ORB-SLAM，都要用到它。而 K 的數值必須透過相機校正（9.2 節）量測得到。

**圖 9-1　針孔相機模型：從空間點到像素**




左圖是整個視覺幾何的起點，而它其實只是一組相似三角形。光心到成像平面的距離是焦距 f，光心到空間點的距離是深度 Z；兩個三角形相似，於是 x / f = X / Z，也就是 x = f·X / Z。所有複雜的視覺 SLAM 數學，追根究柢都是從這兩行推出來的。

請特別注意分母的那個 Z。它是整章、甚至整個第六篇所有困難的根源：投影時 Z 被除掉了，所以同一個像素可能來自近處的小東西，也可能來自遠處的大東西。這就是 9.3 節所說的尺度不確定性，也是第 7 章的 IMU 之所以對單目視覺如此重要的原因——IMU 量得到有物理單位的加速度，能把丟失的尺度補回來。

右圖處理的是另一件事：相機輸出的是像素，不是公尺。像素座標的原點在影像左上角，而光軸穿過的那個點（主點）通常不在正中央——感光元件與鏡頭的裝配總有偏差。因此必

須加上平移量 cx、cy，並用 fx、fy 把物理單位換成像素單位（兩者可能略有差異，因為像素未必是正方形）。

把這四個數字 fx、fy、cx、cy 排成矩陣，就是內參矩陣 K。所以 K 不是什麼抽象的東西——它就是「這台相機怎麼把世界壓成像素」的四個數字。而 9.2 節的校正，做的正是量出這四個數字（外加畸變係數）。

**一個常被混淆的區分**

內參（Intrinsic）描述相機自己：焦距、主點、畸變。只要不換鏡頭、不調焦，它就固定不變。

外參（Extrinsic）描述相機在哪裡：相對於世界座標系的旋轉 R 與平移 t。機器人一動，它就改變。

視覺 SLAM 要解的正是外參——每一幀的 R 與 t 就是機器人的位姿。內參必須事先校正好，否則解出來的外參全錯。

這個分工與第 6.4 節磁力計校正的結構完全相同：先修正感測器本身，再談它與世界的關係。

## 9.2 鏡頭畸變與相機校正

針孔模型假設光線走完美的直線，但真實相機用的是玻璃或塑膠鏡頭。鏡頭雖然能收集更多光線、讓影像更明亮清晰，卻也引入了畸變（Distortion）——影像的幾何變形。

### 兩種畸變

- 徑向畸變（Radial Distortion）：由鏡頭形狀造成，離影像中心越遠變形越嚴重。桶狀畸變讓直線向外凸（廣角鏡與魚眼鏡明顯）、枕狀畸變讓直線向內凹。用手機廣角鏡拍一面磚牆，邊緣的磚縫明顯彎曲，就是徑向畸變。

- 切向畸變（Tangential Distortion）：由鏡頭與感測器未完全平行安裝造成，使影像某些區域看起來被「傾斜拉扯」。通常比徑向畸變輕微。

畸變對視覺 SLAM 是致命的：SLAM 的幾何計算全都假設針孔模型（直線投影），若不先校正畸變，一條真實的直線在影像中是彎的，所有基於直線與角度的幾何推算都會系統性出錯，建出的地圖會扭曲變形。

### 相機校正：找出 K 與畸變係數

相機校正（Camera Calibration）的目的，就是量測出這台相機的內參矩陣 K 與畸變係數。最經典的方法是張正友（Zhang）於 2000 年提出的棋盤格校正法，其原理與第 4 章 AprilTag 的 PnP 一脈相承：

52. 準備一塊尺寸已知的棋盤格標定板（每個方格邊長已知）。
53. 用待校正的相機，從不同角度、距離拍攝棋盤格十至二十張。
54. 演算法自動偵測每張影像中棋盤格角點的像素座標。
55. 由於棋盤格的真實三維座標已知（平面上規律排列），而它們的影像座標也測到了，就能反推出使這些對應關係最吻合的 K 與畸變係數（這又是一個最小平方最佳化問題）。
56. 用求得的畸變係數對影像做「去畸變」（Undistortion），把彎曲的影像矯正回符合針孔模型的樣子。

OpenCV 內建完整的校正工具（calibrateCamera 等函式），ROS 也有 camera_calibration 套件，只要對著棋盤格晃動相機即可自動完成。這是所有視覺 SLAM 實作的第一步，第 30 章實作 RGB-D SLAM 時必定會用到。

**實務提醒：校正不是一勞永逸**

- 換鏡頭、調焦距、甚至鏡頭受碰撞位移，都會使原校正失效，必須重新校正。
- 校正影像應涵蓋畫面各個區域與多種傾斜角度，尤其別忘了邊角——徑向畸變在邊角最嚴重。
- 校正誤差（重投影誤差）應小於 1 像素才算良好；若偏大，多半是棋盤格拍得不夠多樣或偵測到錯誤角點。
- 工業相機出廠常附校正參數，但精度要求高的應用仍建議自行校正驗證。

## 9.3 單目相機、雙目相機、RGB-D 相機

9.1 節說明了單張影像會遺失深度。要找回深度，主流有三條路，對應三種相機。

### 一、單目相機（Monocular）：最簡單也最困難

一顆相機、一張影像。硬體最簡單、最便宜、最省電、最普及（每支手機都有）。但如前所述，單張影像無法直接得到深度——這帶來著名的尺度不確定性（9.5 節）。

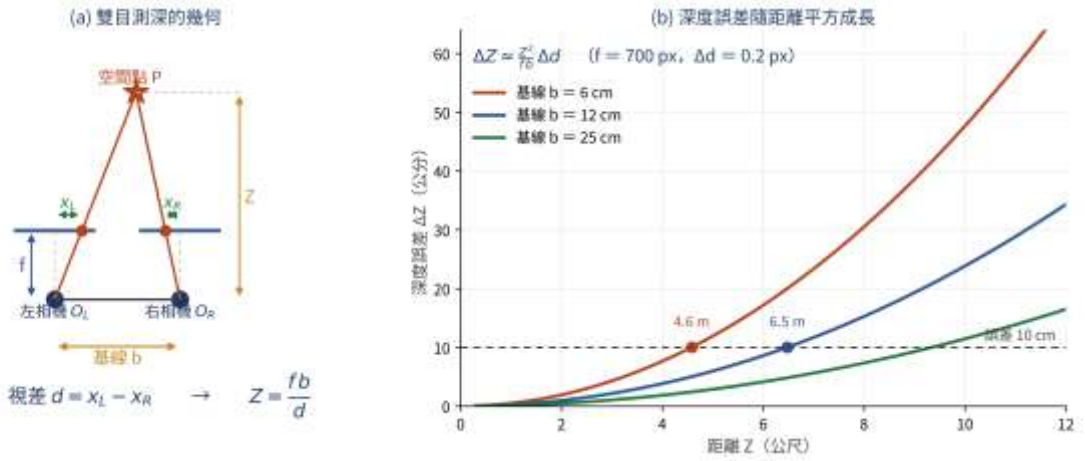
那單目相機如何做 SLAM?答案是利用運動：相機移動時，從不同位置拍攝同一場景，就像人閉上一隻眼、左右晃頭也能感知遠近。透過連續多幀影像的視差變化，可以「三角測量」出深度——但這個深度只有相對比例，沒有絕對尺度（不知道是真實的一公尺還是模型屋的一公分），除非引入其他資訊（IMU、已知物體大小、或輪子里程計）。這正是第 7 章 IMU 為單目視覺提供絕對尺度的價值所在。

### 二、雙目相機（Stereo）：模仿雙眼

兩顆相機以固定的已知距離（基線，Baseline）並排安裝，同時拍攝。同一個空間點在左右兩張影像中的水平位置會有落差，稱為視差（Disparity）。近的物體視差大、遠的物體視差小——把手指放在眼前，輪流閉左右眼，手指相對背景跳動很大；看遠山則幾乎不動，就是這個道理。

由視差配合已知的基線與焦距，即可用三角幾何直接算出深度（9.6 節公式）。雙目的優點是：單張（左右對）影像就能得到深度、有絕對尺度、室內外皆可用。限制是：需要精確校正兩相機的相對位置、對無紋理表面（白牆）因為找不到對應點而失效、且測深精度隨距離增加而快速下降（基線越短衰減越快）。

### 圖 9-2　雙目測深的幾何與深度誤差的成長




左圖說明雙目測深為什麼成立。同一個空間點在左右兩張影像中的水平位置不同，這個差值就是視差 d。點愈近，視差愈大；點愈遠，視差愈小。由相似三角形可得 Z = f·b / d——焦距與基線都是已知常數，所以量出視差就等於量出深度。

右圖則說明它的限制，而這是規格表上看不到、卻決定系統成敗的事。把 Z = f·b / d 對 d 微分，得到 ΔZ ≈ （Z² / f·b）·Δd：深度誤差隨距離的平方成長。以焦距 700 像素、視差量測誤差 0.2 像素計算，基線 6 公分的小型雙目模組在 4.6 公尺處深度誤差就達 10 公分；基線拉到 12 公分可延伸到 6.5 公尺；25 公分則到 9.4 公尺。

這解釋了三件實務上的事。其一，為什麼自駕車的雙目相機基線那麼寬——因為它要看一百公尺外。其二，為什麼手機的雙鏡頭只能做幾公尺內的人像景深（案例 9-1），而不能拿來導航。其三，為什麼 RGB-D 相機幾乎都標示「有效範圍 0.3 至 5 公尺」——超過之後誤差大到無法使用。

同學選相機時可以把這張圖當成速查表：先確定「我要在多遠處、量到多準」，再回推需要

的基線。這個由需求反推規格的流程，與第 8.4 節光達角解析度的判斷完全同構——兩種感測器的物理原理不同，但工程思維是同一套。

## 三、RGB-D 相機：主動測深

RGB-D 相機同時輸出彩色影像（RGB）與每個像素的深度（D）。它以主動方式取得深度，主流有兩種技術：

- 結構光（Structured Light）：投射一組已知的紅外線圖案（如密集光點）到場景，由圖案的變形程度反推深度。初代微軟 Kinect 即採此法，近距離精度高，但強光下失效、範圍較短。
- 主動紅外雙目（Active IR Stereo）：本質是雙目測距，但額外投射紅外線紋理圖案，解決白牆等無紋理表面找不到對應點的問題。Intel RealSense D400 系列即採此法，室內外皆可運作。

RGB-D 相機的最大優點是「開箱即得稠密深度」，不需自己做複雜的深度估計，是室內 RGB-D SLAM（第 30 章 RTAB-Map、ORB-SLAM3）的理想輸入。限制是：多數依賴紅外線，強烈日光下戶外表現不佳（陽光的紅外線會淹沒投射圖案）、有效範圍通常在 0.2 至 10 公尺、且對玻璃與鏡面同樣束手無策（與第 8 章光達的弱點相同）。

| 類型 | 取得深度方式 | 絕對尺度 | 主要優點 | 主要限制 |
|---|---|---|---|---|
| 單目 | 靠運動 + 三角測量 | 無（需外部提供） | 最便宜、最普及、最省電 | 尺度不確定、需運動 |
| 雙目 | 左右視差三角幾何 | 有 | 單幀得深度、室內外可用 | 需校正、怕無紋理、遠處衰減 |
| RGB-D | 結構光 / 主動紅外雙目 | 有 | 直接輸出稠密深度 | 怕強光、範圍有限、怕玻璃 |

**表 9-1 三種相機取得深度的方式比較**

## 9.4 特徵點、邊緣、光流、深度估測

相機給的是一整張由數十萬像素組成的影像。但 SLAM 不可能、也不需要處理每一個像素，而是從影像中提取有用的資訊。本節介紹三種最重要的視覺資訊。

### 一、特徵點（Feature / Keypoint）：視覺 SLAM 的路標

特徵點是影像中「容易被認出來、換個角度也還認得」的顯著點——通常是角點（兩條邊的交會處），例如窗框的角、磁磚的交界、海報的文字邊角。

為什麼偏愛角點？想像在一面白牆上選一個點，下一幀要在新影像中找回它——白牆上到處長得一樣，根本找不到；選在一條直線的邊上，雖能判斷垂直方向卻無法確定沿線位置（這稱為孔徑問題）；唯有角點，在兩個方向上都獨一無二，才能被準確地重新定位。這就是特徵點法視覺 SLAM（第 20、22 章）的基石：把角點當作環境中的「路標」，追蹤它們在連續影像中的移動來推算相機運動。ORB、SIFT、SURF、FAST 等都是知名的特徵偵測與描述方法，第 20 章會詳述。

### 二、邊緣（Edge）：結構的輪廓

邊緣是影像中亮度急遽變化的地方，通常對應物體的輪廓或不同表面的交界。邊緣資訊在某些直接法 SLAM 與線特徵 SLAM 中扮演重要角色，尤其在特徵點稀少的人造環境（如空曠走廊、只有牆線的場景）中，線條反而是穩定的線索。

### 三、光流（Optical Flow）：像素的運動

光流描述影像中亮度模式隨時間的移動——當相機或物體移動，畫面中的紋理會「流動」。想像雨滴打在車窗上向後流的景象，那就是光流的視覺化。光流可用來估計運動、追蹤特徵、偵測動態物體。它是直接法視覺里程計的核心，也是無人機定點懸停（光流定位，第 26 章）的關鍵技術。

### 四、深度估測（Depth Estimation）

除了前述的雙目視差與 RGB-D 主動測深，近年深度學習帶來了單目深度估測的突破：訓練神經網路從單張影像「猜」出每個像素的相對深度。這類模型（如 Depth Anything，第 32 章會提到）利用了大量影像學到的先驗知識——它「知道」天空很遠、地板向遠方延伸、人比螞蟻大。雖然是「猜」的、且多半只有相對深度，但在許多應用中已足夠實用，是傳統幾何方法之外的重要補充。

這四種資訊並非互斥：現代視覺 SLAM 常混合使用——以特徵點做穩健的定位骨架、以光流做快速追蹤、以深度（不論來自雙目、RGB-D 或神經網路）重建稠密地圖。

## 9.5 視覺導航的優點與限制

把相機放進第二篇的感測器全景中（第 8 章表 8-2 已初步比較），本節深入視覺方法特有的優勢與軟肋。

### 視覺導航的獨特優勢

- 資訊最豐富：一張影像同時含形狀、紋理、顏色與語意。這是光達與 IMU 完全給不了的——光達只知道「三公尺外有東西」，相機卻能知道「那是一扇紅色的門」。這正是第 32 章語意導航建立在視覺之上的原因。
- 成本最低、最普及：相機比光達便宜一至兩個數量級，且無所不在，量產成熟。
- 被動感測、低功耗：不像光達需主動發射雷射，相機只接收環境光，省電且無互相干擾問題，特別適合無人機與微型機器人（第 25、26 章）。
- 輕巧：對載重與電力極度受限的無人機而言，「相機 + IMU」是最理想的組合。

### 視覺導航的先天限制

- 尺度不確定性：單目相機最根本的軟肋。如 9.1 節所述，投影除掉了深度，單目影像無法得知絕對尺寸，必須由 IMU、雙目基線或已知物體提供尺度。

- 依賴光線與紋理：這是視覺方法最致命的弱點。黑暗、逆光、過曝會讓相機「看不見」；白牆、純色地板等無紋理表面則找不到特徵點；快速運動造成的影像模糊也會使追蹤失敗。相較之下，光達自帶光源、不怕黑暗。
- 運算負擔：影像資料量大，特徵提取與匹配需要相當運算力，這也是邊緣運算平台（第 33 章 Jetson）的用武之地。
- 怕動態場景：畫面中走動的人、車會干擾以「環境靜止」為假設的視覺 SLAM。

**觀念總結：視覺與光達的互補**

光達：精確幾何、自帶光源不怕黑、但無語意、怕玻璃鏡面、較貴。

相機：豐富語意、便宜輕巧、但無可靠尺度深度、怕黑暗與無紋理。

兩者的弱點幾乎不重疊——這正是「光達建幾何骨架、相機貼語意標籤」的融合邏輯（第 32 章），

也再次印證第二篇的核心信念：沒有全能的感測器，只有恰當的組合。

## 9.6 從影像處理到 Visual SLAM

許多同學在修這門課之前，已經學過影像處理（Image Processing）——濾波、邊緣偵測、二值化、形態學運算等。本節說明這些基礎如何一步步通往視覺 SLAM，把兩門課的知識接起來。

### 知識的階梯

57. 影像處理（Image Processing）：處理單張影像的像素——去雜訊、增強對比、偵測邊緣。這是「讓影像更清楚」的層次。
58. 特徵工程（Feature Extraction）：從影像中抽出角點、描述子，把「一堆像素」濃縮成「少數可辨識的點」。這是第 20 章的內容。

59. 多視圖幾何（Multiple-View Geometry）：分析同一場景的多張影像之間的幾何關係，從特徵點的移動反推相機運動與場景結構。這是第 21 章對極幾何的內容。
60. 視覺里程計（Visual Odometry, VO）：把上一步逐幀串起來，持續估計相機軌跡——但會累積漂移。
61. 視覺 SLAM(Visual SLAM)：在 VO 基礎上加入建圖、閉環偵測與全域最佳化，消除長期漂移，得到一致的軌跡與地圖。這是第六篇的主題。

這個階梯呼應了第 2 章 UZH 課程的鋪陳順序：影像形成 → 特徵 → 多視圖幾何 → 視覺里程計 → SLAM。每一階都是下一階的基礎，缺一不可。本書把影像處理視為先備知識，從第 20 章的特徵開始正式進入視覺 SLAM;但本章打下的相機模型與校正基礎，是理解後續一切的前提。

### 兩大流派：特徵法與直接法

視覺 SLAM 有兩條主要技術路線，此處先建立概念，第 20 章詳述：

- 特徵法（Feature-based）：提取並匹配特徵點，以特徵點的幾何關係求解運動。穩健、成熟，ORB-SLAM 系列為代表。
- 直接法（Direct）：不提取特徵，直接用像素亮度變化（光流概念）估計運動。在弱紋理場景有優勢，但對光線變化敏感，LSD-SLAM、DSO 為代表。

## 9.7 VLM / AI 視覺模型與語意導航初探

本節是第九篇（第 32–34 章，AI 語意與語言導航）的預告。傳統視覺 SLAM 回答的是幾何問題：「相機在哪、環境長什麼形狀」；而 AI 視覺模型能進一步回答語意問題：「畫面裡有什麼、代表什麼意義」。這正是相機相對於光達最珍貴的價值——它承載了語意。

### 從幾何到語意

一台只做傳統 SLAM 的機器人，建出的地圖是「這裡有牆、那裡有障礙」；加上 AI 視覺辨識後，地圖能升級為「這是門、那是樓梯、那邊是滅火器」——這就是語意地圖（第 1

章、第 3 章提過）。這個升級讓機器人能執行人類語言等級的任務。

### 關鍵 AI 視覺模型（第 32 章詳述）

- YOLO：即時物件偵測，在影像中框出「這是人、那是椅子」，是校園場景辨識的主力（附件教材與大學課程的重點）。
- SAM(Segment Anything)：把影像精確分割成一塊塊物件區域。
- Depth Anything：從單張影像估計深度（9.4 節提過）。
- CLIP / DINO：理解影像的語意內容，連結影像與文字。

### VLM：讓機器人「看懂並聽懂」

視覺語言模型（VLM,Vision-Language Model）能同時理解影像與自然語言。這讓機器人可以接受口語任務：「請走到白板前」「避開地上的書包」「幫我找電梯」——VLM 分析相機畫面，理解語言指令，再把任務轉換成導航目標。

這是機器人導航正在發生的典範轉移：從「給定座標、走過去」進化為「聽懂人話、理解場景、自主決定怎麼走」。這也是為什麼本章要放在第二篇的最後——相機不只是又一個測距的感測器，它是機器人通往「理解世界」的窗口。第 32 章會完整展開這個激動人心的方向，包括如何蒐集與標註訓練資料、如何訓練校園場景辨識模型、以及如何整合 VLM 與 ROS 2。

至此，第二篇「定位與導航感測器」全部完成。同學已經認識了機器人的六大感官：里程計與 IMU 感知自身運動（第 7 章），無線電與衛星、地磁提供絕對參考（第 5、6 章），室內定位技術補足室內空缺（第 4 章），光達以雷射精確量測幾何（第 8 章），相機則帶來最豐富的視覺與語意（本章）。下一篇，我們將從「感測器給了什麼」轉向「如何用數學處理這些帶著雜訊的資料」——進入 SLAM 的數學基礎。

## 公式或流程

## 一、針孔投影完整方程

三維點 P_w 在世界座標中的位置，經外參（旋轉 R、平移 t）轉到相機座標，再經內參 K 投影到像素座標，完整關係為：

$$s \cdot [u\, v\, 1]^T = K \cdot [R|t] \cdot [X_w\, Y_w\, Z_w\, 1]^T$$

其中 s 為尺度因子（等於該點在相機座標系中的深度 Z）。這條式子把本章的三個核心概念串在一起：內參 K（9.1）、外參 [R|t]（相機在世界中的位姿，正是 SLAM 要求解的量）、以及尺度 s（9.5 的尺度不確定性來源）。整個視覺 SLAM，本質上就是在大量這樣的投影方程中，同時求解相機位姿 [R|t] 與三維點座標。

## 二、雙目測深公式

雙目相機的深度計算是視覺測距最直觀的公式。設兩相機基線為 b、焦距為 f（像素單位）、某點在左右影像的視差為 d（disparity，單位像素），則深度 Z 為：

$$Z = f \cdot b / d$$

這條式子透露三個重要事實：

- 視差 d 越大（近物），深度 Z 越小；視差趨近 0（遠物），深度趨近無窮——所以雙目對遠物測不準。
- 基線 b 越大，同樣距離的視差越大、測深越準、可測範圍越遠（這是 RealSense D455 加大基線以提升遠距精度的原因）。
- 深度誤差與距離平方成正比：距離加倍，誤差變四倍。這是所有被動視覺測深的共同宿命。

## 三、深度誤差隨距離成長

對視差量測誤差 Δd，深度誤差近似為：

$$\Delta Z \approx (Z^2 / (f \cdot b)) \cdot \Delta d$$

式中 $Z^2$ 說明了為何雙目與 RGB-D 相機都標榜「近距離精確、遠距離快速衰減」。例如某相機在 1 公尺處誤差 1 公分，到 4 公尺處誤差就可能達 16 公分。這條式子是為機器人選相機、定工作範圍的重要依據。

## 四、相機用於視覺 SLAM 的處理流程

**圖 9-3　從相機到視覺 SLAM 的資料流**

① 相機擷取影像（單目 / 雙目 / RGB-D）

↓

② 去畸變：用校正得到的畸變係數矯正影像（9.2）

↓

③ 特徵提取：偵測角點並計算描述子（9.4，第 20 章）

↓

④ 特徵匹配 / 光流追蹤：找出相鄰幀的對應點

↓

⑤ 幾何求解：由對應點與內參 K 推算相機運動（第 21 章）

↓

⑥ 三角測量 / 深度：重建三維點（單目需運動、雙目用視差、RGB-D 直接給）

↓

⑦ 後端最佳化 + 閉環：消除漂移，輸出軌跡與地圖（第 22、23 章）

# 9.8　視覺感測的當代變局：當幾何被模型取代

本章講的是幾何：針孔模型、內參矩陣、視差公式。這些式子自 1990 年代定型以來幾乎沒

變過，也仍然是第六篇所有系統的基礎。但 2024 至 2026 年間，電腦視覺領域發生了一件可能改寫這個基礎的事，同學必須知道。

## 一、單目深度不再是不可能的任務

9.3 節說單目相機「無法得到深度」，這在純幾何的意義下仍然正確——一張影像在數學上確實無法決定尺度。但近年的深度估測基礎模型做到了另一件事：從大量資料中學會「這個場景看起來應該多深」。

它們依靠的不是幾何，而是先驗知識：門的高度大約兩公尺、走廊的寬度大約一點五公尺、人的身高大約一點七公尺。模型看過足夠多的世界之後，就能從一張照片給出相當合理的深度圖，甚至是帶有公尺單位的絕對深度。

這對機器人的意義是實際的：第 30 章的 RGB-D 相機在強烈陽光下會失效（9.3 節提過），而單目深度模型不受此限；它也能為第 20 章的單目 SLAM 提供初始化的尺度參考。但同學務必記住它的本質——那是猜測，不是量測。遇到沒看過的場景（例如模型屋、巨型展場），它會自信地給出錯誤答案。

## 二、VGGT：把整條 SfM 管線換成一個模型

更根本的衝擊來自 2025 年 CVPR 最佳論文所提出的視覺幾何基礎轉換器。它的做法極為直接：輸入一張到數百張影像，模型一次前饋就同時輸出相機內參與外參、深度圖、點雲與三維點軌跡——整個過程在一秒之內完成。

請把這件事與傳統流程對照。傳統上要得到同樣的結果，必須：偵測特徵點（9.4 節）→ 匹配 → 估計對極幾何（第 21 章）→ 三角測量 → 執行集束調整（第 12 章）反覆迭代最佳化。這條管線是第六篇要花四章才講得完的內容。而該研究顯示，在不使用任何幾何最佳化後處理的情況下，模型的表現已超越那些有做後處理的傳統方法。

那麼第六篇還要學嗎？本書的答案非常明確：要，而且更要。理由有三個。其一，這類模型目前針對的是離線重建而非即時 SLAM，機器人要的是每秒三十次的持續定位，不是一次性

的場景重建。其二，它沒有閉環、沒有長時間一致性、沒有不確定度輸出——而這三件事正是 SLAM 的核心。其三，也是最重要的：看不懂幾何的人，無法判斷模型什麼時候在胡說。

### 三、事件相機：另一條不走尋常路的感測器

值得一提的還有事件相機。傳統相機以固定頻率輸出完整畫面，事件相機則讓每個像素獨立運作——只在亮度變化超過門檻時才回報一個事件，時間解析度可達微秒等級，動態範圍遠超過一般相機。

它正好對應 9.5 節列出的兩大視覺限制：動態模糊與光照劇變。無人機高速翻滾時傳統相機拍出一片糊，事件相機仍能清楚追蹤；從暗處衝出隧道的瞬間傳統相機整片過曝，事件相機幾乎不受影響。第 26 章討論無人機視覺慣性導航時會再提到這條路線。

它的代價是輸出的不是影像而是事件串流，本章 9.4 節的所有演算法都不能直接套用，必須重新設計。這使它至今仍主要停留在研究領域——但在高速與高動態範圍的場景中，它是目前唯一的解。

**給同學的三個判斷**

一、幾何沒有過時。模型輸出的位姿仍然要用本章的 K 去解釋，仍然要在第 10 章的座標系裡表達，仍然要經過第 12 章的最佳化才能全域一致。

二、模型最有價值的位置是前端。用它取代脆弱的手工步驟——特徵匹配、單目初始化、深度先驗——而保留後端的幾何最佳化。這與第 7.8 節學習式 IMU 的結論完全一致。

三、判斷力來自幾何。知道 $\Delta Z \propto Z^2$ 的人，看到模型說「那面牆在五十公尺外」時會知道該懷疑；不知道的人只會照單全收。

## 案例

## 案例 9-1 手機人像模式：雙鏡頭的日常魔法

同學口袋裡的雙鏡頭手機，每天都在示範本章的原理。開啟人像模式拍照，主體清晰、背景柔美虛化——這個效果的背後，正是雙目測深：兩顆鏡頭以微小基線同時拍攝，手機計算每個像素的視差，估出深度圖，判斷「哪些像素是近的主體、哪些是遠的背景」，再對背景做模擬的散景虛化。

這個案例也暴露了雙目的限制。人像模式常在髮絲邊緣、鏤空物體、或純色背景「穿幫」——因為這些地方視差難以準確估計（無紋理、細節過小）。這正是 9.3 節所說「雙目怕無紋理表面」的生活寫照。近年手機改用「雙鏡頭視差 + AI 語意分割 + 神經網路深度估計」的混合方案來改善，恰恰對應 9.4 節提到的「多種視覺資訊混合使用」的趨勢。同學下次用人像模式時，不妨觀察它在哪些場景會出錯，這是最貼身的視覺深度實驗。

## 案例 9-2 火星直升機「機智號」：純視覺的星際導航

NASA 的火星直升機機智號（Ingenuity）在火星上完成了數十次飛行，創下地外行星動力飛行的歷史。火星上沒有 GPS、沒有磁場可用的羅盤（火星幾乎沒有全球磁場），它如何知道自己在哪、飛得多穩？

答案是視覺慣性導航：機腹一顆朝下的單目相機以每秒數十幀拍攝地面，追蹤地表岩石與紋理的特徵點（9.4 節），配合 IMU 估計運動——這正是第 2 章提過 UZH 研究群擅長、也是第 26 章無人機導航的核心技術。火星稀薄的大氣（密度僅地球約 1%）讓飛行極不穩定，對導航的即時性與穩健性要求極高。機智號的成功證明了一件事：在完全沒有外部訊號的極端環境，一顆便宜的相機加上 IMU，就能撐起可靠的自主導航。這也是為什麼本書如此重視視覺慣性方法——它是機器人「離開基礎設施、真正自主」的關鍵。

## 案例 9-3 白牆的詛咒：視覺 SLAM 的經典失敗

一個在實驗室反覆上演的場景：同學興沖沖地讓搭載相機的機器人在教室跑視覺 SLAM，一切順利——直到機器人轉向一面乾淨的白牆，系統立刻「跟丟」，地圖崩壞。

原因就是 9.4 與 9.5 節反覆強調的：白牆上沒有任何特徵點，相機看到的每一處都長得一樣，系統無從判斷自己是否移動、移動了多少。這與第 7 章 IMU 初始化需要「運動激勵」、第 8 章光達怕玻璃，並列為 SLAM 三大經典「環境陷阱」。

解法呼應了全書的融合思想：加一顆 IMU，在視覺跟丟的數秒內靠慣性推算撐過去（第 7 章）；或改用光達，牆面對光達反而是清晰的平面；或投射紋理（RGB-D 主動紅外的做法）。這個案例的教學價值在於，它讓同學親身體會「感測器的限制會直接變成系統的失敗」——理解限制，比讚嘆能力更重要。

## 實作活動

### 實作 9-1　相機校正與視覺深度初體驗

目的：親手完成相機校正、觀察畸變矯正效果、並體驗特徵點偵測與雙目測深。本實作以一般網路攝影機或手機即可進行，是第 30 章視覺 SLAM 實作的暖身。

#### 器材

- 網路攝影機或手機相機、電腦（Python + OpenCV 環境）。
- 列印的棋盤格標定板（OpenCV 官網可下載，貼在硬板上保持平整）。
- （選用）雙目相機或 RGB-D 相機（如 RealSense），供進階測深。

#### 第一部分：相機校正與去畸變

48. 用相機從不同角度、距離拍攝棋盤格 15 至 20 張，盡量涵蓋畫面各區域與各種傾斜。
49. 用 OpenCV 的 findChessboardCorners 偵測角點、calibrateCamera 求出內參矩陣 K 與畸變係數，記錄下來。
50. 觀察 K 矩陣：fx、fy 是否接近？cx、cy 是否約為影像寬高的一半？思考這些數字的物理意義。

51. 用求得的參數對一張含直線的影像（如磚牆、書架）做去畸變，把矯正前後並排比較，觀察邊緣彎曲的直線是否被拉直。

52. 檢查重投影誤差是否小於 1 像素；若偏大，增加校正影像數量或重拍再試。

**第二部分：特徵點偵測**

53. 用 OpenCV 的 ORB 偵測器，對一張教室或桌面的影像提取特徵點並畫出，觀察特徵點集中在哪裡（角點、紋理豐富處）、又缺席於哪裡（白牆、純色桌面）。

54. 對同一場景拍兩張（相機小幅移動），用特徵匹配找出對應點並連線，觀察匹配的正確與錯誤情形。

55. 故意拍一張大面積白牆的影像，觀察 ORB 幾乎偵測不到特徵點——親身驗證「白牆的詛咒」（案例 9-3）。

**第三部分：深度體驗（依器材選做）**

56. 若有 RGB-D 相機：讀取並顯示深度圖（以色階呈現遠近），把相機對準桌面與遠處，觀察深度值變化；再對準玻璃或鏡面，觀察深度資料的缺失或錯誤。

57. 若只有單目：把一個已知大小的物體（如 A4 紙）放在不同距離拍攝，記錄它在影像中的像素大小，驗證「距離加倍、像素尺寸減半」的針孔投影關係（9.1）。

**討論問題**

44. 去畸變前後，影像邊緣的直線變化有多明顯？若不做去畸變就拿去跑視覺 SLAM，你預期地圖會出現什麼問題？

45. ORB 特徵點在你的教室裡集中在哪些地方？哪些區域幾乎沒有？這對「機器人該往哪看才不會跟丟」有什麼啟示？

46. （有 RGB-D 者）玻璃與鏡面的深度資料出了什麼問題？這與第 8 章光達的弱點是否相同？說明背後的共同原因。

47. 根據本章表 9-1，若要為一台室內服務機器人選相機，你會選單目、雙目還是 RGB-D?搭配 IMU 嗎？說明理由。

**延伸挑戰（大學部）**

其一：用校正後的相機，實作一個以 ArUco 標記估計相機位姿的程式（呼應第 4 章），把相機相對標記的 6-DoF 位姿即時顯示出來——這串起了第 4 章與本章。其二：若有雙目相機，實作視差計算（OpenCV 的 StereoBM / StereoSGBM），產生深度圖，並用 9.6 節公式驗證某已知距離物體的測深精度，觀察誤差是否隨距離平方成長。這份程式將直接銜接第 30 章。

## 延伸閱讀


55. R. Hartley and A. Zisserman, Multiple View Geometry in Computer Vision, 2nd ed., Cambridge University Press, 2004.（多視圖幾何的權威聖經，第 21 章的主要參考）
56. R. Szeliski, Computer Vision: Algorithms and Applications, 2nd ed., Springer, 2022.（電腦視覺的標準教科書，可免費取得，含相機模型與校正完整章節）
57. Z. Zhang, "A Flexible New Technique for Camera Calibration," IEEE TPAMI, 2000.（張氏棋盤格校正法原始論文，9.2 節依據）
58. OpenCV 官方文件之 Camera Calibration and 3D Reconstruction 教學（docs.opencv.org）.（附完整程式範例，實作首選）
59. L. Keselman et al., "Intel RealSense Stereoscopic Depth Cameras," CVPR Workshops, 2017.（RealSense 系列深度相機的原理與規格，9.3 節依據）
60. 高翔、張濤，《視覺 SLAM 十四講》，電子工業出版社.（華文世界最受歡迎的視覺 SLAM 入門書，含相機模型到 SLAM 的完整脈絡）
61. UZH Robotics and Perception Group, Vision Algorithms for Mobile Robotics 課程（第 2 章介紹）.（影像形成到視覺 SLAM 的完整免費課程）


## 第二篇完結：從感測器到數學

第二篇到此結束。六章走下來，機器人身上的感官已經齊備，而同學應該也發現了一個反覆出現的結構：每一種感測器都有它擅長的情境與必然失效的情境，而不同感測器的失效情境恰好互不重疊。

室內定位需要佈建，GNSS 需要天空，磁力計怕鋼筋，IMU 會漂移，光達沒有語意、在長廊中退化，相機沒有尺度、怕暗怕白牆。把它們列成一張表，會看到一個清楚的事實：沒有任何單一感測器能獨自完成導航，但它們的組合可以。

於是問題變成：當五個感測器同時給出互相矛盾的讀值時，機器人該相信誰？第 11 章的答案是機率——每個讀值帶著它自己的不確定度，融合的結果是不確定度最小的那個估計。而要讓這些讀值能被放進同一條式子裡比較，首先必須把它們表達在同一個座標系中，並用同一套數學描述「位置與朝向」。

這就是下一章的工作。第 10 章會介紹座標系統、旋轉的各種表示法（旋轉矩陣、歐拉角、四元數、李群）與剛體運動——它是全書數學密度最高的一章，也是從此之後每一章都要用到的工具。讀慢一點沒關係，但請務必讀懂。

## 小結

35. 針孔相機模型描述三維點如何投影為二維影像；投影公式的分母是深度 Z，意味著深度在成像時被「除掉」，這是視覺 SLAM 尺度不確定性的幾何根源。內參矩陣 K 是視覺 SLAM 最基礎的物件。
36. 真實鏡頭有徑向與切向畸變，會使直線變彎，破壞針孔模型假設；相機校正（如張氏棋盤格法）量測內參 K 與畸變係數，去畸變是所有視覺 SLAM 的第一步。
37. 三種相機各以不同方式取得深度：單目靠運動三角測量但無絕對尺度、雙目靠左右視差有絕對尺度但怕無紋理、RGB-D 主動投射取得稠密深度但怕強光與玻璃。
38. 視覺 SLAM 從影像提取三種資訊：特徵點（角點，當作路標）、邊緣（輪廓）、光流（像素運動）；深度學習的單目深度估測是新興補充。現代系統常混合使用。

39. 相機資訊最豐富、成本最低、被動省電輕巧；但有尺度不確定性、嚴重依賴光線與紋理（黑暗、白牆、模糊即失效）、運算負擔大、怕動態場景。其弱點與光達幾乎互補。
40. 知識階梯為：影像處理 → 特徵提取 → 多視圖幾何 → 視覺里程計 → 視覺 SLAM;視覺 SLAM 分特徵法（ORB-SLAM）與直接法（DSO）兩大流派。
41. 相機承載語意，是通往語意導航的窗口：YOLO、SAM、Depth Anything、CLIP 等 AI 模型與 VLM，能讓機器人從「知道形狀」進化到「理解場景、聽懂人話」，這是第 32 章的主題。

至此，第二篇「定位與導航感測器」全部完成——我們認識了機器人的所有感官。但感測器給的都是帶著雜訊的原始資料：里程計會漂移、光達有雜點、相機會誤匹配。下一篇「SLAM 數學基礎」將回答一個根本問題：面對這些不完美的量測，機器人如何用數學做出最合理的估測？我們將從座標系與剛體運動開始，一步步建立處理不確定性的數學工具。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第三篇 SLAM 數學基礎

## 第 10 章

## 座標系統與剛體運動

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

## 本章在全書的位置

第二篇把感測器都介紹完了，但它們各說各話——光達說「前方三公尺有牆」，相機說「影像座標 (320, 240) 有個角點」，IMU 說「繞自己的 X 軸轉了 0.3 度」。這些話連單位與參考基準都不同，根本無法放在一起比較。

第三篇要做的，就是讓它們說同一種語言。而本章是第一步，也是最基礎的一步：建立座標系統，並學會把任何一筆資料從一個座標系搬到另一個座標系。

這是全書數學密度最高的一章，但它的回報也最大。從下一章開始，每一條式子裡都會出現本章的符號：第 11 章的狀態 x 就是本章的位姿，第 12 章要最佳化的變數就是一串位姿，第 13 章把光達點標進地圖用的就是本章的 2D 變換，第 17 章的 ICP 求的就是一個 T，第 22 章 ORB-SLAM2 追蹤的也是每一幀的 T。可以說，本章之後的所有章節，都是在求各式各樣的 T。

**圖 10-0　第 10 章與全書的連結地圖**

【承接前章】

第 4 至 9 章　六類感測器，各自用各自的座標與單位說話

本　　章　建立共同語言：座標系、位姿、變換矩陣

【本章負責】

座標系的三個層次：世界　→　機器人　→　感測器

齊次座標：把「旋轉 + 平移」寫成一次矩陣乘法

旋轉的三種表示法與它們的取捨（表 10-1）

TF Tree：ROS 如何管理整棵座標樹

【往下延伸】

→ 第 11 章　位姿成為待估計的「狀態 x」

→ 第 12 章　位姿圖的節點就是本章的 T

→ 第 13 章　光達點進地圖：本章 10.3 節公式的直接應用

→ 第 17 章　ICP 配準：在兩片點雲之間求一個 T

→ 第 28、29 章　TF 樹出錯是 ROS 建圖最常見的故障

## 學習目標

第二篇認識了機器人的所有感官，但每個感測器都在「自己的座標系」中回報資料——相機說「前方兩公尺有東西」、光達說「我的左邊三公尺有牆」。要把這些資訊拼成一張一致的地圖，必須先學會在不同座標系之間精確轉換。本章是第三篇的第一站，也是整個 SLAM 數學的地基。學完本章之後，同學應能夠達成下列目標：

57. 說明世界、機器人、感測器三種座標系的關係，並解釋為何 SLAM 需要多個座標系。

58. 使用齊次座標把旋轉與平移統一表示為單一矩陣運算。

59. 寫出 2D 位姿（x、y、θ）的表示法與座標轉換公式。

60. 說明 3D 旋轉的三種表示法——旋轉矩陣、歐拉角、四元數——及其優缺點與適用場合。

61. 說明座標轉換的組合與反轉，理解 TF 座標樹的概念。

62. 說明 ROS 中 map、odom、base_link、camera_link 等標準座標框架的意義與關係。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 座標系 | Coordinate Frame | 描述位置與方向的參考基準，SLAM 中同時存在多個。 |
| 位姿 | Pose | 位置加姿態，2D 為（x、y、θ），3D 為位置加三軸旋轉。 |
| 剛體運動 | Rigid Body Motion | 不改變物體形狀與大小的運動，只含旋轉與平移。 |
| 齊次座標 | Homogeneous Coordinates | 多加一維的座標表示法，使旋轉與平移能以單一矩陣表示。 |
| 旋轉矩陣 | Rotation Matrix | 描述旋轉的正交矩陣，屬於特殊正交群 SO（3）。 |
| 歐拉角 | Euler Angles | 以滾轉、俯仰、偏航三個角度表示旋轉，直觀但有萬向鎖。 |
| 四元數 | Quaternion | 以四個數表示旋轉，無奇異點、運算高效，SLAM 的標準表示。 |
| 變換矩陣 | Transformation Matrix | 4×4 矩陣，同時包含旋轉 R 與平移 t，屬於 SE（3）。 |
| 萬向鎖 | Gimbal Lock | 歐拉角在特定姿態下喪失一個自由度的奇異現象。 |

| 中文名詞 | 英文名詞 | 概念說明 |
| --- | --- | --- |
| 座標轉換樹 | TF Tree | ROS 中管理各座標框架關係的樹狀結構。 |
| 里程計座標 | odom frame | 連續平滑但長期漂移的參考框架。 |
| 地圖座標 | map frame | 不漂移但可能跳變的全域參考框架。 |

## 10.1 世界座標、機器人座標、感測器座標

想像一台機器人在教室裡，一顆相機裝在它的頭部，拍到「正前方兩公尺處有一張椅子」。這句話裡藏著一個問題：「正前方」是相對於誰？相對於相機、相對於機器人的身體、還是相對於教室？這三者的答案都不一樣——機器人一轉身，相對相機的「前方」變了，但椅子相對教室的位置沒變。這就是為什麼 SLAM 必須同時處理多個座標系。

### 三個層次的座標系

- 世界座標系（World Frame）：固定不動的全域參考，通常以機器人開機的起點或地圖的某個角落為原點。地圖上所有東西的「絕對位置」都以它為基準。它是 SLAM 最終想把一切統一過去的座標系。
- 機器人座標系（Robot / Body Frame）：固定在機器人身上、隨機器人一起移動的座標系，原點通常設在底盤中心。在 ROS 中稱為 base_link。機器人的運動，本質上就是這個座標系相對世界座標系的移動。
- 感測器座標系（Sensor Frame）：固定在各感測器上。相機有相機座標系、光達有光達座標系，原點在各感測器的量測中心。感測器直接量到的資料，都在自己的座標系中。

### 為什麼需要分這麼多層

關鍵在於：感測器裝在機器人上的位置是固定的（相機永遠在頭部前方 20 公分、光達永遠在底盤上方 15 公分），但機器人相對世界的位置卻時刻在變。把「固定的安裝關係」與「變動的機器人位姿」分開處理，數學才會清晰。

於是一筆感測器資料要進地圖，需要走兩步轉換：先從感測器座標系轉到機器人座標系（用固定的安裝關係，稱為外參，第 5 章提過），再從機器人座標系轉到世界座標系（用機器人當前位姿，這正是 SLAM 要估計的）。理解這個「感測器 → 機器人 → 世界」的轉換鏈，是讀懂所有 SLAM 系統的前提。本章接下來就是要把「轉換」這件事的數學工具備齊。

## 10.2 齊次座標

座標轉換包含兩種動作：旋轉（轉個方向）與平移（移個位置）。麻煩的是，旋轉是「乘法」（矩陣乘向量）、平移是「加法」（向量加向量），兩者形式不同，串接多次轉換時會變得很囉嗦。齊次座標（Homogeneous Coordinates）是一個優雅的技巧，能把兩者統一成單一的矩陣乘法。

### 多加一維的魔法

做法很簡單：在原本的座標後面多補一個 1。一個 2D 點（x、y）寫成（x、y、1），一個 3D 點（x、y、z）寫成（x、y、z、1）。

這樣做的好處是，平移也能寫進矩陣裡了。以 2D 為例，「旋轉角度 θ 再平移（tx、ty）」可以合併成單一的 3×3 矩陣乘法：

$$[x';y';1] = [\cos\theta \;\; -\sin\theta \;\; tx;\sin\theta \;\; \cos\theta \;\; ty;0 \;\; 0 \;\; 1]\cdot[x;y;1]$$

這個 3×3 矩陣稱為 2D 的變換矩陣（或齊次變換矩陣）。左上角 2×2 是旋轉、右上角是平移、底下補一列（0 0 1）。原本要「先乘旋轉、再加平移」兩個步驟，現在一次矩陣乘法就完成。

### 為什麼這件事很重要

- 串接變換變成連乘：機器人經過多段移動，或資料經過「感測器→機器人→世界」多層轉換，只要把各段的變換矩陣依序相乘即可，不必分別處理旋轉與平移。
- 反向變換就是反矩陣：要把世界座標轉回機器人座標，只需求變換矩陣的反矩陣。
- 程式實作統一：所有座標處理都是矩陣乘法，程式碼簡潔、不易出錯，也便於 GPU 加速。

3D 的情況完全類似，只是變成 4×4 矩陣：左上角 3×3 是旋轉矩陣 R、右上角 3×1 是平移向量 t、底下補一列（0 0 0 1）。這個 4×4 矩陣是 SLAM 中最核心的數學物件之一，它同時裝著「機器人朝哪、在哪」的完整位姿資訊。

**觀念釐清：變換矩陣的兩種讀法**

同一個變換矩陣 T，可以有兩種解讀，初學者常混淆：

① 座標轉換：把「同一個點」在 A 座標系的座標，換算成在 B 座標系的座標。

② 剛體運動：把「一個點」實際移動、旋轉到新位置（在同一座標系中）。

兩者數學形式相同，但意義相反（把座標系轉一個方向，等於把點反向轉）。

SLAM 中大多是第①種——我們不是真的移動世界，而是換個座標系描述同一件事。

## 10.3 2D 位姿：x、y、θ

掃地機器人、教學小車都在平面上運動，只需 2D 位姿就足以描述。這是理解 3D 的最佳起點，因為 2D 只有一個旋轉自由度，一切都很直觀。

### 三個數描述一切

平面上一台機器人的位姿，用三個數就能完整描述：

- x、y：機器人在世界座標系中的位置。
- θ：機器人的朝向角（航向），即機器人的「正前方」與世界座標 X 軸的夾角。

這正是第 3 章航位推算、第 14 章 Gmapping 粒子所估計的量。每一個粒子、每一個位姿節點，在 2D SLAM 中就是這樣一組（x、y、θ）。

### 2D 座標轉換的實例

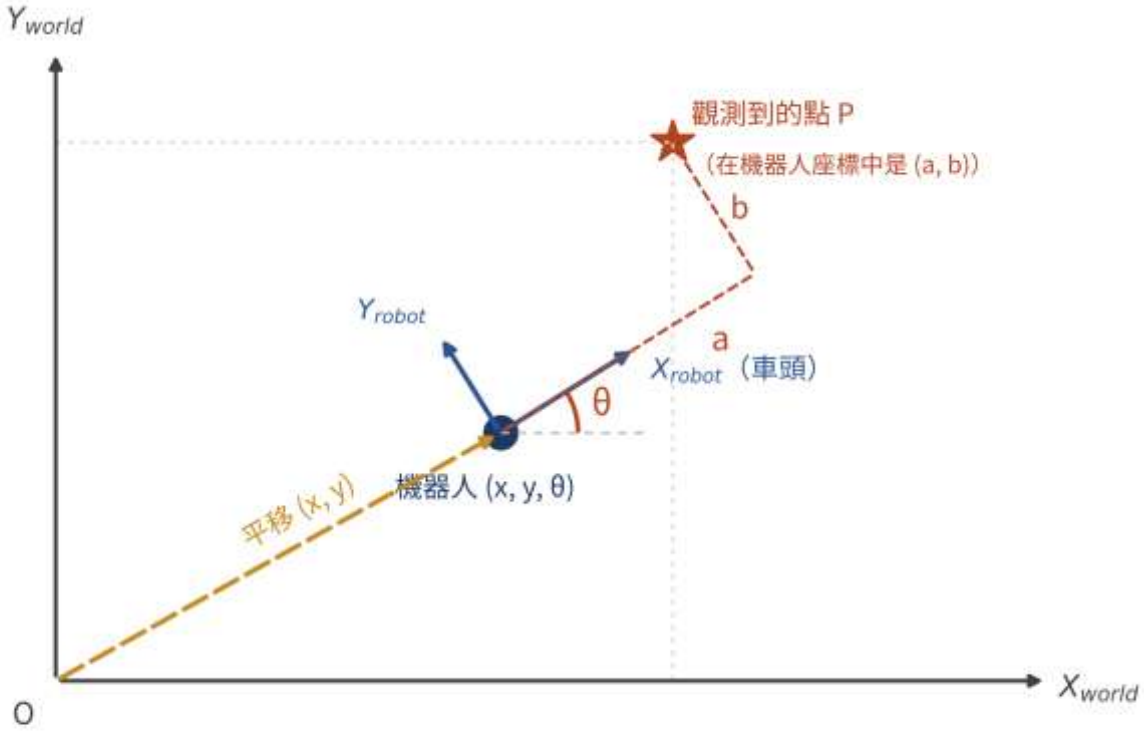
假設光達在機器人座標系中量到一個點 p_robot =（a、b）（機器人前方 a、左方 b），而機器人在世界座標系中的位姿是（x、y、θ），那麼這個點在世界座標系中的位置為：

$$x_world = x + a \cdot \cos\theta - b \cdot \sin\theta$$

$$y_world = y + a \cdot \sin\theta + b \cdot \cos\theta$$

這正是第 8 章「把一圈光達距離值畫進地圖」時，每個點都要做的計算。用齊次座標寫，就是前一節那個 3×3 變換矩陣乘上（a、b、1）。同學可以驗證：當 θ = 0（機器人朝正東）、位姿在原點時，公式退化為 x_world = a、y_world = b，符合直覺。

**圖 10-1　2D 剛體變換：把機器人座標中的點搬到世界座標**




$x_{world} = x + a\cos\theta - b\sin\theta$　　$y_{world} = y + a\sin\theta + b\cos\theta$

＝「先把點旋轉 θ，再平移 (x, y)」——這就是 2D 剛體變換

這張圖是全書使用頻率最高的一組式子的幾何來源。機器人位於世界座標的 (x, y)，車頭朝向與世界 X 軸夾角 θ。它用光達量到一個點，在自己的座標系中記為 (a, b)——a 是前方多遠、b 是左方多遠。問題是：這個點在世界地圖的哪裡？

答案分兩步。先旋轉：機器人的座標軸相對世界轉了 θ，所以 (a, b) 要先乘上旋轉矩陣，得到 (a·cosθ − b·sinθ, a·sinθ + b·cosθ)。再平移：加上機器人自己的位置 (x, y)。合起來就是圖中下方那兩條式子。

請務必記住順序是「先轉後移」而不能顛倒——這一點在第 17 章的 ICP 與第 22 章的位姿

更新中會反覆出現。顛倒順序會得到完全不同的結果，而且錯誤往往不明顯：機器人在原點附近時幾乎看不出來，一旦走遠就整張地圖歪掉。許多同學在第 29 章建圖時遇到「地圖隨著走動愈歪愈誇張」，根源就在這裡。

> **一個把觀念串起來的練習**
>
> 把這兩條式子倒過來想：如果已知點在世界的位置與機器人的位姿，要算它在機器人座標中的 (a, b)，該怎麼做？
>
> 答案是乘上反變換——而 10.6 節會告訴你，反變換不必真的求反矩陣，只要 $T^{-1} = [\, R^{T} \quad -R^{T}t \,]$。
>
> 這兩個方向在 SLAM 中都用得到：建圖時用正向（把觀測搬進地圖），定位時用反向（把地圖點搬進感測器視角去比對）。第 13 章與第 14 章正是這兩者的對照。

### 角度的兩個實務陷阱

- 角度環繞（Angle Wrapping）：θ 是週期性的，+180 度與 -180 度是同一個方向。計算兩個角度的差時，必須把結果規範回$(-\pi, \pi]$區間，否則會出現「轉了 350 度」這種本該是「反向轉 10 度」的荒謬結果。這是 2D SLAM 程式最常見的 bug 之一。
- 弧度與角度：數學運算一律用弧度（radian），但人類閱讀習慣用度（degree）。程式中務必統一單位，混用會造成難以察覺的錯誤。

## 10.4　3D 位姿：位置與姿態

無人機、機器狗、手持相機在三維空間中運動，需要完整的 3D 位姿。3D 位置很簡單，就是（x、y、z）三個數；真正複雜的是 3D 姿態——描述「朝哪」需要三個旋轉自由度，而表示這三個自由度的方法，竟有好幾種，各有優劣。這是本章的數學重點。

### 六個自由度

剛體在三維空間中共有六個自由度（6-DoF）：三個平移（沿 X、Y、Z 移動）加三個旋

轉（繞 X、Y、Z 轉動）。第 7 章、第 23 章反覆出現的「6-DoF 位姿」，指的就是這六個數。位置的三個自由度沒有爭議，以下三節專注於旋轉的三種表示法。

為什麼旋轉這麼麻煩？因為旋轉不是普通的量。兩個平移可以隨意相加、順序無關；但兩個旋轉的合成與順序有關——先繞 X 轉 90 度再繞 Y 轉 90 度，和先繞 Y 再繞 X，結果完全不同（同學可拿手機實際轉轉看）。這種「不可交換性」使得旋轉的數學遠比平移微妙，也是第 2 章 MIT VNAV 課程強調「流形上的最佳化」的原因——旋轉構成的空間是彎曲的，不是平坦的向量空間。

## 10.5 旋轉矩陣、歐拉角、四元數

這是本章最重要的一節。3D 旋轉有三種主要表示法，每一種都在 SLAM 系統中出現，同學必須理解它們各自的角色。

### 一、旋轉矩陣（Rotation Matrix）：理論的基石

用一個 3×3 的矩陣 R 表示旋轉。它把一個向量旋轉到新方向，運算就是矩陣乘向量。旋轉矩陣有嚴格的數學性質：它是正交矩陣（各列互相垂直且長度為 1）、行列式為 +1，這類矩陣構成的集合稱為特殊正交群 SO（3）。

- 優點：數學性質完美、直接可用於座標轉換、無奇異點、旋轉合成就是矩陣相乘。
- 缺點：用 9 個數表示只有 3 個自由度的旋轉，冗餘度高；儲存與最佳化時，還必須維持「正交」這個約束，不能自由地對 9 個數做加減（否則就不再是合法旋轉）。

### 二、歐拉角（Euler Angles）：最直觀但有陷阱

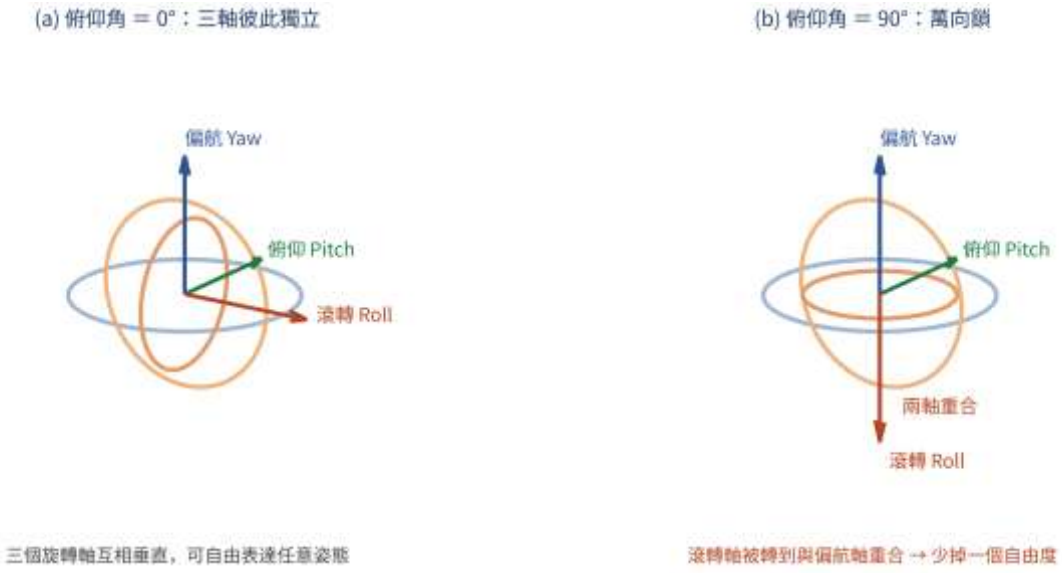
用三個角度——滾轉 φ、俯仰 θ、偏航 ψ（第 7 章介紹過）——表示旋轉。這是最貼近人類直覺的方式，「機頭上仰 10 度、右傾 5 度、朝向東方」一聽就懂。

- 優點：直觀、只用 3 個數（無冗餘）、便於人類理解與介面顯示。

- 缺點：有萬向鎖（Gimbal Lock）——當俯仰角接近 ±90 度時，滾轉與偏航的旋轉軸重合，喪失一個自由度（第 7 章詳述）；此外，旋轉合成不能直接相加，且同一旋轉可能有多組角度表示（週期性），數值最佳化時容易出問題。

因此歐拉角適合「給人看」（介面顯示、除錯），但不適合「給機器算」（內部計算）。這是一個重要的工程分工。

**圖 10-2　萬向鎖：歐拉角為什麼會失去一個自由度**




萬向鎖是歐拉角最惡名昭彰的缺陷，但它其實不難理解——只要看到那三個軸的關係就夠了。左圖中俯仰角為零，偏航、俯仰、滾轉三個旋轉軸互相垂直，任意姿態都能以三個角度唯一表達。

右圖把俯仰角轉到 90 度。注意發生了什麼事：滾轉軸被俯仰軸帶著一起轉，轉到與偏航軸重合了。於是「改變偏航角」與「改變滾轉角」產生完全相同的運動——三個參數只剩下兩個有效的自由度，姿態空間中有一整個方向變得無法表達。

這不是實作上的瑕疵，而是數學上的必然：任何用三個數描述三維旋轉的方法，都一定存在奇異點。想避開它，就必須用超過三個數——旋轉矩陣用九個（帶六個約束），四元數用四個（帶一個約束）。這就是 10.5 節為什麼說四元數是 SLAM 的實戰標準。

案例 10-1 的阿波羅 11 號導航平台正是栽在這件事上。而在機器人身上，它一樣會出事：無

人機做大角度俯衝或爬升、機械手臂的關節轉到特定姿態、四足機器人翻倒時，只要姿態以歐拉角儲存，就可能出現數值爆炸或姿態跳變。第 26 章討論無人機飛控時會再遇到它。

### 三、四元數（Quaternion）：SLAM 的實戰標準

四元數用四個數（w、x、y、z）表示旋轉，可以理解為「繞某個軸轉某個角度」的一種緊湊編碼。它初看不直觀，卻是現代 SLAM、無人機、遊戲、動畫內部一致採用的旋轉表示。

- 優點：無萬向鎖（沒有奇異點）、只用 4 個數（冗餘極小）、旋轉合成只需四元數乘法（運算高效）、數值穩定、便於插值（平滑地從一個姿態轉到另一個）。
- 缺點：不直觀，人難以直接讀懂（w、x、y、z）代表什麼方向；需要維持「單位長度」的約束（但只有一個約束，遠比旋轉矩陣的正交約束簡單）。

實務上的分工非常清楚：內部計算與儲存用四元數、介面顯示與人工判讀轉成歐拉角、座標轉換運算轉成旋轉矩陣。三者可以互相轉換，ROS 的 tf2 與各種數學函式庫都內建轉換函式。同學不必背誦四元數的乘法規則，但必須理解「為什麼實務都用它」——答案就是：它同時避開了旋轉矩陣的冗餘與歐拉角的萬向鎖。

| 表示法 | 數字個數 | 奇異點 | 主要優點 | 主要用途 |
|---|---|---|---|---|
| 旋轉矩陣 | 9(3×3) | 無 | 座標轉換直接、理論完美 | 座標變換運算 |
| 歐拉角 | 3 | 有（萬向鎖） | 最直觀、便於人讀 | 介面顯示、除錯 |
| 四元數 | 4 | 無 | 緊湊、高效、便於插值 | 內部計算與儲存 |

**表 10-1　三種 3D 旋轉表示法比較**

## 10.6　座標轉換與 TF Tree

有了變換矩陣這個工具，現在來看真實機器人系統如何管理眾多座標系之間的關係。答案是一個樹狀結構——TF 座標樹。

### 變換的組合與反轉

先複習兩個關鍵運算，它們是 TF 樹運作的基礎：

- 組合（Composition）：若已知「A 到 B 的變換」與「B 到 C 的變換」，把兩個變換矩陣相乘，就得到「A 到 C 的變換」。這讓我們能把感測器資料一路轉換到世界座標。
- 反轉（Inverse）：若已知「A 到 B 的變換」，求它的反矩陣，就得到「B 到 A 的變換」。

有了這兩個運算，只要知道任意兩個相鄰座標系之間的變換，就能算出任意兩個座標系之間的關係——這正是 TF 樹的核心思想。

### TF Tree：座標系的家譜

在一台真實機器人上，座標系可能有十幾個：世界、地圖、里程計、底盤、每個輪子、光達、相機、機械臂的每一節……ROS 用一個樹狀結構（TF Tree，座標轉換樹）來管理它們的關係。

- 每個座標系是樹上的一個節點。
- 每個父子節點之間，記錄著一個變換（父到子）。
- 查詢任意兩個座標系的關係時，系統自動沿著樹路徑把沿途的變換組合（相乘）起來。

這個設計的巧妙之處在於：每個變換只需由「最了解它的那個模組」負責發布。光達驅動發布「底盤到光達」的固定變換（來自安裝量測）、SLAM 系統發布「地圖到底盤」的變換（來自定位估計）、里程計發布「里程計到底盤」的變換。各模組各司其職，TF 樹把它們自動整合成完整的座標關係網。第 29 章實作時，同學會親手設定這棵樹。

### 時間戳記：一個關鍵細節

TF 不只管「空間關係」，還管「時間」。因為機器人在動，每個變換都是隨時間變化

的，TF 系統為每個變換都標上時間戳記，查詢時可以問「在 0.1 秒前那一刻，相機相對地圖在哪」。這對感測器融合至關重要——相機與光達的資料到達時間不同，必須用正確時刻的變換來對齊，否則高速運動時會產生明顯誤差。這呼應了第 5 章融合中「時間同步」的工程課題。

## 10.7　ROS 中的 map、odom、base_link、camera_link

ROS 為行動機器人定義了一套標準座標框架命名與階層，幾乎所有 ROS 導航系統都遵循它。理解這套慣例，是讀懂與實作 SLAM 系統的基本功。

### 標準框架階層

從上到下的標準鏈是：map → odom → base_link → 各感測器框架。

**圖 10-3　ROS 標準座標框架階層**

map（地圖座標系）：全域固定、不漂移，但會跳變

↓　（由 SLAM / 定位系統發布）

odom（里程計座標系）：連續平滑，但長期漂移

↓　（由里程計發布）

base_link（機器人本體）：固定在機器人底盤中心

↓　（由固定安裝關係發布，即外參）

laser / camera_link / imu_link（各感測器座標系）

### 為什麼要 map 和 odom 兩層

這是整套設計最精妙、也最容易困惑的地方。為什麼不直接用一個座標系，要分成 map 與 odom 兩層？因為它們各自解決不同的需求，而且是互補的：

- odom（里程計座標系）：由里程計與 IMU 積分得到。它的特性是連續、平滑、不跳變——非常適合短期的運動控制與避障。但如第 3、7 章所述，它會長期漂移，越走越不準。
- map（地圖座標系）：由 SLAM 或定位系統（如 AMCL）得到。它的特性是長期正確、不漂移——因為它靠比對地圖來校正。但它會跳變：當 SLAM 突然發現「我其實在這裡」而修正位置時，機器人在 map 中的座標會瞬間跳動一下。

兩層並存的智慧在於：讓需要「平滑」的模組（如避障控制）用 odom，讓需要「準確」的模組（如全域路徑規劃）用 map。SLAM 系統實際發布的，是「map 到 odom」的變換——它不斷微調這個變換來補償里程計的漂移，而 base_link 相對 odom 的運動仍然平滑。這個設計讓「平滑」與「準確」兩個看似衝突的需求，優雅地共存。

### 感測器框架與外參

base_link 之下的每個感測器框架（laser、camera_link、imu_link 等），相對 base_link 的變換是固定的，由感測器在機器人上的安裝位置決定，也就是第 5 章提過的外參。這些變換通常寫在機器人的 URDF 描述檔中（第 29 章），或由靜態變換發布器發布。準確量測這些外參，是系統精度的基礎——安裝位置差幾公分或幾度，建出的地圖就會模糊或錯位。

至此，同學已經備齊了 SLAM 的第一組數學工具：如何在座標系之間精確地轉換位姿。但真實世界的量測都帶著雜訊——里程計會滑、光達有誤差、相機會誤匹配。下一章「機率、雜訊與狀態估測」，將教我們如何用機率的語言描述這些不確定性，並在其中做出最合理的估計。這是 SLAM 從「幾何」邁向「估測」的關鍵轉折。

## 公式或流程

### 一、3D 齊次變換矩陣

3D 剛體運動以 4×4 齊次變換矩陣 T 表示，同時包含旋轉 R（3×3）與平移 t(3×1):

$$T = [\,R\ \ t\,;\,0\ \ 1\,]\quad（左上 3×3 為 R，右上 3×1 為 t，左下為 0，右下為 1）$$

把 A 座標系中的點 p_A（以齊次座標表示）轉到 B 座標系：

$$p_B = T_BA \cdot p_A$$

這裡 T_BA 讀作「從 A 到 B 的變換」。下標的書寫順序是 SLAM 中最容易出錯的地方，建議固定一種慣例並貫徹到底。

## 二、變換的組合與反轉

組合：把 A→B 與 B→C 的變換串接成 A→C:

$$T_CA = T_CB \cdot T_BA$$

注意相乘順序：相鄰的下標要對齊（B 與 B 相鄰）。反轉：

$$T_AB = (T_BA)^{-1}$$

齊次變換矩陣的反矩陣有簡便公式，不需做一般的 4×4 反矩陣運算：

$$T^{-1} = [\,R^{T}\ \ -R^{T}t\,;\,0\ \ 1\,]$$

利用旋轉矩陣的正交性（反矩陣等於轉置），反轉變得非常高效。

## 三、2D 位姿複合（里程計串接）

機器人從位姿 $p_1 = (x_1, y_1, \theta_1)$ 經一段相對運動（$\Delta x, \Delta y, \Delta\theta$，在 $p_1$ 的機體座標系中）後，新位姿為：

$$x_2 = x_1 + \Delta x \cdot \cos\theta_1 - \Delta y \cdot \sin\theta_1$$

$$y_2 = y_1 + \Delta x \cdot \sin\theta_1 + \Delta y \cdot \cos\theta_1$$

$$\theta_2 = \mathrm{normalize}(\theta_1 + \Delta\theta)\quad（\mathrm{normalize}\ 表示規範回\ (-\pi, \pi]）$$

這是第 3 章航位推算、第 14 章 Gmapping 運動更新的具體實現。normalize 步驟即 10.3 節強調的角度環繞處理，不可省略。

## 四、感測器資料進地圖的完整流程

**圖 10-4　一筆感測器量測進入世界地圖的轉換鏈**

① 感測器量測：點 p 在感測器座標系（如 camera_link）

↓ × T（base_link ← camera_link）　外參，固定

② 點 p 在機器人座標系（base_link）

↓ × T（odom ← base_link）　里程計

③ 點 p 在里程計座標系（odom）

↓ × T（map ← odom）　SLAM 修正

④ 點 p 在世界地圖座標系（map）→ 寫入地圖

整條鏈就是多個變換矩陣的連乘，TF 樹自動完成這個計算。

## 10.8　旋轉表示法的當代選擇：從工程到學習

10.5 節比較了旋轉矩陣、歐拉角與四元數。這個比較在傳統 SLAM 的脈絡下已經很完整，但近年有兩個發展值得補充——一個關於工程實作，一個關於深度學習。

### 一、量化的比較：各表示法的真實代價

近年有研究對七種主要的三維旋轉表示法做了系統性的比較，涵蓋歐拉角、軸角向量、四元數、旋轉矩陣、指數映射、六維連續表示法，以及矩陣 Fisher 與 Bingham 等機率式表示法，並從連續性、萬向鎖、運算效率、儲存量與內插品質等面向評估。

其結論與本書 10.5 節一致：就一般用途而言，四元數取得了最佳的整體平衡——運算效率高、儲存精簡、可用球面線性內插（SLERP）平滑過渡，且沒有萬向鎖。具體數字上，四元數的複合運算約 34 微秒、佔用 32 位元組；旋轉矩陣約 306 微秒、72 位元組；六維連續表

示法約 422 微秒、48 位元組。可見四元數在複合運算上快了近一個數量級，這在每秒要做數萬次位姿複合的 SLAM 後端中是實質的差異。

## 二、但在神經網路裡，四元數不是最好的

第二個發展與第 32 章關係密切。當機器人改用神經網路直接預測姿態時——例如從影像回歸相機位姿、或讓視覺語言動作模型輸出機械手的目標姿態——四元數反而成了問題。

原因是連續性。神經網路以梯度下降訓練，要求輸出空間對輸入的變化是連續的；但四元數有「雙重覆蓋」的性質（q 與 -q 代表同一個旋轉），歐拉角更有萬向鎖的跳變。這些不連續處會讓網路在訓練時收到互相矛盾的梯度，學不穩定。

解法是改用六維連續表示法：輸出旋轉矩陣的前兩個欄向量，再以施密特正交化還原出完整的旋轉矩陣。這個表示法在旋轉空間上是連續的，沒有任何奇異點。上述研究指出，在神經網路的姿態估測任務中，它相較四元數可降低約百分之六至十四的誤差。

這帶出一個對同學很重要的觀念：沒有「最好的旋轉表示法」，只有「對這個任務最好的表示法」。SLAM 後端要快速複合，所以用四元數；神經網路要連續可微，所以用六維表示；人要看得懂，所以在使用者介面上顯示歐拉角。同一個旋轉，依用途換不同的衣服。

## 三、李群：讓最佳化在流形上正確進行

還有一條在現代 SLAM 中已成標準、但入門教材常略過的路線：把旋轉與位姿視為李群（SO(3) 與 SE(3)），把它們的微小變化視為李代數上的向量。

為什麼需要這個？因為第 12 章的最佳化要對位姿求導數、要算「兩個位姿相差多少」、要做「位姿加上一個微小增量」。但旋轉矩陣不能直接相加——兩個旋轉矩陣相加之後不再是旋轉矩陣。李代數提供了正確的做法：在流形的切空間上做加減與求導，再用指數映射送回流形。

實務上同學不必從頭推導——Sophus、manif 這類函式庫已經把 SO(3)、SE(3) 的運算封裝好，g2o、Ceres、GTSAM 等最佳化框架也都內建了流形上的參數化。第 12 章與第 23 章

會再具體用到。這裡只要先記住一句話：旋轉不是向量，不能直接加。本章 10.3 節提醒的「角度不能直接平均」，正是這句話在二維的簡化版。

**一張選用速查表**

使用者介面、規格書、人要讀的數字　→　歐拉角（但內部絕不以它儲存）

SLAM 內部狀態、感測器驅動、ROS 訊息　→　四元數

最佳化的求導與增量更新　→　李代數（so(3) / se(3)）

神經網路的輸出層　→　六維連續表示法

點雲與幾何運算　→　旋轉矩陣（直接可乘）

## 案例

### 案例 10-1　阿波羅 11 號的萬向鎖驚魂

萬向鎖不是抽象的數學名詞，它曾在人類登月任務中造成真實危機。阿波羅太空船的慣性導航平台用三個機械環（萬向架）支撐，以歐拉角的方式追蹤太空船姿態。當三個環接近共面時，就會發生萬向鎖——喪失一個旋轉自由度，導航平台可能失去姿態基準。

工程師深知這個風險，在儀表板上設計了警示，並規劃飛行姿態盡量避開危險區域。太空人麥可·柯林斯曾半開玩笑地抱怨這個限制。事後來看，這正是「用歐拉角表示姿態」的先天缺陷在極端任務中的體現。現代太空船、無人機、機器人早已改用四元數，正是為了根除這個問題。

這個案例給同學的啟示是：表示法的選擇不是純粹的數學品味，它有真實的工程後果。10.5 節看似枯燥的三種旋轉表示法比較，背後是無數工程師用教訓換來的智慧——這也是為什麼本書堅持在數學章節中講清楚「為什麼這樣選」，而不只是「怎麼算」。

### 案例 10-2　RViz 裡的座標樹：讓抽象變具體

第一次接觸 TF 樹的同學，常覺得它抽象難懂。但在 ROS 的視覺化工具 RViz 中，座標樹會變成看得見的東西：每個座標系顯示為一組紅綠藍的小軸（對應 X、Y、Z），當機器人移動，base_link 的軸就跟著動；當光達安裝在機器人前方，laser 的軸就固定顯示在 base_link 前方一小段距離。

更有教育意義的是「觀察錯誤」。如果外參設錯了（例如把光達的安裝方向填反），RViz 中光達掃描的點雲就會與實際環境「錯位」——牆明明在前方，點雲卻畫在後方或側面。同學透過調整外參、即時觀察點雲是否貼合環境，能非常直觀地理解座標轉換的意義。這是把 10.1 到 10.7 節所有抽象概念一次串起來的最佳實驗，第 29 章實作會大量用到。

大學課堂上一個常見的除錯口訣是：「地圖歪了先看 TF」。許多 SLAM 建圖失敗，根源不在演算法，而在座標框架設定錯誤——外參填錯、時間戳記沒對齊、TF 樹斷裂。理解本章，能幫同學省下無數除錯的冤枉路。

### 案例 10-3　機械手臂的正向運動學：同一套數學的另一個舞台

本章的座標變換數學，不只用於行動機器人，也是機械手臂的核心。一支六軸機械手臂，從基座到夾爪之間有六個關節，每個關節的轉動都是一個旋轉變換。把基座到第一節、第一節到第二節……一路到夾爪的變換矩陣依序相乘，就能算出「給定六個關節角度，夾爪在空間中的位姿」——這稱為正向運動學（Forward Kinematics）。

這正是 10.6 節「變換組合就是矩陣連乘」的直接應用。同學若在大學同時修習機械手臂相關課程，會發現兩門課用的是同一套齊次變換數學——這並非巧合，而是因為「剛體在空間中如何運動」是機器人學共通的語言。學好本章，等於同時打下了行動機器人與機械手臂的數學基礎。這也呼應第 2 章各校課程「感測器→數學→系統」的共同架構：數學是所有機器人技術的公分母。

## 實作活動

## 實作 10-1 座標轉換動手算與 TF 樹觀察

目的：透過紙筆計算與程式驗證，掌握 2D 座標轉換；並在 ROS 中觀察真實的 TF 樹，把抽象概念變具體。本實作分紙筆、程式、ROS 三部分，適合不同深度的教學。

### 第一部分：紙筆計算（全體）

62. 一台機器人在世界座標系中位姿為（x=3, y=2, θ=90°）。它的光達在機體座標系中量到一個點（前方 a=2，左方 b=0）。用 10.3 節公式手算這個點的世界座標，並在方格紙上畫圖驗證答案合理。

63. 同一台機器人，光達量到另一點（a=1, b=1）。手算其世界座標。畫圖確認。

64. 角度環繞練習：機器人朝向 $\theta_1$=170°，右轉 Δθ=30°。直接相加得 200°，把它規範回(-180°, 180°]應是多少？說明為何不處理會出問題。

### 第二部分：Python 驗證（全體）

65. 用 numpy 建立 10.2 節的 3×3 齊次變換矩陣，把第一部分的計算改用矩陣乘法完成，驗證結果一致。

66. 實作變換的組合：給定「世界→機器人」與「機器人→光達」兩個變換，相乘得到「世界→光達」，並驗證。

67. 實作反轉：求「世界→機器人」變換的反矩陣，驗證它等於「機器人→世界」變換（用 10.公式的簡便法與 numpy 直接求逆兩種方式比較）。

68. （大學部）實作歐拉角、旋轉矩陣、四元數三者的互相轉換（可用 scipy.spatial.transform.Rotation），取一個俯仰角接近 90° 的姿態，觀察歐拉角表示如何出現異常，親身驗證萬向鎖。

### 第三部分：ROS TF 樹觀察（有 ROS 環境者）

69. 啟動 TurtleBot3 模擬（第 28 章）或任一含 TF 的 ROS 系統，用 view_frames 或 rqt_tf_tree 工具產生 TF 樹的圖，找出 map、odom、base_link 與各感測器框架的階層關係。

70. 在 RViz 中顯示 TF，移動機器人，觀察哪些座標軸跟著動（base_link 及其子框架）、哪些不動（map）。
71. 用 tf2 的查詢工具，查詢「camera_link 相對 map」的變換，並在機器人移動前後各查一次，觀察數值變化。

**討論問題**

58. 第一部分中，機器人朝向 90°（正北）時，它機體的「前方」對應世界座標的哪個方向？你的計算結果符合直覺嗎？
59. 為什麼 SLAM 系統發布的是「map→odom」變換，而不是直接發布「map→base_link」？這樣設計對「平滑」與「準確」的兼顧有何幫助？
60. （大學部）你在萬向鎖實驗中觀察到什麼異常？如果無人機的飛控用歐拉角而非四元數，在做翻滾特技時可能發生什麼事？
61. 若一台機器人建出的地圖總是「歪斜」或「牆的點雲對不上」，依本章知識，你會優先檢查哪些地方？

**延伸挑戰（大學部）**

撰寫一個 ROS 2 節點，發布一個自訂感測器相對 base_link 的靜態變換（故意設一個已知的旋轉與平移），在 RViz 中顯示該感測器的模擬點雲，調整外參數值並即時觀察點雲位置變化，直到點雲貼合預期位置。這個練習直接對應第 29 章設定機器人 TF 的實務，能為 2D SLAM 小車實作打下座標基礎。

## 延伸閱讀

48. 高翔、張濤，《視覺 SLAM 十四講》第 3 講（三維空間剛體運動）、第 4 講（李群與李代數），電子工業出版社.（華文世界最清楚的 SLAM 旋轉數學入門）

49. K. M. Lynch and F. C. Park, Modern Robotics: Mechanics, Planning, and Control, Cambridge University Press, 2017.（剛體運動與座標變換的權威教科書，有免費電子版與線上課程）

50. J. Solà, J. Deray, and D. Atchuthan, "A Micro Lie Theory for State Estimation in Robotics," arXiv, 2018.（旋轉群與流形最佳化的友善入門，銜接第 12 章）

51. ROS 官方文件之 tf2 教學（docs.ros.org）.（TF 座標樹的實作指引，附完整範例）

52. REP 105: Coordinate Frames for Mobile Platforms(ros.org/reps).（ROS 標準座標框架 map / odom / base_link 的官方定義文件）

53. 3Blue1Brown, "Quaternions and 3D rotation" 系列影片（YouTube）.（以絕佳動畫直觀解釋四元數，強烈推薦）

54. R. Siegwart et al., Introduction to Autonomous Mobile Robots, 2nd ed., MIT Press, 2011.（第 2、3 章含座標系與運動學的機器人學脈絡）

## 從確定到不確定：本章的位置回顧

本章做完了一件事：讓所有感測器的資料能表達在同一個座標系中。但它悄悄假設了一個在真實世界裡並不成立的前提——每一個數字都是精確的。

圖 10-1 的式子假設機器人的位姿 (x, y, θ) 是已知且正確的。但第 7 章告訴我們 IMU 會漂移，第 5 章告訴我們 GNSS 有 DOP 放大的誤差，第 13 章的輪式里程計會打滑。實際上機器人從來不確切知道自己在哪裡——它只有一個帶著不確定度的估計。

同樣地，圖 10-1 中「觀測到的點 P」也不是一個精確的點。第 8 章的光達有測距誤差與角解析度限制，第 9 章的相機有像素量化與特徵點定位誤差。每一筆觀測都是一團模糊。

於是第 11 章要把本章所有的等號換成機率分布：位姿不再是三個數，而是一個高斯分布；觀測不再是一個點，而是帶著共變異數的量測。而融合多個感測器的工作，就變成「把多個機率分布乘在一起，找出最可能的那個答案」。這就是貝氏濾波，也是卡爾曼濾波與粒子濾

波的共同骨架。本章的變換矩陣不會消失——它會成為機率模型中的運動模型與觀測模型。

## 小結

62. SLAM 同時處理世界、機器人、感測器三層座標系；感測器安裝關係固定（外參）、機器人位姿變動（SLAM 要估的量），一筆資料進地圖需經「感測器→機器人→世界」的轉換鏈。
63. 齊次座標多補一維，把旋轉（乘法）與平移（加法）統一為單一矩陣乘法；2D 用 3×3、3D 用 4×4 變換矩陣，使串接變換成為連乘、反轉成為求反矩陣。
64. 2D 位姿以（x、y、θ）三個數描述，座標轉換用含 cosθ、sinθ 的公式；實務上務必處理角度環繞（規範回 -π 到 π）並統一弧度與角度單位。
65. 3D 剛體有六個自由度（三平移三旋轉）；旋轉的合成不可交換，構成的空間是彎曲的，這是旋轉數學複雜與「流形最佳化」的根源。
66. 3D 旋轉三種表示法各有分工：旋轉矩陣（9 數、無奇異、用於座標轉換）、歐拉角（3 數、直觀但有萬向鎖、用於人機介面）、四元數（4 數、無奇異、高效、用於內部計算與儲存）。
67. TF 樹以樹狀結構管理眾多座標系，每個變換由最了解它的模組發布，查詢時自動沿路徑連乘；變換附時間戳記以支援感測器融合的時間對齊。
68. ROS 標準框架 map→odom→base_link→感測器：odom 連續平滑但漂移、map 準確但跳變；SLAM 發布「map→odom」變換來補償漂移，讓「平滑」與「準確」兩需求優雅共存。

本章備齊了 SLAM 的第一組數學工具——確定性的座標轉換。但真實世界充滿不確定性：里程計會滑、光達有雜點、相機會誤匹配。下一章「機率、雜訊與狀態估測」將引入機率的語言，教機器人如何描述「我大概在這裡，但不太確定」，並在雜訊中做出最合理的估計。這是 SLAM 從幾何邁向估測的關鍵一步，也是貝氏濾波、卡爾曼濾波與粒子濾波的共同

起點。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

## 第 11 章

## 機率、雜訊與狀態估測

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

## 本章在全書的位置

第 10 章讓所有感測器的資料能表達在同一個座標系中，但它假設每個數字都是精確的。本章要拆掉這個假設。

真實的機器人從來不確切知道自己在哪裡。輪子會打滑、IMU 會漂移、光達有測距誤差、相機的特徵點會定位不準。更麻煩的是，這些誤差還會互相矛盾——里程計說走了三公尺，光達比對說只走了二點七公尺，到底該信誰？

本章給的答案是：都信，但依照各自的可信度加權。而「可信度」要能被計算，就必須先把每一個估計都寫成機率分布。於是位姿不再是三個數，而是一個高斯分布；觀測不再是一個點，而是帶著共變異數的量測；融合不再是取平均，而是把分布相乘。

這是全書觀念上最關鍵的轉折。第 12 章的最佳化、第 14 章的 Gmapping、第 15 章的 Cartographer、第 23 章的 ORB-SLAM3，全部建立在本章的框架之上——它們的差別只在於用什麼方式表示那個分布、用什麼方式求出最可能的答案。

**圖 11-0 第 11 章與全書的連結地圖**

【承接前章】

第 10 章 位姿 = 三個（或六個）精確的數字

本 章 位姿 = 一個帶著不確定度的機率分布

【本章負責】

高斯分布與共變異數：用均值 + 橢圓描述「我大概在哪」

運動模型與觀測模型：預測與修正的兩個來源

貝氏濾波 → 卡爾曼 → EKF → 粒子濾波

【往下延伸】

→ 第 12 章 同一個問題的另一種解法：最佳化（濾波的對照組）

→ 第 13 章 佔據格地圖的每一格也是一個機率

→ 第 14 章 Gmapping = 粒子濾波 + 每顆粒子帶一張地圖

→ 第 16 章 AMCL：掃地機器人的重定位就是本章的 MCL

→ 第 23 章 ORB-SLAM3 的 MAP 估測是本章觀念的最佳化版

## 學習目標

上一章我們學會了在座標系之間精確轉換——那是「確定性」的數學。但真實世界沒有這麼乾淨：里程計會打滑、光達有雜點、相機會誤匹配。本章引入機率的語言，教機器人如何說出「我大概在這裡，但不太確定」，並在雜訊中做出最合理的估計。這是 SLAM 從幾何邁向估測的關鍵轉折，也是全書數學的核心。學完本章之後，同學應能夠達成下列目標：

63. 說明 SLAM 為何必須以機率方式處理不確定性，而非追求單一精確答案。
64. 解釋高斯分布的均值與變異數的意義，並理解共變異數如何描述二維以上的不確定性。
65. 說明運動模型與觀測模型如何以機率描述機器人的預測與感知。

66. 說明貝氏濾波「預測—更新」兩步驟的遞迴架構，理解它是所有濾波器的共同骨架。

67. 說明卡爾曼濾波器的運作，並解釋卡爾曼增益如何權衡預測與觀測。

68. 說明 EKF 如何處理非線性，以及 EKF-SLAM 的基本概念與限制。

69. 說明粒子濾波與蒙地卡羅定位的原理，並比較它與卡爾曼家族的差異。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
| --- | --- | --- |
| 機率分布 | Probability Distribution | 描述隨機變數各種可能取值的機率。 |
| 高斯分布 | Gaussian Distribution | 鐘形的常態分布，以均值與變異數完整描述。 |
| 均值 | Mean | 分布的中心，代表最可能的估計值。 |
| 變異數 | Variance | 分布的離散程度，代表不確定性的大小。 |
| 共變異數 | Covariance | 描述多維隨機變數之間的相關性與聯合不確定性。 |
| 運動模型 | Motion Model | 描述機器人執行控制後，狀態如何變化（含雜訊）。 |
| 觀測模型 | Observation Model | 描述在某狀態下，感測器會量到什麼（含雜訊）。 |
| 貝氏濾波 | Bayes Filter | 以預測與更新兩步驟遞迴估計狀態的通用架構。 |
| 卡爾曼濾波 | Kalman Filter | 線性高斯系統下的最佳遞迴估測器。 |
| 卡爾曼增益 | Kalman Gain | 決定相信預測還是觀測的權重。 |
| 擴展卡爾曼濾波 | EKF | 將非線性系統線性化後套用卡爾曼濾波。 |
| 粒子濾波 | Particle Filter | 以大量樣本近似任意分布的非參數濾波器。 |
| 蒙地卡羅定位 | MCL | 以粒子濾波在已知地圖中定位的方法，即 AMCL。 |

## 11.1 為什麼 SLAM 一定要處理不確定性

回想第 7 章那個令人印象深刻的實驗：一台完全靜止的裝置，靠 IMU 積分算出的「位置」在 60 秒內漂移了數公尺。再想第 4 章的 Wi-Fi 定位、第 5 章的 GPS 跳點、第 9 章相機的誤匹配——沒有任何一個感測器給的是「真值」，它們給的都是「帶著雜訊的猜測」。

面對這個現實，有兩種態度。天真的做法是：假裝感測器是準的，直接用它的讀值。這正是航位推算失敗的原因——把每個帶誤差的量測當成真值累加，誤差就永無止境地累積。

成熟的做法是：承認每個量測都不確定，並量化這個不確定性，然後在多個不確定的資訊之間做最佳的權衡。這就是機率機器人學（Probabilistic Robotics）的核心思想，也是現代 SLAM 的數學骨幹。

**關鍵觀念：估計的是「分布」而非「數值」**

這是全章最重要的觀念轉變。傳統思維認為「機器人在（3.5, 2.1）」；機率思維則認為「機器人最可能在（3.5, 2.1）附近，但也可能在周圍一個橢圓範圍內，越靠近中心可能性越高」。

換句話說，SLAM 估計的不是一個點，而是一整個機率分布——它同時告訴我們「最佳猜測」與「這個猜測有多可信」。這個「有多可信」的資訊至關重要，因為它決定了下一步該多相信這個估計。第 1 章給出的 SLAM 機率定義 p(x, m | z, u)，正是在說「給定所有觀測與控制，狀態的機率分布長什麼樣」。

**直觀比喻：蒙眼走路的人**

想像你蒙著眼在熟悉的房間走動。每走一步，你「大概」知道自己前進了多少（運動），

但不太確定——可能多一點少一點（運動雜訊）。這時你對自己位置的把握越來越

模糊。

突然你的手摸到了牆角（觀測）。這個明確的地標，讓你對位置的把握瞬間變清晰。

SLAM 就是在做這件事：走路時不確定性擴大（預測），摸到地標時不確定性收縮（更新）。

本章要學的，就是如何用數學精確地描述「模糊」與「變清晰」這兩個動作。

## 11.2 高斯分布、均值、變異數、共變異數

要用數學描述不確定性，首先需要一種「分布」。SLAM 中最常用的是高斯分布（Gaussian Distribution，又稱常態分布），因為它有兩個無可取代的優點：只需兩個參數就能完整描述、而且在各種運算下仍保持高斯形式（數學上極為友善）。

### 一維高斯：鐘形曲線

一維高斯分布就是熟悉的鐘形曲線，由兩個參數完整決定：

- 均值 μ(mu)：曲線的中心，代表「最可能的值」，也就是我們的最佳估計。
- 變異數 $\sigma^2$（sigma 平方）：曲線的胖瘦，代表不確定性的大小。變異數小，曲線又高又窄，表示很有把握；變異數大，曲線又矮又寬，表示很不確定。

標準差 σ 是變異數的平方根，與量測值同單位，更直觀。「機器人在 3.5 公尺處，標準差 0.2 公尺」意思是：最佳估計是 3.5，而真值有約 68% 的機率落在 3.3 到 3.7 之間（高斯分布的「一個標準差」範圍）。

### 多維高斯：從點到橢圓

機器人的位置是二維的（x、y），不確定性也就變成二維。此時均值是一個點（最可能

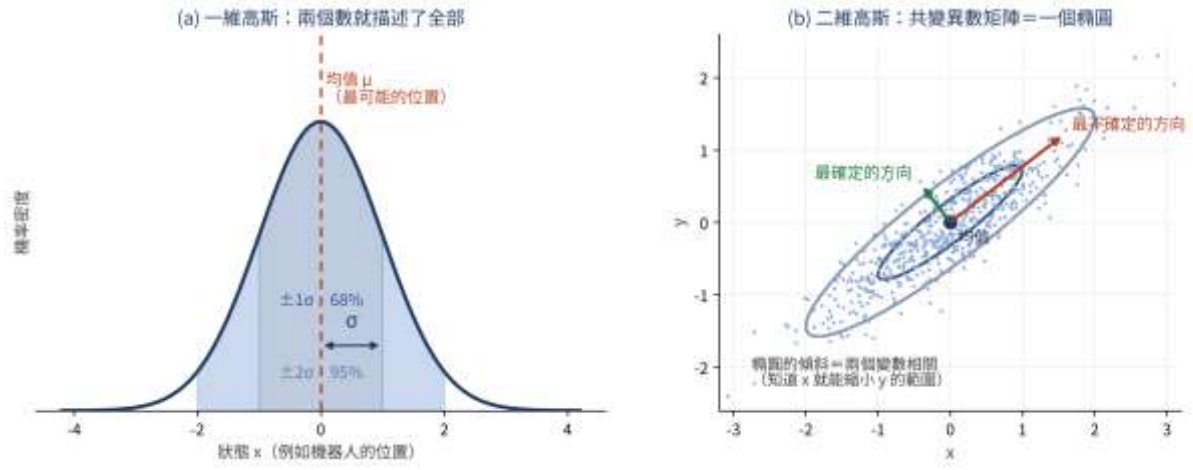
的位置），而不確定性用一個共變異數矩陣（Covariance Matrix）描述。

共變異數矩陣的幾何意義非常直觀：它對應一個不確定性橢圓。橢圓的中心是均值、大小反映不確定程度、形狀與方向反映「哪個方向比較不確定」。

- 對角線元素：各軸自己的變異數（x 方向多不確定、y 方向多不確定）。
- 非對角線元素（共變異數）：x 與 y 的相關性。若為正，表示「x 偏大時 y 也傾向偏大」，橢圓會傾斜。

舉個 SLAM 中的實例：一台機器人沿走廊直行，它對「走了多遠」（沿走廊方向）很不確定（誤差累積），但對「有沒有偏離走廊中線」（垂直方向）相當確定（牆壁一直在校正）。它的不確定性橢圓因此是一個「沿走廊方向拉長」的細長橢圓。這個橢圓會隨機器人運動與觀測不斷變形——SLAM 的每一步，本質上都在更新這個橢圓。

**圖 11-1　高斯分布：從一維鐘形曲線到二維不確定性橢圓**




左圖是一維的情況。整條曲線只由兩個數決定：均值 μ（最可能的位置）與標準差 σ（有多不確定）。68% 的機率落在 ±1σ 之內、95% 落在 ±2σ 之內——這兩個數字值得背起來，因為在實務上判斷「這個估計夠不夠準」時會不斷用到。

右圖是二維的情況，也是本書之後會反覆出現的那個橢圓。注意它是傾斜的——這代表 x 與 y 兩個方向的誤差互相關聯。紅色箭頭指向橢圓的長軸，是最不確定的方向；綠色箭頭指向短軸，是最確定的方向。

這個傾斜在 SLAM 中有具體的物理意義。機器人沿著走廊前進時，橫向位置可以靠兩側的牆面精確鎖定，但縱向位置卻只能靠里程計累積——於是不確定性橢圓會沿著走廊方向被拉長。第 17.5 節的「退化」、第 21 章的「遠處的點測不準」，畫出來都是這種被拉長的橢圓。

請特別記住一件事：共變異數矩陣不是一堆抽象的數字，它就是這顆橢圓。對角線元素決定橢圓在各軸向上的大小，非對角線元素決定它傾斜多少。日後看到 P、Σ 這些符號時，腦中應該直接浮現一個橢圓。

### 為什麼偏愛高斯

- 兩個參數搞定：均值與共變異數就完整描述了不確定性，儲存與計算都輕量。
- 運算封閉性：兩個高斯相乘仍是高斯、高斯經線性變換仍是高斯。這讓濾波器的每一步都能保持高斯形式，不會越算越複雜。這正是卡爾曼濾波如此優雅的數學根源。
- 中央極限定理：許多獨立的小誤差疊加，結果趨近高斯。感測器雜訊往往正是眾多微小因素的總和，因此高斯假設在實務上常常成立。

但要提醒：高斯假設並非萬能。當不確定性是「多峰」的（例如機器人可能在兩個長得一樣的房間之一，但不知是哪個），單一高斯就無法描述——這正是 11.7 節粒子濾波登場的原因。

## 11.3 感測器雜訊與運動模型

要把機器人放進機率框架，需要兩個「模型」，它們是所有濾波器的輸入。這兩個模型分別描述機器人如何「動」與如何「看」。

### 一、運動模型（Motion Model）

運動模型回答：「機器人執行了某個控制（如前進一公尺、左轉 30 度）後，新的位姿會是什麼？」關鍵在於——答案不是一個確定的位姿，而是一個機率分布，因為執行控制時會有雜訊（輪子打滑、地面不平）。

數學上寫成 p(x_t | x_{t-1}, u_t)：給定前一刻位姿 x_{t-1} 與控制 u_t，新位姿 x_t 的機率分布。運動模型的效果是讓不確定性擴大——每動一步，不確定性橢圓就膨脹一點，因為又累積了新的運動雜訊。這正是第 10 章 2D 位姿複合公式，加上一個雜訊項。

## 二、觀測模型（Observation Model）

觀測模型回答：「如果機器人真的在某個位姿，感測器應該量到什麼？」同樣地，答案是機率分布，因為感測器有雜訊。

數學上寫成 p(z_t | x_t)：給定位姿 x_t，觀測 z_t 的機率分布。例如「若機器人在此處，光達應該量到前方 2 公尺有牆」，但實際量測會在 2 公尺附近有些抖動。觀測模型的效果是讓不確定性收縮——一個明確的觀測（摸到牆角）能大幅縮小可能位置的範圍。

### 兩個模型的分工

這兩個模型恰好對應 11.1 節「蒙眼走路」比喻的兩個動作：運動模型讓不確定性擴大（走路時越來越模糊），觀測模型讓不確定性收縮（摸到地標時變清晰）。SLAM 的整個過程，就是這兩個模型交替作用——動一步、看一眼、動一步、看一眼。下一節的貝氏濾波，就是把這個「交替」用數學精確地表達出來。

| | 運動模型 | 觀測模型 |
|---|---|---|
| 數學形式 | p(x_t \| x_{t-1}, u_t) | p(z_t \| x_t) |
| 回答的問題 | 動了之後會到哪 | 在這裡會看到什麼 |
| 輸入 | 前一位姿 + 控制指令 | 當前位姿 |
| 雜訊來源 | 打滑、地面、輪徑誤差 | 感測器量測誤差 |
| 對不確定性 | 擴大（膨脹） | 收縮（縮小） |
| 對應比喻 | 蒙眼走路 | 摸到牆角 |

**表 11-1　運動模型與觀測模型的對照**

## 11.4　Bayes Filter

貝氏濾波（Bayes Filter）是本章的核心——它是卡爾曼濾波、EKF、粒子濾波等所有濾波器的共同骨架。理解了它，後面幾節都只是它的不同實作版本。

### 信念：對狀態的機率描述

貝氏濾波追蹤的核心物件稱為信念（Belief），記為 bel(x_t)——它是「綜合目前所有資訊後，對當前狀態的機率分布」。信念就是 11.1 節說的那個「不確定性橢圓」的數學本體。貝氏濾波的任務，就是每來一筆新資料，就更新這個信念。

### 兩個步驟的遞迴

貝氏濾波以兩個步驟不斷循環，這兩步驟正對應前一節的兩個模型：

72. 預測（Prediction）：機器人執行控制 u_t，用運動模型把信念往前推。這一步讓不確定性擴大——「我動了，所以更不確定自己在哪了」。數學上是把前一刻信念與運動模型做積分（對所有可能的前一位姿加總）。
73. 更新（Update / Correction）：感測器給出觀測 z_t，用觀測模型修正信念。這一步讓不確定性收縮——「我看到了地標，所以更確定自己在哪了」。數學上是把預測信念與觀測模型相乘，再正規化。

這個「預測—更新」的循環，正是第 1 章圖 1-3 SLAM 標準流程、第 7 章卡爾曼濾波「預測—更新」的同一件事，在此得到了精確的機率表述。同學應牢牢記住這個雙步驟結構：它是理解一切遞迴估測的鑰匙。

**圖 11-2　貝氏濾波的遞迴循環**

前一刻信念 bel(x_{t-1})

↓ ① 預測：套用運動模型 p(x_t | x_{t-1}, u_t)

→ 信念變寬（不確定性擴大）

預測信念（先驗）

↓ ② 更新：乘上觀測模型 p(z_t | x_t)，再正規化

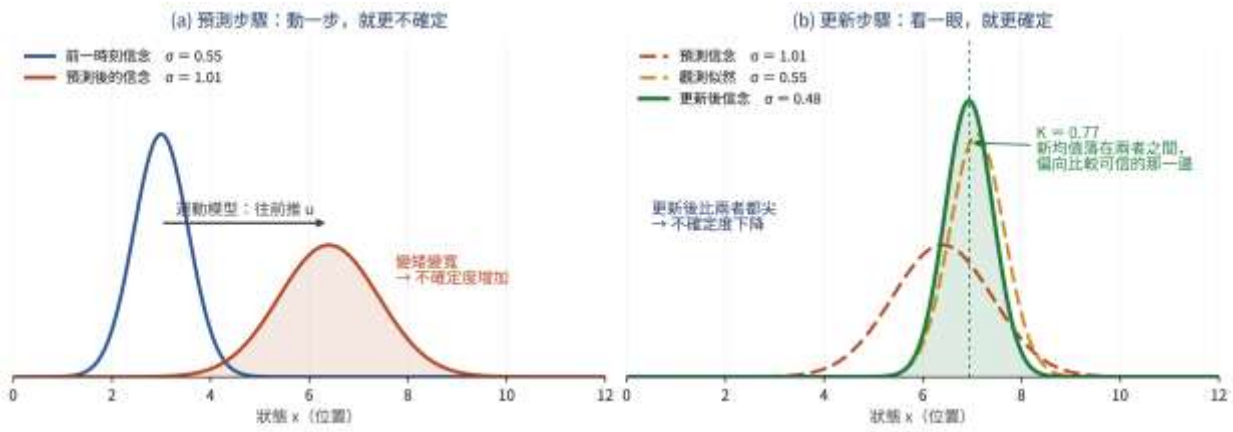
→ 信念變窄（不確定性收縮）

新信念 bel(x_t)

↓

（回到頂端，處理下一時刻）

**圖 11-3　貝氏濾波的兩個步驟：預測使分布變寬，更新使分布變尖**




這張圖是全章——甚至可以說是全書——最值得理解的一張。它用一維的例子把貝氏濾波的整個循環攤開來看。

左圖是預測步驟。機器人原本相當確定自己在位置 3（σ = 0.55），接著里程計說「往前走了 3.4」。於是信念的均值往前移到 6.4，但同時也變矮變寬了（σ 增為 1.01）——因為運動本身帶有雜訊，走得愈多就愈不確定。這正是第 7 章積分漂移與第 13 章里程計誤差在機

率語言中的表達：只靠遞推，不確定度只會單調上升。

右圖是更新步驟。此時感測器回報一個觀測，指出位置應該在 7.1 附近（σ = 0.55）。把預測信念與觀測似然相乘並正規化，得到的後驗信念落在兩者之間（6.94），而且比兩者都更尖（σ 降為 0.48）。

「落在兩者之間的哪裡」由卡爾曼增益決定。本例中 K = 0.77，意思是新均值往觀測那邊移動了兩者差距的 77%——因為觀測（σ = 0.55）比預測（σ = 1.01）更可信。如果觀測雜訊很大，K 就會接近 0，機器人幾乎不理會這次觀測；反之若觀測極準，K 接近 1，機器人幾乎完全採信。11.5 節會給出 K 的完整公式，但它的意義就在這張圖裡。

**三個值得記住的結論**

一、更新後的不確定度一定比更新前小。這是融合的根本價值——多一筆獨立觀測，就少一分不確定。

二、融合後的均值一定落在兩個來源之間，不會超出範圍。所以卡爾曼濾波永遠不會「比兩個來源都離譜」。

三、整個循環只有這兩步：預測變寬、更新變尖。第 14 章的粒子濾波、第 15 章的掃描匹配、第 23 章的視覺慣性最佳化，做的都是同一件事，只是表示法不同。

### 貝氏法則：更新步驟的根據

更新步驟背後的數學，就是機率論中著名的貝氏法則（Bayes' Rule）。它的精神是：把「先驗信念」（預測步驟得到的、看到觀測之前的信念）與「觀測提供的證據」結合，得到「後驗信念」（看到觀測之後、更精確的信念）。

$$\text{後驗} \propto \text{觀測似然} \times \text{先驗} \qquad bel(x_t) \propto p(z_t \mid x_t) \cdot \text{預測信念}$$

這條式子讀作：更新後的信念，正比於「觀測模型」乘上「預測信念」。它是所有濾波器更新步驟的理論根源。貝氏濾波本身是一個抽象框架，實際計算時無法直接處理任意分布

——不同的濾波器，就是對「信念用什麼形式表示、積分與相乘怎麼算」給出不同的答案。接下來三節就是三種答案。

## 11.5 Kalman Filter

卡爾曼濾波器（Kalman Filter,KF）是貝氏濾波在「線性系統 + 高斯雜訊」假設下的精確解。在這個假設下，信念永遠是高斯分布，於是整個濾波只需追蹤兩個量：均值（最佳估計）與共變異數（不確定性）。這讓計算極其高效，也讓卡爾曼濾波成為工程史上最成功的演算法之一——從阿波羅登月導航到今天的手機，無處不在。

### 卡爾曼濾波的兩步驟

它完全遵循貝氏濾波的架構，只是所有運算都在高斯的世界裡進行：

74. 預測：用運動模型把均值往前推（機器人動了，估計位置跟著移動），同時把共變異數加大（加入運動雜訊，不確定性擴大）。
75. 更新：用觀測修正均值，並把共變異數縮小。修正的力道由卡爾曼增益決定。

### 卡爾曼增益：全章最精妙的一個概念

卡爾曼增益（Kalman Gain,K）是整個濾波器的靈魂。它回答一個關鍵問題：預測和觀測不一致時，該相信誰、相信多少？

它的邏輯完全符合直覺：

- 若預測很不確定（共變異數大）、觀測很準（雜訊小）：增益大，多相信觀測——「我本來就沒把握，既然感測器說得這麼肯定，那就聽它的」。
- 若預測很有把握、觀測很雜：增益小，多相信預測——「我很確定自己的推算，這個雜訊很大的量測參考就好」。

換句話說，卡爾曼濾波不是死板地取平均，而是根據雙方的可信度做加權——誰更可信就更聽誰的。這個「按可信度加權」的思想，是本書反覆出現的融合精神（第 4、5、7 章）

的數學化身。第 7 章的姿態濾波、第 5 章的 GNSS/INS 組合導航，底層都是這個增益在運作。

**觀念總結：卡爾曼濾波在做什麼**

一句話：用「上一刻的估計 + 運動」預測現在，再用「感測器觀測」修正，

兩者按各自的可信度（共變異數）加權融合，得到比單獨任一方都更準的估計。

關鍵洞見：融合兩個帶雜訊的資訊，結果會比其中任何一個單獨都更準——

這就是為什麼「多感測器融合」永遠優於「只用最好的那一個」。

## 11.6 Extended Kalman Filter, EKF-SLAM

卡爾曼濾波很美，但有一個嚴格的前提：系統必須是線性的。可惜機器人的世界充滿非線性——第 10 章看過，位姿更新含 cosθ、sinθ;觀測模型「若在此處，光達會量到多遠」也是非線性的。標準卡爾曼濾波無法直接處理。EKF（擴展卡爾曼濾波）就是為此而生。

### EKF 的核心技巧：局部線性化

EKF 的想法很務實：既然系統是非線性的，那就在當前估計點附近，用切線去近似曲線。數學上這叫一階泰勒展開，得到的斜率矩陣稱為雅可比矩陣（Jacobian）。線性化之後，就能套用標準卡爾曼濾波的公式。

這個近似在「非線性不太嚴重、不確定性不太大」時效果良好，是機器人領域最廣泛使用的估測方法——第 7 章提過的無人機飛控 EKF、ROS 的 robot_localization，底層都是它。但它的近似本質也帶來限制：當非線性很強或不確定性很大時，切線偏離曲線太多，估計可能失準甚至發散。

### EKF-SLAM：把地圖也放進狀態

EKF 如何用來做 SLAM?關鍵一步是：把機器人位姿和所有地標的位置，通通放進同一個狀態向量，用同一個高斯分布（同一個大共變異數矩陣）一起估計。

這個設計精妙地抓住了 SLAM 的本質——第 1 章強調的「定位與建圖互相耦合」。當機器人觀測到一個地標並修正自己的位置時，共變異數矩陣中的相關性會把這個修正「傳播」到所有相關的地標與位姿。這正是閉環修正能一次攤平整張地圖誤差的數學機制：回到熟悉地點時，一個強觀測透過共變異數的相關性，同時校正了機器人與沿途所有地標。

### EKF-SLAM 的歷史地位與限制

EKF-SLAM 是 SLAM 發展史上第一個完整可行的解法（1990 年代），奠定了整個領域的理論基礎，地位崇高。但它有兩個致命限制，使它逐漸退出主流：

- 計算複雜度：共變異數矩陣的大小隨地標數量平方成長。地標一多（數千個），矩陣運算就慢到無法即時——這對動輒數萬地標的大場景是硬傷。
- 線性化誤差累積：EKF 每步都做線性近似，誤差會累積，長期或大場景下可能導致估計不一致。

這兩個限制，催生了後續的兩條主流路線：一是第 14 章的粒子濾波（用另一種方式處理），二是第 12 章的圖優化（用批次最佳化取代遞迴濾波）。可以說，理解 EKF-SLAM 的限制，就理解了現代 SLAM 為何走向圖優化——這是承先啟後的關鍵一章。

## 11.7 Particle Filter 與 Monte Carlo Localization

卡爾曼家族都假設信念是「單一高斯」。但 11.2 節末尾提過：有些情況信念是多峰的——機器人可能在兩個一模一樣的房間之一，信念應該是「兩個高峰」。單一高斯無法描述這種情況。粒子濾波（Particle Filter）提供了截然不同的思路。

### 用一群樣本代表信念

粒子濾波的核心洞見是：不用數學公式描述分布，而是用一大群樣本（粒子）來「近似」它。每個粒子代表「機器人可能在這裡」的一個假設。粒子密集的地方，就是可能性高的地方。

這個表示法的最大優點是：它能描述任意形狀的分布——多峰、不規則、非高斯都沒問題。這是卡爾曼家族做不到的。

### 粒子濾波的三步驟

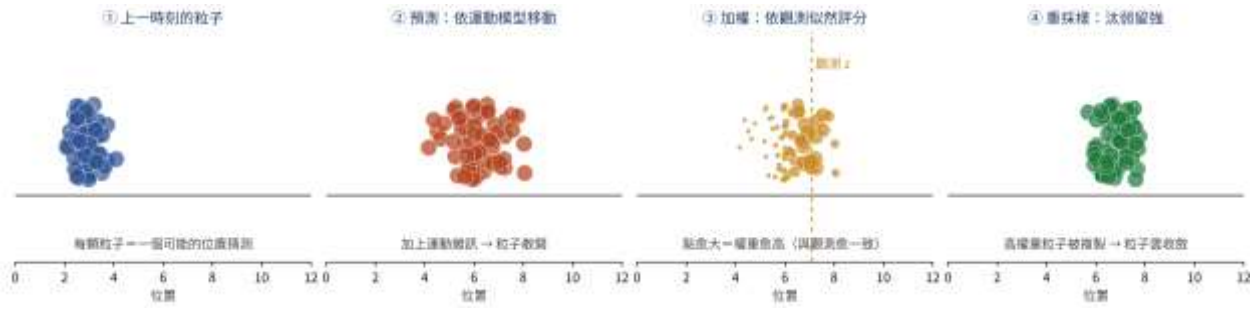
它同樣遵循貝氏濾波架構，但以粒子的方式實現：

76. 預測（移動粒子）：每個粒子依運動模型移動，並各自加上隨機雜訊。於是原本聚集的粒子雲會散開——對應不確定性擴大。

77. 更新（秤重）：用觀測模型給每個粒子打分數（權重）。與觀測吻合的粒子（例如「這個位置預期看到的牆，和實際量到的相符」）得高分，不吻合的得低分。

78. 重採樣（Resampling）：依權重重新抽選粒子——高分粒子被複製多份，低分粒子被淘汰。於是粒子雲逐漸聚集到最可能的位置。這是粒子濾波的精髓：適者生存。

**圖 11-4　粒子濾波的四個步驟：用一群樣本代表機率分布**




粒子濾波放棄了「用公式描述分布」的做法，改用一群樣本來代表它。每一顆粒子就是一個完整的狀態猜測——在定位問題中，就是一個「機器人可能在這裡，朝這個方向」的假設。粒子密集的地方，代表機率高。

四個步驟與圖 11-3 的兩步驟完全對應。① 是上一時刻的粒子雲。② 預測：每顆粒子各自依運動模型往前移動，並加上隨機雜訊——結果是粒子散開，對應高斯版本中的「變寬」。

③ 加權：依觀測似然替每顆粒子評分，與觀測一致的粒子得到高權重（圖中點愈大）。④ 重採樣：依權重重新抽樣，高權重的粒子被複製多份、低權重的被淘汰，粒子雲因而收斂——對應高斯版本中的「變尖」。

那麼它比卡爾曼濾波好在哪？關鍵在於粒子雲可以是任何形狀。卡爾曼濾波被鎖死在「單峰高斯」裡，而真實的定位問題經常是多峰的：機器人剛開機時可能在建築物的任何地方；走在對稱的走廊中時，兩個位置的觀測看起來一模一樣。這些情境下「一個均值加一個橢圓」根本無法表達，粒子雲卻可以同時在兩個地方各留一群粒子，等到更多觀測進來再決定留哪一群。

代價是運算量。粒子數必須隨狀態維度指數成長，所以粒子濾波在二維定位（三個自由度）上很實用，到了高維度就撐不住。這正是第 14 章 Gmapping 必須採用 Rao-Blackwell 化技巧的原因——只對位姿用粒子，地圖的部分改用解析方法處理。

### 蒙地卡羅定位（MCL）與 AMCL

把粒子濾波用於「在已知地圖中定位」，就是蒙地卡羅定位（MCL,Monte Carlo Localization），ROS 中的實作稱為 AMCL（第 3 章提過）。它的運作極為直觀：一開始不知道機器人在哪，就在地圖各處撒滿粒子；機器人一邊移動一邊掃描，與觀測不符的粒子被淘汰，粒子雲逐漸收斂到真實位置。

MCL 天生就能解決第 3 章的「綁架機器人問題」：即使機器人被搬走，只要在地圖各處保留一些粒子，它們就能在新位置重新「勝出」，讓機器人找回自己。這是單一高斯的卡爾曼濾波難以做到的——這也是掃地機器人被抱到別的房間後能重新定位的數學原因。

### 粒子濾波與 SLAM：預告 Gmapping

把粒子濾波用於 SLAM（而非只是定位），需要一個巧妙的技巧，稱為 Rao-Blackwellized 粒子濾波（RBPF）——每個粒子不只代表一個位姿，還各自攜帶一整張地圖。這正是第 14 章 Gmapping 的核心，也是本書第四篇 2D SLAM 的主角。本章先把粒子濾波

的原理講清楚，第 14 章就能專注於它如何建圖。

| 特性 | 卡爾曼 / EKF 家族 | 粒子濾波 |
|---|---|---|
| 信念表示 | 單一高斯（均值 + 共變異數） | 一群粒子樣本 |
| 可處理分布 | 限高斯（單峰） | 任意形狀（可多峰） |
| 非線性 | EKF 靠線性化近似 | 天然支援，不需線性化 |
| 綁架問題 | 難處理 | 天生可解 |
| 計算成本 | 低（維度低時） | 高（需大量粒子） |
| 代表應用 | GNSS/INS、EKF-SLAM | AMCL、Gmapping |

**表 11-2　卡爾曼家族與粒子濾波的比較**

至此，同學已經掌握了 SLAM 的核心估測工具：用機率描述不確定性，用貝氏濾波的「預測—更新」處理它，並認識了卡爾曼、EKF、粒子濾波三種具體實作。這些是遞迴濾波的路線——每來一筆資料就更新一次。但現代 SLAM 還有另一條更強大的路線：不急著逐筆處理，而是累積一批資料後一次性最佳化。下一章「最佳化與圖論 SLAM」，將介紹這條主宰當代 SLAM 的路線——Pose Graph、Bundle Adjustment 與 g2o、Ceres、GTSAM。

## 公式或流程

### 一、一維高斯分布

一維高斯分布的機率密度函數：

$$p(x) = (1 / \sqrt{(2\pi \sigma^2)}) \cdot exp( - (x - \mu)^2 / (2\sigma^2) )$$

其中 μ 為均值（分布中心、最佳估計）、$\sigma^2$ 為變異數（不確定性大小）。約 68% 的機率落在 μ±σ、95% 落在 μ±2σ、99.7% 落在 μ±3σ——這組「68-95-99.7 法則」是判讀不確定性最實用的直覺。

### 二、貝氏法則

狀態估測的理論核心：

$$p(x \mid z) = p(z \mid x) \cdot p(x) \;/\; p(z)$$

讀作：後驗 = ( 似然 × 先驗 ) / 證據。在濾波中，p(x) 是預測步驟得到的先驗信念、p(z|x) 是觀測模型、p(x|z) 是更新後的後驗信念。分母 p(z) 只是正規化常數（確保機率總和為 1），因此常寫成正比形式：

$$p(x \mid z) \propto p(z \mid x) \cdot p(x)$$

## 三、貝氏濾波兩步驟

預測步驟（套用運動模型，對前一狀態積分）：

$$\overline{bel}(x_t) = \int p(x_t \mid x_{t-1}, u_t) \cdot bel(x_{t-1})\, dx_{t-1}$$

更新步驟（乘上觀測模型並正規化）：

$$bel(x_t) = \eta \cdot p(z_t \mid x_t) \cdot \overline{bel}(x_t)$$

其中 $\overline{bel}$ 是預測信念（先驗）、bel 是更新後信念（後驗）、η 是正規化常數。卡爾曼濾波、EKF、粒子濾波都是這兩條式子在不同表示法下的具體實現。

## 四、卡爾曼濾波核心方程（概念版）

預測：

$$\text{均值：} \bar{x} = F \cdot \hat{x} \qquad \text{共變異數：} \bar{P} = F \cdot P \cdot F^{T} + Q$$

更新：

$$\text{卡爾曼增益：} K = \bar{P} \cdot H^{T} \cdot \left( H \cdot \bar{P} \cdot H^{T} + R \right)^{-1}$$

$$\text{均值：} \hat{x} = \bar{x} + K \cdot \left( z - H \cdot \bar{x} \right) \qquad \text{共變異數：} P = \left( I - K \cdot H \right) \cdot \bar{P}$$

其中 F 是運動模型、H 是觀測模型、Q 是運動雜訊、R 是觀測雜訊。細看增益 K 的式子：當觀測雜訊 R 很小（觀測很準），K 變大，更新時就更相信觀測（z - H·x̄ 這個「觀測與預測的差」被乘上大的 K）；當 R 很大，K 變小，更相信預測。這條式子把 11.5 節「按可信度

加權」的直覺精確地數學化了。

### 五、三種濾波器選擇流程

**圖 11-5 濾波器選擇決策流程**

問題一：系統是線性的嗎？

├─ 是 → 卡爾曼濾波（KF）：最佳、最快

└─ 否 → 進入下一問

↓

問題二：非線性溫和、信念單峰、有好的初始估計嗎？

├─ 是 → 擴展卡爾曼濾波（EKF）：線性化近似，高效

└─ 否（強非線性 / 多峰 / 綁架問題）→ 進入下一問

↓

問題三：狀態維度低、可負擔大量粒子嗎？

├─ 是 → 粒子濾波（PF / MCL）：能處理任意分布

└─ 否（高維、大場景）→ 考慮第 12 章的圖優化

## 11.8 狀態估測的當代發展：濾波沒有退場，而是換了位置

本章介紹的卡爾曼家族與粒子濾波，在教科書上常被描述為「經典方法」，彷彿已被第 12 章的圖優化取代。這個印象只對了一半，值得同學仔細分辨。

### 一、後端確實被最佳化接管了

在 SLAM 的後端——也就是維護整條軌跡與整張地圖的那一層——最佳化確實已經全面勝出。原因在 11.6 節提過：EKF-SLAM 的共變異數矩陣隨地標數量平方成長，幾千個地標就

撐不住；而且線性化只做一次，早期的錯誤無法回頭修正。

第 12 章的圖優化保留所有歷史位姿與觀測，可以反覆重新線性化，還能在閉環時一次修正整條軌跡。再加上 iSAM2 這類增量式求解器只更新受影響的部分，使得最佳化在速度上也不再吃虧。今天的 ORB-SLAM3、Cartographer、LIO-SAM 無一例外採用這條路線。

## 二、但前端與高頻迴路仍然是濾波的天下

然而把視角拉近到「每秒數百次的即時估測」，濾波仍然不可取代。無人機飛控的姿態估測、四足機器人的機身狀態估測、第 7 章提到的 IMU 高頻遞推，都還是 EKF 在跑——因為最佳化需要累積一批資料才能求解，而控制迴路等不了。

這帶出一個實務上的分工，同學應該建立起來：高頻、低維度、要求即時 → 濾波；低頻、高維度、要求全域一致 → 最佳化。一個完整的機器人系統兩者都有，而且是分層並存，不是二選一。第 18 章的 LIO-SAM、第 23 章的 ORB-SLAM3 都是這種分層架構。

## 三、不變式 EKF：讓濾波重新變得可靠

濾波本身也在進步。11.6 節提到 EKF 的根本弱點是線性化誤差——當估計偏離真值較遠時，雅可比矩陣算在錯的地方，濾波可能發散。近年的不變式擴展卡爾曼濾波（InEKF）利用第 10.8 節提到的李群結構，把狀態定義在矩陣李群上；其誤差動態在特定條件下與估計值無關，因而具有更好的收斂性質。

這條路線在足式機器人上特別成功，已成為許多四足與人形機器人機身狀態估測的標準做法。對同學而言，它是一個很好的例證：第 10 章看似抽象的李群，在這裡直接換成了更穩定的濾波器。

## 四、神經網路補償：讓濾波處理它建模不了的東西

最新的一條路線，是把學習模型掛在濾波器旁邊，專門處理模型假設失效的情況。2026 年一項針對足式機器人的研究就是典型例子：它在不變式 EKF 之上加了一個以注意力機制為核心的神經補償器。

其設計邏輯值得細看。足式機器人的狀態估測仰賴「觸地的腳不會滑動」這個假設，但在碎石、低摩擦面上這個假設會崩壞。該研究把腳底打滑程度當成明確的條件變數，以 GRU 自動編碼器分別編碼濾波器的狀態歷史與打滑程度，再用交叉注意力產生依打滑情境調整的補償量，最後在濾波器的後驗狀態上加一個九維的殘差修正。

效果相當顯著：在碎石場地上位置誤差較最強基準降低約 93%，在低摩擦的鐵氟龍板上降低約 49%，約一百公尺的室外草地測試降低約 55% 至 73%，而整體仍可在 580 赫茲下即時運行。

請注意它的結構——與第 7.8 節的學習式 IMU、第 9.8 節的深度估測模型完全一致：保留幾何與機率的骨架，讓神經網路負責補償那些「物理模型寫不出來」的部分。這已經是目前機器人領域最穩健的 AI 整合模式，第 32 章會從更高的層次再討論一次。

> **一句話總結這四件事**
>
> 濾波沒有被取代，它只是從「負責整張地圖」退回到「負責即時的那一小段」——而在那個位置上，它仍然無可取代。
>
> 同學學本章不是為了考古：你在第 26 章寫無人機姿態估測、在第 29 章調 AMCL 參數、在第 16 章理解掃地機器人為什麼會「迷路後突然想起自己在哪」，用的都是本章的東西。

## 案例

### 案例 11-1　阿波羅登月：卡爾曼濾波的成名之戰

卡爾曼濾波器由 Rudolf Kálmán 於 1960 年提出，而它第一個震撼世界的應用，就是阿波羅登月計畫。太空船在地月之間航行，導航系統必須融合慣性量測與地面追蹤等多種帶雜訊的資料，即時估出太空船的位置與速度——這正是卡爾曼濾波「按可信度融合多源資訊」的完美舞台。

當時太空船的運算能力極其有限（遠不如今天一支計算機），但卡爾曼濾波只需追蹤均值與共變異數兩個量、且是遞迴計算（不必儲存全部歷史資料），恰好能在如此拮据的硬體上即時運行。可以說，沒有卡爾曼濾波，阿波羅的導航幾乎無法實現。

這個案例呼應了第 10 章的阿波羅萬向鎖——同一個登月任務，同時推動了旋轉表示法與狀態估測兩大數學的發展。它也給同學一個啟示：本章這些看似抽象的機率工具，是人類踏上月球的實際基石。六十多年後的今天，你手機裡的定位、掃地機器人的導航，用的仍是同一套數學。

## 案例 11-2　掃地機器人的「粒子雲」：AMCL 的視覺化

許多支援地圖顯示的掃地機器人 App，若進入除錯或進階模式，能看到一個生動的畫面：機器人開機時，地圖上散布著大量代表「可能位置」的小點（粒子）；隨著機器人移動幾步、掃描幾次，這些小點迅速從「散布全圖」收斂成「緊密的一小團」——機器人「想通了」自己在哪。

這正是 11.7 節 AMCL 的即時視覺化。散開的粒子代表「我還不確定在哪」，收斂的粒子團代表「我很確定了」。如果把機器人抱到別的房間，會看到粒子團先是「困惑」（與觀測不符），然後在正確的新位置重新聚集——綁架問題的解決過程活生生地展現在眼前。

這個案例的教學價值在於：它把本章最抽象的「機率分布」變成看得見的粒子雲。大學課堂上，教師常讓同學在 RViz 中觀察 AMCL 的粒子演化，配合本章理論，抽象的「信念收縮」瞬間變得具體。這也是本書強調「先理解原理、再動手觀察」的又一個例子。

## 案例 11-3　自駕車的感測器融合：增益在真實道路上的權衡

一台自駕車同時擁有 GPS、IMU、輪速計、光達、相機多種感測器，它們對「車現在在哪」各有各的說法，而且各有各的可靠時機。卡爾曼濾波家族（通常是 EKF 或其變體）正是融合這些資訊的主力。

本章的卡爾曼增益概念，在此有極為真實的體現：當車輛開進隧道，GPS 失鎖（觀測雜訊 R 飆升），濾波器自動調低對 GPS 的信任、改倚重 IMU 與輪速計的推算（第 5 章的組合導航）；出隧道後 GPS 恢復，濾波器又逐漸把信任轉回來。這個「依情境動態調整信任」的過程，不需要工程師寫死規則，而是共變異數與增益的數學自然導出的結果。

這正是機率方法相對「寫死規則」的根本優勢：它讓系統依據每個資訊源當下的可信度自動權衡，優雅地適應各種情境。這個思想貫穿全書——從第 4 章的室內定位融合、第 5 章的 GNSS/INS、第 7 章的姿態濾波，到本章的統一數學表述，同學應該能感受到：整個第二篇與第三篇，其實都在講同一件事的不同層面。

## 實作活動

### 實作 11-1　一維卡爾曼濾波與粒子濾波動手做

目的：用最簡單的一維例子，親手實作卡爾曼濾波與粒子濾波，親眼看見「不確定性擴大與收縮」以及「粒子雲收斂」。本實作以 Python 為主，是理解本章所有理論的關鍵一步。

**第一部分：一維卡爾曼濾波（全體）**

62. 設定情境：一台機器人在一維軌道上，真實位置未知。它有一個會漂移的「速度感測器」（運動模型，雜訊大）與一個偶爾回報的「位置感測器」（觀測模型，雜訊中等）。用程式產生一段帶雜訊的模擬資料。
63. 實作卡爾曼濾波的預測步驟：每步用速度更新位置均值，並把變異數加大。畫出「估計位置」與「不確定範圍（均值±標準差）」隨時間的變化，觀察只有預測時不確定範圍如何持續擴大。
64. 加入更新步驟：每當位置感測器回報，就用觀測修正均值、縮小變異數。觀察不確定範圍在每次觀測時如何「收縮」。
65. 把「純預測（航位推算）」與「完整卡爾曼濾波」的估計軌跡畫在一起，與真值比較，量化卡爾曼濾波減少了多少誤差。

66. 實驗卡爾曼增益：把觀測雜訊 R 分別設得很大與很小，觀察增益如何變化、估計軌跡如何更偏向預測或更偏向觀測。

**第二部分：一維粒子濾波（全體或大學部）**

67. 在一維軌道上撒 500 個粒子（均勻分布），代表「一開始不知道機器人在哪」。

68. 實作三步驟：每步讓所有粒子依運動模型移動（加雜訊）、依觀測給每個粒子打權重、再依權重重採樣。

69. 每幾步畫一次粒子分布的直方圖，觀察粒子雲如何從「散布全軌道」收斂到「真實位置附近」。

70. 綁架測試：在中途突然把真實位置換到別處，觀察粒子雲如何先「困惑」、再重新聚集到新位置。

**第三部分：多峰情境（大學部）**

71. 設計一個「對稱走廊」情境：兩個地點的觀測完全相同（對稱環境）。用粒子濾波處理，觀察粒子雲如何維持「兩個高峰」——親身驗證 11.7 節「粒子濾波能處理多峰、卡爾曼不能」的論述。

72. 思考：若用卡爾曼濾波處理這個對稱情境，會發生什麼？（提示：單一高斯會把均值放在兩峰中間——一個機器人根本不可能在的位置。）

**討論問題**

55. 在第一部分，完整卡爾曼濾波比純航位推算減少了多少誤差？這個改善來自哪裡？

56. 當你把觀測雜訊 R 調得很小，估計軌跡發生什麼變化？這符合 11.5 節對卡爾曼增益的描述嗎？

57. 粒子濾波的綁架測試中，粒子雲花了多久重新聚集？若粒子數只有 50 個（而非 500），結果會如何？這說明粒子數的什麼取捨？

58. （大學部）在對稱走廊情境，粒子濾波維持了兩個高峰，而卡爾曼濾波會失敗。這對「選擇哪種濾波器」有什麼實務啟示？

**延伸挑戰（大學部）**

把一維卡爾曼濾波擴展到二維（x、y），追蹤共變異數矩陣，並用 matplotlib 畫出每一步的「不確定性橢圓」，觀察機器人沿直線運動時橢圓如何沿運動方向拉長、觀測後如何收縮。這個二維視覺化直接對應 11.2 節，也為第 14 章 Gmapping 與第 29 章 AMCL 實作打下直覺基礎。

## 延伸閱讀


69. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005.（機率機器人學聖經，本章全部主題的權威來源，第 2 至 8 章尤其相關）
70. Cyrill Stachniss, SLAM 課程之 Bayes Filter、Kalman Filter、Particle Filter 系列影片（YouTube）.（以動畫直觀講解本章三種濾波器，自學首選）
71. R. Faragher, "Understanding the Basis of the Kalman Filter via a Simple and Intuitive Derivation," IEEE Signal Processing Magazine, 2012.（卡爾曼濾波最友善的直觀推導）
72. R. Labbe, "Kalman and Bayesian Filters in Python"（GitHub 免費電子書）.（附完整可執行程式，實作本章的最佳伴讀）
73. D. Fox et al., "Monte Carlo Localization for Mobile Robots," ICRA, 1999.（蒙地卡羅定位 MCL 的原始論文）
74. CMU 16-833 Robot Localization and Mapping 課程（第 2 章介紹）之濾波單元.（EKF-SLAM 與粒子濾波的研究所級教材）
75. 高翔、張濤，《視覺 SLAM 十四講》第 6 講（非線性最佳化）前段.（銜接本章濾波與第 12 章圖優化的橋樑）


## 從濾波到最佳化：本章的位置回顧

本章把 SLAM 的問題寫成了一個機率問題，並給了三種解法。但這三種解法有一個共同的性

格：它們都是遞迴的——只保留「當前」的信念，每來一筆新資料就更新它，然後把舊資料丟掉。

這樣做的好處是記憶體固定、運算量穩定，非常適合即時系統。但代價也很明確：一旦某一步的估計錯了，錯誤就被壓進了當前的信念裡，再也無法回頭修正。機器人繞了一大圈回到起點，發現地圖對不上時，濾波器只能從那一刻起慢慢修正，卻無法回頭把中間那一整段軌跡一起扳正。

第 12 章要介紹的是完全相反的思路：不丟掉任何東西。把每一個時刻的位姿都當成一個待求的變數，把每一筆觀測都當成一條約束，全部放進一個巨大的最佳化問題中一次求解。閉環偵測到「我回到起點了」的那一刻，這條新約束會把整條軌跡一起拉回正確的位置——這是濾波永遠做不到的事。

有趣的是，兩者的數學根源相同：最佳化求的最大後驗估計，與濾波追求的後驗分布峰值，在高斯假設下是同一件事。第 12 章會說明這個等價關係。所以本章學到的一切都不會白費——它只是換了一種計算方式。

## 小結

42. SLAM 必須以機率處理不確定性：每個感測器給的都是帶雜訊的猜測，成熟的做法是量化不確定性再做最佳權衡。關鍵觀念是——估計的是「機率分布」（最佳猜測 + 可信度），而非單一數值。
43. 高斯分布以均值（最佳估計）與變異數（不確定性）兩參數完整描述；多維時共變異數矩陣對應「不確定性橢圓」。高斯因兩參數搞定、運算封閉、中央極限定理而成為 SLAM 首選，但無法描述多峰分布。
44. 運動模型 p(x_t|x_{t-1},u_t) 描述「動了會到哪」使不確定性擴大；觀測模型 p(z_t|x_t) 描述「在這裡會看到什麼」使不確定性收縮。兩者是所有濾波器的輸入。

45. 貝氏濾波是所有濾波器的共同骨架，以「預測（套運動模型、信念變寬）—更新（套觀測模型、信念變窄）」兩步驟遞迴，更新步驟的根據是貝氏法則：後驗 ∝ 似然 × 先驗。
46. 卡爾曼濾波是線性高斯下的最佳解，只需追蹤均值與共變異數；卡爾曼增益依雙方可信度權衡預測與觀測——這是「多感測器融合優於單一感測器」的數學核心。
47. EKF 以雅可比矩陣局部線性化處理非線性，是機器人最廣用的估測法；EKF-SLAM 把位姿與地標放進同一狀態聯合估計，體現定位與建圖的耦合，但共變異數隨地標平方成長且線性化誤差累積，催生了圖優化路線。
48. 粒子濾波以一群樣本近似任意分布，經預測—秤重—重採樣三步驟，能處理多峰與非線性、天生可解綁架問題；蒙地卡羅定位（AMCL）是其定位應用，Rao-Blackwellized 版本（Gmapping）則是其 SLAM 應用。

本章的濾波器都是「遞迴」的——每來一筆資料就更新一次，處理完就丟掉。這高效，但也意味著過去的資訊一旦被壓縮進當前信念，就無法再回頭修正。下一章「最佳化與圖論 SLAM」介紹另一條路線：把一批位姿與觀測全部保留，建成一張「位姿圖」，一次性最佳化求出最一致的解。這條路線精度更高、能更好地處理閉環，正是 ORB-SLAM、Cartographer 等當代主流系統的數學基礎。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

## 第 12 章

## 最佳化與圖論 SLAM

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

## 本章在全書的位置

第 11 章把 SLAM 寫成了機率問題，並給出三種濾波解法。但濾波有一個共同的性格：它只保留「當前」的信念，每更新一次就把舊資料丟掉。這在即時性上很划算，在正確性上卻付出代價——錯誤一旦被壓進信念裡，就再也無法回頭修正。

本章要介紹的是相反的思路：什麼都不丟。把每一個時刻的位姿都當成一個待求的變數，把每一筆觀測都當成一條約束，全部放進同一個最佳化問題中一次求解。當機器人繞一圈回到起點、發現地圖對不上時，這條新的閉環約束能把整條軌跡一起扳正——這是濾波永遠做不到的事。

這就是現代 SLAM 的主流架構。第 14、15 章的 2D SLAM、第 18 章的光達慣性里程計、第 22、23 章的 ORB-SLAM 系列，後端全都是本章的圖優化。可以說，第三篇三章當中，本章是最直接連到實作的一章。

**圖 12-0　第 12 章與全書的連結地圖**

【承接前章】

第 10 章　位姿 T　→　本章成為圖的節點

第 11 章　機率與資訊　→　本章成為邊的資訊矩陣 Ω

第 11 章的濾波「丟掉歷史」　→　本章「保留全部歷史」

【本章負責】

前端 / 後端分工　→　位姿圖　→　最小平方　→　Bundle Adjustment

稀疏性：為什麼上萬個節點也解得動

g2o、Ceres、GTSAM：真正會用到的工具

【往下延伸】

→ 第 15 章　Cartographer 的分支定界閉環 + 位姿圖後端

→ 第 17 章　ICP 配準結果 = 位姿圖的一條邊

→ 第 18 章　LIO-SAM：光達、IMU、GNSS 的因子圖

→ 第 22、23 章　ORB-SLAM 的局部 BA 與全域 BA

→ 第 7.5 節　IMU 預積分的產物，就是本章的一條邊

## 學習目標

上一章的濾波器是「遞迴」的——每來一筆資料就更新一次，處理完就丟掉。本章介紹另一條路線：把一批位姿與觀測全部保留，建成一張「圖」，一次性最佳化求出最一致的解。這條圖優化路線精度更高、更能處理閉環，是 ORB-SLAM、Cartographer 等當代主流系統的數學基礎，也是第三篇的收尾。學完本章之後，同學應能夠達成下列目標：

70. 說明 SLAM 前端與後端的分工，理解圖優化屬於後端。
71. 說明位姿圖（Pose Graph）的節點與邊代表什麼，並看懂它如何描述一個 SLAM 問題。

72. 解釋為什麼 SLAM 可以化為一個最小平方問題。

73. 說明 Bundle Adjustment 的概念與它在視覺 SLAM 中的角色。

74. 認識 g2o、Ceres、GTSAM 三大最佳化函式庫的定位與用途。

75. 說明為什麼現代 SLAM 多採圖優化，而非上一章的濾波器。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
| --- | --- | --- |
| 前端 | Front-End | 把原始感測資料轉成幾何約束（特徵、匹配、里程計）。 |
| 後端 | Back-End | 在所有約束下求出最一致的位姿與地圖。 |
| 位姿圖 | Pose Graph | 以節點表示位姿、邊表示約束的圖結構。 |
| 節點 | Node / Vertex | 待估計的量，如機器人位姿或地標位置。 |
| 邊 | Edge / Constraint | 兩節點之間的相對量測約束。 |
| 最小平方 | Least Squares | 尋找使所有誤差平方和最小的解的方法。 |
| 殘差 | Residual | 量測值與預測值之間的差異。 |
| 光束法平差 | Bundle Adjustment, BA | 同時最佳化相機位姿與三維點的視覺最佳化。 |
| 重投影誤差 | Reprojection Error | 三維點投影到影像與實際觀測像素的差距。 |
| 資訊矩陣 | Information Matrix | 共變異數的反矩陣，代表約束的可信度權重。 |
| 稀疏性 | Sparsity | 每個約束只牽涉少數變數，使大問題可高效求解。 |
| 圖優化函式庫 | g2o / Ceres / GTSAM | 求解圖優化問題的開源工具。 |

## 12.1　前端與後端的分工

現代 SLAM 系統普遍採用「前端 + 後端」的兩段式架構（第 2 章各校課程的共同骨架、第 1 章圖 1-3 的核心）。理解這個分工，是讀懂本章的第一步。

### 前端：把感測資料變成幾何約束

前端（Front-End）直接面對感測器，它的工作是把原始資料轉成「幾何約束」——也就是「A 位置到 B 位置之間，相對移動了多少」或「在 A 位置看到了某個地標」這類幾何關係。

- 對光達 SLAM：前端做掃描匹配（第 13 章），算出「這一幀相對上一幀移動了多少」。
- 對視覺 SLAM：前端做特徵提取與匹配（第 9、20 章），算出相機運動與看到的地標。
- 前端還負責閉環偵測：辨識「這個地方我來過」，產生一條關鍵的閉環約束。

### 後端：在所有約束下求最一致解

後端（Back-End）不直接碰感測器，它接收前端產生的一堆幾何約束，任務是找出一組位姿與地圖，使它們「盡可能同時滿足所有約束」。

問題來了：這些約束往往互相矛盾。里程計說「我從 A 走到 B 移動了 10 公尺」，但閉環偵測說「B 其實就在 A 旁邊」——兩個約束打架。後端的工作，就是在這些矛盾的約束之間找一個「最佳妥協」，讓總體誤差最小。這正是本章的主題。

### 兩種後端路線

如何求這個「最佳妥協」，歷史上有兩條路線：

- 濾波路線（第 11 章）：遞迴地逐筆處理，以 EKF、粒子濾波為代表。優點是即時、記憶體固定；缺點是過去資訊被壓縮後無法回頭修正。

- 最佳化路線（本章）：保留一批約束，一次性求解整個問題，以圖優化為代表。優點是精度高、能反覆修正、能漂亮地處理閉環；代價是計算量較大。

過去十餘年，最佳化路線因為精度與閉環處理的優勢，已成為主流。本章就是要說清楚這條路線的數學與工具。

## 12.2　Pose Graph 的概念

圖優化 SLAM 的核心資料結構，是一張「位姿圖」（Pose Graph）。它把抽象的 SLAM 問題，變成一張直觀的「圖」——就是資料結構課學過的、由節點與邊構成的圖。

### 節點與邊

- 節點（Node / Vertex）：代表「待估計的量」。在最單純的位姿圖中，每個節點是機器人在某個時刻的位姿（第 10 章的 x、y、θ 或 6-DoF）。在含地標的版本中，地標位置也是節點。這些就是我們想求出來的未知數。
- 邊（Edge / Constraint）：代表兩個節點之間的「相對量測約束」。它記錄「根據某個感測器，這兩個節點之間應該是什麼關係」。

### 兩種關鍵的邊

位姿圖中有兩種來源不同、但同等重要的邊：

- 里程計邊（相鄰邊）：連接時間上相鄰的兩個位姿，來自里程計或掃描匹配——「我從第 t 幀走到第 t+1 幀，移動了這麼多」。這些邊像一條鏈子，把位姿串起來。
- 閉環邊（迴路邊）：連接時間上不相鄰、但空間上重逢的兩個位姿，來自閉環偵測——「我現在所在的位置，和很久以前的第 5 幀是同一個地方」。這種邊在圖上形成「迴路」，是消除累積漂移的關鍵。

### 為什麼閉環邊這麼重要

想像機器人繞行大樓一圈回到起點。里程計邊像一條鏈子，一路累積誤差，走完一圈後終點與起點之間出現明顯落差（第 1 章的漂移）。這時閉環偵測產生一條邊，宣告「終點就

是起點」——這條邊與整條里程計鏈子產生矛盾。

後端最佳化的魔法，就是把這個矛盾「均攤」到整條軌跡上：它微調沿途每一個位姿，讓終點回到起點，同時盡量不破壞里程計約束。結果是——整條軌跡被「拉正」，累積的漂移一次性被消除。這就是為什麼閉環對 SLAM 如此關鍵，也是圖優化相對濾波器的一大優勢：它能回頭修正所有歷史位姿。

**圖 12-1　位姿圖的結構**

節點：①—②—③—④—⑤—⑥—⑦—⑧ （每個是一個位姿）

└———里程計邊（相鄰，像鏈子）———┘

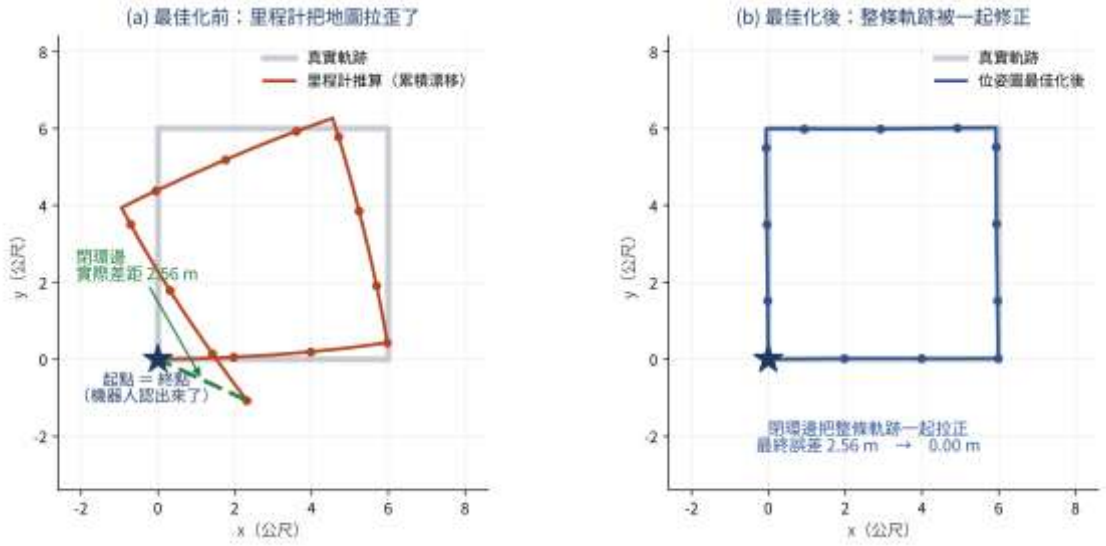
① ←————————閉環邊————————→ ⑧

（偵測到第⑧幀回到了第①幀的位置）

最佳化前：里程計鏈累積漂移，⑧ 與 ① 對不上

最佳化後：閉環邊把矛盾均攤到 ①~⑧，整條軌跡被拉正

**圖 12-2　閉環的威力：一條邊如何把整張地圖拉正**




這張圖是真的跑出來的，不是示意。模擬一台機器人沿著 6 公尺見方的正方形走一圈，里程計帶有每步約 0.013 弧度的系統性角度偏差——這是真實輪式機器人非常典型的誤差來源，例如左右輪胎直徑略有差異。

左圖是只靠里程計推算的結果。注意它的形狀：每一段都還算直，轉角也都接近 90 度，看起來完全合理——但整體已經轉歪了。走完一圈回到起點時，推算位置與真實起點差了 2.56 公尺。這正是第 7 章積分漂移與第 11 章「預測步驟只會讓不確定度上升」在二維平面上的樣子。

關鍵的一步發生在機器人認出「我回到起點了」的那一刻。這個認出的動作由前端完成——在第 14、15 章是掃描匹配，在第 20、22 章是視覺詞袋。前端給出的結論寫成一條閉環邊：節點 N 與節點 0 之間的相對位姿應該是零。

右圖是加入這條邊之後最佳化的結果。整條軌跡被一起拉正，最終誤差從 2.56 公尺降到 0.00 公尺，全程平均誤差也從 1.18 公尺降到 0.04 公尺。請注意「一起」兩個字——被修正的不只是最後一個節點，而是中間每一個節點都分擔了修正量。這就是圖優化相對於濾波最根本的優勢：它能回頭改寫歷史。

**為什麼濾波做不到這件事**

濾波器在第 k 步就把第 k 步以前的資訊壓縮成一個均值與一個共變異數，中間節點的位置已經不存在於狀態中，自然無從修正。

圖優化保留了全部 N 個節點作為變數，閉環邊提供的新資訊可以透過一連串里程計邊「傳播」回去，讓每個節點都移動一點點。

代價是變數變多、要存全部歷史。這就是 12.7 節所說的取捨，也是為什麼實務上常把兩者分層並用：前端用濾波維持高頻，後端用圖優化維持全域一致。

## 12.3 節點、邊、限制條件

本節更仔細地看邊所代表的「限制條件」，以及如何量化「約束被滿足的程度」——這是把位姿圖變成可求解問題的關鍵。

### 殘差：量測與預測的差

每一條邊都帶有一個量測值（感測器說兩節點之間應該是什麼關係）。而根據當前對兩節點位姿的估計，我們也能算出一個預測值（按目前估計，兩節點之間實際是什麼關係）。兩者的差，稱為殘差（Residual）：

$$\textit{殘差} = \textit{量測值} - \textit{預測值}$$

殘差為零，表示這條邊的約束被完美滿足；殘差越大，表示當前的位姿估計越違背這條約束。後端的目標，就是調整所有節點，讓所有邊的殘差盡量小。

### 不是所有約束都一樣可信：資訊矩陣

這裡有一個關鍵細節：不同的邊可信度不同。一條來自高精度光達掃描匹配的邊，遠比一條來自打滑里程計的邊可信。最佳化時，應該更「尊重」可信的邊。

這個可信度用資訊矩陣（Information Matrix）描述——它正是第 11 章共變異數矩陣的反矩陣。共變異數大（不確定）對應資訊矩陣小（權重低）；共變異數小（確定）對應資

訊矩陣大（權重高）。於是可信的約束在最佳化中「說話比較大聲」。這讓圖優化與第 11 章的機率框架完美接軌：兩者其實是同一個機率問題的不同解法。

### 限制條件的一致性

位姿圖的美妙之處在於：它把各種來源的約束——里程計、掃描匹配、閉環、甚至第 4 章的 UWB 測距、第 5 章的 GNSS、第 7 章的 IMU 預積分——通通統一成「節點之間的邊」。無論約束來自哪個感測器，在圖裡都是一條帶殘差與資訊矩陣的邊。這種統一性，正是圖優化能優雅融合多感測器的原因，也呼應了前面各章反覆出現的「緊耦合」思想：把所有原始約束放進同一個最佳化問題一起求解。

## 12.4 Least Squares 最小平方

現在把前面的概念收攏成一個明確的數學問題。既然要讓所有邊的殘差都盡量小，自然的做法是：找一組節點值，使「所有殘差的平方和」最小。這就是最小平方（Least Squares）。

### 為什麼是「平方」和

為什麼不是殘差的絕對值和，而是平方和？有兩個理由：

- 數學理由：平方函數處處可微、形式漂亮，便於用微積分求極小值（令導數為零）。絕對值在零點不可微，處理起來麻煩。
- 機率理由：當雜訊是高斯分布時（第 11 章），「最小化殘差平方和」在數學上恰好等價於「最大化後驗機率」（找最可能的解）。換句話說，最小平方不是隨便選的，它是高斯假設下的最佳估計。這再次顯示圖優化與第 11 章機率框架是同一件事。

加上資訊矩陣作為權重後，更精確的說法是「最小化加權殘差平方和」——可信的邊權重大，對總和的影響也大。

### 如何求解：反覆逼近

這個最小平方問題通常是非線性的（因為殘差含 cosθ、sinθ 等，第 10 章），無法一步解出。實務上用迭代法反覆逼近：

79. 從一個初始猜測開始（通常用里程計串起來的粗略軌跡）。

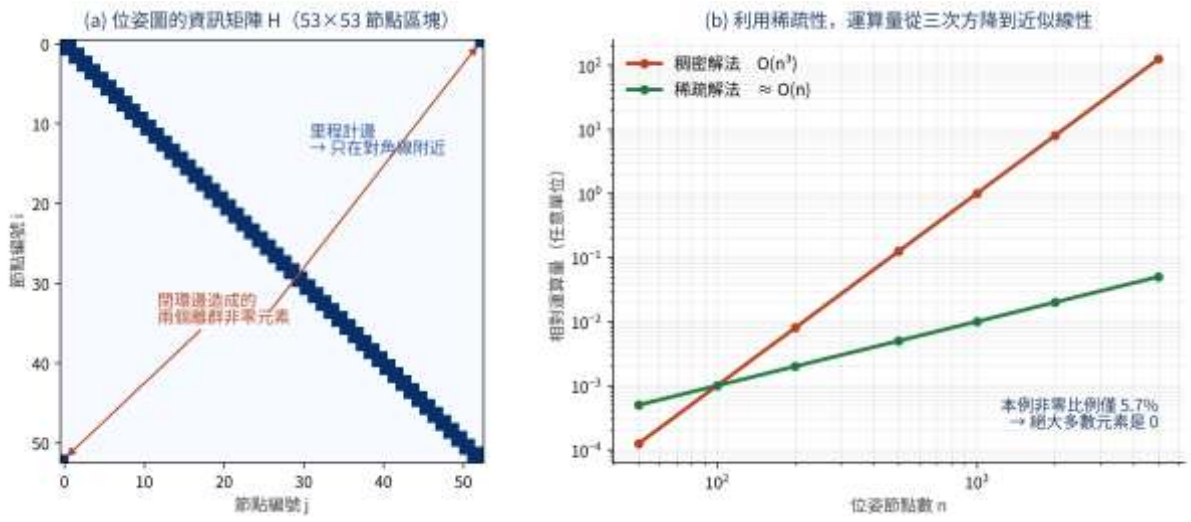
80. 在當前估計附近把問題線性化（求雅可比矩陣，第 11 章 EKF 用過同樣的技巧）。

81. 解一個線性方程組，得到一個「往誤差更小的方向修正」的步伐。

82. 更新估計，回到步驟 2，反覆進行直到殘差不再明顯下降（收斂）。

常用的迭代方法有高斯—牛頓法（Gauss-Newton）與列文伯格—馬夸特法（Levenberg-Marquardt）；後者更穩健，是許多 SLAM 系統的預設選擇。同學不需手算這些方法——這正是 g2o、Ceres、GTSAM 這些函式庫要替我們做的事（12.6 節）。

**圖 12-3　稀疏性：上萬個節點也能即時求解的祕密**




左圖畫出圖 12-2 那個位姿圖的資訊矩陣結構，深色代表非零。可以看到幾乎所有非零元素都擠在對角線附近——因為每條里程計邊只連接相鄰的兩個節點，不會牽涉到其他節點。唯一的例外是右上與左下兩個孤立的方塊，那正是閉環邊造成的，它把第一個與最後一個節點連了起來。

這個 53 個節點的小例子，非零比例已經只有 5.7%；若節點數增加到數千個，非零比例會降到千分之一以下。換句話說，這個矩陣幾乎全是零。

右圖說明這件事為什麼關鍵。一般稠密矩陣的解法運算量是 $O(n^3)$，一千個節點就要做十億次等級的運算，一萬個節點則是一兆次——完全不可能即時。但稀疏解法（稀疏 Cholesky 分解、共軛梯度法等）只處理非零元素，運算量接近 $O(n)$，於是上萬個節點也能在數十毫秒內解完。

這不是細節，而是整個現代 SLAM 得以成立的工程基礎。第 12.6 節的 g2o、Ceres、GTSAM 三套函式庫，核心價值有一大半就在於「把稀疏性用到極致」。同學在第 22 章編譯 ORB-SLAM2 時會安裝 g2o，那時請想起這張圖——它處理的就是這種形狀的矩陣。

**稀疏性：大問題能求解的祕密**

一個大場景的位姿圖可能有數萬個節點，對應數萬個未知數。直接解這麼大的線性方程組，計算量會大到不可行。幸好 SLAM 問題有一個救命的性質：稀疏性（Sparsity）。

稀疏性的意思是：每一條邊只牽涉兩個節點，而不是全部節點。因此線性化後的大矩陣裡，絕大多數元素是零。利用這個結構，有專門的稀疏矩陣演算法能極快地求解——這正是 EKF-SLAM（第 11 章）辦不到而圖優化能辦到的關鍵：EKF 的共變異數矩陣會變稠密（每個地標都與其他相關），而圖優化保持稀疏。這是圖優化能處理大場景的數學根源。

## 12.5 Bundle Adjustment

前面的位姿圖主要處理「位姿之間」的約束。在視覺 SLAM 中，還有一個更精細的最佳化問題，同時處理「相機位姿」與「三維地圖點」——這就是光束法平差（Bundle Adjustment,BA）。它是視覺 SLAM 精度的來源，也是第 22、23 章 ORB-SLAM 的核心。

**名字的由來**

「Bundle」（光束）指的是：從每個三維點，發出許多道光線，分別投射到看見它的各個相機。BA 的任務，就是調整所有相機位姿與所有三維點的位置，讓這些光束盡可能「對齊」——也就是讓每個三維點投影到影像上的位置，盡量接近實際觀測到的像素位置。這個「投影與觀測的差」，稱為重投影誤差（Reprojection Error）。

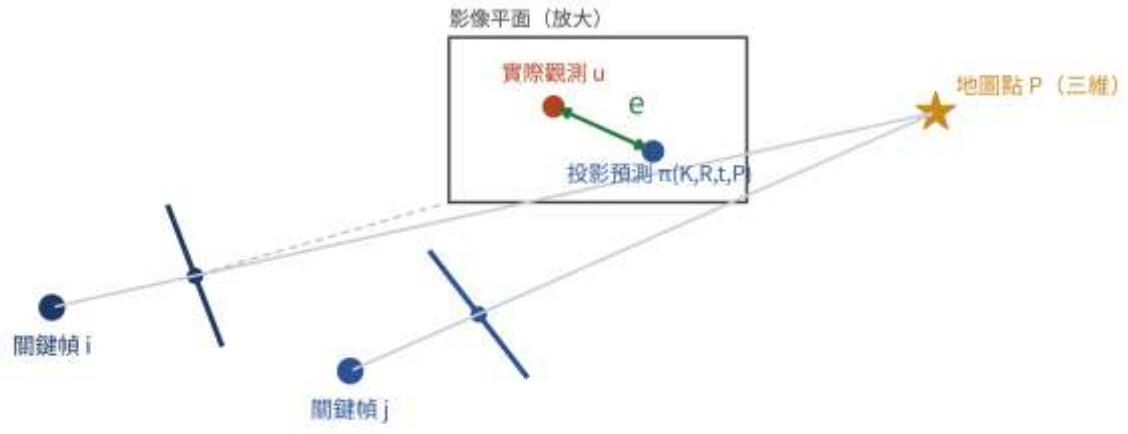
### 重投影誤差：視覺 SLAM 的殘差

重投影誤差是 BA 的核心殘差，它把第 9 章的相機投影模型用上了：

83. 取一個三維地圖點，用當前估計的相機位姿與內參 K（第 9 章），把它投影到影像上，得到一個「預測像素位置」。
84. 這個地圖點在該影像中，其實有一個「實際觀測到的像素位置」（前端特徵提取得到）。
85. 兩者的差（以像素為單位），就是重投影誤差。

BA 同時調整所有相機位姿與所有三維點，最小化所有重投影誤差的平方和。這是一個巨大的最小平方問題——一個場景可能有數百個相機位姿與數萬個三維點——但同樣靠稀疏性（每個點只被少數相機看到）得以高效求解。

**圖 12-4　重投影誤差：Bundle Adjustment 要最小化的那個 e**




Bundle Adjustment 與位姿圖的差別，就在殘差的定義。位姿圖的殘差是「兩個位姿之間的相對關係」，BA 的殘差則是圖中那個綠色箭頭：把三維地圖點 P 用當前估計的相機位姿與內參投影到影像上，投影結果與實際偵測到的特徵點位置之間的差距。

這個定義有兩個重要的性質。其一，它直接以像素為單位——誤差是「差了幾個像素」，

非常直觀，也方便設定門檻（ORB-SLAM 常用的外點判定就是重投影誤差超過某個像素數）。其二，它同時牽涉相機位姿與地圖點座標兩種變數，所以 BA 會一起調整兩者，這正是它名稱中「bundle」的由來——所有光線束一起調。

請把這張圖與第 9 章的圖 9-1 對照著看。那裡的 π 函數是「已知相機位姿與地圖點，算出像素位置」；這裡則是反過來，已知像素位置，去調整相機位姿與地圖點，使算出來的像素位置盡量貼近實際觀測。整個視覺 SLAM 的後端，做的就是這件事。

BA 的變數量遠大於位姿圖——每個地圖點都是三個變數，一個中型場景動輒數萬個點。所幸它的稀疏結構更漂亮：地圖點之間沒有直接的邊，只透過相機相連。利用這個結構做舒爾補（Schur Complement）消元，可以先解出相機位姿再回代地圖點，是 BA 能夠即時運行的關鍵技巧，第 22 章會再提到。

### BA 與 Pose Graph 的關係

- Bundle Adjustment：同時最佳化相機位姿與三維點，精度最高，但變數多、計算量大。適合局部（第 22 章的 Local Mapping 執行緒）或關鍵時刻。
- Pose Graph：只最佳化位姿（把三維點「邊緣化」掉），變數少、速度快。適合大範圍的閉環修正（第 22 章的 Loop Closing）。

現代視覺 SLAM 巧妙地結合兩者：平時用局部 BA 維持高精度，閉環時用位姿圖快速拉正大範圍軌跡。這種「局部 BA ＋全域位姿圖」的架構，是 ORB-SLAM 系列的經典設計，第 22、23 章會完整展開。BA 的理論也正是第 2 章 MIT VNAV、CMU 16-833 課程強調的核心——同學現在應能理解，為什麼那些頂尖課程要花大量篇幅在最佳化上。

## 12.6 g2o、Ceres、GTSAM

前面的數學看似複雜，但好消息是：同學幾乎不需要自己實作這些迭代求解與稀疏矩陣運算。有三個成熟的開源函式庫替我們把重活做好了。認識它們的定位，是進入 SLAM 實作的必備知識。

### 三大函式庫

- g2o(General Graph Optimization)：專為圖優化設計的 C++ 函式庫，由 SLAM 社群開發。使用方式非常貼近本章的概念——你定義節點（位姿、地標）與邊（約束），它替你求解。ORB-SLAM2 即使用 g2o。對學習者而言，g2o 的架構最能對應本章的位姿圖概念，是理解圖優化的好起點。
- Ceres Solver：由 Google 開發的通用非線性最小平方函式庫，不限於 SLAM。它功能強大、文件完善、自動微分方便（不必手算雅可比），被廣泛用於各種最佳化問題。Cartographer（第 15 章）即使用 Ceres。
- GTSAM(Georgia Tech Smoothing and Mapping)：由喬治亞理工學院開發（第 2 章 CMU 16-833 的 Kaess 教授即為核心貢獻者），以因子圖（Factor Graph）為核心概念，並提供 iSAM2 等增量式求解演算法——能在新資料到來時高效更新，而不必每次從頭重算。

### 因子圖：另一種等價觀點

GTSAM 使用的「因子圖」是位姿圖的一個更一般化的表述。在因子圖中，待估計的變數是一種節點，而每個約束（稱為「因子」）是另一種節點，連接它所牽涉的變數。它與位姿圖在數學上等價，但因子圖的表述更靈活，能自然地容納各種感測器約束。第 2 章提過，因子圖是 CMU 16-833 課程的核心工具——這也顯示本章內容與國際頂尖課程完全接軌。

| 函式庫 | 開發者 | 核心概念 | 代表使用者 |
|---|---|---|---|
| g2o | SLAM 社群 | 通用圖優化 | ORB-SLAM2 |
| Ceres | Google | 通用非線性最小平方 | Cartographer |
| GTSAM | 喬治亞理工 | 因子圖 + 增量式求解 | LIO-SAM 等 |

**表 12-1　三大圖優化函式庫比較**

對本書讀者的建議是：高中與大學部入門階段，理解「位姿圖—殘差—最小平方」的

概念即可，不必深入函式庫細節；大學部專題與研究階段，則可從 g2o 入手（概念最貼近本章），再依專案需求選用 Ceres 或 GTSAM。第 22、30 章的實作會實際接觸這些工具。

## 12.7 為什麼現代 SLAM 常用圖優化

本章最後，回答一個貫穿第 11、12 兩章的問題：既然第 11 章的濾波器已經能做 SLAM，為什麼現代主流系統幾乎都改用圖優化？這個對比，是理解當代 SLAM 走向的關鍵。

### 圖優化的四大優勢

86. 能回頭修正歷史：濾波器把過去壓縮進當前信念，無法回頭；圖優化保留整張圖，閉環時能一次性修正所有歷史位姿，把漂移漂亮地攤平。這是精度差異的最大來源。
87. 反覆線性化，精度更高：EKF 對每個量測只線性化一次（且無法重來）；圖優化在迭代中反覆重新線性化，逐步逼近真解，大幅減少第 11 章提到的線性化誤差。
88. 善用稀疏性，能處理大場景：圖優化的矩陣保持稀疏，配合稀疏演算法可處理數萬節點；EKF-SLAM 的共變異數矩陣會變稠密，隨地標平方成長而卡死。
89. 統一融合多感測器：各種約束都是圖上的邊，里程計、視覺、光達、IMU、GNSS 能自然地緊耦合在同一個問題裡（第 7 章的緊耦合思想）。

### 圖優化的代價

圖優化並非全無代價。它需要保留較多歷史資料（記憶體較大）、批次求解的單次計算量較大、且實作較複雜。早期受限於運算能力，濾波器才是主流；是計算能力的提升與稀疏求解演算法的成熟，才讓圖優化成為可能並最終勝出。這也提醒同學：技術的主流會隨硬體與演算法演進而變化，理解「為什麼」比記住「現在用什麼」更重要。

| 比較項目 | 濾波器（第 11 章） | 圖優化（本章） |
| --- | --- | --- |
| 處理方式 | 遞迴，逐筆更新 | 批次，一次性最佳化 |
| 歷史修正 | 無法回頭 | 可修正所有歷史 |

| 比較項目 | 濾波器（第 11 章） | 圖優化（本章） |
|---|---|---|
| 線性化 | 每量測一次 | 迭代中反覆進行 |
| 閉環處理 | 較弱 | 強項 |
| 大場景 | EKF 受限（稠密） | 善用稀疏，可擴展 |
| 計算與記憶體 | 固定、輕量 | 較大 |
| 代表系統 | AMCL、早期 EKF-SLAM | ORB-SLAM、Cartographer |

**表 12-2　濾波器與圖優化的總比較**

至此，第三篇「SLAM 數學基礎」全部完成。同學已經備齊了 SLAM 的完整數學工具箱：第 10 章的座標轉換讓我們能在不同座標系間精確換算、第 11 章的機率與濾波讓我們能處理不確定性、本章的最佳化與圖優化則讓我們能求出最一致的全域解。這三章是抽象的，但它們是後面所有具體系統的共同地基。從下一篇起，我們將把這些數學「用出來」——第四篇從最親民的 2D SLAM 與掃地機器人開始，看 Gmapping 如何用第 11 章的粒子濾波建圖、Cartographer 如何用本章的圖優化做閉環。理論與實作，即將在真實的機器人上會合。

## 公式或流程

### 一、SLAM 的最小平方目標函數

圖優化 SLAM 的核心，是求一組狀態 X（所有位姿與地標），使加權殘差平方和最小：

$$X^* = \operatorname{argmin} \Sigma_k \; e_k(X)^T \cdot \Omega_k \cdot e_k(X)$$

其中 $e_k$ 是第 k 條邊的殘差（量測與預測的差）、$\Omega_k$ 是該邊的資訊矩陣（可信度權重，即共變異數的反矩陣）、X* 是最佳解。這條式子把整個 SLAM 濃縮成一句話：找一組位姿與地圖，使所有約束的加權矛盾最小。第 11 章的機率最大化，在高斯假設下正等價於這個最小平方。

### 二、殘差的一般形式

位姿圖中，一條連接節點 i 與 j 的邊，其殘差為：

$$e_{ij} = \text{量測的相對位姿}(z_{ij}) \ominus \text{預測的相對位姿}(x_i, x_j)$$

符號 ⊖ 表示在位姿空間（第 10 章）中做「相減」——由於位姿含旋轉，這個相減不是普通減法，而需在旋轉群上進行（第 2 章 MIT VNAV 的流形運算）。殘差為零表示約束完美滿足。

## 三、重投影誤差（Bundle Adjustment）

視覺 SLAM 中，三維點 P 在第 i 個相機影像上的重投影誤差為：

$$e = u_observed - \pi(K, R_i, t_i, P)$$

其中 u_observed 是實際觀測到的像素座標、π(·) 是第 9 章的投影函數（用相機位姿 $R_i$、$t_i$ 與內參 K 把三維點 P 投影到影像）。BA 同時對所有相機位姿與三維點最小化所有重投影誤差平方和。

## 四、迭代求解流程（高斯—牛頓 / LM）

**圖 12-5　圖優化的迭代求解流程**

① 初始化：用里程計串起粗略的位姿（初始猜測 $X_0$）

↓

② 線性化：在當前 X 附近計算所有殘差與雅可比矩陣

↓

③ 建立線性系統：$H \cdot \Delta x = -b$　（H 為稀疏資訊矩陣）

↓

④ 稀疏求解：利用稀疏性快速解出修正量 Δx

↓

⑤ 更新：$X \leftarrow X \oplus \Delta x$　（在位姿空間上更新）

↓

⑥ 收斂？否 → 回到 ②　/　是 → 輸出最佳解 X*

這整個流程，就是 g2o、Ceres、GTSAM 在背後替我們執行的。同學理解流程即可，不需手動實作——正如開車不必自己造引擎。

## 12.8　圖優化的當代發展：更快、更大、更不怕錯

本章介紹的框架自 2010 年前後定型以來，骨架幾乎沒有改變。但在三個方向上，它一直在往前推進——速度、規模，以及對錯誤資料的容忍度。

### 一、增量式求解：不必每次都重算整張圖

最直接的問題是：機器人每走一步就多一個節點，難道每次都要重解整張圖？第 12.6 節提到的 GTSAM 給出的答案是 iSAM2——它把因子圖整理成一棵貝氏樹，新加入一條邊時只需要重新計算受影響的那一小部分分支，其餘維持不變。

這讓最佳化從「批次處理」變成「可即時增量更新」，是圖優化能在機器人上線的關鍵。它也解釋了一個現象：平常建圖時 CPU 很輕鬆，一偵測到閉環就突然忙碌起來——因為閉環邊會牽動整棵樹。

## 二、全域式方法重新回到檯面

在離線重建這一側，近年出現了值得注意的翻轉。長期以來，增量式的重建（逐張加入影像、反覆做 BA）被認為精度最高但極慢；全域式方法快但不夠穩。2024 年 ECCV 的一項工作重新設計了全域流程——視圖圖校正、全域定位、結構精修三個步驟，其中全域定位階段同時估計相機位置與影像點。

成果相當驚人：在一組三萬六千張影像、相機跨越兩百五十公尺以上的大型資料集上，它在 5.5 小時內完成重建，一公尺內的召回率達九成；而傳統增量式方法跑超過七天，召回率僅約五成。精度與增量式相當或更好，速度卻快了數個數量級。

對同學的意義是：如果專題需要用數千張照片重建一個校園或一棟建築（第 17 章、第 31 章的專題常有這類需求），現在有了實際可行的工具，不必再等上好幾天。

## 三、強健核函數：當前端給了錯誤的閉環

圖優化有一個致命弱點，本章 12.3 節的「限制條件的一致性」已經提過：它假設每一條邊都是對的。但前端會出錯——第 20 章的視覺詞袋可能把兩個長得很像的走廊誤判為同一處，第 17 章的點雲配準可能在對稱結構中收斂到錯誤的解。

一條錯誤的閉環邊有多嚴重？它會像一條強力的橡皮筋，把兩段本來不相鄰的軌跡硬拉在一起，整張地圖當場扭曲變形。而且因為最小平方對大殘差的懲罰是平方成長，這條錯誤的邊反而擁有最大的話語權——這是最小平方法天生的脆弱之處。

對策有兩個層次。第一層是強健核函數：把殘差的平方懲罰換成 Huber 或 Cauchy 這類成長較慢的函數，使大殘差的影響被壓住，離群值不再主導整個解。第二層更徹底——讓最佳化自己決定要不要相信某條邊，例如為每條閉環邊加一個開關變數，若它與其他約束嚴重

衝突就自動被關掉。

實務上這件事極為重要。同學在第 29 章做實體建圖時，若發現「地圖本來好好的，突然整個扭曲」，第一個該懷疑的就是誤判的閉環。而把求解器的強健核函數打開，往往就能解決。

### 四、與學習方法的結合

最後值得一提的，是圖優化與深度學習的分工正在定型。目前最成熟的模式不是用網路取代最佳化，而是讓網路負責前端——偵測與匹配特徵、判斷閉環、估計深度先驗——再把結果交給傳統的因子圖後端求解。

這個分工的理由與第 7.8 節、第 9.8 節、第 11.8 節完全一致：後端的幾何與機率結構提供可解釋性、不確定度與數學保證，前端的感知部分則交給擅長從資料中學規律的模型。第 23 章的 ORB-SLAM3 與第 32 章的 AI 輔助導航，都可以放在這個框架下理解。

**一個實用的除錯順序**

地圖扭曲 → 先懷疑錯誤的閉環邊（打開強健核函數，或提高閉環的驗證門檻）。

地圖整體歪斜但局部正常 → 懷疑里程計的系統性偏差（如圖 12-2 左圖），需要校正輪徑或輪距（第 29 章）。

閉環偵測不到 → 問題在前端，不在後端（第 15 章的分支定界、第 20 章的視覺詞袋）。

求解很慢 → 檢查是否誤用了稠密解法，或節點數是否該做關鍵幀篩選。

## 案例

### 案例 12-1　閉環的威力：一張歪掉的地圖如何被拉正

這是圖優化最戲劇性的展示，幾乎每個 SLAM 課堂都會演示。一台機器人繞行一棟大樓一圈，全程只靠里程計與掃描匹配建圖。走到一半，地圖看起來還算正常；但快繞完一圈時，

由於漂移累積，地圖開始「歪斜」——本該閉合的走廊，終點與起點錯開了好幾公尺，同一面牆甚至被畫成兩道分岔的線。

然後，閉環偵測啟動：系統認出「我現在看到的，就是一開始的大廳」，產生一條閉環邊。圖優化隨即執行——在螢幕上，同學會看到整張歪掉的地圖「唰」地一下被拉正：分岔的牆合而為一、錯開的走廊接合、整個軌跡變得規整。這一瞬間，本章 12.2 節「把矛盾均攤到整條軌跡」的抽象描述，變成了眼前活生生的畫面。

這個案例的震撼力在於：它讓同學親眼看到「一條邊如何修正整張圖」。這是濾波器做不到的——濾波器沒有保留歷史位姿，無法回頭。圖優化的全部價值，在這一個「拉正」的瞬間展露無遺。第 15 章 Cartographer 實作時，同學將親手觸發這個時刻。

## 案例 12-2　GTSAM 的血脈：從課堂到全球產業

第 2 章介紹 CMU 16-833 課程時提過：Michael Kaess 教授與 Frank Dellaert 教授開發的 GTSAM 函式庫，把因子圖與增量式平滑的理論包裝成開源工具，如今已是全球機器人公司與研究室的標準配備。本章的內容，正是理解 GTSAM 在做什麼的鑰匙。

GTSAM 的一項關鍵創新是 iSAM2（增量式平滑與建圖）。傳統圖優化每次有新資料都要重解整個問題；iSAM2 則聰明地只更新「受新資料影響的那部分」，大幅提升效率，讓大規模、長時間的 SLAM 得以即時運行。這解決了本章 12.7 節提到的「圖優化計算量大」的痛點。

這個案例串起了本書的多條線索：第 2 章的名校課程、本章的圖優化數學、以及真實世界的產業應用。它告訴同學：你正在學的這些數學，不是象牙塔裡的玩具，而是驅動水下機器人、自駕車、AR 裝置的實際引擎。理解本章，你就站在了與那些頂尖工程師相同的知識基礎上。

## 案例 12-3　從測量學到 SLAM:Bundle Adjustment 的百年血統

Bundle Adjustment 常被誤以為是電腦視覺的新發明，其實它的血統可追溯到十九世紀的攝影測量學（Photogrammetry）。當年的測量學家用多張空拍照片重建地形，面對的正是同一個問題：如何調整相機位置與地面點，讓所有照片的投影一致？他們發展出的平差理論，正是今天 BA 的前身。

這段歷史給同學兩個啟示。其一，最小平方與平差的思想歷久彌新——從十九世紀的測繪、二十世紀的阿波羅導航（第 10、11 章），到二十一世紀的 SLAM，人類處理「多個帶誤差量測」的核心方法一脈相承。其二，跨領域的知識常常相通：測量學、攝影測量、電腦視覺、機器人學，在 BA 這個點上交會。這也呼應第 10 章「機械手臂與行動機器人共用同一套數學」的觀察——數學是所有工程領域的共同語言，學好基礎，處處相通。

## 實作活動

### 實作 12-1　位姿圖最佳化動手做

目的：透過紙上推演與程式實作，親身理解位姿圖如何被「拉正」，體會閉環約束的威力。本實作分紙上、程式兩部分，適合不同深度的教學。

#### 第一部分：紙上位姿圖（全體）

73. 在方格紙上畫一個「正方形巡行」：機器人從原點出發，依里程計「向右 4 格、向上 4 格、向左 4 格、向下 4 格」應回到原點。先照里程計畫出四個轉角位姿。

74. 加入漂移：假設每段的轉角都少轉了一點（例如每次只轉 85 度而非 90 度），重新畫出軌跡，觀察終點如何無法回到原點（出現漂移缺口）。

75. 加入閉環邊：宣告「終點就是起點」。用直覺思考：要讓終點回到起點，又盡量不破壞四條邊的長度，四個轉角應該如何微調？動手把「拉正後」的軌跡畫出來。

76. 討論：閉環邊把誤差「均攤」到四個轉角，而不是集中修正某一個。為什麼均攤比集中好？

### 第二部分：Python 位姿圖最佳化（大學部）

77. 用 Python 建立一個簡單的 2D 位姿圖：數個位姿節點、連接它們的里程計邊（含模擬漂移）、以及一條閉環邊。

78. 實作殘差計算：對每條邊，算出「量測的相對位姿」與「當前估計的相對位姿」之差。

79. 用現成的最佳化工具（如 scipy.optimize.least_squares，或安裝 g2o / GTSAM 的 Python 介面）求解，最小化所有殘差平方和。

80. 把「最佳化前」與「最佳化後」的軌跡畫在一起，觀察漂移的軌跡如何被閉環邊拉正回閉合。

81. 實驗資訊矩陣：把閉環邊的權重（資訊矩陣）分別調大與調小，觀察拉正的力道如何變化。權重大時，系統更「相信」閉環，拉得更用力。

### 第三部分：觀察真實 SLAM 的閉環（有 ROS 環境者）

82. 用 Cartographer（第 15 章）或其他支援閉環的 SLAM，在模擬環境中讓機器人繞一圈，在 RViz 中即時觀察：繞行時地圖逐漸漂移，閉環觸發瞬間地圖被拉正。

83. 錄下這個「拉正瞬間」，對照案例 12-1 與本章 12.2 節，指認哪些是里程計邊、哪條是閉環邊。

### 討論問題

59. 第一部分中，加入閉環邊後，你如何微調四個轉角？最佳化把誤差均攤，和「把所有誤差算在最後一段」相比，哪個結果的地圖更合理？為什麼？

60. 第二部分中，調大閉環邊的權重，拉正的力道有何變化？若閉環偵測其實是「誤判」（錯誤的閉環），高權重會造成什麼後果？

61. 本章說圖優化「能回頭修正歷史」，而第 11 章的濾波器不能。從你的實驗結果，說明這個差異如何影響最終地圖品質。

62. （大學部）為什麼稀疏性讓大型位姿圖得以快速求解？若每條邊都牽涉所有節點（不稀疏），會發生什麼？

**延伸挑戰（大學部）**

安裝 g2o 或 GTSAM 的 Python 綁定，載入一個公開的標準位姿圖資料集（如 g2o 格式的 intel、manhattan 資料集），執行最佳化並用 matplotlib 視覺化最佳化前後的差異。進一步嘗試「加入一條錯誤的閉環邊」，觀察它如何破壞整張地圖——這將幫助你理解為何第 20、22 章要花心力在「穩健的閉環偵測」上，以及穩健核函數（Robust Kernel）為何重要。

## 延伸閱讀


76. G. Grisetti, R. Kümmerle, C. Stachniss, and W. Burgard, "A Tutorial on Graph-Based SLAM," IEEE Intelligent Transportation Systems Magazine, 2010.（圖優化 SLAM 最經典的入門教程，本章主要依據）

77. 高翔、張濤，《視覺 SLAM 十四講》第 6 講（非線性最佳化）、第 9-10 講（後端與 BA、位姿圖）.（華文世界最清楚的圖優化實作教材）

78. R. Kümmerle et al., "g2o: A General Framework for Graph Optimization," ICRA, 2011.（g2o 函式庫原始論文）

79. F. Dellaert and M. Kaess, "Factor Graphs for Robot Perception," Foundations and Trends in Robotics, 2017.（GTSAM 與因子圖的權威著作，可免費取得）

80. B. Triggs et al., "Bundle Adjustment — A Modern Synthesis," Vision Algorithms, 2000.（Bundle Adjustment 的權威綜述）

81. Ceres Solver 官方文件（ceres-solver.org）與 GTSAM 官方教學（gtsam.org）.（三大函式庫的第一手實作指引）

82. CMU 16-833 Robot Localization and Mapping 課程（第 2 章介紹）之因子圖與圖優化單元.（研究所級的圖優化教材）

## 第三篇完結：從數學回到機器人

第三篇三章到此結束。第 10 章給了表達位姿的語言，第 11 章給了描述不確定性的語言，本章給了求解的方法。三者合起來，就是整個現代 SLAM 的數學骨架。

回頭看，這三章其實在回答同一個問題的三個層面。「我在哪裡？」——第 10 章說，這是一個位姿 T；第 11 章說，它其實是一個分布；本章說，要找出它，就把所有約束放在一起解一個最小平方問題。

從下一章開始，第四篇要把這些工具真正用起來。第 13 章會把光達的掃描資料變成一張佔據格地圖——那裡會同時用到第 10 章的座標轉換、第 11 章的機率更新，以及本章「用多次觀測求一致解」的思想。第 14 章的 Gmapping 是粒子濾波的實作，第 15 章的 Cartographer 則是本章位姿圖的實作。

換句話說，第三篇學的不是「數學」，而是後面六篇每一章都會用到的工具。如果某個概念現在還覺得抽象，不必焦慮——到第 29 章親手看著一張歪掉的地圖在閉環那一刻被拉正時，本章的一切都會變得具體。

## 小結

49. 現代 SLAM 採前端 + 後端架構：前端把感測資料轉成幾何約束（掃描匹配、特徵匹配、閉環偵測），後端在所有（常互相矛盾的）約束下求最一致解；圖優化屬於後端。
50. 位姿圖以節點（待估的位姿與地標）與邊（相對量測約束）描述 SLAM;里程計邊像鏈子串起位姿、閉環邊形成迴路，是消除累積漂移的關鍵。
51. 每條邊有殘差（量測減預測）與資訊矩陣（可信度權重，即共變異數的反矩陣）；圖優化把各種感測器約束統一成帶權重的邊，實現多感測器緊耦合。

52. SLAM 化為最小平方問題：找一組狀態使加權殘差平方和最小；用平方兼具數學（可微）與機率（高斯下等價於最大後驗）理由，以高斯—牛頓 / LM 迭代求解，並靠稀疏性處理大場景。

53. Bundle Adjustment 同時最佳化相機位姿與三維點，最小化重投影誤差，是視覺 SLAM 精度的來源；現代系統結合「局部 BA（高精度）+ 全域位姿圖（快速閉環）」。

54. g2o（通用圖優化，ORB-SLAM2 使用）、Ceres（Google 通用最小平方，Cartographer 使用）、GTSAM（因子圖 + 增量式 iSAM2）三大函式庫替我們完成求解；因子圖是位姿圖的一般化表述。

55. 現代 SLAM 多採圖優化，因它能回頭修正歷史、反覆線性化提升精度、善用稀疏性處理大場景、統一融合多感測器；代價是記憶體與計算量較大，是硬體與演算法進步後才成為主流。

第三篇「SLAM 數學基礎」到此完成。我們已備齊完整的數學工具箱：座標轉換（第 10 章）、機率與濾波（第 11 章）、最佳化與圖優化（本章）。這三章抽象卻是一切的地基。從下一篇起，理論將在真實機器人上落地——第四篇從最親民的 2D SLAM 與掃地機器人開始，你會看到 Gmapping 如何運用第 11 章的粒子濾波、Cartographer 如何運用本章的圖優化。數學與實作，即將會合。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第四篇 2D SLAM 與掃地機器人

## 第 13 章

## 2D 地圖建構

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

## 本章在全書的位置

第三篇把工具都備齊了：第 10 章的座標轉換、第 11 章的機率估測、第 12 章的最佳化。從本章開始，第四篇要把這些工具真正組裝成一套會動的系統。

而第一件事，是把感測器量到的東西變成一張地圖。本章的佔據格地圖是全書第一種「真的可以存進檔案、交給導航系統使用」的地圖格式——第 14、15 章的 SLAM 系統輸出它，第 16 章的掃地機器人靠它規劃清掃路徑，第 29 章的 Nav2 把它當成導航堆疊的標準輸入。

本章刻意做了一個簡化：假設位姿是已知的。這當然不成立——第 1 章就說過那是雞生蛋問題——但先把「已知位姿怎麼建圖」弄清楚，第 14、15 章才能專心處理「位姿不準怎麼辦」。這是教學上必要的拆解。

**圖 13-0　第 13 章與全書的連結地圖**

【承接前章】

第 10 章　座標轉換　→　把光達點從機體座標搬到世界座標

第 11 章　機率　　→　每一格存的不是 0 / 1，而是佔據機率

第 8 章　　光達規格　→　角解析度決定地圖能畫多細

【本章負責】

佔據格地圖的資料結構　→　對數勝算比更新　→　光線投射

掃描匹配（ICP）　→　里程計與光達的融合

解析度取捨、地圖品質評估

【往下延伸】

→ 第 14 章　Gmapping：把位姿也一起估（粒子濾波路線）

→ 第 15 章　Cartographer：子圖 + 分支定界閉環（最佳化路線）

→ 第 16 章　掃地機器人：佔據格地圖的消費級應用

→ 第 17 章　ICP 的三維版本：點雲配準

→ 第 29 章　Nav2 以本章的地圖為輸入做路徑規劃

## 學習目標

前三篇打好了觀念與數學地基，從本篇開始，理論要在真實機器人上落地了。第四篇聚焦最親民、也最貼近生活的 2D SLAM——掃地機器人用的正是這套技術。本章先講清楚「2D 地圖是什麼、怎麼建」，為後兩章的 Gmapping 與 Cartographer 鋪路。學完本章之後，同學應能夠達成下列目標：

76. 說明佔據格地圖的結構，以及每一格「佔據機率」的意義與更新方式。

77. 說明雷射掃描匹配如何估計機器人的相對移動。

78. 解釋為什麼要融合里程計與光達，兩者如何互補。

79. 說明地圖解析度的選擇如何影響地圖品質、記憶體與運算。

80. 分析走廊、教室、辦公室等典型室內場景的 2D 建圖挑戰。

81. 說明如何評估一張 2D 地圖的品質，辨識變形、斷裂與漂移等常見問題。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
| --- | --- | --- |
| 佔據格地圖 | Occupancy Grid Map | 把空間切成規則格子，每格記錄被佔據的機率。 |
| 佔據機率 | Occupancy Probability | 某格子被障礙物佔據的機率，介於 0 與 1 之間。 |
| 對數勝算比 | Log-odds | 以對數形式表示機率，使地圖更新變成簡單的加減法。 |
| 掃描匹配 | Scan Matching | 比對前後兩次雷射掃描，估計機器人的相對移動。 |
| ICP | Iterative Closest Point | 反覆找對應點並對齊的掃描匹配經典演算法。 |
| 里程計 | Odometry | 由輪速估計的相對位移，連續但會漂移。 |
| 光線投射 | Ray Casting | 沿雷射光路徑更新途經格子的方法。 |
| 地圖解析度 | Map Resolution | 每個格子代表的實際邊長，如 5 公分。 |
| 自由空間 | Free Space | 雷射穿過、確認無障礙的區域。 |
| 未知區域 | Unknown | 感測器尚未觀測到的區域。 |
| 地圖漂移 | Map Drift | 因累積誤差導致地圖逐漸偏移或變形。 |
| 逆感測器模型 | Inverse Sensor Model | 由一次量測推算沿途各格佔據機率的模型。 |

## 13.1 Occupancy Grid Map

第 1 章與第 3 章我們初識過佔據格地圖，本節深入它的運作機制——這是 2D SLAM 最核心的地圖表示法，掃地機器人 App 上那張房間地圖，底層就是它。

### 把空間切成棋盤

佔據格地圖（Occupancy Grid Map）的想法極其樸素：把機器人活動的平面切成一格一格的方格（像棋盤或方格紙），每一格記錄一個數字——這一格「被障礙物佔據的機率」。

- 機率接近 1（習慣畫成黑色）：幾乎確定有障礙，例如牆、桌腳。
- 機率接近 0（畫成白色）：幾乎確定是空的，可以通行。
- 機率為 0.5（畫成灰色）：不知道——感測器還沒看過這一格。

這三種狀態（佔據、自由、未知）的區分至關重要。特別是「未知」——它不等於「自由」。地圖邊緣那一大片灰色，代表機器人還沒探索的地方，不能當成可通行區域，否則規劃路徑時會「以為那裡是空的」而出錯。這是佔據格地圖比單純「有/無障礙」二元地圖高明的地方。

### 每一格獨立處理

佔據格地圖有一個關鍵假設：每一格的佔據狀態互相獨立。這在數學上並不完全成立（相鄰格子顯然相關——牆是連續的），但這個簡化讓計算變得極其高效：每來一筆雷射量測，只需更新它掃到的那幾格，不必考慮格子之間的關聯。這是工程上「用合理的簡化換取可行的計算」的又一個範例，與第 11 章佔據格地圖採高斯假設的精神一致。

### 前提：必須先知道位姿

這裡要強調一個第 1 章反覆出現的觀念：建佔據格地圖的前提，是已知機器人每一刻的位姿。因為一筆雷射量測「前方三公尺有牆」，只有知道機器人在哪、朝哪，才能決定這面牆該畫在地圖的哪一格。

於是問題來了——位姿從哪來？這正是 SLAM 的雞生蛋問題。本章先假設「位姿已由某種方式提供」（例如里程計加掃描匹配），專注於「有了位姿，如何建圖」；至於「如何同時估好位姿」，是第 14、15 章 Gmapping 與 Cartographer 的核心。把「建圖」與「定位」暫時拆開理解，是學習 SLAM 的有效策略。

## 13.2 佔據機率的更新：對數勝算比

機器人不會只看一眼就下結論——它會反覆掃描同一個地方，每次掃描都提供新證據，地圖應該把這些證據累積起來。本節說明這個累積是怎麼做的，以及一個讓計算變簡單的巧妙技巧。

### 累積證據：貝氏更新

每一格的佔據機率，隨著一次次觀測而更新。若某格連續多次被雷射打到（每次都說「這裡有東西」），它的佔據機率就逐漸逼近 1;若連續多次被雷射穿過（每次都說「這裡是空的」），機率就逼近 0。這正是第 11 章貝氏更新的應用——用新證據修正既有信念。

但直接對機率做貝氏更新，需要反覆做乘法與除法（還要正規化），計算較繁瑣，而且機率被限制在 0 到 1 之間，接近邊界時數值容易不穩定。於是實務上採用一個漂亮的技巧。

### 對數勝算比：把乘除變成加減

這個技巧是把機率改寫成對數勝算比（Log-odds）形式。勝算（odds）是「佔據機率 ÷ 空置機率」，再取對數就是對數勝算比。它的好處立竿見影：

- 更新變成加法：每來一筆觀測，只需把對應的「對數勝算比增量」加上去即可——乘除變成了加減，計算極快。
- 數值範圍無界：對數勝算比可以是任意實數（正無窮對應必定佔據、負無窮對應必定自由、0 對應五五波），不再卡在 0 到 1 的邊界，數值穩定。

- 需要時再轉回機率：平時都用對數勝算比累積，只在要顯示或決策時，才轉換回 0 到 1 的機率。

這個「取對數把乘除變加減」的手法，在工程與科學中隨處可見（例如分貝、pH 值），佔據格地圖只是其中一例。同學不必記憶公式，但要理解這個設計動機：讓「累積大量觀測」這件事變得又快又穩。

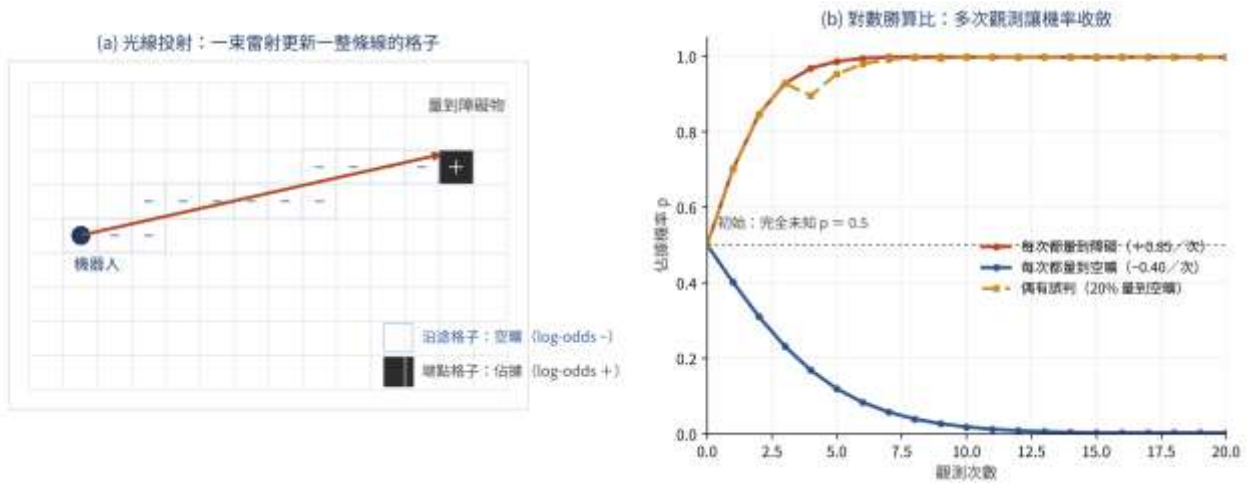
### 光線投射：一筆量測更新一整條線

這裡還有一個容易忽略但很重要的細節。當雷射量到「前方三公尺有牆」，它其實更新了一整條線上的格子，而不只是牆那一格：

- 雷射終點那一格（牆）：增加佔據機率（這裡有東西）。
- 雷射沿途穿過的所有格子（三公尺內）：降低佔據機率（雷射能穿過，表示這些地方是空的）。

這個「沿雷射路徑更新沿途格子」的過程稱為光線投射（Ray Casting），背後的模型稱為逆感測器模型（Inverse Sensor Model）——由一次量測反推沿途各格的佔據狀態。正是這個機制，讓機器人不只知道「哪裡有牆」，也知道「哪裡是確定可走的空間」，這對第 29 章的路徑規劃至關重要。

**圖 13-1　光線投射與對數勝算比更新**




左圖是光線投射的幾何。一束雷射量到「三公尺處有東西」，這句話其實帶來兩種資訊：端

點那一格有障礙物（加上正的 log-odds），而沿途經過的每一格都沒有東西擋住（各自加上負的 log-odds）。所以一束雷射不是只更新一格，而是更新一整條線——這也是為什麼機器人只要在房間裡走一圈，整個空曠區域就會被「掃」成白色。

右圖說明為什麼要用對數勝算比而不是直接存機率。紅線是某一格每次都被量到障礙的情形：第一次觀測後機率從 0.5 跳到約 0.70，三次後超過 0.90，五次後接近 0.99。藍線則是每次都被量到空曠，機率迅速趨近 0。

最值得注意的是黃色虛線：這一格大多數時候被量到障礙，但有 20% 的機會因為雜訊或反光而被誤判為空曠。可以看到曲線只是短暫下滑又被拉回去——單次的誤判無法推翻累積的證據。這正是機率式地圖相對於「看到就標黑」的根本優勢，也是第 11 章貝氏更新在二維格網上的具體實現。

> **為什麼要取對數**
>
> 貝氏更新本身是乘法：新的勝算 = 舊的勝算 × 量測的勝算。乘法做多了容易發生數值下溢，而且運算較慢。
>
> 取對數之後乘法變成加法，每次更新只需要一個整數或浮點數加法——這對每秒要更新數萬格的即時系統是決定性的差異。
>
> 還有一個隱藏好處：log-odds 可以直接設定上下限（例如 ±6），避免某一格因為被觀測上千次而「鎖死」，使地圖永遠無法反映環境變化。13.5 節的動態更新正是靠這個機制。

## 13.3　雷射掃描匹配

上一節假設「位姿已知」。但位姿到底怎麼估？除了里程計，光達本身就能提供強大的位姿線索——這就是掃描匹配（Scan Matching），2D SLAM 前端的核心技術。

### 核心想法：讓兩次掃描對齊

掃描匹配的想法非常直觀。機器人在位置 A 掃描一次，前進一小步到位置 B 再掃描一次。如果環境沒變（牆還是那面牆），那麼「B 的掃描」相對「A 的掃描」的位移與旋轉，就反映了機器人從 A 到 B 的移動。

換句話說：找出一個旋轉與平移，把 B 的掃描疊到 A 的掃描上，使兩者「最吻合」——這個旋轉與平移，就是機器人的相對移動。這是一個最佳化問題（第 12 章的精神）：調整位姿，讓兩次掃描的差異最小。

### ICP：最經典的掃描匹配

最著名的掃描匹配演算法是 ICP（Iterative Closest Point，迭代最近點）。它的流程是一個反覆逼近的循環：

90. 對應（Correspondence）：對 B 掃描中的每個點，在 A 掃描中找最近的點，配成一對。
91. 求解（Alignment）：計算一個旋轉與平移，使所有配對點的距離總和最小。
92. 更新與迭代：套用這個變換，然後回到步驟 1 重新找對應點，反覆進行直到不再改善（收斂）。

ICP 的名字精準描述了它的運作：反覆（Iterative）地找最近點（Closest Point）並對齊。它簡單有效，是點雲配準的基石，第 17 章的 3D 建圖還會再遇到它。

### 掃描匹配的優點與陷阱

- 優點：光達精確，掃描匹配估出的相對移動，通常比里程計準得多，尤其在轉彎時不受輪子打滑影響。
- 陷阱一——環境重複：走廊、空曠大廳等「處處長得一樣」的場景，掃描匹配會找不到唯一的對齊方式（沿走廊滑動，掃描看起來都差不多），導致估計失準。這與第 9 章「白牆的詛咒」是同一類問題：缺乏獨特特徵。

- 陷阱二——初始猜測：ICP 需要一個好的初始猜測才容易收斂到正確解，否則可能陷入錯誤的局部最佳。這正是里程計派上用場的地方——下一節說明。

## 13.4 里程計與光達融合

第二篇反覆強調「感測器融合」，2D SLAM 是這個思想最乾淨的示範：里程計與光達的優缺點恰好互補，合起來遠勝任何一方單獨使用。

### 兩者的互補

| 特性 | 里程計（輪速） | 光達掃描匹配 |
|---|---|---|
| 資料連續性 | 連續、高頻、平滑 | 受掃描頻率限制 |
| 短期精度 | 尚可 | 高（尤其轉彎） |
| 長期行為 | 漂移累積 | 相對穩定 |
| 失效場景 | 打滑、地面不平 | 環境重複、特徵不足 |
| 需要初始猜測 | 不需要 | 需要（否則易發散） |

**表 13-1 里程計與光達掃描匹配的互補**

### 融合的邏輯：里程計給起點，光達來校正

看出關鍵了嗎？里程計不需要初始猜測、且連續平滑，恰好能為「需要初始猜測」的掃描匹配提供起點；而掃描匹配精確、不漂移，恰好能校正里程計的漂移。兩者一拍即合：

93. 機器人移動一步，里程計先給出一個粗略的相對移動估計（初始猜測）。

94. 以這個猜測為起點，執行掃描匹配（ICP），精修出更準確的相對移動。

95. 用精修後的位姿更新地圖（13.2 節），並作為下一步的基礎。

這個「里程計預測、光達校正」的循環，正是第 11 章貝氏濾波「預測—更新」的具體化身：里程計負責預測（運動模型）、光達負責更新（觀測模型）。同學應該能感受到，第

三篇的抽象數學，在此變成了可操作的建圖流程。第 14 章的 Gmapping 與第 15 章的 Cartographer，都是這個基本循環的精緻化。

### 打滑時怎麼辦

當輪子打滑，里程計會突然給出錯誤的大位移。此時掃描匹配的價值更加凸顯：即使里程計說「我前進了半公尺」，只要光達掃描顯示「環境幾乎沒變」，掃描匹配就會把估計拉回「其實幾乎沒動」。這種「以環境觀測否決錯誤運動估計」的能力，正是 SLAM 比純里程計航位推算強韌的根本原因。

## 13.5 地圖解析度與更新

建 2D 地圖時，一個必須做的關鍵決定是：每個格子代表多大的實際範圍？這就是地圖解析度（Map Resolution），它牽動地圖品質、記憶體與運算的三方權衡。

### 解析度的取捨

地圖解析度指每個格子的實際邊長，室內常用 5 公分。這個選擇是一個典型的工程權衡：

- 解析度高（格子小，如 1 公分）：地圖細膩，能呈現細小結構（桌腳、門框），但記憶體用量暴增（格子數與邊長平方成反比——邊長減半，格子數變四倍），運算也變慢。
- 解析度低（格子大，如 20 公分）：記憶體省、運算快，但地圖粗糙，細小障礙可能被整格「抹平」，牆的位置也不精確。

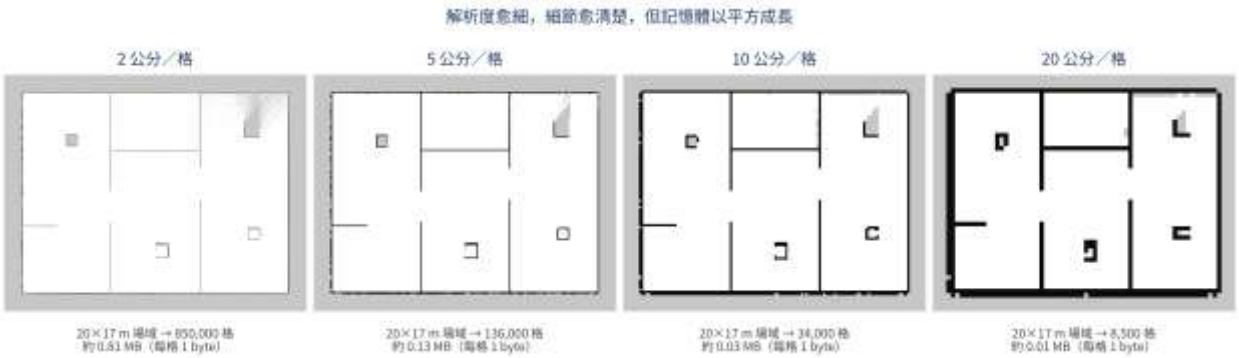
如何選？取決於應用需求。掃地機器人不需分辨一公分的細節，5 公分綽綽有餘；而需要精密對接的機器人，可能需要更高解析度。這呼應第 8 章光達角解析度的討論——地圖解析度再高，也不可能超越感測器本身的解析能力，兩者要匹配。

### 記憶體的現實

讓同學對記憶體用量有具體概念：一個 20 公尺 × 20 公尺的房間，用 5 公分解析度，就

是 400 × 400 = 16 萬個格子。若解析度提高到 1 公分，則變成 2000 × 2000 = 400 萬個格子——整整 25 倍。對大範圍場景（整棟大樓），這個成長會很可觀，這也是第 3 章提到 3D 地圖要用八叉樹（OctoMap）壓縮的原因。2D 佔據格因為只有平面，記憶體壓力相對小得多，這也是它適合教學與掃地機器人的原因之一。

**圖 13-2　解析度的取捨：同一個場域的四種格子大小**




這四張圖是同一趟掃描、同一份資料，只改變格子大小。2 公分的地圖牆面細緻、家具輪廓清楚；20 公分的地圖牆變成粗黑的塊，桌腳與椅子則整個糊成一團，甚至連門口的開口都快要被填平。

但代價寫在每張圖下面。以 20×17 公尺的場域計算，2 公分需要 85 萬格、約 0.81 MB；5 公分降到 13.6 萬格；10 公分只剩 3.4 萬格；20 公分則是 8,500 格。格子邊長減半，格數變四倍——記憶體以平方成長，而更新所需的運算量也同步增加。

實務上的常見選擇是 5 公分，這不是巧合：它剛好能分辨出一般門框（約 80 至 90 公分）、走道（約 1 公尺）與桌腳之間的間隙，而記憶體又還在嵌入式平台能負擔的範圍。掃地機器人為了省記憶體常用 5 至 10 公分；需要穿越狹窄通道的倉儲 AGV 則會用到 2 至 3 公分。

選型的方法與第 8.4 節判斷光達角解析度完全相同：先問「我最小要分辨多寬的空隙」，再回推格子大小。而且兩者必須搭配——用 2 公分的格子去存 1 度角解析度的光達資料，只會得到一張充滿空洞的地圖，因為感測器根本提供不了那麼細的資訊。

### 動態更新：地圖不是拍一張照就好

地圖需要持續更新，原因有二：

- 探索新區域：機器人走到沒去過的地方，把「未知」的灰色格子填成「自由」或「佔據」。
- 修正舊估計：反覆觀測同一區域，累積證據修正之前不確定的格子（13.2 節的貝氏更新）。

但這帶來一個微妙問題：如果環境本身變了怎麼辦？例如有人搬動了椅子。若地圖對舊觀測「記憶」太深（對數勝算比累積太高），就很難更新成新狀態；若記憶太淺，又容易被雜訊干擾。實務上會對對數勝算比設上下限（clamping），讓地圖保持一定的「可塑性」，能反映環境變化。這是第 16 章掃地機器人「動態環境」討論的伏筆。

## 13.6 走廊、教室、辦公室的 2D 建圖

理論講完，來看真實室內場景。不同的空間結構，對 2D 建圖有截然不同的挑戰。理解這些，同學才能預判「哪裡容易出問題」，這是實作前的重要準備。

### 教室：2D SLAM 的友善場景

一般教室是 2D SLAM 最理想的場景：四面牆界線分明、有桌椅提供豐富的幾何特徵、空間封閉不大。掃描匹配在這裡能穩定運作，建出的地圖清晰。這也是為什麼本書的實作活動多以教室為場域（第 14、29 章）——它讓同學在最順利的條件下先建立成功經驗。

### 走廊：掃描匹配的惡夢

長直走廊是 2D SLAM 的經典難題，正是 13.3 節「環境重複」陷阱的典型。走廊兩側是平行的長牆，沿走廊方向移動時，雷射掃描看起來幾乎不變——掃描匹配無法判斷「沿走廊走了多遠」（這個方向缺乏約束），只能確定「有沒有偏離走廊中線」（垂直方向有牆約束）。

這正是第 11 章 11.2 節那個「沿走廊方向拉長的不確定性橢圓」的來源！此時里程計的

價值凸顯——它能提供沿走廊方向的移動估計，補上掃描匹配缺失的約束。這是里程計與光達融合（13.4 節）不可或缺的又一個實例。長走廊建圖若只靠光達，幾乎必定失敗。

### 辦公室：動態與玻璃的挑戰

開放式辦公室綜合了多種挑戰：隔間玻璃讓光達「看穿」或產生錯誤反射（第 8 章的玻璃問題）、人員走動造成動態干擾、相似的格位排列容易混淆。這類環境往往需要更精細的參數調校，或搭配其他感測器。

### 大廳與中庭：空曠的困境

大型空曠空間（如挑高大廳）的問題是：光達的量測範圍有限（教學型約 8 至 12 公尺，第 8 章），當空間大於量測範圍，機器人走到中央時可能「四面都掃不到牆」，完全失去參考。此時掃描匹配無從進行，只能倚賴里程計硬撐，誤差快速累積。

| 場景 | 主要挑戰 | 關鍵對策 |
|---|---|---|
| 教室 | 幾乎無（理想場景） | 直接建圖即可 |
| 長走廊 | 環境重複，沿走廊方向無約束 | 倚賴里程計補足縱向估計 |
| 辦公室 | 玻璃、動態人員、格位相似 | 參數調校、多感測器 |
| 空曠大廳 | 超出光達量程，四周掃不到 | 里程計硬撐、加大量程或加地標 |

**表 13-2　典型室內場景的 2D 建圖挑戰**

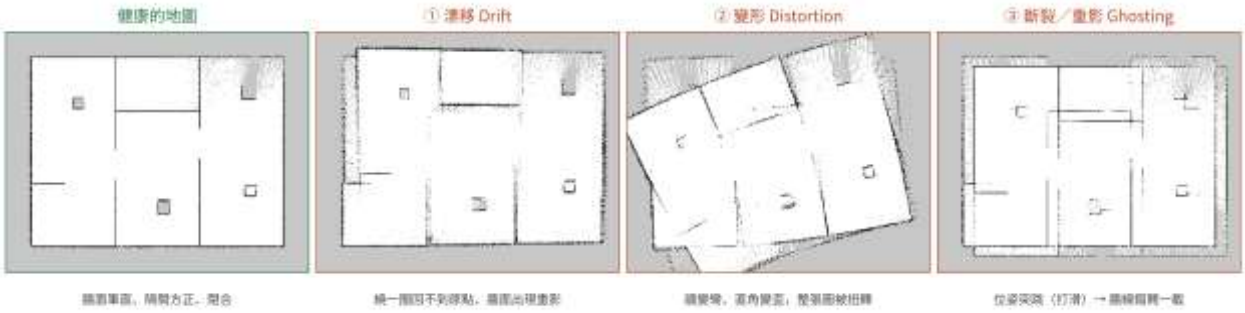
## 13.7　地圖品質評估：變形、斷裂、漂移

建完地圖，怎麼判斷它「建得好不好」？這是實作中極重要卻常被忽略的一環。本節教同學辨識三種常見的地圖病徵，以及如何評估品質——這也是第 31 章專題評分 Rubric 的基礎。

### 三種常見的地圖病徵

- 漂移（Drift）：最常見的問題。累積誤差使地圖逐漸偏移，最典型的症狀是「繞一圈回不到原點」——本該閉合的迴路，終點與起點錯開。這正是第 12 章閉環要解決的問題，也是判斷「需不需要閉環偵測」的依據。
- 變形（Distortion）：地圖的幾何被扭曲，直牆變成彎的、直角變成歪的、房間比例失真。多半來自掃描匹配的系統性誤差或里程計的尺度誤差（輪徑校正不準）。
- 斷裂/重影（Ghosting）：同一面牆在地圖上出現兩道（重影），或牆線斷開。通常是位姿估計在某處突然跳錯，導致同一結構被畫在兩個位置。走廊、空曠處最容易發生。

**圖 13-3 三種常見的地圖病徵**




這四張圖同樣來自同一份感測資料，差別只在位姿。最左邊是位姿正確時該有的樣子：牆面筆直、隔間方正、外框閉合。

① 漂移 Drift：位姿緩慢而持續地偏移，使同一面牆在不同時間被畫在不同位置，最終呈現為兩條平行的線；繞一圈回到原處時，起點與終點也對不上。這是最典型的「沒有閉環」症狀——機器人回來了，卻不知道自己回來了。

② 變形 Distortion：位姿帶有持續的角度偏差，整張地圖被漸進地扭轉。症狀是「每一小段看起來都正常，但整體歪掉」，走廊不再是直的、房間變成菱形。原因通常是輪徑或輪距校正不準（第 29 章會實際校正），或掃描匹配在長走廊中失效。

③ 斷裂 / 重影 Ghosting：位姿在某個瞬間突然跳動——機器人打滑、被人推了一下、或掃描匹配收斂到錯誤的解。症狀很好認：牆面在某個位置整齊地錯開一截，彷彿地圖被剪開後

沒對準就黏回去。

**從病徵反推病因**

看到漂移（牆面重影、起終點對不上）→ 閉環沒有被偵測到，或被偵測到卻沒有被接受。第 15 章的分支定界與第 22 章的視覺詞袋就是在解決這件事。

看到變形（牆彎、直角歪）→ 先懷疑里程計的系統性偏差（輪徑 / 輪距），再懷疑長走廊的掃描匹配退化。

看到斷裂（牆線整齊錯開一截）→ 找那個時間點發生了什麼：打滑、碰撞、或感測器資料掉幀。

這套「看症狀、猜病因」的能力，在第 29 章做實體建圖時會天天用到——而它需要的正是一雙看過正常地圖長什麼樣的眼睛。

## 如何評估地圖品質

評估方法分主觀與客觀兩類：

- 目視檢查（定性）：牆是否筆直？直角是否方正？迴路是否閉合？有無重影與斷裂？這是最快的第一步，有經驗的人一眼就能看出地圖好壞。
- 對照實測（定量）：量測地圖上幾個關鍵距離（走廊長度、房間寬度），與捲尺實測值比較，計算誤差百分比。這是客觀量化地圖精度的方法。
- 閉環一致性：讓機器人繞迴路回到起點，量測地圖上「起點」與「終點」的落差。落差越小，表示漂移控制越好。
- 與真值比對（研究級）：在有動作捕捉系統或已知精確地圖的環境，把估計地圖與真值疊合比對（第 31 章專題會用到）。

養成「建完圖就評估品質」的習慣至關重要。一張看似完整、實則變形的地圖，會讓後續的路徑規劃（第 29 章）全盤出錯——機器人會照著一張錯誤的地圖撞牆。這呼應第 8 章「感測器限制直接變成系統失敗」的觀念：地圖品質，直接決定整個導航系統的成敗。

至此，同學已經理解 2D 地圖「是什麼、怎麼建、怎麼評估」。但本章刻意把「建圖」與「定位」分開，假設位姿已知。真實的 SLAM 必須同時解這兩者——下一章 Gmapping 登場，它用第 11 章的粒子濾波，讓每個粒子各自攜帶一張地圖，一邊定位一邊建圖，漂亮地解開這個雞生蛋問題。理論與演算法，終於要合而為一了。

## 公式或流程

### 一、對數勝算比更新

某格子的佔據機率 p，其對數勝算比定義為：

$$l = \log(p \,/\, (1 - p))$$

每來一筆觀測，新的對數勝算比只需做加法：

$$l_new = l_old + l_measurement$$

其中 l_measurement 是本次觀測帶來的證據（佔據觀測為正、自由觀測為負）。這就是 13.2 節「把乘除變加減」的數學核心。需要顯示或決策時，再轉回機率：

$$p = 1 - 1 \,/\, (1 + \exp(l))$$

l 為正表示傾向佔據、為負表示傾向自由、為零表示五五波。實務上對 l 設上下限，保持地圖對環境變化的可塑性。

### 二、掃描匹配的最佳化目標

掃描匹配求一個相對位姿 T（旋轉 + 平移），使當前掃描 S 對齊參考掃描 R 後的總誤差最小：

$$T^* = \operatorname{argmin} \Sigma_i \,\| T \cdot s_i - r_{j(i)} \|^2$$

其中 $s_i$ 是當前掃描的點、$r_{j(i)}$ 是參考掃描中與之對應的最近點。這是一個最小平方問題（第 12 章），ICP 以「找對應—求解—迭代」反覆逼近其解。

## 三、里程計與光達融合建圖流程

**圖 13-4　2D SLAM 單步建圖流程（本章版本，假設閉環另行處理）**

① 機器人移動一步

↓

② 里程計估出粗略相對移動（初始猜測）

↓

③ 以初始猜測為起點，執行掃描匹配（ICP）精修相對移動

↓

④ 更新機器人位姿（累加精修後的相對移動）

↓

⑤ 光線投射：沿雷射路徑，以對數勝算比更新沿途格子

↓

⑥ 回到 ①，處理下一步（週期性檢查閉環 → 第 14、15 章）

## 四、地圖記憶體估算

一張佔據格地圖的格子總數為：

$$\text{格子數} = (\text{寬} / \text{解析度}) \times (\text{高} / \text{解析度})$$

例如 20 m × 20 m 房間、解析度 0.05 m：格子數 = 400 × 400 = 160,000 格。解析度減半（0.025 m）則格子數變四倍（640,000 格）——這條式子量化了 13.5 節「解析度與記憶體」的平方關係。

# 13.8　2D 建圖的當代實務：從一次性地圖到終身地圖

佔據格地圖的數學在 1980 年代就定型了，本章介紹的對數勝算比更新至今沒有改變。但「地圖怎麼被使用」這件事，近十年有了相當大的轉變，而這個轉變直接影響同學在第 29 章會用到的工具。

### 一、從靜態圖檔到可持續更新的位姿圖

傳統做法是：建圖一次，存成一張 PGM 圖檔加一份 YAML 設定，之後導航時就載入這張靜態地圖來定位。這個流程簡單，但有個明顯的問題——環境會變。桌椅移位、貨架調整、隔間重做，地圖就過時了，只能整個重新建一次。

ROS 2 生態目前的主流工具 slam_toolbox 提出了不同的做法：它不只存地圖，而是把整個位姿圖（第 12 章的節點與邊）序列化存下來。下次開機時載入這個位姿圖，機器人可以選擇繼續建圖、只做定位、或是把新舊地圖合併。這稱為終身建圖（Lifelong Mapping）。

幾個值得知道的規格：其同步模式在約 2,800 平方公尺（三萬平方英尺）的建築中可達五倍以上的即時速度，約 5,600 平方公尺時仍有三倍；已知最大的同步建圖案例約為 18,600 平方公尺。非同步模式則適用於更大的空間，代價是會選擇性丟棄來不及處理的掃描。底層的最佳化求解器預設使用 Ceres，也可切換為 g2o——正是第 12.6 節介紹的那幾套。

### 二、定位也可以不用靜態地圖

slam_toolbox 還提供一種稱為彈性位姿圖定位的模式：載入序列化的地圖後，把進來的掃描與最近一段時間的掃描緩衝區做匹配，而不去永久改動底層地圖。它同樣接受 RViz 的初始位姿設定，使用方式與第 16 章要介紹的 AMCL 相似。

對同學的意義是：第 16 章的 AMCL（粒子濾波定位）不再是唯一選項。不過工具的作者本人建議初學者仍從 AMCL 入手——因為它的行為直觀、資料豐富、出問題時容易診斷。本書的安排也依循這個建議。

### 三、多張地圖的合併

還有一個實務上非常有用的能力：把多次建圖的結果合併成一張全域一致的地圖。大型場域

很難一次掃完——電池會沒電、某些區域暫時進不去——於是分次建圖、事後合併就成了標準做法。合併可以在 RViz 中手動拖曳對齊，也可以用運動學工具自動完成。

這件事背後的數學就是第 12 章的位姿圖最佳化：把兩張圖的位姿圖接在一起，加上幾條人工指定的「這兩個節點是同一個地方」的邊，再重新求解一次。理解了第 12 章，這個功能就不再神祕。

### 四、佔據格之外

最後值得一提的是，佔據格不是唯一的二維地圖表示。近年在導航堆疊中逐漸普及的還有幾種：代價地圖（Costmap）把障礙物向外膨脹一圈，直接存「這裡能不能走」而非「這裡有沒有東西」；距離場（ESDF）每一格存到最近障礙物的距離，讓避障與路徑平滑化的計算更有效率；語意圖層則在幾何之上疊加「這是玻璃」「這是樓梯」「這是禁區」等資訊——第 3.6 節提過的 Nav2 圖層式代價地圖正是這個架構。

但這些表示法幾乎都建立在佔據格之上，或與它並存。所以本章學到的仍然是地基：理解了「每一格存一個機率、以對數勝算比累積證據」，其餘的變體都只是換一種存法。

> **給同學的三個提醒**
>
> 一、地圖是會過時的。任何打算長期運行的機器人都必須處理地圖更新，而不是假設環境永遠不變。
>
> 二、存位姿圖比存圖檔更有價值。圖檔只能看，位姿圖可以繼續最佳化、可以合併、可以回頭修正。
>
> 三、本章的簡化（位姿已知）會在下一章被解除。第 14 章開始，位姿與地圖要一起估計——那才是 SLAM 真正的樣子。

## 案例

## 案例 13-1 掃地機器人 App 上那張地圖：佔據格的日常

打開任何一款支援地圖功能的掃地機器人 App，那張你用來設定禁區、指定房間的房間地圖，底層就是本章的佔據格地圖。黑色的牆線是高佔據機率的格子、白色的可清掃區是低佔據機率的自由空間、而機器人還沒去過的地方則是空白（未知）。

觀察這張地圖的「成長過程」很有教育意義：機器人第一次清掃時，地圖是一點一點「長」出來的——走到哪、掃到哪、畫到哪，未知的區域逐漸被填滿。這正是 13.5 節「動態更新—探索新區域」的視覺化。若機器人在某處打滑或撞到黑色家具（光達看不到，第 8 章），你可能會在地圖上看到牆線歪斜或缺口——這就是 13.7 節的漂移與斷裂，活生生出現在你家客廳。

更進階的觀察：有些掃地機器人第二次清掃同一個家時，地圖會比第一次更精確、更平整。這是因為它累積了更多觀測，對每一格的估計更有把握（13.2 節的貝氏累積），甚至可能執行了閉環修正（第 12 章）。一台掃地機器人，其實是本篇所有理論的隨身教材。

## 案例 13-2 長走廊翻車記：一個真實的教學失敗

大學課堂上，一個幾乎年年上演的場景：同學在教室裡把 2D SLAM 建圖跑得很順，信心滿滿地把機器人開進長廊，結果地圖瞬間「爆掉」——走廊被畫得忽長忽短、牆線扭曲重疊，機器人徹底迷失。

原因就是 13.3 與 13.6 節反覆強調的「環境重複」：長走廊沿縱向缺乏獨特特徵，掃描匹配無法確定「沿走廊走了多遠」。若此時里程計又因地面材質改變（磁磚換成地毯）而不準，兩個資訊都失效，建圖必然失敗。

這個「失敗」其實是最好的一堂課。它讓同學親身體會：SLAM 不是萬能的，環境結構會直接決定成敗。解法呼應全書：融合可靠的里程計（13.4 節）、加大光達量程看到走廊盡頭、或在走廊貼 AprilTag（第 4 章）提供獨特地標。理解為什麼失敗，比在教室裡順利建圖學到的更多——這也是本書為何在每一章都誠實討論「限制與失敗」的原因。

### 案例 13-3 從 2D 到智慧：掃地機器人的演進

回顧掃地機器人的演進，是一部 2D SLAM 的普及史。最早的機型（2002 年前後）根本沒有地圖，採「隨機碰撞」——撞到就轉彎，清掃路線雜亂、重複又遺漏（第3、16章）。2010 年 Neato 首度把 2D 光達裝上掃地機，讓它能建佔據格地圖、規劃系統性路徑。2017 年小米以親民價格普及光達掃地機（第 1 章案例），讓佔據格地圖進入千家萬戶。

近年的機型更進一步：加入 3D ToF 相機辨識並繞開電線、鞋子、寵物排泄物（彌補 2D 光達的高度盲區，第 8 章）；用 AI 辨識房間類型自動分區；甚至記住多層樓的地圖。這條演進線清楚顯示：2D 佔據格地圖是基礎，但真實產品會不斷疊加感測器與 AI 來彌補單一技術的限制——這正是全書貫穿的融合思想，也預告了第九篇 AI 語意導航的方向。同學今天學的 2D 建圖，是理解這一切的起點。

## 實作活動

### 實作 13-1 親手建一張佔據格地圖

目的：透過程式從零實作佔據格地圖的更新，親身理解對數勝算比、光線投射與掃描匹配。本實作以 Python 為主，分模擬與資料兩條路徑，是第 14 章 Gmapping 實作的直接暖身。

#### 第一部分：對數勝算比更新（全體）

84. 用 numpy 建立一個二維陣列當作地圖（例如 200×200 格，每格初始對數勝算比為 0，代表未知）。
85. 模擬單一雷射光束：給定機器人位置、光束方向與量到的距離，實作光線投射——把沿途格子的對數勝算比減少（自由）、終點格子增加（佔據）。
86. 把對數勝算比轉回機率，用 matplotlib 以灰階顯示地圖（黑=佔據、白=自由、灰=未知），確認單束雷射畫出一條「白線到黑點」。
87. 模擬一整圈掃描（360 束）：讓機器人在房間中央，對四周的模擬牆面各發一束，觀察是否畫出房間輪廓。

88. 多次觀測：讓機器人在同一位置重複掃描數次（加入隨機雜訊），觀察牆線如何隨觀測次數增加而變得更「確定」（對數勝算比累積，顏色更黑）。

**第二部分：移動建圖（全體或大學部）**

89. 設計一個簡單的模擬房間（用程式定義幾道牆），讓一台虛擬機器人沿預定路線移動，每步用已知位姿與模擬光達更新地圖，觀察完整房間地圖如何逐步「長」出來。

90. 加入里程計雜訊：故意讓機器人的位姿估計帶有累積誤差（漂移），觀察建出的地圖如何變形、牆線如何重影——親身重現 13.7 節的病徵。

91. 解析度實驗：把同一場景分別用 2 公分、5 公分、20 公分解析度建圖，比較地圖細膩度、記憶體用量（格子數）與運算時間。

**第三部分：ROS 真實/模擬建圖（有 ROS 環境者）**

92. 在 TurtleBot3 模擬（第 28 章）中，啟動內建的 2D SLAM（如 slam_toolbox 或 gmapping），遙控機器人在模擬教室走一圈，在 RViz 中即時觀察佔據格地圖的成長。

93. 把機器人開進模擬長走廊，觀察本章 13.6 節描述的「走廊困境」是否出現（地圖沿走廊方向變形）。

94. 建完圖後，用 13.7 節的方法評估品質：目視檢查牆是否筆直、量測地圖上的房間尺寸與模擬設定值比較。

**討論問題**

63. 第一部分中，同一面牆被觀測 1 次與觀測 10 次，地圖上的顏色（佔據機率）有何差異？這說明對數勝算比累積的什麼作用？

64. 第二部分加入里程計漂移後，地圖出現了哪一種病徵（變形/斷裂/漂移）？若要修正它，你會用第 12 章的什麼技術？

65. 解析度從 5 公分改為 2 公分，格子數增加了幾倍？記憶體與運算時間的變化符合 13.5 節的平方關係嗎？

66. （有 ROS 者）長走廊的地圖出了什麼問題？對照 13.6 節，你認為加入里程計、加大光達量程、貼 AprilTag，哪個對策最有效？為什麼？

**延伸挑戰（大學部）**

實作一個簡化版的 ICP 掃描匹配：給定兩次帶有已知相對位移的模擬掃描，用「找最近對應點—求最佳旋轉平移—迭代」的流程，估計出這個相對位移，並與真值比較。進一步在「長走廊」情境（縱向特徵不足）測試你的 ICP，觀察它為何沿走廊方向估計失準——這將讓你深刻理解為何 Gmapping（第 14 章）要用粒子濾波，而非單純倚賴掃描匹配。

## 延伸閱讀


83. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005.（第 9 章 Occupancy Grid Mapping 為本章的權威依據）

84. Cyrill Stachniss, SLAM 課程之 Occupancy Grid Maps、Scan Matching、ICP 系列影片（YouTube）.（以動畫講解本章核心，自學首選）

85. P. Besl and N. McKay, "A Method for Registration of 3-D Shapes," IEEE TPAMI, 1992.（ICP 掃描匹配的原始論文）

86. A. Elfes, "Using Occupancy Grids for Mobile Robot Perception and Navigation," Computer, 1989.（佔據格地圖的開創性論文）

87. slam_toolbox 開源專案（GitHub）與 ROS 官方文件.（現代 ROS 2 主流的 2D SLAM 工具，取代舊 gmapping，實作首選）

88. ROBOTIS TurtleBot3 e-Manual 之 SLAM 章節（emanual.robotis.com）.（2D 建圖的完整實作步驟）

89. 高翔、張濤，《視覺 SLAM 十四講》之地圖相關章節.（雖以視覺為主，對地圖表示法的討論可與本章互補）


## 從已知位姿到同時估計：本章的位置回顧

本章做完了 SLAM 的後半段：給定位姿，把感測資料變成地圖。這一半其實相當直接——光線投射加上對數勝算比，不到一百行程式就能寫完。

困難的是另一半。本章從頭到尾假設位姿是已知且正確的，而圖 13-3 已經讓同學看到，這個假設一旦破裂，地圖會壞成什麼樣子。真實的機器人沒有人告訴它自己在哪裡——它只有一個會漂移的里程計，以及本章 13.3 節的掃描匹配。

掃描匹配其實已經是 SLAM 的雛形了：它用「把這次掃描對齊到地圖」的方式反過來修正位姿。但它有兩個弱點。其一，它只看局部——在 13.6 節的長走廊中，沿走廊方向怎麼滑都對得上，匹配無從判斷自己走了多遠。其二，它只有一個答案——一旦匹配錯了，沒有任何機制能回頭。

第 14 章的 Gmapping 用第 11 章的粒子濾波解決第一個弱點：同時保留數十個位姿假設，讓觀測逐一淘汰不合理的。第 15 章的 Cartographer 則用第 12 章的位姿圖最佳化解決第二個弱點：保留所有歷史，閉環時一次修正。兩章一起讀，就是第 11 章與第 12 章那兩條路線在同一個問題上的正面對決——而評判的標準，就是本章 13.7 節教的那套地圖品質判準。

## 小結

56. 佔據格地圖把平面切成方格，每格記錄佔據機率（佔據/自由/未知三態）；「未知」不等於「自由」，這是它優於二元地圖之處。建圖的前提是已知位姿，本章暫將建圖與定位分開，專注於「有位姿如何建圖」。

57. 佔據機率隨觀測以貝氏方式累積更新；實務用對數勝算比，把乘除變成加減、數值範圍無界又穩定；一筆雷射量測透過光線投射，更新終點格（佔據）與沿途格（自由）一整條線。

58. 掃描匹配比對前後兩次雷射掃描估計相對移動，經典演算法為 ICP（找對應—求解—迭代）；它精確但怕環境重複（走廊）且需要好的初始猜測。

59. 里程計與光達恰好互補：里程計連續、免初始猜測但漂移，光達精確、不漂移但需初始猜測且怕重複環境；融合邏輯是「里程計預測、光達校正」，正是貝氏濾波預測—更新的化身，並能以環境觀測否決打滑造成的錯誤運動。
60. 地圖解析度是品質、記憶體、運算的三方權衡，格子數與解析度成平方反比；地圖需動態更新以探索新區並修正舊估計，並對對數勝算比設限以保持對環境變化的可塑性。
61. 不同場景挑戰各異：教室理想、長走廊因縱向無約束而最難（需里程計補足）、辦公室有玻璃與動態、空曠大廳超出光達量程；預判挑戰是實作成功的關鍵。
62. 地圖三大病徵為漂移（繞圈回不到原點）、變形（幾何扭曲）、斷裂重影（同牆兩線）；評估方法有目視檢查、對照實測、閉環一致性與真值比對。地圖品質直接決定導航成敗。

本章刻意把建圖與定位分開，假設位姿已知。但真正的 SLAM 必須同時解開這兩者——這正是第 1 章的雞生蛋問題。下一章 Gmapping 登場：它運用第 11 章的粒子濾波，讓每一個粒子各自攜帶一條軌跡與一張地圖，在眾多假設中「適者生存」，漂亮地同時完成定位與建圖。第三篇的粒子濾波理論，即將在第一個完整的 SLAM 系統中大顯身手。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

## 第 14 章

## Gmapping：粒子濾波 SLAM

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

## 本章在全書的位置

第 13 章建了一張地圖，但它靠的是一個不成立的假設——位姿已知。本章要把這個假設拿掉，正面迎戰第 1 章提出的雞生蛋問題。

Gmapping 給的答案是第 11 章的粒子濾波，但做了一個關鍵的轉折：不要讓粒子去表示「位姿與地圖」的聯合分布，而是讓每一顆粒子只負責一條位姿軌跡，地圖則由該粒子自己依第 13 章的方法建出來。這個拆解稱為 Rao-Blackwell 化，是本章的核心。

本章與第 15 章是一組對照實驗：同一個問題（2D 光達 SLAM），Gmapping 走第 11 章的濾波路線，Cartographer 走第 12 章的最佳化路線。讀完這兩章再回頭看第 12.7 節的「濾波與圖優化總比較」，那張表就不再是抽象的條列。

### 圖 14-0　第 14 章與全書的連結地圖

【承接前章】

第 11 章　粒子濾波　→　本章的骨架（預測、加權、重採樣）

第 13 章　佔據格建圖　→　每顆粒子各自建一張

第 13.3 節　掃描匹配　→　本章改良提議分布的關鍵零件

【本章負責】

Rao-Blackwell 化：把聯合分布拆成「位姿用粒子、地圖用解析」

粒子數、權重、重採樣與粒子退化

Gmapping 兩項關鍵改良：改良提議分布、選擇性重採樣

【往下延伸】

→ 第 15 章　Cartographer：同一問題的最佳化解法（對照組）

→ 第 16 章　AMCL：粒子濾波用於「已知地圖中定位」

→ 第 6.7 節　磁力計自動校正：同一套 RBPF 換個應用對象

→ 第 29 章　在實體小車上調 Gmapping 參數

## 學習目標

上一章我們假設「位姿已知」來建圖，但真正的 SLAM 必須同時解開定位與建圖。本章介紹第一個完整的 SLAM 系統——Gmapping。它運用第 11 章的粒子濾波，讓每個粒子各自攜帶一條軌跡與一張地圖，在眾多假設中「適者生存」，漂亮地解開第 1 章的雞生蛋問題。學完本章之後，同學應能夠達成下列目標：

82. 說明 Gmapping 的基本概念與它在 SLAM 發展史上的地位。
83. 解釋 Rao-Blackwellized 粒子濾波的核心思想：為何「每個粒子帶一張地圖」能解開雞生蛋問題。
84. 說明粒子數、權重與重採樣的作用，以及粒子退化與有效粒子數的概念。
85. 說明 Gmapping 兩項關鍵改良：改良提議分布與選擇性重採樣。
86. 說明在 ROS 中執行 Gmapping 建圖的完整流程與必要前置條件。
87. 辨識常見的建圖失敗案例——輪子打滑、場景重複、光達遮蔽——並提出對策。
88. 完成以教學小車或模擬器建立教室地圖的實作。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
| --- | --- | --- |
| 粒子濾波 SLAM | RBPF-SLAM | 以粒子濾波同時估計軌跡與地圖的 SLAM 方法。 |
| Rao-Blackwell 化 | Rao-Blackwellization | 把問題拆解，一部分用粒子取樣、另一部分用解析法求解。 |
| 粒子 | Particle | 一個完整的假設：包含一條軌跡與其對應的一張地圖。 |
| 權重 | Weight | 該粒子的假設與感測器觀測吻合的程度。 |
| 重採樣 | Resampling | 依權重複製高分粒子、淘汰低分粒子的過程。 |

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 粒子退化 | Particle Depletion | 重採樣過度導致粒子多樣性喪失的現象。 |
| 有效粒子數 | Effective Sample Size | 衡量粒子多樣性的指標，用於決定是否重採樣。 |
| 提議分布 | Proposal Distribution | 產生下一步粒子候選位置的機率分布。 |
| 掃描匹配 | Scan Matching | 比對雷射掃描以精修位姿，Gmapping 用它改良提議分布。 |
| 選擇性重採樣 | Selective Resampling | 只在有效粒子數過低時才重採樣，避免退化。 |
| 地圖伺服器 | map_server | ROS 中儲存與載入地圖檔的工具。 |
| 地圖檔 | pgm / yaml | ROS 地圖的影像檔與描述檔格式。 |

## 14.1 Gmapping 的基本概念

Gmapping 是 2D 雷射 SLAM 領域最著名、也最廣泛使用的開源系統之一。它由 Grisetti、Stachniss 與 Burgard 於 2007 年提出——正是第 2 章介紹的德國 Freiburg / Bonn 機率機器人學研究群。同學讀完本章後，若去看 Stachniss 教授在 YouTube 上親自講解粒子濾波 SLAM 的影片，會有「原作者現身說法」的難得體驗。

### Gmapping 解決什麼問題

第 13 章我們刻意假設「位姿已知」，專注於建圖。但真實機器人的位姿正是未知的——而且位姿錯了，地圖就跟著錯。Gmapping 要同時回答兩個互相依賴的問題：機器人走過的軌跡是什麼？環境地圖長什麼樣？

它的答案極具巧思：不要試圖找出「唯一正確」的軌跡，而是同時維持數十到數百個「可能的軌跡假設」，每一個假設都各自建一張地圖；然後看哪些假設與感測器觀測最吻合，讓它們存活繁衍，不吻合的則被淘汰。這正是第 11 章粒子濾波「適者生存」思想的直接應用。

### 輸入與輸出

- 輸入：2D 光達掃描（LaserScan，第 8 章）、里程計（第 3、13 章）、以及機器人的座標框架設定（TF，第 10 章）。
- 輸出：一張佔據格地圖（第 13 章）、以及機器人在地圖中的位姿（即 map→odom 的座標變換，第 10.7 節）。

注意這個輸入輸出清單：它幾乎把前面所有章節都用上了。這正是本篇的意義——理論在此匯流成一個可以真正跑起來的系統。

### 歷史地位

Gmapping 長年是 ROS 生態中 2D SLAM 的預設選擇，無數大學實驗室與教學平台以它入門。近年在 ROS 2 環境中，slam_toolbox 等基於圖優化（第 12 章）的方案逐漸成為

主流，但 Gmapping 的教學價值絲毫未減：它是理解「粒子濾波如何做 SLAM」最清晰的範例，也是連接第 11 章理論與真實系統的最佳橋樑。

## 14.2 Rao-Blackwellized Particle Filter

這是本章最核心的觀念。Gmapping 的全名是「Rao-Blackwellized 粒子濾波 SLAM」，這個拗口的名詞背後，是一個非常聰明的拆解。

### 直接用粒子濾波做 SLAM 的困境

回想第 11 章：粒子濾波用一群樣本近似機率分布。若直接套用到 SLAM，每個粒子必須同時代表「機器人位姿」與「整張地圖」的一個假設。問題是——地圖有數萬個格子，等於數萬個未知數。要用粒子在如此高維的空間中取樣，所需粒子數會是天文數字，完全不可行。這稱為維度詛咒（Curse of Dimensionality）。

### Rao-Blackwell 化：聰明的拆解

Rao-Blackwell 化的核心洞見是：把問題拆成兩部分，一部分用粒子取樣，另一部分用解析方法直接算。

關鍵觀察在於第 13 章的結論：只要知道機器人的完整軌跡，建地圖就是一件確定性的簡單事（用對數勝算比更新即可，不需取樣）。換句話說，地圖並不需要「猜」，它可以由軌跡直接算出來。

於是拆解就很自然了：

- 用粒子取樣的部分：機器人的軌跡（只有每步三個數：x、y、θ）。這是低維的，粒子濾波應付得來。
- 用解析方法算的部分：地圖。每個粒子拿著自己那條軌跡，用第 13 章的方法直接算出對應的地圖。

**觀念核心：每個粒子是一個「平行世界」**

想像 100 個平行世界，每個世界裡的機器人走了略微不同的路線（軌跡假設），

因此各自畫出略微不同的地圖。

新的雷射掃描到來時，每個世界都問自己：「以我的地圖與位置，這筆掃描合理嗎？」

合理的世界得高分（權重高）、不合理的得低分。

重採樣時，高分世界被複製、低分世界消失。存活下來的，就是與觀測最一致的假設。

這就是 Gmapping：用平行世界的競爭，同時找出正確的軌跡與地圖。

### 圖 14-1　Rao-Blackwell 化：每顆粒子都帶著自己的地圖




這四張圖是用同一份光達資料跑出來的，差別只在四顆粒子各自假設的軌跡不同——每顆粒子的里程計誤差模型不一樣，於是走出四條不同的路徑，也就建出四張不同的地圖。

粒子 1 的假設最接近真實，建出的地圖牆面筆直、隔間方正；粒子 4 的假設偏差最大，地圖整個被扭轉、牆面模糊。而權重正是由「這次的雷射掃描與我這張地圖有多吻合」算出來的——權重 0.46、0.29、0.16、0.10，與地圖品質的順序完全一致。

這就是 Rao-Blackwell 化的全部精神。粒子不需要同時猜位姿和地圖（那是一個高到無法處理的維度），它只要猜一條軌跡；地圖是軌跡的必然結果，用第 13 章的方法算出來就好。於是粒子濾波從「不可能」變成「可行」。

請特別注意一件事：被淘汰的不只是錯誤的位姿，還有那張錯誤的地圖。重採樣時粒子 1 被複製多份、粒子 4 被刪除，連帶它那張歪掉的地圖也一起消失。這是 SLAM 與單純定位最大的不同——錯誤的假設會同時污染位姿與地圖，所以必須整組一起淘汰。

> **一個常被問的問題：記憶體夠嗎**
>
> 每顆粒子都帶一張地圖，聽起來很耗記憶體——確實如此。以 30 顆粒子、20×17 公尺場域、5 公分解析度計算，總格數是 30 × 136,000 = 408 萬格，約 4 MB。
>
> 這在筆電上毫無問題，但在只有數十 MB 記憶體的嵌入式平台上就是真實的限制。這也是為什麼 Gmapping 的粒子數通常只設數十而非數千。
>
> 實作上還有一個技巧：粒子重採樣時，被複製的粒子可以共用同一張地圖（寫入時才複製，Copy-on-Write），能大幅節省記憶體。

### 為什麼這樣就解開了雞生蛋問題

第 1 章的雞生蛋問題是：建圖需要位姿、定位需要地圖。RBPF 的解法是——不必先有誰。每個粒子先「假設」一條軌跡（不需要地圖就能假設），依此軌跡建圖（建圖只需軌跡），再用觀測評估這個假設好不好（評估需要地圖，而地圖剛剛建好了）。整個循環自洽，不需要任何一方先存在。這個優雅的拆解，正是 Gmapping 在 SLAM 史上的重要貢獻。

## 14.3 粒子數、權重與重採樣

本節深入 RBPF 的三個實務關鍵：要用幾個粒子？怎麼給分？什麼時候淘汰？

### 一、粒子數：精度與運算的權衡

粒子數是 Gmapping 最重要的參數。它的取捨很直觀：

- 粒子多（如 100 個）：能涵蓋更多假設，對複雜環境與較大不確定性更穩健，但每個粒子都要維護一張地圖，記憶體與運算成本高。
- 粒子少（如 5 至 30 個）：快速輕量，但假設涵蓋不足，遇到大幅漂移或環境重複時，可能所有粒子都「猜錯」而無法挽回。

教學實務上，一般教室與走廊環境用 30 個粒子多半足夠；若環境大或里程計品質差，可增加到 50 至 100 個。這裡有一個重要提醒：粒子數與地圖大小相乘決定記憶體用量——100 個粒子各帶一張 400×400 的地圖，就是 1600 萬個格子。這是 Gmapping 難以用於超大場景的根本原因，也是圖優化方法（第 15 章 Cartographer）後來居上的動機之一。

## 二、權重：用觀測給假設打分數

每個粒子的權重，代表「這個假設與最新觀測的吻合程度」。計算方式是：用該粒子的位姿與地圖，預測「應該會量到什麼」，再與實際雷射掃描比較。

若某粒子的地圖顯示「前方 3 公尺有牆」，而實際掃描也量到 3 公尺，兩者吻合，權重高；若某粒子的地圖說「前方是空的」，實際卻量到牆，顯然這個粒子的位姿假設偏了，權重低。這正是第 11 章觀測模型的具體實現。

## 三、重採樣：適者生存，以及它的陷阱

重採樣依權重複製高分粒子、淘汰低分粒子，讓粒子群逐漸聚集到正確的假設。但這裡藏著一個危險的陷阱：粒子退化（Particle Depletion）。

若每一步都重採樣，粒子的多樣性會迅速流失——最後所有粒子可能都是同一個祖先的複製品，彼此幾乎一樣。這時粒子濾波就失去了「維持多個假設」的意義。更糟的是：如果那個被大量複製的祖先其實猜錯了，系統就再也沒有其他假設可以挽救，SLAM 徹底失敗。

這個問題在 SLAM 中特別嚴重，因為錯誤的軌跡假設一旦獨佔，對應的錯誤地圖也就定案，無法回頭修正（這也是第 12 章指出濾波方法相對圖優化的弱點）。Gmapping 的第二項關鍵改良，正是為了對付它——見下一節。

## 14.4 Gmapping 的兩項關鍵改良

如果只是「粒子濾波加上每粒子一張地圖」，系統會因為需要大量粒子而不實用。Gmapping 之所以成功，關鍵在於兩項改良，它們讓所需粒子數大幅降低。理解這兩項改良，就理解了 Gmapping 的精髓。

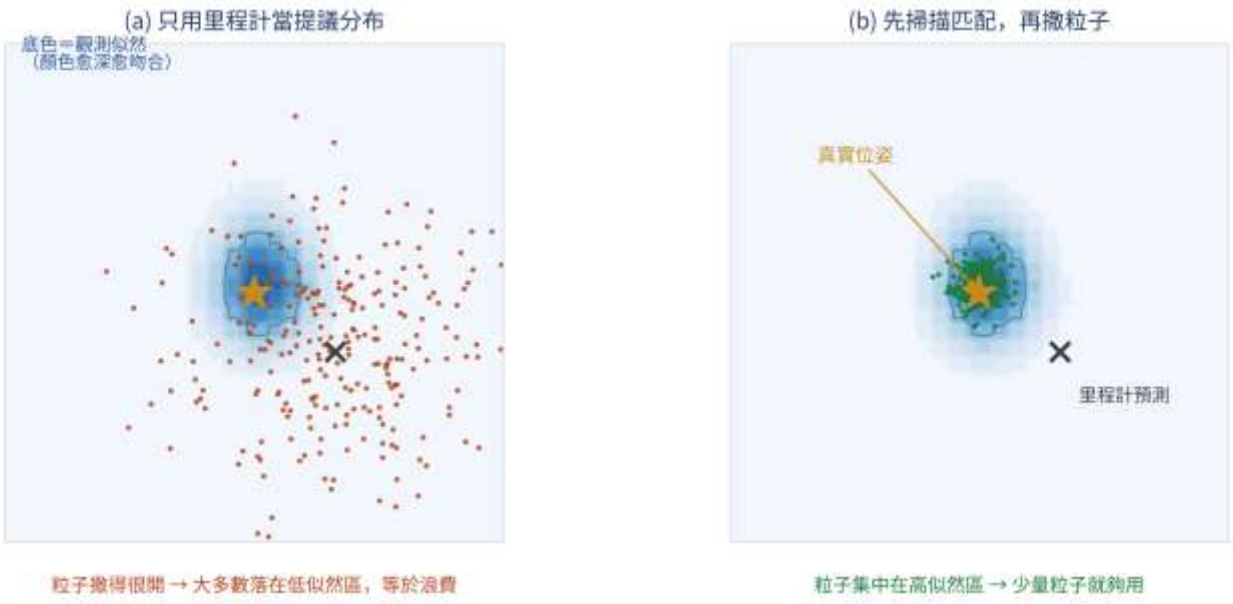
### 改良一：用掃描匹配改良提議分布

提議分布（Proposal Distribution）是指「下一步要把粒子撒在哪裡」的機率分布。最樸素的做法是只依里程計：粒子依里程計移動，再加上隨機雜訊散開。

問題是里程計雜訊往往很大，粒子會撒得很開，而真正正確的位姿只落在其中一小塊區域。大部分粒子被浪費在明顯錯誤的位置上，因此需要大量粒子才能「碰巧」有幾個落在正確處。

Gmapping 的改良是：先用第 13 章的掃描匹配精修位姿，再以精修後的結果為中心，在一個小得多的範圍內撒粒子。

**圖 14-2 提議分布的改良**




兩張圖的底色都是同一份觀測似然——顏色愈深，代表「機器人若在這裡，雷射掃描會與

地圖愈吻合」。金色星號是真實位姿，灰色叉號是里程計的預測位置，兩者之間有明顯的偏差。

左圖是樸素做法：以里程計預測為中心撒粒子。由於里程計雜訊大，粒子散布的範圍遠大於高似然區，絕大多數粒子落在幾乎不可能的位置上——它們會在加權時拿到接近零的權重，等於白算一輪。

右圖是 Gmapping 的做法：先用第 13.3 節的掃描匹配找出似然最高的位置，再以那裡為中心撒一小撮粒子。可以看到幾乎每一顆粒子都落在深色區域內，每一顆都是「有意義的假設」。

效果在數字上相當驚人：所需粒子數從數千降到數十。這不只是省運算，更是讓 Gmapping 能在當年的嵌入式電腦上即時運行的關鍵。同學若把這張圖與第 11 章圖 11-4 的粒子濾波四步驟對照，會發現改良的其實是第②步「預測」——把盲目的撒點換成有依據的撒點。

這個改良的效果驚人：它讓 Gmapping 用數十個粒子就能達到樸素方法需要數千個粒子的效果。同學應該能看出，這正是第 13.4 節「里程計預測、光達校正」思想的延伸應用。

### 改良二：選擇性重採樣

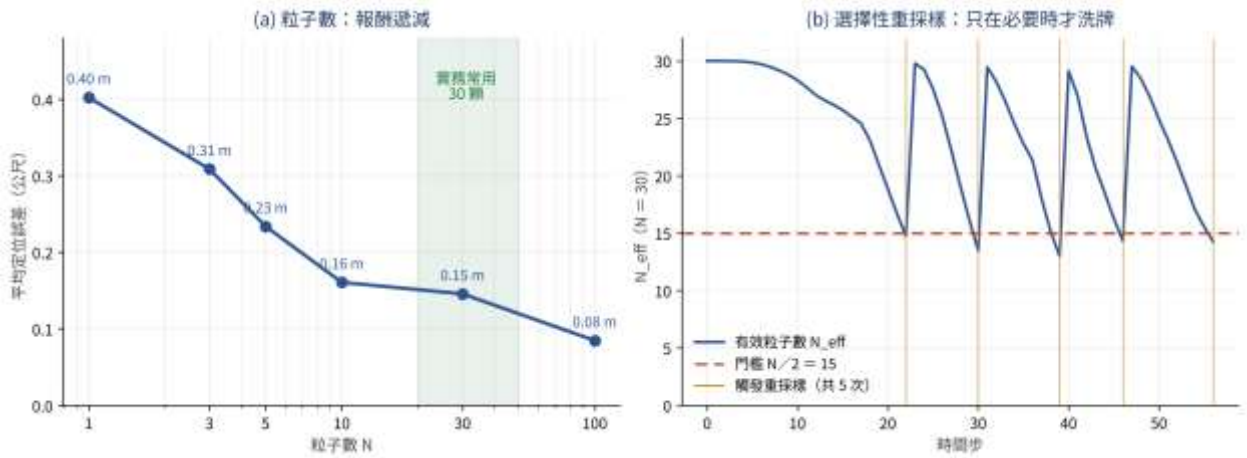
針對 14.3 節的粒子退化問題，Gmapping 採用選擇性重採樣（Selective Resampling）：不是每步都重採樣，而是先評估粒子的多樣性，只在多樣性太低時才重採樣。

衡量多樣性的指標稱為有效粒子數（Effective Sample Size,ESS）。若所有粒子權重相近（表示大家都差不多合理），ESS 高、多樣性好，不需重採樣；若少數粒子權重極高而其餘極低（表示已經分出勝負），ESS 低，此時才重採樣以集中資源。

這個策略在「保持多樣性」與「集中到正確假設」之間取得平衡，大幅減緩了粒子退化。這兩項改良——改良提議分布降低所需粒子數、選擇性重採樣避免退化——正是

Gmapping 論文的核心貢獻，也是它能成為實用系統的原因。

**圖 14-3　粒子數的取捨與選擇性重採樣**




左圖是實際跑出來的結果：同一條軌跡、同一組雜訊設定，只改變粒子數。1 顆粒子時平均定位誤差約 0.40 公尺——這其實就是沒有濾波、只靠單一假設推算；加到 5 顆降為 0.23 公尺；10 顆是 0.16 公尺；30 顆是 0.15 公尺；而 100 顆也只降到 0.08 公尺。

請注意 10 顆到 30 顆之間幾乎沒有改善，而 30 顆到 100 顆（運算量增加三倍以上）只換來 0.07 公尺。這就是報酬遞減——也正是案例 14-2 所說「三十個粒子夠用又不夠用」的量化版本。實務上 Gmapping 的預設粒子數就落在這個甜蜜點附近。

右圖展示選擇性重採樣的運作。藍線是有效粒子數 N_eff 隨時間的變化：剛重採樣完所有權重相等，N_eff 等於粒子總數 30；隨著觀測累積，權重逐漸集中到少數粒子身上，N_eff 一路下滑。當它跌破門檻 N / 2 = 15（紅色虛線）時才觸發重採樣（黃色豎線），整段軌跡共觸發 5 次。

這條鋸齒曲線把 14.3 節的粒子退化講得很清楚：若每一步都重採樣，多樣性會被反覆摧毀，正確的假設可能在還來不及證明自己之前就被刪掉；若從不重採樣，則所有權重都集中到一顆粒子，其餘全成了擺設。選擇性重採樣就是在這兩個極端之間取平衡。

## 14.5　ROS Gmapping 實作流程

理論講完，本節說明在 ROS 中實際執行 Gmapping 建圖的完整流程。這也是第 29 章實作章節的預習。

### 前置條件：三樣東西缺一不可

許多同學第一次跑 Gmapping 失敗，原因幾乎都出在前置條件沒備齊：

96. 雷射掃描資料：光達驅動正常發布 /scan 話題（第 8 章的 LaserScan 格式）。
97. 里程計資料：機器人發布 odom 話題與 odom→base_link 的座標變換（第 10.7 節）。這需要輪速編碼器與正確的輪徑、輪距參數。
98. TF 座標樹完整：特別是 base_link→laser 的固定變換（光達安裝位置的外參，第 10 章）。設錯的話，掃描點會與機器人錯位，地圖必定歪掉——這正是第 10 章「地圖歪了先看 TF」口訣的由來。

### 建圖流程

99. 啟動機器人底層驅動（馬達控制、編碼器里程計、光達）。
100. 確認前置條件：用工具檢查 /scan 有資料、TF 樹完整無斷裂、里程計會隨移動而變化。
101. 啟動 Gmapping 節點，設定關鍵參數（粒子數、地圖解析度、更新距離門檻等）。
102. 開啟 RViz，顯示地圖、雷射掃描、TF 與粒子雲，以便即時觀察建圖狀況。
103. 遙控機器人在環境中緩慢移動，覆蓋所有想建圖的區域。速度要慢——這是成功的關鍵之一。
104. 觀察地圖成長：牆線是否清晰？有無重影？繞迴路後是否閉合？
105. 建圖完成後，用 map_saver 儲存地圖，產生 .pgm 影像檔與 .yaml 描述檔。
106. 日後導航時，以 map_server 載入這張地圖，配合 AMCL（第 3、11 章）進行定位。

**關鍵參數**

| 參數 | 意義 | 調校建議 |
|---|---|---|
| 粒子數 | 維持幾個軌跡假設 | 教室 30 個；環境大或里程計差可增至 50–100 |
| 地圖解析度 | 每格代表的實際邊長 | 室內常用 0.05 m（第 13.5 節） |
| 更新距離 / 角度門檻 | 移動多少才更新一次 | 設太大會漏掉細節，太小則運算浪費 |
| 雷射最大可用距離 | 超過此距離的量測捨棄 | 略小於光達標稱量程較穩健 |
| 里程計雜訊參數 | 描述里程計有多不準 | 打滑嚴重的平台應調大 |

**表 14-1　Gmapping 主要參數與調校建議**

## 14.6　常見失敗案例：輪子打滑、場景重複、光達遮蔽

這一節可能是全章對實作最有用的部分。SLAM 失敗是常態而非例外，能辨識失敗模式並對症下藥，是工程能力的核心。

### 失敗一：輪子打滑與里程計失準

症狀：地圖出現明顯錯位或重影，尤其在轉彎處；機器人「以為」自己走了很遠，地圖卻被拉長。

成因：輪子在光滑地面打滑、地毯與磁磚交界處摩擦力突變、輪徑或輪距參數設定錯誤（這一項最常被忽略，而且會造成系統性的尺度誤差，使整張地圖等比例放大或縮小）。

- 對策：確實校正輪徑與輪距（讓機器人直走 5 公尺、原地轉 10 圈，比對里程計讀值與實測）；放慢速度減少打滑；調大 Gmapping 的里程計雜訊參數，讓系統少信任里程計、多倚賴掃描匹配。

### 失敗二：場景重複與特徵不足

症狀：長走廊被畫得忽長忽短、牆線重疊；空曠大廳中機器人徹底迷失。這正是第 13.6

節的走廊困境。

成因：沿走廊方向缺乏獨特幾何特徵，掃描匹配無法確定縱向移動量；空曠處則四周都超出光達量程，無從匹配。

- 對策：確保里程計品質良好（此時它是縱向資訊的唯一來源）；增加粒子數以維持更多假設；在走廊沿途放置紙箱等臨時障礙物提供特徵（教學情境常用的技巧）；或改用量程更大的光達。

### 失敗三：光達遮蔽與看不見的障礙

症狀：地圖上出現不該有的缺口；機器人撞到地圖上沒畫出來的東西。

成因：玻璃與鏡面讓雷射穿透或反射（第 8 章）；黑色吸光物體反射微弱；障礙物不在光達的掃描平面高度上（第 8 章的高度盲區，案例 8-1 的椅腳問題）；或有人站在光達前方形成移動遮蔽。

- 對策：建圖時清空場地、避免人員在機器人周圍走動；玻璃隔間可在建圖期間暫時貼上紙張或放置紙箱；認知到 2D 地圖只反映單一高度切面，導航時需搭配其他感測器補足。

### 失敗四：移動太快

症狀：掃描匹配跟不上，地圖突然錯位甚至「爆掉」。

成因：機器人移動或轉彎過快，前後兩次掃描差異太大，超出掃描匹配的收斂範圍（第 13.3 節 ICP 需要好的初始猜測）。

- 對策：放慢速度，尤其轉彎時。這是最簡單也最有效的一招——教學經驗中，多數建圖失敗只要「開慢一點」就解決了。

| 失敗模式 | 典型症狀 | 主要對策 |
|---|---|---|
| 輪子打滑 | 地圖錯位、重影、比例失真 | 校正輪徑輪距、放慢、調大里程計雜訊 |
| 場景重複 | 走廊變形、機器人迷失 | 確保里程計品質、增加粒子、加臨時特徵 |
| 光達遮蔽 | 地圖有缺口、撞到未畫出的障礙 | 清場、處理玻璃、認知高度盲區 |
| 移動太快 | 地圖突然錯位或爆掉 | 放慢速度，尤其轉彎時 |

**表 14-2　Gmapping 常見失敗模式與對策**

## 14.7　教學實作：用小車建立教室地圖

本節說明如何把 Gmapping 落實為一堂可操作的課，提供高中與大學部兩種深度的建議。詳細步驟見本章實作活動。

### 高中版：先看懂，再動手

高中課程建議以「觀察與體驗」為主，不必深究數學：

- 先用模擬器（TurtleBot3 Gazebo，第 28 章）讓每位同學都能操作，不受硬體數量限制。
- 重點觀察三件事：地圖如何隨機器人移動而「長」出來、粒子雲如何隨移動散開又收斂、以及開太快時地圖如何出錯。
- 刻意示範失敗：讓同學故意把機器人開很快，親眼看到地圖爆掉——失敗的震撼比成功的順利更能建立直覺。
- 連結生活：對照家中掃地機器人 App 的地圖（案例 13-1），理解兩者是同一回事。

### 大學部版：動手調參，理解機制

大學部應進一步操作實體平台並理解參數影響：

- 親手完成 TF 設定與里程計校正，體會前置條件的重要性（這往往佔掉一半的除錯時間）。

- 系統性地實驗參數：改變粒子數、地圖解析度、里程計雜訊，量化比較地圖品質（用第 13.7 節的評估方法）。
- 在教室、走廊兩種場域各建一次圖，對照第 13.6 節的預測，驗證走廊確實較難。
- 建完地圖後，接續執行 AMCL 定位與導航（第 29 章），完成「建圖→定位→導航」的完整鏈路。

### 教學心法

依多年教學經驗，建議教師掌握三個要點。其一，先講清楚前置條件與失敗模式，再讓同學動手——否則課堂會淹沒在除錯中。其二，強調「慢」：速度是新手成功率最大的變數。其三，務必安排「評估地圖品質」的環節（第 13.7 節），讓同學養成不只建圖、還要判斷好壞的習慣。這三點能讓一堂 SLAM 實作課的成功率大幅提升。

至此，同學已經完整走過第一個 SLAM 系統。Gmapping 用粒子濾波優雅地解開了雞生蛋問題，但它也有明確的限制：粒子數與地圖相乘的記憶體壓力使它難以應付大場景，且濾波方法無法回頭修正歷史（第 12 章）。下一章的 Cartographer 走另一條路——用第 12 章的圖優化與閉環偵測，讓大範圍建圖與「一鍵拉正」成為可能。兩相對照，同學將更深刻理解濾波與最佳化兩條路線的取捨。

## 公式或流程

### 一、Rao-Blackwell 化的機率分解

SLAM 要估計「軌跡與地圖的聯合分布」（第 1 章）。RBPF 的關鍵是把它拆成兩項：

$$p(x(1{:}t), m \mid z(1{:}t), u(1{:}t)) = p(m \mid x(1{:}t), z(1{:}t)) \cdot p(x(1{:}t) \mid z(1{:}t), u(1{:}t))$$

右側第二項是「軌跡的分布」——低維，用粒子取樣；右側第一項是「已知軌跡下的地圖分布」——在第 13 章我們已知它可以用對數勝算比直接算出，不需取樣。這條式子就是 14.2 節「拆解」的數學表述，也是 Gmapping 全部巧思的濃縮。

## 二、粒子權重

第 i 個粒子的權重更新為：

$$w^{(i)} \propto p(z_t \mid m^{(i)}, x_t^{(i)})$$

讀作：權重正比於「以該粒子的地圖 $m^{(i)}$ 與位姿 $x_t^{(i)}$，觀測到 $z_t$ 的可能性」。掃描與地圖越吻合，權重越高。這正是第 11 章觀測模型在 SLAM 中的具體形式。

## 三、有效粒子數與選擇性重採樣

衡量粒子多樣性的指標：

$$N_{eff} = 1 \,/\, \Sigma_i \, (w^{(i)})^2 \quad （權重已正規化）$$

若所有粒子權重相等（多樣性最佳），$N_{eff}$ 等於粒子總數 N;若權重集中在單一粒子，$N_{eff}$ 趨近 1。Gmapping 的選擇性重採樣規則為：

$$若\ N_{eff} < N \,/\, 2，才執行重採樣$$

這條簡單的判斷式，正是 14.4 節避免粒子退化的關鍵機制。

## 四、Gmapping 單步完整流程

**圖 14-4　Gmapping 每一步的處理流程**

① 取得新的里程計讀值與雷射掃描

↓

② 對每個粒子：以里程計為初始猜測，執行掃描匹配精修位姿

↓

③ 在精修結果附近的小範圍取樣，得到該粒子的新位姿（改良提議分布）

↓

④ 計算權重：比較「該粒子地圖預期的掃描」與「實際掃描」的吻合度

↓

⑤ 每個粒子用自己的新位姿，更新自己的地圖（第 13 章對數勝算比）

↓

⑥ 計算 N_eff;若低於門檻則重採樣（選擇性重採樣）

↓

⑦ 輸出權重最高粒子的地圖與位姿 → 回到 ①

### 五、記憶體估算

Gmapping 的記憶體需求為粒子數乘上單張地圖大小：

$$\text{總格子數} = \text{粒子數} \times (\text{寬} / \text{解析度}) \times (\text{高} / \text{解析度})$$

例如 50 個粒子、20 m × 20 m 房間、0.05 m 解析度：50 × 400 × 400 = 800 萬格。這條式子量化了 14.3 節指出的限制——這正是 Gmapping 難以擴展到大場景的根本原因。

## 14.8　Gmapping 的今天：工具會過時，思想不會

對同學要誠實：如果今天要在 ROS 2 上建一張 2D 地圖，大多數人不會選 Gmapping。它

是 ROS 1 時代的套件，官方生態的主流已經轉向第 13.8 節介紹的 slam_toolbox。那為什麼本書還要花一整章講它？

### 一、因為 RBPF 並沒有消失

Gmapping 這個軟體套件會退場，但它背後的 Rao-Blackwell 化粒子濾波仍然活躍在三個地方。

第一，第 16 章的 AMCL——掃地機器人與服務機器人在已知地圖中定位，用的就是粒子濾波，只是不必再建地圖。第二，消費級掃地機器人的定位模組大量採用這條路線，因為它在低運算資源下表現穩定、實作簡單。第三，它持續被移植到新的問題上：第 6.7 節介紹的磁力計自動校正就是一個例子——以因式分解的粒子濾波同時估位姿、磁場地圖與感測器零偏，骨架與本章完全相同。

換句話說，本章學的不是一個軟體的使用方法，而是一種解題的結構：當狀態中有一部分「給定其餘部分後可以解析求解」時，就把它因式分解出去，只用粒子處理真正困難的那一部分。這個結構在往後的研究中會反覆派上用場。

### 二、它的弱點恰好是下一章的主題

多份 2D SLAM 的比較研究得到一致的結論：在走廊、大場域與需要閉環的場景中，以最佳化為後端的方法（Cartographer 一類）在地圖精度上明顯優於 Gmapping。原因在本章已經埋下伏筆——粒子濾波是遞迴的，一旦某一步的所有粒子都偏了，就再也回不來；而重採樣本身又會持續消耗多樣性。

這不是 Gmapping 寫得不好，而是濾波路線的結構性限制，第 11 章的小結已經說過。第 15 章的 Cartographer 正是用第 12 章的位姿圖最佳化補上這一塊：保留全部歷史，閉環時一次修正。兩章連著讀，同學會對「為什麼現代 SLAM 後端幾乎都用最佳化」有切身的理解，而不只是記住結論。

### 三、粒子濾波自己也在進步

值得一提的是，粒子濾波的研究並未停滯，近年的改良大致有三個方向。其一是提議分布的學習化：用神經網路從資料中學出「該往哪裡撒粒子」，取代本章的掃描匹配。其二是平行化：粒子彼此獨立，天生適合 GPU，這使得「幾千顆粒子」在今天不再是負擔。其三是與最佳化的混合：用粒子處理多峰與不確定性，用最佳化處理局部精修——第 18 章的 LIO-SAM 與第 23 章的 ORB-SLAM3 都可以看到這種分層思路的影子。

**給同學的三個判斷**

一、學演算法不要只學工具。slam_toolbox 的使用方式幾年就會變，但「因式分解、用粒子表示多峰」這個結構不會。

二、濾波與最佳化不是誰取代誰。第 11.8 節說過：高頻即時用濾波、全域一致用最佳化，一個完整系統兩者並存。

三、本章的實作仍然值得做。即使最後產品不用 Gmapping，親手看著三十顆粒子逐步收斂、看著 N_eff 跌破門檻觸發重採樣，是理解整個機率 SLAM 家族最快的途徑。

## 案例

### 案例 14-1　原作者的課堂：從論文到 YouTube

Gmapping 的三位作者 Grisetti、Stachniss 與 Burgard，正是第 2 章介紹的 Freiburg / Bonn 機率機器人學研究群核心成員。有趣的是，Stachniss 教授後來把粒子濾波 SLAM 的完整推導錄成影片免費公開在 YouTube 上，成為全球 SLAM 學習者的共同教材。

這意味著同學讀完本章後，可以直接去看演算法的原創者親自講解——他會用動畫展示粒子如何散開、如何被觀測秤重、如何重採樣收斂。這種「讀教科書理解架構、看原作者影片補足直覺」的組合，是本書第 2 章推薦學習路線的最佳實踐。

這個案例也呼應第 2 章「知識公開的力量」：一個 2007 年發表在頂級期刊的演算法，

今天任何一位台灣的高中生或大學生，都能免費取得原始論文、開源程式碼與作者親授影片。學習 SLAM 的門檻，已從「能否進入名校」變成「有沒有一份好的學習地圖與持續的毅力」——這正是本書想扮演的角色。

## 案例 14-2　三十個粒子的教室：為何 Gmapping 夠用又不夠用

在大學的教學實驗中，一個典型的觀察是：用 30 個粒子在一般教室建圖，結果相當漂亮——牆線筆直、迴路閉合、細節清晰。但同樣的設定拿到整層樓（數十公尺見方、多個房間與長走廊），地圖品質就明顯下降，甚至失敗。

原因是兩重的。其一是記憶體與運算：場景變大，每張地圖的格子數暴增，乘上粒子數後負擔沉重（公式五）。其二更根本：大場景意味著更長的軌跡、更多的累積漂移，需要更多粒子才能涵蓋可能的假設；但粒子一多，運算又跟不上，形成兩難。

更關鍵的是：當機器人繞完一大圈回到起點，即使閉環被「偵測」到，粒子濾波也無法像圖優化那樣回頭修正整條歷史軌跡——每個粒子的地圖早已依錯誤軌跡畫定，無法重來（第 12 章的核心論點）。這正是第 15 章 Cartographer 要解決的問題。同學若能親手做這個「教室成功、整層樓失敗」的對照實驗，對兩章的差異會有極深刻的體會。

## 案例 14-3　掃地機器人裡的粒子：理論就在你家

第 11 章案例提過，許多掃地機器人的除錯模式能看到粒子雲收斂的過程。本章讓我們更進一步：掃地機器人的「首次建圖」與「後續清掃」，其實對應兩種不同的技術。

首次進入陌生環境時，機器人必須同時定位與建圖——這是本章的 SLAM。而地圖建好之後，日後每次清掃只需在已知地圖中定位，這就是第 3 章與第 11 章的蒙地卡羅定位（AMCL）。兩者共用粒子濾波的骨架，差別在於「地圖是否已知」：SLAM 的粒子各帶一張成長中的地圖，AMCL 的粒子則共用同一張固定地圖。

理解這個區分很重要，因為它對應 ROS 中兩套不同的工具與流程（14.5 節）：建圖階

段跑 Gmapping、儲存地圖；導航階段用 map_server 載入地圖、跑 AMCL 定位。第 29 章的實作會完整走過這條鏈路。下次看家中掃地機器人第一次清掃時「小心翼翼地探索」、第二次「熟門熟路地直接開工」，你就知道它切換了兩套演算法。

## 實作活動

### 實作 14-1　用 Gmapping 建立教室地圖

目的：完整執行一次粒子濾波 SLAM 建圖，觀察粒子雲的演化，並透過參數實驗與失敗重現，理解 Gmapping 的機制與限制。本實作提供模擬與實體兩條路徑。

**路徑 A：模擬建圖（全班可做，不需硬體）**

95. 啟動 TurtleBot3 Gazebo 模擬環境與 SLAM 節點（第 28 章有完整安裝步驟）。
96. 開啟 RViz，同時顯示：佔據格地圖、雷射掃描、TF 座標軸，以及粒子雲（若工具支援）。
97. 用鍵盤遙控機器人在模擬房間中緩慢移動，一邊移動一邊觀察地圖如何逐格「長」出來、未知的灰色區域如何被填滿。
98. 觀察粒子雲：機器人直線前進時粒子雲如何散開？轉彎或看到明顯特徵時如何收斂？對照第 11 章與 14.3 節的理論。
99. 速度實驗：先用很慢的速度建一次圖，再用最快速度建一次，比較兩張地圖的品質差異，重現 14.6 節「移動太快」的失敗模式。
100. 儲存地圖，用第 13.7 節的方法評估品質：牆是否筆直？迴路是否閉合？量測地圖上的房間尺寸與模擬設定值比較。

**路徑 B：實體小車建圖（有硬體的班級）**

101. 前置檢查：確認 /scan 有資料、TF 樹完整（特別是 base_link→laser）、里程計會隨移動變化。這一步不可跳過。

102. 里程計校正：讓小車直走 3 公尺，比對里程計讀值與捲尺實測；原地轉 10 圈，比對角度讀值。若誤差大，修正輪徑與輪距參數。

103. 清場：請同學退到教室邊緣，移開可能遮蔽光達的物品，若有玻璃隔間可暫時貼紙。

104. 緩慢遙控小車繞行教室一圈（含四個角落），回到起點，觀察迴路是否閉合。

105. 儲存地圖檔（.pgm 與 .yaml），並在地圖上量測教室尺寸，與捲尺實測值比較，計算誤差百分比。

**第二部分：參數實驗（大學部）**

106. 粒子數實驗：分別用 5、30、100 個粒子在同一場域建圖，比較地圖品質與運算負擔（可觀察 CPU 使用率與更新頻率）。

107. 解析度實驗：用 0.02、0.05、0.10 m 三種解析度建圖，比較細節呈現與記憶體用量，驗證公式五的估算。

108. 走廊挑戰：把機器人開進長走廊建圖，對照第 13.6 節與 14.6 節的預測，觀察是否出現變形。嘗試在走廊沿途放置紙箱提供特徵，再建一次，比較改善程度。

109. 打滑實驗：在地面鋪一塊光滑板材或讓一側輪子短暫懸空，製造打滑，觀察地圖如何出錯。

**討論問題**

67. 快速移動與緩慢移動建出的地圖差多少？從 14.6 節與第 13.3 節的角度，解釋為什麼「開慢一點」如此有效。

68. 粒子數從 5 增加到 100，地圖品質改善了多少？運算負擔增加多少？你認為在你的場域，最合適的粒子數是多少？

69. 走廊建圖出現了什麼問題？放置紙箱後改善了嗎？這印證了 14.6 節的哪一項論述？

70. 若機器人繞了一大圈回到起點但地圖沒有閉合，Gmapping 有辦法「事後修正」整張地圖嗎？為什麼？（提示：第 12 章與案例 14-2）

**延伸挑戰（大學部）**

其一：用 Python 實作一個極簡的 RBPF 模擬——在程式中定義一個虛擬房間，建立 20 個粒子（各自有位姿與一張小地圖），實作「移動加雜訊→秤重→選擇性重採樣→各自更新地圖」的循環，把每個粒子的地圖並排畫出，觀察正確的假設如何勝出。其二：建完地圖後，接續啟動 map_server 與 AMCL，讓機器人在自己建的地圖中定位並執行導航（第 29 章），完成「建圖→定位→導航」的完整鏈路。

## 延伸閱讀


90. G. Grisetti, C. Stachniss, and W. Burgard, "Improved Techniques for Grid Mapping with Rao-Blackwellized Particle Filters," IEEE Transactions on Robotics, 2007.（Gmapping 的原始論文，本章主要依據）
91. M. Montemerlo et al., "FastSLAM: A Factored Solution to the Simultaneous Localization and Mapping Problem," AAAI, 2002.（RBPF-SLAM 的開創性工作，與 Gmapping 互為對照）
92. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005.（第 13 章 FastSLAM 與粒子濾波 SLAM 的完整理論）
93. Cyrill Stachniss, Particle Filter 與 FastSLAM 系列課程影片（YouTube）.（原作者親自講解，搭配本章閱讀效果最佳）
94. OpenSLAM Gmapping 開源專案與 ROS slam_gmapping 套件文件.（原始程式碼與參數說明）
95. slam_toolbox 開源專案（GitHub）.（ROS 2 環境中基於圖優化的現代 2D SLAM，可與 Gmapping 對照實驗）
96. ROBOTIS TurtleBot3 e-Manual 之 SLAM 章節.（模擬與實體建圖的完整操作指引）


## 從粒子到圖：本章的位置回顧

本章讓同學看到了 SLAM 第一次真正運作起來的樣子：位姿與地圖同時被估計，而且估得還不錯。這是第四篇的第一個完整系統。

但圖 14-3 左圖那條曲線也誠實地說明了它的天花板。即使把粒子數加到一百，平均誤差也只降到 0.08 公尺，而且運算量已經是三十顆的三倍以上。更根本的問題是：粒子濾波的所有修正都發生在「當下」——它沒有任何機制可以回頭改寫走過的軌跡。

想像機器人繞了整棟樓一圈回到起點，發現眼前的景象與地圖上記錄的起點對不上。Gmapping 能做的只有從這一刻起慢慢修正，中間那一整圈已經寫進每顆粒子地圖裡的誤差，再也拿不出來。這就是第 13.7 節所說的「漂移」病徵，而它的解藥是閉環。

第 15 章的 Cartographer 用三件事解決這個問題：把掃描累積成子圖（Submap）以降低位姿圖的規模、用分支定界法在全圖中高效搜尋閉環候選、再用第 12 章的位姿圖最佳化一次修正所有子圖的位置。同一個教室、同一台小車，第 15 章會讓同學親眼比較兩者建出的地圖差在哪裡——那時再回頭看本章，Gmapping 的優點與極限都會變得非常具體。

## 小結

63. Gmapping 由 Grisetti、Stachniss 與 Burgard 於 2007 年提出，是最著名的 2D 雷射 SLAM 系統之一；輸入為光達掃描、里程計與 TF，輸出為佔據格地圖與 map→odom 變換。
64. Rao-Blackwell 化把 SLAM 拆成兩部分：軌跡（低維，用粒子取樣）與地圖（已知軌跡即可解析求出）；每個粒子是一個「平行世界」，各帶一條軌跡與一張地圖，以觀測評分競爭，漂亮地解開雞生蛋問題。
65. 粒子數是精度與運算的權衡（教室約 30 個）；權重代表假設與觀測的吻合度；重採樣讓好假設存活，但過度重採樣會造成粒子退化，使錯誤假設獨佔而無法挽回。

66. Gmapping 的兩項關鍵改良：以掃描匹配改良提議分布（讓粒子撒在合理位置，所需粒子數從數千降至數十）、以及選擇性重採樣（只在有效粒子數低於門檻時才重採樣，避免退化）。

67. ROS 實作的三個前置條件缺一不可：/scan 雷射資料、里程計與 odom→base_link 變換、完整正確的 TF 樹（尤其 base_link→laser 外參）；建圖後以 map_saver 存成 .pgm 與 .yaml。

68. 四大失敗模式：輪子打滑（校正輪徑輪距、調大里程計雜訊）、場景重複（走廊，靠里程計與增加特徵）、光達遮蔽（玻璃、黑色物體、高度盲區）、移動太快（放慢是最有效的一招）。

69. Gmapping 的根本限制在於記憶體隨「粒子數×地圖大小」成長，難以應付大場景；且身為濾波方法，無法回頭修正歷史軌跡與地圖——這正是下一章圖優化路線要解決的問題。

本章完成了第一個完整的 SLAM 系統。Gmapping 以粒子濾波優雅地解開雞生蛋問題，但也讓我們清楚看到濾波路線的天花板：大場景吃不消、歷史無法修正。下一章 Cartographer 登場——它由 Google 開發，以子地圖（Submap）組織空間、以第 12 章的圖優化執行閉環修正，能在大範圍場景中即時建圖，並在偵測到閉環的瞬間把整張歪掉的地圖「一鍵拉正」。第 12 章那個令人震撼的畫面，即將在真實系統中重現。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

## 第 15 章

## Cartographer：Google 2D / 3D SLAM

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

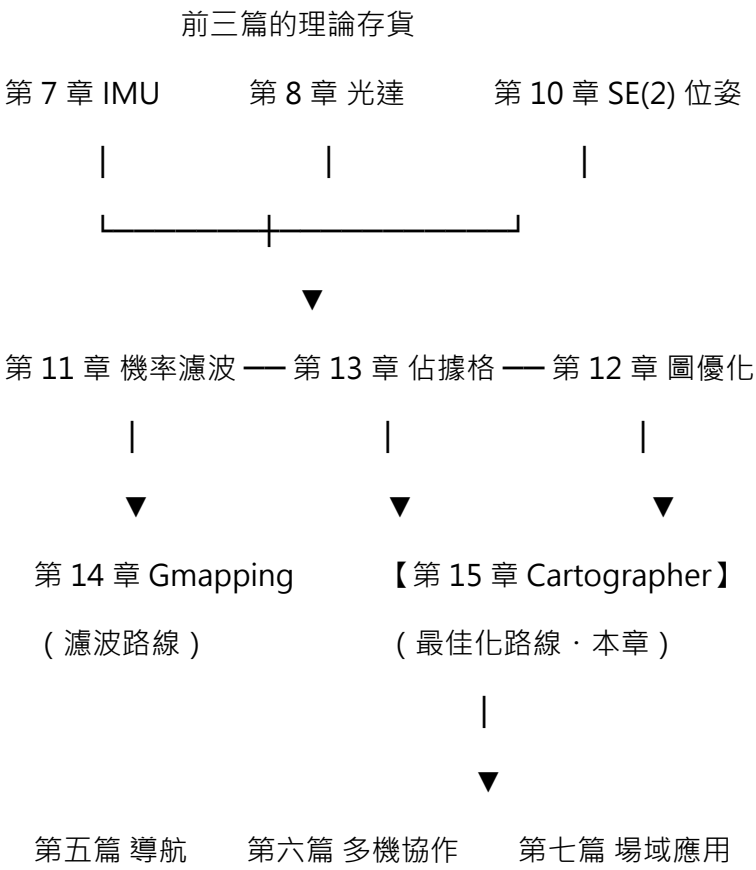
大學智慧機器人學程 / 高中 AI 機器人社團課程

## 本章在全書的位置

第四篇走到這裡，兩條理論主線終於各自長出了一套完整的系統。第 11 章的貝氏濾波，在第 14 章長成了 Gmapping；第 12 章的圖優化，就在本章長成 Cartographer。兩章不是並列的替代品，而是同一個問題在兩種數學框架下的答案，也因此，把它們放在一起讀，比分開讀各自多出一倍的收穫。

往回看，本章幾乎動用了前三篇的全部存貨：第 8 章的光達規格決定了掃描匹配的素材品質，第 7 章的 IMU 在 3D 模式中幾乎是必要條件，第 10 章的 SE(2) 位姿合成是子地圖拼貼的底層語言，第 11 章的機率格概念被第 13 章具體化成子地圖的內部表示，而第 12 章的稀疏位姿調整，就是本章全域 SLAM 的引擎本身。往前看，本章產出的 .pbstream 地圖與閉環修正能力，會在第五篇的導航、第六篇的多機與第七篇的場域應用裡一再被取用。

### 圖 15-0　第 15 章的連結地圖




## 學習目標

上一章的 Gmapping 以粒子濾波解開了雞生蛋問題，但也暴露了濾波路線的天花板：大場景吃不消、歷史無法修正。本章介紹 Google 開發的 Cartographer——它以子地圖組織空間、以第 12 章的圖優化執行閉環修正，能在大範圍場景中即時建圖，並在偵測到閉環的瞬間把整張歪掉的地圖「一鍵拉正」。學完本章之後，同學應能夠達成下列目標：

89. 說明 Cartographer 的雙子系統架構：局部 SLAM 與全域 SLAM 的分工。
90. 解釋子地圖（Submap）的概念，以及為何要把掃描分組成子地圖。
91. 說明局部與全域兩種掃描匹配的目的與差異。
92. 說明分支定界法如何讓即時閉環偵測成為可能。
93. 說明在 ROS 中執行 2D Cartographer 建圖的流程與關鍵設定。
94. 說明 3D Cartographer 與背包式空間建模的應用。
95. 比較 Cartographer 與 Gmapping 的架構、優缺點與適用場合。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 子地圖 | Submap | 由數十筆掃描組成的小塊機率格地圖，是 Cartographer 的基本單元。 |
| 局部 SLAM | Local SLAM | 前端：把新掃描插入當前子地圖，局部一致但會漂移。 |
| 全域 SLAM | Global SLAM | 後端：在背景尋找閉環約束並最佳化整個位姿圖。 |
| 機率格 | Probability Grid | 子地圖的內部表示，每格記錄佔據機率。 |
| 掃描匹配 | Scan Matching | 把新掃描對齊到子地圖或既有子地圖的過程。 |
| 閉環約束 | Loop Closure Constraint | 連接重訪位置的邊，用來消除累積誤差。 |

| 中文名詞 | 英文名詞 | 概念說明 |
| --- | --- | --- |
| 分支定界 | Branch and Bound | 有效率地搜尋最佳匹配、剔除不可能解的演算法。 |
| 多解析度預算格 | Precomputed Grids | 事先算好的不同解析度地圖，加速分支定界搜尋。 |
| 稀疏位姿調整 | Sparse Pose Adjustment | 對子地圖與掃描位姿構成的圖執行的最佳化。 |
| 非線性最小平方 | Nonlinear Least Squares | Cartographer 以 Ceres 求解的最佳化形式。 |
| 背包式建模 | Backpack Mapping | 人員背負感測器組行走以掃描建築空間。 |
| 穩健核函數 | Robust Kernel | 降低錯誤閉環影響的最佳化技巧。 |

## 15.1 Cartographer 系統架構

Cartographer 由 Google 開發並於 2016 年開源，其核心論文標題直接點出了它的主打特色：「2D 光達 SLAM 中的即時閉環」。這個名稱本身就宣示了它與 Gmapping 的根本差異——它把閉環偵測與圖優化做進了系統核心，而且能即時執行。

### 兩個子系統：局部與全域

理解 Cartographer，最重要的是掌握它的雙子系統架構。它可以被視為兩個獨立但相關的子系統：局部 SLAM（也稱為前端或局部軌跡建構器）負責建立一連串的子地圖，每個子地圖在局部上是一致的，但我們接受局部 SLAM 會隨時間漂移；全域 SLAM 則在獨立的執行緒中運行，尋找子地圖之間的閉環約束。

- 局部 SLAM（前端）：把每一筆新掃描匹配並插入「當前子地圖」。它追求即時與局部精確，但明知會隨時間累積漂移——這是刻意的設計取捨。
- 全域 SLAM（後端）：在背景執行緒中，把掃描與附近的子地圖做匹配以建立閉環約束，並持續最佳化由子地圖與掃描位姿構成的約束圖。

關鍵在於：兩者同時運行。前景快速建圖讓操作者立即看到結果，背景默默尋找閉環並修正全域一致性。這正是它能「即時」的原因，也是與 Gmapping 最大的架構差異。

**圖 15-1 Cartographer 雙子系統架構**

【局部 SLAM（前端）——前景執行】

新掃描 → 掃描匹配到當前子地圖 → 插入子地圖 → 子地圖完成後封存

特性：即時、局部一致、會漂移

【全域 SLAM（後端）——背景執行緒】

掃描 ↔ 附近既有子地圖做匹配 → 找到好匹配則建立閉環約束

↓

把子地圖位姿與掃描位姿組成位姿圖（第 12 章）

↓

每隔數秒以 Ceres 求解非線性最小平方 → 整張地圖被拉正

兩者同時運行：前景給即時預覽，背景保證全域一致

### 與第 12 章的連結

同學應該立刻認出來：全域 SLAM 做的正是第 12 章的圖優化。它結合了局部掃描匹配形成的子地圖（以 Ceres 為基礎的非線性最小平方最佳化器精修的機率格），與帶有分支定界閉環偵測的稀疏位姿圖，在一般等級的硬體上達成精確的即時 2D SLAM。第 12 章提到 Cartographer 使用 Ceres 函式庫，在此得到印證。理論與系統，在這一章正式接軌。

## 15.2 Submap 子地圖

子地圖是 Cartographer 最具識別度的設計。理解它為何存在，就理解了 Cartographer 為何能處理大場景。

### 什麼是子地圖

子地圖是一小塊機率格地圖（第 13 章的佔據格），由連續數十筆雷射掃描累積而成。機器人移動時，新掃描不斷插入當前子地圖；當累積到設定的掃描數，這個子地圖就「完成」並被封存，系統改建下一個子地圖。整個環境因此被切成一系列部分重疊的小塊地圖。

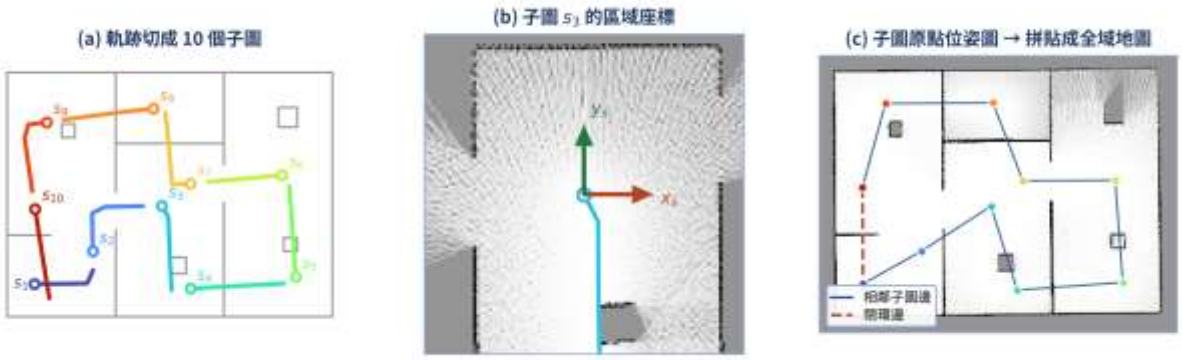
### 為什麼要分子地圖：三個理由

107. 限制誤差累積：由於掃描只與包含少數近期掃描的子地圖做匹配，這個做法會緩慢累積誤差；對於僅僅數十筆連續掃描而言，累積的誤差很小。換句話說，在一個子地圖的短暫生命週期內，漂移小到可以忽略——每個子地圖內部是「乾淨」的。

108. 提供最佳化的單元：全域最佳化時，不需要調整每一筆掃描，只需調整「子地圖的位姿」。一個子地圖代表數十筆掃描，這讓位姿圖的節點數大幅減少，最佳化因此能即時完成。這是效率的關鍵。

109. 成為閉環比對的對象：閉環偵測時，是拿「當前掃描」去比對「過去的子地圖」。子地圖比單筆掃描包含更多資訊、更穩定，比對結果也更可靠。

**圖 15-2　子圖的三個面向：分段、區域座標、拼貼**




這張圖把「子地圖」這個抽象名詞拆成三個可以用眼睛確認的面向。左圖是同一條軌跡被切成十段，每一段就是一個子地圖 $s_1$ 至 $s_{10}$；空心圓標出的是每個子地圖的原點位姿，也就是這段路程開始時機器人所在的位置與朝向。請注意：切分的依據不是走了多遠，而是累積了多少筆掃描——Cartographer 以掃描數為單位封存子地圖，正是為了讓每個子地圖內部的

誤差被控制在可忽略的量級。

中圖把第三個子地圖單獨拿出來，放進它自己的區域座標系。紅色與綠色箭頭就是這個子地圖的 x 軸與 y 軸，原點永遠落在這段路程的起點。子地圖內部完全不知道自己在世界的哪裡，它只記得「相對於我的原點，這面牆在哪、那個轉角在哪」。這個刻意的失憶，正是 Cartographer 能把大場景切小的關鍵：區域的歸區域，全域的歸全域。

右圖是最後的拼貼結果。藍線連起相鄰子地圖的原點，構成一條位姿鏈；紅色虛線則是閉環邊，把最後一個子地圖拉回第一個。真正被最佳化的變數，只有圖上那十個彩色圓點——十個子地圖原點的位姿，合計三十個未知數。相對之下，這段軌跡本身有近六十個時刻；若逐一最佳化每一筆掃描的位姿，未知數會多出一倍，而最佳化的計算量大致隨未知數成長。這就是第 15.2 節所說「提供最佳化的單元」的具體意思。

把三張圖連起來看，子地圖的角色就清楚了：它既是誤差的隔離艙（區域誤差不外溢），也是最佳化的樂高積木（一塊代表數十筆掃描），還是閉環比對的對象（資訊量足夠，比單筆掃描穩定）。一個設計同時解掉三個問題，這是 Cartographer 最值得學習的工程品味。

**子地圖要切多大？**

太小：子地圖數量暴增，位姿圖節點變多，全域最佳化變慢；而且每個子地圖包含的幾何特徵太少，閉環比對容易誤判。

太大：子地圖內部累積的掃描匹配誤差開始顯著，「區域誤差可忽略」的前提被破壞，閉環拉正後牆線仍然歪斜。

實務起點：2D 室內場域常用 70～100 筆掃描封存一個子地圖（表 15-1 的 num_range_data）；走廊等特徵稀少處宜調小，雜亂房間可調大。

判斷方法：在 RViz 中看子地圖之間的重疊區。相鄰子地圖的牆線若能重合，大小就設對了。

### 與 Gmapping 的對比

這裡可以清楚看出兩套系統的哲學差異。Gmapping 讓每個粒子維護一整張完整地圖（第 14 章），記憶體隨粒子數乘上地圖大小成長；Cartographer 則只維護一張由眾多子地圖拼成的地圖，靠調整子地圖位姿來修正全域一致性。

這個差異的後果很直接：Cartographer 的記憶體需求與場景大小成正比（而非乘上粒子數），因此能處理整棟建築；而且因為保留了子地圖與位姿圖，它能「回頭修正」——這正是第 12 章指出的圖優化相對濾波的核心優勢。

## 15.3 Scan Matching

Cartographer 中有兩種掃描匹配，目的截然不同。分清楚它們，是理解系統運作的關鍵。

### 一、局部掃描匹配：插入當前子地圖

這是前端的日常工作。每來一筆新掃描，系統要決定「它該貼在當前子地圖的哪個位置」。做法是以里程計或 IMU 提供初始猜測（第 13.4 節的思想），再用最佳化方法精修，使掃描點與子地圖的機率格盡可能吻合。

Cartographer 把這個匹配表述為非線性最小平方問題，以 Ceres 求解（第 12 章）。這個做法與傳統 ICP（第 13.3 節）不同——它不是找點對點的對應，而是直接最大化「掃描點落在高佔據機率格」的程度，這對雜訊較不敏感。

### 二、全域掃描匹配：尋找閉環

這是後端的工作，難度高得多。問題是：當前掃描，是否與某個很久以前建立的子地圖吻合？若吻合，就找到了閉環。

困難之處在於搜尋空間。閉環候選可能在任何位置、任何角度，而且由於已經累積了漂移，初始猜測可能偏差很大——不能只在小範圍內找。若用暴力搜尋，把每個可能的位置

與角度都試一遍，運算量將大到無法即時執行。

這正是 Cartographer 最精妙的技術貢獻要解決的問題，也是下一節的主題。

### 限制搜尋範圍的實務考量

即使有高效演算法，系統仍需控制計算量。為了限制約束（與計算量）的數量，Cartographer 只考慮所有鄰近節點的一個子取樣集合來建立約束，這由一個取樣比例常數控制；取樣太少可能導致錯過約束、閉環失效，取樣太多則會拖慢全域 SLAM，使即時閉環無法達成。這是實作調參時的重要權衡。

## 15.4 閉環偵測與全域最佳化

本節是全章的技術核心：Cartographer 如何做到「即時」閉環——這正是它論文標題所宣稱的成就。

### 分支定界：剪掉不可能的分支

面對龐大的搜尋空間，Cartographer 採用分支定界法（Branch and Bound）。它以分支定界機制在不同的格地圖解析度上運作，有效率地剔除不正確的匹配。

分支定界的核心思想，可以用一個生活比喻理解：假設你要在一整棟大樓裡找一個人。笨方法是逐間房間搜遍；聰明的做法是先在每一層樓快速掃視（低解析度），若某一層明顯不可能有人（例如整層在裝修），就跳過整層不必逐間找——這就是「剪枝」。

Cartographer 的做法正是如此：它把既有子地圖降尺度成幾種不同解析度（例如四分之一、二分之一大小）作為掃描匹配的輸入，匹配過程從低解析度開始做初步的低精度計算，得到最佳分數的初步結果後，用它跳過高解析度中大量不可能的平移旋轉組合，如此反覆直到抵達完整解析度。系統會預先算好尺寸為 1、4、16、64 的多解析度格網來支援這個搜尋。

### 為什麼這樣就能即時

關鍵在於「上界」的概念。在低解析度地圖上算出的分數，是該區域在高解析度下所有

可能匹配分數的上界（不會低估）。因此若某區域在低解析度下的分數，已經低於目前找到的最佳答案，那麼它在高解析度下也絕不可能更好——整個區域可以安全地跳過，不必展開。

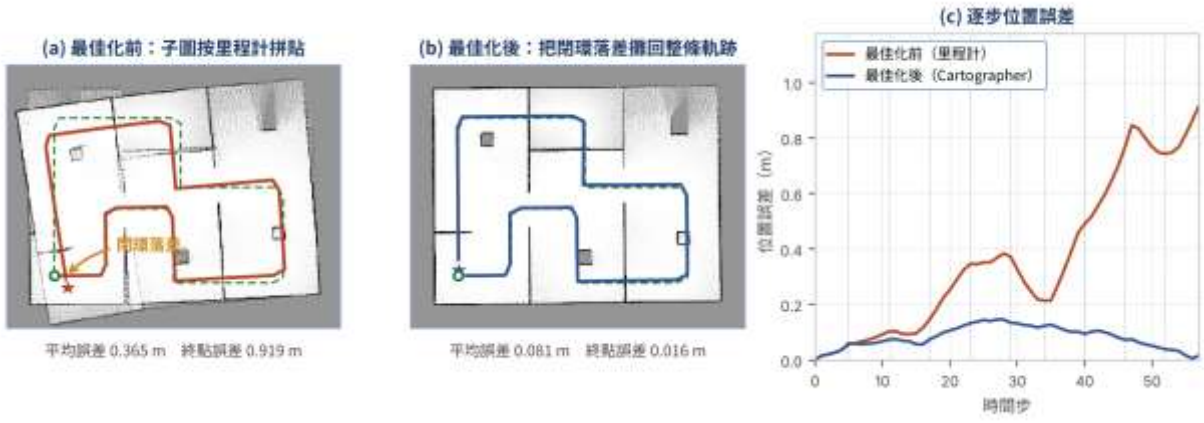
這讓搜尋量從「所有可能組合」降到「少數有希望的分支」，即時閉環因此成為可能。這個機制在 Cartographer 論文中有詳盡的說明，而其專用的快速相關掃描匹配器正是為了讓即時閉環的掃描匹配成為可能而設計的。

### 全域最佳化：把矛盾攤平

找到閉環約束後，就進入第 12 章的圖優化。閉環最佳化與掃描匹配一樣，也被表述為非線性最小平方問題，這讓系統能輕易加入殘差以納入額外資料；每隔幾秒，系統就用 Ceres 計算一次解。

這個最佳化調整的是「子地圖位姿」與「掃描位姿」，稱為稀疏位姿調整（Sparse Pose Adjustment）。當閉環約束與里程計約束矛盾時，最佳化把矛盾均攤到整條軌跡上——第 12 章那個「整張歪掉的地圖唰地被拉正」的畫面，在此真實上演。

**圖 15-3　閉環前後：一條約束如何把整張地圖拉正**




這是本章最值得停下來看的一張圖，因為它把案例 15-1 所說「課堂上最震撼的三秒鐘」變成了可以量化的數字。圖(a)是最佳化之前：子地圖純粹按里程計的推算拼在一起。整張地圖明顯歪斜，外牆沿著對角線整片旋轉，而機器人繞完一圈後的終點（星號）並沒有回到出

發點（圓圈），兩者差了 0.92 公尺。金色箭頭指的就是這個落差——它不是感測器的錯，而是數十次微小角度誤差一路累積的結果。

圖(b)是加入一條閉環邊、重新求解位姿圖之後。請注意變化的性質：不是把終點硬生生搬回起點，而是把這 0.92 公尺的矛盾，依照各段約束的可信度攤回整條軌跡。結果是整張地圖轉正、牆線收斂成單一線條，終點誤差降到 0.016 公尺，平均誤差從 0.365 降到 0.081 公尺——降幅接近八成。這正是第 12.3 節「共變異數決定誰讓步」在真實數據上的樣子。

圖(c)把兩條軌跡的逐步誤差畫在一起，是整張圖最有教育意義的一格。紅線（最佳化前）呈現典型的漂移形狀：前段幾乎貼著零，中段緩升，後段急遽翹起——誤差不是線性成長，而是隨著角度偏差被後續路程放大。藍線（最佳化後）則被壓成一個平緩的拱形：最高點出現在軌跡中段（約 0.15 公尺），兩端則被起點與閉環邊同時錨住而趨近於零。

這個拱形值得同學記住，因為它是位姿圖最佳化的指紋。當一條軌跡的兩端都有強約束、中間只靠里程計串接時，誤差的分布必然是兩端低、中間高。它同時也告訴我們一件實務上的事：如果場域能走出兩個以上的迴路，就會有兩條以上的閉環邊，拱形會被進一步壓低。這就是第 15.5 節建圖流程要求「務必走完整個迴路」的理由，也是下一節比較實驗的伏筆。

**閉環拉正與「地圖突然跳一下」**

為什麼是跳的：全域最佳化每隔數秒執行一次，一旦找到閉環約束，整組子地圖位姿同時被更新，畫面上就是整張地圖瞬間重排——這就是案例 15-1 的三秒鐘。

為什麼導航要小心：地圖與 map→odom 變換同時改變，正在執行的路徑可能瞬間失效。第五篇的導航堆疊必須能承受這種不連續，這也是實務上常把「建圖」與「導航」分成兩個階段的原因。

怎麼確認拉正成功：看迴路接縫處。拉正前牆會有重影或錯開（第 13.7 節的病徵），拉正後應收斂成單一條線。

沒拉正怎麼辦：多半是閉環分數門檻過高或取樣比例過低（表 15-1），先放寬門檻

再試；但門檻放太寬會引來錯誤閉環，請搭配第 15.4 節的穩健核函數一起調。

### 穩健化：防止錯誤閉環的災難

閉環偵測有可能誤判（例如兩個長得很像的房間）。一條錯誤的閉環約束，會把地圖硬拉到錯誤的位置，造成災難性後果（第 12 章實作活動提過）。因此 Cartographer 在最佳化中採用穩健核函數，降低離群約束的權重，讓少數錯誤匹配不至於毀掉整張地圖。這呼應第 12 章延伸挑戰中「加入錯誤閉環邊」的實驗——現在同學知道實務系統是怎麼防範的了。

## 15.5　2D Cartographer 實作

本節說明在 ROS 中執行 2D Cartographer 建圖的實務要點。與第 14.5 節的 Gmapping 流程對照閱讀，能看出兩者的異同。

### 前置條件

與 Gmapping 相同的三要件仍然適用：雷射掃描（/scan）、里程計、完整的 TF 座標樹（第 10 章）。但 Cartographer 有兩點不同：

- IMU 的角色更重要：Cartographer 能有效利用 IMU 提供的姿態資訊（第 7 章），尤其在 3D 模式中幾乎是必要的。2D 模式若有 IMU，建圖穩定度會明顯提升。
- 里程計並非絕對必要：由於局部掃描匹配的能力較強，Cartographer 在特徵豐富的環境中即使沒有里程計也能運作。但在走廊等特徵稀少處（第 13.6 節），里程計仍然關鍵。

### 設定檔：Lua 組態

Cartographer 的一大特色（也是初學者的門檻）是它使用 Lua 語言的設定檔，參數數量遠多於 Gmapping。大部分局部 SLAM 的選項可以在 2D 的 trajectory_builder_2d.lua 與 3D 的 trajectory_builder_3d.lua 設定檔中找到。

關鍵參數群組包含：

| 參數群組 | 控制什麼 | 調校方向 |
|---|---|---|
| 子地圖掃描數 | 每個子地圖累積幾筆掃描 | 太少則子地圖不穩、太多則漂移進子地圖內 |
| 解析度 | 機率格的格子邊長 | 室內常用 0.05 m（第 13.5 節） |
| 局部匹配權重 | 掃描匹配時各項殘差的相對重要性 | 里程計可靠則提高其權重 |
| 閉環取樣比例 | 考慮多少節點來建立約束 | 太低錯過閉環、太高拖慢即時性（15.3 節） |
| 閉環分數門檻 | 匹配分數多高才接受為閉環 | 太低易誤判、太高則閉環偵測不到 |
| 最佳化週期 | 多久執行一次全域最佳化 | 通常數秒一次 |

**表 15-1　2D Cartographer 主要參數群組**

### 建圖流程

110. 確認感測器資料與 TF 樹正常（同第 14.5 節）。

111. 準備 Lua 設定檔，依平台調整感測器組態（有無 IMU、有無里程計、光達數量）。

112. 啟動 Cartographer 節點與 RViz，顯示子地圖、軌跡與雷射掃描。

113. 緩慢遙控機器人建圖。特別留意：走完迴路回到起點時，觀察是否觸發閉環——這是本章最值得觀察的一刻。

114. 建圖完成後，執行「結束軌跡」與最終最佳化，再匯出地圖檔（.pbstream 格式，可再轉為 ROS 標準的 .pgm 與 .yaml）。

### 觀察重點：子地圖與閉環

在 RViz 中，Cartographer 會把各個子地圖以不同方式顯示，同學可以清楚看到地圖是由一塊塊子地圖拼成的。而最精彩的時刻是閉環觸發的瞬間：原本錯開的軌跡與地圖會突然

「跳」到正確位置。務必安排課堂時間讓同學親眼見證這一刻——這是第 12 章抽象理論最有力的具象化。

## 15.6 3D Cartographer 與背包式空間建模

Cartographer 不只做 2D。它同樣支援 3D SLAM，而 Google 開發它的原始動機之一，正是室內空間的三維測繪。

### 3D 模式的差異

- 感測器：需要 3D 光達（多線，第 8 章）。Google 的系統最初搭載兩具 Hokuyo 雷射掃描儀與一個慣性量測單元，後來的版本則改用兩具 Velodyne PUCK 掃描儀。
- IMU 的必要性：3D 模式下，姿態（滾轉與俯仰，第 7 章）的估計至關重要，IMU 幾乎是必備的。
- 子地圖為 3D：機率格從平面變成立體的體素（Voxel）格，記憶體需求大幅上升。
- 運算負擔：3D 掃描匹配與最佳化的計算量遠高於 2D，對硬體要求較高。

### 背包式空間建模

第 8 章案例 8-3 提過背包式移動掃描，Cartographer 正是這類系統的代表。操作員背負搭載光達與 IMU 的背包系統，一邊行走一邊即時建立整棟建築的三維模型。

這個應用之所以重要，是因為它解決了傳統測繪的痛點：傳統靜態雷射掃描儀必須在每個定點架設、掃描、再搬到下一點，耗時費力；背包式系統則「走到哪掃到哪」，效率提升數十倍。

- 應用場景：建築資訊模型（BIM）建置與驗證、古蹟與文化資產數位保存、廠房設施管理、室內導航地圖製作、災後結構評估。

- 對大學場域的意義：這是極具潛力的專題方向——用 3D SLAM 掃描校園建築、實驗室或工廠，產生可用於教學與研究的數位分身，把第五篇（第 17、18 章）的 3D 點雲技術實際應用出來。

**連續時間 SLAM 的延伸研究**

值得一提的是，學術界持續在 Cartographer 的基礎上改良。例如有研究團隊指出，把掃描視為「連續時間軌跡」而非離散時刻的快照，能進一步改善 3D 建圖品質——這類研究正是建立在對 Cartographer 架構的深入理解之上。對有志研究的同學而言，理解本章內容，就是進入這個領域的門票。

## 15.7 Cartographer 與 Gmapping 比較

本篇的兩大主角至此都已介紹完畢。本節做系統性的對照，這也是第四篇最重要的一張表。

| 比較項目 | Gmapping（第 14 章） | Cartographer（本章） |
|---|---|---|
| 理論基礎 | 粒子濾波（第 11 章） | 圖優化（第 12 章） |
| 地圖組織 | 每個粒子一張完整地圖 | 一系列子地圖拼接 |
| 閉環處理 | 無專門機制，難以事後修正 | 核心功能，可即時拉正全圖 |
| 記憶體 | 粒子數 × 地圖大小 | 與場景大小成正比 |
| 大場景 | 受限 | 擅長，可建整棟建築 |
| 3D 支援 | 無（僅 2D） | 支援 2D 與 3D |
| IMU 利用 | 不使用 | 可有效整合，3D 幾乎必備 |
| 參數複雜度 | 較少，易上手 | 較多（Lua 設定檔），門檻較高 |
| 教學價值 | 理解粒子濾波的最佳範例 | 理解圖優化與閉環的最佳範例 |

**表 15-2 Cartographer 與 Gmapping 的系統比較**

**圖 15-4 同一場域、同一條路線：兩套系統的正面對決**

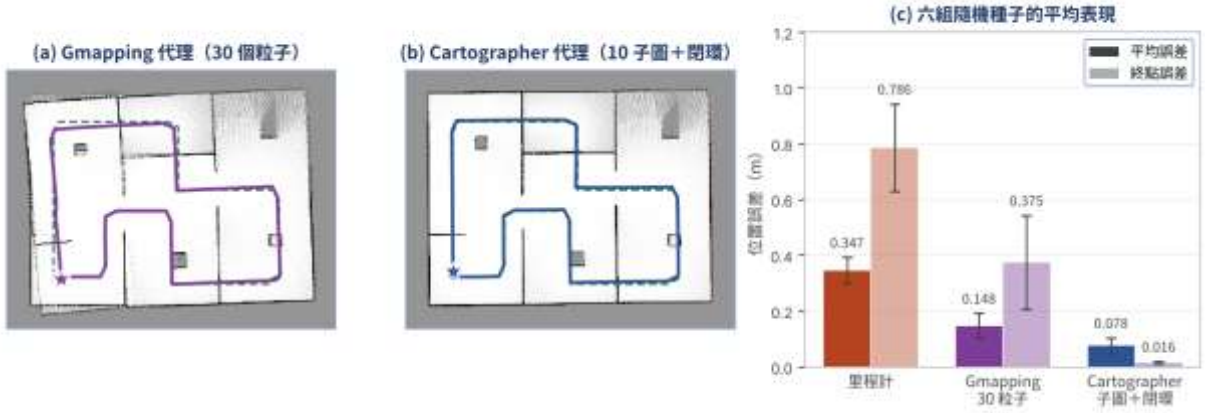



表 15-2 列出了兩套系統的架構差異，這張圖則把差異換算成數字。實驗條件完全一致：同一個 12×9 公尺的 L 形室內場域、同一條繞行一圈回到起點的路線、同一組里程計雜訊參數。圖(a)是第 14 章的 Gmapping 代理（30 個粒子），圖(b)是本章的 Cartographer 代理（10 個子地圖 + 一條閉環邊），圖(c)則是六組不同隨機種子下的統計結果。

先看地圖。兩張都比純里程計的結果好得多，但差別在接縫：Gmapping 的地圖在迴路接合處仍有輕微錯位，外牆帶著一點整體傾斜；Cartographer 的地圖則牆線筆直、迴路乾淨閉合。原因在圖(c)的數字裡——Gmapping 的終點誤差平均 0.375 公尺、標準差 0.168，Cartographer 則是 0.016 公尺、標準差僅 0.004。兩者差了超過二十倍，而且後者幾乎不隨種子波動。

平均誤差的對比同樣明確：里程計 0.347、Gmapping 0.148、Cartographer 0.078 公尺，也就是說粒子濾波把漂移砍掉約六成，而子地圖加閉環又在此之上再砍掉近一半。但更值得注意的是誤差棒的長度。Gmapping 的終點誤差標準差高達 0.168 公尺，接近它自己的平均值一半，這正是粒子濾波的隨機性：同樣的參數，有時運氣好、有時粒子耗盡。Cartographer 的誤差棒則短到幾乎看不見。

這個差異背後有一個結構性的理由，值得同學想清楚：Gmapping 是濾波器，它只能向前走，已經重採樣掉的歷史無法回頭修正，所以終點誤差一旦累積就只能帶著走；Cartographer 保留了整張位姿圖，閉環一旦出現，過去所有子地圖的位姿都可以被重新調

整。「能不能修改歷史」，就是濾波與最佳化兩條路線最根本的分野，也是表 15-2 中所有條目的共同源頭。

**關於這組對照實驗的誠實說明**

這是教學用的代理實作，不是 Gmapping 與 Cartographer 原始碼的效能評比。兩者都以本書的模擬場域重寫，以凸顯架構差異，實際套件的表現受參數、感測器與場域影響甚大。

結果並非在所有條件下都成立。若場域走不出迴路（例如一條直走到底的走廊），Cartographer 沒有閉環可用，優勢會大幅縮小——這正是討論問題第三題要同學思考的。

粒子數與子地圖數都會改變結果。本圖固定 30 粒子與 10 子地圖，同學可在實作 15-1 中自行掃描這兩個參數。

請同學用實作 15-1 第二部分在真實套件上重跑一次，並把你量到的數字與本圖比較。數字不同是正常的，重要的是趨勢是否一致。

### 如何選擇

- 小場域教學、理解粒子濾波：Gmapping。它參數少、概念直觀，是入門的好選擇。
- 大場域、需要閉環、要建整棟樓：Cartographer。它的架構就是為此而生。
- 需要 3D 建模、背包式掃描：Cartographer 3D。
- ROS 2 環境的現代 2D SLAM：除了 Cartographer，也可考慮 slam_toolbox——它同樣基於圖優化，參數較簡潔，近年在 ROS 2 社群中相當流行。

### 兩章的教學意義

把第 14 與 15 章並列閱讀，同學會發現：它們正是第 11 章（濾波）與第 12 章（最佳化）兩條理論路線的實體化身。Gmapping 展示了粒子濾波的巧思與天花板，Cartographer 展示了圖優化如何突破那個天花板。這不只是兩個工具的比較，而是 SLAM

十餘年技術演進的縮影。

至此，同學已經掌握 2D SLAM 的兩大系統。下一章我們把鏡頭轉向最貼近生活的應用——掃地機器人。從隨機碰撞到 SLAM 規劃、從房間分割到覆蓋路徑規劃，看這些理論如何在你家客廳裡運作，並理解家庭場景中桌椅、鏡面與低矮障礙物帶來的真實挑戰。

## 15.8 Cartographer 之後：2024–2026 年的 2D SLAM 新局

Cartographer 的論文發表於 2016 年，以本書寫作的時點來看，已經是一套「十年級」的系統。十年在機器人領域足以改朝換代，但 Cartographer 的核心設計——子地圖、分支定界、稀疏位姿調整——至今仍是 2D 光達 SLAM 的標準答案，幾乎所有後繼系統都是在這個骨架上做局部替換。本節整理近兩年的四條主要變化，讓同學知道學完本章之後，前沿在哪裡。

### 一、維護重心轉移：slam_toolbox 成為 ROS 2 的實務預設

最大的變化不在演算法，而在生態。隨著 ROS 2 成為主流，由 Steve Macenski 開發的 slam_toolbox 逐漸取代 Cartographer，成為多數 ROS 2 專案的預設 2D SLAM。它同樣走圖優化路線、同樣以掃描匹配建立位姿圖，但參數遠比 Cartographer 的 Lua 組態簡潔，並且內建了 Cartographer 較弱的兩項能力：地圖序列化與續建（把建好的位姿圖存檔，下次開機接著建），以及純定位模式（載入既有地圖只做定位，不再更新）。

這對教學的意涵是雙面的。一方面，若同學的目標是在 ROS 2 上把專題跑起來，slam_toolbox 通常是阻力最小的選擇；另一方面，Cartographer 仍是理解「為什麼要這樣設計」的最佳教材——它的論文把分支定界的上界推導寫得最清楚，而 slam_toolbox 的許多設計選擇，其實預設了讀者已經懂 Cartographer 在做什麼。本書的建議是：用 Cartographer 學原理，用 slam_toolbox 做專題。

### 二、終身建圖與多期地圖：地圖不再是一次性產物

第二條變化是問題設定本身的擴張。傳統 SLAM 假設「跑一趟、建一張圖、結束」，但真實場域會變：桌椅被移動、貨架被重排、季節改變了採光。終身建圖（lifelong mapping）處理的正是這件事——如何讓機器人在同一場域反覆行走時，把新觀測融進舊地圖，同時刪去已經不存在的障礙，而不是讓地圖愈建愈髒。slam_toolbox 的序列化與續建能力，正是朝這個方向設計的。

技術上，這把位姿圖從單一軌跡變成多期軌跡的集合，閉環不只發生在同一趟路程內，也發生在不同日期的路程之間（稱為多期 SLAM，multi-session SLAM）。這條線直接通往第六篇的多機協作：多台機器人同時建圖、再把各自的位姿圖合併，本質上就是多期 SLAM 的空間版本。

## 三、聯合最佳化位姿與地圖：Occupancy-SLAM 這一路

第三條是直接針對 Cartographer 架構的學理挑戰。Cartographer 的做法是「先用掃描匹配算出位姿，再把掃描插進地圖」——位姿是變數，地圖是位姿的副產品。澳洲雪梨科技大學團隊提出的 Occupancy-SLAM（發表於 Robotics: Science and Systems 2024，2025 年提出改良版）主張這個分工本身就是精度的瓶頸：既然地圖的品質取決於位姿，位姿的修正也應該回頭由地圖的一致性來驅動，那就應該把「機器人位姿」與「佔據格的每一格數值」當成同一個最佳化問題的變數，一起解。

這個想法在數學上很自然，困難在規模：一張 2D 佔據格動輒數十萬格，把每一格都當變數，問題規模比傳統位姿圖大上好幾個數量級。該團隊的貢獻正是找出這個大型問題的稀疏結構，讓它在可接受的時間內收斂。同學若讀過第 12 章的稀疏性討論（圖 12-3），會認出這是同一套思路的放大版——稀疏性永遠是大型最佳化能不能實用的分水嶺。

## 四、閉環偵測的學習化：從幾何比對到場景描述子

最後一條變化發生在閉環偵測。Cartographer 的分支定界是純幾何方法：它在多解析度格網上窮舉搜尋，靠上界剪枝加速。這個方法的優點是沒有訓練、沒有資料集依賴、行為完全

可預測；缺點是搜尋範圍一旦放大（例如機器人被人抱起來放到另一層樓，也就是第 11 章的綁架問題），計算量仍會爆炸。

近年的主流替代方案是「場景描述子」：把一幀掃描壓縮成一個固定長度的向量指紋，閉環偵測就變成在指紋資料庫中做最近鄰搜尋，與走了多遠無關。3D 領域的 Scan Context 系列是這條路的代表，2024 至 2025 年間的研究則延伸到以點雲密度圖做閉環、以及把地點辨識（place recognition）當成獨立問題系統性整理的綜述。2D 領域受限於單線掃描資訊量少，描述子的辨識力不如 3D，因此分支定界在 2D 仍未被取代——這也解釋了為什麼本章的技術核心到今天仍然值得細讀。

**四條線索，一個共同主題**

工程面（slam_toolbox）：把 Cartographer 的原理包裝得更好用，補上序列化與純定位。

問題面（終身建圖）：地圖從一次性產物變成需要長期維護的資產。

數學面（Occupancy-SLAM）：把地圖本身也拉進最佳化變數，挑戰「位姿先、地圖後」的分工。

感知面（場景描述子）：把閉環偵測從幾何窮舉換成特徵檢索，以應付大範圍與綁架問題。

共同主題：Cartographer 的骨架（子地圖 + 位姿圖 + 閉環）沒有被推翻，被替換的是骨架上的每一個零件。學會骨架，才讀得懂後面所有的零件更換。

## 承先啟後

本章把第 12 章的圖優化從數學式變成了一套可以在走廊上跑起來的系統。回頭看第四篇的安排：第 13 章教我們如何把掃描變成地圖，第 14 章用粒子濾波解開雞生蛋問題，第 15 章則示範了另一條路——保留歷史、容許回頭修正。圖 15-4 的數字說明了這條路在有迴路的場域中的優勢，而 15.8 節也誠實交代了它的邊界：沒有迴路，閉環就無用武之地。

從第 16 章起，鏡頭將從「演算法長什麼樣」轉向「這些演算法在真實產品裡怎麼活下來」。掃地機器人是最好的第一個案例：它便宜、量產、每天在數千萬個家庭裡運行，卻必須在極低的運算預算下完成本章談的每一件事。

## 公式或流程

### 一、局部掃描匹配的最佳化

把新掃描插入子地圖時，求一個位姿 ξ 使掃描點盡量落在子地圖的高佔據機率處：

$$\xi^* = \operatorname{argmin}_{\xi} \sum_k \left(1 - M(T_\xi \cdot h_k)\right)^2$$

其中 $h_k$ 是掃描中的第 k 個點、$T_\xi$ 是把它變換到子地圖座標的位姿變換（第 10 章）、$M(\cdot)$ 是該位置在機率格中的佔據機率。式子的意義是：讓所有掃描點都落在「地圖認為有障礙」的地方。這是一個非線性最小平方問題（第 12 章），以 Ceres 求解。

### 二、全域最佳化（稀疏位姿調整）

全域 SLAM 最佳化子地圖位姿 $\Xi^m$ 與掃描位姿 $\Xi^s$，使所有約束的加權殘差最小：

$$\operatorname{argmin} \tfrac{1}{2} \sum_{ij} \rho\left(E^2(\xi_i^m, \xi_j^s; \Sigma_{ij}, \xi_{ij})\right)$$

其中 E 是殘差（第 j 筆掃描相對第 i 個子地圖的實際關係與量測關係之差）、$\Sigma_{ij}$ 是該約束的共變異數（可信度，第 12.3 節）、ρ 是穩健核函數——它讓誤判的閉環約束不至於毀掉整張地圖（15.4 節）。這條式子與第 12 章的最小平方目標函數形式完全一致，只是把節點換成了子地圖與掃描。

### 三、分支定界搜尋

全域掃描匹配的搜尋空間定義為一個三維視窗（平移 x、平移 y、旋轉 θ）：

$$W = \{-w_x, \ldots, w_x\} \times \{-w_y, \ldots, w_y\} \times \{-w_\theta, \ldots, w_\theta\}$$

分支定界以「分支—定界—剪枝」反覆進行：

**圖 15-5　分支定界閉環搜尋流程**

① 預先計算多解析度格網（尺寸 1、4、16、64）

↓

② 從最低解析度開始，把搜尋空間切成數個大區塊（分支）

↓

③ 對每個區塊，在低解析度格網上計算「分數上界」（定界）

↓

④ 若某區塊的上界 < 目前最佳分數 → 整個區塊剪除，不再展開（剪枝）

↓

⑤ 否則把該區塊再切細，到更高解析度重複 ②–④

↓

⑥ 抵達完整解析度且分數超過門檻 → 接受為閉環約束

## 四、Cartographer 完整資料流

**圖 15-6　Cartographer 端到端流程**

感測器：雷射掃描 + IMU +（可選）里程計

↓

【局部 SLAM】掃描匹配 → 插入當前子地圖 → 子地圖滿了則封存，開新的

↓　（產生局部軌跡，會漂移）

【全域 SLAM，背景執行緒】

取樣節點 → 對既有子地圖做分支定界匹配 → 找到閉環約束

↓

組成位姿圖（子地圖位姿 + 掃描位姿）

↓

每隔數秒以 Ceres 執行稀疏位姿調整（含穩健核函數）

↓

輸出：全域一致的地圖與軌跡（閉環時整張地圖被拉正）

## 案例

### 案例 15-1　閉環的那一刻：課堂上最震撼的三秒鐘

第 12 章案例 12-1 描述過閉環拉正地圖的畫面，而 Cartographer 讓這個畫面成為課堂上可以重現的實驗。

典型的教學場景是這樣的：同學遙控機器人繞行系館一圈。前半程一切正常，地圖工整；繞到後半段時，牆線開始與最初建的部分錯開，走廊出現明顯的重影，同學臉上開始出現疑惑。就在機器人回到起點附近的瞬間——背景執行緒找到了閉環約束，全域最佳化執行——整張地圖在螢幕上「唰」地一下重組，重影合而為一，錯開的走廊接合，軌跡形成一個漂

亮的封閉迴路。

這三秒鐘，把第 12 章的圖優化、第 15.4 節的分支定界與稀疏位姿調整，全部濃縮成一個看得見的瞬間。教學經驗顯示，親眼看過這一幕的同學，對「為什麼要用圖優化」的理解遠勝於任何講述。建議教師務必安排足夠的空間讓機器人繞完一個完整迴路——這是本章最值得投資的課堂時間。

## 案例 15-2　Google 為何要做 Cartographer：室內地圖的野心

Google 開發 Cartographer 並非純學術興趣。當時 Google 已經用街景車繪製了全世界的道路，但室內空間——商場、機場、車站、博物館——仍是地圖的空白區。要繪製室內地圖，傳統測繪太慢太貴，必須有能「邊走邊建圖」的技術。

這解釋了 Cartographer 的幾個設計取向：為什麼要能即時（操作員需要立即知道哪裡還沒掃到）、為什麼要能處理大場景（整座商場）、為什麼要做背包式系統（人能走的地方都要能掃）、以及為什麼閉環如此重要（大範圍行走必然累積漂移）。

這個案例給同學一個重要啟示：技術的設計取向，往往由應用需求決定。Gmapping 出自學術實驗室，追求演算法的優雅；Cartographer 出自產業需求，追求大規模場域的實用性。兩者沒有優劣，只有適不適合——這正是第 8 章「感測器選型是綜合判斷」觀念的再次印證。

## 案例 15-3　感測器組態的實驗：什麼配置最適合什麼場域

有研究團隊系統性地測試了不同感測器組合對 Cartographer 效能的影響，結果頗具啟發性：在雜亂的房間中，三具 2D 光達加上 IMU 與里程計的組合誤差最低；而在長走廊場景中，單具 3D Velodyne 光達加 IMU 則表現最佳。

這個結果與本書前面的討論完全呼應。雜亂房間中，多具 2D 光達從不同角度掃描，能涵蓋更多幾何特徵、減少遮蔽（第 8 章的高度盲區）；而長走廊的困境在於縱向缺乏約束

（第 13.6 節），3D 光達能看到天花板與地面的結構變化，正好補上這個缺口。

對大學專題與大學部實作的啟示是：不要盲目追求「最貴的感測器」，而要依場域特性選擇組態。這也是一個絕佳的專題題目——在校園不同場域（教室、走廊、中庭）實測不同感測器組合的建圖品質，用第 13.7 節的方法量化比較。這樣的研究既有教學價值，也有實用意義。

## 實作活動

### 實作 15-1　Cartographer 建圖與閉環觀察

目的：執行 2D Cartographer 建圖，親眼觀察子地圖的形成與閉環拉正的瞬間，並與第 14 章的 Gmapping 做對照實驗。這是第四篇最重要的一個實作。

**第一部分：模擬建圖與閉環觀察（全班可做）**

1. 在 TurtleBot3 Gazebo 模擬環境（第 28 章）中安裝並啟動 Cartographer 的 2D 設定。
2. 在 RViz 中顯示子地圖與軌跡，先在小範圍移動，觀察子地圖如何一塊塊形成、如何部分重疊。
3. 遙控機器人繞行模擬世界一個完整迴路，全程注意觀察：前半段地圖工整、後半段開始出現錯位。
4. 關鍵時刻：回到起點附近時，留意閉環是否觸發、地圖是否被拉正。錄下這個瞬間並在課堂上重播討論。
5. 若閉環未觸發，嘗試調整閉環分數門檻與取樣比例（表 15-1），再試一次。

**第二部分：對照實驗——Cartographer vs Gmapping（核心）**

6. 在同一個模擬世界、同一條行走路線下，分別用 Gmapping（第 14 章）與 Cartographer 各建一次圖。

7. 用第 13.7 節的方法評估兩張地圖：牆是否筆直？迴路是否閉合？有無重影？量測地圖上的關鍵尺寸與模擬設定值比較。
8. 把兩張地圖並排，標示出差異最明顯的區域，並解釋原因（提示：表 15-2）。
9. 觀察資源用量：比較兩者的 CPU 與記憶體使用情形，對照第 14 章公式五與 15.2 節的討論。

### 第三部分：實體平台建圖（有硬體的班級）

10. 準備 Lua 設定檔，依你的平台調整（有無 IMU、有無里程計、光達型號與安裝位置）。
11. 先在教室建圖驗證設定正確，再挑戰較大場域——理想的是能繞一個完整迴路的走廊或系館一層樓。
12. 緩慢行走，務必走完整個迴路回到起點，觀察閉環。
13. 匯出地圖並評估品質，與同一場域的 Gmapping 結果比較。

### 討論問題

1. 你觀察到閉環拉正的瞬間了嗎？拉正前後，地圖上哪些區域變化最大？為什麼是那些區域？
2. 對照實驗中，Cartographer 與 Gmapping 的地圖在哪些方面差異最明顯？這印證了表 15-2 的哪幾項？
3. 如果場域沒有迴路（例如只是一條走到底的走廊），Cartographer 的閉環機制還能發揮作用嗎？此時它相對 Gmapping 還有優勢嗎？
4. （大學部）15.3 節提到閉環取樣比例「太低錯過閉環、太高拖慢即時性」。請實際調整這個參數，記錄閉環偵測成功率與系統延遲的變化，找出你場域的合適值。

### 延伸挑戰（大學部）

其一：設計一個「錯誤閉環」實驗——在模擬世界中建造兩個外觀幾乎相同的房間，觀察

Cartographer 是否會誤判閉環；若誤判，觀察穩健核函數是否減輕了災難程度。這將深化你對 15.4 節穩健化的理解。其二：若學校有 3D 光達，嘗試 Cartographer 3D 模式，掃描一間實驗室或一段樓梯，產生三維點雲模型，體驗背包式建模的概念（15.6 節），並為第五篇（第 17、18 章）的 3D SLAM 預作準備。

## 延伸閱讀

1. W. Hess, D. Kohler, H. Rapp, and D. Andor, "Real-Time Loop Closure in 2D LIDAR SLAM," ICRA, 2016.（Cartographer 的原始論文，本章主要依據）
2. Cartographer 官方文件（google-cartographer.readthedocs.io）與 Cartographer ROS 文件.（含 Algorithm walkthrough for tuning 調參指南，實作必讀）
3. Google Open Source Blog, "Cartographer" 發布公告.（說明開發動機與室內測繪的應用背景）
4. J. Elseberg and A. Nüchter, "Improving Google's Cartographer 3D Mapping by Continuous-Time SLAM," ISPRS Archives, 2017.（3D 模式的改良研究，含背包式系統的實測）
5. slam_toolbox 開源專案（GitHub）.（ROS 2 環境中另一套基於圖優化的 2D SLAM，可與 Cartographer 對照）
6. Ceres Solver 官方文件（ceres-solver.org）.（Cartographer 使用的最佳化函式庫，呼應第 12 章）
7. Cyrill Stachniss, Graph-Based SLAM 系列課程影片（YouTube）.（圖優化 SLAM 的理論基礎，搭配本章閱讀效果佳）

## 小結

1. Cartographer 由 Google 開發、2016 年開源，核心特色是「即時閉環」；它分為兩個同時運行的子系統：局部 SLAM（前端，建立子地圖，局部一致但會漂移）與全域 SLAM（後端，在背景尋找閉環約束並最佳化位姿圖）。
2. 子地圖是由數十筆掃描組成的機率格小地圖；分子地圖的三個理由是限制誤差累積、提供最佳化的單元（節點數大減故能即時）、以及成為穩定的閉環比對對象。記憶體與場景大小成正比，故能處理整棟建築。
3. 系統中有兩種掃描匹配：局部匹配把新掃描插入當前子地圖（以 Ceres 求解非線性最小平方）；全域匹配則搜尋閉環，難度高得多，需限制取樣節點數以兼顧閉環成功率與即時性。
4. 分支定界法是即時閉環的關鍵：在多解析度預算格上，由低解析度算出分數上界，將不可能勝出的區塊整片剪除，使搜尋量大幅下降；找到約束後以稀疏位姿調整拉正全圖，並用穩健核函數防止錯誤閉環造成災難。
5. 2D 實作需備妥雷射、TF 與（建議）IMU;Cartographer 以 Lua 設定檔組態，參數較多，關鍵在子地圖掃描數、解析度、閉環取樣比例與分數門檻；務必走完整迴路以觀察閉環。
6. 3D 模式需 3D 光達與 IMU，子地圖為體素格，運算量大；背包式空間建模讓操作員邊走邊建立整棟建築的三維模型，應用於 BIM、古蹟保存、設施管理，是大學場域極具潛力的專題方向。
7. 與 Gmapping 相比：Cartographer 基於圖優化而非粒子濾波、以子地圖而非每粒子一張地圖、閉環為核心功能、記憶體與場景成正比、支援 3D 與 IMU，但參數較複雜。兩者正是第 11、12 章兩條理論路線的實體化身。

第四篇的兩大技術主角至此完備。下一章，我們把這些技術帶回最貼近生活的場景——掃地機器人。從隨機碰撞到 SLAM 規劃式清掃、從房間分割與禁區設定到覆蓋路徑規劃，並誠實面對家庭場景的真實挑戰：桌椅腳、鏡面、低矮障礙物與寵物。你會發現，本篇所有

理論都在你家客廳裡默默運作著。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

## 第 16 章

## 掃地機器人 SLAM

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

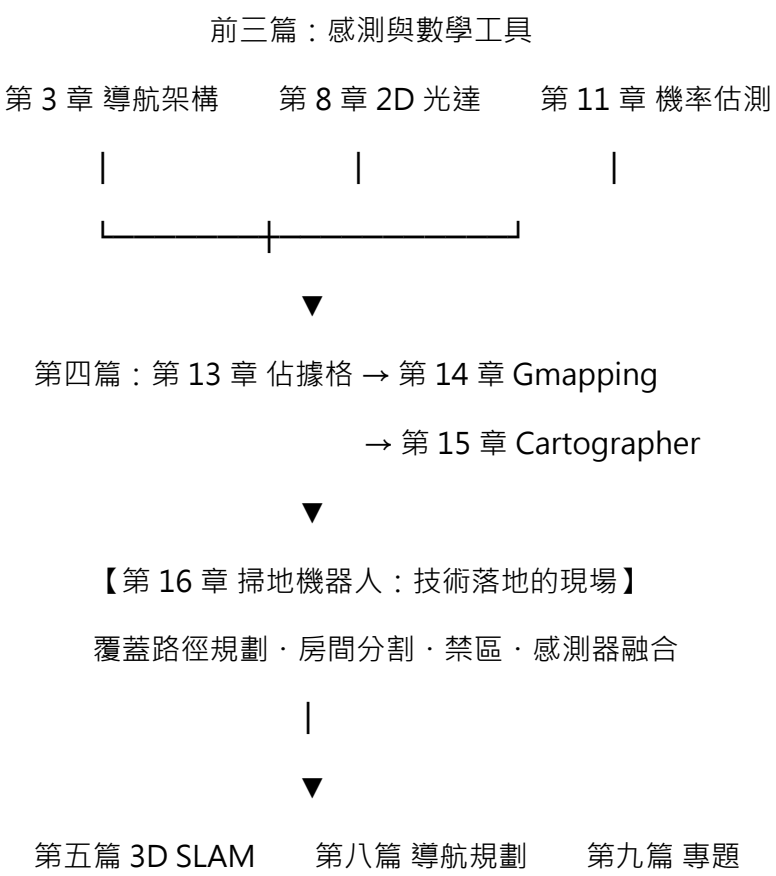
大學智慧機器人學程 / 高中 AI 機器人社團課程

## 本章在全書的位置

本章是第四篇的收尾，也是全書第一次把累積了十五章的技術，整組放進一台你買得到、摸得到的產品裡。前三章談的是演算法本身——佔據格怎麼建、粒子怎麼濾、位姿圖怎麼解；本章問的是另一個問題：當這些演算法必須在一台兩萬元、運算晶片只有手機等級、每天在陌生家庭裡獨自工作八小時的機器上運行時，會發生什麼事。答案往往不是「演算法不夠好」，而是「真實世界比論文裡的走廊複雜得多」。

所以本章的閱讀方式與前三章不同。前三章是由下而上建立工具，本章是由上而下拆解一個產品：看它用了哪些工具、在哪裡妥協、又用什麼方法補救。讀完之後回頭看第 8 章的角解析度、第 13 章的地圖病徵、第 14 章的打滑問題，會發現那些當時看起來抽象的限制，每一條都對應到你家掃地機器人的某個具體行為。

### 圖 16-0　第 16 章的連結地圖




## 學習目標

第四篇的理論走到這裡，終於要回到最貼近生活的場景——你家客廳。掃地機器人是 SLAM 技術最成功的大眾化案例，前面十五章的所有觀念，幾乎都能在這台小小的圓盤機器上找到對應。本章是第四篇的收尾，也是一堂「把理論帶回家」的課。學完本章之後，同學應能夠達成下列目標：

1. 說明掃地機器人為什麼需要 SLAM，以及沒有 SLAM 會有什麼後果。
2. 比較隨機碰撞式與 SLAM 規劃式清掃的差異，並理解覆蓋率與效率的量化意義。
3. 說明光達掃地機器人如何建立房間地圖，並連結第 13 至 15 章的技術。
4. 說明房間分割、禁區設定與覆蓋路徑規劃的原理。
5. 列舉掃地機器人的感測器組合，說明各自的角色與互補關係。
6. 分析家庭場景的真實挑戰：桌椅腳、鏡面、低矮障礙物與動態環境。
7. 從掃地路線理解覆蓋路徑規劃的基本策略與評估指標。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 覆蓋路徑規劃 | Coverage Path Planning | 規劃路徑走遍區域中每一個可通行位置。 |
| 弓字形路徑 | Boustrophedon Path | 像犁田般來回折返的清掃路線，是覆蓋規劃的基本形式。 |
| 隨機碰撞式 | Random Bounce | 早期掃地機的策略：撞到障礙就轉向，無地圖無規劃。 |
| 覆蓋率 | Coverage Rate | 實際清掃到的面積佔可清掃面積的比例。 |
| 重複率 | Redundancy Rate | 重複清掃同一位置的比例，越低效率越高。 |
| 房間分割 | Room Segmentation | 把整張地圖自動切分成有意義的房間區塊。 |
| 禁區 | No-go Zone | 使用者在地圖上劃定、機器人不得進入的區域。 |

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 虛擬牆 | Virtual Wall | 地圖上的虛擬界線，功能與禁區類似。 |
| 懸崖感測器 | Cliff Sensor | 偵測前方地面突然消失（樓梯）的紅外線感測器。 |
| 3D 飛時測距 | 3D ToF | 以立體方式量測前方障礙形狀的感測器。 |
| 結構光 | Structured Light | 投射光線圖案以判讀障礙物形狀與距離。 |
| 多樓層地圖 | Multi-floor Map | 記憶多個樓層地圖並自動辨識當前所在樓層。 |

## 16.1 掃地機器人為什麼需要 SLAM

這個問題乍看簡單：不就是把地板掃乾淨嗎？但仔細想想，「把整個家的地板都掃過一遍、而且不重複太多次」這件事，其實需要回答第 3 章的導航三問：我在哪裡？我要去哪裡？我該怎麼去？

### 沒有 SLAM 的三個困境

- 不知道哪裡掃過了：沒有地圖與定位，機器人無法記錄「我已經走過這一塊」。結果是有些地方掃了五次、有些地方一次都沒到。
- 不知道何時算結束：沒有地圖就不知道房子有多大、還剩多少沒掃。早期機型只能靠計時器——掃滿三十分鐘就收工，完全不管掃完了沒。
- 不知道怎麼回家：電量將盡時要回充電座，沒有地圖就只能亂走碰運氣，常常卡在半路沒電。

### SLAM 帶來的四項能力

1. 系統性清掃：有了地圖，才能規劃弓字形路線逐區覆蓋，大幅提升覆蓋率、降低重複率（16.2 節）。
2. 斷點續掃：電量不足時回充，充飽後能回到「上次掃到的地方」繼續——這需要準確的定位與記憶的地圖。
3. 使用者互動：App 上顯示房間地圖，讓使用者能指定「只掃廚房」、設定禁區、指哪掃哪。這是地圖帶來的產品價值。
4. 多次清掃的一致性：第二次清掃不必重新探索，直接載入既有地圖定位即可（第 14 章案例 14-3 的 AMCL）。

### 把前面的章節串起來

掃地機器人幾乎是本書前十五章的實體總複習：它用第 8 章的 2D 光達感知環境、第 13 章的佔據格地圖記錄空間、第 14 或 15 章的 SLAM 演算法同時定位與建圖、第 10 章的座標

轉換整合感測器、第 11 章的粒子濾波做重定位（綁架問題，第 3 章）。理解掃地機器人，就是理解整個第四篇。

## 16.2 隨機碰撞式掃地 vs SLAM 規劃式掃地

掃地機器人的演進史，是一部從「反應式」走向「審議式」的縮影（第 3 章的架構討論）。

### 隨機碰撞式：反應式架構的極致

2002 年 iRobot 推出第一代 Roomba 時，採用的是隨機碰撞策略：直線前進、撞到東西就轉一個隨機角度、繼續前進。搭配螺旋、沿牆等幾種簡單行為模式交替使用。

這個設計的靈感其實來自第 3 章提過的行為式架構——不建模、不規劃，讓感知直接驅動行為。它的優點是極其便宜、程式簡單、不會因為地圖錯誤而失效；缺點也很明顯：路線雜亂、覆蓋不均、耗時極長。

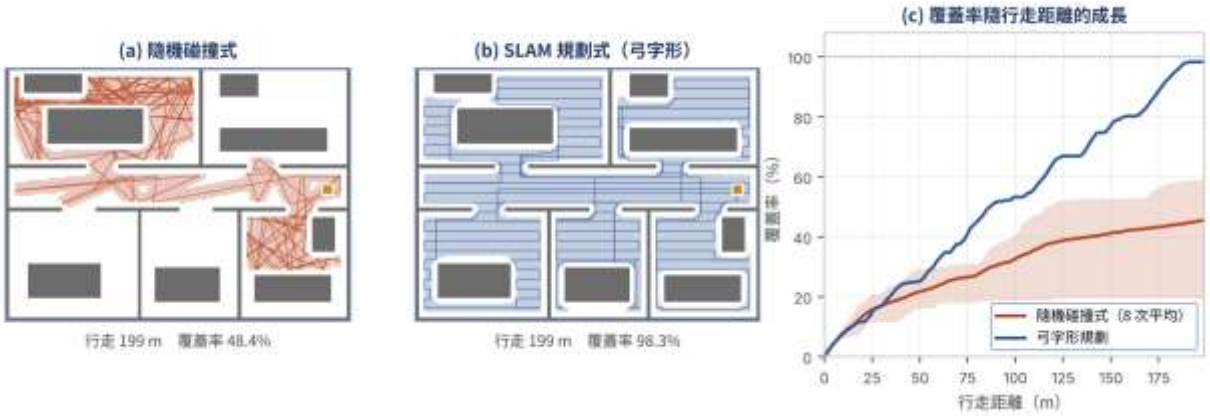
有趣的是，這個策略在數學上並非全無道理：只要時間夠久，隨機行走理論上能覆蓋整個連通區域。問題是「夠久」可能意味著一小時——而且永遠無法保證真的掃完了。

### SLAM 規劃式：審議式架構的勝利

2010 年 Neato 首度把 2D 光達裝上掃地機，讓機器人能建立佔據格地圖並規劃系統性路線。現代機型不再只是靠隨機碰撞轉向的旋轉刷具，而是執行裝置端 AI 來建立地圖、避開障礙、並適應真實的居家生活方式。

規劃式清掃的核心是弓字形路徑（Boustrophedon，原意為「像牛犁田般來回」）：機器人沿著一個方向直線清掃到底，轉 180 度移動一個刷寬，再反向掃回來，如此往復覆蓋整個區域。

**圖 16-1　同一間公寓、同樣走 199 公尺：兩種清掃策略的結果**




這張圖用一次對照實驗，把表 16-1 的文字差異變成可以量的數字。場景是一間 9.0×6.6 公尺的三房公寓，含主臥、次臥、書房、客廳、廚房與走道，並擺了床、沙發、書桌、流理台等家具；扣掉機器人半徑無法進入的牆邊與家具邊，可清掃面積為 27.5 平方公尺。兩種策略都從右側的充電座（金色方塊）出發，都走 199 公尺——也就是說，電池消耗是一樣的。

圖(a)是隨機碰撞式：直線前進、撞到就轉一個隨機角度。路徑呈現典型的亂麻狀，而結果是只掃到 48.4%。更值得注意的是它掃到了哪裡——主臥、走道與廚房有明顯覆蓋，但次臥、書房與客廳幾乎是空白。原因很具體：門口只有 85 公分寬，扣掉機器人半徑後可通行的縫隙不到 60 公分，一個隨機方向要恰好穿過這道縫，機率很低。隨機策略不是「慢慢來總會掃完」，而是會被困在它最先進入的那幾個房間裡。

圖(b)是 SLAM 規劃式：先用地圖把空間切成房間（下一節的圖 16-2），再在每個房間裡走弓字形，房間之間以最短路徑連接。同樣 199 公尺，覆蓋率 98.3%。路徑圖上一條條整齊的來回線，就是「弓字形」這個名字的由來；家具周圍那一圈沒掃到的白邊，則是機器人半徑造成的物理極限——這也解釋了為什麼實務上覆蓋率很難真正到 100%。

圖(c)把覆蓋率隨行走距離的成長畫出來，是本圖最有訊息量的一格。紅色實線是八次隨機實驗的平均，淡紅色帶是八次的最好與最壞範圍。兩條線在前 25 公尺幾乎重疊——這很合理，剛開始時到處都是沒掃過的地方，亂走也會有收穫。但從約 40 公尺之後開始分岔：弓字形持續線性爬升直到接近 100%，隨機式則快速趨於平緩，八次實驗的最終覆蓋率落在 19.5%

到 59.0% 之間。

那條淡紅色帶的寬度，比平均值本身更值得討論。它代表的是「同樣一台機器、同樣的程式、同樣的時間，今天掃完的結果和明天可能差三倍」。對一個每天都要用的家電來說，不可預測比效率低更致命——使用者無法判斷它到底掃完了沒。這正是 16.1 節「不知道何時算結束」那個困境的量化版本，也是 SLAM 為掃地機器人帶來的最大價值：不只是掃得更乾淨，而是掃得可以預期。

**關於這組模擬的說明**

這是教學用的簡化模擬：機器人視為一個直徑 26 公分的圓，有效刷寬 26 公分，不含邊刷的額外涵蓋。

隨機式的實作為「直線前進、碰撞後轉隨機角度」。真實的 Roomba 還會穿插螺旋與沿牆模式，覆蓋率會略高於本圖，但多房間住宅中被困住的結構性問題一樣存在。

弓字形規劃走了 199 公尺，理論最短覆蓋距離為 27.5 ÷ 0.26 ≈ 105.6 公尺，因此重複率約 1.88。這個數字在本章的評估指標裡屬於中等——轉行、繞家具與房間之間的移動都算在內。

同學可在實作 16-1 第三部分自行重現這張圖，並試著把重複率壓到 1.5 以下。

| 比較項目 | 隨機碰撞式 | SLAM 規劃式 |
|---|---|---|
| 是否建圖 | 否 | 是（佔據格地圖） |
| 清掃路線 | 隨機、雜亂 | 弓字形、系統性 |
| 覆蓋率 | 不保證，依賴時間 | 高，可接近完全覆蓋 |
| 重複率 | 高 | 低 |
| 所需時間 | 長 | 短 |
| 能否斷點續掃 | 否 | 是 |

| 比較項目 | 隨機碰撞式 | SLAM 規劃式 |
|---|---|---|
| 成本 | 極低 | 較高（需光達或相機） |
| 對地圖錯誤的容忍 | 不受影響 | 地圖錯則清掃錯 |

**表 16-1　兩種清掃策略的比較**

注意最後一列：規劃式並非全無缺點。若 SLAM 建圖失敗（第 13.7 節的地圖病徵），機器人會照著一張錯誤的地圖清掃，反而可能漏掉整個房間。這呼應第 13 章的結論：地圖品質直接決定系統成敗。

## 16.3　光達掃地機器人與 2D 光達建圖

第 1 章案例 1-1 提過，2017 年小米以親民價格推出光達掃地機器人，讓 SLAM 技術首次大量進入一般家庭。本節深入看它如何運作。

### 硬體配置

典型的光達掃地機器人在機頂有一個凸起的圓塔，裡面是一顆旋轉的 2D 光達（第 8 章），掃描高度約在機身頂部（離地 8 至 10 公分）。它每秒旋轉數圈，量測周圍數百個方向的距離。

值得注意的是成本控制：家用掃地機的光達多採低成本的三角測距原理，量測範圍約 6 至 8 公尺、精度數公分——遠不如工業型號，但對室內建圖已經足夠。這正是第 8 章「解析度要與應用需求匹配」的實例。

### 建圖流程：第 13 至 15 章的實踐

5. 首次進入陌生環境：機器人執行 SLAM，一邊移動一邊建立佔據格地圖並估計自己的位姿（第 13 章）。多數機型會先沿牆走一圈建立房屋輪廓，再處理內部。

6. 演算法選擇：早期機型多用粒子濾波類方法（第 14 章），近年因運算晶片進步，越來越多採用圖優化與閉環偵測（第 15 章），使大坪數住宅的地圖更一致。
7. 地圖儲存：建好的地圖存在機器內部與雲端，日後清掃直接載入。
8. 後續清掃：載入地圖後執行定位（AMCL 類方法，第 11 章），不需重新建圖，可直接規劃路線。

**近年趨勢：光達與視覺並用**

現代高階機型已不只倚賴光達。業界的做法是雷射光達負責建立地圖與追蹤機器人位置，AI 相機則負責視覺 SLAM 的部分——辨識地板上的物體並決定如何處理。這正是第 9 章「光達給幾何、相機給語意」的商業實踐。

這個組合直接呼應第 8 章表 8-2 的互補分析：光達不怕黑暗、幾何精確，但不知道「那是什麼」；相機能辨識物體，卻在昏暗與無紋理處失效。因此有廠商在相機旁加裝 LED 燈以維持在黑暗或低矮空間的視覺能力。

## 16.4 房間分割、禁區設定、路徑覆蓋

有了地圖只是第一步。要讓地圖真正好用，還需要三項功能，它們共同構成了使用者眼中的「智慧」。

**一、房間分割**

房間分割是把一整張佔據格地圖，自動切分成「客廳、臥室、廚房」等有意義的區塊。這對使用者體驗至關重要——沒有分割，使用者只能說「掃全部」；有了分割，才能說「只掃廚房」。

分割的基本思路是找出空間的「瓶頸」：門口與狹窄通道處，自由空間的寬度會突然變窄，這些位置就是天然的分界。常用方法包括依據距離變換找出房間的「核心區」再向外擴張、或是偵測門框的幾何特徵。

現代機型建好地圖後，能為房間貼標籤、排除禁區，甚至記住哪些區域需要更頻繁的清掃。這已經觸及第 3 章提過的語意地圖概念——機器人不只知道「這裡可通行」，還知道「這裡是廚房」。第 32 章會完整討論如何用 AI 做語意標註。

**圖 16-2 房間分割：用距離變換找出門口這個「瓶頸」**




房間分割聽起來像是一件需要人工智慧的事——畢竟「哪裡是臥室」是個語意問題。但這張圖顯示，只用兩步純幾何運算就能把它做得相當好。圖(a)是 SLAM 建好的佔據格地圖，此時它只是一堆黑白格子，沒有任何「房間」的概念。

圖(b)是關鍵的一步：距離變換。它替每一個自由格算出「這一格離最近的牆有多遠」，顏色越亮代表越空曠。看圖就能發現一個規律——房間中央是亮的（離牆遠），而門口是暗的一道窄縫（兩側都是牆，離牆都近）。紅色三角標出的五個位置就是這間公寓的門口，它們在距離變換圖上全部表現為「瓶頸」。白色等高線畫的是 0.45 公尺這個門檻：門口的距離值約 0.42 公尺，恰好被切斷，於是各房間的核心自然分離成獨立的區塊。

圖(c)是第二步：以這些核心為種子做分水嶺（watershed）擴張，讓每個核心沿著距離變換的梯度長回去，直到在門口處彼此相遇為止。結果是六個區塊，恰好對應主臥、次臥、書房、走道、客廳、廚房，連面積都一併算出來了。整個過程沒有用到任何機器學習，只用了距離變換與分水嶺——兩個在 OpenCV 裡各只需一行的運算。

這個結果對產品的意義很直接：有了分割，App 上才能顯示「客廳 3.8 平方公尺」並讓使用者點選「只掃廚房」；有了分割，覆蓋規劃才能逐房間進行（圖 16-1(b) 正是這樣做的），

而不是在整個平面上硬切長條。不過也要誠實說，純幾何方法有它的極限：開放式廚房與客廳之間沒有門，就沒有瓶頸可找，這類格局往往被切成一塊或切錯位置——這正是現代機型開始引入語意資訊的理由。

**距離變換門檻怎麼挑**

門檻的物理意義是「房間核心至少要離牆這麼遠」。它必須大於門口半寬、小於房間半寬。

本圖門寬 0.85 公尺（半寬約 0.42），房間最窄處半寬約 0.6 公尺以上，因此 0.45 公尺是安全的中間值。

門檻太低：門口沒被切斷，相鄰房間連成一塊（本章模擬在 0.40 以下就會發生）。

門檻太高：房間核心被家具切碎，一個房間被分成好幾塊，必須再靠最小面積過濾合併。

實務上門寬相當標準（台灣住宅多為 80～90 公分），所以這個門檻可以寫死；真正難處理的是開放式格局與無門的穿堂。

## 二、禁區與虛擬牆

禁區（No-go Zone）讓使用者在 App 地圖上劃出機器人不得進入的區域——寵物碗、小孩的積木區、易被纏繞的電線區。虛擬牆則是劃一條線禁止跨越。

技術上，這是在代價地圖（第 3 章）上人為加入「障礙」：規劃路徑時把禁區視為不可通行。這是一個聰明的設計——不需要改動 SLAM 或感測器，只需在規劃層加一層使用者定義的約束。同學可以想想：這其實就是把「使用者的知識」注入機器人系統的一種方式。

## 三、覆蓋路徑規劃

覆蓋路徑規劃（Coverage Path Planning,CPP）與一般路徑規劃有本質差異：

- 一般路徑規劃（第 3、29 章）：求「從 A 到 B 的最短路徑」，目標是抵達。

- 覆蓋路徑規劃：求「走遍每一個可通行位置的路徑」，目標是不遺漏。

掃地機器人是後者的典型應用。基本策略是把地圖分割成無障礙的長條區域，在每個區域內走弓字形，再依序連接各區域。機器人利用機載感測器與（若有的話）預先載入的樓層平面圖輪廓，規劃一條能抵達每個可進入房間、走廊與空間的覆蓋路徑。

### 評估指標

- 覆蓋率：實際掃到的面積 ÷ 可清掃面積。理想值接近 100%，但家具下方、角落往往難以完全覆蓋。
- 重複率：總行走距離 ÷ 理論最短覆蓋距離。越接近 1 越有效率。
- 清掃時間：完成全屋所需時間，是使用者最有感的指標。

這三個指標互相拉扯：提高覆蓋率往往增加重複率與時間。工程上的取捨，正是產品設計的核心。這也是第 31 章專題評分 Rubric 可以參考的量化思路。

## 16.5 掃地機器人的感測器：光達、IMU、碰撞、懸崖感測器

一台掃地機器人身上的感測器，幾乎是本書第二篇的濃縮版。本節逐一說明它們的角色與互補關係。

| 感測器 | 角色 | 對應章節 |
|---|---|---|
| 2D 光達 | 建圖與定位的主力，量測周圍距離 | 第 8 章 |
| IMU | 偵測轉向與傾斜，輔助位姿估計 | 第 7 章 |
| 輪速編碼器 | 提供里程計，SLAM 的運動預測來源 | 第 3、13 章 |
| 碰撞感測器 | 光達沒看到時的最後防線 | 本章 |
| 懸崖感測器 | 向下紅外線，防止從樓梯跌落 | 本章 |
| 紅外 / 超音波 | 近距離障礙偵測，補足光達盲區 | 第 8 章 |
| AI 相機 | 辨識物體與語意，避開電線與排泄物 | 第 9、32 章 |

| 感測器 | 角色 | 對應章節 |
|---|---|---|
| 3D ToF / 結構光 | 量測前方障礙的立體形狀 | 第 8、9 章 |

**表 16-2 掃地機器人的感測器組合**

### 為什麼需要這麼多感測器

答案是第 8 章案例 8-1 的「高度盲區」：2D 光達只掃描單一水平高度切面，看不到低於或高於它的東西。因此：

- 低矮障礙（電線、拖鞋、寵物玩具）：在光達掃描平面之下，完全看不到，必須靠前方的紅外線、3D ToF 或相機。
- 懸空障礙（懸掛的窗簾、低矮家具邊緣）：可能剛好在掃描平面上下，偵測不穩定。
- 樓梯與落差：光達水平掃描完全無法偵測「地面突然消失」，必須靠向下的懸崖感測器——這是掃地機器人不可或缺的安全裝置。
- 玻璃與鏡面：第 8 章的老問題，雷射穿透或反射，需靠碰撞感測器兜底。

**圖 16-3 高度剖面：2D 光達看不到的那些東西**

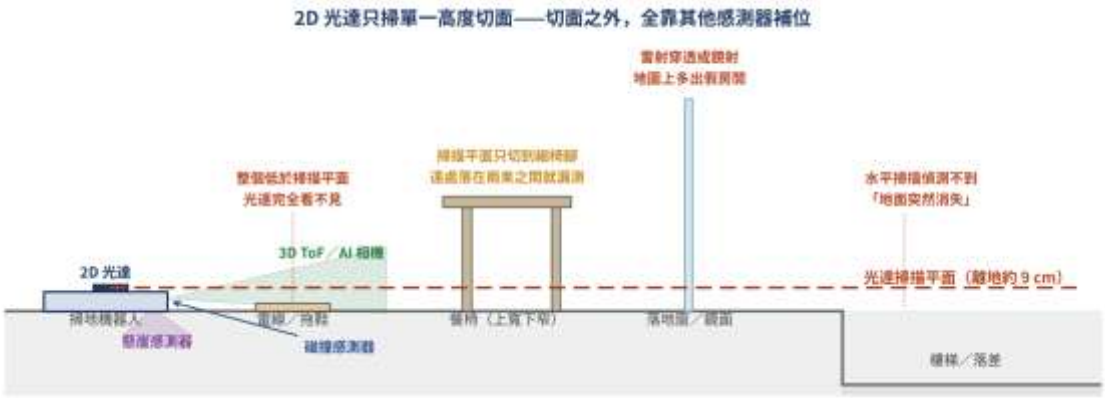



表 16-2 列了八種感測器，初學者的第一個疑問通常是：裝這麼多做什麼？這張側視剖面圖就是答案。紅色虛線是 2D 光達的掃描平面，離地約 9 公分——注意它是一個「平面」，不是一個視野。光達對這個平面上的東西瞭若指掌，對平面之外的一切完全無知。本書第 8 章稱之為高度盲區，而家庭正是高度盲區最致命的場所。

從左往右看四種典型的失效。電線與拖鞋高度只有三、四公分，整個落在掃描平面之下，光達不只是看不清楚，而是根本不知道它們存在——這就是掃地機器人纏住電線、拖著襪子跑遍全屋的物理原因。餐椅則是另一種失效：它上寬下窄，掃描平面只切到細細的椅腳，而第 8 章的角解析度公式告訴我們，距離夠遠時相鄰兩束雷射的間隔會超過椅腳寬度，整根椅腳就落在兩束之間被漏掉。案例 16-2「為什麼你家的掃地機器人總是撞到那張椅子」，講的就是這個幾何事實。

落地窗與鏡面是第 8 章反覆強調的光學難題：雷射要嘛直接穿透、要嘛被鏡面反射到別的方向，結果是地圖上出現一個根本不存在的房間，或是一道看不見的牆。樓梯與落差則更危險——水平掃描完全無法偵測「地面突然消失」，若沒有別的感測器，機器人會直接走下樓梯。

圖中三個彩色區塊就是補位的答案，而它們的擺放位置本身就在說明設計邏輯：3D ToF 與 AI 相機朝前上方張開一個錐狀視野，補的是掃描平面「以下、以上」的立體形狀；懸崖感測器朝正下方，補的是地面本身；碰撞感測器貼在機身前緣，是最後一道防線——當前面所有感測器都失效時，至少還有物理接觸。這三層由遠而近、由精確到粗略的配置，正是第二篇感測器融合思想最生活化的實例：沒有全能的感測器，只有恰當的分工。

**一個可以自己做的觀察**

把一本書平放在地上，看你家掃地機器人是否避開——書厚約 3 公分，低於掃描平面。

換成一個紙箱（高度超過 10 公分），通常會被乾淨地繞開——因為它進入了掃描平面。

這個對照實驗只需要兩樣家裡現成的東西，卻能讓同學親眼確認「掃描平面」是真的存在的一條線。

延伸觀察：把書換成細長的椅腳或立燈底座，從不同距離放置，測試第 8 章角解析

度的影響。

### 現代高階配置

近年高階機型的感測器組合越來越豐富。有機型在單一 AI 相機之外搭配 3D 結構光，構成能辨識大量常見障礙物類型的進階障礙偵測系統。不同感測器各司其職：LED 燈維持相機在黑暗或低矮空間的視力、3D 結構光讀取正前方障礙的形狀、3D ToF 則同時量測高度與寬度並能在強光下維持效能。

這個演進趨勢再次印證本書的核心信念：沒有全能的感測器，只有恰當的組合。掃地機器人正是感測器融合（第二篇）最貼近生活的教材。

## 16.6 家庭場景中的 SLAM 挑戰：桌椅、鏡面、低矮障礙物

實驗室的教室是 SLAM 的友善場景（第 13.6 節），但家庭是另一回事。本節誠實面對真實家庭的挑戰——這也是同學觀察自家掃地機器人時最能印證的部分。

### 一、細瘦的桌椅腳

第 8 章案例 8-1 詳細討論過這個問題：餐椅腳細瘦，在遠處可能整個落在兩束雷射之間而被漏測（第 8 章的角解析度公式）。結果是地圖上沒有這根椅腳，機器人一頭撞上去。

- 對策：近距離時角解析度足夠，加上碰撞感測器兜底；現代機型用 3D ToF 或相機在近距離補足偵測。

### 二、鏡面與落地窗

鏡子會讓光達「看到」鏡中的虛像，在地圖上畫出一個根本不存在的房間；落地窗則讓雷射直接穿透，機器人以為那裡可以通行。這是第 8 章反覆強調的光達弱點，在家庭中極為常見。

- 對策：使用者手動在 App 中設定禁區（16.4 節）；部分機型用超音波偵測玻璃（聲波會被玻璃反射）；建圖時暫時遮蔽鏡面。

### 三、低矮與動態障礙

地上的電線、襪子、寵物玩具、寵物排泄物，是掃地機器人最著名的災難來源。它們低於光達掃描平面，且位置每天都不同——地圖上不會有它們。

- 對策：這正是 AI 相機的價值所在。感測系統融合光達、相機、有時還有結構光感測器的資料，以偵測牆壁、家具腳、玩具甚至電線，而且這個過程是持續進行的。

### 四、動態環境：家會變

椅子被拉開、行李箱放在客廳、門開了又關——家庭環境每天都在變。這對 SLAM 是嚴峻考驗，因為地圖需要既「記得」永久結構（牆），又能「忘記」臨時物件（第 13.5 節的地圖可塑性）。

- 對策：對佔據機率設上下限讓地圖保持可塑性（第 13.5 節）；區分「永久地圖」（牆與家具）與「臨時障礙層」（當次清掃偵測到的物體）；部分機型會在多次清掃後統計哪些障礙是常駐的。

### 五、地面材質變化

磁磚換地毯時，輪子的摩擦力與滑動特性改變，里程計品質下降（第 14.6 節的打滑問題）；地毯還會讓機器人「陷入」而改變機身高度，影響光達的掃描平面。

- 對策：調高 SLAM 對里程計的雜訊假設、多倚賴掃描匹配；部分機型會偵測地毯並調整行為（如提高吸力、抬高拖布）。

| 家庭挑戰 | 根本原因 | 主要對策 |
| --- | --- | --- |
| 細瘦桌椅腳 | 角解析度不足，遠處漏測 | 近距離偵測、碰撞感測器、3D ToF |

| 家庭挑戰 | 根本原因 | 主要對策 |
|---|---|---|
| 鏡面與落地窗 | 雷射穿透或反射，產生虛像 | 使用者設禁區、超音波、暫時遮蔽 |
| 低矮動態障礙 | 在光達掃描平面之下 | AI 相機辨識、3D 結構光 |
| 環境每天在變 | 地圖需兼顧記憶與遺忘 | 機率設限、分層地圖、多次統計 |
| 地毯與地面變化 | 里程計品質下降、機身高度改變 | 調高里程計雜訊、倚賴掃描匹配 |

**表 16-3　家庭場景的 SLAM 挑戰與對策**

## 16.7　教學活動：從掃地路線理解 Coverage Path Planning

本節說明如何把覆蓋路徑規劃設計成一堂不需硬體、卻極有啟發性的課。這是本書認為最適合高中課堂的活動之一，大學部也能延伸為程式實作。

### 為什麼從掃地路線切入

覆蓋路徑規劃是一個「人人都有直覺、但一動手就發現不簡單」的問題。每個人都掃過地、拖過地，直覺上知道「要一排一排來、不要亂跑」——但要精確定義「怎麼排最有效率」、「遇到障礙怎麼繞」、「如何確保不遺漏」，就進入了演算法的領域。

### 紙上活動設計（高中適用）

9. 在方格紙上畫一個「房間」：外框是牆，內部放幾個障礙物（代表沙發、桌子），留一個門口。
10. 讓同學用筆規劃一條路線，要求「經過每一格至少一次、從門口出發回到門口」，並記錄總步數。
11. 比較全班的路線：誰的步數最少？最短路線與最長路線差多少？那些「多繞的步數」就是重複率的具體化。
12. 引入弓字形策略：示範如何把房間切成數個長條區域，每區走弓字形再連接，比較它與直覺路線的效率。

13. 加入變數：如果房間中央多一根柱子，弓字形路線要怎麼調整？如果門口在角落而非中間呢？

### 觀察家中掃地機器人（延伸作業）

14. 觀察並記錄自家（或影片中）掃地機器人的清掃路線：它是隨機式還是弓字形？它如何處理家具？
15. 拍下 App 上的地圖，標示出：哪些區域被分割成一個房間？有沒有明顯的地圖錯誤（第 13.7 節的病徵）？
16. 刻意製造挑戰：在地上放一雙拖鞋或一條（安全的）繩子，觀察機器人是否偵測到、如何反應。
17. 把它抱到另一個房間放下，觀察重定位過程（第 3 章綁架問題、第 11 章案例 11-2）。

### 大學部程式實作方向

- 用 Python 實作弓字形覆蓋規劃：輸入一張含障礙的佔據格地圖，輸出覆蓋路徑，並計算覆蓋率與重複率。
- 實作房間分割：用形態學運算或距離變換找出門口瓶頸，把地圖切成區塊。
- 在 ROS 模擬中，把第 14 或 15 章建好的地圖接上自製的覆蓋規劃器，讓虛擬機器人執行完整清掃。

至此，第四篇「2D SLAM 與掃地機器人」全部完成。我們從第 13 章的佔據格地圖出發，經過第 14 章 Gmapping 的粒子濾波、第 15 章 Cartographer 的圖優化與閉環，最後在本章看到這些技術如何在家庭中落地。同學應該能感受到：SLAM 不是遙不可及的尖端科技，它的核心版本正在你家客廳裡默默運轉。

下一篇，我們把維度從平面提升到立體。第五篇「3D 光達 SLAM 與自主巡檢」將介紹三維點雲、ICP 與 NDT 配準、LOAM 系列演算法，以及機器狗與巡檢機器人如何在複雜的

三維空間中定位建圖。難度會提升，但同學已經備妥了所有必要的基礎。

## 16.8 家用清掃機器人的下一步：2024–2026 年的四個轉向

本章前七節描述的是一套已經成熟的技術組合：2D 光達建圖、房間分割、弓字形覆蓋。這套組合在 2017 年之後基本定型，而真正的變化發生在它的邊界上——當產業發現「把地掃乾淨」這件事的技術瓶頸已經不在 SLAM，而在 SLAM 之外時，創新的方向就整個轉了彎。本節整理四個轉向。

### 一、從「繞開障礙」到「把障礙收走」

最戲劇性的變化發生在 2025 年初的 CES：Roborock 發表了 Saros Z70，在掃地機器人上裝了一支可折疊的五軸機械手臂 OmniGrip，能把地上 300 公克以下的小物件——襪子、小毛巾、衛生紙、拖鞋——夾起來放到指定位置，再回頭清掃那塊地板。

從本書的角度看，這件事的意義不在手臂本身，而在它對地圖的要求整個升級了。要夾起一隻襪子，機器人不只要知道「那裡有障礙」（佔據格做得到），還要知道「那是一隻襪子、它的三維形狀是這樣、可以夾、該放到哪裡」。這已經遠遠超出第 13 章佔據格的表達能力，進入語意地圖與物件層級表示的領域。換句話說，機械手臂逼出來的真正創新，是地圖格式的升級。

### 二、從幾何地圖到語意地圖

承接上一點：近年的研究主流，是在幾何地圖之上疊一層「這是什麼」的資訊。開放詞彙三維場景圖（open-vocabulary 3D scene graph）這一路的工作，把視覺語言模型的辨識能力接到三維地圖上，讓機器人能回應「去沙發旁邊那張桌子」這種用自然語言描述、而非座標指定的任務。

對掃地機器人而言，這條線的價值非常實際。16.4 節的房間分割只能靠幾何瓶頸切塊，遇到開放式廚房就失效；若地圖本身帶有物件標籤，「有流理台與冰箱的區域就是廚房」便成為

可用的判準。禁區也一樣——與其讓使用者在 App 上手動畫框，不如讓機器人自己認出寵物碗並繞開。本書第三篇的機率框架處理「那裡有沒有東西」，語意地圖處理的是「那是什麼」，兩者是疊加而非取代的關係。

## 三、覆蓋路徑規劃的標準化：從自家演算法到開源基礎設施

第三個轉向發生在軟體生態。覆蓋路徑規劃長年是各廠商的獨門配方，但 ROS 2 生態近年出現了通用的覆蓋規劃伺服器（opennav_coverage，建構於 Fields2Cover 之上），把本章談的策略整理成可設定的選項：路徑順序可選 BOUSTROPHEDON（弓字形）、SNAKE、SPIRAL（螺旋）；掃描角度可以指定，也可以用暴力搜尋自動找出最佳角度，而「最佳」的評分標準正好就是本章的三個指標——總長度、覆蓋率，或行數。

這份對應關係值得同學細看：16.4 節講的評估指標，在這套開源工具裡直接成為可選的目標函數；而延伸挑戰要同學比較的「弓字形 vs 螺旋式」，在這裡是一個設定參數。這說明本章的內容並不是產品觀察的雜談，而是一個有標準化工具支撐的工程領域。它原本是為農機（田間作業）設計的，但覆蓋問題的數學結構與室內清掃完全相同。

## 四、掃地機器人成為研究與教學的標準底盤

最後一個轉向與同學最有關。如案例 16-3 所述，掃地機器人底盤正逐漸成為低成本研究平台的首選：底盤成熟、馬達與里程計介面現成、體積適合室內、價格遠低於研究級機器人，而且頂部平坦容易加裝感測器支架。對於經費有限的大學專題與高中社團，這是一條把本書十六章理論真正跑起來的務實路徑。

本書的建議是：不必一開始就追求昂貴的設備。先用一台可程式化的掃地機底盤，加上一顆低成本 2D 光達，就能完整重現第 13 至 16 章的所有內容——建圖、閉環、房間分割、覆蓋規劃。把省下來的預算與時間，放在演算法與場域實驗上，那才是專題真正的價值所在。

**四個轉向的共同訊號**

硬體面（機械手臂）：清掃機器人開始「動手」，而不只是「繞開」。

表示面（語意地圖）：地圖從「哪裡有東西」升級為「那是什麼」，補上純幾何方法的盲點。

軟體面（開源覆蓋規劃）：覆蓋路徑規劃從各家配方變成可設定的標準元件。

教學面（研究底盤）：量產消費硬體回流成為低成本的研究與教學平台。

共同訊號：SLAM 本身已不是瓶頸。本章前七節的技術仍然是地基，但競爭已經移到地基之上——地圖裡還能放進什麼資訊，以及機器人拿這些資訊能做什麼。

### 承先啟後

第四篇到此完成。回頭看這四章的安排：第 13 章把掃描變成地圖，第 14 章用粒子濾波解開雞生蛋問題，第 15 章用圖優化讓歷史可以被修正，而第 16 章把這三件事放進一台真實產品裡，看它在 85 公分的門口、3 公分高的電線與一面鏡子面前如何應對。圖 16-1 到圖 16-3 的三組實驗與剖面，說的都是同一件事：演算法的優雅，最終要由真實世界的幾何來驗收。

從第 17 章起，我們把維度從平面提升到立體。第五篇「3D 光達 SLAM 與自主巡檢」將介紹三維點雲、ICP 與 NDT 配準、LOAM 系列演算法與巡檢應用。本章圖 16-3 那條孤零零的掃描平面，正是下一篇要解決的問題——當掃描不再只有一個高度，機器人終於能看見完整的世界。

## 公式或流程

### 一、覆蓋率與重複率

評估清掃效率的兩個核心指標：

$$\text{覆蓋率} = \text{實際清掃到的面積} \;/\; \text{可清掃總面積} \times 100\%$$

$$\text{重複率} = \text{實際行走距離} \;/\; \text{理論最短覆蓋距離}$$

理論最短覆蓋距離約等於「可清掃面積 ÷ 刷寬」——這是完美弓字形不重複走完全區的距離。重複率為 1 代表完美無重複，實務上因轉彎、繞障礙與返回充電，通常落在 1.2 至 2.0 之間；隨機碰撞式則可能高達 5 以上。

## 二、弓字形路徑的行數

在一個寬 W、長 L 的無障礙矩形區域中，以刷寬 w 走弓字形所需的行數與距離為：

$$行數\ n = \lceil W / w \rceil$$

$$總距離 \approx n \times L + (n - 1) \times w$$

第二項是每次轉行時橫向移動的距離。這條式子說明了為何刷寬越大效率越高——行數減少，轉彎次數也減少。這也是產品設計上「加寬刷頭」的動機。

## 三、掃地機器人的完整運作流程

**圖 16-4 掃地機器人一次清掃的完整流程**

① 從充電座出發，載入既有地圖（首次則進入 SLAM 建圖模式）

↓

② 定位：以 AMCL 類方法確認自己在地圖中的位置（第 11 章）

↓

③ 房間分割與任務規劃：決定清掃順序，排除禁區（16.4 節）

↓

④ 覆蓋路徑規劃：在當前房間內產生弓字形路線

↓

⑤ 執行清掃：沿路徑移動，同時

- 持續定位與地圖更新（SLAM）
- 即時避障（相機 / 3D ToF / 碰撞 / 懸崖感測器）
- 記錄已清掃區域

↓

⑥ 電量不足？→ 規劃返回充電座，充飽後回到中斷點續掃

↓

⑦ 全部房間完成 → 返回充電座，上傳地圖與清掃紀錄

## 四、房間分割的基本思路

**圖 16-5　房間分割的距離變換法**

① 對佔據格地圖執行距離變換：每個自由格記錄它到最近障礙的距離

↓

② 房間中央距離值大（空曠）、門口與走道距離值小（狹窄）

↓

③ 取高於門檻的區域作為各房間的「核心種子」

↓

④ 由各核心向外擴張，直到相遇為止 → 邊界即為房間分界

↓

⑤ 輸出分割後的房間區塊，供使用者命名與個別指定清掃

## 案例

### 案例 16-1　二十年演進：從 Roomba 到 Physical AI

掃地機器人的二十年，是一部機器人技術大眾化的縮影。2002 年第一代 Roomba 以隨機碰撞策略問世——沒有地圖、沒有定位，純粹是第 3 章「行為式架構」的商業化。2010 年 Neato 首度導入 2D 光達與 SLAM。2017 年小米以親民價格普及光達機型（第 1 章案例 1-1）。

而到了 2026 年，情況又有了質變。這一代的家用機器人被歸類為「實體 AI（Physical AI）」——它們不再只是靠隨機碰撞轉向的旋轉刷具，而是執行裝置端 AI，能建立地圖、避開障礙，並適應真實居家生活的樣態。它們用光達與相機 SLAM 掃描房間、建立牆面家具與常用動線的 2D 或 3D 地圖；地圖建好後能為房間貼標籤、排除禁區，甚至記住哪些區域需要更頻繁清掃。

這條演進線清楚顯示：SLAM 是基礎，但真實產品會不斷疊加感測器與 AI 來突破單一技術的限制。從第 13 章的佔據格，到本章的語意標籤與行為適應，同學看到的正是本書從第四篇通往第九篇（AI 語意導航）的完整脈絡。

## 案例 16-2　為什麼你家的掃地機器人總是撞到那張椅子

這是一個幾乎每位使用者都遇過、也最能印證本書理論的日常現象。機器人能靈巧繞開沙發與牆壁，卻總是一頭撞上餐桌的椅腳。

原因是三重的。其一是第 8 章的角解析度問題：細瘦的椅腳在數公尺外可能整個落在兩束雷射之間被漏測。其二是掃描平面高度：有些椅腳上寬下窄，光達掃到的高度剛好是最細的部分。其三是動態性：椅子每天被拉到不同位置，地圖上記錄的舊位置早已失效。

對策也對應三個層次：近距離重掃（角解析度隨距離改善）、加裝 3D ToF 或相機看到完整形狀、以及讓地圖保持可塑性以反映椅子的移動（第 13.5 節）。下次看到機器人撞椅腳，不妨想想這三個原因分別對應本書的哪一章——這是最貼身的 SLAM 複習。

## 案例 16-3　掃地機器人平台在研究與教學中的角色

有趣的是，掃地機器人不只是消費產品，也是研究與教學的實用平台。有研究團隊以可程式化的掃地機器人底盤作為無人地面載具，搭配低成本 360 度 2D 光達，在 ROS 框架下實作 SLAM 與覆蓋路徑規劃研究。

這類平台的優勢很實際：底盤成熟穩定、有現成的馬達與里程計介面、體積小適合室內、價格遠低於研究級機器人，而且結構上容易加裝額外的感測器支架。對大學的專題或高中的進階社團而言，這是一條可行的低成本路徑——用消費級底盤加上教學型光達與單板電腦，就能組出一台能跑第 14、15 章演算法的研究平台。

這個案例呼應第 8 章 8.7 節的教學平台討論，也給同學一個啟示：做機器人研究不一定要昂貴的設備，善用成熟的消費級硬體作為基礎，把心力放在演算法與應用創新上，往往是

更聰明的策略。

## 實作活動

### 實作 16-1　覆蓋路徑規劃：從紙上到程式

目的：透過紙上推演、實地觀察與程式實作三個層次，理解覆蓋路徑規劃的策略與評估指標，並把第四篇的所有理論連結到生活經驗。

#### 第一部分：紙上覆蓋規劃（全體，不需器材）

14. 在方格紙上畫一個 12×10 格的房間，內部放兩個 2×3 格的障礙物（沙發、桌子），在一側邊緣留一格作為充電座位置。
15. 每人獨立規劃一條路線，要求經過每一個自由格至少一次，並回到充電座。記錄總步數。
16. 全班統計：最少步數是多少？最多是多少？計算各自的重複率（總步數 ÷ 自由格數）。
17. 示範弓字形策略：把房間依障礙切成數個長條區，每區走弓字形再連接。比較它與直覺路線的步數差異。
18. 挑戰題：如果把兩個障礙物移到房間正中央並排，弓字形要如何調整？畫出你的解法。

#### 第二部分：觀察真實掃地機器人（家庭作業）

19. 觀察家中（或網路影片中）掃地機器人的清掃路線，判斷它是隨機式還是規劃式，並描述它處理家具的方式。
20. 截圖 App 上的房間地圖，標示：房間分割是否合理？有無第 13.7 節的地圖病徵（變形、斷裂、漂移）？
21. 低矮障礙測試：在地上放一雙拖鞋（安全物品），觀察機器人是繞開、推開還是卡住，並用 16.6 節的知識解釋。

22. 綁架測試：清掃中途把機器人移到另一個房間，計時它花多久重新定位成功（第 3 章綁架問題、第 11 章案例 11-2）。

**第三部分：程式實作（大學部）**

23. 用 Python 讀入一張佔據格地圖（可用第 13 章實作產生的，或自行定義），實作弓字形覆蓋路徑規劃：把地圖切成無障礙長條區，在每區產生來回路徑，再連接各區。

24. 計算並輸出覆蓋率與重複率，用 matplotlib 把路徑畫在地圖上視覺化。

25. 實作房間分割：依圖 16-5 的距離變換法，把地圖自動切分成房間，以不同顏色顯示。

26. 加入禁區功能：讓使用者指定一個矩形區域，規劃時自動排除，驗證路徑確實繞開。

27. （進階）在 ROS 模擬中，把第 14 或 15 章建好的地圖接上你的覆蓋規劃器，讓虛擬機器人實際執行完整清掃。

**討論問題**

5. 第一部分中，弓字形路線比直覺路線省了多少步數？這個差距在真實的一百坪住宅中，會轉換成多少清掃時間？

6. 觀察真實機器人時，你發現了哪些地圖病徵？依第 13.7 節判斷，它們最可能的成因是什麼？

7. 綁架測試花了多久重新定位？若機器人被放在一個與原位置外觀相似的房間（例如兩間格局相同的臥室），你預期會發生什麼？（提示：第 11 章的多峰分布）

8. 依表 16-3，如果要為「養寵物的家庭」設計一台掃地機器人，你會優先加強哪些感測器？為什麼？

**延伸挑戰（大學部）**

其一：比較兩種覆蓋策略——弓字形與螺旋式（從外往內繞）——在同一張地圖上的覆蓋率、重複率與轉彎次數，分析各自適合什麼形狀的房間。其二：實作「斷點續掃」功能—

—在路徑執行到一半時中斷，記錄已完成的區域，重新啟動後只清掃剩餘部分，並驗證覆蓋率仍達 100%。這個功能需要同時用到定位、地圖與任務狀態管理，是整合第四篇所有知識的好題目。

## 延伸閱讀


8. H. Choset, "Coverage for Robotics – A Survey of Recent Results," Annals of Mathematics and Artificial Intelligence, 2001.（覆蓋路徑規劃的經典綜述，含 Boustrophedon 分解法）
9. E. Galceran and M. Carreras, "A Survey on Coverage Path Planning for Robotics," Robotics and Autonomous Systems, 2013.（覆蓋規劃的完整回顧，適合專題參考）
10. R. Bormann et al., "Room Segmentation: Survey, Implementation, and Analysis," ICRA, 2016.（房間分割方法的比較研究，含開源實作）
11. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005.（佔據格與蒙地卡羅定位的理論基礎，呼應本章的建圖與重定位）
12. R. Chuchro, "Simulating Coverage Path Planning with Roomba," arXiv, 2022.（以強化學習探索覆蓋規劃，並與掃地機器人內建演算法比較）
13. ROS Navigation2 官方文件之 Coverage Server 說明（docs.nav2.org）.（ROS 2 生態中的覆蓋路徑規劃工具）
14. OpenCV 官方文件之 Distance Transform 與形態學運算教學.（房間分割實作的技術基礎）


## 小結

8. 掃地機器人需要 SLAM 才能回答「哪裡掃過了、何時算結束、怎麼回家」；SLAM 帶來系統性清掃、斷點續掃、使用者互動與多次清掃一致性四項能力，是本書前十五章的實體總複習。
9. 隨機碰撞式是反應式架構的商業化（2002 年 Roomba），路線雜亂、覆蓋不保證；SLAM 規劃式建圖後走弓字形路徑，覆蓋率高、重複率低，但地圖錯誤會直接導致清掃錯誤。
10. 光達掃地機以低成本 2D 光達建佔據格地圖，首次清掃執行 SLAM、後續清掃載入地圖定位；現代高階機型光達與 AI 相機並用——光達建圖定位、相機辨識地板物體，正是「光達給幾何、相機給語意」的商業實踐。
11. 房間分割靠找出門口等空間瓶頸把地圖切塊，已觸及語意地圖概念；禁區是在代價地圖上注入使用者知識；覆蓋路徑規劃求「走遍每一格」而非「抵達某點」，以覆蓋率、重複率、清掃時間三指標評估。
12. 感測器組合包含 2D 光達、IMU、輪速編碼器、碰撞、懸崖、紅外 / 超音波、AI 相機與 3D ToF；如此多元的原因是 2D 光達的高度盲區——低矮障礙、樓梯落差、玻璃鏡面都需要其他感測器補足。
13. 家庭五大挑戰：細瘦桌椅腳（角解析度不足）、鏡面與落地窗（雷射穿透反射）、低矮動態障礙（在掃描平面之下）、環境每天在變（地圖需兼顧記憶與遺忘）、地面材質變化（里程計品質下降）。
14. 覆蓋路徑規劃是「人人有直覺、一動手就不簡單」的好教材，可從紙上規劃、觀察真實機器人到程式實作分三層次教學，是連結理論與生活的最佳橋樑。

第四篇「2D SLAM 與掃地機器人」到此完成。我們從佔據格地圖出發，經過 Gmapping 的粒子濾波與 Cartographer 的圖優化閉環，最後看到這些技術如何在家庭中落地——SLAM 不是遙不可及的尖端科技，它正在你家客廳裡默默運轉。下一篇，我們把維度從平面提升到立體：第五篇「3D 光達 SLAM 與自主巡檢」將介紹三維點雲處理、ICP 與

NDT 配準、LOAM 系列演算法，以及機器狗與巡檢機器人如何在複雜的三維空間中定位建圖。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第五篇 3D 光達 SLAM 與自主巡檢

## 第 17 章

## 3D 點雲與光達建圖

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

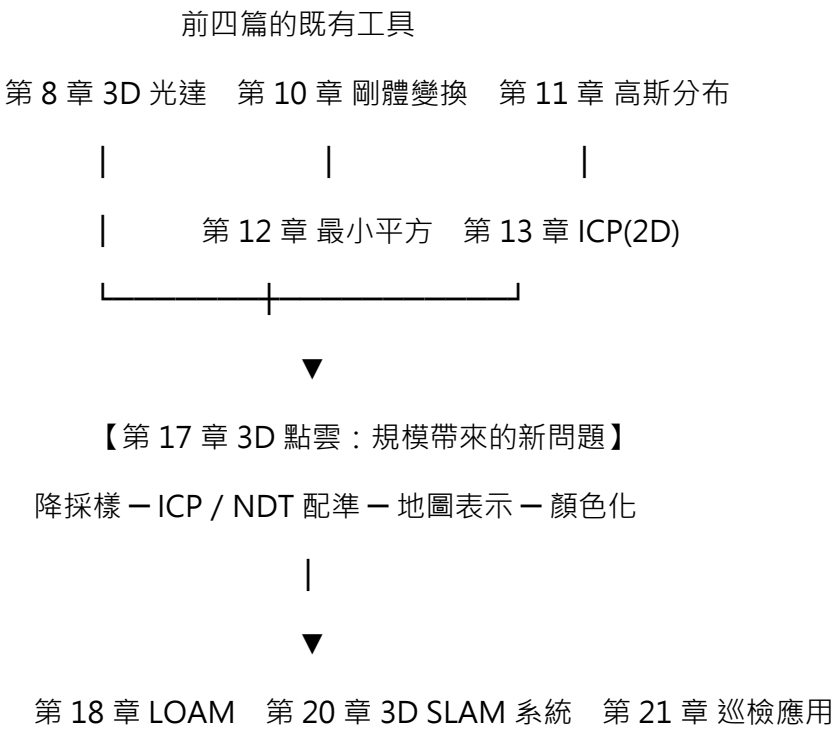
大學智慧機器人學程 / 高中 AI 機器人社團課程

## 本章在全書的位置

第五篇從這一章開始，而這一章要做的事，是把前四篇累積的一切從平面抬進立體。這個轉換聽起來只是多了一個 z 座標，實際上改變的東西比想像中多：地圖不再是一張可以印出來的圖片，而是數千萬個三維點；配準不再是三個自由度，而是六個；資料量從幾 MB 跳到幾 GB。本章的四項工具——降採樣、ICP、NDT、地圖表示——就是為了應付這個量級跳躍而存在的。

值得注意的是，本章幾乎沒有全新的數學。ICP 是第 13.3 節的三維版，NDT 的核心是第 11 章的多維高斯分布，配準的最佳化是第 12 章的最小平方，座標變換是第 10 章的剛體運動。真正新的只有「規模」——以及規模帶來的工程問題。這也是為什麼本章花了整整一節談降採樣：在三維世界裡，「如何把資料變少而不失真」本身就是一門學問。

### 圖 17-0　第 17 章的連結地圖




## 學習目標

第四篇的 2D SLAM 讓機器人在平面上定位建圖，但真實世界是立體的：樓梯有高度、天花

板有管線、機器狗要跨越障礙、無人機要在三維空間穿梭。本章把維度從平面提升到立體，介紹三維點雲的處理與配準技術，為第五篇的 3D SLAM 與自主巡檢揭開序幕。學完本章之後，同學應能夠達成下列目標：

8. 說明 3D 點雲的資料結構、來源與特性，並理解它與 2D 佔據格地圖的根本差異。
9. 說明體素降採樣的原理與必要性，並能估算降採樣的效果。
10. 說明 ICP 點雲配準的流程、變體與限制。
11. 說明 NDT 的核心思想，並比較它與 ICP 的差異與適用場合。
12. 說明 3D 地圖的表示方式與室內空間掃描的流程。
13. 說明點雲顏色化的原理與彩色三維視覺建模的應用。
14. 舉例說明 3D 空間建模在工廠、校園與展場的實際應用價值。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 點雲 | Point Cloud | 由大量三維點組成的空間資料，每點含 x、y、z 座標。 |
| 體素 | Voxel | 三維空間中的立方格，相當於 2D 的像素在立體中的對應。 |
| 體素降採樣 | Voxel Grid Downsampling | 把空間切成體素，每格只保留一個代表點以減量。 |
| 點雲配準 | Point Cloud Registration | 求兩片點雲之間的旋轉與平移，使其對齊。 |
| 迭代最近點 | ICP | 反覆找對應點並求最佳變換的經典配準演算法。 |
| 點對面 ICP | Point-to-Plane ICP | ICP 變體，以點到平面的距離為誤差，收斂較快。 |

| 中文名詞 | 英文名詞 | 概念說明 |
| --- | --- | --- |
| 常態分布轉換 | NDT | 把點雲轉為體素內的高斯分布再匹配的配準方法。 |
| 法向量 | Normal Vector | 描述點所在表面朝向的向量，用於點對面配準與著色。 |
| KD 樹 | KD-Tree | 加速最近鄰搜尋的空間資料結構，ICP 的效率關鍵。 |
| 八叉樹 | Octree | 以遞迴細分表示三維空間的樹狀結構，用於壓縮地圖。 |
| 點雲顏色化 | Colorization | 把相機影像的顏色投影到點雲上，產生彩色模型。 |
| 數位分身 | Digital Twin | 真實空間或設備的高精度三維數位模型。 |

# 17.1　3D 點雲資料

第 8 章介紹過點雲是 3D 光達的輸出格式，本節深入它的結構與特性，這是整個第五篇的資料基礎。

### 點雲是什麼

點雲（Point Cloud）是由大量三維點構成的集合。最基本的形式中，每個點只有三個數字——x、y、z 座標，代表「空間中的這個位置有一個表面」。把數十萬個這樣的點畫出來，物體與環境的立體形狀就浮現了。

每個點還可以附帶額外欄位：

- 反射強度（Intensity）：雷射回波的強弱，反映表面材質與顏色深淺。金屬與反光標誌強度高、黑色物體低。
- 顏色（RGB）：若融合了相機影像，每點可帶顏色，形成彩色點雲（17.6 節）。
- 線號（Ring）：多線光達中，該點來自第幾條雷射線。這個資訊在第 18 章的 LOAM 特徵提取中非常重要。
- 時間戳記：該點被量測的精確時刻，用於運動補償（第 18 章）。

### 點雲的三大特性

- 無序性：點雲是一堆點的集合，沒有固定的排列順序。這與影像的規則像素格不同——影像可以直接套用卷積，點雲不行。這也是深度學習處理點雲時的一大挑戰。
- 稀疏且不均勻：近處的點密集、遠處的點稀疏（第 8 章的角解析度公式）；而且只有「表面」有點，物體內部與沒掃到的地方都是空的。
- 資料量龐大：一台 64 線光達每秒可產生上百萬個點。一趟十分鐘的掃描就是數億個點——這是 17.2 節必須降採樣的原因。

### 與 2D 佔據格地圖的根本差異

| 特性 | 2D 佔據格地圖（第 13 章） | 3D 點雲 |
|---|---|---|
| 資料結構 | 規則格網，每格一個機率值 | 無序的點集合 |
| 空間表示 | 佔據 / 自由 / 未知三態 | 只有「表面在這裡」 |
| 未知區域 | 明確標示（灰色） | 無法區分「沒東西」與「沒掃到」 |
| 記憶體 | 與面積成正比 | 與點數成正比，通常大得多 |
| 直接用於規劃 | 可以 | 不方便，需轉換 |
| 細節保真度 | 受解析度限制 | 極高，保留原始量測 |

**表 17-1　2D 佔據格地圖與 3D 點雲的比較**

請特別注意「未知區域」這一列。第 13 章強調過佔據格地圖能區分「確定空」與「未知」，這對路徑規劃至關重要；但純點雲沒有這個能力——沒有點的地方，可能是真的空，也可能只是雷射沒掃到。這正是實務上常把點雲轉換成 3D 佔據格（如八叉樹）的原因（17.5 節）。

## 17.2　Voxel Grid 降採樣

面對動輒數千萬點的資料，第一件要做的事幾乎總是降採樣（Downsampling）——減少點數，同時盡量保留形狀。體素降採樣是最常用、也最直觀的方法。

### 原理：一格只留一個代表

體素（Voxel）是三維版的像素——把空間切成邊長固定的小立方格。體素降採樣的做法極其簡單：

18. 把整個點雲所在的空間，切成邊長為 d 的體素格網。
19. 把每個點分配到它所在的體素。
20. 每個體素只輸出一個代表點——通常取該體素內所有點的重心（平均座標）。
21. 空的體素直接忽略。

結果是：原本密集區域的大量點被合併成少數幾點，稀疏區域則幾乎不受影響。點雲整體變得均勻而輕量，形狀卻大致保留。

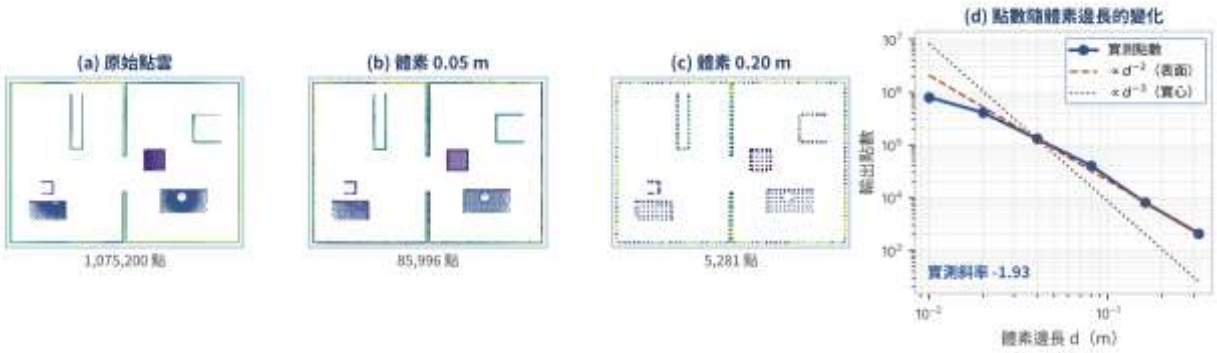
### 取重心還是取中心

這裡有一個看似細微、實則重要的實作差異。取體素內所有點的重心（質心），能較忠實地反映真實表面位置；若圖快而直接取體素中心，速度較快但精度較差。研究比較顯示，精確的體素降採樣結果幾乎與標準實作一致，而近似版本可能產生虛假的點，最多可能多出兩倍的點數。對精度要求高的建圖，建議使用取重心的精確版本。

### 降採樣的效果與代價

- 效果：體素邊長 d 越大，保留的點越少。點數的減少大致與 d 的立方成反比——邊長加倍，點數約降為八分之一。這是三維空間的必然結果。
- 代價：細節流失。若體素邊長大於物體特徵尺寸（例如 20 公分的體素配上 5 公分的管線），該特徵就會消失。
- 取捨：d 應略小於「你關心的最小特徵尺寸」。室內建圖常用 5 至 10 公分，大範圍戶外可用 20 至 50 公分。

**圖 17-1　體素降採樣：看得見的取捨與一條可以驗算的定律**




這張圖來自一次真實的模擬掃描：一間 10×7×2.8 公尺的室內空間，由 64 線光達從七個位置掃描並合併，共 1,075,200 個點。圖(a) 到 (c) 是俯視圖（為了看清楚結構，顯示時濾掉了地板與天花板），顏色代表高度。可以看到牆面、中央隔牆與門口、以及書櫃、工作檯、機櫃等家具的輪廓。

比較三張圖就能理解降採樣的取捨。體素邊長 0.05 公尺時（圖 b），點數從一百零七萬降到八萬六，減量超過九成，但牆線、家具輪廓與門口位置全部保留——這就是 17.2 節所說「點雲整體變得均勻而輕量，形狀卻大致保留」。到了 0.20 公尺（圖 c），點數只剩五千兩百，牆開始變成一串斷續的點，家具退化成幾個色塊，門口的位置已經難以辨認。本章天花板上那三根直徑 18 公分的管線，在這個尺度下也接近消失。

圖(d) 把這件事變成一條可以驗算的定律。橫軸是體素邊長、縱軸是輸出點數，兩軸都取對數。實測點落在一條直線上，斜率 −1.93——非常接近公式一所預測的 −2，而明顯偏離實心體積的 −3（圖中點線）。這印證了 17.2 節那個容易被忽略的重點：光達掃到的是物體「表面」而非內部，所以點雲本質上是一個嵌在三維空間裡的二維曲面，點數隨邊長的平方而非立方遞減。實務上的速算法就是：體素邊長加倍，點數大約降為四分之一。

唯一偏離直線的是最左邊 0.01 公尺那一點。原因不難理解：當體素比原始點的平均間距還小時，大多數體素裡只有一個點，降採樣幾乎沒有作用，點數被原始取樣密度卡住。這給了一個實用的判斷準則——如果你把體素調小卻發現點數不再明顯增加，代表已經觸及感測器本身的解析度下限，再小只是浪費計算。

**體素邊長怎麼選**

準則：d 應略小於「你關心的最小特徵尺寸」。要看到 5 公分的管線，體素就不能用 10 公分。

室內建圖常用 0.05 ~ 0.10 公尺；大範圍戶外或車載可用 0.20 ~ 0.50 公尺。

配準用的點雲可以比展示用的更稀疏——本章圖 17-3 的配準實驗就是在 0.06 公尺下進行的。

檢查方法：把體素逐步調小，記錄點數。當點數不再隨之上升，就是感測器解析度的極限。

KISS-ICP 等近年系統有一個值得注意的細節：體素內保留「一個原始點」而非重

心，以避免重心計算引入的離散化誤差（見 17.8 節）。

### 其他降採樣策略

除了體素法，還有幾種常見策略，各有適用場合：

- 隨機降採樣：隨機丟棄一定比例的點。最快，但可能造成分布不均。
- 均勻降採樣：每隔固定間隔取一點，簡單但不考慮密度。
- 距離門檻法：確保輸出點之間至少相距某個距離。
- 最遠點取樣：反覆挑選離已選點最遠的點，分布最均勻但計算較慢，常用於深度學習前處理。
- 保邊降採樣：近年的改良方向，依點雲特徵動態調整降採樣密度，並特意保留邊緣資訊，避免關鍵特徵流失。

本書實作以體素降採樣為主——它在效果、速度與實作難度之間取得最好的平衡，也是 PCL 與 Open3D 等主流函式庫的預設選擇。

## 17.3 ICP 點雲配準

點雲配準（Registration）是 3D SLAM 的核心運算：給定兩片部分重疊的點雲，求出把其中一片對齊到另一片所需的旋轉與平移。這正是第 13.3 節掃描匹配的三維版本，而 ICP 依然是最經典的方法。

### ICP 的三步驟循環

迭代最近點（Iterative Closest Point,ICP）的流程與第 13 章介紹的 2D 版本完全相同，只是在三維空間中進行：

22. 找對應：對來源點雲中的每個點，在目標點雲中找最近的點，配成一對。

23. 求變換：計算一個旋轉 R 與平移 t（第 10 章的剛體變換），使所有配對點之間的距離平方和最小。
24. 更新並迭代：把變換套用到來源點雲，回到步驟 1 重新找對應，反覆進行直到變換量小於門檻（收斂）。

### 效率關鍵：KD 樹

步驟 1 的「找最近點」是 ICP 最耗時的部分。若用暴力法，每個點都要與目標點雲的所有點比較，對數十萬點的點雲完全不可行。

解法是使用 KD 樹（KD-Tree）——一種把空間遞迴切分的資料結構，能把最近鄰搜尋從線性時間降到對數時間。實務上，建立 KD 樹與搜尋的效率，往往決定了整個 ICP 的速度。這也是為什麼現代配準函式庫會特別強調平行化的 KD 樹實作。

### ICP 的重要變體

- 點對點 ICP(Point-to-Point)：最原始的形式，誤差是兩點之間的直線距離。實作簡單，但在平面滑動時收斂慢。
- 點對面 ICP(Point-to-Plane)：誤差改為「來源點到目標點所在平面的垂直距離」。這個改動讓點能沿著平面自由滑動而不受懲罰，大幅加快收斂——這正是實務上最常用的版本。代價是需要先估計每個點的法向量。
- 廣義 ICP(GICP)：同時考慮來源與目標的局部平面結構，穩健度更高。
- 體素化加速版本：先降採樣再配準，或以體素結構加速對應搜尋，在保持精度的同時提升速度。

### ICP 的三個限制

- 需要好的初始猜測：ICP 是局部最佳化，若兩片點雲的初始位置差太遠，會收斂到錯誤的局部最佳解。這是第 13.3 節就強調過的老問題——實務上靠里程計或 IMU 提供初始值。

- 怕結構退化：在長廊、隧道、平坦大面等缺乏幾何約束的場景，點雲可以沿某個方向自由滑動而誤差不變，ICP 無法確定該方向的位移。這是第 13.6 節走廊困境的三維版本。
- 對離群點敏感：動態物體（走動的人）、雜訊點會拉扯配準結果。實務上需搭配離群點剔除或穩健核函數（第 12、15 章）。

## 17.4　NDT Normal Distributions Transform

ICP 有一個根本的效率瓶頸：每次迭代都要找最近點對應。常態分布轉換（NDT）提出了一個完全不同的思路——不找對應點，而是把點雲轉換成連續的機率分布。

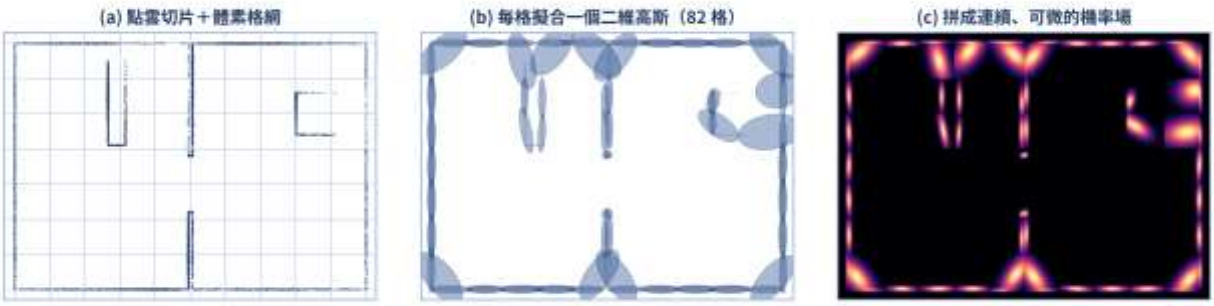
### 核心思想：把點雲變成機率場

NDT 的做法是：

25. 把目標點雲所在空間切成體素（格子）。
26. 對每個體素，計算內部所有點的均值與共變異數——也就是用一個三維高斯分布來描述「這一格的點大致長什麼樣」（第 11 章的高斯分布）。
27. 於是整片點雲變成一個由許多高斯分布拼成的連續機率場：空間中任一位置，都能算出「這裡有表面的機率密度」。
28. 配準時，求一個變換使來源點雲的所有點，落在這個機率場中的總機率最大。

這個轉換帶來一個關鍵優勢：目標函數變成平滑可微的連續函數，可以直接用牛頓法等最佳化方法求解（第 12 章），而且完全不需要在迭代中反覆搜尋最近點。

**圖 17-2　NDT 的核心：把離散的點變成連續的機率場**




「把點雲轉換成機率場」這句話聽起來抽象，但畫出來其實非常直觀。為了看得清楚，這張圖取前一張那間室內空間在高度 1.0 公尺處的一片水平切片，把問題降到二維來示範；三維的做法完全相同，只是高斯分布從橢圓變成橢球。

圖(a) 是切片後的點雲，疊上邊長 1.0 公尺的體素格網。此時資料仍然是離散的點，要問「空間中某一個位置有沒有表面」只能去搜尋附近有沒有點——這正是 ICP 每次迭代都要做、也最耗時的事。圖(b) 則對每一格內的點計算均值與共變異數，用一個二維高斯分布來描述它。請注意橢圓的形狀本身就帶著資訊：落在牆上的格子，橢圓被拉成細長的條狀，長軸方向正是牆面的走向；落在轉角的格子，橢圓則比較接近圓形。換句話說，共變異數矩陣自動記錄了「這一格的表面朝哪個方向延伸」。

圖(c) 把這 82 個高斯分布拼起來，就成了一個連續的機率場。現在，空間中任意一點都能直接代入公式算出「這裡有表面的機率密度」，不需要搜尋、不需要 KD 樹。更關鍵的是，這個函數是平滑可微的，可以直接丟給第 12 章的牛頓法去求極值。ICP 的「找對應—求變換」兩步循環，在 NDT 裡被壓縮成單純的一個最佳化問題。

不過這張圖也誠實暴露了 NDT 的一個弱點：仔細看圖(c) 會發現機率場在體素邊界處是斷掉的，同一面牆跨越兩格時，亮帶會出現接縫。這是因為每一格各算各的高斯，格與格之間並不連續。實際的 NDT 實作會用重疊體素或插值來緩和這個問題，而這也正是下一節要談的「體素解析度」之所以是 NDT 最敏感參數的原因。

**讀懂橢圓：共變異數的幾何意義**

橢圓的長軸方向 = 該格點分布變異最大的方向，也就是表面延伸的方向。

橢圓越扁 = 這一格的點越集中在一個平面上，法向量方向的約束越強。

接近圓形 = 點分布沒有明顯方向性（轉角、雜亂區域或點數太少）。

本書實作對共變異數設了 0.05 公尺的下限：真實表面有量測雜訊，不該被描述成零厚度。少了這道正規化，高斯會退化成一條線，最佳化會變得極不穩定。

每格至少要有五個點才建立高斯——點太少時共變異數估計不可靠，寧可留白。

## NDT 與 ICP 的比較

| 比較項目 | ICP | NDT |
|---|---|---|
| 核心運算 | 找最近點對應 | 體素內的高斯分布 |
| 是否需搜尋對應 | 需要（KD 樹加速） | 不需要 |
| 大量點雲速度 | 較慢 | 較快 |
| 精度 | 通常較高 | 略低於 ICP |
| 對初始值敏感度 | 較敏感 | 較不敏感，收斂範圍較大 |
| 關鍵參數 | 對應距離門檻 | 體素解析度 |
| 適合場景 | 精細配準、點數較少 | 大範圍、大量點雲、初值較差 |

**表 17-2　ICP 與 NDT 的比較**

研究比較指出，NDT 在配準過程中不使用對應點的特徵來計算，對於大量點雲資料，配準速度比 ICP 快，但配準精度不如 ICP。這個結論相當一致地出現在多項實驗中。

**圖 17-3 ICP 與 NDT 的正面對決，以及長廊裡的結構退化**

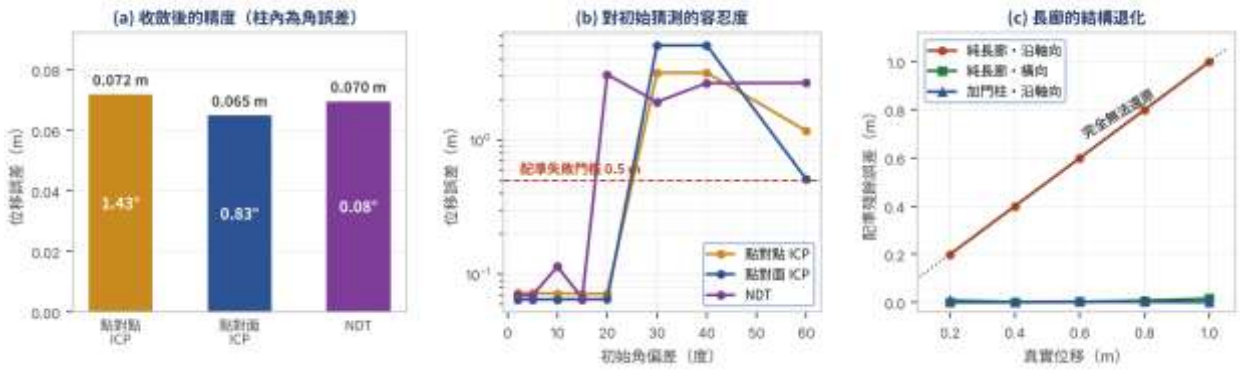



表 17-2 用文字比較了兩種方法，這張圖把它變成實測。實驗設定是真實的 SLAM 情境：光達在同一個室內空間的兩個位置各掃描一次（相距約 3.9 公尺、轉過 34 度），兩片點雲只有部分重疊，再以 0.06 公尺體素降採樣，各約兩萬三千點。任務是把第二片對齊回第一片，而且刻意給它一個不準的初始猜測。

圖(a) 是收斂後的精度。三種方法的位移誤差都落在 6.5 到 7.2 公分之間，差距不大——這個數值其實是兩個視點看到的表面取樣不同所造成的下限，而非演算法的極限。真正拉開差距的是角誤差（柱內數字）：點對點 ICP 1.43 度、點對面 ICP 0.83 度、NDT 0.08 度。點對面明顯優於點對點，印證了 17.3 節「點能沿平面自由滑動而不受懲罰」的論述；而 NDT 因為共變異數本身就編碼了表面方向，在角度上表現最好。

圖(b) 是對初始猜測的容忍度，也是本圖最實用的一格。橫軸是初始角偏差，縱軸是最終位移誤差（對數刻度）。三種方法在 15 度以內都穩定收斂到公分級；NDT 在 20 度開始失敗，兩種 ICP 則撐到 20 度、在 30 度崩潰。曲線的形狀值得注意：它不是緩慢劣化，而是斷崖式失敗——一旦超過吸引域，結果就不是「比較差」而是「完全錯」，誤差直接跳到公尺級。這解釋了為什麼實務上必須靠里程計或 IMU 提供初始猜測（17.3 節的第一個限制）：它們不需要很準，只要把誤差壓在十幾度以內就夠了。

圖(c) 則單獨檢驗 17.3 節的第二個限制：結構退化。場景換成一條 90 公尺長、寬度 3.2 公尺的無特徵長廊，長度遠超過光達 25 公尺的量程，所以機器人永遠看不到兩端。紅線是沿

著長廊軸向移動：殘餘誤差等於真實位移本身——位移 1.0 公尺，誤差就是 1.0 公尺，也就是配準完全沒有還原任何位移。綠線是橫向移動，誤差只有一兩公分，完全正常。藍線則是在同一條長廊沿途加上門洞與柱子之後再測軸向：誤差立刻降到公分級。

這三條線把「結構退化」講得再清楚不過：問題不在演算法，而在幾何資訊本身。無特徵長廊沿軸向平移後，點雲看起來一模一樣，配準演算法沒有任何依據可以判斷它移動了多少——再好的最佳化器也變不出不存在的資訊。對策只有兩條：靠別的感測器（輪速、IMU、UWB）補上那個方向的約束，或是確保環境裡有足夠的幾何特徵。這也回答了案例 17-2 的隧道難題。

**關於這組對照實驗的誠實說明**

速度沒有列入比較。本書的 NDT 是以 Python 撰寫的教學實作，而 ICP 用的是 Open3D 的 C++ 實作，兩者相差一個數量級的工程最佳化，比較執行時間並不公平。文獻中 NDT 較快的結論（17.4 節）來自同等最佳化程度的實作。

容忍度的結果與「NDT 對初值較不敏感」的常見說法不同。本實驗中 ICP 使用由粗到細的對應距離排程（1.5 → 0.75 → 0.3 公尺），這對 ICP 相當有利；NDT 的吸引域則取決於體素大小，本書已試過 2 公尺到 8 公尺的多種排程，以 2 公尺起始最佳。

結論該怎麼讀：三種方法在合理初值下都能用，點對面 ICP 與 NDT 的角度精度優於點對點；而「初值必須夠好」「退化場景無解」這兩個限制，對三者一視同仁。

同學可在實作 17-1 第二、三部分重現這三張圖，數字不同是正常的，重要的是趨勢。

### NDT 的關鍵參數：體素解析度

NDT 最重要的參數是體素（cell）的邊長。太小則每格點數不足，高斯分布估計不可靠；太大則空間描述過於粗糙，精度下降。有研究針對隧道環境的點雲實測發現，當體素解析度

設為 0.5 公尺時能滿足解析度需求，且配準時間最短。這提醒同學：最佳參數與場景尺度密切相關，室內、隧道、戶外各不相同，必須實測調校。

### NDT 與 ICP 的合作

既然 NDT 快而穩健、ICP 精確，自然的想法就是兩者結合：先用 NDT 做粗配準快速拉近，再用 ICP 做精配準提升精度。這種「NDT 粗配準 + ICP 精配準」的組合，在實務系統中相當常見，也再次體現本書反覆強調的思想：沒有萬能的演算法，只有恰當的組合。

## 17.5 3D 地圖與室內空間掃描

有了降採樣與配準工具，就能把一連串的光達掃描拼接成完整的三維地圖。本節說明 3D 地圖的表示方式與室內掃描的實務流程。

### 三種 3D 地圖表示

- 原始點雲地圖：直接保存所有（降採樣後的）點。保真度最高、視覺效果最好，但資料量大、無法區分「空」與「未知」、也不方便直接規劃路徑。適合測繪與展示。
- 3D 佔據格（八叉樹）：把空間切成體素並記錄佔據機率，以八叉樹（Octree）遞迴細分——只在有東西的地方細分，空曠處用大格子表示，大幅節省記憶體。這種表示能區分佔據、自由、未知三態，適合機器人導航與避障。第 3 章提過的 OctoMap 即為代表。
- 網格模型（Mesh）：把點雲轉成三角面片構成的表面模型。檔案小、可貼材質、適合建築與遊戲應用，但重建過程較複雜且可能失真。

### 室內空間掃描流程

29. 規劃掃描路線：確保覆蓋所有想掃描的空間，並在關鍵處（轉角、房間之間）保留足夠的重疊，重疊不足會導致配準失敗。
30. 資料採集：以背包式、手持式或機器人載具緩慢移動掃描（第 15.6 節）。速度要慢，並避免劇烈晃動造成運動變形。

31. 前處理：去除離群點、依需求裁切範圍、體素降採樣（17.2 節）。
32. 配準拼接：以 NDT 或 ICP 把相鄰掃描對齊（17.3、17.4 節），配合 SLAM 的位姿估計。
33. 全域最佳化：偵測閉環並執行圖優化（第 12、15 章），消除累積漂移，確保整體一致。
34. 後處理與輸出：視需求轉為八叉樹、網格或加上顏色（17.6 節），輸出為標準格式（如 PLY、PCD、LAS）。

### 品質評估

與第 13.7 節的 2D 地圖評估相呼應，3D 地圖也需檢查：牆面是否「厚」了（表示配準有誤差，同一面牆被畫成兩層）、直線結構是否筆直、閉環處是否對齊、以及量測關鍵尺寸與實測值比較。「牆變厚」是 3D 建圖最典型的品質指標——理想的牆面應該是薄薄一層點。

## 17.6 點雲顏色化與彩色視覺建模

純幾何的點雲雖然精確，但看起來像灰白色的骨架，不易辨識也不好看。加上顏色，點雲才真正成為「數位分身」。

### 顏色從哪來：投影原理

顏色來自相機影像。原理是第 9 章的針孔投影模型反過來用：

35. 已知相機的內參 K（第 9 章）與相機相對光達的外參（第 10 章的座標變換）。
36. 對每個三維點，先用外參把它從光達座標轉到相機座標。
37. 再用內參把它投影到影像平面，得到對應的像素座標。
38. 讀取該像素的 RGB 值，賦予這個三維點。

這個流程再次印證第 10 章的核心價值：精確的座標轉換與外參校正，是多感測器融合的基礎。外參若差幾度或幾公分，顏色就會「貼歪」——牆的顏色跑到窗戶上。

### 實務挑戰

- 時間同步：光達與相機必須對齊時間戳記。移動中若差幾十毫秒，顏色就會錯位（第 5、10 章的時間同步問題）。
- 遮蔽處理：相機看不到的點（被前方物體擋住）不應被賦予前方物體的顏色。需要做可見性判斷。
- 曝光與色差：不同時刻拍攝的影像亮度不同，拼接後點雲會出現色塊。需做色彩均衡。
- 視野不匹配：光達常為 360 度，相機視野有限。需多台相機或接受部分點沒有顏色。

### 彩色點雲的價值

- 可讀性：彩色模型讓非技術人員也能一眼看懂——這對向客戶或校方展示專案成果極為重要。
- 語意線索：顏色提供材質與物體的線索，是後續 AI 語意分割的重要輸入（第 32 章）。
- 文化資產保存：古蹟與文物的數位典藏，顏色與紋理是不可或缺的資訊。
- 缺陷檢測：牆面裂縫、鏽蝕、水漬等問題，在幾何上不明顯但在顏色上一目了然。

## 17.7 工廠、校園、展場的 3D 空間建模

本節說明 3D 建圖的實際應用價值，特別聚焦於大學場域可以實作的方向。

### 工廠：設施管理與數位分身

工廠的三維模型能用於產線布局規劃（新設備放得下嗎？搬運路徑會不會撞到？）、管線與消防設施記錄、以及設備維護的空間查詢。相關應用會結合預先載入的樓層平面圖輪廓，規劃能抵達每個可進入房間、走廊與機械空間的覆蓋路徑，並在建築物營運時間內確保機器人在人員與設備周圍安全避障。

光達相對相機的關鍵優勢在此格外明顯：不像以相機為基礎的方法在低光或無紋理區域（例如機房與停車場結構）表現不佳，光達不論照明條件如何都能可靠運作，因此成為設施維護應用的明確選擇。

### 校園：大學場域的實作方向

- 建築測繪：掃描系館內部，產生可用於教學、空間管理與無障礙規劃的三維模型。
- 室內導航地圖：為第九篇的語意導航建立空間基礎，讓機器人知道「電梯在哪、樓梯在哪」。
- 實驗室數位分身：掃描智慧機器人教室，作為模擬環境的基礎——這讓同學能在虛擬的教室中測試演算法，再搬到實體驗證。
- 教學素材：掃描成果本身就是絕佳的教材，讓後續學生看到「我們學校長這樣」的三維模型。

### 展場與文化資產

展場的三維模型能用於布展規劃、虛擬看展與動線分析；而古蹟與歷史建築的數位保存，更是 3D 掃描最有社會價值的應用之一——一旦建築因天災或歲月損毀，精確的三維紀錄就成為修復的唯一依據。台灣有豐富的歷史建築資源，這是一個兼具技術與人文意義的專題方向。

### 技術基礎：邊緣運算

值得注意的是，現代 3D 建圖越來越倚賴邊緣運算平台。SLAM 演算法在 NVIDIA Jetson 等邊緣處理器上運行，把個別掃描幀拼接起來，並透過閉環——機器人辨識出先前造訪過的區域，並用這個辨識修正軌跡估計中累積誤差的技術——來修正位置漂移。近年更出現 GPU 加速的體素重建函式庫，能即時建立符號距離場與佔據格，讓機器人在導航的同時重建設施幾何，同時支援建圖與避障。

這意味著 3D SLAM 已經不再是「必須用工作站」的技術，一台掌上型的 Jetson 就能勝

任——這對教學與專題實作是極大的好消息，也是第七篇（邊緣 AI SLAM）與第 32 章要深入的方向。

## 17.8 點雲處理的當代走向：2024–2026 年的四個動態

ICP 的原始論文發表於 1992 年，NDT 是 2003 年——以本章介紹的技術來說，它們都已經是三十年與二十年級的老方法。有趣的是，它們不但沒有被取代，近年反而出現了一股「回歸基本」的風潮。本節整理四個值得注意的動態。

### 一、回歸簡單：KISS-ICP 的「為點對點 ICP 辯護」

2023 年波昂大學團隊發表的 KISS-ICP，論文副標題直接寫著「為點對點 ICP 辯護」。他們的主張相當挑釁：與其不斷往光達里程計上疊加複雜度——特徵提取、深度學習模組、閉環偵測、位姿圖最佳化——不如把最基本的點對點 ICP 的幾個環節做對，效果就能與最先進的方法並駕齊驅。

他們做對的環節有四個：以等速度模型預測運動並補償掃描畸變（不需要 IMU 或輪速計）、以體素降採樣建立局部地圖、用穩健核函數做最佳化，以及最關鍵的「自適應對應距離門檻」——門檻不是手動設定，而是由「里程計實際偏離運動預測多少」的統計量自動算出（取三倍標準差）。結果是一套幾乎不需要調參的系統，同一組設定能跨越車載、無人機、手持等不同平台。

對修課同學而言，這個案例的啟示比技術細節更重要：當你的配準效果不好時，第一反應不該是換一個更複雜的演算法，而是檢查降採樣、初始猜測與對應距離門檻這幾個基本環節。本章圖 17-3(b) 的斷崖式曲線已經說明，初值與門檻的影響遠大於演算法的選擇。

### 二、工程效率：從演算法比較到實作比較

第二個動態是「同一個演算法，不同實作可以差好幾倍」這件事被正式端上檯面。2024 年由日本產業技術總合研究所 Kenji Koide 發表的 small_gicp 函式庫是代表作：它把降採樣、

KD 樹建構與配準全部重寫並平行化，報告指出其體素降採樣比 PCL 快約 1.3 倍（單執行緒）到 3.2 倍（六執行緒），KD 樹建構比 nanoflann 最多快 6 倍，而廣義 ICP 單執行緒就比 PCL 的版本快約 2.4 倍。

這呼應了案例 17-3 的工具選擇討論，也補上一個本章之前沒說的維度：當你在專題報告裡寫「我們用 ICP 做配準」時，這句話其實沒有交代清楚效能——用哪一套實作、有沒有開多執行緒、KD 樹怎麼建，對實際跑起來的速度影響可能比點對點與點對面之別還大。

## 三、地圖表示的新選項：三維高斯潑濺

第三個動態衝擊的是 17.5 節那張「三種 3D 地圖表示」的清單。近兩年，三維高斯潑濺（3D Gaussian Splatting）快速成為第四種選項：它用大量帶有位置、形狀、顏色與透明度的三維高斯來表示場景，既能像點雲一樣由資料直接擬合，又能像網格一樣算繪出逼真的影像。

這條線已經延伸到光達領域。2025 年的 Splat-LOAM 把高斯潑濺直接當成光達里程計與建圖的表示，而 LiV-GS 一類的工作則把光達與相機整合進同一套高斯潑濺 SLAM，用於戶外場景。同學應該會注意到一個有趣的巧合：NDT 用體素內的高斯描述表面（本章圖 17-2），高斯潑濺用空間中的高斯描述場景外觀——兩者相隔二十年，卻都選擇了「用一堆高斯分布來描述世界」這個答案。

## 四、學習式配準與傳統方法的分工

最後一個動態是深度學習在配準問題上的定位逐漸清晰。學習式方法的強項在於「找對應」這一步——學出來的三維描述子能在重疊很少、初始猜測極差的情況下，直接找出兩片點雲的對應關係，這正是 ICP 與 NDT 最弱的地方（圖 17-3(b) 的斷崖）。但在對應找到之後，求解變換仍然是第 12 章的最小平方問題，傳統方法又快又準。

因此目前的實務分工相當明確：全域配準（不知道初值，例如重定位與閉環）傾向用學習式描述子或隨機抽樣一致法（RANSAC）；局部精修（已有不錯初值，例如連續幀之間）仍然

是 ICP 與 NDT 的天下。而 KISS-ICP 的存在則提醒我們：在連續掃描的光達里程計這個特定任務上，初值其實一直都夠好，所以傳統方法至今沒有被取代的理由。

**四個動態的共同線索**

方法面（KISS-ICP）：複雜度不等於效果，基本環節做對比換演算法更有用。

工程面（small_gicp）：同一個演算法的不同實作可以差好幾倍，效能是實作問題而非只是理論問題。

表示面（高斯潑濺）：地圖表示從「點、體素、網格」擴張到「一堆高斯」，與 NDT 的思想遙相呼應。

學習面（描述子）：深度學習補的是「找對應」，不是「求變換」；兩者是分工而非取代。

共同線索：本章的四樣工具沒有一樣被淘汰，被改變的是它們的使用方式與工程品質。把基礎打穩，才讀得懂後面所有的變化。

### 承先啟後

本章備齊了三維點雲的處理工具：降採樣讓資料量回到可處理的範圍，ICP 與 NDT 讓相鄰掃描能對齊，地圖表示與顏色化讓成果能被使用與展示。圖 17-1 到圖 17-3 的三組實驗分別驗證了這三件事的定律、原理與極限——特別是圖 17-3(c) 那條紅線，它提醒我們有些誤差不是演算法的錯，而是幾何資訊本身不存在。

但把相鄰掃描逐幀配準再累加，誤差仍會隨距離累積——這正是第 7 章漂移問題的三維版本。下一章的 LOAM 系列給出了一個優雅的答案：與其對整片點雲做配準，不如先從掃描中提取「邊緣點」與「平面點」兩類特徵，只拿特徵去配準。這個想法讓即時的三維光達里程計成為可能，也開啟了第五篇真正的主角。

## 公式或流程

## 一、體素降採樣的點數估算

把邊長為 d 的體素套用到點雲上，輸出點數約等於「被佔據的體素數」。由於三維空間的關係：

$$\text{輸出點數} \propto 1 \;/\; d^2 \quad (\text{表面型點雲})$$

注意這裡是平方而非立方——因為光達掃到的是「表面」而非實心體積，點主要分布在二維曲面上。實務經驗：體素邊長加倍，點數約降為四分之一。若點雲填滿三維體積（如醫學影像），才會是立方關係。

## 二、ICP 的目標函數

點對點 ICP 求旋轉 R 與平移 t，使配對點的距離平方和最小：

$$(R^*, t^*) = \operatorname{argmin} \Sigma_i \,\| \,(R \cdot p_i + t) - q_i \,\|^2$$

其中 $p_i$ 是來源點雲的點、$q_i$ 是它在目標點雲中的最近點。點對面 ICP 則把誤差改為沿目標點法向量 $n_i$ 的投影：

$$(R^*, t^*) = \operatorname{argmin} \Sigma_i \,((\,(R \cdot p_i + t) - q_i\,) \cdot n_i\,)^2$$

差別在於：點對點懲罰「所有方向的偏離」，點對面只懲罰「垂直於表面的偏離」，讓點能沿平面滑動——這正是它收斂較快的原因。

## 三、NDT 的機率目標

NDT 把目標點雲的每個體素以高斯分布 $(\mu, \Sigma)$ 描述。來源點 p 經變換 T 後落在某體素中，其得分為：

$$\text{score}(p) \propto \exp\!\left(-\,(\,(T \cdot p - \mu)^{\top} \Sigma^{-1} (T \cdot p - \mu)\,)\,/\,2\right)$$

配準目標是最大化所有點的總得分：

$$T^* = \operatorname{argmax} \Sigma_i \,\text{score}(T \cdot p_i)$$

式中的 $(x-\mu)^{\top}\Sigma^{-1}(x-\mu)$ 正是第 11 章多維高斯分布的核心項（馬氏距離）。同

學可以看出：第三篇的機率數學，在此直接成為 3D 配準的工具。

## 四、3D 建圖完整流程

**圖 17-4　從光達掃描到 3D 地圖的完整流程**

① 3D 光達輸出原始點雲（每秒數十萬至上百萬點）

↓

② 前處理：離群點剔除、範圍裁切、運動補償

↓

③ 體素降採樣：減少點數並使分布均勻（17.2 節）

↓

④ 特徵/法向量估計：為點對面 ICP 或後續處理準備

↓

⑤ 配準：NDT 粗配準 → ICP 精配準（17.3、17.4 節）

配合里程計/IMU 提供初始猜測

↓

⑥ 累積拼接：把新掃描併入全域地圖

↓

⑦ 閉環偵測 + 圖優化：消除累積漂移（第 12、15 章）

↓

⑧ 後處理：轉八叉樹/網格、顏色化（17.6 節）、輸出

## 案例

## 案例 17-1 牆為什麼會變厚：3D 建圖的品質指標

第一次做 3D 建圖的同學，常會遇到一個令人困惑的現象：掃描完一個房間，從上方俯視點雲，原本應該是薄薄一層的牆面，卻變成厚達十幾公分的「牆板」；甚至有時會看到兩層平行的牆面，中間空著。

這個現象的成因很直接：配準有誤差。機器人在不同時刻從不同位置掃到同一面牆，若位姿估計準確，這些掃描應該完美重疊成一層；若位姿有偏差，同一面牆就被畫在略微不同的位置，疊起來就「變厚」了。偏差大到一定程度，就會出現明顯的雙層牆（這是三維版的「重影」，對應第 13.7 節的 2D 病徵）。

因此「牆的厚度」成為 3D 建圖最實用的品質指標：量測一段牆面點雲在垂直牆面方向的分布厚度，理想值應接近光達本身的量測精度（數公分）。厚度明顯超出，就表示配準或 SLAM 有問題。這個檢查方法簡單、直觀、不需真值，強烈建議同學在每次建圖後都做一次——它能立刻告訴你「這張圖能不能用」。

## 案例 17-2 從隧道到礦坑：退化場景的三維挑戰

第 13.6 節談過長走廊讓 2D 掃描匹配失效，這個問題在三維世界只會更嚴重。隧道、地下坑道、長廊等場景有一個共同特徵：沿著長軸方向，環境的幾何形狀幾乎不變。點雲可以沿著這個方向自由滑動而不增加配準誤差——這稱為結構退化（Degeneracy）。

針對這類場景，研究者發展出多種對策。有研究團隊在隧道環境中採用「先體素降採樣保持整體結構並減少點數，再以 NDT 求解座標變換並依環境特徵最佳化體素解析度，最後在 ICP 中引入 KD 樹加速點對搜尋並以高斯—牛頓法求解」的組合流程，實測顯示體素解析度設為 0.5 公尺時效果最佳。

這個案例給同學三個啟示：其一，退化場景是 3D SLAM 的共同難題，不是你做錯了；其二，NDT 與 ICP 各有所長，組合使用往往優於單獨使用；其三，參數必須依場景調校，沒有放諸四海皆準的設定。這也預告了第 18 章 LOAM 系列如何用「特徵提取」的方式來對

付這個問題。

### 案例 17-3　工具的選擇：PCL 與 Open3D

做 3D 點雲處理，幾乎一定會用到兩個開源函式庫：PCL（Point Cloud Library）與 Open3D。前者是 C++ 為主的老牌工具，功能極為完整，是 ROS 生態的標準配備；後者由 Intel 開發，以 Python 介面友善、視覺化方便著稱，近年在研究與教學中相當流行。

Open3D 提供多種先進表面配準方法的實作，包括成對的全域配準、成對的局部精修，以及使用位姿圖最佳化的多路配準——同學應該立刻認出最後一項正是第 12 章的圖優化，只是用在點雲拼接上。

效能方面，兩者各有勝場。有基準測試比較了不同函式庫的配準速度，結果顯示點對面 ICP 通常比點對點 ICP 快、而 NDT 的速度表現也相當有競爭力；近年更出現專為平行化設計的輕量配準函式庫，把降採樣、KD 樹建構、法向量估計等前處理步驟全面平行化，多執行緒版本可比單執行緒快數倍。

對本書讀者的建議：教學與快速原型用 Open3D（Python 上手快、視覺化好），整合進 ROS 系統則用 PCL，追求效能的專題可考慮新一代的平行化函式庫。這與第 8 章「先模擬、再便宜硬體、後追求精度」的學習路徑一脈相承。

## 實作活動

### 實作 17-1　點雲處理與配準實作

目的：親手完成點雲的載入、降採樣、視覺化與配準，理解 ICP 與 NDT 的行為差異與參數影響。本實作以 Python 搭配 Open3D 為主，不需 3D 光達硬體即可完成。

#### 第一部分：點雲基礎與降採樣（全體）

28. 安裝 Open3D，載入內建的範例點雲（或從公開資料集下載一片室內掃描），用互動視窗旋轉觀察，感受三維點雲的樣貌。

29. 印出點雲的基本資訊：點數、座標範圍（邊界框）、是否含顏色與法向量。

30. 體素降採樣實驗：分別用邊長 0.01、0.05、0.2 公尺降採樣，記錄每次的輸出點數，並驗證是否符合公式一的平方關係。

31. 視覺比較：把原始點雲與三種降採樣結果並排顯示，觀察在什麼體素大小下，牆角、桌腳等細節開始消失。

32. 估計法向量並以顏色顯示（法向量方向對應顏色），觀察牆面、地面、天花板如何被清楚區分——這是點對面 ICP 的前置作業。

### 第二部分：ICP 配準（全體）

33. 取一片點雲，複製一份並套用一個已知的旋轉與平移（例如繞 Z 軸轉 20 度、平移 0.3 公尺），作為「來源點雲」。

34. 用點對點 ICP 配準，比較估出的變換矩陣與真值，計算誤差，並記錄迭代次數與耗時。

35. 改用點對面 ICP（需先估法向量），比較收斂速度與精度的差異，驗證 17.3 節的論述。

36. 初始值實驗：把初始偏移逐步加大（5 度、20 度、60 度、120 度），找出 ICP 開始失敗的臨界點，親身驗證「ICP 需要好的初始猜測」。

### 第三部分：NDT 與組合配準（大學部）

37. 用 NDT 做同一組配準，比較它與 ICP 在速度、精度與初始值容忍度上的差異，填入你自己的表 17-2。

38. 體素解析度實驗：調整 NDT 的體素大小（0.2、0.5、1.0 公尺），觀察對成功率與精度的影響，對照案例 17-2 的討論。

39. 實作「NDT 粗配準 + ICP 精配準」的組合流程，比較它與單獨使用任一方法的結果。

40. 退化場景測試：自行合成一段「長廊」點雲（兩面平行長牆 + 地板），測試 ICP 沿長廊方向的配準是否失準，重現結構退化現象。

**討論問題**

9. 體素邊長從 0.01 增加到 0.2 公尺，點數減少了幾倍？符合平方關係還是立方關係？為什麼？
10. 點對面 ICP 比點對點快多少？從 17.3 節的原理解釋這個差異。
11. ICP 在初始偏移多大時開始失敗？這對真實機器人系統意味著什麼（提示：里程計與 IMU 的角色）？
12. 長廊測試中，配準在哪個方向失準？這與第 13.6 節的 2D 走廊困境是同一個問題嗎？
13. （大學部）若要為一台校園巡檢機器狗選擇配準方法，你會選 ICP、NDT 還是兩者組合？說明你的理由與場域考量。

**延伸挑戰（大學部）**

其一：下載一組公開的 3D 光達資料集（如 KITTI 的一段序列），實作一個簡易的光達里程計——連續對相鄰兩幀做配準並累加位姿，把估出的軌跡與真值比較，觀察漂移如何隨距離累積（這正是第 18 章 LOAM 要解決的問題）。其二：若學校有 3D 光達，實際掃描一間實驗室，完成圖 17-4 的完整流程並輸出彩色點雲模型，再用案例 17-1 的「牆厚度」方法評估品質。這份成果可直接作為第 31 章專題二的基礎。

## 延伸閱讀

15. P. Besl and N. McKay, "A Method for Registration of 3-D Shapes," IEEE TPAMI, 1992.（ICP 的原始論文，第 13、17 章共同的理論源頭）
16. P. Biber and W. Straßer, "The Normal Distributions Transform: A New Approach to Laser Scan Matching," IROS, 2003.（NDT 的原始論文）

17. Q.-Y. Zhou, J. Park, and V. Koltun, "Open3D: A Modern Library for 3D Data Processing," arXiv, 2018.（Open3D 的介紹論文，含配準工作流程範例）

18. Point Cloud Library (PCL) 官方教學（pointclouds.org）.（降採樣、法向量估計、配準的完整範例）

19. Open3D 官方文件與 Python 教學（open3d.org）.（本章實作的主要工具，教學友善）

20. A. Hornung et al., "OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees," Autonomous Robots, 2013.（3D 佔據格地圖的代表作，呼應 17.5 節）

21. X. Li et al., "A Tutorial Review on Point Cloud Registrations: Principle, Classification, Comparison, and Technology Challenges," Mathematical Problems in Engineering, 2021.（點雲配準的完整綜述，含各種降採樣策略比較）

## 小結

15. 3D 點雲是由大量三維點構成的集合，可附帶反射強度、顏色、線號與時間戳記；它具有無序、稀疏不均、資料量龐大三大特性，且無法像佔據格地圖那樣區分「確定空」與「未知」。

16. 體素降採樣把空間切成立方格，每格以重心作為代表點，能大幅減量並使分布均勻；取重心比取中心精確，近似版本可能產生虛假點。體素邊長應略小於關心的最小特徵尺寸。

17. ICP 以「找對應—求變換—迭代」配準點雲，靠 KD 樹加速最近鄰搜尋；點對面 ICP 因只懲罰垂直表面的偏離而收斂較快，是實務首選。三大限制為需要好初值、怕結構退化、對離群點敏感。

18. NDT 把目標點雲轉成體素內的高斯分布，形成連續可微的機率場，不需搜尋對應點；它比 ICP 快、對初值較不敏感，但精度略低，關鍵參數是體素解析度，須依場景尺度調校。

19. 3D 地圖有三種表示：原始點雲（保真度高）、八叉樹佔據格（能區分三態、適合導航）、網格模型（檔案小、適合展示）；室內掃描流程為規劃路線、採集、前處理、配準、全域最佳化、後處理輸出。

20. 點雲顏色化以第 9 章的相機投影模型，把影像 RGB 賦予三維點，關鍵在精確的內外參與時間同步；彩色點雲提升可讀性、提供語意線索，並用於文化資產保存與缺陷檢測。

21. 3D 建模應用於工廠數位分身、校園測繪與導航地圖、展場與古蹟保存；光達不受照明影響的特性使其在低光與無紋理場域勝過相機，而邊緣運算平台的進步已讓 3D SLAM 走出工作站。

本章備齊了三維點雲的處理工具：降採樣、配準、地圖表示與顏色化。但把相鄰掃描逐幀配準累加，誤差仍會隨距離累積——這正是第 7 章漂移問題的三維版本。下一章的 LOAM 給出了漂亮的解答：它不對所有點做配準，而是聰明地提取「邊緣特徵」與「平面特徵」，並用「高頻里程計 + 低頻建圖」的雙層架構，在即時性與精度之間取得絕佳平衡。這是自主巡檢機器人與自駕車 3D 定位的技術骨幹。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

## 第 18 章

## LOAM 與光達里程計

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

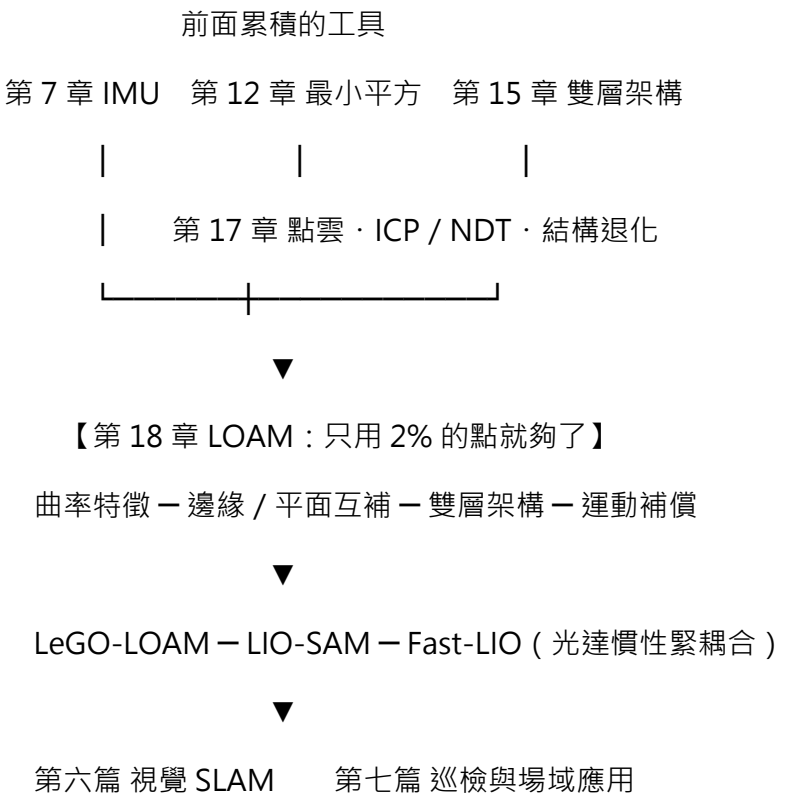
大學智慧機器人學程 / 高中 AI 機器人社團課程

## 本章在全書的位置

第 17 章把點雲處理的工具備齊了，但也留下一個尷尬：ICP 與 NDT 都要花時間在「找對應」上，而一幀 3D 掃描動輒數萬點，逐幀配準再累加，既慢又會持續漂移。本章的 LOAM 給出的答案聽起來幾乎像是取巧——不要用全部的點，只用百分之二三的「有特色」的點。但這個取巧背後有嚴謹的幾何理由，而理解那個理由，就是理解整個第五篇的鑰匙。

本章的結構呼應了 LOAM 的兩大貢獻。18.2 節談「挑哪些點」，用的是曲率這把量尺；18.3 節談「怎麼安排運算」，用的是高頻 / 低頻雙層架構——同學會發現它與第 15 章 Cartographer 的「局部 SLAM + 全域 SLAM」是同一個設計哲學在不同維度的化身。18.4 之後則是這套架構的後代與應用，一路通到第七篇的巡檢專題。

### 圖 18-0　第 18 章的連結地圖




## 學習目標

上一章我們學會了點雲的降採樣與配準，但把相鄰掃描逐幀 ICP 累加，不僅慢、誤差也持續累積。LOAM 給出了漂亮的解答——不對所有點做配準，而是聰明地提取邊緣與平面特徵，

並用「高頻里程計 + 低頻建圖」的雙層架構取得即時性與精度的平衡。這是自主巡檢機器人與自駕車 3D 定位的技術骨幹。學完本章之後，同學應能夠達成下列目標：

15. 說明 LOAM 的核心問題與它在 3D 光達 SLAM 發展史上的地位。
16. 說明如何以曲率（粗糙度）判別邊緣特徵與平面特徵，以及為何只用特徵點就能配準。
17. 解釋高頻里程計與低頻建圖的雙層架構，以及它為何能兼顧即時性與精度。
18. 比較 LeGO-LOAM、LIO-SAM、Fast-LIO 等後續系統的改良重點。
19. 說明光達與 IMU 緊耦合的動機、優勢與實作要點。
20. 說明 3D SLAM 如何應用於自主巡檢機器人。
21. 規劃一個以大學場域為對象的巡檢專題（走廊、電梯、實驗室）。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 光達里程計與建圖 | LOAM | 以邊緣與平面特徵進行即時光達定位建圖的經典方法。 |
| 光達里程計 | LiDAR Odometry | 僅由連續光達掃描推算機器人運動的技術。 |
| 曲率 / 粗糙度 | Curvature / Smoothness | 衡量一點周圍局部起伏程度的指標，用來分類特徵。 |
| 邊緣特徵 | Edge Feature | 曲率高的點，位於角落、柱子、樹幹等尖銳處。 |
| 平面特徵 | Planar Feature | 曲率低的點，位於牆面、地面等平坦處。 |
| 點到線距離 | Point-to-Edge Distance | 邊緣特徵配準時使用的誤差量。 |
| 點到面距離 | Point-to-Plane Distance | 平面特徵配準時使用的誤差量。 |
| 運動變形 | Motion Distortion | 光達旋轉期間載具移動造成同一幀點雲扭 |

| 中文名詞 | 英文名詞 | 概念說明 |
| --- | --- | --- |
| | | 曲。 |
| 地面分離 | Ground Segmentation | 把地面點與非地面點分開處理，LeGO-LOAM 的關鍵。 |
| 兩步最佳化 | Two-step Optimization | 先由地面平面解部分自由度，再由邊緣解其餘自由度。 |
| 光達慣性里程計 | LIO | 光達與 IMU 緊耦合的里程計方法。 |
| 迭代擴展卡爾曼濾波 | iEKF | Fast-LIO 採用的高效緊耦合估測架構。 |

## 18.1 LOAM：LiDAR Odometry and Mapping

LOAM 由 Ji Zhang 與 Sanjiv Singh 於 2014 年在 CMU 機器人研究所提出，論文標題為「LOAM: Lidar Odometry and Mapping in Real-time」。它是 3D 光達 SLAM 領域影響最深遠的一篇工作——後續幾乎所有主流系統都建立在它的思想之上。

### 它要解決的問題

第 17 章末尾留下一個難題：把相鄰的 3D 掃描逐幀用 ICP 配準再累加，會遇到兩個瓶頸。其一是速度——每幀數十萬點，ICP 的最近鄰搜尋負擔沉重，難以即時。當掃描包含大量點時，ICP 可能承受過高的計算成本。其二是精度——逐幀累加，誤差持續漂移。

LOAM 的核心洞見是：不需要用上所有的點。環境中真正提供幾何約束的，是那些「有特色」的點——牆角、柱子、樹幹（邊緣），以及牆面、地面（平面）。只要抓住這些特徵點來配準，就能用不到原始點數百分之一的資料量，達到相當甚至更好的精度。

### LOAM 的兩大貢獻

- 特徵提取：以曲率（粗糙度）把點分為邊緣特徵與平面特徵，大幅減少參與運算的點數（18.2 節）。
- 雙層架構：把問題拆成「高頻但粗略的里程計」與「低頻但精確的建圖」兩個並行任務，兼顧即時性與精度（18.3 節）。

LOAM 把 SLAM 問題拆成兩個平行任務——里程計與建圖——並藉由從光達掃描中提取邊緣與平面特徵、在長期最佳化中處理特徵對應，在結構化環境中以低運算成本達成高精度。這個「特徵提取 + KD 樹對應搜尋」的架構，已被後續大量光達里程計研究採用為核心流程。

### 歷史地位

自 Zhang 與 Singh 提出 LOAM 以來，以光達為基礎的 SLAM 研究快速推進；LOAM 以曲率提取邊緣與平面特徵，並透過與 KD 樹的掃描對地圖匹配估計 6 自由度位姿，藉由把

里程計與建圖分離到不同的執行頻率，同時達成運算效率與精度。

對本書讀者而言，理解 LOAM 就等於掌握了整個第五篇的鑰匙：第 18.4 節的 LeGO-LOAM、LIO-SAM、Fast-LIO 全都是它的後代，而大學自主巡檢機器人的相關研究，也以這條技術脈絡為基礎。

## 18.2 邊緣特徵與平面特徵

本節是 LOAM 最具識別度的設計：如何從數十萬點中，挑出少數真正有用的特徵點。

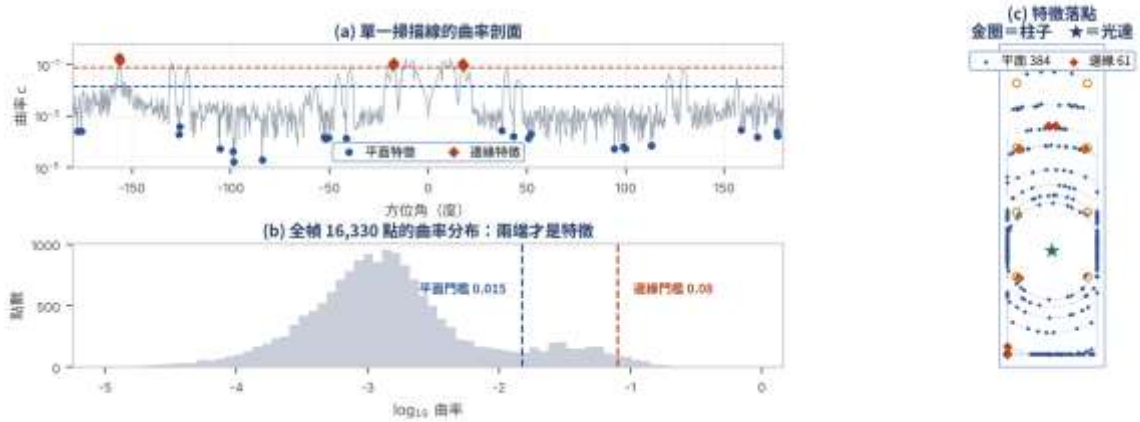
### 曲率：判別特徵的量尺

LOAM 用一個簡單而巧妙的指標來分類每個點——局部粗糙度（常稱為曲率）。做法是：取某個點與它在同一條掃描線上的鄰居們，計算「這個點偏離鄰居平均位置的程度」。

- 粗糙度高：表示這個點與周圍鄰居的排列不共線，位於尖銳處——這是邊緣特徵。典型例子：牆角、柱子邊、樹幹、桿件。
- 粗糙度低：表示這個點與鄰居近乎排成一直線，位於平坦處——這是平面特徵。典型例子：牆面、地面、天花板、大型平板。

每一幀點雲中，只需挑出粗糙度最高的少數點作為邊緣特徵、最低的少數點作為平面特徵。LeGO-LOAM 論文中的圖示清楚展示了這個過程：原始點雲經分割後，以藍色與黃色分別標示邊緣與平面特徵。

**圖 18-1　曲率這把量尺：從一條掃描線到少數幾個特徵點**




這張圖來自一次真實的模擬：一條寬 7 公尺、長 80 公尺的走廊，沿途有柱列、門框與一根方柱，用 32 線光達掃描一幀，得到 32,603 個有效點。圖(a) 取其中一條掃描線，把公式一算出的曲率沿方位角畫出來（縱軸為對數刻度）。整條曲線絕大部分時間都在 $10^{-3}$ 附近低吟——那是平坦的牆面；而每當掃到柱子或門框，曲線就出現一根尖峰，瞬間躍升兩個數量級。

圖(b) 把全幀所有點的曲率畫成直方圖，可以看到一個單峰分布，峰值落在 $10^{-3}$ 左右。LOAM 要的不是這個峰，而是分布的「兩端」：右端尾巴（曲率高）是邊緣特徵，左端尾巴（曲率低）是平面特徵。中間那一大坨佔了絕大多數的點，對配準幾乎沒有額外貢獻——這就是案例 18-1 所說「大部分點是冗餘的」最直接的視覺證據。

圖(c) 驗證了這套篩選是否真的挑到對的東西。紅色菱形是邊緣特徵，可以看到它們準確地落在金圈標出的柱子上，以及門框與方柱的位置；藍色小點是平面特徵，均勻鋪在兩側長牆與地面上。最後的數字是：邊緣 61 個、平面 384 個，合計僅佔原始點數的 2.7%。換句話說，LOAM 丟掉了 97% 的資料，而丟掉的那些，正是提供不了新資訊的那些。

本書也實測了這個減量的代價。在同一組掃描上，用全部 32,513 點做點對面配準，角誤差 0.031 度、位移誤差 1.1 公釐、耗時 0.18 秒；只用 768 個平面特徵（2.4%），角誤差 0.042 度、位移誤差 1.3 公釐、耗時 0.038 秒。精度幾乎沒有損失，時間卻降到五分之一。這就是 LOAM 能夠「即時」的根本原因。

**曲率門檻與那兩道剔除**

本圖的門檻取邊緣 c > 0.08、平面 c < 0.015，並限制每條掃描線的每個子區段最多取 2 個邊緣、4 個平面點，避免特徵擠在同一處（18.2 節的「均勻分布」）。

LOAM 原論文另有兩道剔除，本圖也實作了：一是遮擋邊界點（相鄰距離跳變處，較遠一側其實是被擋住的，下一幀就會變樣）；二是掃描線近乎平行於表面的點（量測極不穩定）。

剔除比例約 8%。少了這兩道，遠處牆面因量測雜訊造成的假尖峰會被誤判成邊緣特徵，本書初次實作時正是如此——邊緣點散落在空牆上，而不是柱子上。

門檻需隨光達線數與場域調整。線數越少、角解析度越粗，曲率的雜訊就越大，門檻要放寬。

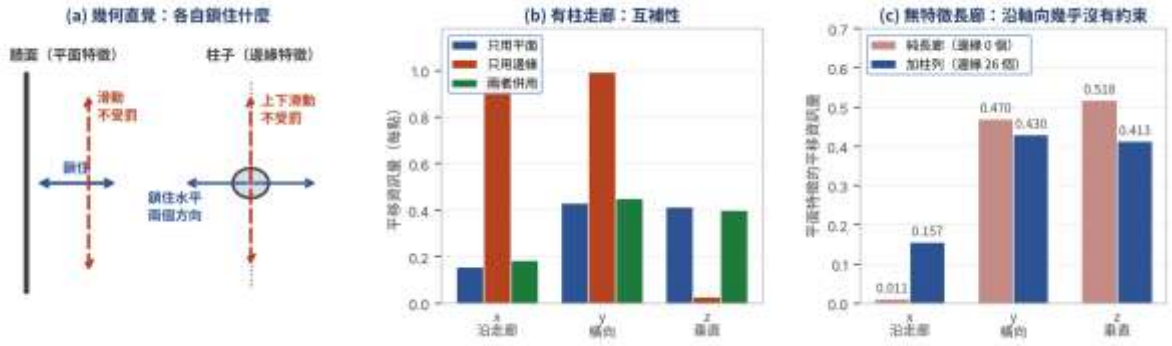
### 為什麼兩種特徵都需要

這是本節最重要的觀念。兩種特徵提供的幾何約束互補：

- 平面特徵約束「垂直於平面的移動」。一面牆能告訴你「你離牆多遠」，但無法告訴你「你沿著牆走了多遠」——沿牆滑動時，平面約束完全不變。
- 邊緣特徵約束「垂直於邊線的移動」。一根柱子能鎖定你在水平兩個方向的位置，但沿柱子上下移動它管不著。

把兩者結合，才能鎖定完整的六個自由度。這也解釋了第 17.3 節提到的結構退化：在長廊中，牆面提供的都是「同方向的平面約束」，而缺乏沿走廊方向的邊緣特徵，沿廊方向的位移因而無法確定。

**圖 18-2　為什麼兩種特徵缺一不可：鎖住的方向正好互補**




這是本章最重要的觀念，而它可以被精確地量化。做法是把配準誤差對位姿的敏感度寫成一個「資訊矩陣」——某個方向上的資訊量越大，代表只要往那個方向偏一點，誤差就會明顯上升，也就是那個方向被鎖得越緊。圖(a) 先給出幾何直覺：一面牆只能鎖住垂直於牆的那一個方向，沿著牆面滑動它完全不受懲罰；一根柱子則鎖住水平的兩個方向，但沿著柱子上下滑動它管不著。

圖(b) 是實測。只用平面特徵時，沿走廊方向的資訊量是 0.157，橫向 0.430、垂直 0.413——沿走廊明顯是最弱的一環。只用邊緣特徵時情況恰恰相反：水平兩個方向分別是 0.980 與 0.994，幾乎滿格，而垂直方向只有 0.026，等於沒有約束。兩組數字的形狀互為補集，這正是 18.2 節那句「兩種特徵提供的幾何約束互補」的量化版本。

圖(c) 則把結論推到極端，順便回答了第 17 章留下的問題。把走廊裡的柱子與門框全部移除，變成一條純粹的空長廊，重新計算：平面特徵沿走廊方向的資訊量只剩 0.011，而橫向 0.470、垂直 0.518——相差超過四十倍。更說明問題的是，在這條空長廊裡，曲率篩選找到的邊緣特徵數量是零：環境裡根本沒有任何尖銳結構可以提供沿軸向的約束。

這就是第 17.3 節「結構退化」在 LOAM 框架下的精確描述：不是演算法不夠好，而是幾何資訊本身不存在。而一旦把柱列加回去，邊緣特徵立刻出現 26 個，平面特徵沿走廊的資訊量也從 0.011 升到 0.157（門框與方柱貢獻了垂直於走廊的面）。這也預告了 18.5 節的結論——當環境本身提供不了約束時，唯一的辦法是從別的感測器拿，而 IMU 正是那個來源。

**怎麼讀「資訊量」這個數字**

它是每個特徵點平均提供的約束強度，無單位，介於 0 與 1 之間。1 代表該方向被完全鎖住，0 代表完全自由。

平面特徵的資訊矩陣是 $\Sigma\, n\, n^{T}$（$n$ 為表面法向量）：法向量指向哪，就鎖住哪個方向。

邊緣特徵的資訊矩陣是 $\Sigma\,(I - u\,u^{T})$（$u$ 為邊線方向）：鎖住所有垂直於邊線的方向。本圖中柱子都是鉛直的，所以 $u$ 幾乎都是 $z$ 軸，鎖住的就是水平面。

這個分析也是現代系統「退化偵測」的基礎：即時算出資訊矩陣的最小特徵值，一旦低於門檻就知道自己正走進退化場景，可以提高對 IMU 的信任（見 18.7 節）。

### 均勻分布：避免特徵集中

LOAM 還有一個實務細節：把每條掃描線分成數個子區段，在每個子區段中各挑選一定數量的特徵點。這確保特徵均勻分布於整個視野，而不會全部集中在某個角落——特徵若集中在一側，配準的幾何條件會很差（這與第 5 章 GNSS 的 DOP 概念異曲同工）。

### 配準的誤差定義

特徵提取後，配準的方式也隨特徵類型而不同：

- 邊緣特徵：在上一幀（或地圖）中找出對應的邊線（由兩個鄰近的邊緣點決定），誤差定義為「點到這條線的垂直距離」。
- 平面特徵：在上一幀（或地圖）中找出對應的平面（由三個鄰近的平面點決定），誤差定義為「點到這個平面的垂直距離」。

把所有特徵點的誤差平方和最小化，就得到最佳的位姿變換——這又是第 12 章的非線性最小平方問題。同學應該能感受到：第三篇的數學工具，在每一個具體系統中反覆現身。

## 18.3　高頻里程計與低頻地圖最佳化

LOAM 的第二項核心創新，是把 SLAM 拆成兩個並行運行、頻率不同的任務。這個架構設計極為經典，值得同學仔細理解。

### 兩個任務的分工

- 光達里程計（高頻）：比較相鄰兩幀掃描的特徵，快速估出「這 0.1 秒我移動了多少」。它只用兩幀資料，運算輕、速度快，但精度較低、會漂移。
- 光達建圖（低頻）：把當前掃描與「累積的全域地圖」比對，做更精細的配準。地圖包含大量歷史資訊，配準更準，但運算重、頻率低。

在 LOAM 的原始設計中，建圖模組以 1 Hz 的頻率把點雲投影到地圖上；最後，兩個演算法發布的位姿變換被整合起來，產生約 10 Hz 的輸出，代表光達相對於地圖的位姿。

**圖 18-3　LOAM 的雙層架構**

【光達里程計】高頻（約 10 Hz）

相鄰兩幀特徵匹配 → 快速估出相對運動

特性：即時、輕量、精度較低、會漂移

↓ 提供運動估計與初始猜測

【光達建圖】低頻（約 1 Hz）

當前掃描 ↔ 全域地圖特徵匹配 → 精細配準

特性：精確、運算重、頻率低

↓

【位姿整合】

把兩者的變換整合 → 輸出約 10 Hz 的高頻且精確位姿

### 為什麼這樣設計是聰明的

這個架構的精妙之處，在於它讓「即時性」與「精確性」不再互相衝突：

39. 高頻里程計滿足即時控制的需求。機器人的運動控制迴路需要高頻位姿，不能等 1 秒才更新一次。

40. 低頻建圖負責消除漂移。它有充裕的時間做精細配準，把高頻里程計累積的誤差拉回來。

41. 高頻里程計為低頻建圖提供良好的初始猜測。這正是第 17.3 節強調的——ICP 類方法需要好的初值才能收斂，而里程計恰好提供了它。

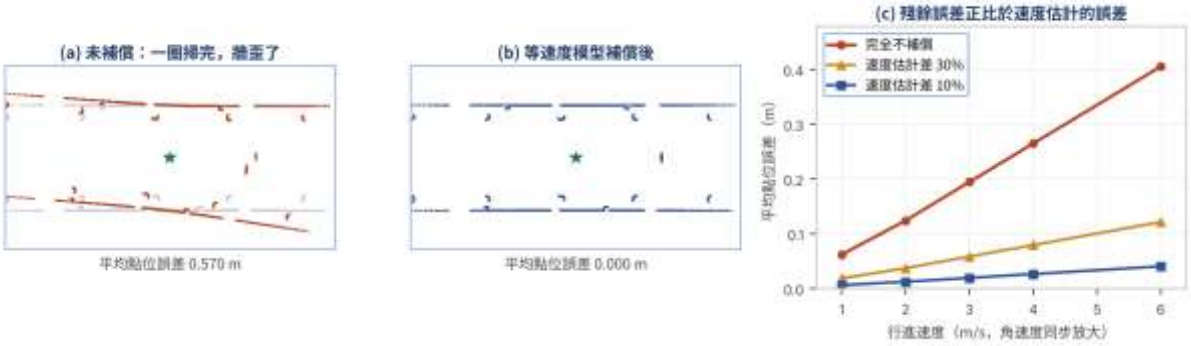
同學應該注意到：這個「快速但粗略的前端 + 緩慢但精確的後端」架構，與第 15 章 Cartographer 的「局部 SLAM + 全域 SLAM」雙子系統如出一轍。這不是巧合，而是機器人系統設計的一個普遍模式：用不同的時間尺度處理不同性質的問題。

### 運動變形補償

還有一個 3D 光達特有的問題必須處理。機械式光達旋轉一圈需要約 0.1 秒（10 Hz），在這段時間內，載具本身也在移動。結果是：同一「幀」點雲中的各個點，其實是在不同位置量到的——點雲因此被扭曲，稱為運動變形（Motion Distortion，第 8.4 節提過）。

LOAM 的處理方式是：利用里程計估出的運動，把每個點依其量測時刻反推回「幀起始時刻」的座標系，消除變形。這正是第 17.1 節提到「點雲需要時間戳記」的原因。後續系統如 LIO-SAM 也明確要求：當光達以 10 Hz 旋轉時，每個點的時間戳記應在 0 到 0.1 秒之間變化。

**圖 18-4　運動變形：光達轉一圈的時間裡，車子也在動**




機械式光達轉一圈約需 0.1 秒，而在這 0.1 秒內載具本身也在移動與轉向。結果是：同一「幀」點雲裡的點，其實是在不同位姿下量到的。若天真地把它們全部當成同一時刻、同一位姿的量測，整片點雲就會被扭曲。圖(a) 是這個現象的模擬：載具以 6 公尺 / 秒前進、同時以 1.6 弧度 / 秒轉彎，淺灰色是真實幾何、紅色是未補償的點雲——兩側長牆明顯被拉斜，平均每點偏離真實位置 0.570 公尺。

圖(b) 是補償之後。做法正是 18.3 節所述：以等速度模型推算每個點量測時刻的位姿，把它反推回「幀起始時刻」的座標系。藍色點雲與灰色真值完全重合，平均誤差回到零。這裡要誠實說明：之所以能回到零，是因為補償用的速度正好等於模擬時使用的速度——這是理想狀況，用來驗證補償公式本身沒有寫錯。

真正有意思的是圖(c)。紅線是完全不補償時的平均點位誤差，它與速度成正比：1 公尺 / 秒時 0.062 公尺，6 公尺 / 秒時 0.405 公尺。黃線與藍線則是速度估計分別有 30% 與 10% 誤差時的殘餘誤差。三條線呈現一個乾淨的比例關係：補償後的殘餘誤差，大約等於「速度估計的誤差百分比」乘上「完全不補償時的誤差」。速度估計差 10%，殘餘誤差就是不補償時的 10%。

這條比例關係把 18.3 與 18.5 兩節扣在一起。它說明運動補償的品質完全取決於速度估計的品質，而速度估計正是 IMU 最擅長的事——數百赫茲的更新率，短時間內極為精確。這就是為什麼現代的光達系統幾乎都要配 IMU：不只是為了配準的初值，也是為了讓每一幀點雲在進入配準之前，就已經是一片沒有變形的乾淨資料。

**運動變形的三個實務重點**

速度越快、轉得越急，變形越嚴重；其中「轉」比「直行」影響更大，因為旋轉會讓遠處的點產生與距離成正比的位移。本圖在 6 m/s 時最大單點誤差達 2.36 公尺。

補償需要每個點的時間戳記。這正是第 17.1 節提到的「時間戳記」欄位的用途，也是案例 18-3 所說「缺少必要欄位就跑不起來」的原因之一。

固態光達沒有旋轉機構，但掃描仍需時間，一樣有變形問題，只是模式不同。

等速度模型是最簡單的假設。急加速或急轉彎時它會失準，這時 IMU 提供的實測角速度與加速度就比模型推算更可靠。

## 18.4 LeGO-LOAM、LIO-SAM、Fast-LIO 簡介

LOAM 開創了道路，後續十餘年間出現了大量改良系統。本節介紹三個最具代表性的後代，它們各自解決了 LOAM 的不同弱點。

### 一、LeGO-LOAM：輕量化與地面最佳化

LeGO-LOAM 全名為「輕量化且地面最佳化的光達里程計與建圖」，針對地面載具的即時 6 自由度位姿估計而設計。它輕量，能在低功耗嵌入式系統上達成即時位姿估計；它地面最佳化，因為它在分割與最佳化步驟中善用了地平面的存在。

它的兩項關鍵改良是：

- 點雲分割與地面分離：先進行點雲分割以濾除雜訊，再做特徵提取以取得具鑑別度的平面與邊緣特徵。分割能有效去除植被、雜訊等不穩定的點——這對戶外環境特別重要。
- 兩步最佳化：採用兩步驟的 Levenberg-Marquardt 最佳化，用平面與邊緣特徵分別求解 6 自由度變換的不同分量。具體而言，先用地面點提取的平面特徵求出垂直位移

與滾轉、俯仰角，再用分割後點雲的邊緣特徵估計水平位移與偏航角，並以前一步的結果作為約束。

實測顯示，LeGO-LOAM 在多變地形的地面載具資料集上，能以更低的運算成本達到與 LOAM 相當或更好的精度。它特別針對水平放置於地面載具上的 16 線光達最佳化，並假設掃描中總是存在地平面——這個假設限制了它的適用範圍，但也正是它高效的原因。

## 二、LIO-SAM：光達慣性緊耦合與因子圖

LIO-SAM 的創新在於以因子圖搭配貝氏樹進行 IMU 緊耦合，實現具運動感知的子地圖匹配，並整合考量漂移的閉環偵測。

- 緊耦合 IMU：把 IMU 預積分（第 7 章）作為因子加入圖中，大幅提升在快速運動下的穩健度。
- 因子圖架構：採用第 12 章介紹的 GTSAM 與 iSAM2 增量式求解，能高效地融合光達、IMU、GPS 與閉環等多種約束。
- 閉環偵測：其閉環功能改編自 LeGO-LOAM，是基於 ICP 的方法；由於 ICP 執行較慢，官方建議測試時把播放速度設為原速。

值得注意的實作限制：與原始 LOAM 實作相同，LIO-SAM 只能搭配九軸 IMU（能提供滾轉、俯仰與偏航估計），且目前僅支援機械式光達。這類細節在專題實作時非常關鍵，同學選用硬體前務必確認。

## 三、Fast-LIO 系列：效率的極致

Fast-LIO 走的是另一條路線：不做特徵提取，直接使用原始點雲，並以迭代擴展卡爾曼濾波（iEKF）進行緊耦合估測。它的關鍵創新是一種高效的卡爾曼增益計算公式，使運算複雜度與狀態維度而非量測數量相關，大幅提升速度。

Fast-LIO2 進一步引入增量式 KD 樹（ikd-Tree）資料結構，能高效地增量更新地圖，達成極高的處理頻率，特別適合運算資源受限的平台與快速運動的載具（如無人機）。這條

路線證明了：在 IMU 提供良好運動先驗的前提下，不做特徵提取也能又快又準。

| 系統 | 年份 | 關鍵改良 | 適用場景 |
|---|---|---|---|
| LOAM | 2014 | 特徵提取 + 雙層架構 | 通用，奠基之作 |
| LeGO-LOAM | 2018 | 地面分離、兩步最佳化、輕量化 | 地面載具、嵌入式平台 |
| LIO-SAM | 2020 | IMU 緊耦合、因子圖、多源融合 | 需高穩健度與閉環的場域 |
| Fast-LIO2 | 2021 | 免特徵、iEKF、增量式 KD 樹 | 運算受限平台、快速運動 |

**表 18-1　LOAM 系列主要系統比較**

## 18.5　光達 + IMU 緊耦合

第 18.4 節的三個系統中，有兩個都在強調 IMU。本節說明為什麼光達 SLAM 越來越離不開 IMU，以及緊耦合的意義。

### 光達單獨使用的三個弱點

- 快速運動下失效：純光達方法在快速動態運動下表現會劣化；掃描之間差異太大，配準難以收斂（第 13.3 節 ICP 需要好初值的老問題）。
- 幾何退化環境：在長走廊或開闊空間等幾何退化的環境中，不足的幾何約束會導致位姿估計失敗。這是第 17.3 節結構退化的再次出現。
- 運動變形：如 18.3 節所述，旋轉期間的載具運動會扭曲點雲，需要運動資訊才能補償。

### IMU 恰好補上這三個缺口

回顧第 7 章：IMU 更新率極高（數百 Hz）、短時間內極為精確、完全不受環境結構影響。它恰好能：

42. 在兩幀光達之間提供高頻運動估計，作為配準的優良初值，解決快速運動問題。

43. 在幾何退化的方向上提供獨立的約束——走廊裡光達說不清「走了多遠」，IMU 的加速度積分卻能給出估計。

44. 提供每個點量測時刻的精確姿態，用於運動變形補償。

反過來，光達也回報 IMU 它最需要的東西：不漂移的位姿觀測，用來持續估計並補償 IMU 的零偏（第 7.3 節）。光達慣性里程計（LIO）因而受到高度重視，它緊密融合光達量測與 IMU 資料，結合了光達對光線變化的穩健性與捕捉稠密幾何資訊的能力。

### 鬆耦合 vs 緊耦合

第 7.6 節在視覺慣性 SLAM 中討論過這組概念，在光達領域完全適用：

- 鬆耦合：光達 SLAM 與 IMU 各自算出位姿，再加權融合。實作簡單，但資訊已被壓縮，精度受限。
- 緊耦合：把 IMU 預積分約束與光達特徵配準誤差，放進同一個最佳化問題（或同一個濾波器）一起求解。精度顯著較佳，是現代系統的一致選擇。

LIO-SAM 以因子圖實現緊耦合（最佳化路線，第 12 章）、Fast-LIO 以 iEKF 實現緊耦合（濾波路線，第 11 章）——兩者殊途同歸，再次印證第 11、12 兩章正是 SLAM 的兩條主幹。

### 實作要點

- 時間同步：光達與 IMU 的時間戳記必須精確對齊，誤差幾毫秒就會顯著影響精度（第 5、10 章反覆強調的工程細節）。
- 外參校正：IMU 與光達的相對安裝位置與姿態必須準確量測（第 10 章的外參），否則融合結果會系統性偏差。
- IMU 規格：如前所述，部分系統要求九軸 IMU。此外 IMU 的品質（第 7 章的等級表）直接影響在退化場景中能撐多久。

## 18.6 自主巡檢機器人的 3D SLAM

3D 光達 SLAM 最重要的實際應用之一，就是自主巡檢機器人。本節說明這類系統的需求與設計考量。

## 為什麼巡檢需要 3D 而非 2D

- 地形起伏：巡檢機器人（尤其是機器狗）需要上下樓梯、跨越門檻、走過斜坡。2D 地圖無法描述高度變化，也無法判斷「這個台階跨得過去嗎」。
- 立體障礙：工廠的懸空管線、機房的機櫃上緣、樹枝，都不在 2D 掃描平面上。第 8 章的高度盲區在巡檢場域會直接造成碰撞。
- 巡檢目標的三維位置：要記錄「三樓東側牆面第二根管線有鏽蝕」，需要三維座標而非平面位置。
- 資產建模：巡檢的副產品往往是設施的三維模型（第 17.7 節的數位分身），本身就有價值。

## 典型系統架構

**圖 18-5　自主巡檢機器人的 3D SLAM 系統架構**

【感測層】3D 光達 ＋ IMU ＋ （可選）相機、GNSS、輪速計

↓

【前端】特徵提取（邊緣 / 平面）或直接點雲配準

↓

【緊耦合估測】LIO：光達約束 ＋ IMU 預積分 → 高頻 6-DoF 位姿

↓

【後端】因子圖最佳化 ＋ 閉環偵測（第 12 章）

↓

【地圖】3D 點雲地圖 → 轉八叉樹供導航使用（第 17.5 節）

↓

【應用層】路徑規劃、避障、巡檢點導航、影像/熱像資料採集

### 平台選擇：輪型、履帶、四足

- 輪型：效率最高、續航最好，但受限於平坦地面，無法上樓梯。適合工廠平面與走廊巡檢。
- 履帶型：可跨越較大障礙與斜坡，常見於災害現場與戶外巡檢。
- 四足（機器狗）：地形適應力最強，能上樓梯、跨障礙，是近年巡檢的熱門選擇。代價是續航較短、成本高，且劇烈的步態震動對 SLAM 是嚴峻考驗——這正是必須用 IMU 緊耦合的原因。

### 巡檢任務的特殊需求

- 重複性：巡檢是週期性任務，同一條路線每天走。因此「首次建圖、後續定位」的模式特別適用（第 14 章案例 14-3），且能比較不同時間的資料找出變化（例如新出現的裂縫）。
- 精確定位到巡檢點：機器人需要停在特定位置、以特定角度拍攝儀表或設備。定位誤差需控制在數公分，這是 3D SLAM 精度要求的來源。
- 長時間運行：巡檢任務可能持續數小時，對漂移控制與系統穩定性要求極高——閉環偵測因此不可或缺。
- 資料關聯：巡檢採集的影像、熱像、聲音資料，都需要標上精確的三維位置，才能在數位分身中定位問題。

## 18.7　LOAM 之後：2024–2026 年光達慣性里程計的四個走向

LOAM 發表於 2014 年，十餘年來它的兩大設計——特徵提取與雙層架構——一直是光達 SLAM 的標準骨架。但近兩年，這兩根支柱都受到了認真的挑戰，而挑戰的方向恰好都指向同一件事：運算資源變便宜了，所以當年為了即時性所做的妥協，現在未必還划算。本節整理四個走向。

### 一、特徵提取正在被放棄

第一個走向是 18.4 節介紹 Fast-LIO 時已經埋下的伏筆：不做特徵提取，直接用原始點雲。這條路線在近年幾乎成為主流。理由很現實——2014 年提取特徵是為了把數十萬點降到數千點，否則算不動；但在今天的處理器與 GPU 上，直接拿幾萬點做體素化的配準已經完全可行，而且少了特徵提取這一步，就少了一組需要調整的門檻，也少了「柱子太細被漏掉」這類失效模式。

這與第 17.8 節提到的 KISS-ICP 是同一股風潮：與其設計精巧的篩選規則，不如把最基本的配準做紮實。不過本章圖 18-2 的分析仍然沒有過時——免特徵的方法一樣要面對幾何退化，只是它不再用「邊緣 / 平面」這組語彙來描述，而是直接看體素分布的共變異數。概念是同

一個，換了個說法。

## 二、全域一致性回到前台：GLIM

第二個走向是對「雙層架構」本身的修正。LOAM 的低頻建圖雖然會修正漂移，但前端一旦把某一幀的位姿定下來，就不再回頭。產業技術總合研究所 Kenji Koide 團隊於 2024 年發表於《Robotics and Autonomous Systems》的 GLIM，主張這個「定下來就不改」的設計正是退化場景下崩潰的根源。

GLIM 的做法有兩個關鍵：其一是在里程計端採用固定延遲平滑（五秒視窗），讓最近幾秒的位姿持續被修正，而不是立即定案；其二是全域最佳化時直接最小化子地圖之間的配準誤差，而不是先把它們壓縮成位姿圖上的高斯約束。再加上 GPU 加速的體素化 GICP，整套系統在一般配備下每幀前處理 15.1 毫秒、里程計 29.3 毫秒。

效果在退化場景最明顯。該論文以固態光達在八段退化序列上測試，GLIM 的絕對軌跡誤差落在 0.017 到 0.389 公尺之間，而 FAST-LIO2 的範圍是 0.297 到 15.551 公尺——最差的一段差了四十倍。這個對比值得同學記住：在條件良好時各家系統差距不大，真正拉開差距的永遠是最壞情況。

## 三、退化不再只是被動承受，而是主動偵測

第三個走向直接承接本章圖 18-2 的分析。既然退化可以用資訊矩陣的最小特徵值量化，那就可以即時算出來：一旦發現某個方向的約束不足，系統就自動降低對光達的信任、提高對 IMU 的權重，甚至在該方向上乾脆不更新。這類「退化感知」的光達慣性里程計是 2025 年前後相當活躍的題目。

對專題而言，這提供了一個門檻不高但很有價值的方向：不必自己寫一套完整的 LIO，只要在既有系統上加一個資訊矩陣的監看器，把最小特徵值隨時間畫出來，再對照機器人實際走過的場域，就能做出一份「這棟建築哪裡會讓 SLAM 失效」的地圖。這對第 19 章的巡檢專題直接有用。

## 四、從光達慣性到光達慣性視覺

第四個走向是把相機也拉進同一個緊耦合框架。FAST-LIVO2 一類的系統在 Fast-LIO 的迭代卡爾曼濾波架構上，再加入影像的直接光度誤差，形成光達、慣性、視覺三者的緊耦合。動機與第二篇的感測器互補分析完全一致：光達給幾何、相機給紋理與語意、IMU 給高頻運動，三者的失效模式幾乎不重疊。

特別值得注意的是它對退化場景的意義。長走廊讓光達說不清「走了多遠」，但牆上的海報、門牌、地板紋理對相機而言是清楚的特徵——這正好補上圖 18-2(c) 裡那個 0.011 的空缺。換句話說，第六篇即將介紹的視覺 SLAM，不只是另一條平行的路線，也是本章這套系統在退化場景下的救援方案。

> **四個走向的共同線索**
>
> 運算面（免特徵）：當年為即時性而做的篩選，在今天的硬體上未必划算。
>
> 架構面（GLIM）：前端不該把位姿立刻定死，保留修正空間才撐得過退化區段。
>
> 診斷面（退化感知）：退化從「事後才發現的災難」變成「可即時量測的指標」。
>
> 感測面（光達慣性視覺）：單一感測器補不了的洞，就用失效模式不重疊的另一種來補。
>
> 共同線索：本章的曲率、互補約束與運動補償三樣基本功沒有被淘汰，它們反而成了判斷新系統好壞的標準——任何新方法都要回答同樣的三個問題：哪些點有用、哪個方向沒約束、以及掃描期間車子動了多少。

## 承先啟後

本章完成了 3D 光達 SLAM 的核心技術。從第 17 章的點雲處理到本章的 LOAM 系列，同學已經具備在三維世界中定位建圖的完整知識：圖 18-1 說明哪些點值得留下，圖 18-2 說明為什麼兩種特徵缺一不可，圖 18-4 說明掃描期間的運動必須被補償。這三件事也正是所有後續系統——LeGO-LOAM、LIO-SAM、Fast-LIO、GLIM——各自改良的起點。

但圖 18-2(c) 那個 0.011 的數字，是本章無法靠自己解決的問題：當環境的幾何本身提供不了約束，再好的光達演算法也變不出資訊。18.5 節給了第一個答案——IMU，而第六篇將給出第二個答案——相機。

在那之前，下一章要先回答另一個問題：這些技術放進一台每天要跑一年的機器人裡，會遇到什麼？第 19 章把 3D SLAM 帶進自主巡檢的場域，處理巡檢點排序、重複定位精度與長期地圖維護這三件「演算法課本不教、但決定系統成敗」的事——而本章圖 18-2 的退化分析，到了那裡會變成某個巡檢點每兩次就拍歪一次的具體後果。

## 公式或流程

### 一、局部粗糙度（曲率）

對第 i 個點 $p_i$，取同一掃描線上鄰近的點集合 S，其粗糙度定義為：

$$c = (1 / (|S| \cdot \|p_i\|)) \cdot \| \Sigma_{j \in S} (p_i - p_j) \|$$

式子的意義是：把 $p_i$ 與所有鄰居的位置差向量加總。若 $p_i$ 位於平坦處，左右鄰居的差向量會互相抵消，c 值很小；若位於尖角，差向量同向疊加，c 值很大。分母的 $\|p_i\|$ 用來正規化距離的影響（遠處點間距大，不應因此被誤判為邊緣）。

- c 大於門檻 → 邊緣特徵
- c 小於門檻 → 平面特徵

### 二、點到線與點到面的距離

邊緣特徵的誤差為「點到對應邊線的垂直距離」。設當前邊緣點為 p，對應邊線由地圖中兩點 a、b 決定：

$$d_{edge} = \| (p - a) \times (p - b) \| / \| a - b \|$$

分子是三點構成的平行四邊形面積（以外積計算），除以底邊長即得高——也就是點到直線的距離。

平面特徵的誤差為「點到對應平面的垂直距離」。設平面由地圖中三點 a、b、c 決定，其法向量 n = （b - a）×（c - a）：

$$d_plane = |(p - a) \cdot n| / ||n||$$

## 三、LOAM 的最佳化目標

把所有特徵點的誤差組合起來，求位姿變換 T 使總誤差最小：

$$T^* = \text{argmin} \left( \Sigma\, d_edge(T \cdot p_i)^2 + \Sigma\, d_plane(T \cdot p_j)^2 \right)$$

這是一個非線性最小平方問題，以 Levenberg-Marquardt 法迭代求解（第 12 章）。同學可以看出：從第 12 章的抽象理論、第 13 章的 2D 掃描匹配、第 17 章的 ICP，到本章的 LOAM，都是同一套數學在不同資料上的應用。

## 四、LOAM 完整處理流程

**圖 18-6　LOAM 每一幀的處理流程**

① 接收一幀 3D 點雲（含每點的時間戳記與線號）

↓

② 運動變形補償：依運動估計把各點反推回幀起始時刻

↓

③ 計算每點的局部粗糙度 c

↓

④ 分區選取特徵：每個子區段挑出粗糙度最高（邊緣）與最低（平面）的點

↓

⑤【里程計，10 Hz】與上一幀特徵匹配 → 快速估出相對運動

↓

⑥【建圖，1 Hz】與全域地圖特徵匹配 → 精細位姿，並更新地圖

↓

⑦ 整合兩者輸出 → 約 10 Hz 的精確位姿 ＋ 全域 3D 地圖

## 案例

### 案例 18-1　為什麼只用百分之一的點，反而更準

初學者聽到 LOAM 「只用少數特徵點配準」時，直覺反應往往是：丟掉這麼多資料，精度不會下降嗎？答案出乎意料——不但不會，往往還更準。

原因有三。其一，大部分點是「冗餘」的：一面平坦的牆上有上萬個點，但它們提供的幾何約束其實只有一個——「你離這面牆多遠」。用一萬個點或一百個點來表達這個約束，結果幾乎相同，但運算量差了一百倍。

其二，有些點是「有害」的：植被的葉子、飄動的塑膠袋、走動的人，這些點在下一幀就不在原位了，強行配準只會把誤差引入。LeGO-LOAM 的點雲分割步驟正是為了濾除這類不穩定的點。

其三，特徵點是「精選」的：曲率極高或極低的點，恰好是最容易在下一幀被正確辨識出來的點——這與第 9 章視覺 SLAM 偏愛角點的道理完全相同（「白牆上找不回任何一個點」）。

這個案例給同學一個超越技術的啟示：更多資料不等於更好的結果，關鍵在於資料的品質與代表性。這個原則在機器學習、統計、工程判斷上處處適用。

## 案例 18-2　地面的價值：LeGO-LOAM 的巧思

LeGO-LOAM 的名字裡有「Ground-Optimized（地面最佳化）」，這個看似樸素的設計其實非常聰明。

思考一下：對一台在地面行駛的機器人而言，地面是什麼？它是一個永遠存在、極為穩定、面積巨大的平面特徵。而且它提供的約束非常明確——地面能精準地告訴機器人「你離地多高、機身有沒有前後傾或左右傾」，也就是垂直位移與滾轉、俯仰這三個自由度。

LeGO-LOAM 因此把 6 個自由度拆成兩組分別求解：先用地面點的平面特徵求出垂直位移與滾轉、俯仰角，再用其他邊緣特徵估計水平位移與偏航角，並把前一步的結果當作約束。這樣做的好處是：每一步要解的未知數變少，問題更好收斂、運算更快、也更穩定。

這個案例展示了一種重要的工程思維：善用場域的先驗知識。地面載具「一定在地上」是一個免費的、可靠的假設，不利用它就浪費了。當然代價是通用性——這個假設在無人機上就完全不成立。這再次呼應第 15 章的觀察：技術的設計取向，由應用需求決定。

## 案例 18-3　實作的魔鬼細節：硬體規格的坑

許多同學在專題中興沖沖地下載 LIO-SAM 或 LeGO-LOAM，卻卡在「跑不起來」。原

因往往不是演算法，而是硬體規格不符。以下是幾個真實的坑：

- IMU 軸數不足：與原始 LOAM 實作相同，LIO-SAM 只能搭配九軸 IMU（能提供滾轉、俯仰與偏航估計）。買了六軸 IMU 的同學會直接卡關。
- 光達類型不支援：目前僅支援機械式光達。固態光達（第 8 章）的資料格式不同，需要額外適配。
- 缺少必要欄位：系統需要每點的時間戳記與線號（ring）。官方文件明確說明：當光達以 10 Hz 旋轉時，每點的時間戳記應在 0 到 0.1 秒之間；線號則指出該點屬於感測器的哪一個通道，用來正確地組織點雲。若驅動程式沒輸出這些欄位，就必須自行改寫。
- 特定光達最佳化：LeGO-LOAM 特別針對水平放置於地面載具上的 16 線光達最佳化，並假設掃描中總是存在地平面。用在其他配置上效果會打折。

這個案例的教學意義在於：開源系統不是「下載就能用」。閱讀官方文件的硬體需求章節，是專題開始前必做的功課。這個習慣能省下數週的挫折——這也是本書每章都詳述「前置條件」與「常見失敗」的原因。

## 實作活動

### 實作 18-1　光達里程計與特徵提取

目的：親手實作曲率特徵提取、觀察邊緣與平面特徵的分布，並執行完整的光達里程計系統，理解漂移與閉環的作用。本實作以公開資料集為主，不需 3D 光達硬體。

#### 第一部分：特徵提取實作（全體）

41. 下載一段公開的 3D 光達資料（如 KITTI 或 LeGO-LOAM 官方提供的校園資料集），取出單一幀點雲。
42. 用 Python 實作公式一的粗糙度計算：對每個點，取同一線號上左右各數個鄰居，計算 c 值。

43. 依 c 值分類：取最高的 2% 為邊緣特徵、最低的 20% 為平面特徵，用不同顏色顯示（可用 Open3D），觀察它們分別落在環境的什麼位置。

44. 驗證直覺：邊緣特徵是否集中在柱子、樹幹、牆角？平面特徵是否集中在地面與牆面？若不是，調整鄰居數量與門檻再試。

45. 統計特徵點數佔原始點數的比例，體會 LOAM 的減量效果。

**第二部分：執行光達里程計（全體或大學部）**

46. 在 ROS 環境中安裝 LeGO-LOAM 或 A-LOAM（注意檢查案例 18-3 的硬體與資料格式需求）。

47. 播放資料集，在 RViz 中即時觀察：特徵點雲、估計軌跡、逐步累積的 3D 地圖。

48. 觀察漂移：讓資料播放完一個大迴路，比較起點與終點的位置差距，量化累積漂移。

49. 開啟閉環偵測後重跑一次，觀察閉環觸發時整條軌跡與地圖如何被拉正（第 12、15 章的震撼時刻）。

**第三部分：系統比較與退化實驗（大學部）**

50. 在同一段資料上分別執行 LeGO-LOAM、LIO-SAM（若有 IMU 資料）與 Fast-LIO，比較三者的軌跡精度、運算負擔與地圖品質。

51. 用第 17 章案例 17-1 的「牆厚度」方法，量化比較三張地圖的品質。

52. 退化場景實驗：找一段以長走廊為主的資料（或自行採集），比較純光達與光達加 IMU 的表現差異，驗證 18.5 節的論述。

53. 若有 IMU 資料，刻意關閉 IMU 融合再跑一次，量化 IMU 帶來的改善幅度。

**討論問題**

14. 你的特徵點佔原始點數的百分之幾？若把這個比例提高一倍，配準精度會提升嗎？運算時間增加多少？

15. 觀察你提取的特徵：在植被茂密或雜亂的區域，特徵分布合理嗎？這印證了 LeGO-LOAM 為何要先做點雲分割？

16. 大迴路的累積漂移是多少公尺？佔總行走距離的百分之幾？閉環修正後改善了多少？

17. （大學部）在長走廊場景，哪一個方向的誤差最大？這與第 13.6 節的 2D 走廊困境、第 17.3 節的結構退化是否為同一問題？

18. （大學部）依表 18-1，若要為一台大學的巡檢機器狗選系統，你會選哪一個？考量因素有哪些？

**延伸挑戰（大學部）**

其一：實作一個最簡化的光達里程計——只用平面特徵，以點到面 ICP（第 17 章）連續配準相鄰兩幀並累加位姿，把軌跡與資料集真值比較，繪製誤差隨距離成長的曲線。接著加入邊緣特徵，觀察精度改善多少，親身驗證 18.2 節「兩種特徵互補」的論述。其二：採集一段大學走廊的實測資料（依第 19.8 節階段一的做法），在其上執行本章的系統並評估品質，作為第 31 章專題四的前導研究。

## 延伸閱讀

22. J. Zhang and S. Singh, "LOAM: Lidar Odometry and Mapping in Real-time," Robotics: Science and Systems, 2014.（LOAM 原始論文，本章主要依據，CMU 機器人研究所出版）

23. T. Shan and B. Englot, "LeGO-LOAM: Lightweight and Ground-Optimized Lidar Odometry and Mapping on Variable Terrain," IROS, 2018.（地面最佳化與兩步最佳化的原始論文）

24. T. Shan et al., "LIO-SAM: Tightly-coupled Lidar Inertial Odometry via Smoothing and Mapping," IROS, 2020.（因子圖緊耦合的代表作）

25. W. Xu and F. Zhang, "FAST-LIO: A Fast, Robust LiDAR-inertial Odometry Package by Tightly-Coupled Iterated Kalman Filter," IEEE RA-L, 2021.（以及 FAST-LIO2, IEEE T-RO, 2022）

26. LeGO-LOAM 與 LIO-SAM 開源專案（GitHub）.（含官方資料集、硬體需求說明與參數設定，實作必讀）

27. A-LOAM 開源專案（GitHub, HKUST Aerial Robotics）.（LOAM 的簡潔重實作，程式碼易讀，適合教學研讀）

28. KITTI Vision Benchmark Suite 官方網站.（自駕車 3D 光達的標準評測資料集，含真值軌跡）

## 小結

22. LOAM 由 Zhang 與 Singh 於 2014 年提出，核心洞見是「不需要用上所有的點」；它把 SLAM 拆成里程計與建圖兩個平行任務，藉由提取邊緣與平面特徵，在結構化環境中以低運算成本達成高精度，成為後續光達 SLAM 研究的共同基礎。

23. 特徵以局部粗糙度（曲率）分類：高曲率為邊緣特徵（柱子、牆角）、低曲率為平面特徵（牆面、地面）；兩者提供互補的幾何約束——平面約束垂直於面的移動、邊緣約束垂直於線的移動，缺一則自由度無法完全鎖定。

24. 雙層架構是 LOAM 的第二項創新：高頻里程計（約 10 Hz）提供即時位姿與良好初值，低頻建圖（約 1 Hz）與全域地圖精細配準以消除漂移，兩者整合後輸出高頻且精確的位姿。此外必須做運動變形補償。

25. LeGO-LOAM 以點雲分割濾除雜訊、善用地平面做兩步 Levenberg-Marquardt 最佳化，在低功耗嵌入式系統上即時運行，以更低運算成本達到相當或更好的精度，但假設掃描中總是存在地平面。

26. LIO-SAM 以因子圖搭配貝氏樹實現 IMU 緊耦合與漂移感知的閉環整合；Fast-LIO 系列則走免特徵提取、以迭代擴展卡爾曼濾波緊耦合的路線，適合運算受限平台與快速運動。

27. 純光達在快速運動與幾何退化環境（長廊、開闊地）會失效，IMU 恰好補上這三個缺口（提供初值、退化方向的獨立約束、運動變形補償），而光達回報以不漂移的觀測校正 IMU 零偏；緊耦合優於鬆耦合。

28. 自主巡檢需要 3D 而非 2D，因為要處理地形起伏、立體障礙、三維巡檢目標與資產建模；平台可選輪型、履帶或四足，而四足的步態震動使 IMU 緊耦合成為必要。

本章完成了 3D 光達 SLAM 的核心技術。從第 17 章的點雲處理到本章的 LOAM 系列，同學已經具備在三維世界中定位建圖的完整知識。下一章我們把這些技術帶進真實場域——第 19 章的自主巡檢機器人，處理巡檢點排序、重複定位精度與長期地圖維護這三件決定系統成敗的事，並以此為第五篇作結。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

## 第 19 章

## 自主巡檢機器人與場域實務

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程　/　高中 AI 機器人社團課程

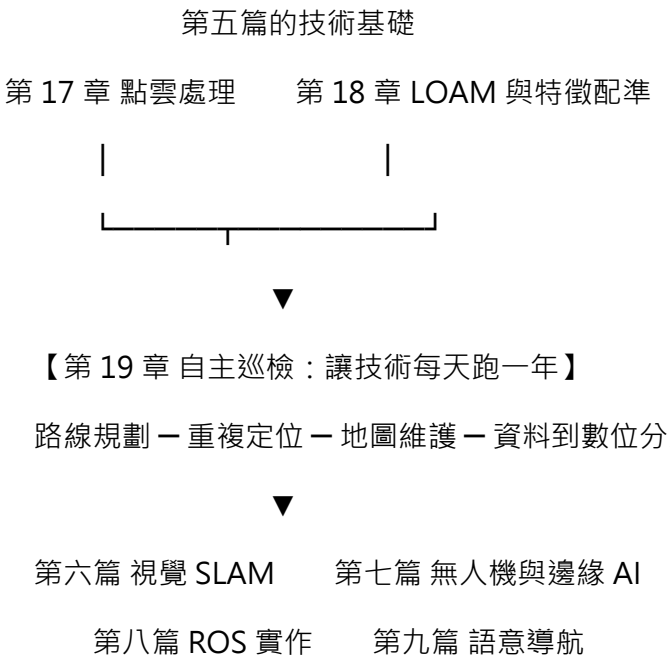
## 本章在全書的位置

第 17 與 18 章把三維光達 SLAM 的技術備齊了：點雲怎麼處理、特徵怎麼挑、位姿怎麼估、掃描期間的運動怎麼補償。本章要問的是另一個問題——當這些技術必須每天在同一棟建築裡自己跑上三小時、連續跑一年，會發生什麼事。

答案是：技術指標不再是唯一的重點。一台巡檢機器人的成敗，往往不取決於它的 SLAM 有多準，而取決於三件更瑣碎的事——巡檢點的順序排得好不好、每天能不能停在同一個位置、以及三個月前建的地圖今天還能不能用。這三件事正是本章 19.4、19.5、19.6 三節的主題，也是本章三張實驗圖要回答的問題。

本章在全書的角色，與第 16 章的掃地機器人對稱：第 16 章是 2D SLAM 的落地應用，本章則是 3D SLAM 的落地應用。兩章都在示範同一件事——把演算法放進每天要用的產品裡，真正的難題才開始。

### 圖 19-0　第 19 章的連結地圖




## 學習目標

巡檢是 3D SLAM 最成熟的商業應用，也是最能把本書前十八章串起來的場域。本章從需求

出發，依序處理巡檢機器人的技術堆疊、平台選型、路線規劃、重複定位與長期維護，最後以台灣的研究與產業現況，以及一個可執行的校園專題作結。讀完本章，同學應能：

1. 說明巡檢自主化要解決的問題，以及它與一般導航任務的根本差異。
2. 說明巡檢機器人的技術堆疊，並指出每一層對應本書的哪些章節。
3. 比較輪型、履帶、四足與無人機四種平台的適用場域與代價。
4. 說明巡檢點路線規劃的問題本質，並能估算排序對續航的影響。
5. 解釋重複定位精度為何是巡檢的關鍵指標，以及它與場域幾何的關係。
6. 說明長期運行下地圖維護的必要性，並能設計地圖更新策略。
7. 舉例說明台灣在巡檢機器人上的研究與產業成果，並據以規劃一個校園巡檢專題。

## 核心概念

| 名詞 | 意義 | 關聯章節 |
|---|---|---|
| 巡檢點（Waypoint） | 機器人必須抵達並以特定姿態採集資料的位置 | 19.4、19.5 |
| 重複定位精度 | 多次回到同一巡檢點時，位姿的離散程度 | 19.5、第 11 章 |
| 覆蓋與巡檢的差異 | 覆蓋要走遍每一格；巡檢只需走訪指定的點 | 19.4、第 16 章 |
| 路線最佳化 | 在巡檢點之間找出最短的巡迴順序（旅行推銷員問題） | 19.4 |
| 地圖維護 | 依環境變動更新先驗地圖，維持長期可定位 | 19.6、第 13 章 |
| 多期 SLAM | 跨日期的多趟軌跡合併為同一張地圖 | 19.6、第 15 章 |
| 數位分身 | 以巡檢資料建立的三維模型與資產紀錄 | 19.7、第 17 章 |
| 退化場域 | 幾何約束不足、定位易失敗的區域（長走廊、空曠大廳） | 19.5、第 17、18 章 |

# 19.1　為什麼巡檢要自主化

巡檢是一項古老而乏味的工作：有人拿著紀錄表，每天沿著固定路線走一圈，看儀表讀數、聽設備聲音、摸管線溫度、確認門窗與消防設備。這件事看似簡單，卻有三個難以靠人力克服的問題。

## 人工巡檢的三個結構性問題

- 一致性：人會疲勞、會分心、會「昨天看過今天就跳過」。同一個巡檢點由不同人、在不同時間檢查，標準很難一致。
- 頻率：人力有限，巡檢頻率往往被壓到最低。一週兩次的巡檢，代表設備最長可能有三天處在無人察覺的異常狀態。
- 環境風險：變電所、高壓設備、地下涵洞、災後建築、輻射區——有些地方本來就不該讓人常去。

自主巡檢機器人正面回應了這三點。它的紀錄完全一致、可以每天甚至每小時跑一趟、而且可以進入人不該進的地方。更重要的是，它把巡檢從「發現故障」升級為「預測故障」——因為只有在資料夠密、夠一致時，趨勢才看得出來。

## 巡檢與一般導航的四個差異

同學容易把巡檢當成「導航 + 拍照」，但兩者的工程重點其實相當不同：

| 面向 | 一般導航 | 巡檢任務 |
|---|---|---|
| 路線 | 每次可能不同，求抵達 | 每天相同，求重複 |
| 定位要求 | 公尺級即可安全行走 | 公分級，否則拍不到同一個角度 |
| 地圖 | 跑一趟建一次 | 建一次用一年，必須維護 |
| 成功標準 | 有沒有到 | 資料有沒有可比性 |

其中「重複」兩個字是關鍵。巡檢資料的價值來自於跨時間比較——今天的紅外熱像要和

上週的比，才知道溫度是不是在上升。若兩次拍攝的位置與角度差了半公尺，影像就無法直接比對，所有的趨勢分析都失去基礎。這就是 19.5 節要處理的問題。

## 19.2　巡檢機器人的技術堆疊

一台巡檢機器人是本書前十八章的總和。本節由下而上拆解它的五層結構，並指出每一層對應哪些章節。

| 層級 | 內容 | 本書章節 |
|---|---|---|
| 感測層 | 3D 光達、IMU、輪速編碼器、RGB 與紅外熱像、麥克風 | 第 7、8、9 章 |
| 定位建圖層 | 3D SLAM 建圖、先驗地圖定位、閉環修正 | 第 17、18 章 |
| 導航層 | 全域路徑規劃、區域避障、巡檢點抵達控制 | 第 3 章、第八篇 |
| 任務層 | 巡檢點管理、路線排程、電量與回充決策 | 19.4、19.6 |
| 資料層 | 資料標註位置與時間、異常偵測、數位分身與報表 | 19.7、第九篇 |

### 巡檢任務層：本章新增的一層

前三層在第 17、18 章與第八篇都有完整處理，本章真正新增的是任務層與資料層。任務層要回答的問題聽起來很行政，實際上全是最佳化問題：十二個巡檢點該用什麼順序走？電量只剩四成時要先回充還是先跑完？某個點今天定位失敗了，要重試幾次才放棄？

這些決策的品質，直接決定了機器人每天能完成多少工作。19.4 節會用一個實驗說明：光是把巡檢點的順序排對，就能省下四成五的行走距離——而行走距離就是電池，電池就是一天能跑幾趟。

## 19.3　平台選型：輪型、履帶、四足、無人機

巡檢機器人沒有通用平台，選型幾乎完全由場域決定。本節比較四種主流選擇。

| 平台 | 優勢 | 限制 | 典型場域 |
|---|---|---|---|
| 輪型 | 續航最佳、成本最低、SLAM 最穩定 | 無法上樓梯、越障能力差 | 廠房、走廊、機房、變電所 |
| 履帶 | 越障與斜坡能力佳、穩定度高 | 體積重量大、轉彎磨耗地面 | 戶外、災後現場、工地 |
| 四足 | 地形適應力最強、可上樓梯 | 續航短、成本高、步態震動不利 SLAM | 多樓層、涵洞、不平地面 |
| 無人機 | 可達高處與橋下、速度快 | 續航以分鐘計、室內定位困難 | 橋梁、塔架、大型結構外部 |

### 四足平台的 SLAM 難題

四足機器人近年成為巡檢的熱門選擇，但它對 SLAM 是嚴峻考驗。原因是步態震動：每一步落地都讓機身產生高頻的俯仰與滾轉，而第 18.3 節談過的運動變形補償，在這種非等速的劇烈運動下特別容易失準。這也是為什麼四足平台幾乎一定要搭配高品質 IMU 與緊耦合的光達慣性里程計（第 18.5 節）。

**選型的三個提問**

場域有沒有高低差？有樓梯或門檻就排除輪型。

一趟巡檢要多久？超過一小時，四足與無人機的續航就會成為瓶頸。

巡檢對象在哪個高度？在天花板或橋下，地面平台再強也看不到。

最後一問最實際：這個場域的 SLAM 會不會退化？長走廊與空曠大廳（第 17.3 節、圖 18-2）對任何平台都是難題，必須先確認再採購。

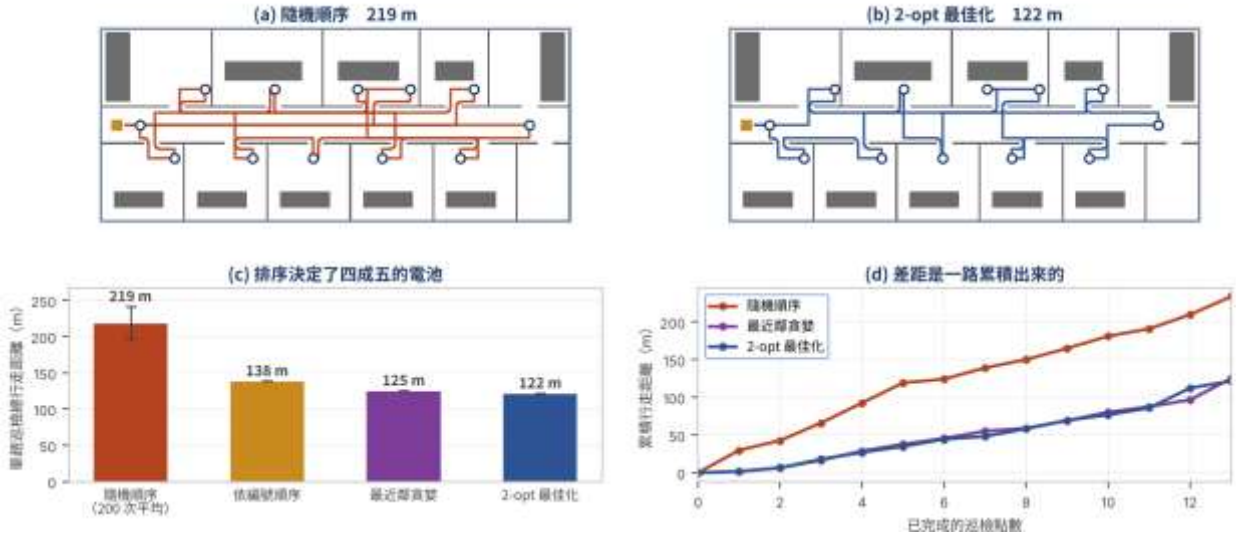
## 19.4　巡檢點規劃與路線最佳化

巡檢與第 16 章的掃地任務有一個根本差異：掃地要「走遍每一格」（覆蓋路徑規劃），巡

檢只需「走訪指定的點」。這個差異把問題從覆蓋問題換成了旅行推銷員問題——給定一組必須造訪的點，求最短的巡迴順序。

這個問題在理論上是困難的（隨點數成階乘成長），但在巡檢的實務規模下非常好處理。一層樓通常只有十幾個巡檢點，用最近鄰貪婪法得到初始解、再以 2-opt 反覆改良，幾毫秒內就能得到接近最佳的路線。

**圖 19-1　巡檢點排序：一條路線決定了四成五的電池**




本書以一層 34×14 公尺的系館樓層做實驗：一條貫穿的走廊、上下兩排教室與實驗室、一座電梯井與一間樓梯間，沿途設定十二個巡檢點——電梯門、門禁讀卡機、配電箱、消防栓、空調主機與實驗室門口。機器人從走廊左端的充電座出發，走訪全部十二點後回到充電座。

圖(a) 是使用者在 App 上隨手點選巡檢點、未經排序的結果：路徑在走廊上來回穿梭，總長 219 公尺。圖(b) 是同樣十二個點、經 2-opt 最佳化後的路線：機器人沿走廊單向前進，順路進出兩側房間，總長 121.5 公尺。兩者造訪的點完全相同，差別只在順序。

圖(c) 把四種排序並列。隨機順序（200 次平均）219.3 公尺，依編號順序 138.5 公尺，最近鄰貪婪 125.2 公尺，2-opt 最佳化 121.5 公尺。從隨機到最佳化省下 45%，而從貪婪再到 2-opt 只多省 3%——這說明一件很實用的事：排序的大部分效益來自「別亂走」，而不是

來自精巧的演算法。一個十行的最近鄰法就能拿走九成以上的好處。

圖(d) 顯示差距是如何累積的。三條曲線在前兩三個點幾乎重疊，但隨機順序每多走一個點就多付一次代價，到第十二點時已經落後近百公尺。換算成實務影響：以 0.5 公尺 / 秒的巡檢速度計，這條多出來的路要多花三分多鐘，而若機器人一天要跑八趟，就是每天多耗 26 分鐘的電池與馬達壽命。

**實作上的三個修正**

本實驗的距離是走廊與房間的實際可通行最短路徑（以 BFS 在佔據格上計算），不是直線距離。用直線距離排序會在有隔牆的樓層嚴重失真。

真實巡檢還有順序限制：某些點必須在特定時間拍攝（例如上班前的門禁），這屬於帶時間窗的旅行推銷員問題，需在最佳化時加入限制。

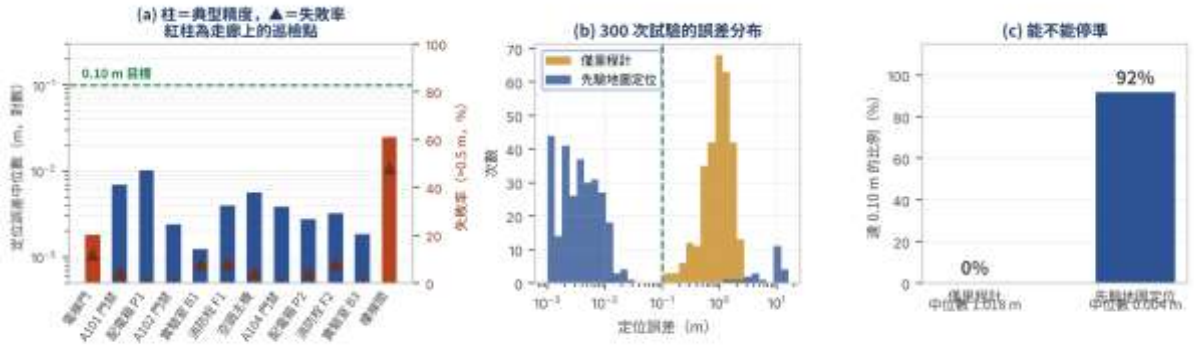
跨樓層時電梯是瓶頸節點，等待時間往往比行走時間長，應把「同一層的點一次走完」設為硬性限制。

電量不足以跑完全程時，問題變成「在有限里程下最大化巡檢價值」，此時需要為每個巡檢點設定優先權。

## 19.5　重複定位精度：能不能每天停在同一個位置

這是巡檢最關鍵、也最容易被低估的指標。機器人要拍攝同一個儀表、比對同一面牆的熱像，就必須每天停在同一個位置、用同一個角度。業界常見的要求是數公分——這遠高於一般導航「不要撞到東西」的公尺級需求。

**圖 19-2　同一個巡檢點、重複 25 次：停得準不準**




實驗設定如下：機器人從充電座出發抵達某個巡檢點，途中里程計累積了約 1 公尺的誤差（相當於 60 公尺路程、1.5% 的漂移率）。此時有兩種選擇——直接相信里程計，或以光達掃描比對先驗地圖重新定位。十二個巡檢點各重複 25 次，共 300 次試驗。

圖(b) 的兩個分布說明了一切。僅靠里程計時，誤差集中在 1 公尺附近，沒有任何一次達到 0.10 公尺的目標；改用先驗地圖定位後，絕大多數試驗落在 1 公分以下，中位數 0.004 公尺，92% 的試驗達標。圖(c) 把這個對比濃縮成一個數字：0% 對 92%。

但圖(a) 才是本節真正的重點。它逐點顯示定位精度，而同學會立刻注意到：藍色的房間內巡檢點，誤差中位數在 1 到 6 公釐之間，失敗率 0 到 8%；而紅色的走廊巡檢點——電梯門與樓梯間——明顯較差，其中樓梯間的失敗率高達 48%，接近一半的嘗試會定位失敗。

原因同學應該已經很熟悉了：長走廊沿軸向缺乏幾何約束（第 17.3 節的結構退化、圖 18-2 的資訊矩陣分析）。機器人知道自己貼著哪一面牆，卻說不清沿走廊走了多遠。這個在第 17、18 章還只是理論限制的現象，到了巡檢場域就變成具體的產品缺陷：那個巡檢點每兩次就有一次拍歪。

這帶出一個重要的設計原則：巡檢點的位置不該只由「要檢查什麼」決定，也要考慮「那裡定位得準不準」。把巡檢點從走廊中央往門口挪兩公尺，讓機器人能同時看到門框與兩側牆面，往往比升級感測器更有效，而且不用花錢。

**提高重複定位精度的四個手段**

移動巡檢點：優先選擇幾何特徵豐富的位置（轉角、門框、設備密集處）。

加裝人工地標：在退化區域貼上 AprilTag 或反光標籤（第 4 章），用極低的成本提供絕對位置參考。

多感測器互補：在走廊這類光達退化的地方，相機的視覺特徵（第六篇）或 UWB 標籤（第 4 章）可以補上缺少的那個方向。

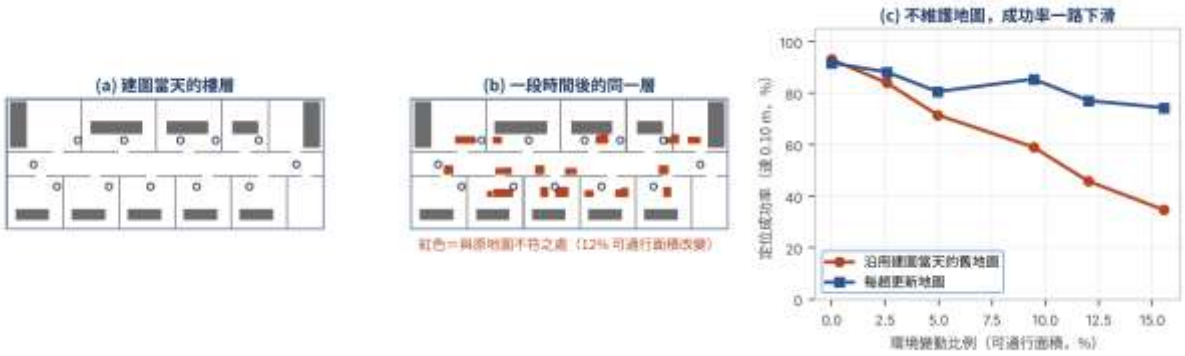
接受並處理失敗：定位信心不足時，讓機器人後退一公尺重新嘗試，或標記該筆資料為「位置不可靠」，而不是拍一張沒人知道拍歪了的照片。

## 19.6　長期運行：地圖維護與環境變動

前兩節處理的是「今天這一趟」，本節要問的是「一年後還能不能跑」。這是巡檢與一次性建圖任務最大的差別：地圖不是成果，而是需要維護的資產。

真實場域會變。桌椅被搬動、走廊堆了紙箱、實驗室換了設備、門今天開著明天關著。這些變動單獨看都很小，累積起來卻足以讓定位失效——因為掃描匹配比對的正是這些東西。

**圖 19-3　地圖會過期：環境變動對定位成功率的影響**




圖(a) 是建圖當天的樓層，圖(b) 是一段時間後的同一層——紅色標出與原地圖不符之處：房間裡多出的桌椅、走廊上暫置的物品，合計改變了約 12% 的可通行面積。從人的角度看，這層樓「還是那層樓」；但從掃描匹配的角度看，有八分之一的觀測對不上。

圖(c) 是量化的結果，也是本章最該記住的一張圖。紅線是沿用建圖當天舊地圖的定位成功率：環境變動 0% 時 93%，變動 5% 時降到 72%，變動 15% 時只剩 35%——不到一年，系統就廢了一半。藍線則是每趟巡檢後更新地圖：即使環境變動 15%，成功率仍維持在 74%。

兩條線的分岔說明了地圖維護不是錦上添花，而是巡檢系統能否長期運作的分水嶺。值得注意的是，藍線也不是水平的——它從 92% 緩降到 74%，因為環境變動越大，暫時性的物品（紙箱、推車）在地圖裡占比越高，而這些東西明天就不在了。這正是第 15.8 節提過的終身建圖問題：地圖既要「記得」永久結構，又要「忘記」臨時障礙。

### 三種地圖維護策略

- 定期重建：每隔固定期間（例如每季）重新建一次圖。做法最簡單，但重建期間系統無法運作，且會丟失歷史對應關係。
- 增量更新：每趟巡檢後把新觀測併入地圖，以佔據機率的上下限控制舊資訊的遺忘速度（第 13.5 節）。本實驗的藍線即屬此類。
- 分層地圖：把地圖分成「永久層」（牆、柱、門框）與「暫態層」（桌椅、堆置物）。定位只用永久層，避障則兩層都用。這是目前商用系統最常見的做法，也最能同時滿足記得與遺忘。

## 19.7　巡檢機器人的功能與產業應用

前面六節處理的是「怎麼走到定點」。但一台巡檢機器人到了定點之後要做什麼、能做到什麼程度，才是使用者真正買單的理由。本節把巡檢機器人的功能攤開來看，再依場域說明不同產業的需求差異，最後介紹這條產業鏈由哪些角色組成。

### 一、巡檢機器人的核心功能

巡檢機器人的功能可分為「移動」「採集」「判讀」「管理」四類。以下清單整理了目前商用機型普遍具備或正在導入的能力，並標出每一項背後所需的技術與本書對應章節。

| 類別 | 功能 | 所需技術 | 本書章節 |
|---|---|---|---|
| 移動 | 自主導航與避障 | 3D SLAM、路徑規劃、代價地圖 | 第 17、18 章、第八篇 |
| 移動 | 定點停靠（公分級重複定位） | 先驗地圖定位、精定位控制 | 19.5、第 11 章 |
| 移動 | 自動回充與續航管理 | 電量模型、排程決策 | 19.4、19.6 |
| 移動 | 跨樓層移動（電梯整合） | 拓撲地圖、設備通訊介面 | 19.4、19.8 |
| 採集 | 可見光影像定點拍攝 | 雲台控制、曝光補償 | 第 9 章 |
| 採集 | 紅外熱像量測 | 熱像儀、溫度標定 | 第 9 章、19.7 |
| 採集 | 異音擷取與頻譜記錄 | 麥克風陣列、聲學分析 | 19.7 |
| 採集 | 氣體、溫濕度與輻射偵測 | 環境感測器整合 | 19.7 |
| 判讀 | 儀表指針與數字讀取 | 影像辨識、OCR | 第九篇 |
| 判讀 | 設備外觀缺陷偵測 | 物件偵測、異常偵測 | 第九篇 |
| 判讀 | 熱點與溫升趨勢分析 | 跨時間資料比對 | 19.5、19.6 |
| 管理 | 資料標註三維座標與時間 | 位姿紀錄、時間同步 | 第 10 章、19.6 |
| 管理 | 異常分級與工單派送 | 規則引擎、後台整合 | 19.7 |
| 管理 | 數位分身與資產紀錄 | 三維重建、點雲顏色化 | 第 17.5、17.6 節 |

同學可以從這張表看出一個清楚的分界：「移動」與「採集」兩類功能，目前的商用機型已經相當成熟，而且技術基礎全部落在本書第一至八篇；「判讀」一類則是現階段的前沿，多數機型仍只負責把資料傳回後台由人判讀，自動判讀的能力正在逐步導入——這正是第九篇要處理的主題。

## 二、不同場域的功能需求差異

巡檢機器人沒有通用規格。同樣是「巡檢」，電力場域與地下管道所需的功能幾乎沒有重疊。下表整理四種常見場域的差異，可作為專題選題或需求書撰寫的起點。

| 場域 | 主要巡檢對象 | 關鍵功能 | 環境難題 |
|---|---|---|---|
| 電力設施 | 變壓器、開關、電容器、接頭 | 紅外熱像、異音偵測、儀表判讀 | 戶外全天候、電磁干擾、高壓安全距離 |
| 橋梁與結構 | 裂縫、鏽蝕、鋼纜索力 | 高解析影像、三維定位、表面正射影像 | 無衛星訊號、高空作業、需達次毫米解析度 |
| 廠房與倉儲 | 管線、機台、消防設備、料架 | 定點影像、溫度、氣體偵測 | 人車混流、環境每日變動、產線不可停 |
| 地下管道與涵洞 | 結構變形、積水、氣體 | 防水防塵、氣體偵測、照明 | 無光源、狹窄、通訊中斷、無法救援 |

其中橋梁檢測的需求最能說明「功能決定平台」。要辨識 0.2 至 0.3 公釐的裂縫，代表載具必須能貼近結構表面、相機必須有足夠解析度；而橋下與結構內部收不到衛星訊號，載具又必須完全依靠自身的 SLAM 維持位置。這兩個要求互相拉扯——飛得越近定位越困難——正是結構檢測無人機最核心的工程難題，也是第五篇與第七篇的交會點。

## 三、量化效益：功能如何轉換成價值

巡檢機器人的採購決策通常不看技術指標，而看三個可量化的效益。以下數字來自國內已實際部署的案例，可作為估算的參考。

- 巡檢頻率：國內某電力事業單位導入戶外巡檢機器人後，同一座變電所的巡檢頻率由人員每週兩次提升為機器人每日一次。這代表異常被發現的最長延遲從三天縮短為一天，也讓預測性維護第一次有足夠密集的資料基礎。
- 運算資源：國內某法人研究機構報告，其為巡檢機器人開發的導航演算法僅占用機器人運算資源的 25%，而傳統演算法需要 65% 至 70%。對一台要同時跑導航與影像

辨識的機器人而言，省下的運算資源直接決定了它能不能即時反應。

- 檢測涵蓋率：以橋梁為例，國內車行橋梁超過 2.3 萬座，主管機關的目標是 100% 檢測。這個數量級使人工檢測在本質上不可行，也說明自動化在某些場域不是效率問題，而是可行性問題。

## 四、產業鏈由哪些角色組成

巡檢機器人是少數在台灣具備完整供應鏈的機器人應用。以下依行業別說明各環節的角色，讓同學了解這個產業的樣貌與可能的就業方向。

| 環節 | 行業別 | 在巡檢機器人中的角色 |
|---|---|---|
| 感測元件 | 光達製造業 | 提供 2D / 3D 光達，決定建圖品質與成本 |
| 感測元件 | 光學與熱像模組業 | 可見光相機、紅外熱像儀、雲台 |
| 驅動元件 | 電機與馬達製造業 | 輪組、關節馬達，決定續航與越障能力 |
| 控制元件 | 工業控制器與機器人控制器業 | 運動控制、即時系統 |
| 運算平台 | 邊緣運算與嵌入式系統業 | 車載電腦，決定能跑多複雜的演算法 |
| 整機整合 | 電子代工與系統整合業 | 機構設計、整機組裝、量產與驗證 |
| 軟體與演算法 | 機器人軟體與 AI 服務業 | SLAM、導航、影像辨識、後台平台 |
| 研究法人 | 工業技術研究法人 | 平台技術開發、產業串聯、場域驗證 |
| 終端用戶 | 公用事業、製造業、交通主管機關 | 提出需求、提供驗證場域、採購導入 |

值得注意的是「研究法人」這個環節。在台灣，法人研究機構往往扮演平台開發與產業串聯的角色：由法人設計本體與 AI 核心，再由各行業的廠商供應零組件與整機整合，最後由公用事業或製造業提供場域驗證。這種分工讓資源有限的單一廠商也能參與高複雜度的機器人

開發，也是同學日後若投入這個領域最可能接觸到的合作模式。

**從功能到規格：寫一份巡檢需求書**

先定義巡檢對象與判讀標準：要看什麼、看到什麼程度算異常。這一步決定了相機解析度、熱像精度與停靠精度的要求。

再定義場域條件：有無高低差、是否戶外、有無衛星訊號、環境多久變動一次。這一步決定平台型式與地圖維護策略。

然後定義頻率與續航：一天幾趟、一趟多久、充電要多久。這一步決定電池容量與路線最佳化的重要性。

最後才談技術規格。先寫需求再挑規格，是避免「買了最貴的感測器卻解決不了問題」最有效的方法——這也是第 8 章感測器選型討論的延伸。

## 19.8　場域專題：南部科技大學的走廊、電梯與實驗室

本節把前面七節落實成一個可執行的專題規劃。場域設定為台灣南部一所科技大學的系館——這類校園是極佳的巡檢教學場域：有長走廊（退化場景）、有電梯與樓梯（跨樓層）、有實驗室（貴重設備與門禁）、也有真實的管理需求。

### 場域分析：三種環境的不同挑戰

| 場域 | 主要挑戰 | 技術對策 | 對應章節 |
|---|---|---|---|
| 長走廊 | 沿軸向幾何退化、定位失敗率高 | IMU 緊耦合、人工地標、巡檢點改設於門口 | 第 17.3、18.5、19.5 |
| 電梯與樓梯間 | 跨樓層、地圖不連續、金屬反射 | 分層地圖、電梯作為拓撲節點、四足或輪型分工 | 19.4、19.6 |
| 實驗室 | 環境每日變動、人員走動、設備更換 | 分層地圖、動態物體剔除、每趟增量更新 | 19.6、第 13.5 |

| | | | |
|---|---|---|---|
| 大廳與中庭 | 空曠、遠處回波稀疏、玻璃帷幕 | 降低對光達的依賴、視覺輔助、縮短巡檢點間距 | 第 8 章、第六篇 |

## 五階段專題規劃

1. 階段 1——資料採集：以手推車或機器人載具搭載 3D 光達與 IMU，分別在走廊、樓梯間與實驗室錄製資料集。這一步不需要自主導航，人推著走即可，適合作為大學部的入門作業。
2. 階段 2——離線建圖與比較：以錄好的資料分別執行 LeGO-LOAM、LIO-SAM 與 Fast-LIO（第 18.4 節），比較三者在各場域的軌跡精度、地圖品質與運算負擔，並以第 17 章案例的「牆厚度」方法量化地圖品質。
3. 階段 3——退化場景實驗：專門針對長走廊，比較純光達與光達加 IMU 緊耦合的差異，量化 IMU 帶來的改善。這是最有研究價值的一環，也直接對應本章圖 19-2(a) 那兩根紅色長柱。
4. 階段 4——巡檢點設定與路線最佳化：在建好的地圖上設定十至十五個巡檢點，實作本章 19.4 節的最近鄰加 2-opt 規劃，並實際量測排序前後的行走距離與耗時，重現圖 19-1。
5. 階段 5——長期運行與地圖維護：連續數週每日執行巡檢，記錄每個巡檢點的定位成功率，觀察它如何隨環境變動而下降，再導入增量更新或分層地圖，量化改善幅度。這一階段可延伸為研究所題目。

## 教學與研究價值

這個專題的優點是層次分明：階段一、二適合大學部課程作業，階段三、四適合專題生，階段五可延伸為碩士論文。而且成果具體可展示——一份校園的三維模型、一套每日運行的巡檢紀錄，對外說明時遠比任何投影片有說服力。

更重要的是，它把本書前十八章的內容全部用上了：第 8 章選光達、第 7 章配 IMU、第 10

章校外參、第 17 章處理點雲、第 18 章跑 LOAM、第 12 章做閉環最佳化，再加上本章的路線規劃、重複定位與地圖維護。一個做完這五階段的學生，已經具備進入業界巡檢機器人團隊的完整能力。

## 公式或流程

### 一、巡檢任務的三個評估指標

巡檢系統的品質可用三個指標衡量，三者互相拉扯：

巡檢完成率 = 成功採集資料的巡檢點數 / 總巡檢點數

重複定位精度 = 多次造訪同一巡檢點時，位姿的標準差

單趟耗時 = 總行走距離 / 平均速度 + Σ 各點的採集時間

提高完成率往往要增加重試次數，因而拉長單趟耗時；而縮短耗時又可能犧牲定位精度。工程上的取捨正是產品設計的核心，與第 16.4 節掃地機器人的三個指標完全同構。

### 二、巡檢路線的最佳化問題

設巡檢點集合為 $P = \{ p_1, \dots, p_n \}$，充電座為 $p_0$，$d(i, j)$ 為兩點之間的可通行最短路徑長度（非直線距離），求一個排列 $\pi$ 使總行走距離最小：

$$\min \; d(p_0, p_{\pi(1)}) + \sum_{k=1}^{n-1} d(p_{\pi(k)}, p_{\pi(k+1)}) + d(p_{\pi(n)}, p_0)$$

這是旅行推銷員問題。實務解法為兩步：先以最近鄰貪婪法求初始解，再以 2-opt 反覆交換任兩段路徑的順序，直到無法再改善。本章圖 19-1 的實驗顯示，貪婪法已能取得九成以上的效益。

### 三、地圖維護的佔據機率更新

增量更新的核心是讓舊資訊能被新觀測覆蓋。沿用第 13.2 節的對數勝算比，但為其設定上下限：

$l_t = \text{clamp}(l_{t-1} + l_{meas}, -l_{max}, +l_{max})$

上下限 $l_{max}$ 決定了地圖的「可塑性」：$l_{max}$ 越大，地圖越相信歷史、越不容易被新觀測改變；$l_{max}$ 越小，地圖越容易遺忘。長期巡檢系統通常取較小的 $l_{max}$，以換取對環境變動的適應力。

## 四、巡檢機器人的一日流程

| 階段 | 動作 |
|---|---|
| ① 啟動 | 自充電座出發，以先驗地圖執行全域重定位（第 11 章） |
| ② 規劃 | 依巡檢點清單與電量，計算當趟路線（19.4 節） |
| ③ 巡檢 | 前往各巡檢點 → 精定位至公分級 → 採集影像、熱像與聲音 |
| ④ 監看 | 即時記錄定位信心；信心不足時重試或標記資料不可靠 |
| ⑤ 回充 | 電量門檻觸發後返回充電座，並續跑未完成的點 |
| ⑥ 更新 | 把當趟掃描併入地圖（增量更新或分層地圖，19.6 節） |
| ⑦ 報表 | 資料標上三維座標與時間，輸出異常清單與趨勢圖（第九篇） |

# 案例

## 案例 19-1　從每週兩次到每天一次：巡檢頻率改變了什麼

國內某電力事業單位自 2017 年起投入智慧巡檢機器人研究，與法人研究機構及國內機器人業者合作，歷時約三年完成一台設計與製造全由國內廠商負責的戶外巡檢機器人，部署於一座二次變電所。

它的感測配置幾乎是本書第二篇的完整實作：3D 光達建立周遭三維地形、3D 相機偵測障礙避免碰撞、雲台上的雙影像模組（含紅外熱像）擔任「眼睛」、麥克風陣列擷取設備聲音、天線支援遠端遙控。它沿固定路線行走、在設定點位拍攝，任務結束後返回專屬小屋無線充電——正是 19.4 與 19.5 兩節所描述的巡檢點與重複定位需求。

但這個案例真正的價值不在硬體，而在制度上的改變：同一座變電所的巡檢頻率，從人員每週兩次提升為機器人每日一次。這個看似平凡的數字意味著異常被發現的最長延遲從三天縮短為一天，也讓「預測性維護」第一次有了足夠密集的資料基礎——例如變壓器接頭開始劣化時，可以在故障發生前排定停電更換。

這個案例也誠實呈現了當時的限制：機器人只負責傳回影像，尚無法自行辨識設備或判讀聲音，後續規劃才是導入 AI 判讀儀表指針與偵測變壓器異音。這個「先解決位置、再解決理解」的順序，正好就是本書的結構——第一至八篇處理「在哪裡」，第九篇才處理「那是什麼」。

## 案例 19-2　當巡檢對象有兩萬三千座

國內車行橋梁超過 2.3 萬座，而主管機關的目標是全部檢測。這個數字本身就說明了自動化的必要性——若每座橋的人工檢測需要數天，全國檢測一輪就是以年計的工程。這不是效率問題，而是可行性問題。

國內研究團隊發展的做法是把問題拆成兩半：先以無人機把橋「數位化」，建立三維模型並製作表面正射影像；再讓 AI 在影像上找裂縫與劣化，並把發現的缺陷定位回三維模型上。系統還能自動偵測被遮蔽的區域、以額外的影像補償提高覆蓋率——這與第 16 章覆蓋路徑規劃的思路一致，只是從地面搬到了空中。

兩個技術限制值得同學記住。其一是定位：橋下與結構內部收不到衛星訊號，無人機必須完全依靠自身的 SLAM 維持位置，這正是第五篇與第七篇的交集。其二是解析度：要辨識 0.2 至 0.3 公釐的裂縫，代表無人機必須飛得夠近、相機必須夠好，而飛得近又讓定位更困難。這組矛盾是結構檢測無人機的核心工程問題。

另一條路線是放棄通用平台，改用專用機構。針對鋼纜檢測，國內工程顧問機構開發了爬索車：適用鋼纜直徑最大 300 公釐、機體重 13.5 公斤、爬升角度 60 度以上、爬升速度每秒 40 公分以上，在同一根鋼纜上裝設多點感測器，直接量測模態、頻率與索力，取代傳統的

震動法。這提醒同學：巡檢不一定要用會走路的機器人，針對單一任務設計的專用機構往往更可靠也更便宜。

### 案例 19-3　國產四足平台：產業分工與模擬先行

2026 年 7 月，國內主管機關公布了國產四足機器人研發平台的成果，由法人研究機構主導，歷時約一年半建成兩台國產原型機。這個案例的價值不在單一技術，而在它揭示的產業結構。

參與分工相當清楚：電子代工業者負責整機整合、視覺系統業者提供影像酬載、電機業者供應關節馬達、光達業者提供感測元件、控制器業者負責運動控制，法人研究機構則包辦本體設計、AI 核心、模擬環境與慣性量測單元。換句話說，從感測到驅動、從控制器到整機，整條供應鏈都在國內，而平台採開放架構，讓其他團隊能在其上開發應用。

已驗證的應用包括地下涵洞巡檢、自主導航巡檢、群體協作與火場救災——前三項正是本章的主題，而火場救災則呼應第七篇的極端環境討論。法人機構預期第一波需求將出現在工業巡檢、半導體高載重搬運與物流。

最值得同學注意的是訓練方式：團隊以超級電腦搭配通用模擬平台，同時平行模擬四千多隻虛擬機器狗進行 AI 訓練，再部署到實機。這個「先模擬、再部署」的流程將是第 33 章的完整主題，而它已經是國家級專案的標準做法，不再是研究室裡的實驗。

## 實作活動

### 實作 19-1　巡檢路線規劃與重複定位評估

目的：以程式實作巡檢任務層的兩個核心問題——路線最佳化與重複定位評估——並在模擬或真實場域中量測它們的影響。本實作以 Python 為主，不需 3D 光達即可完成第一、二部分。

#### 第一部分：路線最佳化（全體）

1. 取一張樓層平面圖（可用第 13 章實作產生的佔據格地圖，或自行以方格紙定義），

在上面標出 10 至 15 個巡檢點與一個充電座位置。

2. 以 BFS 或 A* 計算任兩點之間的可通行最短路徑長度，建立距離矩陣。請特別注意：必須用可通行距離，不能用直線距離。

3. 實作最近鄰貪婪法求初始巡迴順序，計算總距離。

4. 實作 2-opt 改良：反覆嘗試反轉路線中的任一段，若總距離變短就接受，直到無法再改善。記錄改善幅度。

5. 與隨機順序（取 100 次平均）比較，重現圖 19-1(c) 的四根長條，並計算排序為你的場域省下多少距離。

**第二部分：重複定位評估（全體或大學部）**

1. 在模擬環境（或真實機器人）中，讓機器人從同一起點前往同一個巡檢點 20 次，每次加入不同的里程計雜訊。

2. 記錄兩種條件下的最終位姿誤差：僅用里程計，以及用掃描比對先驗地圖重新定位。

3. 畫出兩者的誤差分布（建議用對數橫軸），計算中位數與達標率，重現圖 19-2(b)(c)。

4. 換到不同的巡檢點重做，特別挑一個位於長走廊中央的點與一個位於房間轉角的點，比較兩者的失敗率，驗證 19.5 節關於場域幾何的論述。

5. 依結果調整巡檢點位置：把表現最差的點移動一至兩公尺，重測並記錄改善幅度。

**第三部分：長期地圖維護（大學部與專題生）**

1. 在模擬環境中隨機加入數個新物件，模擬環境變動，並計算可通行面積的改變比例。

2. 以原始地圖執行定位，記錄成功率；再以更新後的地圖執行，比較兩者，重現圖 19-3(c)。

3. 實作分層地圖：把連續多趟都觀測到的格子標為永久層，只出現一兩趟的標為暫態層。定位僅使用永久層，比較它與全圖定位的成功率。

4. （進階）在真實場域連續一週每日建圖，觀察哪些區域最常變動，畫出一張「變動

熱區圖」，作為巡檢點設置與地圖維護策略的依據。

### 討論問題

1. 在你的場域中，2-opt 相對於隨機順序省下多少距離？若機器人電池能跑 2 公里，排序最佳化讓它每天多跑幾趟？
2. 哪一個巡檢點的重複定位最差？它周圍的幾何有什麼特徵？這與第 17.3 節的結構退化、圖 18-2 的資訊矩陣分析是同一回事嗎？
3. 若把巡檢點從走廊中央移到門口兩公尺處，定位改善了多少？這個「免費」的改善，相對於升級感測器的成本，說明了什麼？
4. 環境變動到什麼比例時，舊地圖的定位成功率開始明顯下降？依這個比例推估，你的場域多久需要更新一次地圖？
5. （大學部）依 19.3 節的平台比較，若要為你的系館選一台巡檢機器人，你會選哪一種？跨樓層的需求會如何改變你的選擇？

### 延伸挑戰（大學部）

其一：實作帶時間窗的巡檢排程——某些巡檢點必須在指定時段造訪（例如門禁需在上班前、空調需在運轉時），把這個限制加入 2-opt 的接受條件，比較有無限制時的路線長度差異。其二：設計一個「定位信心監看器」，即時計算掃描匹配的資訊矩陣最小特徵值（圖 18-2 的方法），當它低於門檻時主動標記該筆巡檢資料，並把整層樓的信心值畫成熱區圖，找出場域中所有的退化區域。

## 延伸閱讀

- 國內電力事業戶外巡檢機器人相關報導.（國產戶外巡檢機器人的完整案例，含感測配置與巡檢頻率的改變）
- 國內法人研究機構之巡檢機器人與國產四足機器人研發平台相關發布.（巡檢機器人的產業生態系與分工）

- 國內橋梁檢測無人機與 AI 劣化辨識系統相關報導.（結構檢測的自動化路線與技術限制）
- T. Shan and B. Englot, "LeGO-LOAM," IROS 2018；T. Shan et al., "LIO-SAM," IROS 2020.（巡檢機器人最常用的兩套 3D SLAM 系統，第 18 章延伸）
- S. Macenski et al., slam_toolbox 與 Nav2 官方文件.（地圖序列化、純定位模式與導航堆疊，巡檢系統的軟體基礎）
- D. Applegate et al., The Traveling Salesman Problem: A Computational Study, Princeton Univ. Press.（路線最佳化的權威參考，2-opt 與更進階方法）
- 長期定位與地圖維護的相關文獻（lifelong mapping、multi-session SLAM）.（本章 19.6 節的理論基礎）

## 小結

- 人工巡檢有一致性、頻率與環境風險三個結構性問題；自主巡檢機器人以完全一致的紀錄、可提高的頻率與可進入危險場域回應這三點，並把巡檢從「發現故障」升級為「預測故障」。
- 巡檢與一般導航的四個差異是：路線求重複而非抵達、定位要求公分級而非公尺級、地圖要維護而非用完即丟、成功標準是資料的可比性。
- 技術堆疊分為感測、定位建圖、導航、任務與資料五層；前三層由第 17、18 章與第八篇處理，本章新增的是任務層（巡檢點與路線）與資料層（數位分身與報表）。
- 平台選型由場域決定：輪型續航最佳但無法上樓梯、履帶越障佳、四足地形適應最強但續航短且步態震動不利 SLAM、無人機可達高處但續航以分鐘計。
- 巡檢是旅行推銷員問題而非覆蓋問題。實驗顯示：隨機順序 219 公尺、最近鄰貪婪 125 公尺、2-opt 最佳化 121.5 公尺，排序省下 45%，而其中九成效益來自簡單的貪婪法。

- 重複定位精度是巡檢最關鍵的指標。僅靠里程計無法達到 0.10 公尺目標（0%），以先驗地圖定位可達 92%、中位數 0.004 公尺；但走廊上的巡檢點因結構退化，失敗率可高達 48%——巡檢點的位置應同時考慮「要檢查什麼」與「那裡定位得準不準」。
- 地圖是需要維護的資產。沿用舊地圖時，定位成功率隨環境變動從 93% 降到 35%；每趟更新地圖則可維持在 74% 以上。三種維護策略為定期重建、增量更新與分層地圖。
- 巡檢機器人的功能可分為移動、採集、判讀、管理四類；移動與採集已相當成熟，判讀仍以傳回後台由人判讀為主，自動判讀是現階段的前沿（第九篇）。
- 不同場域的功能需求差異極大：電力設施重熱像與異音、橋梁結構需次毫米解析度與無衛星定位、廠房倉儲面對每日變動、地下管道則需防水防塵與氣體偵測。
- 國內巡檢機器人具備完整供應鏈，由光達、光學、馬達、控制器、邊緣運算、整機整合、軟體演算法、研究法人與終端用戶共同組成；已實際部署的案例顯示，巡檢頻率可由每週兩次提升為每日一次。

## 承先啟後

第五篇到此完成。從第 17 章的點雲處理、第 18 章的 LOAM 與特徵配準，到本章把這些技術放進每天要跑一年的巡檢系統裡，同學應該能看出一條清楚的線索：演算法決定了系統的上限，而任務層的瑣碎決策決定了它實際能發揮多少。

本章圖 19-2(a) 那兩根特別長的紅柱，也再次指出光達的極限——在幾何退化的長走廊裡，再好的三維光達也說不清自己走了多遠。下一篇我們換一種感測器：從第 20 章起，改用相機「看」。視覺 SLAM 有它自己的難題（白牆、重複紋理、尺度不確定性），但它恰好能在光達束手無策的地方派上用場——牆上的海報、門牌與地板紋理，對相機而言都是清楚的特徵。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第六篇 Visual SLAM 與視覺導航機器人

## 第 20 章

## Visual SLAM 入門

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

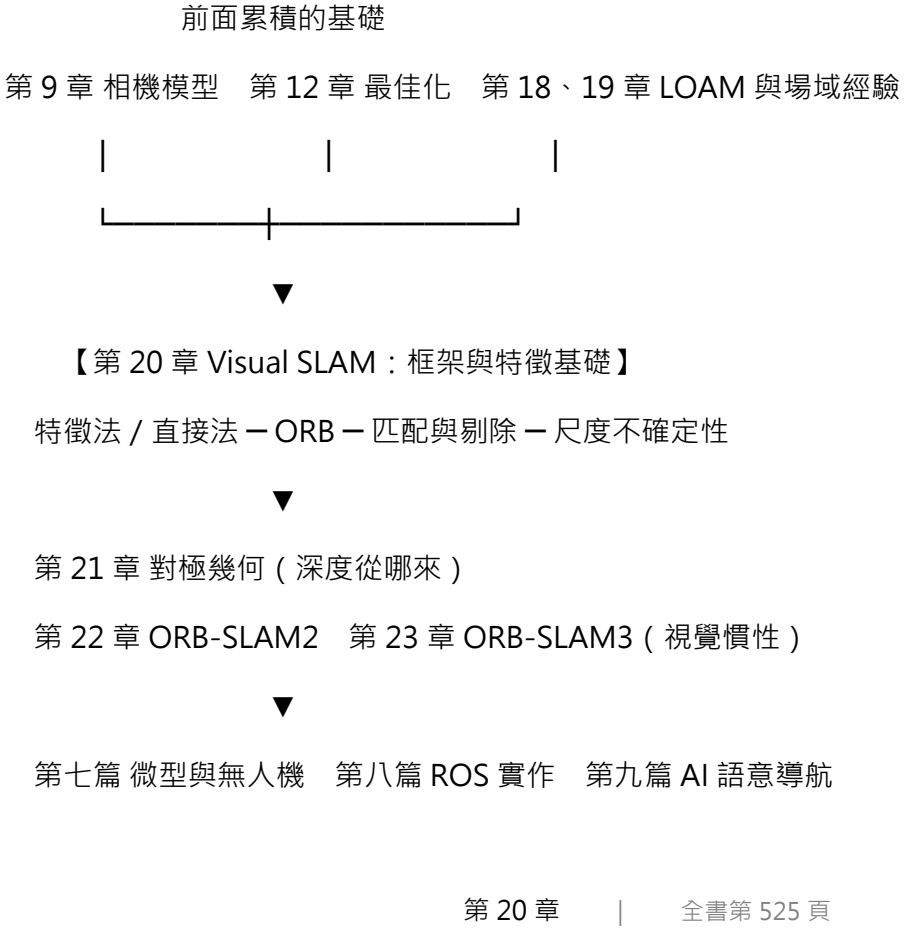
大學智慧機器人學程 / 高中 AI 機器人社團課程

## 本章在全書的位置

第六篇換了感測器，但沒有換思路。第五篇用雷射「量」出三維世界，並在第 19 章把那套技術送進了每天要跑一年的巡檢現場；本篇改用相機「看」。然而同學很快會發現，LOAM 的四個步驟——提取特徵、前端快速估計、後端精細最佳化、閉環修正——在視覺 SLAM 裡一步都沒少。差別只在於特徵的性質：光達的特徵是幾何的，一出生就帶著三維座標；相機的特徵是外觀的，只有二維像素位置，深度得另外想辦法。

這個差別造就了本篇的兩個主題。其一是「怎麼挑特徵、怎麼配對」，也就是本章 20.3 與 20.4 節的內容；其二是「沒有深度怎麼辦」，這是 20.5 節的尺度不確定性，也是第 21 章對極幾何要正面回答的問題。另外值得先放在心上的是：第 19.5 節量到的那件事——長走廊因為幾何退化而讓光達說不清自己走了多遠——恰恰是相機的主場，因為牆上的海報、門牌與地板紋理都是清楚的視覺特徵。在那之前，本章先把整個框架搭起來，讓同學知道後面四章各自在補哪一塊。

### 圖 20-0　第 20 章的連結地圖



## 學習目標

第五篇我們用雷射「量」出了三維世界；從本篇起，改用相機「看」。感測器換了，但同學會發現核心思想完全相通——特徵、配準、最佳化、閉環，只是從幾何點雲換成了影像像素。本章是第六篇的入口，建立視覺 SLAM 的整體框架與特徵技術基礎。學完本章之後，同學應能夠達成下列目標：

22. 說明視覺 SLAM 的基本流程，並指出各階段對應本書的哪些章節。
23. 比較特徵法與直接法兩大流派的原理、優缺點與適用場合。
24. 說明 ORB、SIFT、SURF、FAST、BRIEF 的角色與特性，並理解為何 ORB 成為 SLAM 的主流選擇。
25. 區分特徵追蹤與特徵匹配的差異，並說明常見的誤匹配剔除策略。
26. 解釋單目 SLAM 的尺度不確定性，以及三種解決途徑。
27. 比較雙目 SLAM 與 RGB-D SLAM 的差異與適用場景。
28. 舉例說明視覺 SLAM 在 AR / VR 與機器人中的應用。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
| --- | --- | --- |
| 視覺 SLAM | Visual SLAM | 以相機為主要感測器的同步定位與建圖。 |
| 視覺里程計 | Visual Odometry | 由連續影像估計相機運動，會累積漂移。 |
| 特徵法 | Feature-based / Indirect | 提取特徵點並匹配，再由幾何關係求解運動。 |
| 直接法 | Direct Method | 不提特徵，直接以像素亮度差估計運動。 |
| 關鍵點 | Keypoint | 影像中顯著且可重複偵測的位置，如角點。 |
| 描述子 | Descriptor | 描述關鍵點周圍外觀的向量，用於比對。 |

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 尺度不變性 | Scale Invariance | 物體遠近改變時仍能偵測到同一特徵的能力。 |
| 旋轉不變性 | Rotation Invariance | 影像旋轉後仍能正確匹配的能力。 |
| 漢明距離 | Hamming Distance | 兩個二元描述子相異位元的個數，比對極快。 |
| 特徵匹配 | Feature Matching | 在兩張影像間找出對應的同一特徵點。 |
| 特徵追蹤 | Feature Tracking | 在連續影像中持續追蹤同一特徵的位置。 |
| 尺度不確定性 | Scale Ambiguity | 單目相機無法確定地圖真實大小的先天限制。 |
| 詞袋模型 | Bag of Words | 把影像表示為視覺單詞的統計，用於地點辨識。 |

## 20.1 視覺 SLAM 的基本流程

第 9.6 節我們畫過一條知識階梯：影像處理 → 特徵提取 → 多視圖幾何 → 視覺里程計 → 視覺 SLAM。本節把這條階梯展開成完整的系統流程。

### 五個階段

- 感測器資料讀取與預處理：讀取影像、去畸變（第 9.2 節的相機校正）。若是雙目或 RGB-D，還需做立體匹配或深度對齊。
- 視覺里程計（前端）：估計相鄰影像之間的相機運動，並三角測量出局部的三維點。這一階段快速但會累積漂移（第 18 章的光達里程計是它的雷射版本）。
- 後端最佳化：對一批位姿與地圖點做非線性最佳化（第 12 章的 Bundle Adjustment），消除累積誤差。
- 閉環偵測：辨識「這個地方我來過」，產生閉環約束以修正全域漂移（第 12、15 章）。視覺 SLAM 常用詞袋模型來做這件事。
- 建圖：依應用需求輸出地圖——稀疏的特徵點地圖、稠密的點雲、或語意地圖（第 1、3 章）。

**圖 20-1　視覺 SLAM 的標準流程**

影像輸入（單目 / 雙目 / RGB-D）

↓

① 預處理：去畸變、立體匹配（第 9 章）

↓

② 視覺里程計（前端）：特徵提取與匹配 → 估計相機運動（第 21 章）

↓

③ 後端最佳化：Bundle Adjustment 精修位姿與地圖點（第 12 章）

↓

④ 閉環偵測：詞袋模型辨識重訪地點 → 全域修正（第 22 章）

↓

⑤ 建圖輸出：稀疏點雲 / 稠密地圖 / 語意地圖（第 32 章）

### 與光達 SLAM 的對照

把這個流程與第 18 章的 LOAM 並排，會發現架構驚人地一致：兩者都是「特徵提取 → 前端快速估計 → 後端精細最佳化 → 閉環修正」。差別只在於特徵的性質：

45. 光達的特徵是幾何的（邊緣、平面），直接帶有三維座標與距離。
46. 視覺的特徵是外觀的（角點與其周圍紋理），只有二維像素座標，深度需要另外求得。

這個差別導致了視覺 SLAM 的核心難題：如何從「沒有深度的二維觀測」中，恢復出三維的相機運動與場景結構。這正是第 21 章對極幾何要回答的問題。

## 20.2　特徵法與直接法

視覺 SLAM 有兩條主要技術路線，理解它們的分歧，就理解了這個領域的地形。

### 特徵法（Feature-based，又稱間接法）

特徵法的流程是三個步驟：

- 偵測：在影像中找出角點等關鍵點。
- 描述：為每個關鍵點計算描述子，記錄它周圍長什麼樣。
- 匹配：依描述子的相似度，在不同影像間找出對應的點。

特徵法在視覺里程計中被廣泛採用，因為它對相機運動、光線變化與動態物體具有穩健性。它之所以稱為「間接」法，是因為它不直接使用像素值，而是先把影像抽象成「一組特徵點」這個中間表示，再用幾何方法求解運動。

47. 優點：對光線變化穩健（描述子多有一定的亮度不變性）、可處理較大的視角變化、易於做閉環偵測（用描述子做詞袋比對）、成熟穩定。
48. 缺點：計算特徵與描述子耗時、丟棄了大部分像素資訊、在紋理稀少處（白牆）找不到特徵而失效（第 9 章的「白牆詛咒」）。

### 直接法（Direct Method）

直接法完全跳過特徵提取，直接使用像素亮度。它的核心假設是灰度不變性：同一個空間點在不同影像中的亮度應該相同。於是估計運動就變成：找一組相機運動，使得「把上一幀的像素依這個運動投影到當前幀後，亮度差最小」。

49. 優點：不需計算特徵與描述子，速度可以很快；能利用影像中所有有梯度的像素（不只角點），因此在紋理稀少但有微弱梯度的場景仍可運作；能自然地產生半稠密或稠密地圖。
50. 缺點：灰度不變性假設脆弱——光線變化、自動曝光、非朗伯表面都會破壞它；對大幅度運動收斂困難（需要好的初值）；較難做閉環偵測。

代表系統包括 LSD-SLAM 與 DSO（Direct Sparse Odometry）。

| 比較項目 | 特徵法（間接法） | 直接法 |
|---|---|---|
| 使用的資訊 | 特徵點與描述子 | 像素亮度 |
| 核心假設 | 特徵可重複偵測與匹配 | 灰度不變性 |
| 對光線變化 | 較穩健 | 敏感 |
| 弱紋理場景 | 易失效 | 仍可利用微弱梯度 |
| 大幅運動 | 較能處理 | 易發散，需好初值 |
| 閉環偵測 | 容易（詞袋模型） | 較困難 |
| 地圖密度 | 稀疏 | 半稠密或稠密 |
| 代表系統 | ORB-SLAM 系列 | LSD-SLAM、DSO |

**表 20-1　特徵法與直接法的比較**

本書第六篇以特徵法為主線（第 21 至 23 章的 ORB-SLAM 系列），因為它成熟穩定、資源豐富、也最適合教學。但同學應知道直接法的存在——近年也出現結合兩者優點的混合方法。

## 20.3　ORB、SIFT、SURF、FAST、BRIEF

這五個名詞常一起出現，但它們的角色其實不同：有的只負責「偵測」、有的只負責「描述」、有的兩者都做。釐清這一點是本節的第一個重點。

### 偵測器與描述子：分工不同

51. FAST：只是偵測器。它極快地找出角點，但不產生描述子。

52. BRIEF：只是描述子。它為給定的關鍵點產生二元描述向量，但不負責找點。

53. SIFT、SURF、ORB：兩者兼具，既偵測也描述。

實務上這些方法可以獨立、分開地作為偵測器或描述子使用，不同組合會有不同的效能表現。這解釋了為何文獻中會出現「FAST-BRIEF」「SIFT-SURF」這類組合寫法。

### 五大演算法簡介

54. SIFT（尺度不變特徵轉換）：經典中的經典。它以高斯差分找出不同尺度下的極值點，產生 128 維的描述子，具有優異的尺度與旋轉不變性。SIFT 本質上把特徵偵測與描述向量產生合併為單一的關鍵推導步驟。缺點是計算量大。

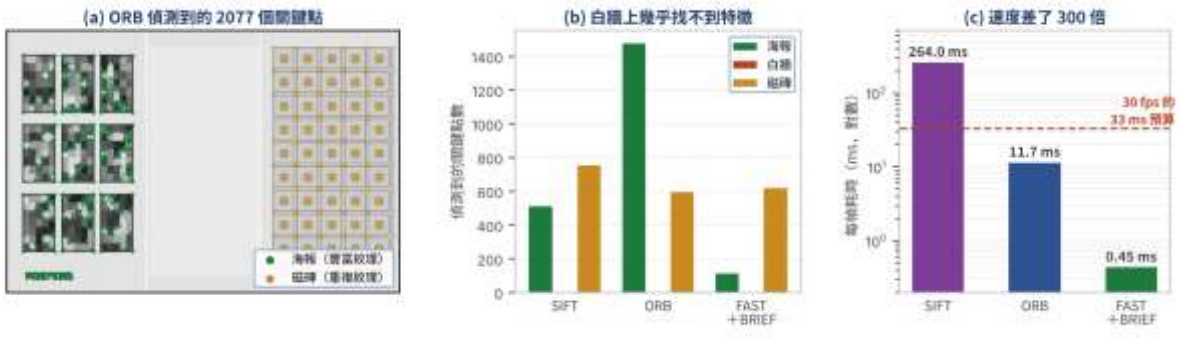
55. SURF（加速穩健特徵）：SIFT 的加速版，以積分影像與盒式濾波器近似高斯運算。整體而言 SURF 的計算需求低於 SIFT。

56. FAST（加速分段測試特徵）：它簡單而快速，把一個像素與其周圍 Bresenham 圓上的像素比較，先檢查該像素是否比圓上連續 N 個像素更亮或更暗；若是，它就可能是一個角點。演算法接著採用非極大值抑制來辨識並保留最具鑑別度的關鍵點。

57. BRIEF（二元穩健獨立基本特徵）：以「隨機挑選點對比較亮度」產生二元字串描述子。優點是產生極快、比對只需算漢明距離（位元運算），極為高效。缺點是原始版本不具旋轉與尺度不變性。

58. ORB（定向 FAST 與旋轉 BRIEF）：把 FAST 與 BRIEF 結合並各自改良——為 FAST 角點加上方向資訊（定向 FAST），並讓 BRIEF 描述子隨方向旋轉（旋轉 BRIEF），因而獲得旋轉不變性。

**圖 20-2　同一面牆、三種特徵演算法：數量、分布與代價**




為了讓比較有對照組，本書合成了一面教室牆，刻意分成三個區域：左側是九張各不相同的海報（豐富且獨特的紋理）、中間是一片幾乎無紋理的白牆、右側是整排一模一樣的磁磚（重複紋理）。這三種區域正好對應視覺 SLAM 會遇到的三種處境。

圖(a) 把 ORB 偵測到的 2,077 個關鍵點畫在影像上，結果一目了然：海報區密密麻麻，磁磚區也不少，而中間那片白牆上，一個點都沒有。圖(b) 把三種演算法的分區統計並列，白牆區的數字分別是 SIFT 1 個、ORB 0 個、FAST 0 個——這就是第 9 章「白牆詛咒」最直接的證據。注意這不是哪個演算法不夠好，而是沒有亮度變化的地方本來就沒有資訊可以提取，三種方法一視同仁地失效。

圖(c) 則是選擇 ORB 的理由。在同一張 960×640 的影像上，SIFT 每幀要 264 毫秒、ORB 11.7 毫秒、單純的 FAST 偵測只要 0.45 毫秒。SIFT 比 ORB 慢了二十餘倍，比 FAST 慢了近六百倍。紅色虛線標出的是 30 fps 系統每幀 33 毫秒的總預算——而這 33 毫秒裡還得塞進匹配、位姿求解與地圖更新。SIFT 單是提取特徵就超支八倍，ORB 則只用掉三分之一。案例 20-1 所說「速度的代價與回報」，講的就是這條紅線。

不過圖(b) 也藏著一個陷阱，值得同學先記下來：磁磚區的特徵數量其實很可觀，ORB 在那裡找到 598 個點、FAST 甚至找到 621 個。數量多不代表有用——下一節的圖 20-3 會揭曉，這些點幾乎全是災難的來源。

**這張圖的實驗條件**

影像為 960×640 灰階合成圖，ORB 上限設 2000 點，SIFT 與 FAST 用 OpenCV 預設參數，耗時為同一張影像重複五次的平均。

絕對數字會隨影像大小、參數與硬體而變，重要的是數量級的差距：SIFT 與 ORB 之間差一個數量級以上，這個結論相當穩健。

ORB 原論文報告它比 SURF 快約 14 倍、SURF 又比 SIFT 快約 24 倍（20.3 節），本實驗的 SIFT / ORB 比值與這個量級一致。

同學可在實作 20-1 第一部分用自己拍的教室照片重跑，並特別找一面白牆與一片磁磚地板來重現圖(b) 的三種處境。

### 為什麼 ORB 成為 SLAM 的主流

答案是速度。ORB 的開發者報告它比 SURF 快 14 倍，而 SURF 又比 SIFT 快 24 倍；這個較新的演算法在表現上也大致相當，某些情況下甚至好得多。另有文獻指出 ORB 的提取速度約比 SURF 快 15 倍，同時保持對旋轉與尺度變化的穩健性，這個效率使它特別適合即時應用。

在視覺 SLAM 的各種特徵提取演算法中，ORB 特徵的優勢在於提取與匹配快速，而相較 SIFT 與 SURF 並未損失太多精度。ORB 因此成為 SLAM 系統中主流的特徵點偵測方法——它以部分精度與穩健度換取運算速度的提升，這個取捨對即時系統極為划算。

### ORB 描述子的巧思

值得一提 ORB 描述子的設計細節。在 ORB 中，BRIEF 描述子是 256 位元的向量，每個位元代表同一個 31×31 像素區塊中兩個 5×5 像素視窗平均強度比較的結果；一個區塊中有 676 個這樣的視窗，因此有超過二十萬種可能的成對比較。為了最大化描述子的鑑別力，ORB 從這二十萬種組合中，挑選出「離平均最遠且彼此相關性最低」的 256 個比較結果。

這個設計展現了一種重要的工程思維：不是隨機選，而是用資料驅動的方式挑出最有資訊量的比較。同學若日後設計自己的特徵或指標，這個「最大化鑑別力、最小化冗餘」的原則值得參考。

| 演算法 | 角色 | 描述子形式 | 速度 | 不變性 |
|---|---|---|---|---|
| SIFT | 偵測 + 描述 | 128 維浮點 | 最慢 | 尺度、旋轉 |
| SURF | 偵測 + 描述 | 64 / 128 維浮點 | 中 | 尺度、旋轉 |
| FAST | 僅偵測 | 無 | 極快 | 無（原始版） |
| BRIEF | 僅描述 | 二元字串 | 極快 | 無（原始版） |
| ORB | 偵測 + 描述 | 256 位元二元 | 快 | 旋轉，部分尺度 |

**表 20-2 五種特徵演算法的比較**

## 20.4 Feature Tracking 與 Feature Matching

找到特徵之後，下一步是在不同影像間建立對應關係。這裡有兩種思路，同學常混淆，本節釐清它們。

### 特徵匹配（Matching）：比對描述子

匹配的做法是：在兩張影像中各自獨立提取特徵與描述子，然後對第一張的每個特徵，在第二張中找描述子最相似的那一個。

59. 相似度度量：浮點描述子（SIFT、SURF）用歐氏距離；二元描述子（ORB、BRIEF）用漢明距離——只需計算兩個位元串有幾位不同，用 XOR 加位元計數即可完成，速度極快。這是二元描述子的最大優勢。
60. 適用場景：兩張影像差異較大時（視角改變、時間間隔長、閉環偵測），匹配是唯一可行的方法。

### 特徵追蹤（Tracking）：在鄰近位置找

追蹤的做法是：在第一張影像提取特徵後，不在第二張重新提取，而是假設特徵只移動了一小段距離，直接在原位置附近搜尋最相似的影像區塊。典型方法是光流（第 9.4 節）。

61. 優點：不需計算描述子，速度更快；在連續影像中效果好。
62. 限制：只適用於影像變化小的情況（高幀率、緩慢運動）；追蹤久了會累積漂移，特徵可能「跟丟」或漂到別的地方。

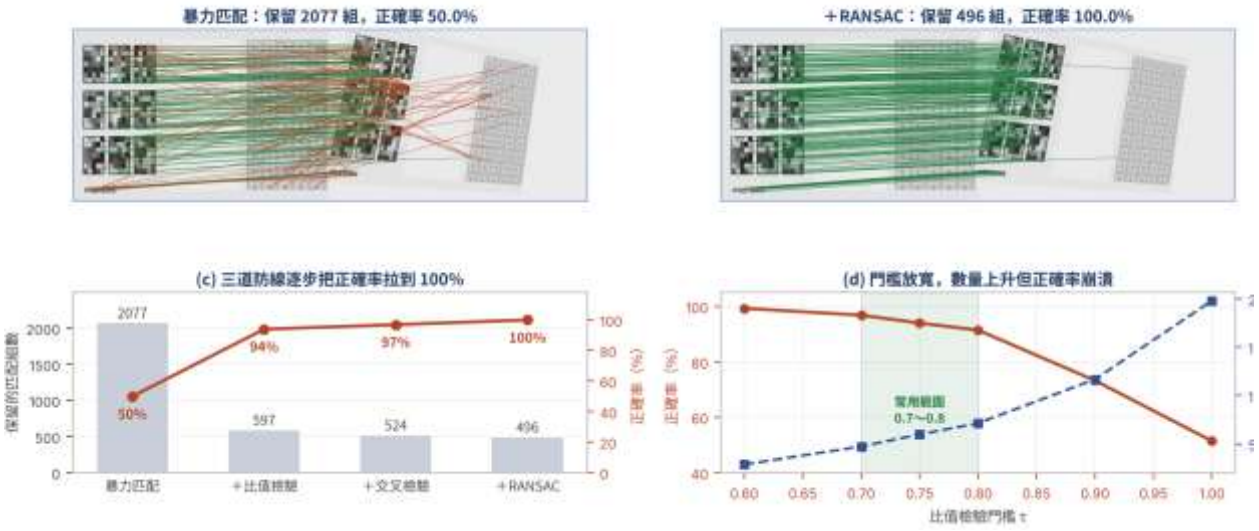
### 誤匹配：必須面對的現實

無論匹配或追蹤，都必然產生錯誤的對應。原因包括：重複紋理（磁磚、欄杆、樹葉，處處長得一樣）、視角變化太大、光線改變、動態物體。而在幾何求解中，少數幾個嚴重的誤匹配就可能把結果拉偏——這是視覺 SLAM 最常見的失敗原因之一。

### 三種誤匹配剔除策略

- 比值檢驗：對每個特徵，找出最相似與次相似的兩個候選。若兩者的距離很接近，表示「難以區分」，這個匹配不可靠，予以捨棄。這是 SIFT 論文提出的經典做法，簡單而有效。
- 交叉檢驗：A 影像的特徵 a 匹配到 B 的特徵 b，同時 b 反過來也匹配到 a，才認可。單向的匹配視為不可靠。
- 幾何驗證（RANSAC）：最重要的一招。利用「所有正確匹配必須符合同一個相機運動」這個幾何約束，反覆隨機取樣求解並統計符合者，把不符合的判為離群點剔除。實務上以 RANSAC 估計本質矩陣與內點集合，是從含有離群點的特徵對應中估計相機相對位姿的標準做法。這個方法將在第 21 章詳細展開。

**圖 20-3　誤匹配的三道防線：從 50% 正確率到 100%**




誤匹配剔除的效果通常只能靠眼睛看連線順不順，但如果第二張影像是用一個已知的單應矩陣從第一張變換出來的，那每一組匹配的對錯就有了標準答案——把第一張的特徵點用該矩陣投影過去，落點與匹配到的點相距四個像素以內就算正確。本圖正是這樣建立真值的。

圖(a) 是未經任何剔除的暴力匹配：2,077 組匹配，綠線是正確的、紅線是錯誤的，正確率恰好 50.0%——等於擲硬幣。注意紅線的分布並不均勻，大量紅線從右側磁磚區橫跨到左側海報區，呈現明顯的系統性錯亂。圖(b) 是走完三道防線之後：496 組匹配，正確率 100%，

連線整齊地呈現同一個方向的平移。

圖(c) 把過程拆開。比值檢驗一口氣把 2,077 組砍到 597 組，正確率從 50% 跳到 94%——這一刀最關鍵。交叉檢驗再修到 524 組、96.9%；最後 RANSAC 以幾何一致性把殘餘的外點清掉，剩下 496 組、100%。三道防線總共丟掉了 76% 的匹配，換來的是完全可信的 496 組對應。對後端最佳化而言，這筆交易非常划算：少量正確的約束遠勝大量摻雜錯誤的約束。

圖(d) 回答了討論問題第二題。比值門檻 $\tau$ 從 0.6 放寬到 1.0，保留的匹配數從 285 升到 1,981，但正確率從 99.3% 一路崩到 51.4%。曲線在 0.8 之後明顯轉折——這解釋了為什麼文獻上的建議值幾乎都落在 0.7 到 0.8 之間（圖中綠色區塊）。$\tau$ 的意義其實很直觀：它問的是「最佳候選有沒有明顯優於次佳候選」，而 $\tau = 1.0$ 等於完全不問，自然退化成暴力匹配。

**磁磚區發生了什麼：案例 20-2 的量化版**

分區來看，暴力匹配在海報區的正確率是 69.8%，在磁磚區只有 1.0%——598 組匹配裡只有 6 組是對的。重複紋理不是「比較難」，而是幾乎必然出錯。

比值檢驗在兩區的表現也截然不同：海報區從 1,479 組保留 587 組（約四成），磁磚區卻從 598 組只保留 10 組（不到 2%）。

這正是比值檢驗的設計意圖：在處處長得一樣的地方，最佳與次佳候選的距離必然接近，演算法會老實地說「我分不出來」而放棄，而不是硬選一個。

最終 RANSAC 之後，磁磚區只剩 2 組匹配。少，但是對的——這比留下 598 組有 99% 是錯的要安全得多。

## 20.5 Monocular SLAM 的尺度不確定性

這是單目視覺 SLAM 最根本、也最無法迴避的限制。第 9.1 節從針孔投影公式指出了它的幾何根源，本節說明它的後果與解法。

### 問題的本質

回顧第 9.1 節的投影公式：x = f·X/Z。注意這個式子中，若把場景中所有點的三維座標同時放大 k 倍，同時把相機的所有平移也放大 k 倍，投影到影像上的結果完全相同。

換句話說，單目相機拍到的一段影像序列，可能對應「真實房間中的真實運動」，也可能對應「一個模型屋中的微小運動」——兩者在影像上完全無法區分。單目 SLAM 因此只能恢復出「差一個未知比例因子」的軌跡與地圖。

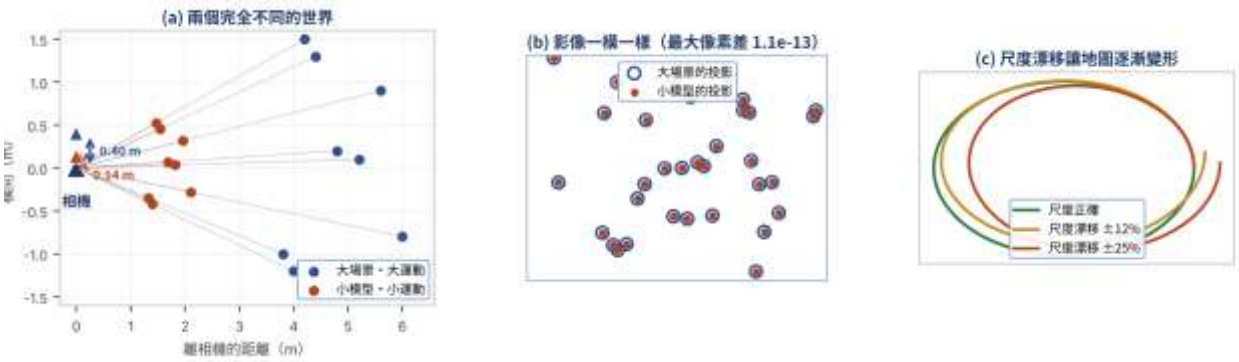
### 後果

63. 地圖沒有真實尺寸：建出的地圖可以說「這面牆到那面牆是 3.7 個單位」，但不知道一個單位是幾公尺。

64. 無法直接用於導航：機器人不知道「前方障礙物到底在 0.5 公尺還是 5 公尺外」，自然無法安全避障。

65. 尺度漂移：更麻煩的是，這個未知的比例因子在長時間運行中還會緩慢改變（稱為尺度漂移），使地圖逐漸變形。

**圖 20-4　尺度不確定性：單目相機先天無法分辨的兩個世界**




20.5 節說「把場景放大 k 倍、相機平移也放大 k 倍，影像完全相同」，這句話可以直接驗算。圖(a) 是側視圖：藍色是一個真實房間，點分布在離相機 3.6 到 6.2 公尺處，相機平移 0.40 公尺；紅色則是把整個場景與平移同時縮小到 0.35 倍——變成一個擺在眼前的模型屋，相機只移動 0.14 公尺。兩者在物理上完全是不同的世界。

圖(b) 把兩組三維點用同一組相機內參投影成影像。藍色空心圈是大場景的投影、紅色實心點是小模型的投影——它們不是「很接近」，而是完全重合。本書實際計算了兩組投影的最大像素差：$1.1\times10^{-13}$，也就是浮點運算的捨入誤差，在數學上就是零。這不是演算法不夠聰明，而是投影方程式本身決定的：單目影像裡根本沒有攜帶尺度資訊。

圖(c) 則展示這個不確定性在實務上的具體樣貌——尺度漂移。綠線是尺度正確時的真實軌跡，一個閉合的橢圓。若每一段位移的尺度因子以隨機漫步方式緩慢偏移 ±12%（黃線）或 ±25%（紅線），軌跡就逐漸變形、首尾無法接合。請注意：這裡的每一步方向都是正確的，錯的只有「每一步走了多遠」，但累積下來足以讓整張地圖失真。

這就是為什麼單目 SLAM 幾乎不會單獨使用。20.5 節的三種解法——加 IMU、改用雙目或 RGB-D、或引入已知尺寸的參考物——本質上都是從外部補進一個「公尺」的定義。值得一提的是，這個限制至今沒有被打破：即使是 20.8 節將介紹的、以基礎模型為核心的最新單目 SLAM 系統，仍然要在 Sim(3) 上最佳化、並在評測時先做尺度對齊——因為它面對的是同一個幾何事實。

**尺度與「深度估計」的差別**

深度學習模型（如 Depth Anything 類）能從單張影像「猜」出深度，但那是從訓練資料學到的先驗——看到門框就推測約 2 公尺高，而不是量出來的。

這種先驗在一般場景好用，在尺寸異常的場景會系統性出錯：模型屋、放大的道具、空拍的俯視畫面，都是典型的失敗案例。

相對地，IMU 的加速度計量到的是帶物理單位的加速度，雙目的基線是用尺量過的固定距離——這兩者提供的是量測，不是猜測。

實務建議：需要安全保證的場合（避障、抓取、導航）用量測；需要視覺效果的場合（AR 貼圖、稠密重建）可接受先驗。

### 三種解決途徑

- 引入 IMU（視覺慣性）：加速度計量測的是有物理單位的加速度，積分後即可提供真實尺度。這是最通用的解法，也是第 7 章與第 23 章的核心主題。代價是需要足夠的運動激勵才能觀測到尺度（第 7.7 節）。
- 使用雙目或 RGB-D 相機：雙目的基線是已知的固定距離，RGB-D 直接輸出以公尺為單位的深度。硬體上直接解決問題，是最省事的做法（20.6 節）。
- 已知尺寸的參考物：若場景中有已知大小的物體（如第 4 章的 AprilTag、棋盤格），或已知相機離地高度，就能反推尺度。這在特定應用中很實用。

值得補充的是，近年也有以深度學習從單張影像估計深度（第 9.4 節的 Depth Anything 類模型）來提供尺度先驗的做法，但其精度與可靠度仍不及前三種物理方法。

## 20.6 Stereo SLAM 與 RGB-D SLAM

第 9.3 節介紹過三種相機取得深度的方式，本節聚焦它們在 SLAM 系統中的差異。

### 雙目 SLAM

雙目相機以已知基線的兩個鏡頭同時拍攝，由左右影像的視差直接算出深度（第 9.6 節的公式 $Z = f \cdot b/d$）。

66. 優點：單幀即可得深度、有絕對尺度、室內外皆可運作（不依賴主動投射）、量測範圍可透過加大基線延伸。
67. 限制：需精確校正兩相機的相對位置（外參）、在無紋理表面找不到對應而無深度、深度誤差隨距離平方成長（第 9.6 節）、運算量較大（需做立體匹配）。
68. 適用：戶外機器人、無人機、自駕車——凡是需要在陽光下運作、且距離範圍較大的場景。

### RGB-D SLAM

RGB-D 相機主動投射紅外線圖案取得稠密深度（第 9.3 節的結構光或主動紅外雙目）。

69. 優點：直接輸出每個像素的深度，不需自己做立體匹配；在無紋理的白牆上也有深度（因為是主動投射）；有絕對尺度；能輕易建出稠密的三維地圖。

70. 限制：多依賴紅外線，強烈日光下失效；有效範圍通常在 0.2 至 10 公尺；對玻璃鏡面同樣束手無策（與第 8 章光達的弱點相同）；功耗較高。

71. 適用：室內服務機器人、桌面級三維重建、室內測繪——這也是第 30 章實作選擇 RGB-D 的原因。

| 比較項目 | 單目 SLAM | 雙目 SLAM | RGB-D SLAM |
|---|---|---|---|
| 絕對尺度 | 無（需外部提供） | 有 | 有 |
| 深度來源 | 運動三角測量 | 左右視差 | 主動投射 |
| 無紋理表面 | 失效 | 無深度 | 仍有深度 |
| 戶外強光 | 可用 | 可用 | 多半失效 |
| 有效距離 | 不限（但無尺度） | 較遠 | 約 0.2–10 m |
| 地圖密度 | 稀疏 | 可半稠密 | 稠密 |
| 硬體成本 | 最低 | 中 | 中 |

**表 20-3　三種視覺 SLAM 配置的比較**

ORB-SLAM2 與 ORB-SLAM3（第 22、23 章）的一大優點，就是同時支援這三種模式——同樣一套系統，換個輸入模式即可。這讓同學能在同一個框架下比較三者的差異，是絕佳的教學素材。

## 20.7　視覺 SLAM 在 AR / VR 與機器人中的應用

第 1 章 1.4 節提過 AR / VR 是 SLAM 四大應用之一，本節深入視覺 SLAM 在其中的角色，並延伸到機器人應用。

### AR / VR：口袋裡的視覺 SLAM

AR 眼鏡或手機 AR 要把虛擬物件穩定地「釘」在真實桌面上，前提是裝置必須即時知道自己在空間中的六自由度位姿。業界慣稱這個能力為「空間定位」或「由內而外追蹤」（Inside-Out Tracking），而它的技術本質就是視覺慣性 SLAM。

72. 為什麼是視覺而非光達：AR 裝置對重量、功耗、成本極度敏感，相機加 IMU 是唯一可行的組合（第 9.5 節的視覺優勢）。
73. 特殊需求：延遲必須極低（否則虛擬物件會「漂移」造成不適）、且必須在使用者快速轉頭時仍維持追蹤——這對視覺 SLAM 是嚴苛考驗，也是 IMU 不可或缺的原因（第 7.6 節）。
74. VR 頭盔的演進：早期 VR 需要外部基地台定位，現代頭盔改用機身相機做視覺 SLAM，使用者因此能自由走動——這是 SLAM 直接改變產品形態的例子。

**機器人應用**

75. 室內服務機器人：視覺 SLAM 提供的不只是幾何，還有語意（第 9.7 節）——機器人能辨識「那是門、那是電梯」，這是光達給不了的（第 32 章的主題）。
76. 無人機：載重與電力極度受限，相機加 IMU 的組合是最佳選擇（第 26 章的主題）。
77. 低速自駕與園區載具：視覺 SLAM 在低速自駕情境中的應用是活躍的研究方向，特徵法因其對相機運動、光線變化與動態物體的穩健性而被廣泛採用。
78. 地點辨識與重定位：視覺特徵天生適合做「這裡我來過嗎」的判斷。詞袋技術是最普遍的做法之一，把局部特徵描述子當作視覺單詞使用；而 DBoW2 引入高效的 FAST 偵測器搭配 BRIEF 描述子，以二元單詞袋的形式，相較傳統使用 SURF 與 SIFT 的做法大幅減少特徵提取所需時間，系統更有效率也更穩健。這正是 ORB-SLAM 閉環偵測的技術基礎（第 22 章）。

**趨勢：深度學習的加入**

傳統手工設計的特徵方法有一個問題：它們無法有效適應各種複雜且無法預期的情況；因此許多研究者採用卷積神經網路來提取影像的深層特徵，應用於視覺里程計、位姿估計與

閉環偵測等各個階段。

這是視覺 SLAM 目前最活躍的方向之一：用學習來的特徵取代手工特徵、用神經網路做地點辨識、乃至於端到端的位姿估計。不過在實務系統中，傳統幾何方法因其可解釋性與穩定性，目前仍是主流；學習方法多作為輔助或用於特定困難場景。第 32 章會回到這個主題，討論 AI 如何與 SLAM 結合。

至此，同學已建立視覺 SLAM 的整體框架。下一章我們深入前端的數學核心——對極幾何：兩張影像之間到底藏著什麼幾何關係？本質矩陣與基礎矩陣是什麼？PnP 問題如何求解？三角測量如何恢復三維點？這些是把「二維像素對應」變成「三維相機運動」的關鍵橋樑。

## 20.8 特徵之後：2024–2026 年視覺前端的三個轉向

本章介紹的 SIFT、ORB 都是手工設計的特徵，而案例 20-3 已經預告了深度學習帶來的轉變。這個轉變在近兩年明顯加速，而且變化的層次比「換一種特徵」更深。本節整理三個轉向。

### 一、學習式特徵終於跑得動了

學習式局部特徵（如 SuperPoint）準確度早就超越 ORB，但長年卡在同一個問題上：太慢，而且需要 GPU。這正是圖 20-2(c) 那條 33 毫秒紅線所設下的門檻。2024 年的 XFeat 直接針對這一點而設計——它主打在「一台便宜的筆電 CPU 上即時運行」，不需要專門的硬體加速，而速度比既有的輕量深度特徵最多快五倍，準確度則與 SuperPoint 這類大得多的模型相當。

在匹配端也有類似的演進。LightGlue 一類的方法讓深度匹配器能依影像的難易度自適應地提前結束運算，容易的影像對就少算幾層。兩者合起來的意義是：圖 20-3 那三道防線（比值檢驗、交叉檢驗、RANSAC）正在被一個學會判斷「這組對應可不可信」的網路取代。不過要強調的是，RANSAC 代表的幾何一致性驗證仍然沒有消失，它只是從「主要防線」退

為「最後確認」。

## 二、從「兩張影像之間」到「直接吐出三維」

第二個轉向更根本。傳統視覺 SLAM 的前端是一條長鏈：提特徵 → 匹配 → 剔除 → 解對極幾何 → 三角測量。而 DUSt3R 與 MASt3R 這一系列工作提出：乾脆讓網路直接從兩張影像吐出每個像素的三維座標，把中間那一整串步驟一次跳過。

建構於其上的 MASt3R-SLAM（CVPR 2025）就是一套完整的單目稠密 SLAM：它以稠密的點圖匹配取代 KD 樹或暴力搜尋，甚至不需要事先知道相機內參。在 TUM RGB-D 資料集上，已校正情況下平均絕對軌跡誤差 0.030 公尺，未校正時 0.060 公尺；在一般桌機搭配高階顯示卡上約可達 15 fps。

但這裡有一個同學一定要看清楚的細節，而且它直接呼應圖 20-4：MASt3R-SLAM 並沒有解決尺度不確定性。該論文明言尺度是預測之間不一致的主要來源，因此在 Sim(3) 上最佳化位姿，單目的評測結果也是在做過尺度對齊之後才報告的。換句話說，就算把整個前端換成基礎模型，單目相機看不出絕對尺度這件事依然成立——因為那是幾何的性質，不是演算法的缺陷。

## 三、地圖表示：從稀疏點雲到可算繪的場景

第三個轉向與第 17.8 節提過的三維高斯潑濺是同一股潮流，只是在視覺領域更為自然。傳統特徵法 SLAM 建出的是稀疏的特徵點地圖——夠用來定位，但看起來只是一團散點，既無法用於渲染，也不利於後續的語意理解。近兩年的稠密視覺 SLAM 則直接把場景表示成可算繪的形式，讓「定位」與「重建逼真場景」變成同一件事的兩面。

這對第 20.7 節談的 AR / VR 應用意義特別大：AR 需要的從來不只是相機位姿，還需要知道虛擬物件會被哪些真實表面遮擋。稀疏點雲給不了這個資訊，稠密可算繪的表示則天生具備。不過代價也很明確——記憶體與運算量都遠高於稀疏地圖，這也是為什麼機器人導航目前仍以稀疏或半稠密為主。

**三個轉向的共同線索**

前端元件（XFeat / LightGlue）：學習式特徵終於跨過即時性門檻，開始取代 ORB。

前端結構（MASt3R-SLAM）：整條「特徵—匹配—幾何」的鏈條被端到端網路一次跨過。

地圖表示（稠密可算繪）：地圖從定位用的稀疏點，變成能看、能遮擋、能理解的場景。

沒有改變的事：尺度不確定性仍在（圖 20-4）、幾何一致性驗證仍需要、而白牆上依然沒有資訊可以提取。

共同線索：被取代的是「怎麼算」，不是「算什麼」。本章的四個基本問題——哪裡有特徵、哪些對應可信、深度從哪來、尺度誰給——依然是評斷任何新系統的標準。

### 承先啟後

本章建立了視覺 SLAM 的整體框架與特徵基礎。圖 20-2 說明了哪裡找得到特徵、以及為什麼是 ORB；圖 20-3 說明了匹配必然出錯、以及三道防線如何把正確率從 50% 救回 100%；圖 20-4 則說明了單目相機先天的極限。這三件事也正是第 22、23 章 ORB-SLAM 系列每一個設計決策的背景。

但我們還沒回答最核心的問題：有了一組可信的二維對應，究竟如何算出相機移動了多少、以及那些特徵點在三維空間的哪裡？下一章進入對極幾何——本質矩陣、基礎矩陣、單應矩陣與三角測量，這是整個視覺 SLAM 的數學心臟，也是圖 20-3 裡 RANSAC 所驗證的那個「幾何約束」的真面目。

## 公式或流程

## 一、漢明距離：二元描述子的比對

兩個二元描述子 $d_i$、$d_j$ 的相似度以漢明距離衡量——即相異位元的個數：

$$Hamming(d_i, d_j) = popcount(d_i \ XOR \ d_j)$$

XOR 後計算 1 的個數（popcount）。對 256 位元的 ORB 描述子而言，這只需數個位元運算指令，遠快於浮點描述子的歐氏距離計算。這正是 ORB 能達到即時效能的關鍵之一。距離越小表示越相似。

## 二、比值檢驗

對某個特徵，設它與最相似候選的距離為 $d_1$、與次相似候選的距離為 $d_2$，則接受此匹配的條件為：

$$d_1 \ / \ d_2 < \tau \quad (\tau \text{ 常取 } 0.7 \text{ 至 } 0.8)$$

直觀理解：若最佳與次佳的差距不夠明顯（比值接近 1），表示這個特徵在影像中有多個相似的候選，難以區分，應予捨棄。這條簡單的判準能剔除大量誤匹配。

## 三、直接法的光度誤差

直接法不比對描述子，而是最小化光度誤差——把上一幀的像素依相機運動 T 投影到當前幀後的亮度差：

$$T^* = argmin \ \Sigma_i \ || \ I_2(\pi(T \cdot P_i)) \ - \ I_1(u_i) \ ||^2$$

其中 $I_1$、$I_2$ 是兩幀影像的亮度、$u_i$ 是第一幀中的像素、$P_i$ 是它對應的三維點、$\pi(\cdot)$ 是第 9 章的投影函數。與特徵法比較：特徵法最小化的是「重投影誤差」（幾何距離，以像素為單位），直接法最小化的是「光度誤差」（亮度差）。兩者都是最小平方問題（第 12 章），只是誤差的定義不同。

## 四、視覺 SLAM 前端處理流程

**圖 20-5 特徵法前端的每幀處理流程**

① 讀取影像 → 去畸變（第 9.2 節）

↓

② 偵測關鍵點：以 FAST 找角點並做非極大值抑制

↓

③ 計算方向與描述子：定向 FAST + 旋轉 BRIEF → 256 位元描述子

↓

④ 與上一幀（或地圖）匹配：漢明距離比對

↓

⑤ 誤匹配剔除：比值檢驗 → 交叉檢驗 → RANSAC 幾何驗證

↓

⑥ 位姿求解：由通過驗證的匹配估計相機運動（第 21 章）

↓

⑦ 三角測量新的三維點 → 交給後端最佳化（第 12、22 章）

## 案例

### 案例 20-1 速度的代價與回報：ORB 為何贏得 SLAM 戰場

如果單看特徵的品質，SIFT 幾乎在各項指標上都優於 ORB——它的尺度不變性更完整、匹配更準確、對視角變化更耐受。那為什麼絕大多數即時 SLAM 系統選擇 ORB?

答案是即時性的殘酷要求。一個以 30 幀每秒運行的 SLAM 系統，每一幀只有約 33 毫秒的預算，而這段時間還得完成特徵提取、匹配、位姿求解、地圖更新等所有工作。ORB 開發者報告它比 SURF 快 14 倍，而 SURF 又比 SIFT 快 24 倍——換算下來，ORB 大約比

SIFT 快兩個數量級。這個差距不是「稍微快一點」，而是「能不能即時運行」的分野。

更關鍵的是，ORB 犧牲的精度在 SLAM 系統中可以被彌補。SLAM 有後端最佳化與閉環修正（第 12 章），即使單次匹配略遜一籌，大量的冗餘觀測與全域最佳化能把誤差壓下來。換句話說：系統層級的設計，補償了元件層級的不足。

這個案例給同學一個重要的工程觀：選擇技術元件時，不能只看該元件的孤立指標，而要看它在整個系統中的貢獻與代價。這與第 8 章「感測器選型是綜合判斷」、第 15 章「設計取向由應用需求決定」一脈相承。

## 案例 20-2　重複紋理的陷阱：磁磚地板與欄杆

一個在大學課堂上容易重現的失敗實驗：讓機器人在鋪滿相同磁磚的地板上，或沿著整排相同的欄杆移動，執行視覺 SLAM。結果往往是位姿估計突然跳錯，地圖出現嚴重錯位。

原因是重複紋理造成的系統性誤匹配。每一塊磁磚的角點看起來都一模一樣，描述子幾乎相同。演算法在比對時，很可能把「這一塊磁磚的角」匹配到「隔壁那一塊磁磚的角」——而且它「看起來」是一個完美的匹配，描述子距離極小。

這正是比值檢驗存在的意義：當最佳與次佳候選的距離非常接近時（磁磚場景必然如此），比值檢驗會判定「無法區分」而捨棄這個匹配，寧可少一些匹配，也不要錯誤的匹配。若比值門檻設得太寬鬆，這類場景就會出問題。

對照第 13.6 節的長走廊、第 17.3 節的結構退化、第 9 章的白牆詛咒，同學應該看出一個貫穿全書的模式：環境的重複性與特徵匱乏，是 SLAM 的共同天敵，不分光達或視覺。理解這一點，你就能在專題規劃階段預判哪些場域會出問題。

## 案例 20-3　從手工特徵到深度學習：一個正在發生的轉變

本章介紹的 SIFT、SURF、ORB 都是「手工設計」的特徵——人類研究者根據對影像的理解，設計出偵測與描述的規則。這些方法主導了電腦視覺與 SLAM 二十餘年。

但它們有一個共同弱點：無法有效適應各種複雜且無法預期的情況；因此許多研究者轉而採用卷積神經網路來提取影像的深層特徵，應用於視覺里程計、位姿估計與閉環偵測等階段。學習來的特徵能適應手工規則難以描述的情境——季節變化、日夜差異、天氣影響。

不過同學要注意：這個轉變尚未完成，而且未必會全面取代。傳統幾何方法有幾個學習方法目前難以匹敵的優點：可解釋（出錯時知道為什麼）、不需訓練資料、在未見過的場景中行為可預期、運算資源需求明確。目前主流實務系統仍以幾何方法為骨幹，學習方法作為特定環節的增強。

對本書讀者的建議是：先把傳統方法學紮實。理解 ORB 為何這樣設計、RANSAC 為何有效、Bundle Adjustment 在解什麼——這些原理是不會過時的思考框架。有了這個基礎，再去看深度學習的方法，你會知道它們在取代什麼、又保留了什麼。第 32 章會回到這個主題。

## 實作活動

### 實作 20-1　特徵提取、匹配與誤匹配剔除

目的：親手實作視覺 SLAM 前端的完整流程，比較不同特徵演算法的效能，並見證誤匹配剔除的威力。本實作以 Python 搭配 OpenCV，只需一般相機或現成影像即可完成，是第 21 至 23 章的直接暖身。

**第一部分：特徵演算法比較（全體）**

54. 準備兩張同一場景但視角略有不同的照片（可用手機拍攝教室，前後移動一小步）。
55. 分別用 SIFT、ORB、FAST 偵測關鍵點，把結果畫在影像上並排比較：各偵測到幾個點？分布在哪裡？
56. 計時比較：記錄三種方法在同一張影像上的提取時間，驗證本章提到的速度差異（ORB 遠快於 SIFT）。

57. 對一張大面積白牆的照片重複實驗，觀察三種方法都幾乎偵測不到特徵——再次驗證第 9 章的「白牆詛咒」。

58. 旋轉測試：把第二張影像旋轉 45 度，比較 ORB 與原始 BRIEF 的匹配成功率，驗證「旋轉不變性」的價值。

**第二部分：匹配與誤匹配剔除（全體）**

59. 用 ORB 對兩張影像做暴力匹配（以漢明距離），畫出所有匹配連線，目視觀察有多少條線明顯是錯的。

60. 套用比值檢驗（門檻 0.75），重新畫出匹配結果，統計剔除了多少匹配、剩下的品質如何。

61. 加入交叉檢驗，再次觀察改善程度。

62. 用 RANSAC 估計兩張影像之間的幾何關係（可先用單應矩陣），把內點與外點以不同顏色顯示，觀察 RANSAC 剔除了哪些匹配。

63. 比值門檻實驗：把門檻設為 0.6、0.75、0.9，比較保留的匹配數量與正確率，找出你的場景最合適的值。

**第三部分：重複紋理與追蹤（大學部）**

64. 重複紋理實驗：拍攝磁磚地板、欄杆或格子狀的窗戶，執行匹配並觀察誤匹配現象，重現案例 20-2。比較比值檢驗在此類場景的重要性。

65. 光流追蹤：錄製一段緩慢移動的影片，用 OpenCV 的 Lucas-Kanade 光流追蹤特徵點，把軌跡畫出來，觀察哪些點被穩定追蹤、哪些跟丟。

66. 比較追蹤與匹配：對同一段影片，分別用「每幀重新提取並匹配」與「光流追蹤」，比較兩者的耗時與穩定度，對照 20.4 節的討論。

**討論問題**

19. ORB 與 SIFT 的提取時間相差多少倍？與本章引述的數據是否相符？若你的 SLAM 系統要跑 30 fps，哪一種可行？

20. 比值檢驗剔除了百分之幾的匹配？剩下的匹配品質有明顯提升嗎？若把門檻放寬到 0.9，會發生什麼？
21. 在重複紋理場景中，誤匹配呈現什麼模式（是隨機的，還是系統性地偏向某個方向）？為什麼系統性誤匹配比隨機誤匹配更危險？
22. 光流追蹤在什麼情況下跟丟？這對「相機移動速度」與「幀率」的關係有什麼啟示？
23. （大學部）依表 20-3，若要為大學的室內導覽機器人選視覺配置，你會選單目 + IMU、雙目還是 RGB-D？說明理由。

**延伸挑戰（大學部）**

其一：實作一個最簡單的視覺里程計——連續讀取影片幀，用 ORB 匹配相鄰幀、以 RANSAC 剔除誤匹配、估計相對運動並累加，把軌跡畫出來。觀察漂移如何隨時間累積（這正是第 21 至 22 章要解決的問題）。其二：實作一個簡易的地點辨識——把一段影片的所有幀提取 ORB 描述子，建立簡單的詞袋表示，測試能否正確找出「同一地點的不同時刻影像」。這是第 22 章閉環偵測的雛形。

## 延伸閱讀

29. E. Rublee et al., "ORB: An Efficient Alternative to SIFT or SURF," ICCV, 2011.（ORB 原始論文，本章 20.3 節主要依據）
30. D. G. Lowe, "Distinctive Image Features from Scale-Invariant Keypoints," IJCV, 2004.（SIFT 經典論文，含比值檢驗的提出）
31. E. Rosten and T. Drummond, "Machine Learning for High-Speed Corner Detection," ECCV, 2006.（FAST 偵測器原始論文）
32. M. Calonder et al., "BRIEF: Binary Robust Independent Elementary Features," ECCV, 2010.（二元描述子的開創性工作）

33. D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," IEEE T-RO, 2012.（DBoW2,ORB-SLAM 閉環偵測的基礎）

34. 高翔、張濤，《視覺 SLAM 十四講》第 7 講（視覺里程計：特徵點法）、第 8 講（直接法）.（華文世界最清楚的特徵法與直接法教材，含完整程式碼）

35. OpenCV 官方文件之 Feature Detection and Description 教學（docs.opencv.org）.（本章實作的主要工具，附完整範例）

## 小結

29. 視覺 SLAM 的標準流程為：預處理去畸變 → 視覺里程計（前端） → 後端最佳化 → 閉環偵測 → 建圖；其架構與第 18 章的光達 SLAM 高度一致，差別在於視覺特徵只有二維像素座標，深度需另外求得。

30. 特徵法先提取關鍵點與描述子再以幾何求解，對光線變化與動態物體穩健但在弱紋理處失效；直接法以像素亮度最小化光度誤差，能利用微弱梯度並產生稠密地圖，但依賴脆弱的灰度不變性假設。

31. FAST 只偵測、BRIEF 只描述、SIFT / SURF / ORB 兩者兼具；ORB 結合定向 FAST 與旋轉 BRIEF，以 256 位元二元描述子與漢明距離達成極高效率——比 SURF 快約 14 倍、SURF 又比 SIFT 快約 24 倍，因而成為 SLAM 的主流選擇。

32. 匹配比對描述子，適用於視角差異大與閉環偵測；追蹤在鄰近位置搜尋，適用於連續影像且更快。誤匹配必然發生，三種剔除策略為比值檢驗、交叉檢驗與 RANSAC 幾何驗證。

33. 單目 SLAM 有先天的尺度不確定性：場景與運動同時放大 k 倍，影像完全相同；三種解法為引入 IMU、使用雙目或 RGB-D、或利用已知尺寸的參考物。

34. 雙目以已知基線的視差測深，有絕對尺度且室內外皆可用，但怕無紋理且誤差隨距離平方成長；RGB-D 主動投射取得稠密深度，在白牆上仍有效，但怕強光與玻璃、範圍受限。
35. 視覺 SLAM 是 AR / VR 由內而外追蹤的技術本質，也廣泛用於服務機器人、無人機與低速自駕；詞袋模型（如 DBoW2 以二元單詞袋）是地點辨識與閉環偵測的關鍵技術，而深度學習特徵正逐步進入這個領域。

本章建立了視覺 SLAM 的整體框架與特徵基礎。但我們還沒回答最核心的問題：有了兩張影像之間的特徵對應，究竟如何算出相機移動了多少？下一章「對極幾何與相機運動估測」將補上這塊拼圖——本質矩陣與基礎矩陣描述了兩視圖之間的幾何約束、PnP 問題處理三維點到二維影像的對應、三角測量恢復三維座標、RANSAC 則讓這一切在有誤匹配的現實中依然可行。這是視覺 SLAM 最數學、也最優美的一章。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

## 第 21 章

## 對極幾何與相機運動估測

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

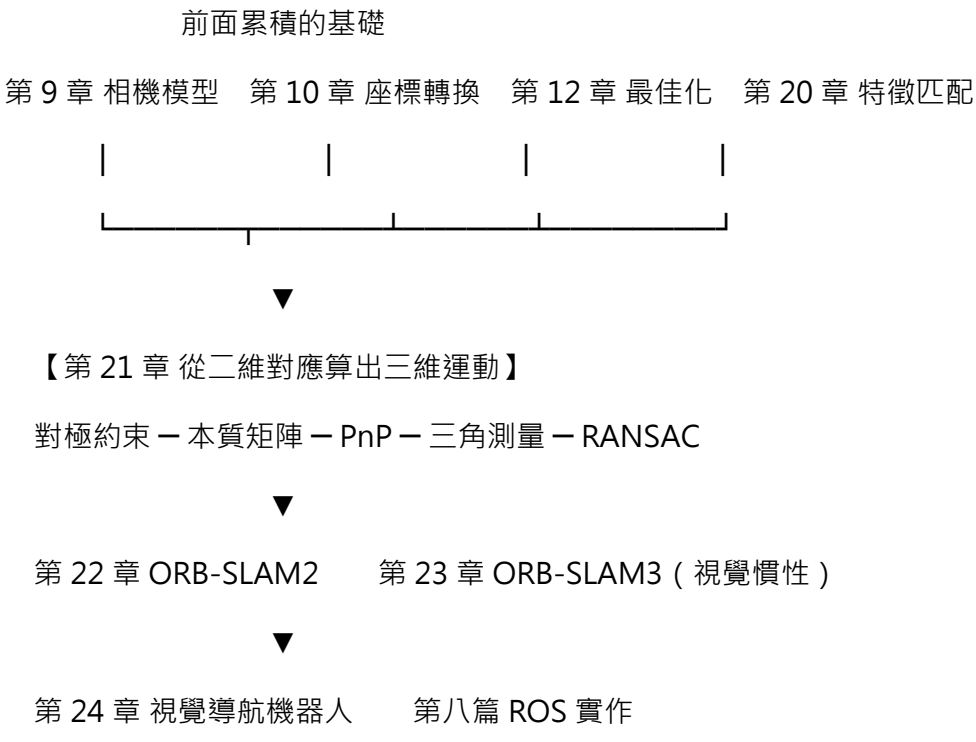
## 本章在全書的位置

第 20 章教會了機器人「在兩張影像之間找到同一個東西」。本章回答下一個問題：找到了之後，怎麼算出相機移動了多少。

這是全書數學密度最高的一章，但它的回報也最直接——本章的三張實驗圖會證明一件事：對極幾何不是近似、不是經驗法則，而是一組在無雜訊時誤差為零的精確關係。圖 21-2 會示範，給它乾淨的對應點，八點法恢復出的旋轉與平移方向誤差是 0.000 度；四組數學解當中，正深度檢驗會以 223 比 0 的票數挑出唯一正確的那一組。

但同樣重要的是它的脆弱。圖 21-3 會量出雜訊怎麼放大成位姿誤差，以及為什麼基線短的時候深度完全不能信；圖 21-4 則會顯示，只要有一成的錯誤匹配，這套優美的數學就會整個垮掉——這正是 RANSAC 存在的理由，也是第 22 章 ORB-SLAM2 每一個穩健性設計的動機。

### 圖 21-0　第 21 章的連結地圖




## 學習目標

上一章我們學會了如何在兩張影像間找出對應的特徵點，但還沒回答最核心的問題：有了這些對應，究竟如何算出相機移動了多少？本章補上這塊拼圖。這是視覺 SLAM 最數學、也最優美的一章——一旦理解對極幾何，你會發現兩張看似平凡的照片裡，竟藏著如此嚴謹的三維約束。學完本章之後，同學應能夠達成下列目標：

29. 說明從兩張影像推估相機運動的整體思路與適用條件。
30. 說明對極幾何的核心元素：基線、對極點、對極線與對極約束。
31. 區分本質矩陣與基礎矩陣的差異，並說明如何由它們恢復相機運動。
32. 說明 PnP 問題的定義、求解方法與它在 SLAM 中的角色。
33. 說明三角測量的原理與其精度受什麼因素影響。
34. 說明 RANSAC 的運作流程，並理解它為何能對抗大量誤匹配。
35. 把影像處理課程的知識銜接到視覺 SLAM 的幾何求解。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
| --- | --- | --- |
| 對極幾何 | Epipolar Geometry | 描述同一場景兩個視角之間內在幾何關係的理論。 |
| 基線 | Baseline | 連接兩個相機光心的直線。 |
| 對極點 | Epipole | 基線與各影像平面的交點。 |
| 對極線 | Epipolar Line | 一影像中的點，在另一影像中對應的一條直線。 |
| 對極約束 | Epipolar Constraint | 對應點必落在對極線上的幾何限制。 |
| 本質矩陣 | Essential Matrix, E | 已知內參時描述兩視圖關係的 3×3 矩陣。 |
| 基礎矩陣 | Fundamental Matrix, F | 未知內參時描述兩視圖關係的 3×3 矩陣。 |
| 單應矩陣 | Homography, H | 描述平面場景或純旋轉下兩視圖關係的矩 |

| 中文名詞 | 英文名詞 | 概念說明 |
| --- | --- | --- |
| | | 陣。 |
| PnP 問題 | Perspective-n-Point | 由已知三維點與其影像投影求相機位姿。 |
| 三角測量 | Triangulation | 由兩個視角的對應點恢復三維座標。 |
| 視差角 | Parallax Angle | 同一點對兩相機光心的夾角，影響三角測量精度。 |
| 隨機取樣一致 | RANSAC | 以隨機取樣與投票剔除離群點的穩健估計方法。 |
| 內點與外點 | Inlier / Outlier | 符合與不符合估計模型的資料點。 |

## 21.1 兩張影像如何推估相機移動

先建立本章的整體圖像。假設相機在位置 A 拍了一張照片，移動到位置 B 又拍了一張。第 20 章已經幫我們找出了兩張影像中的對應特徵點。現在的問題是：從這些對應點，能不能算出「相機從 A 到 B 的旋轉與平移」？

### 直覺：對應點的移動洩漏了運動

直覺上答案是肯定的。想像你站在教室前方拍一張照，往右走兩步再拍一張：畫面中所有物體都往左移了。若你是往前走，則畫面中的物體會向四周放射狀擴散（近的物體移動大、遠的移動小）。不同的運動，會在影像上產生不同的位移模式——這個模式就是我們要解讀的線索。

### 三種求解情境

視覺 SLAM 中，依「手上有什麼資訊」不同，會用到三種不同的求解方法。這個分類是本章的骨架，同學務必記牢：

| 情境 | 已知資訊 | 求解方法 | 本章節次 |
|---|---|---|---|
| 2D–2D | 兩張影像的對應像素點 | 對極幾何（本質 / 基礎矩陣） | 20.2–20.3 |
| 3D–2D | 已知三維點 + 其影像投影 | PnP 問題 | 20.4 |
| 3D–3D | 兩組對應的三維點 | ICP（第 17 章） | 第 17 章 |

**表 21-1 三種相機運動估測情境**

- 2D–2D：系統剛啟動時，還沒有任何三維地圖點，只能靠兩張影像的像素對應來初始化。這是單目 SLAM 的起手式。
- 3D–2D：一旦地圖建立起來，後續每一幀都能用「地圖中已知的三維點」與「當前影像看到的二維投影」求解位姿。這是 SLAM 運行期間最常用的方法，效率高且穩定。

- 3D–3D：當兩個視角都有深度資訊（雙目、RGB-D）時，可直接對兩組三維點做配準——這正是第 17 章的 ICP，同學已經學過。

本章聚焦前兩者。理解這個分類，同學就能看懂 ORB-SLAM（第 22 章）的原始碼在什麼時候呼叫哪一種求解器。

## 21.2 Epipolar Geometry

對極幾何是本章的理論核心。它回答一個看似簡單卻極深刻的問題：第一張影像中的某個點，在第二張影像中可能出現在哪裡？

### 搜尋範圍從二維降到一維

直覺上，答案似乎是「任何地方」——畢竟我們不知道那個點有多遠。但對極幾何告訴我們一個驚人的結論：它只可能出現在一條特定的直線上。

為什麼？考慮第一張影像中的一個像素點 $p_1$。它對應的三維點 P，必定落在「從第一個相機光心穿過 $p_1$ 的那條射線」上——只是不知道在射線上的哪個位置（這正是第 9 章的深度遺失）。現在把這整條射線投影到第二張影像上，會得到一條直線。無論 P 實際在射線上的哪個位置，它在第二張影像中的投影，必定落在這條直線上。

這條直線就是對極線（Epipolar Line），而這個約束就是對極約束（Epipolar Constraint）。它的實用價值極大：原本要在整張影像（二維）中搜尋對應點，現在只需沿著一條線（一維）搜尋。這既加快了搜尋，也大幅減少了誤匹配的機會。

### 對極幾何的四個元素

- 基線（Baseline）：連接兩個相機光心 $O_1$、$O_2$ 的直線。它是整個幾何的骨幹。
- 對極平面（Epipolar Plane）：由兩個光心與空間點 P 三者決定的平面。不同的 P 會產生不同的對極平面，但它們都包含基線——所有對極平面繞著基線形成一束。

- 對極點（Epipole）：基線與各影像平面的交點，記為 $e_1$、$e_2$。幾何意義是「另一個相機的光心在本影像中的投影位置」。有趣的是：同一張影像中所有的對極線，都會通過該影像的對極點。
- 對極線（Epipolar Line）：對極平面與影像平面的交線。$p_1$ 在第二張影像中的對極線記為 $l_2$。

**圖 21-1　對極幾何的結構**

空間點 P

├──── 投影到影像 1 → 像素點 $p_1$

└──── 投影到影像 2 → 像素點 $p_2$

$O_1$ ──────基線──────→ $O_2$　（兩相機光心）

|　　|

└→ 對極點 $e_1$　　└→ 對極點 $e_2$

對極平面 = $O_1$、$O_2$、P 三點決定的平面

對極線 $l_2$ = 對極平面與影像 2 的交線

【核心約束】$p_2$ 必定落在對極線 $l_2$ 上

→ 搜尋範圍從二維（整張影像）降到一維（一條線）

## 一個生活化的驗證

同學可以親手驗證這個現象：閉上左眼用右眼看前方一枝筆，記住它在視野中的位置；

再閉右眼用左眼看，筆會「跳」到另一個位置。若你把筆前後移動再重複，會發現它在視野中跳動的軌跡，始終沿著一條固定的直線——那就是對極線。人的雙眼，就是一組天然的雙目系統。

## 21.3 Essential Matrix 與 Fundamental Matrix

對極約束若只是文字描述，無法用來計算。本節把它寫成矩陣形式——這就是本質矩陣與基礎矩陣。

### 對極約束的矩陣表示

對極約束的數學形式極為簡潔優美。設 $p_1$、$p_2$ 是兩張影像中的一組對應點（以齊次座標表示），則：

$$p_2^T \cdot F \cdot p_1 = 0 \qquad （基礎矩陣形式，使用像素座標）$$

$$x_2^T \cdot E \cdot x_1 = 0 \qquad （本質矩陣形式，使用歸一化座標）$$

這兩條式子是本章最重要的公式。它們說：對應點與矩陣相乘的結果必須為零——換句話說，任何一組正確的對應點，都必須「滿足」這個約束。反過來，若我們蒐集足夠多的對應點，就能反解出這個矩陣。

### 兩者的差異

- 基礎矩陣 F：作用於原始的像素座標，不需要知道相機內參 K（第 9 章）。它同時混合了「相機的內部參數」與「兩相機的相對運動」兩種資訊。自由度為 7。
- 本質矩陣 E：作用於歸一化座標（已用內參 K 把像素座標轉換過），因此只包含「相對運動」的資訊，更為純粹。自由度為 5（三個旋轉、三個平移，但平移只知方向不知大小，故減一）。

兩者的關係為：

$$E = K_2^T \cdot F \cdot K_1$$

實務上，SLAM 系統的相機都經過校正（第 9.2 節），內參已知，因此多直接使用本質

矩陣。基礎矩陣則在內參未知的情況下使用（例如處理網路上來源不明的照片）。

### 從 E 恢復相機運動

求出本質矩陣後，可以透過奇異值分解（SVD）把它分解為旋轉 R 與平移 t。但這裡有兩個重要的性質必須注意：

- 平移只知方向，不知大小：E 的定義中，把 t 乘上任意非零常數，約束仍然成立。這正是第 20.5 節單目尺度不確定性在數學上的體現——對極幾何本身無法告訴我們「相機移動了幾公尺」，只能告訴我們「往哪個方向移動」。
- 分解有四組解：數學上，分解 E 會得到四組可能的（R, t）組合。要從中挑出正確的那一組，需要用一個物理判準：把對應點三角測量出三維座標後，只有正確的那組解會讓所有點都落在兩個相機的前方（深度為正）。這稱為正深度檢驗（Cheirality Check）。

### 特殊情況：單應矩陣

對極幾何有一個重要的例外情形。當場景中所有點都落在同一個平面上（例如相機對著一面牆、一張桌面、地板），或者相機只做純旋轉沒有平移時，對極約束會發生退化——本質矩陣的求解變得不穩定甚至無解。

此時應改用單應矩陣（Homography,H），它直接描述「一張影像的點如何映射到另一張影像」：

$$p_2 = H \cdot p_1$$

ORB-SLAM 在初始化時的一個聰明設計，就是同時計算基礎矩陣與單應矩陣，依據兩者的擬合分數自動判斷「這個場景比較像平面還是立體」，再選用合適的模型。這個細節在第 22 章會再提到。

### 把整條數學鏈實際算一遍

前面三節都是推導。推導讀起來優美，但同學心裡難免有個疑問：這套東西真的算得出來嗎？

本小節用一組可完整重現的合成資料，把「投影 → 八點法 → 分解 → 正深度檢驗 → 三角測量」整條鏈走一遍。

設定是這樣：相機內參為焦距 520 像素、主點在影像中心、解析度 640×480；場景裡隨機散布 240 個三維點，深度介於 4 至 12 公尺；第二個相機相對第一個旋轉約 9 度、平移 0.66 公尺。把這些點投影到兩張影像上，加入 0.3 像素的高斯雜訊，然後假裝不知道真實運動，只用這些二維對應點把它反推回來。

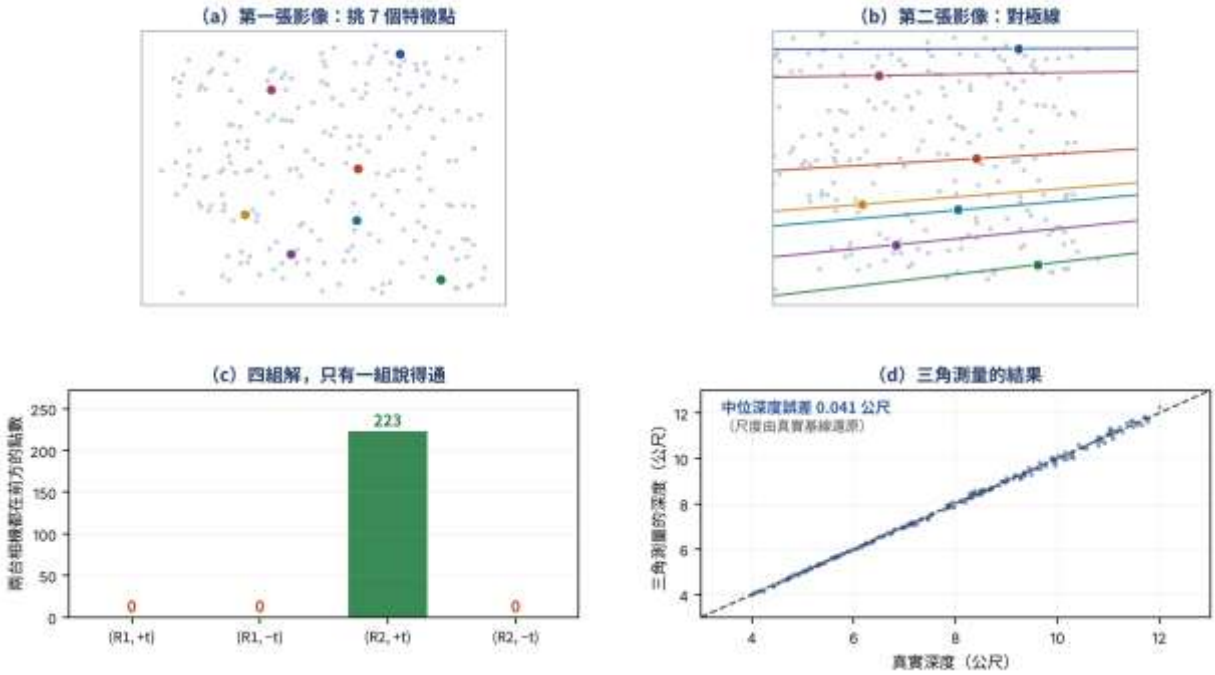



**圖 21-2　對極約束的數值驗證：從對應點到三維座標**

（a）是第一張影像，從 240 個點裡挑了 7 個標上顏色。（b）是第二張影像，每一條彩色直線就是對應顏色那個點的對極線——由基礎矩陣依 $l_2 = F \cdot p_1$ 算出來的。請注意每個彩色圓點都精確落在同色的直線上：這就是對極約束在影像上的樣子。原本要在整張 640×480 的影像裡找對應點，現在只要沿著一條線找。

（b）還透露了一個細節：七條對極線並不平行，它們微微地朝同一側收斂。延長出去，它們會交於同一點——那就是對極點，也就是第一個相機光心在第二張影像中的投影位置。本例的相機主要是橫向平移加小幅旋轉，所以對極點落在影像外的遠處，線看起來幾乎平行。如果相機是朝正前方推進，對極點就會落在影像中央，所有對極線會像放射狀的光芒一樣從

中心散開——同學可以自己改參數試試。

（c）是本章最能說明「數學歸數學、物理歸物理」的一張圖。把本質矩陣做奇異值分解，數學上會得到四組 (R, t) 組合，它們全都滿足對極約束，純粹從代數無法分辨誰對誰錯。但把每一組拿去三角測量，再檢查「這些三維點是不是在兩台相機的前方」，結果是：第三組有 223 個點深度為正，其餘三組一個都沒有。223 比 0 比 0 比 0——這不是勉強勝出，是壓倒性的。正深度檢驗之所以有效，正是因為錯誤的那三組會把整個場景放到相機背後，而那在物理上根本不可能被拍到。

（d）是最後一步：用挑出來的 (R, t) 把 223 個點三角測量回三維，再與真實深度比較。點幾乎全部貼在對角線上，中位深度誤差 0.041 公尺——在 4 至 12 公尺的場景裡約是千分之四。必須說明的是，橫軸的「真實深度」能對上，是因為我們用已知的真實基線把尺度還原了；若是真正的單目系統，整張圖會保持形狀但整體縮放一個未知倍數，這就是第 20.5 節的尺度不確定性。

**零雜訊時，誤差真的是零**

把上面的實驗把雜訊設成 0，八點法恢復出的旋轉誤差是 0.0000 度、平移方向誤差也是 0.0000 度。

這件事值得停下來想一想。它表示對極幾何不是一個「夠用的近似」，而是一組精確的代數關係——誤差完全來自量測，不來自方法本身。這與第 13 章的佔據格地圖、第 14 章的粒子濾波那種「本質上就是近似」的方法，是兩種完全不同的東西。

也因此，當同學在實作中發現位姿估得很差時，不該懷疑公式，而該去查三件事：特徵定位的精度、內參校正的準確度、以及有沒有混進錯誤匹配。接下來兩節就分別處理前兩件與第三件。

## 21.4　PnP 問題

對極幾何處理的是「兩張影像」的情形。但 SLAM 運行起來之後，我們手上已經有一張三維地圖了——此時求位姿有更好的方法。

### 問題定義

PnP（Perspective-n-Point，透視 n 點）問題的定義是：已知 n 個三維空間點的座標，以及它們在影像上的二維投影位置，求相機的位姿（旋轉 R 與平移 t）。

同學應該覺得眼熟——第 4.6 節介紹 AprilTag 時就用過這個概念：標記的四個角點三維座標已知（正方形邊長已知）、它們在影像上的像素座標也偵測得到，於是能解出相機相對標記的完整 6-DoF 位姿。那正是一個 n = 4 的 PnP 問題。

### 為什麼 PnP 比對極幾何好用

- 有絕對尺度：三維點的座標是有單位的（公尺），因此解出的平移也有真實大小——不像對極幾何只能給方向。
- 不需退化判斷：不會有平面場景退化的問題。
- 所需點數少：最少三個點就有解（P3P），遠少於對極幾何所需的點數。
- 效率高：在 SLAM 中，每一幀只需把「地圖中已知的點」投影比對即可，是追蹤階段的主力方法。

### 常見求解方法

- P3P：用三個點求解，會得到最多四組解，需第四個點來排除。是許多方法的基礎。
- EPnP：一種高效的線性解法，能處理任意數量的點，速度快且精度良好，是 OpenCV 與許多 SLAM 系統的預設選擇。
- 直接線性變換（DLT）：把問題轉為線性方程組求解，概念簡單但精度較差，常作為初值。
- 非線性最佳化（BA）：以上述方法求得初值後，再最小化重投影誤差（第 12 章）進行精修。這是實務上必做的最後一步。

### 重投影誤差：PnP 的最佳化目標

PnP 的精確解法，本質上就是最小化重投影誤差——第 12.5 節 Bundle Adjustment 的核心概念：

79. 用當前估計的位姿，把每個三維點投影到影像上，得到「預測的像素位置」。

80. 與「實際偵測到的像素位置」比較，兩者的差就是重投影誤差。

81. 調整位姿，使所有點的重投影誤差平方和最小。

同學可以看出：PnP 其實是「只最佳化相機位姿、固定三維點」的簡化版 BA。而完整的 BA 則同時最佳化位姿與三維點。兩者是同一套數學的不同應用範圍——這也是第 12 章數學工具的又一次現身。

## 21.5 三角測量

前面兩節都是「由對應點求相機運動」。本節是反過來：已知相機運動，由對應點求三維座標。這就是三角測量（Triangulation），它讓單目 SLAM 能夠建立地圖。

### 原理：兩條射線的交點

原理極其直觀。已知相機在兩個位置的位姿後：

82. 從第一個相機光心，穿過像素 $p_1$，畫一條射線。三維點 P 必在這條射線上。

83. 從第二個相機光心，穿過像素 $p_2$，再畫一條射線。P 也必在這條射線上。

84. 兩條射線的交點，就是 P 的三維位置。

理論上完美，實務上卻有個問題：由於像素座標有量測誤差，兩條射線通常不會精確相交，而是「擦身而過」。因此實際做法是求「離兩條射線都最近的點」，或以最小化重投影誤差的方式求解。

### 精度的關鍵：視差角

這是本節最重要的觀念。三角測量的精度，取決於兩條射線的夾角——稱為視差角

（Parallax Angle）。

- 視差角大（相機移動距離相對於物體距離較大）：兩條射線接近垂直相交，交點位置明確，深度估計精準。
- 視差角小（相機幾乎沒動，或物體非常遠）：兩條射線近乎平行，像素的微小誤差就會讓交點沿射線方向大幅滑動，深度極不準確。

這解釋了幾個重要的實務現象：

85. 遠處的點測不準：同樣的相機移動，遠處物體的視差角小得多，因此天空、遠山的深度幾乎無法估計。這與第 9.6 節「雙目深度誤差隨距離平方成長」是同一件事。
86. 純旋轉無法建圖：若相機只轉頭不移動，兩個光心重合，視差角為零，完全無法三角測量。這是單目 SLAM 初始化失敗的常見原因——同學在第 30 章實作時，若一直初始化不成功，多半是「只轉不移」。
87. 需要足夠的基線：因此 SLAM 系統會等到相機移動足夠距離（產生足夠視差）後，才三角測量新的地圖點。這是關鍵幀選擇策略的考量之一（第 22 章）。

### 矛盾與解法

這裡存在一個微妙的矛盾：視差角要大，就需要相機移動較遠；但移動越遠，兩張影像的外觀差異越大，特徵匹配就越困難（第 20 章）。

實務系統的解法是：先用小基線的相鄰幀做穩定追蹤，再用相隔較遠的關鍵幀做三角測量。這種「近幀追蹤、遠幀建圖」的策略，正是 ORB-SLAM 區分「普通幀」與「關鍵幀」的原因之一。

### 量一量：雜訊與基線各值多少

21.5 節說「視差角小的時候深度不準」，這句話對，但不夠具體。不準到什麼程度？基線要多長才夠？本小節用兩組掃描實驗把這兩個問題量出來。

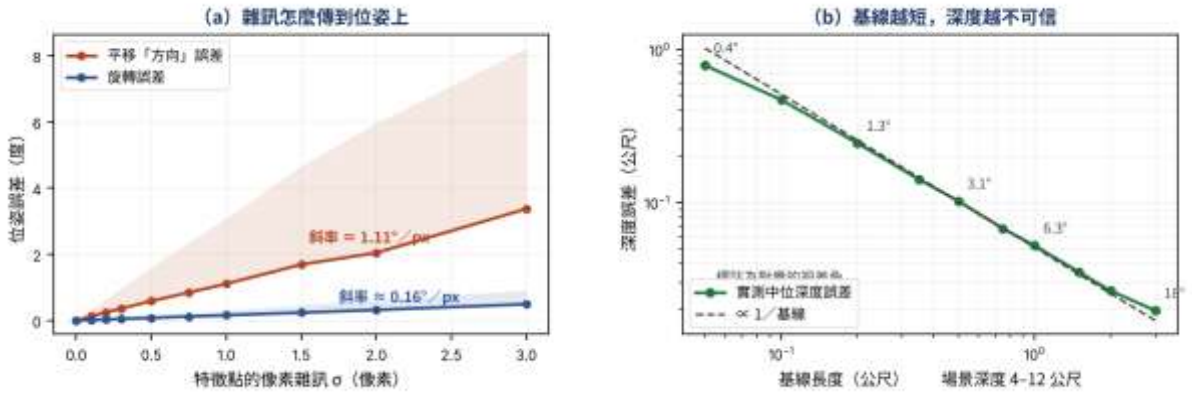



**圖 21-3　雜訊如何傳到位姿，以及基線如何決定深度精度**

（a）固定場景與運動，只改變特徵點的像素雜訊 σ，每個設定跑 40 次取中位數，淡色帶是第 90 百分位。兩條線都幾乎是直線——位姿誤差與像素雜訊成正比，沒有突然崩潰的門檻。這是好消息：特徵定位精度改善一倍，位姿精度就改善一倍。

真正值得記住的是兩條線的斜率差距。旋轉誤差約 0.16 度 / 像素，平移方向誤差約 1.11 度 / 像素——平移方向對雜訊的敏感度是旋轉的七倍。這個不對稱在實務上影響很大：視覺里程計的軌跡之所以會慢慢偏掉，主要不是因為轉角估錯，而是因為每一步的前進方向都歪了一點點。它也解釋了為什麼第 23 章的視覺慣性 SLAM 要把 IMU 加進來——IMU 的加速度計能對平移方向提供獨立的約束，恰好補在視覺最弱的地方。

（b）固定 0.5 像素的雜訊，改變基線長度，量三角測量的深度誤差。兩軸都是對數刻度，實測點幾乎貼著「與基線成反比」的虛線：基線 5 公分時深度誤差 0.79 公尺（在 4 至 12 公尺的場景裡幾乎沒有參考價值），基線 0.5 公尺時降到 0.10 公尺，基線 3 公尺時降到 0.020 公尺。每個點旁邊標的是對應的視差角，從 0.4 度一路到 18 度。

把這張圖倒過來讀更實用：若同學的專題要求深度誤差在 5 公分以內、場景深度約 10 公尺，那麼基線至少要 1 公尺——也就是說，相機必須移動一公尺之後才適合三角測量新的地圖點。這正是 21.5 節「需要足夠的基線」那句話的具體數字，也是第 22 章關鍵幀選擇策略背後的計算依據。

**為什麼實務上用視差角而不用基線**

圖 21-3(b) 的橫軸是基線長度，但真正決定精度的是視差角——也就是基線與場景深度的比值。同樣是 0.5 公尺的基線，對 5 公尺外的物體是 5.7 度的視差，對 50 公尺外只剩 0.57 度。

所以 SLAM 系統判斷「能不能三角測量了」時，看的是視差角而非移動距離。常見的門檻是 1 至 2 度：低於這個值，新點先不建，等相機再走一段。

這也說明了為什麼開闊的戶外場景對單目 SLAM 特別不友善——遠處的建築與地平線永遠達不到足夠的視差，能用來建圖的只有近處的路面與路邊物件。

## 21.6 RANSAC 排除錯誤匹配

前面幾節的數學都很優美，但它們有一個共同的前提：輸入的對應點是正確的。而第 20.4 節告訴我們，誤匹配必然存在。本節介紹對抗誤匹配的關鍵武器。

### 為什麼誤匹配如此致命

傳統的最小平方法（第 12 章）會把所有資料點的誤差平方和最小化。問題在於：平方會放大大的誤差。一個嚴重的誤匹配，可能產生數十甚至上百像素的誤差，平方之後主導整個目標函數，把估計結果硬生生拉偏。

打個比方：全班九十九位同學的身高都在 160 至 180 公分，但資料中誤植了一筆「1800 公分」。用平均值來估計班級身高，結果會被這一筆離譜的資料嚴重扭曲。最小平方法面對誤匹配，就是這個處境。

### RANSAC 的核心思想

RANSAC（RANdom SAmple Consensus，隨機取樣一致）採用完全不同的策略：與其讓所有資料都參與，不如「隨機挑幾個、看看有多少人同意」。

88. 隨機取樣：從所有對應點中，隨機挑出剛好足夠求解模型的最少數量（例如求本質矩陣需要 5 或 8 點，求 PnP 需要 3 或 4 點）。
89. 求解模型：用這幾個點算出一個候選的相機運動（或矩陣）。
90. 投票統計：用這個候選模型檢驗「所有」對應點，計算每個點的誤差。誤差小於門檻的稱為內點（Inlier），記錄內點數量。
91. 反覆迭代：回到步驟 1，重複數十到數百次。
92. 選出最佳：保留內點數最多的那個模型，並用它的所有內點重新做一次精確的最小平方求解。

### 為什麼這樣就有效

關鍵洞見在於：如果隨機挑的幾個點「碰巧全是正確的匹配」，那麼算出的模型就是正確的，而其他正確的匹配也都會同意它（成為內點），因此內點數會很高。反之，若挑到了誤匹配，算出的模型是錯的，幾乎沒有其他點會同意，內點數很低。

於是「內點數最多」就成為判斷模型正確與否的有效準則。RANSAC 不試圖修正誤匹配，而是直接把它們排除在外——這種「多數決」的精神，讓它能容忍極高比例的離群點（有時超過一半）。

### 實務要點

- 迭代次數：需要多少次才能有高機率至少抽中一次「全是內點」的樣本？這可由機率公式估算（見公式或流程）。內點比例越低、所需樣本點數越多，就需要越多次迭代。
- 門檻設定：誤差門檻決定了「多接近才算內點」。太嚴則正確的匹配也被排除、太鬆則誤匹配混入。通常依像素雜訊水準設定，例如 1 至 3 像素。

- 與穩健核函數的關係：RANSAC 是「先剔除再最佳化」的硬性做法；第 12、15 章提過的穩健核函數則是「降低離群點權重」的軟性做法。兩者常搭配使用——先用 RANSAC 剔除明顯的誤匹配，再在最佳化中用穩健核函數處理殘餘的離群值。
- 通用性：RANSAC 不限於視覺。第 17 章的點雲配準、第 13 章的平面擬合、甚至任何「有離群點的模型擬合」問題都能用它。它是工程師工具箱中最實用的方法之一。

## 量一量：RANSAC 到底救回了多少

21.6 節說誤匹配「致命」。這一節把「致命」兩個字量出來。

實驗方法是：在 0.5 像素雜訊的正確匹配中，刻意把一部分對應點的第二張影像座標換成隨機位置（模擬誤匹配），再比較兩種做法——直接把全部匹配丟進八點法，以及先用 RANSAC 篩出內點再估。每個比例跑 25 次取中位數。

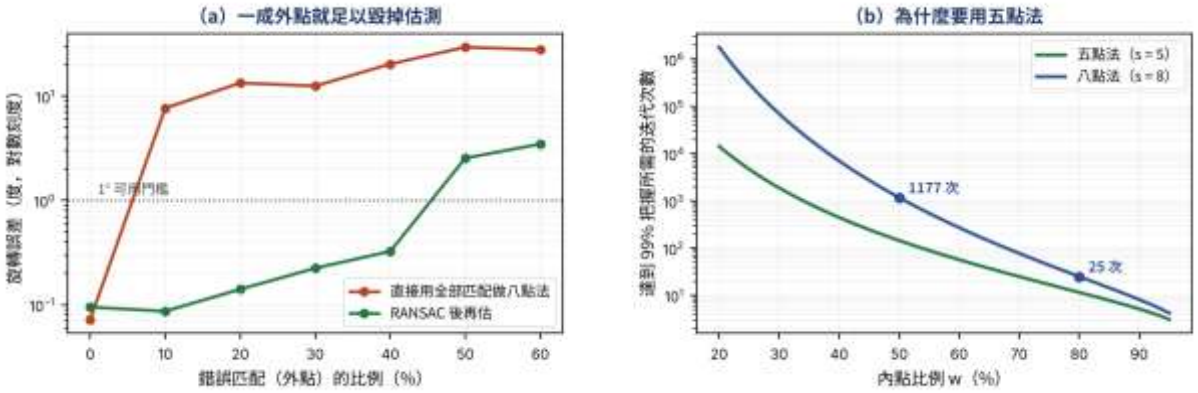



**圖 21-4　誤匹配的代價，以及取樣點數為何關鍵**

（a）的紅線說明了問題的嚴重性。零外點時，直接八點法的旋轉誤差是 0.07 度，相當精準。但只要混進 10% 的誤匹配，誤差就暴增到 7.68 度——一百倍。到 20% 時是 13.4 度，到 50% 時接近 30 度，也就是完全失去意義。請注意縱軸是對數刻度：這條線不是緩慢劣化，而是從一成外點就直接崩掉。

原因在 21.6 節講過：最小平方法把誤差平方後求和，而一個誤匹配可能造成上百像素的誤差，平方之後主導了整個目標函數。換句話說，這不是「精度稍微下降」，而是最佳化的目標本身被少數幾個點綁架了。

綠線是加上 RANSAC 之後。在 10% 外點時誤差是 0.087 度，甚至比零外點時的直接估計還略好一點（因為 RANSAC 順便剔掉了雜訊較大的點）；一路到 40% 外點都還維持在 0.33 度以內，遠低於 1 度的可用門檻。到 50% 時升到 2.5 度，開始吃力——這與理論一致：內點比例掉到一半時，隨機抽到「八個全是內點」的機率只有 $0.5^8 \approx 0.4\%$。

一個誠實的補充：在 50% 與 60% 外點的情況下，RANSAC 判定為內點的比例分別只有 21% 與 12%，明顯低於真實的內點比例。這表示它找到的是一個偏小的一致集合——模型大致正確，但沒有把所有好點都收進來。實務上的對策是增加迭代次數，或改用對外點更穩健的變形（如 MAGSAC、LO-RANSAC）。

（b）回答一個 21.6 節提過但沒算過的問題：要迭代幾次才夠？依內點比例 $w$ 與最少取樣點數 $s$，達到 99% 把握所需的次數為 $N = \log(1-0.99) \,/\, \log(1-w^s)$。內點比例 50% 時，八點法需要 1,177 次，五點法只要 145 次；內點比例 80% 時則分別是 25 次與 12 次。這就是為什麼本質矩陣的五點法在實務上幾乎完全取代了八點法——不是因為它比較準，而是因為它讓 RANSAC 便宜了一個數量級。

> **取樣點數為什麼這麼關鍵**
>
> 關鍵在於 $w^s$ 這一項：$s$ 出現在指數上。內點比例 50% 時，抽中八個全是內點的機率是 $0.5^8 = 0.39\%$，抽中五個全是內點則是 $0.5^5 = 3.1\%$，差了八倍。
>
> 這個觀察在 21.8 節會再出現一次：近年利用單目深度估計的新解法只需要三個點，在同樣 50% 內點比例下只要 34 次迭代——比八點法少了三十四倍。
>
> 所以當同學在設計任何 RANSAC 類的演算法時，第一個該問的問題永遠是：我能不能用更少的點把模型定下來？

## 21.7 從影像處理課程銜接到視覺 SLAM

本章的數學密度較高，本節退一步，幫同學把整條知識脈絡連起來，並說明如何從已學

過的影像處理課程，一步步走到視覺 SLAM。

## 知識的接續

| 階段 | 核心問題 | 關鍵技術 | 本書章節 |
|---|---|---|---|
| 影像處理 | 如何讓影像更清楚 | 濾波、邊緣偵測、形態學 | 先備知識 |
| 相機模型 | 三維如何變成二維 | 針孔模型、內參、畸變校正 | 第 9 章 |
| 特徵工程 | 如何找出可重複辨識的點 | ORB、描述子、匹配 | 第 20 章 |
| 多視圖幾何 | 兩張影像藏著什麼幾何關係 | 對極約束、E / F、PnP、三角測量 | 本章 |
| 視覺里程計 | 如何連續估計相機軌跡 | 逐幀求解與累加 | 第 20、22 章 |
| 視覺 SLAM | 如何消除漂移、建一致地圖 | BA、閉環、關鍵幀 | 第 22、23 章 |

**表 21-2　從影像處理到視覺 SLAM 的知識階梯**

## 三個關鍵的觀念轉折

93. 從「處理影像」到「理解幾何」：影像處理把影像當作二維訊號來處理（濾波、增強）；視覺 SLAM 則把影像視為「三維世界的投影」，關心的是背後的幾何結構。這是最根本的視角轉變。

94. 從「單張影像」到「多視圖關係」：本章的對極幾何是這個轉折的核心——單張影像遺失了深度，但兩張影像之間的關係，足以恢復運動與結構。

95. 從「精確假設」到「穩健估計」：影像處理的演算法多假設輸入是乾淨的；真實的 SLAM 必須面對誤匹配、雜訊與動態物體，因此 RANSAC 與穩健最佳化成為必備工具。這也呼應第 11 章「以機率處理不確定性」的精神。

## 學習建議

- 先建立幾何直覺，再看公式：對極幾何的公式很簡潔，但直覺更重要。務必動手做本章的實作，把對極線畫出來看。
- 善用視覺化：OpenCV 能畫出對極線、內外點、重投影結果。看到圖，抽象的矩陣就有了意義。
- 與光達 SLAM 對照：回頭比較第 17 章的 ICP（3D–3D）與本章的 PnP（3D–2D）、對極幾何（2D–2D），你會發現它們是同一個問題在不同資訊條件下的變形。
- 不必手推所有公式：本質矩陣的 SVD 分解、EPnP 的推導等細節，OpenCV 都已實作。理解「它在解什麼、什麼時候用、會失敗在哪」比背推導更重要。有志研究者可再深入 Hartley 與 Zisserman 的經典教科書。

至此，視覺 SLAM 的數學基礎已備妥。下一章我們把這些零件組裝成一個完整的系統——ORB-SLAM2。它如何用三個執行緒協同工作？關鍵幀與地圖點如何管理？共視圖是什麼？閉環如何偵測與修正？一個真正能跑的視覺 SLAM 系統，即將完整呈現在同學眼前。

## 21.8 對極幾何之後：2024–2026 年兩視圖估測的三個轉向

本章的數學在 1990 年代就已經成熟，而且至今仍是每一套視覺 SLAM 的底層。但這幾年它的「使用方式」正在改變。本節整理三個值得同學知道的轉向。

### 一、更少的點：讓單目深度參與求解

第一個轉向利用了一件新事實：單目深度估計網路（第 9 章提過）現在已經相當好用，雖然它給的深度帶有未知的尺度與偏移。2025 年的一項研究把這個「不完美的深度」直接寫進求解器：在估計相對位姿的同時，一併把深度的尺度與偏移當成未知數解出來。

結果是所需的最少對應點從五個降到三個。依 21.6 節的迭代公式，在 50% 內點比例下，三點法只需 34 次迭代，五點法需 145 次，八點法需 1,177 次。論文報告的實測是：在標準資料集上，三點解法的中位旋轉誤差為 0.89 度、耗時 22.4 毫秒；而搭配當前最佳稠密匹配器

的五點法雖然更準（0.56 度），卻需要 184.4 毫秒。也就是說，精度換到八倍的速度——這在嵌入式平台上是關鍵的取捨。

## 二、更密的匹配：從幾百個特徵到整張影像

第二個轉向來自匹配端。第 20 章介紹的是「偵測特徵點再比對描述子」，一張影像通常得到幾百到一兩千個對應。近年的稠密匹配方法則直接為影像中幾乎每一個像素找對應，一次產生數萬到數十萬組，而且在弱紋理與大視角變化下遠比特徵點穩健。

這對本章的數學產生了一個新問題：RANSAC 原本是為「幾百個對應」設計的，面對數十萬組對應，光是每次迭代要檢驗一遍所有點就慢得無法接受。因此 2025 年出現了專門處理這件事的研究方向——先把稠密匹配「摘要」成少量具代表性的對應，再餵給傳統的兩視圖估測器。這是一個很好的例子：新的前端並沒有淘汰舊的幾何，而是逼著舊的幾何重新設計它的介面。

## 三、直接回歸：跳過本質矩陣

第三個轉向最激進：乾脆不解本質矩陣，而是訓練一個網路，輸入兩張影像、直接輸出它們之間的相對位姿。

2025 年的一項代表性工作以約 800 萬組影像配對訓練，在室內資料集上的相對位姿精度（AUC@20）達到 75.56，明顯高於同類方法的 65.57；在室內定位任務上的平均中位誤差是 0.04 公尺與 1.02 度，而先前最好的同類方法是 0.36 公尺與 18.38 度。速度方面，它每一對影像只要 42 毫秒（約每秒 24 對），而稠密匹配方法需要 300 毫秒、重建式方法需要 441 毫秒。

這是否意味著本章可以不用學了？恰恰相反，有三個理由。其一，這類網路的訓練標註，正是用本章的方法產生的。其二，它們的評估指標（旋轉誤差、平移方向誤差、AUC）完全建立在本章的位姿表示上。其三，也是最實際的：當它估錯時，你需要本章的知識才能判斷錯在哪裡——是視差不足、是場景退化成平面、還是外點太多。

| | 傳統對極幾何 | 稠密匹配 + 幾何 | 直接回歸位姿 |
|---|---|---|---|
| 輸入 | 數百個特徵對應 | 數萬至數十萬組對應 | 兩張影像 |
| 最少點數 | 5 至 8（有深度時 3） | 同左 | 不適用 |
| 可解釋性 | 高，每步可追溯 | 中 | 低 |
| 退化處理 | 需自行偵測（單應） | 同左 | 隱含於訓練資料 |
| 典型耗時 | 數毫秒 | 數百毫秒 | 數十毫秒 |
| 目前角色 | 所有系統的底層 | 高精度離線重建 | 快速初始化與重定位 |

**表 21-3　三種兩視圖位姿估測路線的比較**

務實的結論是：三條路線正在分工而非互相取代。傳統對極幾何因為快、可解釋、不需要訓練資料，仍然是即時 SLAM 前端的預設選擇；稠密匹配用在離線的高精度重建；直接回歸則適合初始化與重定位這類「要快、容錯高」的場合。第 22、23 章要剖析的 ORB-SLAM 系列，走的是第一條路——也是同學最應該先徹底理解的一條。

## 公式或流程

### 一、對極約束

設 $p_1$、$p_2$ 為兩張影像中的一組對應點（齊次像素座標），$K_1$、$K_2$ 為相機內參（第 9 章），則對極約束為：

$$p_2^{T} \cdot F \cdot p_1 = 0$$

把像素座標用內參轉為歸一化座標 $x = K^{-1} \cdot p$，則約束改寫為本質矩陣形式：

$$x_2^{T} \cdot E \cdot x_1 = 0$$

兩矩陣的關係與本質矩陣的組成為：

$$E = K_2^{T} \cdot F \cdot K_1 \quad \cdot E = [t]_x \cdot R$$

其中 $[t]_\times$ 是平移向量的反對稱矩陣（第 10 章）、R 是旋轉矩陣。這條式子清楚顯示：本質矩陣正是「平移與旋轉」的緊湊編碼。由 E 分解出 R 與 t 即得相機運動，但 t 只有方向沒有大小（尺度不確定性）。

## 二、對極線的計算

第一張影像中的點 $p_1$，在第二張影像中的對極線為：

$$l_2 = F \cdot p_1 \qquad （直線方程式 a·u + b·v + c = 0 的係數）$$

反之，$p_2$ 在第一張影像中的對極線為 $l_1 = F^T \cdot p_2$。實作時把 $l_2$ 畫在第二張影像上，就能親眼看見「對應點必落在這條線上」的約束——這是本章實作最有啟發性的一步。

## 三、重投影誤差（PnP 與 BA 的目標）

設三維點 P、相機位姿（R, t）、內參 K、實際觀測到的像素 u_obs，則重投影誤差為：

$$e = u_obs - \pi(K, R, t, P)$$

PnP 求解即最小化所有點的重投影誤差平方和：

$$(R^*, t^*) = \arg\min \Sigma_i \, \| u_obs_i - \pi(K, R, t, P_i) \|^2$$

注意：PnP 只最佳化（R, t），三維點固定；完整的 Bundle Adjustment（第 12.5 節）則同時最佳化位姿與三維點。兩者是同一數學框架的不同應用。

## 四、RANSAC 的迭代次數

設內點比例為 w、每次取樣需要 s 個點、希望以機率 p 至少抽中一次全是內點的樣本，則所需迭代次數為：

$$N = \log(1 - p) \,/\, \log(1 - w^s)$$

舉例：內點比例 $w = 0.5$、求本質矩陣需 $s = 8$ 點、希望成功機率 $p = 0.99$，則 $N \approx 1177$ 次。若改用只需 $s = 5$ 點的方法，N 降至約 145 次——這說明了為何「所需點數少的求解器」在 RANSAC 中如此珍貴，也是五點法比八點法更受青睞的原因。

## 五、視覺前端的完整求解流程

**圖 21-5　從特徵對應到相機位姿的完整流程**

① 特徵匹配（第 20 章）→ 得到一組可能含誤匹配的對應點

↓

② 判斷情境：

- 系統初始化，無地圖點 → 走 2D–2D（對極幾何）
- 已有地圖點 → 走 3D–2D(PnP)

↓

③ RANSAC 迭代：隨機取樣 → 求解模型 → 統計內點

↓

④ 取內點數最多的模型，剔除外點

↓

⑤ 用全部內點做非線性最佳化（最小化重投影誤差）精修位姿

↓

⑥ 三角測量：對視差角足夠的新對應點，恢復三維座標並加入地圖

↓

⑦ 輸出相機位姿 + 新增地圖點 → 交給後端（第 22 章）

# 案例

## 案例 21-1　為什麼站著原地轉圈，單目 SLAM 就掛了

這是同學實作單目視覺 SLAM 時最常遇到的挫折。拿著手機或相機，系統一直顯示「初

始化中」，怎麼樣都成功不了。檢查了相機校正、特徵數量、光線，全都正常——問題到底出在哪？

答案往往極其簡單：你只在原地轉，沒有走動。

依 21.5 節的原理：純旋轉時，兩次拍攝的光心位置完全重合，基線長度為零，視差角為零。此時從兩個光心穿過對應像素的射線完全重疊，根本沒有交點可言——三角測量在數學上無解，系統當然無法建立初始地圖。

更微妙的是，純旋轉時對極幾何也會退化：平移向量 t 為零，本質矩陣 $E = [t]_x \cdot R$ 也變成零矩陣，求解自然失敗。此時應該用的是單應矩陣（21.3 節）——這也正是 ORB-SLAM 初始化時要同時計算兩種模型並自動選擇的原因。

解法很簡單：平移移動。拿著相機小幅度左右或前後平移數十公分，同時保持特徵在視野中，通常幾秒內就能成功初始化。這個案例呼應第 7.7 節「IMU 初始化需要運動激勵」——兩者的共同教訓是：視覺與慣性系統都需要「有內容的運動」才能觀測到某些量。

## 案例 21-2　對極線的視覺化：一眼看懂三維約束

對極幾何的公式看起來抽象，但把它畫出來的瞬間，一切都清晰了。這是大學課堂上最受歡迎的示範之一。

做法是：對同一場景拍兩張視角略不同的照片，用 ORB 找出對應點，以 RANSAC 估計基礎矩陣 F，然後對第一張影像中的每個特徵點，用公式二算出它在第二張影像中的對極線並畫出來。

結果會呈現一組壯觀的畫面：第二張影像上出現許多條直線，而每一個對應的特徵點，都精準地落在它自己那條線上。更神奇的是，若相機是往前移動，這些線會全部匯聚到畫面中的某一點——那就是對極點，也就是「另一個相機光心的投影位置」，幾何上正對應著你行進的方向。

這個實驗的教學價值極高：它讓同學親眼看到「兩張平凡的照片裡，竟然藏著如此嚴謹的三維約束」。抽象的矩陣 F，忽然變成了看得見的線。強烈建議教師把這個示範排進課堂，並讓同學自己拍照實作——本章的實作活動就以此為核心。

### 案例 21-3　RANSAC 的普世價值：超越視覺的方法論

RANSAC 是 1981 年由 Fischler 與 Bolles 提出的方法，原本就是為了電腦視覺中的位姿估計問題（有趣的是，他們提出時處理的正是 PnP 類問題）。但四十餘年來，它的影響力遠遠超出了視覺領域。

它被用在：點雲的平面與圓柱擬合（第 17 章）、光達的地面分離（第 18 章的 LeGO-LOAM）、機器人的感測器校正、生物資訊的序列比對、金融資料的趨勢擬合——任何「資料中混有離群點、需要找出主流模型」的場合。

RANSAC 之所以如此通用，是因為它體現了一個深刻的方法論：當你無法信任每一筆資料時，不要試圖用全部資料求平均，而要尋找「最多資料同意的解釋」。這是一種從「相信資料」轉向「相信共識」的思維轉變。

對同學的啟發是：學習演算法時，除了記住它怎麼做，更要理解它背後的思維模式。RANSAC 的「隨機取樣 + 多數決」、卡爾曼濾波的「按可信度加權」（第 11 章）、圖優化的「把矛盾均攤」（第 12 章）——這些思維框架，遠比具體的公式更能長久受用。

## 實作活動

### 實作 21-1　對極幾何與相機運動估測

目的：親手計算基礎矩陣、把對極線畫出來、執行 PnP 與三角測量，並見證 RANSAC 剔除誤匹配的威力。本實作以 Python 搭配 OpenCV，只需一支手機拍照即可完成，是全書最能建立幾何直覺的實驗之一。

**第一部分：對極線視覺化（全體，本章核心）**

67. 拍攝素材：對教室或走廊拍兩張照片，第二張相對第一張向右平移約 30 公分（注意是平移，不是原地轉）。確保畫面有豐富紋理。

68. 用 ORB 提取特徵並匹配（第 20 章的成果），套用比值檢驗初步剔除誤匹配。

69. 用 OpenCV 的 findFundamentalMat 搭配 RANSAC 估計基礎矩陣 F，同時取得內外點的標記。

70. 依公式二，對第一張影像中的內點，計算它們在第二張影像中的對極線並畫出來；同時把對應的特徵點也畫上。

71. 觀察與驗證：每個對應點是否都落在自己的對極線上？誤差有多大（以像素計）？所有對極線是否匯聚到同一點（對極點）？

72. 對照實驗：改拍一組「原地旋轉」的照片，重複上述流程，觀察估計是否變得不穩定——親身驗證案例 21-1 的退化現象。

### 第二部分：RANSAC 的威力（全體）

73. 不用 RANSAC（改用最小平方法）估計 F，把對極線畫出來，觀察是否明顯偏移。

74. 比較內點數：RANSAC 判定的內點佔全部匹配的百分之幾？被剔除的外點在影像上是什麼樣的匹配（把它們單獨畫出來看）？

75. 人為加入誤匹配：故意在匹配清單中加入若干隨機的錯誤對應，重跑 RANSAC，觀察它是否仍能正確剔除。加到多少比例才會失敗？

76. 依公式四計算：在你的內點比例下，理論上需要多少次迭代才能有 99% 的成功機率？與 OpenCV 的預設值比較。

### 第三部分：PnP 與三角測量（大學部）

77. PnP 實作：用第 9 章校正好的相機拍攝一張含棋盤格（或 ArUco 標記）的照片，已知格點的三維座標，用 solvePnP 求相機位姿，並把座標軸投影回影像驗證。

78. 三角測量實作：用第一部分求得的相機運動，對匹配的內點做三角測量（OpenCV 的 triangulatePoints），把恢復的三維點用 matplotlib 或 Open3D 顯示。

79. 視差角實驗：分別用「平移 10 公分」與「平移 50 公分」的兩組照片做三角測量，比較恢復的三維點雲品質（散布程度、深度合理性），驗證 21.5 節視差角對精度的影響。

80. 尺度驗證：量測場景中某兩點的真實距離，與三角測量結果比較——你會發現比例對但絕對值不對，親身體驗單目的尺度不確定性（第 20.5 節）。

**討論問題**

24. 對極線視覺化的結果中，對應點偏離對極線多少像素？這個誤差來自哪裡（特徵定位誤差？相機校正誤差？）

25. 原地旋轉的那組照片，基礎矩陣的估計為何不穩定？依 21.3 節，此時應該改用什麼模型？

26. RANSAC 剔除的外點，在影像上呈現什麼特徵（是隨機分布，還是集中在某些區域如重複紋理處）？

27. 平移 10 公分與 50 公分的三角測量結果差多少？這對「SLAM 系統何時該三角測量新地圖點」有什麼啟示？

28. （大學部）三角測量恢復的三維點，比例正確但絕對值不對。若要修正，依第 20.5 節你有哪三種選擇？各自的成本如何？

**延伸挑戰（大學部）**

其一：把本章的零件串成一個完整的兩幀視覺里程計——特徵匹配 → RANSAC 估計本質矩陣 → 分解出 R 與 t → 正深度檢驗選出正確解 → 三角測量建立初始地圖點。這正是 ORB-SLAM 初始化模組的簡化版，完成後你會對第 22 章的系統架構有極深刻的理解。其二：實作 ORB-SLAM 的模型選擇策略——同時計算基礎矩陣與單應矩陣，依擬合分數自動判斷場景是平面還是立體，並用合適的模型初始化。用一面白牆與一個立體場景分別測試，驗證選擇是否正確。

## 延伸閱讀

36. R. Hartley and A. Zisserman, Multiple View Geometry in Computer Vision, 2nd ed., Cambridge University Press, 2004.（多視圖幾何的權威聖經，本章所有主題的完整推導來源）

37. M. A. Fischler and R. C. Bolles, "Random Sample Consensus," Communications of the ACM, 1981.（RANSAC 原始論文，提出時即用於位姿估計問題）

38. D. Nistér, "An Efficient Solution to the Five-Point Relative Pose Problem," IEEE TPAMI, 2004.（五點法求本質矩陣，大幅降低 RANSAC 所需迭代次數）

39. V. Lepetit, F. Moreno-Noguer, and P. Fua, "EPnP: An Accurate O(n) Solution to the PnP Problem," IJCV, 2009.（EPnP 原始論文，OpenCV 的預設 PnP 解法）

40. 高翔、張濤，《視覺 SLAM 十四講》第 7 講（視覺里程計：特徵點法）.（對極幾何、PnP、三角測量的華文最佳教材，含完整程式碼）

41. OpenCV 官方文件之 Camera Calibration and 3D Reconstruction 模組（docs.opencv.org）.（findFundamentalMat、solvePnP、triangulatePoints 等函式的使用說明）

42. UZH Vision Algorithms for Mobile Robotics 課程之 Multiple View Geometry 單元（第 2 章介紹）.（免費公開的課程影片與講義，可與本章對照學習）

## 小結

36. 相機運動估測依已知資訊分三種情境：2D–2D 用對極幾何（系統初始化）、3D–2D 用 PnP（SLAM 運行期間的主力）、3D–3D 用 ICP（第 17 章）。理解這個分類，就能看懂 SLAM 系統何時呼叫哪一種求解器。

37. 對極幾何的核心是對極約束：第一張影像中的一個點，在第二張影像中必落在一條對極線上，使搜尋範圍從二維降到一維。四個元素為基線、對極平面、對極點與對極線，且同一影像中所有對極線都通過對極點。

38. 對極約束的矩陣形式為 $p_2^T \cdot F \cdot p_1 = 0$ 與 $x_2^T \cdot E \cdot x_1 = 0$；基礎矩陣用像素座標且不需內參，本質矩陣用歸一化座標且更純粹。由 $E = [t]_x \cdot R$ 分解可得運動，但平移只知方向（尺度不確定），且有四組解需以正深度檢驗篩選。平面場景或純旋轉時應改用單應矩陣。

39. PnP 由已知三維點與其影像投影求相機位姿，優點是有絕對尺度、不會退化、所需點數少、效率高；常用解法包含 P3P、EPnP、DLT，並以最小化重投影誤差的非線性最佳化精修——它本質上是固定三維點的簡化版 Bundle Adjustment。

40. 三角測量由兩條射線的交點恢復三維座標，精度取決於視差角：視差角小（相機幾乎沒動或物體很遠）時深度極不準；純旋轉時視差角為零，完全無法三角測量——這是單目 SLAM 初始化失敗最常見的原因。

41. RANSAC 以「隨機取樣、求解模型、統計內點、反覆迭代、取內點最多者」對抗誤匹配；它不修正離群點而是直接排除，能容忍極高的離群比例。所需迭代次數由內點比例與取樣點數決定，這也是五點法優於八點法的原因。

42. 從影像處理到視覺 SLAM 有三個觀念轉折：從「處理影像」到「理解幾何」、從「單張影像」到「多視圖關係」、從「精確假設」到「穩健估計」。理解演算法在解什麼、何時用、會失敗在哪，比背推導更重要。

## 承先啟後

但這些都還是零件——特徵、對極幾何、PnP、三角測量、RANSAC，如何組裝成一個能長時間穩定運行、能建立一致地圖、能從跟丟中恢復的完整系統？下一章我們剖析 ORB-SLAM2：它如何以三個執行緒協同工作、關鍵幀與地圖點如何管理、共視圖如何組織空間關係、閉環如何偵測與修正。建議帶著本章的三張圖去看它的每一個設計決策：關鍵幀為什麼要等視差夠大才建新點（圖 21-3(b)）、為什麼每一步都要跑 RANSAC（圖 21-4(a)）、以及初始化時為什麼要同時算基礎矩陣與單應矩陣再挑一個。一個真正能跑的視覺 SLAM 系

統，即將完整呈現。

本章備齊了視覺 SLAM 的幾何工具。但這些都還是零件——特徵、對極幾何、PnP、三角測量、RANSAC，如何組裝成一個能長時間穩定運行、能建立一致地圖、能從跟丟中恢復的完整系統？下一章我們剖析 ORB-SLAM2：它如何以三個執行緒協同工作、關鍵幀與地圖點如何管理、共視圖如何組織空間關係、閉環如何偵測與修正。一個真正能跑的視覺 SLAM 系統，即將完整呈現。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

## 第 22 章

## ORB-SLAM2 理論與實作

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

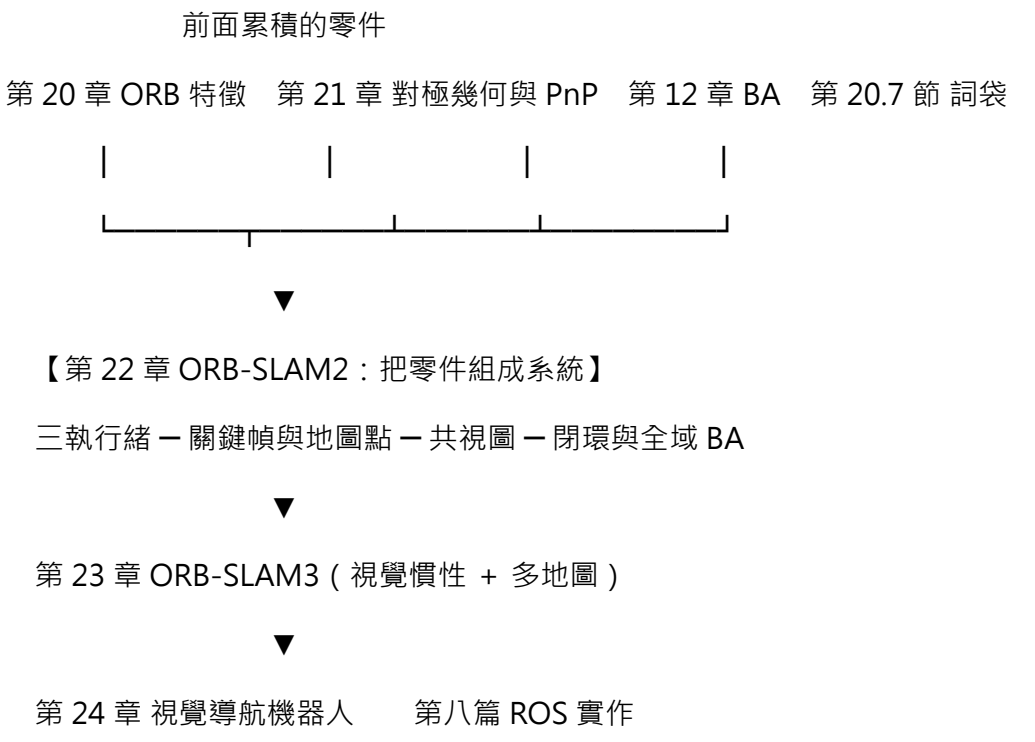
## 本章在全書的位置

第 20 章給了特徵，第 21 章給了幾何。本章把它們組裝成一個真的能跑的系統。

ORB-SLAM2 值得花一整章來讀，不是因為它的某個演算法特別聰明，而是因為它示範了「把正確的零件組起來」需要多少工程判斷：關鍵幀該什麼時候插、新的地圖點該信到什麼程度、地圖長太大了怎麼辦、閉環偵測錯了會怎樣。這些問題在論文的公式裡找不到答案，只能靠系統設計來回答。

本章的三張圖就是為了把這些工程判斷量化。圖 22-1 會數出新地圖點通過五道關卡後還剩多少、擋掉了多少誤匹配；圖 22-2 會示範一件違反直覺的事——閉環閉上了，地圖卻沒修好；圖 22-3 則把共視圖畫出來，說明為什麼它不等於「時間上相鄰的幀」。

### 圖 22-0　第 22 章的連結地圖




## 學習目標

第 20、21 章我們備齊了視覺 SLAM 的所有零件：特徵、對極幾何、PnP、三角測量、RANSAC。但零件不等於系統——如何讓它們協同運作、長時間穩定、跟丟能恢復、閉環

能修正？本章剖析 ORB-SLAM2 這個公認最成熟的開源視覺 SLAM 系統，把理論組裝成真正能跑的架構。學完本章之後，同學應能夠達成下列目標：

36. 說明 ORB 特徵在 ORB-SLAM2 中的角色，以及「同一套特徵貫穿三大任務」的設計優點。
37. 說明單目、雙目、RGB-D 三種模式的初始化差異。
38. 說明追蹤執行緒的工作流程與三種追蹤策略。
39. 說明局部建圖執行緒如何新增地圖點、執行局部 BA 與剔除冗餘。
40. 說明閉環執行緒如何偵測閉環、驗證相似變換並修正全域漂移。
41. 說明關鍵幀、地圖點、共視圖與生成樹的資料結構與作用。
42. 規劃並執行 ORB-SLAM2 在公開資料集上的實作與評測。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 關鍵幀 | KeyFrame | 被保留下來代表某個視角的影像幀，含位姿與特徵。 |
| 地圖點 | MapPoint | 地圖中的三維特徵點，記錄座標、描述子與觀測資訊。 |
| 共視圖 | Covisibility Graph | 連接觀測到共同地圖點的關鍵幀所形成的圖。 |
| 生成樹 | Spanning Tree | 連接所有關鍵幀的最小生成樹，結構輕量。 |
| 本質圖 | Essential Graph | 共視強度高的邊加上生成樹與閉環邊，用於位姿圖優化。 |
| 追蹤 | Tracking | 對每一幀估計相機位姿的前端執行緒。 |
| 局部建圖 | Local Mapping | 管理與最佳化局部地圖的執行緒。 |
| 閉環 | Loop Closing | 偵測重訪並修正全域漂移的執行緒。 |

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 僅運動 BA | Motion-only BA | 固定地圖點、只最佳化相機位姿的束調整。 |
| 局部 BA | Local BA | 最佳化局部關鍵幀與其所見地圖點的束調整。 |
| 全域 BA | Full BA | 最佳化所有關鍵幀與地圖點的束調整。 |
| 地點辨識 | Place Recognition | 以詞袋比對判斷是否曾經來過的模組。 |
| 重定位 | Relocalization | 追蹤失敗後在地圖中重新找回位置。 |
| 相似變換 | Similarity Transform | 含尺度的變換，單目閉環修正時使用。 |

## 22.1 ORB 特徵提取

ORB-SLAM2 由西班牙薩拉戈薩大學的 Mur-Artal 與 Tardós 開發，是視覺 SLAM 領域最具影響力的開源系統。它的名字直接點出核心：整套系統建立在 ORB 特徵之上。

### 為什麼選 ORB：三個理由

第 20 章已經談過 ORB 的速度優勢，但在 ORB-SLAM2 中，選擇 ORB 還有更深的系統層面考量。這些特徵對旋轉與尺度具有穩健性，對相機自動增益、自動曝光與光線變化也有良好的不變性；此外它們提取與匹配快速，能支援即時運作，並在詞袋地點辨識中展現良好的精確率與召回率表現。

- 速度：即時系統的硬性需求（第 20.3 節）。
- 對曝光與光線變化的不變性：這一點常被忽略卻極為關鍵——相機的自動增益與自動曝光會不斷改變影像亮度，若特徵不具此種不變性，系統會頻繁跟丟。這也正是直接法（第 20.2 節）的軟肋。
- 適合詞袋：二元描述子能高效地建立詞彙樹，使地點辨識又快又準。

### 關鍵設計：一套特徵、三種用途

這是 ORB-SLAM2 架構上最優雅的決定之一。系統對追蹤、建圖與地點辨識三項任務，使用的是同一套 ORB 特徵。

這個設計的好處是巨大的：

96. 省下重複計算：影像只需提取一次特徵，三個模組共用。若追蹤用 ORB、閉環用 SIFT，則每幀要算兩套特徵，運算量倍增。
97. 資料結構統一：地圖點只需儲存一種描述子，關鍵幀只需一種詞袋表示。
98. 行為一致：追蹤能匹配上的特徵，閉環也能匹配上，不會出現「追蹤看得見但閉環認不出」的矛盾。

**實作細節：金字塔與均勻分布**

- 影像金字塔：為了獲得尺度不變性，ORB-SLAM2 把影像縮放成多層金字塔，在每一層都提取 FAST 角點。這讓系統能匹配遠近不同的相同物體。
- 均勻分布：直接提取 FAST 角點會造成特徵集中在紋理豐富處（如一幅畫上密集出現），其他區域則空無一物。ORB-SLAM2 用四元樹把影像遞迴切分，確保特徵均勻散布全畫面——這與第 18.2 節 LOAM 分區選取特徵的用意完全相同（避免幾何條件退化）。

## 22.2 單目、雙目、RGB-D 初始化

ORB-SLAM2 的一大特色是同一套系統支援三種相機。但三者的初始化差異極大，這是理解系統的關鍵之一。

**單目：最困難的初始化**

單目沒有深度資訊，必須靠兩張影像的視差來建立初始地圖（第 21 章）。而如同第 21 章案例 21-1 所述，若場景是平面或相機純旋轉，對極幾何會退化。

ORB-SLAM2 的解法極為聰明：它平行計算兩個幾何模型——針對平面場景的單應矩陣，以及針對非平面場景的基礎矩陣——然後依據兩者的相對分數選擇其中之一。

- 若單應矩陣的分數明顯較高，判定場景為平面，以單應矩陣分解出運動。
- 否則以基礎矩陣（本質矩陣）求解。
- 若兩者都不夠好（例如視差不足），則放棄本次初始化，等待下一幀重試。

這正是同學實作時「一直初始化不成功」的機制原因：系統其實在保護你，不讓它用不可靠的模型建出錯誤的地圖。解法一如第 21 章所述——平移移動，製造足夠視差。

**雙目與 RGB-D：立刻就能開始**

雙目與 RGB-D 相機的初始化簡單得多：第一幀就有深度資訊，可以直接把有深度的特

徵點變成地圖點，建立初始地圖，系統立即進入正常追蹤。

- 雙目：由左右影像視差計算深度（第 9.6 節）。
- RGB-D：直接讀取深度圖。
- 共同優點：有絕對尺度、不需等待視差累積、不會因純旋轉而失敗。

### 近點與遠點的區分

ORB-SLAM2 在雙目與 RGB-D 模式中有一個重要的設計：依深度把特徵點分為近點與遠點。近點（深度小於某個倍數基線）的深度量測可靠，可直接三角化為地圖點；遠點的深度不可靠（第 9.6 節深度誤差隨距離平方成長），則像單目一樣，等到多視角觀測累積足夠視差後再三角化。

這個設計展現了工程上的務實：不是所有量測都同等可信，系統應該依可信度分別處理——這與第 11 章卡爾曼增益「按可信度加權」的精神一脈相承。

## 22.3 Tracking 執行緒

ORB-SLAM2 以三個平行執行緒運作。系統有三個主要的平行執行緒：追蹤，對每一幀透過與局部地圖的特徵匹配並套用僅運動 BA 最小化重投影誤差來定位相機；局部建圖，管理並最佳化局部地圖，執行局部 BA；閉環，偵測大迴路並以位姿圖優化修正累積漂移。本節先看第一個。

### 追蹤的任務

追蹤執行緒是前端，它的唯一任務是：對每一張進來的影像，盡快算出「相機現在在哪裡」。它必須即時（與相機幀率同步），因此運算必須輕量。

### 三種追蹤策略

ORB-SLAM2 依情況採用三種策略，由易到難依序嘗試：

99. 依運動模型追蹤：假設相機以等速運動，用上一幀的速度預測當前位姿，再與上一幀的地圖點匹配並精修。最快，也是正常情況下的主要路徑。

100. 依參考關鍵幀追蹤：若運動模型失敗（例如相機突然改變運動），改用詞袋加速與最近的參考關鍵幀做特徵匹配，再求解位姿。

101. 重定位：若前兩者都失敗（追蹤丟失），啟動地點辨識模組，在整個地圖的關鍵幀資料庫中搜尋相似畫面，以 PnP 加 RANSAC 求解位姿，把自己「找回來」。

**追蹤局部地圖**

求出初步位姿後，追蹤執行緒還會做一件關鍵的事：把「局部地圖」中的更多地圖點投影到當前幀，尋找額外的匹配，再做一次最佳化。這讓位姿估計使用更多觀測，精度顯著提升。

局部地圖的取得，靠的是共視圖（22.6 節）：找出與當前幀共視程度高的關鍵幀，以及它們所見的地圖點。這些圖結構讓系統能取得關鍵幀的局部視窗，使追蹤與局部建圖能在局部範圍內運作，因而能在大型環境中工作。

**關鍵幀的選擇**

追蹤執行緒的最後一項工作，是決定「這一幀該不該成為關鍵幀」。判準大致包括：距離上次插入關鍵幀已過足夠幀數、局部建圖執行緒處於閒置、當前幀追蹤到的地圖點數量足夠但與參考關鍵幀的共視比例已下降（表示視角有明顯變化）。

ORB-SLAM2 的策略是「寬鬆插入、嚴格剔除」：先盡量插入關鍵幀以保證追蹤穩健，再由局部建圖執行緒剔除冗餘者（22.4 節）。這個設計讓系統在快速運動時不易跟丟，同時避免關鍵幀無限膨脹。

## 22.4 Local Mapping 執行緒

局部建圖執行緒是中端，它不需即時，但負責維護地圖的品質。局部建圖執行緒把新的點插入局部地圖，並透過執行局部束調整來最佳化局部地圖。

### 五個步驟

102. 插入關鍵幀：新的關鍵幀被插入共視圖、連接關鍵幀與擁有最多共同點的關鍵幀的生成樹，並建立該關鍵幀的詞袋表示（用於三角化新點時的資料關聯）。

103. 剔除劣質地圖點：新建立的地圖點必須通過嚴格檢驗才能留下——必須被足夠多的關鍵幀觀測到、追蹤成功率必須夠高。不合格者立即刪除，避免污染地圖。

104. 建立新地圖點：新地圖點由共視圖中相連關鍵幀的 ORB 特徵三角化產生；某關鍵幀中未匹配的 ORB 與其他關鍵幀中未匹配的 ORB 比較，配對必須滿足對極約束才算有效；成對的 ORB 被三角化後，還要檢查在兩幀中的深度是否為正，並檢驗視差、重投影誤差與尺度一致性。

105. 局部束調整：對當前關鍵幀、共視圖中與它相連的關鍵幀，以及這些關鍵幀所見的所有地圖點，執行局部 BA（第 12.5 節）。這是精度的主要來源。

106. 剔除冗餘關鍵幀：若某關鍵幀所見的地圖點中，有九成以上也被其他至少三個關鍵幀看到，則它是冗餘的，予以刪除。這讓地圖規模不會隨時間無限成長。

### 嚴格的品質控管

同學應該注意到：步驟 2、3、5 都在「刪東西」。這正是 ORB-SLAM2 精度的祕密——它對地圖品質極為嚴苛，寧可少一些地圖點與關鍵幀，也不要不可靠的資料。

三角化新點時的多重檢驗尤其嚴謹：對極約束、正深度、視差角（第 21.5 節）、重投影誤差、尺度一致性——五道關卡缺一不可。這反映了一個工程原則：錯誤的地圖點比沒有地圖點更危險，因為它會持續誤導後續的位姿估計。

### 基線的考量

實作上還有一個細節值得一提：三角化新點時，系統會從共視圖中選取鄰近的關鍵幀（單目模式取較多、非單目取較少），並且只使用其中基線足夠大的那些。這正是第 21.5 節「視差角決定三角測量精度」的直接應用——基線太小的關鍵幀對，三角化出來的點深度不可靠。

### 量一量：五道關卡各擋掉多少

22.4 節說新地圖點必須通過五道檢驗，而且「五道關卡缺一不可」。這句話聽起來很有道理，但到底每一道各擋掉多少？本小節用一個可重現的模擬把它數出來。

設定是一間 14×10 公尺的實驗室，牆面、地板與家具上共有 1,400 個地標；相機沿橢圓軌跡繞行兩圈（第二圈半徑略小，模擬真實的重訪不會走在同一條線上），依 22.3 節的判準選出關鍵幀。接著對系統真的會嘗試的每一對共視關鍵幀，把它們共同看到的地標當作正確候選匹配，再混入 25% 的誤匹配——其中一半是隨機的錯配，另一半刻意製造成「落在對極線上但深度錯誤」的錯配，因為真實的誤匹配往往正是後者（匹配本來就沿著對極線搜尋）。

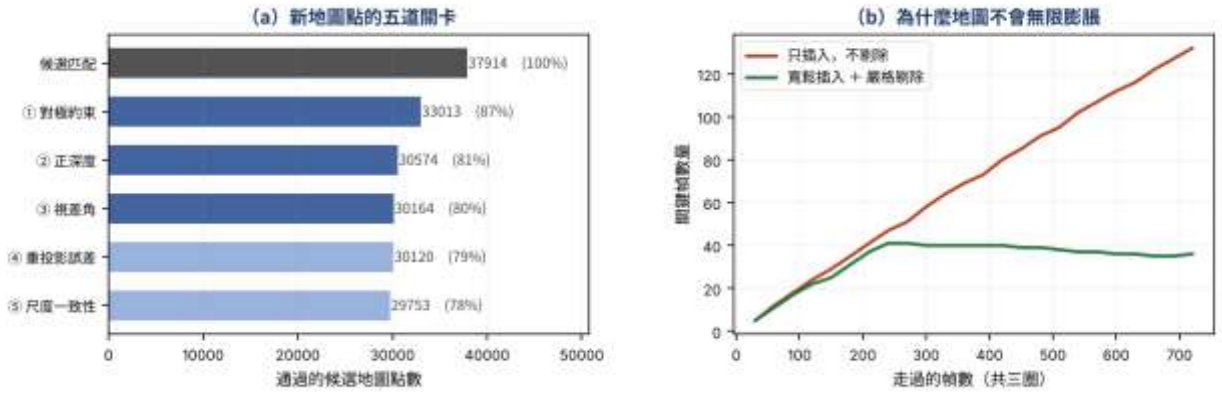



**圖 22-1　新地圖點的五道關卡，以及關鍵幀為何不會無限成長**

（a）是漏斗。37,914 個候選匹配裡，第①道對極約束擋掉 4,901 個（剩 87%），第②道正深度再擋掉 2,439 個（剩 81%），第③道視差角擋掉 410 個（剩 80%），第④道重投影誤差與第⑤道尺度一致性各擋掉數十到數百個，最後剩 29,753 個（78%）。

這張圖最值得討論的是前三道與後兩道的落差。對極約束、正深度、視差角三道合計擋掉了九成以上被擋的候選，而重投影誤差與尺度一致性加起來只擋掉不到 1.5%。這是不是表示後兩道可以省掉？不行，理由有二。其一，被它們擋掉的那幾百個點，正是「通過了前三道卻仍然錯誤」的點——這種點最危險，因為它看起來完全合理。其二，本模擬沒有建模金字塔層級，而真實系統的尺度一致性檢驗正是比對「觀測距離比」與「金字塔層級差」是否

吻合，在真實資料上它擋掉的比例會高得多。

更重要的是品質而非數量。這 29,753 個存活的候選裡，正確匹配佔 93.9%——比原始的 75% 大幅提升；9,475 個誤匹配中有 7,671 個被擋下（81%），而 28,439 個正確匹配裡只有 490 個被誤殺（1.7%）。這個「寧可嚴格一點」的比例配置，正是 22.4 節「錯誤的地圖點比沒有地圖點更危險」那句話的工程實現。

（b）回答另一個問題：既然關鍵幀是「寬鬆插入」，那麼在同一個地方繞三圈，關鍵幀會不會暴增？紅線是只插入不剔除的情況——三圈下來累積 132 個關鍵幀，而且還在線性成長。綠線加上九成規則的冗餘剔除（若某關鍵幀所見的地圖點有九成以上也被其他至少三個關鍵幀看到，就刪掉），結果在第一圈末尾就停在約 40 個，之後三圈都維持在 36 至 41 之間。

這正是 ORB-SLAM2 能長時間運行的關鍵。地圖的規模由「環境有多大」決定，而不是由「跑了多久」決定——這與第 19.6 節巡檢機器人的地圖維護、第 15 章 Cartographer 的子地圖管理，是同一個思想。

**寬鬆插入、嚴格剔除：為什麼不乾脆一開始就嚴格**

一個自然的疑問是：既然最後要刪掉七成，為什麼不一開始就少插一點？

因為插入的時機與剔除的時機，掌握的資訊量完全不同。追蹤執行緒在決定「這一幀要不要當關鍵幀」時，只看得到當下；它不知道接下來相機會往哪走、這個視角日後會不會變得重要。而局部建圖執行緒在剔除時，已經看過後續的關鍵幀，能判斷某一幀是否真的冗餘。

所以這不是「先浪費再收拾」，而是「把決策延後到資訊足夠的時候」。同樣的思想在第 14 章的粒子濾波重取樣、第 15 章的子地圖凍結時機都出現過，是系統設計上一個相當通用的原則。

## 22.5 Loop Closing 執行緒

閉環執行緒是後端，它處理 SLAM 最根本的問題：累積漂移。閉環執行緒負責偵測重訪區域，並使用 g2o 框架以位姿圖優化修正累積誤差；在位姿圖優化之後，它會啟動第四個執行緒執行完整束調整，最佳化整個地圖與關鍵幀之間的軌跡估計。

### 第一步：偵測候選閉環

ORB-SLAM2 基於 DBoW2 函式庫辨識重訪的地點；它把當前影像的特徵以二元格式，與階層式 DBoW2 資料庫中儲存的二元特徵比較，若當前幀與資料庫中某個表示的相似度達到一定分數，即偵測到閉環。

這裡的關鍵是效率：資料庫可能有數千個關鍵幀，不可能逐一做特徵匹配。詞袋模型把每張影像壓縮成一個「視覺單詞的統計向量」，比對只需計算向量相似度，速度極快——這正是第 20.7 節介紹 DBoW2 的價值所在。

### 第二步：幾何驗證

詞袋相似只是「看起來像」，不足以確認。系統接著做嚴格的幾何驗證：對候選閉環幀與當前幀做特徵匹配，並求解兩者之間的變換。若相似度得到足夠多內點的支持，閉環才被接受。

這一步至關重要。第 12 章與第 15.4 節都強調過：一條錯誤的閉環會把地圖硬拉到錯誤位置，造成災難性後果。因此寧可漏掉閉環，也不能接受錯誤的閉環。

### 第三步：相似變換與尺度修正

這裡有一個單目 SLAM 特有的細節。由於單目有尺度漂移（第 20.5 節），閉環時當前的尺度可能與初始尺度不同。因此求解的不是一般的剛體變換（旋轉加平移，第 10 章），而是相似變換——多加一個尺度因子。

修正時，系統會把這個尺度差異也一併攤平到整條軌跡上，同時修正位置與尺度漂移。

雙目與 RGB-D 模式因為有絕對尺度，則只需一般的剛體變換。

### 第四步：位姿圖優化與全域 BA

- 位姿圖優化：在本質圖（22.6 節）上執行位姿圖優化（第 12 章），快速把整條軌跡拉正。因為只最佳化位姿（不含地圖點），速度快，能即時完成——這正是第 15 章 Cartographer 的相同策略。
- 全域 BA：位姿圖優化後，系統啟動第四個執行緒執行完整的束調整，同時最佳化所有關鍵幀位姿與所有地圖點，得到最佳的整體結構與運動解。這一步較慢，因此放在背景執行，不阻塞前端追蹤。

這個「先快速拉正、再精細最佳化」的兩段式設計，同學應該覺得熟悉——它與第 18 章 LOAM 的雙層架構、第 15 章 Cartographer 的雙子系統，是同一個系統設計模式在不同系統中的體現。

### 量一量：閉環到底修好了什麼

22.5 節說閉環「修正累積漂移」。這句話對，但有一個常被誤解的地方：閉環把頭尾接上，不等於中間那一段就正確了。本小節把這件事量出來。

實驗用一個簡化的平面位姿圖（相機在水平面上繞一圈，因此可用二維位姿表示，這讓實驗能完整重現而不失去要點）。里程計每一步加入雜訊，繞完一圈後累積出明顯的漂移。接著比較三種位姿圖優化的設定：只加一條閉環邊、加上本質圖的邊、以及加上完整共視圖的邊。所有殘差都除以該項量測的標準差（第 11、12 章的資訊加權）。

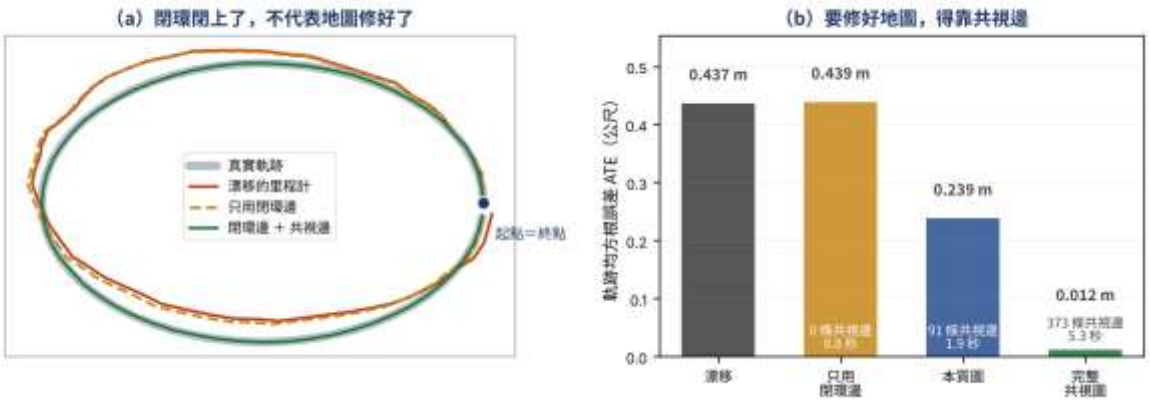



**圖 22-2　閉環閉上了，不代表地圖修好了**

（a）的灰色粗線是真實軌跡，紅線是漂移後的里程計估計，整條軌跡的均方根誤差（ATE）是 0.437 公尺，走回起點時與真正的起點差了 0.220 公尺。

橘色虛線是「只加一條閉環邊」的結果。它確實把頭尾接上了——閉環處的殘餘落差從 0.220 公尺降到 0.018 公尺，終點誤差也從 0.213 公尺降到 0.017 公尺。從「軌跡有沒有閉合」這個角度看，它完全成功。

但看整條軌跡：橘色虛線幾乎與紅線重疊，ATE 是 0.439 公尺——跟沒優化之前一模一樣。換句話說，閉環只把誤差從終點搬走了，卻沒有把它從中間消掉。原因很直觀：一條閉環邊只約束了「第一幀與最後一幀」的相對關係，中間那幾十個位姿彼此之間除了里程計之外沒有任何其他約束，優化器沒有理由去動它們。

綠線是加上共視邊之後。ATE 降到 0.012 公尺，是漂移的三十六分之一，而且與真實軌跡幾乎完全重合。關鍵在於共視邊把「不相鄰但看到同一批地標」的位姿直接綁在一起，形成一張網；閉環的修正量因此能沿著這張網傳遍整條軌跡，而不是只停在接縫處。

（b）把三種設定的代價也一起列出來。只用閉環邊：0 條共視邊、0.8 秒、ATE 0.439 公尺。本質圖：91 條邊、1.9 秒、ATE 0.239 公尺。完整共視圖：373 條邊、5.3 秒、ATE 0.012 公尺。必須誠實地說：在這個模擬裡，本質圖只把誤差改善到一半左右，離完整共視圖差得很遠。它換到的是四分之一的邊數與不到四成的時間。

這個取捨正是 ORB-SLAM2 的設計：閉環發生時先在本質圖上做位姿圖優化，因為它快到可以即時完成、不會卡住前端追蹤；接著在背景執行緒啟動全域 BA，用完整的資訊把地圖精修到位。先用便宜的方法把形狀大致拉正，再用昂貴的方法補完精度——這與第 18 章 LOAM 的雙層架構、第 15 章 Cartographer 的雙子系統，是同一個模式的第三次出現。

> **一個對專題很實用的診斷**
>
> 如果同學的 SLAM 系統「閉環之後看起來有閉上，但地圖還是歪的」，圖 22-2 就是在描述你的情況。
>
> 該檢查的不是閉環偵測，而是共視關係：系統有沒有正確建立非相鄰關鍵幀之間的邊？共視門檻是不是設得太高，導致圖太稀疏？位姿圖優化用的是本質圖還是只有里程計加閉環？
>
> 反過來，如果閉環處有明顯的「接縫斷裂」，那才是閉環偵測或幾何驗證的問題。兩種症狀的成因完全不同，先分清楚再動手。

## 22.6 KeyFrame、MapPoint、Covisibility Graph

一個系統的資料結構，決定了它能做什麼。本節說明 ORB-SLAM2 的四個核心資料結構。

### 關鍵幀（KeyFrame）

關鍵幀是被保留下來的影像幀，代表一個具代表性的視角。它儲存：相機位姿（第 10 章的變換矩陣）、所有 ORB 特徵點與描述子、該幀的詞袋向量、以及與其他關鍵幀的連接關係。

為什麼不保留所有幀？因為相鄰幀的資訊高度重複，全部保留會使 BA 的規模爆炸。關鍵幀是「以最少的幀涵蓋最多視角」的取捨結果。

### 地圖點（MapPoint）

地圖點是三維空間中的特徵點。它儲存：三維座標、代表性描述子（從所有觀測中選出最具代表性的那個）、平均觀測方向、以及「哪些關鍵幀看到了我」的觀測清單。

最後一項至關重要：它建立了「地圖點↔關鍵幀」的雙向連結，使系統能快速查詢「這個關鍵幀看到哪些點」與「這個點被誰看到」。共視圖正是由這個關係推導出來的。

### 共視圖（Covisibility Graph）

系統維護一個共視圖，連接任何兩個觀測到共同地圖點的關鍵幀，以及一個連接所有關鍵幀的最小生成樹。

- 節點：每個關鍵幀。
- 邊：兩個關鍵幀若共同觀測到足夠多的地圖點，就連一條邊；邊的權重是共同觀測的地圖點數量。

共視圖的作用是定義「鄰居」：與當前幀共視程度高的關鍵幀，就是空間上鄰近、視角相似的幀。追蹤與局部建圖都只在這個局部範圍內運作，因此能在大型環境中維持即時性（22.3 節引述的原理）。

### 生成樹與本質圖

- 生成樹（Spanning Tree）：連接所有關鍵幀的最小生成樹，每個關鍵幀連到「與它共視最多的那個關鍵幀」。它極為輕量，保證所有關鍵幀連通。
- 本質圖（Essential Graph）：閉環時做位姿圖優化的結構。它包含生成樹的所有邊、共視權重高於門檻的邊，以及閉環邊。相較於完整共視圖，本質圖的邊少得多，使位姿圖優化能快速完成。

這個「完整共視圖用於局部操作、精簡本質圖用於全域優化」的分層設計，是 ORB-SLAM2 能兼顧精度與效率的關鍵。這些圖結構在閉環時，也作為位姿圖優化所使用的結構。

| 資料結構 | 內容 | 主要用途 |
|---|---|---|
| KeyFrame | 位姿、ORB 特徵、詞袋向量、連接關係 | 地圖的骨架、BA 的變數 |
| MapPoint | 三維座標、描述子、觀測清單 | 追蹤的參考、BA 的變數 |
| 共視圖 | 關鍵幀節點 + 共視權重邊 | 取得局部地圖、定義鄰居 |
| 生成樹 | 最小生成樹的邊 | 保證連通、輕量結構 |
| 本質圖 | 生成樹 + 強共視邊 + 閉環邊 | 閉環時的位姿圖優化 |

**表 22-1　ORB-SLAM2 的核心資料結構**

### 把共視圖畫出來

共視圖是本章最抽象的概念，也最容易被誤解成「時間上相鄰的關鍵幀」。本小節把它實際畫出來。使用的是與圖 22-1 相同的模擬：相機在實驗室裡繞兩圈，第二圈的半徑比第一圈小三成。




**圖 22-3　共視圖、局部地圖與本質圖**

（a）是完整的共視圖：88 個關鍵幀、536 條邊（共視地圖點數達 15 個以上才連線，線的粗細代表共視的強度）。請注意內外兩圈之間密密麻麻的連線——那些就是「跨圈」的共視邊，連接的是時間上相隔數十個關鍵幀、但空間上看著同一面牆的兩幀。這張圖最重要的訊息是：共視關係由空間決定，不是由時間決定。

（b）是局部地圖。紅色星號是當前關鍵幀，橘色是它的局部地圖——共視最強的鄰居，加上這些鄰居的鄰居，一共 23 個關鍵幀。追蹤執行緒只需要把這 23 個關鍵幀所見的地圖點投

影到當前幀，而不是把全部 88 個關鍵幀的地圖點都拿來試。請注意橘色的點同時橫跨內外兩圈：機器人此刻能用上第一圈經過這裡時建立的地圖點，這正是 SLAM 與純里程計的根本差別。

更重要的是，局部地圖的大小不隨地圖成長。這張圖裡是 23 / 88；如果機器人繼續繞十圈、關鍵幀變成四百個，局部地圖仍然只會是當前位置附近的那二十幾個。這就是 22.6 節說共視圖「使系統能在大型環境中工作」的具體意思——追蹤的計算量與地圖總規模無關，只與局部密度有關。

（c）是本質圖：從 536 條邊精簡到 180 條，約三分之一。保留的是最小生成樹（確保每個關鍵幀都連得上）加上共視強度最高的那些邊。它保留了圖的整體連通結構與主要的跨圈連接，卻丟掉了大量冗餘的短邊——這就是閉環時位姿圖優化所使用的圖，也是圖 22-2(b) 裡那 91 條邊的來源。

**三張圖對應三個執行緒**

（a）完整共視圖：由局部建圖執行緒維護，是整個地圖的連接結構。

（b）局部地圖：由追蹤執行緒使用，決定每一幀要拿哪些地圖點來匹配。

（c）本質圖：由閉環執行緒使用，是位姿圖優化的對象。

同一份連接關係，三個執行緒各取所需——這就是 22.6 節「資料結構決定了系統能做什麼」的意思。如果共視圖建錯了，三個執行緒會同時出問題，而且症狀各不相同。

## 22.7 ORB-SLAM2 實作與資料集測試

本節說明實作要點，為第 30 章的完整實作與第 31 章專題預作準備。

### 編譯與相依套件

ORB-SLAM2 是 C++ 專案，編譯前需備妥幾個相依套件：

- OpenCV：影像處理與特徵提取（第 9、20 章）。
- Eigen3：線性代數運算。
- Pangolin：三維視覺化，用於顯示地圖與軌跡。
- g2o 與 DBoW2：已包含在專案的 Thirdparty 資料夾中，分別負責圖優化（第 12 章）與地點辨識（第 20.7 節）。

實作提醒：ORB-SLAM2 發布於數年前，在較新的作業系統與 OpenCV 版本上可能需要修改少量程式碼才能編譯成功。建議參考社群維護的分支版本，或使用容器化環境以避免相依性問題——這是同學實作時最常卡關的地方。

### 組態檔：相機參數

執行前必須準備一個 YAML 組態檔，內容包括：

- 相機內參：fx、fy、cx、cy（第 9.1 節的 K 矩陣）。
- 畸變係數：k1、k2、p1、p2（第 9.2 節）。
- 影像尺寸與幀率。
- 雙目的基線乘焦距、RGB-D 的深度縮放因子。
- ORB 參數：每幀特徵數（常用 1000 至 2000）、金字塔層數與縮放比例、FAST 門檻。

這裡再次強調第 9 章的重要性：相機校正不準，整個系統的精度就無從談起。務必用自己的相機做過完整校正，不要直接套用範例參數。

### 公開資料集

| 資料集 | 相機類型 | 場景 | 特色 |
|---|---|---|---|
| TUM RGB-D | RGB-D | 室內辦公室 | 有動作捕捉真值，適合精度評測 |
| KITTI | 雙目 | 戶外道路 | 自駕車場景，大範圍長軌跡 |
| EuRoC MAV | 雙目 + IMU | 室內機房與廠房 | 無人機視角，含快速運動序列 |

**表 22-2　常用的視覺 SLAM 公開資料集**

建議的學習順序是：先跑 TUM RGB-D 的簡單序列（緩慢運動、室內、有深度）建立成功經驗，再挑戰 EuRoC 的快速運動序列，最後嘗試 KITTI 的大範圍戶外場景。

### 評測方法

- 絕對軌跡誤差（ATE）：把估計軌跡與真值軌跡對齊後，計算對應時刻位置差的均方根。這是最常用的整體精度指標。
- 相對位姿誤差（RPE）：計算固定時間間隔內相對運動的誤差，反映局部漂移程度。
- 工具：evo 是目前最方便的軌跡評測工具，支援多種資料集格式，能自動對齊、計算指標並繪圖。
- 單目特別注意：單目軌跡沒有絕對尺度（第 20.5 節），評測時需先與真值做尺度對齊（Sim3 對齊），否則誤差會毫無意義。

### 觀察重點

執行時，建議同學在 Pangolin 視窗中留意幾件事：

107. 初始化過程：單目模式下，觀察系統需要多少幀、多大的運動才能成功初始化（對照 22.2 節）。
108. 關鍵幀的分布：觀察關鍵幀在軌跡上如何分布——是否在轉彎與視角變化大處較密集？

109. 共視關係：視覺化中關鍵幀之間的連線就是共視圖，觀察它的結構。

110. 閉環時刻：繞回起點時，觀察軌跡與地圖被拉正的瞬間（第 12、15 章的震撼畫面，在此再現）。

111. 追蹤丟失與重定位：刻意遮住相機數秒再放開，觀察系統如何丟失又如何重定位回來——這是 ORB-SLAM2 最令人印象深刻的能力之一。

至此，一個完整的視覺 SLAM 系統已經完整呈現。但 ORB-SLAM2 仍有兩個限制：它是純視覺的（沒有 IMU，因此單目有尺度問題、快速運動易跟丟），而且一旦嚴重跟丟且無法重定位，系統就無法繼續。下一章的 ORB-SLAM3 正是為解決這兩點而生——緊耦合的視覺慣性融合，以及能在跟丟時開啟新地圖、日後再合併的多地圖系統。

## 22.8 ORB-SLAM2 之後：2024–2026 年視覺 SLAM 的兩條岔路

ORB-SLAM2 發表於 2017 年。近十年過去，它仍是比較新方法時最常用的基準線——這本身就說明了它的份量。但視覺 SLAM 的研究確實往兩個方向岔了出去，本節說明這兩條路各自要解決什麼，以及它們與本章內容的關係。

### 一、第一條路：把稀疏地圖換成可算繪的稠密地圖

ORB-SLAM2 的地圖是稀疏的——幾萬個三維點，足以定位，卻不足以「看」。近年的一條主線是把三維高斯潑濺（3D Gaussian Splatting）這類可算繪的表示法接進 SLAM，讓系統在定位的同時也建出能重新算繪成影像的稠密地圖。

值得同學注意的是，這類系統並沒有丟掉本章的東西。2024 年底的一項代表性工作明確地把 ORB 特徵與高斯潑濺緊耦合在同一套最佳化裡：追蹤時同時最小化光度誤差與重投影誤差，後端保留輕量的束調整來抑制累積漂移。作者的理由很直白——純粹依賴光度的高斯潑濺 SLAM 容易產生假影與雜訊、視角選擇不佳、而且缺乏全域最佳化，而這三件事正是本章的關鍵幀策略、共視圖與閉環在處理的。

效果上，該工作在 TUM 資料集的平均軌跡誤差是 0.91 公分，ORB-SLAM2 是 1.00 公分—

—定位精度只是略勝，真正的差別在算繪品質（相較純高斯潑濺的基線提升約 3.93 分貝）。作者也誠實指出限制：訓練耗時可觀，系統尚無法做到即時定位。這是一個很重要的提醒——稠密地圖的代價目前仍然很高。

## 二、第二條路：把前端換成學習式的三維重建模型

第二條路更激進：連特徵匹配與對極幾何都交給網路。2025 年出現的一類方法以學習式的三維重建模型作為前端，直接從兩張影像預測稠密的三維對應，再接上 SLAM 的後端。

它最引人注目的特性是不需要相機校正——系統使用通用的中心相機模型，因此可以直接處理來源不明的影片，這對第 9 章花了整節講內參校正的同學會有點衝擊。速度方面，在高階顯示卡上約每秒 15 幀，在中階卡上約每秒 5 到 7 幀；精度則在部分資料集上與 ORB-SLAM3 及其他稠密方法互有勝負。

## 三、本章的東西還剩多少

| | ORB-SLAM2（本章） | 高斯潑濺 SLAM | 學習式前端 SLAM |
|---|---|---|---|
| 地圖形式 | 稀疏特徵點 | 可算繪的稠密場 | 稠密點雲 |
| 前端 | ORB 匹配 + PnP | 光度 + 特徵 | 神經網路預測對應 |
| 需要內參 | 要 | 要 | 可以不要 |
| 關鍵幀機制 | 有 | 有 | 有 |
| 共視圖與閉環 | 有 | 多半保留 | 多半保留 |
| 即時性 | CPU 即可即時 | 多半尚未即時 | 需高階 GPU |
| 目前定位 | 成熟的基準線 | 算繪品質優先 | 免校正、泛化優先 |

**表 22-3　三條路線的比較**

從這張表可以看出一件事：兩條新路線換掉的是「地圖怎麼表示」與「對應怎麼找」，但關鍵幀、共視關係、閉環偵測與位姿圖優化——也就是本章 22.3 到 22.6 節的全部內容——幾

乎都原封不動地保留著。

理由不難理解。這些機制解決的不是「怎麼算出位姿」，而是「怎麼在長時間運行中管理資訊」：哪些幀值得記住、哪些資料彼此相關、什麼時候該承認自己回到了舊地方、以及誤差該怎麼重新分配。不論前端換成什麼，這些問題都還在。這也是本章值得仔細讀的理由——它教的是系統設計，而系統設計比演算法活得久。

## 公式或流程

### 一、三種 BA 的比較

ORB-SLAM2 在不同階段使用三種束調整（第 12.5 節），差別在於「哪些變數被最佳化」：

$$\text{僅運動 BA：} \operatorname{argmin}_{R,t} \Sigma_i \rho( \| u_i - \pi(K, R, t, P_i) \|^2 )$$

固定地圖點 $P_i$，只求相機位姿（R, t）。用於追蹤執行緒的每一幀，速度最快。

$$\text{局部 BA：} \operatorname{argmin}_{R,t,P} \Sigma \text{局部關鍵幀與其地圖點的重投影誤差}$$

同時最佳化局部關鍵幀的位姿與它們所見的地圖點。用於局部建圖執行緒，是精度的主要來源。

$$\text{全域 BA：} \operatorname{argmin}_{\text{全部 } R,t,P} \Sigma \text{所有關鍵幀與地圖點的重投影誤差}$$

最佳化整張地圖，運算量最大，只在閉環後的第四個執行緒中執行。式中 ρ 為穩健核函數（第 12、15 章），用來降低離群觀測的影響。

### 二、位姿圖優化（閉環修正）

閉環時在本質圖上執行位姿圖優化，只最佳化關鍵幀位姿（不含地圖點）：

$$\operatorname{argmin} \Sigma_{(i,j)\in \text{本質圖}} \| \log( \Delta T_{ij}^{-1} \cdot T_i^{-1} \cdot T_j ) \|^2_{\Sigma}$$

其中 $\Delta T_{ij}$ 是邊所記錄的相對位姿量測、$T_i$ 與 $T_j$ 是待最佳化的關鍵幀位姿。單目模式下 T

為含尺度的相似變換（Sim3），雙目與 RGB-D 則為剛體變換（SE3）。這正是第 12 章位姿圖優化的具體實現。

## 三、三執行緒協同流程

**圖 22-4　ORB-SLAM2 的三執行緒架構**

【追蹤執行緒】即時，每幀執行

提取 ORB → 依運動模型 / 參考幀 / 重定位求初始位姿

→ 追蹤局部地圖 → 僅運動 BA 精修 → 判斷是否插入關鍵幀

↓（新關鍵幀）

【局部建圖執行緒】非即時，關鍵幀觸發

插入共視圖與生成樹 → 剔除劣質地圖點 → 三角化新地圖點

→ 局部 BA → 剔除冗餘關鍵幀

↓（新關鍵幀）

【閉環執行緒】非即時，背景執行

DBoW2 偵測候選 → 幾何驗證 → 求解相似變換（Sim3）

→ 本質圖位姿圖優化（拉正軌跡）

↓

【第四執行緒】全域束調整，最佳化整張地圖與軌跡

## 四、地圖點三角化的五道檢驗

**圖 22-5 新地圖點必須通過的檢驗**

① 對極約束：配對的 ORB 必須滿足對極幾何（第 21 章）

↓

② 正深度檢驗：三角化後在兩幀中的深度都必須為正

↓

③ 視差角檢驗：視差必須足夠大，否則深度不可靠（第 21.5 節）

↓

④ 重投影誤差檢驗：投影回兩張影像的誤差必須夠小

↓

⑤ 尺度一致性檢驗：觀測到的尺度與距離必須合理

↓

五道全過 → 成為地圖點；任一失敗 → 捨棄

## 案例

### 案例 22-1 遮住相機再放開：重定位的魔術時刻

這是展示 ORB-SLAM2 能力最有戲劇效果的示範，幾乎每個課堂都值得做一次。

讓系統正常運行、地圖已建立一段之後，用手完全遮住相機鏡頭。畫面全黑，特徵瞬間歸零，系統立刻顯示追蹤丟失（TRACK LOST），軌跡停止更新。此時把手移開，讓相機重新看到之前走過的區域——不到一秒，系統就「跳」回正確的位置，軌跡無縫接續。

這一秒鐘裡發生了什麼？系統啟動了地點辨識模組：把當前影像的 ORB 特徵轉成詞袋向量，在關鍵幀資料庫中快速搜尋最相似的候選，對候選做特徵匹配，再以 PnP 加 RANSAC（第 21 章）求解位姿，通過驗證後恢復追蹤。

更有趣的變化是：遮住相機後，把相機搬到房間的另一個角落再放開。系統同樣能找回自己——這正是第 3 章「綁架機器人問題」的視覺版本。相較之下，若是純視覺里程計（沒有地圖與地點辨識），遮擋後就永遠回不來了。

這個示範讓同學親身體會「SLAM 與 VO 的差別」：VO 只能一路往前推算，SLAM 卻擁有地圖與記憶，能從失敗中恢復。這也是第 20.1 節流程圖中「閉環偵測」模組價值的最佳註腳。

## 案例 22-2　寬鬆插入、嚴格剔除：一個工程哲學

ORB-SLAM2 對關鍵幀的處理策略，乍看有些矛盾：插入時判準寬鬆（常常插入），但局部建圖又會積極剔除冗餘的關鍵幀。為什麼不乾脆一開始就嚴格一點？

原因在於兩者服務的目標不同。插入時的目標是追蹤穩健：若關鍵幀太稀疏，快速運動或大幅轉向時，當前幀與最近關鍵幀的視角差異過大，匹配就會失敗導致跟丟。寬鬆插入等於為系統買了保險。

剔除時的目標則是地圖精簡：關鍵幀無限增加會使 BA 規模膨脹、記憶體耗盡、系統逐漸變慢。所以在確認某個關鍵幀的資訊已被其他關鍵幀充分涵蓋（九成地圖點被至少三個其他關鍵幀觀測到）之後，就把它移除。

這個「先求穩、後求精」的策略，是一個值得記住的工程哲學：在不確定的當下寧可多做（保留選項），等資訊充足後再回頭精簡。同學會發現這與第 12 章「圖優化能回頭修正歷史」的優勢相呼應——正因為系統能事後修正，當下才敢寬鬆。

## 案例 22-3　從 ORB-SLAM2 長出的家族

ORB-SLAM2 開源之後，成為無數研究的基礎。觀察這個「技術家族」的開枝散葉，能看出視覺 SLAM 的研究地形。

- 加入其他特徵：有研究在點特徵之外加入線段特徵。PL-SLAM 提出的系統同樣基於三個執行緒：視覺里程計、局部建圖與閉環；其地圖由關鍵幀、偵測到的三維地標（同時包含關鍵點與線段）、共視圖與生成樹組成。線特徵在人造環境（走廊、建築）中特別有價值，因為那裡點特徵可能不足但線條豐富。
- 加速與輕量化：有研究提出免描述子的變體。FastORB-SLAM 的結構基於 ORB-SLAM2 並實作三個執行緒，但改以灰階匹配而非描述子匹配來建立相鄰幀之間的關鍵點對應，再以兩階段、與描述子無關的關鍵點匹配方法處理離群點問題，最後透過最小化重投影誤差恢復相機運動。
- 應用延伸：有研究把 ORB-SLAM2 用於視障者的穿戴式導航裝置——由明眼人配戴裝置行走以建立「虛擬導盲路徑」，再供視障者依循。該研究說明追蹤執行緒能透過與局部地圖的特徵匹配定位相機，局部建圖執行緒把新點插入並以局部 BA 最佳化，閉環執行緒偵測大迴路並最佳化位姿圖。
- 持久化地圖：也有研究為自駕車的視覺定位加上地圖儲存功能，讓建好的地圖能存檔重用，系統在定位模式下關閉局部建圖與閉環執行緒，僅執行追蹤。

這些延伸的共同點是：它們都保留了 ORB-SLAM2 的三執行緒骨架，只替換或增添其中的模組。這說明了一個好架構的價值——它不只解決當下的問題，更成為後續創新的平台。同學若要做視覺 SLAM 專題，以 ORB-SLAM2 為基礎進行改良，是一條務實且有前人可循的路徑。

## 實作活動

### 實作 22-1　ORB-SLAM2 系統實作與評測

目的：完整編譯並執行 ORB-SLAM2，在公開資料集上評測精度，觀察三執行緒的協同運作，並實驗系統的極限。這是第六篇最重要的實作，也是第 30、31 章的基礎。

**第一部分：環境建置與資料集執行（全體）**

81. 安裝相依套件（OpenCV、Eigen3、Pangolin），編譯 ORB-SLAM2。若遇到編譯錯誤，記錄問題與解法——這是實務工程的必經訓練。

82. 下載 TUM RGB-D 資料集的一個簡單序列（建議從 freiburg1_xyz 或 freiburg2_desk 開始），以 RGB-D 模式執行，觀察 Pangolin 視窗中的地圖與軌跡。

83. 同一段序列改用單目模式執行，比較兩者的差異：初始化耗時、軌跡完整度、地圖點密度。

84. 觀察並記錄：系統建立了多少關鍵幀？多少地圖點？關鍵幀在軌跡上的分布有什麼規律？

**第二部分：三執行緒行為觀察（全體）**

85. 初始化觀察（單目）：記錄系統從啟動到成功初始化所需的幀數與相機移動距離，對照 22.2 節的原理。

86. 共視圖觀察：在視覺化中觀察關鍵幀之間的連線，判斷哪些關鍵幀共視程度高，並與相機軌跡對照。

87. 閉環觀察：選一段含迴路的序列（如 KITTI 序列 00），觀察閉環觸發瞬間軌跡與地圖被拉正的過程，錄影記錄。

88. 重定位實驗：執行時人為中斷影像輸入數秒（或選用含遮擋的序列），觀察追蹤丟失與重定位的完整過程，重現案例 22-1。

**第三部分：精度評測（大學部）**

89. 安裝 evo 工具，把 ORB-SLAM2 輸出的軌跡與資料集真值比較，計算絕對軌跡誤差（ATE）與相對位姿誤差（RPE）。

90. 繪製軌跡對照圖與誤差隨時間變化圖，找出誤差最大的區段，並回頭觀察該區段的影像，推測原因（快速運動？弱紋理？動態物體？）

91. 單目評測注意：記得先做尺度對齊（Sim3），否則誤差無意義。比較「有無尺度對齊」的誤差數值差異，親身理解尺度不確定性。

92. 三模式比較：若資料集支援，分別以單目、雙目、RGB-D 執行同一場景，比較三者的 ATE 與運算負擔，填入你自己的比較表。

**第四部分：參數實驗與極限測試（大學部）**

93. 特徵數實驗：把每幀提取的 ORB 特徵數分別設為 500、1000、2000，比較精度、幀率與地圖點數量的變化，找出你場景的合適值。

94. 弱紋理挑戰：自行錄製一段包含大面積白牆的影片，觀察系統在哪裡跟丟，驗證第 9、20 章的「白牆詛咒」。

95. 快速運動挑戰：錄製一段快速轉向的影片，觀察追蹤是否失敗，並思考 IMU 能如何幫助（預告第 23 章）。

**討論問題**

29. 單目模式的初始化需要多少幀？若相機只做純旋轉，結果如何？這印證了 22.2 節與第 21.5 節的哪些論述？

30. 系統剔除了多少關鍵幀？若關閉冗餘關鍵幀剔除功能，長時間運行會發生什麼？

31. 閉環拉正的瞬間，軌跡上哪些區段位移最大？為什麼是那些區段？（提示：第 12 章「把矛盾均攤」）

32. 三種模式的 ATE 差多少？RGB-D 是否一定最準？在什麼情況下雙目會勝出？

33. （大學部）特徵數從 500 增至 2000，精度提升多少？幀率下降多少？這個取捨對即時系統意味著什麼？

**延伸挑戰（大學部）**

其一：閱讀 ORB-SLAM2 的 Tracking.cc 原始碼，找出三種追蹤策略（運動模型、參考關鍵幀、重定位）的實作位置，並在關鍵處加上輸出訊息，執行時觀察系統在什麼情況下切換策

略、切換頻率如何。這是把本章理論與真實程式碼對應起來的最佳練習。其二：嘗試用自己的相機（手機或網路攝影機）執行 ORB-SLAM2——先完成第 9 章的相機校正、撰寫 YAML 組態檔、再錄製影片或即時輸入，把你們系館的教室或走廊建成地圖。這份成果可直接作為第 31 章專題的素材。

## 延伸閱讀


43. R. Mur-Artal and J. D. Tardós, "ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo and RGB-D Cameras," IEEE T-RO, 2017.（本章主要依據的原始論文）
44. R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM: A Versatile and Accurate Monocular SLAM System," IEEE T-RO, 2015.（前身之作，單目版本的完整設計）
45. ORB-SLAM2 開源專案（GitHub: raulmur/ORB_SLAM2）.（原始程式碼、編譯說明與資料集執行範例）
46. D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," IEEE T-RO, 2012.（DBoW2，閉環與重定位的技術基礎）
47. H. Strasdat, J. M. M. Montiel, and A. J. Davison, "Scale Drift-Aware Large Scale Monocular SLAM," RSS, 2010.（共視圖與 Sim3 閉環修正的來源）
48. 高翔、張濤，《視覺 SLAM 十四講》第 9 講（後端）與實踐章節.（華文最佳的 ORB-SLAM 系統剖析輔助教材）
49. evo 軌跡評測工具（GitHub: MichaelGrupp/evo）.（ATE 與 RPE 計算、軌跡繪圖，實作評測必備）


## 小結

43. ORB-SLAM2 對追蹤、建圖與地點辨識三項任務使用同一套 ORB 特徵；ORB 對旋轉、尺度、相機自動增益與曝光及光線變化具良好不變性，提取匹配快速，並在詞袋地點辨識中有良好的精確率與召回率。

44. 單目初始化平行計算單應矩陣（平面場景）與基礎矩陣（非平面場景），依相對分數擇一；雙目與 RGB-D 第一幀即有深度，可立即建圖，並依深度把特徵分為可靠的近點與需累積視差的遠點。

45. 追蹤執行緒以僅運動 BA 最小化重投影誤差來定位相機，依序嘗試運動模型、參考關鍵幀與重定位三種策略，並藉共視圖取得局部地圖做進一步精修；關鍵幀採「寬鬆插入」策略。

46. 局部建圖執行緒插入關鍵幀至共視圖與生成樹並建立詞袋、剔除劣質地圖點、由相連關鍵幀三角化新點（須通過對極約束、正深度、視差、重投影誤差與尺度一致性五道檢驗）、執行局部 BA，並剔除冗餘關鍵幀。

47. 閉環執行緒以 DBoW2 比對二元特徵偵測重訪，經幾何驗證取得足夠內點後接受閉環，求解相似變換（單目含尺度修正），在本質圖上做位姿圖優化，並啟動第四執行緒執行全域 BA。

48. 核心資料結構為關鍵幀、地圖點、共視圖（連接觀測共同地圖點的關鍵幀）、最小生成樹與本質圖；共視圖用於局部操作以支援大型環境，精簡的本質圖則用於閉環的位姿圖優化。

49. 實作需備妥 OpenCV、Eigen3、Pangolin，並以 YAML 提供精確的相機內參與畸變係數；常用資料集為 TUM RGB-D、KITTI 與 EuRoC，評測以 ATE 與 RPE 為指標，單目須先做尺度對齊。

## 承先啟後

本章把第 20、21 章的零件組成了一個真能跑的系統，也量出了幾個關鍵數字：新地圖點的

五道關卡把候選從 37,914 個篩到 29,753 個，正確率由 75% 提升到 93.9%，而正確匹配只誤殺了 1.7%；九成規則的冗餘剔除讓繞三圈的關鍵幀數停在 36 個而非 132 個；圖 22-2 則示範了一件違反直覺的事——只有一條閉環邊時，軌跡確實閉合了（接縫落差 0.220 降到 0.018 公尺），整體誤差卻完全沒改善（ATE 0.437 對 0.439 公尺）；要把誤差真正消掉，得靠共視邊把修正量傳遍整條軌跡（ATE 降到 0.012 公尺）。

ORB-SLAM2 是視覺 SLAM 成熟度的標竿，但它仍有兩個明確限制：純視覺架構使單目有尺度問題、快速運動易跟丟；而且嚴重跟丟且無法重定位時，系統便無以為繼。下一章的 ORB-SLAM3 正是為此而生——以最大後驗估測為基礎的緊耦合視覺慣性融合（第 7 章的 IMU 預積分在此登場），以及跟丟時開新地圖、日後再無縫合併的多地圖系統。讀第 23 章時可以帶著一個問題：圖 22-3 的共視圖只有一張，當系統同時持有好幾張互不相連的地圖時，這套機制要怎麼改？第六篇的最高峰，即將登場。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

## 第 23 章

## ORB-SLAM3 與 Visual-Inertial SLAM

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

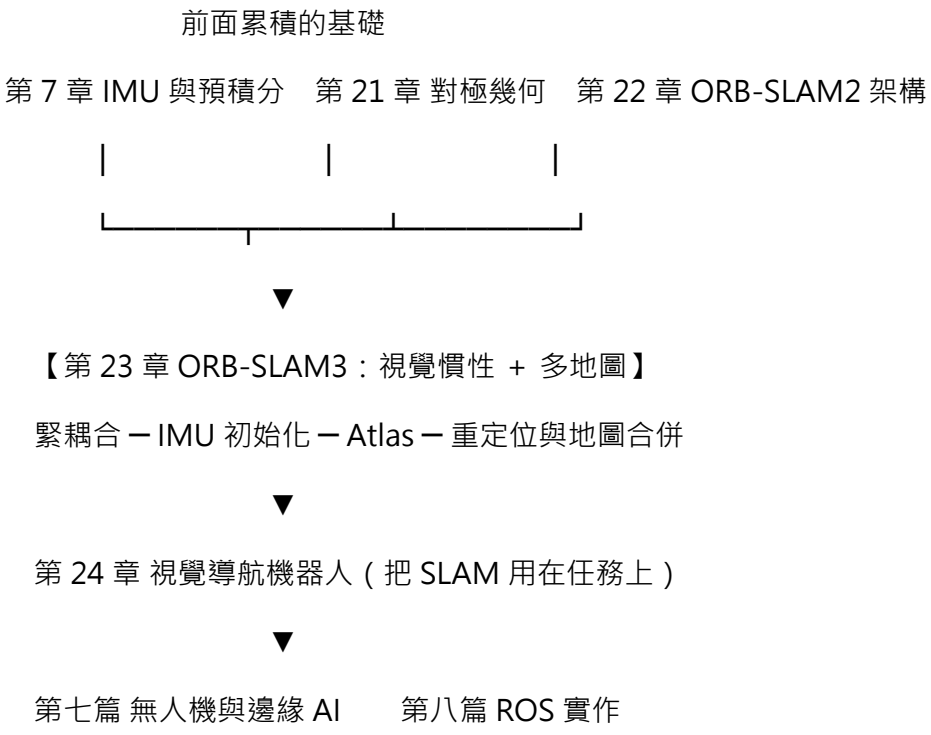
## 本章在全書的位置

第 22 章的 ORB-SLAM2 已經是一套完整的系統，但它有兩個硬傷：單目沒有絕對尺度，而且一旦嚴重跟丟又重定位不了，整個系統就到此為止。本章的 ORB-SLAM3 正是針對這兩件事而生——前者靠 IMU 解決，後者靠多地圖解決。

這也是第六篇的最高峰，以及第二篇與第六篇的交會點。第 7 章講了一整章的 IMU：它為什麼漂、零偏是什麼、為什麼需要運動激勵才能初始化——那些結論在本章全部派上用場，而且會被量化。

本章的三張圖分別回答三個問題：圖 23-1 量出 IMU 在視覺失效時能撐多久，以及零偏估不準的代價；圖 23-2 把「等速直線時尺度不可觀測」從一句口訣變成可以算出來的條件數；圖 23-4 則比較跟丟之後的三種命運——停住、開新圖不合併、以及 Atlas 的無縫合併。

### 圖 23-0 第 23 章的連結地圖




## 學習目標

上一章的 ORB-SLAM2 已是視覺 SLAM 成熟度的標竿，但它仍有兩個限制：純視覺架構使

單目有尺度問題、快速運動易跟丟；而嚴重跟丟且無法重定位時，系統便無以為繼。ORB-SLAM3 正是為此而生。本章是第六篇的最高峰，也是全書視覺路線的集大成。學完本章之後，同學應能夠達成下列目標：

43. 說明 ORB-SLAM3 相對 ORB-SLAM2 的三項核心突破。
44. 說明四種資料關聯（短期、中期、長期、多地圖）的意義與價值。
45. 說明單目慣性與雙目慣性 SLAM 的架構與 IMU 帶來的貢獻。
46. 說明以最大後驗估測為基礎的 IMU 初始化三階段流程。
47. 說明 Atlas 多地圖系統的組成與地圖合併的兩階段策略。
48. 說明追蹤失敗後的重定位與地圖合併機制。
49. 舉例說明視覺慣性 SLAM 在手機、無人機與穿戴裝置上的應用。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 視覺慣性 SLAM | Visual-Inertial SLAM | 緊耦合相機與 IMU 的同步定位與建圖。 |
| 多地圖系統 | Atlas | 同時維護多張地圖的架構，含一張活躍地圖與多張非活躍地圖。 |
| 活躍地圖 | Active Map | 追蹤執行緒當前用來定位的地圖。 |
| 非活躍地圖 | Inactive Map | Atlas 中暫時未使用、但保留待日後合併的地圖。 |
| 短期資料關聯 | Short-term | 匹配最近數秒內的地圖元素。 |
| 中期資料關聯 | Mid-term | 匹配相機附近、帶有輕微累積漂移的地圖元素。 |
| 長期資料關聯 | Long-term | 以地點辨識匹配先前造訪區域，不受累積漂移影響。 |
| 多地圖資料關聯 | Multi-map | 匹配並使用先前建圖階段的地圖元素。 |

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 最大後驗估測 | MAP Estimation | 在已知觀測下尋找機率最大的狀態估計。 |
| IMU 初始化 | IMU Initialization | 估出尺度、重力方向、零偏與速度的啟動程序。 |
| 地圖合併 | Map Merging | 把兩張地圖無縫融合為一的操作。 |
| 焊接視窗 | Welding Window | 地圖合併時先行融合的局部關鍵幀範圍。 |
| 多階段作業 | Multi-session | 跨多次執行、重複使用先前地圖的運作模式。 |

## 23.1 ORB-SLAM3 的系統特色

ORB-SLAM3 由 Campos、Elvira、Rodríguez、Montiel 與 Tardós 於 2021 年發表，是第 7.7 節已初步介紹過的系統。它是特徵法 SLAM 這條血脈的集大成之作，從 PTAM、ORB-SLAM、ORB-SLAM2 一路演進而來，發布時是當時最完整、最精確的開源 SLAM 函式庫。

### 三項核心突破

- 緊耦合的視覺慣性 SLAM：第一個以最大後驗估測（MAP）為基礎、連 IMU 初始化階段也建立在 MAP 之上的緊耦合系統（第 7.7 節）。這解決了單目的尺度問題與快速運動下的穩健度。
- 多地圖系統（Atlas）：跟丟時不再崩潰，而是開啟新地圖繼續運作，日後重回舊區域時再無縫合併。這解決了 ORB-SLAM2「跟丟即結束」的致命弱點。
- 四種資料關聯的全面運用：這是最根本的架構創新。ORB-SLAM3 是第一個在所有演算法階段都運用短期、中期、長期與多地圖資料關聯的系統。

### 支援的感測器組態

第 7.7 節提過，ORB-SLAM3 支援極為廣泛的組態，這是它實用價值的重要來源：

112. 相機類型：單目、雙目、RGB-D。
113. 鏡頭模型：針孔與魚眼（第 9 章）。魚眼支援對室內大視角應用特別有價值。
114. 是否含 IMU：每一種相機模式都有對應的視覺慣性版本。

把這些組合起來，系統支援單目、單目慣性、雙目、雙目慣性、RGB-D、RGB-D 慣性等多種模式——同一套程式碼，換個參數即可。這讓它成為研究與教學的絕佳平台：同學能在完全相同的框架下，比較不同感測器組態的表現。

### 精度表現

第 7.7 節已引述過關鍵數據，此處補充多階段作業的表現。在 EuRoC 多階段實驗中，

Atlas 的地圖合併使精度較 CCM-SLAM 與 VINS-Mono 提升一倍以上；相對 VINS-Mono 的優勢從單階段的 2.6 倍成長到多階段的 3.2 倍。整體而言，論文摘要宣稱其精度較先前方法提升二至十倍；開源釋出後，它成為數百篇後續論文的標準比較基準。

這個「單階段已勝出、多階段優勢更大」的現象，清楚說明了多地圖架構的價值：能重複使用先前的地圖，精度自然更高。這對實際部署的機器人（每天在同一場域運行）意義重大。

## 23.2 四種資料關聯：系統的架構主軸

本節是理解 ORB-SLAM3 設計哲學的關鍵。作者把「資料關聯」——也就是「判斷不同時刻的觀測是否對應同一個地圖元素」——分為四個層次，並主張先前的系統往往只用了其中一兩種。

| 類型 | 匹配對象 | 技術手段 | 解決什麼 |
|---|---|---|---|
| 短期 | 最近數秒內的地圖元素 | 投影匹配 | 基本的逐幀追蹤 |
| 中期 | 相機附近、有輕微漂移的元素 | 局部地圖投影匹配 | 減少漂移累積，提升精度 |
| 長期 | 先前造訪區域（不論漂移多大） | 地點辨識（DBoW2） | 閉環修正、重定位 |
| 多地圖 | 先前建圖階段的地圖元素 | 地點辨識 + 地圖合併 | 跨階段重用、跟丟後恢復 |

**表 23-1 四種資料關聯的比較**

### 短期與中期：日常運作的基礎

幀與活躍地圖之間的短期與中期資料關聯，是由追蹤與建圖執行緒例行找出的——把地圖點投影到估計的相機位姿，並在影像中僅數個像素的視窗內搜尋匹配。

中期關聯是 ORB-SLAM3 精度優勢的重要來源。實驗顯示，在 EuRoC 資料集的單目慣性單階段運作中，ORB-SLAM3 的精度是 VINS-Mono 的 2.6 倍，這歸功於它使用中期資料關聯的能力。

為什麼中期關聯如此重要？因為許多視覺里程計系統只做短期關聯（只與前幾幀比對），一旦某個地圖點暫時離開視野再回來，系統就把它當成新的點——這浪費了資訊，也讓漂移持續累積。ORB-SLAM3 則能重新匹配上這些「還在附近但有些漂移」的舊點，等於不斷地做小規模修正。

### 長期與多地圖：恢復與重用

為了達成重定位與閉環偵測所需的長期資料關聯，ORB-SLAM 使用 DBoW2 詞袋地點辨識系統，這個方法也被多數近期的 VO 與 SLAM 系統所採用。

多地圖關聯則是 ORB-SLAM3 獨有的第四層。它讓系統能匹配「不同建圖階段」的地圖元素——無論是同一次執行中跟丟後開的新地圖，還是昨天建好存下來的地圖。這開啟了多階段作業的可能。

### 觀念總結

這四層關聯可以用一個記憶的比喻來理解：短期是「剛剛看到的」、中期是「這附近熟悉的」、長期是「以前來過的」、多地圖是「別次經歷的」。一個好的 SLAM 系統，應該像人一樣善用所有層次的記憶——這正是 ORB-SLAM3 的設計哲學。

## 23.3 Monocular / Stereo Visual-Inertial SLAM

第 7.6 節已經說明過 IMU 在視覺慣性 SLAM 中的四大貢獻（絕對尺度、重力方向、填補視覺失效空隙、高頻輸出），本節聚焦 ORB-SLAM3 如何實現緊耦合。

### 為什麼需要緊耦合

IMU 量測角速度與線加速度，但存在顯著漂移，使得單靠 IMU 資料的二次積分無法用

於長期位姿估計。這正是第 7.3 節的核心結論。

反過來說，純視覺在快速運動、弱紋理、光線劇變時會失效。因此兩者必須融合——而如第 7.6 節所述，緊耦合（把 IMU 預積分約束與視覺重投影誤差放進同一個最佳化問題）的精度顯著優於鬆耦合。

### ORB-SLAM3 的緊耦合實現

系統的三個執行緒都因 IMU 而擴充：

115. 追蹤執行緒：除了最小化重投影誤差，還加入 IMU 預積分的慣性誤差項（第 7.5 節），同時估計位姿、速度與 IMU 零偏。

116. 局部建圖執行緒：局部建圖模組把關鍵幀與地圖點加入當前活躍地圖、移除冗餘的幀，並在當前幀的鄰近關鍵幀上，以視覺束調整或視覺慣性束調整最佳化地圖。

117. 閉環與地圖合併執行緒：在慣性模式下，閉環修正還需處理重力方向與尺度的一致性。

### 狀態向量的擴充

與純視覺相比，視覺慣性 SLAM 每個關鍵幀要估計的量變多了：

118. 純視覺：位姿（旋轉 R、平移 t），共 6 個自由度。

119. 視覺慣性：位姿 + 速度 + 加速度計零偏 + 陀螺儀零偏，共 15 個自由度。

狀態變多雖然增加了運算量，但換來的是：絕對尺度（單目不再有尺度不確定性）、重力方向已知（地圖自動擺正，滾轉與俯仰角可觀測）、以及在視覺短暫失效時仍能維持追蹤數秒的能力。這個交換在絕大多數應用中都極為划算。

### 雙目慣性：最強健的組態

若不計成本與體積，雙目慣性是 ORB-SLAM3 最強健的組態：雙目提供絕對尺度與單幀深度（不需等待視差累積）、IMU 提供高頻運動與快速運動下的穩健度。第 7.7 節引述的 EuRoC 3.5 公分精度，正是雙目慣性模式的成績。對校園的巡檢機器人或無人機專題，若條

件允許，這是首選組態。

## 量一量：IMU 到底能撐多久

23.3 節說緊耦合能「在視覺短暫失效時仍能維持追蹤數秒」。「數秒」是多少秒？本小節把它算出來，順便回答一個更重要的問題：這個能力取決於什麼。

實驗設定：讓載體做一段含加減速與旋轉的運動，IMU 以 200 Hz 輸出比力讀值（含零偏與雜訊）。從某一刻起假設視覺完全失效，持續 τ 秒，期間只靠 IMU 二次積分推算位置，最後比較推算位置與真實位置的誤差。關鍵變因是「系統對加速度計零偏估得多準」，因為推算時要先把估計的零偏扣掉。

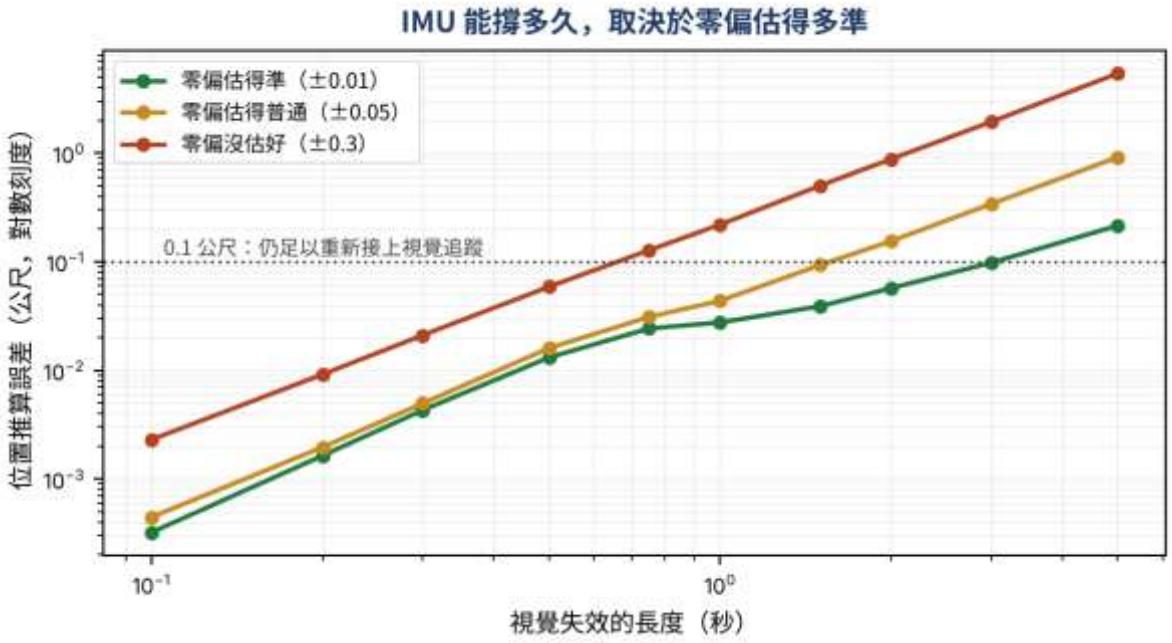



**圖 23-1　IMU 能撐多久，取決於零偏估得多準**

三條線的形狀都是二次成長——這是第 7.3 節的必然結果：加速度的誤差積分兩次變成位置誤差，所以誤差隨時間平方累積。真正的差別在於三條線之間的垂直距離。

若零偏估到 ±0.01 公尺 / 秒² 的水準（綠線），視覺失效 1 秒的位置誤差是 0.028 公尺、2 秒是 0.057 公尺、即使 5 秒也只有 0.21 公尺。這個精度足以讓系統在視覺恢復時直接把地圖點投影回畫面重新接上——也就是「使用者幾乎不會察覺中斷」的技術基礎。

若零偏只估到 ±0.3 公尺 / 秒²（紅線），同樣的 1 秒誤差是 0.22 公尺、2 秒是 0.88 公尺、5 秒是 5.39 公尺。與綠線相比，2 秒時差了 15 倍。0.88 公尺的誤差意味著重新投影時完全對不上原來的地圖點，系統只能走重定位流程，甚至直接跟丟。

這組數字解釋了一件事：IMU 在視覺慣性 SLAM 裡不是「備援感測器」那麼簡單。它能不能救場，取決於零偏有沒有被持續而正確地估計——而零偏的估計又回頭依賴視覺提供的位置約束。兩者是互相依賴的，這正是 23.3 節強調「緊耦合」而非「鬆耦合」的根本理由：把 IMU 當成獨立模組接上來，零偏就永遠估不準。

> **為什麼零偏比雜訊重要得多**
>
> IMU 的隨機雜訊是零均值的，積分之後會互相抵消一部分，誤差大致隨 $\sqrt{t}$ 成長。但零偏是常數偏移，積分兩次之後誤差隨 $t^2$ 成長——兩者完全不是同一個量級。
>
> 以本實驗的數字為例：加速度計雜訊是 0.02 公尺 / 秒²，而未校正的零偏可達 0.1 公尺 / 秒²以上。五秒之後，雜訊貢獻的誤差不到幾公分，零偏貢獻的卻是公尺級。
>
> 所以同學在調校視覺慣性系統時，若發現「靜止時位置會慢慢飄走」，要查的是零偏估計而不是雜訊參數。

## 23.4 IMU 初始化與預積分

第 7.7 節已介紹過 IMU 初始化的困難與運動激勵的必要性，本節深入其三階段流程——這是 ORB-SLAM3 最重要的技術貢獻之一。

### 初始化要解出什麼

系統啟動時，有一堆彼此耦合的未知數必須同時確定：

120. 尺度：單目模式下地圖的真實大小（雙目與 RGB-D 已知，故較單純）。
121. 重力方向：哪個方向是「下」，用來從加速度計讀值中扣除重力（第 7.1 節的比力觀念）。

122. IMU 零偏：加速度計與陀螺儀的偏移（第 7.3 節）。

123. 初始速度：每個關鍵幀的速度。

### 三階段流程

ORB-SLAM3 把初始化拆成三個階段，逐步逼近：

- 純視覺 MAP 估測：先只用視覺，建立一段短時間的地圖與軌跡（單目模式下尺度未知）。這提供了後續慣性估計所需的運動基準。
- 純慣性 MAP 估測：固定視覺軌跡不動，只用 IMU 量測來估計尺度、重力方向、零偏與速度。因為視覺軌跡已定，這個子問題變得相對簡單。
- 聯合視覺慣性 BA：最後把所有變數（位姿、速度、零偏、尺度、重力、地圖點）放進同一個最佳化問題聯合精修。

這個「先分開解、再聯合精修」的策略，大幅提升了初始化的速度與可靠度。

### 初始化的表現

IMU 初始化在 2 秒內達到 5% 的尺度誤差，經精修後約為 1%。這相較第 7.7 節提到的早期系統（ORB-SLAM-VI 需約 15 秒）是巨大的進步——2 秒對使用者體驗而言幾乎無感，15 秒則明顯令人不耐。

### 實作提醒：運動激勵

再次強調第 7.7 節的關鍵提醒：IMU 初始化需要足夠的運動激勵。若載體只是等速直線移動或完全靜止，加速度計讀到的幾乎只有重力，系統無法把「尺度」與「零偏」區分開來（數學上不可觀測）。

實務操作：啟動後拿著裝置做數秒帶有加減速與旋轉的運動（小幅度前後左右移動並轉動），系統才能順利完成初始化。這與第 21 章「純旋轉無法三角測量」並列為視覺慣性 SLAM 兩大實作陷阱——同學在第 30 章實作時務必記得。

### 量一量：為什麼非得晃一晃不可

23.4 節說「若載體只是等速直線移動，系統無法把尺度與零偏區分開來（數學上不可觀測）」。這句話很重要，但「不可觀測」聽起來像個抽象的名詞。本小節把它變成一個可以算出來的數字。

做法是把初始化的第二階段（固定視覺軌跡、純慣性估計）寫成一個線性方程組。未知數是：每個關鍵幀的速度、重力向量、尺度因子、以及加速度計零偏。方程式則來自相鄰關鍵幀之間的預積分關係——位置式與速度式各一組。把這個方程組的係數矩陣做奇異值分解，最大與最小奇異值的比值（條件數）就直接告訴我們問題好不好解。




**圖 23-2　運動激勵決定了初始化能不能解**

（a）畫出四秒內的三種運動：等速直線（紅）、輕微晃動（橘，橫向擺動約 3 公分）、以及充分激勵（綠，含前後加減速、橫向與上下移動、加上偏航與俯仰的轉動）。請注意這三條軌跡在視覺上都「有在動」——光看軌跡，很難預料它們的初始化結果會差這麼多。

（b）是結果，縱軸是對數刻度。等速直線的尺度誤差是 100.3%——不是「不夠準」，是完全解不出來；實際解出的尺度甚至可能是負值。條件數高達 3,820。輕微晃動把尺度誤差拉到 1.2%，充分激勵則是 0.2%，條件數降到 157，比等速直線小了二十四倍。

為什麼等速直線會完全失敗？道理其實很直白。加速度計量到的是比力，也就是「真實加速度減去重力」在機體座標系下的投影。等速直線運動時真實加速度是零，而姿態又固定不變，所以加速度計讀到的永遠是同一個常數向量。這個常數可以解釋成「重力的投影」，也可以

解釋成「零偏」，兩種解釋在數學上完全等價——方程組根本沒有足夠的資訊把它們分開。

這裡還有一個值得注意的細節：輕微晃動雖然把尺度救回來了（1.2%），但它的條件數仍有 3,559，幾乎與等速直線一樣高，而且加速度計零偏的誤差反而是三者中最大的。換句話說，稍微晃一下足以讓尺度變得可觀測，卻不足以把零偏分離出來。這正好呼應圖 23-1 的結論——零偏估不準，IMU 就撐不過視覺空隙。

（c）回答另一個問題：初始化要花多久。固定「充分激勵」的運動，把時間窗從 0.5 秒掃到 10 秒。0.5 秒時尺度誤差是 15.6%，0.8 秒降到 0.9%，2 秒時是 0.7%，之後曲線明顯轉平——10 秒才降到 0.06%。也就是說，兩秒左右是報酬遞減的轉折點：再等下去精度改善有限，使用者的耐心卻是線性消耗的。這就是 23.4 節引述「2 秒內達到 5% 尺度誤差」之所以被視為重要突破的原因。

**本實驗與真實系統的差距**

本實驗為了能完整重現，做了幾個簡化：沒有陀螺儀零偏、視覺軌跡的雜訊是獨立同分布、預積分對零偏的敏感度只取一階近似。因此這裡的絕對數字（例如 2 秒時 0.7%）比真實系統樂觀。

真實系統在 2 秒時約為 5%，再經第三階段的聯合視覺慣性 BA 精修到約 1%。但三個定性結論完全一致：等速直線不可觀測、晃一下就能救回尺度、兩秒之後報酬遞減。

同學若要在自己的裝置上重做這個實驗，最有價值的產出不是絕對數字，而是「你的裝置需要晃多大、晃多久」——這會直接寫進你的操作手冊。

## 23.5 Multi-Map 多地圖系統（Atlas）

這是 ORB-SLAM3 的第二項核心創新，也是最能體現「工程智慧」的設計。

### Atlas 的組成

Atlas 是由一組離散地圖構成的多子地圖系統。它維護一張活躍地圖，供追蹤執行緒定位當前幀，而局部建圖執行緒持續以新的關鍵幀最佳化並更新該地圖；Atlas 中的其他地圖則為非活躍地圖。系統並建立一個以詞袋模型為基礎的關鍵幀資訊資料庫，用於重定位、閉環偵測與地圖合併。

124. 活躍地圖（Active Map）：正在使用的那一張，追蹤與局部建圖都在它上面運作。

125. 非活躍地圖（Inactive Map）：曾經建立、目前未使用，但完整保留在記憶體中等待日後合併。

126. 共用的詞袋資料庫：所有地圖的關鍵幀都登錄在同一個資料庫中——這是能跨地圖比對的關鍵。

### 跟丟時發生什麼

這正是 Atlas 存在的理由。回顧 ORB-SLAM2 的處境：嚴重跟丟且重定位失敗時，系統就停在那裡，之後的資料全部浪費。

ORB-SLAM3 的做法是：跟丟後若短時間內無法重定位，就把當前地圖設為非活躍、開啟一張全新的地圖，從零開始繼續建圖與追蹤。使用者幾乎不會察覺中斷。而當機器人日後走回舊地圖涵蓋的區域，地點辨識會偵測到跨地圖的匹配，系統隨即把兩張地圖合併，恢復成一張完整一致的地圖。

### 與其他多地圖方案的差異

值得注意的是，也有其他系統提出 ORB-SLAM2 的多地圖擴充，但它們把子地圖保留為分離的實體；ORB-SLAM3 則執行無縫的地圖合併，建立更精確的全域地圖。

這個差異很關鍵。若只是把地圖各自存著不合併，雖然資料沒丟，但你會得到好幾張互不相干的地圖，無法規劃跨區域的路徑，精度也無法互相提升。無縫合併才能真正發揮多地圖的價值。

### 多階段作業的價值

Atlas 的另一個重要應用是跨執行階段的重用：今天建好的地圖存起來，明天啟動時載入，系統能直接在舊地圖中重定位並繼續擴充。這對每天在同一場域運行的服務機器人、巡檢機器人極為實用，也解釋了 23.1 節提到「多階段精度優勢更大」的現象。

## 23.6 失去追蹤後的重定位與地圖融合

本節說明 Atlas 運作的技術細節：如何偵測跨地圖的關聯，以及如何合併。

### 地點辨識：高召回率的設計

每當建圖執行緒建立一個新的關鍵幀，地點辨識就會啟動，嘗試偵測與 Atlas 中任何既有關鍵幀的匹配。若找到的匹配關鍵幀屬於活躍地圖，就執行閉環；否則就是多地圖資料關聯，此時活躍地圖與匹配地圖會被合併。

這是一個優雅的統一設計：閉環與地圖合併，其實是同一個機制的兩種結果——差別只在於匹配到的關鍵幀屬於同一張地圖還是不同地圖。

### 精確率與召回率的取捨

地點辨識面臨一個經典的兩難：

127. 召回率要高：必須盡量找到所有真實的匹配，否則會錯過閉環與合併的機會。
128. 精確率必須是百分之百：一個錯誤的匹配會把地圖硬拉到錯誤位置，造成災難（第 12、15、22 章反覆強調）。

ORB-SLAM3 的做法是：為了達到高召回率，對每個新的活躍關鍵幀，向 DBoW2 資料庫查詢 Atlas 中數個相似的關鍵幀；而為了達到百分之百的精確率，每一個候選都要通過數個步驟的幾何驗證。

幾何驗證透過直接的 Sim(3) 或 SE(3) 對齊執行（使用 RANSAC 與 Horn 方法）。這裡再次看到第 21 章 RANSAC 的身影，以及第 22 章單目需用相似變換（含尺度）的道理。

### 地圖合併的兩階段策略

確認跨地圖關聯後，合併就要開始了。但這裡有一個工程難題：非活躍地圖可能很大，若一次全部合併，運算會阻塞系統。

ORB-SLAM3 的解法是把合併拆成兩個步驟：首先，在由匹配關鍵幀與其鄰居所定義的焊接視窗中執行合併；第二階段，再透過位姿圖優化把修正傳播到合併後地圖的其餘部分。

**圖 23-3　地圖合併的兩階段策略**

偵測到跨地圖關聯：活躍地圖 Ma 的關鍵幀 Ka ↔ 非活躍地圖 Mm 的關鍵幀 Km

↓ 估出對齊變換 Tam

【第一階段：焊接視窗】

在 Ka 與 Km 的鄰居所構成的局部視窗中執行合併

在此視窗中密集搜尋中期資料關聯，提升精度

→ 局部立即可用，追蹤執行緒不中斷

↓

【第二階段：傳播修正】

以位姿圖優化把修正傳播到合併後地圖的其餘部分

→ 全域一致性恢復

這個「先局部焊接、再全域傳播」的策略，讓合併能即時進行而不阻塞追蹤。同學應該注意到，這與第 15 章 Cartographer「局部即時、全域背景」、第 18 章 LOAM「高頻里程計、低頻建圖」的架構思想完全一致——這是機器人系統設計的普遍模式。

### 一個細節：中期關聯的強化

作者提出的第二項創新是：一旦估出新關鍵幀與匹配地圖之間的相對位姿，便以匹配關

鍵幀及其在共視圖中的鄰居定義一個局部視窗，並在此視窗中密集搜尋中期資料關聯，以提升精度。

這再次呼應 23.2 節的主軸：ORB-SLAM3 的精度優勢，很大程度來自它在每一個環節都盡可能運用中期資料關聯。

### 量一量：跟丟之後的三種命運

23.5 與 23.6 節描述了 Atlas 的運作。本小節用一個模擬把它的價值量化：同一條軌跡、同一次跟丟，三種系統設計各自會得到什麼。

設定是機器人在室內繞兩圈共 120 個關鍵幀，在第 66 至 78 幀之間視覺完全失效（例如快速轉動或面對白牆）。三種情況分別是：ORB-SLAM2 的做法（重定位失敗後停住）、開新地圖但不合併、以及 Atlas 的無縫合併。最後用兩個指標比較：涵蓋了多少條軌跡，以及最終地圖的均方根誤差。

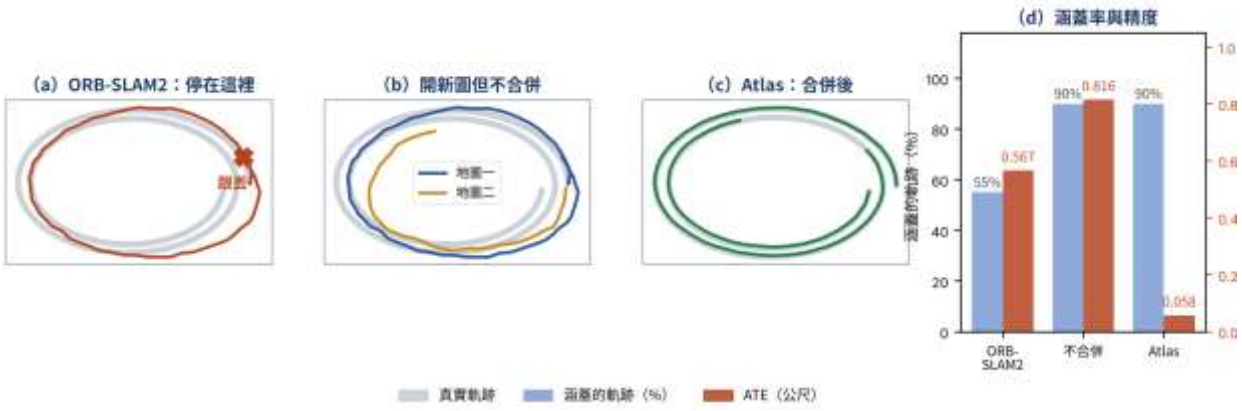



**圖 23-4　跟丟之後的三種命運**

（a）是 ORB-SLAM2 的處境。系統走到第 66 幀跟丟，重定位失敗後就停在那裡，只涵蓋了 55% 的軌跡，之後的資料全部浪費。已建好的部分 ATE 是 0.567 公尺——注意這個誤差不算小，因為它沒有機會靠第二圈的閉環來修正。

（b）是「開新地圖但不合併」。涵蓋率提升到 90%，資料確實沒丟。但兩張地圖各自有自己的原點，把它們放在同一個座標系下看，整體 ATE 是 0.816 公尺——比只建一半還糟。

圖上藍色與橘色兩段明顯錯開，這正是 23.5 節說「無法規劃跨區域路徑」的視覺呈現。

（c）是 Atlas。機器人走回第一張地圖涵蓋的區域時，地點辨識偵測到跨地圖的匹配，系統把兩張圖合併成一張。涵蓋率同樣是 90%，但 ATE 降到 0.058 公尺——是不合併的十四分之一，也比只建一半的 ORB-SLAM2 好了十倍。圖上可以看到合併後的綠色軌跡與真實軌跡幾乎重合，只在跟丟的那一段留下缺口。

（d）把兩個指標並排。這張圖要傳達的重點是：Atlas 的價值不只是「資料沒丟」，更是「資料能互相校正」。第二張地圖之所以能被拉回正確位置，靠的是它與第一張地圖之間的跨地圖共視關係——這與第 22 章圖 22-2 的結論完全一致：誤差要靠圖的連通性才能重新分配。多地圖如果不合併，就只是把資料存起來而已。

> **為什麼「合併」比「保留」難得多**
>
> 把跟丟後的資料另存成一張新地圖，工程上很容易——開一個新容器就好。真正困難的是合併：兩張地圖的座標系不同、尺度可能不同（單目）、而且合併時不能卡住正在運行的追蹤執行緒。
>
> 這就是 23.6 節那個兩階段策略存在的理由：先在焊接視窗內做局部融合（範圍小、可即時完成），再把修正量沿著位姿圖往外傳播（範圍大、放背景做）。同樣是「先快速拉正、再精細最佳化」——這個模式在本書已經出現第四次了。

## 23.7 GoPro、手機、無人機上的視覺慣性 SLAM

視覺慣性 SLAM 已不只是研究題目，它就在你我的日常裝置中運作。本節說明幾個代表性應用。

### 手機與 AR / VR

第 20.7 節提過 AR 的「由內而外追蹤」，其技術本質正是視覺慣性 SLAM。手機的相機加上內建 IMU（第 7 章），構成完美的視覺慣性組合：相機提供不漂移的環境參考、IMU

提供高頻運動與快速轉頭時的穩健度。

ORB-SLAM3 論文中一個特別有說服力的數據，正是針對這類應用。在魚眼的 TUM-VI 基準測試上，它整體表現優於 VINS-Mono 與 Basalt；在類似 AR / VR 的房間序列中，雙目慣性模式的平均絕對軌跡誤差為 0.009 公尺，單目慣性模式為 0.011 公尺。

毫米級的精度——這正是把虛擬物件穩定「釘」在真實桌面上所需要的水準。同學可以想想：你手機裡那個能量測桌子長度的 AR 尺，背後就是這個等級的技術。

### 穿戴式相機與運動攝影

運動攝影機（如 GoPro）配戴在頭盔或胸前，在騎車、跑步、攀岩時錄影。這類影片對 SLAM 極具挑戰：劇烈震動、快速轉向、動態模糊、強烈的光線變化（進出樹蔭、隧道）。

純視覺方法在這類序列上幾乎必然跟丟，而視覺慣性 SLAM 因為有 IMU 在視覺失效時撐住數秒（第 7.6 節的第三項貢獻），得以維持連續軌跡。這類應用的典型用途包括：運動路線的三維重建、極限運動的軌跡分析、以及第五篇提過的背包式空間測繪。

### 無人機

無人機是視覺慣性 SLAM 最重要的應用之一，也是第 26 章的主題。它的特殊需求包括：

129. 重量與功耗極度受限：相機加 IMU 是唯一可行的組合（第 9.5 節）。
130. 六自由度快速運動：純視覺難以應付，IMU 不可或缺。
131. GPS 拒止環境：室內、橋下、隧道中，視覺慣性是唯一的定位手段（第 5 章案例 5-3）。
132. 即時性要求嚴苛：定位延遲直接影響飛行穩定與安全。

第 7 章案例 7-3 提過的 EuRoC 資料集，正是以微型飛行器錄製，而 ORB-SLAM3 在其上的優異表現（第 7.7 節的 3.5 公分精度），說明了它在無人機應用上的實用性。

### 機器人與校園場域

對本書讀者而言，幾個具體的專題方向值得考慮：

133. 視覺導航機器人：以雙目慣性 ORB-SLAM3 建立校園地圖，再結合第 24 章的視覺導航與第 32 章的語意理解。

134. 多階段建圖：利用 Atlas 的多階段能力，分多次建立整棟系館的地圖，驗證地圖合併的效果。

135. 跟丟恢復實驗：設計一個「刻意讓系統跟丟」的實驗流程，量化 Atlas 相對 ORB-SLAM2 在恢復能力上的優勢——這是一個很好的比較型研究題目。

136. 與光達 SLAM 對照：在同一場域分別執行 ORB-SLAM3（視覺慣性）與第 18 章的 LIO 系統（光達慣性），比較兩者在不同環境（明亮走廊、昏暗機房、玻璃帷幕大廳）的表現。這個對照能讓同學深刻體會第 8、9 章反覆強調的感測器互補性。

至此，第六篇的核心系統已完整介紹。下一章我們把視野從「定位建圖」擴展到「導航與應用」——視覺導航機器人如何利用這些能力執行任務？線追蹤、視覺伺服、AprilTag 導航、YOLO 語意導航、乃至於 VLM 輔助導航，將把本書的技術與真實的機器人任務串接起來。

## 23.8 視覺慣性之後：2024–2026 年的兩個方向

ORB-SLAM3 發表於 2021 年，至今仍是視覺慣性 SLAM 的標準比較對象。這幾年的進展主要往兩個方向走，而且方向幾乎相反。

### 一、往下：擠進毫瓦級的晶片

第一個方向是把視覺慣性里程計縮小到極限。2026 年的一項工作把完整的單目視覺慣性管線（ORB 特徵、八點法與 EPnP 搭配 RANSAC、IMU 預積分、滑動視窗的位姿圖優化）部署到一顆 RISC-V 的超低功耗晶片上，時脈只有 370 MHz。

成績是每秒 20 幀、功耗低於 100 毫瓦、記憶體用量不到 1 MB（第一層約 110 KB、第二層約 850 KB）。把這幾個數字放在一起看相當驚人——一套完整的視覺慣性 SLAM，跑在比

手機弱兩個數量級的晶片上。

但代價也很誠實。該系統省掉了閉環偵測，在標準的 EuRoC 資料集上平均軌跡誤差約 3.46 公尺，而 ORB-SLAM3 是 0.25 公尺左右——差了一個數量級。作者自己指出，原因正是缺少全域修正。這組對照很適合放在本章末尾：它說明了閉環與全域最佳化不是錦上添花，而是長時間一致性的唯一來源。第 25 章的毫瓦級平台會再回到這個主題。

## 二、往上：讓學習式模組接手前端

第二個方向是把前端的某些環節交給神經網路：用學習式的特徵與匹配取代 ORB、用學習式的光流一致性檢驗取代純幾何的外點剔除、或在光線劇變的場景中加入學習式的線特徵。

這些工作有一個共同點值得同學注意：它們換掉的幾乎都是「對應怎麼找」這一層，而 IMU 預積分、緊耦合的最佳化、關鍵幀與地圖管理——也就是本章 23.3 到 23.6 節的全部內容——大多原封不動。這與第 22 章 22.8 節觀察到的現象完全相同。

| | ORB-SLAM3（本章） | 毫瓦級 VIO | 學習式前端 VIO |
|---|---|---|---|
| 前端 | ORB + RANSAC | ORB + RANSAC（精簡） | 學習式特徵 / 光流 |
| IMU 融合 | 緊耦合，含零偏估計 | 緊耦合（滑動視窗） | 緊耦合 |
| 閉環與多地圖 | 有 | 無 | 多半保留 |
| 典型硬體 | 桌機或 Jetson | RISC-V，370 MHz | 需要 GPU |
| 功耗 | 數瓦至數十瓦 | 低於 100 毫瓦 | 數十瓦 |
| EuRoC 平均誤差 | 約 0.25 公尺 | 約 3.46 公尺 | 與 ORB-SLAM3 相當 |
| 適用 | 一般機器人與 AR | 微型無人機、穿戴式 | 光線劇變、弱紋理場景 |

**表 23-2　三條路線的比較**

從這張表可以讀出一個對專題選題很實用的判斷：如果你的載體有數瓦的功耗預算，ORB-SLAM3 仍然是最省事也最可靠的選擇；如果你做的是克級的微型平台，那麼你需要的是第 25 章的技術，並且要接受沒有閉環的後果；如果你的場域光線條件惡劣，學習式前端值得一試，但系統架構仍是本章這一套。

## 公式或流程

### 一、視覺慣性 SLAM 的狀態向量

視覺慣性模式下，每個關鍵幀 i 需估計的狀態為：

$$S_i = \{ R_i, p_i, v_i, b_g_i, b_a_i \}$$

其中 $R_i$ 為旋轉、$p_i$ 為位置（合計 6 自由度）、$v_i$ 為速度（3）、b_g 與 b_a 為陀螺儀與加速度計零偏（各 3），共 15 個自由度——相較純視覺的 6 個自由度多出速度與零偏。單目慣性模式在初始化階段還額外要估尺度 s 與重力方向 R_wg。

### 二、緊耦合的聯合最佳化目標

ORB-SLAM3 的視覺慣性束調整，同時最小化視覺與慣性兩類誤差：

$$\min \Sigma \rho\left( \| e_視覺 \|^2_\Sigma \right) + \Sigma \| e_慣性 \|^2_\Sigma + \| e_先驗 \|^2$$

137. 視覺誤差 e_視覺：重投影誤差（第 21、22 章），要求地圖點投影位置與實際觀測相符。

138. 慣性誤差 e_慣性：IMU 預積分殘差（第 7.5 節），要求相鄰關鍵幀的位姿、速度變化與 IMU 量測一致。

139. 先驗誤差 e_先驗：保留先前最佳化累積的資訊。

$\rho$ 為穩健核函數。這條式子正是第 7.6 節「緊耦合」的具體形式，也是第 12 章最小平方框架的又一次應用——同學至此應能看出，整本書的數學是高度統一的。

## 三、IMU 初始化的三階段

**圖 23-5 IMU 初始化流程**

① 純視覺 MAP 估測

只用相機建立短時間的地圖與軌跡（單目尺度未知）

↓ 提供運動基準

② 純慣性 MAP 估測（固定視覺軌跡）

估計：尺度 s、重力方向、陀螺儀零偏、加速度計零偏、各幀速度

↓ 取得慣性參數初值

③ 聯合視覺慣性束調整

所有變數（位姿、速度、零偏、尺度、重力、地圖點）一起精修

↓

完成：2 秒內尺度誤差約 5%，精修後約 1%

【前提】必須有足夠的運動激勵，否則尺度與零偏不可觀測

## 四、閉環與地圖合併的統一流程

**圖 23-6 地點辨識觸發的兩種結果**

建圖執行緒建立新關鍵幀

↓

地點辨識：向 DBoW2 資料庫查詢 Atlas 中的相似關鍵幀（高召回率）

↓

幾何驗證：多步驟驗證 + Sim（3）/ SE(3) 對齊（RANSAC + Horn 方法）

↓ （追求 100% 精確率）

匹配幀屬於哪張地圖？

├─ 活躍地圖 → 【閉環】位姿圖優化 + 全域 BA（第 22 章）

└─ 其他地圖 → 【地圖合併】焊接視窗融合 + 位姿圖傳播修正

## 案例

### 案例 23-1 跟丟不再是終點：Atlas 的實測體驗

這是對比 ORB-SLAM2 與 ORB-SLAM3 最有戲劇性的實驗，強烈建議在課堂上並排演示。

實驗設計很簡單：讓兩套系統執行同一段影片，而這段影片中間有一段嚴重的視覺失效——例如相機被完全遮住十秒、或快速甩過一大片白牆（第 9 章的白牆詛咒）。

ORB-SLAM2 的表現：跟丟後嘗試重定位，但因為離開原區域太遠而失敗，系統就此停擺。畫面凍結，之後的影片內容完全沒有被利用——即使後面有精彩的場景，也全部浪費。

ORB-SLAM3 的表現：跟丟後短暫嘗試重定位，失敗後隨即開啟一張新地圖，繼續正常建圖與追蹤，使用者幾乎察覺不到中斷。而當相機日後繞回舊區域，地點辨識偵測到跨地圖關聯，兩張地圖無縫合併——螢幕上會看到「兩段原本分離的軌跡忽然接在一起」的震撼

畫面。

這個對比讓同學直觀理解「穩健度」在真實系統中的價值。實驗室裡的乾淨資料集上，兩者精度差距或許只是數字；但在真實世界的挑戰性場景中，差別是「能不能用」。這也呼應第 15 章 Cartographer 的觀察：產品化的系統，穩健度往往比極致精度更重要。

## 案例 23-2　兩秒鐘的價值：初始化速度為何是關鍵

IMU 初始化從十五秒縮短到兩秒，聽起來只是「快了一點」。但這十三秒的差距，在實務上是決定性的。

想像一位使用者打開手機的 AR 應用。若需要拿著手機晃動十五秒才能開始使用，絕大多數人會在第五秒就關掉 App——這個產品根本無法上市。而兩秒鐘的初始化，使用者通常還在調整握姿，幾乎無感。

對機器人也一樣。一台無人機起飛前若要在地上晃十五秒，不僅浪費時間，在某些場景（例如緊急救援）根本不可行；而兩秒的初始化，可以在起飛滑行的過程中順帶完成。

更深一層看，快速初始化還帶來一個間接好處：當系統跟丟並需要在新地圖上重新初始化時（23.5 節），快速初始化意味著中斷時間極短。若初始化要十五秒，Atlas 的「無縫切換」就名不副實了。這說明了系統設計中各項技術的相互支撐——初始化速度、多地圖架構、資料關聯，三者共同構成了 ORB-SLAM3 的穩健度。

對同學的啟示是：評估一項技術改進時，不要只看單一指標的數字變化，要問「這個改變讓什麼原本不可能的事變成可能？」——這才是技術價值的真正衡量。

## 案例 23-3　從 PTAM 到 ORB-SLAM3：一條血脈的二十年

ORB-SLAM3 不是憑空出現的，它是特徵法視覺 SLAM 這條技術血脈二十年演進的結晶。它是自 PTAM 與 ORB-SLAM 開始的特徵法 SLAM 血脈的集大成之作，發布時是當時最完整、最精確的開源 SLAM 函式庫。

140. PTAM(2007)：首創「追蹤與建圖分離為兩個執行緒」的架構，讓即時視覺 SLAM 成為可能。這個雙執行緒思想，是後續所有系統的基礎。

141. ORB-SLAM(2015)：以 ORB 特徵統一追蹤、建圖與地點辨識三項任務（第 22.1 節），加入閉環執行緒，並引入共視圖與本質圖（第 22.6 節）。

142. ORB-SLAM2(2017)：擴展到雙目與 RGB-D，成為最廣泛使用的開源視覺 SLAM（第 22 章）。

143. ORB-SLAM3(2021)：加入緊耦合視覺慣性與 Atlas 多地圖，並全面運用四種資料關聯。

觀察這條血脈，同學能學到一個重要的研究方法論：每一代都不是推翻前代，而是保留有效的架構、針對明確的弱點做突破。PTAM 的雙執行緒、ORB-SLAM 的共視圖與詞袋、ORB-SLAM2 的多相機支援——這些設計在 ORB-SLAM3 中全都還在。

這也給有志研究的同學一個務實的建議：與其追求「發明全新的東西」，不如深入理解一個成熟系統，找出它明確的弱點，提出針對性的改良。ORB-SLAM3 之所以能提出 Atlas，正是因為作者深知 ORB-SLAM2「跟丟即結束」的痛點。理解限制，才能突破限制。

## 實作活動

### 實作 23-1　ORB-SLAM3 視覺慣性 SLAM 實作

目的：執行 ORB-SLAM3 的多種模式，親身體驗 IMU 帶來的差異、Atlas 的跟丟恢復能力，並完成與 ORB-SLAM2 的對照實驗。這是第六篇的總結實作，也是第 30、31 章的重要基礎。

**第一部分：環境建置與基本執行（全體）**

96. 編譯 ORB-SLAM3（相依套件與第 22 章類似：OpenCV、Eigen3、Pangolin；注意版本相容性問題）。

97. 下載 EuRoC 資料集的一個序列（建議從 MH_01_easy 開始），先以純雙目模式執行，記錄軌跡與地圖。

98. 同一序列改以雙目慣性模式執行，比較兩者的軌跡、地圖點數量與系統穩定度。

99. 觀察初始化：單目慣性模式下，記錄從啟動到 IMU 初始化完成所需的時間與影像幀數，對照 23.4 節的「兩秒」數據。

### 第二部分：IMU 的價值（全體或大學部）

100. 挑戰序列：選一段快速運動的困難序列（如 EuRoC 的 V1_03_difficult 或 MH_05_difficult），分別以純視覺與視覺慣性模式執行，比較是否跟丟、跟丟幾次。

101. 尺度驗證：單目模式與單目慣性模式各執行一次，用 evo 評測時，前者需做尺度對齊、後者不需要——親身驗證 IMU 提供絕對尺度的價值（第 20.5 節）。

102. 運動激勵實驗：若使用自己的相機與 IMU，分別嘗試「靜止不動」「等速直線移動」「帶加減速與旋轉的移動」三種啟動方式，觀察哪一種能成功初始化，驗證 23.4 節的論述。

### 第三部分：Atlas 多地圖實驗（核心，大學部）

103. 人為製造跟丟：執行資料集時，以程式暫時中斷影像輸入數秒（或選用含嚴重遮擋的序列），觀察系統是否開啟新地圖。

104. 觀察 Atlas 狀態：在終端輸出或視覺化中，確認當前有幾張地圖、哪一張是活躍地圖。

105. 地圖合併：讓相機回到舊地圖涵蓋的區域，觀察合併發生的瞬間，並錄影記錄——這是本章最值得展示的畫面。

106. 對照實驗：對同一段含嚴重跟丟的序列，分別執行 ORB-SLAM2 與 ORB-SLAM3，比較最終軌跡的完整度，重現案例 23-1。

107. 多階段實驗：利用 EuRoC 同一場景的多個序列（如 MH_01 至 MH_05），依序執行並讓系統合併，觀察多階段作業如何逐步擴充地圖。

**討論問題**

34. 純雙目與雙目慣性的精度（ATE）差多少？在哪一種序列上差距最大？為什麼？
35. 困難序列中，純視覺跟丟了幾次？加入 IMU 後改善多少？對照第 7.6 節，IMU 的哪一項貢獻在此發揮了作用？
36. IMU 初始化在你的實驗中花了多久？若初始化時載體靜止不動，會發生什麼？這印證了 23.4 節的哪一項論述？
37. 地圖合併發生時，軌跡與地圖有什麼變化？這與第 22 章的閉環修正有何異同？（提示：23.6 節的統一流程）
38. （大學部）依 23.2 節的四種資料關聯，說明 ORB-SLAM3 相對 VINS-Mono 精度優勢的主要來源是哪一種關聯？為什麼它如此重要？

**延伸挑戰（大學部）**

其一：校園場域實測——用自備的雙目慣性相機（或含 IMU 的深度相機）錄製系館走廊與大廳的資料，執行 ORB-SLAM3 建圖。刻意設計一條「經過昏暗機房造成跟丟、再回到明亮走廊」的路線，測試 Atlas 的合併能力，並以第 13.7 節的方法評估地圖品質。其二：跨模態比較——在同一場域同時錄製視覺慣性與光達慣性資料，分別執行 ORB-SLAM3 與第 18 章的 LIO 系統，在明亮走廊、昏暗機房、玻璃帷幕大廳三種環境下比較兩者表現，量化驗證第 8、9 章的感測器互補論述。這個題目兼具教學與研究價值，適合作為專題主軸。

## 延伸閱讀

50. C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM," IEEE T-RO, 2021.（本章主要依據）
51. R. Elvira, J. D. Tardós, and J. M. M. Montiel, "ORBSLAM-Atlas: A Robust and Accurate Multi-Map System," IROS, 2019.（Atlas 多地圖架構的前導工作）

52. C. Campos, J. M. M. Montiel, and J. D. Tardós, "Inertial-Only Optimization for Visual-Inertial Initialization," ICRA, 2020.（IMU 初始化三階段流程的理論來源）

53. C. Forster et al., "On-Manifold Preintegration for Real-Time Visual-Inertial Odometry," IEEE T-RO, 2017.（IMU 預積分，第 7.5 節的理論基礎）

54. ORB-SLAM3 開源專案（GitHub: UZ-SLAMLab/ORB_SLAM3）.（原始程式碼、編譯說明與各模式執行範例）

55. T. Qin, P. Li, and S. Shen, "VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator," IEEE T-RO, 2018.（另一套經典視覺慣性系統，可與本章對照）

56. EuRoC MAV 與 TUM-VI 資料集官方網站.（本章實驗與論文評測所用的標準基準）

## 小結

50. ORB-SLAM3 有三項核心突破：以最大後驗估測為基礎的緊耦合視覺慣性 SLAM、Atlas 多地圖系統、以及在所有演算法階段全面運用四種資料關聯；支援單目 / 雙目 / RGB-D 與針孔 / 魚眼鏡頭的多種組態。

51. 四種資料關聯為短期（最近數秒）、中期（附近有輕微漂移的元素）、長期（以地點辨識匹配先前造訪區域）與多地圖（跨建圖階段）；中期關聯是精度優勢的關鍵來源，使其在 EuRoC 單目慣性單階段運作中精度達 VINS-Mono 的 2.6 倍。

52. 視覺慣性模式下狀態向量從純視覺的 6 自由度擴充為 15 自由度（加上速度與兩組零偏）；IMU 帶來絕對尺度、重力方向、視覺失效時的支撐與高頻輸出，而視覺回報以不漂移的觀測校正 IMU 零偏。

53. IMU 初始化採三階段：純視覺 MAP 估測 → 固定視覺軌跡的純慣性 MAP 估測 → 聯合視覺慣性 BA;2 秒內達 5% 尺度誤差、精修後約 1%，但必須有足夠的運動激勵，否則尺度與零偏不可觀測。

54. Atlas 由一張活躍地圖與多張非活躍地圖組成，共用以詞袋為基礎的關鍵幀資料庫；跟丟時開啟新地圖繼續運作，日後偵測到跨地圖關聯時執行無縫合併——不同於其他方案把子地圖保留為分離實體。

55. 地點辨識對每個新關鍵幀查詢 Atlas 中的相似幀以求高召回率，再以多步驟幾何驗證與 Sim（3）/ SE(3) 對齊確保百分之百精確率；匹配屬活躍地圖則閉環，否則合併。合併分焊接視窗融合與位姿圖傳播兩階段，以免阻塞追蹤。

56. 視覺慣性 SLAM 廣泛應用於手機 AR、穿戴式相機與無人機；在類似 AR / VR 的 TUM-VI 房間序列中，雙目慣性平均 ATE 為 0.009 公尺、單目慣性為 0.011 公尺，達到毫米級精度。

## 承先啟後

本章把第 7 章的 IMU 與第 22 章的系統架構接了起來，也量出了三組數字：IMU 要撐過視覺空隙，關鍵在零偏——零偏估到 ±0.01 時兩秒的推算誤差是 0.057 公尺，估到 ±0.3 時是 0.88 公尺，差了十五倍；等速直線運動下初始化的條件數高達 3,820、尺度誤差 100%，充分激勵則把條件數降到 157、誤差降到 0.2%；而跟丟之後，Atlas 的合併把 ATE 從不合併的 0.816 公尺壓到 0.058 公尺，同時涵蓋 90% 的軌跡。

第六篇的核心系統至此完整介紹。但 SLAM 本身不是目的——機器人建了地圖、知道自己在哪之後，要拿它來做什麼？下一章我們把視野從「定位建圖」擴展到「導航與任務」：視覺導航與 SLAM 有何不同？線追蹤、視覺伺服、人工標記導航如何運作？物件偵測與視覺語言模型如何讓機器人理解「請去教務處」這樣的指令？機器狗與校園巡檢機器人如何整合這一切？第 24 章將把本書的技術與真實的機器人任務串接起來。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

## 第 24 章

## 視覺導航機器人

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

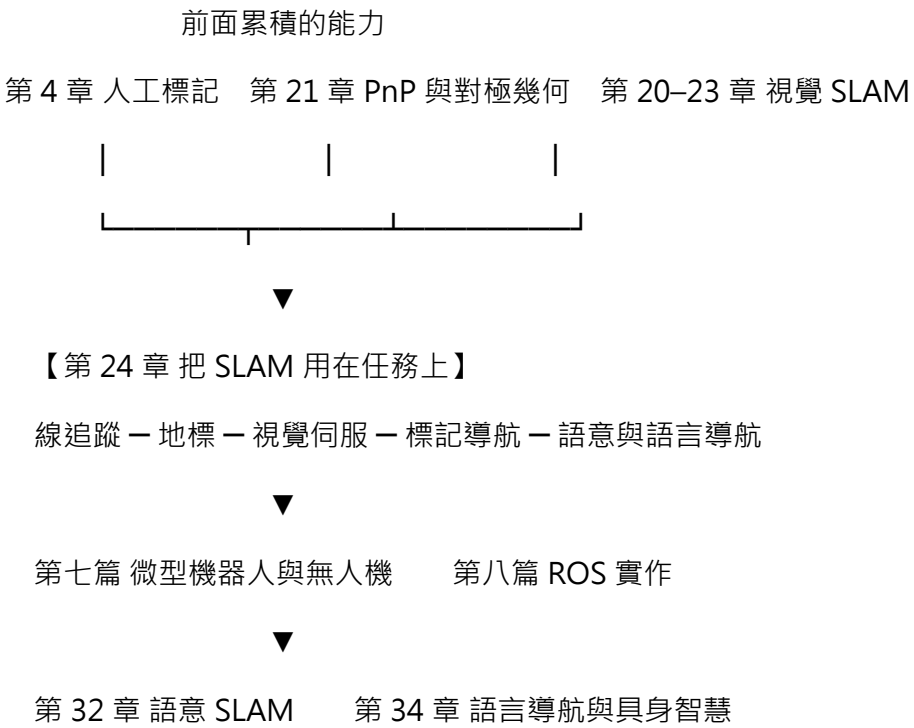
## 本章在全書的位置

第 20 到 23 章把視覺 SLAM 從特徵一路建到完整系統。本章問的是下一個問題：機器人知道自己在哪之後，要拿它來做什麼。

這一章的性格與前四章很不一樣。前四章在追求精度與穩健，本章則在談取捨——很多導航任務根本不需要 SLAM。貼一條線、貼幾個標記、或者直接對著目標做視覺伺服，在對的場合比完整的 SLAM 更便宜也更可靠。工程判斷的重點不是「用上最強的技術」，而是「用剛好夠的技術」。

本章的三張圖都在量化這些取捨：圖 24-1 比較兩種視覺伺服的收斂行為，說明為什麼精密對接要挑對方法；圖 24-2 量出標記的位姿精度如何隨距離與視角變化；圖 24-3 則回答一個很實際的問題——標記到底該多久貼一個。

### 圖 24-0　第 24 章的連結地圖




## 學習目標

前四章我們建立了完整的視覺 SLAM 能力：機器人能知道自己在哪、環境長什麼樣。但

SLAM 本身不是目的——建了地圖之後，要拿它來做什麼？本章是第六篇的最後一章，把技術與真實的機器人任務串接起來，從最樸素的線追蹤，一路走到能聽懂人話的 VLM 導航。學完本章之後，同學應能夠達成下列目標：

50. 說明視覺導航與視覺 SLAM 的差異與互補關係。
51. 比較線追蹤、地標導航與視覺伺服三種基本視覺導航策略。
52. 說明 AprilTag / ArUco 在機器人導航中的四種典型用途。
53. 說明 YOLO 物件辨識如何支援語意導航，並理解其在機器人視角下的限制。
54. 說明 VLM 視覺語言模型如何讓機器人理解自然語言導航指令。
55. 說明機器狗視覺導航的特殊挑戰與對策。
56. 規劃一個校園巡檢或資源回收辨識機器人的系統架構。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 視覺導航 | Visual Navigation | 以視覺資訊引導機器人移動到目標的技術。 |
| 線追蹤 | Line Following | 沿地面標線行進的最基本視覺導航方式。 |
| 視覺伺服 | Visual Servoing | 以影像回授直接控制機器人運動的方法。 |
| 影像式視覺伺服 | IBVS | 直接以影像特徵誤差作為控制訊號。 |
| 位置式視覺伺服 | PBVS | 先解出三維位姿誤差再控制。 |
| 地標導航 | Landmark Navigation | 依據可辨識的地標決定行進方向。 |
| 語意導航 | Semantic Navigation | 依物體與場景意義而非幾何座標導航。 |
| 物件目標導航 | Object-Goal Navigation | 接受「去找某個物體」的任務指令。 |
| 視覺語言導航 | VLN | 依自然語言指令在環境中導航。 |

| 中文名詞 | 英文名詞 | 概念說明 |
| --- | --- | --- |
| 視覺語言模型 | VLM | 能同時理解影像與語言的多模態 AI 模型。 |
| 視覺語言動作模型 | VLA | 直接由影像與語言產生動作指令的模型。 |
| 邊界探索 | Frontier Exploration | 朝已知與未知交界處前進以探索環境。 |
| 零樣本 | Zero-Shot | 未經任務特定訓練即可執行新任務的能力。 |

## 24.1 視覺導航與 SLAM 的差異

這是本章第一個必須釐清的觀念。許多同學把「視覺 SLAM」與「視覺導航」混為一談，但兩者回答的是不同的問題。

### 兩個不同的問題

- 視覺 SLAM 回答的是：「我在哪裡？環境長什麼樣？」它產出的是位姿與地圖——這是第 20 至 23 章的主題。
- 視覺導航回答的是：「我該往哪走？」它產出的是運動指令——這是本章的主題。

用第 3 章的導航三問來對照：SLAM 專門解決第一問（我在哪裡），而視覺導航則涵蓋第二、三問（我要去哪裡、我該怎麼去）。

### 視覺導航不一定需要 SLAM

這是一個重要但常被忽略的事實。有些視覺導航方法完全不建地圖：

- 線追蹤機器人只看地上的線，不知道自己在哪，也不需要知道（24.2 節）。
- 視覺伺服直接把「影像中目標的偏移」轉成馬達指令，中間不經過任何地圖。
- 依循視覺地標的走廊機器人，只需認得「看到滅火器就右轉」，不需全域座標。

這些方法屬於第 3 章介紹的反應式架構——感知直接驅動行為，不建模、不規劃。它們簡單、快速、穩健，在受控環境中非常實用。

### SLAM 讓導航能力升級

但反應式導航有明顯的天花板：無法規劃全域路徑、無法處理「去一個沒有標線通往的地方」、無法在中途改變目標。加上 SLAM 之後，機器人才具備：

144. 全域路徑規劃：在地圖上算出最佳路線（第 3、29 章）。
145. 任意目標導航：能前往地圖上任何座標，而非只能沿預設路線。
146. 斷點恢復：跟丟後能重定位並繼續任務（第 22、23 章）。

147. 語意任務：結合語意地圖，能理解「去教務處」這種指令（24.5 節）。

| 比較項目 | 視覺 SLAM | 視覺導航 |
|---|---|---|
| 核心問題 | 我在哪？環境長什麼樣？ | 我該往哪走？ |
| 輸出 | 位姿與地圖 | 運動指令（速度、方向） |
| 是否需地圖 | 產出地圖 | 不一定需要 |
| 架構類型 | 審議式（需建模） | 可反應式，也可審議式 |
| 對應導航三問 | 第一問 | 第二、三問 |
| 本書章節 | 第 19–22 章 | 本章、第 29 章 |

**表 24-1　視覺 SLAM 與視覺導航的對照**

實務系統多半兩者並用：以 SLAM 提供全域定位與規劃，以反應式視覺導航處理即時避障與精確對接。這正是第 3 章「混合式架構」的體現。

## 24.2　線追蹤、地標導航、視覺伺服

本節介紹三種不需要完整 SLAM 的基本視覺導航策略，由簡到繁。

### 一、線追蹤（Line Following）

這是最樸素、也是無數同學第一個做出來的機器人：在地面貼一條黑色膠帶，機器人以向下的相機或紅外線感測器偵測線的位置，若線偏左就左轉、偏右就右轉。

- 原理：把影像轉為灰階或二值化（影像處理的基本功），找出線的中心與畫面中心的偏差，轉為轉向指令。
- 優點：成本極低、程式極簡、極為穩健、不需地圖與定位。
- 限制：只能走預設路線、無法繞開障礙、環境必須改造（貼線）。

- 實務應用：工廠 AGV 的磁條或色帶導引至今仍廣泛使用——在可控環境中，它便宜可靠得無可取代（呼應第 4 章亞馬遜倉儲案例的觀念：環境越可控，技術選型越能簡化）。

## 二、地標導航（Landmark Navigation）

比線追蹤進階一階：不鋪連續的線，而是在關鍵位置放置可辨識的地標，機器人「看到地標 A 就右轉、看到地標 B 就停」。

地標可以是人工的（第 4.6 節的 AprilTag、QR Code、色塊）或自然的（門、滅火器、消防栓）。前者可靠但需改造環境，後者不需改造但辨識較難——這正是 YOLO 等 AI 辨識技術的用武之地（24.4 節）。

地標導航本質上建立了一張拓樸地圖（第 3 章）：節點是地標，邊是「從這個地標到下一個地標的行進動作」。它不需要精確的座標，卻能完成相當複雜的路線任務。

## 三、視覺伺服（Visual Servoing）

視覺伺服是三者中最精密的，它把相機直接放進控制迴路：持續比較「當前影像」與「目標影像」的差異，並產生運動指令以縮小差異，直到兩者一致。

它有兩大流派：

- 影像式視覺伺服（IBVS）：直接以影像上的特徵誤差（例如目標點應該在畫面中央，現在偏左 30 像素）作為控制訊號。優點是不需重建三維資訊、對相機校正誤差不敏感；缺點是機器人的實際運動軌跡可能不直觀。
- 位置式視覺伺服（PBVS）：先由影像解出「相機相對目標的三維位姿誤差」（用第 21 章的 PnP），再據此規劃運動。優點是運動軌跡在三維空間中合理；缺點是需要準確的相機校正與目標模型。

視覺伺服的典型應用是精確對接：機器人手臂抓取物件、掃地機器人回充電座對位、無人機精準降落。它能達到公釐級的相對定位精度——遠高於 SLAM 的全域定位精度，因為

它只關心「相對於眼前目標」的位置。

| 方法 | 需要環境改造 | 精度 | 彈性 | 典型應用 |
|---|---|---|---|---|
| 線追蹤 | 需要（貼線） | 中 | 低（固定路線） | 工廠 AGV、教學機器人 |
| 地標導航 | 視地標而定 | 中 | 中（拓樸路線） | 走廊巡邏、展場導覽 |
| 視覺伺服 | 不需要 | 極高（公釐級） | 低（僅對目標） | 精確對接、抓取、降落 |

**表 24-2　三種基本視覺導航策略比較**

### 量一量：兩種視覺伺服差在哪

24.2 節說影像式視覺伺服「機器人的實際運動軌跡可能不直觀」，位置式則「運動軌跡在三維空間中合理」。這兩句話聽起來像是風格差異，其實是會決定成敗的實質差別。本小節把它算出來。

實驗設定：相機要從一個偏離的起始位姿，移動到正對目標、距離 0.65 公尺的位置。目標是一個邊長 20 公分的方形上的四個角點。兩種控制律都用標準形式——影像式以影像雅可比矩陣的偽逆乘上特徵誤差，位置式則先解出三維位姿誤差再直接命令速度。關鍵變因是起始時相機繞光軸轉了多少度。

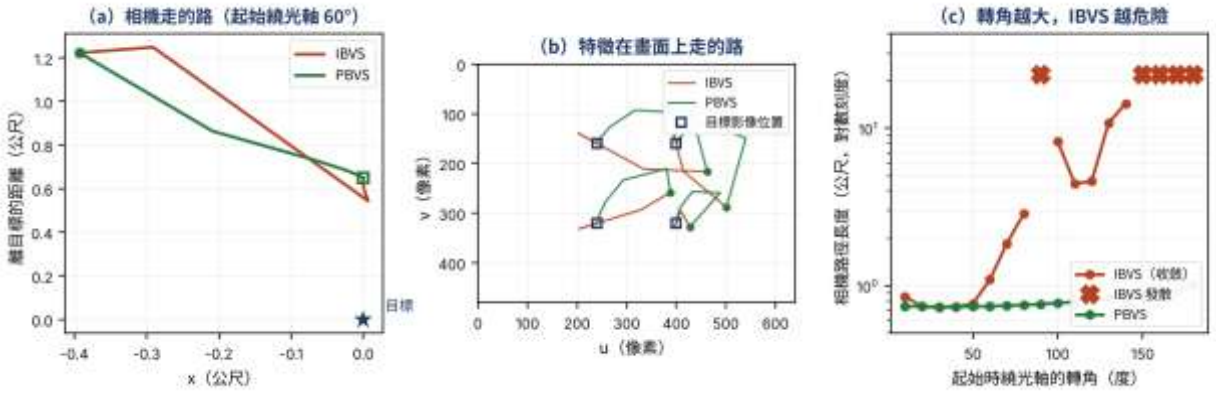



**圖 24-1　影像式與位置式視覺伺服的收斂行為**

（a）是相機在空間中走的路，起始時繞光軸轉了 60 度。位置式（綠）平順地由遠而近收斂

到目標距離；影像式（紅）則先衝過頭、距離一度縮到 0.54 公尺（比目標的 0.65 還近），再退回來。路徑長度分別是 0.74 與 1.09 公尺。這個「先衝過頭再退回」正是影像式視覺伺服著名的相機後退現象。

（b）是同一次過程在影像平面上的樣子，而這裡的勝負完全相反。影像式的四條特徵軌跡幾乎是直線——這正是它的設計目標：它最小化的是影像上的誤差，所以特徵會沿著畫面上的最短路徑移動。位置式的特徵軌跡則明顯繞彎，因為它只關心三維位姿，不管特徵在畫面上怎麼走。

這組對照說明了兩者的本質：影像式保證畫面上的行為漂亮（特徵不容易跑出視野），但相機的實際運動可能很怪；位置式保證相機的運動漂亮，但特徵可能在過程中跑出畫面外。選哪一個，取決於你的失敗模式是什麼——手臂在狹窄空間作業怕撞到，就該選位置式；目標只佔畫面一小塊、怕追丟，就該選影像式。

（c）把起始轉角從 10 度掃到 180 度。位置式的路徑長度全程維持在 0.74 至 1.03 公尺之間，幾乎就是起點到終點的直線距離，而且每一次都收斂。影像式在 50 度以內同樣漂亮（約 0.73 公尺），超過之後路徑迅速膨脹——60 度是 1.09 公尺、80 度是 2.86 公尺、140 度是 14.4 公尺，而在 90、150、160、170、180 度這幾個設定上直接發散。

### 180 度那個著名的反例

影像式視覺伺服最廣為人知的失效案例，正是「目標相對相機剛好轉了 180 度」。此時影像雅可比矩陣的偽逆會命令相機沿光軸直直後退——因為對影像而言，退得越遠，四個角點就越靠近畫面中心，誤差「看起來」在減少。結果是相機一路退到無窮遠，誤差卻始終降不下來。

本實驗在 150 度以上重現了這個現象：路徑長度動輒數百萬公尺，那不是機器人真的走了那麼遠，而是控制律已經失控。

實務上的對策有三種：切換式控制（遠距離用位置式、近距離用影像式）、混合式視

覺伺服（平移用影像式、旋轉用位置式）、或最簡單的——先用其他方式把姿態大致對正，再交給視覺伺服收尾。

## 24.3 AprilTag / ArUco 導航

第 4.6 節介紹過視覺基準標記的原理——由四個角點與相機內參解出完整 6-DoF 位姿（第 21 章的 PnP 問題）。本節聚焦它們在導航任務中的實際用法。

### 四種典型用途

148. 絕對位姿校正：在走廊、電梯口、實驗室門口貼標記。機器人只要看到任一標記，就能取得不漂移的絕對位姿，一舉消除 SLAM 的累積誤差。這是最常見也最實用的用法（第 4.7 節的融合思想）。

149. 精確對接：充電站、取放料點、電梯按鈕前貼標記，配合視覺伺服（24.2 節）完成公釐級的最後對位。SLAM 的定位精度通常在數公分，不足以應付需要精密對接的動作。

150. 樓層與區域識別：第 5.7 節提過，光達 SLAM 無法分辨長得一樣的不同樓層。在每層樓的電梯口貼上不同 ID 的標記，問題迎刃而解。

151. SLAM 精度評估：在場域中密集布置已知座標的標記，可作為近似的真值軌跡，用來評測 SLAM 系統的精度（第 22.7 節的 ATE 評測需要真值）。這對沒有動作捕捉系統的教學實驗室極為實用。

### 實作要點

- 標記尺寸與偵測距離：偵測距離大致與標記邊長成正比。走廊用的標記若太小，機器人要走到很近才看得到，失去校正的意義。實務上會依需求製作不同尺寸。
- 安裝高度與角度：標記應貼在機器人相機視野的合理高度，並盡量正對走道。斜視角度過大時，四個角點的偵測精度下降，解出的位姿誤差變大。

- 光線與反光：標記應避免直接反光（護貝的標記在燈光下可能完全無法辨識）。建議用消光材質列印。
- 唯一性管理：每個標記的 ID 與其世界座標必須建檔管理。這份「標記地圖」本身就是一種簡易的語意地圖（第 3 章）。

### 與 SLAM 的融合層次

第 4.7 節提過三種融合層次，在此具體化：

- 鬆耦合：SLAM 正常運作，只在看到標記時把位姿「拉」到標記給出的值。實作最簡單，適合教學。
- 緊耦合：把標記觀測當作約束加入 SLAM 的後端最佳化（第 12 章的一條邊）。精度最佳，是研究級做法。
- 輔助初始化與重定位：最務實——SLAM 平時自主運作，只在啟動時或跟丟時用標記提供絕對參考。以極低成本解決最頭痛的問題。

### 量一量：標記貼多大、貼多斜、貼多密

24.3 節的「實作要點」給了三條原則：標記太小看不遠、斜視角度過大精度會下降、以及唯一性要管理。前兩條都是可以量化的，而量化之後會得到比原則更有用的東西——可以直接寫進規劃文件的數字。

實驗方法：取一個邊長 16 公分的標記，以 520 像素焦距、640×480 解析度的相機觀測，四個角點加入定位雜訊後解 PnP，與真實位姿比較。角點雜訊的模型是：標記在畫面上投影得越小（距離遠或斜視造成前縮），雜訊越大——這比「固定雜訊」更貼近真實偵測器的行為。

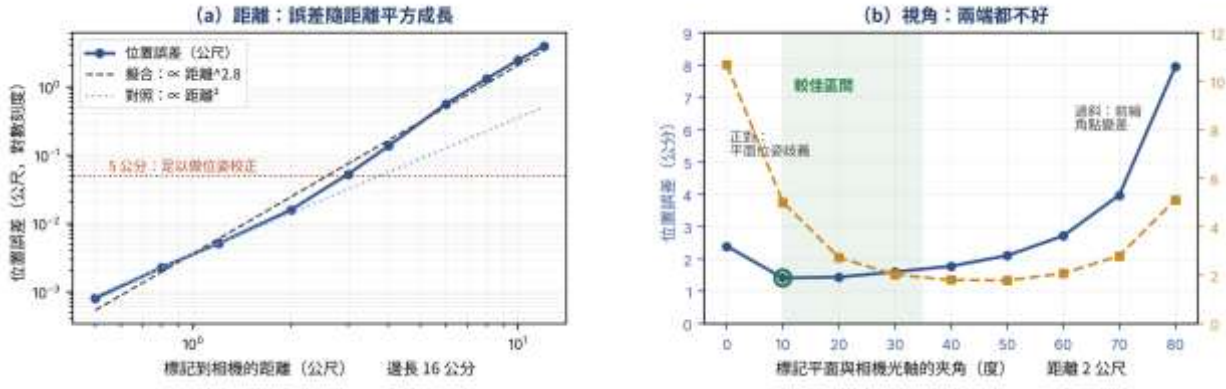



**圖 24-2　標記的位姿精度如何隨距離與視角變化**

（a）是距離的影響，兩軸都是對數刻度。0.5 公尺時位置誤差只有 0.8 公釐，1.2 公尺時 5 公釐，3 公尺時 5.2 公分，6 公尺時 0.56 公尺，12 公尺時 3.9 公尺。擬合出的指數是 2.76——比單純的距離平方更陡。多出來的那 0.76 次方來自角點偵測本身：標記在畫面上越小，角點位置就越難定準，雜訊與幾何放大效應相乘。

這條曲線給出一個很實用的設計規則。若把「位置誤差 5 公分」當作足以做位姿校正的門檻（SLAM 本身的定位誤差通常也在數公分），那麼 16 公分的標記有效距離大約是 3 公尺。走廊若要在 6 公尺外就能用，標記邊長得加倍到 32 公分左右。這就是 24.3 節「偵測距離大致與標記邊長成正比」那句話的具體換算。

（b）是視角的影響，而結果比 24.3 節原本的說法更細緻。原則上「斜視角度過大時位姿誤差變大」是對的——80 度時位置誤差是 8 公分，比最佳值差了五倍以上。但完全正對也不好：0 度時角度誤差高達 10.7 度，是整條曲線的最差點。

正對之所以糟，是因為平面目標的位姿歧義：當標記正對相機時，「標記往左傾一點」與「標記往右傾一點」投影出來的四個角點幾乎一模一樣，旋轉方向在數學上接近不可觀測。稍微斜一點，這個對稱性就被打破了。兩端各有各的問題，最佳區間落在大約 10 至 35 度。

> **一條可以直接用的安裝規則**
>
> 把圖 24-2 的兩個結論合起來，走廊標記的安裝建議是：邊長取「預期偵測距離除以

20」（16 公分對應 3 公尺、32 公分對應 6 公尺）；不要正貼在走道正前方的牆上，而是略微側貼或讓機器人斜向經過，使視角落在 10 至 35 度之間。

這兩件事都不花錢，卻能讓同一批標記的精度差上好幾倍。同學在規劃場域時值得先用自己的相機跑一次圖 24-2 的實驗——不同的焦距與解析度會得到不同的數字。

### 量一量：標記該多久貼一個

前面量的是單一標記的精度，接下來是更實際的系統問題：一條巡邏路線要貼幾個標記才夠？

實驗設定是一條 90 公尺的ㄇ字形走廊巡邏路線，共 360 個關鍵幀。里程計每一步都有雜訊，繞完整條路線會累積出明顯漂移。標記每隔固定距離貼一個，量測到的絕對位姿帶有雜訊（位置標準差 2 公分、角度 2 度，由圖 24-2 的結果推得）。比較三種做法：完全不用標記、鬆耦合（看到標記就把位姿拉過去）、以及緊耦合（把標記觀測當成最佳化的約束，與里程計一起解）。

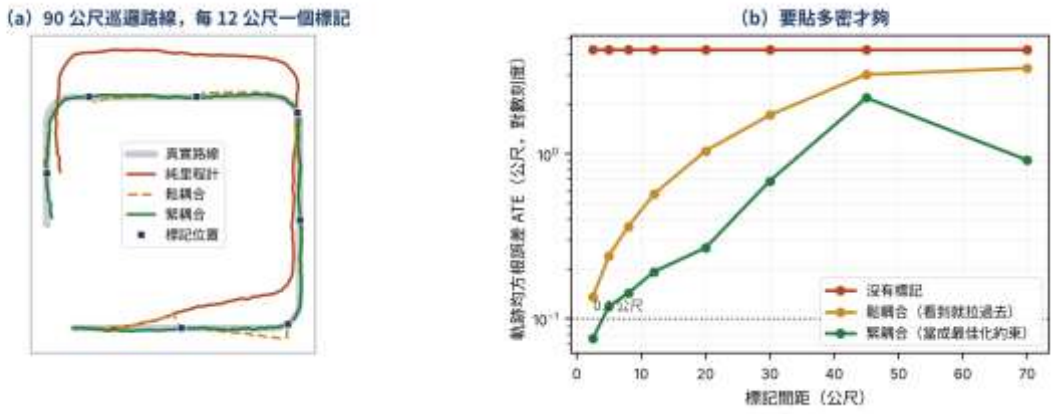



**圖 24-3　標記間距對定位精度的影響**

（a）畫出每 12 公尺貼一個標記時的實際軌跡。純里程計（紅）在走完一圈後已經明顯偏離路線；鬆耦合（橘虛線）在每個標記處被拉回來，但兩個標記之間仍會漂開，形成鋸齒狀的軌跡；緊耦合（綠）則全程貼合真實路線。

（b）掃描標記間距。沒有標記時 ATE 固定是 4.29 公尺。鬆耦合在間距 2.5 公尺時是 0.136 公尺，5 公尺時 0.241、12 公尺時 0.571、20 公尺時 1.04 公尺。緊耦合在相同間距下分別

是 0.076、0.119、0.193、0.269 公尺——每一點都比鬆耦合好，而且間距越大、差距越明顯：20 公尺間距時緊耦合的誤差只有鬆耦合的四分之一。

為什麼緊耦合在稀疏的情況下優勢更大？因為鬆耦合只能「在看到標記的那一瞬間」修正，修正量無法往回傳給前面那些已經走過的位姿；緊耦合則把標記當成最佳化裡的一條約束，誤差會沿著整條位姿鏈重新分配。這與第 22 章圖 22-2 的結論是同一件事——修正量要靠圖的連通性才能傳開。

必須誠實地指出一個細節：在 45 公尺與 70 公尺的間距下，整條路線只剩一個標記，結果就由「那個標記剛好貼在哪裡」主宰，而不再反映間距本身。圖上 70 公尺那一點的誤差反而比 45 公尺低，就是這個原因。所以這張圖只在「標記數量足夠形成鏈」的範圍內可以直接讀。

**給場域規劃的三個數字**

以本實驗的設定（90 公尺路線、數公分級的里程計雜訊）為例：

要把 ATE 壓在 10 公分以內，緊耦合需要約 5 公尺的間距、鬆耦合則需要 2.5 公尺以內——也就是說，採用緊耦合可以讓標記數量減半。

若只要求 30 公分以內（多數巡檢任務足夠），緊耦合可以放寬到約 20 公尺的間距，整條路線只需要四、五個標記。這個差別直接決定了布建成本與維護工作量。

## 24.4 YOLO 物件辨識與語意導航

前三節的導航，機器人都不「理解」它看到的東西——線就是線、標記就是編號。加入物件辨識之後，機器人才開始理解場景的意義。

### 從幾何到語意

第 1、3 章提過語意地圖的概念：在幾何地圖之上，標註「這是門、那是樓梯、那邊是滅火器」。YOLO 這類即時物件偵測模型，正是產生這些標註的工具。

有了語意，導航任務的表達方式就徹底改變了：

- 幾何導航：「前往座標（12.5, 8.3）」——使用者必須知道座標。
- 語意導航：「前往最近的飲水機」——使用者只需說出物體名稱。

後者才是人類自然的溝通方式，也是服務機器人真正可用的前提。

### YOLO 在導航中的三種角色

152. 語意標註：偵測到的物體連同其三維位置（由 RGB-D 或雙目提供深度）一起寫入地圖，形成語意地圖。

153. 動態物體處理：辨識出「人」「車」等會移動的物體，在 SLAM 中把這些區域的特徵剔除，避免動態物體干擾位姿估計（第 9.5 節提過的視覺 SLAM 弱點）。這是一個極為實用的技巧。

154. 任務目標偵測：巡檢機器人辨識儀表、裂縫、垃圾；資源回收機器人辨識瓶罐種類——這是任務本身的核心功能。

### 機器人視角的挑戰

這裡有一個容易被忽略、但在實務上極為關鍵的問題：大多數視覺模型是用「人類視角」的照片訓練的，而機器人的視角往往截然不同。

研究指出，四足機器人的自主導航需要穩健的語意感知，然而由於這類平台在低於 50 公分的高度運作，以人類中心視角訓練的視覺模型往往難以良好運作。

這個問題對掃地機器人、機器狗、小型輪型機器人都成立：從 30 公分高往上看的椅子、從地面看到的門，與訓練資料中的照片差異很大，辨識率因此下降。

- 對策一：蒐集機器人視角的資料重新訓練或微調模型——這正是第 31 章專題四階段四（訓練 YOLO 辨識巡檢目標）與第 32 章語意標註的用意。
- 對策二：使用具備開放詞彙能力的較新模型，它們的泛化能力較強。

- 對策三：調整相機安裝高度與俯仰角，盡量接近訓練資料的視角。

這個案例再次印證本書反覆強調的觀念：技術的效能高度依賴部署情境，不能只看論文上的數字。

## 24.5 VLM 視覺語言模型輔助導航

YOLO 讓機器人認得物體，但它仍然只能回答「畫面中有什麼」。VLM（視覺語言模型）則更進一步——它能理解場景、推理常識、並接受自然語言指令。這是機器人導航正在發生的典範轉移。

### VLM 帶來什麼

VLM 在大規模影像與文字語料上預訓練，展現出統合視覺與語言模態的強大能力，對需要豐富語意感知的導航任務日益重要。早期模型把影像與文字特徵對齊到共同的嵌入空間，而 CLIP 引入對比學習實現開放詞彙的視覺辨識，大幅提升多模態表徵的泛化能力；近期的 VLM 更進一步提供視覺語言推理與決策支援的統一介面。

對導航而言，VLM 的關鍵價值在於常識推理。舉例來說，若指令是「去找洗手台」，機器人不需事先標註過洗手台的位置——VLM 知道洗手台通常在廁所或廚房，因此會優先探索這類區域。這種能力稱為語意先驗。

### 三種應用模式

155. 視覺語言導航（VLN）：接受自然語言指令，如「走到走廊盡頭，右轉後在第二扇門停下」。系統需理解指令、對應到當前視野、逐步執行。

156. 物件目標導航：接受「去找一個垃圾桶」這類目標，系統需在未知環境中探索並找到目標。VLM 的常識推理讓探索更有效率。

157. 社交感知導航：理解場景中的人與情境，決定合宜的行為（不從人群中硬穿、在教室外放慢速度）。

### 零樣本能力：不需任務特定訓練

VLM 導航最具突破性的特質，是零樣本能力——不需要為每個新環境或新任務重新訓練。

相關研究以 VLM 作為全域規劃器，把多個機器人不同視角的子地圖聚合為統一的全域地圖，編碼機器人狀態與邊界區域；VLM 據此在機器人之間分配邊界，促成協調而高效的探索。實驗顯示這類方法在成功率與導航效率上優於既有基準，且不需要任務特定的訓練；消融實驗進一步證實了 VLM 語意先驗的重要性。

這與傳統的學習式導航形成鮮明對比：後者往往需要在目標環境中蒐集大量資料訓練，換個場域就要重來。

### 前沿方向：視覺語言動作模型（VLA）

更新的發展是 VLA（Vision-Language-Action）模型——不只理解，而是直接由影像與語言產生動作指令。

近期研究提出能實現全模態目標條件化的訓練框架：以高容量的視覺語言動作骨幹，同時以 2D 位姿、第一人稱影像與自然語言三種目標模態（以及它們的組合）進行訓練。這呼應了人類的能力——我們能靈活地解讀並組合不同形式的目標指定，無論是語言指令、空間座標還是視覺參照。

另有研究展示同一套代理核心能部署到四種異質的真實機器人上——雙臂輪式平台、四足機器人、人形機器人與自建的四旋翼無人機——只需更換底層控制器。這種跨平台的通用性，是機器人基礎模型（Robotics Foundation Model）願景的具體展現。

### 實務限制：別忘了工程現實

VLM 導航令人興奮，但同學務必認識它的限制：

- 運算負擔：大型 VLM 推論耗時，難以在機器人上即時運行。因此出現了各種效率優化研究，例如以注意力為基礎的權杖重要性評估與查詢引導的時空過濾，在不重新訓

練或修改預訓練模型的前提下保留導航相關資訊，並在真實的四足機器人上驗證了可靠且低延遲的指令跟隨導航。

- 幻覺風險：VLM 可能「看到」不存在的東西或做出錯誤推理。若機器人完全信任它，可能撞牆或走錯路。實務系統必須保留幾何層的安全檢查（第 29 章的避障）。
- 延遲與安全：VLM 適合做高階決策（去哪裡），不適合做即時控制（閃避突然出現的障礙）。分層架構是必然選擇——這正是第 3 章混合式架構的現代版本。

這些限制正是第 32 章要深入討論的主題。同學此刻只需建立一個核心認知：VLM 讓機器人「聽得懂、想得通」，但「走得穩、不撞人」仍然要靠前面二十三章的幾何與控制技術。兩者缺一不可。

## 24.6 機器狗視覺導航

四足機器人（機器狗）近年成為視覺導航研究與應用的熱門平台。本節說明它的特殊挑戰與對策。

### 為什麼選機器狗

- 地形適應力：能上下樓梯、跨越門檻與碎石、走過不平地面——這是輪型機器人做不到的（第 3.6 節）。
- 校園與工廠場域適用：有樓梯、斜坡、戶外通道的校園場域，機器狗的通行能力明顯優於輪型。
- 平台成熟：商用四足機器人已相當普及，並提供開放的開發介面，適合作為研究與教學平台。

### 四項特殊挑戰

158. 步態震動：四足行走時機身持續上下起伏與左右擺動，造成影像模糊與劇烈的位姿變化。這對視覺 SLAM 是嚴峻考驗——這正是第 23 章 IMU 緊耦合不可或缺的原因，也呼應第 18 章機器狗需要光達慣性緊耦合的論述。

159. 低視角問題：如 24.4 節所述，機器狗在低於 50 公分的高度運作，以人類中心視角訓練的視覺模型往往難以良好運作。語意感知的準確度因此下降。

160. 三維地形理解：輪型機器人只需知道「前方能不能走」，機器狗還需要知道「這個台階多高、踏得上去嗎、落腳點穩不穩」。導航從 2D 升級為 2.5D 甚至完整 3D（第 3.6 節）。

161. 續航與運算限制：四足行走的能耗遠高於輪型，可搭載的運算平台也受限。這使得輕量化的演算法與邊緣運算格外重要（第七篇的主題）。

### 對策與系統設計

- 感測器組合：3D 光達或深度相機（地形）+ IMU（姿態與震動補償）+ RGB 相機（語意）。多感測器融合在此不是選項而是必需。
- 緊耦合估測：以第 18 章的 LIO 或第 23 章的視覺慣性 SLAM 為骨幹，IMU 是穩定度的關鍵。
- 分層控制：上層做語意決策與路徑規劃（可用 VLM），中層做地形評估與落腳點規劃，底層做步態控制。各層時間尺度不同，這是第 3 章混合式架構的典型實現。
- 語意模型的在地化：蒐集機器狗視角的資料微調模型，是提升辨識率最直接的方法。

### 研究前沿

機器狗已成為驗證前沿導航技術的標準平台。近年的視覺語言導航研究，普遍會在真實的四足機器人上驗證方法的實用性——因為它同時具備真實世界的複雜度與可控的實驗條件。已有研究把專為腿式機器人設計的視覺語言動作導航模型作為代表性工作提出，顯示這個方向的活躍程度。

## 24.7 校園巡檢與資源回收辨識機器人

本節把本章技術整合成兩個具體的專題規劃，作為第六篇的收尾，也為第 31 章的專題預作準備。

### 專題一：校園巡檢機器人

這是第 5.7 節、第 18.7 節都提過的主軸，本節從視覺導航的角度完整規劃。

**圖 24-4 校園巡檢機器人的系統架構**

【感知層】RGB-D 或雙目相機 + IMU +（可選）光達、GNSS

↓

【定位建圖】ORB-SLAM3 視覺慣性 SLAM（第 23 章）

+ AprilTag 絕對位姿校正（24.3 節）

↓

【語意理解】YOLO 物件偵測 → 語意地圖（24.4 節）

↓

【任務決策】巡檢點清單 / VLM 自然語言任務（24.5 節）

↓

【路徑規劃】全域規劃（地圖）+ 局部避障（即時感知）

↓

【執行與紀錄】移動至巡檢點 → 拍照 / 量測 → 標上三維座標存檔

#### 巡檢任務的視覺重點

- 儀表判讀：偵測並讀取壓力表、電表、溫度計的數值。這需要精確對位（視覺伺服，24.2 節）加上數字辨識。
- 異常偵測：裂縫、漏水痕跡、鏽蝕、雜物堆放——這些多半只有視覺看得出來（第 8 章提過光達給不了語意）。

- 變化偵測：比較不同時間的同一位置影像，找出「昨天沒有、今天出現」的異常。這需要精確的重複定位能力（第 23 章 Atlas 的多階段作業正好適用）。
- 門禁與安全：偵測門是否關好、消防通道是否堵塞、滅火器是否在位。

### 專題二：資源回收辨識機器人

這是一個兼具技術挑戰與教育意義的題目，特別適合高中與大學部合作。

- 任務定義：機器人在校園中巡行，辨識並記錄（或收集）可回收物，分類為寶特瓶、鋁罐、紙類、一般垃圾。
- 視覺挑戰：物體姿態多變（躺著、立著、壓扁的）、材質反光（鋁罐）、透明（寶特瓶，這對深度相機也是難題，呼應第 9 章的限制）、光線多變（戶外）。
- 資料集建置：這是專題的核心工作——在校園實地拍攝各種角度、光線、背景下的回收物，建立標註資料集，訓練 YOLO 模型。這正是第 31 章專題四階段四的方法。
- 系統整合：SLAM 提供位置（知道在哪裡發現的）、YOLO 提供辨識、導航讓機器人自主巡行、資料庫記錄成果。

### 教學層次的規劃

| 階段 | 內容 | 適合對象 |
|---|---|---|
| 一 | 線追蹤與 AprilTag 導航，理解基本視覺導航 | 高中 |
| 二 | 訓練 YOLO 辨識校園物件，建立資料集 | 高中進階 / 大學部 |
| 三 | SLAM 建圖 + 語意標註，產生語意地圖 | 大學部 |
| 四 | 自主巡檢：規劃路線、執行任務、產出報告 | 大學部專題 |
| 五 | VLM 自然語言任務、跨階段變化偵測 | 專題 / 研究生 |

**表 24-3 視覺導航專題的五個教學階段**

這個層次規劃的優點是：每個階段都有可展示的成果，學生不必等到最後才看到東西動起來；而且各階段技術逐步疊加，前面的成果不會白費。

至此，第六篇「Visual SLAM 與視覺導航機器人」全部完成。我們從第 20 章的特徵基礎出發，經過第 21 章的幾何、第 22 章的 ORB-SLAM2、第 23 章的 ORB-SLAM3，到本章把技術落實為真實任務。下一篇，我們把視野轉向極端受限的平台——微型機器人與無人機。當運算資源只有毫瓦等級、當載重只有數十公克時，SLAM 該如何實現？第七篇「微型機器人、無人機與邊緣 AI SLAM」將帶來截然不同的思考。

## 24.8 語言導航的現況：2025 年的一次誠實評估

24.5 節描述了視覺語言模型輔助導航的願景。本節補上一件同樣重要的事：以目前的技術，它做得到什麼程度。

### 一、一個具身導航的基準

2025 年底發表的一個基準提供了相當清楚的答案。它的設計是：給模型一張真實場景的照片、一段文字指令，以及一個載體類型（人、四足機器人、輪型機器人、腳踏車），要模型在影像上畫出應該走的軌跡。資料集有 1,000 個情境與超過 3,000 條專家軌跡，評分同時考慮軌跡形狀、終點準確度，以及「這個載體不該走的地方」的懲罰。

| 方法 | 分數 |
| --- | --- |
| 人類專家 | 75.40 |
| 只預測路徑（直接告訴它目標在哪） | 56.55 |
| 已知目標的直線基準 | 51.89 |
| 完整軌跡預測（最佳模型） | 34.38 |
| 只預測目標點 | 29.65 |

**表 24-4　具身導航基準的評估結果（分數越高越好）**

### 二、三個應該被記住的數字

第一，最佳模型的 34.38 分與人類專家的 75.40 分差了 41.02 分。這不是「再過半年就追上」的差距。

第二，也是最值得玩味的：一條「已知目標的直線」拿到 51.89 分，遠高於最佳模型完整軌跡的 34.38 分。一條不避障、不繞路、完全不看環境的直線，勝過當代最強模型的完整規劃。這說明模型的失分主要不在路徑形狀，而在它沒搞清楚目標在哪。

第三點佐證了這個判斷：當目標被直接告知時，同一個模型只預測路徑能拿到 56.55 分；反過來只預測目標點則掉到 29.65 分。模型「會走」但「不會認」——把語言裡的目標對應到影像中正確的位置，才是目前真正的瓶頸。

### 三、這對本章的分層架構意味著什麼

這三個數字直接支持本章 24.1 到 24.3 節的立場：既然模型最弱的環節是目標定位，那麼能用標記確定的位置，就不要讓模型去猜。一個貼在教務處門口的標記，比任何視覺語言模型都更可靠地回答「教務處在哪裡」。

反過來說，模型相對擅長的事情應該交給它：理解口語指令、判讀畫面內容、生成回報、以及在已知目標的情況下描述路線。務實的問題不是「語言模型能不能做導航」，而是「導航的哪些部分適合交給它」。

這也呼應案例 24-2 的低視角問題：模型在機器人視角下的表現，與在人類視角下有明顯落差。第 32 章會回到資料蒐集與微調的做法，第 34 章則會把這整條語言到行動的鏈路完整展開。

## 公式或流程

### 一、線追蹤的比例控制

設影像寬度為 W、偵測到的線中心像素為 x_line、畫面中心為 x_center = W / 2，則橫向偏差為：

$$e = x_line - x_center$$

最基本的比例控制器直接把偏差轉為角速度指令：

$$\omega = -K_p \cdot e$$

其中 K_p 為比例增益。負號表示線偏右（e 為正）時應右轉。實務上常加入微分項以抑制震盪（PD 控制），或依偏差大小調整線速度——偏差大時放慢、走直線時加速。這是最簡單的視覺回授控制，也是同學理解視覺伺服的最佳起點。

## 二、影像式視覺伺服的控制律

IBVS 的目標是讓當前影像特徵 s 逼近目標特徵 s*。定義特徵誤差：

$$e = s - s^*$$

控制律為（以指數方式收斂誤差）：

$$v = -\lambda \cdot L^{+} \cdot e$$

其中 v 是相機的六自由度速度指令、λ 是收斂速率、L 是影像雅可比矩陣（描述「相機動一單位，影像特徵會移動多少」）、$L^{+}$ 為其偽反矩陣。這條式子的精神與第 11、12 章的最佳化一脈相承：定義誤差、求出誤差對控制量的敏感度、沿誤差下降方向移動。

## 三、語意導航的目標轉換

語意導航的核心是把「語意目標」轉為「幾何目標」：

**圖 24-5 從語言指令到運動指令的完整流程**

使用者指令：「請去最近的飲水機」

↓

① 語言理解（VLM / LLM）：解析出目標物件類別 = 飲水機

↓

② 語意地圖查詢：地圖中是否已標註飲水機？

├─ 有 → 取得其三維座標，直接進入 ④

└─ 無 → 進入 ③ 探索

↓

③ 語意探索：以 VLM 常識推理選擇有希望的探索方向

（飲水機通常在走廊、茶水間附近）→ 邊界探索 + YOLO 偵測

↓

④ 幾何路徑規劃：在地圖上算出到目標的路徑（第 3、29 章）

↓

⑤ 局部避障與運動控制：即時感知，輸出速度指令

↓

⑥ 抵達後精確對接（視覺伺服，如需要）

這條流程清楚呈現了本書的整體架構：上層由 AI 理解語意，中層由 SLAM 與規劃提供幾何，底層由控制執行動作。三層缺一不可——這也是 24.5 節末尾強調的核心認知。

## 四、AprilTag 位姿校正的融合

當機器人偵測到已知世界座標的標記時，可由標記推算自身位姿：

$$T_{world\leftarrow robot} = T_{world\leftarrow tag} \cdot T_{tag\leftarrow camera} \cdot T_{camera\leftarrow robot}$$

其中 T_world←tag 由標記地圖查得（已知）、T_tag←camera 由 PnP 解出（第 21 章）、T_camera←robot 為外參（第 10 章的相機安裝位置）。三個變換相乘即得機器人在世界座標中的位姿——這正是第 10 章「變換組合就是矩陣連乘」的直接應用。

## 案例

### 案例 24-1　從貼一條線到聽懂人話：視覺導航的四十年

把本章的技術依時間排列，會看到一條清晰的演進脈絡，而且每一步都對應著「機器人理解世界的深度」在提升。

- 1980 年代——線追蹤：機器人只認得亮度差異，環境必須為它改造（貼線）。它不知道自己在哪、也不知道要去哪，只是不斷修正偏差。
- 2000 年代——地標與視覺伺服：機器人開始能辨識特定圖案，並精確地相對定位。環境改造需求降低（只需貼標記），但仍需改造。
- 2010 年代——視覺 SLAM：機器人能自行建立地圖並定位，不再需要改造環境。但它理解的是「幾何」，不是「意義」。
- 2020 年代——語意與 VLM 導航：機器人開始理解「那是門、那是飲水機」，並能接受自然語言指令。它終於能用人類的方式溝通。

觀察這條脈絡，同學會發現一個有趣的規律：每一代技術降低的，都是「人類為機器人做的準備工作」。從改造環境、到標註地標、到只需下達指令——機器人越來越能適應人類的世界，而非要求人類遷就它。

但這裡有一個重要提醒：新技術並未淘汰舊技術。今天的工廠 AGV 仍大量使用磁條導引，因為在可控環境中它最可靠、最便宜；精確對接仍需視覺伺服，因為 SLAM 的精度不夠。工程師的任務不是永遠用最新的技術，而是為每個問題選擇最合適的技術——這是本書從第 8 章開始就反覆強調的判斷力。

## 案例 24-2 低視角的困境：當模型不認識機器人眼中的世界

這是一個看似技術細節、實則影響深遠的問題，也是同學做專題時最容易踩到的坑。

狀況是這樣的：同學下載了一個在標準資料集上表現優異的物件偵測模型，興沖沖裝到機器狗或掃地機器人上，結果辨識率慘不忍睹——明明是椅子，模型卻認不出來；明明前方是門，模型標成了牆。

原因如研究所指出：四足機器人在低於 50 公分的高度運作，而以人類中心視角訓練的視覺模型往往難以良好運作。訓練資料中的椅子照片，多半是站著的人以約 150 公分高度拍攝的；從 30 公分高往上看，椅子的形狀、透視、遮蔽關係都完全不同。

這個問題還有幾個變形：掃地機器人看到的是家具的下半部與地板紋理；無人機看到的是物體的頂視圖；水下機器人面對的是色偏嚴重的影像。共同點是：部署情境與訓練情境的分布不一致。

解法沒有捷徑——蒐集目標視角的資料重新訓練或微調。這聽起來麻煩，但正是第 31 章專題四階段四的核心，也是專題最有價值的部分：一份「在地的機器人視角校園物件資料集」，本身就是可發表、可延續的研究成果。

這個案例給同學的更深啟示是：AI 模型不是萬用的黑盒子。理解它的訓練資料分布與部署情境的差異，是負責任地使用 AI 的基本功。

## 案例 24-3 多機器人協作探索：VLM 作為全域規劃器

本章 24.5 節提到的一項研究，值得單獨展開，因為它展示了 VLM 導航的完整潛力。

該研究提出的框架把視覺語言模型整合為全域規劃器，實現具備常識推理的多機器人視覺目標導航。它把多個機器人不同視角的子地圖聚合為統一的全域地圖，編碼機器人狀態與邊界區域；VLM 據此在機器人之間分配邊界，促成協調而高效的探索。

這個設計精彩之處在於：VLM 不是用來做低階控制，而是扮演「指揮官」的角色。它

看著全域地圖，運用常識判斷「這幾個未探索方向中，哪一個最可能找到目標」，再把任務分配給不同的機器人。低階的移動與避障，仍由傳統的規劃與控制負責。

實驗結果顯示，這個方法在成功率與導航效率上都優於既有基準，且不需要任務特定的訓練；消融實驗進一步證實了 VLM 語意先驗的重要性。研究團隊也在真實場景中以四足機器人驗證了這個框架。

對校園場域的啟發是：這正是「多台機器狗校園巡檢」專題（第 27、31 章）的技術路徑。多台機器人分頭探索、VLM 統籌分配、各自以 SLAM 定位建圖、最後合併成完整的校園語意地圖——這是一個層次豐富、可分工進行、且與國際前沿接軌的專題題目。

## 實作活動

### 實作 24-1　從線追蹤到語意導航的階梯式實作

目的：依照 24.7 節的五階段規劃，逐步建立視覺導航能力。本實作可依班級程度與資源選擇進行到哪一階段，每一階段都有可展示的成果。

**階段一：線追蹤（高中適用，不需 SLAM）**

108. 準備一台附相機的教學小車（或在模擬環境中），在地面貼出一條含直線與轉彎的路線。

109. 以 OpenCV 讀取影像、轉灰階、二值化，找出線的中心位置。

110. 依公式一實作比例控制器，把橫向偏差轉為轉向指令，讓小車自動循線。

111. 參數實驗：調整比例增益 K_p，觀察太小（反應遲鈍、出彎道）與太大（左右震盪）的現象，找出合適值。

112. 挑戰：在路線中加入十字交叉或分岔，設計規則讓小車做出選擇。

**階段二：AprilTag 導航與位姿校正（高中進階 / 大學部）**

113. 在教室四周貼上數個 AprilTag，量測並記錄每個標記的世界座標（建立標記地圖）。

114. 用第 9 章校正過的相機偵測標記，以 PnP 解出相機相對標記的位姿（第 21 章）。

115. 依公式四，把三個變換相乘，算出機器人在世界座標中的位姿，並即時顯示。

116. 與 SLAM 對照：同時執行視覺 SLAM，把兩條軌跡畫在一起——觀察 SLAM 軌跡平滑但漸漸偏離、標記軌跡跳動但始終正確（重現第 4 章案例 4-3）。

117. 簡易融合：實作鬆耦合校正——平時用 SLAM，每當看到標記就把位姿拉回標記給出的值，觀察漂移是否被抑制。

**階段三：YOLO 語意標註（大學部）**

118. 蒐集資料：用機器人視角的相機（注意安裝高度）在校園拍攝目標物件——門、飲水機、滅火器、垃圾桶、樓梯口，每類至少 200 張，涵蓋不同角度與光線。

119. 標註並訓練 YOLO 模型，在驗證集上評估準確率。

120. 視角對照實驗：另外拍一組「人類視角」（約 150 公分高）的測試影像，比較模型在兩種視角下的辨識率，量化重現案例 24-2 的現象。

121. 語意標註：把 YOLO 偵測結果與 RGB-D 深度結合，計算物件的三維位置，標註到 SLAM 建好的地圖上，產生語意地圖。

**階段四：語意導航任務（大學部專題）**

122. 建立語意地圖後，實作一個簡單的語意導航：輸入物件名稱（如「飲水機」），系統查詢地圖中該物件的座標，規劃路徑並導航前往。

123. 加入動態物體處理：用 YOLO 偵測「人」，在 SLAM 中剔除人身上的特徵點，觀察定位穩定度是否改善（24.4 節的第二種角色）。

124. 實作巡檢任務：設定數個巡檢點，讓機器人依序前往、拍照記錄、標上三維座標，最後產出巡檢報告。

### 階段五：VLM 輔助（專題 / 研究生）

125. 把機器人當前影像送入 VLM，以自然語言提問：「前方是否可通行？」「畫面中有樓梯嗎？」「哪裡可能是門？」記錄回答的正確率。

126. 比較 VLM 與 YOLO：在同一批影像上，比較兩者對「開放詞彙物件」（訓練時沒見過的類別）的辨識能力。

127. 延遲量測：記錄 VLM 推論的耗時，評估它適合放在系統的哪一層（高階決策？即時控制？），對照 24.5 節的限制討論。

### 討論問題

39. 階段一中，比例增益太大與太小各造成什麼現象？這與控制理論的哪些概念相關？

40. 階段二中，SLAM 軌跡與 AprilTag 軌跡的差異隨時間如何變化？加入校正後改善多少？

41. 階段三的視角對照實驗中，兩種視角的辨識率差多少？這對「直接套用網路上的預訓練模型」有什麼警示？

42. 階段四中，剔除人身上的特徵點後，定位穩定度改善了嗎？若場景中人很多（如下課時間走廊），系統還能運作嗎？

43. （專題）VLM 的推論延遲是多少？若機器人以每秒 0.5 公尺移動，這個延遲代表機器人在「思考」期間移動了多遠？這對安全設計有什麼意涵？

### 延伸挑戰（大學部）

其一：資源回收辨識機器人——依 24.7 節專題二的規劃，建立校園回收物資料集（特別注意透明寶特瓶與反光鋁罐的拍攝），訓練辨識模型，並整合 SLAM 讓機器人記錄「在哪裡發現了什麼」。這個題目同時涵蓋 AI、SLAM 與環境教育，展示效果佳。其二：多機器人協作探索的簡化版——用兩台機器人（或兩個模擬實例）分頭探索同一場域，各自建圖後合併（可用第 23 章 Atlas 的概念），並比較「協作探索」與「單機探索」完成覆蓋所需的時間，呼應案例 24-3 的研究方向。

## 延伸閱讀

57. F. Chaumette and S. Hutchinson, "Visual Servo Control, Part I: Basic Approaches" 與 "Part II: Advanced Approaches," IEEE Robotics & Automation Magazine, 2006–2007.（視覺伺服的權威教程，IBVS 與 PBVS 的完整說明）

58. E. Olson, "AprilTag: A Robust and Flexible Visual Fiducial System," ICRA, 2011.（第 4、24 章共同的基準標記理論來源）

59. A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision (CLIP)," ICML, 2021.（開放詞彙視覺辨識的里程碑，VLM 導航的基礎）

60. B. Yu et al., "Co-NavGPT: Multi-Robot Cooperative Visual Semantic Navigation Using Vision Language Models," arXiv, 2023（2025 修訂）.（案例 24-3 的來源，多機器人 VLM 協作探索）

61. N. Yokoyama et al., "VLFM: Vision-Language Frontier Maps for Zero-Shot Semantic Navigation," ICRA, 2024.（零樣本語意導航的代表作）

62. Ultralytics YOLO 官方文件（docs.ultralytics.com）.（物件偵測模型的訓練與部署，第 31 章專題四與第 32 章語意標註的主要工具）

63. OpenCV 官方文件之 ArUco 模組與影像處理教學（docs.opencv.org）.（線追蹤與標記偵測的實作基礎）

## 小結

57. 視覺 SLAM 回答「我在哪、環境長什麼樣」，視覺導航回答「我該往哪走」；視覺導航不一定需要 SLAM（線追蹤、視覺伺服屬反應式架構），但加上 SLAM 後才具備全域規劃、任意目標導航、斷點恢復與語意任務的能力。

58. 三種基本策略：線追蹤成本極低但只能走預設路線；地標導航建立拓樸地圖、彈性較高；視覺伺服以影像回授直接控制，能達公釐級的相對定位精度，分 IBVS 與 PBVS 兩流派。

59. AprilTag / ArUco 在導航中有四種用途：絕對位姿校正、精確對接、樓層與區域識別、以及作為 SLAM 精度評估的近似真值；與 SLAM 的融合可分鬆耦合、緊耦合與輔助初始化三個層次。

60. YOLO 在導航中扮演語意標註、動態物體剔除與任務目標偵測三種角色；但因四足與小型機器人在低於 50 公分的高度運作，以人類中心視角訓練的模型往往難以良好運作，必須蒐集機器人視角資料微調。

61. VLM 在大規模影像文字語料上預訓練，提供語意先驗與常識推理，使機器人能接受自然語言指令並零樣本執行新任務；但受限於運算負擔、幻覺風險與延遲，適合做高階決策而非即時控制。

62. 機器狗視覺導航面臨四項特殊挑戰：步態震動、低視角語意問題、三維地形理解與續航運算限制；對策是多感測器緊耦合估測與分層控制架構。

63. 校園巡檢與資源回收辨識是整合本篇技術的理想專題，可依線追蹤、AprilTag、YOLO 語意、自主巡檢、VLM 任務五階段規劃，兼顧高中到研究生的不同層次。

## 承先啟後

本章把前四章的技術帶進真實任務，也量出了三組可以直接用的數字：視覺伺服方面，位置式的相機路徑全程維持在 0.74 至 1.03 公尺、每次都收斂，影像式則在起始轉角超過 50 度後路徑迅速膨脹、並在數個角度上直接發散；標記方面，16 公分的標記在 3 公尺處的位置誤差約 5 公分，誤差隨距離的 2.76 次方成長，而最佳視角落在 10 至 35 度（正對會遇到平面位姿歧義）；布建方面，緊耦合在 20 公尺間距下的 ATE 是 0.269 公尺，只有鬆耦合的四分之一，等於讓標記數量減半。

第六篇「Visual SLAM 與視覺導航機器人」至此全部完成。我們從特徵與幾何出發，建立了完整的視覺 SLAM 系統，最後把它落實為真實的機器人任務。但前面所有的討論，都預設機器人有足夠的運算資源與電力。下一篇，我們走向另一個極端：當機器人只有 44 公克、運算功耗只有毫瓦等級時，SLAM 還可能嗎？第七篇「微型機器人、無人機與邊緣 AI SLAM」將展示這個領域最令人驚嘆的工程成就。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第七篇 微型機器人、無人機與邊緣 AI SLAM

## 第 25 章

# NanoSLAM 與低功耗 SLAM

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

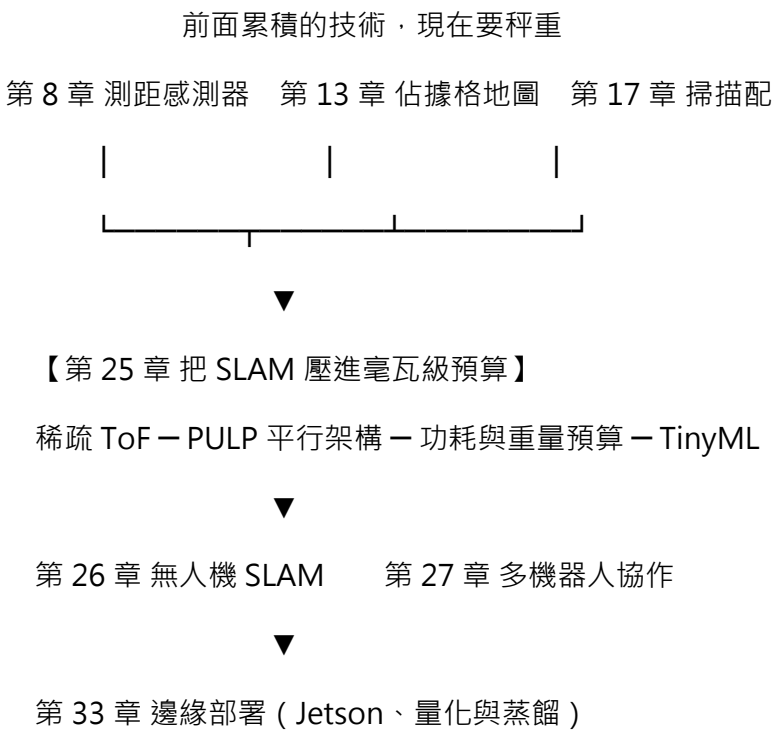
## 本章在全書的位置

第六篇的視覺 SLAM 已經相當成熟，但它有一個沒說出口的前提：機器人身上有一台夠力的電腦，而且插得到電或背得動電池。本章把這個前提拿掉，看看會剩下什麼。

第七篇的主角是極限受限的平台——一台 27 公克的微型無人機、一顆功耗 66 毫瓦的晶片、七分鐘的電池。在這個預算下，第 13 章的佔據格地圖、第 17 章的掃描配準、第 22 章的關鍵幀管理，每一項都必須重新秤過。

本章的三張圖就在秤這些東西：圖 25-1 解釋為什麼多核慢跑比單核快跑省電，圖 25-3 問「這麼稀疏的量測還能定位建圖嗎」，以及 GAP9 的能效究竟領先多少；圖 25-4 則算出一件可能讓同學意外的事——運算模組的「重量」比它的「功耗」更吃續航。

### 圖 25-0　第 25 章的連結地圖




## 學習目標

前六篇的所有系統，都預設機器人有足夠的運算資源與電力——ORB-SLAM3 需要一顆完整的處理器、光達 SLAM 需要數十瓦的運算平台。但如果機器人只有 44 公克重、掌心大小、

電力預算只有毫瓦等級呢？本章展示這個領域最令人驚嘆的工程成就，也開啟第七篇的極限挑戰。學完本章之後，同學應能夠達成下列目標：

57. 說明微型機器人執行 SLAM 的三大限制與傳統解法的問題。
58. 說明 NanoSLAM 的設計目標與五項核心貢獻。
59. 說明 RISC-V 與平行超低功耗運算典範如何支撐毫瓦級 SLAM。
60. 說明毫瓦級 SLAM 的系統設計取捨與各模組的功耗分配。
61. 舉例說明微型無人機、群體機器人與 AR 眼鏡的應用場景。
62. 說明邊緣 AI 與 TinyML 對 SLAM 發展的影響。
63. 規劃一個高中程度的微型無人機地圖建構模擬專題。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 微型無人機 | Nano-UAV / Nano-drone | 掌心大小、數十公克重的超小型飛行器。 |
| 機載運算 | Onboard Computation | 所有運算都在機器人自身完成，不依賴外部設施。 |
| 運算卸載 | Computation Offload | 把運算送到外部電腦執行，受限於通訊頻寬與延遲。 |
| 平行超低功耗 | PULP | 以多核心平行處理換取能效的運算典範。 |
| 系統單晶片 | SoC | 把處理器、記憶體與周邊整合在單一晶片上。 |
| 開放指令集架構 | RISC-V | 開放且可擴充的處理器指令集架構。 |
| 飛時測距矩陣 | ToF Matrix Sensor | 一次量測多個方向距離的小型深度感測器。 |
| 掃描匹配 | Scan Matching | 比對前後量測以估計相對運動（第 13、17 章）。 |

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 位姿圖優化 | Pose Graph Optimization | 以圖優化修正累積誤差（第 12 章）。 |
| 動態頻率調節 | Dynamic Frequency Scaling | 依負載即時調整時脈與電壓以省電。 |
| 深度睡眠 | Deep Sleep | 閒置時把功耗降到極低的省電狀態。 |
| 邊緣人工智慧 | Edge AI | 在裝置端而非雲端執行 AI 推論。 |
| 微型機器學習 | TinyML | 在微控制器等極受限硬體上執行機器學習。 |

## 25.1 為什麼微型機器人很難跑 SLAM

第七篇的第一個問題是：為什麼把 SLAM 裝進微型機器人這麼困難？答案不只是「晶片比較慢」這麼簡單，而是三重限制的疊加。

### 三大限制

- 運算能力：SLAM 的前端（特徵提取、掃描匹配）與後端（圖優化、BA）都極耗運算。第 22、23 章的 ORB-SLAM 系列需要完整的多核心處理器，微控制器等級的晶片根本跑不動。
- 記憶體：第 13 章算過，一張 20×20 公尺、5 公分解析度的 2D 地圖就有 16 萬格；第 14 章的 Gmapping 還要乘上粒子數。而微型平台的記憶體可能只有數百 KB 至數 MB。
- 電力：這是最嚴苛的限制。微型無人機的電池能量極少，而飛行本身就消耗絕大部分電力。若運算再耗掉數瓦，飛行時間會從數分鐘掉到數十秒——系統形同無用。

### 傳統解法：運算卸載，以及它的問題

由於執行 SLAM 演算法所需的龐大運算負載與記憶體需求，目前完全機載的建圖只能在能搭載高瓦數處理器的機器人平台上實現；正因如此，口袋大小、硬體受限的機器人只能把 SLAM 的執行卸載到外部設施。

這個做法有明顯的代價：

- 依賴基礎設施：必須有無線網路與一台外部電腦，機器人無法真正自主。這在災害現場、地下坑道、野外環境根本不可行。
- 通訊延遲：影像或感測資料傳出去、指令傳回來，延遲可能達數十至數百毫秒。對高速飛行的無人機而言，這個延遲可能導致撞牆。
- 頻寬與功耗：持續傳輸大量資料本身也耗電，而且多台機器人同時運作時頻寬會成為瓶頸（第 27 章的多機器人議題）。

- 可靠度：網路一斷，機器人就失去導航能力。

### 本章的核心問題

因此本章要回答的問題是：能不能讓 SLAM 完全在機載完成，而且功耗低到微型無人機負擔得起？

答案是肯定的，而且已經被實現了。這需要三個層面的同步突破：演算法必須輕量化、硬體必須高能效、感測器必須低功耗。接下來的幾節，就是這三者如何協同的故事。

## 25.2 NanoSLAM 的設計目標

NanoSLAM 由蘇黎世聯邦理工學院（ETH Zürich）與波隆那大學的團隊於 2023 年提出，是第 7、25 章反覆提及的代表性工作。它的目標明確而大膽：讓微型無人機完全靠自己執行 SLAM。

### 核心宣言

NanoSLAM 是一套輕量化且最佳化的端到端 SLAM 方法，專為在僅 87.9 毫瓦的功耗預算下，於公分尺寸的機器人上運作而設計。研究團隊在真實場景中展示其建圖能力，並把 NanoSLAM 部署在一台重 44 公克、搭載名為 GAP9 的新型商用 RISC-V 低功耗平行處理器的微型無人機上。

讓同學對這些數字有感：87.9 毫瓦大約是一顆 LED 指示燈的功耗等級；44 公克比一顆雞蛋還輕。在這樣的條件下執行完整的 SLAM——這是本章最令人驚嘆之處。

### 五項核心貢獻

NanoSLAM 論文明確列出五項貢獻，它們構成了本章的技術骨架：

162. 平行化的最佳化實作：針對演算法進行最佳化的平行實作，並評估其執行時間與記憶體需求。

163. 機載 ICP：對掃描匹配的標竿演算法 ICP（第 13、17 章）進行平行實作與評估，能在機載以 55 毫秒完成。

164. 感測擴充板：為商用的 Crazyflie 2.1 微型無人機設計客製化的外掛伴隨板，以 4 顆 ToF 矩陣感測器擴充無人機的感測能力，使其能執行掃描匹配與自主導航。

165. 通訊協定：設計一套通訊協定，協調無人機原廠微控制器與 GAP9 系統單晶片之間的整合與資料交換，規範如何儲存圖結構、交換圖的位姿、新增邊，以及執行圖優化。

166. 實地驗證：廣泛的實地實驗評估，證明系統的閉環建圖功能——運用 NanoSLAM 達成軌跡誤差最多降低 67%，以及 4.5 公分的建圖精度。

**關鍵成果數據**

這個演算法設計上善用 RISC-V 處理核心的平行能力，能以 4.5 公分的精度建立一般環境的地圖，端到端執行時間小於 250 毫秒。

把這些數字與前面章節對照，更能體會其價值：第 23 章 ORB-SLAM3 在 EuRoC 上的 3.5 公分精度，是在完整的電腦上跑出來的；NanoSLAM 的 4.5 公分，則是在毫瓦級的微型平台上達成。研究團隊指出，藉由結合新型低功耗深度感測器與 GAP9 的平行能力，系統達成了與更大、運算能力更強的無人機（通常稱為微型飛行器或標準尺寸無人機）相當的室內建圖精度。

## 25.3 RISC-V GAP9 與低功耗平行運算

演算法再輕量，也需要合適的硬體才能實現。本節介紹讓毫瓦級 SLAM 成為可能的關鍵硬體。

**微控制器設計的新趨勢**

近年微控制器設計的趨勢強調平行處理、硬體加速器與能源效率。GreenWaves Technologies 的 GAP9 系統單晶片正是這些趨勢的體現，適用於微型無人機的專門應用；

它以 RISC-V 核心提供先進的平行能力、電源最佳化與感測器整合，賦予微型無人機即時邊緣運算、更長的飛行時間與更強的資料處理能力。

### PULP：平行超低功耗運算典範

GAP9 建立在平行超低功耗（Parallel Ultra-Low-Power,PULP）運算典範之上，體積小，功耗低於 180 毫瓦。

PULP 的核心思想值得同學理解，因為它與直覺相反：與其用一顆跑得很快的核心，不如用多顆跑得慢的核心平行工作。原因在於晶片的功耗大致與電壓平方及頻率成正比——把頻率降低一半、用兩顆核心平行處理，總運算量相同，但功耗可能降到原本的一半以下。這是「以平行換能效」的工程智慧。

### 晶片規格對照

把微型無人機常見的幾種晶片放在一起比較，能清楚看出 GAP9 的定位：

| 晶片（架構） | CPU 核心數 | 最高頻率 | 功耗 | 定位 |
|---|---|---|---|---|
| STM32(ARM) | 1 | 168 MHz | 116 mW | 飛控與狀態估計 |
| ESP32(Xtensa) | 2 | 240 MHz | 300 mW | 通訊與輕量運算 |
| GAP8(RISC-V) | 8 + 1 | 175 MHz | 96 mW | 前一代 AI 加速 |
| GAP9(RISC-V) | 9 + 1 | 370 MHz | 66 mW | 本章的主角 |

**表 25-1　微型無人機常見系統單晶片比較**

這張表格說明了三件事：第一，GAP9 的核心數遠多於傳統微控制器，體現 PULP 的平行思想；第二，即使核心多、頻率高，它的功耗反而更低，能效優勢明顯；第三，飛控用的 STM32 與運算用的 GAP9 是分工的關係——這正是 25.2 節第四項貢獻（通訊協定）存在的原因。

### 為什麼是 RISC-V

RISC-V 是一套開放的指令集架構，任何人都能自由使用與擴充，不需授權費。這對研究與新創極為有利：

- 可客製化：能為特定運算（如 SLAM 中的矩陣運算）加入專用指令，大幅提升能效。
- 無授權成本：對學術研究與小量產品的成本結構影響顯著。
- 生態成長快：近年 RISC-V 在邊緣 AI 與物聯網領域快速崛起，工具鏈日趨成熟。

對本書讀者而言，認識 RISC-V 的價值在於：它代表了「運算硬體不再由少數廠商壟斷」的趨勢，也是台灣半導體與機器人產業值得關注的方向。

### 進階功能：動態調節與深度睡眠

GAP9 支援執行時期調整時脈頻率（最高 370 MHz）與電壓，以及自動時脈閘控；閒置功耗可降至 45 微瓦。

這意味著系統能「該快則快、該省則省」：執行 SLAM 時全速運轉，飛行平穩無需建圖時降頻甚至進入深度睡眠。45 微瓦的閒置功耗幾乎可以忽略不計——這種細緻的電源管理，正是延長飛行時間的關鍵。

### 量一量：多核慢跑真的比較省電嗎

25.3 節說「與其用一顆跑得很快的核心，不如用多顆跑得慢的核心平行工作」，理由是「功耗大致與電壓平方及頻率成正比」。這個推論值得實際算一遍，因為它還有一個同樣重要的下半段：核心不是越多越好。

模型很簡單。要達成固定的總吞吐量，用 N 顆核心時每顆只需跑 1 / N 的頻率。動態功耗是 $N \times C \times V^2 \times f$，而在近閾值區電壓大致隨頻率線性下降，但有一個下限（電晶體不能低於閾值電壓）。此外每顆核心都會貢獻漏電流，與頻率無關。

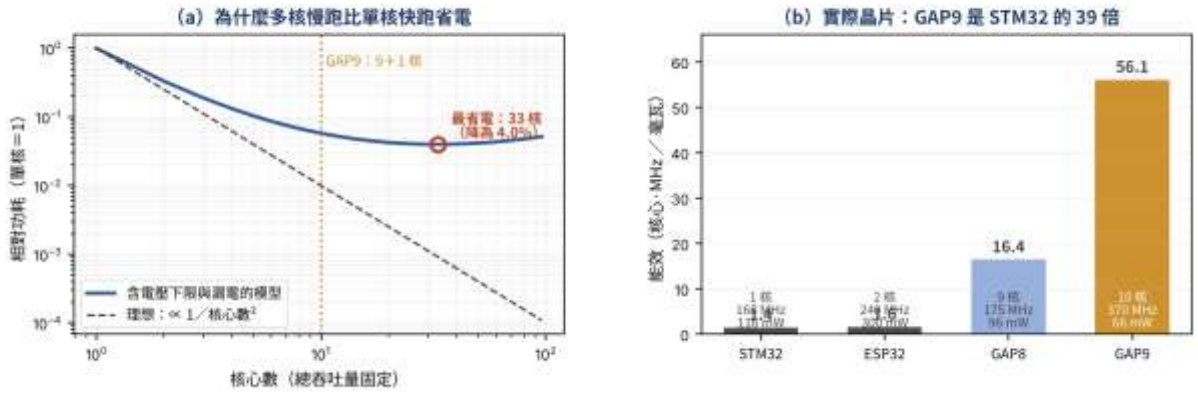



**圖 25-1　PULP 的能效原理與實際晶片的對照**

（a）是模型的結果，兩軸都是對數刻度。灰色虛線是理想情況——若電壓能隨頻率無限下降，功耗會正比於核心數的平方分之一，也就是核心加倍、功耗降為四分之一。藍線是加入電壓下限與漏電之後的實際曲線：前段貼著理想線走，從 1 核到 8 核功耗降到 6.8%，之後逐漸偏離，在 32 核左右觸底（降到 4.0%），再往上反而因為漏電而回升。

這個最低點解釋了一個同學可能沒想過的問題：既然多核省電，為什麼 GAP9 只放 9＋1 顆核心，不放九十顆？因為過了最佳點之後，每多一顆核心帶來的漏電超過它省下的動態功耗。實際晶片的最佳核心數還要再考慮面積成本、記憶體頻寬與平行化效率——真實的工作負載很少能完美切成九十份。

（b）用表 25-1 的真實晶片數據驗證這個原理。把「核心數乘以頻率」當作吞吐量的粗略指標，除以功耗，得到的能效是：STM32 每毫瓦 1.4、ESP32 1.6、GAP8 16.4、GAP9 56.1。GAP9 是 STM32 的 39 倍、ESP32 的 35 倍。

這個倍數不是全部來自平行度。GAP8 到 GAP9 的能效從 16.4 跳到 56.1（3.4 倍），而核心數只從 9 增加到 10——差距主要來自製程、記憶體階層與專用加速器的改進。換句話說，PULP 的平行思想提供了第一個數量級，後續的硬體最佳化又提供了一個數量級。兩者疊加，才讓 SLAM 從瓦級降到毫瓦級。

## 25.4　毫瓦級 SLAM 的系統設計

本節說明 NanoSLAM 如何把第 13、17 章的技術，壓縮進毫瓦級的預算。

### 感測器選擇：ToF 矩陣而非光達或相機

第八章介紹的旋轉式 2D 光達雖然好用，但對微型無人機而言太重、太耗電。相機則需要大量運算才能提取深度（第 9 章）。NanoSLAM 的選擇是 ToF 矩陣感測器——一種小型的飛時測距晶片，一次能量測多個方向的距離。

- 優點：體積僅數公釐、功耗極低、直接輸出距離（不需運算）、無移動部件。
- 配置：如 25.2 節所述，以 4 顆 ToF 矩陣感測器分別朝向四個方向，合起來提供近似 360 度的距離量測——這相當於一個極簡版的 2D 光達。
- 限制：解析度與量測範圍遠低於真正的光達，因此演算法必須能在稀疏的量測下運作。

### 演算法管線

NanoSLAM 的整體流程，是同學已經熟悉的架構，只是每個模組都被極致輕量化：

**圖 25-2 NanoSLAM 的系統管線**

【感測】4 顆 ToF 矩陣感測器 → 近似 360 度的稀疏距離量測

↓

【前端】平行化 ICP 掃描匹配（第 13、17 章）

估出相鄰時刻的相對位姿 → 機載 55 毫秒完成

↓

【圖建構】把位姿存為節點、掃描匹配結果存為邊（第 12 章的位姿圖）

↓

【閉環偵測】辨識重訪位置 → 新增閉環邊

↓

【後端】平行化位姿圖優化（第 12 章）

修正累積漂移 → 軌跡誤差最多降低 67%

↓

【輸出】機載完成的地圖與軌跡 端到端 < 250 毫秒，87.9 毫瓦

### 三個關鍵的輕量化策略

1. 平行化而非簡化：NanoSLAM 沒有把 ICP 或圖優化「改成簡單的近似版」，而是把標準演算法平行化到多核心上。這保住了精度，又善用了 PULP 硬體的特性。

2. 稀疏而非稠密：用 ToF 矩陣提供的稀疏量測取代稠密點雲，大幅減少資料量與運算量——這與第 18 章 LOAM「不需要用上所有的點」的思想一脈相承。

3. 分工而非全包：飛控與狀態估計由原廠的 STM32 負責，SLAM 運算由 GAP9 負責，兩者以專門的通訊協定協調。各司其職，避免單一晶片超載。

### 功耗分配的啟示

把相關數據放在一起，能看出系統設計的取捨：

| 任務 | 功耗 / 延遲 | 說明 |
|---|---|---|
| NanoSLAM 建圖 | 約 87.9 mW，延遲 < 250 ms | 本章主角 |
| 蒙地卡羅定位 | 平均 23 mW | 已知地圖時的定位（第 11 章） |
| YOLO 物件偵測 | 低於 100 mW，最短 17 ms | 邊緣 AI 推論（第 24、32 章） |
| 深度睡眠 | 45 μW | 閒置時的極低功耗 |

**表 25-2　微型平台上各項任務的功耗表現**

系統維持在低於 100 毫瓦的功耗範圍——尤其是對運算密集的任務而言——對延長飛行時間與支援複雜的即時任務負載至關重要。

值得注意的是：定位（23 mW）遠比建圖（87.9 mW）省電。這印證了第 14 章案例 14-3 的觀察——首次建圖與後續定位是不同的任務。對微型機器人而言，這意味著一個實用策略：先建好地圖（可以慢一點、耗電一點），之後的任務只需低功耗的定位即可。

### 量一量：這麼稀疏的量測，還能定位嗎

25.4 節說 ToF 矩陣「解析度與量測範圍遠低於真正的光達，因此演算法必須能在稀疏的量測下運作」。這句話留下兩個沒回答的問題：稀疏到什麼程度還能用？以及如果要取捨，該保射線數還是保量程？本小節把兩者分開來量。

實驗設定是一間 10×7 公尺、含六件家具的室內場景。機器人沿橢圓軌跡移動，每個位置用似然場掃描配準估計自己的位姿（第 17 章的方法），與真實位姿比較。兩種感測器配置：4 顆 ToF 矩陣（各 45 度視野、合計約 180 度），以及一台旋轉式光達（360 度、12 公尺）。

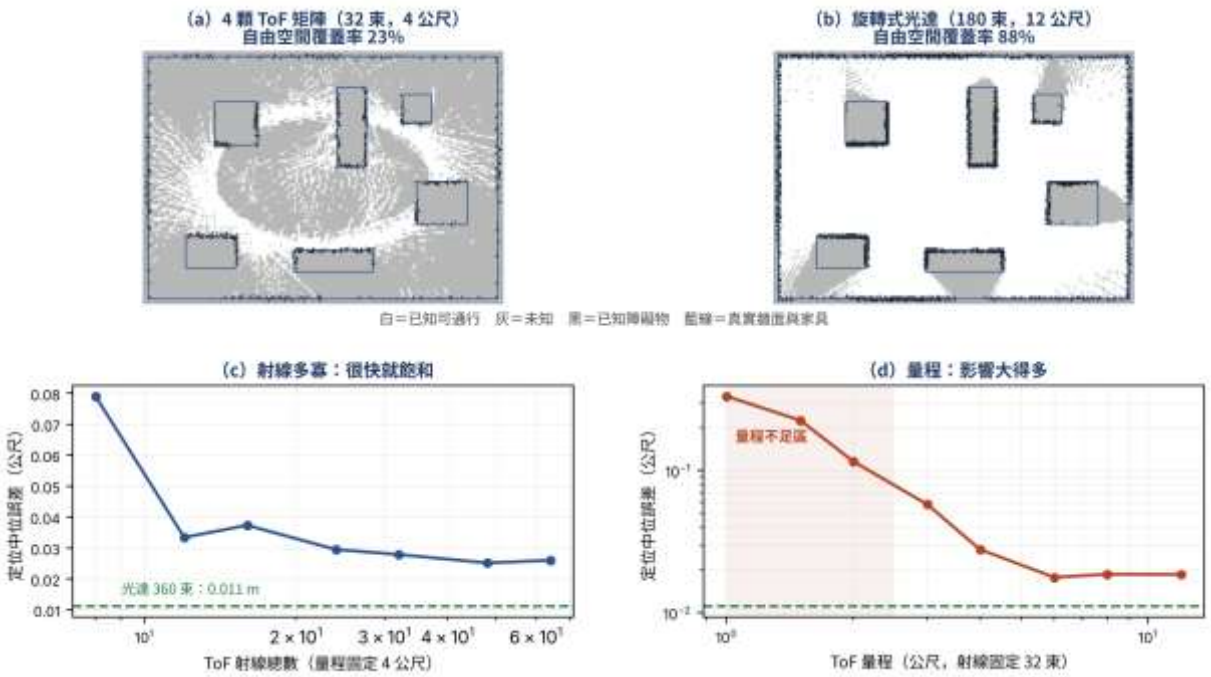



**圖 25-3　稀疏 ToF 與旋轉式光達的建圖與定位**

（a）與（b）是兩者建出來的地圖。光達把房間掃得乾乾淨淨，自由空間覆蓋率 88%，六件家具的輪廓都清楚。ToF 的地圖則明顯殘缺：覆蓋率只有 23%，白色的可通行區域呈放射狀，大片區域仍是未知的灰色。原因有兩個——4 顆感測器只涵蓋約一半的方位，而且量程只有 4 公尺，掃不到遠處的牆。

（c）回答第一個問題：射線數。只有 8 束時定位誤差是 0.079 公尺，增加到 12 束就降到 0.033 公尺，之後幾乎走平——32 束是 0.028、64 束是 0.026 公尺。也就是說，射線從 12 束加到 64 束（五倍以上），精度只改善兩成。這是一個相當重要的結論：對定位而言，十幾束量測就已經接近飽和。

（d）回答第二個問題：量程，而答案完全不同。量程 1 公尺時定位誤差是 0.334 公尺（根本不能用），2 公尺時 0.116、4 公尺時 0.028、6 公尺以上才走平在 0.018 公尺。從 1 公尺到 4 公尺，誤差改善了十二倍。把（c）與（d）並排看，結論很清楚：在微型平台上挑 ToF 感測器時，量程比區段數重要得多。

最後把 ToF 與光達放在一起：4×8 的 ToF 配置在 4 公尺量程下定位誤差是 0.028 公尺，光達是 0.011 公尺——只差 2.5 倍，而射線數只有十一分之一。這正是毫瓦級 SLAM 得以成立

的關鍵：定位這件事，對量測密度的需求遠低於直覺。

**定位夠用，不代表地圖夠用**

這張圖裡有一個必須講清楚的落差：ToF 的定位精度只比光達差 2.5 倍，但地圖覆蓋率只有光達的四分之一（23% 對 88%）。

原因是兩件事需要的量測量不同。定位是把當前這一幀對上已知的結構，少數幾個正確的距離量測就能把六個自由度釘住；建圖卻要把每一塊空間都實際掃過一遍，沒有捷徑。

這也解釋了真實的毫瓦級系統為什麼多半只建「夠用的粗地圖」——不是演算法不行，而是感測器的總量測量就那麼多。同學在規劃微型平台的專題時，應該把目標訂成「能回到起點、能避開障礙」，而不是「建出和光達一樣漂亮的地圖」。

## 25.5 微型無人機、Swarm Robots、AR 眼鏡應用

毫瓦級 SLAM 開啟了哪些原本不可能的應用？本節說明三個代表性方向。

### 微型無人機：安全與可及性

微型無人機能放進手掌，適合室內環境，且與人互動時很安全；然而它們常因缺乏足夠的感測與運算硬體，而無法在複雜任務中提供所需的效能。

「安全」這一點特別值得強調。一台數公斤的標準無人機若失控撞到人，可能造成嚴重傷害；而 44 公克的微型無人機幾乎不具危險性。這使得它們能在教室、辦公室、醫院等有人的空間中運作——這是大型無人機做不到的。

- 室內探索與測繪：在人無法或不宜進入的空間（狹窄管道、災後建築、天花板夾層）進行探索與建圖。
- 農業巡檢：已有研究以多台自主微型無人機視覺偵測害蟲，再由單一較慢的重型載具前往偵測到的爆發點施藥；為因應微型無人機的極端限制（如低解析度感測器與低於

100 毫瓦的運算功耗預算），團隊設計並最佳化了一個微型的影像式卷積神經網路用於害蟲偵測。

- 教育與研究：微型無人機價格相對親民、安全性高，是極佳的教學平台（25.7 節）。

### 群體機器人（Swarm Robots）

當單台機器人的成本與體積都降低，「用很多台」就成為可行的策略。群體機器人的價值在於：

- 覆蓋效率：多台分頭探索，完成大範圍任務的時間大幅縮短（第 24 章案例 24-3 的多機器人協作探索）。
- 容錯性：單台故障不影響整體任務——這是單一昂貴機器人無法比擬的優勢。
- 分散式感知：從多個視角同時觀測，能建立更完整的環境模型。

但群體機器人有一個關鍵前提：每台機器人必須能自主定位。若全部依賴外部設施或中央電腦，通訊頻寬會隨機器人數量成長而崩潰（25.1 節提過）。機載 SLAM 因此是群體機器人可規模化的必要條件——這正是 NanoSLAM 這類技術的戰略價值。第 27 章會深入多機器人 SLAM 的協作議題。

### AR 眼鏡與穿戴裝置

第 20、23 章談過 AR 需要視覺慣性 SLAM，但 AR 眼鏡的限制比手機更嚴苛：必須輕到能長時間配戴、電池必須小、且不能發熱燙臉。

這與微型無人機的限制驚人地相似——都是「極小的功耗預算下要跑 SLAM」。因此本章的技術路線（專用低功耗晶片、平行超低功耗運算、輕量化演算法）也直接適用於穿戴裝置。

這個跨領域的共通性，是本章一個重要的觀念：低功耗 SLAM 不只是微型無人機的專門技術，它是所有「受限平台」的共同需求。掌握這條路線，同學就掌握了通往 AR 眼鏡、智

慧手錶、微型感測節點等眾多應用的鑰匙。

## 量一量：運算的代價，真的是功耗嗎

25.1 節說微型機器人的困難在於「運算、功耗與重量三重限制」。三者當中哪一個最致命？直覺上會說功耗，但算過之後會發現答案是重量——而且差距很大。

計算基礎是一台 27 公克等級的微型無人機：電池約 0.89 瓦時、懸停功耗約 7.6 瓦、純懸停續航約 7.0 分鐘。依動量理論，懸停功耗大致與質量的 1.5 次方成正比，所以多掛一塊板子不只是多耗那塊板子的電，還會讓四個馬達全部更費力。

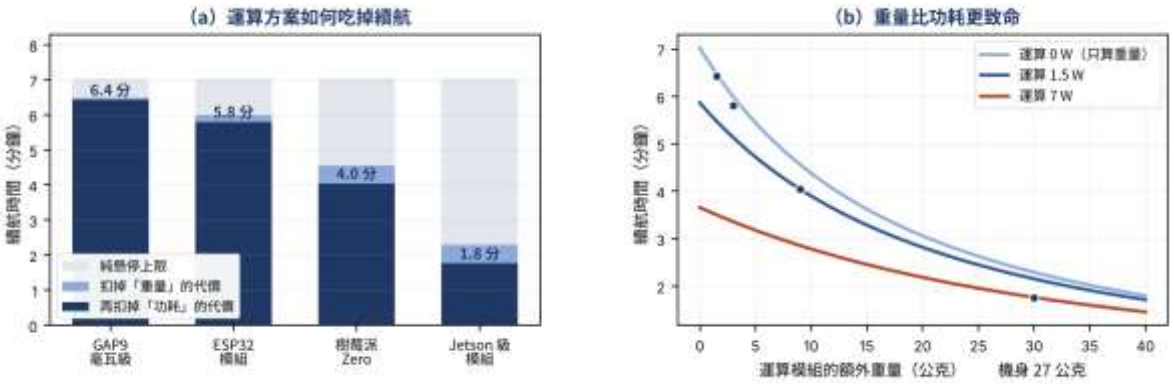



**圖 25-4　運算方案的重量與功耗如何吃掉續航**

（a）把四種運算方案的續航拆成兩段損失。以 GAP9 為例：它只有 1.5 公克、66 毫瓦，重量讓續航從 7.03 分降到 6.48 分，功耗再降到 6.43 分——總共只損失 8.5%。

Jetson 等級的模組則是另一個極端：30 公克、7 瓦。重量把續航從 7.03 分壓到 2.29 分，功耗再壓到 1.76 分。請注意這兩段的比例——重量造成 4.74 分的損失，功耗只造成 0.53 分，前者是後者的 9 倍。四種方案的這個比例分別是 10.6、5.3、4.7 與 9.0 倍，沒有一個例外。

（b）把重量單獨拉出來看。即使運算功耗是 0 瓦（想像一塊不耗電的板子），光是掛上 30 公克就會讓續航從 7.0 分掉到 2.3 分。三條曲線之間的垂直距離（也就是功耗的影響）遠小於曲線本身的下降幅度。

這個結論對專題選型有直接的意義。同學在為微型平台挑運算模組時，第一個該問的不是「它耗幾瓦」，而是「它多重」。一塊省電但笨重的板子，可能比一塊耗電但輕巧的板子更傷續航。這也是 25.2 節為什麼要強調「毫瓦級」與「數公克」必須同時成立——兩者缺一，微型 SLAM 就不成立。

> **這條規則在什麼時候會反過來**
>
> 重量優先的結論，前提是「懸停功耗遠大於運算功耗」。對 27 公克的無人機而言，7.6 瓦對 0.066 瓦，相差一百倍以上，所以重量完全主導。
>
> 但換成地面機器人就不一樣了。一台 5 公斤的輪型機器人，移動功耗可能只有 10 瓦，而多掛 100 公克幾乎不影響能耗——此時運算功耗反而成為主要考量。
>
> 所以這不是一條普遍規則，而是一條「飛行平台的規則」。同學在套用前，先算一下自己平台的「移動功耗對運算功耗」比值，以及「額外重量對自身重量」比值。

## 25.6 Edge AI 與 TinyML 對 SLAM 的影響

本節把視野從 SLAM 擴大到整個邊緣智慧的趨勢，並說明它與本書後續章節的連結。

### 邊緣 AI 的意義

邊緣 AI 指的是在裝置端而非雲端執行 AI 推論。它的價值與 25.1 節討論機載 SLAM 的理由完全一致：不依賴網路、低延遲、隱私保護、以及可規模化。

TinyML 則是邊緣 AI 的極端版本——在微控制器等記憶體只有數百 KB 的硬體上執行機器學習。這聽起來不可思議，但透過模型量化、剪枝、知識蒸餾等技術，已經能在極小的硬體上執行實用的推論。

### 與 SLAM 的三種結合

1. 感知增強：在微型平台上執行物件偵測。相關評測顯示，推論任務如 YOLO 物件偵測能在低於 100 毫瓦下完成，最短推論時間為 17 毫秒、約 1.59 毫焦耳。這讓微型機器人也能做語意感知（第 24 章）。
2. 學習取代運算：用小型神經網路直接估計某些量（如光流、深度、甚至位姿），取代傳統的幾何運算。若模型夠小，推論可能比傳統演算法更省電。
3. 線上學習：更前沿的方向是讓機器人在飛行中自我學習調適。已有研究探討在極低功耗預算下於微型無人機上進行裝置端自監督學習——這代表機器人不只是執行預先訓練好的模型，還能持續適應新環境。

### 硬體與演算法的共同設計

本章最重要的方法論啟示是：在極限受限的條件下，硬體與演算法不能分開設計。

NanoSLAM 之所以成功，不是因為演算法特別聰明，也不是因為晶片特別快，而是因為兩者互相配合：演算法為 RISC-V 多核心的平行架構量身打造，硬體則提供了演算法所需的能效與整合度。這種「硬體軟體共同設計」（Hardware-Software Co-design）的思維，是邊緣運算領域的核心能力。

對智慧機器人學程的同學而言，這是一個重要的提醒：除了學會使用現成的演算法，理解底層硬體的特性（平行度、記憶體階層、功耗行為）同樣重要。這兩者的交會處，正是台灣半導體優勢與機器人應用能夠結合的地方。

### 與後續章節的連結

- 第 26 章：微型無人機的視覺慣性導航，延續本章的平台限制討論。
- 第 27 章：多機器人與雲端邊緣協作，呼應 25.5 節的群體機器人。
- 第 32–34 章：語意 SLAM、模擬訓練與邊緣 AI 部署（Jetson、NPU，第 33 章），以及 VLM 語言導航（第 34 章），是本章邊緣運算討論的延伸。

## 25.7 高中專題：微型無人機地圖建構模擬

本章的技術聽起來遙不可及，但它其實非常適合作為高中專題——因為核心觀念可以用模擬完整體驗，不需昂貴硬體。

### 為什麼適合高中

- 觀念直觀：「用四個方向的距離感測器建地圖」比視覺 SLAM 的特徵與幾何容易理解得多。
- 可完全模擬：用 Python 就能寫出一個微型無人機的 2D 模擬，不需飛行器。
- 連結有感：微型無人機、群體機器人、AR 眼鏡都是學生感興趣的主題。
- 層次分明：從最簡單的「感測與畫圖」，到「加入漂移」，再到「閉環修正」，難度可調。

### 專題設計：四個階段

1. 階段一——建立模擬環境：用 Python 畫出一個 2D 房間（牆與障礙），放入一台虛擬的微型無人機，實作四個方向的距離感測（從無人機位置往四個方向找到最近的牆）。
2. 階段二——理想建圖：讓無人機沿預定路線飛行，每一步用四個方向的距離量測，把偵測到的牆點畫在地圖上。此階段假設位置完全準確，學生會看到一張漂亮的房間地圖逐漸浮現。
3. 階段三——加入漂移：在無人機的位置估計中加入小小的隨機誤差（模擬里程計漂移，第 1、7 章），重新建圖。學生會親眼看到地圖開始扭曲、牆線重影——這是第 13.7 節地圖病徵的具體重現。
4. 階段四——閉環修正：讓無人機繞一圈回到起點，加入「終點應該等於起點」的約束，用簡化的方式把誤差均攤回整條軌跡（第 12 章實作 12-1 的紙上方法可直接套用），觀察地圖被「拉正」的效果。

### 延伸方向

- 功耗預算計算：給定電池容量與各模組的功耗（參考表 25-2），計算飛行時間，並實驗「若把建圖頻率降低一半，能飛多久？」——這是很好的工程權衡練習。
- 群體模擬：加入多台虛擬無人機分頭探索，比較完成全區覆蓋所需的時間（呼應 25.5 節與第 27 章）。
- 感測器數量實驗：比較用 4 個、8 個、16 個方向的距離感測，對地圖品質的影響——這對應真實系統中「感測器數量與功耗、重量」的取捨。

### 教學建議

這個專題的教學價值，在於它讓學生在最簡化的情境中，完整走過 SLAM 的核心邏輯：感測 → 建圖 → 發現漂移 → 閉環修正。這正是本書第一篇到第四篇的縮影。

建議教師先讓學生做出階段二的「完美地圖」建立成就感，再刻意加入漂移製造「破壞」——那個「原本漂亮的地圖忽然歪掉」的瞬間，會讓學生真切理解為什麼 SLAM 是一個困難的問題。這種先建立、再打破、最後修復的教學節奏，比直接講解理論有效得多。

至此，同學已經看到 SLAM 在極限受限平台上的實現。下一章我們聚焦無人機本身的導航挑戰：GPS 拒止環境如何飛行？高速運動與震動如何處理？視覺慣性里程計在無人機上如何運作？第 26 章將把第七篇的討論帶向飛行的核心。

## 25.8 毫瓦級之後：2025–2026 年的三個進展

NanoSLAM 證明了毫瓦級 SLAM 可行。這幾年的進展主要在三個方向，每一個都值得同學知道它的代價。

### 一、把完整的視覺慣性里程計搬上 RISC-V

2026 年的一項工作把完整的單目視覺慣性管線部署到 GAP9 上：ORB 特徵、八點法與 EPnP 搭配 RANSAC、IMU 預積分、以及滑動視窗的位姿圖優化——也就是第 21、23 章的那一整套，塞進 370 MHz 的晶片。

成績是每秒 20 幀、功耗低於 100 毫瓦、記憶體用量不到 1 MB（第一層約 110 KB、第二層約 850 KB）。但代價很明確：它省掉了閉環偵測，在標準的 EuRoC 資料集上平均軌跡誤差約 3.46 公尺，而有閉環的 ORB-SLAM3 約 0.25 公尺——差了一個數量級。作者自己把原因歸給缺少全域修正。

這個對照值得與第 22 章圖 22-2 一起讀。那張圖證明了閉環與共視邊是誤差重新分配的唯一途徑；而微型平台之所以放棄它們，不是因為不重要，而是因為它們需要保留整張地圖與整段軌跡——那正是 1 MB 記憶體裡放不下的東西。

## 二、感測器往更稀疏、更便宜走

第二個方向是把感測器再簡化。圖 25-3(c) 的結論——十幾束量測就接近飽和——正是這個方向的理論基礎。近年出現的做法包括：用單顆多區段 ToF 搭配旋轉機構取代多顆固定感測器、以及用極低解析度的灰階相機（例如 160×160）做光流與場所辨識。

這類設計的共同邏輯是：與其追求單次量測的品質，不如增加量測的次數與多樣性，再用第 11 章的機率方法把它們融合起來。這與第八、九章「沒有全能的感測器，只有恰當的組合」是同一個信念，只是在毫瓦級的預算下被推到了極致。

## 三、運算往「該算才算」走

第三個方向不在演算法也不在硬體，而在排程。25.3 節提過 GAP9 的閒置功耗可以降到 45 微瓦，這讓「事件驅動」的運算變得划算：平穩飛行時降頻甚至休眠，只在轉彎、遇到障礙或偵測到定位不確定度上升時才全速運轉。

這個思想與第 19.6 節的地圖維護、第 33.8 節的錯開執行是一脈相承的——都是在問「這一步真的需要現在做嗎」。對續航只有七分鐘的平台而言，省下一半的運算時間，就等於多飛將近半分鐘。

| | 本章的 NanoSLAM | RISC-V 上的完整 VIO | 事件驅動排程 |
|---|---|---|---|

| | | | |
|---|---|---|---|
| 感測器 | 4 顆 ToF 矩陣 | 單目相機 + IMU | 不限 |
| 有無閉環 | 有（輕量） | 無 | 視配置 |
| 記憶體 | 數百 KB | 不到 1 MB | 不變 |
| 功耗 | 毫瓦級 | 低於 100 毫瓦 | 平均可再降 |
| 精度 | 公分級（小場域） | EuRoC 約 3.46 公尺 | 不變 |
| 主要代價 | 地圖稀疏 | 無全域修正 | 實作複雜度 |

**表 25-3　三個方向的比較**

## 公式或流程

### 一、功耗與飛行時間的關係

微型無人機的飛行時間，由電池能量與總功耗決定：

$$\text{飛行時間} = \text{電池能量} \;/\; (\text{飛行功耗} + \text{運算功耗} + \text{感測功耗})$$

以 Crazyflie 2.1 為例，搭配 380 mAh 電池時單次飛行可達約 7 分鐘。假設飛行本身的功耗遠大於運算，則：

$$\text{若運算功耗佔總功耗比例為 } r\text{，則飛行時間下降約 } r$$

這條簡單的關係式解釋了為什麼「毫瓦」如此關鍵。若 SLAM 耗電 5 瓦，對一台總功耗約 10 瓦的微型無人機而言，飛行時間會直接砍半；而 87.9 毫瓦僅佔約 1%，幾乎不影響續航。這就是 NanoSLAM 把功耗壓到毫瓦級的工程意義。

### 二、平行加速的效益上限

把演算法平行化到 N 個核心上，理論加速比受限於「無法平行的部分」：

$$\text{加速比 } S(N) = 1 \;/\; ((1 - p) + p \;/\; N)$$

其中 p 為可平行化的比例。這稱為阿姆達爾定律（Amdahl's Law）。例如 p = 0.9、N

= 8 時，加速比約為 4.7 倍，而非 8 倍。

這條式子對 NanoSLAM 的設計有直接意涵：ICP 的最近點搜尋高度可平行（每個點獨立搜尋），因此 p 很高、平行效益顯著；而圖優化中的稀疏矩陣分解則較難完全平行。理解哪些部分值得平行化，是善用 PULP 架構的關鍵。

## 三、能效指標

比較不同平台的運算能力時，單看速度不夠，要看「每焦耳能做多少運算」：

*能效 = 運算量 ／ 能量消耗　（常用單位：GOPS／W 或 Ops／J）*

這正是 PULP 典範追求的目標（25.3 節）。以本章的資料為例，YOLO 物件偵測最短推論時間 17 毫秒、約 1.59 毫焦耳——用「每次推論消耗多少能量」來衡量，遠比「每秒能跑幾次」更能反映受限平台的實際可用性。

## 四、低功耗 SLAM 的設計決策流程

**圖 25-5 受限平台的 SLAM 設計決策流程**

① 功耗預算是多少？（電池容量 ÷ 目標運行時間 - 移動所需功耗）

↓

② 感測器選擇：在預算內能負擔什麼？

- 數瓦 → 3D 光達 / 深度相機
- 數百毫瓦 → 2D 光達 / 單目相機
- 數十毫瓦 → ToF 矩陣感測器（本章方案）

↓

③ 演算法選擇：資料量與運算量能否負擔？

- 稠密方法 → 需要大量記憶體與運算，受限平台不可行
- 稀疏 / 特徵法 → 大幅減量（第 18、20 章的共同思想）

↓

④ 平行化評估：哪些部分可平行？（依阿姆達爾定律估算效益）

↓

⑤ 電源管理：動態調頻、深度睡眠、依任務切換模式

↓

⑥ 分工設計：飛控與運算是否分離到不同晶片？通訊如何協調？

## 案例

### 案例 25-1 一顆 LED 的電力：重新理解「運算成本」

本章最值得同學停下來思考的，是那個 87.9 毫瓦的數字。

讓我們做幾個對照：一顆手機 LED 手電筒約消耗數百毫瓦；一台筆記型電腦執行 ORB-

SLAM3 時，處理器功耗可能達到數十瓦——是 NanoSLAM 的數百倍。而 NanoSLAM 在這樣的預算下，達成了 4.5 公分的建圖精度，與更大、運算能力更強的無人機相當。

這個對比顛覆了一個常見的直覺：我們往往認為「更好的結果需要更多的運算」。但 NanoSLAM 證明，透過演算法與硬體的共同設計，可以在功耗降低兩三個數量級的同時，維持相當的精度。

這給同學兩個啟示。第一，效能瓶頸往往不在「算力不夠」，而在「用錯方法」。第 18 章 LOAM「不需要用上所有的點」、第 20 章 ORB「以部分精度換速度」、本章「以平行換能效」——都是同一種思維：找到問題的本質，只做必要的計算。

第二，在資源受限的條件下創新，往往比在資源充裕時更能激發真正的工程智慧。這對台灣的機器人與半導體產業特別有啟發意義：我們或許無法在算力上與大廠競爭，但在「用更少資源做更多事」這條路上，有充分的發揮空間。

## 案例 25-2　誰負責飛、誰負責想：雙晶片的分工哲學

NanoSLAM 系統中一個容易被忽略、卻極具工程智慧的設計，是它的雙晶片架構：原廠的 STM32 負責飛控與狀態估計，新增的 GAP9 負責 SLAM 運算，兩者以專門設計的通訊協定協調——包括如何儲存圖結構、交換圖的位姿、新增邊，以及執行圖優化。

為什麼不用一顆晶片全包？原因有三：

- 即時性要求不同：飛控迴路必須以極高頻率、極低延遲執行，任何延遲都可能導致墜機；SLAM 則可以慢一點（250 毫秒也能接受）。把兩者放在同一顆晶片上，SLAM 的運算尖峰可能拖累飛控。
- 可靠度隔離：即使 SLAM 當掉，飛控仍能維持飛行安全——這是安全設計的基本原則。
- 各取所長：STM32 適合即時控制，GAP9 適合平行運算。用對的晶片做對的事，整體能效最佳。

同學應該注意到：這個「快速反應層 + 慢速思考層」的分工，與第 3 章的混合式架構、第 15 章 Cartographer 的局部與全域子系統、第 18 章 LOAM 的高頻里程計與低頻建圖，是完全相同的設計模式。

這說明了一件事：好的系統設計原則具有跨層次的普遍性。無論是在演算法層、軟體架構層還是硬體層，「依時間尺度分層、各層各司其職」都是有效的策略。掌握這種模式思維，比記住任何單一系統的細節都更有價值。

## 案例 25-3　微型無人機的田野應用：從實驗室到農田

微型無人機的價值，在一個具體的農業應用中展現得淋漓盡致。

有研究團隊提出這樣的構想：高效的作物生產需要及早偵測害蟲爆發並及時處理，他們考慮以一支由多台自主微型無人機組成的機隊視覺偵測害蟲，再由單一台較慢的重型載具前往偵測到的爆發點施藥。

這個「輕騎兵偵察、重裝部隊處理」的分工設計極為聰明：微型無人機便宜、安全、可大量部署，適合大範圍快速搜尋；重型載具雖然慢，但能攜帶藥劑執行實際處理。兩者結合，效率遠高於單獨使用任一種。

技術上的挑戰同樣值得注意：為因應微型無人機的極端限制（如低解析度感測器與低於 100 毫瓦的運算功耗預算），團隊設計、微調並最佳化了一個微型的影像式卷積神經網路用於害蟲偵測。這個網路每次推論僅需 0.58 GOps，在他們的資料集上偵測有害昆蟲的平均精度均值達 0.79——相較文獻中表現最佳的網路，平均精度低了 14%，但運算量少了 32 倍。

這個「精度降低 14%、運算量減少 32 倍」的取捨，正是受限平台工程的縮影。這個案例也提示了一個在地化的專題方向：台灣的農業場域（果園、稻田、溫室）同樣面臨病蟲害偵測的需求，而微型無人機的安全性與低成本，使它比大型無人機更適合在密集種植的環境中作業。

# 實作活動

## 實作 25-1 微型無人機地圖建構模擬

目的：以純模擬的方式完整體驗低功耗 SLAM 的核心邏輯——從稀疏感測建圖、觀察漂移、到閉環修正，並進行功耗與感測器數量的工程權衡分析。本實作只需 Python，適合高中到大學部各層次。

### 第一部分：建立模擬環境與理想建圖（全體）

128. 用 Python 定義一個 2D 房間：以線段表示牆與障礙物，並在其中放置一台虛擬微型無人機（以位置與朝向表示）。

129. 實作四方向距離感測：從無人機位置沿前、後、左、右四個方向射出射線，計算與最近牆面的交點距離。這模擬了 25.4 節的 4 顆 ToF 矩陣感測器。

130. 讓無人機沿一條預定路線（例如繞房間一圈）移動，每一步記錄四個距離值，並依第 10 章的座標轉換把偵測到的牆點畫在地圖上。

131. 用 matplotlib 顯示逐步成長的地圖，觀察房間輪廓如何浮現。此階段假設位置完全準確，地圖應該相當漂亮。

### 第二部分：漂移與閉環（全體）

1. 加入里程計漂移：在無人機每一步的位置與角度估計中加入小的隨機誤差（例如位置誤差標準差 2 公分、角度誤差 0.5 度），重新建圖。

2. 觀察病徵：地圖出現什麼問題？牆線是否重影？繞完一圈後，終點與起點是否對不上？量測這個落差有多大。對照第 13.7 節的三種地圖病徵。

3. 實作簡化的閉環修正：當無人機回到起點附近時，加入「終點應等於起點」的約束，把累積誤差平均分攤到整條軌跡的每一步（第 12 章實作 12-1 的方法），重新繪製地圖。

4. 量化改善：比較修正前後的「終點與起點落差」以及地圖的目視品質，計算誤差降低的百分比，與 NanoSLAM 論文的 67% 對照。

**第三部分：工程權衡分析（大學部，本章特色）**

1. 感測器數量實驗：把四方向感測改為 8、16、32 個方向，比較地圖品質（牆面完整度、細節解析度）。找出「品質提升趨緩」的轉折點。

2. 功耗估算：假設每個 ToF 感測器耗電 20 毫瓦、每次掃描匹配耗電依感測點數線性成長，計算不同感測器數量下的總功耗。

3. 飛行時間試算：依公式一，假設電池能量固定、飛行本身耗電 5 瓦，計算各種配置下的飛行時間，畫出「感測器數量 vs 地圖品質 vs 飛行時間」的三方權衡圖。

4. 決策練習：若任務要求「至少飛行 5 分鐘且地圖誤差小於 10 公分」，你會選擇哪一種配置？說明理由。

**第四部分：延伸探索（進階）**

1. 建圖頻率實驗：把 SLAM 執行頻率從每步一次改為每三步、每五步一次，觀察地圖品質下降多少、功耗節省多少。這對應真實系統的動態調頻策略（25.3 節）。

2. 群體模擬：加入第二台、第三台虛擬無人機分頭探索，各自建圖後合併，比較「協作探索」與「單機探索」完成全區覆蓋所需的步數。這呼應 25.5 節與第 27 章。

3. 定位模式：先用一台無人機建好地圖，再讓第二台在已知地圖中只做定位（不建圖），比較兩者的運算量。對照表 25-2 中「定位 23 mW、建圖 87.9 mW」的差異。

**討論問題**

44. 加入漂移後，地圖出現了哪些病徵？繞一圈的終點落差有多大？這與第 13.7 節的描述是否吻合？

45. 閉環修正把誤差「均攤」到整條軌跡，而不是集中修正最後一步。為什麼均攤比較好？（提示：第 12 章案例）

46. 感測器從 4 個增加到 32 個，地圖品質提升了多少？功耗增加多少？你認為最合適的數量是多少？為什麼？

47. 若你是微型無人機的系統設計者，在「更好的地圖」與「更長的飛行時間」之間，你會如何取捨？這個取捨是否依任務而異？

48. （大學部）依阿姆達爾定律（公式二），若 ICP 有 95% 可平行化，用 GAP9 的 9 個核心能加速多少倍？這與理想的 9 倍差多少？差距從何而來？

**延伸挑戰（大學部）**

其一：實作一個簡化的位姿圖優化——把模擬中每一步的位姿當作節點、相鄰步之間的運動當作邊、閉環當作額外的邊，用 scipy 的最小平方求解器最佳化整條軌跡（第 12 章實作 12-1 的程式版本），比較它與「平均分攤」的簡易方法在精度上的差異。其二：若學校有 Crazyflie 或類似的微型無人機平台，嘗試讀取其感測器資料並在電腦端執行你的 SLAM 程式，體驗真實資料與模擬的差異——真實的感測雜訊、時間延遲與飛行擾動，會讓你對本章的工程挑戰有更深的體會。

## 延伸閱讀

64. V. Niculescu et al., "NanoSLAM: Enabling Fully Onboard SLAM for Tiny Robots," IEEE Internet of Things Journal, 2023(arXiv:2309.12008).（本章主要依據，程式碼公開於 GitHub: ETH-PBL/NanoSLAM）

65. H. Müller et al., "GAP9Shield: A 150GOPS AI-capable Ultra-low Power Module for Vision and Ranging Applications on Nano-drones," arXiv:2407.13706, 2024.（微型無人機的感測與運算模組，含功耗實測數據）

66. D. Rossi et al., PULP 平台相關論文與 pulp-platform.org 官方網站.（平行超低功耗運算典範的技術來源）

67. Bitcraze Crazyflie 官方文件（bitcraze.io）.（本章實驗平台的硬體規格、開發環境與社群資源）

68. GreenWaves Technologies GAP9 產品文件.（RISC-V 低功耗處理器的規格與開發工具）

69. P. Warden and D. Situnayake, TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers, O'Reilly, 2019.（TinyML 的入門經典）

70. L. Crupi et al., "Training on the Fly: On-device Self-supervised Learning aboard Nano-drones within 20 mW," arXiv:2408.03168, 2024.（微型無人機上的裝置端學習，25.6 節的前沿方向）

## 小結

64. 微型機器人執行 SLAM 面臨運算、記憶體與電力三重限制；傳統做法是把運算卸載到外部設施，但這帶來依賴基礎設施、通訊延遲、頻寬功耗與可靠度的問題，使機器人無法真正自主。

65. NanoSLAM 是輕量化且最佳化的端到端 SLAM 方法，專為在僅 87.9 毫瓦的功耗預算下於公分尺寸機器人上運作而設計；它被部署在重 44 公克、搭載 RISC-V 低功耗平行處理器 GAP9 的微型無人機上，達成 4.5 公分建圖精度與小於 250 毫秒的端到端執行時間。

66. GAP9 建立在平行超低功耗（PULP）運算典範上，以多核心平行換取能效，功耗低於 180 毫瓦；它支援動態調整時脈與電壓、自動時脈閘控，閒置功耗可降至 45 微瓦。RISC-V 的開放性使其可客製化且無授權成本。

67. NanoSLAM 以 4 顆 ToF 矩陣感測器取代光達、以平行化 ICP 做掃描匹配（機載 55 毫秒）、以位姿圖優化修正漂移（軌跡誤差最多降低 67%）；三個輕量化策略為平行化而非簡化、稀疏而非稠密、分工而非全包。

68. 微型無人機能放進手掌、適合室內且與人互動安全；機載 SLAM 是群體機器人可規模化的必要條件；而 AR 眼鏡的限制與微型無人機驚人地相似，使低功耗 SLAM 成為所有受限平台的共同需求。

69. 邊緣 AI 與 TinyML 與 SLAM 有三種結合：感知增強（YOLO 可在低於 100 毫瓦、最短 17 毫秒完成推論）、以學習取代運算、以及線上自我學習；核心方法論是硬體與演算法的共同設計。

70. 微型無人機地圖建構模擬極適合高中專題：觀念直觀、可完全模擬、層次分明；建議的教學節奏是先建立完美地圖、再刻意加入漂移製造破壞、最後以閉環修復。

## 承先啟後

本章把 SLAM 壓進毫瓦級的預算，也量出了三組關鍵數字：定位對量測密度的需求遠低於直覺——12 束 ToF 就接近飽和（0.033 公尺），32 束（0.028 公尺）與光達的 0.011 公尺只差 2.5 倍；但量程的影響大得多，從 1 公尺放寬到 4 公尺可以把誤差改善十二倍。在硬體上，固定吞吐量時多核慢跑可以把功耗降到 4%，但過了約 32 核就會被漏電反噬；實際晶片上 GAP9 的能效是 STM32 的 39 倍。而在平台層面，運算模組的重量對續航的影響是功耗的 4.7 至 10.6 倍——挑板子時第一個該問的是它多重，不是它耗幾瓦。

本章展示了 SLAM 在極限受限平台上的實現，也帶出一個貫穿全書的觀念：效能瓶頸往往不在算力不足，而在方法未經最佳化。下一章我們聚焦無人機導航本身的挑戰——GPS 拒止環境如何飛行？高速運動、震動與影像模糊如何處理？光流、視覺慣性里程計與視覺 SLAM 在無人機上如何分工？第 26 章將把第七篇的討論帶向飛行的核心。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

## 第 26 章

## 無人機 SLAM 與視覺慣性導航

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

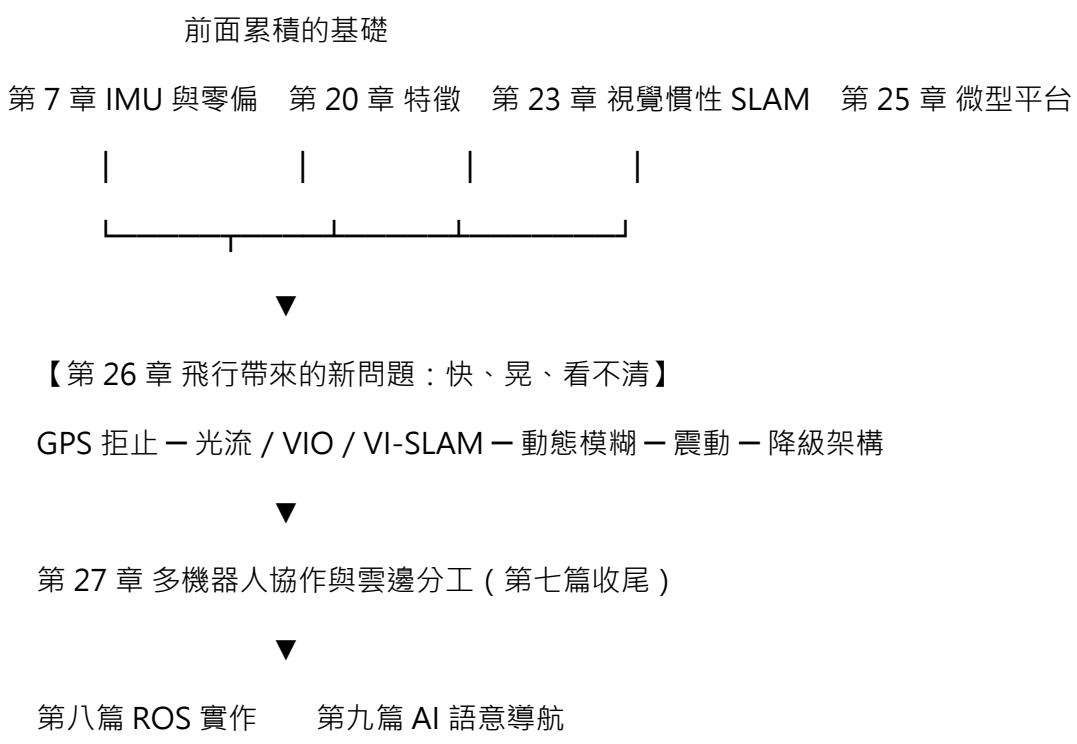
## 本章在全書的位置

第 25 章把 SLAM 壓進了毫瓦級的運算預算。本章換一個維度：速度。

地面機器人的典型速度是每秒 0.5 公尺，而無人機可以是它的十倍甚至五十倍。這個差異不是「同一套方法跑快一點」而已——它會讓第 20 章賴以生存的清晰角點糊成一片、讓第 22 章倚重的中期資料關聯根本來不及建立、也會讓第 7 章談過的 IMU 零偏以一種意想不到的方式被螺旋槳製造出來。

本章的三張圖就在量這些事：圖 26-1 比較光流、VIO 與 VI-SLAM 三層技術的漂移行為；圖 26-2 把 26.4 節的「三重打擊」拆開來量，並找出最有效的那一個對策；圖 26-3 則說明一件反直覺的事——提高 IMU 取樣率救不了震動造成的混疊。

### 圖 26-0　第 26 章的連結地圖




## 學習目標

上一章我們看到 SLAM 如何被壓縮進毫瓦級的預算。本章聚焦無人機導航本身的挑戰：在天空中，機器人失去了地面的支撐、擁有完整的六自由度、速度快得多、而且任何定位失誤都

可能導致墜機。學完本章之後，同學應能夠達成下列目標：

64. 說明無人機定位相對地面機器人的四項特殊困難。
65. 說明 GPS 拒止環境的成因與典型場景，並列舉可用的定位手段。
66. 比較光流、VIO 與視覺 SLAM 三種技術的能力層次與適用場合。
67. 說明高速運動、震動與影像模糊如何影響視覺定位，並提出對策。
68. 規劃一次無人機室內導航實驗的流程與安全措施。
69. 說明 EuRoC 與 TUM-VI 兩大視覺慣性資料集的特性與用途。
70. 說明從微型無人機到視覺導航無人機的技術演進脈絡。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 微型飛行器 | MAV | 小型無人飛行器，是視覺慣性導航研究的主要平台。 |
| GPS 拒止環境 | GPS-denied | 衛星訊號被遮蔽或干擾而無法使用的環境。 |
| 光流 | Optical Flow | 影像中亮度模式隨時間移動的向量場。 |
| 光流定位 | Optical Flow Positioning | 以向下相機的光流估計水平速度以維持懸停。 |
| 視覺慣性里程計 | VIO | 緊耦合相機與 IMU 估計運動，會累積漂移。 |
| 視覺慣性 SLAM | VI-SLAM | VIO 加上建圖與閉環，可消除長期漂移。 |
| 動態模糊 | Motion Blur | 曝光期間相機移動造成的影像模糊。 |
| 特徵軌跡縮短 | Shortened Feature Tracks | 高速下特徵快速離開視野，追蹤時間變短。 |
| 滾動快門 | Rolling Shutter | 感測器逐列曝光，高速時造成影像變形。 |
| 全域快門 | Global Shutter | 全畫面同時曝光，適合高速運動。 |

| 中文名詞 | 英文名詞 | 概念說明 |
| --- | --- | --- |
| 積極機動 | Aggressive Maneuver | 大角度、高加速度的飛行動作。 |
| 光度校正 | Photometric Calibration | 校正相機的亮度響應與暗角，對直接法重要。 |
| 動作捕捉系統 | Motion Capture | 以外部相機提供毫米級真值位姿的實驗設備。 |

## 26.1 無人機定位的特殊困難

第 3.6 節曾比較過五種平台的導航需求，本節深入無人機為何是其中最嚴苛的一種。

### 四項特殊困難

- 完整的六自由度運動：地面機器人在平面上運動，只有三個自由度（第 10.3 節的 x、y、θ）；無人機能在三維空間中任意平移與旋轉。狀態變多、運動更複雜，估測難度直接上升。
- 沒有里程計：這是最容易被忽略卻極為關鍵的一點。輪型機器人有輪速編碼器提供里程計（第 3、13 章），即使不完美也是穩定的運動參考；無人機在空中沒有任何接觸，完全沒有等效的感測器。它只能靠 IMU 積分——而第 7.3 節告訴我們，IMU 單獨使用會迅速漂移。
- 速度快、動態劇烈：無人機的飛行速度遠高於室內地面機器人，而且姿態變化劇烈。這直接導致影像模糊與特徵追蹤困難（26.4 節）。
- 失誤的後果嚴重：輪型機器人定位錯誤最多是撞牆停下；無人機定位錯誤可能直接墜機，造成設備損壞甚至人員傷害。這使得系統的穩健度要求遠高於精度要求。

### 載重與電力的雙重限制

第 25 章已詳述受限平台的問題，對無人機而言更為尖銳：每增加一公克的感測器重量，就要消耗額外的推力與電力；而飛行本身已經消耗了絕大部分電量。這使得無人機的感測器選擇極度受限——這也是第 9.5 節指出「相機加 IMU 是無人機最理想組合」的根本原因。

### 即時性的嚴苛要求

地面機器人的定位若延遲 0.5 秒，它只是慢半拍地停下；無人機的定位若延遲 0.5 秒，在飛行過程中可能已經移動了數公尺，控制迴路會失去穩定。

因此無人機系統對延遲的容忍度極低。這解釋了為什麼第 25 章提到 NanoSLAM 要把端到端執行時間壓在 250 毫秒以內，也解釋了為什麼高階飛控的姿態控制迴路要以數百赫茲

執行——這正是第 7 章 IMU 高頻輸出價值的體現。

## 26.2 GPS-denied 環境導航

第 5 章詳細討論過 GNSS 的原理與限制，本節聚焦無人機在衛星訊號不可用時的處境與對策。

### 典型的 GPS 拒止場景

167. 室內：建材完全遮蔽衛星訊號（第 5.5 節）。工廠、倉庫、商場、體育館的巡檢與盤點任務都屬此類。

168. 橋樑與結構物下方：橋樑檢測是無人機的重要應用，但橋面本身就是遮蔽物。

169. 隧道與地下空間：礦坑、下水道、地下停車場——完全沒有衛星訊號。

170. 都市峽谷：第 5.5 節提過的高樓夾道，不只遮蔽，還有嚴重的多路徑造成錯誤定位——這比完全沒訊號更危險，因為系統可能不知道自己錯了。

171. 森林與植被下：樹冠遮蔽使訊號微弱且不穩定。

172. 人為干擾：第 5.5 節提過的 Jamming 與 Spoofing，對無人機是嚴肅的資安議題。

### 可用的定位手段

在 GPS 拒止環境中，無人機能倚賴的定位手段依可靠度排序如下：

| 手段 | 提供什麼 | 限制 | 本書章節 |
|---|---|---|---|
| IMU | 高頻姿態與運動 | 單獨使用快速漂移 | 第 7 章 |
| 氣壓計 | 相對高度 | 受氣流與溫度影響 | 本章 |
| 光流（向下相機） | 水平速度 | 需地面紋理，不提供絕對位置 | 26.3 節 |
| 測距感測器（向 | 離地高度 | 地面起伏會誤判 | 第 8 章 |

| 手段 | 提供什麼 | 限制 | 本書章節 |
|---|---|---|---|
| 下） | | | |
| VIO（相機 + IMU） | 完整 6-DoF 位姿 | 會漂移，需紋理與光線 | 第 23 章 |
| VI-SLAM | 位姿 + 地圖 + 閉環 | 運算量較大 | 第 23 章 |
| UWB / 視覺標記 | 絕對位置校正 | 需事先部署基礎設施 | 第 4 章 |

**表 26-1　GPS 拒止環境中無人機的定位手段**

### 分層策略

實務系統採用分層設計，把這些手段組合起來：

4. 底層（數百 Hz）：IMU 提供姿態，維持飛行穩定。這一層不能有任何中斷，否則直接失控。
5. 中層（數十 Hz）：光流與測距提供速度與高度，支撐定點懸停。即使沒有地圖，無人機也能「待在原地不飄走」。
6. 上層（10 至 30 Hz）：VIO 或 VI-SLAM 提供完整位姿與地圖，支撐路徑規劃與自主導航。
7. 校正層（不定期）：UWB、視覺標記或回到 GPS 可用區域時，提供絕對位置校正（第 4、5 章的融合思想）。

這個分層的重要性在於容錯：若上層的 VIO 失效，無人機仍能靠中層的光流維持懸停，不至於墜機；若中層也失效，底層的 IMU 至少能維持姿態穩定，讓操作者接手。安全設計的核心原則，就是讓失效逐級降級而非瞬間崩潰。

## 26.3　Optical Flow、VIO、Visual SLAM

本節釐清三種視覺定位技術的能力層次——它們常被混用，但提供的資訊差異很大。

### 光流：最基本的一層

第 9.4 節介紹過光流是「影像中亮度模式隨時間的移動」。在無人機上，它有一個極為實用的應用：向下光流定位。

做法是在機腹裝一顆朝下的相機，持續計算地面紋理在畫面中的移動速度。配合測距感測器提供的離地高度，就能換算出無人機的水平移動速度。

173. 提供什麼：水平速度（不是位置）。

174. 能做什麼：定點懸停。若光流顯示地面在往後移，表示無人機在往前飄，飛控就往後修正。這讓無人機能在沒有 GPS 的室內穩定停在原地。

175. 限制：只給速度，積分後才是位置，因此仍會漂移；需要地面有足夠紋理（在純白地板或水面上會失效）；高度量測不準會直接造成速度換算錯誤。

光流定位是消費級無人機室內懸停的標準配備，運算量極小，是「用最少資源解決最關鍵問題」的好例子。

### VIO：完整的六自由度里程計

VIO 緊耦合相機與 IMU（第 7.6、23.3 節），輸出完整的 6-DoF 位姿與速度。相較光流，它提供的資訊完整得多，能支撐路徑追蹤與自主飛行。

但 VIO 本質上仍是里程計——它會累積漂移（第 20.1 節提過視覺里程計的特性）。飛行時間一長，估測位置就會偏離真實位置。

### 視覺慣性 SLAM：加上地圖與閉環

VI-SLAM 在 VIO 之上加入建圖與閉環偵測（第 23 章），能在重訪同一地點時修正累積漂移。代價是運算量與記憶體需求上升。

| 技術 | 輸出 | 會漂移嗎 | 運算需求 | 典型用途 |
|---|---|---|---|---|
| 光流 | 水平速度 | 積分後會 | 極低 | 定點懸停 |
| VIO | 6-DoF 位姿 + 速度 | 會累積 | 中 | 路徑追蹤、自主飛行 |
| VI-SLAM | 位姿 + 地圖 + 閉環修正 | 閉環可修正 | 較高 | 探索建圖、長時間任務 |

**表 26-2　三種視覺定位技術的能力比較**

### 如何選擇

176. 任務是「懸停拍攝」：光流就夠了，省電省運算。

177. 任務是「飛一段預定航線」：需要 VIO 提供位置追蹤。

178. 任務是「探索未知空間並建圖」：需要 VI-SLAM。

179. 任務是「長時間反覆巡檢同一場域」：VI-SLAM 加上地圖重用（第 23 章 Atlas 的多階段作業）最為合適。

這個層次選擇的思維，呼應了本書反覆強調的原則：不要用超過任務需求的技術，因為每一分運算與電力，在無人機上都是飛行時間的代價。

### 量一量：三層技術各會漂多少

表 26-2 用「會漂移嗎」這一欄區分了三種技術，但沒說會漂多少、以什麼形式漂。本小節用一次完整的飛行模擬把它量出來。

設定是無人機沿一條橢圓航線繞六圈、共 72 秒，平均速度約每秒 2.5 公尺。三種估測各依自己的物理特性建模：光流量到的是機體座標的水平速度，必須乘上測距感測器量到的高度才能換算成真實速度（高度有 4% 誤差），再用陀螺儀積分出的航向轉回世界座標（航向會漂）；VIO 直接輸出相對位姿，誤差逐步累積；VI-SLAM 在 VIO 之上，每繞完一圈就與前一圈的同一位置建立閉環，做位姿圖優化。每種設定各跑五次取中位數。

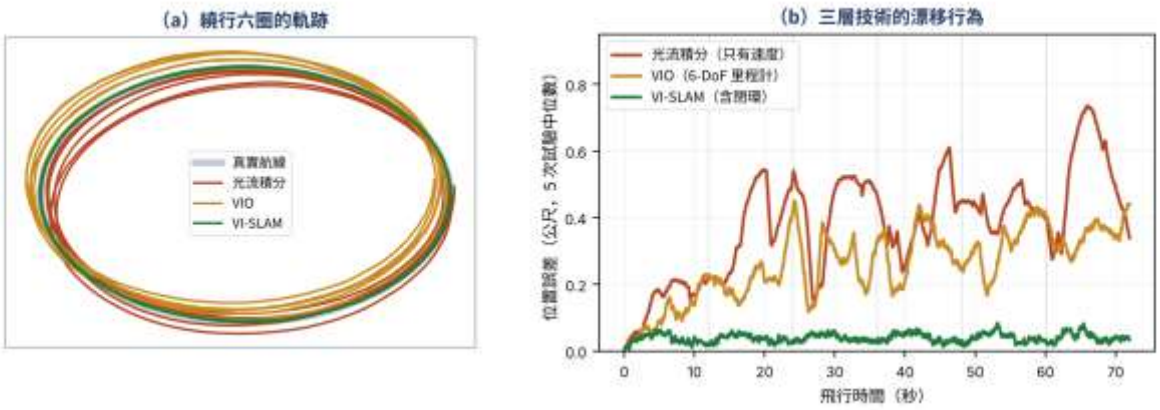



**圖 26-1　光流、VIO 與 VI-SLAM 的漂移行為**

（a）是六圈疊在一起的軌跡。綠色的 VI-SLAM 幾乎與真實航線重合，六圈畫在一起看不出分岔；黃色的 VIO 一圈一圈往外偏；紅色的光流則偏得更明顯，而且偏的方向不固定——這是航向誤差在轉動整個速度向量的後果。

（b）是位置誤差隨時間的變化。後半段（36 秒之後）的平均誤差是：光流 0.452 公尺、VIO 0.324 公尺、VI-SLAM 0.041 公尺；峰值分別是 0.736、0.441 與 0.084 公尺。VI-SLAM 比前兩者好了 8 到 11 倍。

但更值得注意的是三條線的「形狀」而不是高度。綠線從頭到尾是平的——它的誤差不隨飛行時間成長，而是被閉環鎖在一個固定的水準上。紅線與黃線則呈現明顯的鋸齒：每繞一圈誤差先上升、再部分回落。回落不是因為被修正了，而是因為航線是封閉的，繞回起點附近時累積的誤差碰巧部分抵消——這是巧合，不是機制。若航線是單程的，這兩條線只會一路往上。

這張圖也說明了 26.3 節「如何選擇」那幾條建議的根據。懸停時不需要位置、只需要速度，光流的漂移完全不構成問題；但只要任務需要「飛到某個座標」或「飛完就回來」，沒有閉環的方案就會在幾十秒內累積出公尺級的誤差。

> **光流為什麼會比 VIO 還糟**
>
> 表面上看，光流的量測比較單純，誤差應該比較小。但它有兩個 VIO 沒有的弱點。

第一，它只給速度，沒有任何姿態資訊，所以要把速度轉到世界座標必須借用陀螺儀積分出的航向——而航向誤差會直接旋轉整個速度向量，造成的位置誤差與飛行距離成正比，而不是與時間的平方根成正比。

第二，它的速度換算依賴高度量測。本模擬只給了 4% 的高度誤差，就造成全程 4% 的尺度偏差。若無人機飛過一張桌子，測距感測器量到的高度突然變小，光流會誤判成「飛得更慢」——這是消費級無人機室內飄移最常見的原因。

## 26.4 高速運動、震動與影像模糊

本節是無人機視覺定位最核心的技術挑戰。

### 三重打擊

研究指出，現有的 VIO 系統在高速下會因為動態模糊、特徵軌跡縮短與控制迴路延遲增加而效能劣化。這三者恰好構成一個惡性循環：

8. 動態模糊：曝光期間相機移動，影像變糊。在如此的速度下，新的問題會浮現，既有的困難也被加劇——動態模糊變得更為明顯，損害視覺特徵追蹤。模糊的影像中，角點變成一片糊，特徵提取直接失敗（第 20 章的特徵需要清晰的角點）。
9. 特徵軌跡縮短：高速飛行時，同一個特徵點快速掃過視野就消失了。第 23.2 節強調的「中期資料關聯」因此難以建立，系統只能倚賴短期關聯，精度下降。
10. 控制迴路延遲：視覺處理本身需要時間，高速下這個延遲對應的位移更大，控制品質惡化，可能引發震盪。

### 資料集反映的現實

這些挑戰在標準評測資料集上有清楚的呈現。有研究觀察到，由於無人機的高速移動，影像會有些模糊，在困難序列如 MH_04、MH_05、V1_03 與 V2_03 中尤其明顯。

更有趣的是一個細節：EuRoC 的機房序列在開頭都有快速運動以協助 IMU 初始化（第

23.4 節），但由此產生的動態模糊對視覺追蹤是一大挑戰；因此 DSO 的作者裁掉開頭，只使用無人機在空中的部分，但對 VIO 系統而言 IMU 初始化不可或缺，無法捨棄開頭。

這個細節同時說明了兩件事：第一，IMU 初始化需要運動激勵（第 23.4 節反覆強調），但運動激勵本身會造成模糊，兩者互相拉扯；第二，不同方法在同一資料集上的評測條件可能不同，同學閱讀論文比較數據時要留意這類細節。

### 速度的量級

讓同學對「高速」有具體概念：EuRoC 微型飛行器資料集的速度很少超過每秒 5 公尺；競速無人機資料集可達每秒 28 公尺，但通常是室內結構化場地或短程戶外賽道的 RGB 相機。有研究以雙目 RGB 的 VIO 支援到每秒 17.5 公尺，但它依賴雙目視覺並假設有利的光線條件。

對照之下，室內地面機器人的速度通常在每秒 0.5 公尺以下——無人機的速度是它的十倍甚至數十倍。這就是為什麼無人機的視覺定位困難得多。

### 五種對策

11. 提高快門速度：縮短曝光時間直接減少模糊，代價是影像變暗，需要更好的感光元件或更亮的環境。

12. 使用全域快門：滾動快門逐列曝光，高速旋轉時會造成影像歪斜；全域快門全畫面同時曝光，是高速應用的必要選擇。

13. 倚重 IMU：有研究整合以 IMU 為基礎的運動預測模組來改善跨幀的特徵追蹤，特別是在高速運動期間。IMU 能預測特徵「應該移動到哪裡」，大幅縮小搜尋範圍，提升追蹤成功率。這是第 7.6 節「IMU 為視覺提供初值」的具體實現。

14. 影像增強：對模糊或低對比的影像做預處理。有研究結合對比度受限的自適應直方圖等化（CLAHE）、梯度濾波與描述子驗證，以在動態模糊下維持穩定的特徵軌跡。

15. 自適應權重：依影像品質動態調整視覺與慣性的信任比例。有研究在 EuRoC 與 TUM VI 資料集以及實地測試中驗證了自適應共變異數方法，在有動態模糊的情境中顯著改善無人機的導航精度。

### 震動的問題

除了飛行速度，螺旋槳與馬達造成的高頻震動也是無人機特有的問題：

180. 對 IMU：震動會污染加速度計讀值，若頻率接近取樣率還可能造成混疊。實務上需要機械減震（軟膠墊）與濾波處理。

181. 對相機：高頻震動造成的微小模糊會降低特徵品質，通常以減震座與較快的快門緩解。

182. 對安裝：感測器與機身的相對位置（外參，第 10 章）若因震動而鬆動改變，整個系統的精度都會受影響。

### 量一量：三重打擊各打掉多少

26.4 節把高速飛行的困難歸納為動態模糊、特徵軌跡縮短與控制迴路延遲三者。這一節把前兩者分開來量，順便檢驗五種對策裡哪一個最有效。

實驗方法：合成一張帶有多尺度紋理與幾何邊角的地面影像（模擬磚縫、標線、門框），依「速度 × 曝光時間 × 焦距 ÷ 深度」算出對應的像素位移，施以線性運動模糊，再用 ORB 偵測與交叉比對，統計有多少特徵能與清晰影像正確配對。相機參數取無人機常用的廣角配置（焦距 380 像素），場景深度 4 公尺。

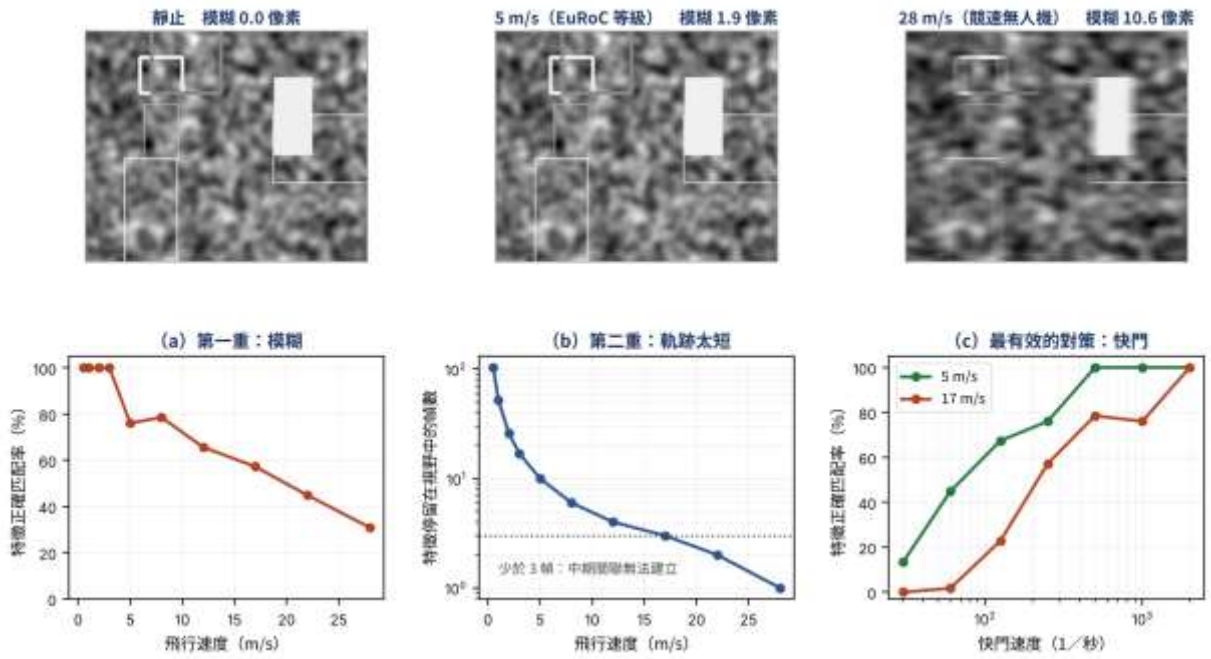



**圖 26-2　速度造成的三重打擊，以及最有效的對策**

上排是同一塊紋理在三種速度下的樣子。靜止時角點清晰；每秒 5 公尺（EuRoC 微型飛行器資料集的典型速度）時模糊 1.9 像素，肉眼已經看得出邊緣變軟；每秒 28 公尺（競速無人機）時模糊 10.6 像素，細紋理幾乎完全消失，只剩大塊的明暗。

（a）量化第一重打擊。每秒 3 公尺以內特徵匹配率還是 100%，5 公尺時降到 76%，12 公尺 66%，17 公尺 57%，28 公尺只剩 31%。請注意轉折點落在每秒 3 到 5 公尺之間——這恰好就是一般室內無人機的巡航速度，也解釋了為什麼 EuRoC 的困難序列正是那些飛得比較快的。

（b）量化第二重打擊，而這一重其實更致命。縱軸是一個特徵點從進入視野到離開視野能被看到幾幀（相機 30 fps）。每秒 0.5 公尺時是 104 幀（3.5 秒），2 公尺時 26 幀，5 公尺時 10 幀，17 公尺時只剩 3 幀，28 公尺時——只剩 1 幀。

一幀是什麼意思？意思是這個特徵在這一幀被偵測到，下一幀它已經不在畫面上了。沒有第二次觀測，就沒有三角測量（第 21.5 節需要視差）、沒有中期資料關聯（第 23.2 節）、也沒有辦法建立地圖點。換句話說，在每秒 28 公尺的速度下，就算影像完全清晰，視覺 SLAM 一樣做不下去。模糊只是讓情況雪上加霜。

（c）檢驗 26.4 節提到的第一種對策：提高快門速度。以每秒 17 公尺為例，快門 1/30 秒時匹配率是 0%，1/60 秒 1.7%，1/125 秒 22.9%，1/250 秒 57.2%，1/500 秒 78.5%，1/1000 秒 76.0%，1/2000 秒 100%。從 1/125 提高到 1/500，匹配率從兩成多跳到近八成——這是五種對策裡成本最低、效果最立即的一個。

代價當然也在：快門每快一倍，進光量就少一半。所以無人機的高速視覺定位實際上是一個光學問題——要嘛環境夠亮，要嘛換大光圈鏡頭與高感光元件。這也是為什麼競速無人機的視覺研究多半限定在「室內結構化場地或光線良好的戶外賽道」。

> **一個對專題的務實建議**
>
> 把（a）與（b）放在一起看，會得到一個很清楚的速度上限：每秒 3 公尺以內，模糊與軌跡長度都還健康（100% 匹配、17 幀以上）；每秒 5 公尺是堪用的邊界（76%、10 幀）；超過每秒 10 公尺，就必須同時處理快門、光源與演算法三件事。
>
> 同學在規劃無人機專題時，與其追求飛得快，不如先把速度限制在每秒 2 到 3 公尺，把定位與建圖做穩；速度是最後才該放寬的那個參數。

### 量一量：螺旋槳的震動會變成什麼

26.4 節最後提到震動「會污染加速度計讀值，若頻率接近取樣率還可能造成混疊」。這句話值得展開，因為混疊的後果比「雜訊變大」嚴重得多。

實驗設定：在一段緩慢的真實加速度訊號上疊加螺旋槳震動（基頻加二次諧波，振幅 2.5 公尺 / 秒²），以 400 Hz 或 1000 Hz 取樣，比較取樣後的訊號與真值的平均差——這個平均差，在下游的估測器眼中就是一個加速度計零偏。

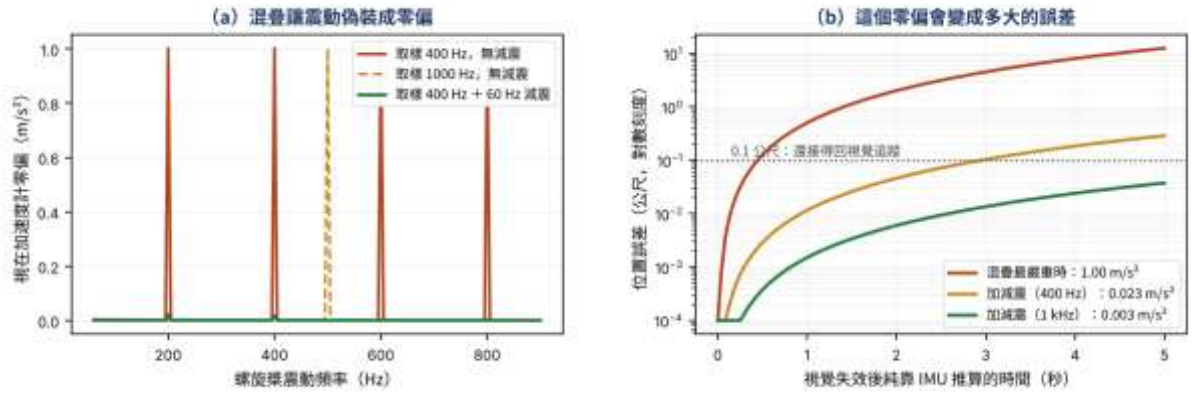



**圖 26-3　震動的混疊，以及它為什麼偽裝成零偏**

（a）掃描震動頻率。紅線是 400 Hz 取樣、沒有減震：大多數頻率下視在零偏接近零，但在 200、400、600、800 Hz 這幾個點上突然竄到 1.00 公尺 / 秒²。這些正是取樣率的整數倍與半整數倍——震動在這些頻率上被折疊成直流分量，也就是一個看起來完全像零偏的常數偏移。

黃色虛線是把取樣率提高到 1000 Hz。尖峰的位置改變了，但高度完全沒變——同樣是 1.00 公尺 / 秒²。這是本圖最重要的訊息：提高取樣率不能解決混疊，只能把問題搬到別的頻率去。這是一個相當常見的誤解。

綠線是加上 60 Hz 截止頻率的機械減震（軟膠墊加類比低通）。視在零偏的最大值從 1.00 降到 0.023 公尺 / 秒²，降了四十倍以上。關鍵在於這個濾波發生在「取樣之前」——高頻成分在進入類比數位轉換器之前就被擋掉了，因此根本沒有機會被折疊。取樣之後再做數位濾波，對混疊完全無效，因為訊號已經壞了。

（b）說明這 1.00 公尺 / 秒² 有多嚴重。零偏積分兩次就是位置誤差：視覺失效 1 秒會產生 0.50 公尺的誤差、2 秒 2.0 公尺、4 秒 8.0 公尺。對照第 23 章圖 23-1 的結論——零偏估到 ±0.01 時 2 秒的誤差只有 0.057 公尺——震動造成的這個「假零偏」比那裡討論的最差情況還糟三倍以上。

更麻煩的是，這個偏移不是隨機的，而是隨轉速改變的。無人機加速時轉速上升、震動頻率跟著移動，假零偏可能從零突然跳到 1.00 再掉回零。第 23.4 節的 IMU 零偏估計假設零偏

是緩慢變化的，面對這種跳變完全無能為力。所以減震不是「調校的細節」，而是系統能不能運作的前提。

**一個可以在實驗室做的檢查**

把無人機固定在桌上（槳葉拆掉或加裝防護），從怠速緩慢加到全速，同時記錄加速度計原始讀值。

然後對每個轉速區間算平均值。如果平均值在某幾個轉速上明顯偏離重力，那就是混疊——而且你剛好找到了必須避開的轉速區間。

更完整的做法是對原始訊號做快速傅立葉轉換，看看螺旋槳的基頻與諧波落在哪裡，再據此選擇減震座的軟硬度（也就是它的截止頻率）。這個實驗只需要一台無人機與一支記錄程式，卻能省下後續數週的除錯時間。

## 26.5　無人機室內導航實驗

本節說明如何規劃一次安全而有教學價值的無人機室內導航實驗，這也是專題實作的基礎。

### 安全第一：必須先講的事

無人機實驗與地面機器人最大的差異在於風險。以下是不可妥協的原則：

183. 防護網或防護籠：室內實驗應在有防護網的空間進行，或使用附有槳葉保護框的機型。

184. 選用微型平台：第 25 章介紹的微型無人機重量僅數十公克，即使失控撞到人也幾乎不會受傷。教學實驗強烈建議從微型平台開始。

185. 緊急停止：操作者必須隨時能一鍵切斷動力，且此功能不依賴需要除錯的自主系統。

186. 分階段測試：先在模擬中驗證、再繫繩測試、最後才自由飛行。跳過階段是意外的主因。

187. 人員位置：實驗時所有非必要人員應在防護網外，操作者站在無人機後方而非前方。

**實驗場地的準備**

188. 紋理：地面與牆面需有足夠的視覺紋理（第 9、20 章的白牆詛咒在無人機上同樣成立）。必要時可在地面鋪設有圖案的墊子。

189. 光線：均勻且充足，避免逆光與強烈明暗對比。窗戶可能造成過曝，建議拉上窗簾以維持穩定的照明。

190. 氣流：避免冷氣直吹或開窗形成的氣流，它會嚴重干擾微型無人機的穩定。

191. 真值系統（選用）：若有動作捕捉系統，能提供毫米級的真值位姿，用於量化評測（26.6 節）。沒有的話，可用第 4 章的 AprilTag 提供近似真值（第 24.3 節的第四種用途）。

**實驗流程**

16. 系統檢查：確認 IMU 校正、相機校正（第 9.2 節）、感測器外參（第 10 章）、以及通訊連線正常。

17. 懸停測試：最基本的能力。啟動光流定位，讓無人機在原地懸停 30 秒，量測位置漂移。若懸停都不穩，後續的自主飛行不必嘗試。

18. VIO 初始化：依第 23.4 節，以帶加減速與旋轉的運動完成 IMU 初始化，確認尺度估計正確（可用已知距離驗證）。

19. 簡單航線：設定一條簡單的方形或直線航線，讓無人機自主飛行，記錄軌跡。

20. 閉環測試：讓無人機繞一圈回到起點，量測終點與起點的落差，評估累積漂移（第 13.7 節的評估方法）。

21. 挑戰測試：逐步增加難度——提高速度、加入急轉、暫時遮蔽相機——觀察系統在什麼條件下失效。這是最有教學價值的環節。

### 資料記錄

每次飛行都應完整記錄：影像、IMU 原始資料、飛控狀態、估測軌跡、以及（若有）真值軌跡。這些資料能離線反覆分析，也能作為自製資料集——對專題研究而言，一份自己場域的資料集本身就是有價值的成果。

## 26.6 視覺慣性資料集：EuRoC、TUM-VI

沒有無人機硬體，同樣能做無人機 SLAM 的研究——這正是公開資料集的價值。本節介紹兩大標準資料集。

### EuRoC MAV 資料集

第 7 章案例 7-3 已介紹過 EuRoC，此處補充細節。它包含十一段以微型飛行器錄製的序列，提供雙目相機影像、IMU 量測與來自動作捕捉系統的真值位姿。

每段序列涵蓋 30 至 60 秒的飛行，包含來自外部動作捕捉的高保真 6-DoF 真值位姿，以及運作環境的高解析度點雲掃描，使軌跡估計與稠密三維地圖重建都能以幾何一致的參考資料進行評估。典型的評測流程是以 20 Hz 的單目影像與 200 Hz 的 IMU 資料，交由特徵法 VIO（例如關閉閉環的 ORB-SLAM3）處理，產出具真實尺度的相機軌跡與稀疏三維地標。

192. 序列分類：機房（MH）序列在工業廠房錄製，房間（V1、V2）序列在有動作捕捉的實驗室錄製。

193. 難度分級：每組序列都分 easy、medium、difficult 三級，難度來自速度、光線與運動劇烈程度。這讓同學能循序漸進地測試演算法。

194. 使用建議：從 MH_01_easy 開始建立成功經驗，再挑戰 V1_03_difficult 等困難序列。

### TUM-VI 資料集

TUM-VI 是另一套廣泛使用的基準。它提供 1024×1024 解析度、20 Hz 的相機影像，具備高動態範圍與光度校正；IMU 以 200 Hz 量測三軸加速度與角速度，相機與 IMU 在硬體層級時間同步。

相較之下，EuRoC MAV 的序列較短、變化較少，只包含一個機房與一個實驗室的錄製；此外 EuRoC MAV 不含光度校正，而這對評測直接法（第 20.2 節）相當重要。

195. 特色：涵蓋室內外多種場景、序列較長、有光度校正、時間同步在硬體層級完成。

196. AR / VR 相關：第 23.7 節引述過 ORB-SLAM3 在 TUM-VI 房間序列的毫米級精度，那正是這個資料集的房間場景。

| 特性 | EuRoC MAV | TUM-VI |
|---|---|---|
| 平台 | 微型飛行器 | 手持感測器組 |
| 影像 | 雙目，20 Hz | 雙目 1024×1024,20 Hz |
| IMU | 200 Hz | 200 Hz，硬體時間同步 |
| 真值 | 動作捕捉 / 雷射追蹤 | 動作捕捉（部分序列） |
| 光度校正 | 無 | 有 |
| 場景多樣性 | 一個機房 + 一個實驗室 | 室內外多種場景 |
| 額外資料 | 環境點雲掃描 | 高動態範圍影像 |

**表 26-3　EuRoC 與 TUM-VI 資料集比較**

### 資料集的使用建議

197. 演算法比較：在同一資料集上比較不同系統（第 23 章實作 23-1 的做法），數據才有可比性。

198. 留意評測條件：如 26.4 節提過的「是否裁掉開頭」，不同論文的評測條件可能不同，直接比較數字要謹慎。

199. 自製資料集：公開資料集無法涵蓋你的特定場域。用第 26.5 節的方法錄製大學場域的資料，是專題極有價值的貢獻。

200. 新興資料集：近年也出現速度更快、感測模態更多的資料集，如競速無人機資料集（速度可達每秒 28 公尺）與含熱像的資料集，適合進階研究。

## 26.7 從 nanoSLAM 到視覺導航無人機

本節把第 24、25 兩章連起來，梳理無人機自主導航的技術演進與未來方向。

### 三個層次的無人機自主

| 層次 | 能力 | 關鍵技術 | 代表平台 |
|---|---|---|---|
| 穩定 | 不飄走、能懸停 | IMU + 光流 + 測距 | 消費級無人機 |
| 自主 | 能沿航線飛、能避障 | VIO + 路徑規劃 | 專業無人機 |
| 智慧 | 能探索、建圖、理解任務 | VI-SLAM + 語意 + VLM | 研究平台、未來產品 |

**表 26-4 無人機自主能力的三個層次**

這個層次表能幫同學定位自己的專題目標。多數教學專題應從「穩定」層次確認基礎，再挑戰「自主」層次；「智慧」層次適合研究所或長期專題。

### 技術演進的三條主線

22. 運算的下沉：從外部電腦運算（卸載）、到機載高性能運算（如 Jetson）、再到第 25 章的毫瓦級專用晶片。這條線讓自主能力得以進入更小、更便宜的平台。

23. 感測的輕量化：從笨重的光達、到相機加 IMU、再到第 25 章的 ToF 矩陣感測器。這條線降低了重量與功耗的門檻。

24. 智慧的上升：從幾何導航、到語意理解（第 24 章）、再到 VLM 驅動的任務級自主。這條線提升了機器人能執行的任務複雜度。

值得注意的是，這三條線正在交會：第 25 章的微型平台已經能執行 YOLO 物件偵測（低於 100 毫瓦、最短 17 毫秒），意味著「小、省電、且有語意理解能力」的無人機正在成為現實。

### 大學場域的專題方向

201. 室內測繪：以微型無人機掃描系館內部（如挑高中庭、樓梯間），產生三維模型。這些是地面機器人難以到達的空間，能展現無人機的獨特價值。

202. 設施巡檢：屋頂太陽能板、外牆、高處管線的檢查——無人機能安全地取代人員登高作業。

203. 農業應用：呼應第 25 章案例 25-3，台灣的果園與溫室是微型無人機的理想場域。

204. 教學資料集：錄製並公開一份大學場域的視覺慣性資料集，供後續學生與研究者使用。這是相對容易入手、卻能長期產生價值的貢獻。

205. 地面與空中協作：讓機器狗（第 18、24 章）與無人機協同探索——地面機器人續航長、承載大，無人機視野廣、能到高處。這是第 27 章多機器人協作的具體應用。

### 前沿方向

最後提幾個值得關注的研究方向，供有志深入的同學參考：

206. 高速飛行的感知：如 26.4 節所述，現有 VIO 在高速下會劣化。以熱像儀等非傳統感測器突破速度限制，是活躍的研究方向。

207. 學習與幾何的結合：第 20.7 節提過深度學習特徵正進入 SLAM。有研究整合深度特徵提取器與稀疏特徵匹配器，並結合傳統的手工關鍵點偵測與光流追蹤，以 TensorRT 加速推論，在快速相機運動與光線變化下改善前端追蹤效能。

208. 學習式慣性里程計：純 IMU 的學習方法在特定情境下已能接近 VIO 的表現——這對視覺失效的場景（黑暗、煙霧）是重要的備援。

209. 群體協作：多台微型無人機協同建圖與探索，是第 27 章的主題，也是本篇的最後一塊拼圖。

至此，無人機導航的核心挑戰與技術路線已完整呈現。下一章我們把視野從單機擴大到群體：多台機器人如何協同建圖？各自的地圖如何融合？雲端與邊緣如何分工？通訊、同步與安全又有哪些課題？第 27 章將完成第七篇的最後一塊拼圖。

## 26.8　無人機視覺定位的現況：速度的上限在哪裡

本章量了很多「會壞掉」的地方。作為收尾，本節整理目前的技術邊界，以及突破它的三條路。

### 一、現況的三個速度帶

| 速度帶 | 典型場景 | 視覺定位的狀態 | 主要限制 |
|---|---|---|---|
| 每秒 3 公尺以內 | 室內巡檢、拍攝 | 成熟，VI-SLAM 可穩定運行 | 運算與續航 |
| 每秒 3 至 10 公尺 | 戶外巡檢、測繪 | 可行，需快門與減震的配套 | 動態模糊 |
| 每秒 10 至 20 公尺 | 競速、快速搜救 | 研究階段，需雙目與良好光線 | 特徵軌跡過短 |
| 每秒 20 公尺以上 | 競速極限 | 視覺幾乎無法單獨支撐 | 一幀即消失 |

**表 26-5　無人機視覺定位的四個速度帶**

這張表的分界點，正是圖 26-2 量出來的那兩個轉折：每秒 3 到 5 公尺之間模糊開始咬人，每秒 17 公尺左右特徵軌跡跌破三幀。兩個限制的性質不同——前者可以用光學手段緩解，

後者是幾何上的硬限制。

### 二、三條突破的路

第一條是換感測器。事件相機不輸出影格，而是在每個像素亮度變化時輸出一個事件，時間解析度可達微秒級，原理上不存在動態模糊。它的代價是輸出形式完全不同，第 20 到 23 章的整套方法都要重寫。

第二條是讓 IMU 承擔更多。26.4 節提過「以 IMU 為基礎的運動預測改善跨幀特徵追蹤」，本質上是用慣性資訊預測特徵「應該移動到哪裡」，把搜尋範圍從整張影像縮到一個小窗口。在高速下這個做法的價值特別大——但它的前提是 IMU 的零偏要估得準，而圖 26-3 剛好說明了震動會怎麼破壞這件事。兩者互相牽制，這是無人機系統設計最棘手的一環。

第三條是接受限制、改變任務。既然高速時視覺撐不住，那就在高速段只靠 IMU 與氣壓計維持姿態，定期減速到每秒 2 到 3 公尺做一次視覺定位修正。這與第 19.6 節巡檢機器人的地圖維護、第 25.8 節的事件驅動運算是同一個思路——不是讓系統時時刻刻都達到最高標準，而是讓它在需要的時候達到就好。

對同學的專題而言，第三條路最務實。它不需要特殊硬體，也不需要改寫演算法，只需要在任務規劃時把「什麼時候該慢下來」寫進去。

## 公式或流程

### 一、光流速度換算

向下相機量到的光流是「影像上每秒移動幾個像素」，要換算成真實的水平速度，需要離地高度與相機焦距：

$$v = (\mathit{flow_pixels} / f) \times h$$

其中 flow_pixels 是每秒的像素位移、f 是焦距（像素單位，第 9.1 節的內參）、h 是離地高度（由測距感測器提供）。

這條式子透露一個關鍵風險：高度 h 出現在分子。若測距感測器誤判高度（例如飛過桌面時高度突然變小），換算出的速度就會錯誤，飛控可能因此做出錯誤修正而發生漂移甚至撞擊。這是光流定位最常見的失效原因，同學在實驗時務必注意場地的地面起伏。

## 二、動態模糊的量化

曝光期間影像上的模糊長度，約為：

$$blur_pixels \approx (v / h) \times f \times t_exposure$$

其中 v 為飛行速度、h 為到場景的距離、t_exposure 為曝光時間。

代入實際數字感受一下：飛行速度 5 m/s、距離場景 5 公尺、焦距 400 像素、曝光 10 毫秒，則模糊約 4 像素——已足以嚴重影響角點偵測（第 20 章的特徵需要清晰的局部結構）。若速度提高到 20 m/s，模糊達 16 像素，特徵提取幾乎完全失效。

這條式子也直接指出對策：縮短曝光時間 t_exposure 能線性減少模糊——這正是 26.4 節第一項對策的數學依據。

## 三、VIO 的狀態與誤差項

無人機 VIO 的狀態向量與第 23.3 節相同，每個關鍵幀為：

$$S = \{R, p, v, b_g, b_a\} \quad (\text{共 } 15 \text{ 自由度})$$

緊耦合最佳化同時最小化視覺與慣性誤差：

$$\min \Sigma \rho(\| e_\text{視覺} \|^2) + \Sigma \| e_\text{慣性} \|^2$$

對無人機而言，一個實務重點是兩者權重的動態調整：當偵測到影像模糊或特徵數不足時，應降低視覺項的權重、提高慣性項的比重——這正是 26.4 節第五項對策（自適應權重）的實現方式，也呼應第 11 章卡爾曼增益「按可信度加權」的核心思想。

## 四、無人機 GPS 拒止導航的分層架構

**圖 26-4 無人機導航的分層架構與失效降級**

【校正層】不定期：UWB / 視覺標記 / 重回 GPS 區 → 絕對位置校正

↓ 失效則降級

【上層】10–30 Hz:VI-SLAM / VIO → 完整 6-DoF 位姿 + 地圖

支撐：路徑規劃、自主導航、探索建圖

↓ 失效則降級

【中層】數十 Hz：光流 + 測距 → 水平速度 + 離地高度

支撐：定點懸停（至少不飄走）

↓ 失效則降級

【底層】數百 Hz:IMU → 姿態

支撐：維持飛行穩定，讓操作者接手

【設計原則】讓失效逐級降級，而非瞬間崩潰

## 案例

### 案例 26-1 沒有輪子的代價：無人機為何比地面機器人難

同學若同時做過地面機器人與無人機的 SLAM，會強烈感受到後者困難得多。原因不只是「飛比較快」，而是一個常被忽略的結構性差異：無人機沒有里程計。

回顧本書的脈絡：第 13.4 節說明 2D SLAM 靠「里程計預測、光達校正」的循環運作；第 14.5 節指出 Gmapping 的三個前置條件中，里程計是必要的；第 18.3 節的 LOAM 也需要運動估計作為配準初值。里程計雖然會漂移，但它提供了穩定、連續、免費的運動參考。

無人機完全沒有這個奢侈品。它唯一的內部運動來源是 IMU，而第 7.3 節已經證明：消

費級 IMU 單獨積分，數秒內就會漂移數公尺。這意味著無人機的視覺系統必須「獨力承擔」原本由里程計分擔的責任——一旦視覺失效，系統就沒有可靠的後備。

這解釋了三件事：為什麼無人機領域如此重視 VIO 的緊耦合（第 23 章）、為什麼光流定位是消費級無人機的標準配備（26.3 節）、以及為什麼 26.2 節的分層降級架構如此關鍵。

對同學的啟示是：理解一個平台的「缺什麼」，往往比理解它「有什麼」更能解釋系統設計的取捨。這種從限制反推設計的思維，是工程判斷力的核心。

## 案例 26-2　初始化的兩難：動得不夠不行，動太多也不行

這是一個極具教學價值的矛盾，而且同學在實作時必定會遇到。

一方面，第 23.4 節反覆強調：IMU 初始化需要足夠的運動激勵，否則尺度與零偏不可觀測。靜止或等速直線移動，系統就是初始化不了。

另一方面，26.4 節提到的實例顯示：EuRoC 的機房序列在開頭都有快速運動以協助 IMU 初始化，但由此產生的動態模糊對視覺追蹤是一大挑戰。快速運動雖然滿足了慣性系統的需求，卻破壞了視覺系統的輸入品質。

這個矛盾的存在，恰恰說明了視覺慣性融合的本質：兩種感測器的需求並不一致，甚至互相拉扯。IMU 喜歡有加速度的運動，相機喜歡平穩清晰的畫面。

實務上的解法是折衷：做「中等幅度、帶加減速與旋轉、但不過快」的運動。具體而言，拿著裝置或操控無人機做小幅度的前後左右移動並轉動數秒，速度以「畫面不明顯模糊」為上限。

這個案例也提醒同學：當論文中的方法在你手上跑不起來時，問題未必在程式，而可能在「你的資料不符合方法的隱含假設」。理解方法對輸入的要求，是成功實作的前提。

## 案例 26-3　速度的邊界：VIO 能飛多快

「無人機視覺定位能支援多快的飛行？」這個問題的答案，近年正被不斷推進，也清楚

呈現了技術的演進。

先看基準：EuRoC 微型飛行器資料集的速度很少超過每秒 5 公尺——這是多數 VIO 論文的評測條件。以此為基準，同學看到的多數精度數據，都是在相對溫和的速度下取得的。

往上推進：有研究以雙目 RGB 的 VIO 支援到每秒 17.5 公尺，但它依賴雙目視覺並假設有利的光線條件。競速無人機資料集則可達每秒 28 公尺，但通常是室內結構化場地或短程戶外賽道的 RGB 相機。

突破限制：更前沿的研究開始改用非傳統感測器。有研究以單目熱像慣性里程計，在低空飛行中達成每秒 30 公尺的即時運作。熱像的優勢在於不受光線影響、且對某些模糊來源較不敏感。

但這條路也有代價。研究指出，雖然以深度學習為基礎的方法在熱像上展現了不錯的效能提升，但它們的推論時間仍與無人機機載硬體嚴格的延遲與功耗限制不相容——某些方法在筆電等級的 GPU 上每幀就需要超過 30 毫秒。因此手工設計的管線仍是嵌入式高速閉環飛行最實際的解法。

這個案例濃縮了本書一再出現的主題：前沿技術的精彩數據，往往建立在特定條件之上；而真正能部署的系統，必須同時滿足精度、速度、功耗與可靠度的多重約束。學會閱讀論文時同時看「達成了什麼」與「在什麼條件下達成」，是研究能力的重要一環。

## 實作活動

### 實作 26-1　無人機視覺慣性導航實驗

目的：透過公開資料集與（可選的）實體飛行，理解無人機視覺定位的挑戰與對策。第一、二部分不需硬體，適合全體；第三部分需要微型無人機平台。

**第一部分：資料集分析（全體，不需硬體）**

132. 下載 EuRoC 資料集，選取 MH_01_easy 與 V1_03_difficult 兩段序列，分別以第 23 章的 ORB-SLAM3 雙目慣性模式執行，用 evo 計算 ATE。

133. 比較兩者的精度差異，並播放影像觀察 difficult 序列在哪些片段出現明顯的動態模糊，對照 26.4 節的討論。

134. 模糊量化：寫程式計算每一幀影像的清晰度（可用拉普拉斯運算子的變異數），畫出「清晰度隨時間變化」的曲線，標出最模糊的片段。

135. 關聯分析：把清晰度曲線與軌跡誤差曲線畫在一起，觀察兩者是否相關——影像最模糊的時刻，是否也是誤差最大的時刻？

136. 初始化觀察：留意序列開頭的快速運動段，記錄系統完成 IMU 初始化所需的時間，並觀察此段的影像模糊程度，親身驗證案例 26-2 的兩難。

**第二部分：光流與模糊模擬（全體）**

25. 用 OpenCV 的 Lucas-Kanade 光流，對一段向下拍攝的影片（可用手機在桌面上方平移拍攝）計算光流，依公式一換算水平速度。

26. 高度敏感度實驗：故意把高度值設錯（例如實際 1 公尺卻填 0.5 公尺），觀察換算速度的誤差，理解 26.5 節提到的失效風險。

27. 模糊模擬：對清晰影像人為加入動態模糊（用 OpenCV 的運動模糊核），模糊長度依公式二計算，測試 ORB 特徵偵測數量隨模糊程度如何下降。

28. 繪製「模糊像素數 vs 可偵測特徵數」的曲線，找出特徵數急遽下降的臨界點。這個臨界值就是你的系統能容忍的速度上限依據。

**第三部分：實體飛行（有微型無人機平台的班級）**

務必先閱讀 26.5 節的安全原則，並在防護網內、以微型平台進行。

1. 懸停測試：啟動光流定位，讓無人機在原地懸停 30 秒，以地面標記量測水平漂移範圍。

2. 地面紋理實驗：分別在有圖案的地毯、純色地板、與反光地面上懸停，比較漂移程度，驗證光流對紋理的依賴。

3. 高度實驗：在不同離地高度（0.5、1.0、1.5 公尺）懸停，觀察穩定度差異，思考公式一中高度誤差的影響。

4. 簡單航線：設定一條方形航線自主飛行，回到起點後量測落差，評估累積漂移。

5. 挑戰測試：在安全前提下逐步提高飛行速度，記錄系統開始不穩定的速度，對照案例 26-3 的討論。

**討論問題**

49. easy 與 difficult 序列的 ATE 差多少倍？影像清晰度與軌跡誤差的相關性如何？

50. 依公式二，若你的相機曝光 5 毫秒、焦距 400 像素、距離場景 3 公尺，飛行速度多快時模糊會達到 5 像素？這對速度上限有什麼意涵？

51. 光流換算實驗中，高度誤差 50% 造成速度誤差多少？若無人機依此錯誤速度做修正，會發生什麼？

52. （有飛行者）在哪種地面上懸停最穩定？最不穩定的原因是什麼？這印證了 26.3 節的哪一項限制？

53. 依圖 26-1 的分層架構，若你的無人機在飛行中 VIO 突然失效，系統應如何反應？請設計一套降級流程。

**延伸挑戰（大學部）**

其一：自適應權重實作——在 VIO 系統中加入影像品質評估（如第二部分的清晰度指標），當影像模糊時自動降低視覺項權重、提高慣性項比重，在 EuRoC difficult 序列上測試是否改善精度。這對應 26.4 節的第五項對策，也是一個完整的小型研究題目。其二：大學場域資料集——依 26.5 節的方法，以微型無人機或手持感測器組錄製系館內部的視覺慣性資料（含影像、IMU、以及用 AprilTag 提供的近似真值），整理成標準格式並公開，供後續學生使用。這是一份能長期產生價值的貢獻，也是專題成果的良好呈現形式。

## 延伸閱讀

71. M. Burri et al., "The EuRoC Micro Aerial Vehicle Datasets," International Journal of Robotics Research, 2016.（EuRoC 資料集的官方說明，本章 26.6 節依據）

72. D. Schubert et al., "The TUM VI Benchmark for Evaluating Visual-Inertial Odometry," IROS, 2018.（TUM-VI 資料集，含光度校正與硬體時間同步的設計說明）

73. C. Campos et al., "ORB-SLAM3," IEEE T-RO, 2021.（第 23 章的系統，本章實作的主要工具）

74. T. Qin, P. Li, and S. Shen, "VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator," IEEE T-RO, 2018.（無人機視覺慣性系統的另一代表作）

75. D. Scaramuzza and Z. Zhang, "Visual-Inertial Odometry of Aerial Robots," Encyclopedia of Robotics, Springer, 2019.（空中機器人視覺慣性導航的權威綜述，UZH 研究群，呼應第 2 章）

76. K. Sun et al., "Robust Stereo Visual Inertial Odometry for Fast Autonomous Flight," IEEE RA-L, 2018.（高速飛行 VIO 的代表性工作，案例 26-3 提及）

77. PX4 與 ArduPilot 開源飛控之視覺定位整合文件.（把 VIO 輸出接入飛控的實務指引，實體飛行必讀）

## 小結

71. 無人機定位有四項特殊困難：完整六自由度運動、沒有里程計（只能靠會漂移的 IMU）、速度快且動態劇烈、以及失誤後果嚴重；加上載重電力限制與嚴苛的即時性要求，使它成為最難的 SLAM 平台之一。

72. GPS 拒止場景包含室內、橋下、隧道、都市峽谷、植被下與人為干擾；可用手段依序為 IMU、氣壓計、光流、測距、VIO、VI-SLAM 與 UWB / 視覺標記，實務採分層架構並讓失效逐級降級而非瞬間崩潰。

73. 三種視覺定位技術的層次：光流只給水平速度但運算極省，支撐定點懸停；VIO 給完整 6-DoF 位姿但會漂移；VI-SLAM 加上地圖與閉環可修正長期漂移。應依任務需求選擇，不用超過需求的技術。

74. 高速飛行造成三重打擊：動態模糊損害特徵追蹤、特徵軌跡縮短使中期關聯難以建立、控制迴路延遲惡化；對策包含提高快門速度、使用全域快門、以 IMU 預測改善追蹤、影像增強、以及自適應調整視覺與慣性的權重。

75. 室內飛行實驗的安全原則不可妥協：防護網、微型平台、緊急停止、分階段測試；場地需注意紋理、光線與氣流；實驗流程應由懸停測試逐步進展到挑戰測試。

76. EuRoC 提供十一段微型飛行器序列、雙目影像與動作捕捉真值，速度很少超過每秒 5 公尺；TUM-VI 則提供 1024×1024 影像、光度校正與硬體時間同步，場景多樣性較高。比較數據時須留意各研究的評測條件。

77. 無人機自主可分穩定、自主、智慧三個層次；技術演進沿運算下沉、感測輕量化、智慧上升三條主線推進，而三者正在交會——小、省電且具語意理解能力的無人機正在成為現實。

## 承先啟後

本章把飛行帶來的三個新問題量化了：漂移方面，72 秒的飛行後半段，光流的平均誤差是 0.452 公尺、VIO 0.324 公尺、VI-SLAM 0.041 公尺，而真正的差別在形狀——只有 VI-SLAM 的誤差不隨時間成長；速度方面，特徵匹配率從每秒 3 公尺的 100% 掉到 28 公尺的 31%，而更致命的是特徵停留在視野中的幀數從 104 幀掉到 1 幀，最有效的對策是快門（17 m/s 下由 1/125 提到 1/500，匹配率從 23% 升到 79%）；震動方面，混疊會讓螺旋

槳的高頻振動偽裝成 1.00 公尺 / 秒² 的假零偏（4 秒即造成 8 公尺的位置誤差），而且提高取樣率無效，只有取樣前的機械減震能把它壓到 0.023。

本章完成了單機無人機導航的討論。但無論多先進，單一機器人的視野、續航與運算能力終究有限。下一章我們把視野擴大到群體：多台機器人如何協同建圖？各自的地圖如何融合成一致的全域地圖？雲端與邊緣如何分工？通訊頻寬、時間同步與資料安全又有哪些課題？第 27 章將完成第七篇的最後一塊拼圖，也為第八篇的實作與第九篇的 AI 語意導航鋪路。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

## 第 27 章

## 多機器人 SLAM 與雲端 / 邊緣協作

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程　/　高中 AI 機器人社團課程

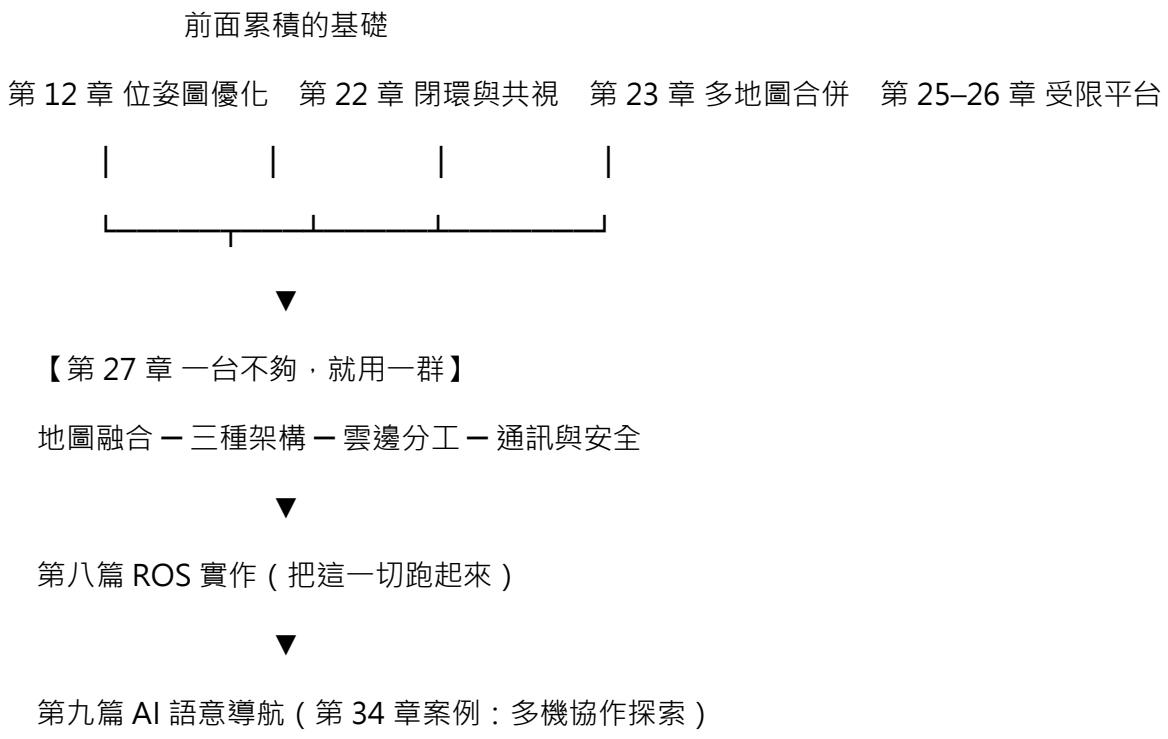
## 本章在全書的位置

第 25 章把機器人做小，第 26 章讓它飛起來。兩章的共同結論是：單一平台的能力終究有限。本章的答案不是把它做得更強，而是多來幾台。

但「多來幾台」不是把一台的能力乘以 N。多機器人系統有它自己的問題——地圖怎麼對齊、誰跟誰說話、頻寬不夠時要先傳什麼、以及一台機器人認錯地方時會不會拖垮其他所有機器人。這些問題在單機系統裡完全不存在。

本章的三張圖分別量化其中三件事：圖 27-1 算出多台機器人協同探索到底能快多少，以及「共享地圖」這件事值多少；圖 27-2 量出一條錯誤的跨機器人閉環能造成多大破壞；圖 27-3 則回答一個很實際的問題——頻寬只夠傳幾條閉環時，該傳哪幾條。

### 圖 27-0　第 27 章的連結地圖




## 學習目標

前面二十五章討論的都是「一台機器人」。但單機的視野、續航與運算終究有限：一台機器狗要花數小時才能巡完整棟系館，而十台可能只需二十分鐘。本章把視野擴大到群體，也完

成第七篇的最後一塊拼圖。學完本章之後，同學應能夠達成下列目標：

71. 說明多機器人 SLAM 相對單機的優勢與新增的三大挑戰。
72. 說明地圖融合的流程與跨機器人閉環偵測的關鍵。
73. 比較集中式、分散式與去中心化三種協作架構的優缺點。
74. 說明雲端 SLAM 與邊緣 SLAM 的分工原則與適用場合。
75. 規劃一個多台機器狗校園巡檢的系統架構與任務分配。
76. 說明 SLAM 與 AI Agent 如何協作，以及 VLM 在多機協調中的角色。
77. 說明多機器人系統在隊形、通訊、資料同步與安全上的課題。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 協作式 SLAM | Collaborative SLAM | 多台機器人共同建立一致地圖的技術。 |
| 集中式架構 | Centralized | 所有資料匯集到中央伺服器處理。 |
| 分散式架構 | Distributed | 運算分散在多個節點，彼此交換資訊。 |
| 去中心化 | Decentralized | 無中央節點，機器人直接彼此通訊並各自融合。 |
| 地圖融合 | Map Fusion / Merging | 把多張地圖對齊並合併為一致的全域地圖。 |
| 跨機器人閉環 | Inter-robot Loop Closure | 不同機器人觀測到同一地點所建立的約束。 |
| 相對變換 | Relative Transform | 兩台機器人座標系之間的旋轉與平移關係。 |
| 會合 | Rendezvous | 兩台機器人相遇並交換資訊的事件。 |
| 感知混淆 | Perceptual Aliasing | 不同地點看起來相同而造成的錯誤關聯。 |
| 離群點剔除 | Outlier Rejection | 排除錯誤閉環以免破壞全域地圖。 |
| 邊緣運算 | Edge Computing | 在靠近機器人的本地節點進行運算。 |

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 雲端運算 | Cloud Computing | 在遠端資料中心進行運算與儲存。 |
| 異質團隊 | Heterogeneous Team | 由不同類型機器人（地面、空中）組成的團隊。 |

## 27.1　單機 SLAM 與多機 SLAM

本節說明為什麼要用多台機器人，以及這個選擇帶來什麼新問題。

### 多機器人的四項優勢

- 速度：多台分頭探索，完成大範圍任務的時間大幅縮短。這是最直接的動機——第 24 章案例 24-3 的多機器人協作探索正是這個思路。
- 容錯：單台故障不影響整體任務。相較之下，單一昂貴機器人一旦故障，整個任務就中止。
- 視角互補：從多個位置同時觀測，能建立更完整的環境模型，也能減少遮蔽造成的盲區。
- 能力互補：異質團隊能結合不同平台的長處——地面機器人續航長、承載大；無人機視野廣、能到高處（第 26.7 節提過的地空協作）。

### 三大新挑戰

但多機器人不是「單機乘以 N」這麼簡單，它帶來三個單機不存在的問題：

210. 座標系統一：每台機器人開機時各自以自己的起點為原點（第 10 章）。要把它們的地圖合在一起，必須先知道彼此座標系之間的相對變換——而這個變換一開始是未知的。

211. 資料關聯的擴大：單機 SLAM 只需判斷「我現在看到的，是不是我以前看過的」（第 23 章的長期關聯）；多機器人還要判斷「我看到的，是不是別人看過的」。搜尋空間變大，錯誤關聯的風險也隨之上升。

212. 通訊限制：機器人之間的頻寬有限、連線可能中斷、延遲不固定。所有協作機制都必須在這個現實下運作——這是本章反覆出現的主軸。

### 與單機技術的關係

好消息是：多機器人 SLAM 的核心技術，同學都已經學過了。

213. 每台機器人的自身定位建圖：第 13 至 26 章的各種單機 SLAM。

214. 跨機器人閉環偵測：第 20、22 章的地點辨識（詞袋模型）與幾何驗證。

215. 地圖融合的最佳化：第 12 章的位姿圖優化。

216. 錯誤閉環的處理：第 12、15 章的穩健核函數與離群點剔除。

換句話說，多機器人 SLAM 是把單機的技術「擴充到多個座標系與多個資料來源」。理解這一點，同學就不會覺得本章的內容陌生——它是前面所有章節的自然延伸。

## 量一量：多幾台到底能快多少

27.1 節列出了多機器人 SLAM 的四項優勢，第一項是「速度」。這一項聽起來最理所當然，卻也最容易被高估。本小節把它算出來。

實驗設定是一層 34×18 公尺的系館樓層（中央走廊加兩側十二個房間）。N 台機器人從走廊上不同位置出發，各自以「前往最近的前沿」策略探索，感測半徑 3 公尺，直到自由空間被探索 98.5% 為止。兩種設定：共享同一張地圖（看得到別人探過哪裡），以及各自建圖、完全不交換資訊。

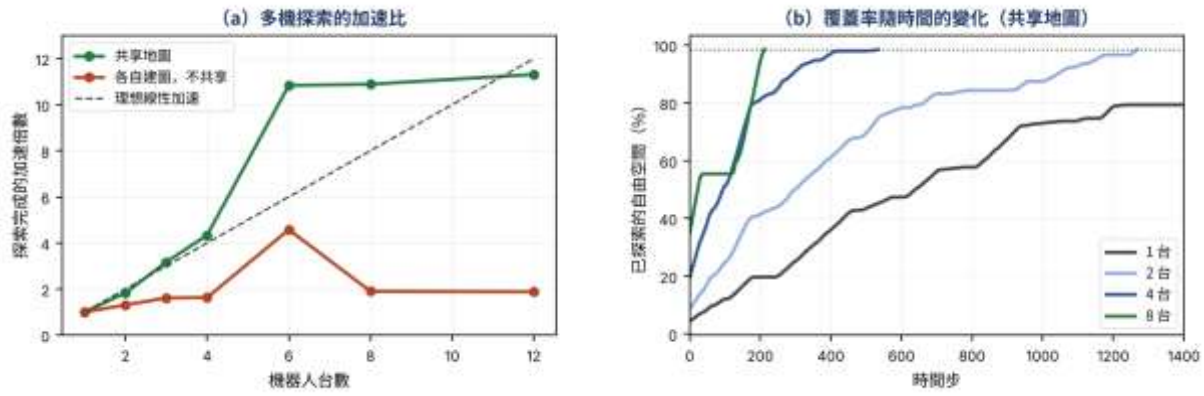



**圖 27-1　多機器人協同探索的加速比**

（a）是加速比。共享地圖時，兩台是 1.83 倍、三台 3.17 倍、四台 4.33 倍、六台 10.8 倍，之後就停住了——八台 10.9 倍、十二台 11.3 倍。也就是說，這層樓的「平行度上限」大約在六台。再多的機器人不是沒用，而是它們抵達最後幾個未探索角落所需的移動時間，已經

成為無法再壓縮的瓶頸。

三台與四台出現略高於線性的加速（3.17 與 4.33），原因是起點分散在走廊各處，省下了單機必須從頭走到尾的那段路程。這提醒我們：多機系統的效益有一部分來自「分散部署」，而不只是「平行運算」。

真正值得注意的是紅線。不共享地圖時，兩台只有 1.31 倍、四台 1.63 倍，而且曲線上下起伏不定（六台 4.57 倍，八台卻掉回 1.90 倍）。這個不穩定本身就是結論：沒有共享地圖的多機器人會重複探索彼此已經走過的區域，最終表現完全取決於它們碰巧從哪裡出發。以八台為例，共享地圖讓完成時間縮短了 83%。

這就是為什麼 27.2 節的地圖融合是整章的核心操作。沒有融合，多機器人只是多台各自為政的單機；有了融合，它們才真正成為一個系統。

**加速比飽和之後，多出來的機器人做什麼**

圖 27-1(a) 顯示六台以上就不再變快。但這不代表第七台沒有價值。

27.1 節提到的另外三項優勢——容錯、視角多樣性與任務分工——都不隨台數飽和。第七台可以在某一台故障時接手，可以從不同角度觀測同一個物件以提升語意判讀的可靠度（第 32 章），也可以專門負責充當通訊中繼。

所以規劃機群規模時，應該分開問兩個問題：「要多快完成」決定探索所需的台數，「要多可靠」與「要做哪些事」決定額外的台數。

## 27.2　Map Fusion 地圖融合

地圖融合是多機器人 SLAM 的核心操作：把兩張（或多張）各自建立的地圖，對齊並合併成一致的全域地圖。

### 與單機閉環的異同

同學應該立刻聯想到第 23.6 節的 Atlas 地圖合併。兩者的數學本質相同——找出兩張地圖之間的相對變換，再以位姿圖優化把修正傳播出去。差別在於：

217. 單機的多地圖：兩張地圖來自同一台機器人的不同時段，感測器與參數完全相同。

218. 多機器人的地圖融合：兩張地圖來自不同機器人，可能使用不同的感測器、不同的視角、甚至不同的地圖表示。異質團隊的融合尤其困難。

**融合的三步驟**

6. 偵測相遇：判斷兩台機器人是否觀測到同一個地點。這可以是實體的會合（兩台機器人互相看見），也可以是「地點辨識的重疊」——A 機器人現在看到的場景，B 機器人稍早也看過。

7. 估計相對變換：確認重疊後，由共同觀測到的特徵或掃描，解出兩台機器人座標系之間的旋轉與平移（第 21 章的幾何求解，或第 17 章的點雲配準）。

8. 融合與最佳化：把兩張位姿圖用這條「跨機器人的邊」連起來，執行位姿圖優化（第 12 章），使整個系統達成全域一致。

**關鍵風險：錯誤的跨機器人閉環**

第 12、15、22 章都強調過：一條錯誤的閉環會把地圖硬拉到錯誤位置。在多機器人系統中，這個風險更高，因為：

219. 候選空間更大：要比對的不只是自己的歷史，還有所有其他機器人的觀測。

220. 感知混淆更嚴重：不同機器人可能在不同的相似地點（例如兩間格局相同的教室），產生看似正確的錯誤匹配。

221. 後果更嚴重：一條錯誤的跨機器人邊，會同時破壞多台機器人的地圖。

因此穩健的離群點剔除是多機器人 SLAM 的必備元件。相關系統會整合穩健的離群點剔除機制來強化這個做法，並使其適應零星的機器人間通訊。另有系統採用分散式的漸進非凸化演算法，並把語意資料納入產出的地圖中。

### 通訊效率的考量

地圖融合需要交換資料，而頻寬有限。因此如何「用最少的資料達成最好的融合」，是這個領域的核心課題。有系統引入基於代數連通度最大化的新穎跨機器人閉環優先排序技術，以減少通訊並加速收斂；它也採用去中心化的鄰居管理與位姿圖優化方式，適合零星的機器人間通訊，並支援不同感測器類型的描述子——影像用 CosPlace 與 NetVLAD、光達掃描用 Scan Context。

這裡的核心洞見是：不是所有閉環候選都同等重要。有些閉環對改善全域一致性貢獻很大，有些則影響甚微。優先傳輸高價值的候選，能在有限頻寬下獲得最大效益——這是一個極具工程智慧的思路。

### 量一量：一條錯誤的跨機邊有多危險

27.2 節說「一條錯誤的跨機器人邊，會同時破壞多台機器人的地圖」。這句話值得用數字說一次，因為它的嚴重程度超過多數人的直覺。

實驗設定：三台機器人各走一條部分重疊的環形路線，每台 50 個關鍵幀，里程計有雜訊。它們之間有 42 條正確的跨機器人閉環。接著刻意混入若干條「感知混淆」造成的錯誤邊——機器人以為看到的是對方的第 $j$ 幀，實際上那是長得很像的另一處。比較一般最小平方與穩健核（漸進非凸化）兩種求解方式。

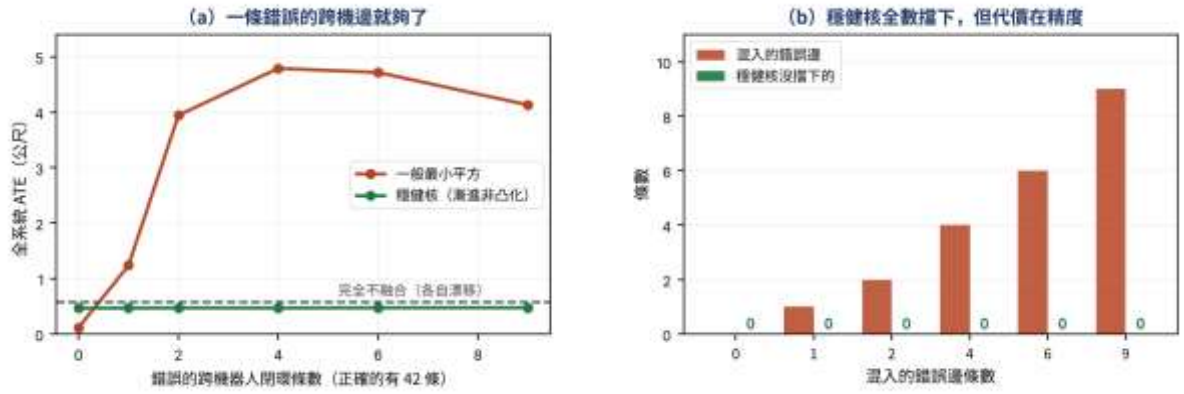



**圖 27-2　錯誤的跨機器人閉環與穩健剔除**

（a）的紅線是重點。沒有錯誤邊時，融合後的全系統 ATE 是 0.119 公尺，比完全不融合的

0.576 公尺好了近五倍——這是地圖融合的正面價值。但只要混入一條錯誤邊，ATE 就跳到 1.238 公尺；兩條是 3.946 公尺；四條以上停在約 4.8 公尺。

請注意一條錯誤邊造成的破壞：從 0.119 跳到 1.238，放大了十倍以上，而且比完全不融合還糟兩倍。換句話說，在這個情境下，「融合但混進一條錯誤邊」比「根本不融合」更差。這就是 27.2 節把離群點剔除列為「必備元件」的理由——它不是用來提升精度的，是用來避免災難的。

綠線是加上穩健核之後。不論混入幾條錯誤邊（測到九條），ATE 都穩定在 0.47 公尺左右，完全不隨錯誤邊增加而惡化。（b）顯示它的篩選能力：所有錯誤邊都被擋下，沒有一條漏網。

但這裡有一個必須誠實說明的代價。即使在完全沒有錯誤邊的情況下，穩健核的 ATE 也是 0.469 公尺，而一般最小平方是 0.119 公尺——差了四倍。原因是穩健核會降低「所有」大殘差的權重，包括那些只是雜訊比較大的正確邊。它用精度換取了對離群點的免疫力。

實務上的處理方式是兩階段：先用穩健核判別出內點集合，再只用內點重做一次一般最小平方。這樣理論上能同時拿到兩者的好處，但判別門檻要訂得很小心——門檻一鬆就會放進錯誤邊，而放進一條就前功盡棄。這是多機器人 SLAM 實作中最需要調校、也最容易出事的地方。

### 量一量：頻寬不夠時該傳哪幾條

27.2 節最後提到「不是所有閉環候選都同等重要」，並介紹了以代數連通度最大化來排序候選的做法。這個想法相當抽象，本小節用實驗讓它變得具體。

代數連通度是圖的拉普拉斯矩陣第二小的特徵值，直覺上衡量「這張圖連得有多緊」——值越大，表示圖越不容易被切成兩半，誤差也越容易在整張圖上重新分配（第 22 章圖 22-2 的道理）。實驗比較兩種挑選策略：隨機挑 k 條候選，以及貪婪地每次挑「最能提升代數連通度」的那一條。

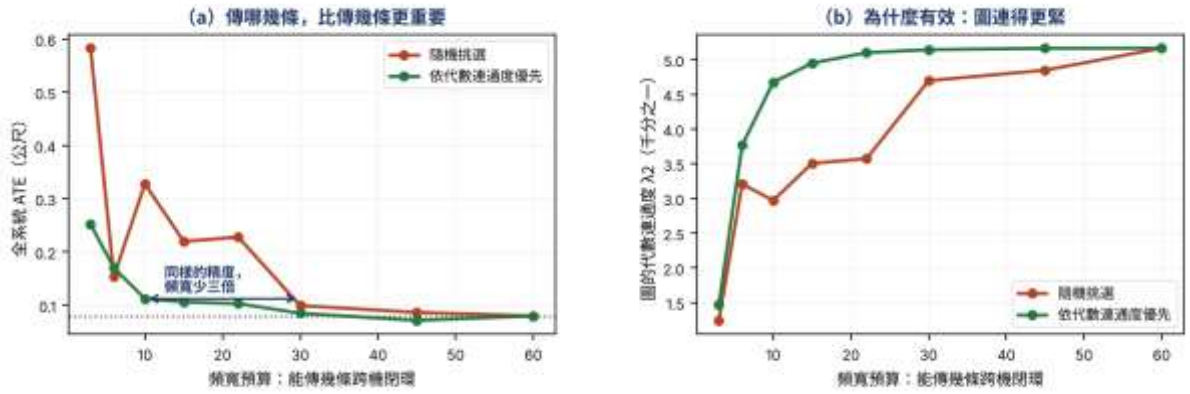



**圖 27-3　有限頻寬下，該優先傳哪些閉環候選**

（a）是結果。預算只有 3 條時，隨機挑選的 ATE 是 0.583 公尺，依連通度排序是 0.252 公尺；預算 10 條時分別是 0.328 與 0.112 公尺；預算 22 條時是 0.228 與 0.103 公尺。在中低預算下，排序讓誤差減半到減三分之二。

更有用的讀法是橫著看：依連通度排序只要 10 條就能達到 0.112 公尺，而隨機挑選要到 30 條才達到相近的水準。也就是說，光是「挑對要傳哪幾條」，就能把頻寬需求降到三分之一。對於只有零星通訊機會的機器人團隊，這個差別是決定性的。

（b）解釋為什麼有效。縱軸是圖的代數連通度，綠線在每個預算下都高於紅線——排序策略用同樣的邊數換到了更緊密的圖結構。兩條線在預算 60（全部傳完）時重合，因為此時兩者挑的是同一個集合。

這個實驗也示範了一個通用的工程思路：當資源有限時，與其問「能不能多拿一點資源」，不如先問「現有的資源有沒有花在刀口上」。同樣的思路在第 22 章的關鍵幀剔除、第 25 章的事件驅動運算都出現過。

**代數連通度為什麼是對的指標**

直覺上我們會想挑「最確定的閉環」或「最近的閉環」。但那些閉環往往連接的是已經靠得很近的兩個節點——資訊是重複的，對全域一致性幫助不大。

代數連通度抓到的是另一件事：這條邊有沒有把圖上原本相距很遠的兩塊連起來。把

兩塊鬆散的區域縫在一起，遠比在已經很緊的地方再加一針有用。

這與第 22 章「共視邊讓修正量傳遍整條軌跡」是同一個道理，只是從定性的觀察變成了一個可以計算、可以排序的數字。

## 27.3　Collaborative SLAM 的三種架構

多機器人 SLAM 的系統架構，是本章最重要的設計決策。多機器人 SLAM 架構的光譜，從集中式一路到完全去中心化。

### 一、集中式架構

所有機器人把資料傳給一台中央伺服器，由伺服器統一處理與最佳化，再把結果分發回去。

像 CCM-SLAM 這類集中式系統把資料匯集到伺服器進行聯合最佳化；這簡化了地圖融合，但引入了通訊瓶頸與單點故障。

222. 優點：地圖融合簡單（所有資料都在同一處）、全域最佳化容易、機器人端運算負擔輕。

223. 缺點：通訊瓶頸（所有資料都要傳）、單點故障（伺服器一掛全系統停擺）、擴展性差。

224. 代表系統：CCM-SLAM、COVINS（針對視覺慣性感測、以移除冗餘關鍵幀來稀疏化）、LAMP 2.0（光達系統，以圖神經網路預測每個跨機器人閉環候選的優先順序）。

### 二、分散式與去中心化架構

在分散式系統中，處理分散在多個節點或代理上；機器人蒐集資料並執行自己的定位與建圖，同時透過持續通訊彼此分享資訊，以建立完整的全域地圖。

去中心化的做法中，機器人直接彼此交換資訊，並在機載完成所有資料融合，不依賴中央實體來協調系統。

225. 優點：無單點故障、擴展性佳、對通訊中斷較有韌性。

226. 缺點：同步複雜度高、需要分散式最佳化演算法、每台機器人的運算負擔較重。

227. 代表系統：DOOR-SLAM 與 Kimera-Multi 執行點對點的位姿圖優化並具離群點剔除，以增加擴展性為代價換來更高的同步複雜度；Swarm-SLAM 提供了支援多種感測模態的 ROS 2 去中心化框架；LAMP 與 LAMP 2.0 則處理 DARPA 地下挑戰賽中的地下多機器人 SLAM（第 1 章案例 1-2）。

**架構選擇的關鍵：通訊條件**

集中式架構的缺點在於，它們需要與共同後端保持良好的通訊，而若該節點失效，整個系統就會失效；此外它們無法擴展到大型機器人團隊。

因此，只依賴機器人之間零星、間歇通訊的去中心化解法，更適合大範圍的場景。

| 架構 | 運算位置 | 優點 | 缺點 | 適用場景 |
|---|---|---|---|---|
| 集中式 | 中央伺服器 | 融合簡單、機器人端輕量 | 通訊瓶頸、單點故障 | 通訊良好的小型團隊 |
| 分散式 | 多節點協同 | 無單點故障、擴展性佳 | 同步複雜 | 中大型團隊 |
| 去中心化 | 各機器人機載 | 對通訊中斷有韌性 | 機載運算負擔重 | 通訊零星的大範圍任務 |

**表 27-1　三種協作 SLAM 架構的比較**

**實務上的混合**

真實系統往往不是純粹的某一種。例如：機器人在機載執行自己的 SLAM（去中心化的部分），但當通訊良好時把關鍵幀傳給伺服器做全域最佳化（集中式的部分）。這種混合設

計能兼顧兩者的優點，也是本書反覆強調的工程思維——不必在教科書式的分類中二選一。

## 27.4　Cloud SLAM 與 Edge SLAM

上一節談的是「機器人之間」如何分工，本節談的是「機器人與外部運算資源」如何分工。

### 三層運算的分工

| 層級 | 位置 | 延遲 | 運算能力 | 適合的任務 |
|---|---|---|---|---|
| 機載 | 機器人本身 | 極低 | 受限（第 25 章） | 即時控制、姿態估計、避障 |
| 邊緣 | 本地伺服器 / 基地台 | 低（數毫秒至數十毫秒） | 中至高 | SLAM 後端、地圖融合、AI 推論 |
| 雲端 | 遠端資料中心 | 高（數十至數百毫秒） | 極高 | 大規模地圖儲存、模型訓練、跨場域分析 |

**表 27-2　機載、邊緣、雲端三層運算的分工**

### 分工的基本原則

決定「什麼該在哪一層做」，有三個判準：

9. 即時性要求：控制迴路必須在機載（第 26.1 節的無人機案例最為明顯）；可以容忍數十毫秒延遲的任務才適合送到邊緣。

10. 資料量：傳輸大量原始資料成本高。若能在機載先做壓縮或特徵提取，再傳輸精簡後的資料，效率會好得多（這正是 27.2 節通訊效率的思路）。

11. 安全與隱私：涉及個人隱私的影像資料，是否適合傳上雲端？這在醫院、學校等場域是必須考慮的問題。

### 邊緣輔助的協作 SLAM

邊緣運算特別適合多機器人 SLAM，因為它同時解決了兩個問題：機器人端運算不足、以及集中式架構的延遲問題。

有系統以卸載繁重運算來最佳化伺服器端，同時以機載的視覺里程計維持代理的自主性。這是一個典型的分工：機器人自己做即時的前端（不依賴網路），把耗時的後端最佳化與地圖融合交給邊緣伺服器。

通訊技術的進步也在推動這個方向。有研究運用 5G 通訊以處理來自多台機器人的即時稠密體積資料，顯示高吞吐與低延遲如何讓大型感測器串流的卸載成為可能。第 5.4 節提過 5G 在機器人定位的應用，此處則是它在協作運算上的價值。

### 通訊品質作為規劃資源

一個近年重要的觀念轉變值得同學注意。有研究的核心想法是把通訊品質視為一種規劃資源，並結合多代理協作，使「導航斷崖」成為可管理的降級事件，而非硬性的運作極限。

這個思路極具啟發性：與其被動接受「這裡沒訊號」，不如主動把通訊品質納入路徑規劃——例如讓一台機器人停在訊號良好處作為中繼、或安排機器人定期回到通訊區交換資料。這與第 26.2 節「讓失效逐級降級」的設計哲學一脈相承。

## 27.5　多台機器狗校園巡檢

本節把前面的技術整合成一個具體的系統規劃，這也是第 31 章專題四的基礎。

### 為什麼需要多台

以一般大學的校園規模估算：一台機器狗以每秒 0.5 公尺的巡檢速度，要走遍一棟系館的所有走廊與樓層，可能需要數小時，而電池續航往往不足一小時。多台機器狗分頭進行，才能在單次充電內完成任務。

### 系統架構

**圖 27-4　多機器狗校園巡檢的系統架構**

【機載層】每台機器狗

- 光達慣性 SLAM（第 18 章）或視覺慣性 SLAM（第 23 章）
- 即時避障與步態控制
- 語意感知：YOLO 偵測巡檢目標（第 24 章）

↓ 傳送：關鍵幀描述子、位姿圖、巡檢發現

【邊緣層】校園內的本地伺服器

- 跨機器狗閉環偵測與地圖融合（27.2 節）
- 全域位姿圖優化（第 12 章）
- 任務分配與路線協調

↓ 傳送：精簡後的地圖、巡檢報告

【雲端層】（選用）

- 長期地圖儲存與跨時間比對（變化偵測）
- AI 模型訓練與更新
- 管理介面與報表

### 任務分配策略

228. 依區域劃分：最簡單——每台機器狗負責一個樓層或一個區域。優點是互不干擾、實作容易；缺點是無法動態調整，若某區提早完成，該機器狗就閒置。

229. 依邊界動態分配：第 24 章案例 24-3 提過的做法——把已探索與未探索的交界（邊界）動態分配給最合適的機器人。效率較高，但需要即時的協調機制。

230. 依能力分配：異質團隊的做法——機器狗負責樓梯與室內，無人機負責中庭與屋頂，輪型機器人負責平坦的長廊（續航最佳）。

**實務課題**

12. 起始座標對齊：讓所有機器狗從同一個充電站出發，或在起點放置 AprilTag（第 24.3 節），即可直接建立共同的座標系，大幅簡化地圖融合。這是一個「用簡單方法避開困難問題」的好例子。

13. 通訊覆蓋：校園的 Wi-Fi 未必處處覆蓋（地下室、樓梯間）。系統必須能容忍斷線，並在恢復連線時補傳資料——這正是 27.3 節去中心化架構的價值。

14. 安全與互相避讓：多台機器狗可能在走廊相遇，需要協調通行。此外還要考慮與行人的互動（第 24 章的社交感知導航）。

15. 電池管理：機器狗續航短，系統需規劃充電輪替——例如兩台巡檢、一台充電，輪流交替以維持持續覆蓋。

## 27.6　SLAM 與 AI Agent 協作

第 24.5 節介紹過 VLM 輔助導航，本節聚焦 AI 在多機器人協調中的角色。

### VLM 作為全域規劃器

第 24 章案例 24-3 已詳述過這個方向：VLM 不做低階控制，而是扮演「指揮官」——看著全域地圖，運用常識推理判斷哪些方向最可能找到目標，再把任務分配給不同的機器人。

這個分工的優雅之處在於：VLM 的優勢是語意理解與常識推理（慢但聰明），機器人的優勢是即時感知與控制（快但不理解意義）。讓各自做擅長的事，是最合理的架構。

### AI Agent 在多機系統中的三種角色

16. 任務規劃者：把高階目標（「巡檢三樓所有教室」）分解為具體的子任務，並分配給各機器人。這需要理解場域的語意結構（語意地圖，第 3、24 章）。

17. 異常判讀者：巡檢機器人回傳大量影像，由人工逐一檢視不切實際。AI Agent 能篩選出「需要人類注意」的異常（裂縫、漏水、堵塞的通道），大幅減輕人力負擔。

18. 協調仲裁者：當多台機器人的需求衝突（都想去同一區域、都需要充電），Agent 依全域目標做出決策。

**語意地圖的共享**

多機器人協作的一個進階方向，是共享語意而非只共享幾何。有系統把語意資料納入產出的地圖中，能在僅依賴局部通訊的情況下，即時建立全域一致的度量語意三維網格模型，並對錯誤的閉環具有韌性。

共享語意的價值在於：機器人 A 辨識出「這是電梯」之後，機器人 B 不需要重新辨識，直接沿用即可。這不只省運算，也讓整個團隊對環境的理解保持一致。

**務實的提醒**

第 24.5 節強調過的限制，在多機器人系統中同樣適用甚至更嚴重：

231. 延遲：VLM 推論耗時，若每個決策都要等它，系統反應會過慢。因此 AI 應處理「秒級以上」的決策，即時避讓仍由傳統方法負責。

232. 幻覺：AI 判斷錯誤可能導致整個團隊往錯誤方向探索。系統應保留幾何層的驗證與人類的監督介面。

233. 通訊：AI 決策通常在邊緣或雲端執行，這意味著它依賴通訊。斷線時，機器人必須能退回預設的自主行為（27.4 節的降級思維）。

## 27.7　隊形、通訊、資料同步與安全

本節收攏多機器人系統的四項工程課題，它們雖不像演算法那樣光鮮，卻往往決定系統能否真正運作。

**一、隊形與空間協調**

234. 避免碰撞：多台機器人在同一空間移動，必須避免互撞。做法包括共享即時位置、劃分行進區域、以及在狹窄處（走廊、門口）設定通行規則。

235. 維持通訊鏈：若團隊需要保持連線，隊形必須考慮通訊範圍。有時會刻意讓一台機器人停在中繼位置（27.4 節提過的把通訊視為規劃資源）。

236. 覆蓋效率：隊形也影響探索效率。分散能加快覆蓋，但過度分散會失去互相支援與通訊的能力。

**二、通訊**

237. 頻寬限制：原始影像與點雲的資料量極大，不可能全部傳輸。實務上傳輸的是精簡的表示——關鍵幀描述子、位姿圖、稀疏特徵（27.2 節的效率考量）。

238. 間歇連線：實際場域中，連線會斷。系統設計必須假設「隨時可能斷線」，並在恢復時能補傳與重新同步。

239. 通訊拓樸：是所有機器人都能互相通訊，還是只有相鄰的能？這直接影響架構選擇（27.3 節）。

**三、資料同步**

240. 時間同步：這是多機器人系統最容易被低估的難題。每台機器人的時鐘略有差異，若不校準，融合時會出現時間錯位。第 5、10 章反覆強調過時間同步的重要性，在多機系統中影響更大。常用做法是網路時間協定或以 GNSS 授時（第 5.1 節提過 GNSS 同時提供精確授時）。

241. 座標同步：即 27.1 節提過的座標系統一問題。除了靠地圖融合估計，也可用共同的起始點或環境中的已知標記（27.5 節的實務做法）。

242. 版本一致：當地圖被更新（例如閉環修正後），所有機器人都需要拿到最新版本。若某台機器人用著舊地圖行動，可能導致衝突。

**四、安全**

243. 通訊安全：機器人之間的通訊若被竊聽或竄改，攻擊者可能注入錯誤的地圖資料，誤導整個團隊。加密與身分驗證是必要的。

244. 資料隱私：巡檢機器人拍攝的影像可能包含人臉與私人空間。在校園、醫院等場域，必須明確規範資料的蒐集、儲存與存取權限。這不只是技術問題，也是倫理與法規問題。

245. 實體安全：多台機器人同時運作，對周圍人員的風險增加。系統需有全體緊急停止機制，且每台機器人都應有獨立的安全底線（即使失去通訊也不會失控）。

246. 故障隔離：一台機器人故障或行為異常時，系統應能將其隔離，避免它的錯誤資料污染全域地圖——這正是 27.2 節離群點剔除在系統層級的延伸。

### 第七篇的收束

第七篇從第 25 章的毫瓦級微型平台出發，經過第 26 章的無人機導航，到本章的群體協作，呈現了一條清晰的線索：當單一機器人的能力受限時，答案不是把它做得更大更強，而是讓它更小、更省電、然後用更多台協同工作。

這個思路與自然界的策略不謀而合——螞蟻與蜜蜂個體能力有限，群體卻能完成驚人的任務。本篇介紹的技術，正是讓機器人團隊具備這種集體智慧的基礎。

下一篇，我們回到最實際的層面：如何把這一切真正跑起來。第八篇「ROS / Python / C++ 實作」將帶同學建置開發環境、實作 2D 光達建圖與導航、執行視覺 SLAM，並完成完整的專題。理論走到這裡，是時候讓機器人真的動起來了。

## 27.8　多機器人 SLAM 的三個實務邊界

本章量了三件事，每一件都指向一個實務上的邊界。本節把它們收攏起來。

### 一、台數的邊界：由場域決定，不由預算決定

圖 27-1 顯示加速比在六台左右飽和。這個數字不是普適的——它由場域的大小、拓樸複雜度與感測半徑共同決定。同學在規劃機群時，應該先用自己的場域跑一次圖 27-1 的模擬，找出屬於自己的飽和點，再決定要買幾台。

一個粗略的估計方式是：把場域的總路徑長度除以「單台在續航時間內能走的距離」，得到的就是「至少需要幾台」；而飽和點通常落在這個數字的一到兩倍之間。超過飽和點之後增加的台數，價值在容錯與分工，不在速度。

## 二、信任的邊界：寧可少融合，不可錯融合

圖 27-2 給出的數字很極端：一條錯誤邊讓系統比完全不融合還糟兩倍。這個不對稱決定了多機器人系統的設計原則——融合的門檻應該訂得比單機閉環更嚴格。

具體的做法包括：要求更多的內點數才接受跨機匹配、要求連續多幀都偵測到同一個匹配、以及在接受之前先用幾何一致性檢查（例如三台機器人之間的變換要能首尾相接回到原點）。這些檢查都會讓系統錯過一些正確的閉環，但依圖 27-2 的數字，這個代價遠小於接受一條錯誤邊。

## 三、頻寬的邊界：先排序，再壓縮

面對頻寬限制，直覺的做法是壓縮資料——降低影像解析度、減少特徵點數、稀疏化關鍵幀。圖 27-3 指出另一個常被忽略的維度：挑選。在傳同樣多資料的前提下，挑對要傳的內容可以讓效果提升兩到三倍。

務實的順序是：先排序（決定傳哪些），再壓縮（決定每一筆傳多細）。順序顛倒的話，你會把頻寬平均花在所有候選上，結果每一條都傳得不夠清楚，反而都用不上。

| | 本章量到的數字 | 對設計的意義 |
|---|---|---|
| 台數 | 六台飽和於 11 倍加速 | 飽和點由場域決定，先模擬再採購 |
| 共享地圖 | 八台時縮短 83% 時間 | 地圖融合是多機系統的前提，不是加分項 |
| 錯誤閉環 | 一條就讓 ATE 由 0.119 升到 1.238 公尺 | 融合門檻要比單機閉環更嚴 |

| | | |
|---|---|---|
| 穩健核 | 全數擋下，但無錯誤時精度由 0.119 降為 0.469 | 保險有保費，兩階段處理可兼顧 |
| 候選排序 | 10 條排序過 ≒ 30 條隨機 | 先排序再壓縮 |

**表 27-3　本章三個實驗的設計意涵**

## 公式或流程

### 一、跨機器人的座標變換

設機器人 A 與 B 各自的世界座標系為 W_A、W_B。地圖融合的核心，是求出兩者之間的相對變換：

$$T_{W_A \leftarrow W_B}$$

取得這個變換後，B 的任一位姿即可轉換到 A 的座標系：

$$p_{W_A} = T_{W_A \leftarrow W_B} \cdot p_{W_B}$$

這正是第 10 章「變換組合就是矩陣連乘」在多機器人上的應用。若團隊有 N 台機器人，原則上只需 N - 1 個相對變換就能把所有地圖統一到同一座標系（構成一棵生成樹，呼應第 22.6 節的概念）。

### 二、多機器人位姿圖的目標函數

把所有機器人的位姿圖合併後，整體最佳化的目標為：

$$\min \Sigma_{機器人內部邊} \|e\|^2_\Sigma + \Sigma_{跨機器人邊} \rho(\|e\|^2_\Sigma)$$

第一項是各機器人自己的里程計與閉環約束，第二項是跨機器人的閉環約束。

請特別注意第二項外面的穩健核函數 ρ——跨機器人邊的錯誤風險遠高於機器人內部的邊（27.2 節），因此必須用穩健核函數保護。這個細節體現了「依可信度差異化處理」的思想，與第 11 章卡爾曼增益、第 12 章資訊矩陣一脈相承。

## 三、通訊量的估算

多機器人系統的通訊需求，可粗估為：

$$\text{總通訊量} \approx N \times (N-1) \times \text{每對機器人交換的資料量} \times \text{頻率}$$

注意 N 的平方成長——這正是集中式與全連通架構難以擴展的數學原因。以 10 台機器人為例，兩兩通訊的配對數是 45 對；若增加到 20 台，配對數躍升至 190 對。

這條式子解釋了 27.2 節「閉環候選優先排序」與 27.3 節「去中心化架構」的必要性：當 N 變大，必須設法讓實際通訊量遠低於這個上限——只在需要時通訊、只傳最有價值的資料。

## 四、多機器人 SLAM 的完整流程

**圖 27-5　多機器人協作 SLAM 的處理流程**

【各機器人機載】

每台獨立執行單機 SLAM（第 13–25 章）→ 各自的位姿圖與地圖

↓ 提取精簡表示（關鍵幀描述子、稀疏特徵）

【通訊】

依優先順序交換候選資訊（頻寬有限，只傳高價值的）

↓

【跨機器人閉環偵測】

地點辨識比對 → 幾何驗證 → 離群點剔除

↓ 取得跨機器人的相對變換

【地圖融合】

建立跨機器人邊 → 合併位姿圖

↓

【全域最佳化】

位姿圖優化（含穩健核函數）→ 全域一致的地圖

↓

【分發】把修正後的結果回傳各機器人

## 案例

### 案例 27-1　地下挑戰賽：多機器人 SLAM 的極限考場

第 1 章案例 1-2 介紹過 DARPA 地下挑戰賽，此處從多機器人的角度重看這場比賽。

比賽要求機器人團隊在人造隧道、地下城市與天然洞穴中自主探索、建圖並尋找目標物。

這個場域幾乎集合了本章所有挑戰的最惡劣版本：

247. 通訊極差：地下環境無線電訊號衰減嚴重，機器人往往在深入後就完全失聯——這正是 27.3 節「零星、間歇通訊」的極端情況。

248. 環境退化：隧道的幾何高度重複（第 17.3 節的結構退化），感知混淆風險極高，錯誤閉環的機率大增。

249. 異質團隊：參賽隊伍普遍組合輪型機器人、四足機器狗與無人機，各自適應不同的地形（27.1 節的能力互補）。

這場比賽直接催生了多個重要的系統。LAMP 與 LAMP 2.0 正是為了處理 DARPA 地下挑戰賽中的地下多機器人 SLAM 而發展；LAMP 2.0 引入以圖神經網路為基礎的集中式優先排序機制，預測每個跨機器人閉環候選經過位姿圖優化後的結果。

這個「用 AI 預測哪些閉環候選值得處理」的做法極具啟發性：當資源（通訊、運算）有限時，與其平均對待所有候選，不如用學習方法預測價值再分配資源。這與 27.2 節的優先排序思想相同，只是用了不同的技術手段。

對同學的啟示是：極端場景往往是技術突破的溫床。地下挑戰賽的參賽系統，如今已成為多機器人 SLAM 的重要基礎——困難的問題逼出了好的答案。

## 案例 27-2　通訊不可靠時，系統會壞到什麼程度

多機器人 SLAM 的論文，多半在通訊良好的條件下評測。但真實場域中，通訊往往時斷時續——這對系統的影響有多大？

有研究專門量化了這個問題，結果相當驚人：相對於可靠通訊的基準，Swarm-SLAM 的誤差增加為 5.08 倍、Kimera-Multi 為 6.17 倍。

換句話說，同一套系統在通訊不可靠時，誤差可能惡化五到六倍。這個數字提醒我們：實驗室的漂亮數據，未必能在真實場域重現。

這也解釋了為何近年的研究越來越重視通訊條件。第 27.4 節提過的「把通訊品質視為規劃資源」正是一個回應：與其被動接受通訊不良的後果，不如主動規劃路線以維持連線，或設計能在斷線時優雅降級的架構。

對做專題的同學而言，這是一個重要提醒：評測你的多機器人系統時，務必加入「通訊不良」的情境測試。只在理想條件下測試，會嚴重高估系統的實際能力。這與第 14.6 節「常見失敗案例」、第 26.5 節「挑戰測試」的精神一致——主動尋找失效邊界，是負責任的工程實踐。

### 案例 27-3　從模擬到現場：異質團隊的開發工具

多機器人系統的開發有一個現實困難：硬體成本。十台機器狗的價格對多數實驗室都是沉重負擔，而且調度十台機器人做實驗的人力與場地成本同樣可觀。

模擬因此成為多機器人研究的必要工具。近年已有專為異質多機器人設計的開源模擬環境出現。例如有開源模擬器與資料蒐集管線，建立在虛幻引擎 5 的模擬器之上，解決了先前框架的關鍵架構限制，以支援在大規模、照片級真實、動態環境中同時運作無人空中與地面載具；它引入了新的路徑點追蹤地面載具控制器以對應既有的無人機控制介面，並為異質平台提供共享的導航堆疊，涵蓋建圖、可通行性分析、規劃與控制。

這類工具對教學特別有價值：同學能在模擬中建立十台、二十台機器人的團隊，測試任務分配、地圖融合與通訊策略，完全不需要對應數量的實體硬體。等到演算法在模擬中驗證可行，再用兩三台實體機器人驗證關鍵環節即可。

這正是本書從第 8 章就建議的學習路徑：先模擬、再便宜硬體、後追求規模。多機器人系統把這個原則的價值放大到極致——沒有模擬，多機器人研究幾乎寸步難行。

## 實作活動

### 實作 27-1　多機器人協作建圖模擬

目的：透過模擬完整體驗多機器人 SLAM 的核心議題——地圖融合、通訊限制、任務分配與錯誤閉環的風險。本實作以模擬為主，不需多台實體機器人。

**第一部分：兩機器人地圖融合（全體）**

137. 沿用第 25 章實作 25-1 的模擬環境（或第 13 章的 2D 建圖模擬），建立兩台虛擬機器人，各自從房間的不同角落出發，以不同路線探索並建立自己的地圖。

138. 觀察問題：把兩張地圖直接疊在一起顯示，會發現它們完全對不上——因為各自以自己的起點為原點（27.1 節的座標系統一問題）。

139. 人工融合：假設你「知道」兩台機器人起點的相對位置，依公式一做座標變換，把 B 的地圖轉到 A 的座標系，再疊合顯示。觀察融合後的地圖是否完整。

140. 自動估計變換：讓兩台機器人的探索路線有部分重疊，用第 17 章的 ICP 對兩張地圖（或重疊區域的點）做配準，自動估出相對變換。與真值比較誤差。

141. 加入漂移：讓兩台機器人的位姿估計都帶有累積誤差，重新融合，觀察地圖如何錯位，並用位姿圖優化（第 12 章）修正。

**第二部分：通訊限制實驗（全體或大學部）**

19. 設定通訊範圍：規定兩台機器人只有在距離小於某個值時才能交換資料。模擬它們各自探索、偶爾相遇的過程。

20. 比較三種通訊條件：全時連線、只在相遇時通訊、以及隨機斷線（例如每步有 30% 機率無法通訊），比較最終地圖的品質差異。

21. 量化影響：計算三種條件下的地圖誤差，與案例 27-2 提到的「誤差增加五到六倍」對照，看看你的模擬是否呈現類似趨勢。

22. 資料量統計：記錄三種條件下的總通訊量，思考「通訊量」與「地圖品質」的取捨。

**第三部分：任務分配與錯誤閉環（大學部）**

23. 探索效率比較：讓 1 台、2 台、4 台機器人分別探索同一個房間直到全區覆蓋，記錄各自所需的步數。畫出「機器人數量 vs 完成時間」的曲線，觀察效益是否線性成長（提示：通常不會，因為會有重複探索）。

24. 任務分配策略：比較「隨機探索」與「依區域劃分」兩種策略的效率差異，對照 27.5 節的討論。

25. 錯誤閉環實驗：人為加入一條錯誤的跨機器人閉環邊（例如把 A 的某位姿錯誤地連到 B 的不相關位姿），執行位姿圖優化，觀察地圖如何被破壞。

26. 穩健化：在最佳化中加入穩健核函數（可用 scipy 的 soft_l1 等損失函數），重跑一次，觀察錯誤閉環的破壞是否被抑制。這直接驗證公式二中 ρ 的作用。

**討論問題**

54. 未做座標變換時，兩張地圖疊起來是什麼樣子？這說明了多機器人 SLAM 相對單機的什麼新問題？

55. ICP 自動估計的相對變換誤差有多大？若兩台機器人的探索路線幾乎沒有重疊，會發生什麼？

56. 通訊條件從「全時連線」變成「只在相遇時通訊」，地圖誤差增加了幾倍？與案例 27-2 的數據相比如何？

57. 機器人從 1 台增加到 4 台，完成時間縮短為幾分之一？為什麼不是四分之一？

58. （大學部）加入錯誤閉環後，地圖的哪些部分被破壞得最嚴重？穩健核函數把破壞抑制了多少？這對真實系統的設計有什麼意涵？

**延伸挑戰（大學部）**

其一：實作簡易的閉環優先排序——當通訊量有限，只能傳輸部分閉環候選時，設計一個評分機制（例如依據「這條閉環能連接多遠的兩段軌跡」或「涉及的機器人是否已有其他連結」），優先傳輸高分候選，比較它與隨機選擇的效果差異。這對應 27.2 節的核心思想。

其二：異質團隊模擬——加入一台「速度快但感測範圍小」與一台「速度慢但感測範圍大」

的機器人，設計任務分配策略以發揮各自優勢，並與同質團隊比較效率。這呼應 27.1 節的能力互補與第 26.7 節的地空協作。

## 延伸閱讀

78. Y. Tian et al., "Kimera-Multi: Robust, Distributed, Dense Metric-Semantic SLAM for Multi-Robot Systems," IEEE T-RO, 2022.（分散式多機器人語意 SLAM 的代表作，程式碼公開）

79. P.-Y. Lajoie and G. Beltrame, "Swarm-SLAM: Sparse Decentralized Collaborative SLAM Framework for Multi-Robot Systems," IEEE RA-L, 2024.（ROS 2 去中心化框架，含閉環優先排序技術）

80. Y. Chang et al., "LAMP 2.0: A Robust Multi-Robot SLAM System for Operation in Challenging Large-Scale Underground Environments," IEEE RA-L, 2022.（DARPA 地下挑戰賽的代表系統）

81. P. Schmuck and M. Chli, "CCM-SLAM: Robust and Efficient Centralized Collaborative Monocular SLAM," Journal of Field Robotics, 2019.（集中式協作 SLAM 的經典）

82. P.-Y. Lajoie et al., "DOOR-SLAM: Distributed, Online, and Outlier Resilient SLAM for Robotic Teams," IEEE RA-L, 2020.（分散式架構與離群點剔除）

83. awesome-Collaborative-SLAM 開源清單（GitHub）.（協作 SLAM 論文與程式碼的完整整理，研究入門的好起點）

84. S. Macenski et al., "Robot Operating System 2: Design, Architecture, and Uses in the Wild," Science Robotics, 2022.（ROS 2 的多機器人支援，第八篇實作的基礎）

## 小結

78. 多機器人 SLAM 的四項優勢為速度、容錯、視角互補與能力互補；三大新挑戰為座標系統一、資料關聯範圍擴大、以及通訊限制。其核心技術是單機技術（SLAM、地點辨識、位姿圖優化、離群點剔除）在多座標系與多資料來源上的擴充。

79. 地圖融合分三步驟：偵測相遇、估計相對變換、融合與最佳化；最大風險是錯誤的跨機器人閉環——候選空間更大、感知混淆更嚴重、後果更嚴重，因此穩健的離群點剔除是必備元件，而閉環候選的優先排序能在有限頻寬下獲得最大效益。

80. 三種架構各有取捨：集中式融合簡單但有通訊瓶頸與單點故障、無法擴展到大型團隊；分散式與去中心化執行點對點位姿圖優化，擴展性佳且對通訊中斷有韌性，代價是同步複雜度與機載運算負擔；只依賴零星間歇通訊的去中心化解法更適合大範圍場景。

81. 機載、邊緣、雲端三層依即時性、資料量與隱私分工；邊緣輔助的做法是以機載視覺里程計維持自主性、把繁重運算卸載到伺服器；近年更把通訊品質視為規劃資源，使導航斷崖成為可管理的降級事件而非硬性極限。

82. 多機器狗校園巡檢採機載、邊緣、雲端三層架構；任務分配可依區域、依邊界動態或依能力；實務課題包含起始座標對齊（可用共同充電站或 AprilTag 簡化）、通訊覆蓋、互相避讓與電池輪替。

83. AI Agent 在多機系統中扮演任務規劃者、異常判讀者與協調仲裁者；共享語意而非只共享幾何能讓團隊對環境的理解保持一致，但須留意延遲、幻覺與通訊依賴三項限制。

84. 四項工程課題：隊形需兼顧避撞、通訊鏈與覆蓋效率；通訊須面對頻寬限制與間歇連線；資料同步含時間、座標與版本一致；安全涵蓋通訊加密、資料隱私、實體安全與故障隔離。

## 承先啟後

本章把「多來幾台」這件事的三個關鍵參數量化了：協同探索的加速比在六台左右飽和於 11.3 倍，而且共享地圖與否的差距極大——八台時共享讓完成時間縮短 83%；地圖融合的正面價值是把 ATE 從 0.576 公尺降到 0.119 公尺，但只要混入一條錯誤的跨機邊就會惡化到 1.238 公尺，比完全不融合還糟兩倍，而穩健核能把它穩定在 0.47 公尺；頻寬方面，依代數連通度排序後只需十條跨機閉環，就能達到隨機挑選三十條的效果。

第七篇到此完成。從第 25 章的毫瓦級微型平台、第 26 章的無人機導航，到本章的群體協作，呈現了一條清晰的線索：當單一機器人的能力受限時，答案不是把它做得更大更強，而是讓它更小、更省電，然後用更多台協同工作——這與螞蟻和蜜蜂的策略不謀而合。

下一篇，我們回到最實際的層面：如何把這一切真正跑起來。第八篇「ROS / Python / C++ 實作」將帶同學建置開發環境、實作 2D 光達建圖與導航、執行視覺 SLAM，並完成完整的專題與評量。理論走到這裡，是時候讓機器人真的動起來了。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第八篇 ROS / Python / C++ 實作

## 第 28 章

## SLAM 程式設計環境

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

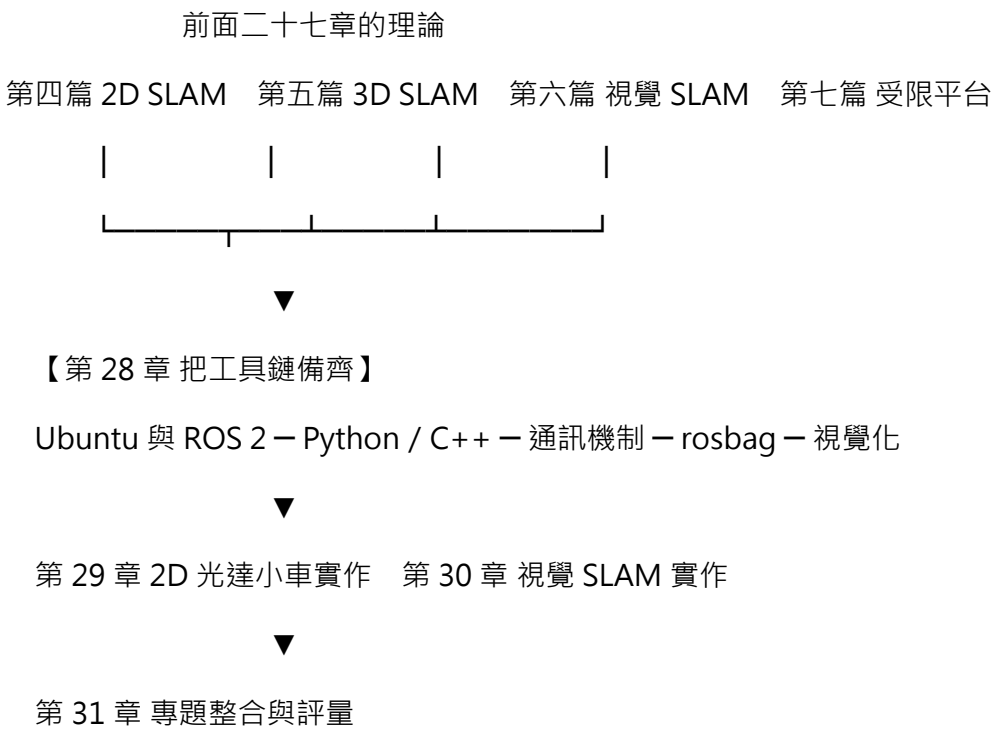
## 本章在全書的位置

前面二十七章講的都是「該怎麼做」。從本章起，第八篇要處理「怎麼真的跑起來」。

這一篇的性格與前面完全不同。它不談新的演算法，而談那些會讓專題卡上三天的事情：版本對不上、話題沒有資料、座標轉換樹斷掉、錄了兩小時的資料發現硬碟滿了。這些問題在論文裡不會出現，卻佔掉實作時間的一大半。

本章的三張圖都是真的跑出來的量測，而不是從規格表抄來的數字：圖 28-1 用同一段掃描配準的核心，分別以純 Python、NumPy 與 C++ 實作並計時；圖 28-2 用真實的行程間通訊量出「話題」與「服務」兩種模式的差別；圖 28-3 則產生逼真的感測資料，實測錄製一小時要多少空間。

### 圖 28-0　第 28 章的連結地圖




## 學習目標

前七篇建立了完整的理論與系統知識。從本篇開始，我們回到最實際的層面：如何把這一切真正跑起來。本章是第八篇的入口，負責把開發環境備妥、把工具鏈打通——這是所有後

續實作的地基。學完本章之後，同學應能夠達成下列目標：

78. 說明 Ubuntu 與 ROS 2 的版本對應關係，並完成環境安裝。
79. 說明 Python 與 C++ 在 ROS 2 中的分工，並理解 CMake 與建置系統的角色。
80. 說明話題、服務、動作三種通訊機制的差異與適用場合。
81. 使用 rosbag2 錄製與重播感測資料，並理解 MCAP 格式的意義。
82. 使用 RViz2、Gazebo 與 Foxglove 進行視覺化、模擬與除錯。
83. 說明閱讀 GitHub 開源專案的系統性方法。
84. 規劃從高中積木式實作到大學原始碼實作的學習路徑。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 長期支援版 | LTS | 有較長維護期限的穩定版本，適合教學與產品。 |
| 工作空間 | Workspace | 存放 ROS 套件原始碼與建置產物的目錄結構。 |
| 套件 | Package | ROS 中程式碼組織的基本單位。 |
| 建置工具 | colcon | ROS 2 用來編譯工作空間中所有套件的工具。 |
| 節點 | Node | 執行特定功能的獨立程式（第 3 章）。 |
| 話題 | Topic | 節點間以發布訂閱模式交換資料的通道。 |
| 服務 | Service | 一問一答式的同步通訊機制。 |
| 動作 | Action | 可回報進度、可取消的長時間任務通訊機制。 |
| 啟動檔 | Launch File | 一次啟動多個節點並設定參數的腳本。 |
| 資料錄製 | rosbag2 | 錄製與重播 ROS 話題資料的工具。 |
| 訊息擷取格式 | MCAP | 自我描述、可快速隨機存取的記錄檔格式。 |
| 視覺化工具 | RViz2 | ROS 2 內建的三維視覺化工具。 |

| 中文名詞 | 英文名詞 | 概念說明 |
| --- | --- | --- |
| 模擬器 | Gazebo | 提供物理模擬與感測器模擬的環境。 |

# 28.1 Ubuntu 與 ROS / ROS 2

第 3.5 節介紹過 ROS 的角色與核心概念，本節聚焦環境建置的實務。

### 為什麼是 Ubuntu

ROS 2 支援多種作業系統，但 Ubuntu 是所謂的第一級平台（Tier 1）——也就是官方主要開發與測試的平台。實務上，絕大多數開源 SLAM 專案的說明文件、相依套件與社群支援都以 Ubuntu 為前提。因此本書的實作一律以 Ubuntu 為基準。

### 版本對應：最容易踩的第一個坑

ROS 2 與 Ubuntu 的版本是綁定的——每個 ROS 2 發行版只支援特定的 Ubuntu 版本。裝錯版本會導致一連串難以排查的相依性錯誤，這是初學者最常見的挫折來源。

ROS 2 每年 5 月發布新版本，偶數年為長期支援版（LTS）。截至本書撰寫時，主要的版本對應如下：

| ROS 2 發行版 | 發布年 | Ubuntu 版本 | 類型 | 備註 |
|---|---|---|---|---|
| Humble Hawksbill | 2022 | 22.04 | LTS | 支援至 2027 年 5 月，最成熟穩定 |
| Jazzy Jalisco | 2024 | 24.04 | LTS | rosbag2 預設改用 MCAP 格式 |
| Kilted Kaiju | 2025 | 24.04 | 一般版 | rosbag2 效能改善 |
| Lyrical Luth | 2026 | 26.04 | LTS | 最新的長期支援版 |

**表 28-1 ROS 2 發行版與 Ubuntu 版本對應**

### 版本選擇建議

- 教學與初學：建議選擇成熟的 LTS 版本。Humble 是目前教學資源最豐富的選擇——網路上絕大多數教學、開源專案與問答都以它為基準，遇到問題最容易找到答案。

- 新專案：若要長期維護，可選最新的 LTS 版本以獲得較長的支援期。
- 沿用既有專案：直接對齊該專案指定的版本，不要嘗試「順便升級」——這往往會浪費大量時間在相依性問題上。
- 重要提醒：全班應統一版本。不同版本之間的錄製資料未必能互通，教學時版本不一致會造成極大困擾。

### 安裝方式的三種選擇

250. 原生安裝：直接在實體電腦上安裝 Ubuntu 與 ROS 2。效能最好、硬體支援最完整，是連接實體機器人時的必要選擇。缺點是需要重灌或雙系統。

251. 虛擬機：在 Windows 或 macOS 上以虛擬機執行 Ubuntu。安裝容易、不影響原系統，適合純軟體的模擬練習。缺點是 3D 加速效能較差（Gazebo 可能會卡）、USB 裝置連接較麻煩。

252. 容器（Docker）：以容器封裝完整環境。優點是環境一致、可快速重建、不污染主系統——這對教學特別有價值，教師可以提供統一的容器映像，免去全班各自安裝的混亂。缺點是需要額外設定才能使用圖形介面與 USB 裝置。

### 教學上的實務建議

依多年教學經驗，建議的策略是：高中與大學部入門階段使用容器或虛擬機，讓所有同學在一致的環境中操作模擬；進入實體機器人實作階段，再改用原生安裝。這能把「環境問題」的干擾降到最低，讓課堂時間花在 SLAM 本身而非除錯安裝。

## 28.2 Python、C++、CMake

ROS 2 同時支援 Python 與 C++，兩者在 SLAM 開發中有清楚的分工。

### 兩種語言的分工

| 面向 | Python(rclpy) | C++(rclcpp) |
|---|---|---|
| 開發速度 | 快，適合快速原型 | 較慢，需編譯 |
| 執行效能 | 較慢 | 快，適合高頻運算 |
| 適合的任務 | 控制邏輯、資料分析、視覺化 | SLAM 核心、影像處理、即時控制 |
| 學習門檻 | 低 | 較高（記憶體管理、模板） |
| 本書用途 | 實作練習、資料處理 | 閱讀開源 SLAM 原始碼 |

**表 28-2　Python 與 C++ 在 ROS 2 中的分工**

一個實用的原則是：先用 Python 把邏輯跑通，若效能不足再用 C++ 重寫關鍵部分。同學會發現主流的 SLAM 系統（ORB-SLAM3、Cartographer、LOAM 系列）幾乎都是 C++ 寫的——因為它們需要處理高頻的影像與點雲資料，Python 的效能無法勝任。

### CMake 與建置系統

C++ 專案需要編譯，而管理編譯過程的工具就是 CMake。它負責：找出相依的函式庫在哪裡、決定編譯順序、產生實際的編譯指令。

對 SLAM 學習者而言，CMake 最常出現的場合是「編譯開源專案時出錯」。第 22.7 節提過 ORB-SLAM2 在新系統上可能編不過，問題往往就出在 CMakeLists.txt 中指定的函式庫版本與系統上的版本不符。

- 必備技能：看懂 CMakeLists.txt 中的 find_package（尋找相依套件）與 target_link_libraries（連結函式庫）兩個指令，就能解決大部分編譯問題。
- 常見錯誤：找不到套件（需安裝或指定路徑）、版本不符（需修改版本要求或安裝對應版本）、C++ 標準不符（需調整編譯選項）。

### 工作空間與 colcon

ROS 2 的程式碼組織成「工作空間」，其中放置多個「套件」。建置工具 colcon 會自

動分析套件之間的相依關係，依正確順序編譯全部套件。

典型的工作流程是：建立工作空間目錄 → 把套件原始碼放進 src 子目錄 → 執行 colcon 建置 → 載入環境設定（source）→ 執行節點。這個流程同學會在整個第八篇反覆使用。

### 環境設定的常見陷阱

- 忘記 source：建置完成後必須載入環境設定，系統才找得到新編譯的套件。這是最常見的「明明編譯成功卻找不到節點」原因。
- 多個工作空間疊加：若同時有多個工作空間，載入順序會影響哪個版本的套件被使用。出現詭異行為時，先檢查環境變數。
- 修改程式後未重新建置：Python 節點在某些設定下可以直接生效，C++ 則一定要重新編譯。這個差異常讓初學者困惑。

## 量一量：Python 與 C++ 到底差多少

表 28-2 建議「Python 做原型與整合、C++ 做效能關鍵」。這個分工是對的，但「效能關鍵」的界線在哪裡？本小節用一段真實的程式碼把它量出來。

測試的對象是掃描配準的內層迴圈——把 N 個點依位姿變換後，查距離場並累加似然值（第 17 章的似然場評分，第 19、25 章的實驗都用過）。同一段邏輯寫三個版本：純 Python 迴圈、NumPy 向量化、以及 C++（g++ 加 -O2 最佳化），在同一台機器上計時。

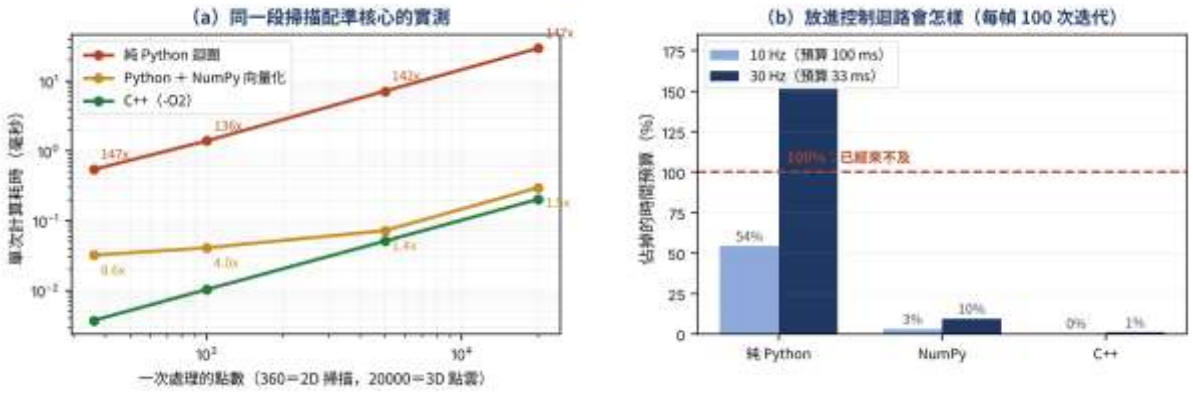



**圖 28-1　同一段運算在三種語言下的實測效能**

（a）是單次計算的耗時，兩軸都是對數刻度。以 360 點（典型的 2D 光達掃描）為例：C++ 是 0.0037 毫秒、NumPy 是 0.0320 毫秒、純 Python 是 0.5431 毫秒。純 Python 比 C++ 慢了 147 倍。

但真正有用的資訊是兩條紅、黃線的「相對位置會變」。純 Python 與 C++ 的倍數幾乎固定在 136 到 147 倍之間，不隨點數改變；NumPy 與 C++ 的差距卻從 360 點時的 8.6 倍，一路縮小到 20000 點（3D 點雲等級）時的 1.5 倍。

原因是 NumPy 的成本有兩塊：一塊是每次呼叫的固定開銷（型別檢查、記憶體配置），另一塊才是實際的運算。點數少的時候固定開銷佔大半，所以看起來慢；點數多的時候固定開銷被攤薄，效能就逼近 C++。這給了一條很實用的判準：NumPy 適合「一次處理一大批」，不適合「每次處理一點點但呼叫很多次」。

（b）把這些數字放回真實的控制迴路。假設掃描配準每一幀要做 100 次迭代（這是很常見的設定）。在 10 Hz（每幀 100 毫秒預算）下，純 Python 佔掉 54%、NumPy 3%、C++ 0.4%。54% 聽起來還能用，但那表示只剩一半的時間給導航、通訊與其他所有事。

到了 30 Hz（每幀 33 毫秒），純 Python 需要 163% 的時間預算——也就是根本來不及，系統會開始掉幀。NumPy 是 10%、C++ 是 1%，都還游刃有餘。這張圖給出的實務界線很清楚：純 Python 的迴圈只適合離線分析與原型驗證；一旦要進入即時迴路，最低限度是向量化。

**什麼時候真的需要 C++**

圖 28-1(b) 顯示 NumPy 在 30 Hz 下只佔 10% 的預算——對絕大多數教學與專題用途，這已經夠了。所以「一定要用 C++」這個想法，多半是過慮。

真正需要 C++ 的是三種情況：一是迭代次數遠超過 100 次（例如大規模的束調整）；二是運算無法向量化（例如有分支的樹狀搜尋、增量式的資料結構更新）；三是跑在第 25 章那種毫瓦級平台上，連 Python 直譯器都放不下。

務實的做法是：先全部用 Python 寫出能跑的版本，量出哪一段真的吃掉時間，只把那一段改寫成 C++。這與第 31 章的除錯順序是同一個原則——先量測，再優化。

## 28.3 ROS Topic、Service、Action

第 3.5 節簡介過這三種通訊機制，本節說明它們的差異與選用時機——這是設計 ROS 系統的核心判斷。

### 話題（Topic）：持續的資料流

話題採發布訂閱模式：發布者持續送出資料，訂閱者持續接收。雙方不需要知道彼此存在，只要約定好話題名稱與訊息格式。

- 特性：非同步、一對多、持續流動、不保證接收方收到。
- 適合：感測器資料（光達掃描、影像、IMU）、狀態廣播（位姿、地圖）、控制指令（速度）。
- SLAM 中的例子：光達驅動發布 /scan、SLAM 節點訂閱它並發布地圖（第 14.5 節的實作流程）。

### 服務（Service）：一問一答

服務是同步的請求回應模式：客戶端送出請求，等待伺服器回覆。

- 特性：同步、一對一、有明確的回覆、呼叫端會等待。

- 適合：短時間就能完成的查詢或設定，例如「取得目前地圖」「重設里程計」「切換模式」。
- 注意：因為呼叫端會等待，不適合用在耗時的操作——會阻塞整個節點。

### 動作（Action）：可監控的長任務

動作專為「需要花時間、且過程中想知道進度」的任務設計。它結合了三個元素：目標（要做什麼）、回饋（進行到哪了）、結果（最後如何）。

- 特性：非同步、可回報進度、可中途取消。
- 適合：導航到某個目標點、執行一段軌跡、完成一次掃描任務。
- SLAM 中的例子：第 29 章的 Nav2 導航就是以動作介面提供——你送出「去這個座標」的目標，系統持續回報剩餘距離，你也可以隨時取消。

| 機制 | 通訊模式 | 是否等待 | 可否取消 | 典型用途 |
|---|---|---|---|---|
| 話題 | 發布訂閱，一對多 | 不等待 | 不適用 | 感測器資料、狀態、指令 |
| 服務 | 請求回應，一對一 | 等待回覆 | 不可 | 快速查詢與設定 |
| 動作 | 目標回饋結果 | 不等待，可監控 | 可以 | 導航、長時間任務 |

**表 28-3　三種通訊機制的比較**

### 選用的判斷流程

遇到「這個功能該用哪種機制」時，依序問三個問題：

27. 這是持續流動的資料嗎？是 → 話題。
28. 是不是很快就能回答的一次性請求？是 → 服務。
29. 是不是耗時且需要監控進度的任務？是 → 動作。

這個判斷看似簡單，卻是 ROS 系統設計品質的關鍵。用錯機制（例如用服務執行耗時

的導航）會導致系統阻塞、難以除錯——這是實作時值得留意的細節。

### 量一量：話題與服務的代價差在哪

表 28-3 比較了三種通訊機制的特性，其中最關鍵的一欄是「是否阻塞」。這一欄看起來只是個技術細節，但它對系統設計的影響比其他所有欄位加起來還大。本小節把它量出來。

實驗方法：用真實的行程間通訊（兩個獨立的作業系統行程，以管線連接），傳送 4 KB 的訊息。話題模式是「發了就走」，不等回應；服務模式是「送出請求後等待回覆」，並讓伺服端花掉不同長度的處理時間。量測呼叫端實際被擋住多久，以及每秒能完成幾次。

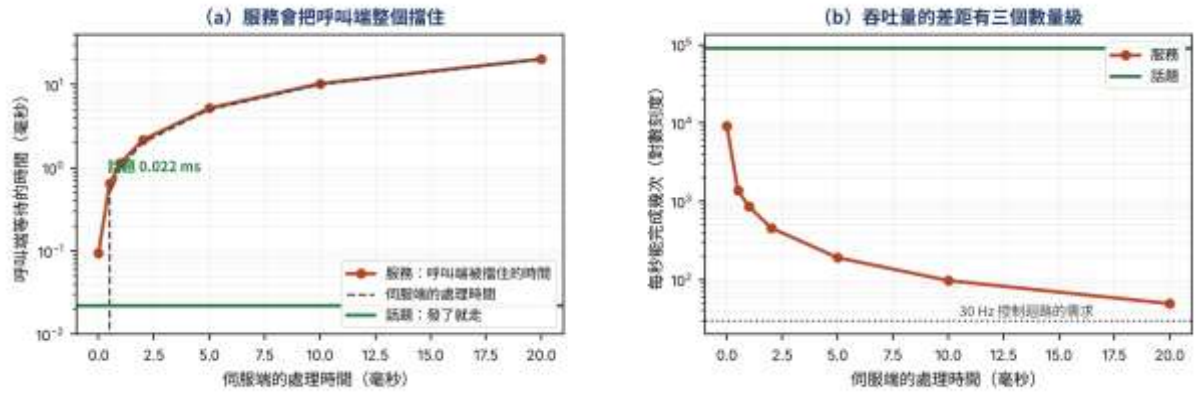



**圖 28-2　話題與服務的延遲與吞吐量實測**

（a）的綠線是話題模式：延遲 0.022 毫秒，而且不論伺服端在忙什麼都不會改變——因為發送端根本不等。紅線是服務模式，它幾乎完全貼著灰色虛線（伺服端的處理時間）：伺服端花 5 毫秒，呼叫端就被擋 5.2 毫秒；花 20 毫秒，就被擋 20.3 毫秒。

這條線看起來理所當然，卻是最容易被忽略的陷阱。很多同學會在控制迴路裡呼叫一個服務去「問一下目前的地圖」或「請求規劃一條路徑」，然後納悶為什麼機器人的反應變得遲鈍——因為整個迴路在等那個服務回來。

（b）把它換算成吞吐量，縱軸是對數刻度。話題模式每秒可以送 89,862 則；服務模式即使伺服端完全不做事也只有 9,101 次（差了十倍，純粹來自一來一回的往返成本）；伺服端花 20 毫秒時，只剩每秒 49 次。灰色點線標出 30 Hz 控制迴路的需求——伺服端的處理時間只

要超過約 30 毫秒，服務就跟不上了。

這就是「動作」這個機制存在的理由。動作把「送出請求」與「等待結果」拆開：呼叫端送出目標之後立刻回到自己的迴路，伺服端則週期性地回報進度，完成時再送出結果。它本質上是用話題的非阻塞特性，包裝出服務的「有來有回」語意。

**三個選用的判準**

依圖 28-2 的量測，選擇通訊機制時可以問三個問題：

一、下游需不需要回覆？不需要就用話題——它快了十倍以上，而且不會把上游卡住。感測資料、位姿、速度指令全部屬於這一類。

二、需要回覆，但很快就能回來嗎？若伺服端的處理時間遠小於控制週期（例如查一個參數、切換一個模式），服務是最簡單的選擇。

三、需要回覆，而且要花很久？那就必須用動作。導航到目標點、執行一段掃描、充電對接——這些動輒數秒到數分鐘的任務，用服務會把整個系統鎖死。

## 28.4 rosbag 資料錄製與重播

rosbag2 是 SLAM 開發最重要的工具之一，它讓「錄一次、分析無數次」成為可能。

### 為什麼 rosbag 如此重要

- 可重現性：同一段資料反覆重播，能公平比較不同演算法或參數——第 15 章實作 15-1 的「Cartographer vs Gmapping 對照實驗」正需要這個能力。
- 離線開發：資料錄好後，不需要機器人也能開發演算法。這對硬體有限的教學環境極為實用。
- 除錯：出問題時把資料錄下來，可以反覆重播找出原因，不必在現場一次次重跑實驗。
- 資料集建立：第 26.5 節建議的「自製場域資料集」，正是以 rosbag 錄製的。

### MCAP：新一代的記錄格式

rosbag2 的儲存格式近年有重要演進。MCAP（Message Capture）是來自 Foxglove 的記錄容器格式，自 ROS 2 Jazzy 起成為 rosbag2 的預設儲存格式。

它的特性包含：經過索引以支援快速隨機存取、內嵌訊息定義因而具自我描述性，並支援 ROS、protobuf、JSON、flatbuffers 等多種格式。

對同學而言，「自我描述」這一點特別重要：記錄檔本身就包含了訊息格式的定義，因此即使在沒有安裝對應訊息套件的電腦上，也能正確讀取——這大幅改善了資料交換與長期保存的可行性。

#### 錄製的實務要點

30. 錄什麼：SLAM 開發至少要錄感測器原始資料（/scan 或影像）、IMU、里程計、以及 TF 座標轉換。TF 常被遺漏，但缺少它重播時什麼都對不上。

31. 不要錄什麼：輸出結果（地圖、估測軌跡）通常不需要錄，因為重播時會重新產生。錄了反而佔空間並可能造成混淆。

32. 時間標記：確認所有資料的時間戳記正確——這是第 5、10、27 章反覆強調的工程細節，在重播時尤其關鍵。

33. 檔案大小：影像與點雲資料量龐大，一分鐘可能就數 GB。錄製前先估算並確認儲存空間充足。

#### 重播的常見問題

實務上常見的三個問題值得同學留意：ROS 版本不符——在某個版本錄製的資料可能無法在另一版本順暢重播；缺少 TF 轉換——許多資料集缺少轉換資訊，導致 RViz 視覺化失效；以及話題名稱不符——若你的演算法預期 /scan 但資料中是 /lidar_scan，就必須做話題重新對應。

第三點特別實用：重播時可以「改名」，讓錄好的資料餵給預期不同話題名稱的演算法，

不必修改程式碼。這是整合他人資料集時的常用技巧。

### 量一量：錄一小時要多大

28.4 節說明了 rosbag 的價值，但沒談它的代價。這一節把代價算出來，因為「硬碟滿了」是實驗現場最常見的意外之一。

實驗方法：產生逼真的合成感測資料（不是隨機雜訊——隨機資料不會壓縮，會嚴重低估壓縮率），包含平滑牆面加不連續的 2D 光達掃描、16 線的 3D 點雲、帶紋理與幾何結構的 RGB 影像、平滑的深度圖，以及 IMU。量測每種資料的原始位元率，以及用一般壓縮（zlib）與影像專用編碼（JPEG）之後的位元率。

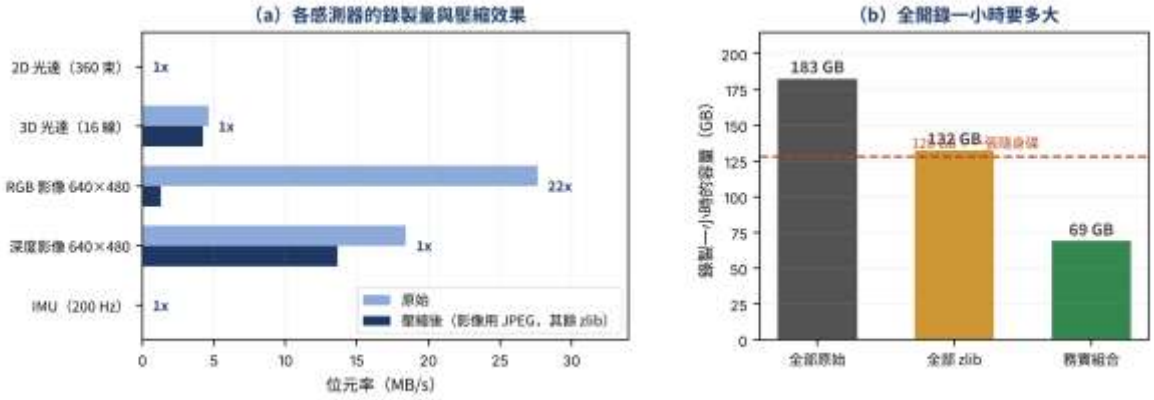



**圖 28-3　各感測器的錄製量與壓縮效果**

（a）第一個發現是量級的懸殊。2D 光達每秒只有 0.01 MB，IMU 也是 0.01 MB——這兩種資料可以錄上好幾天都不成問題。而 RGB 影像是每秒 27.65 MB、深度影像 18.43 MB、3D 點雲 4.61 MB。單一台 RGB-D 相機就佔掉了全部資料量的九成以上。

第二個發現更重要：一般壓縮對感測資料幾乎沒用。3D 點雲的壓縮率只有 1.1 倍、深度影像 1.4 倍、RGB 影像 1.5 倍。原因是這些資料本質上是高熵的浮點數或接近隨機的低位元，zlib 找不到可以利用的重複樣式。IMU 甚至出現壓縮後變大的情況（0.8 倍），因為訊息太小，壓縮格式的標頭反而佔了便宜。

真正有效的是領域專用的編碼。同一批 RGB 影像用 JPEG（品質 85）壓縮，位元率從 27.65

MB/s 降到 1.28 MB/s——22 倍。這個對比說明了一件事：壓縮的效果取決於編碼器懂不懂資料的結構，而不是取決於壓縮等級調多高。

（b）把它換算成實際的容量。所有感測器全開、完全不壓縮，錄一小時是 183 GB；全部套上 zlib 是 132 GB，只省下 28%；而「影像用 JPEG、其餘用 zlib」的務實組合是 69 GB，只有原始的 38%。紅色虛線是 128 GB——一支常見隨身碟的容量。也就是說，不做取捨的話，一支隨身碟連一小時都錄不滿。

**錄製前的三個決定**

依圖 28-3 的數字，規劃錄製時應該先做三個決定。

一、哪些話題真的需要錄？多數 SLAM 的離線分析只需要光達、IMU 與里程計——那加起來不到每秒 5 MB，一小時只有 17 GB。影像除非要做視覺 SLAM 或語意標註，否則不必全程錄。

二、影像要用什麼格式？壓縮影像話題（JPEG）與原始影像話題差了二十倍。若是為了之後跑特徵點法，JPEG 品質 85 以上通常足夠；若是要做光度法（第 20.2 節的直接法），就必須錄原始影像。

三、要不要降頻？30 fps 的影像對多數離線分析是浪費，10 fps 往往就夠了，這一刀直接砍掉三分之二。

## 28.5　RViz、Gazebo、Foxglove

本節介紹三個貫穿第八篇的工具：視覺化、模擬與除錯分析。

### RViz2：看見機器人的世界

RViz2 是 ROS 2 內建的三維視覺化工具，也是本書從第 10 章開始就反覆提到的除錯利器。它能同時顯示：

- 感測器資料：光達掃描點、點雲、影像。

- 座標框架：TF 樹的各個座標軸（第 10 章「地圖歪了先看 TF」的檢查方法）。
- SLAM 輸出：佔據格地圖、估測軌跡、粒子雲（第 11 章案例 11-2）、關鍵幀與共視圖（第 22 章）。
- 導航資訊：規劃路徑、代價地圖、目標點（第 29 章）。

近年 RViz2 也持續改進，例如加入基於正規表示式的 TF 過濾器讓你能依名稱顯示或隱藏座標轉換、話題頻率會自動顯示在狀態列、以及可訂閱 point_cloud_transport 話題等功能。

### Gazebo：沒有機器人也能實驗

Gazebo 提供物理模擬與感測器模擬，讓同學能在虛擬環境中完整驗證 SLAM 演算法。本書從第 8 章就一再建議「先模擬、再便宜硬體、後追求精度」，Gazebo 正是這條路徑的第一站。

- 模擬什麼：機器人的運動學與動力學、光達與相機的量測（含雜訊）、環境的幾何與碰撞。
- 教學價值：每位同學都能有一台「自己的機器人」，不受硬體數量限制；可以任意設計場景（長走廊、玻璃隔間）來重現第 13、14 章討論的各種困難情境；也能安全地做危險實驗。
- 版本搭配：ROS 2 從 Jazzy 起，官方會為每個 ROS 版本推薦特定的 Gazebo 版本，並提供 Gazebo 供應商套件讓使用者能快速安裝上手。安裝時務必依官方建議的搭配，以免相容性問題。

### Foxglove：更強大的除錯與分析

Foxglove Studio 是由前自駕車公司工程師於 2021 年創辦的公司所開發的多協定視覺化工具，同時支援桌面、網頁與雲端，並支援 ROS 1、ROS 2、MCAP、MAVLink、MQTT 與原始 WebSocket，提供 3D、繪圖、影像、狀態轉換、地圖等多種面板。

它之所以受到重視，是因為 RViz2 雖然功能足夠但有其限制：它依賴 ROS，難以檢視非 ROS 的系統；記錄檔分析能力較弱；難以協作或分享（只能靠截圖與錄影）；且沒有網頁或平板的支援。

- 對教學的價值：網頁版讓同學不必安裝就能檢視資料；能分享連結給老師或同學，方便討論與批改。
- 對研究的價值：強大的繪圖與時間序列分析，適合做第 22.7 節的軌跡誤差分析。
- 建議：RViz2 用於即時的三維視覺化（執行時看機器人在做什麼），Foxglove 用於事後的資料分析（找出哪裡出了問題）。兩者互補而非取代。

## 28.6 GitHub 開源專案閱讀方法

SLAM 領域的一大幸運，是幾乎所有重要系統都開源。但面對一個有數萬行程式碼的專案，同學常不知從何下手。本節提供一套系統性的閱讀方法。

### 五個步驟

34. 先讀論文，再讀程式：這是最重要的建議。ORB-SLAM3、Cartographer、LOAM 都有對應的論文，先理解演算法架構（如本書第 15、18、22、23 章所做的），再看程式碼會事半功倍。直接跳進程式碼往往會迷失在細節中。

35. 讀 README 與文件：確認相依套件、支援的平台版本、以及執行範例的指令。特別注意硬體需求——第 18 章案例 18-3 提過，許多同學卡關的原因是硬體規格不符，而這在文件中往往寫得清清楚楚。

36. 先跑起來，再讀程式：用官方提供的資料集把系統跑通，建立成功經驗。能跑之後再讀程式碼，你會有具體的參照——「剛剛畫面上那條軌跡，是這段程式產生的」。

37. 從進入點往下追：找到 main 函式或 ROS 節點的進入點，順著資料流往下讀。不要企圖一次讀懂全部，而是追蹤「一幀資料進來後，依序經過哪些函式」。

38. 加輸出訊息驗證理解：在關鍵位置加上輸出訊息，執行後觀察它們的觸發時機與頻率。第 22 章實作 22-1 的延伸挑戰正是這個方法——追蹤 ORB-SLAM2 在什麼情況下切換三種追蹤策略。

### 閱讀時的重點對照

讀 SLAM 專案時，建議依本書的架構去找對應的模組：

- 前端在哪：特徵提取或掃描匹配的程式（第 13、20 章）。
- 後端在哪：呼叫最佳化函式庫的程式（第 12 章的 g2o、Ceres、GTSAM）。
- 閉環在哪：地點辨識與驗證的程式（第 22、23 章）。
- 資料結構：關鍵幀、地圖點、位姿圖如何定義（第 22.6 節）。

把程式碼的模組對應到本書的章節，你會發現理論與實作的對應關係非常清楚——這也是本書刻意以「架構」而非「數學細節」為主軸的原因。

### 善用社群資源

- Issues：專案的 issue 區是寶庫。你遇到的問題，很可能別人已經問過並得到解答。搜尋錯誤訊息往往能直接找到答案。
- Fork 與分支：原始專案可能已停止維護，但社群常有維護中的分支（第 22.7 節提過的建議）。留意 star 數較多、近期有更新的 fork。
- 貢獻回饋：若你解決了一個未被記錄的問題，把解法寫成 issue 或 pull request 回饋社群。這是參與開源生態最直接的方式，也是很好的履歷素材。

## 28.7 從高中積木式實作到大學原始碼實作

本章最後，規劃一條從高中到大學、乃至研究所的完整學習路徑。這也是全書教學設計的總綱。

| 階段 | 目標 | 主要工具 | 本書章節 |
|---|---|---|---|
| 一：觀察體驗 | 理解 SLAM 在做什麼 | 模擬器、掃地機器人觀察 | 第 1–3、16 章 |
| 二：積木式操作 | 會用現成系統建圖導航 | Gazebo、啟動檔、參數調整 | 第 14、15、29 章 |
| 三：資料與分析 | 會錄資料、評估品質 | rosbag2、Python、evo | 第 13.7、22.7 節 |
| 四：讀原始碼 | 理解系統如何實現 | GitHub、除錯工具 | 第 22、23 章 |
| 五：修改與創新 | 能改良或整合系統 | C++、CMake、演算法設計 | 第 30、31 章 |

**表 28-4　SLAM 學習的五個階段**

### 各階段的教學建議

- 階段一（高中適用）：不碰程式碼，重點是建立直覺。用模擬器操作、觀察家中掃地機器人（第 16.7 節）、做紙上活動（第 12、16、25 章的實作）。目標是讓同學「知道 SLAM 在解什麼問題、為什麼困難」。
- 階段二（高中進階/大學部）：使用現成系統，學會啟動、設定參數、觀察結果。重點是理解參數的意義與影響，而非撰寫程式。第 14、15 章的實作正是設計在這個層次。
- 階段三（大學部）：開始寫程式——但寫的是分析與工具程式，而非 SLAM 核心。錄製資料、計算誤差、繪製圖表。這個階段培養的是「用資料說話」的工程素養。
- 階段四（大學部後期/專題）：閱讀開源系統的原始碼，理解理論如何落實為程式。此時前面三階段的經驗會成為理解的支架。
- 階段五（專題/研究生）：修改既有系統或整合多個系統以解決特定問題。第 31 章的專題與第 32 章的 AI 整合，都屬於這個層次。

### 三個教學心法

39. 先成功、再深入：讓同學在每個階段都有可展示的成果。太早要求讀原始碼，容易造成挫折而放棄。

40. 刻意示範失敗：本書每一章都討論「常見失敗」，是有意的設計。看到系統在什麼情況下壞掉，比看它順利運作學到更多——第 14.6 節、第 26.5 節的挑戰測試都是這個精神。

41. 環境問題與學習內容分開：如 28.1 節建議，用容器或統一環境把安裝問題降到最低。課堂時間應該花在 SLAM，而不是排除相依性錯誤。

### 承先啟後

本章把工具鏈的三個代價量出來了：語言方面，同一段掃描配準核心，純 Python 比 C++ 慢 147 倍，在 30 Hz 迴路中會超出時間預算 63%，但 NumPy 在大批量時只比 C++ 慢 1.5 倍，在 30 Hz 下只佔 10% 的預算；通訊方面，話題的延遲是 0.022 毫秒、吞吐 89,862 則 / 秒，而服務會把呼叫端擋住伺服端的整段處理時間（伺服端 20 毫秒，呼叫端就等 20.3 毫秒，吞吐掉到每秒 49 次）；錄製方面，全部感測器不壓縮錄一小時是 183 GB，而一般壓縮只省 28%，只有影像專用編碼能帶來 22 倍的差距。

環境與工具至此備齊。下一章，我們把第四篇的 2D SLAM 理論真正跑起來：從硬體組裝、TF 設定、里程計發布，到 Gmapping 與 Cartographer 建圖、AMCL 定位、以及完整的導航堆疊。第 29 章將是本書第一個從頭到尾的完整系統實作。

## 28.8　工具鏈的三個實務原則

本章量了三件事，每一件都對應一個可以直接套用的原則。

### 一、先量測，再優化

圖 28-1 顯示 NumPy 在 30 Hz 控制迴路中只佔 10% 的時間預算。這表示大多數「我應該改寫成 C++ 嗎」的糾結是多餘的。正確的順序是：先用 Python 寫出能跑的版本，用計時器

量出哪一段真的慢，再只改寫那一段。

這個原則聽起來老生常談，但在 SLAM 專題裡特別重要，因為瓶頸的位置常常違反直覺。很多人以為瓶頸在矩陣運算，實際上往往在資料的複製、序列化，或是第 28.3 節那種被服務擋住的等待。

## 二、阻塞比慢更危險

圖 28-2 的核心訊息是：一個「慢」的模組只是拖慢自己，一個「阻塞」的呼叫卻會拖垮整個迴路。前者可以靠加快硬體解決，後者不行——它是架構問題。

這與第 33.8 節的「錯開執行」、第 34.5 節的「非同步設計」是同一個原則，也是為什麼本書在談每一個系統時都要先問「這一層的更新頻率是多少」。不同頻率的模組之間，介面必須是非阻塞的。

## 三、資料是會滿出來的

圖 28-3 顯示不做取捨時一小時就是 183 GB。這個數字對規劃實驗有直接的意義：一天的實驗若錄滿八小時，就是 1.5 TB。而這些資料還要備份、要傳輸、要長期保存。

務實的做法是分層：平時只錄輕量的話題（光達、IMU、里程計、位姿），需要做視覺分析時才開啟影像，而且降頻與壓縮。這呼應了第 19.6 節巡檢機器人的地圖維護與第 27 章的頻寬排序——在資源有限的系統裡，「選擇要留下什麼」本身就是一項設計工作。

| 量到的事 | 數字 | 實務原則 |
|---|---|---|
| 純 Python 的代價 | 比 C++ 慢 147 倍，30 Hz 下超出預算 63% | 即時迴路至少要向量化 |
| NumPy 的代價 | 大批量時只比 C++ 慢 1.5 倍 | 多數情況不需要改寫 C++ |
| 服務的阻塞 | 伺服端花 20 ms，呼叫端就被擋 20.3 ms | 長任務一律用動作 |

| | | |
|---|---|---|
| 話題與服務的吞吐 | 89,862 對 9,101 次 / 秒 | 不需回覆就用話題 |
| 錄製的資料量 | 全開一小時 183 GB | 先選話題、再壓縮、必要時降頻 |
| 壓縮的效果 | zlib 只有 1.1–1.5 倍，JPEG 有 22 倍 | 用懂資料結構的編碼器 |

**表 28-5　本章三個實驗的實務對照**

## 公式或流程

### 一、環境建置的完整流程

**圖 28-4　從零到能跑 SLAM 的環境建置流程**

① 確認版本對應：選定 ROS 2 發行版 → 查出對應的 Ubuntu 版本（表 28-1）

↓

② 安裝 Ubuntu：原生 / 虛擬機 / 容器（依用途選擇，28.1 節）

↓

③ 安裝 ROS 2：依官方文件加入軟體來源並安裝桌面版（含 RViz2）

↓

④ 安裝模擬器：依該 ROS 版本官方推薦的 Gazebo 版本

↓

⑤ 建立工作空間：建立目錄 → 放入套件原始碼 → colcon 建置

↓

⑥ 載入環境設定（source）→ 驗證：啟動範例節點，確認能通訊

↓

⑦ 安裝 SLAM 套件：slam_toolbox / Cartographer / Nav2 等

↓

⑧ 冒煙測試：啟動模擬器與 SLAM，在 RViz2 中看到地圖成長

第八步「冒煙測試」是關鍵的檢查點：若能看到地圖逐步長出來，表示整條工具鏈都通了，可以放心進入第 29 章的實作。若卡住，依 27.1 至 28.2 節的常見陷阱逐項排查。

## 二、通訊機制的選用判斷

**圖 28-5　話題、服務、動作的選用決策**

這個功能需要什麼樣的通訊？

↓

① 是持續流動的資料嗎？（感測器、狀態、控制指令）

├─ 是 → 【話題】發布訂閱，非同步，一對多

└─ 否 → 進入 ②

↓

② 是很快就能回答的一次性請求嗎？（查詢、設定）

├─ 是 → 【服務】請求回應，呼叫端等待

└─ 否 → 進入 ③

↓

③ 是耗時且需監控進度的任務嗎？（導航、掃描任務）

└─ 是 → 【動作】可回報進度、可取消

## 三、rosbag 錄製與重播的完整流程

**圖 28-6　資料錄製到分析的工作流程**

【錄製階段】

① 確認要錄的話題：感測器資料 + IMU + 里程計 + TF（勿漏 TF）

② 估算檔案大小與儲存空間

③ 執行錄製 → 產生 MCAP 格式的記錄檔

↓

【重播階段】

④ 檢查話題名稱是否與演算法預期相符，必要時做重新對應

⑤ 重播資料 → 同時啟動 SLAM 節點

⑥ 在 RViz2 中即時觀察，或以 Foxglove 事後分析

↓

【分析階段】

⑦ 輸出估測軌跡 → 與真值比較 → 計算 ATE / RPE（第 22.7 節）

⑧ 調整參數 → 回到 ⑤ 重播同一份資料 → 公平比較

注意第八步：因為重播的是同一份資料，不同參數的比較才是公平的。這正是 rosbag 對研究與教學最大的價值——把「感測資料的隨機性」這個變因固定下來。

## 案例

### 案例 28-1　半堂課的代價：版本不符的連鎖反應

這是幾乎每個 SLAM 課程都會發生的場景，值得同學引以為戒。

狀況是這樣的：同學下載了某個開源 SLAM 專案，依 README 執行編譯，結果出現一長串錯誤訊息。上網搜尋、逐一嘗試各種解法、安裝了幾個看起來相關的套件——兩個小

時過去，問題不但沒解決，系統還被裝壞了。

根源往往極其單純：專案是為某個 ROS 2 版本寫的，而同學裝的是另一個版本。相依的函式庫版本不同，連鎖反應下產生數十個看似無關的錯誤。

這個案例的教訓有三：

- 第一，先確認版本對應。開始編譯前，先花五分鐘確認專案支援的 ROS 版本與你的環境是否相符——這五分鐘能省下兩小時。
- 第二，錯誤訊息要從第一個看起。編譯錯誤常有連鎖效應，後面數十個錯誤可能都源於第一個。從最後往回看是常見的誤區。
- 第三，容器是好朋友。用容器封裝環境，裝壞了就重建，不會污染主系統，也能確保全班環境一致（28.1 節的建議）。

更深一層的啟示是：工程實作中，「環境問題」佔用的時間往往遠超預期。這不是同學能力不足，而是這個領域的常態。有經驗的工程師與初學者的差別，不在於不會遇到問題，而在於能快速判斷問題的類別並找到對應的解法。

## 案例 28-2　從 rosbag 到論文：一份資料的多重價值

rosbag 的價值，在一個真實的研究流程中展現得最清楚。

假設一位同學要做「不同 SLAM 系統在系館走廊的表現比較」的專題。若沒有 rosbag，他必須：為每個系統各跑一次實體實驗——但每次實驗的路線、速度、光線、人流都不完全相同，比較結果毫無說服力；而且每次都要借用機器人、佈置場地，耗時費力。

有了 rosbag，流程完全不同：錄製一次高品質的資料（含影像、IMU、光達、TF），之後所有系統都用同一份資料測試。這帶來四個好處：

- 公平比較：所有系統面對完全相同的輸入，差異純粹來自演算法。
- 可重現：別人拿到你的資料，能重現你的結果——這是科學研究的基本要求。

- 效率：參數調整可以反覆進行，不需重跑實驗。一個下午能測試數十組參數。
- 長期價值：這份資料明年、後年的學生仍能使用。第 26.7 節建議的「錄製並公開在地場域資料集」，正是這個思路。

具體的建議是：每一屆的專題生都貢獻一份自己場域的資料，累積下來就是一個有價值的在地資料集。這對後續研究、教學示範、甚至與其他學校交流，都是實質的資產。

### 案例 28-3　工具的演進：從 rosbag 到 MCAP、從 RViz 到 Foxglove

本章介紹的工具鏈，近年有顯著的演進，值得同學留意這個趨勢。

記錄格式方面，MCAP 已成為事實上的新標準，取代了原本的 rosbag 格式，並自 ROS 2 Jazzy 起成為 rosbag2 的預設儲存格式。它的關鍵優勢是自我描述——記錄檔內嵌訊息定義，不需額外的套件就能解讀。

視覺化方面，Foxglove 補足了 RViz2 的限制：RViz2 依賴 ROS、難以檢視非 ROS 系統、記錄檔分析能力弱、難以協作分享、且無網頁或平板支援。Foxglove 則同時支援桌面、網頁與雲端，並能處理 ROS 1、ROS 2、MCAP、MAVLink、MQTT 等多種協定。

這個演進背後有一個值得注意的模式：推動改變的往往不是 ROS 核心團隊，而是產業界的實務需求。Foxglove 由前自駕車公司的工程師創辦，而他們正是把自駕車開發中的痛點——大量資料的分析、跨團隊協作、非 ROS 系統的整合——轉化為工具。

對同學的啟示是：工具會不斷演進，不必執著於某個特定工具。重要的是理解「這個工具解決什麼問題」——當你清楚問題的本質，就能快速上手新工具，也能判斷什麼時候該換工具。這與本書一貫的主張相同：理解原理比記憶操作重要。

## 實作活動

### 實作 28-1　建置 SLAM 開發環境並完成冒煙測試

目的：完整建置可用的 ROS 2 開發環境，熟悉三種通訊機制與 rosbag 工具，並完成第一次

模擬建圖。本實作是第 29、30 章的必要前置作業。

**第一部分：環境建置（全體）**

142. 依表 28-1 選定 ROS 2 版本與對應的 Ubuntu 版本，記錄你的選擇與理由。

143. 依 28.1 節選擇安裝方式（原生 / 虛擬機 / 容器），完成 Ubuntu 與 ROS 2 桌面版的安裝。

144. 驗證安裝：啟動 ROS 2 內建的範例節點（如 talker 與 listener），確認兩者能正常通訊。

145. 建立工作空間：建立目錄結構，用 colcon 建置一個空的工作空間，並載入環境設定。

146. 安裝模擬器與 TurtleBot3 套件，啟動模擬環境，確認能在畫面中看到虛擬機器人。

**第二部分：通訊機制探索（全體）**

42. 列出目前系統中所有活躍的話題，找出 /scan、/odom、/tf 等 SLAM 相關的話題。

43. 檢視 /scan 話題的訊息內容，對照第 8.3 節的 LaserScan 格式，找出 angle_min、angle_increment、ranges 等欄位。

44. 觀察話題的發布頻率，記錄光達、里程計、TF 各自的頻率，思考為什麼它們不同。

45. 列出可用的服務與動作，找出與導航相關的動作介面（第 29 章會用到）。

46. 用 Python 撰寫一個最簡單的節點：訂閱 /scan，計算並印出「當前最近障礙物的距離與方向」。這是第 8 章實作 8-1 延伸挑戰的 ROS 版本。

**第三部分：rosbag 錄製與重播（全體）**

47. 遙控模擬機器人在環境中移動，同時用 rosbag2 錄製 /scan、/odom、/tf 與 /imu（若有），觀察產生的檔案格式與大小。

48. 停止模擬器，單獨重播錄好的資料，在 RViz2 中顯示光達掃描與 TF，確認能正常重現。

49. 話題重新對應練習：重播時把 /scan 改名為 /laser_scan，觀察原本訂閱 /scan 的節點是否還能運作，理解 28.4 節提到的重新對應技巧。

50. 刻意不錄 TF 再重播一次，觀察 RViz2 中會發生什麼——親身驗證 28.4 節「勿漏 TF」的提醒。

**第四部分：冒煙測試——第一次建圖（全體）**

51. 啟動模擬器與 SLAM 套件（建議用 slam_toolbox 或 Cartographer），在 RViz2 中顯示地圖、光達與 TF。

52. 緩慢遙控機器人繞行環境一圈，觀察佔據格地圖如何逐步長出來（第 13 章的理論在此具象化）。

53. 儲存地圖，並用第 13.7 節的方法評估品質：牆是否筆直？迴路是否閉合？

54. 把整個流程寫成一個啟動檔，讓下次能一鍵啟動模擬器、SLAM 與 RViz2。

**討論問題**

59. 你選擇了哪個 ROS 2 版本？依 28.1 節的建議，這個選擇對「找教學資源」與「長期維護」各有什麼利弊？

60. 光達、里程計、TF 三者的發布頻率各是多少？為什麼它們的頻率不同？（提示：第 7、13 章的感測器特性）

61. 不錄 TF 的資料重播時發生了什麼？這印證了第 10 章的哪個觀念？

62. 依圖 28-2 的決策流程，若要實作「讓機器人執行一次完整的房間掃描任務，過程中可查詢進度也可中止」，應該用哪種通訊機制？為什麼？

63. 第一次建圖的結果如何？若不理想，依第 14.6 節的失敗模式，你認為最可能的原因是什麼？

**延伸挑戰（大學部）**

其一：容器化你的開發環境——撰寫一個容器設定檔，封裝完整的 ROS 2 環境與所需的 SLAM 套件，讓同學能一鍵取得一致的環境。這對教學助教與專題團隊極為實用，也是現代軟體工程的必備技能。其二：開源專案閱讀練習——選一個本書介紹過的開源 SLAM 專案（建議從 slam_toolbox 開始，程式規模適中），依 28.6 節的五個步驟閱讀，並繪製一張「資料如何從輸入流到輸出」的流程圖。把你的流程圖與本書對應章節的架構圖比較，看看理論與實作的對應關係。

## 延伸閱讀


85. ROS 2 官方文件（docs.ros.org）.（安裝指引、教學與 API 文件，實作的第一手參考）
86. REP-2000: ROS 2 Releases and Target Platforms(ros.org/reps).（版本與平台對應的官方規範，28.1 節的依據）
87. S. Macenski et al., "Robot Operating System 2: Design, Architecture, and Uses in the Wild," Science Robotics, 2022.（ROS 2 設計理念的權威介紹）
88. Gazebo 官方文件之 Installing Gazebo with ROS(gazebosim.org).（ROS 與 Gazebo 版本搭配的官方建議）
89. MCAP 官方文件（mcap.dev）與 Foxglove 文件（docs.foxglove.dev）.（新一代記錄格式與視覺化工具）
90. slam_toolbox 開源專案（GitHub）.（ROS 2 主流的 2D SLAM，程式規模適中，適合作為原始碼閱讀的入門）
91. ROBOTIS TurtleBot3 e-Manual(emanual.robotis.com).（教學平台的完整實作指引，含模擬與實體機器人）


## 小結

85. ROS 2 與 Ubuntu 的版本是綁定的，裝錯會造成連鎖的相依性錯誤；ROS 2 每年 5 月發版、偶數年為 LTS，教學建議選用資源最豐富的成熟 LTS 版本，且全班應統一版本。安裝方式有原生、虛擬機與容器三種，教學階段建議用容器以確保環境一致。

86. Python 適合快速原型與資料分析、C++ 適合 SLAM 核心與高頻運算；CMake 管理 C++ 的編譯，看懂 find_package 與 target_link_libraries 就能解決多數編譯問題；colcon 依相依關係建置工作空間，建置後務必載入環境設定。

87. 三種通訊機制各有適用：話題用於持續流動的資料（感測器、狀態、指令）、服務用於快速的一次性請求、動作用於耗時且需監控進度的任務（如導航）；用錯機制會導致系統阻塞與除錯困難。

88. rosbag2 讓「錄一次、分析無數次」成為可能，支撐可重現性、離線開發、除錯與資料集建立；MCAP 自 ROS 2 Jazzy 起成為預設格式，具索引與自我描述特性；錄製時勿漏 TF，重播時留意版本、TF 與話題名稱三個常見問題。

89. RViz2 用於即時三維視覺化、Gazebo 提供無需硬體的模擬（版本須依官方推薦搭配）、Foxglove 補足 RViz2 在記錄檔分析、協作分享與非 ROS 系統上的限制；三者互補而非取代。

90. 閱讀開源專案的五步驟：先讀論文再讀程式、讀 README 確認相依與硬體需求、先跑起來再讀程式、從進入點順資料流往下追、加輸出訊息驗證理解；並善用 issue 區與社群維護的分支。

91. 學習路徑分五階段：觀察體驗、積木式操作、資料與分析、讀原始碼、修改與創新；教學心法為先成功再深入、刻意示範失敗、以及把環境問題與學習內容分開。

環境與工具至此備齊，同學已經有能力把前面所有理論真正跑起來。下一章，我們把第四篇的 2D SLAM 從頭到尾實作一次：硬體組裝與 TF 設定、里程計發布、Gmapping 與 Cartographer 建圖、AMCL 定位，最後接上完整的導航堆疊，讓機器人真正自己走到目標點。這將是本書第一個完整的端到端系統實作。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

## 第 29 章

## 2D 光達小車建圖實作

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

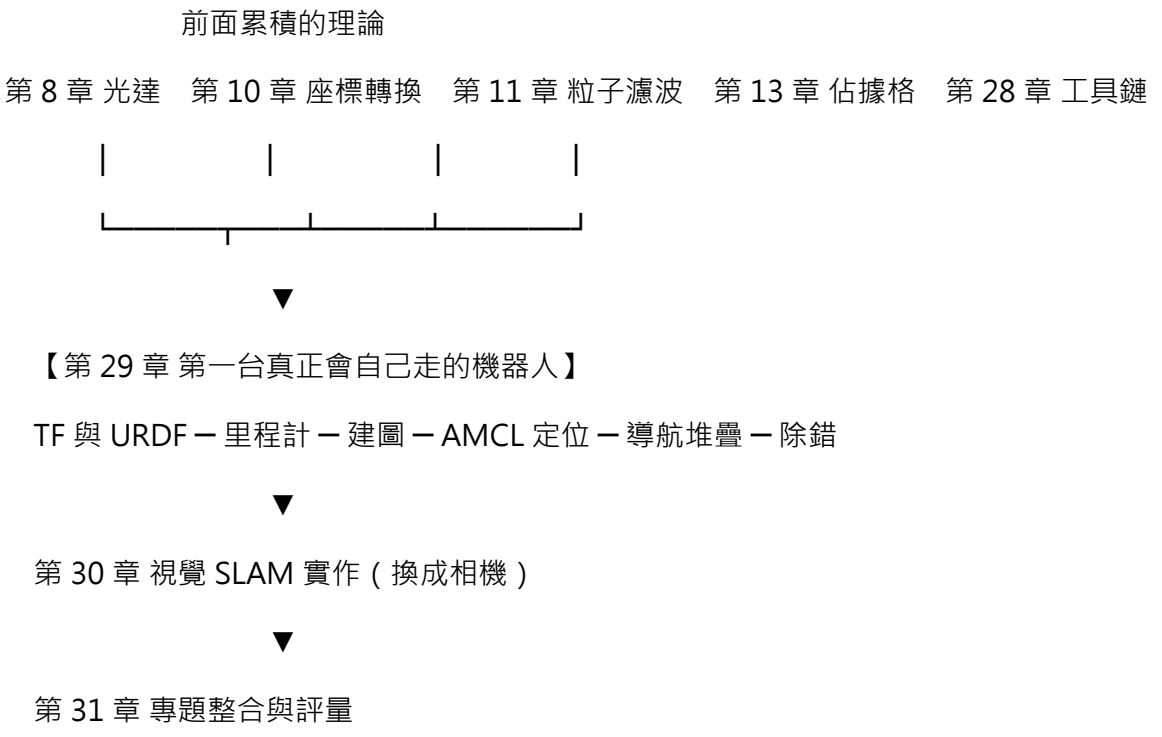
## 本章在全書的位置

這是全書第一個端到端的系統實作——從輪子轉動到機器人自己走到目標點，中間每一個環節都要親手接起來。

本章與前面各章最大的不同，在於它處理的是「接線的地方」。演算法本身在第 13、14、15 章都講過了，真正會卡住專題的是座標轉換設錯、里程計參數沒校正、以及系統不動時不知道該從哪裡查起。這些問題的共同點是：它們不會報錯，只會讓結果「看起來怪怪的」。

本章的三張圖就是為了讓「怪怪的」變成「可以量的」：圖 29-2 示範里程計校正的標準方法，並說明為什麼一定要順時針與逆時針各走一次；圖 29-3 把四種常見的座標轉換錯誤建出來的地圖並排，當成一張症狀對照表；圖 29-4 則實際跑一次全域定位，看粒子要幾步才收斂。

### 圖 29-0　第 29 章的連結地圖




## 學習目標

前二十七章的理論與工具至此全部備齊。本章是本書第一個從頭到尾的完整系統實作：從硬

體與座標設定開始，經過建圖、定位，最後讓機器人自己走到目標點。學完本章之後，同學應能夠達成下列目標：

85. 說明 2D SLAM 小車的硬體組成與各元件的角色。
86. 撰寫 URDF 機器人描述檔並正確設定 TF 座標樹。
87. 由輪速編碼器計算並發布里程計資料。
88. 執行 2D SLAM 建圖並儲存地圖檔。
89. 說明 AMCL 定位的設定要點與常見問題。
90. 說明 Nav2 導航堆疊的架構與一次導航任務的完整資料流。
91. 依系統性的順序排除導航系統的故障。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 機器人描述檔 | URDF | 以 XML 描述機器人的連桿、關節與感測器安裝位置。 |
| 連桿與關節 | Link / Joint | URDF 的基本元素：剛體與連接它們的關節。 |
| 狀態發布器 | robot_state_publisher | 依 URDF 發布各連桿之間 TF 變換的節點。 |
| 差速驅動 | Differential Drive | 以左右兩輪速度差控制方向的驅動方式。 |
| 里程計 | Odometry | 由輪速積分推算的位姿，提供 odom→base_link 變換。 |
| 代價地圖 | Costmap | 在佔據格上加註通行成本的導航用地圖。 |
| 膨脹層 | Inflation Layer | 把障礙物向外擴張安全距離的代價地圖層。 |
| 全域規劃器 | Global Planner | 在靜態地圖上規劃起點到目標的完整路徑。 |
| 控制器 | Controller | 把路徑轉為速度指令的局部規劃元件。 |

| 中文名詞 | 英文名詞 | 概念說明 |
| --- | --- | --- |
| 行為樹 | Behavior Tree | Nav2 用來組織導航任務流程的架構。 |
| 復原行為 | Recovery Behavior | 導航失敗時嘗試脫困的動作，如原地旋轉、後退。 |
| 生命週期節點 | Lifecycle Node | 有明確啟動狀態轉換的 ROS 2 節點類型。 |
| 速度指令 | cmd_vel | 送給底盤的線速度與角速度指令。 |

## 29.1　硬體：小車底盤、馬達、編碼器、2D 光達

本節說明一台能跑 SLAM 的 2D 小車需要哪些元件，以及選型時的考量。

### 五大元件

- 底盤與馬達：最常見的是差速驅動——左右兩個驅動輪加一到兩個萬向輪。它結構簡單、控制直觀（第 10.3 節的 2D 位姿正好對應），是教學平台的標準選擇。
- 編碼器：裝在馬達軸或輪軸上，計算輪子轉了多少。這是里程計的資料來源（29.3 節），也是第 13.4 節「里程計預測、光達校正」中不可或缺的一半。
- 2D 光達：第 8 章介紹過的旋轉式雷射掃描器，提供 /scan 話題。教學用的入門級型號量測範圍約 8 至 12 公尺，對室內建圖已足夠。
- 微控制器：負責馬達驅動與編碼器讀取，並與上層電腦通訊。它處理的是毫秒級的即時控制，不適合放在單板電腦上做。
- 單板電腦：執行 ROS 2 與 SLAM 演算法。這個「微控制器負責即時控制、單板電腦負責運算」的分工，正是第 25 章案例 25-2 討論過的雙晶片架構在教學平台上的體現。

### 選型的三個考量

253. 光達安裝高度與遮蔽：光達必須有清晰的 360 度視野。若機身或其他元件擋住部分角度，那些方向就永遠是盲區。安裝時務必實際檢查掃描是否被遮擋。

254. 編碼器解析度：解析度太低，里程計的精度就差。一般教學平台的編碼器解析度足夠，但若使用自製底盤，需確認每圈的脈衝數足以分辨小幅度的移動。

255. 運算能力：2D SLAM 對運算的需求不高（相較第五、六篇的 3D 與視覺 SLAM），常見的單板電腦即可勝任。若要同時跑 Nav2 導航，則需留意記憶體是否充足。

### 三種平台選擇

- 現成教學平台：整合完整、文件齊全、有社群支援，能讓同學快速進入 SLAM 本身而非硬體除錯。適合課程時間有限的情況。
- 自組平台：成本較低、學習深度更高——同學能親手處理每一個環節，對系統的理解遠比使用現成平台深刻。適合專題與有充裕時間的課程。
- 純模擬：完全不需硬體，但能完整體驗 28.2 至 29.7 節的所有流程。第 28 章建議的「先模擬、再硬體」路徑，在此章尤其適用——先在模擬中把整條管線跑通，再搬到實體車上，能省下大量除錯時間。

## 29.2 TF 設定與 URDF

第 10 章強調過「地圖歪了先看 TF」，本節說明如何正確建立座標樹。這是整個實作最容易出錯、也最需要謹慎的環節。

### URDF：機器人的骨架描述

URDF（統一機器人描述格式）以 XML 描述機器人的物理結構：有哪些剛體（連桿）、它們如何連接（關節）、以及各自的尺寸與相對位置。

對 SLAM 而言，URDF 最重要的功能是描述感測器的安裝位置——也就是第 5、10 章反覆提到的外參。若 URDF 中寫的光達位置與實際安裝差了幾公分或幾度，建出的地圖就會模糊或錯位。

### 典型的座標樹結構

第 10.7 節介紹過 ROS 的標準框架階層，在實體小車上的具體形式為：

**圖 29-1 2D SLAM 小車的完整 TF 樹**

map（地圖座標系） 全域固定、不漂移、會跳變

↓ 由 SLAM 或 AMCL 發布

odom（里程計座標系） 連續平滑、長期漂移

↓ 由里程計節點發布（29.3 節）

base_footprint（機器人在地面的投影，選用）

↓

base_link（機器人本體中心）

├──→ laser（光達） ↖

├──→ imu_link（IMU） 由 URDF 與 robot_state_publisher

├──→ left_wheel 依固定的安裝關係發布

└──→ right_wheel ↙

### 誰負責發布哪一段

這是初學者最常混淆的地方，務必弄清楚：

- map → odom：由 SLAM 系統（建圖時）或 AMCL（導航時）發布。官方文件明確指出，這個變換是 Nav2 系統的主要需求之一。
- odom → base_link：由里程計系統發布。里程計系統的角色就是提供這個變換，而其來源可以是光達、雷達、輪速編碼器、視覺慣性里程計或 IMU；目標是提供一個基於機器人運動的平滑連續局部座標系。
- base_link → 各感測器：由 robot_state_publisher 依據 URDF 提供，它會發布 base_link 到各感測器的變換。

### 常見的 TF 錯誤

55. 座標樹斷裂：某一段沒有節點發布，導致 map 到 laser 的完整鏈路斷開。RViz 中會顯示「找不到變換」的錯誤。

56. 外參填錯：光達的安裝位置或朝向填錯。症狀是掃描點與實際環境錯位——把機器人放在牆邊，若 RViz 中的掃描點與牆的位置對不上，就是這個問題。

57. 重複發布：兩個節點同時發布同一段變換，造成衝突與跳動。這在整合多個套件時容易發生。

58. 時間戳記問題：變換的時間戳記與感測資料不同步，導致高速移動時明顯錯位（第 10.6 節提過時間戳記的重要性）。

### 驗證的方法

- 視覺化檢查：在 RViz 中同時顯示 TF 座標軸與光達掃描，確認掃描點與機器人的相對位置合理。
- 產生 TF 樹圖：用工具產生座標樹的圖形，確認結構完整、沒有斷裂或重複。
- 靜態驗證：把機器人靜置在已知位置（例如緊靠牆角），檢查掃描點是否落在正確的距離上。

## 29.3 里程計資料發布

里程計是 2D SLAM 的另一半基石。本節說明它如何從編碼器讀值變成 ROS 的里程計訊息。

### 從編碼器到位姿

差速驅動小車的里程計計算，是第 3 章航位推算與第 10.3 節座標轉換的直接應用：

59. 讀取編碼器：取得左右輪在這個時間間隔內各轉了多少。

60. 換算輪子位移：轉動圈數乘上輪子周長，得到左右輪各自走了多遠。

61. 計算機體運動：由左右輪位移的平均值得到前進距離、由兩者的差除以輪距得到轉角（公式見本章公式一）。

62. 更新位姿：依第 10.3 節的 2D 位姿複合公式，把相對運動累加到當前位姿，並記得處理角度環繞。

63. 發布：發布里程計訊息（含位姿與速度），同時發布 odom → base_link 的 TF 變換。

### 校正：精度的關鍵

第 14.6 節提過「輪子打滑與里程計失準」是建圖失敗的首要原因，而其中最常被忽略的是參數校正。兩個關鍵參數是：

- 輪徑：若設定值與實際不符，直線行走的距離就會系統性偏差，導致整張地圖等比例放大或縮小。校正方法是讓小車直走一段固定距離（例如 3 公尺），比較里程計讀值與捲尺實測值。
- 輪距：兩個驅動輪的中心距離。若設錯，轉彎角度就會偏差，這對建圖的影響比輪徑更嚴重——因為角度誤差會透過後續的移動被放大（第 7.3 節討論過姿態誤差的放大效應）。校正方法是讓小車原地轉 10 圈，比較里程計累計角度與 3600 度的差異。

這兩項校正應該在第一次建圖之前完成。許多同學花數小時調 SLAM 參數，卻沒發現問題出在未校正的里程計——這是實作中最常見的冤枉路。

### 里程計的品質要求

里程計的目標是提供一個平滑而連續的局部座標系，而全域定位系統會更新相對於全域座標系的變換以補償里程計的漂移。

請注意「平滑而連續」這個要求：里程計不需要絕對準確（漂移會被 SLAM 修正），但必須平滑不跳變。若里程計因為程式錯誤或通訊丟包而出現跳躍，SLAM 的運動預測就會失效，建圖必然失敗。

### 進階：融合 IMU

純輪速里程計的角度估計容易因打滑而失準。實務上常以濾波器融合輪速與 IMU 的角

速度（第 7、11 章），得到更穩健的里程計。ROS 2 生態中可用 robot_localization 套件的 EKF 節點來提供 odom → base_link 變換，這在地面不平或容易打滑的場域特別有幫助。

### 量一量：里程計參數不準會怎樣

29.3 節說里程計要校正，但沒說不校正會差多少、也沒說怎麼知道是哪個參數錯了。本小節用標準的方形路徑測試把兩件事一起回答。

差動輪里程計只有兩個關鍵參數：輪半徑與輪距。它們各自會造成不同性格的誤差：輪距設太小，機器人以為自己轉得比實際多，於是每次轉彎都轉不夠；左右輪直徑不等，機器人直行時會偏向一側，但它的里程計以為自己走直線。麻煩的是——光看一次實驗，這兩種錯誤看起來一模一樣。

實驗方法是經典的方形路徑測試：命令機器人依「自己的里程計」走一個 2 公尺見方的正方形回到起點，然後量測它實際停在哪裡。關鍵在於順時針與逆時針各走一次。

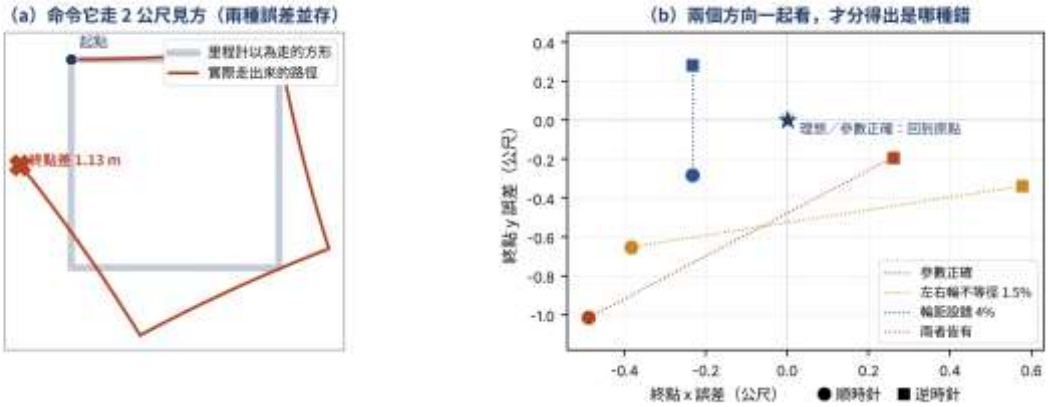



**圖 29-2　方形路徑測試：里程計誤差的指紋**

（a）畫出一台同時有兩種誤差的小車（輪距設定值偏小 4%、右輪實際比左輪大 1.5%）順時針走方形的結果。灰色粗線是里程計相信自己走的路徑——一個標準的正方形；紅線是它實際走出來的路徑。每個轉角都轉不夠，而且直線段微微偏彎，四個邊累積下來，終點與起點差了 1.13 公尺。在一個 2 公尺見方的場地裡，這個誤差超過邊長的一半。

（b）是本圖的重點。四種情況各跑順時針（圓點）與逆時針（方塊）兩次，把終點誤差畫

在同一張圖上。參數正確時兩點都落在原點（誤差 0.001 公尺）。

輪距設錯 4%（藍色）的兩點落在 x 相同、y 正負對稱的位置（−0.233, −0.281）與（−0.233, +0.281）。這個「同 x、鏡像 y」的排列，就是純輪距誤差的指紋——因為轉彎角度的誤差在兩個方向上是對稱的。

左右輪不等徑 1.5%（橘色）則完全不同：兩點落在（−0.384, −0.652）與（+0.577, −0.339），位置差很遠而且沒有明顯的對稱關係。因為直行時的偏彎方向不隨行進方向改變，但轉彎的順序改變了它累積的方式。

兩種誤差並存時（紅色），順時針的終點誤差是 1.13 公尺，逆時針卻只有 0.32 公尺。這正是為什麼只跑一個方向會被騙：如果同學只試了逆時針，看到 0.32 公尺可能覺得「還可以接受」，完全不知道另一個方向會差到 1.13 公尺。

**校正的順序**

依圖 29-2(b) 的指紋，校正可以分兩步進行。

第一步先處理左右輪不等徑：調整左右輪的半徑比，直到順時針與逆時針的終點落在「對原點大致對稱」的位置為止。此時殘留的就主要是輪距誤差。

第二步調整輪距：若轉彎轉不夠（終點偏向路徑內側），把輪距設定值調大；反之調小。每次調整後重跑兩個方向，直到兩點都靠近原點。

一個實務提醒：校正結果會隨地面材質改變（地毯與磁磚的打滑程度不同），也會隨輪胎磨損而漂移。因此這個測試值得在每學期開始、或換場地時重做一次——它只需要十分鐘，卻能省下後面所有「地圖為什麼會歪」的除錯時間。

## 29.4　2D SLAM 建圖

前置作業完成後，終於可以建圖了。本節說明流程與工具選擇。

### 工具選擇

第 14、15 章介紹過 Gmapping 與 Cartographer，而在現代 ROS 2 環境中，還有第三個常用選擇。Nav2 官方把 SLAM Toolbox 列為預設的 SLAM 演算法，用於定位並產生靜態地圖。

| 工具 | 理論基礎 | 特點 | 適用 |
|---|---|---|---|
| slam_toolbox | 圖優化（第 12 章） | ROS 2 預設、參數簡潔、支援續建地圖 | 教學首選 |
| Cartographer | 圖優化 + 子地圖 | 閉環強、支援 3D、參數較多 | 大場域、需閉環 |
| Gmapping | 粒子濾波（第 11 章） | 概念直觀、ROS 1 時代主流 | 理解粒子濾波 |

**表 29-1 三種 2D SLAM 工具的比較**

教學建議：先用 slam_toolbox 建立成功經驗（參數少、容易上手），再用 Cartographer 體驗閉環的威力（第 15 章實作 15-1），最後可用 Gmapping 對照理解粒子濾波與圖優化的差異。

### 建圖流程

64. 啟動硬體：機器人驅動、光達、里程計節點全部啟動，確認 /scan 與 /odom 都有資料。

65. 檢查 TF：確認座標樹完整（29.2 節的驗證方法）。這一步不可跳過。

66. 啟動 SLAM：啟動選定的 SLAM 節點。要確認它有提供 map → odom 變換並發布 /map 話題。

67. 啟動 RViz：顯示地圖、掃描、TF 與機器人模型，以便即時觀察。

68. 緩慢遙控建圖：第 14.6 節強調過，「開慢一點」是新手成功率最大的變數。轉彎時尤其要慢。

69. 評估品質：依第 13.7 節檢查牆是否筆直、迴路是否閉合、有無重影。

70. 儲存地圖：用 nav2_map_server 儲存，會產生 .yaml 與 .pgm 兩個檔案來表示地圖。

### 地圖檔的兩個檔案

- .pgm：灰階影像檔，就是佔據格地圖的視覺化——黑色為障礙、白色為自由空間、灰色為未知（第 13.1 節的三態）。可以用一般看圖軟體開啟檢視。
- .yaml：描述檔，記錄影像檔名、解析度（每像素幾公尺）、原點座標、以及判定佔據與自由的機率門檻。

這兩個檔案就是第 14.5 節提過的地圖產物。日後導航時由 map_server 載入，供 AMCL 定位與路徑規劃使用。

### 量一量：座標轉換設錯時，地圖長什麼樣

29.2 節設定了座標轉換樹，29.7 節把它列為除錯檢查表的第一項。但「座標轉換設錯」到底會長什麼樣子？本小節刻意設錯四種常見的情況，把建出來的地圖並排，當成一張症狀對照表。

實驗設定是一間 10×7 公尺的室內場景，機器人沿橢圓軌跡繞一圈，用 240 束的 2D 光達建佔據格地圖。真實的光達裝在車體中心，而系統「以為」的外參被刻意設錯。

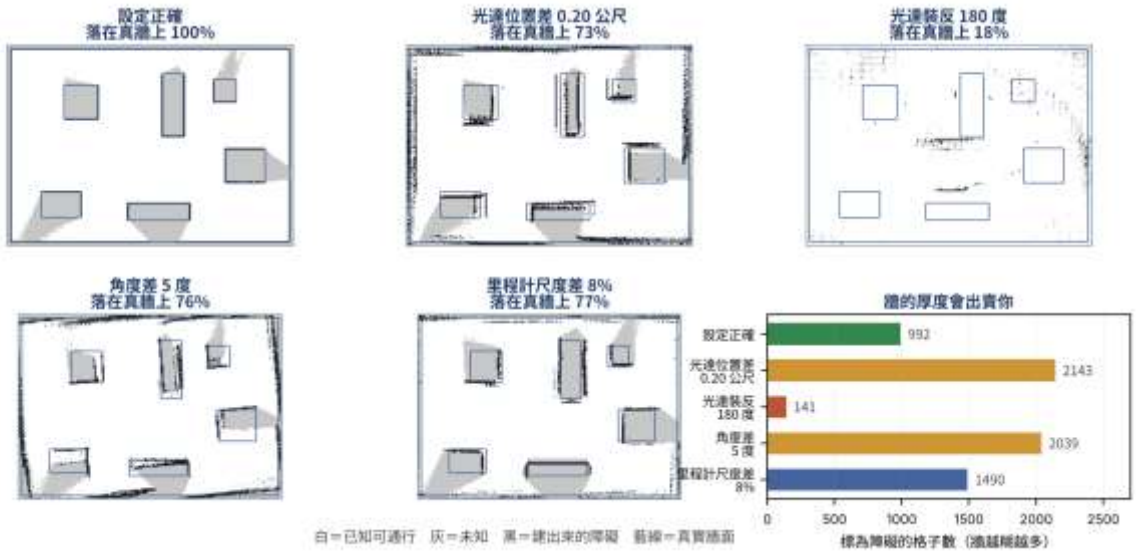



**圖 29-3　座標轉換設錯的四種症狀**

設定正確時，牆面是乾淨的一條線，標為障礙的格子有 992 個，其中 100% 落在真實牆面上。這是基準。

光達位置差 0.20 公尺時——這是最常見的錯誤，通常來自「量測安裝位置時少量了法蘭的厚度」或「URDF 裡填的是光達外殼位置而非光學中心」。地圖看起來仍然像那間房間，牆也還在該在的地方，但障礙格數從 992 暴增到 2,143（2.2 倍），只有 73% 落在真實牆面上。症狀是：牆變厚了，而且是雙層的。

角度差 5 度的症狀幾乎一樣（2,039 格、76%）。這說明一件重要的事：從地圖的外觀，你分不出是位置錯還是角度錯——只能分辨出「有錯」。要分辨是哪一種，得回頭檢查座標轉換樹的數值。

光達裝反 180 度是最戲劇性的一種。這種錯誤在實務上並不罕見——光達的安裝方向與它的座標系定義不一定一致。結果是所有量測都被投影到房間外面，地圖上只剩 141 個障礙格，而且只有 18% 落對位置。這種錯誤反而最容易發現，因為地圖根本不成形。

里程計尺度差 8% 則是另一種類型——它不是座標轉換的錯，而是 29.3 節的校正沒做好，但症狀同樣表現在地圖上：1,490 格、77% 正確。牆同樣變厚，但厚的程度隨離起點的距離而增加。

把這五張圖放在一起，可以歸納出一條很實用的診斷規則：地圖的牆有多厚，就是系統總誤差有多大。一張乾淨的單線牆代表所有環節都對；雙層或模糊的牆代表某處有系統性偏差；而完全不成形的地圖，反而通常是最容易找的那種錯。

**一個不用寫程式的檢查法**

把機器人放在定點，不要移動它，只讓它原地旋轉一圈並建圖。

如果外參正確，旋轉時所有量測都會落在同一條牆線上，地圖是乾淨的。如果光達的安裝位置填錯，旋轉時量測會在地圖上畫出一個圓環——圓環的半徑就是你填錯的距離。

這個檢查只需要三十秒，卻能一次驗證圖 29-3 裡最常見的那兩種錯誤，而且圓環的半徑直接告訴你差多少。建議列入每次換硬體後的標準流程。

## 29.5　AMCL 定位

地圖建好之後，日常運行不需要再建圖，只需在已知地圖中定位——這就是第 3、11 章介紹過的 AMCL。

### AMCL 是什麼

AMCL 是一套機率式的 2D 移動機器人定位系統，實作了自適應（或 KLD 取樣）的蒙地卡羅定位方法，以粒子濾波追蹤機器人在已知地圖上的位置。它已不再是 SLAM 演算法，因為它不建立環境地圖。

演算法的每次迭代，會比較地圖的樣貌與當前的雷射掃描，只保留那些讓資料與地圖最吻合的位置。這正是第 11 章粒子濾波「預測—秤重—重採樣」循環的實現。

### AMCL 與 SLAM 的分工

| 項目 | SLAM（建圖階段） | AMCL（導航階段） |
|---|---|---|
| 是否建圖 | 是 | 否，使用既有地圖 |
| 地圖來源 | 即時建立 | 由 map_server 載入 |
| 運算負擔 | 較高 | 較低 |
| 提供的 TF | map → odom | map → odom |
| 使用時機 | 首次進入新環境 | 日常運行 |

**表 29-2 SLAM 與 AMCL 的分工**

注意兩者都提供 map → odom 變換——這正是第 10.7 節設計的巧妙之處：對下游的導航系統而言，不論是 SLAM 還是 AMCL 在提供定位，介面完全相同，因此可以無縫切換。

### 關鍵參數

- 粒子數：與第 14.3 節的討論相同，多則穩健但耗運算。AMCL 的「自適應」特性能依據不確定性動態調整粒子數——不確定時多用粒子，收斂後減少。
- 運動雜訊模型：描述里程計有多不準的參數。其中控制旋轉對機器人旋轉估計影響的參數，若調高會讓系統對旋轉運動更謹慎、假設更大的不確定性；調低則對旋轉估計更有信心，但若感測器不夠準確會有風險。
- 更新門檻：機器人移動或轉動多少才執行一次濾波更新。設定較小的角度門檻（例如 0.05 弧度而非預設的 0.2，約 11.5 度）能讓機器人在較少的旋轉後就更新，適合緩慢或精確的移動；調高則減少更新次數以節省運算。
- 變換容許時間：發布的變換被視為有效的時間長度，預設 1 秒是合理值；調高有助於處理通訊延遲但可能使用過時的位置，調低則確保資料較新但可能造成抖動。

### 初始位姿：最常見的問題

AMCL 啟動時需要一個初始位姿的猜測。若沒有給，粒子會撒滿整張地圖，可能需要機器人移動一段距離才能收斂，甚至收斂到錯誤位置（第 11 章的多峰問題）。

- 手動設定：在 RViz 中用「2D Pose Estimate」工具點選機器人的大致位置與朝向。這是最常用的做法。
- 固定起點：讓機器人總是從同一個位置（例如充電站）啟動，並在設定中寫死初始位姿。這是第 27.5 節提過的實務技巧。
- 視覺標記輔助：用 AprilTag 提供絕對位姿（第 24.3 節），自動完成初始化。

### 量一量：全域定位要幾步才收斂

29.5 節說明了 AMCL 的原理與它與 SLAM 的分工。這一節實際跑一次全域定位——機器人完全不知道自己在哪，從均勻散布的粒子開始，看它要走幾步才能確定位置。

實驗設定：同樣的 10×7 公尺場景，已有先驗地圖。4,000 個粒子隨機散布在所有可通行的位置上，朝向也完全隨機。機器人沿一條約 5.5 公尺的路徑移動，每一步以 60 束的光達量測更新權重並重取樣。

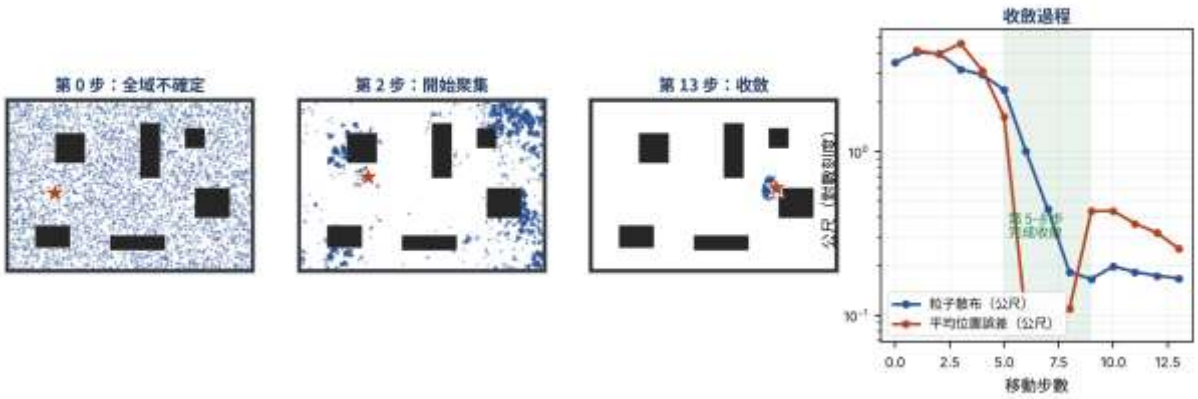



**圖 29-4　AMCL 全域定位的收斂過程**

第 0 步時粒子鋪滿整個可通行區域，散布是 3.49 公尺——系統對自己的位置一無所知。第 2 步時粒子已經開始聚成幾團，散布降到約 3.0 公尺；這些團代表「幾個看起來都說得通的可能位置」。到第 13 步，粒子收斂成一小團，散布 0.17 公尺，平均位置誤差 0.26 公尺。

右邊的收斂曲線顯示一個值得注意的現象：前四步幾乎沒有進展（散布甚至一度略為上升），真正的收斂集中在第 5 到 8 步。原因是機器人在前幾步只看到走廊的一小段，而場景裡有好

幾處長得差不多；直到它移動到能同時看見兩面不同牆的位置，多個假設才被一次排除。

這個「要走一段路才定得出來」的特性，對專題操作有直接的意義：啟動 AMCL 之後不要期待它立刻正確，先用遙控讓機器人在原地附近走動幾公尺，等粒子雲收斂了再交給導航堆疊。很多「導航一啟動就亂衝」的狀況，原因只是定位還沒收斂就下了目標點。

> **實作時踩過的一個坑**
>
> 本實驗第一次跑的時候，粒子確實收斂了，卻收斂到錯誤的位置。
>
> 原因有兩個，都很典型。其一是運動更新寫錯了：里程計給的是世界座標的位移，但每個粒子的朝向都不同，必須先把位移轉到各粒子自己的機體座標再套用。直接把世界座標位移加到所有粒子上，等於假設它們朝向都一樣。
>
> 其二是沒有淘汰落在牆裡或地圖外的粒子。那些粒子的似然值是從被裁切的距離場查出來的，數值虛高，結果整團粒子一路飄出房間。
>
> 這兩個錯誤都不會讓程式當掉，只會讓定位「看起來在動但就是不對」——正是 29.7 節所說的、最難查的那一類問題。

## 29.6　Navigation Stack 導航

最後一步：讓機器人自己走到目標點。這是第 3.4 節導航系統架構的實際落地。

### Nav2 的整體架構

Nav2 是 ROS 2 的導航堆疊，由多個模組協同運作。一個導航目標從送出到執行，會流經行為樹導航器、規劃器、控制器、代價地圖、復原行為、速度指令，最後到達底盤。

**圖 29-5 一次導航任務的完整資料流**

① 使用者送出目標（在 RViz 點選，或由程式以動作介面送出）

↓

② 行為樹導航器：依據行為樹決定執行流程

↓

③ 全域規劃器：在全域代價地圖上規劃完整路徑

（輸入：靜態地圖 + AMCL 提供的當前位姿）

↓

④ 控制器：依局部代價地圖，把路徑轉為速度指令

（局部代價地圖即時納入光達偵測到的動態障礙）

↓

⑤ 發布 /cmd_vel → 底盤執行 → 機器人移動

↓

⑥ 若卡住或路徑失效 → 觸發復原行為（原地旋轉、後退、清除代價地圖）

↓

⑦ 抵達目標 → 回報成功；失敗則回報失敗原因

### 代價地圖：導航的核心資料結構

代價地圖套件運用感測器資訊，以佔據格的形式提供機器人環境的表示；格子儲存 0 到 254 之間的成本值，代表通過該區域的代價。成本 0 表示格子是自由的，254 表示致命佔據，而介於兩者之間的值則被導航演算法用來像位能場一樣引導機器人遠離障礙物。

代價地圖以多個圖層實作，每個圖層有特定功能，共同貢獻於格子的整體成本。常見的圖層包括：

- 靜態層：載入建好的地圖（29.4 節的產物）。
- 障礙層：即時納入光達偵測到的障礙，包含地圖上沒有的動態物體。
- 膨脹層：把障礙物向外擴張一圈安全距離。這一層極為重要——沒有它，規劃出的路徑會緊貼牆面，機器人必定擦撞。膨脹半徑應設為機器人半徑加上安全餘裕。

### 全域與局部的分工

- 全域代價地圖：涵蓋整張地圖，以靜態地圖為主，供全域規劃器算出完整路徑。更新頻率低。
- 局部代價地圖：只涵蓋機器人周圍數公尺，以即時感測為主，供控制器閃避動態障礙。更新頻率高。

這個「全域慢而完整、局部快而即時」的分工，同學應該覺得熟悉——它與第 15 章 Cartographer 的雙子系統、第 18 章 LOAM 的雙層架構、第 26 章無人機的分層設計，是同一個系統設計模式。

## 29.7　系統性除錯：當機器人不動的時候

本章最實用的一節。導航系統元件眾多，出問題時若沒有系統性的排查順序，很容易亂槍打鳥浪費時間。

### 排查的正確順序

建議的除錯順序是：依序檢查生命週期狀態、TF、地圖伺服器、AMCL、代價地圖、規劃器輸出、控制器輸出，最後才是底盤運動。

更重要的是一個常被忽略的原則：規劃器與控制器的調參，應該等到生命週期、TF、定位、地圖與代價地圖都健康之後才進行。

這個順序背後的邏輯是：下游的問題往往源於上游。若 TF 有問題，再怎麼調規劃器參數都沒用。同學務必養成「由上游往下游排查」的習慣。

**圖 29-6 導航系統的除錯檢查表**

① 生命週期狀態：所有 Nav2 節點是否都已啟動並進入 active 狀態？

↓ 通過

② TF:map → odom → base_link → laser 的鏈路是否完整？

↓ 通過

③ 地圖：map_server 是否成功載入？RViz 中看得到地圖嗎？

↓ 通過

④ 定位：AMCL 是否收斂？粒子雲是否聚集在正確位置？

↓ 通過

⑤ 代價地圖：全域與局部代價地圖是否正確顯示？障礙是否有膨脹？

↓ 通過

⑥ 規劃器：送出目標後，是否有規劃出路徑？

↓ 通過

⑦ 控制器：是否有發布 /cmd_vel？數值是否合理？

↓ 通過

⑧ 底盤：收到 /cmd_vel 後輪子是否轉動？方向是否正確？

### 常見症狀與對應的檢查點

| 症狀 | 最可能的原因 | 檢查順序 |
|---|---|---|
| 完全無反應 | 節點未啟動或未進入 active | ① |
| RViz 顯示變換錯 | TF 斷裂或外參錯誤 | ② |

| 症狀 | 最可能的原因 | 檢查順序 |
|---|---|---|
| 誤 | | |
| 機器人位置亂跳 | AMCL 未收斂或初始位姿錯誤 | ④ |
| 規劃不出路徑 | 目標在障礙上、代價地圖膨脹過大 | ⑤⑥ |
| 規劃出路徑但不動 | 控制器問題、/cmd_vel 未送達底盤 | ⑦⑧ |
| 一直撞牆 | 膨脹半徑太小、光達外參錯誤 | ②⑤ |
| 原地打轉 | 角度控制參數不當、里程計角度失準 | 29.3 節校正 |

**表 29-3　常見故障症狀與檢查順序**

### 每個階段都要留下證據

一個好的實作習慣是：每個檢查點都應該產出可驗證的證據——一段指令輸出、RViz 中一個看得見的座標框架、一個地圖檔、一台移動中的模擬機器人、一份乾淨的程式碼倉庫，或一段簡短的展示錄影。

這個習慣有三重價值：除錯時能快速定位問題在哪一段；報告與專題時有具體的成果可展示；而且累積下來就是一份完整的學習歷程。建議同學從本章開始就養成這個習慣——第 31 章的專題評量正需要這些證據。

至此，同學已經完成第一個從感測器到自主導航的完整系統。下一章我們把感測器換成相機，實作 RGB-D 與雙目的視覺 SLAM:RealSense 相機設定、相機校正、點雲產生、RTAB-Map 與 ORB-SLAM3 實作，並比較視覺 SLAM 與光達 SLAM 在同一場域的表現差異。

## 29.8　把三張圖變成一份開機檢查表

本章的三個實驗各自回答一個問題，合起來正好構成一套每次開機都該跑一遍的檢查流程。

| 順序 | 檢查 | 方法 | 合格標準 |
|---|---|---|---|
| 1 | 座標轉換 | 原地旋轉一圈建圖 | 牆是單線，不出現圓環 |
| 2 | 里程計 | 順逆時針各走一次 2 公尺方形 | 兩次終點都在 5 公分內 |
| 3 | 建圖 | 繞場一圈 | 障礙格數接近基準值，牆不變厚 |
| 4 | 定位 | 在已知地圖上做全域定位 | 5 至 10 步內粒子散布小於 0.3 公尺 |
| 5 | 導航 | 下一個 3 公尺外的目標點 | 能抵達且不碰撞 |

**表 29-4　開機檢查表**

這份檢查表的順序不是隨意排的，而是依「錯誤會往下游傳播」的方向排列。座標轉換錯了，里程計校正的結果就沒有意義；里程計不準，建出來的地圖就會歪；地圖歪了，定位就收斂不了；定位不準，導航一定失敗。從下游開始查，會浪費大量時間在症狀上。

### 每一項該花多久

第一項三十秒、第二項十分鐘、第三項五分鐘、第四項兩分鐘、第五項兩分鐘——整套跑完不到二十分鐘。相較於專題後期花三天找「機器人為什麼會撞牆」，這是極划算的投資。

更重要的是，這份檢查表把「感覺怪怪的」變成了可以記錄的數字。障礙格數、終點誤差、粒子散布——這三個數字每次都記下來，就能看出系統是不是在退化（輪胎磨損、感測器鬆動、地面變髒）。這與第 19.6 節巡檢機器人的地圖維護是同一個思路：會長期運行的系統，需要可追蹤的健康指標。

### 給教學現場的建議

對帶實作課的老師，這份檢查表還有一個額外的用途：它可以當成評分的客觀依據。與其評

「機器人有沒有跑起來」（受運氣影響很大），不如評「這五項檢查的數字是多少」——前者是二元的，後者是連續的，也更能區分出學生真正理解到哪一層。

## 公式或流程

### 一、差速驅動的里程計計算

設在一個取樣週期內，左右輪各走了 d_L 與 d_R（由編碼器脈衝數乘上每脈衝距離得到），輪距為 b，則機體的運動為：

$$\Delta s = (d_R + d_L) / 2 \quad (\text{前進距離})$$

$$\Delta\theta = (d_R - d_L) / b \quad (\text{轉角})$$

再依第 10.3 節的 2D 位姿複合公式更新位姿：

$$x \leftarrow x + \Delta s \cdot \cos(\theta + \Delta\theta/2)$$

$$y \leftarrow y + \Delta s \cdot \sin(\theta + \Delta\theta/2)$$

$$\theta \leftarrow \text{normalize}(\theta + \Delta\theta)$$

式中取 θ + Δθ / 2 是為了用「這段運動的平均朝向」來計算位移，比直接用起始朝向精確。normalize 則是第 10.3 節強調的角度環繞處理，不可省略。

注意 Δθ 的分母是輪距 b——這解釋了為什麼 29.3 節說「輪距設錯對建圖的影響比輪徑更嚴重」：輪距誤差直接造成角度誤差，而角度誤差會被後續移動放大。

### 二、每脈衝距離的換算

編碼器輸出的是脈衝數，換算為距離需要：

$$\text{每脈衝距離} = (\pi \times D) / (N \times G)$$

其中 D 為輪徑、N 為編碼器每圈脈衝數、G 為減速比（若編碼器裝在馬達軸上）。這個常數是里程計精度的基礎，務必以實測校正而非只依規格書。

## 三、輪徑與輪距的校正

**圖 29-7　里程計參數校正流程**

【輪徑校正】

① 讓小車沿直線行走一段固定距離（建議 3 公尺以上）

② 以捲尺量測實際距離 L_real，讀取里程計距離 L_odom

③ 修正：新輪徑 = 舊輪徑 × （L_real / L_odom）

④ 重複驗證，直到誤差小於 1%

【輪距校正】

① 讓小車原地旋轉 10 圈（順時針與逆時針各做一次）

② 讀取里程計累計角度 θ_odom，理論值為 3600 度

③ 修正：新輪距 = 舊輪距 × （θ_odom / 3600°）

④ 重複驗證，直到誤差小於 2%

【順序】先校輪徑、再校輪距；兩者都完成後才開始建圖

## 四、代價地圖的膨脹半徑

膨脹層的關鍵參數是膨脹半徑，它決定障礙物向外擴張多遠：

$$膨脹半徑 \geq 機器人外接圓半徑 + 安全餘裕$$

設定過小，規劃出的路徑會緊貼牆面導致擦撞；設定過大，狹窄通道會被完全封死而規劃不出路徑（表 29-3 的常見症狀）。實務上先設為機器人半徑的 1.2 至 1.5 倍，再依場域的通道寬度微調。

## 五、從零到自主導航的完整流程

**圖 29-8　2D SLAM 小車的完整實作流程**

【階段一：硬體與座標】

① 組裝硬體 → ② 撰寫 URDF → ③ 驗證 TF 樹完整

↓

【階段二：里程計】

④ 實作編碼器讀取與里程計計算 → ⑤ 校正輪徑與輪距（圖 29-4）

↓

【階段三：建圖】

⑥ 啟動 SLAM → ⑦ 緩慢遙控建圖 → ⑧ 評估品質 → ⑨ 儲存地圖

↓

【階段四：定位】

⑩ 載入地圖 → ⑪ 啟動 AMCL → ⑫ 設定初始位姿 → ⑬ 確認收斂

↓

【階段五：導航】

⑭ 啟動 Nav2 → ⑮ 調整代價地圖 → ⑯ 送出目標 → ⑰ 觀察執行

↓

【階段六：除錯與調校】依圖 29-3 的順序排查與優化

# 案例

## 案例 29-1　三公分的代價：一個未校正的輪徑

這是課堂上真實發生過、也極具代表性的案例。

一組同學的建圖結果總是「怪怪的」：地圖看起來完整，牆線也算筆直，但把地圖上的教室尺寸與實際量測比較，發現地圖上的教室比實際大了約 8%。更奇怪的是，他們花了整個下午調整 SLAM 的各種參數，結果毫無改善。

問題出在輪徑。他們使用的輪子標稱直徑 65 公釐，但實際裝上後因為橡膠輪胎受壓變形，有效直徑只有約 60 公釐。里程計以 65 公釐計算，因此「以為」自己走得比實際遠，建出的地圖就被等比例放大。

這個案例的教訓有三層：

- 第一，規格書不等於實際值。輪胎變形、製造公差、安裝誤差都會造成差異，必須實測校正（29.3 節與公式三）。
- 第二，系統性誤差不會被 SLAM 修正。SLAM 能修正隨機的漂移（靠掃描匹配與閉環），但對「每一步都固定放大 8%」的系統性誤差無能為力——因為所有資料都一致地錯，系統無從察覺。
- 第三，調參之前先檢查基礎。這正是 29.7 節強調的排查順序：上游的問題，調下游的參數永遠解決不了。

建議教師在課程一開始就要求同學完成里程計校正並記錄數據——這半小時能省下後續數小時的困惑。

## 案例 29-2　從模擬到實體：預期之外的落差

第 28 章建議「先模擬、再實體」，但同學常會遇到一個挫折：在 Gazebo 中跑得完美的系統，搬到實體小車上卻問題百出。這個落差來自哪裡？

主要有四個來源：

- 感測器雜訊：模擬中的光達雖然可加雜訊，但真實光達還有第 8 章談過的各種問題——玻璃穿透、黑色物體反射微弱、量測距離的隨機跳動。這些在模擬中往往未被完整建模。
- 里程計品質：模擬中的輪速通常是理想的；真實世界有打滑、地面材質變化、輪胎變形（案例 29-1）、以及通訊丟包造成的資料斷續。
- 時間同步：模擬中所有節點共用同一個模擬時鐘，時間戳記完美一致；真實系統中，微控制器、單板電腦、感測器各有時鐘，同步問題會浮現（第 5、10、27 章反覆強調的工程細節）。
- 環境複雜度：模擬場景通常乾淨簡單；真實教室有桌椅腳（第 8 章的角解析度問題）、玻璃窗、走動的人。

但這不表示模擬沒有價值——恰恰相反。模擬的價值在於：它讓你先確認「邏輯與架構是對的」，把 TF 設定、話題連接、參數配置等問題先解決掉。搬到實體時，剩下的就只是「真實世界的物理問題」，問題範圍大幅縮小。

務實的建議是：模擬中通過的系統，實體上仍要預留一到兩倍的除錯時間。這不是失敗，而是工程的常態。

## 案例 29-3　為什麼機器人規劃出路徑卻不動

這是導航實作中最令人困惑的症狀之一：RViz 中明明看到一條漂亮的綠色路徑，機器人卻紋風不動。

依 29.7 節的排查順序，問題必然在第七或第八個檢查點——控制器或底盤。常見的原因包括：

71. 控制器沒有發布速度指令：可能是控制器節點未啟動、或參數設定錯誤導致它認為「無法執行這條路徑」。檢查方法是直接監看 /cmd_vel 話題有無資料。

72. 速度指令發布了但底盤沒收到：話題名稱不符（第 28.4 節提過的重新對應問題）、或底盤驅動節點未訂閱。

73. 底盤收到了但不動：速度指令的數值太小，低於馬達的啟動門檻；或馬達驅動器未啟用、電源未開。

74. 速度限制設得太保守：Nav2 的最大速度參數設得過小，或加速度限制過嚴，使機器人「動得太慢看起來像不動」。

這個案例的價值在於它示範了系統性除錯的威力：同樣的症狀，若隨機猜測可能花數小時；依循「監看 /cmd_vel → 判斷是控制器還是底盤 → 再往下細查」的順序，通常十分鐘內就能定位。

這也呼應第 28.6 節閱讀開源專案的方法——順著資料流追蹤。無論是讀程式碼還是除錯系統，「follow the data」都是最可靠的策略。

## 實作活動

### 實作 29-1 從建圖到自主導航的完整實作

目的：完成本書第一個端到端的系統實作。依圖 29-5 的六個階段循序進行，每階段都產出可驗證的證據。本實作可在模擬中完成，有硬體者建議先模擬再實體。

#### 階段一：座標系統設定（全體）

147. 若使用現成平台，檢視其 URDF，找出 base_link 與 laser 之間的變換，並與實際安裝位置核對。若自組平台，則撰寫自己的 URDF。

148. 啟動系統並產生 TF 樹圖，確認 map → odom → base_link → laser 鏈路完整無斷裂。

149. 外參驗證：把機器人靜置在距離牆面已知距離處（例如 1 公尺），在 RViz 中檢查掃描點是否落在正確位置。若有偏差，回頭修正 URDF。

150. 刻意破壞實驗：故意把 URDF 中光達的位置改錯 20 公分，觀察 RViz 中掃描點如何錯位——親身理解外參的重要性。改回正確值後繼續。

**階段二：里程計校正（有硬體者必做，模擬者可略）**

75. 依圖 29-4 執行輪徑校正：直線行走 3 公尺，比較里程計與捲尺實測值，修正參數後重驗。記錄修正前後的誤差百分比。

76. 依圖 29-4 執行輪距校正：原地旋轉 10 圈，比較累計角度與 3600 度，修正輪距後重驗。

77. 平滑度檢查：遙控機器人移動，觀察里程計輸出是否連續平滑、有無跳變（29.3 節的品質要求）。

**階段三：建圖（全體）**

78. 啟動 slam_toolbox 建圖，緩慢遙控機器人繞行場域一圈，在 RViz 中觀察地圖成長。

79. 依第 13.7 節評估地圖品質：牆是否筆直？迴路是否閉合？量測地圖上的關鍵尺寸與實際比較，計算誤差百分比。

80. 速度對照實驗：分別以「很慢」與「較快」的速度各建一次圖，比較品質差異，驗證第 14.6 節「開慢一點」的建議。

81. 儲存地圖，檢視產生的 .yaml 與 .pgm 兩個檔案，用看圖軟體開啟 .pgm 觀察佔據格的三態表示。

**階段四：定位（全體）**

82. 啟動 map_server 載入剛才儲存的地圖，確認 RViz 中能看到地圖。

83. 啟動 AMCL，在 RViz 中顯示粒子雲。觀察未設定初始位姿時，粒子如何散布全圖。

84. 用 RViz 的位姿估計工具設定初始位姿，遙控機器人移動數步，觀察粒子雲如何收斂——這是第 11 章案例 11-2 的親身重現。

85. 綁架測試：把機器人（或模擬中的機器人）瞬移到地圖上的另一個位置，觀察 AMCL 是否能重新收斂，以及花了多久。

**階段五：自主導航（全體）**

1. 啟動 Nav2，在 RViz 中顯示全域與局部代價地圖，觀察障礙物的膨脹效果。
2. 用 RViz 的目標點工具送出一個導航目標，觀察規劃出的路徑，以及機器人如何沿路徑移動。
3. 動態避障測試：在機器人行進途中放置一個障礙物（模擬中可加入物件），觀察局部代價地圖如何更新、機器人如何繞行。
4. 膨脹半徑實驗：分別把膨脹半徑設為機器人半徑的 0.8 倍、1.2 倍、2.5 倍，觀察路徑的變化與是否出現「規劃不出路徑」的情況（公式四的討論）。
5. 復原行為觀察：把機器人困在死角或送出無法抵達的目標，觀察 Nav2 觸發哪些復原行為。

**階段六：除錯練習（大學部）**

1. 人為製造故障：依表 29-3 選三種症狀，分別人為製造（例如關閉某個節點、改錯話題名稱、把膨脹半徑設到極大），再依圖 29-3 的順序排查，記錄從發現症狀到定位問題所花的時間。
2. 建立你自己的檢查表：依本次實作的經驗，整理一份適合你的平台的除錯檢查表，供日後與同學使用。

**討論問題**

64. 外參改錯 20 公分後，RViz 中的掃描點如何錯位？若不修正就建圖，地圖會出現什麼問題？
65. （有硬體者）輪徑校正前後的誤差各是多少？這個誤差若不修正，對地圖尺寸的影響有多大？（對照案例 29-1）
66. 慢速與快速建圖的品質差多少？從第 13.3 節掃描匹配的原理解釋這個差異。

67. 綁架測試中 AMCL 花了多久收斂？若地圖中有兩個外觀相似的區域，你預期會發生什麼？（提示：第 11 章的多峰分布）

68. 膨脹半徑過大時，發生了什麼？過小時呢？你的場域最合適的值是多少？

69. （大學部）在除錯練習中，依系統性順序排查比隨機猜測快多少？這對你日後的工程習慣有什麼啟示？

**延伸挑戰（大學部）**

其一：完整的系館場域導航系統——在系館的一個樓層完成建圖，設定數個具名的目標點（如「實驗室門口」「電梯口」「茶水間」），並撰寫一個 Python 節點，讓使用者輸入地點名稱後機器人自動導航前往。這實際上是第 24 章語意導航的簡化版，也是第 31 章專題的良好起點。其二：里程計融合實驗——若平台有 IMU，設定 robot_localization 的 EKF 節點融合輪速與 IMU，比較「純輪速里程計」與「融合後里程計」在快速轉彎與地面打滑情況下的差異，量化 IMU 帶來的改善。這是第 7、11 章理論的直接應用。

## 延伸閱讀

92. Nav2 官方文件（docs.nav2.org）.（架構說明、設定指引與教學，本章主要依據）

93. Nav2 官方文件之 Setup Guides:Setting Up Sensors、Mapping and Localization.（從零建立機器人的完整步驟）

94. slam_toolbox 開源專案（GitHub: SteveMacenski/slam_toolbox）.（ROS 2 預設的 2D SLAM，程式規模適中適合閱讀）

95. S. Macenski et al., "The Marathon 2: A Navigation System," IROS, 2020.（Nav2 的官方論文，說明設計理念）

96. ROS 2 官方文件之 URDF 教學（docs.ros.org）.（機器人描述檔的撰寫方法）

97. ROBOTIS TurtleBot3 e-Manual 之 SLAM 與 Navigation 章節.（教學平台的完整實作指引）

98. robot_localization 套件官方文件.（以 EKF 融合多感測器提供里程計，29.3 節的進階內容）

## 小結

92. 2D SLAM 小車由底盤馬達、編碼器、2D 光達、微控制器與單板電腦組成；微控制器負責毫秒級即時控制、單板電腦負責 SLAM 運算，這個分工與第 25 章的雙晶片架構思想一致。

93. URDF 描述機器人結構與感測器安裝位置（外參）；TF 樹的分工是：SLAM 或 AMCL 發布 map→odom、里程計系統發布 odom→base_link、robot_state_publisher 依 URDF 發布 base_link 到各感測器。常見錯誤為斷裂、外參填錯、重複發布與時間戳記不同步。

94. 里程計由編碼器讀值經差速驅動公式計算並發布；輪徑與輪距必須實測校正，輪距誤差因直接造成角度誤差而影響更嚴重；里程計不需絕對準確但必須平滑連續，可用 EKF 融合 IMU 提升穩健度。

95. slam_toolbox 是 Nav2 的預設 SLAM 演算法、參數簡潔適合教學；建圖流程須先確認 TF 完整，緩慢遙控並持續評估品質；地圖以 .yaml 與 .pgm 兩檔儲存。

96. AMCL 以粒子濾波在已知地圖中定位、不建立地圖，與 SLAM 同樣提供 map→odom 變換因而可無縫切換；關鍵參數包含粒子數、運動雜訊模型、更新門檻與變換容許時間；初始位姿必須設定，否則可能收斂緩慢或錯誤。

97. Nav2 的導航目標流經行為樹導航器、規劃器、控制器、代價地圖、復原行為與速度指令；代價地圖以 0 至 254 的成本值表示，0 為自由、254 為致命佔據，並以靜態層、障礙層、膨脹層等多層疊加。

98. 除錯必須依系統性順序：生命週期、TF、地圖、定位、代價地圖、規劃器、控制器、底盤；規劃器與控制器的調參應等到上游都健康之後才進行；每個檢查點都應留下可驗證的證據。

## 承先啟後

本章把三個最容易卡住專題的環節量化了：里程計方面，輪距設錯 4% 加上左右輪不等徑 1.5%，走一個 2 公尺見方的方形就會偏 1.13 公尺，而且順時針與逆時針的誤差差了三倍多（1.13 對 0.32 公尺）——只跑一個方向會被騙；座標轉換方面，光達位置差 0.20 公尺或角度差 5 度，都會讓障礙格數從 992 增加到兩千多、正確率掉到七成多，而且兩者的症狀從外觀分不出來；定位方面，全域定位的粒子散布從 3.49 公尺收斂到 0.17 公尺，但真正的收斂發生在第 5 到 8 步——啟動後必須先走一段路，定位才會穩。

本章完成了第一個從感測器到自主導航的端到端系統。同學應該能感受到：前面二十七章的理論，在這一章全部匯流成一台真正會動的機器人——第 8 章的光達、第 10 章的座標轉換、第 11 章的粒子濾波、第 12 章的圖優化、第 13 章的佔據格、第 15 章的 SLAM 系統、第 28 章的工具鏈，缺一不可。

下一章，我們把感測器換成相機。RealSense 與 ZED 等相機的設定、相機校正、RGB-D 點雲產生、RTAB-Map 與 ORB-SLAM3 的實作，以及最重要的——視覺 SLAM 與光達 SLAM 在同一場域的對照比較。第 30 章將讓同學親身驗證第 8、9 章反覆討論的感測器互補性。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

## 第 30 章

## RGB-D / 雙目相機 SLAM 實作

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

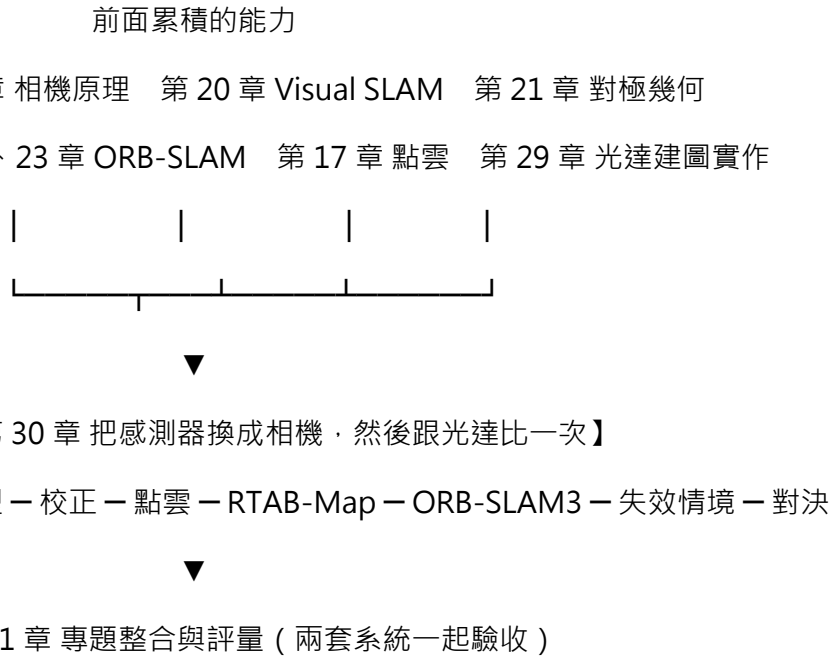
## 本章在全書的位置

這是第八篇的最後一章技術實作，也是全書唯一一次讓兩套感測器在同一個場域、走同一條路徑、正面比一次。

上一章的光達小車，困難在「接線」——座標轉換、里程計參數、一層壓一層的系統整合。本章換成相機之後，困難換了一種性質：相機不像光達那樣自帶光源、也不像光達那樣直接量距離。它的深度是「算」出來的，而凡是算出來的東西，就會在某些條件下算得很準、在另一些條件下算得離譜，而且常常不會告訴你它算錯了。

本章的三張圖，就是要把這三種「條件」量出來：圖 30-1 實測雙目深度誤差怎麼隨距離成長，順便回答「有效範圍五公尺」這句話到底是什麼意思；圖 30-2 把深度與彩色沒對齊的後果畫出來，並指出它的嚴重程度與距離的關係正好和深度誤差相反；圖 30-4 則是本書醞釀了二十幾章的對決——貼滿海報、白牆、玻璃、昏暗四種房間，光達與視覺各跑一圈。

### 圖 30-0　第 30 章的連結地圖




## 學習目標

上一章我們用光達完成了完整的建圖與導航。本章把感測器換成相機，實作視覺 SLAM——

並在最後做一件本書醞釀已久的事：在同一個場域，讓光達與視覺兩套系統正面對決。學完本章之後，同學應能夠達成下列目標：

92. 比較 RealSense、ZED、OAK-D 三類深度相機的原理與適用場合。
93. 完成相機校正並驗證內參與畸變係數的正確性。
94. 由 RGB 與深度影像產生彩色點雲，並理解對齊的重要性。
95. 執行 RTAB-Map 建圖並儲存三維地圖。
96. 執行 ORB-SLAM3 並完成軌跡評測。
97. 規劃室內場景的測試流程，並辨識視覺 SLAM 的典型失效情境。
98. 在同一場域比較視覺 SLAM 與光達 SLAM 的表現差異。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 深度影像 | Depth Image | 每個像素記錄該點距離的影像。 |
| 深度對齊 | Depth Alignment | 把深度影像對齊到彩色影像的座標系。 |
| 彩色點雲 | Colored Point Cloud | 每個三維點帶有 RGB 顏色的點雲。 |
| 相機資訊 | CameraInfo | ROS 中攜帶內參與畸變係數的訊息。 |
| 視覺里程計 | Visual Odometry | 由影像估計相機運動，提供 odom→base_link。 |
| RGB-D 里程計 | RGBD Odometry | RTAB-Map 提供的以 RGB-D 影像估計位姿的模組。 |
| 外觀式建圖 | Appearance-based Mapping | 以影像外觀相似度偵測閉環的建圖方法。 |
| 記憶體管理 | Memory Management | RTAB-Map 控制地圖規模以維持即時性的機制。 |
| 資料庫檔 | Database File | RTAB-Map 儲存地圖與影像的檔案格式。 |

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 定位模式 | Localization Mode | 載入既有地圖只做定位、不再擴充地圖。 |
| 全域快門 | Global Shutter | 全畫面同時曝光，適合運動中的視覺。 |
| 主動紅外投射 | IR Projector | 投射紋理圖案以協助立體匹配的裝置。 |
| 近似同步 | Approximate Sync | 容許不同話題時間戳記略有差異的同步方式。 |

## 30.1 RealSense / ZED / OAK-D 相機介紹

第 9.3 節介紹過三種取得深度的方式，本節聚焦實際可用的產品與選型考量。

### 三大類深度相機

- RealSense 系列：採主動紅外雙目（第 9.3 節）。它以改良的硬體與軟體達成室內外皆可用的十公尺以上量測範圍，並具備全域快門與寬視野，對機器人與無人機應用相當有用。部分型號內建 IMU，可直接支援視覺慣性 SLAM（第 23 章）。
- ZED 系列：採被動雙目，不投射紅外線，因此在戶外強光下仍可運作（這正好補上 RealSense 的弱點）。它一般能提供比 RealSense 更佳的深度精度與更寬的視野，但代價是需要 GPU 運算——ZED 的深度計算倚賴 CUDA，必須搭配 NVIDIA 顯示卡。
- OAK-D 系列：雙目加上內建的 AI 推論晶片。它能在相機端直接執行物件偵測等神經網路，把運算從主機卸載出去——這與第 25 章邊緣運算的思想一致，對運算資源有限的平台特別有價值。

### 選型的四個考量

256. 使用環境：室內為主可用 RealSense；需要戶外強光下運作則考慮 ZED 或被動雙目。第 9.3 節提過主動紅外投射會被陽光淹沒。

257. 運算資源：ZED 需要 NVIDIA GPU;RealSense 的深度由相機內部處理，對主機需求較低；OAK-D 甚至能把 AI 推論也搬到相機上。

258. 是否需要 IMU：若要做視覺慣性 SLAM（第 23 章），選內建 IMU 的型號能省去外部 IMU 的同步與外參校正麻煩。

259. 有效範圍：以 RealSense D435 為例，它的量測範圍達 10 公尺，但在 5 公尺範圍內精度較佳——實務上常把有效範圍設為 5 公尺。這呼應第 9.6 節「深度誤差隨距離平方成長」的討論。

| 類型 | 深度原理 | 強光下 | 運算需求 | 特色 |
|---|---|---|---|---|
| RealSense | 主動紅外雙目 | 較弱 | 低（相機內處理） | 部分型號內建 IMU |
| ZED | 被動雙目 | 可用 | 需 NVIDIA GPU | 精度與視野較佳 |
| OAK-D | 雙目 + AI 晶片 | 可用 | 極低（相機端推論） | 可在相機上跑神經網路 |

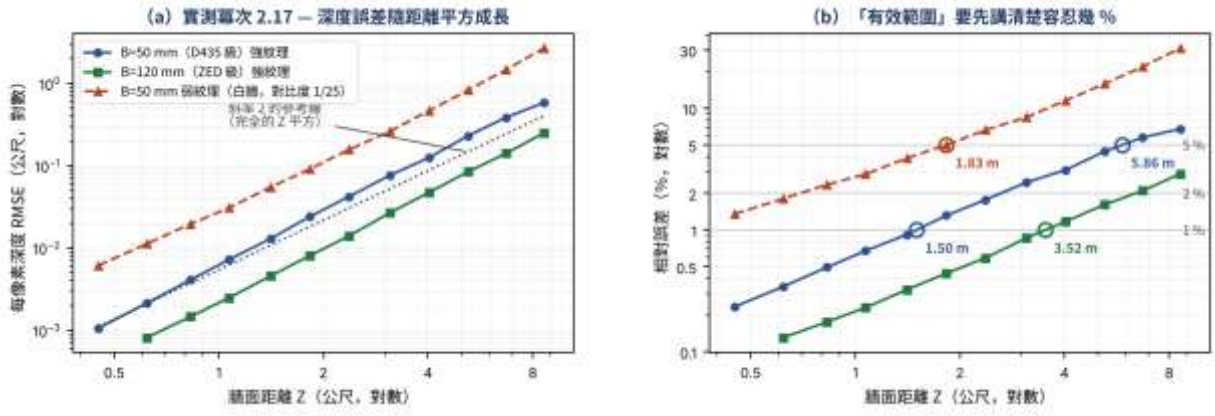
**表 30-1　三類深度相機的比較**

在往下談校正與點雲之前，先把選型這一節最關鍵、卻也最常被含糊帶過的一句話量出來：「有效範圍五公尺」。

前面說 D435 的量測範圍達十公尺、但五公尺內精度較佳。這句話其實不完整——精度從來不是在某個距離突然「變差」，它是連續惡化的，所以「有效範圍」這四個字只有在講清楚「容忍幾 % 的誤差」之後才有意義。下面這組數字是實際跑出來的，不是套公式。

實驗這樣做：用隨機斑點紋理產生一面傾斜的牆，依雙目幾何算出每個像素應有的視差，渲染出左右兩張 640×480 影像，加上標準差 2.5 灰階的感測雜訊，再交給 OpenCV 的 StereoSGBM 去匹配，最後把它算出來的視差換回深度，和真值逐像素相減。三組條件：基線 50 mm（D435 等級）、基線 120 mm（ZED 等級）、以及同樣 50 mm 但把紋理對比度降到二十五分之一（模擬白牆）。

**圖 30-1　雙目深度誤差的實測成長：距離平方、基線、紋理三者的作用**




圖（a）是對數座標。三條線都是直的，而且斜率量出來是 2.17、2.19、2.05——這就是教科書上「深度誤差隨距離平方成長」那句話的實測版本。要注意的是這個平方關係並非來自任何人為設定：程式裡沒有任何一行寫過 Z 的平方，它是雙目幾何（深度與視差成反比）加上匹配誤差自然長出來的。基線 50 mm 的深度 RMSE 從 0.45 m 處的 1.1 公釐，一路長到 8.6 m 處的 58 公分，相差五百多倍。

圖（b）把同樣的數字換成相對誤差，「有效範圍」這句話就有了可以回答的形式。以 50 mm 基線、強紋理來說：要求 1 % 的相對精度，有效範圍只有 1.50 公尺；放寬到 2 % 是 2.61 公尺；放寬到 5 % 才終於接近課本上常說的五公尺（實測 5.86 公尺）。所以「D435 的有效範圍五公尺」這句話，隱含的其實是「可以接受 5 % 誤差」——在五公尺處那是 25 公分，用來避障綽綽有餘，用來做三維量測則完全不行。

第二條線回答了選型那一節的另一個問題：基線加大到底買到什麼。120 mm 基線（2.4 倍）在 1 % 精度下的有效範圍是 3.52 公尺，恰好是 50 mm 的 2.35 倍。這不是巧合：相對誤差正比於 $Z / B$，所以在固定精度要求下，有效距離與基線成正比。要把有效範圍加倍，最直接的辦法就是把兩顆鏡頭拉開一倍——這也是為什麼戶外用的雙目相機都又寬又大。

第三條線是本章後面「五種典型失效情境」的第一個量化證據。同一顆相機、同一個距離，只是把紋理對比度降到二十五分之一，五公尺處的誤差就從 4.4 % 惡化到 15.6 %，差了 3.5 倍；1 % 的有效範圍更是直接掉到一公尺以內。但真正要注意的不是誤差變大，而是下面這

件事。

**白牆不會給你「無效」，它會給你一個錯得很離譜的數字**

弱紋理那一組，SGBM 回報的有效像素比例仍然是 99.5 %。

也就是說，面對白牆時相機並沒有拒絕作答，它照樣填滿整張深度圖，只是填進去的值在五公尺處平均錯了 80 公分。

這是視覺感測器最危險的性格：光達打到吸光的黑色物體會直接回報無效點，你一眼就看得出來；相機卻會「有自信地答錯」。

實務上的對策是不要只看深度圖有沒有洞，而要看驅動提供的信心度或視差有效遮罩，並對同一面牆重複量測看數值穩不穩。

### 教學上的建議

對大學與高中的教學而言，建議優先考慮內建 IMU 的 RealSense 型號：它價格相對親民、ROS 2 驅動成熟、社群資源豐富、且不需額外的 GPU。等到專題階段需要戶外運作或更高精度時，再考慮 ZED。

## 30.2 相機校正

第 9.2 節說明過相機校正的原理，本節聚焦實作要點——這是視覺 SLAM 的第一道關卡，也是精度的基礎。

### 出廠參數 vs 自行校正

現代深度相機出廠時已經過校正，並會透過 ROS 的 CameraInfo 訊息提供內參與畸變係數。對多數應用而言，直接使用出廠參數即可。

但在三種情況下，建議自行校正：

- 相機受過碰撞或鏡頭被調整過。

- 使用一般的網路攝影機或自組的雙目系統（沒有出廠參數）。
- 對精度要求極高的研究應用，想確認出廠參數的準確性。

### 校正流程回顧

依第 9.2 節的張氏棋盤格法：

3. 列印棋盤格並貼在硬板上（務必保持平整，彎曲會嚴重影響校正精度）。
4. 從不同角度、距離拍攝 15 至 20 張，涵蓋畫面各區域——尤其別忘了邊角，徑向畸變在邊角最嚴重。
5. 執行校正程式，取得內參矩陣 K 與畸變係數。
6. 檢查重投影誤差，應小於 1 像素；若偏大，增加影像數量或重拍。
7. 驗證：對含直線的場景做去畸變，確認彎曲的直線被拉直。

### ROS 中的校正工具

ROS 提供 camera_calibration 套件，能以互動方式完成校正：對著棋盤格移動相機，工具會即時提示涵蓋度（左右、上下、遠近、傾斜），達標後即可計算並儲存參數。這比自己寫程式方便得多，是實作首選。

### 雙目的額外校正

雙目相機除了各自的內參，還需要兩相機之間的相對位姿（外參）——這決定了基線長度，直接影響深度計算（第 9.6 節的公式 $Z = f \cdot b / d$）。雙目校正同樣可用 camera_calibration 套件完成，但需要同時拍攝左右影像。

### 常見的校正陷阱

- 棋盤格不平整：貼在紙板或曲面上會導致校正結果錯誤。務必用硬質平板。
- 影像涵蓋不足：只在畫面中央拍攝，邊角的畸變係數就估不準。務必讓棋盤格出現在畫面的各個角落。

- 解析度不一致：校正時與實際使用時的影像解析度必須相同，否則內參的像素單位會不匹配。
- 忘記更新：校正完成後，務必確認 SLAM 系統實際讀取的是新的參數檔，而非舊的或預設值。

## 30.3 RGB-D 點雲產生

有了 RGB 影像、深度影像與相機內參，就能產生彩色點雲——這是視覺 SLAM 的核心資料形式，也是第 17 章點雲處理技術的來源之一。

### 從深度影像到三維點

原理是第 9.1 節針孔投影模型的逆運算。已知某像素的座標與該點的深度，即可反推出它的三維座標（公式見本章公式一）。對整張深度影像的每個像素執行這個運算，就得到一片點雲。

再把對應像素的 RGB 顏色賦予每個點，就成為彩色點雲——這正是第 17.6 節點雲顏色化的簡化版：因為 RGB 與深度來自同一台相機，不需要處理跨感測器的外參與時間同步問題。

### 關鍵前提：深度對齊

這裡有一個初學者常忽略的細節：彩色相機與深度感測器在物理上位於不同位置，因此同一個像素座標在兩張影像中對應的並非同一個空間點。

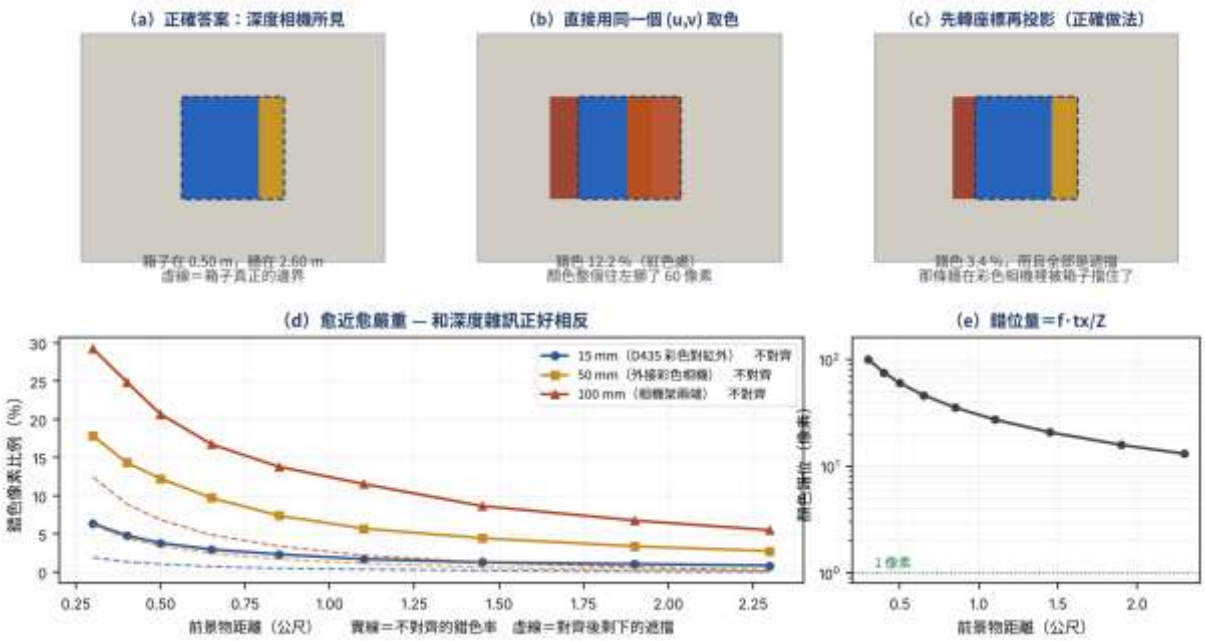
解決方法是深度對齊——把深度影像投影到彩色相機的視角，使兩者的像素一一對應。實務上這對應到相機驅動中的對齊設定，產生的話題會是「對齊到彩色影像的深度影像」；RTAB-Map 也正是訂閱這個對齊後的深度話題。

若忘了開啟對齊，產生的點雲會出現顏色與幾何錯位——牆的顏色跑到窗戶上，這與第 17.6 節提過的外參錯誤症狀相同。

「忘了開啟對齊，點雲就會顏色與幾何錯位」這句話，聽起來像是美觀問題。把它量出來之後會發現它不只是美觀問題，而且它的嚴重程度與距離的關係，和剛剛圖 30-1 的結論正好相反。

實驗場景很簡單：一面 2.6 公尺遠的牆，前方放一個箱子，箱子正面漆藍色、右側漆橘色，牆是米白色。深度感測器與彩色鏡頭在物理上錯開 tx。上色分兩種做法：正確做法是把深度像素反投影成三維點、轉到彩色相機座標、再用彩色內參投影回去取色；錯誤做法則是直接拿同一組 (u, v) 去彩色影像取色。

**圖 30-2　深度沒對齊到彩色，彩色點雲會錯成什麼樣**




圖（a）是正確答案，虛線框標出箱子真正的邊界。圖（b）是直接用同一個 (u, v) 取色的結果，紅色就是上錯顏色的像素。可以清楚看到整塊顏色往左挪了 60 個像素：箱子右側有一條帶狀區域被塗上了牆的顏色，而箱子左邊的牆面則被塗成了箱子的藍色。錯位量有一條很乾淨的公式，就是圖（e）畫的 f·tx / Z——這個例子裡是 600 × 0.050 ÷ 0.50 = 60 像素。整張畫面有 12.2 % 的像素上錯了顏色。

圖（c）是先轉座標再投影的正確做法。錯色率從 12.2 % 掉到 3.4 %——但沒有掉到零，而剩下的那 3.4 % 值得單獨講一下。

把遮擋單獨算出來之後發現：對齊之後剩下的錯色像素，100 % 都落在遮擋區，非遮擋的殘餘是 0.00 %。原因是幾何上的必然——靠近箱子邊緣的那一條牆，在彩色相機的視角裡被箱子擋住了，彩色影像上根本沒有它的顏色可取。這條帶子無論對齊做得多好都補不回來，它是兩顆鏡頭分處兩地這件事本身的代價。這也解釋了為什麼實務上的彩色點雲在物體邊緣總有一圈顏色可疑的點：那不是你的程式寫壞了。

圖（d）是本節最重要的一張。三條實線是三種鏡頭間距（15 mm 的 D435 彩色對紅外、50 mm 與 100 mm 的外接彩色相機）在不同物距下的錯色率，虛線是對齊後剩下的遮擋下限。三條線都是「愈近愈嚴重」：D435 那條在 0.30 公尺處錯色 6.3 %，到 2.30 公尺只剩 0.84 %；100 mm 那條在 0.30 公尺處更高達 29.3 %，也就是將近三分之一的點顏色是錯的。

**兩種誤差往相反方向拉，所以沒有一個「最佳距離」**

圖 30-1 說：深度誤差隨距離平方成長，所以愈遠愈糟。

圖 30-2 說：顏色錯位量是 f·tx / Z，所以愈近愈糟。

以 D435 為例，從 0.5 公尺退到 2.3 公尺，深度 RMSE 大約放大 38 倍，錯色率卻縮小到四分之一。

兩者的方向相反，代表「把相機拿近一點比較準」這個直覺只對了一半：它對深度對，對顏色錯。

真正的結論是——先決定這份點雲要拿來做什麼。要做幾何量測就離遠一點不行、離近一點顏色就不能信；要做語意或材質辨識則反過來。兩個都要的話，唯一的解法是把對齊確實打開，把兩條曲線中可以消掉的那一條消掉。

## ROS 中的資料流

**圖 30-3 RGB-D 相機到點雲的資料流**

相機驅動節點

├──→ /color/image_raw 彩色影像

├──→ /aligned_depth_to_color/image_raw 對齊後的深度影像

├──→ /color/camera_info 內參與畸變係數

└──→ /imu（若相機內建） 慣性資料

↓

點雲產生節點（或 SLAM 系統內部）

依公式一把每個像素反投影為三維點，並賦予顏色

↓

└──→ /points PointCloud2 格式的彩色點雲（第 8.3 節）

### 資料量的現實

一張 640×480 的深度影像會產生約 30 萬個點，以 30 幀每秒計算，每秒就是 900 萬個點——這正是第 17.2 節必須降採樣的原因。實務上會先做體素降採樣，再交給後續處理。

另一個實務考量是有效範圍的裁切：如 30.1 節提過，深度相機在較遠處的精度快速下降，因此常設定一個最大有效距離（例如 5 公尺），超過的點直接捨棄——保留不可靠的遠點，反而會拉低整體精度。

## 30.4 RTAB-Map 實作

RTAB-Map 是 RGB-D SLAM 最常用的開源系統，也是本章的實作主角。

### RTAB-Map 是什麼

RTAB-Map（Real-Time Appearance-Based Mapping）是一套基於增量式外觀閉環

偵測器的 RGB-D 圖優化 SLAM 方法，設計為可通用於室內與室外環境。

它使用相機接收到的深度影像執行圖優化 SLAM，產生稠密的彩色點雲與相機里程計；它也能以詞袋與特徵偵測器即時完成閉環。它是一套支援 RGB-D 相機的圖優化 SLAM 框架，同時運用視覺外觀與深度資訊進行三維建圖與閉環偵測。

對照本書的架構，同學應該能立刻定位它：圖優化是第 12 章、詞袋閉環偵測是第 20、22 章、稠密點雲是第 17 章——RTAB-Map 是這些技術的整合實現。

### 三項實用特性

- 可用外部里程計：它允許你傳入自己的里程計話題，而非由它自行產生——若你有更精確的里程計（例如第 29 章的輪速里程計或融合 IMU 的結果），可直接使用。這對輪型機器人特別有價值。
- 可融合雷射：你也能使用自己的 2D 或 3D 雷射掃描來改善地圖品質。這意味著 RTAB-Map 能同時吃視覺與光達資料——正是第 8、9 章反覆強調的感測器互補在實作上的體現。
- 多種地圖格式：它相容於八叉樹與體素等資料結構，這些對三維規劃與多地圖網格化的效率都優於原始點雲。這呼應第 17.5 節三種地圖表示的討論。

### RGB-D 里程計

RTAB-Map 套件也提供名為 RGBD 里程計的視覺里程計，用來提供位姿資訊與 odom→base_link 變換，在室內運作已展現可靠的結果。

這一點很重要：即使機器人沒有輪速編碼器（例如手持相機掃描），RTAB-Map 也能自己產生里程計，完整運作。這讓它成為「一台相機就能做 SLAM」的實用方案。

### 執行流程

8. 啟動相機驅動，務必開啟深度對齊（30.3 節）。

9. 確認三個必要話題有資料：彩色影像、對齊後的深度影像、相機資訊。

10. 啟動 RTAB-Map，指定上述話題名稱與座標框架。首次執行建議加上「啟動時刪除資料庫」的參數，避免載入舊地圖造成混淆。

11. 在 RViz 或 RTAB-Map 自己的視覺化介面中觀察：即時點雲、相機軌跡、以及閉環偵測的發生。

12. 緩慢移動相機掃描整個空間，注意保持特徵豐富的視野。

13. 完成後儲存地圖（RTAB-Map 以資料庫檔案儲存，內含影像、特徵與位姿圖）。

14. 日後可切換到定位模式，載入既有地圖只做定位、不再擴充——這與第 29.5 節 AMCL 的角色相同。

### 關鍵參數

- 同步方式：若彩色與深度影像的時間戳記完全一致，可用精確同步；若來自不同來源或有微小差異，則需啟用近似同步，否則系統會因等不到配對而完全不動。這是實作中常見的卡關點。
- 最大深度：設定有效範圍上限（30.3 節），超過的點捨棄。
- 記憶體管理：RTAB-Map 有機制控制地圖規模以維持即時性——當地圖過大時，較舊的部分會被移到「長期記憶」中，只在需要時取回。這讓它能處理大範圍場景。

## 30.5 ORB-SLAM3 實作

第 23 章完整剖析過 ORB-SLAM3 的原理，本節聚焦實作要點。

### 與 RTAB-Map 的定位差異

| 面向 | RTAB-Map | ORB-SLAM3 |
|---|---|---|
| 主要輸出 | 稠密彩色點雲地圖 | 稀疏特徵點地圖 |
| ROS 整合 | 原生支援，開箱即用 | 需社群封裝或自行整合 |
| 適合任務 | 三維重建、導航地圖 | 高精度定位、軌跡評測 |

| 面向 | RTAB-Map | ORB-SLAM3 |
|---|---|---|
| 視覺化 | 點雲模型直觀 | 特徵點與關鍵幀 |
| 本書章節 | 本章 | 第 23 章原理、本章實作 |

**表 30-2　RTAB-Map 與 ORB-SLAM3 的定位比較**

簡言之：要「看得見的三維模型」用 RTAB-Map；要「精確的軌跡與定位」用 ORB-SLAM3。兩者不是替代關係，而是不同任務的工具。

### 實作要點

15. 編譯：相依套件與第 22.7 節相同（OpenCV、Eigen3、Pangolin）。注意版本相容性，建議參考社群維護的分支或使用容器（第 28.1 節的建議）。

16. 組態檔：準備 YAML 檔，填入 30.2 節校正得到的內參與畸變係數、影像尺寸、以及 ORB 特徵參數（每幀特徵數、金字塔層數）。

17. 模式選擇：依相機能力選擇單目、雙目、RGB-D，以及是否啟用慣性模式（需相機內建 IMU）。

18. IMU 初始化：若使用慣性模式，啟動後務必做帶加減速與旋轉的運動，否則初始化不會成功（第 23.4 節反覆強調的運動激勵）。

19. 觀察：在 Pangolin 視窗中觀察特徵點、關鍵幀、共視圖與閉環——第 22.6 節的資料結構在此具象化。

20. 軌跡輸出與評測：輸出估測軌跡，以 evo 工具計算 ATE 與 RPE（第 22.7 節）。單目模式記得先做尺度對齊。

### 三種模式的實測比較

同學可在同一場域依序測試三種模式，親身驗證第 20.6 節表 20-3 的理論比較：

- 單目：初始化最困難（需足夠視差，第 21.5 節）、軌跡無絕對尺度、但硬體最簡單。

- RGB-D：初始化最容易（第一幀就有深度）、有絕對尺度、地圖最稠密，但受限於深度相機的有效範圍與強光。
- 雙目慣性：最穩健、精度最佳（第 23.7 節提過的毫米級成績），但需要相機支援與較多的設定。

## 30.6 室內場景測試

本節說明如何設計有教學價值的測試，特別是「刻意尋找失效邊界」的實驗。

### 基本測試流程

21. 環境準備：選擇紋理豐富、光線均勻的空間開始（例如有書架、海報的教室），先建立成功經驗。
22. 緩慢掃描：手持相機或裝在小車上，以緩慢平穩的速度移動，涵蓋整個空間。
23. 閉環測試：繞行一圈回到起點，觀察閉環是否觸發、地圖是否被拉正（第 12、15 章的震撼畫面在此重現）。
24. 品質評估：量測地圖上的關鍵尺寸與實際比較；檢查牆面是否「變厚」（第 17 章案例 17-1 的三維品質指標）。

### 五種典型失效情境

這是本節最有價值的部分——刻意製造失敗，理解系統的邊界。

25. 白牆與弱紋理：對著大面積白牆掃描，觀察特徵點如何歸零、系統如何跟丟。這是第 9、20 章反覆提過的「白牆詛咒」，在此親身驗證。
26. 快速運動：這是視覺 SLAM 最常見的失效原因。移動或轉動太快會中斷成功的點雲匹配序列，導致系統跟丟。同學可逐步提高移動速度，找出系統開始失敗的臨界點。
27. 光線變化：從明亮處走進昏暗處，或正對窗戶造成過曝，觀察追蹤是否中斷。這是第 9.5 節「依賴光線」的具體呈現。

28. 玻璃與鏡面：對著落地窗或鏡子掃描，觀察深度資料出現什麼異常。這與第 8 章光達的玻璃問題是同一類困難。

29. 動態物體：讓人在畫面中走動，觀察是否影響定位。第 24.4 節提過可用物件偵測剔除動態物體的特徵，此處先觀察不處理時的後果。

### 一個實務上的重要提醒

關於資料錄製，有一個值得注意的經驗：相較於錄製資料再離線處理，線上即時建圖有一個優勢——RTAB-Map 在追蹤失敗時，能藉由把相機移回上次成功追蹤的位置來恢復里程計。

這個「人在迴路中」的即時修正能力，是離線處理錄好的資料所沒有的。因此教學上建議：先以線上方式建立成功經驗與手感，理解「相機要怎麼移動系統才不會跟丟」；等到需要做演算法比較時，再錄製資料以確保公平性（第 28.4 節 rosbag 的價值）。

### 掃描的手感

視覺 SLAM 有一個光達 SLAM 沒有的特點：操作者的手法會顯著影響結果。幾個實用建議：

- 轉彎時放慢：直線移動時特徵變化平緩，轉彎時視野快速改變，最容易跟丟。
- 保持特徵在視野中：不要突然把相機轉向空白牆面，而應沿著有紋理的方向移動。
- 避免只旋轉不平移：純旋轉無法三角測量（第 21.5 節、案例 21-1），單目模式尤其致命。
- 回頭確認：繞行時偶爾回頭看一下走過的區域，有助於觸發閉環並鞏固地圖。

## 30.7 視覺 SLAM 與光達 SLAM 比較

這是本書醞釀已久的一節：第 8、9 章比較過感測器特性，第 29 章實作了光達 SLAM，本章實作了視覺 SLAM——現在讓兩者在同一場域正面對決。

### 實測比較的設計

公平比較需要控制變因：

- 同一場域、同一條路線：最好用同一台機器人同時搭載光達與相機，一次移動同時錄下兩種資料。
- 相同的評估指標：軌跡誤差（ATE）、地圖品質（第 13.7 節的 2D 指標與第 17 章的牆厚度）、以及系統穩健度（跟丟次數）。
- 多種環境：這是關鍵——單一環境的比較會誤導，必須涵蓋各自的優勢與劣勢場景。

### 四種環境的預期表現

| 環境 | 光達 SLAM | 視覺 SLAM | 原因 |
|---|---|---|---|
| 明亮有紋理的教室 | 良好 | 良好 | 兩者都在舒適區 |
| 昏暗的機房 | 良好 | 失效 | 光達自帶光源、相機需環境光 |
| 大面積白牆走廊 | 尚可 | 失效 | 光達看幾何、相機找不到特徵 |
| 玻璃帷幕大廳 | 失效 | 尚可 | 雷射穿透玻璃、相機看得到反光與框 |

**表 30-3　兩種 SLAM 在四種環境的預期表現**

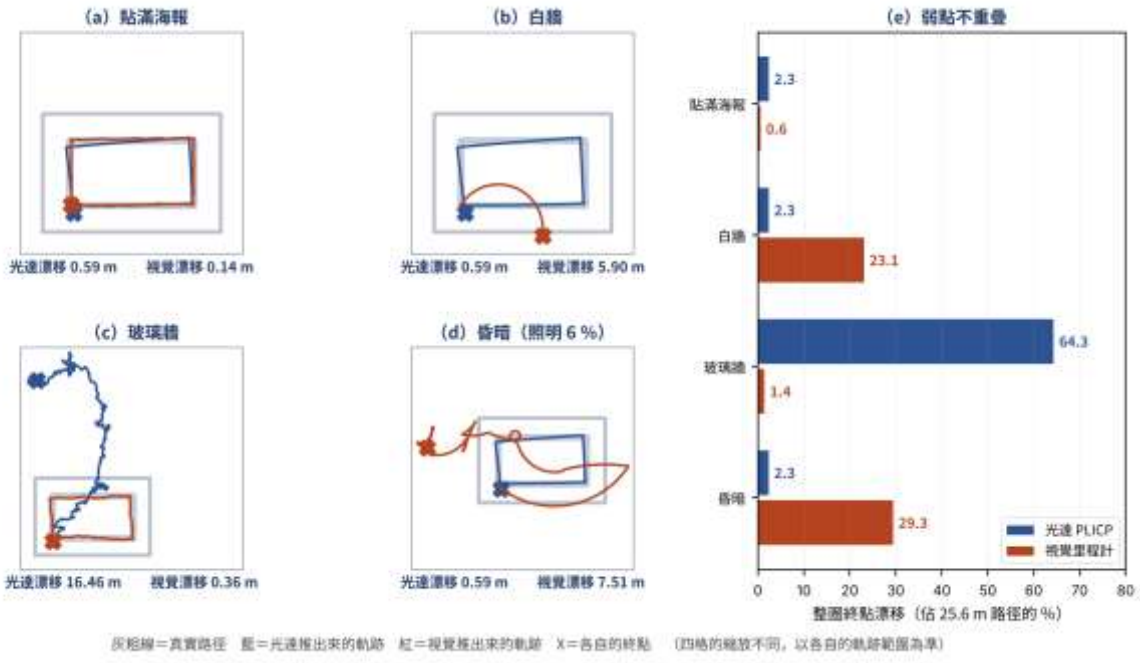
請特別注意最後一列——這是唯一視覺勝過光達的情境，也最能說明「互補」的真義：兩者的弱點不重疊，因此組合起來能涵蓋更多場景。

上面那張表是「預期表現」。本節把它真的跑一次。

實驗設計刻意做得很乾淨：四間一模一樣的 12 × 8 公尺房間，機器人沿同一條貼牆的矩形路徑走一圈（全長 25.6 公尺），每一步同時取得光達掃描與相機觀測。四間房的差別只有

牆面性質：貼滿海報（紋理 1.00、照明 1.00）、白牆（紋理 0.04）、玻璃（85 % 的雷射光束直接穿透，未穿透者有一半發生鏡射、回報錯誤距離）、以及昏暗（紋理 1.00，但照明只有 6 %）。光達走點對線 ICP 做掃描匹配，視覺則由兩幀共同看到的地標方位角與距離解相對位姿。

**圖 30-4　同一場域的對決：四種環境、兩套感測器、同一條路徑**




先看光達。四間房裡有三間的終點漂移完全相同，都是 0.59 公尺。這不是三次獨立實驗恰好吻合——而是因為光達的模擬根本沒有讀取紋理與照明這兩個參數，三次執行在位元層級是同一回事。把這件事講白了其實正是重點：所謂「光達在昏暗處表現良好」，精確的說法不是它「克服」了黑暗，而是黑暗對它根本不存在。雷射自己帶光源，關燈與否和它完全無關。

光達唯一垮掉的是玻璃房：漂移 16.46 公尺，佔路徑全長的 64.3 %，終點已經飛到房間外面去了。圖（c）裡那條藍線在半途整個斷開往上衝，就是掃描匹配失去參考之後的典型樣子。每幀可用的光達點從 306 個掉到 38 個，少了八成七——剩下的那些還混著鏡射回來的假距離。這一格非常值得在課堂上停下來看：它說明光達的失效不是「精度下降」，而是「突然什麼都不知道」。

再看視覺。白牆房漂移 5.90 公尺、昏暗房 7.51 公尺，而且 342 步裡分別有 340 步與 316 步判定跟丟——也就是幾乎整趟都在用等速模型硬推。兩間房的平均可用觀測分別只有 1.3 個與 2.6 個地標，連解一個三自由度位姿的最低需求都達不到。注意白牆與昏暗這兩種情況在相機眼中其實是同一件事：對比度等於紋理乘上照明，紋理 0.04 配滿照明，和紋理 1.00 配 6 % 照明，算出來的對比度分別是 0.04 與 0.06，難怪結果如此接近。這個觀察在實務上很有用：白牆上貼海報和把燈打亮，是同一種補救措施的兩種形式。

玻璃房則反過來，視覺只漂移 0.36 公尺，比光達好了 45 倍。原因在模擬裡也寫得很明白：玻璃對雷射是透明的，對相機卻不是——窗框、汙漬、反光、貼紙都是可以追蹤的特徵。這一格是整本書裡唯一一次視覺大幅勝過光達，也是「互補」這兩個字最具體的證據。

**每步最準的那一個，整圈不一定最準**

貼滿海報那一間房有個反直覺的結果值得留意：光達的每步相對位姿 RMSE 是 0.42 公分，視覺是 1.86 公分，光達準了 4.4 倍；但整圈下來光達漂移 0.59 公尺，視覺只有 0.14 公尺，反而是視覺贏。

原因是兩者的誤差性質不同。掃描匹配在每一步都往同一個方向偏一點點（因為相鄰兩幀的光束打在不同的表面點上），這種系統性偏差會一路累積；視覺的誤差比較大但比較隨機，走一圈之後正負相抵掉不少。

這件事對專題評分很重要：用「每幀誤差」來評斷一套里程計是不夠的，一定要跑閉環、看回到起點差多少。但閉環落差也有它的盲區——里程計的尺度偏差會讓整張地圖等比例放大或縮小，而迴路照樣完美閉合，所以閉環落差必須與捲尺量到的尺寸誤差一起看（第 31.8 節有這件事的實測）。第 31 章的評分標準就是照這兩項互補的原則訂的。

## 其他維度的比較

| 比較維度 | 光達 SLAM | 視覺 SLAM |
|---|---|---|
| 成本 | 較高 | 較低 |
| 地圖資訊 | 精確幾何，無語意 | 含顏色與語意線索 |
| 運算需求 | 2D 較低、3D 較高 | 較高（影像處理） |
| 深度精度 | 公分級且穩定 | 隨距離平方衰減 |
| 對操作手法 | 較不敏感 | 敏感（30.6 節的手感） |
| 後續應用 | 導航、避障 | 語意理解、AR、三維重建 |

**表 30-4　兩種 SLAM 的其他維度比較**

### 融合：最務實的答案

比較的結論不是「哪個比較好」，而是「如何組合」。30.4 節提過 RTAB-Map 支援同時使用視覺與雷射資料，這正是融合的實作途徑。

- 光達提供精確幾何與穩定的里程計，在昏暗與弱紋理處撐住系統。
- 相機提供顏色、語意與閉環辨識能力，在玻璃環境與需要理解場景時發揮作用。
- IMU（第 7、23 章）在兩者都短暫失效時提供支撐。

這正是本書從第二篇就反覆強調的核心信念：沒有全能的感測器，只有恰當的組合。走過二十九章，同學應該能親手驗證這句話——而不只是接受它。

不過這場對決還沒打完——剛才那場比賽的規則，其實暗中偏袒了其中一方。下面兩節先把規則改公平，再談這個領域接下來要往哪裡走。

### 把禮物收回去：如果特徵會配錯呢

上面那場對決對視覺其實是有優待的——程式裡用地標編號直接配對，等於假設特徵匹配永遠不會出錯。真實的 ORB 描述子匹配在重複紋理、視角變化大或動態物體出現時，一成到兩成的錯配是家常便飯。把這份禮物收回去，結果會變成下面這樣。

**表 30-5　把錯配率加進去之後，視覺里程計的整圈漂移（公尺）**

| 環境 | 錯配 0 % | 錯配 10 % | 錯配 20 % | 錯配 35 % | 同一間房的光達 |
|---|---|---|---|---|---|
| 貼滿海報 | 0.14 | 0.52 | 1.22 | 2.50 | 0.59 |
| 白牆 | 5.90 | 5.90 | 5.90 | 5.90 | 0.59 |
| 玻璃牆 | 0.36 | 1.10 | 3.10 | 5.50 | 16.46 |
| 昏暗 | 7.51 | 7.51 | 7.98 | 17.32 | 0.59 |

第一列是最該注意的一列。在零錯配的理想世界裡，貼滿海報的房間是視覺大勝（0.14 對 0.59）；但只要加進 10 % 的錯配，視覺就掉到 0.52 公尺，和光達的 0.59 公尺幾乎打平；錯配到 20 % 就反過來輸了。換句話說，前面那句「有紋理的環境視覺比較準」成立的前提是特徵匹配要夠乾淨，而這正是 ORB-SLAM 花那麼多力氣在 RANSAC、詞袋驗證與共視圖一致性檢查上的理由（第 22 章）。

第二、四列幾乎不隨錯配率改變，原因也很實際：白牆與昏暗的房間裡平均只有一兩個可用地標，連「配錯」的機會都沒有——系統早就跟丟了。錯配率只有在特徵夠多的時候才是個問題；特徵不夠的時候，問題在更前面。

第三列則說明了一個令人安心的事實：即使錯配高達 20 %，玻璃房裡的視覺（3.10 公尺）仍然遠勝光達（16.46 公尺）。互補性不是建立在視覺很完美的假設上，而是建立在兩者的失效原因毫不相干這件事上。

## 30.8　2024–2026：視覺 SLAM 正在被重寫

本章教的 RTAB-Map 與 ORB-SLAM3 是目前產線與課堂上最可靠的選擇，這一點近期內不會改變。但同一時間，視覺 SLAM 的研究主線已經搬到了兩個新的地方，而且兩者都快到了會影響選型的程度。這一節把幾個具體的數字放在一起，讓同學知道自己正站在什麼位置。

## 一、地圖的長相變了：從點雲到 3D 高斯

傳統的稠密視覺 SLAM 把地圖存成點雲或體素。2024 年起，3D Gaussian Splatting 開始取代它們：地圖變成數百萬顆帶顏色與不透明度的橢球，可以用光柵化即時算繪出照片級的畫面。

SplaTAM 是第一個把這件事做進 SLAM 迴圈的代表作。在 Replica 資料集上，它的相機軌跡誤差（ATE RMSE）平均 0.36 公分，勝過 Point-SLAM 的 0.52 公分、ESLAM 的 0.63 公分與 NICE-SLAM 的 1.06 公分，算繪速度在 876×584 解析度下可達每秒 400 幀。但在真實感測器拍攝的 TUM-RGBD 上，它的 5.48 公分卻輸給 ORB-SLAM2 的 1.98 公分——論文自己也指出，在這個基準上以特徵為基礎的方法依然勝過稠密方法。這個對比很能說明本章選擇教 ORB-SLAM3 的理由：論文榜上的第一名，換個資料集就不是第一名了。

速度方面的進展更快。2025 年底發表的 Gaussian-plus-SDF SLAM 在 Azure Kinect 的真實序列上跑到每秒 150 幀以上，而同期的高斯式 SLAM 多半還在每秒 20 幀以下；它用的高斯數量少了約一半，最佳化迭代次數少了約 75 %。

## 二、不校正也能跑：視覺基礎模型進入 SLAM

本章 30.2 節花了不少篇幅談相機校正，因為內參錯了後面全錯。2025 年 CVPR 的 MASt3R-SLAM 直接挑戰了這個前提：它用一個預訓練的三維重建基礎模型取代傳統的特徵匹配與三角化，在「不知道相機內參」的情況下也能做稠密 SLAM。

數字是這樣的：在 RTX 4090 上約每秒 15 幀。已知內參時，TUM-RGBD 的 ATE RMSE 平均 0.030 公尺、7-Scenes 0.047 公尺，都略勝 DROID-SLAM 的 0.038 與 0.049；完全不知道內參時分別退到 0.060 與 0.066 公尺，仍與許多需要校正的方法相當。代價也很清楚：換到 RTX 3080 Ti 就掉到每秒 5 到 7 幀，顯示記憶體要 9 到 11 GB，而且 EuRoC 這種快速運動的序列上（0.041）仍輸給 DROID-SLAM（0.022）。

同一條路線上的另一半是單目深度估測。Depth Anything V2 與 2025 年的 Video-Depth-

Anything 讓一般的 RGB 相機也能輸出具備時間一致性的深度。以後者的公制版本為例，大型模型在 NYUv2 上的 δ1 達 0.908、KITTI 上 0.910；最小的模型只有 2,840 萬個參數，在 A100 上半精度推論延遲 7.5 毫秒。不過要留意它的即時串流模式仍有明顯代價：ScanNet 上的 δ1 從離線的 0.926 掉到 0.836。

**這對同學的專題意味著什麼**

不要急著把 RTAB-Map 換掉。上面這些方法共同的特徵是：論文裡的數字很漂亮，但都需要一張高階顯示卡，而且多半還沒有成熟的 ROS 2 封裝與長期維護承諾。

可以現在就用上的是單目深度估測：它能當作深度相機失效時的備援（例如白牆或超出有效範圍的區域），也能讓沒有深度相機的隊伍先把流程跑起來。

要寫進專題報告的是比較：用本章的方法在自己的場域量一次 ATE，再把它和論文上的數字並排，然後解釋為什麼不一樣。這件事的學習價值遠高於把最新的模型跑起來。

## 三、硬體這一端也在變

30.1 節的選型建議建立在一個假設上：RealSense 便宜、ROS 2 驅動成熟、社群資源豐富。這個假設在 2025 年 7 月有了變化——RealSense 自 Intel 獨立成為新公司，取得 5,000 萬美元投資（來自 Intel Capital 與聯發科創新基金），Intel 保留少數股權與一席董事。

對教學現場而言，這件事短期是好消息：新公司有 130 名員工、三千多家客戶，並且明確把自主移動機器人的視覺 SLAM 列為投資方向，新推出的 D555 支援乙太網路供電，並內建一顆 5 TOPS 的自研 AI 晶片，走的正是第 25 章邊緣運算那條路。但長期的維護承諾與供貨穩定度還需要時間驗證，因此在規劃多年期的課程或專題設備時，建議不要把整個流程綁死在單一廠牌的 SDK 上——盡量讓節點只依賴標準的 ROS 2 影像與 CameraInfo 介面，這樣換相機時改的是驅動，不是整套系統。

# 公式或流程

## 一、由深度影像反投影為三維點

這是第 9.1 節針孔投影模型的逆運算。設某像素座標為 ( u, v ) 、該像素的深度值為 d、相機內參為 fx、fy、cx、cy，則對應的三維點座標為：

$$Z = d \,/\, scale$$

$$X = (u - cx) \times Z \,/\, fx$$

$$Y = (v - cy) \times Z \,/\, fy$$

其中 scale 是深度縮放因子——深度影像常以毫米為單位的整數儲存，而點雲以公尺為單位，因此需要除以 1000。這個縮放因子填錯是初學者常見的錯誤，症狀是「點雲的尺度完全不對」( 例如整個房間變成一公釐大 ) 。

把這個運算套用到深度影像的每個像素，再賦予對應的 RGB 顏色，就得到彩色點雲 ( 30.3 節 ) 。

## 二、深度品質與有效範圍

第 9.6 節推導過雙目深度誤差隨距離平方成長：

$$\Delta Z \approx (Z^2 \,/\, (f \cdot b)) \times \Delta d$$

這解釋了 30.1 節提到的實務做法——雖然相機標稱十公尺量測範圍，但常把有效範圍設為五公尺。以此式估算：距離加倍，誤差變四倍。若在 2 公尺處誤差 1 公分，到 10 公尺處誤差就可能達 25 公分，已不足以支撐精確建圖。

實務建議：依你的精度需求，用這條式子反推可用的最大距離，並在系統中設定為深度裁切門檻。

## 三、視覺 SLAM 實作的完整流程

**圖 30-5　RGB-D 視覺 SLAM 的完整實作流程**

【階段一：相機準備】

① 安裝相機驅動 → ② 確認內參（出廠或自行校正，30.2 節）

③ 開啟深度對齊 → ④ 驗證彩色與深度影像的像素對應正確

↓

【階段二：資料驗證】

⑤ 檢查三個必要話題：彩色影像、對齊深度、相機資訊

⑥ 在 RViz 中顯示點雲，確認幾何與顏色都正確

↓

【階段三：SLAM 執行】

⑦ 啟動 RTAB-Map 或 ORB-SLAM3，注意同步方式設定

⑧ 緩慢掃描，觀察軌跡、地圖與閉環

↓

【階段四：評估與比較】

⑨ 儲存地圖 → ⑩ 量測誤差（尺寸比對、牆厚度、ATE）

⑪ 與光達 SLAM 在同場域比較（30.7 節）

## 四、視覺 SLAM 的故障排除順序

與第 29.7 節的導航除錯相同，視覺 SLAM 也應依系統性順序排查：

**圖 30-6　視覺 SLAM 的除錯檢查表**

① 相機驅動：三個必要話題都有資料嗎？頻率正常嗎？

↓

② 內參：CameraInfo 的數值合理嗎？（fx、fy 應相近；cx、cy 約為影像寬高的一半）

↓

③ 深度對齊：彩色與深度是否對齊？點雲的顏色有沒有錯位？

↓

④ 深度縮放：點雲的尺度正確嗎？量一個已知物體驗證（公式一的 scale）

↓

⑤ 同步：若系統完全不動，檢查是否需要啟用近似同步

↓

⑥ 特徵：當前畫面偵測到多少特徵點？太少表示環境紋理不足

↓

⑦ 運動：移動是否過快？是否只旋轉未平移？（30.6 節的手感）

↓

⑧ 參數：確認上游都健康後，才開始調 SLAM 參數

## 案例

### 案例 30-1　動得快一點就跟丟：視覺 SLAM 最常見的挫折

這是同學實作視覺 SLAM 時幾乎必定遇到的狀況，值得完整說明。

典型情境是這樣的：同學拿著深度相機在實驗室慢慢移動，系統運作良好，點雲逐步建

立。但只要稍微加快移動或轉動速度，畫面上的座標軸就開始亂飄，點雲停止記錄，終端顯示追蹤品質為零。

這個現象在社群中被反覆討論。有使用者描述：若非常緩慢地移動或旋轉相機，地圖能建立但尺寸不對；若稍微加快一點，相機的 TF 就開始隨機亂飄，點雲停止記錄。官方文件也明確警告：移動或轉動太快會中斷成功的點雲匹配序列，導致系統跟丟。

原因是第 26.4 節討論過的三重打擊在此重現：快速運動造成動態模糊使特徵提取失敗、特徵軌跡縮短使匹配困難、而深度相機的幀率限制也使前後幀差異過大，超出配準的收斂範圍（第 17.3 節 ICP 需要好初值）。

對策有三個層次：

- 操作層面：放慢速度，尤其轉彎時（30.6 節的手感建議）。這是最直接有效的一招。
- 設定層面：社群提出的一個解法是分別使用相機的兩個紅外線影像，以紅外線組態而非深度組態執行 SLAM——這相當於改用雙目模式，能提供更穩定的特徵追蹤。
- 系統層面：加入 IMU 融合（第 23 章）。IMU 能在視覺短暫失效時提供運動預測，大幅提升快速運動下的穩健度——這正是第 7.6 節「IMU 填補視覺失效空隙」的價值。

這個案例的教學價值在於：它讓同學親身體會「論文上的精度數字，是在特定條件下取得的」。同一套系統，慢速掃描能達公分級精度，快速移動則直接失效——理解這個落差，是負責任地使用技術的前提。

## 案例 30-2　三公尺長的房間變成三公釐：深度縮放的陷阱

這是一個看似荒謬、卻極為常見的錯誤，幾乎每個第一次自己寫點雲產生程式的同學都會遇到。

症狀是：點雲產生出來了，形狀看起來也對，但整個場景小得離譜——在 RViz 中要把

視角拉到極近才看得見，而量測的尺寸完全不合理。

原因就是公式一中的深度縮放因子。深度影像為了節省空間，通常以 16 位元整數儲存，單位是毫米；而 ROS 的點雲以公尺為單位。若程式中忘了除以 1000，整個場景就會縮小一千倍——三公尺長的房間變成三公釐。

這個錯誤的診斷方法極為簡單：量測點雲中一個已知尺寸的物體（例如一張 A4 紙或桌子的寬度），若數值差了三個數量級，就是縮放因子的問題。

更廣泛的啟示是：單位錯誤是工程中最常見也最容易避免的錯誤之一。第 10.3 節提過弧度與角度的混用、本章的毫米與公尺、第 5 章的經緯度與公尺——養成「每個數值都標註單位」的習慣，能省下無數除錯時間。這個習慣在寫程式、寫報告、看規格書時都同樣受用。

## 案例 30-3　同一場域的對決：當光達與相機各自失效

第 30.7 節提出的對照實驗，在大學課堂上有極高的教學價值。這裡描述一次典型的實測結果。

實驗設計：同一台小車同時搭載 2D 光達與 RGB-D 相機，沿相同路線走過四個區域——明亮的教室、昏暗的機房、貼滿白色隔音板的走廊、以及玻璃帷幕的中庭。

結果相當戲劇性：

- 教室：兩者都表現良好，地圖清晰、軌跡準確。同學此時可能會覺得「兩者差不多」。
- 昏暗機房：視覺 SLAM 在進入後數秒內跟丟，特徵數歸零；光達則完全不受影響，因為它自帶雷射光源（第 8 章）。
- 白牆走廊：視覺 SLAM 再次失效（白牆詛咒）；光達雖然也面臨走廊的結構退化問題（第 13.6 節），但至少能持續運作。

- 玻璃中庭：局勢逆轉——光達的雷射穿透玻璃，在地圖上畫出根本不存在的空間；相機反而能看到玻璃的反光、窗框與框外的景物，建出較合理的地圖。

這個實驗把第 8、9 章的理論從「表格上的比較」變成「親眼看到的失效」。同學普遍反映，做完這個實驗後，對感測器選型的理解遠比讀十遍表格深刻。

更重要的是後續討論：既然兩者的弱點不重疊，那麼組合起來會如何？這自然引導到 RTAB-Map 同時使用視覺與雷射的融合實作，也呼應第 27 章多感測器與多機器人協作的思想。建議教師務必安排這個實驗，它是第八篇最有說服力的一堂課。

## 實作活動

### 實作 30-1　視覺 SLAM 實作與跨感測器比較

目的：完成 RGB-D 視覺 SLAM 的完整實作，並在同一場域與第 29 章的光達 SLAM 進行對照比較。這是第八篇的高潮實驗，也是第 31 章專題的重要基礎。

**第一部分：相機設定與點雲產生（全體）**

151. 安裝深度相機驅動，確認彩色影像、深度影像與相機資訊三個話題都有資料，並記錄各自的發布頻率。

152. 檢視 CameraInfo 中的內參數值，依圖 30-3 的第二步驗證合理性（fx 與 fy 是否相近？cx、cy 是否約為影像寬高的一半？）。

153. 開啟深度對齊，在 RViz 中顯示彩色點雲，檢查顏色與幾何是否正確對應。刻意關閉對齊再看一次，觀察錯位現象。

154. 尺度驗證：在點雲中量測一個已知尺寸的物體（A4 紙、桌面寬度），確認數值正確——親身避開案例 30-2 的陷阱。

155. （進階）自己寫程式：依公式一，從深度影像與內參產生點雲，不使用現成節點。故意把 scale 設錯，重現案例 30-2 的現象。

### 第二部分：RTAB-Map 建圖（全體）

30. 啟動 RTAB-Map，指定正確的話題名稱。首次執行加上刪除舊資料庫的參數。

31. 在教室中緩慢移動相機掃描，觀察即時建立的彩色點雲與相機軌跡。

32. 閉環測試：繞行一圈回到起點，觀察閉環偵測是否觸發、地圖是否被拉正。

33. 速度實驗：逐步加快移動速度，找出系統開始跟丟的臨界速度，重現案例 30-1。記錄這個速度值。

34. 儲存地圖，並切換到定位模式重新啟動，驗證系統能在既有地圖中定位（對照第 29.5 節的 AMCL）。

### 第三部分：失效情境探索（全體，本章特色）

依 30.6 節的五種情境，逐一測試並記錄系統行為：

35. 白牆測試：把相機對準大面積白牆，記錄特徵點數量的變化，以及系統跟丟前能撐多久。

36. 光線測試：從明亮處走入昏暗處，記錄追蹤中斷的時機；再對著窗戶造成過曝，觀察結果。

37. 玻璃測試：掃描玻璃窗或鏡子，觀察深度資料出現什麼異常（缺失？錯誤的深度值？）。

38. 動態物體測試：讓一位同學在畫面中走動，觀察是否影響定位精度與地圖品質。

39. 純旋轉測試：原地旋轉相機（不平移），觀察系統的反應，驗證第 21.5 節的三角測量限制。

### 第四部分：跨感測器比較（核心實驗，大學部）

40. 場域選擇：選出四個特性不同的區域——明亮有紋理處、昏暗處、大面積白牆處、玻璃處（參考表 30-3）。

41. 資料採集：以同一台小車同時搭載光達與相機，沿相同路線移動，同時錄製兩種資料（第 28.4 節的 rosbag）。

42. 分別執行：用同一份資料，分別執行第 29 章的光達 SLAM 與本章的視覺 SLAM。

43. 量化比較：記錄四個區域中各自的跟丟次數、地圖品質、以及（若有真值）軌跡誤差，填入你自己的比較表。

44. 融合嘗試：設定 RTAB-Map 同時使用視覺與雷射資料（30.4 節），觀察是否能涵蓋兩者各自失效的區域。

**討論問題**

70. 關閉深度對齊後，點雲的顏色如何錯位？錯位的方向與相機的物理結構有什麼關係？

71. 你的系統在多快的移動速度下開始跟丟？這個速度對「裝在移動機器人上」意味著什麼？

72. 五種失效情境中，哪一種最容易發生？哪一種後果最嚴重？為什麼？

73. （核心問題）在四個區域的比較中，光達與視覺各在哪些區域勝出？這是否符合表 30-3 的預期？有沒有出乎意料的結果？

74. 融合視覺與雷射後，系統的表現如何？它是否真的涵蓋了兩者各自的弱點？融合帶來什麼新的代價（運算、設定複雜度）？

**延伸挑戰（大學部）**

其一：大學場域三維數位分身——用 RTAB-Map 掃描一間實驗室或一段走廊，產生完整的彩色三維模型，並依第 17.5 節的方法轉為八叉樹或網格格式輸出。這份成果可用於系所展示、空間管理，也是第 31 章專題二的直接產物。其二：視覺與光達融合的量化研究——設計一套嚴謹的實驗，在多種環境中比較「純光達」「純視覺」「兩者融合」三種配置的表現，以 ATE 與跟丟次數量化，並撰寫成一份完整的研究報告。這個題目層次完整、成果具體，是極佳的專題主軸，也與第八篇強調的「用資料說話」相呼應。

## 延伸閱讀

99. M. Labbé and F. Michaud, "RTAB-Map as an Open-Source Lidar and Visual SLAM Library for Large-Scale and Long-Term Online Operation," Journal of Field Robotics, 2019.（RTAB-Map 的權威論文，含視覺與雷射融合的說明）

100. RTAB-Map 官方網站與 ROS 2 套件文件（introlab.github.io/rtabmap）.（參數說明、教學與各種相機的設定範例）

101. C. Campos et al., "ORB-SLAM3," IEEE T-RO, 2021.（第 23 章的系統，本章的實作對象之一）

102. Intel RealSense ROS 2 套件文件與 SLAM 教學 Wiki(GitHub: IntelRealSense/realsense-ros).（相機設定、深度對齊與 IMU 融合的官方指引）

103. ROS 2 camera_calibration 套件文件.（相機校正的互動式工具，30.2 節的實作基礎）

104. L. Keselman et al., "Intel RealSense Stereoscopic Depth Cameras," CVPR Workshops, 2017.（深度相機的原理與規格，第 9、30 章共同參考）

105. evo 軌跡評測工具（GitHub: MichaelGrupp/evo）.（ATE 與 RPE 計算，跨感測器比較的量化工具）

## 小結

99. 三類深度相機各有定位：RealSense 採主動紅外雙目、室內表現佳且部分型號內建 IMU;ZED 採被動雙目、戶外可用且精度較佳但需 GPU;OAK-D 內建 AI 晶片可在相機端推論。選型應考量環境、運算資源、是否需 IMU 與有效範圍。

100. 現代深度相機出廠已校正並透過 CameraInfo 提供參數，但在碰撞後、使用自組系統或高精度需求時應自行校正；校正的常見陷阱包含棋盤格不平整、影像涵蓋不足、解析度不一致與忘記更新參數。

101. 由深度影像反投影產生點雲是針孔模型的逆運算，關鍵前提是深度對齊——彩色與深度感測器物理位置不同，未對齊會造成顏色與幾何錯位；深度縮放因子填錯會使場景尺度差三個數量級。

102. RTAB-Map 是基於增量式外觀閉環偵測的 RGB-D 圖優化 SLAM，產生稠密彩色點雲與 RGB-D 里程計；它能使用外部里程計、融合 2D 或 3D 雷射掃描、並支援八叉樹與體素等結構，是視覺與光達融合的實作途徑。

103. RTAB-Map 與 ORB-SLAM3 定位不同：前者輸出稠密彩色地圖適合三維重建與導航，後者輸出稀疏特徵地圖適合高精度定位與軌跡評測；三種模式（單目、RGB-D、雙目慣性）的難易與精度差異可實測驗證。

104. 視覺 SLAM 的五種典型失效為白牆弱紋理、快速運動、光線變化、玻璃鏡面與動態物體；移動或轉動太快會中斷點雲匹配序列導致跟丟，而線上建圖相較離線處理的優勢是能靠人為把相機移回上次成功位置來恢復追蹤。

105. 光達與視覺的弱點不重疊：昏暗與白牆處光達勝出、玻璃處視覺勝出，這正是互補的真義；融合（如 RTAB-Map 同時使用視覺與雷射）是最務實的答案，印證了本書「沒有全能的感測器，只有恰當的組合」的核心信念。

106. 評估一套里程計或 SLAM，單一指標不夠：每幀誤差看不出累積漂移，而閉環落差看不出里程計的尺度偏差——整張地圖等比例放大或縮小時，迴路照樣完美閉合。閉環落差要與捲尺量到的尺寸誤差一起看，兩者的盲區才不會重疊（第 31.8 節有這件事的實測數據）。

第八篇的技術實作到這裡告一段落。同學已經親手建立了兩套完整的 SLAM 系統，並且在同一個場域裡量出了它們各自在哪裡失效、失效時是什麼樣子、以及為什麼那些弱點不會同時出現。這比讀任何一張比較表格都更深刻——因為當你親眼看過光達在玻璃房裡把軌跡甩出房間外、也看過相機在白牆前整趟都在跟丟，「沒有全能的感測器，只有恰當的組合」這句話就不再是一句口號。

下一章，我們把所有能力整合成完整的專題：從高中的教室 2D 建圖、大學部的三維空間掃描、掃地機器人路徑分析、機器狗校園巡檢、視覺導航無人機，到多感測器融合定位，並提出涵蓋地圖品質、定位誤差、系統穩定性、報告表達與程式碼品質的完整評分標準。第 31 章將是本書實作的總驗收。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

## 第 31 章

## SLAM 專題與評量

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程　/　高中 AI 機器人社團課程

## 本章在全書的位置

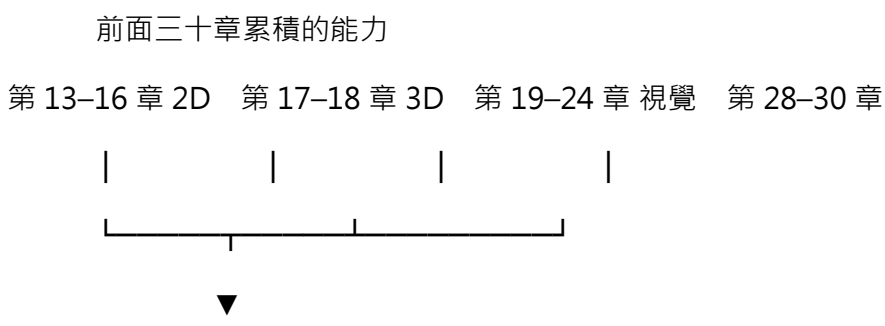
這是第八篇的最後一章，也是全書實作部分的總驗收。

前面三十章，我們一路從「SLAM 是什麼」走到「我親手把它跑起來了」：第一篇到第三篇建立觀念、感測器與數學基礎，第四篇到第七篇依序處理 2D 建圖、3D 光達、視覺 SLAM 與微型平台，第 28 章備妥開發環境，第 29 章讓光達小車真的動起來，第 30 章把相機與光達放在同一個場域正面比了一次。

本章要處理的是一個不一樣的問題：這些能力要怎麼組成一件「作品」，而這件作品要怎麼被評斷好壞。這兩件事看似是課程行政，其實是工程訓練裡最難教、也最容易被跳過的部分——多數同學的專題不是敗在技術，而是敗在題目太大、時程沒有緩衝、以及最後交出一份沒有數據的報告。

所以本章有三張用實驗算出來的圖，分別回答三個常被含糊帶過的問題：圖 31-1 回答「不同策略的效率差多少」，用的是專題三的覆蓋規劃；圖 31-2 回答「時程該留多少緩衝」，用的是四個里程碑的蒙地卡羅模擬；圖 31-3 回答「你用的評估方法本身可不可靠」——這一張的結論可能會讓不少同學重新檢視自己的評估方式。

**圖 31-0　第 31 章的連結地圖**




## 學習目標

走過三十章的理論與實作，同學已經具備完整的 SLAM 能力。本章是本書實作的總驗收：六個難度遞增的專題，一套五向度的評分標準，以及三組量化實驗，用來說明「怎麼做」與「怎麼被評」背後的理由。它既是課程的結業設計，也是同學規劃自己研究方向的地圖。學完本章之後，同學應能夠達成下列目標：

1. 依自身的能力與資源，選擇合適難度的專題並規劃執行流程。
2. 完成教室 2D 地圖建構專題，並量化評估地圖品質。
3. 完成三維空間掃描專題，產出可展示的數位模型。
4. 設計並執行掃地機器人路徑分析與覆蓋率評估，並以實驗說明不同策略的效率差距。
5. 規劃機器狗校園巡檢與視覺導航無人機的系統架構。
6. 設計多感測器融合定位的實驗與比較方法。
7. 依五向度評分標準自我檢核，並準備專題報告與展示。

8. 判斷一個評估方法是否足以支撐結論，並說明閉環落差、尺寸誤差與 ATE 各自能看出什麼、看不出什麼。

9. 以機率的觀點規劃里程碑與緩衝，並估算專題準時完成的機會。

## 核心概念

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 專題導向學習 | Project-Based Learning | 以完成真實任務為核心的學習方式。 |
| 評分標準 | Rubric | 以明確向度與等級描述評分依據的工具。 |
| 絕對軌跡誤差 | ATE | 估計軌跡與真值對齊後的位置誤差（第 22.7 節）。 |
| 相對位姿誤差 | RPE | 固定間隔內相對運動的誤差，反映局部漂移。 |
| 閉環落差 | Loop Closure Gap | 繞一圈回到原點時，地圖上起終點的距離。 |
| 覆蓋率 | Coverage Rate | 實際走過的面積佔可行區域的比例（第 16 章）。 |
| 重複率 | Redundancy Rate | 實際行走距離與理論最短距離的比值。 |
| 地圖品質指標 | Map Quality Metric | 牆面厚度、尺寸誤差、閉環一致性等評估項目。 |
| 系統穩定性 | Robustness | 長時間運行不跟丟、能從失敗恢復的能力。 |
| 可重現性 | Reproducibility | 他人能依你的紀錄重現相同結果的程度。 |
| 成果證據 | Evidence | 可驗證的產出：地圖檔、軌跡圖、錄影、程式碼。 |
| 技術債 | Technical Debt | 為求快速而累積的、日後需償還的程式品質 |

| | | |
|---|---|---|
| | | 問題。 |
| 專題里程碑 | Milestone | 把長期專題切分成可檢核的階段性目標。 |
| 緩衝 | Schedule Buffer | 為吸收工期不確定性而刻意預留的時間。 |

**表 31-1　本章的核心名詞**

### 專題總覽：如何選擇

本章提供六個專題，難度由淺入深。它們不是彼此獨立的選項，而是一條可以循序漸進的路徑——後面的專題往往以前面的成果為基礎。

| 專題 | 主題 | 難度 | 適合對象 | 主要依據章節 |
|---|---|---|---|---|
| 一 | 教室 2D 地圖建構 | ★ | 高中、大學部入門 | 第 13–15、28 章 |
| 二 | 三維空間掃描 | ★★ | 大學部 | 第 17、30 章 |
| 三 | 掃地機器人路徑分析 | ★★ | 高中進階、大學部 | 第 16 章 |
| 四 | 機器狗校園巡檢 | ★★★★ | 專題生 | 第 18、24、27 章 |
| 五 | 視覺導航無人機 | ★★★★ | 專題生 | 第 23、26 章 |
| 六 | 多感測器融合定位 | ★★★★★ | 專題生、研究生 | 第 5–7、11、29 章 |

**表 31-2　六個專題的難度與適用對象**

### 選擇專題的三個判準

- 資源：有哪些硬體？有多少時間？若只有電腦，專題一與三可完全以模擬完成；專題四、五需要實體平台。
- 能力：對照第 28.7 節的五階段學習路徑，誠實評估自己目前在哪一階。挑戰高一階的專題最能成長，跳兩階則容易挫折。
- 興趣與延續性：專題最好能延續成長期的研究方向。若你對 AI 有興趣，專題四、五

能自然銜接第 32 章的語意 SLAM 與第 34 章的語言導航。

**共通的執行原則**

- 切分里程碑：把專題拆成三到五個可檢核的階段，每階段都有可展示的成果。這能避免「做到最後才發現方向錯了」。
- 留下證據：第 29.7 節強調過，每個階段都應產出可驗證的證據——地圖檔、誤差數據、錄影、程式碼倉庫。這既是除錯的依據，也是報告的素材。
- 先模擬再實體：第 28、29 章反覆建議的路徑。在模擬中把流程與參數調通，能省下大量實體除錯時間。
- 記錄失敗：失敗的嘗試與其原因，往往比成功的結果更有價值。專題報告中若能誠實呈現「我試過什麼、為什麼不行、最後怎麼解決」，遠比只報告成功結果更有說服力。

## 31.1　專題一：教室 2D 地圖建構

這是最基礎也最重要的專題。它涵蓋 SLAM 的完整循環，而且成果直觀可見——適合作為所有同學的第一個專題。

**專題目標**

以 2D 光達建立一間教室（或一段走廊）的佔據格地圖，並量化評估其品質。進階目標是完成建圖後的自主導航。

**三個階段**

- 階段一——模擬建圖：在模擬環境中完成建圖流程，熟悉工具鏈與參數（第 28、29 章）。產出：模擬地圖檔與操作紀錄。
- 階段二——實體建圖：在真實教室以小車建圖。務必先完成里程計校正（第 29.3 節）。產出：地圖檔、建圖過程錄影、品質評估報告。

- 階段三——自主導航：載入地圖，以 AMCL 定位並用 Nav2 導航到指定目標點（第 28.5、29.6 節）。產出：導航錄影、成功率統計。

**必做的量化評估**

- 尺寸誤差：量測地圖上教室的長寬，與捲尺實測比較，計算誤差百分比。目標是小於 3%。
- 閉環一致性：繞行一圈回到起點，量測地圖上起點與終點的落差。
- 目視檢查：依第 13.7 節檢查牆是否筆直、有無重影與斷裂。
- 導航成功率（若做階段三）：設定五個目標點，各嘗試三次，統計成功抵達的比例與平均耗時。

這四項要一起做，不能只挑一項。理由在 31.8 節會以實驗說明：尺寸誤差與閉環落差各自只能看見一種錯誤，只量其中一項，有很大的機會把一張錯了 3% 的地圖判成滿分。

**進階挑戰**

- 演算法比較：用同一份錄製資料（rosbag），分別執行 slam_toolbox、Cartographer 與 Gmapping，比較地圖品質與運算負擔（第 15 章實作 15-1）。
- 困難場域：挑戰長走廊或空曠大廳，重現第 13.6 節的困境，並嘗試提出對策（增加特徵、調整參數、融合 IMU）。
- 參數研究：系統性地改變粒子數、解析度、更新門檻等參數，量化其對品質與效能的影響，繪製成圖表。

**評量重點**

這個專題的評量重點不在「有沒有建出地圖」，而在「能不能說清楚地圖為什麼好或不好」。同學應能指出誤差的來源（里程計？掃描匹配？環境結構？），並提出改善的依據。

> **常見的三個起步錯誤**

一、沒有校正里程計就直接建圖。第 29.3 節量過：輪徑誤差 2% 在繞行一圈後就足以讓地圖產生可見的扭曲，而且這種扭曲看起來像是「演算法不好」，很容易讓同學往錯的方向調參數。

二、推得太快。掃描匹配需要相鄰兩幀有足夠重疊；推車速度超過感測器更新率所能負擔的範圍時，匹配會失敗而產生跳躍。先以慢速走完一次，再逐步加速找出自己系統的上限。

三、只走一次就下結論。同一個場域至少走三次，看結果穩不穩定。三次結果差異很大，代表系統處在邊界上，這件事本身就值得寫進報告。

## 31.2　專題二：智慧機器人教室 3D 點雲掃描

這個專題把維度提升到三維，產出的成果視覺效果極佳，適合作為系所展示與招生素材。

### 專題目標

以 RGB-D 相機或 3D 光達掃描一間實驗室或一段走廊，產出完整的彩色三維點雲模型，並評估其精度。

### 三個階段

- 階段一——設備準備與校正：相機校正、深度對齊驗證、點雲尺度確認（第 29.2、30.3 節）。產出：校正參數檔與驗證紀錄。
- 階段二——掃描建模：以 RTAB-Map 掃描目標空間。注意規劃路線確保覆蓋完整、重疊充足（第 17.5 節）。產出：三維模型檔與掃描過程錄影。
- 階段三——後處理與輸出：降採樣、離群點剔除、轉為八叉樹或網格格式，產出可分享的模型（第 17.2、17.5 節）。

### 必做的量化評估

- 牆面厚度：第 17 章案例 17-1 的核心指標。量測一段牆面點雲在垂直方向的分布厚度，理想值應接近感測器精度（數公分）。
- 尺寸誤差：量測模型中的關鍵尺寸（門寬、房間長度）與實測比較。
- 覆蓋完整性：檢查是否有明顯的空洞，並說明成因（遮蔽？玻璃？掃描路線不足？）。
- 資料規模：記錄原始點數、降採樣後點數、以及各階段的處理時間。

**進階挑戰**

- 多次掃描合併：分次掃描不同區域再合併，體驗第 23 章 Atlas 多階段作業或第 27 章地圖融合的概念。
- 跨感測器比較：同一空間分別以 RGB-D 相機與 3D 光達掃描，比較模型的精度、密度與完整性（第 30.7 節的三維版本）。
- 應用延伸：把模型匯入模擬環境，建立教室的「數位分身」，供後續學生在虛擬環境中測試演算法（第 17.7 節）。這條路也正是第 33 章 Sim-to-Real 的入口——你掃出來的這間教室，之後可以變成訓練場。

**成果的長期價值**

這個專題的產出有實際用途：系所空間管理、無障礙路線規劃、招生展示、以及作為模擬環境的基礎。建議同學把成果整理成可分享的格式並附上說明文件——這比一份只有自己看得懂的檔案有價值得多。

## 31.3　專題三：掃地機器人路徑分析

這個專題不需要昂貴硬體，卻能深入覆蓋路徑規劃的核心議題，是高中與大學部入門的理想選擇。

**專題目標**

分析並改善覆蓋路徑規劃演算法，以覆蓋率與重複率量化評估，並（若可能）與真實掃地機器人的行為比較。

### 三個階段

- 階段一——紙上與模擬分析：依第 16.7 節的紙上活動，再以 Python 實作弓字形覆蓋規劃，計算覆蓋率與重複率。產出：程式碼與量化數據。
- 階段二——策略比較：實作並比較至少三種策略——隨機碰撞、弓字形、螺旋式，在不同房間形狀下比較效率。產出：比較圖表與分析。
- 階段三——真實觀察（選用）：觀察並記錄真實掃地機器人的路線與 App 地圖，與你的模擬結果比較，推測它使用的策略。產出：觀察紀錄與對照分析。

### 一、這個專題該做出什麼樣的圖：一次示範

以下是本書為這個專題實際跑出來的結果，可以作為同學階段二產出的樣板——注意它長什麼樣子：有明確的實驗設定、有對照組、有兩個互補的指標、而且結論可以用一句話講完。

實驗設定：8 公尺 × 6 公尺的房間，切成 0.25 公尺的格子；障礙以 3×3 格的家具方塊隨機擺放，佔地比例分別取 0%、6%、12%、20%；每種設定跑 20 次隨機配置。三種策略：隨機碰撞（直行，撞到才隨機轉向，另加每步 5% 的自發轉向以模擬打滑）、弓字形（逐列上下掃掠，遇障礙以最短路徑繞到下一個未掃格）、螺旋式（由外圈往內收）。橫軸一律換算成「已行走距離 ÷ 理論最短覆蓋距離」，這樣三種策略才能公平比較。

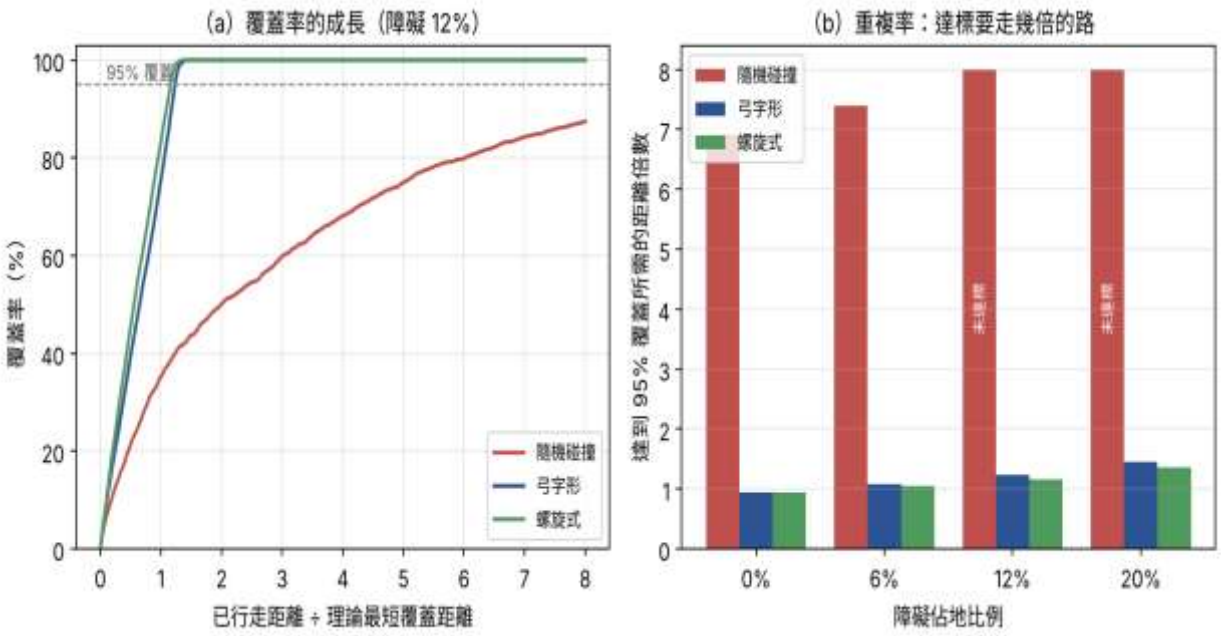



**圖 31-1　三種覆蓋策略的效率差距**

（a）是障礙 12% 時覆蓋率隨行走距離的成長曲線；（b）是四種障礙密度下，達到 95% 覆蓋所需的距離倍數。隨機碰撞在 12% 與 20% 的設定中，走到 8 倍距離仍未達標，故標為「未達標」。

## 二、三個可以直接寫進報告的結論

1. 系統性策略的效率是隨機策略的五倍以上。走完一倍的理論最短距離時，螺旋式已覆蓋 83.2%、弓字形 75.6%，隨機碰撞只有 35.3%。要達到 95% 覆蓋，弓字形在無障礙房間只需 0.95 倍距離，隨機碰撞需要 6.9 倍——而在障礙 12% 以上的房間，隨機碰撞走到 8 倍都到不了。

2. 隨機策略的真正問題不是慢，是尾巴收不起來。看（a）的紅線：它在前段其實爬得不算太差，但越接近 100% 越平——最後那幾格永遠靠運氣。這正是早期掃地機器人「掃很久卻總有一塊沒掃到」的數學原因，也是後來各家廠商全部改用建圖加規劃的理由。

3. 障礙會讓系統性策略變貴，但不會讓它失效。弓字形達標距離從 0.95 倍（無障礙）升到 1.45 倍（障礙 20%），螺旋式從 0.95 升到 1.36——多出來的部分就是繞路成本。換句話說，障礙讓重複率上升約五成，但覆蓋率仍然保證達標；隨機策略則是直

接失去保證。

第三點值得多說一句，因為它牽涉到第 16 章的理論：弓字形與螺旋式之所以「保證」達標，是因為它們走的是一條事先算好、涵蓋所有可行格的路徑；繞路只增加成本，不改變涵蓋性。隨機策略沒有這個保證，它的覆蓋率是一個機率量——這就是為什麼它的曲線會有一條永遠貼不到 100% 的漸近線。

**為什麼螺旋式在這個實驗中略優於弓字形**

螺旋式每一圈都沿著「已掃區域的邊界」前進，在矩形房間中轉彎次數較少、繞路較短，因此達標距離略低（12% 障礙時 1.16 倍 vs 1.23 倍）。

但這個優勢與房間形狀高度相關。在狹長走廊或多房間連通的格局中，螺旋式會反覆穿越已掃區域，優勢消失甚至反轉。

這正是值得同學自己驗證的地方：換一種房間形狀重跑一次，看結論會不會翻盤。一個會隨設定翻盤的結論，只要你說清楚條件，仍然是好結論；沒說清楚條件的結論，才是錯的。

## 三、必做的量化評估

- 覆蓋率：實際走過的格數佔可行格數的比例，目標接近 100%。
- 重複率：總行走距離除以理論最短覆蓋距離（第 16 章公式一）。建議改用「達到 95% 覆蓋所需的距離倍數」，因為最後 5% 往往佔掉隨機策略一半以上的時間，會把平均值拉得失真。
- 轉彎次數：轉彎耗時且耗電，是實務上的重要指標。
- 對障礙的適應性：在障礙物數量與配置不同的房間中測試，觀察策略的穩健度。

## 四、進階挑戰

- 房間分割：實作第 16.4 節的距離變換法，把地圖自動切分成房間，再依房間逐一覆

蓋，比較與「整體覆蓋」的效率差異。

- 斷點續掃：實作中斷後記錄已完成區域、重啟後只掃剩餘部分的功能，並驗證覆蓋率仍達 100%。
- 與實體整合：把你的覆蓋規劃器接上第 29 章建好的地圖與 Nav2，讓實體小車真的執行覆蓋清掃。
- 加入定位誤差：把本節的實驗重跑一次，但讓機器人的位置帶有 SLAM 誤差（例如每公尺 2 公分的隨機偏移）。這會讓「已掃過」的紀錄本身變得不可靠——這正是真實掃地機器人最難的部分，也是本書前三十章存在的理由。

### 五、教學上的價值

這個專題的獨特之處是「人人都有直覺、一動手就發現不簡單」。同學可能自認「不就是一排一排掃」，但一遇到不規則房間、障礙物、多房間連通，就會發現問題的複雜度。這種從直覺到嚴謹的落差，是很好的演算法思維訓練。

## 31.4　專題四：機器狗校園巡檢 SLAM

這是難度較高的整合型專題，涵蓋第 18、24、27 章的技術，適合專題生投入一到兩個學期。

### 專題目標

以四足或輪型機器人在校園執行自主巡檢，建立三維地圖、辨識巡檢目標、並產出巡檢報告。

### 五個階段

- 階段一——資料採集：以手推車或機器人載具搭載光達與 IMU，在走廊、樓梯間、實驗室錄製資料集（第 18.7 節）。這一步不需演算法能力，適合起步。
- 階段二——離線建圖比較：以錄好的資料執行 LeGO-LOAM、LIO-SAM、Fast-LIO，比較軌跡精度、地圖品質與運算負擔（第 18 章）。
- 階段三——退化場景研究：專門針對長走廊，比較純光達與光達加 IMU 緊耦合的差

異，量化 IMU 的貢獻。這是最有研究價值的一段。

- 階段四——語意感知：訓練 YOLO 模型辨識巡檢目標（消防設備、門、異常），注意蒐集機器人視角的資料（第 24.4 節的低視角問題）。
- 階段五——自主巡檢與報告：整合定位、導航與辨識，設定巡檢點自動執行，產出含三維位置標註的巡檢報告。

特別提醒階段一的價值：它是整個專題中唯一「不需要任何演算法能力就能做、而且做完就一定有產出」的階段。一份錄製良好、標註清楚的校園資料集，本身就是可以傳給學弟妹的資產，也是整個專題最穩的保險——即使後面階段全部卡住，你手上仍有一份能寫進報告的東西。

**必做的量化評估**

- 軌跡精度：以閉環一致性或 AprilTag 提供的近似真值（第 24.3 節）評估。請一併閱讀 31.8 節：只用閉環一致性是不夠的。
- 地圖品質：牆面厚度、樓層結構是否正確、跨樓層是否對齊。
- 辨識準確率：YOLO 模型在機器人視角測試集上的精度，並與人類視角資料集的表現對照。
- 任務完成率：設定十個巡檢點，統計自主抵達並成功採集資料的比例。

**進階挑戰**

- 多機器人協作：兩台以上機器人分頭巡檢再融合地圖（第 27 章），比較與單機的效率差異。
- 跨時間變化偵測：間隔數週重複巡檢同一路線，自動比對找出環境的變化（第 23 章 Atlas 的多階段作業）。
- 語言任務介面：讓使用者以自然語言指定巡檢任務（第 24.5 節），直接銜接第 34 章的語言導航。

**安全與倫理提醒**

校園巡檢涉及公共空間與人員，執行前務必注意：取得場域使用許可、避開人流尖峰、機器人明顯標示、緊急停止機制、以及影像資料的隱私規範（第 27.7 節）。

影像資料的處理要特別留意：巡檢錄影必然會拍到師生。建議在資料進入訓練集之前就做人臉模糊，並在報告中說明你的處理方式——這一段寫得好，評審一定會注意到。

這些不是形式，而是專業工程師的基本素養。

## 31.5　專題五：視覺導航無人機

這個專題結合第 23、26 章的視覺慣性導航，難度高但成果吸引力極強。

### 專題目標

讓無人機在 GPS 拒止的室內環境自主定位與飛行，並完成指定的航線或探索任務。

### 四個階段

- 階段一——資料集分析：先以 EuRoC 或 TUM-VI 資料集執行 ORB-SLAM3，熟悉系統行為與評測方法（第 26.6 節）。不需硬體即可完成。
- 階段二——懸停穩定：以微型無人機在防護網內測試光流定位懸停，量測漂移範圍（第 26.5 節的安全原則）。
- 階段三——VIO 整合：讓 VIO 輸出的位姿接入飛控，實現以視覺定位為基礎的位置保持與簡單航線飛行。
- 階段四——自主任務：執行指定航線、簡單避障、或探索建圖任務。

### 必做的量化評估

- 懸停精度：定點懸停 60 秒的水平漂移範圍。

- 軌跡誤差：以 AprilTag 或動作捕捉提供真值，計算 ATE。
- 初始化時間：從啟動到 IMU 初始化完成所需的時間與運動量（第 23.4 節）。
- 失效邊界：逐步提高飛行速度，記錄系統開始不穩定的臨界速度（第 26.4 節）。

**進階挑戰**

- 影像品質自適應：實作依影像清晰度動態調整視覺與慣性權重的機制（第 26.4 節第五項對策）。
- 地空協作：與地面機器人協同探索，比較各自的覆蓋能力（第 25.7、27.1 節）。
- 自製資料集：錄製自己校園場域的視覺慣性資料集並公開，供後續學生使用。

> **安全是第一優先**
>
> 第 26.5 節的安全原則在此必須嚴格遵守：防護網或防護籠、微型平台優先、隨時可用的緊急停止、分階段測試、人員位置管理。
>
> 任何自主飛行的測試，都應先在繫繩或手動可隨時接管的狀態下驗證。跳過這個步驟是意外的主因。
>
> 專題的成功不值得用任何人的安全去換。把「今天不飛」列為一個隨時可以做的決定，並且事先講好什麼情況下一定不飛（風大、人多、電量不足、有人沒戴護目鏡）。

## 31.6　專題六：地磁 + IMU + 視覺融合定位

這是難度最高、也最能展現整合能力的專題，涵蓋第 5 至 7 章的感測器與第 11 章的融合理論。

**專題目標**

設計並實作一套多感測器融合的定位系統，量化各感測器的貢獻，並在多種環境中驗證融合

的價值。

### 四個階段

- 階段一——單感測器基準：分別以地磁、IMU、視覺（或光達）各自定位，量測各自的誤差特性與失效情境。這一步建立比較的基準。
- 階段二——兩兩融合：實作 IMU 加地磁（第 6.6 節的 AHRS）、IMU 加視覺（第 23 章的 VIO），量化融合帶來的改善。
- 階段三——完整融合：以 EKF 或圖優化整合三者（第 11、12 章），並加入依情境調整權重的機制（第 5.6 節的信任度切換）。
- 階段四——多環境驗證：在明亮教室、昏暗機房、鋼構旁、戶外等環境測試，驗證融合系統是否在各環境都優於單一感測器。

### 必做的量化評估

- 各配置的軌跡誤差：單一感測器與各種融合組合，在各環境的 ATE 比較表。
- 失效次數：各配置在各環境的跟丟或發散次數。
- 權重變化：記錄融合系統在不同環境中，各感測器權重的動態變化，驗證機制是否如預期運作。
- 運算成本：各配置的運算負擔，評估融合的代價。

### 這個專題的研究價值

本專題最有價值之處，在於它能產出一張「感測器在什麼環境該相信誰」的實測地圖。這不只是課堂練習——它正是第 4.7、5.6、7.6、30.7 節反覆討論的融合原則，在你自己的場域中的具體答案。

建議把成果整理成完整的研究報告：實驗設計、資料、圖表、分析與結論。這樣的報告已具備投稿學生研討會或作為研究所申請素材的水準。

### 與第九篇的銜接

完成這個專題的同學，已經具備進入第九篇的所有基礎。第 32 章要討論的語意 SLAM、第 33 章的模擬訓練與 Sim-to-Real、第 34 章的 VLM 語言導航，都是在這套穩固的幾何定位之上，再疊加理解與決策的能力——而幾何層若不穩，上面那幾層就是空中樓閣。

## 31.7　評分 Rubric：五個向度

本節提供一套完整的評分標準。它既是教師評量的依據，也是同學自我檢核的清單——建議在專題開始時就閱讀，而非最後才看。

| 向度 | 權重建議 | 評什麼 |
|---|---|---|
| 地圖品質 | 25% | 尺寸誤差、牆面清晰度、閉環一致性、完整性 |
| 定位誤差 | 25% | 軌跡精度、重複性、失效次數 |
| 系統穩定性 | 20% | 長時間運行、從失敗恢復、對環境變化的耐受 |
| 報告表達 | 20% | 問題定義、方法說明、數據呈現、分析深度 |
| 程式碼品質 | 10% | 可讀性、模組化、文件、可重現性 |

**表 31-3　專題評分的五個向度**

### 一、地圖品質（25%）

- 優秀：尺寸誤差小於 2%、牆線清晰無重影、迴路完美閉合、覆蓋完整無異常空洞，並能說明殘餘誤差的來源。
- 良好：尺寸誤差小於 5%、牆線大致清晰、迴路閉合、僅有少量可解釋的空洞。
- 及格：地圖可辨識環境結構，但有明顯變形或重影，且能指出問題所在。
- 待改進：地圖嚴重扭曲或不完整，且無法說明原因。

### 二、定位誤差（25%）

- 優秀：有真值比對的量化數據（ATE / RPE），誤差在合理範圍，並分析誤差隨時

間與環境的變化。

- 良好：有量化評估（如閉環落差、尺寸比對），數據合理。
- 及格：有定性評估（目視軌跡合理），但缺乏量化。
- 待改進：未做任何定位評估。

**三、系統穩定性（20%）**

- 優秀：能長時間穩定運行、跟丟後能自動恢復、並主動測試過失效邊界（如第 30.6 節的五種情境）。
- 良好：能完成完整任務，偶有失效但能人工恢復。
- 及格：能在理想條件下完成任務。
- 待改進：僅能片段運作，無法完成完整流程。

**四、報告表達（20%）**

- 優秀：清楚定義問題、說明方法選擇的理由、以圖表呈現數據、深入分析結果、並誠實討論限制與失敗的嘗試。
- 良好：結構完整、數據清楚、有基本分析。
- 及格：能說明做了什麼，但缺乏分析與反思。
- 待改進：僅有操作紀錄，無法看出理解程度。

**五、程式碼品質（10%）**

- 優秀：結構清晰模組化、有註解與說明文件、版本控制良好、他人能依文件重現結果。
- 良好：程式可執行、有基本註解與說明。
- 及格：程式可執行，但難以閱讀或缺乏說明。
- 待改進：程式無法執行或無法取得。

### 六、特別強調：誠實報告失敗

評量中一個常被忽略卻極重要的原則：誠實報告失敗的嘗試，應該加分而非扣分。

一份寫著「我們嘗試了 A 方法，因為某某原因失敗；分析後改用 B 方法，並驗證了改善」的報告，展現的理解深度遠高於只寫「我們用 B 方法，得到很好的結果」。前者展現了工程判斷，後者可能只是照抄教學。

本書從第一章就反覆討論「常見失敗」，正是這個精神。教師在評量時應明確傳達這個標準，鼓勵同學誠實面對困難。

### 七、展示的建議

- 三分鐘能講完核心：先說「解決什麼問題、怎麼做、結果如何」，細節放後面。
- 用影片而非現場演示：現場演示風險高（硬體故障、環境不同），事先錄好的影片更可靠，也能剪出重點。
- 數據要有對照：「誤差 5 公分」沒有意義，「誤差 5 公分，而未校正前是 30 公分」才有說服力。
- 準備一個「如果重來」：能說出「若有更多時間，我會如何改進」，展現的是持續思考的能力。

## 31.8　評估方法本身可靠嗎

31.7 節的前兩個向度合計佔了一半的分數，而它們都建立在一個假設上：同學報上來的數字，真的反映了系統的好壞。本節要檢查這個假設。

這不是吹毛求疵。在實際的專題評審中，最常見的狀況不是同學沒做評估，而是做了一個「看起來很科學、但其實證明不了任何事」的評估。以下兩個實驗各揭露一種這樣的陷阱。

### 一、陷阱一：閉環落差為零，不代表地圖是對的

閉環落差——繞一圈回到起點，看地圖上起終點差多少——是同學最愛用的指標，因為它不需要真值設備，拿捲尺就能量。問題是：它只對某一類錯誤敏感。

實驗設定：在一條 12 公尺 × 8 公尺的橢圓形迴路上模擬兩種里程計誤差。家族 A 是尺度偏差（輪徑量錯，走的距離一致地偏大或偏小 0.5% 到 6%）；家族 B 是航向隨機游走（每步的角度量測帶有隨機誤差，逐步累積）。每個家族各 150 次試驗，對每一次都算出三個數字：閉環落差、真實 ATE（與真值軌跡逐點比對）、以及地圖尺寸誤差。

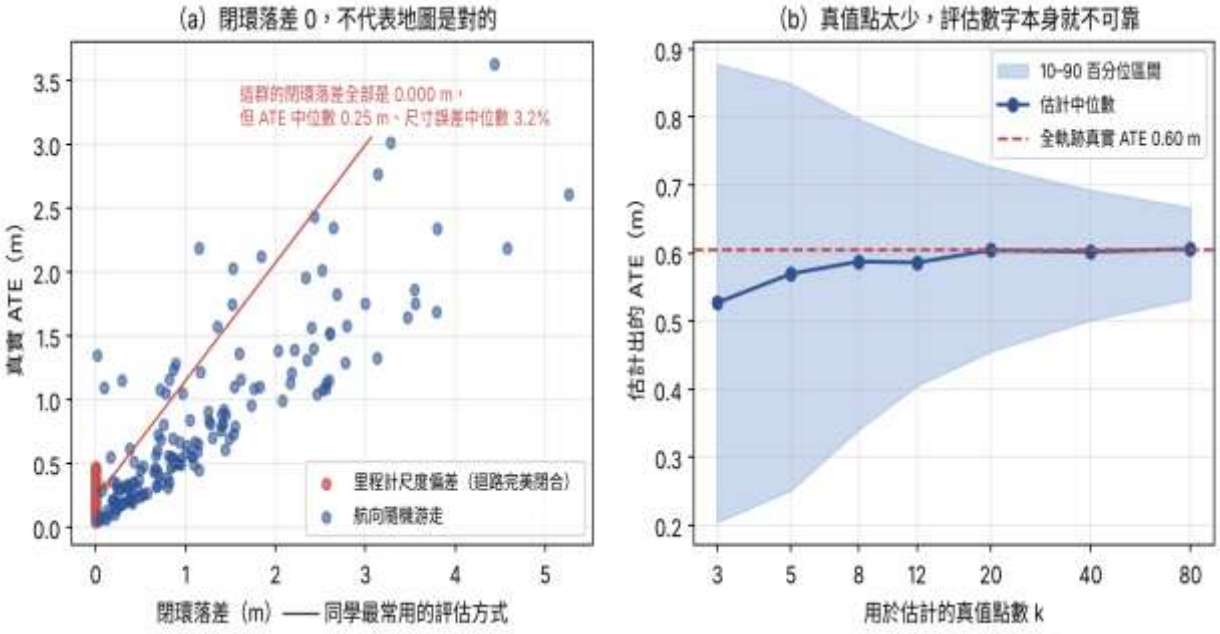



**圖 31-2　兩種評估方式各自看得見什麼**

（a）每一個點是一次試驗。紅點是尺度偏差家族，藍點是航向游走家族。（b）同一條軌跡，改變用於估計 ATE 的真值點數 k，各重複 800 次抽樣。

（a）的紅點全部擠在橫軸的零點上：尺度偏差家族的閉環落差在數值上是 0.000 公尺（最大 $1.5 \times 10^{-14}$ 公尺，純粹是浮點誤差）。理由很單純——把整張地圖等比例放大或縮小，迴路還是會閉合。但這些地圖的真實 ATE 中位數是 0.25 公尺，尺寸誤差中位數 3.2%、最大到 6.0%。依 31.7 節的標準，一張尺寸誤差 6% 的地圖連「良好」都勾不到，可是它的閉環一致性是滿分。

藍點則呈現另一種樣貌：閉環落差與 ATE 的相關係數 0.84，確實是個有用的指標。把兩個

家族合在一起看，相關係數 0.88 看起來更漂亮，但這個數字是假的——它完全來自藍點那一群，紅點那一整排垂直的點對相關係數幾乎沒有貢獻，卻正是最容易被漏掉的錯誤。

**結論：兩個指標要一起看，因為它們互補**

閉環落差看得見：航向誤差、掃描匹配的累積漂移、迴路偵測失敗。看不見：尺度偏差。

尺寸誤差（捲尺比對）看得見：尺度偏差。看不見：局部的扭曲與重影——地圖可以長寬都對，中間卻歪掉。

這就是 31.1 節要求「尺寸誤差、閉環一致性、目視檢查、導航成功率」四項一起做的原因。四項各自都有盲區，但盲區不重疊。

若能借到真值設備（動作捕捉、全站儀、或第 24.3 節的 AprilTag 方案），就直接算 ATE / RPE——但請先讀下一小節。

## 二、陷阱二：真值點太少，ATE 這個數字本身就在晃

假設你借到了設備，可以在場域中佈幾個真值點。問題來了：幾個才夠？

圖 31-2（b）用的是家族 B 的一條代表性軌跡，它的真實 ATE 是 0.60 公尺（用全部 1200 個取樣點算出來）。接著模擬「只量得到 k 個真值點」的情況：從軌跡上隨機抽 k 點算 ATE，每個 k 重複 800 次，畫出估計值的 10 到 90 百分位區間。

結果：k = 3 時，估計值的 10–90 區間是 0.20 到 0.88 公尺——同一套系統，可以報出 0.2 也可以報出 0.9，差了四倍多。k = 8 時區間收到 0.34–0.80，k = 20 時 0.46–0.73，k = 80 時才收到 0.53–0.67。

這代表什麼？如果你用三個真值點量出 ATE = 0.25 公尺，而隔壁組用三個真值點量出 0.55 公尺，你不能說你的系統比較好——這兩個數字都在同一個分布裡，差異完全可能只是抽樣運氣。要讓「我的系統優於對照組」這句話站得住，真值點的數量必須多到區間不重疊為

止。

| 真值點數 k | ATE 估計的 10–90 區間（m） | 相對真值 0.60 m 的偏離 |
|---|---|---|
| 3 | 0.20 – 0.88 | −67% ~ +45% |
| 5 | 0.25 – 0.85 | −58% ~ +41% |
| 8 | 0.34 – 0.80 | −44% ~ +32% |
| 12 | 0.40 – 0.76 | −33% ~ +26% |
| 20 | 0.46 – 0.73 | −25% ~ +20% |
| 40 | 0.50 – 0.69 | −17% ~ +15% |
| 80 | 0.53 – 0.67 | −12% ~ +10% |

**表 31-4　真值點數與 ATE 估計的不確定度**

實務建議：若要做組間或方法間的比較，至少 20 個真值點；若只是要報一個概略的量級，8 到 12 點可以接受，但報告中必須寫明點數，並說明這個數字帶有約 ±30% 的不確定度。把不確定度寫出來，不會讓分數變低——依 31.7 節的標準，那是「優秀」等第的特徵。

### 三、這兩個實驗要同學帶走的事

1. 評估方法有盲區，而且盲區是方法的性質，不是運氣。選一個指標之前，先問它對哪一類錯誤不敏感。
2. 用兩個以上盲區不重疊的指標，比把一個指標做得很精細更有效。
3. 任何估計值都有不確定度，包括你用來證明系統很好的那個數字。樣本太少時，兩組數字的差異可能完全沒有意義。
4. 在報告中寫出「我的評估方法看不見什麼」，比多報三個小數位更能顯示你真的懂。

## 給教師的課程規劃建議

本節提供三種課程長度的專題安排建議，供教師參考。

| 課程類型 | 時數 | 建議專題 | 重點調整 |
|---|---|---|---|
| 高中選修 | 18 週 × 1–2 小時 | 專題一（模擬）或三 | 以模擬與紙上活動為主，不碰原始碼 |
| 大學部 | 18 週 × 3 小時 | 專題一 + 二 或 三 | 完整走過建圖、定位、導航、評估 |
| 專題 / 研究 | 一至兩學期 | 專題四、五或六 | 強調研究方法、量化比較與報告撰寫 |

**表 31-5　三種課程的專題安排建議**

### 時程規劃的原則

- 前三分之一講理論與工具：讓同學備妥環境與基礎知識，此時可穿插小型的模擬練習維持動機。
- 中間三分之一做基礎實作：完成第 29、30 章的引導式實作，建立成功經驗。
- 最後三分之一做專題：此時同學已有能力自主探索，教師轉為顧問角色。
- 預留緩衝：實作課程的意外極多（硬體故障、環境問題、相依性錯誤）。建議預留至少兩週的緩衝時間——下一節會說明「至少兩週」這個數字其實還偏樂觀。

### 分組的建議

- 三到四人一組較理想：太少則工作量過重，太多則容易有人搭便車。
- 明確分工但輪替：例如分為建圖、導航、評估、報告四個角色，但中途輪替，確保每人都接觸到完整流程。
- 把「評估」獨立成一個角色：這是最常被犧牲的工作，因為它不像建圖那樣有立即的成就感。指定專人負責，並要求他在每個里程碑都交出數據。

# 案例

## 案例 31-1　一張滿分的地圖，錯了 4%

某組的專題一成果：教室地圖牆線筆直、無重影、繞行一圈起終點完全重合，目視 check 無可挑剔，組員在報告中寫下「閉環誤差為零，地圖品質優秀」。

評審拿捲尺量了教室：實際長 9.2 公尺，地圖上量出 8.83 公尺，誤差 4.0%。原因是輪徑參數用了規格書上的標稱值，而輪胎因為充氣不足實際半徑偏小——這正是圖 31-2 中紅點那一族的真實版本。

這個案例的教訓不是「同學偷懶」，恰恰相反：他們做了評估，只是做的那一項對這種錯誤沒有感覺。補救也很簡單——第 29.3 節的里程計校正程序跑一次，重新建圖後尺寸誤差降到 0.8%。整件事花不到一個下午，但如果沒有人拿捲尺去量，這張地圖會一路帶著 4% 的誤差進到導航階段，然後在 Nav2 反覆錯過目標點時，讓他們去懷疑導航參數。

## 案例 31-2　沒有對照組的「改善」

某組在專題三中實作了一套改良的覆蓋規劃，報告寫「覆蓋率達 97%，效果良好」。評審問：原本的方法是多少？答不出來。

這是最常見的失分方式。97% 這個數字本身不帶任何資訊——從圖 31-1 可以看到，在 12% 障礙的房間裡，最陽春的弓字形走到 1.23 倍距離就超過 95%，而隨機碰撞走到 8 倍也達不到。97% 到底是了不起還是理所當然，取決於你跟誰比、在什麼條件下比。

正確的寫法是：「在相同的房間配置與相同的行走距離預算下，基準弓字形覆蓋 92.4%、本方法 97.1%，在 20 組隨機配置中本方法有 17 組較優。」這句話裡有對照組、有控制變因、有重複次數，三者缺一，結論就只是一個感想。

## 案例 31-3　三種典型的專題失敗模式

**模式一：目標過大**

開學時信心滿滿地設定「做一台能自主巡檢全校的機器狗」，期末交出一份只有模擬的報告。問題不在志向，而在沒有定義「最小可行成果」——當第一個階段就卡住時，沒有任何東西可以撤退。

**模式二：環境問題吃掉時間**

相依性衝突、驅動程式不相容、硬體到貨延遲、場域借不到。這些事情單一來看都是小事，加起來卻能吃掉半個學期。下一節的模擬會告訴你：這不是運氣差，這是常態。

**模式三：沒有量化評估**

做了很多、也真的跑起來了，但整份報告只有照片與操作紀錄。依 31.7 節的標準，這樣的專題在地圖品質與定位誤差兩個向度上合計 50% 的分數會嚴重失血，而這五成正是最容易補起來的——一把捲尺加一個下午就能把「及格」拉到「良好」。

這三種模式的共同解方是：在專題開始時就想清楚「最後要交出什麼」。把 31.7 節的評分標準與圖 31-4 的檢核表放在眼前，逆推每個階段該做什麼——這是專案管理的基本功，也是同學進入業界或研究所後最實用的能力之一。

## 案例 31-4　時程要留多少緩衝：一次模擬

「預留兩週緩衝」是常見的建議，但兩週是怎麼來的？這一節用蒙地卡羅模擬給一個有依據的答案。

模型設定：一個專題切成四個里程碑，規劃工期分別是 3、4、4、3 週，合計 14 週。每個里程碑的實際工期服從對數常態分布，中位數等於規劃值——也就是說，規劃並沒有系統性低估，只是有不確定度。以 $\sigma$ 代表不確定度的大小：$\sigma = 0.25$ 是順利的軟體專題，$\sigma = 0.45$ 是一般情況，$\sigma = 0.65$ 是牽涉硬體、場域與外部相依的專題。每種設定跑 40000 次。

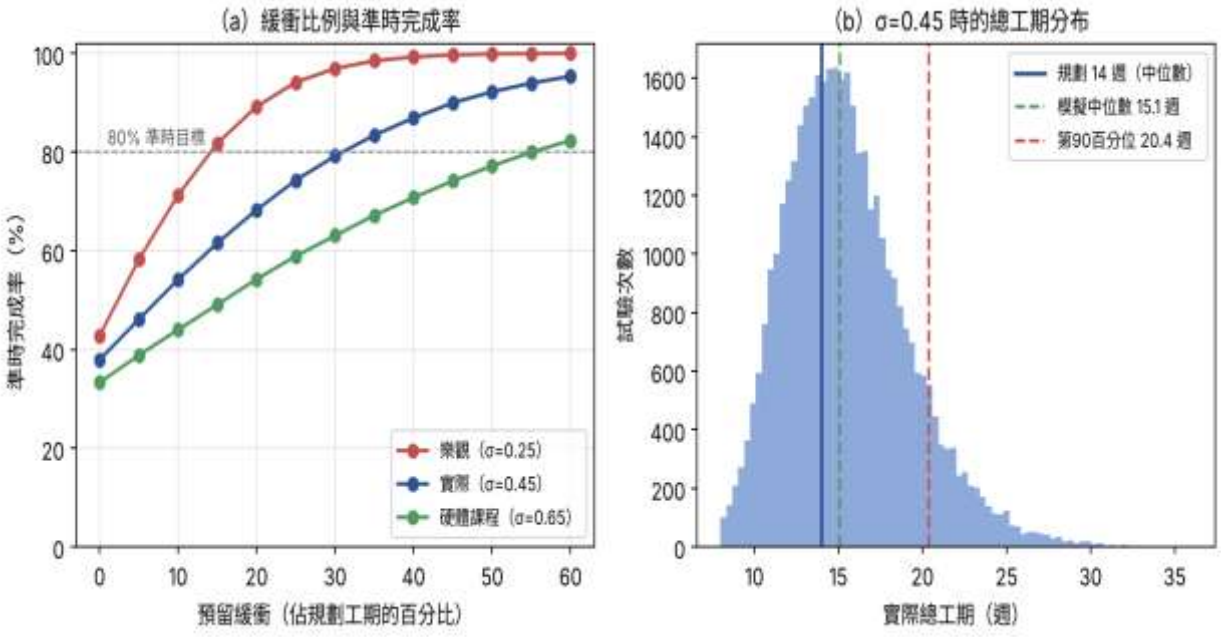



**圖 31-3　緩衝比例與準時完成率**

（a）橫軸是預留緩衝佔規劃工期的百分比，縱軸是在緩衝內完成的機率。（b）是 σ = 0.45 時總工期的分布。

## 一、三個反直覺的數字

1. 不留緩衝，準時機率只有三到四成。σ = 0.45 時是 37.9%，σ = 0.65 時只剩 33.3%。注意這裡每個里程碑的中位數都等於規劃值——沒有人低估，但合起來仍然大概率超時。

2. 原因是分布不對稱。看（b）的直方圖：左邊被 0 擋住，右邊拖著一條長尾。一個階段提早完成最多省幾天，但卡住可以卡三週。所以總工期的中位數 15.1 週已經高於規劃的 14 週，而第 90 百分位是 20.4 週——規劃值的 1.46 倍。

3. 要達到 80% 準時，σ = 0.45 需要約 30% 的緩衝（14 週要變 18.2 週），σ = 0.65 需要約 55%。換算回來：一個 14 週的專題，「兩週緩衝」只能撐到 55% 的準時率；要有八成把握，軟體專題要留四週，硬體專題要留將近八週。

## 二、這對專題規劃的四個意涵

- 緩衝放在最後，不要分散到各階段。分散的緩衝會被帕金森定律吃掉（工作會膨脹

填滿可用時間）；集中的緩衝才看得見剩多少。

- 定義最小可行成果，而且要在第一個里程碑就達成。模擬告訴我們超時是常態，所以必須保證「超時的時候手上已經有東西」。
- 把不確定度最高的階段排在前面。硬體到貨、場域申請、相依性安裝這些 σ 很大的事，越早踩到坑，剩下的時間越能反應。
- 每個里程碑結束就重估。第一個階段若花了 1.5 倍時間，那是你這個專題 σ 偏大的證據，後面三個階段的預期也要跟著上修，而不是指望它們會「補回來」。

**這個模型的限制（也請同學在自己的報告中這樣寫）**

模型假設四個階段的工期彼此獨立。實際上它們常常相關——硬體不穩定會同時拖慢好幾個階段，這會讓長尾更重，所以上面的數字是偏樂觀的估計。

模型也假設各階段的中位數等於規劃值。若規劃本身偏樂觀（多數人都是），實際情況會比圖 31-3 更差。

對數常態是一個方便的選擇，不是從資料擬合出來的。若你手上有前幾屆專題的實際工期紀錄，換上真實分布重跑一次，那會是一份相當有價值的課程改進報告。

## 實作活動

### 實作 31-1　專題規劃與提案

目的：在正式投入專題之前，完成一份完整的規劃提案。這個練習本身就是重要的訓練——多數專題的成敗，在規劃階段就已決定。

**第一部分：選題與可行性評估**

- 依表 31-2 選出兩到三個候選專題，對每一個評估：所需硬體是否可取得？所需能力是否具備（對照第 28.7 節的五階段）？時間是否充足？

- 對最有興趣的專題，列出「最小可行成果」與「理想完整成果」。前者是必定要達成的底線，後者是行有餘力的目標。
- 風險評估：列出三個最可能失敗的環節，並為每個準備備案。例如「若實體小車無法運作，改以模擬完成階段二」。

**第二部分：里程碑規劃**

- 把專題切分成三到五個里程碑，每個里程碑明確定義：要完成什麼、產出什麼證據、預計花多少時間。
- 在時程表上標出檢核點。緩衝比例請依案例 31-4：純軟體或模擬的專題留 30%，牽涉硬體、場域或外部相依的專題留 50% 以上，並把緩衝集中放在最後。
- 對每個里程碑，列出「若這一步卡住超過一週，我會怎麼辦」的應對方案。

**第三部分：評估方法設計（關鍵）**

- 依 31.7 節的五個向度，為你的專題設計具體的評估方法。例如地圖品質要量什麼、定位誤差用什麼當真值。
- 依 31.8 節檢查你選的指標：它對哪一類錯誤不敏感？你有沒有第二個盲區不重疊的指標來補？
- 估算真值點數：你打算量幾個點？依表 31-4，這個點數下你的 ATE 估計有多大的不確定度？這個不確定度小到足以支撐你想下的結論嗎？
- 設計對照組：你的成果要與什麼比較？（不同方法？不同參數？有無某個元件？）沒有對照，數據就沒有意義（案例 31-2）。
- 規劃資料收集：每個里程碑要收集哪些數據？用什麼格式記錄？這能避免最後才發現數據不足（案例 31-3 的模式三）。

**第四部分：提案發表**

- 把上述內容整理成一份兩頁的提案，並準備五分鐘的口頭報告。

- 向同學與教師報告，重點在說服他們：這個題目有價值、你有能力完成、而且你知道會遇到什麼困難。
- 接受提問並修正提案。他人的質疑往往能點出你的盲點——這比自己埋頭做三週才發現問題有效率得多。

**討論問題**

1. 你的「最小可行成果」是什麼？若時間只剩一半，你會捨棄哪些部分？
2. 你設計的評估方法中，真值（ground truth）從何而來？若沒有真值，你如何論證成果的品質？
3. 你的指標看不見哪一類錯誤？用什麼補？（對照圖 31-2）
4. 你的對照組是什麼？為什麼這個對照能凸顯你的成果價值？
5. 依案例 31-3 的三種失敗模式，你的專題最可能踩到哪一種？你的預防措施是什麼？
6. 依案例 31-4，你的專題 σ 大概落在哪一檔？你留的緩衝夠嗎？

**延伸挑戰**

把你的提案寫成正式的研究計畫格式：研究背景與動機、文獻回顧（可直接引用本書各章的延伸閱讀）、研究方法、預期成果、時程規劃、參考文獻。這份文件不只是專題的起點，也是日後撰寫研究所申請資料、國科會大專生研究計畫、或競賽企劃書的基礎。撰寫這份文件的過程，本身就會逼你把想法釐清——這是本活動最大的價值。

## 實作 31-2　把評估方法做一次壓力測試

目的：用本章 31.8 節的方法，檢驗你自己打算用的評估方式。這個活動只需要一台電腦，一節課可以做完，但做過的同學通常會回頭改掉原本的評估設計。

1. 取一條你自己的軌跡資料（或用模擬產生一條繞一圈的軌跡當作真值）。
2. 人為加入兩種錯誤：（甲）把所有步長乘以 1.03，模擬輪徑量錯；（乙）在每一步的航向加上平均為零的隨機誤差並累積。兩種各產生 50 條軌跡。

3. 對每一條算三個數字：閉環落差、尺寸誤差、以及與真值比對的 ATE。
4. 畫出「閉環落差 vs ATE」與「尺寸誤差 vs ATE」兩張散布圖，並分別計算相關係數。
5. 回答：你原本打算用的那個指標，對哪一種錯誤是瞎的？要加上哪一個指標才補得起來？
6. 進階：從你的軌跡中隨機抽 k 個點估計 ATE（k 取 3、5、10、20、50），每個 k 重複 500 次，畫出估計值的分布。你實際打算佈幾個真值點？那個點數下，你有把握區分「誤差 0.3 公尺」與「誤差 0.5 公尺」兩套系統嗎？

這個活動的產出——兩張散布圖加一段說明——可以直接放進專題報告的「評估方法」一節。依 31.7 節的標準，能寫出「我知道我的指標看不見什麼」的報告，在報告表達這個向度上就已經站到「優秀」的位置。

## 延伸閱讀

- J. Sturm et al., "A Benchmark for the Evaluation of RGB-D SLAM Systems," IROS, 2012.（ATE 與 RPE 評測方法的權威來源，TUM RGB-D 資料集的說明論文）
- evo 軌跡評測工具（GitHub: MichaelGrupp/evo）。（專題評估的必備工具，支援多種資料集格式與圖表輸出）
- Z. Zhang and D. Scaramuzza, "A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry," IROS, 2018.（軌跡評測的完整教程，含尺度對齊等細節；本章 31.8 節的問題意識與這篇一致）
- C. Cadena et al., "Past, Present, and Future of SLAM," IEEE T-RO, 2016.（第 1 章即引用的權威回顧，適合作為專題文獻回顧的起點）
- 本書第 13.7 節（地圖品質評估）、第 17 章案例 17-1（牆面厚度指標）、第 22.7 節（軌跡評測）。（專題評估方法的內部參照）

- S. Macenski et al., "The Marathon 2: A Navigation System," IROS, 2020.（Nav2 論文，含系統性能評估的方法論可供參考）
- E. Galceran and M. Carreras, "A Survey on Coverage Path Planning for Robotics," Robotics and Autonomous Systems, 2013.（覆蓋路徑規劃的回顧，專題三的理論依據）
- 各校大專學生研究計畫與專題競賽的優秀作品集。（觀摩他人如何組織一份完整的專題報告）

## 兩張可以貼在桌前的圖

**圖 31-4　專題從規劃到展示的完整流程**

【規劃階段】

① 依表 31-2 選定專題 → ② 評估資源與能力 → ③ 切分三至五個里程碑

④ 依案例 31-4 決定緩衝比例（軟體 30%、硬體 50% 以上，集中在最後）

↓

【執行階段】

⑤ 模擬驗證（先把流程與參數調通）

⑥ 實體實作（預留兩倍除錯時間）

⑦ 每個里程碑產出可驗證的證據（第 29.7 節），並重估剩餘時程

↓

【評估階段】

⑧ 依 31.7 節的五向度自我檢核

⑨ 量化評估：ATE / RPE / 尺寸誤差 / 覆蓋率（依專題而定）

⑩ 依 31.8 節檢查指標盲區與真值點數是否足夠

⑪ 誠實記錄失敗的嘗試與原因

↓

【展示階段】

⑫ 撰寫報告：問題、方法、數據、分析、限制

⑬ 準備三分鐘核心說明 + 展示影片

⑭ 準備「如果重來會怎麼做」的反思

**圖 31-5　繳交前的自我檢核清單**

□ 地圖品質：有量化的尺寸誤差數據嗎？能說明誤差來源嗎？

□ 定位精度：有 ATE 或閉環落差的數據嗎？兩者都量了嗎？

□ 指標盲區：我的指標看不見哪一類錯誤？有補上第二個指標嗎？

□ 真值點數：點數夠支撐我想下的結論嗎？不確定度寫出來了嗎？

□ 系統穩定：測試過失效邊界嗎？知道系統在什麼條件下會壞嗎？

□ 對照組：有與其他方法或參數設定比較嗎？控制變因說清楚了嗎？

□ 失敗紀錄：誠實記錄了失敗的嘗試與解決過程嗎？

□ 證據完整：地圖檔、軌跡圖、錄影、程式碼都齊全嗎？

□ 可重現：他人依你的文件能重現結果嗎？

□ 反思：能說出「如果重來會怎麼做」嗎？

## 小結

- 六個專題難度遞增：教室 2D 建圖（★）、三維空間掃描（★★）、掃地機器人路徑分析（★★）、機器狗校園巡檢（★★★★）、視覺導航無人機（★★★★）、多感測器融合定位（★★★★★）；選擇時應評估資源、能力與延續性。
- 專題一涵蓋 SLAM 完整循環且成果直觀，評量重點不在「有沒有建出地圖」，而在「能不能說清楚地圖為什麼好或不好」；專題二的產出對系所空間管理與模擬環境有長期價值，也可以直接成為第 33 章模擬訓練的場景。
- 專題三的實驗結果：走完一倍的理論最短距離時，螺旋式覆蓋 83.2%、弓字形 75.6%、隨機碰撞只有 35.3%；要達到 95% 覆蓋，弓字形在無障礙房間需 0.95 倍距離、隨機碰撞需 6.9 倍，而在障礙 12% 以上的房間隨機碰撞走到 8 倍仍未達標。系統性策略的優勢不只是快，而是它有涵蓋性的保證。
- 評分採五個向度：地圖品質（25%）、定位誤差（25%）、系統穩定性（20%）、報告表達（20%）、程式碼品質（10%）；建議在專題開始時就閱讀標準並逆推規劃。

- 評估方法本身有盲區：尺度偏差造成的地圖錯誤，閉環落差完全看不見——實驗中這一族的閉環落差是 0.000 公尺，但 ATE 中位數 0.25 公尺、尺寸誤差中位數 3.2%、最大 6.0%。閉環落差與尺寸誤差必須一起用，因為兩者的盲區不重疊。
- 評估數字本身帶有不確定度：同一條真實 ATE 為 0.60 公尺的軌跡，用 3 個真值點估計的 10–90 區間是 0.20–0.88 公尺，用 20 點才收到 0.46–0.73。要做組間比較，真值點至少 20 個，並在報告中寫明點數與不確定度。
- 時程的不確定度是不對稱的：四個里程碑、規劃 14 週的專題，在一般不確定度下不留緩衝只有 37.9% 準時，總工期中位數 15.1 週、第 90 百分位 20.4 週。要有八成把握，軟體專題約需 30% 緩衝、硬體專題約需 55%，且緩衝應集中放在最後而非分散。
- 專題失敗的三種典型模式為目標過大、環境問題吃掉時間、以及沒有量化評估；共同解方是在開始時就想清楚「最後要交出什麼」，並依評分標準逆推每階段的工作。
- 誠實報告失敗的嘗試應該加分而非扣分——展現發現問題、分析原因、驗證改善的完整歷程，遠比只呈現漂亮結果更能說明理解深度；寫出「我的評估方法看不見什麼」同樣是優秀等第的特徵。
- 展示的建議：三分鐘能講完核心、用預錄影片而非現場演示、數據要有對照組、並準備一個「如果重來會怎麼做」的反思。

## 第八篇結語

第八篇「ROS / Python / C++ 實作」至此全部完成。從第 28 章的環境建置、第 29 章的光達建圖導航、第 30 章的視覺 SLAM 與跨感測器比較，到本章的專題整合與評量，同學已經走完了從理論到實作的完整路徑。

也請記得本章 31.8 節的那兩張散布圖。前面三十章教的是怎麼讓機器人知道自己在哪裡；這一章補上的是怎麼知道「它真的知道」——而後者比前者更容易被跳過，也更能分辨一份作品的成色。

最後一篇，我們把視野投向未來。傳統 SLAM 解決的是「我在哪裡、地圖長什麼樣」；而 AI 與 VLM 要解決的是「我看到了什麼、這個地方代表什麼、下一步該怎麼做」。第九篇共有三章：第 32 章「語意 SLAM：讓地圖帶有意義」讓幾何地圖升級為語意地圖；第 33 章「模擬訓練與 Sim-to-Real」讓機器人在虛擬環境中學會技能並部署到真實機身；第 34 章「VLM、語言導航與具身智慧」則讓機器人聽懂「請去教務處」這樣的指令，並為全書作結。

你在本章建立的那套評估習慣，到了第九篇只會更重要。語意與語言這一層的失敗，往往不會當場報錯——它只是安靜地把機器人帶到錯的房間。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第九篇　AI、VLM 與語意導航 SLAM

## 第 32 章

## 語意 SLAM：讓地圖帶有意義

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程　/　高中 AI 機器人社團課程

## 學習目標

走過三十一章，同學已經能讓機器人回答「我在哪裡、地圖長什麼樣」。本章要補上第三個問題：「這個地方是什麼」。我們從傳統 SLAM 的侷限談起，說明 AI 視覺模型在 SLAM 中的分工、語意地圖的四種表示法，再處理動態物體與語意代價圖兩個實務難題，最後回顧國際研究地圖。讀完本章，同學應能：

1. 說明幾何地圖的侷限，以及機器人智慧的三個層次為何是疊加而非取代。
2. 區分偵測、分割、深度、描述、場景理解五類 AI 視覺模型在 SLAM 中的分工。
3. 說明語意地圖的四個層次，以及開放詞彙與場景圖為何是近年的研究重點。
4. 說明動態物體為何會汙染地圖，並能設計多趟一致性的剔除策略。
5. 說明語意如何進入代價地圖並改變導航決策，並能估算其代價與效益。
6. 說明語意標註的誤差如何經由外參與定位傳播到地圖上。
7. 舉例說明國際上語意 SLAM 的主要研究路線與代表系統。

走過三十一章，同學已經能讓機器人回答「我在哪裡、地圖長什麼樣」。但這還不夠——真正實用的機器人，還必須回答「我看到了什麼、這個地方代表什麼、下一步該怎麼做」。本章是第九篇的第一章，也是通往下一個十年的入口。學完本章之後，同學應能夠達成下列目標：

99. 說明傳統 SLAM 的限制，以及 AI 為何成為必要的補充。
100. 說明 YOLO、SAM、Depth Anything、DINO、CLIP 在機器人感知中的分工。
101. 說明從 Visual SLAM 到 Semantic SLAM 再到具身 AI 導航的演進脈絡。
102. 規劃機器人訓練資料的蒐集與標註流程，並訓練校園場景辨識模型。
103. 說明 VLM 如何理解自然語言導航指令，以及 RAG 如何提供校園空間知識。
104. 說明邊緣 AI 部署的關鍵技術：量化、蒸餾與記憶體管理。

105. 分析 AI 輔助 SLAM 的風險，並展望 SLAM 與基礎模型結合的未來。

## 核心概念

| 名詞 | 意義 | 關聯章節 |
|---|---|---|
| 語意地圖 | 在幾何地圖上附加類別、實例與關係資訊 | 32.3 |
| 實例分割 | 把影像切成一個個獨立物件，而非只有類別 | 32.2 |
| 開放詞彙 | 類別不必事先定義在固定清單中 | 32.3、32.6 |
| 三維場景圖 | 以「物件—區域—樓層」階層與關係描述空間 | 32.3 |
| 動態物體剔除 | 把會移動的東西排除在地圖之外 | 32.4 |
| 語意代價地圖 | 把「那是什麼」轉換成導航成本 | 32.5 |
| 標註傳播誤差 | 語意正確但外參或定位不準，標籤仍會貼歪 | 32.5、第 10 章 |
| 幾何—語意—任務 | 機器人智慧的三個層次，下層支撐上層 | 32.1 |

| 中文名詞 | 英文名詞 | 概念說明 |
|---|---|---|
| 語意 SLAM | Semantic SLAM | 在幾何地圖上加註物件與場景意義的 SLAM。 |
| 具身智慧 | Embodied AI | 透過與實體環境互動來學習與行動的人工智慧。 |
| 開放詞彙 | Open-Vocabulary | 不受預設類別限制，能辨識訓練時未見過的物件。 |
| 物件目標導航 | ObjectNav | 接受「去找某個物體」指令的導航任務。 |
| 場景圖 | Scene Graph | 以節點與關係描述場景中物件及其空間關聯的結構。 |
| 視覺語言模型 | VLM | 能同時理解影像與語言的多模態模型。 |
| 視覺語言動作模型 | VLA | 直接由影像與語言產生動作指令的模型。 |

| 中文名詞 | 英文名詞 | 概念說明 |
| --- | --- | --- |
| 檢索增強生成 | RAG | 先檢索相關資料再生成回答，以提升正確性。 |
| 模型蒸餾 | Distillation | 把大模型的能力轉移到小模型上的訓練方法。 |
| 量化 | Quantization | 以較少位元表示模型參數以降低記憶體與運算。 |
| 邊緣部署 | Edge Deployment | 在機器人本體的運算平台上執行 AI 推論。 |
| 幻覺 | Hallucination | AI 產生看似合理但實際錯誤內容的現象。 |
| 世界模型 | World Model | 學習環境動態、能預測未來狀態的模型。 |

## 32.1　為什麼傳統 SLAM 還需要 AI

本書前三十章建立的是一套極為紮實的幾何系統。它能精確地告訴你機器人在哪、環境的形狀如何——但它有一個根本的盲點。

### 傳統 SLAM 知道形狀，不知道意義

一台跑著 Cartographer 的機器人，面對一扇門時，它「知道」的是：前方 1.2 公尺處有一段寬 0.9 公尺的空隙，兩側是連續的平面。它不知道那是「門」——不知道門可以通過、門可能會關上、門後面通常是另一個房間。

這個差別在簡單任務中無關緊要（避開障礙不需要知道障礙是什麼），但在真實的服務任務中卻致命：

- 使用者說「去教務處」，而地圖上只有座標，沒有名字。
- 地上有一攤水，幾何上它是平坦的地面，語意上機器人應該繞開。
- 前方是玻璃門，光達看不到（第 8 章），但視覺能辨識出門框與把手。
- 走廊上站著一個人，幾何上是障礙物，語意上機器人應該減速並禮讓，而非把他當成牆壁繞過去。

### 三個層次的落差

| 層次 | 回答的問題 | 技術 | 本書章節 |
|---|---|---|---|
| 幾何 | 我在哪？環境形狀如何？ | SLAM | 第 1–30 章 |
| 語意 | 我看到了什麼？這裡是什麼地方？ | 物件偵測、分割、VLM | 本章 |
| 任務 | 下一步該怎麼做？ | LLM Agent、VLA | 本章 |

**表 32-1　機器人智慧的三個層次**

這三層是疊加而非取代的關係——這是本章最重要的觀念。語意層若沒有穩固的幾何層支撐，機器人會「知道那是門，卻走不到門口」；而幾何層若沒有語意層，機器人只能執行座標指令，無法與人自然互動。

### AI 帶來什麼

260. 感知的擴充：辨識物件、分割場景、估計深度——把像素轉成有意義的資訊。

261. 語言的介面：讓使用者以自然語言下達任務，而非輸入座標。

262. 常識的推理：「洗手台通常在廁所或茶水間」——這類知識無法從幾何地圖推導，卻能大幅提升探索效率（第 24.5 節）。

263. 泛化的能力：面對訓練時未見過的環境與物件，仍能做出合理判斷。

## 32.2　AI 視覺模型在 SLAM 中的角色

第 23.4、24.5 節介紹過 YOLO 與 VLM，本節系統性地整理各類視覺模型的分工——這是規劃 AI 感知系統的地圖。

### 五類模型的分工

近年的整理相當清楚地指出各家族的專長與弱點：影像文字對比模型（如 CLIP、SigLIP）是視覺語言模型的預設前端，擅長語意檢索但在稠密預測上較弱；自監督視覺變換器（如 DINOv2、DINOv3）因訓練目標保留了細緻的空間結構，在分割、深度、對應等稠密預測上表現優異，但其特徵未與語言對齊；分割基礎模型（如 SAM、SAM 2）是物件提議與追蹤的專家；稠密預測專家（如 Depth Anything、MiDaS）則處理深度。

| 模型類型 | 代表 | 擅長 | 在 SLAM 中的角色 |
|---|---|---|---|
| 物件偵測 | YOLO | 即時框出物件類別 | 語意標註、動態物體剔除 |
| 影像文字對比 | CLIP、SigLIP | 開放詞彙的語意檢索 | 地點辨識、語言查詢 |

| 模型類型 | 代表 | 擅長 | 在 SLAM 中的角色 |
|---|---|---|---|
| 自監督視覺 | DINOv2 / v3 | 稠密預測、空間結構 | 特徵提取、分割 |
| 分割基礎模型 | SAM、SAM 2 | 物件提議與追蹤 | 精確的物件邊界、動態追蹤 |
| 深度估計 | Depth Anything | 單張影像估深度 | 單目補足深度、稠密重建 |

**表 32-2　五類 AI 視覺模型在 SLAM 中的角色**

### 一個關鍵的工程限制

這裡有一個對機器人部署極為重要的現實：在穿戴裝置、機器人或智慧相機上的實用視訊系統，不可能為每一種能力各裝一個骨幹網路；既不能在能力上妥協，也無法承受記憶體與延遲的代價。

換句話說，雖然每個任務都有最佳的專用模型，但機器人的運算資源不允許同時跑五個大模型。實務上必須取捨——選一到兩個關鍵模型，或使用能兼顧多任務的統一骨幹。這正是第 25 章「硬體與演算法共同設計」思想在 AI 時代的延續。

### 對本書讀者的建議

- 入門：從 YOLO 開始。它成熟、輕量、資源豐富，能立即產生可見的成果（32.4 節的實作）。
- 進階：加入深度估計模型補足單目相機的深度（第 9.4 節），或用分割模型取得精確的物件邊界。
- 研究：探索開放詞彙模型，讓機器人能辨識訓練時未定義的物件——這是語意導航能否實用化的關鍵。

## 32.3　從 Visual SLAM 到 Semantic SLAM 到具身 AI 導航

本節梳理這條演進脈絡，讓同學看清自己所學的位置與前方的方向。

## 三個階段

264. 視覺 SLAM（第 20 至 23 章）：輸出相機軌跡與稀疏或稠密的幾何地圖。地圖中的每個點只有座標，沒有意義。

265. 語意 SLAM：在建圖的同時進行物件辨識與分割，把語意標籤附加到地圖元素上。地圖中的點不再只是「有東西」，而是「這是椅子的一部分」。

266. 具身 AI 導航：機器人不只有語意地圖，還能理解任務、推理常識、規劃行動。它能接受「去把會議室的燈關掉」這樣的指令並自主完成。

## 語意 SLAM 的兩種做法

- 後處理式：先用傳統 SLAM 建好幾何地圖，再把物件偵測的結果投影上去。實作簡單、模組解耦，是教學與專題的首選（32.4 節的實作路徑）。
- 緊耦合式：把語意資訊納入 SLAM 的估測過程——例如用物件層級的關聯取代點層級的關聯，或利用「牆是平面、地板是水平」等語意先驗來約束最佳化。精度更高但實作複雜。

**圖 32-1　語意地圖的四個層次，以及一句話如何變成導航目標**




語意地圖常被誤解成「在地圖上貼標籤」，但真正有用的語意是有階層的。圖的左半部把它分成四層：最底下是幾何層，只回答「這一格是空的還是被占住」；往上是物件層，把一叢點認定為一個實例，例如「這是滅火器」；再往上是區域層，把一群物件歸屬到一個有機能

的空間，例如「實驗室 B1」；最上層是建築層，記錄樓層與拓撲關係，例如「B1 在三樓，經 A 電梯可達」。

這個階層是由下而上建立的，順序不能顛倒：沒有正確的幾何，物件實例會落在錯的位置；沒有物件，區域就無從歸納。這也是為什麼本書把語意放在第九篇——它需要前面八篇全部到位才能成立。

圖的右半部則示範了階層的用途：由上而下解析一句話。使用者說「去三樓實驗室旁邊那個滅火器」，系統先在建築層選對樓層地圖，再到區域層找出「實驗室」，接著在該區域內的物件層搜尋「滅火器」，最後取出該實例的三維座標作為導航目標。每一層只需要回答一個簡單的問題，而不是讓一個模型一次處理整句話。

這正是三維場景圖的價值所在。若地圖只是一堆標了類別的格子，「實驗室旁邊」這個空間關係根本無法表達；而開放詞彙讓最後一步的搜尋不受固定類別清單限制——即使訓練時沒有「滅火器」這個類別，仍能以文字特徵比對找到它。

**四個層次的實務意義**

幾何層決定安全：撞不撞得到東西，只看這一層。語意出錯不致命，幾何出錯會。

物件層決定互動：能不能「去拿那個東西」，取決於實例是否被正確切分與定位。

區域層決定效率：有了房間概念，才能說「只掃廚房」或「先巡三樓」（第 16、19 章）。

建築層決定可達性：跨樓層的導航需要拓撲，而電梯是典型的拓撲節點而非幾何障礙。

專題建議：先做到物件層就很有價值，區域層可用第 16.4 節的距離變換法自動切分，建築層則往往用人工標注即可。

## 前沿：開放詞彙與場景圖

近年的研究正朝兩個方向推進：

- 開放詞彙線上語意建圖：讓 SLAM 系統不受預定義類別限制，能在建圖過程中辨識並標註任意物件——這對真實環境的複雜多樣至關重要。
- 場景圖表示：不只標註「這裡有一張椅子」，還記錄「椅子在桌子旁邊、桌子在會議室裡」的關係結構。近期研究已展示以多模態三維場景圖支撐零樣本的具身導航。

場景圖的價值在於它能支撐推理：當使用者說「去拿桌上的杯子」，機器人需要理解「桌上」這個空間關係——這在純幾何地圖中無法表達。

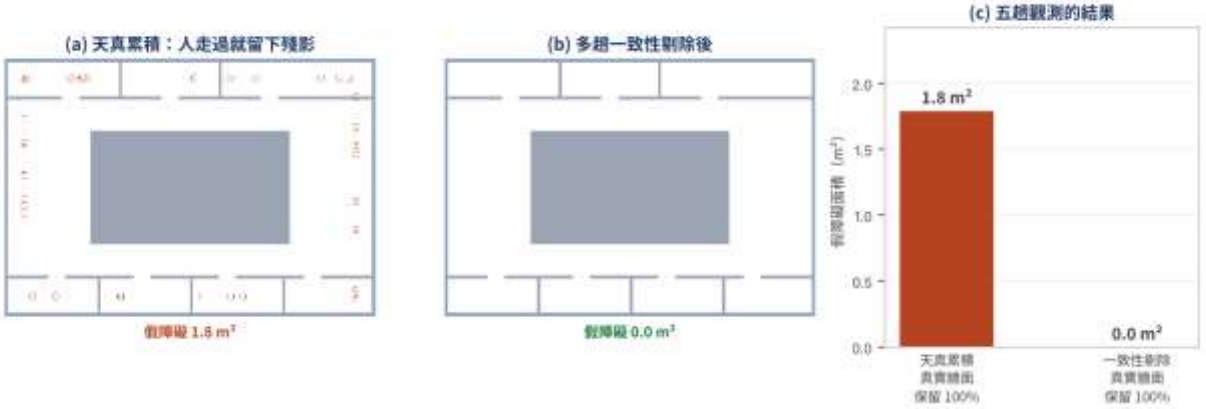
**學術與產業的匯流**

值得注意的是，這條演進線上，學術界與產業界正在快速匯流。相關研究已把 SLAM 與語意、三維高斯、鳥瞰視角、導航、大型語言模型、多階段作業等主題整合為同一個研究社群關注的範疇。對同學而言，這意味著：本書所學的幾何基礎，正是進入這個熱門領域的門票。

## 32.4　動態環境：會動的東西不該進地圖

語意最直接、也最早被產業採用的用途，其實不是「認出那是什麼」，而是「認出那個不該被記下來」。第 13.5 節談過佔據機率的上下限讓地圖保持可塑性，但那是被動的遺忘；有了語意，機器人可以主動判斷：這是人，所以不要寫進地圖。

**圖 32-2　人走過留下的殘影，以及怎麼把它清掉**




這個實驗讓機器人在同一層樓繞行五趟，每趟環境中都有七個隨機位置的人。圖(a) 是天真累積的結果：只要任何一趟觀測到某一格被占住，就把它記成障礙。灰色是真實牆面，紅色是地圖上多出來的假障礙——總共 1.8 平方公尺。它們散落在走廊各處，每一塊都是某個人在某一趟剛好站過的位置。

圖(b) 是加上「多趟一致性」判準之後：一格必須在五趟中至少三趟被觀測為占住，才會被記成障礙。結果是假障礙從 1.8 平方公尺降到 0，而真實牆面的保留率仍是 100%。道理很簡單——牆每一趟都在，人不會。

這個判準之所以有效，是因為它利用了一個語意上的事實：永久結構與臨時物體在時間上的行為完全不同。值得注意的是，這裡並沒有用到任何深度學習模型，只用了「多趟投票」這個統計方法——這正是第 19.6 節分層地圖策略的具體實作。

那麼什麼時候才真正需要 AI？答案是當你等不及五趟的時候。多趟投票需要時間，而機器人在第一趟就必須避開迎面走來的人。這時就需要即時的實例分割：在單一幀影像中認出「這是人」，直接把對應的點排除在建圖之外。兩種方法在實務上是並用的——AI 處理當下，投票處理長期。

**動態物體的三道防線**

即時層（毫秒）：以實例分割或物件偵測在單幀中剔除人、推車等已知動態類別。優

點是反應快，缺點是只認得訓練過的類別。

短期層（秒到分）：以連續幀之間的不一致性偵測移動物體，不需要類別知識，但對緩慢移動的物體不敏感。

長期層（趟與趟之間）：本圖的多趟一致性投票。不需要任何模型，能處理所有類型的變動，代價是需要時間累積。

三道防線解決的是不同時間尺度的問題，實務系統通常三者並用，而不是挑一個最好的。

## 32.5　語意如何改變導航決策

本節說明語意如何真的改變規劃器的行為，以及其中的陷阱。

### 從類別到成本：三種注入方式

- 區域層注入：把整個區域標上類別（禁區、人流熱區），再把類別映射成代價。最簡單，也最常用於人工設定的禁區。
- 物件層注入：偵測到特定物件（玻璃、鏡面、樓梯口）時，在其周圍加上一圈代價。適合處理感測器會失效的物體。
- 可通行性學習：直接從影像預測「這裡好不好走」，不經過明確的類別。近年在戶外與非結構化地形上相當流行，代價是難以解釋出錯的原因。

### 一個容易被忽略的陷阱：標註會貼歪

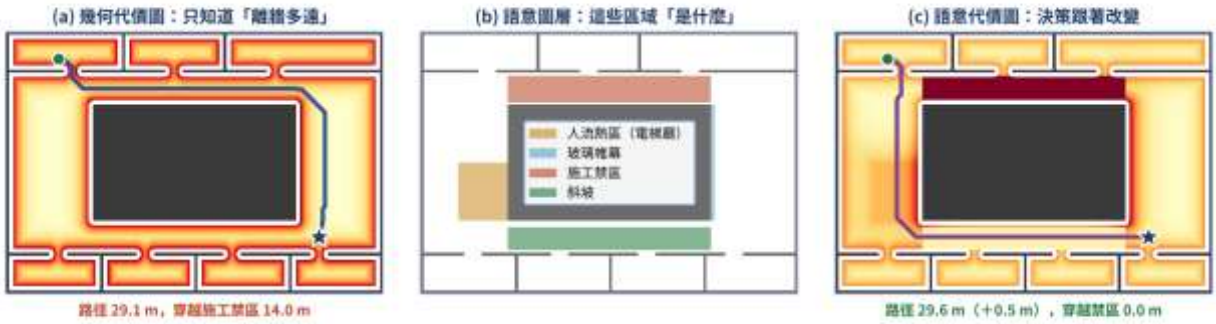
語意分割的準確率常被當成唯一指標，但對 SLAM 而言，「認得對」與「貼得準」是兩回事。一個把滅火器認得完美的模型，若相機與光達的外參差了兩度，標註落在地圖上的位置就會偏掉數十公分。

公式二已經寫出了這條傳播鏈：影像上的偵測結果要先經相機內參反投影、再經相機到底盤

的外參、底盤到里程計、里程計到地圖，才會落在最終的地圖座標上。這條鏈上任何一環的誤差都會累加。換句話說，語意地圖的品質同時取決於 AI 的辨識率與第 10 章的座標轉換品質——而後者往往才是實務上的瓶頸。

這帶出一個很實用的除錯順序：當語意標註貼歪時，先不要急著換更大的模型。先用一個已知位置的物件（例如貼在牆上的標靶）驗證外參與定位，確認幾何鏈沒問題之後，再去檢討辨識模型。本書第一至八篇的內容，在這裡仍然是地基。

**圖 32-3　同一張地圖，加上語意之後決策就不一樣了**




本書以一層回字形走廊的樓層做實驗：中央是電梯與機房構成的核心，四周環繞走廊，外圈是教室與實驗室。機器人要從左上角的房間送東西到右下角——繞核心左右兩側都走得通，於是「選哪一邊」就成了一個真正的決策。

圖(a) 是只有幾何資訊的代價地圖。顏色越紅代表越靠近障礙、成本越高，規劃器因此傾向走在走廊中央。它選了順時針的上方路線，全長 29.1 公尺——以純幾何的標準來看，這是正確答案。問題在於，這條路有 14.0 公尺穿過了施工禁區，而幾何地圖完全不知道那裡是禁區：對它而言，那只是一段沒有障礙物的走廊。

圖(b) 是加上語意圖層之後的同一層樓。四個區域被標示出來：電梯廳的人流熱區、玻璃帷幕、施工禁區，以及無障礙斜坡。這些資訊在幾何上完全看不出來——玻璃帷幕甚至是光達看不到的東西，它之所以出現在地圖上，正是因為有別的感測模態與語意模型把它標了出

來。

圖(c) 是把語意轉換成代價之後的結果。禁區被設成不可通行、人流熱區與玻璃帷幕被加上隨距離衰減的額外成本、斜坡則因為好走而降低成本。規劃器於是改走逆時針的下方路線，全長 29.6 公尺——只多了 0.5 公尺，卻把 14 公尺的禁區穿越降為零。這 0.5 公尺就是語意的價格，而它買到的是一條合規、安全、也更符合人類預期的路線。

**語意代價怎麼設才合理**

硬約束 vs 軟成本：禁區、施工區應設為不可通行（硬約束）；人流熱區、玻璃帷幕則適合設為可被權衡的額外成本（軟成本）。

衰減距離比數值更重要：本圖對玻璃帷幕設 1.0 公尺的衰減距離、人流熱區 1.3 公尺。這個距離決定了機器人「離多遠才開始閃避」，比成本數值本身更影響行為。

負成本也有用：無障礙斜坡被設為低成本，等於主動鼓勵機器人走它。語意不只用來避開壞的地方，也用來偏好好的地方。

小心過度加權：若把每個語意區都設成高成本，規劃器會變得保守甚至找不到路。實務上建議一次只加入一到兩層語意，確認行為合理後再疊加。

## 32.6　國際研究地圖：語意 SLAM 的三條路線

語意 SLAM 在國際上大致沿三條路線發展，各有代表性的研究群與系統。本節提供一張地圖，讓同學知道該往哪裡找資料。

### 一、度量—語意 SLAM：把語意接進既有的 SLAM 框架

這條路線的主張是：不要另起爐灶，而是在成熟的 SLAM 系統上加一層語意。代表性的工作把實例分割接到視覺慣性 SLAM 的後端，在建立幾何地圖的同時維護物件層的實例，並以三維場景圖組織它們。優點是繼承了既有系統的穩定度，也最符合本書第一至八篇的架構。

## 二、開放詞彙地圖：擺脫固定類別清單

傳統語意分割只能認出訓練時定義的那幾十個類別，但真實場域裡的東西遠不止於此。開放詞彙這條路線把視覺語言模型的特徵直接存進地圖的每一個元素，查詢時再以文字比對——於是「找紅色的滅火器」這種事先沒定義過的查詢也能運作。代價是記憶體用量大幅上升，而且查詢結果難以保證。

## 三、神經場與可算繪表示：定位與重建合一

第 17.8 與第 20.8 節都提過的三維高斯潑濺，在語意領域也有對應發展：把語意特徵一併存進每個高斯元素，讓地圖既能算繪出逼真影像、又能回答語意查詢。這條路線在重建品質上最有優勢，但運算與記憶體成本最高，目前多用於離線重建而非即時導航。

三條路線的取捨可以這樣理解：第一條最穩、最適合部署；第二條最靈活、最適合開放場域；第三條最漂亮、最適合展示與離線分析。對專題而言，建議從第一條開始——它與本書的技術棧完全相容，而且出錯時你知道該去看哪一層。

**怎麼追這個領域的進展**

會議：ICRA、IROS、RSS、CVPR、CoRL 是語意 SLAM 的主要發表場域。

關鍵字：semantic SLAM、open-vocabulary mapping、3D scene graph、instance-aware mapping、dynamic SLAM。

評估資料集：室內以 ScanNet、Replica 為主；戶外與自駕則看 SemanticKITTI、nuScenes。

實作起點：先在既有的 ORB-SLAM 或 RTAB-Map 上接一個現成的分割模型，把結果投影到地圖，親手走一遍公式二的傳播鏈——這比讀十篇論文更有感覺。

## 公式或流程

## 一、物件偵測的評估指標

訓練 YOLO 等偵測模型時，最常用的指標是平均精度均值：

$$mAP = (1/C) \Sigma_c AP_c$$

其中 C 為類別數、AP_c 為第 c 類的平均精度（精確率對召回率曲線下的面積）。常見的變體是以交併比門檻 0.5 計算的 mAP@0.5，以及在多個門檻上平均的 mAP@0.5:0.95（較嚴格）。

對機器人應用而言，還應關注兩個實務指標：

- 漏檢率：沒偵測到的比例。對安全相關的物件（人、樓梯口），漏檢比誤檢危險得多。
- 每類別的表現：總體 mAP 高不代表每一類都好。若「樓梯」這一類的精度特別低，而它正是安全關鍵，那麼整體數字就有誤導性。

## 二、把偵測結果投影到三維地圖

語意標註的核心運算，是把影像上的偵測框轉為三維座標。以偵測框中心像素 (u, v) 與其深度 d 為例，依第 30 章公式一反投影：

$$P_{camera} = ((u - cx) \cdot Z / fx,\ (v - cy) \cdot Z / fy,\ Z) \quad ,\ Z = d / scale$$

再依第 10 章的座標轉換鏈，轉到地圖座標系：

$$P_{map} = T_{map \leftarrow odom} \cdot T_{odom \leftarrow base} \cdot T_{base \leftarrow camera} \cdot P_{camera}$$

這條式子把本章的 AI 與第 10 章的座標轉換連在一起——語意標註看似是 AI 的工作，但它能否正確落在地圖上，取決於外參與定位的品質。這再次印證 32.1 節的觀點：語意層必須建立在穩固的幾何層之上。

## 四、AI 輔助 SLAM 的完整資料流

**圖 32-4　從感測到語意決策的端到端流程**

① 感測器：相機、光達、IMU

↓

② 幾何 SLAM：定位與建圖（第 13–22 章）→ 位姿 + 幾何地圖

↓

③ AI 感知：YOLO 偵測 / 分割 / 深度估計（32.2 節）

↓ 依公式二把偵測結果投影到地圖座標

④ 語意地圖：幾何 + 物件標籤 + 空間關係

↓

⑤ RAG 知識庫：校園拓樸、地點屬性、通行規則（32.5 節）

↓

⑥ VLM / LLM Agent：理解指令、推理、選擇目標（32.6 節）

↓ 輸出結構化的幾何目標

⑦ 路徑規劃與控制（第 29 章）→ 速度指令

↓

⑧ 執行 → 環境改變 → 回到 ①

【安全把關】步驟 ⑦ 有權否決 ⑥ 的決策

## 案例

## 案例

### 案例 32-1　語意對了，位置錯了

一個在專題中極常見的失敗：同學接上了準確率九成以上的物件偵測模型，機器人也確實認出了走廊上的滅火器，但在地圖上一看，標註點飄在牆外一公尺處，甚至落到隔壁房間去了。

直覺反應通常是「模型不夠好」，於是換更大的模型、重新標更多資料。但真正的原因幾乎都不在模型。偵測給的是影像上的一個框，要把它變成地圖上的一個點，必須經過深度估計、相機內參、相機到底盤的外參、以及當下的定位結果——四個環節。其中最常出錯的是外參：相機裝上去之後沒有重新校正，或是用了設計值而非實測值，差個兩三度就足以造成數十公分的偏移。

診斷方法很簡單：把一個已知位置的物件（貼在牆上的標靶）當成測試對象，讓機器人從不同角度偵測它十次，看標註點是否收斂在同一個位置。若標註點隨視角散開成一片，問題在外參或定位；若集中但整體偏移，問題在外參的平移；只有在標註點本身就跳來跳去時，才需要回頭檢討模型。

這個案例的教訓與第 19.5 節的重複定位精度相同：AI 的輸出要落在正確的地方，靠的是第 10 章的座標轉換與第 11 章的定位。語意層再強，也站在幾何層的肩膀上。

## 案例 32-2　為什麼掃地機器人不把你的腳記成牆

一個每天都在發生、卻很少被注意的語意應用。掃地機器人工作時，家裡的人會走來走去；若它把每一次偵測到的腳都記成障礙，地圖很快就會布滿不存在的牆，第 16 章談的覆蓋規劃也會跟著失效。

它的做法正是圖 32-2 的三道防線。即時層：以紅外線或相機判斷前方是移動物體，暫時繞開但不寫入地圖。短期層：同一位置在連續幾次掃描中時有時無，判定為不穩定觀測。長期層：多次清掃都沒再出現的障礙，其佔據機率會被逐步降低，最終從地圖上消失（第 13.5 節的上下限機制）。

值得注意的是，低階機型往往只有長期層，所以你會看到它把臨時放在地上的背包記成障礙，而且好幾天都繞著那個已經不存在的東西走。這個差異就是語意能力在產品上的具體價值——

—它不一定表現為「更聰明」，而是表現為「比較不會做蠢事」。

### 案例 32-3　開放詞彙的代價

開放詞彙地圖聽起來近乎萬能：不必事先定義類別，使用者想找什麼就用文字描述。但它的代價相當具體，值得同學事先知道。

其一是記憶體。傳統語意地圖每個元素只存一個類別編號（幾個位元組），開放詞彙地圖則要存一個視覺語言模型的特徵向量——常見維度是數百維的浮點數，也就是每個元素上千位元組。同樣一張地圖，記憶體用量可能差上兩三個數量級，這正是第 33 章要處理的邊緣部署難題。

其二是可靠性。固定類別的模型至少有明確的評估指標與失敗邊界；開放詞彙查詢則可能對相近的描述給出完全不同的結果，而且沒有一個好方法可以事先知道哪些查詢會失敗。對需要安全保證的導航任務而言，這是個嚴重的問題。

目前的務實做法是混合：安全關鍵的類別（人、樓梯、禁區）仍用固定類別的模型、有明確的評估與保證；開放詞彙則用於使用者互動與查詢這類「錯了也不會出事」的場合。這個分工原則同學可以直接套用在專題上。

## 實作活動

## 實作活動

### 實作 32-1　語意代價地圖與動態物體剔除

目的：親手把語意資訊接進地圖與規劃器，驗證它如何改變決策，並實作動態物體的多趟一致性剔除。本實作以 Python 為主，第一、二部分不需要機器人硬體。

**第一部分：語意代價地圖（全體）**

1. 取一張佔據格地圖（可用第 13 章實作的成果），以距離變換產生幾何代價圖，並用

A* 規劃一條起點到終點的路徑，記錄長度。

2. 在地圖上以矩形標出二至三個語意區域（禁區、人流熱區、偏好路線），為每一類指定成本與衰減距離。

3. 把語意代價疊加到幾何代價圖上，重新規劃，比較兩條路徑的長度、以及各自穿越語意區域的距離，重現圖 32-3 的三個數字。

4. 敏感度實驗：把人流熱區的成本從 0 逐步加大，找出路徑「跳」到另一條路線的臨界值，並解釋這個臨界值的意義。

**第二部分：動態物體剔除（全體）**

1. 在模擬環境中隨機放置數個「人」，執行一趟掃描建圖，重複五趟。

2. 以天真累積（任一趟觀測到即記為障礙）建圖，與真實地圖比對，計算假障礙的面積，重現圖 32-2(a)。

3. 改用多趟一致性投票（至少 k 趟觀測到才記為障礙），掃描 k 從 1 到 5，畫出「假障礙面積」與「真實牆面保留率」兩條曲線，找出最佳的 k。

4. 討論：若把趟數從 5 增加到 20，最佳的 k 會怎麼變？這對巡檢機器人（第 19 章）的地圖維護策略有什麼啟示？

**第三部分：語意標註投影（大學部）**

1. 以 RGB-D 相機或模擬資料，對一張影像執行現成的物件偵測模型，取得偵測框與深度。

2. 依公式二把偵測結果投影到地圖座標系，把標註點畫在地圖上。

3. 刻意把外參的旋轉改動 1、2、5 度，記錄標註點偏移了多少，量化外參誤差的影響，重現案例 32-1 的現象。

4. （進階）對同一個物件從五個不同視角偵測並投影，計算標註點的離散程度，作為整條傳播鏈品質的綜合指標。

## 討論問題

1. 語意路徑比幾何路徑多走了多少？這個代價換到了什麼？若多走的距離是 50% 而不是 2%，你還會選語意路徑嗎？
2. 多趟一致性投票的最佳 k 值是多少？k 太小與太大各會發生什麼？
3. 外參旋轉差 2 度時，十公尺外的標註點偏移多少？這個數字與第 19.5 節要求的公分級定位精度相比如何？
4. 哪些類別適合用固定清單的模型、哪些適合開放詞彙？請以「出錯的後果」為判準分類。
5. （大學部）若要為校園導覽機器人設計語意地圖，你會放入哪四個語意層？每一層的成本與衰減距離怎麼設？

## 延伸挑戰（大學部）

其一：實作一個「語意圖層編輯器」——讓使用者在地圖上畫出區域、選擇類別、調整成本，並即時看到規劃路徑如何改變。這類工具是巡檢與服務機器人產品的標準配備。其二：把動態物體剔除從「投票」升級為「分層地圖」：維護永久層與暫態層兩張地圖，定位只用永久層、避障兩層都用，並比較它與單純投票法在定位成功率上的差異（可沿用第 19.6 節的評估方法）。

# 延伸閱讀

106. N. Yokoyama et al., "VLFM: Vision-Language Frontier Maps for Zero-Shot Semantic Navigation," ICRA, 2024.（零樣本語意導航的代表作，第 24、32 章共同參考）

107. T. B. Martins, M. R. Oswald, and J. Civera, "Open-Vocabulary Online Semantic Mapping for SLAM," IEEE RA-L, 2025.（開放詞彙語意建圖，32.3 節的前沿方向）

108. D. Shah et al., "LM-Nav: Robotic Navigation with Large Pre-trained Models of Language, Vision, and Action," CoRL, 2022.（語言模型驅動導航的開創性工作）

109. Embodied AI with Foundation Models for Mobile Service Robots: A Systematic Review, arXiv, 2025.（具身智慧與基礎模型的完整綜述，適合作為研究起點）

110. Ultralytics YOLO 官方文件（docs.ultralytics.com）.（物件偵測模型的訓練、評估與部署，本章實作的主要工具）

111. awesome-and-novel-works-in-slam 開源清單（GitHub）.（語意 SLAM、三維高斯、導航、大型語言模型等新方向的持續更新整理）

112. NVIDIA Jetson 開發者文件與 TensorRT 最佳化指南.（邊緣 AI 部署的實務參考，第 33.8 節的技術基礎）

## 小結

## 小結

- 傳統 SLAM 知道形狀卻不知道意義；機器人智慧可分幾何、語意、任務三個層次，三者是疊加而非取代——語意層若無穩固的幾何層支撐，標籤只會貼歪。
- AI 視覺模型在 SLAM 中有五類分工：偵測、分割、深度、描述與場景理解；選型的關鍵限制是邊緣平台的運算與記憶體預算。
- 語意地圖可分四個層次：幾何層、物件層、區域層與建築層，由下而上建立；開放詞彙讓類別不必事先定義，場景圖則讓空間關係可被查詢與推理。
- 動態物體若直接寫入地圖會留下殘影。本章實驗顯示，天真累積產生 1.8 平方公尺的假障礙，而多趟一致性投票可將其降為零，同時保留 100% 的真實牆面。
- 動態物體的處理有三道防線——即時的實例分割、短期的幀間不一致、長期的多趟投票，三者處理不同時間尺度，實務上並用而非擇一。

- 語意進入導航是透過代價地圖。本章實驗顯示，語意路徑只多走 0.5 公尺（29.1 → 29.6 公尺），卻把 14 公尺的禁區穿越降為零；硬約束與軟成本應分開設計，而衰減距離比成本數值更影響行為。
- 語意標註的品質同時取決於辨識率與座標轉換鏈；外參差兩三度就足以造成數十公分的偏移，除錯時應先驗證幾何鏈再檢討模型。
- 國際研究沿三條路線發展：度量—語意 SLAM 最穩定、開放詞彙最靈活、神經場與可算繪表示重建品質最佳但成本最高。

## 承先啟後

本章把地圖從「形狀」推進到「意義」。圖 32-1 整理了語意地圖的四個層次、圖 32-2 說明了哪些東西不該進地圖、圖 32-3 則示範了語意如何改變決策。但所有這些都預設了一件事：你手上已經有一個夠好的 AI 模型。

模型從哪裡來？真實資料昂貴、標註耗時，而機器人在真實世界裡試錯的代價更高。下一章我們進入模擬：第 33 章介紹如何在模擬環境中大規模訓練機器人，再把學到的策略部署到真實的邊緣硬體上——這條「先模擬、再部署」的路線，已經是從國家級專案到產業量產的標準流程。

（本章到此結束，下一章進入模擬訓練與邊緣部署。）

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

## 第 33 章

## 模擬訓練與 Sim-to-Real

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程　/　高中 AI 機器人社團課程

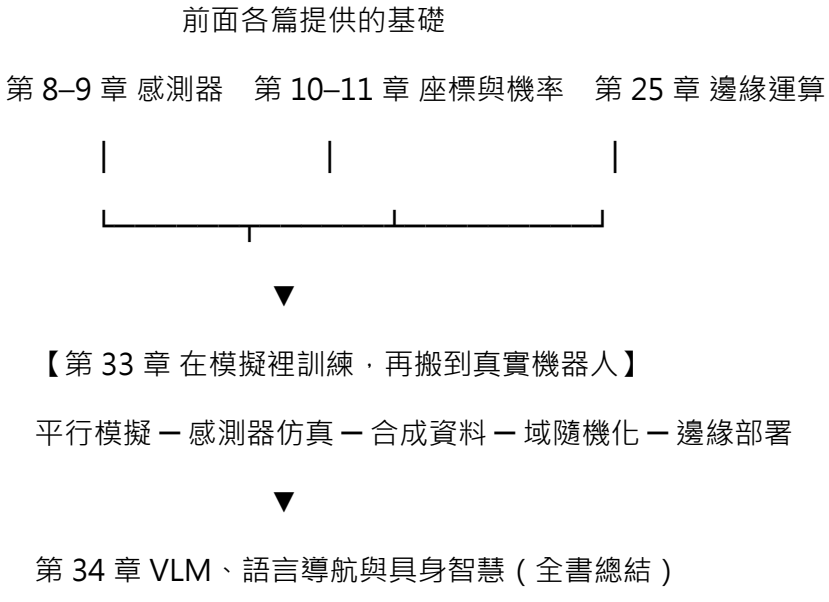
## 本章在全書的位置

第 32 章讓地圖有了意義——機器人知道「那是一扇門」。但知道之後呢？要讓機器人真的走過去、把門推開、在不平的地面上站穩，背後需要一套控制策略，而這套策略幾乎不可能在真實機器人上從零學會。原因很簡單：強化學習需要幾千萬次的嘗試，真實機器人摔不起那麼多次，也沒有那麼多時間。

本章要處理的就是這個矛盾：先在模擬裡練，再把學到的東西搬到真實機器人上。這條路線稱為 sim-to-real，它在 2020 年之後幾乎成為足式機器人與靈巧操作的標準做法。本章以輝達的 Isaac Sim 與 Isaac Lab 生態為主線，因為它是目前最完整、也最容易取得的一套工具；但本章要講的觀念——平行模擬為什麼有效、模擬與真實的落差從哪裡來、怎麼把落差補起來——不隨工具改變。

本章與前面各篇的關係很直接：第十、十一章的座標與機率是模擬器的語言，第八、九章的感測器模型決定模擬器要仿真什麼，第 25 章的邊緣運算則是本章 33.8 節「訓練完怎麼部署」的硬體基礎。可以說，本章是把全書的技術從「能跑」推到「能學」的一章。

### 圖 33-0　第 33 章的連結地圖




## 學習目標

本章從「為什麼要先模擬」出發，依序處理平行模擬的原理、感測器仿真、合成資料生成、sim-to-real 落差的來源與對策，最後落到邊緣裝置的部署。本章有三張以實驗算出來的圖，分別回答三個問題：平行度到底帶來什麼、模擬跟真實差在哪、以及域隨機化付出了什麼代價。讀完本章，同學應能：

1. 說明為什麼強化學習的機器人策略幾乎都必須先在模擬中訓練，並能估算所需的互動次數。
2. 說明 Isaac Sim 與 Isaac Lab 的分工，以及 GPU 原生模擬與傳統模擬器的差異。
3. 解釋平行環境數如何同時影響吞吐量、梯度雜訊與訓練穩定度，並知道三者各自的極限。
4. 說明機器人常用感測器在模擬中如何被仿真，以及哪些部分仿真得好、哪些仍有落差。
5. 說明合成資料生成與真實場景重建的做法，並能為自己的專題規劃資料來源。
6. 指出 sim-to-real 落差的主要來源，並能分辨「系統識別」與「域隨機化」兩種對策的適用時機。
7. 說明域隨機化的代價：穩健度與名目效能之間的取捨。
8. 規劃一條從雲端訓練到 Jetson 部署的完整路徑，並能估算量化後的記憶體需求。

## 核心概念

| 名詞 | 意義 | 關聯章節 |
| --- | --- | --- |
| Sim-to-Real | 在模擬中訓練策略，再部署到真實機器人 | 33.1、33.6 |
| 平行模擬 | 在同一張 GPU 上同時推進數千個環境 | 33.3 |

| | | |
|---|---|---|
| GPU 原生 | 狀態以張量形式常駐 GPU，不經 CPU 往返 | 33.3 |
| OpenUSD | 描述場景與機器人的開放場景格式 | 33.2 |
| 域隨機化 | 訓練時隨機抽樣環境參數，換取部署穩健度 | 33.6 |
| 系統識別 | 量測真實機器人並反推模擬參數 | 33.6、第 11 章 |
| 合成資料 | 由模擬器產生並自動標註的訓練資料 | 33.5、第 32 章 |
| 域落差 | 訓練分布與部署分布不一致造成的效能下降 | 33.5、33.6 |
| 量化 | 以較少位元表示模型參數以節省記憶體 | 33.8 |
| 蒸餾 | 把大模型的能力轉移到可在邊緣執行的小模型 | 33.8 |

**表 33-1　本章的核心名詞**

## 33.1　為什麼機器人要先在模擬裡學

先把問題的規模講清楚。

### 一、數字的問題

以強化學習訓練一個足式機器人的行走策略，典型的樣本需求是十億（$10^9$）等級的環境互動步數。假設控制頻率是 50 Hz，十億步相當於 2,000 萬秒，也就是約 231 天的連續運行。這還是在機器人不會壞、電池不用充、摔倒後會自己站起來的理想假設下。

真實世界還有三個模擬沒有的成本：硬體會壞（一次摔倒可能損壞減速機或外殼）、人力要

全程看顧（不能無人運行）、以及實驗不可重複（今天的地板濕度跟昨天不同，無法乾淨地比較兩個演算法）。這三點加起來，使得「在真機上做強化學習」在絕大多數情況下不是成本問題，而是根本不可行。

## 二、模擬提供了什麼

- 時間可以加速：模擬不受真實時間限制，一秒鐘可以推進數千個環境的數十步。
- 失敗沒有成本：摔倒只是把狀態重設，不會壞零件。
- 狀態完全可知：模擬器知道機器人的真實位姿與速度，這讓「非對稱訓練」成為可能——評價網路可以看到真實狀態，而策略網路只能看到感測器資料。
- 實驗可以重複：固定隨機種子，同一個實驗可以跑一百次，這是比較演算法的前提（第 31 章的評估方法）。
- 環境可以任意生成：地形、光線、物件擺放都能程式化地產生，這是真實世界做不到的多樣性。

## 三、模擬不提供什麼

必須同樣誠實地講清楚模擬的限制。模擬器解的是你寫進去的方程式，不是真實世界的物理。凡是沒有寫進方程式的東西——馬達的溫升、齒輪的背隙、線材的拉扯、腳墊磨損後摩擦係數的改變、韌體裡那些工程師為了好用而加上的補償——模擬器都不知道。這些「未建模的動力學」正是 sim-to-real 落差的主要來源，33.6 節會逐一處理。

> **為什麼 SLAM 比控制更晚用上模擬訓練**
>
> 本書前三十二章的 SLAM 演算法，大多不需要訓練——它們是推導出來的，不是學出來的。這是 SLAM 與控制策略的根本差異：SLAM 有明確的機率模型與最佳化目標（第 11、12 章），而「怎麼在碎石地上走穩」沒有人寫得出閉式解。
>
> 但這條界線正在模糊。深度學習的視覺里程計、學習式的迴路閉合描述子、以及第 32 章的語意分割，都需要大量資料；而 33.5 節的合成資料生成，正是讓 SLAM 也

開始受益於模擬的途徑。

## 33.2　Isaac Sim 與 Isaac Lab 是什麼

這兩個名字經常被混用，但分工其實很清楚。

### 一、兩層的分工

| | Isaac Sim | Isaac Lab |
|---|---|---|
| 定位 | 模擬器與場景編輯環境 | 機器人學習的框架 |
| 負責 | 物理求解、感測器算繪、場景與機器人匯入 | 任務定義、觀測與獎勵、訓練流程 |
| 使用者 | 要做場景、要接 ROS 2、要產生資料的人 | 要訓練策略的人 |
| 底層 | PhysX 物理 + RTX 算繪 + OpenUSD 場景 | 建構在 Isaac Sim 之上 |
| 類比 | 實驗室與儀器 | 實驗手冊與評分表 |

表 33-2　Isaac Sim 與 Isaac Lab 的分工

講得更白一點：Isaac Sim 是「世界」，Isaac Lab 是「課程」。你可以只用 Isaac Sim 做一個場景、放一台機器人、接上 ROS 2 來測試你的導航堆疊，完全不碰強化學習；也可以用 Isaac Lab 直接取用它內建的數十個訓練環境，不必自己從零搭場景。

### 二、三根支柱

- PhysX：物理求解器，處理剛體、關節、接觸與摩擦。對機器人而言最關鍵的是接觸求解的品質——腳掌踩地、夾爪抓物都靠它。
- RTX：光線追蹤算繪，負責產生相機影像。它同時也是光達與深度相機仿真的基礎（33.4 節）。

- OpenUSD：開放的場景描述格式，由皮克斯提出、現已成為跨工具的場景交換標準。它的價值在於「一份場景、多個工具共用」，也讓機器人模型（連桿、關節、致動器、感測器）能以標準結構描述。

### 三、2025 年的重要變化

2025 年 8 月 11 日的 SIGGRAPH，Isaac Sim 5.0 與 Isaac Lab 2.2 開放一般取用，同時把 Isaac Sim 的擴充套件開源到 GitHub（isaac-sim/IsaacSim 與 isaac-sim/IsaacLab）。這對教學與研究是一件大事：原本只能當成黑盒子使用的模擬器，現在可以讀原始碼、可以改、也可以提交修改。

但有一點要說清楚，免得同學踩空：底層的 Omniverse Kit 並未全部開源，目前也不接受外部提交。也就是說，你可以修改 Isaac Sim 的擴充層，但不能修改最底層的核心。對課程專題而言這通常不構成限制，但在規劃長期研究時值得先知道。

同一版本也帶進了幾項與機器人直接相關的功能：OpenUSD 的機器人結構描述與匯入精靈，讓 URDF 轉換更不容易出錯；關節摩擦模型，讓減速機的真實阻力可以被寫進模擬；以及 ROS 2 Jazzy 的支援與標準化的模擬介面，讓同一套 ROS 2 節點可以在不同模擬器之間切換（第八篇）。

**Isaac Lab 2.2 帶進來的東西**

可一鍵啟動的雲端訓練環境（Brev launchable），不必自己裝 CUDA 與驅動程式。

GR00T-Mimic 的雙臂訓練環境，示範從人類示範資料學習操作。

Omniverse Fabric 的整合，降低大規模場景的記憶體與同步開銷。

張量化的吸盤夾爪模型，讓真空吸取的任務能平行模擬。

開源的策略評估框架，讓訓練完的策略有一套標準的比較方式（呼應第 31 章）。

## 33.3　GPU 原生的平行模擬

這是整個生態的技術核心，也是本章第一張實驗圖要回答的問題。

## 一、「GPU 原生」的意思

傳統模擬器（如 Gazebo）的流程是：CPU 算物理 → 把狀態複製給學習程式 → 學習程式算出動作 → 再複製回模擬器。每一步都要在 CPU 與 GPU 之間搬資料，當環境數量一多，搬資料的時間就會超過算物理的時間。

GPU 原生的做法是：把所有環境的狀態以張量形式直接存在 GPU 記憶體裡，物理求解、觀測計算、獎勵計算、神經網路推論全部在 GPU 上完成，整個訓練迴圈不需要把資料搬回 CPU。這個差別在環境數少的時候看不出來，但在數千個環境時就是數量級的差距。

## 二、平行度到底帶來什麼

「多開幾個環境就會比較快」這句話人人會說，但快在哪裡、快多少、以及什麼時候就不再變快，是三個不同的問題。以下用一個可以完整重現的實驗把三件事分開來看。

實驗設定：一個單自由度的關節追蹤任務。關節有轉動慣量、黏滯阻尼、庫倫摩擦與致動延遲四個參數；控制器是一組含摩擦前饋的 PD 增益，也就是要「學」的三個參數。訓練方式是最單純的隨機爬山法，每次迭代用 N 個隨機抽樣的環境來評估候選參數。這個任務比真正的足式運動簡單得多，但平行度的三個效果在這裡已經完全呈現。

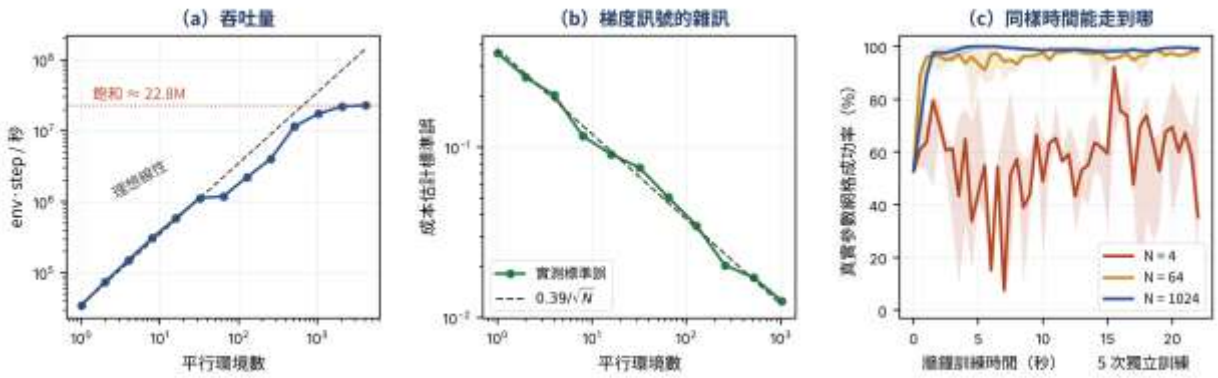



**圖 33-1　平行環境數的三個效果：吞吐量、梯度雜訊與訓練穩定度**

先看（a）。橫軸是同時推進的環境數 N，縱軸是每秒能推進的環境步數，兩軸都是對數刻度。虛線是「理想線性」——如果平行度完全免費，加倍環境數就該加倍吞吐量。實測結

果在 N 小於約 32 時幾乎貼著理想線走：從 N=1 的每秒 3.5 萬步，一路漲到 N=32 的每秒 113 萬步，三十二倍的環境換來三十二倍的吞吐量。這段之所以「免費」，是因為單一環境根本填不滿 GPU，多開的環境只是把原本閒置的運算單元用起來。

N 超過 32 之後曲線開始偏離虛線，到 N=4096 時吞吐量停在每秒約 2,285 萬步。與 N=1 相比是 654 倍，但與理想線性的 1.43 億步相比只有六分之一。飽和的原因是 GPU 的運算單元已經被填滿，再加環境只能排隊。值得注意的是：從 N=1 到 N=4096，每一批次的牆鐘時間只從 0.023 秒漲到 0.143 秒，也就是說環境數增加 4,096 倍，等待時間只增加 6.3 倍。這正是平行模擬最直觀的價值——幾乎白拿的樣本。

（b）回答第二個問題：這些樣本拿來做什麼。強化學習每次更新都要估計「這組參數有多好」，而這個估計是從有限個環境的平均算出來的，必然帶有雜訊。縱軸是這個估計的標準誤，實測點幾乎完美落在 1/√N 的虛線上——把每個點的標準誤乘上 √N，十一個點的結果都落在 0.32 至 0.43 之間。從 N=1 到 N=1024，標準誤由 0.357 降到 0.0126，正好接近 √1024 = 32 倍。這就是公式一。

（c）把前兩件事合起來看：給定相同的牆鐘時間預算（22 秒），不同平行度能走到哪裡。為了避免單次訓練的運氣成分，每個設定跑五次獨立訓練，圖中粗線是中位數、淡色帶是四分位距。N=4 的中位數只到 35.3%，而且四分位距從 15.8% 橫跨到 85.6%——換句話說，你不知道這次訓練會成功還是失敗。N=64 的中位數是 98.4%，N=1024 是 99.2%，且四分位距只有 98.3% 到 99.4%。

（c）要傳達的重點不是「平行度高比較快」，而是「平行度高比較可靠」。N=4 不是走得慢，是走得亂——梯度雜訊太大，每一步都可能走錯方向，最後落在哪裡幾乎是運氣。這個現象在真實的強化學習訓練中更嚴重，也是為什麼 Isaac Lab 這類框架把「能同時開幾千個環境」當作核心指標。

**一個誠實的但書**

上面三張圖是在一個單自由度的玩具任務上算出來的，不是足式機器人。真實任務的絕對數字會完全不同：飽和點取決於你的 GPU 與環境的複雜度，所需的環境數取決於動作維度與獎勵的稀疏程度。

但三個定性結論是通用的：吞吐量先線性成長後飽和、梯度雜訊依 1/√N 下降、以及平行度不足時訓練結果的變異會大到無法比較。同學在自己的任務上做同樣的三張圖，是規劃運算資源最有效的方法。

## 三、真實規模的數字

作為對照，看一下工業級的數字。Isaac Lab 的技術報告（2025 年 11 月）在 8 張 GPU、每張 16,384 個環境的配置下，靈巧手抓取任務的教師策略達到每秒超過 90 萬步，Franka 機械臂開櫃任務則超過每秒 160 萬步，並且在多機多卡的情況下接近線性擴展。

另一個對同學更實際的數字是：在開櫃任務上，一張消費級的 GeForce 5090 的吞吐量約等於兩張專業卡；在靈巧手任務上甚至快約 25%，因為該任務的瓶頸在 CPU 而非 GPU。這意味著一台配備單張高階顯示卡的桌機，已經足以支撐相當認真的機器人學習研究——這在五年前是不可想像的。

## 四、公式一：平行度與估計品質

設單一環境的回報估計為隨機變數，其標準差為 σ；以 N 個獨立環境的平均作為估計時，估計量的標準誤為：

$$\mathrm{SE}(N) = \sigma \,/\, \sqrt{N}$$

要把估計的雜訊降為原來的十分之一，需要一百倍的環境數。這個「平方根的懲罰」解釋了為什麼平行度要以千為單位計算：從 64 個環境提升到 4,096 個環境，雜訊只降為八分之一。它同時也解釋了（a）圖的飽和為什麼可以接受——即使吞吐量只剩理想值的六分之一，多出來的環境仍在降低雜訊。

## 33.4　感測器模擬：從 RTX 光達到觸覺

對本書的讀者而言，這一節大概是最切身的：前面三十二章用到的每一種感測器，在模擬裡長什麼樣子？

### 一、感測器的四個家族

| 家族 | 模擬中的實作 | 對應章節 | 仿真品質 |
|---|---|---|---|
| 本體感測 | 直接由物理狀態讀出 | 第 7、10 章 | 高 |
| 相機 | RTX 光線追蹤算繪 | 第 9、20–23 章 | 中高 |
| 測距 | Warp 光線投射 | 第 8、17–18 章 | 中 |
| 接觸與觸覺 | 接觸求解器 + 視觸覺模型 | 第 19 章 | 中低 |

**表 33-3　感測器在模擬中的四個家族**

### 二、逐一來看

本體感測最容易，也仿真得最好。IMU、關節編碼器、接觸偵測、座標轉換（frame transformer）都可以直接從物理狀態讀出來，再依第 7 章的誤差模型加上偏差與雜訊即可。但要注意：模擬器給的是「乾淨的真值加上你寫的雜訊」，而真實 IMU 的誤差有溫度漂移與振動耦合，不是獨立同分布的高斯雜訊。

相機是 RTX 算繪的主場。除了 RGB，還能免費拿到深度、語意分割、實例分割、表面法向量與運動向量——這些在真實世界需要額外標註的資料，在模擬裡是算繪管線的副產品。這正是 33.5 節合成資料的技術基礎。鏡頭模型同時支援針孔與魚眼，後者對第 9 章提過的廣角視覺導航很重要。

測距感測器（光達、深度相機的立體匹配）用的是 Warp 光線投射：從感測器原點依指定的掃描樣式射出光線，算出與場景的交點。這能正確處理遮擋與幾何，也能仿真各家光達不同

的線束排列；足式機器人常用的「地形高度掃描」也是用同一套機制實作。但有兩件事它不容易做對：第 8 章談過的鏡面反射與穿透（玻璃、水面、高反射標牌），以及第 18 章談過的運動畸變——後者需要模擬器在掃描期間持續更新感測器位姿，而不是在一個瞬間射出所有光線。

接觸與觸覺是目前落差最大的一類。接觸力本身由 PhysX 求解，但真實的接觸牽涉到材料的微觀變形、表面粗糙度與黏滯效應，這些在剛體求解器裡只能用摩擦係數近似。視觸覺感測器（以相機拍攝彈性膜的變形來推算接觸力）的模擬已經可用，但如果同學的專題要做精細的插拔或裝配，必須預期這一類任務的 sim-to-real 落差最大。

> **一個常被忽略的細節：感測器的時間**
>
> 真實感測器不是同時觸發的。光達轉一圈要 100 毫秒、相機曝光要幾毫秒、IMU 是 200 Hz、輪速編碼器是 50 Hz——這些在第 7 章與第 18 章都討論過。
>
> 模擬器預設常把所有感測器當成同一瞬間取樣。如果你的演算法對時間同步敏感（例如緊耦合的光達慣性里程計），那麼「模擬裡好好的、真機上就飄」很可能不是演算法的問題，而是模擬沒有把時間差建進去。
>
> 對策是在模擬中明確設定各感測器的更新率與相位，並刻意加入隨機的時間抖動。

## 33.5　合成資料生成與真實場景重建

第 32 章談過自己蒐集資料的必要性，也談過標註的成本。這一節提供另一條路：讓模擬器把資料跟標註一起生出來。

### 一、合成資料的誘惑與陷阱

誘惑很明顯：在模擬裡，每一個像素屬於哪個物件是已知的，因此分割標註、邊界框、深度圖、六自由度位姿全部免費且零誤差。想要一萬張不同光線、不同視角、不同物件擺放的訓練影像，只是一段腳本與幾個小時的運算。

陷阱也同樣明顯：模擬出來的影像和真實影像不一樣。材質的反射、感測器的雜訊與壓縮痕跡、鏡頭的髒污與色偏、真實場景裡那些沒有人會刻意建模的雜物——這些差異合稱域落差（domain gap）。一個只在合成資料上訓練的模型，放到真實影像上往往表現得很差。

## 二、三條縮小域落差的路

1. 真實場景重建：不要自己建模，直接把真實場景掃進來。目前的做法是以實拍影片重建出可互動的三維場景（例如 NuRec 函式庫與開源的 3DGUT 方法），這樣背景與材質本來就是真的，只有動態物件是合成的。
2. 視覺域轉換：用生成模型把模擬影像「翻譯」成看起來像真實拍攝的影像。Isaac Sim 的資料寫出器已能直接串接這類模型（如 Cosmos Transfer）。
3. 視覺域隨機化：反過來做——不去模仿真實，而是把模擬的材質、光線、背景隨機到極端誇張的程度，逼模型只能依賴物件的形狀而非外觀。這條路最簡單，也最常用。

## 三、合成與真實的混合策略

實務上最有效的通常不是二選一，而是混合：以大量合成資料做預訓練、再以少量真實資料做微調。這呼應第 32 章提過的一個重點——從預訓練權重微調，所需的真實資料量遠低於從零訓練。一個對校園專題務實的配方是：合成資料數千張用於預訓練，真實資料每類 200 至 500 張用於微調，並且測試集一律只用真實資料。

**圖 33-2　資料從蒐集到部署的迭代流程**

① 定義類別與蒐集規劃

↓

② 取得資料（真實拍攝 / 模擬生成 / 兩者混合）

↓

③ 標註（真實資料要訂規範並抽查；合成資料自動產生）

↓

④ 分割資料集（依地點而非隨機分割；測試集只用真實資料）

↓

⑤ 訓練（合成資料預訓練 → 真實資料微調）

↓

⑥ 評估（在真實測試集上計算精度；並在真實場域實測）

↓

⑦ 錯誤分析（哪些類別最常錯？什麼情況下錯？）

↓

⑧ 針對弱點補充資料 → 回到 ⑤ 迭代

↓

⑨ 部署最佳化（量化、加速）→ 邊緣裝置（33.8 節）

請特別注意第七與第八步：錯誤分析與針對性補資料，往往比盲目增加訓練輪數有效得多。合成資料在這一步特別有價值——當你發現模型在「逆光的樓梯」上最常錯，你可以在模擬裡專門生成一千張逆光樓梯，這在真實世界需要等天氣。

**兩個值得知道的工具**

MobilityGen：自動生成移動機器人的軌跡與對應的感測資料，適合用來產生 SLAM 與導航的訓練或評估資料集。

以視覺語言模型自動描述場景：模擬器算繪出影像後，由 VLM 產生文字描述作為標註——這把第 32 章的語意標註與本章的合成資料接了起來，也是第 34 章語言導航

資料的主要來源之一。

## 33.6　域隨機化與 sim-to-real 落差

這是本章最重要的一節。前面五節都在講「怎麼在模擬裡訓練」，這一節講「訓練完之後，為什麼搬到真機上會失敗，以及怎麼辦」。

### 一、落差從哪裡來

| 落差來源 | 具體內容 | 本書關聯 |
|---|---|---|
| 致動器動力學 | 韌體補償造成的虛擬慣量、電流環頻寬、扭矩飽和 | 本節 |
| 連桿慣量誤差 | CAD 模型低估實際慣量（電線、外殼、感測器） | 本節 |
| 摩擦與阻尼 | 減速機的庫倫摩擦與隨狀態改變的阻尼 | 本節 |
| 時間延遲 | 指令傳輸、濾波與控制週期造成的總延遲 | 第 25 章 |
| 關節偏差 | 組裝與校正誤差造成的零位偏移 | 第 10 章 |
| 感測器特性 | 雜訊模型、取樣時間、視角與遮擋 | 第 7–9 章 |
| 環境差異 | 地面摩擦、地形、負載、溫度與老化 | 第 19 章 |

**表 33-4　sim-to-real 落差的七個來源**

### 二、對策一：系統識別

第一條路是把模擬改對：量測真實機器人的反應，反推模擬應該用的參數。標準做法是把機

器人吊在空中（讓腳不接觸地面以排除接觸力的干擾），對每個關節送出掃頻訊號（頻率由低到高連續變化的正弦波），錄下編碼器的回應，再以最佳化反推參數。近期的研究指出，每台機器人只需要約 20 秒的空中掃頻資料就足夠，待識別的參數約為每關節四個（電樞慣量、黏滯阻尼、庫倫摩擦、關節偏差）加上一個全域的指令延遲。

以下用一個可完整重現的實驗示範這個流程。「真實機器人」是一個含庫倫摩擦與 7.5 毫秒指令延遲的關節模型；「URDF 模型」則只有連桿慣量，沒有阻尼、沒有摩擦、沒有延遲——這正是從 CAD 直接匯出的 URDF 的典型狀態。

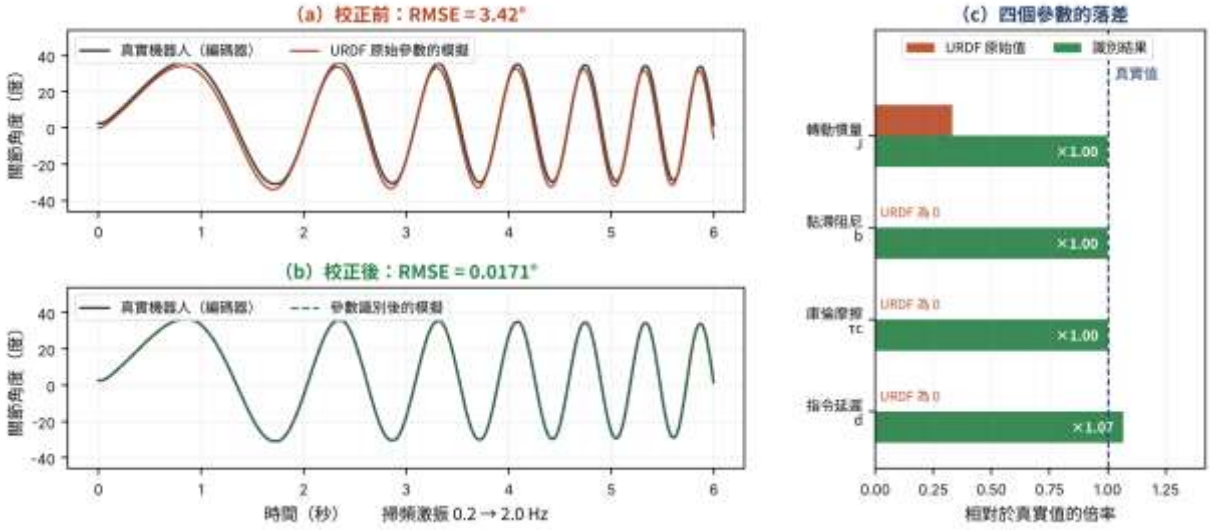



**圖 33-3　系統識別：從 URDF 的猜測到擬合後的模擬**

（a）是校正前。灰線是「真實機器人」編碼器錄到的角度，紅線是用 URDF 參數模擬出來的同一段動作。兩條線的形狀相似，但紅線明顯超前——這是延遲沒有被建模的後果；振幅也略大，因為模擬裡沒有摩擦與阻尼在消耗能量。整段的均方根誤差是 3.42 度。對一個行走策略而言，每個關節三度多的系統性偏差足以讓步態完全走樣。

（b）是校正後。綠色虛線與灰線完全重合，均方根誤差降到 0.0171 度，是校正前的兩百分之一。要強調的是：這不是把模型改複雜了，而是把同一個模型的參數調對了——方程式的形式從頭到尾沒有改變。

（c）說明參數到底差多少。紅柱是 URDF 的原始值相對於真實值的倍率，綠柱是識別結果。

轉動慣量上，URDF 只有真實值的 0.33 倍——少掉的三分之二來自韌體補償造成的虛擬慣量、以及 CAD 沒算進去的線材與外殼。黏滯阻尼、庫倫摩擦、指令延遲這三項，URDF 裡根本就是零。識別之後，慣量、阻尼、摩擦三項都還原到真實值的 1.00 倍；延遲因為是以 2 毫秒為間隔做格點搜尋，落在 8.0 毫秒（真值 7.5 毫秒），誤差一個格點。

> **一個實作上的坑：延遲不能用梯度法擬合**
>
> 指令延遲在離散時間的模擬裡是整數個時間步，因此殘差對延遲是階梯函數——在每個台階內部的梯度是零。把延遲丟進 least_squares 這類梯度法裡，它會停在起始值動都不動。
>
> 正確的做法是外層對延遲做整數格點搜尋，內層再以梯度法擬合連續參數。本節的實驗第一次做錯時，擬合出來的延遲是 0.0001 秒（真值 0.0075），而且慣量被高估了 2.3 倍去「補」那個沒被識別出來的延遲——軌跡看起來吻合，參數卻全錯。這是系統識別最典型的陷阱：誤差小不代表參數對，不同參數之間可能互相補償。

## 三、對策二：域隨機化

第二條路反過來：不去追求模擬正確，而是讓策略對參數不確定本身免疫。做法是在訓練時，每個環境都從一個分布裡隨機抽取動力學參數，逼策略學會一套「對任何合理參數都能用」的行為。

問題是：隨機的範圍該多寬？以下實驗比較三種設定。同一個關節追蹤任務，訓練三組控制器：（a）只在名目參數上訓練；（b）在名目值 ±10% 的範圍內隨機；（c）在慣量 0.5 至 2.0 倍、摩擦 0.3 至 2.0 倍、延遲 0 至 28 毫秒的寬範圍內隨機。然後把三組控制器放到同一張「真實參數」網格上評估。

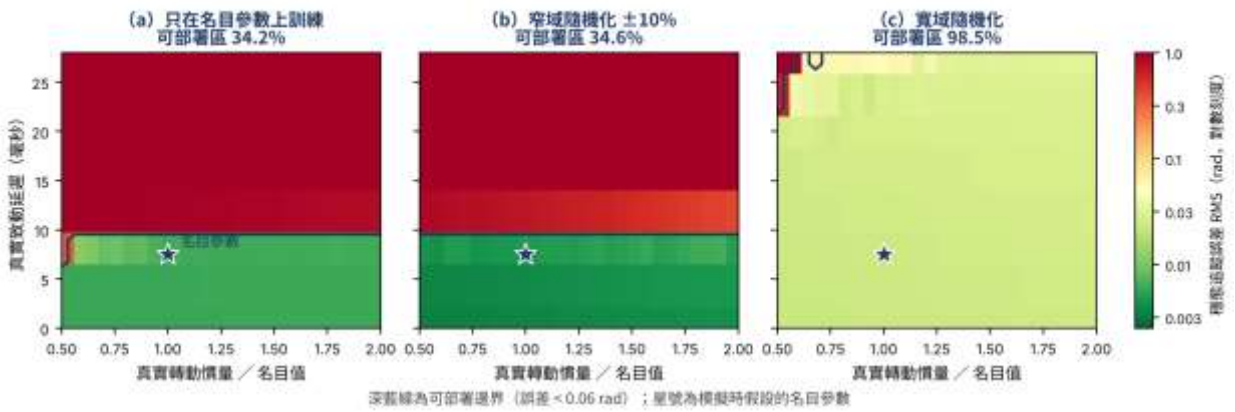



**圖 33-4　域隨機化的範圍對部署成功率的影響**

先看最明顯的結論：可部署區（深藍線包住的範圍）從（a）的 34.2%、（b）的 34.6%，跳到（c）的 98.5%。窄域隨機化幾乎沒有幫助——±10% 的範圍太窄，學到的策略和只在名目點訓練的策略差不多。這一點值得同學記住：域隨機化不是「加一點隨機就好」，範圍必須寬到涵蓋你對真實參數的實際不確定性。

第二個結論藏在顏色裡。（a）與（b）在可部署區內是深綠色，代表追蹤誤差極小；（c）全圖則是淡綠色，代表「到處都能用，但到處都不夠好」。量化來看：在名目參數這一點上，只在名目參數訓練的控制器誤差是 0.0061 弧度，寬域隨機化的控制器是 0.0245 弧度——四倍。這就是域隨機化的代價：穩健度不是免費的，它用名目效能去換。

第三個結論是失敗邊界的形狀。（a）與（b）的邊界幾乎是一條水平線，落在約 9.5 毫秒的延遲上，而且幾乎與轉動慣量無關。換句話說，在這個任務裡，慣量估錯兩倍不會讓策略失效，但延遲多幾毫秒就會。這個訊息對專題規劃很實用：與其花力氣去秤每個連桿的重量，不如先把控制迴路的延遲量出來。

## 四、兩條路怎麼選

| | 系統識別 | 域隨機化 |
|---|---|---|
| 做法 | 量測真機，把模擬改對 | 不改模擬，把策略訓得耐操 |
| 需要 | 能量測的真機與激振設備 | 對參數不確定範圍的估計 |

| | | |
|---|---|---|
| 優點 | 名目效能高，可解釋 | 不需要真機即可訓練 |
| 缺點 | 參數可能互相補償；會隨磨損失效 | 名目效能下降；範圍難訂 |
| 適合 | 已有真機、要榨出最佳效能 | 真機尚未到位、或機群個體差異大 |

**表 33-5　系統識別與域隨機化的比較**

實務上兩者通常並用：先以系統識別把模擬的中心點校正到正確位置，再在這個正確的中心點周圍做適度的隨機化，涵蓋磨損、溫度與個體差異。這樣既不會像純域隨機化那樣犧牲名目效能，也不會像純系統識別那樣在零件磨損後失效。

## 33.7　怎麼上手、怎麼參加社群

這一節很實際：如果同學看完前面六節想動手，下一步該做什麼。

### 一、三條入門路徑

| 路徑 | 需要什麼 | 適合誰 |
|---|---|---|
| 雲端一鍵啟動 | 瀏覽器與帳號 | 沒有獨立顯示卡的同學 |
| 容器部署 | 具 CUDA 的 Linux 主機 + 容器環境 | 實驗室有工作站 |
| 原始碼安裝 | 同上，另需自行處理相依套件 | 要修改模擬器的人 |

**表 33-6　三條入門路徑**

對大多數同學，建議從雲端一鍵啟動開始。它把驅動程式、CUDA 版本、相依套件這些最容易卡住的環節都處理好了，讓你在第一天就能看到機器人在模擬裡動起來，而不是花三天在裝環境。等到流程熟悉、確定要長期投入，再轉到本地部署。

本地部署時有一個細節值得先知道：模擬器的畫面是透過 WebRTC 串流出來的，預設使用

49100 與 47998 這兩個連接埠。如果你在學校的網路環境下連不上畫面，十之八九是防火牆擋了這兩個埠，而不是軟體裝壞了。

## 二、一條建議的學習順序

1. 先跑通內建範例：不要改任何東西，確認你的環境能訓練出一個會走路的機器人。這一步的目的只是建立基準線。
2. 換機器人不換任務：把範例裡的機器人換成另一個內建型號，觀察同一套獎勵在不同形態上的表現差異。框架內建約十一種足式機器人形態，夠你比較很久。
3. 改獎勵：這是最能學到東西的一步。把速度追蹤的權重調高、把能耗懲罰調高，觀察步態怎麼改變。你會很快體會到獎勵設計是一門手藝。
4. 匯入自己的機器人：用匯入精靈把自己的 URDF 轉成模擬可用的格式，這時會開始遇到關節限制、慣量、摩擦設定的真實問題。
5. 接上 ROS 2：讓模擬器當成你導航堆疊的測試台（第八篇）。這一步不需要強化學習，但對做 SLAM 的同學價值最高。
6. 做系統識別與 sim-to-real：有真機之後，按 33.6 節的流程走一遍。

## 三、怎麼參加社群

機器人模擬是一個變動很快的領域，文件常常落後於程式碼。因此「知道去哪裡問」跟「知道怎麼做」幾乎一樣重要。

- 開發者論壇：官方的機器人開發者論壇是提問最有效的地方，工程師會直接回覆。提問時附上版本號、顯示卡型號與完整錯誤訊息，得到有用回覆的機率會高很多。
- GitHub：Isaac Sim 與 Isaac Lab 的程式碼在 GitHub 上公開。遇到行為與文件不符時，直接讀原始碼往往比搜尋快。也可以提交問題回報與修改——這是最實質的參與方式。
- 官方課程與電子報：免費的機器人基礎課程適合完全沒有背景的同學；機器人電子

報則是追蹤版本變動最省力的方式。

- 論文：技術報告與論文裡的數字（如 33.3 節引用的吞吐量）是規劃硬體採購最可靠的依據，比行銷資料準確得多。

> **給專題同學的三個提醒**
>
> 版本要鎖定。這個生態更新很快，不同版本之間的 API 會變。專題開始時就把版本號寫進文件，並且整個團隊用同一版。
>
> 不要在模擬裡追求完美。模擬的價值在於快速迭代，不在於像素級的真實。花三週把材質調得很漂亮，對策略的 sim-to-real 成功率幫助通常是零。
>
> 先想清楚要量什麼。訓練出一個會走路的機器人只是起點，專題的價值在於你量了什麼、比較了什麼（第 31 章）。

## 33.8　邊緣部署：Jetson、量化與蒸餾

訓練在雲端或工作站，執行卻在機器人身上。這一節處理這條「訓練完就部署」的一條龍最後一段：怎麼把模型塞進一台靠電池供電、巴掌大的運算模組裡。

### 一、問題的規模

視覺語言模型的運算需求通常使它們只能在雲端執行，而雲端執行又帶來第 25.1 節討論過的所有問題：依賴網路、延遲、頻寬與可靠度。對一台在地下停車場或廠房裡工作的機器人而言，「網路斷了就停住」不是可以接受的設計。

### 二、硬體的選擇

| 模組 | AI 效能 | 記憶體與頻寬 | 功耗 | 典型用途 |
|---|---|---|---|---|
| AGX Thor T5000 | 2,070 TFLOPS（FP4 稀疏） | 128 GB，273 GB/s | 40–130 W | 機上跑大型模型 |
| AGX Orin 64GB | 275 TOPS | 64 GB，204.8 | 15–60 W | SLAM + 多模型 |

| | （INT8） | GB/s | | 並行 |
|---|---|---|---|---|
| Orin NX 16GB | 100 / 157 TOPS | 16 GB，102.4 GB/s | 10–40 W | 巡檢機器人主流 |
| Orin Nano 8GB | 40 / 67 TOPS | 8 GB，68 GB/s | 7–25 W | 教學與專題 |

**表 33-7　邊緣運算模組的比較**

看這張表時有兩個陷阱要避開。第一，Thor 標示的是 FP4 稀疏的 TFLOPS，Orin 系列標示的是 INT8 的 TOPS，兩者不能直接相除比較——數字差了七倍不代表效能差七倍。第二，也是更重要的：對同時跑 SLAM 與 AI 模型的機器人而言，決定能不能跑的往往不是算力那一欄，而是記憶體那一欄。案例 33-2 會說明這件事。

## 三、三大關鍵技術

量化：以較少的位元表示模型參數。目前邊緣端的視覺語言模型與視覺語言動作模型，預設採用 4 位元權重搭配 16 位元啟動值的組態；4 位元權重加 4 位元啟動值的格式能解鎖新一代硬體平台，是支援時的產品級升級選擇；8 位元權重加 8 位元啟動值則仍是成熟視覺與分割模型較安全的備選。

蒸餾：把大模型的能力轉移到小模型。有研究提出蒸餾策略，把大型前沿模型的複雜空間語意推理轉移到輕量的 40 億參數本地模型上，以在嵌入式 GPU 裝置上執行；實驗顯示，以約 500 條前沿模型的推理軌跡微調小模型，即能有效賦予導航能力，成功率達 34.5%，相較大型雲端模型的 39.7% 已相當接近。

記憶體管理：這是常被忽略卻極關鍵的一環。有研究在 Jetson Orin Nano 上同時執行 SLAM 與 VLM，發現以幾何為基礎的地圖管理策略會發生記憶體不足的失敗與執行停滯；改採語意分區的關鍵幀載入卸載機制後，不僅消除了這兩個問題、維持了定位穩定性，還使吞吐量提升並降低了 21.7% 的延遲。

## 四、公式二：量化的記憶體估算

模型量化後的記憶體需求約為：

記憶體 ≈ 參數量 × 每參數位元數 / 8　（位元組）

以 40 億參數的模型為例：16 位元精度需約 8 GB、8 位元約 4 GB、4 位元約 2 GB。這解釋了為何 4 位元量化能讓大模型擠進嵌入式平台——差距是四倍。對照表 33-7：一個 4 位元量化的 40 億參數模型約佔 2 GB，在 8 GB 的 Orin Nano 上還要與作業系統、SLAM 的地圖、影像緩衝區共用剩下的 6 GB，已經相當吃緊。

但要注意：實際記憶體用量還包含啟動值、中間結果與框架開銷，通常是上述估算的 1.2 至 2 倍。規劃時務必預留餘裕。

## 五、三種對策

1. 限制地圖規模：依語意分區只載入當前需要的區域。這呼應第 23 章 Atlas 的多地圖與第 15 章 Cartographer 的子地圖思想——把大問題切成可管理的小塊。
2. 選用量化後的小模型：不必為每個任務各裝一個大模型。一個量化過的通用模型，往往比三個未量化的專用模型更務實。
3. 錯開執行：AI 推論不需要每幀都跑。每秒一到兩次通常已足夠支撐語意層的決策，而 SLAM 的前端仍維持原本的頻率。

**部署前的檢查清單**

在目標硬體上量過記憶體峰值了嗎？（不是平均值，是峰值）

把 SLAM 與 AI 模型同時跑過了嗎？分開跑都正常不代表合起來正常。

量過端到端延遲了嗎？從影像進來到決策出去，不只是模型推論那一段。

斷網的情況測過了嗎？如果有任何雲端相依，這是必測項目。

長時間運行測過了嗎？記憶體洩漏與地圖成長要跑數小時才看得出來（第 19.6 節）。

# 案例

## 案例 33-1　沒有校正的模擬，訓練出來的策略會直接摔倒

這是一個把 33.6 節的系統識別講得最清楚的真實實驗。

研究團隊對多款足式機器人做了同一件事：把機器人吊在空中，對關節送出 20 到 60 秒的掃頻訊號，只用編碼器的資料，以演化式最佳化在 4,096 個平行環境中擬合動力學參數。待識別的參數包含每個關節的電樞慣量、黏滯阻尼、庫倫摩擦與零位偏差，加上一個全域的指令延遲，總共約四十九個。

關鍵的對照實驗是這樣的：用未經校正的 URDF 參數訓練出來的策略，放到真實的四足機器人上，連向前行走都失敗，機器人必須由吊車接住；而以識別後的參數訓練出來的策略則能正常行走。同一套演算法、同一組獎勵、同一個訓練流程，差別只在模擬參數對不對。

團隊同時量化了落差的來源，每一項都呼應表 33-4：韌體的補償行為造成約 $8.1\times10^{-3}$ kg·m² 的「虛擬慣量」，CAD 低估了大腿連桿的慣量，全域指令延遲約 7.5 毫秒，而關節零位偏差最大可達約 5 度。這些數字沒有一項能從 CAD 圖面讀出來。

校正帶來的不只是「能走」。同一套流程在不同機型上量到的運輸成本（CoT）從 1.86 降到 1.27，約三成的能耗改善；續航方面，一款四足機器人單次充電跑了 4.12 公里，另一款達 6.10 公里；一台 4.2 公斤的小型四足也能在 46 秒內爬完二十階、每階 18 公分的樓梯。

對同學的啟示有兩點。其一，約 20 秒的資料就能完成識別——這是一個學期專題完全負擔得起的實驗。其二，也是更重要的一點：團隊在識別之後並未使用動力學的域隨機化。這與 33.6 節圖 33-4 的結論一致——當你能把模擬校正到足夠準，就不必用名目效能去換穩健度。域隨機化是在你無法校正時的保險，不是預設選項。

## 案例 33-2　記憶體才是真正的瓶頸

這是許多同學在專題後期才會撞上的問題，提前認識能省下大量挫折。

情境是這樣的：同學在 Jetson 上成功跑起了 SLAM，也成功跑起了物件偵測模型，但一旦兩者同時執行，系統就變得極度緩慢，甚至直接崩潰。檢查處理器與圖形處理器的使用率，卻發現都沒有滿載——這讓人困惑。

真正的瓶頸是記憶體。相關研究對這個現象有清楚的量化：在 Jetson Orin Nano 上同時執行 SLAM 與視覺語言模型時，以幾何為基礎的地圖管理策略會遭遇記憶體不足的失敗與執行停滯；而改採語意分區的關鍵幀載入卸載機制後，不僅消除了這兩個問題並維持定位穩定性，還提升了吞吐量並降低了 21.7% 的延遲。

問題的根源是：SLAM 的地圖會隨探索持續成長（第 13.5 節的記憶體估算、第 14 章的粒子數乘地圖大小），而 AI 模型本身就佔用大量記憶體。兩者相爭，系統必然崩潰。

解法的巧妙之處在於「語意分區」：機器人不需要隨時記得整棟樓的細節，只需載入當前所在區域的地圖。這個思想與第 23 章 Atlas 的多地圖、第 15 章 Cartographer 的子地圖一脈相承——把大問題切成可管理的小塊。

對專題的具體建議：在規劃階段就用公式二估算記憶體需求，並優先考慮限制地圖規模的機制。等到系統崩潰才處理，往往需要大幅重構。

## 案例 33-3　把大模型裝進機器人：蒸餾的威力

這個案例回應同學最可能有的疑問：大模型跑不動，是不是就只能放棄？

問題的背景很清楚：視覺語言模型在機器人領域已成為強大的工具，能以語言脈絡理解環境，催化了物件目標導航這類開放詞彙任務；但它們的運算量通常使其只能在雲端執行。

研究團隊的做法是蒸餾：先以大型前沿模型建立一套基於場景圖的高水準管線，在標準的開放詞彙物件導航基準上達到 39.7% 的成功率；接著把這套推理能力轉移到僅 40 億參數的本地模型上，以便在嵌入式 GPU 裝置上執行。

關鍵的結果是：僅以約 500 條前沿模型的推理軌跡微調小模型，就能有效賦予它導航能力，

成功率達到 34.5%，大幅縮小了與大型雲端模型之間的差距。

五百條資料——這個數字值得細細品味。相較於從零訓練所需的數十萬筆標註，這是極小的成本。而換來的是「能在機器人上跑」這個質變：不需要網路、沒有雲端延遲、資料不外流。

這個案例的方法論啟示，與第 25 章 NanoSLAM 的案例完全一致：效能瓶頸往往不在算力不足，而在方法未經最佳化。當你覺得「這個技術太重、跑不動」時，先問的不該是「要不要放棄」，而是「有沒有更聰明的做法」。

## 實作：從模擬訓練到邊緣部署

本章的實作分三個階段，難度遞增。階段一不需要任何特殊硬體，階段三需要一台嵌入式運算模組。

### 階段一　平行度的三張圖（不需要機器人）

重現本章圖 33-1 的三個量測。任務可以自己訂，最簡單的是本章用的單關節追蹤，也可以用任何你熟悉的向量化環境。

1. 量吞吐量：固定策略，對 N = 1, 2, 4, ..., 4096 各跑一批，記錄牆鐘時間並換算成每秒環境步數。畫在對數座標上，標出飽和點。
2. 量梯度雜訊：固定策略，對每個 N 重複抽樣四十次，計算成本估計的標準差。驗證它是否依 1/√N 下降（乘上 √N 看是否接近常數）。
3. 量訓練穩定度：固定牆鐘時間預算，對不同 N 各跑五次獨立訓練，畫出中位數與四分位距。重點不是誰的中位數高，而是誰的四分位距窄。

這個階段的產出是一張「你的硬體的平行度曲線」。它會告訴你：在你的機器上，環境數開到多少之後就不再划算。這是後續所有訓練實驗的規劃依據。

## 階段二　系統識別與域隨機化（需要一個有編碼器的關節）

最低需求是一個帶編碼器的伺服馬達或減速機模組，能送出位置指令並讀回實際角度。一個機械手臂的單關節就夠了。

1. 錄掃頻資料：讓關節不接觸任何負載，送出 0.2 至 2 Hz 的掃頻位置指令，以盡可能高的頻率記錄指令與編碼器讀值，錄 20 至 60 秒。
2. 建模型：寫出含轉動慣量、黏滯阻尼、庫倫摩擦與指令延遲的離散時間模型。先用 CAD 的慣量值、其餘設為零，模擬一次並與實測比較——這就是你的圖 33-3（a）。
3. 做識別：外層對延遲做整數格點搜尋，內層以最小平方法擬合連續參數。記得驗證參數是否互相補償：換一段不同的激振訊號重新驗證，如果換了訊號參數就跑掉，表示識別還不夠穩。
4. 量代價：用識別後的參數訓練一組控制器，再用寬範圍域隨機化訓練另一組，在真實關節上比較兩者的追蹤誤差。你應該會看到與圖 33-4 相同的現象：隨機化的那組比較耐操，但名目精度較差。

## 階段三　邊緣部署（需要嵌入式運算模組）

1. 建立基準線：在模組上單獨跑 SLAM，記錄記憶體峰值、延遲與更新率。再單獨跑你的 AI 模型，同樣記錄三項。
2. 合併執行：兩者同時跑，再量一次三項。比較「合併後」與「分開時的總和」，差額就是競爭造成的損失。
3. 量化：把模型量化到 8 位元與 4 位元，各量一次記憶體與精度。用公式二的估算值對照實測值，看看框架開銷佔多少。
4. 套用對策：依 33.8 節的三種對策逐一嘗試，每次只改一項並重新量測，建立一張「對策 × 效果」的表。

這個階段的產出是一份部署報告，內容是你自己量到的數字。它比任何規格表都有說服力，也是本書技術路線最實際的收尾。

## 延伸閱讀

- Isaac Lab 技術報告（2025）。（GPU 原生平行模擬的架構與吞吐量數據，33.3 節的依據）
- Isaac Sim 與 Isaac Lab 的 GitHub 專案與官方文件。（33.2、33.7 節的實作基礎）
- 足式機器人的系統化 sim-to-real 轉移研究（2025）。（33.6 節與案例 33-1 的來源）
- 學習式足式運動控制的綜述論文。（本章 33.1 節的背景，含各家方法的比較）
- OpenUSD 規格與機器人結構描述文件。（33.2 節場景與機器人描述格式的參考）
- NVIDIA Jetson 開發者文件與推論最佳化指南。（33.8 節邊緣部署的技術基礎）
- 第 25 章的邊緣運算、第 31 章的評估方法、第 32 章的資料蒐集與標註。（本章與全書其他章節的直接銜接點）

## 本章重點

- 強化學習的機器人策略需要十億等級的環境互動，在真機上相當於約 231 天的連續運行，因此必須先在模擬中訓練。模擬提供時間加速、零成本失敗、完全可知的狀態、可重複的實驗與可生成的環境；但它解的是你寫進去的方程式，不是真實物理。
- Isaac Sim 是模擬器與場景環境（PhysX 物理、RTX 算繪、OpenUSD 場景），Isaac Lab 是建構其上的機器人學習框架（任務、觀測、獎勵、訓練流程）。兩者於 2025 年 8 月開放一般取用並將擴充層開源，但底層的 Omniverse Kit 並未全部開源，也不接受外部提交。
- GPU 原生模擬的關鍵是狀態以張量常駐 GPU，整個訓練迴圈不需往返 CPU。實測顯示吞吐量在 $N \le 32$ 時近乎線性（每秒 3.5 萬步增至 113 萬步），其後飽和於每秒約 2,285 萬步；但環境數增加 4,096 倍時，每批次的等待時間只增加 6.3 倍。

- 平行度的真正價值是降低梯度雜訊。實測的成本估計標準誤精確地依 1/√N 下降（乘上 √N 後十一個點都落在 0.32 至 0.43）。在相同的 22 秒預算下，N=4 的成功率中位數只有 35.3% 且四分位距橫跨 15.8–85.6%，N=64 達 98.4%，N=1024 達 99.2% 且四分位距僅 98.3–99.4%——平行度不足時，訓練結果的變異大到無法比較。

- 感測器仿真可分四個家族：本體感測（高）、相機（中高）、測距（中）、接觸與觸覺（中低）。相機的分割、深度、法向量是算繪的副產品，這是合成資料的技術基礎；而光達的鏡面反射、運動畸變與感測器之間的時間差，是最常被忽略的落差來源。

- 合成資料的誘惑是標註免費且零誤差，陷阱是域落差。三條縮小落差的路是真實場景重建、視覺域轉換與視覺域隨機化；實務上最有效的是合成資料預訓練加少量真實資料微調，且測試集一律只用真實資料。

- sim-to-real 落差有七個來源，其中致動器動力學、連桿慣量誤差與時間延遲最常被低估。系統識別實驗顯示：URDF 的轉動慣量只有真實值的 0.33 倍，而阻尼、摩擦、延遲三項根本是零；校正後模擬與真實的均方根誤差由 3.42 度降到 0.0171 度，約兩百分之一。

- 系統識別有一個典型陷阱：延遲在離散時間下是階梯函數，用梯度法會停在起始值不動，而慣量會被高估去補償它——軌跡吻合但參數全錯。正確做法是外層對延遲做格點搜尋。

- 域隨機化的範圍必須寬到涵蓋真實的不確定性：±10% 的窄範圍幾乎無效（可部署區 34.6%，與不隨機化的 34.2% 相當），寬範圍則達 98.5%。但代價是名目點的追蹤誤差從 0.0061 弧度增為 0.0245 弧度，約四倍——穩健度是用名目效能換來的。

- 實測的失敗邊界幾乎只由延遲決定（約 9.5 毫秒），與轉動慣量幾乎無關。這說明規劃時應優先量測控制迴路的延遲，而非精算每個連桿的質量。

- 兩種對策通常並用：先以系統識別把模擬的中心點校正正確，再在其周圍做適度隨機化以涵蓋磨損、溫度與個體差異。案例 33-1 顯示，校正做得夠好時可以完全不用動力學的域隨機化。

- 邊緣部署的三大技術為量化（4 位元權重是目前邊緣視覺語言模型的預設）、蒸餾（約 500 條推理軌跡微調 40 億參數模型即可達 34.5%，接近雲端大模型的 39.7%）、以及記憶體管理（語意分區的關鍵幀載入卸載能消除記憶體不足並降低 21.7% 延遲）。
- 選擇邊緣模組時，決定能不能跑的往往是記憶體那一欄而非算力那一欄；且不同世代的標示單位不同（FP4 稀疏的 TFLOPS 與 INT8 的 TOPS 不可直接相比）。實際記憶體用量約為公式二估算值的 1.2 至 2 倍。

## 承先啟後

本章把全書的技術從「能跑」推到「能學」：第 32 章讓地圖有了意義，本章則讓機器人有了在模擬中取得技能的途徑，並把訓練好的模型一路送到機器人身上的運算模組裡。

但還有最後一塊拼圖沒有補上。本章訓練出來的是「怎麼走」的策略，第 32 章建立的是「這裡是什麼」的地圖——中間缺的是「為什麼要去那裡」。當使用者說「去會議室把投影機關掉」，機器人需要把一句話拆成一連串可執行的子目標，需要在沒見過的環境裡推理「會議室通常在哪」，也需要在找不到目標時知道該往哪裡探索。

這就是第 34 章的主題：視覺語言模型、語言導航與具身智慧。它也是全書的最後一章——我們會把第 1 章到第 34 章的線索收攏起來，用一次完整的任務說明這三十四章各自在哪一個環節發揮作用。

台灣自編大學教科書

@prof

# 《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

## 第 34 章

## VLM、語言導航與具身智慧

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程　/　高中 AI 機器人社團課程

## 本章在全書的位置

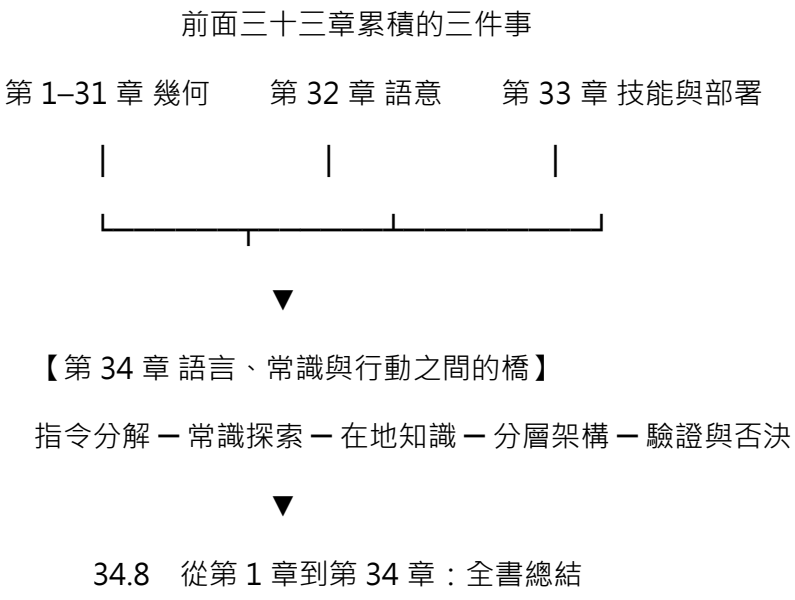
這是全書的最後一章，也是把前面三十三章收攏起來的一章。

到目前為止，我們有了三樣東西：第 1 至 31 章給了機器人一張準確的幾何地圖，讓它知道「我在哪、環境長什麼形狀」；第 32 章讓地圖有了意義，讓它知道「那是一扇門、那是一個人」；第 33 章讓它能在模擬中學會技能，並把模型送進機身上的運算模組。

還缺最後一樣：目的。當使用者說「去會議室把投影機關掉」，機器人必須把一句話變成一連串可執行的子目標，必須在沒去過的樓層推理「會議室大概在哪一側」，也必須在找不到目標時知道該往哪裡繼續找。這就是本章的主題——語言、常識與行動之間的橋。

本章同時承擔全書的總結。34.8 節會用一次完整的任務，把第 1 章到第 34 章的每一篇放回它該在的位置，說明為什麼這九篇缺一不可。

### 圖 34-0　第 34 章的連結地圖




## 學習目標

本章從「一句話怎麼變成座標」開始，依序處理常識先驗、在地知識庫、分層架構與驗證機制，再誠實地檢視目前視覺語言模型在導航上的真實水準，最後以全書總結作結。本章有三張以實驗算出來的圖，分別回答三個問題：常識值多少、常識錯了會怎樣、以及為什麼一定

要有驗證層。讀完本章，同學應能：

1. 說明自然語言指令轉為導航目標的各個階段，並指出每一階段可能的失敗模式。
2. 解釋常識先驗如何縮短物件目標導航的搜尋成本，並能量化其效益。
3. 說明一個指錯方向的先驗會造成什麼代價，並據以判斷何時不該相信語意層。
4. 設計一個在地空間知識庫，並說明檢索增強生成相較於微調模型的三個優點。
5. 說明依時間尺度分層的系統架構，並指出每一層的更新頻率與職責。
6. 以連乘模型估算多步驟任務的整體成功率，並說明驗證與重試為何能大幅改善它。
7. 說明目前視覺語言模型在具身導航上與人類表現的差距，並能解讀相關基準的結果。
8. 把全書九篇的內容對應到一次完整任務的各個環節。

## 核心概念

| 名詞 | 意義 | 關聯章節 |
|---|---|---|
| 具身智慧 | 智慧體透過身體與環境互動而產生的智慧 | 34.1、第 33 章 |
| 語言導航 | 依自然語言指令在未知環境中導航 | 34.2、34.6 |
| 物件目標導航 | 給定一個物件名稱，找到它的位置 | 34.3 |
| 常識先驗 | 「某類物件通常在某類空間」的統計知識 | 34.3 |
| 語意探索 | 以語意先驗決定下一個探索方向 | 34.3、第 16 章 |
| 檢索增強生成 | 推論前先從知識庫檢索相關資訊 | 34.4 |

| | | |
|---|---|---|
| 場景圖 | 以節點與關係描述場景的結構化表示 | 34.1、第 32 章 |
| 分層架構 | 依時間尺度把系統切成數層，各司其職 | 34.5、第 3 章 |
| 否決機制 | 下層可推翻上層的決策以保障安全 | 34.5、34.7 |
| 視覺語言動作模型 | 直接由影像與語言輸出動作的模型 | 34.6、第 33 章 |

**表 34-1　本章的核心名詞**

## 34.1　從「知道那是什麼」到「知道該做什麼」

第 32 章結束時，機器人手上有一張標了語意的地圖。本節說明，從那裡到「能執行一句話」，中間還差什麼。

### 一、三個層次，再看一次

第 32 章用三個層次描述機器人的智慧：幾何層回答「我在哪、環境什麼形狀」，語意層回答「我看到了什麼」，任務層回答「下一步該怎麼做」。前兩層本書已經處理完畢，本章處理第三層。

任務層與前兩層有一個本質差異：前兩層的輸入是感測器，輸出是對世界的描述；任務層的輸入是人的意圖，輸出是行動。描述可以求準確，意圖卻常常是模糊的——「把會議室的燈關掉」沒說是哪一間會議室、也沒說燈的開關在哪裡。任務層的工作有一大半是在補上這些沒說出口的東西。

### 二、一句話裡藏了多少假設

以「去會議室把投影機關掉」為例，機器人必須自行補上的資訊至少有：

- 哪一間會議室？（若有多間，可能要依使用者所在位置或行事曆推斷）

- 會議室在哪？（需要在地的空間知識，34.4 節）
- 投影機長什麼樣子？（需要語意層的辨識能力，第 32 章）
- 投影機在會議室的哪裡？（需要常識：通常在天花板或講桌上）
- 「關掉」要怎麼做？（按遙控器？按機身按鈕？還是發紅外線訊號？）
- 怎麼確認真的關掉了？（需要驗證，34.5 節）
- 關不掉怎麼辦？（需要降級與回報，34.7 節）

這七項裡面，只有第三項是第 32 章處理過的。其餘六項分別需要在地知識、常識推理、動作知識與驗證機制——這正是本章要補齊的部分。

## 三、為什麼場景圖比語意地圖更有用

第 32 章提過場景圖：不只記錄「這裡有一張椅子」，還記錄「椅子在桌子旁邊、桌子在會議室裡」的關係結構。到了任務層，這個差異就變得關鍵。

理由是：語言裡的空間關係是相對的。「桌上的杯子」、「門旁邊的滅火器」、「靠窗那排的第三個座位」——這些在平面的語意地圖上無法表達，因為語意地圖只記錄「某座標有某類物件」。要回答「桌上的杯子」，系統必須知道哪個杯子的支撐關係指向哪張桌子。

> **一個區分：導航的三種目標形式**
>
> 點目標：直接給定座標（「去（12.4, 8.1）」）。這是第 1 至 31 章處理的形式，也是所有上層最終都必須轉成的形式。
>
> 物件目標：給定一個物件類別（「去找飲水機」）。目標位置未知，需要搜尋——這是 34.3 節的主題。
>
> 指令目標：給定一段自然語言（「沿著走廊走到底，右轉進第二間教室」）。需要逐步理解並執行——這是 34.6 節評估的對象，也是目前最困難的一種。

## 34.2　語言指令怎麼變成導航目標

本節把這條轉換鏈拆開來看，因為每一個環節都是一個可能的失敗點。

### 一、五個階段

| 階段 | 輸入 | 輸出 | 典型失敗 |
|---|---|---|---|
| ① 意圖解析 | 自然語言 | 結構化的任務描述 | 指代不明、動詞歧義 |
| ② 地點定位 | 任務描述 | 語意地圖上的區域 | 地點名稱不在知識庫中 |
| ③ 目標定位 | 區域 + 物件類別 | 三維座標 | 物件不在原處、辨識錯誤 |
| ④ 路徑規劃 | 三維座標 | 可行路徑 | 目標不可達、門關著 |
| ⑤ 執行與確認 | 路徑 | 完成或回報 | 抵達了但沒確認、中途被打斷 |

**表 34-2　從語言到行動的五個階段**

值得注意的是：第④階段之後的一切，本書前三十一章已經完全處理過了。本章與第 32 章新增的，是第①到③階段。換句話說，語言導航不是取代 SLAM，而是在 SLAM 的前面接上一段轉換。

### 二、輸出要結構化，不要自由文字

一個實作上的重要原則：上層送給下層的，應該是結構化的資料（目標座標、物件清單、動作代號），而不是自由文字。

理由很實際。自由文字沒有辦法可靠地解析，也沒有辦法驗證。如果語言模型回答「往左前方那間看起來像茶水間的房間走」，下游系統要怎麼把它變成座標？而如果它回答的是「目標區域：茶水間，信心 0.72；候選座標 （23.3, 13.9）」，下游就能查詢地圖驗證該座標是否可達、是否在禁區內、是否與代價地圖衝突。

這也是為什麼「讓語言模型直接輸出速度指令」在工程上是糟糕的設計。它跳過了所有可以驗證的中間表示，把一個可以被檢查的系統變成一個不可檢查的黑盒子。

### 三、不確定性要往下傳

第 11 章花了整章講機率與不確定性，這個觀念在任務層同樣適用，而且常常被忽略。語言模型通常會用同樣肯定的語氣說出「會議室在三樓東側」，無論它是從知識庫查到的、還是猜的。

務實的做法是：讓上層輸出的每個目標都帶一個信心值與一個來源標記（查自知識庫 / 由常識推得 / 純粹猜測）。下層依這個標記決定要多謹慎——查自知識庫的目標可以直接前往，純粹猜測的目標應該先派出一趟低成本的探索再決定。

## 34.3　常識先驗與語意探索

「洗手台通常在廁所或茶水間」——這類知識無法從幾何地圖推導，卻能大幅改變機器人的搜尋效率。本節用一個可完整重現的實驗來量化它值多少。

### 一、實驗設定

場景是一層系館樓層：中央走廊加上兩側十二個房間，房間有六種類型（教室、實驗室、辦公室、茶水間、會議室、廁所、儲藏室）。機器人從入口出發，要找到一個指定的物件（飲水機、滅火器、投影機、急救箱、梯子、影印機之一）。

關鍵的設定是：兩種策略拿到的是同一張地圖。這不是「有地圖」對「沒地圖」的比較——那不公平，而是同一張先驗地圖之下，搜尋順序的比較：

- 幾何順序：每次前往最近的未搜尋房間。這是不使用任何語意資訊的基準。
- 語意順序：依「這類物件出現在這類房間的機率」由高到低排序，機率相同時取較近的房間。

物件依一組真實的條件機率分布隨機放置（例如飲水機有 55% 的機率在茶水間、30% 在走

廊、15% 在會議室）。機器人進入房間後執行來回掃掠（掃掠長度依第 16 章的覆蓋規劃估算），當物件進入 3.5 公尺的感測範圍且視線不被牆擋住時視為找到。共跑 600 次隨機試驗。

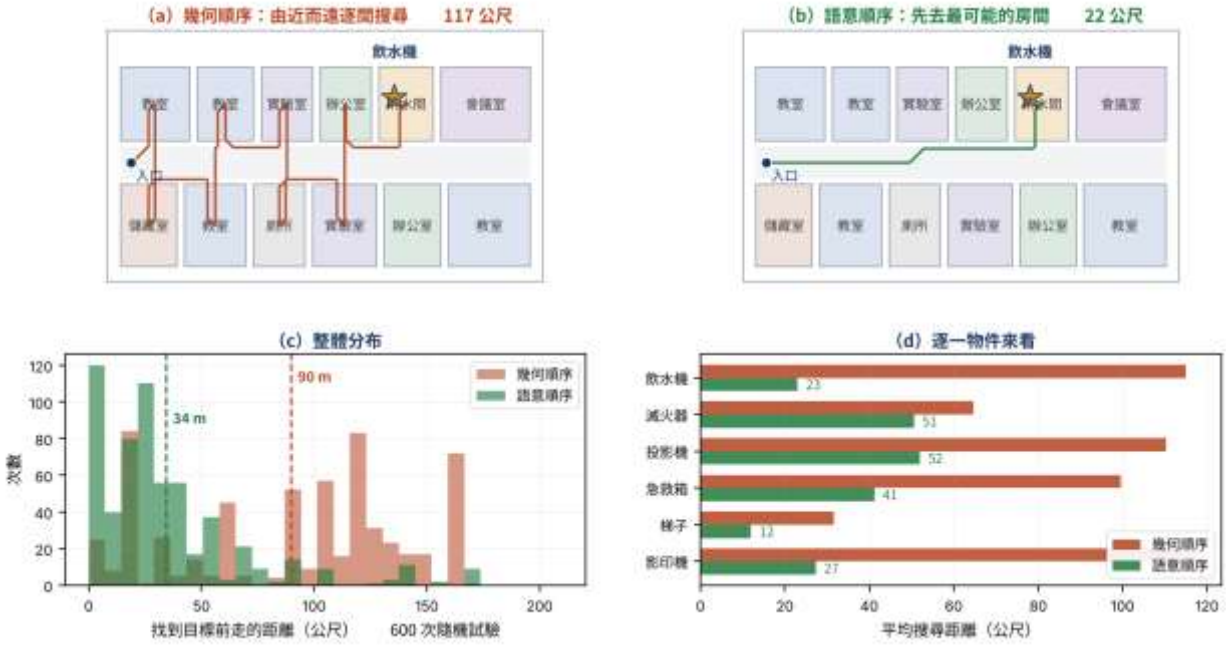



**圖 34-1　同一張地圖，兩種搜尋順序**

（a）與（b）畫的是同一次試驗：飲水機落在茶水間。幾何順序從入口旁最近的教室開始，一間一間往東推進，走了 117 公尺才抵達茶水間。語意順序直接前往茶水間，只走了 22 公尺。圖上畫的是房間之間的移動路徑，房內的來回掃掠沒有畫出來，但距離有計入。

（c）是 600 次試驗的整體分布。幾何順序的平均是 90 公尺，語意順序是 34 公尺，平均縮短 61.7%；若看中位數則是 102.5 公尺對 26.5 公尺，縮短 74.2%。兩個分布的形狀也不同：幾何順序呈現明顯的多峰，每一個峰對應「物件落在第幾間被搜尋的房間」；語意順序則大量集中在 30 公尺以內。

（d）逐一物件來看，效益差異很大。梯子從 32 公尺降到 12 公尺、影印機從 118 公尺降到 27 公尺，因為它們的分布高度集中（梯子有 70% 在儲藏室、影印機有 65% 在辦公室）。滅火器只從 65 公尺降到 51 公尺，因為它有 60% 的機率在走廊上——而走廊是機器人無論如何都會經過的地方，語意先驗在這裡幫不上什麼忙。

這個差異本身就是一個有用的結論：常識先驗的價值，取決於該物件的分布有多集中。對於

「到處都有」的物件（滅火器、垃圾桶、消防栓），花力氣建立語意推理的報酬率很低；對於「只在特定地方」的物件（梯子、影印機、特定儀器），報酬率就很高。

**一個常被跳過的數字**

在這 600 次試驗裡，語意順序有 12.2% 的情況反而比幾何順序走得更遠。

原因是：機率最高的房間不一定是這一次物件真正所在的房間。當飲水機那 15% 的機率落在會議室時，先衝去茶水間的機器人就白跑了一趟。

這提醒我們：語意先驗改善的是「期望值」，不是「每一次」。在評估任何語意導航系統時，只看平均值會低估它的變異——這也是第 31 章一再強調的評估原則。

## 二、先驗錯了會怎樣

前面的實驗假設機器人的常識是對的。但視覺語言模型的常識是從網路資料學來的，不一定符合你的建築。「影印機在辦公室」在多數場合成立，但如果這棟樓把影印機統一放在走廊的開放區域呢？

下面的實驗掃描先驗的正確率 $\alpha$：以機率 $\alpha$ 使用真實的分布，否則把房間類型的排序隨機打亂。另外再加一個極端情況——先驗「反著指」，也就是真實機率越高的房間，排序越往後。

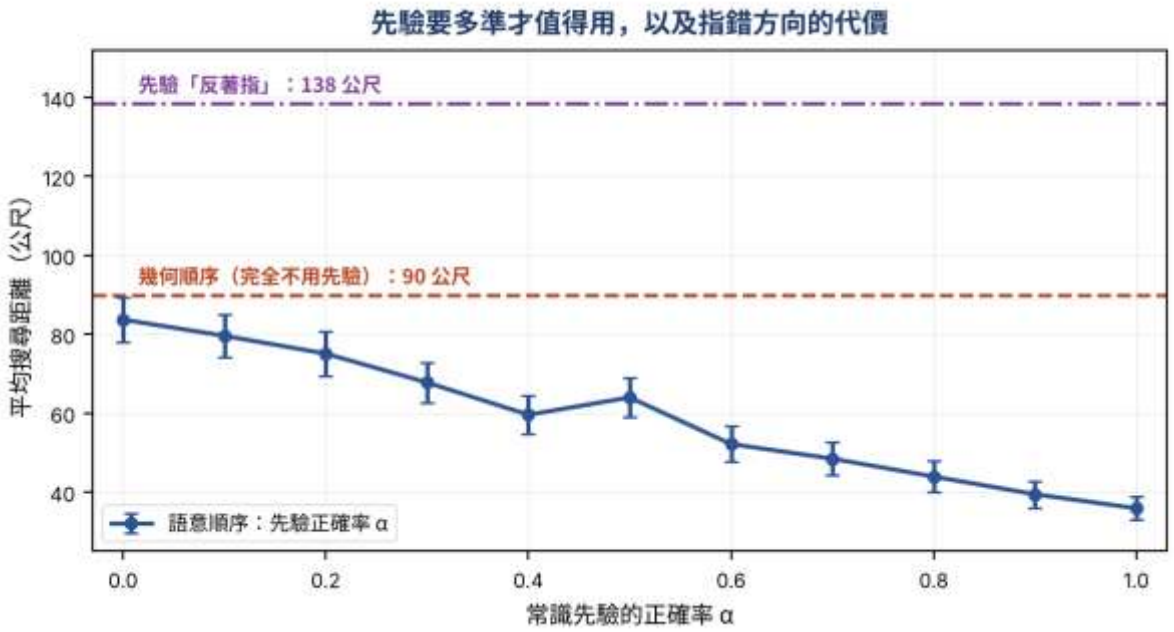



**圖 34-2　先驗要多準才值得用，以及指錯方向的代價**

藍線是語意順序隨 α 的變化，誤差棒是 95% 信賴區間。從 α=0 的 84 公尺一路降到 α=1 的 36 公尺，大致呈線性。這條線性關係本身是個好消息：常識先驗不需要完美才有用，正確率每提升一成，就省下約五公尺。

紅色虛線是完全不用先驗的幾何順序（90 公尺）。注意藍線即使在 α=0 時也略低於它——一個完全隨機的搜尋順序，和「由近而遠」的順序差不多好。這有點反直覺，但道理不難懂：物件是隨機放置的，「先去近的房間」並不會提高找到它的機率，只是先省下一點移動距離而已。

真正的風險在紫色點線：當先驗反著指時，平均搜尋距離是 138 公尺，比完全不用先驗還糟 54%。這就是語意層最危險的失效模式——它不是沒幫上忙，而是主動把機器人帶往錯誤的方向，而且它會用同樣肯定的語氣這麼做。

對系統設計的啟示很明確：語意層的建議必須可被推翻。合理的做法是設一個預算——若依語意順序搜尋了前兩個建議仍未找到，就退回幾何順序，不再相信先驗。這正是 34.5 節「否決機制」的具體形式。

## 34.4　RAG：讓機器人讀懂在地的空間知識

上一節的常識先驗處理的是「一般而言」的知識。本節處理另一半：這一棟樓特有的知識。

### 一、問題與解法

這裡有一個實務問題：視覺語言模型再聰明，也不知道「這所學校的教務處在哪一棟哪一樓」——這是特定機構的知識，不在模型的訓練資料中。

解法是檢索增強生成（RAG）：建立一份在地的空間知識庫，當機器人收到指令時，先從知識庫檢索相關資訊，再交給模型推理。模型負責語言理解與推理，知識庫負責事實。

**圖 34-3　校園空間知識庫的範例結構**

【樓層拓樸】

一樓：校門口 → 穿堂 → 教務處 → 學務處 → 樓梯 → 電梯

二樓：電梯 → 走廊 → 電腦教室 → 會議室

【地點屬性】

教務處：一樓、穿堂東側、開放時間 08:00–17:00、門為玻璃自動門

智慧機器人教室：三樓、入口 → 工作桌 → 機器狗測試區 → 光達建圖區 → 充電區

【通行規則】

機器人不可進入：實驗室內部、樓梯（除非為爬樓梯平台）

需減速區域：穿堂、教室門口、轉角

【常見查詢對應】

「辦公室」→ 教務處、學務處　　「充電」→ 各樓層充電站

### 二、RAG 的三個優點

1. 可維護：校園配置改變時，只需更新知識庫，不需重新訓練模型。一間教室改了用途，改一行文字就好。

2. 可驗證：機器人的回答可以追溯到知識庫的哪一條，便於除錯與問責。這與 34.7 節的可解釋性問題直接相關。
3. 低成本：相較於微調大模型，建立知識庫的成本低得多，也更適合教學專題——一份結構化的文字檔就能起步。

## 三、三個實用的查詢範例

- 「從入口到機器狗測試區怎麼走？」——RAG 檢索到智慧機器人教室的內部拓樸，模型據此生成路線描述，再轉為導航目標序列。
- 「如果前方有學生坐在地上，機器人應該如何繞路？」——結合即時感知（偵測到人）與知識庫的通行規則（需減速、禮讓），產生繞行策略。
- 「掃地機器人如何規劃覆蓋整間教室？」——結合地圖（第 13 章）與覆蓋規劃（第 16 章），這是語意層與幾何層協作的典型例子。

## 四、務實的架構建議

對專題實作而言，建議的分工是：RAG 知識庫用結構化的文字檔或簡單資料庫即可（不必一開始就上向量資料庫）；語言模型負責語言理解與視覺判斷；而所有涉及安全的決策（避障、速度限制）仍由傳統的幾何系統把關。這是下一節反覆強調的原則。

> **知識庫與常識先驗的分工**
>
> 一個容易混淆的地方：34.3 節的常識先驗與本節的知識庫，都是「模型之外的知識」，但兩者的性質完全不同。
>
> 常識先驗是統計性的、跨場域通用的（「梯子通常在儲藏室」），它回答的是「該先去哪裡找」。知識庫是確定性的、場域專屬的（「儲藏室在一樓西側」），它回答的是「那裡在哪裡」。
>
> 一個好的系統兩者都要：用知識庫把能查到的查清楚，查不到的才交給常識先驗去猜——而且要記得標明這是猜的（34.2 節）。

## 34.5　分層架構與驗證機制

本節說明如何把前面各節的元件組裝成一個不會因為上層出錯而失控的系統。

### 一、依時間尺度分層

**圖 34-4　語言導航系統的分層架構**

【任務層】秒級以上　語言模型 Agent

- 理解自然語言指令、查詢知識庫（34.2、34.4 節）
- 分解任務、選擇探索方向、判讀異常（34.3 節）

↓ 輸出：語意目標（去哪、找什麼）+ 信心值 + 來源

【語意層】0.1–1 秒　視覺 AI 模型

- 物件偵測、分割、深度估計（第 32 章）
- 語意地圖的建立與查詢

↓ 輸出：幾何目標座標、動態物體標註

【幾何層】10–30 Hz　SLAM + 導航（第 1–31 章）

- 定位建圖、路徑規劃、代價地圖

↓ 輸出：速度指令

【控制層】100 Hz 以上　底盤 / 飛控（第 33 章）

- 運動控制、即時避障、安全停止

【安全原則】上層可以出錯，下層必須守住安全底線

這個分層與第 3 章的混合式架構、第 15 章 Cartographer 的雙子系統、第 18 章 LOAM 的雙層設計、第 26 章無人機的降級架構，是同一個系統設計模式在 AI 時代的延伸——依時間尺度分層、各層各司其職。同學至此應該能深刻體會：好的架構原則具有跨時代的普遍性。

### 二、整合的四個實務要點

1. 非同步設計：AI 推論耗時，絕不能阻塞導航迴路。應以非同步方式呼叫，並在等待期間讓機器人維持安全行為（繼續執行上一個目標或停下）。

2. 明確的介面：AI 層輸出的應該是結構化的資料，而非自由文字（34.2 節）。這讓下游系統能可靠地解析與驗證。

3. 可否決機制：幾何層應能否決 AI 的決策。若 AI 說「往前走」但代價地圖顯示前方是障礙，幾何層必須勝出。圖 34-2 的紫色點線說明了沒有否決機制的代價。

4. 降級路徑：AI 服務不可用時（斷線、逾時、模型崩潰），系統應退回純幾何導航，而非停擺——這正是第 26.2 節「失效逐級降級」的原則。

## 三、為什麼多步驟任務一定要有驗證

前面講的「可否決機制」聽起來像是一個安全上的保險。但它其實還有一個更基本的作用：沒有它，多步驟任務根本不可能有堪用的成功率。下面用一個簡單的模型說明。

假設一個任務有 n 個步驟，每一步的成功率都是 p，且各步驟獨立。那麼整體成功率是 p 的 n 次方——這是公式一。這個連乘關係的破壞力，遠比直覺來得大。

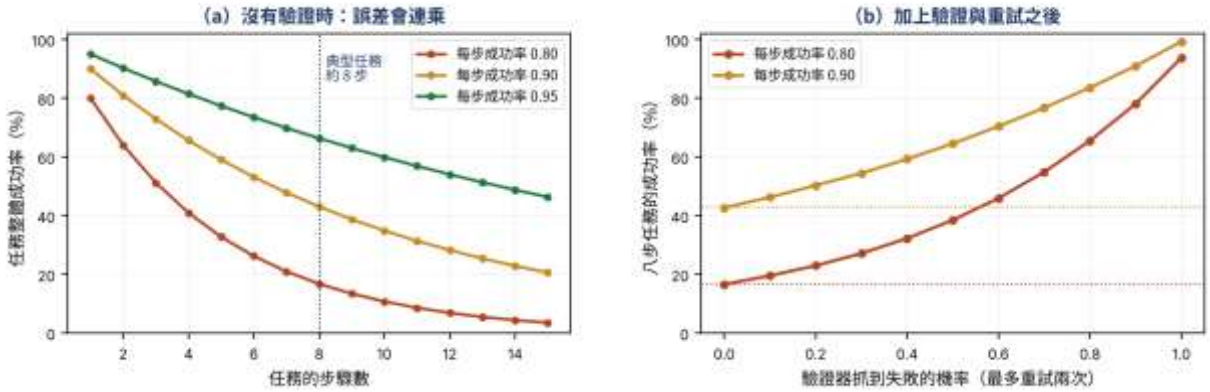



**圖 34-5　多步驟任務的成功率，以及驗證與重試的效果**

（a）畫的就是連乘的後果。一個八步的任務（34.2 節的五個階段再加上開門、對位、回報，八步是很保守的估計），即使每一步都有 90% 的成功率，整體成功率只有 43.0%；每步 80% 的話更只剩 16.8%。要讓八步任務達到 66.3%，每一步必須做到 95%。

這解釋了一個常見的困惑：為什麼機器人的每個子系統看起來都不錯，整體卻老是失敗？不是哪一個環節特別糟，而是連乘本身就很殘酷。它也解釋了為什麼「把任務拆得更細」不一定是好事——拆得越細，步驟越多，連乘的懲罰越重。

（b）是對策。在每個步驟後面加上一個驗證：若驗證器判定這一步失敗，就重試（最多兩次）。橫軸是驗證器抓到失敗的機率，點線是沒有驗證時的水準。以每步 80% 的系統為例，整體成功率從 16.6% 提升到驗證率 0.5 時的 38.5%、驗證率 1.0 時的 93.7%。每步 90% 的系統則從 42.7% 提升到 99.3%。

請注意這個改善的幅度：一個完美的驗證器，把一個「十次有八次失敗」的系統變成「十次有九次成功」，而底層每一步的能力完全沒有改變。這是本章最重要的工程結論之一——對多步驟任務而言，投資在「知道自己做錯了」上面，報酬率遠高於投資在「做得更好」上面。

更何況驗證通常比執行容易得多。「走到會議室門口」需要完整的導航堆疊，但「確認自己是不是在會議室門口」可能只需要讀一次門牌、或比對一次當前位姿與目標位姿。這種「驗證比執行簡單」的不對稱，是分層架構能夠成立的根本原因。

### 四、公式一：多步驟任務的成功率

設任務有 n 個步驟，第 i 步的成功率為 $p_i$，各步獨立，無驗證與重試，則：

$$P_{\text{任務}} = p_1 \times p_2 \times \dots \times p_n$$

當各步成功率相同時簡化為 $P = p^n$。若加入驗證（偵測率 r）與最多 k 次重試，單一步驟的有效成功率提升為：

$$p_{\text{eff}} = p + (1-p) \cdot \Sigma_{j=1..k} \left(r(1-p)\right)^{j-1} \cdot r \cdot p$$

其中 $r \cdot p$ 是「被抓到並重試成功」的機率。當 $r \to 1$ 且 k 夠大時，$p_{\text{eff}} \to 1$，整體成功率趨近 1；當 $r \to 0$ 時 $p_{\text{eff}} \to p$，退回（a）的情況。圖 34-5(b) 的曲線就是這條式子在 n=8、k=2 下的數值結果。

## 34.6　現況：視覺語言模型在導航上到底有多行

前面五節描述了一個理想的架構。本節誠實地檢視：以 2025 至 2026 年的技術水準，這個

架構裡最上面那一層做得如何。

## 一、一個具身導航的基準

2025 年底發表的一個具身導航基準提供了相當清楚的答案。它的設計是：給模型一張真實場景的照片、一段文字指令，以及一個載體類型（人、四足機器人、輪型機器人、腳踏車），要模型在影像上畫出應該走的軌跡。資料集包含 1,000 個情境與超過 3,000 條專家軌跡，評分方式結合了軌跡的動態時間校正距離、終點誤差，以及依語意分割計算的「載體不該走的地方」的懲罰。

八個當代的視覺語言模型被評估。結果如下：

| 方法 | 分數 |
|---|---|
| 人類專家 | 75.40 |
| 只預測路徑（直接告訴它目標在哪） | 56.55 |
| 已知目標的直線基準 | 51.89 |
| 完整軌跡預測（最佳模型） | 34.38 |
| 只預測目標點 | 29.65 |

**表 34-3　具身導航基準的評估結果（分數越高越好）**

## 二、三個應該被記住的數字

第一，最佳模型的 34.38 分與人類專家的 75.40 分之間，差距是 41.02 分。這不是一個可以用「再過半年就追上」來形容的差距。2026 年的視覺語言模型在具身導航上仍遠不及人類。

第二，也是最值得玩味的一個：一條「已知目標的直線」拿到 51.89 分，遠高於最佳模型完整軌跡的 34.38 分。一條完全不看環境、不避障、不繞路的直線，竟然勝過當代最強模型的完整規劃。這說明模型的失分主要不在路徑形狀，而在它根本沒搞清楚目標在哪。

第三點佐證了這個判斷：當目標被直接告知時，同一個模型只預測路徑能拿到 56.55 分，反

過來只預測目標點則掉到 29.65 分。也就是說，模型「會走」但「不會認」——把語言裡的目標對應到影像中正確的位置，才是目前真正的瓶頸。

### 三、這對系統設計意味著什麼

這三個數字直接支持本章 34.4 與 34.5 節的設計原則。既然模型最弱的環節是「目標定位」，那麼系統就應該盡可能不讓它做這件事——能從知識庫查到的地點，絕不讓模型猜（34.4 節）；必須猜的時候，要標明是猜的並設定搜尋預算（34.2、34.3 節）；而猜錯的代價，要由驗證與否決機制吸收（34.5 節）。

反過來說，模型相對擅長的事情應該交給它：理解語言、判讀畫面、生成報告、在已知目標的情況下描述路線。這就是一個務實的分工——不是問「語言模型能不能做導航」，而是問「導航的哪些部分適合交給它」。

> **另一個現況：真實部署仍然稀少**
>
> 2026 年的一篇綜述指出，語言導航的研究進展「大多仍侷限於軟體層面的真實轉模擬設定」，對模擬轉真實與真實世界部署的研究相對有限，而「邊緣部署的實用方案也相當稀缺」。
>
> 這與第 33 章的結論互相呼應：模擬裡的成績與真機上的表現是兩回事。對同學而言，這其實是機會——一個在真實校園裡跑得起來的語言導航系統，即使功能簡單，在目前也具有實質的研究價值。

## 34.7　風險、限制與未來發展

在收尾之前，必須談三件事：風險、限制、以及方向。

### 一、四大風險

1. 幻覺：AI 可能「看到」不存在的東西或做出錯誤推理，而且它通常不會表達不確定。若機器人完全信任 AI，可能撞牆或走錯路。圖 34-2 的紫色點線正是這個風險的量化

形式。對策是保留幾何層的驗證與否決權（34.5 節）。

2. 誤判的系統性：AI 的錯誤往往不是隨機的，而是有系統的偏差——例如在特定光線下總是把某類物件認錯。這比隨機錯誤更危險，因為它會持續發生，而且重試也救不了（34.5 節的驗證模型假設重試是獨立的，系統性錯誤違反這個假設）。對策是充分的場域測試與錯誤分析（第 32 章、第 33.5 節）。

3. 延遲：大模型推論耗時，若放在即時控制迴路中會造成災難。對策是嚴格的分層架構——AI 只做秒級以上的決策（34.5 節）。

4. 安全性：包含資安（模型或知識庫被竄改）、隱私（機器人拍攝的影像可能含個人資訊）、以及可問責性（出事時能否追溯決策依據）。這些不只是技術問題，也是倫理與法規問題。

## 二、當前的四個限制

- 資料需求：訓練或微調模型需要標註資料，而標註成本高昂。這是專題與小型團隊的主要障礙——也是為何蒸餾、少量樣本微調與合成資料（第 33.5、33.8 節）如此受重視。

- 泛化的落差：模型在實驗室表現良好，換個場域就下降。第 33.5 節的域落差討論同樣適用於語意層。

- 可解釋性：AI 為什麼做出這個判斷？當它出錯時，往往難以診斷。這與傳統幾何方法「每一步都可追溯」形成鮮明對比，也是 34.4 節主張用知識庫而非微調的理由之一。

- 運算與能源：如第 33.8 節所述，即使經過量化與蒸餾，AI 仍是機器人上最耗資源的元件之一。

## 三、未來的四個方向

1. SLAM 與基礎模型的深度整合：不再是「SLAM 加上 AI 模組」，而是統一的架構——由基礎模型同時處理感知、建圖與規劃。

2. 世界模型：讓機器人不只理解「現在是什麼」，還能預測「接下來會發生什麼」。這對動態環境中的規劃至關重要，也與第 33 章的模擬訓練直接相通——一個夠好的世界模型，本身就是一個可以在裡面訓練的模擬器。
3. 機器人基礎模型：跨平台的通用代理——同一套模型能部署到輪型、四足、人形、飛行等不同平台。這將大幅降低機器人開發的門檻。
4. 終身學習：機器人在部署後持續學習與適應，而非停留在出廠時的能力。第 25.6 節提過的裝置端自監督學習是這個方向的早期嘗試，而第 19.6 節的地圖維護則是它最務實的雛形。

## 34.8　從第 1 章到第 34 章：一台機器人的完整旅程

作為全書的最後一節，讓我們用一次完整的任務，把三十四章的內容串起來。

### 一、一次任務，九篇知識

情境：台灣南部一所科技大學的服務機器人接到指令——「請去三樓的智慧機器人教室，檢查充電區的設備狀態，並回報異常」。

1. 理解指令：模型解析出目標地點與任務內容（34.2 節）。輸出是結構化的任務描述，不是自由文字。
2. 查詢知識庫：RAG 檢索出「智慧機器人教室在三樓、內部拓樸為入口→工作桌→機器狗測試區→光達建圖區→充電區」（34.4 節）。這一步讓機器人不必用猜的。
3. 規劃路線：在既有地圖上規劃從當前位置到三樓的路徑，包含搭電梯（第 3 章、第 29 章）。
4. 移動與定位：光達 SLAM 或視覺慣性 SLAM 持續估計位姿（第 13 至 23 章），在既有地圖中定位（第 11 章、第 29 章），IMU 在轉彎與震動時提供支撐（第 7 章）。
5. 通過電梯口：看到人工標記，取得絕對位姿校正並確認樓層（第 4 章、第 24 章）。
6. 避開行人：偵測到人，一方面剔除其特徵以免干擾定位，一方面觸發減速禮讓（第

24 章、第 32 章）。

7. 抵達充電區：依語意地圖找到目標，以視覺伺服完成精確對位（第 24 章）。若語意地圖上找不到，則依常識先驗決定往哪裡找，並在兩次嘗試後退回幾何搜尋（34.3 節）。
8. 檢查與判讀：拍攝設備影像，AI 判斷是否異常，並把發現標上三維座標（第 31 章、第 32 章）。所有移動與操作的技能，來自第 33 章的模擬訓練。
9. 驗證每一步：每個階段完成後確認結果，失敗則重試（34.5 節）。沒有這一步，上面這八步的整體成功率會低到不堪用。
10. 回報：以自然語言生成報告，附上位置與影像證據（34.2 節）。

這一趟任務中，本書的每一篇都有貢獻：第一篇的觀念、第二篇的感測器、第三篇的數學、第四至六篇的 SLAM 系統、第七篇的邊緣運算、第八篇的實作工具、第九篇的 AI 語意、模擬訓練與語言導航。

而最關鍵的是：這九篇缺一不可。少了幾何層，機器人會撞牆；少了語意層，它聽不懂指令；少了模擬訓練，它學不會動作；少了工具鏈，一切都無法運行。這正是本書從第一頁到最後一頁想傳達的訊息——SLAM 不是單一技術，而是一整套互相支撐的知識體系。

## 二、一條貫穿全書的主線

本書從第 1 章的「機器人沒有地圖，也不知道自己在哪裡」開始，走過感測器、數學、2D 與 3D 系統、視覺方法、微型與群體機器人、完整實作，最後來到 AI 語意導航、模擬訓練與語言理解。

如果要說這三十四章有一個貫穿的主線，那就是：理解限制，才能突破限制。

我們反覆討論每種感測器的弱點（第二篇）、每種演算法的失敗模式（第四至六篇）、每個平台的資源約束（第七篇）、每個系統的除錯順序（第八篇）、以及每個模型的落差來源（第九篇）。這不是悲觀，而是因為工程的本質就是在限制中求解。

AI 的加入沒有改變這個本質。語言模型讓機器人「聽得懂、想得通」，但「走得穩、不撞人」仍然要靠前面三十一章的幾何與控制。本章 34.6 節的數字說得很清楚：在具身導航上，當代最強的模型仍遠不及人類，而一條已知目標的直線甚至勝過它的完整規劃。兩者缺一不可——而且可預見的未來也將如此。

### 三、給同學的最後幾句話

本書的兩個場域——大學智慧機器人學程與高中 AI 機器人社團課程——代表了兩種學習階段，但共享同一個信念：SLAM 不是遙不可及的尖端科技。它的核心版本可以跑在一顆手機等級的處理器上，它的原理可以用紙筆推演，它的成果就在你家客廳的掃地機器人裡默默運轉。

本書裡的每一張實驗圖，都是用一台普通的電腦算出來的。第 15 章的迴路閉合、第 17 章的配準、第 18 章的運動補償、第 19 章的巡檢路線、第 33 章的平行模擬與參數識別、以及本章的語意搜尋——沒有一張需要昂貴的設備。需要的只是把問題寫清楚、把實驗做乾淨、把數字誠實地看進去。

願同學帶著這本書打下的基礎，去建造屬於自己的機器人——無論它是巡檢校園的機器狗、掃描古蹟的無人機，還是一台只會沿線走的小車。每一台能自己找到路的機器人，背後都是同一套人類花了四十年才建立起來的智慧。

現在，輪到你了。

## 案例

### 案例 34-1　一句話、八個步驟：為什麼驗證比能力更重要

這個案例把 34.5 節的數字放回一個具體的情境裡。

假設同學做了一台校園送件機器人，任務是「把這份文件送到三樓教務處」。拆解之後有八個步驟：理解指令、查詢地點、規劃路線、搭電梯、走到門口、辨識門牌、進入、交付並確

認。

假設每一步都做到 80% 的成功率——這聽起來不差，每個子系統十次有八次成功。但整體成功率只有 0.8 的八次方，也就是 16.8%。十次任務裡有八次會在某個環節出錯。

團隊的直覺通常是去改善最弱的那一環。假設花了一整個學期，把每一步從 80% 提升到 90%——這是相當可觀的工程投入。結果呢？整體成功率從 16.8% 提升到 43.0%。進步很大，但十次仍有將近六次失敗。

另一條路是：每一步都不改，但在每一步後面加一個檢查。「電梯按了嗎？」比對樓層感測器。「到門口了嗎？」比對當前位姿與目標位姿。「門牌對嗎？」拍一張照辨識文字。檢查失敗就重做這一步，最多兩次。

以 34.5 節的模型計算：若檢查能抓到一半的失敗，整體成功率從 16.6% 提升到 38.5%；若檢查接近完美，則提升到 93.7%。而底層每一步的能力，一點都沒有改變。

這個案例要傳達的，不只是一個數字上的技巧。它是一個關於工程優先順序的判斷：當一個多步驟系統的整體表現不佳時，先問「它知不知道自己做錯了」，通常比先問「怎麼讓它做得更好」更有效。這與第 31 章的除錯順序、第 26 章的降級設計，是同一個思路。

## 案例 34-2　當機器人的常識指錯方向

這個案例處理一個同學在專題中很可能遇到、卻很少被事先提醒的失效模式。

情境：同學為巡檢機器人加上了語意搜尋的能力。機器人要找「影印機」時，會先問語言模型「影印機通常在哪裡」，得到「辦公室」的答案，於是優先搜尋所有辦公室。在測試的那棟樓，這個策略表現極好——搜尋距離縮短了七成。

接著他們把系統搬到另一棟樓。那棟樓為了省電與管理方便，把所有影印機集中放在每層樓電梯旁的開放區域，辦公室裡一台都沒有。結果機器人每次都先把所有辦公室跑一遍，然後才在「最不可能」的走廊上找到目標。

圖 34-2 量化了這個情況。當先驗完全反著指時，平均搜尋距離是 138 公尺，而完全不用先驗的幾何順序是 90 公尺——語意層讓系統退步了 54%。更糟的是，系統不會知道自己錯了：它每一次都「合理地」先去辦公室，每一次都合理地失敗。

值得注意的是中間那段。圖 34-2 的藍線在 α=0（先驗完全隨機）時是 84 公尺，略優於幾何順序的 90 公尺。也就是說，一個無知的先驗並不可怕，可怕的是一個有把握但錯誤的先驗。這正是語言模型最容易產生的那一種——它很少說「我不知道」。

對策有三個層次。最低限度是設搜尋預算：依語意建議搜尋前兩個地點，未果就退回幾何順序。進一步是把在地的觀察寫回知識庫：第一次在走廊找到影印機之後，這個事實應該被記下來，而不是下次再用通用常識猜一遍（34.4 節）。最徹底的做法是讓先驗本身可被學習——這正是第 33 章終身學習方向要處理的問題。

## 案例 34-3　一條直線，勝過最強的模型

這是一個適合放在全書最後的案例，因為它同時說明了 AI 的價值與它的邊界。

2025 年底的一個具身導航基準，要求模型在真實場景的照片上畫出機器人應該走的軌跡。評估的對象包含八個當代最強的視覺語言模型，評分同時考慮軌跡的形狀、終點的準確度，以及「這個載體不該走的地方」的懲罰。

結果中最令人意外的一項是：一條「已知目標位置的直線」得到 51.89 分，而最佳模型的完整軌跡預測只有 34.38 分。一條不避障、不繞路、完全不看環境的直線，贏過了當代最強模型的完整規劃。

第一反應可能是嘲笑模型。但仔細想，這個結果說的不是「模型很笨」，而是「評分裡絕大部分的分數來自目標的位置」。補充的實驗證實了這一點：同一個模型，在被直接告知目標位置後只預測路徑，分數躍升到 56.55；反過來只預測目標點，則掉到 29.65。

換句話說，模型「會走」但「不會認」。它能理解「沿著走廊往前、避開那堆箱子」這樣的

空間描述，但無法可靠地判斷「使用者說的那間會議室，是畫面裡的哪一扇門」。

這個診斷對系統設計極有價值，因為它指出了該把工程力氣花在哪裡。既然目標定位是瓶頸，那麼能查知識庫就不要猜（34.4 節）、能用人工標記就不要純靠視覺（第 4 章、第 24 章）、必須猜的時候要標明信心並設預算（34.2、34.3 節）。

同時它也說明了 AI 的真正價值在哪裡。模型拿到的 56.55 分（已知目標、只預測路徑）是高於直線基準的 51.89 分的——也就是說，在目標明確的前提下，模型確實能做出比直線更好的規劃。這就是本章 34.6 節的結論：不要問「語言模型能不能做導航」，而要問「導航的哪些部分適合交給它」。

最後值得補一句。人類專家在這個基準上的分數是 75.40，而最佳模型是 34.38——差距 41.02 分。這本書寫到最後一頁，技術仍在快速前進；但在可預見的未來，那套人類花了四十年建立起來的幾何方法，仍然是機器人能不能安全走到目的地的根本保證。

## 實作：校園語意導航系統

這是全書的最終實作，分四個階段，可依課程層次選擇進行到哪一階段。它同時用上第 32、33、34 三章的內容。

### 階段一　校園物件辨識資料集（高中適用）

1. 定義類別：與同學討論並決定五到十個校園常見物件（建議：教室門、白板、講桌、椅子、樓梯、電梯、走廊、垃圾桶、消防栓、飲水機、路障）。
2. 視角規劃：決定拍攝高度與角度，務必模擬機器人的實際視角（30 至 50 公分高），而非站著拍——這是本實作最重要的細節。
3. 分組蒐集：每組負責二至三類，每類拍攝至少 200 張，涵蓋不同角度、距離、光線與遮蔽情況，並刻意包含困難樣本。
4. 標註：先訂定標註規範（半掩的門算不算？框到哪裡？），再進行標註，並互相抽

查至少 10%。

5. 分割資料集：依「拍攝地點」而非隨機分割，避免評估結果虛高。

6. 訓練模型，在測試集上評估，並針對表現最差的類別做錯誤分析。

## 階段二　語意搜尋與常識先驗（本章核心）

這個階段不需要真實機器人，在一張地圖上就能完成，是本章三張圖的重現。

1. 建立樓層的語意地圖：把你們系館的平面圖數位化，標出每個房間的類型與門的位置。

2. 建立物件分布：調查五到八個物件在各類房間的實際出現頻率。這一步本身就是一個有價值的田野調查。

3. 實作兩種搜尋順序：幾何順序與語意順序，在隨機放置的物件上各跑數百次，比較搜尋距離的分布。這是你們的圖 34-1。

4. 掃描先驗品質：刻意把先驗打亂或反轉，量測搜尋距離如何變化，找出「先驗要多準才值得用」的交叉點。這是你們的圖 34-2。

5. 設計退場機制：實作「搜尋前兩個語意建議未果則退回幾何順序」的策略，量測它在先驗錯誤時能挽回多少。

## 階段三　在地知識庫與語言介面

1. 建立知識庫：依圖 34-3 的結構，寫出你們系館的樓層拓樸、地點屬性、通行規則與常見查詢對應。用純文字檔即可。

2. 實作檢索：給定一句指令，從知識庫檢索相關條目。先用關鍵字比對就好，不必一開始就上向量檢索。

3. 接上語言模型：讓模型根據檢索結果輸出結構化的目標（區域名稱、座標、信心值、來源標記）。

4. 測試失敗情況：故意問知識庫裡沒有的地點，確認系統會標明「這是猜的」而不是

假裝知道。

### 階段四　驗證層與端到端整合

1. 量測每一步的成功率：把你們的任務拆成 n 步，每一步單獨測 30 次以上，得到各步的 p_i。
2. 預測整體成功率：用公式一算出理論值，再跑 30 次完整任務，比較預測與實測。差異本身就是很好的討論題材。
3. 加上驗證：為每一步設計一個檢查，量測檢查的偵測率 r（故意製造失敗，看檢查抓到幾次）。
4. 量測改善：加上驗證與重試之後重測整體成功率，與公式一的預測比較。這是你們的圖 34-5。
5. 撰寫報告：報告的核心不是「我們做出來了」，而是「我們量到了什麼、哪裡與預測不符、為什麼」。這是本書從第 1 章到第 34 章一貫的要求。

## 延伸閱讀

- 具身導航的視覺語言模型評估基準（2025）。（34.6 節與案例 34-3 的來源，含完整的評分方法）
- 大模型增強的視覺語言導航綜述（2026）。（34.6 節關於真實部署現況的依據）
- 開放詞彙零樣本物件導航的相關研究。（34.3 節語意探索的理論基礎）
- 第 32 章的語意地圖與場景圖、第 33 章的模擬訓練與邊緣部署。（本章的直接前置，三章構成第九篇）
- 第 3 章的混合式架構、第 26 章的降級設計、第 31 章的評估與除錯方法。（34.5 節分層架構與驗證機制的思想來源）
- 第 11 章的機率與不確定性。（34.2 節「不確定性要往下傳」的理論基礎）

## 本章重點

- 任務層與感知層的本質差異在於：感知的輸入是感測器、輸出是對世界的描述；任務層的輸入是人的意圖、輸出是行動。意圖常是模糊的，任務層的工作有一大半是補上沒說出口的假設。
- 語言到行動可分五階段：意圖解析、地點定位、目標定位、路徑規劃、執行與確認。第四階段之後本書前三十一章已完全處理，語言導航是在 SLAM 前面接一段轉換，不是取代它。
- 上層送給下層的必須是結構化資料（座標、清單、信心值、來源標記），不是自由文字——自由文字無法解析也無法驗證。讓語言模型直接輸出速度指令，是把可檢查的系統變成不可檢查的黑盒子。
- 常識先驗的效益可觀：同一張地圖之下，語意順序的平均搜尋距離是 34 公尺，幾何順序是 90 公尺，縮短 61.7%（中位數縮短 74.2%）。但有 12.2% 的試驗語意順序反而更遠——它改善的是期望值，不是每一次。
- 效益取決於物件分布的集中程度：梯子（70% 在儲藏室）縮短 62%、影印機（65% 在辦公室）縮短 77%，但滅火器（60% 在走廊）只縮短 22%。對「到處都有」的物件，建立語意推理的報酬率很低。
- 先驗的正確率 α 與搜尋距離大致呈線性（α=0 時 84 公尺，α=1 時 36 公尺）。完全隨機的先驗（84 公尺）並不比不用先驗（90 公尺）差；真正危險的是反著指的先驗——138 公尺，比不用先驗還糟 54%。語言模型最容易產生的正是這種「有把握但錯誤」的先驗。
- RAG 以在地知識庫補足模型沒有的機構知識，三個優點是可維護、可驗證、低成本。常識先驗（統計性、跨場域、回答「先去哪找」）與知識庫（確定性、場域專屬、回答「那裡在哪」）分工不同，好的系統兩者都要。
- 分層架構依時間尺度切分：任務層（秒級）、語意層（0.1–1 秒）、幾何層（10–30

Hz）、控制層（100 Hz 以上）。四個實務要點是非同步設計、明確介面、可否決機制、降級路徑。安全原則是「上層可以出錯，下層必須守住底線」。

- 多步驟任務的成功率是連乘的：八步任務在每步 90% 時只有 43.0%，每步 80% 時只剩 16.8%。這解釋了為什麼每個子系統都不錯、整體卻老是失敗。
- 驗證與重試的效果遠大於提升單步能力：每步 80% 的八步任務，加上偵測率 0.5 的驗證（最多重試兩次）從 16.6% 升到 38.5%，偵測率接近 1 時升到 93.7%——底層能力完全沒變。驗證通常比執行容易得多，這種不對稱是分層架構能成立的根本原因。
- 2025 年底的具身導航基準顯示：最佳視覺語言模型得 34.38 分，人類專家 75.40 分，差距 41.02 分。更值得注意的是「已知目標的直線」得 51.89 分，勝過模型的完整軌跡；而同一模型在已知目標時只預測路徑可得 56.55 分，只預測目標點則掉到 29.65 分。模型「會走」但「不會認」——目標定位才是當前真正的瓶頸。
- 四大風險為幻覺、誤判的系統性、延遲、安全性。其中系統性誤判最危險，因為它違反「重試是獨立的」這個假設，重試救不了它。
- 全書的主線是「理解限制，才能突破限制」。AI 讓機器人聽得懂、想得通，但走得穩、不撞人仍要靠前三十一章的幾何與控制。在可預見的未來，兩者缺一不可。

## 全書結語

第九篇到此完成，全書也到此完成。

從第 32 章讓地圖有了意義、第 33 章讓機器人學會動作、到本章讓它聽懂了人話，這三章把前面三十一章累積的幾何基礎推向了應用的前沿。但請記得本章 34.6 節那幾個數字：最前沿的模型在具身導航上仍然遠不及人類，而一條已知目標的直線甚至勝過它的完整規劃。

這不是要澆冷水，而是要把位置看清楚。你們手上這套從第 1 章學到第 31 章的幾何與機率方法，不是即將被 AI 取代的舊技術，而是任何語言模型都必須站上去的地面。

三十四章，九篇，走到這裡。接下來要建造什麼，由你們決定。